

StarForth

1.0.0

Generated on Wed Jun 17 2026 18:04:02 for StarForth by Doxygen 1.9.8

Wed Jun 17 2026 18:04:02

1 StarForth	1
1.1 What makes it different	1
1.2 Quick Start	1
1.3 Documentation	1
1.4 License	2
2 Topic Index	3
2.1 Topics	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	9
4.1 File List	9
5 Topic Documentation	15
5.1 Architecture Detection	15
5.2 Unified API Wrappers	15
5.3 Runtime Architecture Detection	15
5.4 Build Information	15
5.5 VM Input Operations	15
5.5.1 Detailed Description	16
5.5.2 Function Documentation	16
5.5.2.1 vm_addr_from_ptr()	16
5.5.2.2 vm_input_ensure()	16
5.5.2.3 vm_input_in()	17
5.5.2.4 vm_input_load_line()	18
5.5.2.5 vm_input_source()	18
5.5.2.6 vm_input_span()	19
5.5.2.7 vm_input_tib()	19
5.6 Stack Operations	20
5.6.1 Detailed Description	20
5.6.2 Function Documentation	21
5.6.2.1 vm_add_check_overflow()	21
5.6.2.2 vm_cpuid()	21
5.6.2.3 vm_divmod()	22
5.6.2.4 vm_has_sse42()	22
5.6.2.5 vm_max_asm()	22
5.6.2.6 vm_memcpy_asm()	22
5.6.2.7 vm_memzero_asm()	23
5.6.2.8 vm_min_asm()	23
5.6.2.9 vm_mul_double()	23
5.6.2.10 vm_pop_asm()	23
5.6.2.11 vm_push_asm()	24

5.6.2.12 vm_rpop_asm()	24
5.6.2.13 vm_rpush_asm()	25
5.6.2.14 vm_strcmp_asm()	25
5.6.2.15 vm_sub_check_overflow()	25
5.7 Fast Primitive Word Implementations	26
5.7.1 Detailed Description	26
5.7.2 Macro Definition Documentation	27
5.7.2.1 PRIM_2DROP_ARM64	27
5.7.2.2 PRIM_2DUP_ARM64	27
5.7.2.3 PRIM_AND_ARM64	27
5.7.2.4 PRIM_BRANCH_ARM64	27
5.7.2.5 PRIM_C_FETCH_ARM64	27
5.7.2.6 PRIM_C_STORE_ARM64	27
5.7.2.7 PRIM_DROP_ARM64	27
5.7.2.8 PRIM_DUP_ARM64	28
5.7.2.9 PRIM_EQUALS_ARM64	28
5.7.2.10 PRIM_EXECUTE_ARM64	28
5.7.2.11 PRIM_FETCH_ARM64	28
5.7.2.12 PRIM_GREATER_ARM64	28
5.7.2.13 PRIM_INVERT_ARM64	28
5.7.2.14 PRIM_LESS_ARM64	28
5.7.2.15 PRIM_MINUS_ARM64	29
5.7.2.16 PRIM_MOD_ARM64	29
5.7.2.17 PRIM_NEGATE_ARM64	29
5.7.2.18 PRIM_OR_ARM64	29
5.7.2.19 PRIM_OVER_ARM64	29
5.7.2.20 PRIM_PLUS_ARM64	29
5.7.2.21 PRIM_R_FETCH_ARM64	29
5.7.2.22 PRIM_R_FROM_ARM64	30
5.7.2.23 PRIM_ROT_ARM64	30
5.7.2.24 PRIM_SLASH_ARM64	30
5.7.2.25 PRIM_STAR_ARM64	30
5.7.2.26 PRIM_STORE_ARM64	30
5.7.2.27 PRIM_SWAP_ARM64	30
5.7.2.28 PRIM_TO_R_ARM64	31
5.7.2.29 PRIM_XOR_ARM64	31
5.7.2.30 PRIM_ZBRANCH_ARM64	31
5.7.2.31 PRIM_ZERO_EQUALS_ARM64	31
5.7.2.32 PRIM_ZERO_LESS_ARM64	31
5.8 Control Flow Stack	31
5.8.1 Detailed Description	32
5.8.2 Macro Definition Documentation	32

5.8.2.1 CF_STACK_MAX	32
5.8.3 Enumeration Type Documentation	32
5.8.3.1 cf_tag_t	32
5.9 Runtime Branch Helpers	32
5.9.1 Detailed Description	33
5.9.2 Function Documentation	33
5.9.2.1 control_forth_0branch()	33
5.9.2.2 control_forth_again()	34
5.9.2.3 control_forth_begin()	34
5.9.2.4 control_forth_branch()	34
5.9.2.5 control_forth_case()	34
5.9.2.6 control_forth_do()	34
5.9.2.7 control_forth_else()	34
5.9.2.8 control_forth_endcase()	35
5.9.2.9 control_forth_endof()	35
5.9.2.10 control_forth_EXIT()	35
5.9.2.11 control_forth_I()	35
5.9.2.12 control_forth_if()	35
5.9.2.13 control_forth_J()	35
5.9.2.14 control_forth_leave()	36
5.9.2.15 control_forth_loop()	36
5.9.2.16 control_forth_of()	36
5.9.2.17 control_forth_plus_loop()	36
5.9.2.18 control_forth_qdo()	36
5.9.2.19 control_forth_repeat()	36
5.9.2.20 control_forth_runtime_do()	37
5.9.2.21 control_forth_runtime_leave()	37
5.9.2.22 control_forth_runtime_loop()	37
5.9.2.23 control_forth_runtime_plus_loop()	37
5.9.2.24 control_forth_runtime_qdo()	37
5.9.2.25 control_forth_then()	37
5.9.2.26 control_forth_UNLOOP()	38
5.9.2.27 control_forth_until()	38
5.9.2.28 control_forth_while()	38
5.9.2.29 register_control_words()	38
5.9.3 Variable Documentation	38
5.9.3.1 addr	38
5.9.3.2 tag	39
5.10 FORTH-79 Control Flow Words	39
5.11 FORTH-79 Defining Words	39
5.12 FORTH-79 Dictionary Manipulation Words	39
5.13 FORTH-79 Dictionary & Compilation Words	40

5.14 FORTH-79 Line Editor Words	41
5.15 FORTH-79 Logical Operations	41
5.16 FORTH-79 Memory Operations	41
5.17 FORTH-79 Return Stack Operations	42
5.18 FORTH-79 System & Environment Words	42
5.19 FORTH-79 Vocabulary System Words	42
6 Data Structure Documentation	45
6.1 _EFI_BOOT_SERVICES Struct Reference	45
6.1.1 Field Documentation	46
6.1.1.1 AllocatePages	46
6.1.1.2 AllocatePool	46
6.1.1.3 CheckEvent	46
6.1.1.4 CloseEvent	46
6.1.1.5 CreateEvent	46
6.1.1.6 Exit	46
6.1.1.7 ExitBootServices	46
6.1.1.8 FreePages	46
6.1.1.9 FreePool	46
6.1.1.10 GetMemoryMap	46
6.1.1.11 GetNextMonotonicCount	46
6.1.1.12 HandleProtocol	46
6.1.1.13 Hdr	46
6.1.1.14 InstallConfigurationTable	46
6.1.1.15 InstallProtocolInterface	47
6.1.1.16 LoadImage	47
6.1.1.17 LocateDevicePath	47
6.1.1.18 LocateHandle	47
6.1.1.19 RaiseTPL	47
6.1.1.20 RegisterProtocolNotify	47
6.1.1.21 ReinstallProtocolInterface	47
6.1.1.22 Reserved	47
6.1.1.23 RestoreTPL	47
6.1.1.24 SetTimer	47
6.1.1.25 SetWatchdogTimer	47
6.1.1.26 SignalEvent	47
6.1.1.27 Stall	47
6.1.1.28 StartImage	47
6.1.1.29 UninstallProtocolInterface	47
6.1.1.30 UnloadImage	48
6.1.1.31 WaitForEvent	48
6.2 _EFI_FILE_PROTOCOL Struct Reference	48

6.2.1 Field Documentation	48
6.2.1.1 Close	48
6.2.1.2 Delete	48
6.2.1.3 Flush	48
6.2.1.4 GetInfo	48
6.2.1.5 GetPosition	48
6.2.1.6 Open	48
6.2.1.7 Read	49
6.2.1.8 Revision	49
6.2.1.9 SetInfo	49
6.2.1.10 SetPosition	49
6.2.1.11 Write	49
6.3 _EFI_RUNTIME_SERVICES Struct Reference	49
6.3.1 Field Documentation	49
6.3.1.1 ConvertPointer	49
6.3.1.2 GetNextHighMonotonicCount	49
6.3.1.3 GetNextVariableName	49
6.3.1.4 GetTime	50
6.3.1.5 GetVariable	50
6.3.1.6 GetWakeupTime	50
6.3.1.7 Hdr	50
6.3.1.8 ResetSystem	50
6.3.1.9 SetTime	50
6.3.1.10 SetVariable	50
6.3.1.11 SetVirtualAddressMap	50
6.3.1.12 SetWakeupTime	50
6.4 _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL Struct Reference	50
6.4.1 Field Documentation	50
6.4.1.1 OpenVolume	50
6.4.1.2 Revision	50
6.5 _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL Struct Reference	51
6.5.1 Field Documentation	51
6.5.1.1 ClearScreen	51
6.5.1.2 EnableCursor	51
6.5.1.3 Mode	51
6.5.1.4 OutputString	51
6.5.1.5 QueryMode	51
6.5.1.6 Reset	51
6.5.1.7 SetAttribute	51
6.5.1.8 SetCursorPosition	51
6.5.1.9 SetMode	51
6.5.1.10 TestString	52

6.6 _EFI_SYSTEM_TABLE Struct Reference	52
6.6.1 Field Documentation	52
6.6.1.1 BootServices	52
6.6.1.2 ConfigurationTable	52
6.6.1.3 ConIn	52
6.6.1.4 ConOut	52
6.6.1.5 ConsoleInHandle	52
6.6.1.6 ConsoleOutHandle	52
6.6.1.7 FirmwareRevision	52
6.6.1.8 FirmwareVendor	53
6.6.1.9 Hdr	53
6.6.1.10 NumberOfTableEntries	53
6.6.1.11 RuntimeServices	53
6.6.1.12 StandardErrorHandle	53
6.6.1.13 StdErr	53
6.7 BayesianLatencyPosterior Struct Reference	53
6.7.1 Detailed Description	53
6.7.2 Field Documentation	53
6.7.2.1 credible_lower_95	53
6.7.2.2 credible_lower_99	54
6.7.2.3 credible_upper_95	54
6.7.2.4 credible_upper_99	54
6.7.2.5 mean_ns_q48	54
6.7.2.6 median_ns_q48	54
6.7.2.7 sample_count	54
6.7.2.8 stddev_ns_q48	54
6.8 blk_meta_t Struct Reference	54
6.8.1 Field Documentation	55
6.8.1.1 acl_block	55
6.8.1.2 app_data	55
6.8.1.3 chain_length	55
6.8.1.4 checksum	55
6.8.1.5 content_length	55
6.8.1.6 content_type	55
6.8.1.7 created_time	55
6.8.1.8 encoding	55
6.8.1.9 entropy	55
6.8.1.10 flags	55
6.8.1.11 hash	55
6.8.1.12 magic	56
6.8.1.13 modified_time	56
6.8.1.14 next_block	56

6.8.1.15 owner_id	56
6.8.1.16 padding	56
6.8.1.17 parent_block	56
6.8.1.18 permissions	56
6.8.1.19 prev_block	56
6.8.1.20 reserved1	56
6.8.1.21 signature	56
6.8.1.22 write_count	56
6.9 blk_volume_meta_t Struct Reference	56
6.9.1 Field Documentation	57
6.9.1.1 _pad	57
6.9.1.2 bam_devblocks	57
6.9.1.3 bam_start	57
6.9.1.4 created_time	57
6.9.1.5 devblock_base	57
6.9.1.6 first_free	57
6.9.1.7 flags	57
6.9.1.8 free_blocks	57
6.9.1.9 hdr_crc	57
6.9.1.10 label	57
6.9.1.11 last_allocated	58
6.9.1.12 magic	58
6.9.1.13 mounted_time	58
6.9.1.14 reserved_disk_lo	58
6.9.1.15 reserved_ram_lo	58
6.9.1.16 total_blocks	58
6.9.1.17 total_devblocks	58
6.9.1.18 total_volumes	58
6.9.1.19 tracked_blocks	58
6.9.1.20 version	58
6.10 blkcfg_t Struct Reference	58
6.10.1 Field Documentation	59
6.10.1.1 backend	59
6.10.1.2 blocks	59
6.10.1.3 fbs	59
6.10.1.4 file_path	59
6.10.1.5 read_only	59
6.10.1.6 size_bytes	59
6.11 blkio_dev Struct Reference	59
6.11.1 Field Documentation	59
6.11.1.1 forth_block_size	59
6.11.1.2 is_open	59

6.11.1.3 state	59
6.11.1.4 total_blocks	60
6.11.1.5 vt	60
6.12 blkio_file_state_t Struct Reference	60
6.12.1 Field Documentation	60
6.12.1.1 fbs	60
6.12.1.2 fd	60
6.12.1.3 path	60
6.12.1.4 read_only	60
6.12.1.5 total_blocks	60
6.13 blkio_info_t Struct Reference	60
6.13.1 Field Documentation	61
6.13.1.1 forth_block_size	61
6.13.1.2 phys_sector_size	61
6.13.1.3 phys_size_bytes	61
6.13.1.4 read_only	61
6.13.1.5 total_blocks	61
6.14 blkio_params_t Struct Reference	61
6.14.1 Field Documentation	61
6.14.1.1 forth_block_size	61
6.14.1.2 opaque	61
6.14.1.3 total_blocks	62
6.15 blkio_ram_state_t Struct Reference	62
6.15.1 Field Documentation	62
6.15.1.1 base	62
6.15.1.2 fbs	62
6.15.1.3 read_only	62
6.15.1.4 total_blocks	62
6.16 blkio_vtable_t Struct Reference	62
6.16.1 Field Documentation	62
6.16.1.1 close	62
6.16.1.2 flush	63
6.16.1.3 info	63
6.16.1.4 open	63
6.16.1.5 read	63
6.16.1.6 write	63
6.17 BlockBuffer Struct Reference	63
6.17.1 Detailed Description	63
6.17.2 Field Documentation	63
6.17.2.1 accessed	63
6.17.2.2 block_number	64
6.17.2.3 data	64

6.17.2.4 dirty	64
6.17.2.5 hw_address	64
6.17.2.6 type	64
6.18 BootInfo Struct Reference	64
6.18.1 Field Documentation	64
6.18.1.1 acpi_table	64
6.18.1.2 framebuffer	64
6.18.1.3 memory_map	64
6.18.1.4 memory_map_descriptor_size	65
6.18.1.5 memory_map_size	65
6.18.1.6 runtime_services	65
6.18.1.7 uefi_boot_services_exited	65
6.19 cache_slot_t Struct Reference	65
6.19.1 Field Documentation	65
6.19.1.1 data	65
6.19.1.2 devblock	65
6.19.1.3 dirty	65
6.19.1.4 loaded	65
6.19.1.5 meta	66
6.19.1.6 meta_dirty	66
6.19.1.7 valid	66
6.20 CapsuleDirHeader Struct Reference	66
6.20.1 Field Documentation	66
6.20.1.1 arena_base	66
6.20.1.2 arena_size	66
6.20.1.3 desc_capacity	66
6.20.1.4 desc_count	66
6.20.1.5 dir_hash	66
6.20.1.6 magic	67
6.20.1.7 name_count	67
6.20.1.8 reserved	67
6.21 CapsuleNameEntry Struct Reference	67
6.21.1 Field Documentation	67
6.21.1.1 name	67
6.22 CapsuleRunRecord Struct Reference	67
6.22.1 Field Documentation	67
6.22.1.1 capsule_hash	67
6.22.1.2 capsule_id	68
6.22.1.3 ended_ns	68
6.22.1.4 flags	68
6.22.1.5 post_dict_hash	68
6.22.1.6 pre_dict_hash	68

6.22.1.7 reserved	68
6.22.1.8 result_code	68
6.22.1.9 run_id	68
6.22.1.10 started_ns	68
6.22.1.11 vm_id	68
6.23 cf_item_t Struct Reference	68
6.24 CLIConfig Struct Reference	69
6.24.1 Detailed Description	69
6.24.2 Field Documentation	69
6.24.2.1 benchmark	69
6.24.2.2 benchmark_iterations	69
6.24.2.3 break_me	69
6.24.2.4 disk_img_path	69
6.24.2.5 doe_experiment	69
6.24.2.6 fail_fast	70
6.24.2.7 heartbeat_log_mode	70
6.24.2.8 log_level	70
6.24.2.9 log_level_explicitly_set	70
6.24.2.10 ram_disk_mb	70
6.24.2.11 run_tests	70
6.24.2.12 run_tests_flag_observed	70
6.24.2.13 script_mode	70
6.25 DictEntry Struct Reference	70
6.25.1 Field Documentation	71
6.25.1.1 acl_allow	71
6.25.1.2 acl_default	71
6.25.1.3 acl_mode	71
6.25.1.4 acl_pinned	71
6.25.1.5 acl_ttl	71
6.25.1.6 execution_heat	71
6.25.1.7 flags	71
6.25.1.8 func	71
6.25.1.9 link	72
6.25.1.10 name	72
6.25.1.11 name_len	72
6.25.1.12 physics	72
6.25.1.13 transition_metrics	72
6.25.1.14 word_id	72
6.26 DictPhysics Struct Reference	72
6.26.1 Field Documentation	73
6.26.1.1 acl_hint	73
6.26.1.2 avg_latency_ns	73

6.26.1.3 last_active_ns	73
6.26.1.4 last_decay_ns	73
6.26.1.5 mass_bytes	73
6.26.1.6 pubsub_mask	73
6.26.1.7 state_flags	73
6.26.1.8 temperature_q8	73
6.27 DoeMetrics Struct Reference	73
6.27.1 Detailed Description	75
6.27.2 Field Documentation	75
6.27.2.1 actual_window_size	75
6.27.2.2 adaptive_shrink_rate	75
6.27.2.3 avg_word_heat	75
6.27.2.4 bucket_hit_percent	75
6.27.2.5 bucket_hits	75
6.27.2.6 bucket_search_latency_ns	75
6.27.2.7 bucket_search_stddev_ns	75
6.27.2.8 cache_demotions	75
6.27.2.9 cache_hit_latency_ns	75
6.27.2.10 cache_hit_percent	75
6.27.2.11 cache_hit_stddev_ns	76
6.27.2.12 cache_hits	76
6.27.2.13 cache_promotions	76
6.27.2.14 ci_lower_95	76
6.27.2.15 ci_upper_95	76
6.27.2.16 context_accuracy_percent	76
6.27.2.17 context_correct	76
6.27.2.18 context_predictions_total	76
6.27.2.19 cpu_freq_delta_mhz_q48	76
6.27.2.20 cpu_temp_delta_c_q48	76
6.27.2.21 decay_min_interval_ns	76
6.27.2.22 decay_rate_q16	76
6.27.2.23 decay_slope	77
6.27.2.24 dictionary_lookups	77
6.27.2.25 early_exit_count	77
6.27.2.26 enable_hotwords_cache	77
6.27.2.27 enable_loop_1_heat_tracking	77
6.27.2.28 enable_loop_2_rolling_window	77
6.27.2.29 enable_loop_3_linear_decay	77
6.27.2.30 enable_loop_4_pipelining	77
6.27.2.31 enable_loop_5_window_inference	77
6.27.2.32 enable_loop_6_decay_inference	77
6.27.2.33 enable_loop_7_adaptive_heartrate	77

6.27.2.34 enable_pipelining	77
6.27.2.35 final_effective_window_size	78
6.27.2.36 heat_cache_demotion_threshold	78
6.27.2.37 hot_word_count	78
6.27.2.38 inference_run_count	78
6.27.2.39 memory_allocated_bytes	78
6.27.2.40 prefetch_accuracy_percent	78
6.27.2.41 prefetch_attempts	78
6.27.2.42 prefetch_hits	78
6.27.2.43 rolling_window_size	78
6.27.2.44 rolling_window_width	78
6.27.2.45 speedup_vs_baseline	78
6.27.2.46 stale_word_count	78
6.27.2.47 stale_word_ratio	79
6.27.2.48 tick_count	79
6.27.2.49 tick_target_ns	79
6.27.2.50 total_executions	79
6.27.2.51 total_heat	79
6.27.2.52 total_lookups	79
6.27.2.53 total_runtime_ms	79
6.27.2.54 vm_workload_duration_ns_q48	79
6.27.2.55 window_diversity_percent	79
6.27.2.56 window_final_size_bytes	79
6.27.2.57 window_tuning_checks	79
6.27.2.58 window_variance_q48	79
6.27.2.59 words_executed	80
6.28 EFI_CONFIGURATION_TABLE Struct Reference	80
6.28.1 Field Documentation	80
6.28.1.1 VendorGuid	80
6.28.1.2 VendorTable	80
6.29 EFI_FILE_INFO Struct Reference	80
6.29.1 Field Documentation	80
6.29.1.1 Attribute	80
6.29.1.2 CreateTime	80
6.29.1.3 FileName	81
6.29.1.4 FileSize	81
6.29.1.5 LastAccessTime	81
6.29.1.6 ModificationTime	81
6.29.1.7 PhysicalSize	81
6.29.1.8 Size	81
6.30 EFI_GUID Struct Reference	81
6.30.1 Field Documentation	81

6.30.1.1 Data1	81
6.30.1.2 Data2	81
6.30.1.3 Data3	81
6.30.1.4 Data4	82
6.31 EFI_LOADED_IMAGE_PROTOCOL Struct Reference	82
6.31.1 Field Documentation	82
6.31.1.1 DeviceHandle	82
6.31.1.2 FilePath	82
6.31.1.3 ImageBase	82
6.31.1.4 ImageCodeType	82
6.31.1.5 ImageDataType	82
6.31.1.6 ImageSize	82
6.31.1.7 LoadOptions	82
6.31.1.8 LoadOptionsSize	82
6.31.1.9 Reserved	83
6.31.1.10 Unload	83
6.32 EFI_MEMORY_DESCRIPTOR Struct Reference	83
6.32.1 Field Documentation	83
6.32.1.1 Attribute	83
6.32.1.2 NumberOfPages	83
6.32.1.3 PhysicalStart	83
6.32.1.4 Type	83
6.32.1.5 VirtualStart	83
6.33 EFI_TABLE_HEADER Struct Reference	83
6.33.1 Field Documentation	84
6.33.1.1 CRC32	84
6.33.1.2 HeaderSize	84
6.33.1.3 Reserved	84
6.33.1.4 Revision	84
6.33.1.5 Signature	84
6.34 EFI_TIME Struct Reference	84
6.34.1 Field Documentation	84
6.34.1.1 Day	84
6.34.1.2 Daylight	84
6.34.1.3 Hour	85
6.34.1.4 Minute	85
6.34.1.5 Month	85
6.34.1.6 Nanosecond	85
6.34.1.7 Pad1	85
6.34.1.8 Pad2	85
6.34.1.9 Second	85
6.34.1.10 TimeZone	85

6.34.1.11 Year	85
6.35 Elf64_Ehdr Struct Reference	85
6.35.1 Field Documentation	86
6.35.1.1 e_ehsize	86
6.35.1.2 e_entry	86
6.35.1.3 e_flags	86
6.35.1.4 e_ident	86
6.35.1.5 e_machine	86
6.35.1.6 e_phentsize	86
6.35.1.7 e_phnum	86
6.35.1.8 e_phoff	86
6.35.1.9 e_shentsize	86
6.35.1.10 e_shnum	86
6.35.1.11 e_shoff	86
6.35.1.12 e_shstrndx	86
6.35.1.13 e_type	86
6.35.1.14 e_version	87
6.36 Elf64_Phdr Struct Reference	87
6.36.1 Field Documentation	87
6.36.1.1 p_align	87
6.36.1.2 p_filesz	87
6.36.1.3 p_flags	87
6.36.1.4 p_memsz	87
6.36.1.5 p_offset	87
6.36.1.6 p_paddr	87
6.36.1.7 p_type	87
6.36.1.8 p_vaddr	87
6.37 Elf64_Rel Struct Reference	88
6.37.1 Field Documentation	88
6.37.1.1 r_info	88
6.37.1.2 r_offset	88
6.38 Elf64_Rela Struct Reference	88
6.38.1 Field Documentation	88
6.38.1.1 r_addend	88
6.38.1.2 r_info	88
6.38.1.3 r_offset	88
6.39 Elf64_Shdr Struct Reference	88
6.39.1 Field Documentation	89
6.39.1.1 sh_addr	89
6.39.1.2 sh_addralign	89
6.39.1.3 sh_entsize	89
6.39.1.4 sh_flags	89

6.39.1.5 sh_info	89
6.39.1.6 sh_link	89
6.39.1.7 sh_name	89
6.39.1.8 sh_offset	89
6.39.1.9 sh_size	89
6.39.1.10 sh_type	89
6.40 Elf64_Sym Struct Reference	90
6.40.1 Field Documentation	90
6.40.1.1 st_info	90
6.40.1.2 st_name	90
6.40.1.3 st_other	90
6.40.1.4 st_shndx	90
6.40.1.5 st_size	90
6.40.1.6 st_value	90
6.41 ExecResult Struct Reference	90
6.41.1 Field Documentation	90
6.41.1.1 dsp	90
6.41.1.2 error	91
6.41.1.3 rsp	91
6.41.1.4 stack	91
6.42 FramebufferInfo Struct Reference	91
6.42.1 Field Documentation	91
6.42.1.1 base	91
6.42.1.2 height	91
6.42.1.3 pixels_per_scanline	91
6.42.1.4 size	91
6.42.1.5 width	91
6.43 gdt_entry Struct Reference	92
6.43.1 Detailed Description	92
6.43.2 Field Documentation	92
6.43.2.1 access	92
6.43.2.2 base_high	92
6.43.2.3 base_low	92
6.43.2.4 base_mid	92
6.43.2.5 granularity	92
6.43.2.6 limit_low	92
6.44 gdt_r Struct Reference	92
6.44.1 Field Documentation	92
6.44.1.1 base	92
6.44.1.2 limit	93
6.45 heap_block Struct Reference	93
6.45.1 Field Documentation	93

6.45.1.1 free	93
6.45.1.2 next	93
6.45.1.3 prev	93
6.45.1.4 requested	93
6.45.1.5 size	93
6.46 HeartbeatSnapshot Struct Reference	93
6.46.1 Field Documentation	94
6.46.1.1 actual_window_size	94
6.46.1.2 decay_slope_q48	94
6.46.1.3 hot_word_count	94
6.46.1.4 published_ns	94
6.46.1.5 published_tick	94
6.46.1.6 stale_word_count	94
6.46.1.7 total_heat	94
6.46.1.8 window_width	94
6.47 HeartbeatState Struct Reference	94
6.47.1 Field Documentation	95
6.47.1.1 bucket_collapse_flag	95
6.47.1.2 bucket_mode_transitions	95
6.47.1.3 bucket_sum_heat_variance	95
6.47.1.4 bucket_sum_K	95
6.47.1.5 bucket_sum_K_squared	95
6.47.1.6 bucket_sum_window_variance	96
6.47.1.7 bucket_tick_count	96
6.47.1.8 check_counter	96
6.47.1.9 dictionary_lookups	96
6.47.1.10 early_exit_count	96
6.47.1.11 heartbeat_enabled	96
6.47.1.12 inference_run_count	96
6.47.1.13 last_inference_tick	96
6.47.1.14 m5_time_trust	96
6.47.1.15 m5_variance	96
6.47.1.16 run_start_ns	96
6.47.1.17 snapshot_index	96
6.47.1.18 snapshots	97
6.47.1.19 tick_buffer	97
6.47.1.20 tick_buffer_size	97
6.47.1.21 tick_buffer_write_index	97
6.47.1.22 tick_count	97
6.47.1.23 tick_count_total	97
6.47.1.24 tick_number_offset	97
6.47.1.25 tick_target_ns	97

6.47.1.26 words_executed	97
6.47.1.27 worker	97
6.48 HeartbeatTickSnapshot Struct Reference	97
6.48.1 Field Documentation	98
6.48.1.1 actual_window_size	98
6.48.1.2 avg_word_heat	98
6.48.1.3 bucket_hits_delta	98
6.48.1.4 cache_hits_delta	98
6.48.1.5 elapsed_ns	98
6.48.1.6 estimated_jitter_ns	98
6.48.1.7 hot_word_count	98
6.48.1.8 l8_mode	98
6.48.1.9 predicted_label_hits	99
6.48.1.10 tick_interval_ns	99
6.48.1.11 tick_number	99
6.48.1.12 window_width	99
6.48.1.13 word_executions_delta	99
6.49 HeartbeatWorker Struct Reference	99
6.49.1 Field Documentation	99
6.49.1.1 running	99
6.49.1.2 stop_requested	99
6.49.1.3 tick_ns	99
6.50 HotwordsCache_s Struct Reference	100
6.50.1 Field Documentation	100
6.50.1.1 cache	100
6.50.1.2 cache_count	100
6.50.1.3 enabled	100
6.50.1.4 lru_index	100
6.50.1.5 stats	100
6.51 HotwordsStats Struct Reference	100
6.51.1 Field Documentation	101
6.51.1.1 bucket_hits	101
6.51.1.2 bucket_reorders	101
6.51.1.3 bucket_search_samples	102
6.51.1.4 bucket_search_total_ns_q48	102
6.51.1.5 bucket_search_variance_sum_q48	102
6.51.1.6 cache_hit_samples	102
6.51.1.7 cache_hit_total_ns_q48	102
6.51.1.8 cache_hit_variance_sum_q48	102
6.51.1.9 cache_hits	102
6.51.1.10 evictions	102
6.51.1.11 max_bucket_search_ns	102

6.51.1.12 max_cache_hit_ns	103
6.51.1.13 min_bucket_search_ns	103
6.51.1.14 min_cache_hit_ns	103
6.51.1.15 misses	103
6.51.1.16 promotions	103
6.51.1.17 total_lookups	103
6.52 idt_entry Struct Reference	103
6.52.1 Field Documentation	104
6.52.1.1 ist	104
6.52.1.2 offset_high	104
6.52.1.3 offset_low	104
6.52.1.4 offset_mid	104
6.52.1.5 selector	104
6.52.1.6 type_attr	104
6.52.1.7 zero	104
6.53 idtr Struct Reference	104
6.53.1 Field Documentation	104
6.53.1.1 base	104
6.53.1.2 limit	104
6.54 InferenceInputs Struct Reference	105
6.54.1 Field Documentation	105
6.54.1.1 hot_word_count	105
6.54.1.2 last_stale_count	105
6.54.1.3 last_total_heat	105
6.54.1.4 prefetch_attempts	105
6.54.1.5 prefetch_hits	105
6.54.1.6 stale_word_count	105
6.54.1.7 total_heat	105
6.54.1.8 trajectory_length	105
6.54.1.9 vm	105
6.54.1.10 window	106
6.54.1.11 word_count	106
6.55 InferenceOutputs Struct Reference	106
6.55.1 Field Documentation	106
6.55.1.1 adaptive_decay_slope	106
6.55.1.2 adaptive_window_width	106
6.55.1.3 early_exited	106
6.55.1.4 last_check_tick	106
6.55.1.5 slope_fit_quality_q48	106
6.55.1.6 window_variance_q48	107
6.56 kmalloc_prefix_t Struct Reference	107
6.56.1 Field Documentation	107

6.56.1.1 block	107
6.56.1.2 magic	107
6.57 kmalloc_stats_t Struct Reference	107
6.57.1 Detailed Description	107
6.57.2 Field Documentation	107
6.57.2.1 free_bytes	107
6.57.2.2 peak_bytes	107
6.57.2.3 total_bytes	108
6.57.2.4 used_bytes	108
6.58 MemoryProfile Struct Reference	108
6.58.1 Field Documentation	108
6.58.1.1 cache_hits	108
6.58.1.2 cache_misses	108
6.58.1.3 reads	108
6.58.1.4 total_bytes_read	108
6.58.1.5 total_bytes_written	108
6.58.1.6 writes	108
6.59 ParityPacket Struct Reference	108
6.59.1 Detailed Description	109
6.59.2 Field Documentation	109
6.59.2.1 bootstrap_result	109
6.59.2.2 header_hash64	109
6.59.2.3 here_offset	109
6.59.2.4 latest_word_id	109
6.59.2.5 tests_errors	109
6.59.2.6 tests_failed	109
6.59.2.7 tests_passed	109
6.59.2.8 tests_skipped	109
6.59.2.9 tests_total	110
6.59.2.10 window_hash64	110
6.59.2.11 word_count	110
6.60 physics_analytics_event_header Struct Reference	110
6.60.1 Detailed Description	110
6.60.2 Field Documentation	110
6.60.2.1 channel	110
6.60.2.2 payload_bytes	110
6.60.2.3 reserved	110
6.60.2.4 timestamp_ns	111
6.61 physics_analytics_header Struct Reference	111
6.61.1 Detailed Description	111
6.61.2 Field Documentation	112
6.61.2.1 consume_seq	112

6.61.2.2	dropped_events	112
6.61.2.3	flags	112
6.61.2.4	heap_bytes	112
6.61.2.5	magic	112
6.61.2.6	produce_seq	112
6.61.2.7	read_offset	112
6.61.2.8	ring_bytes	112
6.61.2.9	ring_offset	112
6.61.2.10	scratch_bytes	112
6.61.2.11	scratch_offset	113
6.61.2.12	summary_bytes	113
6.61.2.13	summary_offset	113
6.61.2.14	version_major	113
6.61.2.15	version_minor	113
6.61.2.16	write_offset	113
6.62	physics_analytics_heap Struct Reference	113
6.62.1	Detailed Description	114
6.62.2	Field Documentation	114
6.62.2.1	base	114
6.62.2.2	bytes	114
6.62.2.3	header	114
6.62.2.4	ring	114
6.62.2.5	ring_bytes	114
6.62.2.6	scratch	114
6.62.2.7	scratch_bytes	114
6.62.2.8	summary	115
6.62.2.9	summary_bytes	115
6.63	physics_host_snapshot Struct Reference	115
6.63.1	Detailed Description	116
6.63.2	Field Documentation	116
6.63.2.1	backend	116
6.63.2.2	backend_seq	116
6.63.2.3	cgroup_cpu_usage_us	116
6.63.2.4	cgroup_memory_current_bytes	117
6.63.2.5	cpu_count	117
6.63.2.6	cpu_idle_jiffies	117
6.63.2.7	cpu_total_jiffies	117
6.63.2.8	flags	117
6.63.2.9	load_avg_milli	117
6.63.2.10	monotonic_time_ns	117
6.63.2.11	psi_cpu_full_avg10_milli	117
6.63.2.12	psi_cpu_full_avg300_milli	117

6.63.2.13 psi_cpu_full_avg60_milli	117
6.63.2.14 psi_cpu_some_avg10_milli	118
6.63.2.15 psi_cpu_some_avg300_milli	118
6.63.2.16 psi_cpu_some_avg60_milli	118
6.63.2.17 psi_io_full_avg10_milli	118
6.63.2.18 psi_io_full_avg300_milli	118
6.63.2.19 psi_io_full_avg60_milli	118
6.63.2.20 psi_io_some_avg10_milli	118
6.63.2.21 psi_io_some_avg300_milli	118
6.63.2.22 psi_io_some_avg60_milli	118
6.63.2.23 psi_mem_some_avg10_milli	118
6.63.2.24 psi_mem_some_avg300_milli	119
6.63.2.25 psi_mem_some_avg60_milli	119
6.63.2.26 realtime_ns	119
6.63.2.27 runnable_threads	119
6.63.2.28 scheduler_policy	119
6.63.2.29 scheduler_priority	119
6.63.2.30 scheduler_quantum_ns	119
6.64 physics_runtime_state Struct Reference	119
6.64.1 Field Documentation	119
6.64.1.1 backend_seq	119
6.64.1.2 heap	120
6.64.1.3 heap_ready	120
6.65 physics_seed_definition Struct Reference	120
6.65.1 Field Documentation	120
6.65.1.1 acl_hint	120
6.65.1.2 avg_latency_ns	120
6.65.1.3 name	120
6.65.1.4 pubsub_mask	120
6.65.1.5 temperature_q8	120
6.66 PhysicsSnapshot Struct Reference	120
6.66.1 Field Documentation	121
6.66.1.1 decay_slope_q48	121
6.66.1.2 eff_window_size	121
6.66.1.3 heat_25th	121
6.66.1.4 heat_50th	121
6.66.1.5 heat_75th	121
6.66.1.6 prefetch_attempts	121
6.66.1.7 tick_target_ns	121
6.66.1.8 total_executions	121
6.67 PipelineGlobalMetrics Struct Reference	121
6.67.1 Field Documentation	122

6.67.1.1 last_checked_accuracy	122
6.67.1.2 last_checked_window_size	122
6.67.1.3 prefetch_attempts	122
6.67.1.4 prefetch_hits	122
6.67.1.5 suggested_next_size	122
6.67.1.6 window_tuning_checks	122
6.68 pmm_stats_t Struct Reference	122
6.68.1 Field Documentation	123
6.68.1.1 free_bytes	123
6.68.1.2 free_pages	123
6.68.1.3 total_bytes	123
6.68.1.4 total_pages	123
6.68.1.5 used_bytes	123
6.68.1.6 used_pages	123
6.69 Profiler Struct Reference	123
6.69.1 Field Documentation	124
6.69.1.1 branch_count	124
6.69.1.2 category_timers	124
6.69.1.3 compile_operations	124
6.69.1.4 [struct]	124
6.69.1.5 dict_lookups	124
6.69.1.6 instruction_count	124
6.69.1.7 level	124
6.69.1.8 loop_iterations	124
6.69.1.9 memory_allocations	124
6.69.1.10 memory_stats	124
6.69.1.11 profiling_overhead	124
6.69.1.12 stack_operations	124
6.69.1.13 total_execution_time	125
6.69.1.14 vm_cycles	125
6.69.1.15 word_profile_capacity	125
6.69.1.16 word_profile_count	125
6.69.1.17 word_profiles	125
6.70 ProfileScope Struct Reference	125
6.70.1 Field Documentation	125
6.70.1.1 category	125
6.70.1.2 word_name	125
6.71 ProfileTimer Struct Reference	125
6.71.1 Detailed Description	126
6.71.2 Field Documentation	126
6.71.2.1 call_count	126
6.71.2.2 max_time	126

6.71.2.3 min_time	126
6.71.2.4 name	126
6.71.2.5 start_time	126
6.71.2.6 total_time	126
6.72 reloc_block Struct Reference	126
6.72.1 Detailed Description	127
6.72.2 Field Documentation	127
6.72.2.1 block_size	127
6.72.2.2 entry	127
6.72.2.3 page_rva	127
6.73 rolling_window_view_t Struct Reference	127
6.73.1 Field Documentation	127
6.73.1.1 effective_window_size	127
6.73.1.2 history	127
6.73.1.3 is_warm	127
6.73.1.4 total_executions	128
6.73.1.5 window_pos	128
6.74 RollingWindowOfTruth Struct Reference	128
6.74.1 Field Documentation	128
6.74.1.1 adaptive_check_accumulator	128
6.74.1.2 adaptive_pending	128
6.74.1.3 effective_window_size	128
6.74.1.4 execution_history	129
6.74.1.5 is_warm	129
6.74.1.6 last_pattern_diversity	129
6.74.1.7 pattern_diversity_check_count	129
6.74.1.8 snapshot_buffers	129
6.74.1.9 snapshot_effective_window_size	129
6.74.1.10 snapshot_index	129
6.74.1.11 snapshot_is_warm	129
6.74.1.12 snapshot_pending	129
6.74.1.13 snapshot_total_executions	129
6.74.1.14 snapshot_window_pos	129
6.74.1.15 total_executions	130
6.74.1.16 window_pos	130
6.75 sf_alloc_stats Struct Reference	130
6.75.1 Field Documentation	130
6.75.1.1 alloc_count	130
6.75.1.2 free_count	130
6.75.1.3 peak_bytes	130
6.75.1.4 total_bytes	130
6.75.1.5 used_bytes	130

6.76 sf_mutex Struct Reference	131
6.76.1 Field Documentation	131
6.76.1.1 handle	131
6.77 sf_time_backend_t Struct Reference	131
6.77.1 Field Documentation	131
6.77.1.1 format_timestamp	131
6.77.1.2 get_monotonic_ns	131
6.77.1.3 get_realtime_ns	132
6.77.1.4 has_rtc	132
6.77.1.5 set_realtime_ns	132
6.78 SpeedupEstimate Struct Reference	132
6.78.1 Detailed Description	133
6.78.2 Field Documentation	133
6.78.2.1 credible_lower_95_q48	133
6.78.2.2 credible_upper_95_q48	133
6.78.2.3 probability_gt_10pct_q48	133
6.78.2.4 probability_gt_double_q48	133
6.78.2.5 speedup_factor_q48	133
6.79 ssm_config_t Struct Reference	133
6.79.1 Field Documentation	133
6.79.1.1 L2_rolling_window	133
6.79.1.2 L3_linear_decay	133
6.79.1.3 L5_window_inference	134
6.79.1.4 L6_decay_inference	134
6.80 ssm_l8_metrics_t Struct Reference	134
6.80.1 Field Documentation	134
6.80.1.1 cv	134
6.80.1.2 entropy	134
6.80.1.3 stability_score	134
6.80.1.4 temporal_decay	134
6.81 ssm_l8_state_t Struct Reference	134
6.81.1 Field Documentation	135
6.81.1.1 current_mode	135
6.81.1.2 hysteresis_counter	135
6.81.1.3 pending_mode	135
6.82 TestCase Struct Reference	135
6.82.1 Detailed Description	135
6.82.2 Field Documentation	135
6.82.2.1 contract	135
6.82.2.2 expected	136
6.82.2.3 implemented	136
6.82.2.4 input	136

6.82.2.5 name	136
6.82.2.6 should_error	136
6.82.2.7 type	136
6.83 TestModule Struct Reference	136
6.83.1 Detailed Description	136
6.83.2 Field Documentation	137
6.83.2.1 module_name	137
6.83.2.2 reserved	137
6.83.2.3 reserved_int	137
6.83.2.4 run_module_tests	137
6.84 TestStats Struct Reference	137
6.84.1 Detailed Description	137
6.84.2 Field Documentation	137
6.84.2.1 total_error	137
6.84.2.2 total_fail	138
6.84.2.3 total_pass	138
6.84.2.4 total_skip	138
6.84.2.5 total_tests	138
6.85 time_trust_state Struct Reference	138
6.85.1 Field Documentation	138
6.85.1.1 expected_delta	138
6.85.1.2 last_tsc	138
6.85.1.3 ticks	138
6.85.1.4 total_samples	139
6.85.1.5 trust	139
6.85.1.6 variance	139
6.85.1.7 window	139
6.86 time_window Struct Reference	139
6.86.1 Field Documentation	139
6.86.1.1 count	139
6.86.1.2 deltas	139
6.86.1.3 pos	139
6.87 timer_calibration_record Struct Reference	139
6.87.1 Field Documentation	140
6.87.1.1 converged	140
6.87.1.2 cv_hpet_ppm	140
6.87.1.3 cv_pit_ppm	140
6.87.1.4 diff_ppm	140
6.87.1.5 hpet_hz	140
6.87.1.6 pit_hz_mean	140
6.87.1.7 reserved	140
6.87.1.8 trust	140

6.87.1.9 tsc_hz_mean	140
6.87.1.10 vm_mode	141
6.87.1.11 windows_used	141
6.88 VM Struct Reference	141
6.88.1 Detailed Description	143
6.88.2 Field Documentation	143
6.88.2.1 abort_requested	143
6.88.2.2 base	143
6.88.2.3 base_addr	143
6.88.2.4 compile_buffer	144
6.88.2.5 compile_pos	144
6.88.2.6 compile_size	144
6.88.2.7 compiling_word	144
6.88.2.8 current_executing_entry	144
6.88.2.9 current_word_name	144
6.88.2.10 data_stack	144
6.88.2.11 decay_slope_direction	144
6.88.2.12 decay_slope_q48	144
6.88.2.13 dict_fence_here	145
6.88.2.14 dict_fence_latest	145
6.88.2.15 dict_lock	145
6.88.2.16 doe_row_printing	145
6.88.2.17 dsp	145
6.88.2.18 ecw_nesting	146
6.88.2.19 emergency_console	146
6.88.2.20 error	146
6.88.2.21 exit_colon	147
6.88.2.22 halted	147
6.88.2.23 heartbeat	147
6.88.2.24 heartbeat_decay_cursor_id	148
6.88.2.25 heat_threshold_25th	148
6.88.2.26 heat_threshold_50th	148
6.88.2.27 heat_threshold_75th	148
6.88.2.28 here	148
6.88.2.29 host	148
6.88.2.30 hot_word_count_at_check	148
6.88.2.31 hotwords_cache	149
6.88.2.32 in_var	149
6.88.2.33 input_buffer	149
6.88.2.34 input_length	149
6.88.2.35 input_pos	149
6.88.2.36 interpreter_enabled	149

6.88.2.37 ip	149
6.88.2.38 last_bucket_reorg_ns	149
6.88.2.39 last_decay_check_ns	149
6.88.2.40 last_inference_outputs	149
6.88.2.41 latest	150
6.88.2.42 lookup_strategy	150
6.88.2.43 memory	150
6.88.2.44 mode	150
6.88.2.45 next_word_id	150
6.88.2.46 pipeline_metrics	150
6.88.2.47 recycled_word_id_count	151
6.88.2.48 recycled_word_ids	151
6.88.2.49 return_stack	151
6.88.2.50 rolling_window	151
6.88.2.51 rsp	151
6.88.2.52 scr_addr	151
6.88.2.53 span_var	152
6.88.2.54 ssm_config	152
6.88.2.55 ssm_l8_state	152
6.88.2.56 stale_word_count_at_check	152
6.88.2.57 state_addr	152
6.88.2.58 state_var	152
6.88.2.59 tib_buf	152
6.88.2.60 tib_cap	152
6.88.2.61 total_heat_at_last_check	152
6.88.2.62 tuning_lock	152
6.88.2.63 word_count_at_check	153
6.88.2.64 word_id_map	153
6.88.2.65 zuse_session	153
6.89 VMHostServices Struct Reference	153
6.89.1 Detailed Description	153
6.89.2 Field Documentation	153
6.89.2.1 alloc	153
6.89.2.2 free	154
6.89.2.3 is_executable_ptr	154
6.89.2.4 monotonic_ns	154
6.89.2.5 mutex_destroy	154
6.89.2.6 mutex_init	154
6.89.2.7 mutex_lock	154
6.89.2.8 mutex_unlock	154
6.89.2.9 owns_xt_entry	154
6.89.2.10 panic	154

6.89.2.11 parity_mode	154
6.89.2.12 putc	154
6.89.2.13 puts	154
6.89.2.14 verbose	154
6.90 vmm_page_info_t Struct Reference	155
6.90.1 Field Documentation	155
6.90.1.1 executable	155
6.90.1.2 present	155
6.90.1.3 writable	155
6.91 VMRegistryEntry Struct Reference	155
6.91.1 Field Documentation	155
6.91.1.1 birth_capsule_id	155
6.91.1.2 birth_dict_hash	155
6.91.1.3 birth_timestamp_ns	155
6.91.1.4 flags	156
6.91.1.5 reserved	156
6.91.1.6 state	156
6.91.1.7 vm_id	156
6.92 WordContract Struct Reference	156
6.92.1 Field Documentation	156
6.92.1.1 flags	156
6.92.1.2 fuzz_rounds	156
6.93 WordProfile Struct Reference	156
6.93.1 Detailed Description	157
6.93.2 Field Documentation	157
6.93.2.1 avg_time	157
6.93.2.2 call_count	157
6.93.2.3 max_time	157
6.93.2.4 min_time	157
6.93.2.5 percentage	157
6.93.2.6 total_time	157
6.93.2.7 word_name	157
6.94 WordStats Struct Reference	157
6.94.1 Detailed Description	158
6.94.2 Field Documentation	158
6.94.2.1 call_count	158
6.94.2.2 entry	158
6.94.2.3 max_time_ns	158
6.94.2.4 min_time_ns	158
6.94.2.5 total_time_ns	158
6.95 WordTestSuite Struct Reference	158
6.95.1 Detailed Description	159

6.95.2 Field Documentation	159
6.95.2.1 suite_contract	159
6.95.2.2 test_count	159
6.95.2.3 tests	159
6.95.2.4 word_name	159
6.96 WordTransitionMetrics Struct Reference	159
6.96.1 Field Documentation	160
6.96.1.1 actual_window_size	160
6.96.1.2 context_transitions	160
6.96.1.3 context_window	160
6.96.1.4 context_window_pos	161
6.96.1.5 max_transition_probability_q48	161
6.96.1.6 misprediction_cost_q48	161
6.96.1.7 most_likely_next_word_id	161
6.96.1.8 prefetch_attempts	161
6.96.1.9 prefetch_hits	161
6.96.1.10 prefetch_latency_saved_q48	161
6.96.1.11 prefetch_misses	161
6.96.1.12 total_context_transitions	162
6.96.1.13 total_transitions	162
6.96.1.14 transition_heat	162
6.97 XXHash64State Struct Reference	162
6.97.1 Detailed Description	162
6.97.2 Field Documentation	162
6.97.2.1 buffer	162
6.97.2.2 buffer_size	163
6.97.2.3 seed	163
6.97.2.4 total_len	163
6.97.2.5 v1	163
6.97.2.6 v2	163
6.97.2.7 v3	163
6.97.2.8 v4	163
7 File Documentation	165
7.1 include/arch_detect.h File Reference	165
7.1.1 Detailed Description	166
7.1.2 Macro Definition Documentation	166
7.1.2.1 ARCH_ARM64	166
7.1.2.2 ARCH_NAME	166
7.1.2.3 ARCH_RISCV64	166
7.1.2.4 ARCH_X86_64	166
7.1.2.5 NEXT_OPT	166

7.1.2.6 PRIM_BRANCH_OPT	166
7.1.2.7 PRIM_DROP_OPT	166
7.1.2.8 PRIM_DUP_OPT	166
7.1.2.9 PRIM_FETCH_OPT	166
7.1.2.10 PRIM_MINUS_OPT	167
7.1.2.11 PRIM_PLUS_OPT	167
7.1.2.12 PRIM_R_FETCH_OPT	167
7.1.2.13 PRIM_R_FROM_OPT	167
7.1.2.14 PRIM_STAR_OPT	167
7.1.2.15 PRIM_STORE_OPT	167
7.1.2.16 PRIM_SWAP_OPT	167
7.1.2.17 PRIM_TO_R_OPT	167
7.1.2.18 PRIM_ZBRANCH_OPT	167
7.1.2.19 vm_abs_opt	167
7.1.2.20 vm_add_overflow_opt	167
7.1.2.21 vm_clz_opt	167
7.1.2.22 vm_ctz_opt	167
7.1.2.23 vm_divmod_opt	168
7.1.2.24 vm_max_opt	168
7.1.2.25 vm_memcpy_opt	168
7.1.2.26 vm_memzero_opt	168
7.1.2.27 vm_min_opt	168
7.1.2.28 vm_mul_double_opt	168
7.1.2.29 vm_pop_opt	168
7.1.2.30 vm_popcnt_opt	168
7.1.2.31 vm_prefetch_opt	168
7.1.2.32 vm_prefetch_stream_opt	168
7.1.2.33 vm_push_opt	169
7.1.2.34 vm_rpop_opt	169
7.1.2.35 vm_rpush_opt	169
7.1.2.36 vm_save_registers_opt	169
7.1.2.37 vm_setup_registers_opt	169
7.1.2.38 vm_strcmp_opt	169
7.1.2.39 vm_sub_overflow_opt	169
7.1.3 Function Documentation	169
7.1.3.1 vm_get_arch_name()	169
7.1.3.2 vm_get_build_info()	169
7.1.3.3 vm_has_asm_opt()	170
7.1.3.4 vm_has_direct_threading()	170
7.1.3.5 vm_has_sse42_opt()	170
7.2 include/blkcfg.h File Reference	170
7.2.1 Macro Definition Documentation	170

7.2.1.1 BLKCFG_DEFAULT_FBS	170
7.2.1.2 BLKCFG_PATH_MAX	171
7.2.2 Enumeration Type Documentation	171
7.2.2.1 blkcfg_backend_t	171
7.2.3 Function Documentation	171
7.2.3.1 blkcfg_format_summary()	171
7.2.3.2 blkcfg_init_from_options()	171
7.2.4 Variable Documentation	171
7.2.4.1 g_blkcfg	171
7.3 include/blkio.h File Reference	172
7.3.1 Macro Definition Documentation	172
7.3.1.1 BLKIO_FORTH_BLOCK_SIZE	172
7.3.2 Typedef Documentation	172
7.3.2.1 blkio_dev_t	172
7.3.3 Enumeration Type Documentation	172
7.3.3.1 anonymous enum	172
7.3.4 Function Documentation	173
7.3.4.1 blkio_bzero()	173
7.3.4.2 blkio_close()	173
7.3.4.3 blkio_flush()	173
7.3.4.4 blkio_info()	173
7.3.4.5 blkio_open()	173
7.3.4.6 blkio_read()	173
7.3.4.7 blkio_write()	174
7.4 include/blkio_factory.h File Reference	174
7.4.1 Function Documentation	174
7.4.1.1 blkio_factory_open()	174
7.5 include/block_subsystem.h File Reference	175
7.5.1 Macro Definition Documentation	176
7.5.1.1 BLK_DEVICE_SECTOR	176
7.5.1.2 BLK_DISK_START	176
7.5.1.3 BLK_DISK_SYS_RESERVED	176
7.5.1.4 BLK_FORTH_SIZE	176
7.5.1.5 BLK_FORTH_SYS_RESERVED	176
7.5.1.6 BLK_META_PER_BLOCK	176
7.5.1.7 BLK_META_TOTAL	176
7.5.1.8 BLK_PACK_RATIO	176
7.5.1.9 BLK_RAM_BLOCKS	176
7.5.2 Enumeration Type Documentation	176
7.5.2.1 anonymous enum	176
7.5.3 Function Documentation	177
7.5.3.1 blk_allocate()	177

7.5.3.2 blk_flush()	177
7.5.3.3 blk_get_buffer()	177
7.5.3.4 blk_get_empty_buffer()	177
7.5.3.5 blk_get_meta()	177
7.5.3.6 blk_get_total_blocks()	178
7.5.3.7 blk_get_volume_meta()	178
7.5.3.8 blk_is_allocated()	178
7.5.3.9 blk_is_valid()	178
7.5.3.10 blk_mark_allocated()	178
7.5.3.11 blk_mark_free()	178
7.5.3.12 blk_set_meta()	178
7.5.3.13 blk_set_volume_meta()	179
7.5.3.14 blk_subsys_attach_device()	179
7.5.3.15 blk_subsys_init()	179
7.5.3.16 blk_subsys_shutdown()	179
7.5.3.17 blk_update()	179
7.6 include/cli.h File Reference	179
7.6.1 Enumeration Type Documentation	180
7.6.1.1 HeartbeatLogMode	180
7.6.2 Function Documentation	180
7.6.2.1 cli_parse()	180
7.6.2.2 cli_print_usage()	180
7.7 include/dictionary_heat_optimization.h File Reference	181
7.7.1 Macro Definition Documentation	181
7.7.1.1 SF_FC_BUCKETS	181
7.7.2 Function Documentation	181
7.7.2.1 dict_adaptive_optimization_pass()	181
7.7.2.2 dict_find_word_heat_aware()	182
7.7.2.3 dict_reorganize_buckets_by_heat()	182
7.7.2.4 dict_update_heat_percentiles()	182
7.7.3 Variable Documentation	182
7.7.3.1 sf_fc_cap	182
7.7.3.2 sf_fc_count	182
7.7.3.3 sf_fc_list	182
7.8 include/dictionary_management.h File Reference	183
7.8.1 Function Documentation	183
7.8.1.1 dictionary_clear()	183
7.8.1.2 dictionary_init()	183
7.8.1.3 dictionary_lookup()	183
7.8.1.4 dictionary_register()	184
7.9 include/doe_metrics.h File Reference	184
7.9.1 Detailed Description	185

7.9.2 Function Documentation	185
7.9.2.1 metrics_from_vm()	185
7.9.2.2 metrics_get_cpu_freq_mhz()	186
7.9.2.3 metrics_get_cpu_temp_c()	186
7.9.2.4 metrics_get_timestamp()	186
7.9.2.5 metrics_print_text()	186
7.9.2.6 metrics_write_csv_header()	187
7.9.2.7 metrics_write_csv_row()	187
7.9.2.8 metrics_write_james_law_csv_row()	187
7.10 include/inference_engine.h File Reference	188
7.10.1 Function Documentation	189
7.10.1.1 compute_variance_q48()	189
7.10.1.2 find_variance_inflection()	189
7.10.1.3 infer_decay_slope_q48()	189
7.10.1.4 inference_engine_run()	190
7.10.1.5 inference_outputs_to_string()	191
7.10.1.6 inference_outputs_validate()	191
7.11 include/io.h File Reference	191
7.11.1 Macro Definition Documentation	192
7.11.1.1 BLOCK_COUNT	192
7.11.1.2 MEMORY_SIZE	192
7.11.2 Function Documentation	192
7.11.2.1 io_init()	192
7.11.2.2 io_read_block()	192
7.11.2.3 io_write_block()	193
7.12 include/log.h File Reference	194
7.12.1 Macro Definition Documentation	194
7.12.1.1 LOG_LINE_MAX	194
7.12.2 Enumeration Type Documentation	194
7.12.2.1 LogLevel	194
7.12.2.2 TestResult	195
7.12.3 Function Documentation	195
7.12.3.1 log_get_level()	195
7.12.3.2 log_message()	195
7.12.3.3 log_set_level()	197
7.12.3.4 log_set_vm()	197
7.12.3.5 log_test_result()	197
7.13 include/math_portable.h File Reference	198
7.13.1 Macro Definition Documentation	198
7.13.1.1 Q48_SCALE	198
7.13.2 Function Documentation	198
7.13.2.1 erf_q48()	198

7.13.2.2 <code>sqrt_q48()</code>	198
7.14 <code>include/memory_management.h</code> File Reference	199
7.14.1 Detailed Description	199
7.14.2 Function Documentation	199
7.14.2.1 <code>vm_align()</code>	199
7.14.2.2 <code>vm_allot()</code>	200
7.15 <code>include/physics_execution_hooks.h</code> File Reference	200
7.15.1 Function Documentation	201
7.15.1.1 <code>physics_on_lookup()</code>	201
7.15.1.2 <code>physics_post_execute()</code>	201
7.15.1.3 <code>physics_pre_execute()</code>	201
7.16 <code>include/physics_hotwords_cache.h</code> File Reference	201
7.16.1 Macro Definition Documentation	202
7.16.1.1 <code>HOTWORDS_CACHE_SIZE</code>	202
7.16.1.2 <code>HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD</code>	202
7.16.1.3 <code>HOTWORDS_EXECUTION_HEAT_THRESHOLD</code>	203
7.16.2 Typedef Documentation	203
7.16.2.1 <code>HotwordsCache</code>	203
7.16.3 Function Documentation	203
7.16.3.1 <code>hotwords_bayesian_report()</code>	203
7.16.3.2 <code>hotwords_bucket_reorder()</code>	203
7.16.3.3 <code>hotwords_cache_cleanup()</code>	203
7.16.3.4 <code>hotwords_cache_init()</code>	203
7.16.3.5 <code>hotwords_cache_lookup()</code>	203
7.16.3.6 <code>hotwords_cache_promote()</code>	204
7.16.3.7 <code>hotwords_cache_set_enabled()</code>	204
7.16.3.8 <code>hotwords_posterior_bucket_searches()</code>	204
7.16.3.9 <code>hotwords_posterior_cache_hits()</code>	204
7.16.3.10 <code>hotwords_speedup_estimate()</code>	204
7.16.3.11 <code>hotwords_stats_print()</code>	205
7.16.3.12 <code>hotwords_stats_reset()</code>	205
7.17 <code>include/physics_metadata.h</code> File Reference	205
7.17.1 Function Documentation	206
7.17.1.1 <code>physics_decay_slope_load()</code>	206
7.17.1.2 <code>physics_execution_heat_increment()</code>	206
7.17.1.3 <code>physics_execution_heat_load()</code>	206
7.17.1.4 <code>physics_metadata_apply_linear_decay()</code>	206
7.17.1.5 <code>physics_metadata_apply_seed()</code>	206
7.17.1.6 <code>physics_metadata_init()</code>	206
7.17.1.7 <code>physics_metadata_record_latency()</code>	206
7.17.1.8 <code>physics_metadata_refresh_state()</code>	207
7.17.1.9 <code>physics_metadata_set_mass()</code>	207

7.17.1.10 physics_metadata_touch()	207
7.18 include/physics_pipelining_metrics.h File Reference	207
7.18.1 Macro Definition Documentation	208
7.18.1.1 ENABLE_PIPELINING	208
7.18.1.2 MIN_SAMPLES_FOR_SPECULATION	208
7.18.1.3 MINIMUM_PREFETCH_ROI	209
7.18.1.4 MISPREDICTION_COST_Q48	209
7.18.1.5 SPECULATION_DEPTH	209
7.18.1.6 SPECULATION_THRESHOLD_Q48	209
7.18.1.7 TRANSITION_WINDOW_SIZE	209
7.18.2 Typedef Documentation	209
7.18.2.1 WordTransitionMetrics	209
7.18.3 Function Documentation	209
7.18.3.1 transition_metrics_binary_chop_suggest_window()	209
7.18.3.2 transition_metrics_cleanup()	210
7.18.3.3 transition_metrics_context_accuracy_string()	210
7.18.3.4 transition_metrics_get_probability_q48()	210
7.18.3.5 transition_metrics_init()	211
7.18.3.6 transition_metrics_record()	211
7.18.3.7 transition_metrics_record_context()	212
7.18.3.8 transition_metrics_record_prefetch_hit()	212
7.18.3.9 transition_metrics_record_prefetch_miss()	213
7.18.3.10 transition_metrics_reset()	213
7.18.3.11 transition_metrics_should_speculate()	213
7.18.3.12 transition_metrics_stats_string()	214
7.18.3.13 transition_metrics_update_cache()	214
7.18.3.14 transition_metrics_update_context_window()	214
7.19 include/physics_runtime.h File Reference	215
7.19.1 Macro Definition Documentation	216
7.19.1.1 HOLA_PROTOCOL_MAJOR	216
7.19.1.2 HOLA_PROTOCOL_MINOR	216
7.19.1.3 HOLA_SHARED_MAGIC	216
7.19.1.4 PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT	216
7.19.1.5 PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES	216
7.19.1.6 PHYSICS_HOST_FLAG_CGROUP	216
7.19.1.7 PHYSICS_HOST_FLAG_CPU_STAT	216
7.19.1.8 PHYSICS_HOST_FLAG_PSI	216
7.19.2 Typedef Documentation	216
7.19.2.1 physics_analytics_event_header_t	216
7.19.2.2 physics_analytics_header_t	217
7.19.2.3 physics_analytics_heap_t	217
7.19.2.4 physics_host_backend_t	217

7.19.2.5 physics_host_snapshot_t	217
7.19.3 Enumeration Type Documentation	217
7.19.3.1 physics_host_backend	217
7.19.4 Function Documentation	217
7.19.4.1 physics_analytics_heap_info()	217
7.19.4.2 physics_analytics_publish_event()	217
7.19.4.3 physics_host_snapshot()	218
7.19.4.4 physics_runtime_init()	218
7.19.4.5 physics_runtime_shutdown()	218
7.20 include/platform_alloc.h File Reference	219
7.20.1 Typedef Documentation	219
7.20.1.1 sf_alloc_stats_t	219
7.20.2 Function Documentation	219
7.20.2.1 sf_alloc_get_stats()	219
7.20.2.2 sf_alloc_init()	219
7.20.2.3 sf_calloc()	219
7.20.2.4 sf_free()	220
7.20.2.5 sf_malloc()	220
7.20.2.6 sf_realloc()	220
7.21 include/platform_lock.h File Reference	220
7.21.1 Typedef Documentation	220
7.21.1.1 sf_mutex_t	220
7.21.2 Function Documentation	221
7.21.2.1 sf_mutex_destroy()	221
7.21.2.2 sf_mutex_init()	221
7.21.2.3 sf_mutex_lock()	221
7.21.2.4 sf_mutex_unlock()	221
7.22 include/platform_time.h File Reference	221
7.22.1 Macro Definition Documentation	222
7.22.1.1 SF_TIME_STAMP_SIZE	222
7.22.2 Typedef Documentation	222
7.22.2.1 sf_time_ns_t	222
7.22.3 Function Documentation	222
7.22.3.1 sf_format_timestamp()	222
7.22.3.2 sf_has_rtc()	222
7.22.3.3 sf_monotonic_ns()	222
7.22.3.4 sf_ns_to_ms()	223
7.22.3.5 sf_ns_to_seconds()	223
7.22.3.6 sf_ns_to_us()	223
7.22.3.7 sf_realtime_ns()	223
7.22.3.8 sf_seconds_to_ns()	223
7.22.3.9 sf_set_realtime_ns()	223

7.22.3.10 sf_time_init()	223
7.22.4 Variable Documentation	223
7.22.4.1 sf_time_backend	223
7.22.4.2 sf_time_backend_l4re	224
7.22.4.3 sf_time_backend_posix	224
7.23 include/profiler.h File Reference	224
7.23.1 Macro Definition Documentation	225
7.23.1.1 PROFILE_ENABLED	225
7.23.1.2 PROFILE_END	225
7.23.1.3 PROFILE_INC_CYCLES	225
7.23.1.4 PROFILE_INC_DICT_LOOKUP	226
7.23.1.5 PROFILE_INC_STACK_OP	226
7.23.1.6 PROFILE_MEMORY_READ	226
7.23.1.7 PROFILE_MEMORY_WRITE	226
7.23.1.8 PROFILE_START	226
7.23.1.9 PROFILE_WORD_ENTER	226
7.23.1.10 PROFILE_WORD_EXIT	226
7.23.2 Typedef Documentation	226
7.23.2.1 DictEntry	226
7.23.2.2 VM	226
7.23.3 Enumeration Type Documentation	226
7.23.3.1 ProfileCategory	226
7.23.3.2 ProfileLevel	227
7.23.4 Function Documentation	227
7.23.4.1 profiler_analyze_performance()	227
7.23.4.2 profiler_end_timer()	227
7.23.4.3 profiler_generate_report()	227
7.23.4.4 profiler_get_time_ns()	228
7.23.4.5 profiler_inc_branch()	228
7.23.4.6 profiler_inc_dict_lookup()	228
7.23.4.7 profiler_inc_loop_iter()	228
7.23.4.8 profiler_inc_stack_op()	228
7.23.4.9 profiler_inc_vm_cycles()	228
7.23.4.10 profiler_init()	228
7.23.4.11 profiler_memory_alloc()	229
7.23.4.12 profiler_memory_read()	229
7.23.4.13 profiler_memory_write()	229
7.23.4.14 profiler_print_hotspots()	229
7.23.4.15 profiler_print_memory_analysis()	229
7.23.4.16 profiler_print_summary()	229
7.23.4.17 profiler_print_word_analysis()	229
7.23.4.18 profiler_reset()	229

7.23.4.19 profiler_set_level()	230
7.23.4.20 profiler_shutdown()	230
7.23.4.21 profiler_start_timer()	230
7.23.4.22 profiler_suggest_optimizations()	230
7.23.4.23 profiler_word_count()	230
7.23.4.24 profiler_word_enter()	230
7.23.4.25 profiler_word_exit()	231
7.23.5 Variable Documentation	231
7.23.5.1 g_profiler	231
7.24 include/rolling_window_knobs.h File Reference	231
7.24.1 Macro Definition Documentation	232
7.24.1.1 ADAPTIVE_CHECK_FREQUENCY	232
7.24.1.2 ADAPTIVE_GROWTH_THRESHOLD	232
7.24.1.3 ADAPTIVE_MIN_WINDOW_SIZE	232
7.24.1.4 ADAPTIVE_SHRINK_RATE	232
7.25 include/rolling_window_of_truth.h File Reference	232
7.25.1 Function Documentation	233
7.25.1.1 rolling_window_check_adaptive_shrink()	233
7.25.1.2 rolling_window_cleanup()	233
7.25.1.3 rolling_window_count_transition()	233
7.25.1.4 rolling_window_export_execution_history()	234
7.25.1.5 rolling_window_find_hottest_word()	234
7.25.1.6 rolling_window_get_recent_sequence()	234
7.25.1.7 rolling_window_init()	235
7.25.1.8 rolling_window_is_warm()	235
7.25.1.9 rolling_window_measure_diversity()	236
7.25.1.10 rolling_window_pattern_capture_rate()	236
7.25.1.11 rolling_window_record_execution()	237
7.25.1.12 rolling_window_reset()	237
7.25.1.13 rolling_window_service()	238
7.25.1.14 rolling_window_stats_string()	238
7.26 include/ssm_jacquard.h File Reference	238
7.26.1 Macro Definition Documentation	239
7.26.1.1 SSM_CV_HIGH_THRESHOLD	239
7.26.1.2 SSM_ENTROPY_HIGH_THRESHOLD	239
7.26.1.3 SSM_HYSTERESIS_TICKS	239
7.26.1.4 SSM_TEMPORAL_DECAY_LOW_THRESHOLD	239
7.26.1.5 SSM_TEMPORAL_DECAY_THRESHOLD	239
7.26.2 Enumeration Type Documentation	239
7.26.2.1 ssm_l8_mode_t	239
7.26.3 Function Documentation	240
7.26.3.1 ssm_apply_mode()	240

7.26.3.2 ssm_l8_init()	240
7.26.3.3 ssm_l8_mode_name()	240
7.26.3.4 ssm_l8_update()	240
7.27 include/starforth_config.h File Reference	241
7.27.1 Macro Definition Documentation	242
7.27.1.1 ADAPTIVE_CHECK_FREQUENCY	242
7.27.1.2 ADAPTIVE_GROWTH_THRESHOLD	242
7.27.1.3 ADAPTIVE_MIN_WINDOW_SIZE	242
7.27.1.4 ADAPTIVE_SHRINK_RATE	242
7.27.1.5 DECAY_MIN_INTERVAL	242
7.27.1.6 DECAY_RATE_PER_US_Q16	242
7.27.1.7 ENABLE_HOTWORDS_CACHE	242
7.27.1.8 ENABLE_PIPELINING	242
7.27.1.9 HEARTBEAT_CHECK_FREQUENCY	242
7.27.1.10 HEARTBEAT_INFERENCE_FREQUENCY	242
7.27.1.11 HEARTBEAT_SLOPE_VALIDATION_FREQUENCY	242
7.27.1.12 HEARTBEAT_THREAD_ENABLED	242
7.27.1.13 HEARTBEAT_TICK_NS	242
7.27.1.14 HEARTBEAT_WINDOW_TUNING_FREQUENCY	243
7.27.1.15 INITIAL_DECAY_SLOPE_Q48	243
7.27.1.16 ROLLING_WINDOW_SIZE	243
7.27.1.17 STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT	243
7.27.1.18 STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT	243
7.27.1.19 STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT	243
7.27.1.20 STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT	243
7.27.1.21 STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT	243
7.27.1.22 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT	243
7.27.1.23 STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT	243
7.27.1.24 STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT	243
7.27.1.25 STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT	243
7.27.1.26 STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT	243
7.27.1.27 STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT	243
7.27.1.28 STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT	243
7.27.1.29 STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT	244
7.27.1.30 STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT	244
7.27.1.31 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT	244
7.27.1.32 STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT	244
7.27.1.33 STARFORTH_CONFIG_STRICT_PTR_DEFAULT	244
7.27.1.34 STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT	244
7.27.1.35 STRICT_PTR	244
7.27.1.36 TRANSITION_WINDOW_SIZE	244
7.27.1.37 VM_STRICT_PTR	244

7.28 include/starkernel/apic.h File Reference	244
7.28.1 Macro Definition Documentation	245
7.28.1.1 APIC_TIMER_VECTOR	245
7.28.2 Function Documentation	245
7.28.2.1 apic_eoi()	245
7.28.2.2 apic_init()	245
7.28.2.3 apic_timer_init()	245
7.28.2.4 apic_timer_period_tsc()	245
7.28.2.5 apic_timer_start()	245
7.28.2.6 apic_timer_stop()	246
7.29 include/starkernel/arch.h File Reference	246
7.29.1 Function Documentation	246
7.29.1.1 arch_disable_interrupts()	246
7.29.1.2 arch_early_init()	246
7.29.1.3 arch_enable_interrupts()	246
7.29.1.4 arch_halt()	247
7.29.1.5 arch_interrupts_init()	247
7.29.1.6 arch_mmu_init()	247
7.29.1.7 arch_read_timestamp()	247
7.29.1.8 arch_relax()	247
7.30 include/starkernel/capsule.h File Reference	247
7.30.1 Macro Definition Documentation	249
7.30.1.1 CAPSULE_BIRTH_ELIGIBLE	249
7.30.1.2 CAPSULE_DESC_MAGIC	249
7.30.1.3 CAPSULE_DIR_MAGIC	249
7.30.1.4 CAPSULE_DOE_ELIGIBLE	249
7.30.1.5 CAPSULE_FLAG_ACTIVE	249
7.30.1.6 CAPSULE_FLAG_DEPRECATED	249
7.30.1.7 CAPSULE_FLAG_EXPERIMENT	249
7.30.1.8 CAPSULE_FLAG_MAMA_INIT	249
7.30.1.9 CAPSULE_FLAG_PINNED	250
7.30.1.10 CAPSULE_FLAG_PRODUCTION	250
7.30.1.11 CAPSULE_FLAG_REVOKED	250
7.30.1.12 CAPSULE_IS_MAMA_INIT	250
7.30.1.13 CAPSULE_MAGIC_GET_HASHALG	250
7.30.1.14 CAPSULE_MAGIC_GET_SIG	250
7.30.1.15 CAPSULE_MAGIC_GET_VERSION	250
7.30.1.16 CAPSULE_MAGIC_PACK	250
7.30.1.17 CAPSULE_MAX_COUNT	250
7.30.1.18 CAPSULE_MODE_VALID	250
7.30.1.19 CAPSULE_NAME_MAX	250
7.30.1.20 CAPSULE_VERSION_0	251

7.30.2 Enumeration Type Documentation	251
7.30.2.1 CapsuleHashAlg	251
7.30.2.2 CapsuleValidateResult	251
7.30.3 Function Documentation	251
7.30.3.1 __attribute__()	251
7.30.3.2 capsule_find_by_id()	251
7.30.3.3 capsule_find_by_name()	252
7.30.3.4 capsule_find_mama_init()	252
7.30.3.5 capsule_get_payload()	252
7.30.3.6 capsule_validate()	253
7.30.3.7 capsule_validate_result_str()	253
7.30.4 Variable Documentation	253
7.30.4.1 CapsuleDesc	253
7.31 include/starkernel/capsule_birth.h File Reference	253
7.31.1 Typedef Documentation	254
7.31.1.1 CapsuleDictHashFn	254
7.31.1.2 CapsuleExecFn	254
7.31.1.3 CapsuleVMAllocFn	255
7.31.2 Function Documentation	255
7.31.2.1 capsule_birth_baby()	255
7.31.2.2 capsule_birth_mama()	256
7.31.2.3 capsule_birth_set_hooks()	256
7.31.2.4 capsule_run_experiment()	257
7.31.2.5 capsule_vm_hooks_register()	257
7.31.2.6 capsule_vm_registry_count()	257
7.31.2.7 capsule_vm_registry_get()	258
7.31.2.8 capsule_vm_registry_init()	258
7.32 include/starkernel/capsule_generated.h File Reference	258
7.32.1 Function Documentation	258
7.32.1.1 __attribute__()	258
7.33 include/starkernel/capsule_loader.h File Reference	258
7.33.1 Function Documentation	259
7.33.1.1 capsule_clear_blocks()	259
7.33.1.2 capsule_exec_init()	259
7.33.1.3 capsule_load_blocks()	260
7.34 include/starkernel/capsule_run.h File Reference	260
7.34.1 Macro Definition Documentation	261
7.34.1.1 CAPSULE_MAX_RUN_RECORDS	261
7.34.2 Enumeration Type Documentation	261
7.34.2.1 CapsuleRunResult	261
7.34.2.2 VMState	262
7.34.3 Function Documentation	262

7.34.3.1 capsule_parity_log_birth()	262
7.34.3.2 capsule_parity_log_birth_failed()	262
7.34.3.3 capsule_parity_log_mama_init()	262
7.34.3.4 capsule_parity_log_run()	263
7.34.3.5 capsule_parity_set_output()	263
7.34.3.6 capsule_run_log_count()	263
7.34.3.7 capsule_run_log_get()	263
7.34.3.8 capsule_run_log_init()	264
7.34.3.9 capsule_run_log_record()	264
7.35 include/starkernel/console.h File Reference	264
7.35.1 Function Documentation	265
7.35.1.1 console_getc()	265
7.35.1.2 console_init()	265
7.35.1.3 console_poll()	265
7.35.1.4 console_println()	265
7.35.1.5 console_putc()	265
7.35.1.6 console_puts()	265
7.36 include/starkernel/elf64.h File Reference	266
7.36.1 Macro Definition Documentation	267
7.36.1.1 EI_ABIVERSION	267
7.36.1.2 EI_CLASS	267
7.36.1.3 EI_DATA	268
7.36.1.4 EI_MAG0	268
7.36.1.5 EI_MAG1	268
7.36.1.6 EI_MAG2	268
7.36.1.7 EI_MAG3	268
7.36.1.8 EI_NIDENT	268
7.36.1.9 EI_OSABI	268
7.36.1.10 EI_PAD	268
7.36.1.11 EI_VERSION	268
7.36.1.12 ELF64_R_INFO	268
7.36.1.13 ELF64_R_SYM	268
7.36.1.14 ELF64_R_TYPE	268
7.36.1.15 ELFCLASS32	268
7.36.1.16 ELFCLASS64	268
7.36.1.17 ELFDATA2LSB	269
7.36.1.18 ELFDATA2MSB	269
7.36.1.19 ELFMAG0	269
7.36.1.20 ELFMAG1	269
7.36.1.21 ELFMAG2	269
7.36.1.22 ELFMAG3	269
7.36.1.23 ELFOSABI_NONE	269

7.36.1.24 EM_AARCH64	269
7.36.1.25 EM_RISCV	269
7.36.1.26 EM_X86_64	269
7.36.1.27 ET_CORE	269
7.36.1.28 ET_DYN	269
7.36.1.29 ET_EXEC	269
7.36.1.30 ET_NONE	269
7.36.1.31 ET_REL	269
7.36.1.32 EV_CURRENT	270
7.36.1.33 PF_R	270
7.36.1.34 PF_W	270
7.36.1.35 PF_X	270
7.36.1.36 PT_DYNAMIC	270
7.36.1.37 PT_INTERP	270
7.36.1.38 PT_LOAD	270
7.36.1.39 PT_NOTE	270
7.36.1.40 PT_NULL	270
7.36.1.41 PT_PHDR	270
7.36.1.42 PT_SHLIB	270
7.36.1.43 PT_TLS	270
7.36.1.44 R_AARCH64_ABS64	270
7.36.1.45 R_AARCH64_NONE	270
7.36.1.46 R_AARCH64_RELATIVE	270
7.36.1.47 R_RISCV_64	271
7.36.1.48 R_RISCV_NONE	271
7.36.1.49 R_RISCV_RELATIVE	271
7.36.1.50 R_X86_64_32	271
7.36.1.51 R_X86_64_32S	271
7.36.1.52 R_X86_64_64	271
7.36.1.53 R_X86_64_NONE	271
7.36.1.54 R_X86_64_PC32	271
7.36.1.55 R_X86_64_RELATIVE	271
7.36.1.56 SHT_DYNAMIC	271
7.36.1.57 SHT_DYNSYM	271
7.36.1.58 SHT_HASH	271
7.36.1.59 SHT_NOBITS	271
7.36.1.60 SHT_NOTE	271
7.36.1.61 SHT_NULL	271
7.36.1.62 SHT_PROGBITS	272
7.36.1.63 SHT_REL	272
7.36.1.64 SHT_RELA	272
7.36.1.65 SHT_SHLIB	272

7.36.1.66 SHT_STRTAB	272
7.36.1.67 SHT_SYMTAB	272
7.36.2 Typedef Documentation	272
7.36.2.1 Elf64_Addr	272
7.36.2.2 Elf64_Half	272
7.36.2.3 Elf64_Off	272
7.36.2.4 Elf64_Sword	272
7.36.2.5 Elf64_Sxword	272
7.36.2.6 Elf64_Word	272
7.36.2.7 Elf64_Xword	272
7.37 include/starkernel/elf_loader.h File Reference	272
7.37.1 Function Documentation	273
7.37.1.1 elf_load_kernel()	273
7.38 include/starkernel/hal/hal.h File Reference	273
7.39 include/starkernel/hal_memory.h File Reference	273
7.39.1 Function Documentation	274
7.39.1.1 hal_mem_alloc()	274
7.39.1.2 hal_mem_alloc_aligned()	274
7.39.1.3 hal_mem_free()	274
7.40 include/starkernel/kmalloc.h File Reference	274
7.40.1 Function Documentation	274
7.40.1.1 kfree()	274
7.40.1.2 kmalloc()	275
7.40.1.3 kmalloc_aligned()	275
7.40.1.4 kmalloc_get_stats()	275
7.40.1.5 kmalloc_heap_base_addr()	275
7.40.1.6 kmalloc_heap_end_addr()	275
7.40.1.7 kmalloc_init()	275
7.40.1.8 kmalloc_is_initialized()	275
7.41 include/starkernel/pmm.h File Reference	275
7.41.1 Macro Definition Documentation	276
7.41.1.1 PMM_PAGE_SIZE	276
7.41.2 Function Documentation	276
7.41.2.1 pmm_alloc_contiguous()	276
7.41.2.2 pmm_alloc_page()	276
7.41.2.3 pmm_free_contiguous()	276
7.41.2.4 pmm_free_page()	277
7.41.2.5 pmm_get_stats()	277
7.41.2.6 pmm_init()	277
7.41.2.7 pmm_is_initialized()	277
7.42 include/q48_16.h File Reference	277
7.42.1 Typedef Documentation	278

7.42.1.1 q48_16_t	278
7.42.2 Function Documentation	278
7.42.2.1 q48_abs()	278
7.42.2.2 q48_add()	278
7.42.2.3 q48_div()	279
7.42.2.4 q48_exp_approx()	279
7.42.2.5 q48_from_double()	279
7.42.2.6 q48_from_u64()	280
7.42.2.7 q48_is_valid()	280
7.42.2.8 q48_log_approx()	280
7.42.2.9 q48_mul()	281
7.42.2.10 q48_sqrt_approx()	281
7.42.2.11 q48_sub()	282
7.42.2.12 q48_to_double()	282
7.42.2.13 q48_to_string()	282
7.42.2.14 q48_to_u64()	282
7.43 include/starkernel/q48_16.h File Reference	284
7.43.1 Macro Definition Documentation	284
7.43.1.1 Q48_ONE	284
7.43.2 Typedef Documentation	285
7.43.2.1 q48_16_t	285
7.43.3 Function Documentation	285
7.43.3.1 q48_abs()	285
7.43.3.2 q48_add()	285
7.43.3.3 q48_div()	285
7.43.3.4 q48_exp_approx()	285
7.43.3.5 q48_from_u64()	286
7.43.3.6 q48_log_approx()	286
7.43.3.7 q48_mul()	286
7.43.3.8 q48_sqrt_approx()	287
7.43.3.9 q48_sub()	287
7.43.3.10 q48_to_u64()	287
7.44 include/repl.h File Reference	288
7.44.1 Function Documentation	288
7.44.1.1 vm_repl()	288
7.45 include/starkernel/repl.h File Reference	288
7.45.1 Function Documentation	289
7.45.1.1 sk_repl()	289
7.46 include/starkernel/timer.h File Reference	289
7.46.1 Macro Definition Documentation	290
7.46.1.1 TIME_TRUST_HIGH	290
7.46.1.2 TIME_TRUST_LOW	290

7.46.1.3 TIME_WINDOW_SIZE	290
7.46.2 Typedef Documentation	290
7.46.2.1 time_trust_t	290
7.46.2.2 timer_calibration_record_t	290
7.46.2.3 timer_trust_level_t	290
7.46.2.4 TimeTrustState	290
7.46.2.5 TimeWindow	290
7.46.3 Enumeration Type Documentation	291
7.46.3.1 timer_trust_level	291
7.46.4 Function Documentation	291
7.46.4.1 heartbeat_init()	291
7.46.4.2 heartbeat_state()	291
7.46.4.3 heartbeat_tick()	291
7.46.4.4 heartbeat_ticks()	291
7.46.4.5 heartbeat_trust()	291
7.46.4.6 timer_calibration_record()	292
7.46.4.7 timer_check_drift_now()	292
7.46.4.8 timer_init()	292
7.46.4.9 timer_now_ns()	292
7.46.4.10 timer_tsc_hz()	292
7.47 include/starkernel/uefi.h File Reference	292
7.47.1 Macro Definition Documentation	295
7.47.1.1 EFI_ABORTED	295
7.47.1.2 EFI_BAD_BUFFER_SIZE	295
7.47.1.3 EFI_BUFFER_TOO_SMALL	295
7.47.1.4 EFI_DEVICE_ERROR	295
7.47.1.5 EFI_FILE_MODE_READ	295
7.47.1.6 EFI_INVALID_PARAMETER	295
7.47.1.7 EFI_LOAD_ERROR	295
7.47.1.8 EFI_MEMORY_RP	295
7.47.1.9 EFI_MEMORY_RUNTIME	295
7.47.1.10 EFI_MEMORY_UC	295
7.47.1.11 EFI_MEMORY_UCE	295
7.47.1.12 EFI_MEMORY_WB	295
7.47.1.13 EFI_MEMORY_WC	295
7.47.1.14 EFI_MEMORY_WP	296
7.47.1.15 EFI_MEMORY_WT	296
7.47.1.16 EFI_MEMORY_XP	296
7.47.1.17 EFI_NOT_FOUND	296
7.47.1.18 EFI_NOT_READY	296
7.47.1.19 EFI_OUT_OF_RESOURCES	296
7.47.1.20 EFI_SUCCESS	296

7.47.1.21 EFI_UNSUPPORTED	296
7.47.1.22 EFI_WRITE_PROTECTED	296
7.47.1.23 EFI_API	296
7.47.1.24 FALSE	296
7.47.1.25 TPL_APPLICATION	296
7.47.1.26 TPL_CALLBACK	296
7.47.1.27 TPL_HIGH_LEVEL	296
7.47.1.28 TPL_NOTIFY	296
7.47.1.29 TRUE	297
7.47.2 Typedef Documentation	297
7.47.2.1 BOOLEAN	297
7.47.2.2 CHAR16	297
7.47.2.3 EFI_ALLOCATE_PAGES	297
7.47.2.4 EFI_ALLOCATE_POOL	297
7.47.2.5 EFI_BOOT_SERVICES	297
7.47.2.6 EFI_EXIT_BOOT_SERVICES	297
7.47.2.7 EFI_FILE_CLOSE	297
7.47.2.8 EFI_FILE_GET_INFO	297
7.47.2.9 EFI_FILE_OPEN	297
7.47.2.10 EFI_FILE_PROTOCOL	297
7.47.2.11 EFI_FILE_READ	297
7.47.2.12 EFI_FREE_PAGES	297
7.47.2.13 EFI_FREE_POOL	297
7.47.2.14 EFI_GET_MEMORY_MAP	298
7.47.2.15 EFI_HANDLE	298
7.47.2.16 EFI_HANDLE_PROTOCOL	298
7.47.2.17 EFI_LOCATE_HANDLE	298
7.47.2.18 EFI_PHYSICAL_ADDRESS	298
7.47.2.19 EFI_RAISE_TPL	298
7.47.2.20 EFI_RESTORE_TPL	298
7.47.2.21 EFI_RUNTIME_SERVICES	298
7.47.2.22 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL	298
7.47.2.23 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME	298
7.47.2.24 EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL	298
7.47.2.25 EFI_STATUS	298
7.47.2.26 EFI_SYSTEM_TABLE	298
7.47.2.27 EFI_TEXT_RESET	298
7.47.2.28 EFI_TEXT_STRING	299
7.47.2.29 EFI_TPL	299
7.47.2.30 INT16	299
7.47.2.31 INT32	299
7.47.2.32 INT64	299

7.47.2.33 INT8	299
7.47.2.34 INTN	299
7.47.2.35 UINT16	299
7.47.2.36 UINT32	299
7.47.2.37 UINT64	299
7.47.2.38 UINT8	299
7.47.2.39 UINTN	299
7.47.2.40 VOID	299
7.47.3 Enumeration Type Documentation	299
7.47.3.1 EFI_ALLOCATE_TYPE	299
7.47.3.2 EFI_LOCATE_SEARCH_TYPE	300
7.47.3.3 EFI_MEMORY_TYPE	300
7.47.4 Variable Documentation	300
7.47.4.1 EFI_ACPI_20_TABLE_GUID	300
7.47.4.2 EFI_ACPI_TABLE_GUID	300
7.47.4.3 EFI_FILE_INFO_GUID	301
7.47.4.4 EFI_LOADED_IMAGE_PROTOCOL_GUID	301
7.47.4.5 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID	301
7.48 include/starkernel/vm/arena.h File Reference	301
7.49 include/starkernel/vm/bootstrap/sk_vm_bootstrap.h File Reference	301
7.50 include/starkernel/vm/parity.h File Reference	301
7.50.1 Macro Definition Documentation	302
7.50.1.1 FNV1A_64_OFFSET_BASIS	302
7.50.1.2 FNV1A_64_PRIME	302
7.50.1.3 SK_BOOTSTRAP_ARENA_FAIL	302
7.50.1.4 SK_BOOTSTRAP_DICT_FAIL	302
7.50.1.5 SK_BOOTSTRAP_INIT_FAIL	302
7.50.1.6 SK_BOOTSTRAP_OK	302
7.50.2 Function Documentation	302
7.50.2.1 fnv1a_64()	302
7.50.2.2 sk_dict_canonical_hash()	303
7.50.2.3 sk_dict_word_count()	303
7.50.2.4 sk_parity_collect()	304
7.50.2.5 sk_parity_print()	304
7.51 include/starkernel/vmm.h File Reference	304
7.51.1 Macro Definition Documentation	305
7.51.1.1 VMM_FLAG_NX	305
7.51.1.2 VMM_FLAG_PRESENT	305
7.51.1.3 VMM_FLAG_USER	305
7.51.1.4 VMM_FLAG_WRITABLE	305
7.51.1.5 VMM_PAGE_SIZE	305
7.51.2 Function Documentation	305

7.51.2.1 vmm_get_paddr()	305
7.51.2.2 vmm_init()	305
7.51.2.3 vmm_map_page()	306
7.51.2.4 vmm_map_range()	306
7.51.2.5 vmm_query_page()	306
7.51.2.6 vmm_unmap_page()	306
7.52 include/starkernel/xxhash64.h File Reference	306
7.52.1 Macro Definition Documentation	307
7.52.1.1 XXHASH64_CAPSULE_SEED	307
7.52.2 Function Documentation	307
7.52.2.1 xxhash64()	307
7.52.2.2 xxhash64_capsule()	307
7.52.2.3 xxhash64_digest()	308
7.52.2.4 xxhash64_reset()	308
7.52.2.5 xxhash64_update()	308
7.53 include/startup.h File Reference	309
7.53.1 Function Documentation	309
7.53.1.1 startup_blkio_cleanup()	309
7.53.1.2 startup_blkio_init()	309
7.53.1.3 startup_block_subsys_init()	309
7.53.1.4 startup_vm_init()	310
7.54 include/vm.h File Reference	310
7.54.1 Macro Definition Documentation	314
7.54.1.1 ACL_MODE_STRICT	314
7.54.1.2 ACL_MODE_TTL	314
7.54.1.3 ACL_TTL_OPEN	314
7.54.1.4 ACL_USER_DEFAULT	314
7.54.1.5 BLOCK_SIZE	314
7.54.1.6 CHARGE_NEUTRAL	314
7.54.1.7 COMPILE_BUFFER_SIZE	314
7.54.1.8 DICTIONARY_BLOCKS	314
7.54.1.9 DICTIONARY_MEMORY_SIZE	314
7.54.1.10 DICTIONARY_SIZE	314
7.54.1.11 HEARTBEAT_TICK_BUFFER_SIZE	314
7.54.1.12 HEAT_CACHE_DEMOTION_THRESHOLD	314
7.54.1.13 INPUT_BUFFER_SIZE	314
7.54.1.14 LIKELY	314
7.54.1.15 LOG_BLOCKS	314
7.54.1.16 LOG_BLOCKS_END	315
7.54.1.17 LOG_BLOCKS_START	315
7.54.1.18 LOG_LAYER1_MAX_LINES	315
7.54.1.19 LOG_LINE_MAX	315

7.54.1.20 LOG_LINES_PER_BLOCK	315
7.54.1.21 MAX_BLOCKS	315
7.54.1.22 PHYSICS_STATE_COMPILED	315
7.54.1.23 PHYSICS_STATE_HIDDEN	315
7.54.1.24 PHYSICS_STATE_IMMEDIATE	315
7.54.1.25 PHYSICS_STATE_PINNED	315
7.54.1.26 SPIN_IDLE	315
7.54.1.27 STACK_SIZE	315
7.54.1.28 UNLIKELY	315
7.54.1.29 USER_BLOCKS_START	315
7.54.1.30 VM_MEMORY_SIZE	315
7.54.1.31 VM_POP	316
7.54.1.32 VM_PUSH	316
7.54.1.33 VM_RPOP	316
7.54.1.34 VM_RPUSH	316
7.54.1.35 WORD_COMPILED	316
7.54.1.36 WORD_FROZEN	316
7.54.1.37 WORD_HIDDEN	316
7.54.1.38 WORD_ID_INVALID	316
7.54.1.39 WORD_IMMEDIATE	316
7.54.1.40 WORD_NAME_MAX	316
7.54.1.41 WORD_PINNED	316
7.54.1.42 WORD_SMUDGED	316
7.54.2 Typedef Documentation	316
7.54.2.1 cell_t	316
7.54.2.2 HeartbeatWorker	317
7.54.2.3 HotwordsCache	317
7.54.2.4 InferenceOutputs	317
7.54.2.5 ucell_t	317
7.54.2.6 vaddr_t	317
7.54.2.7 VM	317
7.54.2.8 VMHostServices	317
7.54.2.9 word_func_t	317
7.54.2.10 WordTransitionMetrics	317
7.54.3 Enumeration Type Documentation	317
7.54.3.1 vm_mode_t	317
7.54.4 Function Documentation	317
7.54.4.1 __attribute__()	317
7.54.4.2 CELL()	317
7.54.4.3 execute_colon_word()	318
7.54.4.4 heartbeat_capture_tick_snapshot()	318
7.54.4.5 heartbeat_emit_tick_row()	318

7.54.4.6 VM_ADDR()	319
7.54.4.7 vm_addr_ok()	319
7.54.4.8 vm_addr_to_block()	319
7.54.4.9 vm_align()	320
7.54.4.10 vm_allot()	320
7.54.4.11 vm_bootstrap_root_vocabulary()	321
7.54.4.12 vm_cleanup()	321
7.54.4.13 vm_compile_call()	321
7.54.4.14 vm_compile_exit()	321
7.54.4.15 vm_compile_literal()	321
7.54.4.16 vm_compile_word()	322
7.54.4.17 vm_create_word()	322
7.54.4.18 vm_dictionary_find_by_func()	322
7.54.4.19 vm_dictionary_find_latest_by_func()	323
7.54.4.20 vm_dictionary_get_data_field()	323
7.54.4.21 vm_dictionary_lookup_by_word_id()	323
7.54.4.22 vm_dictionary_track_entry()	323
7.54.4.23 vm_dictionary_untrack_entry()	323
7.54.4.24 vm_enable_interpreter()	324
7.54.4.25 vm_enter_compile_mode()	324
7.54.4.26 vm_exit_compile_mode()	324
7.54.4.27 vm_find_word()	324
7.54.4.28 vm_get_block_addr()	325
7.54.4.29 vm_heartbeat_start()	325
7.54.4.30 vm_hide_word()	325
7.54.4.31 vm_init()	325
7.54.4.32 vm_init_with_host()	326
7.54.4.33 vm_interpret()	327
7.54.4.34 vm_interpret_word()	327
7.54.4.35 vm_load_cell()	327
7.54.4.36 vm_load_u8()	328
7.54.4.37 vm_make_immediate()	328
7.54.4.38 vm_parse_number()	328
7.54.4.39 vm_parse_word()	328
7.54.4.40 vm_pin_execution_heat()	329
7.54.4.41 vm_pop()	329
7.54.4.42 vm_pop_fast()	330
7.54.4.43 vm_ptr()	330
7.54.4.44 vm_push()	330
7.54.4.45 vm_push_fast()	331
7.54.4.46 vm_repl()	331
7.54.4.47 vm_rpop()	332

7.54.4.48 vm_rpop_fast()	332
7.54.4.49 vm_rpush()	332
7.54.4.50 vm_rpush_fast()	333
7.54.4.51 vm_run_smoke_tests()	333
7.54.4.52 vm_smudge_word()	333
7.54.4.53 vm_snapshot_read()	333
7.54.4.54 vm_store_cell()	333
7.54.4.55 vm_store_u8()	333
7.54.4.56 vm_tick()	334
7.54.4.57 vm_tick_inference_engine()	334
7.54.4.58 vm_tick_slope_validator()	335
7.54.4.59 vm_tick_window_tuner()	335
7.54.4.60 vm_unpin_execution_heat()	335
7.54.5 Variable Documentation	335
7.54.5.1 __attribute__	335
7.54.5.2 acl_allow	335
7.54.5.3 acl_default	335
7.54.5.4 acl_mode	336
7.54.5.5 acl_pinned	336
7.54.5.6 acl_ttl	336
7.54.5.7 execution_heat	336
7.54.5.8 flags	336
7.54.5.9 func	336
7.54.5.10 link	336
7.54.5.11 name	336
7.54.5.12 name_len	336
7.54.5.13 physics	336
7.54.5.14 transition_metrics	336
7.54.5.15 word_id	337
7.55 include/vm_api.h File Reference	337
7.55.1 Detailed Description	337
7.56 include/vm_asm_opt.h File Reference	337
7.57 include/vm_asm_opt_arm64.h File Reference	338
7.57.1 Function Documentation	339
7.57.1.1 vm_abs_asm()	339
7.57.1.2 vm_add_check_overflow()	339
7.57.1.3 vm_clz()	339
7.57.1.4 vm_ctz()	339
7.57.1.5 vm_dc_zva()	339
7.57.1.6 vm_divmod()	340
7.57.1.7 vm_has_neon()	340
7.57.1.8 vm_is_cortex_a72()	340

7.57.1.9	vm_max_asm()	340
7.57.1.10	vm_memcpy_asm()	340
7.57.1.11	vm_memzero_asm()	340
7.57.1.12	vm_min_asm()	341
7.57.1.13	vm_mul_double()	341
7.57.1.14	vm_pop_asm()	341
7.57.1.15	vm_popcnt()	341
7.57.1.16	vm_prefetch()	341
7.57.1.17	vm_prefetch_stream()	341
7.57.1.18	vm_push_asm()	342
7.57.1.19	vm_read_midr()	342
7.57.1.20	vm_rpop_asm()	342
7.57.1.21	vm_rpush_asm()	342
7.57.1.22	vm_strcmp_asm()	343
7.57.1.23	vm_strcmp_short_asm()	343
7.57.1.24	vm_sub_check_overflow()	343
7.58	include/vm_asm_opt_riscv64.h File Reference	343
7.58.1	Function Documentation	344
7.58.1.1	vm_abs_asm()	344
7.58.1.2	vm_add_check_overflow()	344
7.58.1.3	vm_clz()	344
7.58.1.4	vm_ctz()	344
7.58.1.5	vm_divmod()	344
7.58.1.6	vm_has_riscv_b_ext()	344
7.58.1.7	vm_has_riscv_m_ext()	345
7.58.1.8	vm_max_asm()	345
7.58.1.9	vm_memcpy_asm()	345
7.58.1.10	vm_memzero_asm()	345
7.58.1.11	vm_min_asm()	345
7.58.1.12	vm_mul_double()	345
7.58.1.13	vm_pop_asm()	345
7.58.1.14	vm_popcnt()	345
7.58.1.15	vm_prefetch()	345
7.58.1.16	vm_prefetch_stream()	345
7.58.1.17	vm_push_asm()	346
7.58.1.18	vm_rpop_asm()	346
7.58.1.19	vm_rpush_asm()	346
7.58.1.20	vm_strcmp_asm()	346
7.58.1.21	vm_sub_check_overflow()	346
7.59	include/vm_debug.h File Reference	346
7.59.1	Function Documentation	346
7.59.1.1	vm_debug_dump_state()	346

7.59.1.2 vm_debug_install_signal_handlers()	347
7.59.1.3 vm_debug_set_current_vm()	347
7.60 include/vm_host.h File Reference	348
7.60.1 Macro Definition Documentation	349
7.60.1.1 SK_ALLOC	349
7.60.1.2 SK_ALLOC_ALIGNED	349
7.60.1.3 SK_FREE	349
7.60.1.4 SK_MUTEX_DESTROY	349
7.60.1.5 SK_MUTEX_INIT	349
7.60.1.6 SK_MUTEX_LOCK	349
7.60.1.7 SK_MUTEX_UNLOCK	349
7.60.1.8 SK_PUTC	349
7.60.1.9 SK_PUTS	349
7.60.1.10 SK_TIME_NS	349
7.60.2 Typedef Documentation	350
7.60.2.1 sk_alloc_fn	350
7.60.2.2 sk_free_fn	350
7.60.2.3 sk_mutex_destroy_fn	350
7.60.2.4 sk_mutex_init_fn	350
7.60.2.5 sk_mutex_lock_fn	350
7.60.2.6 sk_mutex_unlock_fn	350
7.60.2.7 sk_panic_fn	350
7.60.2.8 sk_putc_fn	350
7.60.2.9 sk_puts_fn	350
7.60.2.10 sk_time_fn	350
7.60.2.11 sk_xt_entry_owns_fn	350
7.60.2.12 sk_xt_validate_fn	350
7.60.2.13 VMHostServices	351
7.60.3 Function Documentation	351
7.60.3.1 sk_host_services()	351
7.61 include/vm_inner_interp_arm64.h File Reference	351
7.61.1 Macro Definition Documentation	352
7.61.1.1 NEXT_ARM64	352
7.61.2 Function Documentation	352
7.61.2.1 vm_docol_arm64()	352
7.61.2.2 vm_exit_arm64()	352
7.61.2.3 vm_save_registers_arm64()	352
7.61.2.4 vm_setup_registers_arm64()	352
7.62 include/vm_inner_interp_asm.h File Reference	352
7.62.1 Macro Definition Documentation	353
7.62.1.1 NEXT_ASM	353
7.62.1.2 PRIM_2DROP	354

7.62.1.3 PRIM_2DUP	354
7.62.1.4 PRIM_AND	354
7.62.1.5 PRIM_BRANCH	354
7.62.1.6 PRIM_C_FETCH	354
7.62.1.7 PRIM_C_STORE	354
7.62.1.8 PRIM_DROP	355
7.62.1.9 PRIM_DUP	355
7.62.1.10 PRIM_EQUALS	355
7.62.1.11 PRIM_EXECUTE	355
7.62.1.12 PRIM_FETCH	355
7.62.1.13 PRIM_GREATER	355
7.62.1.14 PRIM_INVERT	356
7.62.1.15 PRIM_LESS	356
7.62.1.16 PRIM_MINUS	356
7.62.1.17 PRIM_MOD	356
7.62.1.18 PRIM_NEGATE	356
7.62.1.19 PRIM_OR	356
7.62.1.20 PRIM_OVER	357
7.62.1.21 PRIM_PLUS	357
7.62.1.22 PRIM_R_FETCH	357
7.62.1.23 PRIM_R_FROM	357
7.62.1.24 PRIM_ROT	357
7.62.1.25 PRIM_SLASH	357
7.62.1.26 PRIM_STAR	358
7.62.1.27 PRIM_STORE	358
7.62.1.28 PRIM_SWAP	358
7.62.1.29 PRIM_TO_R	358
7.62.1.30 PRIM_XOR	358
7.62.1.31 PRIM_ZBRANCH	358
7.62.1.32 PRIM_ZERO_EQUALS	359
7.62.1.33 PRIM_ZERO_LESS	359
7.62.2 Function Documentation	359
7.62.2.1 vm_docol_asm()	359
7.62.2.2 vm_exit_asm()	359
7.62.2.3 vm_save_registers()	359
7.62.2.4 vm_setup_registers()	359
7.63 include/vm_inner_interp_riscv64.h File Reference	360
7.63.1 Macro Definition Documentation	360
7.63.1.1 NEXT_RV64	360
7.63.1.2 PRIM_2DROP_RV64	361
7.63.1.3 PRIM_2DUP_RV64	361
7.63.1.4 PRIM_AND_RV64	361

7.63.1.5 PRIM_BRANCH_RV64	361
7.63.1.6 PRIM_C_FETCH_RV64	361
7.63.1.7 PRIM_C_STORE_RV64	361
7.63.1.8 PRIM_DROP_RV64	361
7.63.1.9 PRIM_DUP_RV64	362
7.63.1.10 PRIM_EQUALS_RV64	362
7.63.1.11 PRIM_EXECUTE_RV64	362
7.63.1.12 PRIM_FETCH_RV64	362
7.63.1.13 PRIM_GREATER_RV64	362
7.63.1.14 PRIM_INVERT_RV64	362
7.63.1.15 PRIM_LESS_RV64	363
7.63.1.16 PRIM_MINUS_RV64	363
7.63.1.17 PRIM_MOD_RV64	363
7.63.1.18 PRIM_NEGATE_RV64	363
7.63.1.19 PRIM_OR_RV64	363
7.63.1.20 PRIM_OVER_RV64	363
7.63.1.21 PRIM_PLUS_RV64	363
7.63.1.22 PRIM_R_FETCH_RV64	364
7.63.1.23 PRIM_R_FROM_RV64	364
7.63.1.24 PRIM_ROT_RV64	364
7.63.1.25 PRIM_SLASH_RV64	364
7.63.1.26 PRIM_STAR_RV64	364
7.63.1.27 PRIM_STORE_RV64	364
7.63.1.28 PRIM_SWAP_RV64	365
7.63.1.29 PRIM_TO_R_RV64	365
7.63.1.30 PRIM_XOR_RV64	365
7.63.1.31 PRIM_ZBRANCH_RV64	365
7.63.1.32 PRIM_ZERO_EQUALS_RV64	365
7.63.1.33 PRIM_ZERO_LESS_RV64	365
7.63.2 Function Documentation	366
7.63.2.1 vm_docol_rv64()	366
7.63.2.2 vm_exit_rv64()	366
7.63.2.3 vm_save_registers_rv64()	366
7.63.2.4 vm_setup_registers_rv64()	366
7.64 include/word_registry.h File Reference	366
7.64.1 Function Documentation	366
7.64.1.1 register_forth79_words()	366
7.64.1.2 register_word()	367
7.65 README.md File Reference	367
7.66 src/blkio_factory.c File Reference	367
7.66.1 Function Documentation	368
7.66.1.1 blkio_factory_open()	368

7.66.1.2 blkio_file_init_state()	368
7.66.1.3 blkio_file_state_size()	369
7.66.1.4 blkio_file_vtable()	369
7.66.1.5 blkio_ram_init_state()	369
7.66.1.6 blkio_ram_state_size()	369
7.66.1.7 blkio_ram_vtable()	369
7.66.1.8 nonempty()	369
7.67 src/blkio_file.c File Reference	369
7.67.1 Macro Definition Documentation	370
7.67.1.1 PATH_MAX	370
7.67.2 Function Documentation	370
7.67.2.1 blkio_file_init_state()	370
7.67.2.2 blkio_file_state_size()	371
7.67.2.3 blkio_file_vtable()	371
7.67.2.4 file_close()	371
7.67.2.5 file_flush()	371
7.67.2.6 file_info()	371
7.67.2.7 file_open()	371
7.67.2.8 file_read()	371
7.67.2.9 file_write()	372
7.67.2.10 full_read_at()	372
7.67.2.11 full_write_at()	372
7.67.3 Variable Documentation	372
7.67.3.1 BLKIO_FILE_VT	372
7.68 src/blkio_ram.c File Reference	372
7.68.1 Function Documentation	373
7.68.1.1 blkio_ram_init_state()	373
7.68.1.2 blkio_ram_state_size()	373
7.68.1.3 blkio_ram_vtable()	373
7.68.1.4 in_bounds()	373
7.68.1.5 ram_close()	373
7.68.1.6 ram_flush()	374
7.68.1.7 ram_info()	374
7.68.1.8 ram_open()	374
7.68.1.9 ram_read()	374
7.68.1.10 ram_write()	374
7.68.2 Variable Documentation	374
7.68.2.1 BLKIO_RAM_VT	374
7.69 src/block_subsystem.c File Reference	374
7.69.1 Macro Definition Documentation	376
7.69.1.1 BLK_DISK_SYS_RESERVED	376
7.69.1.2 BLK_FORTH_SYS_RESERVED	376

7.69.1.3 DISK_CACHE_SLOTS	376
7.69.1.4 META_PER_BLOCK	376
7.69.1.5 META_REGION_OFFSET	377
7.69.1.6 META_REGION_SIZE	377
7.69.2 Function Documentation	377
7.69.2.1 __attribute__()	377
7.69.2.2 bam_clr()	377
7.69.2.3 bam_flush_to_disk()	377
7.69.2.4 bam_load_from_disk()	377
7.69.2.5 bam_set()	377
7.69.2.6 bam_test()	377
7.69.2.7 blk_allocate()	378
7.69.2.8 blk_flush()	378
7.69.2.9 blk_format_or_load_v2()	378
7.69.2.10 blk_get_buffer()	378
7.69.2.11 blk_get_empty_buffer()	378
7.69.2.12 blk_get_meta()	378
7.69.2.13 blk_get_timestamp()	379
7.69.2.14 blk_get_total_blocks()	379
7.69.2.15 blk_get_volume_meta()	379
7.69.2.16 blk_is_allocated()	379
7.69.2.17 blk_is_valid()	379
7.69.2.18 blk_mark_allocated()	379
7.69.2.19 blk_mark_free()	379
7.69.2.20 blk_set_meta()	380
7.69.2.21 blk_set_volume_meta()	380
7.69.2.22 blk_subsys_attach_device()	380
7.69.2.23 blk_subsys_init()	380
7.69.2.24 blk_subsys_shutdown()	380
7.69.2.25 blk_update()	380
7.69.2.26 cache_get_slot()	380
7.69.2.27 cache_load_devblock()	381
7.69.2.28 cache_writeback()	381
7.69.2.29 choose_B()	381
7.69.2.30 compute_crc64()	381
7.69.2.31 compute_totals_from_B()	381
7.69.2.32 crc64_init()	381
7.69.2.33 devblock4k_to_lba1k()	382
7.69.2.34 fblock_pack_offset()	382
7.69.2.35 fblock_to_devblock()	382
7.69.2.36 lbn_in_range()	382
7.69.2.37 lbn_to_pbn()	382

7.69.2.38 meta_from_slice()	382
7.69.2.39 meta_to_slice()	382
7.69.2.40 minzu()	383
7.69.2.41 read_devblock_4k()	383
7.69.2.42 read_header_4k()	383
7.69.2.43 udiv_floor()	383
7.69.2.44 volmeta_from_buf()	383
7.69.2.45 volmeta_to_buf()	383
7.69.2.46 write_devblock_4k()	383
7.69.2.47 write_header_4k()	384
7.69.3 Variable Documentation	384
7.69.3.1 bam	384
7.69.3.2 bam_bytes	384
7.69.3.3 bam_dirty	384
7.69.3.4 cache	384
7.69.3.5 crc64_initd	384
7.69.3.6 crc64_table	384
7.69.3.7 devblock_base_4k	384
7.69.3.8 dirty_ram	384
7.69.3.9 disk_dev	384
7.69.3.10 [struct]	384
7.69.3.11 initialized	385
7.69.3.12 ram_base	385
7.69.3.13 ram_size	385
7.69.3.14 ram_user	385
7.69.3.15 total_blkio_blocks_1k	385
7.69.3.16 total_user_lbn	385
7.69.3.17 vm	385
7.69.3.18 vol_meta	387
7.69.3.19 vol_meta_dirty	387
7.70 src/cli.c File Reference	387
7.70.1 Function Documentation	388
7.70.1.1 cli_parse()	388
7.70.1.2 cli_print_usage()	388
7.70.1.3 parse_u32()	388
7.71 src/dictionary_heat_optimization.c File Reference	388
7.71.1 Function Documentation	389
7.71.1.1 compare_dict_entry_heat_desc()	389
7.71.1.2 compare_heat_ascending()	389
7.71.1.3 dict_adaptive_optimization_pass()	389
7.71.1.4 dict_find_word_heat_aware()	389
7.71.1.5 dict_reorganize_buckets_by_heat()	390

7.71.1.6 dict_update_heat_percentiles()	390
7.72 src/dictionary_management.c File Reference	390
7.72.1 Macro Definition Documentation	391
7.72.1.1 SF_FC_BUCKETS	391
7.72.2 Function Documentation	391
7.72.2.1 sf_fc_clear_all()	391
7.72.2.2 sf_fc_reserve()	391
7.72.2.3 sf_rebuild_fc_index()	392
7.72.2.4 sf_try_fast_append()	392
7.72.2.5 sf_xrealloc()	392
7.72.2.6 vm_create_word()	392
7.72.2.7 vm_dictionary_acquire_word_id()	393
7.72.2.8 vm_dictionary_find_by_func()	393
7.72.2.9 vm_dictionary_find_latest_by_func()	393
7.72.2.10 vm_dictionary_get_data_field()	393
7.72.2.11 vm_dictionary_index_reset()	394
7.72.2.12 vm_dictionary_lookup_by_word_id()	394
7.72.2.13 vm_dictionary_track_entry()	394
7.72.2.14 vm_dictionary_untrack_entry()	394
7.72.2.15 vm_find_word()	394
7.72.2.16 vm_hide_word()	395
7.72.2.17 vm_pin_execution_heat()	395
7.72.2.18 vm_smudge_word()	395
7.72.2.19 vm_unpin_execution_heat()	395
7.72.3 Variable Documentation	396
7.72.3.1 sf_cached_latest	396
7.72.3.2 sf_fc_cap	396
7.72.3.3 sf_fc_count	396
7.72.3.4 sf_fc_list	396
7.73 src/dae_metrics.c File Reference	396
7.73.1 Detailed Description	397
7.73.2 Function Documentation	397
7.73.2.1 extract_cache_metrics()	397
7.73.2.2 isqrt()	397
7.73.2.3 metrics_from_vm()	397
7.73.2.4 metrics_get_cpu_freq_mhz()	398
7.73.2.5 metrics_get_cpu_temp_c()	398
7.73.2.6 metrics_get_timestamp()	398
7.73.2.7 metrics_print_text()	398
7.73.2.8 metrics_write_csv_header()	399
7.73.2.9 metrics_write_csv_row()	399
7.73.2.10 metrics_write_james_law_csv_row()	399

7.73.3 Variable Documentation	400
7.73.3.1 bucket_hits	400
7.73.3.2 cache_hits	400
7.73.3.3 [struct]	400
7.73.3.4 total_lookups	400
7.74 src/heartbeat_export.c File Reference	400
7.74.1 Function Documentation	400
7.74.1.1 heartbeat_capture_tick_snapshot()	400
7.74.1.2 heartbeat_emit_tick_row()	401
7.75 src/inference_engine.c File Reference	401
7.75.1 Function Documentation	402
7.75.1.1 compute_fit_quality()	402
7.75.1.2 compute_levene_statistic()	402
7.75.1.3 compute_mean_q48()	402
7.75.1.4 compute_median_q48()	402
7.75.1.5 compute_variance_q48()	402
7.75.1.6 extract_heat_trajectory()	404
7.75.1.7 find_variance_inflection()	404
7.75.1.8 has_variance_stabilized()	404
7.75.1.9 infer_decay_slope_q48()	405
7.75.1.10 inference_engine_run()	405
7.75.1.11 inference_outputs_to_string()	406
7.75.1.12 inference_outputs_validate()	406
7.76 src/io.c File Reference	407
7.76.1 Function Documentation	407
7.76.1.1 io_init()	407
7.76.1.2 io_write_block()	407
7.77 src/log.c File Reference	408
7.77.1 Function Documentation	409
7.77.1.1 get_timestamp()	409
7.77.1.2 log_get_level()	409
7.77.1.3 log_message()	409
7.77.1.4 log_set_level()	410
7.77.1.5 log_test_result()	411
7.77.2 Variable Documentation	411
7.77.2.1 color_reset	411
7.77.2.2 current_level	411
7.77.2.3 level_colors	411
7.77.2.4 level_names	411
7.77.2.5 test_fail_color	411
7.77.2.6 test_pass_color	411
7.77.2.7 test_skip_color	412

7.78 src/main.c File Reference	412
7.78.1 Macro Definition Documentation	413
7.78.1.1 STARFORTH_STATE_BYTES	413
7.78.2 Function Documentation	413
7.78.2.1 __attribute__()	413
7.78.2.2 cleanup_and_exit()	413
7.78.2.3 cleanup_blkio()	413
7.78.2.4 init_blkio()	413
7.78.2.5 init_vm_and_subsystem()	413
7.78.2.6 main()	414
7.78.2.7 print_usage()	414
7.78.2.8 run_benchmark_mode()	414
7.78.2.9 run_doe_experiment()	414
7.78.2.10 run_optional_tests()	414
7.78.2.11 signal_handler()	415
7.78.3 Variable Documentation	415
7.78.3.1 g_blkio_opened	415
7.78.3.2 g_ram_base	415
7.78.3.3 g_ram_bytes	415
7.78.3.4 g_ram_is_mmap	415
7.79 src/math_portable.c File Reference	415
7.79.1 Function Documentation	415
7.79.1.1 erf_q48()	415
7.79.1.2 exp_q48()	416
7.79.1.3 sqrt_q48()	416
7.80 src/memory_management.c File Reference	416
7.80.1 Function Documentation	416
7.80.1.1 vm_addr_to_block()	416
7.80.1.2 vm_align()	417
7.80.1.3 vm_allot()	417
7.80.1.4 vm_get_block_addr()	418
7.81 src/physics_execution_hooks.c File Reference	418
7.81.1 Function Documentation	418
7.81.1.1 physics_on_lookup()	418
7.81.1.2 physics_post_execute()	418
7.81.1.3 physics_pre_execute()	419
7.82 src/physics_hotwords_cache.c File Reference	419
7.82.1 Function Documentation	420
7.82.1.1 calculate_posterior()	420
7.82.1.2 hotwords_bayesian_report()	420
7.82.1.3 hotwords_bucket_reorder()	420
7.82.1.4 hotwords_cache_cleanup()	421

7.82.1.5 hotwords_cache_init()	421
7.82.1.6 hotwords_cache_lookup()	421
7.82.1.7 hotwords_cache_promote()	421
7.82.1.8 hotwords_cache_set_enabled()	421
7.82.1.9 hotwords_posterior_bucket_searches()	421
7.82.1.10 hotwords_posterior_cache_hits()	422
7.82.1.11 hotwords_speedup_estimate()	422
7.82.1.12 hotwords_stats_print()	422
7.82.1.13 hotwords_stats_reset()	422
7.82.1.14 q48_to_ns_double()	423
7.82.2 Variable Documentation	423
7.82.2.1 g_hotwords_cache	423
7.83 src/physics_metadata.c File Reference	423
7.83.1 Typedef Documentation	423
7.83.1.1 physics_seed_definition_t	423
7.83.2 Function Documentation	424
7.83.2.1 clamp_u32()	424
7.83.2.2 derive_state_flags()	424
7.83.2.3 physics_metadata_apply_linear_decay()	424
7.83.2.4 physics_metadata_apply_seed()	424
7.83.2.5 physics_metadata_init()	424
7.83.2.6 physics_metadata_record_latency()	424
7.83.2.7 physics_metadata_refresh_state()	424
7.83.2.8 physics_metadata_set_mass()	425
7.83.2.9 physics_metadata_touch()	425
7.83.2.10 temperature_from_execution_heat()	425
7.83.3 Variable Documentation	425
7.83.3.1 physics_seed_table	425
7.84 src/physics_pipeline_metrics.c File Reference	426
7.84.1 Function Documentation	427
7.84.1.1 ns_to_q48()	427
7.84.1.2 q48_div_u64()	427
7.84.1.3 q48_mul_q48()	427
7.84.1.4 q48_to_ns()	427
7.84.1.5 transition_metrics_binary_chop_suggest_window()	427
7.84.1.6 transition_metrics_cleanup()	428
7.84.1.7 transition_metrics_context_accuracy_string()	428
7.84.1.8 transition_metrics_get_probability_q48()	428
7.84.1.9 transition_metrics_init()	429
7.84.1.10 transition_metrics_record()	429
7.84.1.11 transition_metrics_record_context()	429
7.84.1.12 transition_metrics_record_prefetch_hit()	430

7.84.1.13 transition_metrics_record_prefetch_miss()	430
7.84.1.14 transition_metrics_reset()	431
7.84.1.15 transition_metrics_should_speculate()	431
7.84.1.16 transition_metrics_stats_string()	431
7.84.1.17 transition_metrics_update_cache()	432
7.84.1.18 transition_metrics_update_context_window()	432
7.85 src/physics_runtime.c File Reference	432
7.85.1 Macro Definition Documentation	433
7.85.1.1 PATH_MAX	433
7.85.2 Typedef Documentation	434
7.85.2.1 physics_runtime_state_t	434
7.85.3 Function Documentation	434
7.85.3.1 align_up()	434
7.85.3.2 analytics_heap_allocate()	434
7.85.3.3 analytics_heap_release()	434
7.85.3.4 analytics_ring_free()	434
7.85.3.5 parse_avg_milli()	434
7.85.3.6 physics_analytics_heap_info()	435
7.85.3.7 physics_analytics_publish_event()	435
7.85.3.8 physics_host_snapshot()	435
7.85.3.9 physics_runtime_init()	435
7.85.3.10 physics_runtime_shutdown()	437
7.85.3.11 read_cgroup_cpu_usage_us()	437
7.85.3.12 read_cgroup_memory_current()	437
7.85.3.13 read_proc_stat_totals()	437
7.85.3.14 read_psi_file()	437
7.85.3.15 resolve_cgroup_path()	438
7.85.3.16 snapshot_posix()	438
7.85.4 Variable Documentation	438
7.85.4.1 g_runtime	438
7.86 src/platform/alloc_host.c File Reference	438
7.86.1 Function Documentation	439
7.86.1.1 sf_alloc_get_stats()	439
7.86.1.2 sf_alloc_init()	439
7.86.1.3 sf_calloc()	439
7.86.1.4 sf_free()	439
7.86.1.5 sf_malloc()	439
7.86.1.6 sf_realloc()	439
7.86.2 Variable Documentation	439
7.86.2.1 g_stats	439
7.87 src/platform/alloc_kernel.c File Reference	439
7.87.1 Macro Definition Documentation	440

7.87.1.1 SF_ALIGN	440
7.87.1.2 SF_ALIGN_UP	440
7.87.1.3 SF_ARENA_SIZE	440
7.87.2 Function Documentation	440
7.87.2.1 sf_alloc_get_stats()	440
7.87.2.2 sf_alloc_init()	440
7.87.2.3 sf_calloc()	440
7.87.2.4 sf_free()	440
7.87.2.5 sf_malloc()	441
7.87.2.6 sf_realloc()	441
7.87.3 Variable Documentation	441
7.87.3.1 __attribute__	441
7.87.3.2 g_arena_offset	441
7.87.3.3 g_initialized	441
7.87.3.4 g_stats	441
7.88 src/starkernel/vm/alloc_kernel.c File Reference	441
7.88.1 Macro Definition Documentation	442
7.88.1.1 SF_ALIGN	442
7.88.1.2 SF_ALIGN_UP	442
7.88.1.3 SF_ARENA_SIZE	442
7.88.2 Function Documentation	442
7.88.2.1 sf_alloc_get_stats()	442
7.88.2.2 sf_alloc_init()	442
7.88.2.3 sf_calloc()	442
7.88.2.4 sf_free()	442
7.88.2.5 sf_malloc()	443
7.88.2.6 sf_realloc()	443
7.88.3 Variable Documentation	443
7.88.3.1 __attribute__	443
7.88.3.2 g_arena_offset	443
7.88.3.3 g_initialized	443
7.88.3.4 g_stats	443
7.89 src/platform/l4re/time.c File Reference	443
7.89.1 Function Documentation	443
7.89.1.1 sf_time_init_l4re()	443
7.89.2 Variable Documentation	444
7.89.2.1 sf_time_backend_l4re	444
7.90 src/platform/linux/time.c File Reference	444
7.90.1 Macro Definition Documentation	444
7.90.1.1 _POSIX_C_SOURCE	444
7.90.2 Function Documentation	444
7.90.2.1 posix_format_timestamp()	444

7.90.2.2 posix_get_monotonic_ns()	444
7.90.2.3 posix_get_realtime_ns()	444
7.90.2.4 posix_has_rtc()	445
7.90.2.5 posix_set_realtime_ns()	445
7.90.3 Variable Documentation	445
7.90.3.1 sf_time_backend_posix	445
7.91 src/platform/platform_init.c File Reference	445
7.91.1 Function Documentation	445
7.91.1.1 sf_time_init()	445
7.91.2 Variable Documentation	445
7.91.2.1 sf_time_backend	445
7.92 src/platform/threading.c File Reference	446
7.92.1 Function Documentation	446
7.92.1.1 sf_mutex_destroy()	446
7.92.1.2 sf_mutex_init()	446
7.92.1.3 sf_mutex_lock()	446
7.92.1.4 sf_mutex_unlock()	446
7.93 src/profiler.c File Reference	446
7.93.1 Detailed Description	448
7.93.2 Macro Definition Documentation	448
7.93.2.1 _POSIX_C_SOURCE	448
7.93.3 Typedef Documentation	448
7.93.3.1 DictEntry	448
7.93.4 Function Documentation	449
7.93.4.1 compare_by_calls()	449
7.93.4.2 compare_by_time()	449
7.93.4.3 get_time_ns()	449
7.93.4.4 get_word_stats()	449
7.93.4.5 print_word_name_safe()	450
7.93.4.6 profiler_analyze_performance()	450
7.93.4.7 profiler_end_timer()	450
7.93.4.8 profiler_generate_report()	450
7.93.4.9 profiler_get_time_ns()	450
7.93.4.10 profiler_inc_branch()	451
7.93.4.11 profiler_inc_dict_lookup()	451
7.93.4.12 profiler_inc_loop_iter()	451
7.93.4.13 profiler_inc_stack_op()	451
7.93.4.14 profiler_inc_vm_cycles()	451
7.93.4.15 profiler_init()	451
7.93.4.16 profiler_memory_alloc()	451
7.93.4.17 profiler_memory_read()	451
7.93.4.18 profiler_memory_write()	452

7.93.4.19 profiler_print_hotspots()	452
7.93.4.20 profiler_print_memory_analysis()	452
7.93.4.21 profiler_print_summary()	452
7.93.4.22 profiler_print_word_analysis()	452
7.93.4.23 profiler_reset()	452
7.93.4.24 profiler_set_level()	452
7.93.4.25 profiler_shutdown()	453
7.93.4.26 profiler_start_timer()	453
7.93.4.27 profiler_suggest_optimizations()	453
7.93.4.28 profiler_word_count()	453
7.93.4.29 profiler_word_enter()	453
7.93.4.30 profiler_word_exit()	453
7.93.5 Variable Documentation	454
7.93.5.1 current_start_time	454
7.93.5.2 current_word	454
7.93.5.3 enabled	454
7.93.5.4 g_profiler	454
7.93.5.5 level	454
7.93.5.6 [struct]	454
7.93.5.7 total_dict_lookups	454
7.93.5.8 total_memory_reads	454
7.93.5.9 total_memory_writes	454
7.93.5.10 total_stack_ops	454
7.93.5.11 word_capacity	455
7.93.5.12 word_count	455
7.93.5.13 word_stats	455
7.94 src/rolling_window_of_truth.c File Reference	455
7.94.1 Function Documentation	456
7.94.1.1 rolling_window_check_adaptive_shrink()	456
7.94.1.2 rolling_window_cleanup()	456
7.94.1.3 rolling_window_count_transition()	457
7.94.1.4 rolling_window_export_execution_history()	457
7.94.1.5 rolling_window_find_hottest_word()	457
7.94.1.6 rolling_window_get_recent_sequence()	458
7.94.1.7 rolling_window_init()	458
7.94.1.8 rolling_window_is_warm()	459
7.94.1.9 rolling_window_measure_diversity()	459
7.94.1.10 rolling_window_measure_diversity_view()	459
7.94.1.11 rolling_window_pattern_capture_rate()	459
7.94.1.12 rolling_window_publish_snapshot()	460
7.94.1.13 rolling_window_publish_snapshot_if_needed()	460
7.94.1.14 rolling_window_record_execution()	460

7.94.1.15 rolling_window_reset()	461
7.94.1.16 rolling_window_run_adaptive_pass()	461
7.94.1.17 rolling_window_seed_hotwords_cache()	461
7.94.1.18 rolling_window_seed_pipelining_context()	462
7.94.1.19 rolling_window_service()	462
7.94.1.20 rolling_window_service_internal()	462
7.94.1.21 rolling_window_snapshot_view()	462
7.94.1.22 rolling_window_stats_string()	462
7.95 src/ssm_jacquard.c File Reference	463
7.95.1 Function Documentation	463
7.95.1.1 ssm_apply_mode()	463
7.95.1.2 ssm_l8_init()	463
7.95.1.3 ssm_l8_mode_name()	464
7.95.1.4 ssm_l8_update()	464
7.96 src/stack_management.c File Reference	464
7.96.1 Detailed Description	465
7.96.2 Function Documentation	465
7.96.2.1 vm_pop()	465
7.96.2.2 vm_push()	466
7.96.2.3 vm_rpop()	467
7.96.2.4 vm_rpush()	467
7.97 src/starkernel/arch/amd64/apic.c File Reference	467
7.97.1 Macro Definition Documentation	468
7.97.1.1 APIC_REG_EOI	468
7.97.1.2 APIC_REG_ID	469
7.97.1.3 APIC_REG_LVT_TIMER	469
7.97.1.4 APIC_REG_SIVR	469
7.97.1.5 APIC_REG_TIMER_CCR	469
7.97.1.6 APIC_REG_TIMER_DCR	469
7.97.1.7 APIC_REG_TIMER_ICR	469
7.97.1.8 LAPIC_DEFAULT_PHYS	469
7.97.1.9 LAPIC_VIRT_BASE	469
7.97.1.10 LVT_MASKED	469
7.97.1.11 LVT_MODE_ONESHOT	469
7.97.1.12 LVT_MODE_PERIODIC	469
7.97.1.13 TIMER_DIV_1	469
7.97.1.14 TIMER_DIV_128	469
7.97.1.15 TIMER_DIV_16	469
7.97.1.16 TIMER_DIV_2	470
7.97.1.17 TIMER_DIV_32	470
7.97.1.18 TIMER_DIV_4	470
7.97.1.19 TIMER_DIV_64	470

7.97.1.20	TIMER_DIV_8	470
7.97.2	Function Documentation	470
7.97.2.1	apic_eoi()	470
7.97.2.2	apic_init()	470
7.97.2.3	apic_timer_init()	470
7.97.2.4	apic_timer_period_tsc()	471
7.97.2.5	apic_timer_start()	471
7.97.2.6	apic_timer_stop()	471
7.97.2.7	calibrate_apic_timer()	471
7.97.2.8	lapic_read()	471
7.97.2.9	lapic_write()	471
7.97.3	Variable Documentation	472
7.97.3.1	lapic_base	472
7.97.3.2	lapic_phys_base	472
7.97.3.3	timer_initial_count	472
7.97.3.4	timer_period_tsc_ticks	472
7.97.3.5	timer_tick_hz	472
7.98	src/starkernel/arch/amd64/arch.c File Reference	472
7.98.1	Function Documentation	473
7.98.1.1	__attribute__([1/2])	473
7.98.1.2	__attribute__([2/2])	473
7.98.1.3	arch_disable_interrupts()	473
7.98.1.4	arch_early_init()	473
7.98.1.5	arch_enable_interrupts()	473
7.98.1.6	arch_halt()	473
7.98.1.7	arch_mmu_init()	473
7.98.1.8	arch_read_timestamp()	474
7.98.1.9	gdt_init()	474
7.98.1.10	gdt_set_entry()	474
7.98.2	Variable Documentation	474
7.98.2.1	access	474
7.98.2.2	base	474
7.98.2.3	base_high	474
7.98.2.4	base_low	474
7.98.2.5	base_mid	474
7.98.2.6	granularity	474
7.98.2.7	limit	475
7.98.2.8	limit_low	475
7.99	src/starkernel/arch/amd64/interrupts.c File Reference	475
7.99.1	Macro Definition Documentation	476
7.99.1.1	IDT_ENTRIES	476
7.99.1.2	INTERRUPT_GATE	476

7.99.2 Function Documentation	476
7.99.2.1 __attribute__()	476
7.99.2.2 arch_interrupts_init()	476
7.99.2.3 decode_pf_error()	476
7.99.2.4 isr_common_handler()	476
7.99.2.5 isr_stub0()	476
7.99.2.6 lidt()	477
7.99.2.7 outb()	477
7.99.2.8 pic_disable()	477
7.99.2.9 print_dec_u64()	477
7.99.2.10 print_hex64()	477
7.99.2.11 print_q48_16()	477
7.99.2.12 set_idt_entry()	477
7.99.3 Variable Documentation	478
7.99.3.1 base	478
7.99.3.2 idt	478
7.99.3.3 isr_stub_table	478
7.99.3.4 ist	478
7.99.3.5 limit	478
7.99.3.6 offset_high	478
7.99.3.7 offset_low	478
7.99.3.8 offset_mid	478
7.99.3.9 selector	478
7.99.3.10 type_attr	478
7.99.3.11 zero	478
7.100 src/starkernel/arch/amd64/timer.c File Reference	478
7.100.1 Macro Definition Documentation	480
7.100.1.1 CV_EPSILON_PPM	480
7.100.1.2 CV_STABLE_COUNT	480
7.100.1.3 DRIFT_EPSILON_PPM	480
7.100.1.4 HPET_GEN_CAP_ID	481
7.100.1.5 HPET_GEN_CONFIG	481
7.100.1.6 HPET_MAIN_COUNTER	481
7.100.1.7 HPET_PHYS_BASE	481
7.100.1.8 HPET_VIRT_BASE	481
7.100.1.9 PIT_CHANNEL0_DATA	481
7.100.1.10 PIT_COMMAND	481
7.100.1.11 PIT_FREQ_HZ	481
7.100.1.12 PMTIMER_FREQ_HZ	481
7.100.1.13 PMTIMER_IO_PORT	481
7.100.1.14 PMTIMER_MASK	481
7.100.1.15 PMTIMER_WIDTH_BITS	481

7.100.1.16 SAMPLE_WINDOWS	482
7.100.1.17 TIMER_REQUIRE_PIT	482
7.100.1.18 TIMER_VM_STRICT_INVARIANT_TSC	482
7.100.1.19 WARMUP_WINDOWS	482
7.100.1.20 WINDOW_TICKS_HPET	482
7.100.1.21 WINDOW_TICKS_PIT	482
7.100.2 Function Documentation	482
7.100.2.1 calibrate_tsc_with_pmtimer()	482
7.100.2.2 calibrate_window_hpet()	482
7.100.2.3 calibrate_window_pit()	482
7.100.2.4 converge_tsc_bare_metal()	482
7.100.2.5 cpu_has_invariant_tsc()	483
7.100.2.6 cpu_has_rdtscp()	483
7.100.2.7 cpuid()	483
7.100.2.8 derive_tsc_hz_from_cpuid()	483
7.100.2.9 enable_hpet()	483
7.100.2.10 heartbeat_init()	483
7.100.2.11 heartbeat_state()	484
7.100.2.12 heartbeat_tick()	484
7.100.2.13 heartbeat_ticks()	484
7.100.2.14 heartbeat_trust()	484
7.100.2.15 hpet_read()	484
7.100.2.16 hpet_write()	484
7.100.2.17 inb()	485
7.100.2.18 init_timer_vm_mode()	485
7.100.2.19 inl()	485
7.100.2.20 muldiv64()	485
7.100.2.21 outb()	485
7.100.2.22 pit_probe_running()	485
7.100.2.23 pit_program_reload()	485
7.100.2.24 pit_read_counter16()	486
7.100.2.25 pmtimer_delta()	486
7.100.2.26 pmtimer_read()	486
7.100.2.27 print_dec()	486
7.100.2.28 rdtsc()	486
7.100.2.29 rdtscp_aux()	486
7.100.2.30 running_under_hypervisor()	486
7.100.2.31 sanity_check_hpet_running_quick()	487
7.100.2.32 timer_calibration_record()	487
7.100.2.33 timer_check_drift_now()	487
7.100.2.34 timer_drift_check_bare_metal()	487
7.100.2.35 timer_fatal()	487

7.100.2.36 timer_init()	487
7.100.2.37 timer_now_ns()	487
7.100.2.38 timer_now_ns_vm_relative()	488
7.100.2.39 timer_tsc_hz()	488
7.100.2.40 update_stats()	488
7.100.2.41 variance_to_trust()	488
7.100.2.42 vm_timebase_init_from_pmtimer()	488
7.100.2.43 window_push()	488
7.100.2.44 window_variance_q48()	488
7.100.3 Variable Documentation	489
7.100.3.1 calib_record	489
7.100.3.2 g_heartbeat	489
7.100.3.3 hpet_freq_hz	489
7.100.3.4 hpet_regs	489
7.100.3.5 pit_available	489
7.100.3.6 pit_tsc_hz_mean	489
7.100.3.7 tsc_hz_locked	489
7.100.3.8 vm_ns_base	489
7.100.3.9 vm_pm_start	489
7.100.3.10 windows_used	489
7.101 src/starkernel/boot/elf_loader.c File Reference	490
7.101.1 Function Documentation	490
7.101.1.1 elf_apply_relocations()	490
7.101.1.2 elf_load_kernel()	490
7.101.1.3 elf_load_segments()	491
7.101.1.4 elf_memcpy()	491
7.101.1.5 elf_memset()	491
7.101.1.6 elf_validate_header()	491
7.102 src/starkernel/boot/reloc_stub.c File Reference	491
7.102.1 Function Documentation	492
7.102.1.1 __attribute__([1/2])	492
7.102.1.2 __attribute__([2/2])	492
7.102.2 Variable Documentation	492
7.102.2.1 block_size	492
7.102.2.2 entry	492
7.102.2.3 page_rva	492
7.103 src/starkernel/boot/uefi_loader.c File Reference	493
7.103.1 Macro Definition Documentation	493
7.103.1.1 KERNEL_MAX_SIZE	493
7.103.1.2 RAW_LOG	493
7.103.2 Typedef Documentation	493
7.103.2.1 KernelEntry	493

7.103.3 Function Documentation	493
7.103.3.1 efi_main()	493
7.103.3.2 guid_equals()	494
7.103.3.3 load_kernel_from_esp()	494
7.103.4 Variable Documentation	494
7.103.4.1 g_boot_info	494
7.103.4.2 kernel_elf_buffer	494
7.103.4.3 kernel_elf_size	494
7.104 src/starkernel/capsule/capsule_birth.c File Reference	494
7.104.1 Macro Definition Documentation	495
7.104.1.1 MAX_VMS	495
7.104.2 Function Documentation	495
7.104.2.1 capsule_birth_baby()	495
7.104.2.2 capsule_birth_mama()	496
7.104.2.3 capsule_birth_set_hooks()	496
7.104.2.4 capsule_run_experiment()	497
7.104.2.5 capsule_vm_registry_count()	497
7.104.2.6 capsule_vm_registry_get()	498
7.104.2.7 capsule_vm_registry_init()	498
7.104.2.8 dispatch_init_forth()	498
7.104.2.9 vm_registry_alloc()	498
7.104.3 Variable Documentation	498
7.104.3.1 next_vm_id	498
7.104.3.2 vm_alloc_fn	498
7.104.3.3 vm_dict_hash_fn	498
7.104.3.4 vm_exec_fn	499
7.104.3.5 vm_registry	499
7.104.3.6 vm_registry_count	499
7.105 src/starkernel/capsule/capsule_find.c File Reference	499
7.105.1 Function Documentation	500
7.105.1.1 capsule_find_by_id()	500
7.105.1.2 capsule_find_by_name()	500
7.105.1.3 capsule_find_mama_init()	500
7.105.1.4 capsule_get_payload()	501
7.105.1.5 capsule_streq()	501
7.106 src/starkernel/capsule/capsule_loader.c File Reference	501
7.106.1 Macro Definition Documentation	502
7.106.1.1 CAPSULE_RAM_OFFSET	502
7.106.1.2 LOADER_BLOCK_SIZE	502
7.106.2 Function Documentation	502
7.106.2.1 capsule_clear_blocks()	502
7.106.2.2 capsule_exec_init()	502

7.106.2.3 capsule_load_blocks()	503
7.106.2.4 copy_block_to_ram()	504
7.106.2.5 copy_device_to_ram()	504
7.106.2.6 is_block_header()	504
7.106.2.7 u32_to_dec()	504
7.106.2.8 write_ramdrive_block()	504
7.107 src/starkernel/capsule/capsule_run.c File Reference	505
7.107.1 Function Documentation	505
7.107.1.1 capsule_parity_log_birth()	505
7.107.1.2 capsule_parity_log_birth_failed()	506
7.107.1.3 capsule_parity_log_mama_init()	506
7.107.1.4 capsule_parity_log_run()	506
7.107.1.5 capsule_parity_set_output()	506
7.107.1.6 capsule_run_log_count()	506
7.107.1.7 capsule_run_log_get()	507
7.107.1.8 capsule_run_log_init()	507
7.107.1.9 capsule_run_log_record()	507
7.107.1.10 parity_put_hex64()	507
7.107.1.11 parity_put_u32()	508
7.107.1.12 parity_put_u64()	508
7.107.2 Variable Documentation	508
7.107.2.1 parity_putc	508
7.107.2.2 parity_puts	508
7.107.2.3 run_id_counter	508
7.107.2.4 run_log	508
7.107.2.5 run_log_count	508
7.107.2.6 run_log_head	508
7.108 src/starkernel/capsule/capsule_validate.c File Reference	508
7.108.1 Function Documentation	509
7.108.1.1 capsule_validate()	509
7.108.1.2 capsule_validate_result_str()	509
7.108.2 Variable Documentation	509
7.108.2.1 validate_result_strings	509
7.109 src/starkernel/capsule/capsule_vm_hooks.c File Reference	510
7.109.1 Function Documentation	510
7.109.1.1 capsule_dict_hash_hook()	510
7.109.1.2 capsule_exec_hook()	510
7.109.1.3 capsule_vm_alloc_hook()	510
7.109.1.4 capsule_vm_hooks_register()	511
7.110 src/starkernel/capsule/mama_forth_words.c File Reference	511
7.110.1 Detailed Description	511
7.111 src/starkernel/hal/console.c File Reference	511

7.111.1 Macro Definition Documentation	512
7.111.1.1 SERIAL_SUPPORTED	512
7.111.2 Function Documentation	512
7.111.2.1 console_getc()	512
7.111.2.2 console_init()	512
7.111.2.3 console_poll()	512
7.111.2.4 console_println()	512
7.111.2.5 console_putc()	512
7.111.2.6 console_puts()	513
7.111.2.7 inb()	513
7.111.2.8 outb()	513
7.112 src/starkernel/hal/hal.c File Reference	513
7.113 src/starkernel/hal/host_services.c File Reference	513
7.114 src/starkernel/hal/memory.c File Reference	513
7.114.1 Function Documentation	513
7.114.1.1 hal_mem_alloc()	513
7.114.1.2 hal_mem_alloc_aligned()	514
7.114.1.3 hal_mem_free()	514
7.115 src/starkernel/hash/xxhash64.c File Reference	514
7.115.1 Macro Definition Documentation	515
7.115.1.1 XXHASH64_PRIME1	515
7.115.1.2 XXHASH64_PRIME2	515
7.115.1.3 XXHASH64_PRIME3	515
7.115.1.4 XXHASH64_PRIME4	515
7.115.1.5 XXHASH64_PRIME5	515
7.115.2 Function Documentation	515
7.115.2.1 xxh64_avalanche()	515
7.115.2.2 xxh64_merge_round()	515
7.115.2.3 xxh64_read32()	515
7.115.2.4 xxh64_read64()	515
7.115.2.5 xxh64_rotl()	516
7.115.2.6 xxh64_round()	516
7.115.2.7 xxhash64()	516
7.115.2.8 xxhash64_digest()	516
7.115.2.9 xxhash64_reset()	517
7.115.2.10 xxhash64_update()	517
7.116 src/starkernel/kernel_main.c File Reference	517
7.116.1 Macro Definition Documentation	518
7.116.1.1 KERNEL_HEAP_SIZE	518
7.116.2 Function Documentation	518
7.116.2.1 is_ram_type()	518
7.116.2.2 itoa_simple()	518

7.116.2.3 kernel_main()	518
7.116.2.4 print_banner()	519
7.116.2.5 print_boot_info()	519
7.116.2.6 print_heap_stats()	519
7.116.2.7 print_pmm_stats()	519
7.116.2.8 print_uint()	519
7.117 src/starkernel/math/q48_16.c File Reference	519
7.117.1 Function Documentation	520
7.117.1.1 q48_div()	520
7.117.1.2 q48_exp_approx()	520
7.117.1.3 q48_log_approx()	520
7.117.1.4 q48_mul()	521
7.117.1.5 q48_sqrt_approx()	521
7.118 src/starkernel/memory/kmalloc.c File Reference	522
7.118.1 Macro Definition Documentation	523
7.118.1.1 KMALLOC_DEFAULT_HEAP_SIZE	523
7.118.1.2 KMALLOC_MAGIC	523
7.118.1.3 KMALLOC_MIN_ALIGN	523
7.118.2 Typedef Documentation	523
7.118.2.1 heap_block_t	523
7.118.3 Function Documentation	523
7.118.3.1 account_allocation()	523
7.118.3.2 account_free()	523
7.118.3.3 align_up_size()	523
7.118.3.4 allocate_from_block()	523
7.118.3.5 can_split()	524
7.118.3.6 coalesce_neighbors()	524
7.118.3.7 kfree()	524
7.118.3.8 kmalloc()	524
7.118.3.9 kmalloc_aligned()	524
7.118.3.10 kmalloc_get_stats()	524
7.118.3.11 kmalloc_heap_base_addr()	524
7.118.3.12 kmalloc_heap_end_addr()	525
7.118.3.13 kmalloc_init()	525
7.118.3.14 kmalloc_is_initialized()	525
7.118.3.15 minSplittablePayload()	525
7.118.3.16 normalize_alignment()	525
7.118.3.17 pointer_within_heap()	525
7.118.3.18 update_free_bytes()	525
7.118.4 Variable Documentation	525
7.118.4.1 heap_base	525
7.118.4.2 heap_head	526

7.118.4.3 heap_initialized	526
7.118.4.4 heap_size	526
7.118.4.5 heap_stats	526
7.119 src/starkernel/memory/pmm.c File Reference	526
7.119.1 Macro Definition Documentation	527
7.119.1.1 PMM_BITMAP_BITS	527
7.119.1.2 PMM_MAX_BITMAP_BYTES	527
7.119.2 Function Documentation	527
7.119.2.1 bitmap_clear()	527
7.119.2.2 bitmap_mark_range()	527
7.119.2.3 bitmap_set()	527
7.119.2.4 bitmap_test()	527
7.119.2.5 find_contiguous_free()	528
7.119.2.6 is_ram_type()	528
7.119.2.7 is_usable_memory()	528
7.119.2.8 pmm_alloc_contiguous()	528
7.119.2.9 pmm_alloc_page()	528
7.119.2.10 pmm_bytes_to_pages()	528
7.119.2.11 pmm_free_contiguous()	528
7.119.2.12 pmm_free_page()	529
7.119.2.13 pmm_get_stats()	529
7.119.2.14 pmm_init()	529
7.119.2.15 pmm_is_initialized()	529
7.119.2.16 pmm_memset()	529
7.119.2.17 recompute_free_pages()	529
7.119.3 Variable Documentation	529
7.119.3.1 __kernel_end	529
7.119.3.2 __kernel_start	530
7.119.3.3 pmm_alloc_floor_page	530
7.119.3.4 pmm_bitmap	530
7.119.3.5 pmm_free_pages	530
7.119.3.6 pmm_initialized	530
7.119.3.7 pmm_total_pages	530
7.119.3.8 pmm_tracked_pages	530
7.120 src/starkernel/memory/vmm.c File Reference	530
7.120.1 Macro Definition Documentation	531
7.120.1.1 ADDR_MASK	531
7.120.1.2 PTE_ACCESSED	531
7.120.1.3 PTE_DIRTY	531
7.120.1.4 PTE_GLOBAL	531
7.120.1.5 PTE_NX	531
7.120.1.6 PTE_PCD	531

7.120.1.7 PTE_PRESENT	532
7.120.1.8 PTE_PS	532
7.120.1.9 PTE_PWT	532
7.120.1.10 PTE_USER	532
7.120.1.11 PTE_WRITABLE	532
7.120.2 Function Documentation	532
7.120.2.1 alloc_page_zeroed()	532
7.120.2.2 get_or_alloc_table()	532
7.120.2.3 get_table()	532
7.120.2.4 invlpg()	532
7.120.2.5 is_ram_type()	533
7.120.2.6 load_cr3()	533
7.120.2.7 make_pte()	533
7.120.2.8 paddr_to_virt()	533
7.120.2.9 pd_index()	533
7.120.2.10 pdpt_index()	533
7.120.2.11 pml4_index()	533
7.120.2.12 pt_index()	533
7.120.2.13 resolve_pt()	534
7.120.2.14 vmm_get_paddr()	534
7.120.2.15 vmm_init()	534
7.120.2.16 vmm_map_page()	534
7.120.2.17 vmm_map_range()	534
7.120.2.18 vmm_query_page()	534
7.120.2.19 vmm_unmap_page()	535
7.120.3 Variable Documentation	535
7.120.3.1 hpet_phys_base	535
7.120.3.2 lapic_phys_base	535
7.120.3.3 vmm_root_pml4_phys	535
7.121 src/repl.c File Reference	535
7.121.1 Function Documentation	535
7.121.1.1 __attribute__()	535
7.121.1.2 vm_repl()	535
7.122 src/starkernel/repl.c File Reference	536
7.122.1 Function Documentation	536
7.122.1.1 sk_fault_handler()	536
7.122.1.2 sk_readline()	536
7.122.1.3 sk_repl()	537
7.123 src/starkernel/vm/arena.c File Reference	537
7.124 src/starkernel/vm/bootstrap/sk_vm_bootstrap.c File Reference	537
7.125 src/starkernel/vm/host/shim.c File Reference	537
7.126 src/starkernel/vm/parity.c File Reference	537

7.126.1 Macro Definition Documentation	538
7.126.1.1 MAX_DICT_ENTRIES	538
7.126.1.2 SK_PARITY_DEBUG	538
7.126.1.3 sk_parity_debug_check_ptr	538
7.126.1.4 sk_parity_debug_log_msg	539
7.126.2 Function Documentation	539
7.126.2.1 fnv1a_64()	539
7.126.2.2 fnv1a_64_u32()	539
7.126.2.3 fnv1a_64_u8()	539
7.126.2.4 hash_colon_body()	539
7.126.2.5 print_hex64()	540
7.126.2.6 print_nl()	540
7.126.2.7 print_str()	540
7.126.2.8 print_u32()	540
7.126.2.9 sk_dict_canonical_hash()	540
7.126.2.10 sk_dict_word_count()	541
7.126.2.11 sk_parity_collect()	541
7.126.2.12 sk_parity_print()	541
7.127 src/starkernel/vm/q48_stubs.c File Reference	542
7.127.1 Function Documentation	542
7.127.1.1 q48_from_double()	542
7.127.1.2 q48_to_double()	542
7.128 src/starkernel/vm/vm_core.c File Reference	542
7.128.1 Macro Definition Documentation	544
7.128.1.1 vm_check_arena	544
7.128.2 Function Documentation	544
7.128.2.1 execute_colon_word()	544
7.128.2.2 hosted_alloc()	544
7.128.2.3 hosted_free()	544
7.128.2.4 hosted_mutex_destroy()	544
7.128.2.5 hosted_mutex_init()	545
7.128.2.6 hosted_mutex_lock()	545
7.128.2.7 hosted_mutex_unlock()	545
7.128.2.8 hosted_panic()	545
7.128.2.9 hosted_putc()	545
7.128.2.10 hosted_puts()	545
7.128.2.11 hosted_time_ns()	545
7.128.2.12 hosted_xt_entry_owned()	545
7.128.2.13 hosted_xt_is_executable()	545
7.128.2.14 vm_addr_ok()	545
7.128.2.15 vm_assert_interpreter_enabled()	546
7.128.2.16 vm_cleanup()	546

7.128.2.17	vm_compile_call()	546
7.128.2.18	vm_compile_exit()	546
7.128.2.19	vm_compile_literal()	546
7.128.2.20	vm_compile_word()	546
7.128.2.21	vm_default_host_services()	547
7.128.2.22	vm_enable_interpreter()	547
7.128.2.23	vm_enter_compile_mode()	547
7.128.2.24	vm_exit_compile_mode()	547
7.128.2.25	vm_get_base()	547
7.128.2.26	vm_host()	547
7.128.2.27	vm_host_alloc()	547
7.128.2.28	vm_host_calloc()	547
7.128.2.29	vm_host_free()	548
7.128.2.30	vm_interpret()	548
7.128.2.31	vm_interpret_word()	548
7.128.2.32	vm_load_cell()	548
7.128.2.33	vm_load_u8()	548
7.128.2.34	vm_make_immediate()	548
7.128.2.35	vm_monotonic_ns()	549
7.128.2.36	vm_parse_number()	549
7.128.2.37	vm_parse_word()	549
7.128.2.38	vm_ptr()	549
7.128.2.39	vm_reset_hosted_fake_ns()	550
7.128.2.40	vm_set_base()	550
7.128.2.41	vm_store_cell()	550
7.128.2.42	vm_store_u8()	550
7.128.3	Variable Documentation	550
7.128.3.1	hosted_fake_ns	550
7.128.3.2	hosted_services	550
7.129	src/starkernel/vm/vm_runtime.c File Reference	551
7.129.1	Function Documentation	551
7.129.1.1	heartbeat_publish_snapshot()	551
7.129.1.2	heartbeat_snapshot_index_load()	551
7.129.1.3	heartbeat_snapshot_index_store()	552
7.129.1.4	vm_heartbeat_run_cycle()	552
7.129.1.5	vm_heartbeat_update_l8()	552
7.129.1.6	vm_snapshot_read()	552
7.129.1.7	vm_tick()	552
7.129.1.8	vm_tick_apply_background_decay()	553
7.129.1.9	vm_tick_inference_engine()	553
7.129.1.10	vm_tick_slope_validator()	553
7.129.1.11	vm_tick_window_tuner()	553

7.130 src/test_runner/include/test_common.h File Reference	554
7.130.1 Detailed Description	555
7.130.2 Macro Definition Documentation	555
7.130.2.1 MAX_TEST_INPUT	555
7.130.2.2 MAX_TEST_OUTPUT	555
7.130.2.3 MAX_TESTS_PER_WORD	555
7.130.3 Enumeration Type Documentation	555
7.130.3.1 TestType	555
7.130.4 Function Documentation	555
7.130.4.1 assert_stack_depth()	555
7.130.4.2 assert_stack_top()	557
7.130.4.3 assert_vm_error()	557
7.130.4.4 print_module_summary()	559
7.130.4.5 restore_dict_state()	559
7.130.4.6 restore_vm_state()	560
7.130.4.7 run_single_test()	560
7.130.4.8 run_test_suite()	561
7.130.4.9 run_test_suite_m()	562
7.130.4.10 save_dict_state()	562
7.130.4.11 save_vm_state()	563
7.130.5 Variable Documentation	563
7.130.5.1 fail_fast	563
7.131 src/test_runner/include/test_contracts.h File Reference	563
7.131.1 Macro Definition Documentation	564
7.131.1.1 CONTRACT_DEFAULT_FUZZ_ROUNDS	564
7.131.1.2 CONTRACT_HEARTBEAT_SAFE	564
7.131.1.3 CONTRACT_NONE	564
7.131.1.4 CONTRACT_PHYSICS_TRANSPARENT	564
7.131.2 Typedef Documentation	564
7.131.2.1 contract_flags_t	564
7.131.3 Function Documentation	564
7.131.3.1 assert_heartbeat_safe()	564
7.131.3.2 capture_exec_result()	564
7.131.3.3 check_physics_transparent()	565
7.131.3.4 exec_results_equal()	565
7.131.3.5 perturb_physics()	565
7.131.3.6 restore_physics()	565
7.131.3.7 save_physics()	565
7.131.4 Variable Documentation	566
7.131.4.1 global_contract_violations	566
7.132 src/test_runner/include/test_runner.h File Reference	566
7.132.1 Function Documentation	567

7.132.1.1 enable_benchmark_mode()	567
7.132.1.2 run_acl_words_tests()	568
7.132.1.3 run_all_tests()	568
7.132.1.4 run_arithmetic_words_tests()	568
7.132.1.5 run_block_words_tests()	568
7.132.1.6 run_break_me_tests()	569
7.132.1.7 run_control_words_tests()	569
7.132.1.8 run_defining_words_tests()	569
7.132.1.9 run_dictionary_manipulation_words_tests()	570
7.132.1.10 run_dictionary_words_tests()	570
7.132.1.11 run_double_words_tests()	570
7.132.1.12 run_format_words_tests()	571
7.132.1.13 run_integration_tests()	571
7.132.1.14 run_io_words_tests()	571
7.132.1.15 run_logical_words_tests()	572
7.132.1.16 run_mama_forth_words_tests()	572
7.132.1.17 run_memory_words_tests()	572
7.132.1.18 run_mixed_arithmetic_words_tests()	572
7.132.1.19 run_module_tests()	573
7.132.1.20 run_return_stack_words_tests()	573
7.132.1.21 run_stack_words_tests()	573
7.132.1.22 run_starforth_words_tests()	574
7.132.1.23 run_stress_tests()	574
7.132.1.24 run_string_words_tests()	575
7.132.1.25 run_system_words_tests()	575
7.132.1.26 run_vocabulary_words_tests()	575
7.132.1.27 run_word_tests()	575
7.132.2 Variable Documentation	576
7.132.2.1 global_test_stats	576
7.133 src/test_runner/modules/acl_words_test.c File Reference	576
7.133.1 Macro Definition Documentation	576
7.133.1.1 ACL_ASSERT	576
7.133.2 Function Documentation	577
7.133.2.1 run_acl_words_tests()	577
7.133.2.2 test_acl_adaptive_ttl()	577
7.133.2.3 test_acl_deny_aborts()	577
7.133.2.4 test_acl_inherit()	577
7.133.2.5 test_acl_pin_blocks_shadow()	577
7.133.2.6 test_acl_pin_immutable()	578
7.133.2.7 test_acl_strict_mode()	578
7.133.2.8 test_acl_ttl_hot_path()	578
7.133.2.9 test_emergency_console_bypass()	578

7.133.3 Variable Documentation	578
7.133.3.1 tests_failed	578
7.133.3.2 tests_passed	578
7.134 src/test_runner/modules/arithmetic_words_test.c File Reference	578
7.134.1 Function Documentation	579
7.134.1.1 run_arithmetic_words_tests()	579
7.134.2 Variable Documentation	579
7.134.2.1 arithmetic_word_suites	579
7.135 src/test_runner/modules/block_words_test.c File Reference	579
7.135.1 Function Documentation	579
7.135.1.1 run_block_words_tests()	579
7.135.2 Variable Documentation	580
7.135.2.1 block_word_suites	580
7.136 src/test_runner/modules/break_me_tests.c File Reference	580
7.136.1 Function Documentation	581
7.136.1.1 finalize_report()	581
7.136.1.2 init_report()	581
7.136.1.3 run_break_me_tests()	581
7.136.2 Variable Documentation	581
7.136.2.1 report_file	581
7.136.2.2 test_end_time	581
7.136.2.3 test_start_time	581
7.136.2.4 total_tests_run	581
7.137 src/test_runner/modules/control_words_test.c File Reference	582
7.137.1 Function Documentation	582
7.137.1.1 run_control_words_tests()	582
7.137.2 Variable Documentation	582
7.137.2.1 control_word_suites	582
7.138 src/test_runner/modules/defining_words_tests.c File Reference	582
7.138.1 Function Documentation	583
7.138.1.1 run_defining_words_tests()	583
7.138.2 Variable Documentation	583
7.138.2.1 defining_word_suites	583
7.139 src/test_runner/modules/dictionary_manipulation_words_test.c File Reference	583
7.139.1 Function Documentation	583
7.139.1.1 run_dictionary_manipulation_words_tests()	583
7.139.2 Variable Documentation	584
7.139.2.1 dict_manip_word_suites	584
7.140 src/test_runner/modules/dictionary_words_test.c File Reference	584
7.140.1 Function Documentation	584
7.140.1.1 run_dictionary_words_tests()	584
7.140.2 Variable Documentation	585

7.140.2.1 dictionary_word_suites	585
7.141 src/test_runner/modules/double_words_test.c File Reference	585
7.141.1 Function Documentation	585
7.141.1.1 run_double_words_tests()	585
7.141.2 Variable Documentation	585
7.141.2.1 double_word_suites	585
7.142 src/test_runner/modules/format_words_test.c File Reference	585
7.142.1 Function Documentation	586
7.142.1.1 run_format_words_tests()	586
7.142.2 Variable Documentation	586
7.142.2.1 format_word_suites	586
7.143 src/test_runner/modules/integration_tests.c File Reference	586
7.143.1 Detailed Description	586
7.143.2 Function Documentation	586
7.143.2.1 run_integration_tests()	586
7.143.3 Variable Documentation	588
7.143.3.1 integration_suites	588
7.144 src/test_runner/modules/io_words_test.c File Reference	588
7.144.1 Function Documentation	588
7.144.1.1 run_io_words_tests()	588
7.144.2 Variable Documentation	589
7.144.2.1 io_word_suites	589
7.145 src/test_runner/modules/logical_words_test.c File Reference	589
7.145.1 Function Documentation	589
7.145.1.1 run_logical_words_tests()	589
7.145.2 Variable Documentation	589
7.145.2.1 logical_word_suites	589
7.146 src/test_runner/modules/mama_forth_words_test.c File Reference	589
7.146.1 Detailed Description	590
7.146.2 Macro Definition Documentation	590
7.146.2.1 MAMA_IMPLEMENTED	590
7.146.3 Function Documentation	590
7.146.3.1 run_mama_forth_words_tests()	590
7.146.4 Variable Documentation	590
7.146.4.1 mama_word_suites	590
7.147 src/test_runner/modules/memory_words_test.c File Reference	591
7.147.1 Function Documentation	591
7.147.1.1 run_memory_words_tests()	591
7.147.2 Variable Documentation	591
7.147.2.1 memory_word_suites	591
7.148 src/test_runner/modules/mixed_arithmetic_words_test.c File Reference	591
7.148.1 Function Documentation	592

7.148.1.1 run_mixed_arithmetic_words_tests()	592
7.148.2 Variable Documentation	592
7.148.2.1 mixed_arithmetic_word_suites	592
7.149 src/test_runner/modules/return_stack_words_test.c File Reference	592
7.149.1 Function Documentation	592
7.149.1.1 run_return_stack_words_tests()	592
7.149.2 Variable Documentation	592
7.149.2.1 return_stack_word_suites	592
7.150 src/test_runner/modules/stack_words_test.c File Reference	593
7.150.1 Detailed Description	593
7.150.2 Function Documentation	593
7.150.2.1 run_stack_words_tests()	593
7.150.3 Variable Documentation	593
7.150.3.1 stack_word_suites	593
7.151 src/test_runner/modules/starforth_words_test.c File Reference	593
7.151.1 Function Documentation	594
7.151.1.1 run_starforth_words_tests()	594
7.151.2 Variable Documentation	594
7.151.2.1 starforth_word_suites	594
7.152 src/test_runner/modules/stress_tests.c File Reference	594
7.152.1 Detailed Description	595
7.152.2 Function Documentation	595
7.152.2.1 run_stress_tests()	595
7.152.3 Variable Documentation	596
7.152.3.1 stress_suites	596
7.153 src/test_runner/modules/string_words_test.c File Reference	596
7.153.1 Function Documentation	596
7.153.1.1 run_string_words_tests()	596
7.153.2 Variable Documentation	596
7.153.2.1 string_word_suites	596
7.154 src/test_runner/modules/system_words_test.c File Reference	597
7.154.1 Function Documentation	597
7.154.1.1 run_system_words_tests()	597
7.154.2 Variable Documentation	597
7.154.2.1 system_word_suites	597
7.155 src/test_runner/modules/vocabulary_words_test.c File Reference	597
7.155.1 Function Documentation	598
7.155.1.1 run_vocabulary_words_tests()	598
7.155.2 Variable Documentation	598
7.155.2.1 vocabulary_word_suites	598
7.156 src/test_runner/test_common.c File Reference	598
7.156.1 Macro Definition Documentation	599

7.156.1.1 DIE	599
7.156.1.2 test_fatal_exit	599
7.156.2 Function Documentation	599
7.156.2.1 assert_stack_depth()	599
7.156.2.2 assert_stack_top()	600
7.156.2.3 assert_vm_error()	600
7.156.2.4 dump_and_die()	600
7.156.2.5 fail_and_exit()	601
7.156.2.6 print_module_summary()	601
7.156.2.7 resolve_contract()	602
7.156.2.8 restore_dict_state()	602
7.156.2.9 restore_vm_state()	602
7.156.2.10 run_single_test()	603
7.156.2.11 run_test_suite()	603
7.156.2.12 run_test_suite_m()	603
7.156.2.13 save_dict_state()	604
7.156.2.14 save_vm_state()	604
7.156.3 Variable Documentation	605
7.156.3.1 fail_fast	605
7.157 src/test_runner/test_contracts.c File Reference	605
7.157.1 Function Documentation	605
7.157.1.1 assert_heartbeat_safe()	605
7.157.1.2 capture_exec_result()	605
7.157.1.3 check_physics_transparent()	606
7.157.1.4 contract_rand()	606
7.157.1.5 exec_results_equal()	606
7.157.1.6 perturb_physics()	606
7.157.1.7 restore_physics()	606
7.157.1.8 rng_seed()	607
7.157.1.9 save_physics()	607
7.157.2 Variable Documentation	607
7.157.2.1 global_contract_violations	607
7.157.2.2 rng_state	607
7.158 src/test_runner/test_runner.c File Reference	607
7.158.1 Function Documentation	608
7.158.1.1 enable_benchmark_mode()	608
7.158.1.2 get_time_ns()	608
7.158.1.3 run_all_tests()	608
7.158.1.4 run_module_tests()	609
7.158.1.5 run_word_tests()	609
7.158.2 Variable Documentation	609
7.158.2.1 benchmark_iterations	609

7.158.2.2 benchmark_mode	609
7.158.2.3 global_test_stats	609
7.158.2.4 test_modules	610
7.159 src/vm.c File Reference	610
7.159.1 Function Documentation	611
7.159.1.1 acl_recheck()	611
7.159.1.2 execute_colon_word()	611
7.159.1.3 vm_addr_ok()	611
7.159.1.4 vm_compile_call()	612
7.159.1.5 vm_compile_exit()	612
7.159.1.6 vm_compile_literal()	612
7.159.1.7 vm_compile_word()	612
7.159.1.8 vm_enter_compile_mode()	612
7.159.1.9 vm_exit_compile_mode()	613
7.159.1.10 vm_get_base()	613
7.159.1.11 vm_interpret()	613
7.159.1.12 vm_interpret_word()	613
7.159.1.13 vm_load_cell()	614
7.159.1.14 vm_load_u8()	614
7.159.1.15 vm_make_immediate()	614
7.159.1.16 vm_parse_number()	614
7.159.1.17 vm_parse_word()	615
7.159.1.18 vm_ptr()	615
7.159.1.19 vm_set_base()	615
7.159.1.20 vm_store_cell()	615
7.159.1.21 vm_store_u8()	616
7.160 src/vm_api.c File Reference	616
7.160.1 Function Documentation	617
7.160.1.1 vm_api_create_word()	617
7.160.1.2 vm_api_find_word()	617
7.160.1.3 vm_api_hide_word()	618
7.160.1.4 vm_api_pop()	618
7.160.1.5 vm_api_push()	618
7.160.1.6 vm_api_rpop()	618
7.160.1.7 vm_api_rpush()	619
7.160.1.8 vm_api_smudge_word()	619
7.161 src/starkernel/vm/vm_bootstrap.c File Reference	619
7.161.1 Function Documentation	620
7.161.1.1 vm_bootstrap_root_vocabulary()	620
7.161.1.2 vm_bootstrap_scr()	620
7.161.1.3 vm_init()	620
7.161.1.4 vm_init_with_host()	620

7.162 src/vm_bootstrap.c File Reference	621
7.162.1 Function Documentation	621
7.162.1.1 vm_bootstrap_scr()	621
7.162.1.2 vm_cleanup()	621
7.162.1.3 vm_init()	622
7.163 src/vm_debug.c File Reference	622
7.163.1 Function Documentation	623
7.163.1.1 dbg_get_base()	623
7.163.1.2 dump_block_preview()	623
7.163.1.3 mode_str()	623
7.163.1.4 sig_handler()	624
7.163.1.5 vm_debug_dump_state()	624
7.163.1.6 vm_debug_install_signal_handlers()	624
7.163.1.7 vm_debug_set_current_vm()	625
7.163.2 Variable Documentation	625
7.163.2.1 g_vm	625
7.164 src/starkernel/vm/vm_internal.h File Reference	625
7.164.1 Macro Definition Documentation	626
7.164.1.1 HEARTBEAT_DECAY_BATCH	626
7.164.1.2 HEARTBEAT_HAS_THREADS	626
7.164.1.3 PARITY_MODE	626
7.164.1.4 SK_PARITY_DEBUG	626
7.164.2 Typedef Documentation	626
7.164.2.1 HeartbeatWorker	626
7.164.3 Function Documentation	626
7.164.3.1 heartbeat_publish_snapshot()	626
7.164.3.2 heartbeat_snapshot_index_load()	626
7.164.3.3 heartbeat_snapshot_index_store()	626
7.164.3.4 vm_default_host_services()	626
7.164.3.5 vm_get_base()	627
7.164.3.6 vm_heartbeat_run_cycle()	627
7.164.3.7 vm_host()	627
7.164.3.8 vm_host_alloc()	627
7.164.3.9 vm_host_calloc()	627
7.164.3.10 vm_host_free()	627
7.164.3.11 vm_monotonic_ns()	627
7.164.3.12 vm_reset_hosted_fake_ns()	628
7.164.3.13 vm_set_base()	628
7.165 src/vm_internal.h File Reference	628
7.165.1 Macro Definition Documentation	629
7.165.1.1 HEARTBEAT_DECAY_BATCH	629
7.165.1.2 HEARTBEAT_HAS_THREADS	629

7.165.2 Typedef Documentation	629
7.165.2.1 HeartbeatWorker	629
7.165.3 Function Documentation	629
7.165.3.1 execute_colon_word()	629
7.165.3.2 heartbeat_snapshot_index_load()	629
7.165.3.3 heartbeat_snapshot_index_store()	629
7.165.3.4 vm_cleanup()	629
7.165.3.5 vm_get_base()	629
7.165.3.6 vm_heartbeat_publish_snapshot()	630
7.165.3.7 vm_heartbeat_run_cycle()	630
7.165.3.8 vm_heartbeat_start_thread()	630
7.165.3.9 vm_heartbeat_stop_thread()	630
7.165.3.10 vm_init()	630
7.165.3.11 vm_set_base()	631
7.165.3.12 vm_snapshot_read()	631
7.165.3.13 vm_tick()	631
7.165.3.14 vm_tick_inference_engine()	632
7.165.3.15 vm_tick_slope_validator()	632
7.165.3.16 vm_tick_window_tuner()	633
7.166 src/vm_time.c File Reference	633
7.166.1 Function Documentation	634
7.166.1.1 vm_heartbeat_publish_snapshot()	634
7.166.1.2 vm_heartbeat_run_cycle()	634
7.166.1.3 vm_heartbeat_start_thread()	634
7.166.1.4 vm_heartbeat_stop_thread()	634
7.166.1.5 vm_heartbeat_update_l8()	634
7.166.1.6 vm_snapshot_read()	635
7.166.1.7 vm_tick()	635
7.166.1.8 vm_tick_apply_background_decay()	635
7.166.1.9 vm_tick_inference_engine()	635
7.166.1.10 vm_tick_slope_validator()	637
7.166.1.11 vm_tick_window_tuner()	637
7.167 src/word_registry.c File Reference	637
7.167.1 Function Documentation	638
7.167.1.1 register_forth79_words()	638
7.167.1.2 register_word()	639
7.168 src/word_source/acl_words.c File Reference	639
7.169 src/word_source/arithmetic_words.c File Reference	639
7.169.1 Function Documentation	640
7.169.1.1 arithmetic_word_abs()	640
7.169.1.2 arithmetic_word_max()	640
7.169.1.3 arithmetic_word_min()	640

7.169.1.4 arithmetic_word_minus()	640
7.169.1.5 arithmetic_word_multiply()	640
7.169.1.6 arithmetic_word_negate()	641
7.169.1.7 arithmetic_word_one_minus()	641
7.169.1.8 arithmetic_word_one_plus()	641
7.169.1.9 arithmetic_word_plus()	641
7.169.1.10 arithmetic_word_star_slash()	641
7.169.1.11 arithmetic_word_star_slash_mod()	641
7.169.1.12 arithmetic_word_two_divide()	642
7.169.1.13 arithmetic_word_two_minus()	642
7.169.1.14 arithmetic_word_two_multiply()	642
7.169.1.15 arithmetic_word_two_plus()	642
7.169.1.16 mixed_math_word_mod()	642
7.169.1.17 register_arithmetic_words()	642
7.169.1.18 word_divide()	643
7.169.1.19 word_divmod()	643
7.170 src/word_source/block_words.c File Reference	643
7.170.1 Function Documentation	644
7.170.1.1 block_word_block()	644
7.170.1.2 block_word_buffer()	644
7.170.1.3 block_word_empty_buffers()	644
7.170.1.4 block_word_flush()	644
7.170.1.5 block_word_list()	644
7.170.1.6 block_word_load()	644
7.170.1.7 block_word_next_block()	645
7.170.1.8 block_word_save_buffers()	645
7.170.1.9 block_word_scr()	645
7.170.1.10 block_word_thru()	645
7.170.1.11 block_word_update()	645
7.170.1.12 empty_all_buffers()	645
7.170.1.13 get_block_buffer()	645
7.170.1.14 get_empty_buffer()	646
7.170.1.15 init_block_system()	646
7.170.1.16 mark_buffer_dirty()	646
7.170.1.17 register_block_words()	647
7.170.1.18 save_all_buffers()	647
7.170.1.19 set_scr()	647
7.171 src/word_source/control_words.c File Reference	647
7.171.1 Function Documentation	649
7.171.1.1 cf_epoch_sync()	649
7.171.1.2 cf_peek_item()	649
7.171.1.3 cf_pop_item()	649

7.171.1.4 cf_push_item()	650
7.171.1.5 emit_cell()	650
7.171.2 Variable Documentation	650
7.171.2.1 cf_last_mode	650
7.171.2.2 cf_sp	650
7.171.2.3 cf_stack	650
7.171.2.4 endof_addrs	650
7.171.2.5 endof_mark_sp	650
7.171.2.6 endof_mark_stack	650
7.171.2.7 endof_sp	650
7.171.2.8 leave_addrs	651
7.171.2.9 leave_mark_sp	651
7.171.2.10 leave_mark_stack	651
7.171.2.11 leave_sp	651
7.172 src/word_source/defining_words.c File Reference	651
7.172.1 Function Documentation	652
7.172.1.1 defining_runtime_constant()	652
7.172.1.2 defining_runtime_create()	652
7.172.1.3 defining_runtime_dodoes()	652
7.172.1.4 defining_runtime_does_rt()	653
7.172.1.5 defining_runtime_lit()	653
7.172.1.6 defining_runtime_variable()	653
7.172.1.7 defining_word_bracket_compile()	653
7.172.1.8 defining_word_colon()	654
7.172.1.9 defining_word_compile()	654
7.172.1.10 defining_word_constant()	654
7.172.1.11 defining_word_create()	654
7.172.1.12 defining_word_does()	654
7.172.1.13 defining_word_immediate()	654
7.172.1.14 defining_word_left_bracket()	655
7.172.1.15 defining_word_literal()	655
7.172.1.16 defining_word_right_bracket()	655
7.172.1.17 defining_word_semicolon()	655
7.172.1.18 defining_word_state()	655
7.172.1.19 defining_word_variable()	655
7.172.1.20 dictionary_word_forget()	655
7.172.1.21 register_defining_words()	656
7.173 src/word_source/dictionary_heat_diagnostic_words.c File Reference	656
7.173.1 Function Documentation	656
7.173.1.1 forth_COMPARE_LOOKUPS()	656
7.173.1.2 forth_HEAT_PERCENTILES()	656
7.173.1.3 forth_LOOKUP_STRATEGY_FETCH()	657

7.173.1.4 forth_LOOKUP_STRATEGY_STORE()	657
7.173.1.5 forth_REORG_BUCKETS()	657
7.173.1.6 forth_SHOW_HEAT_OPTIMIZATION()	657
7.173.1.7 register_dictionary_heat_diagnostic_words()	657
7.174 src/word_source/dictionary_manipulation_words.c File Reference	657
7.174.1 Function Documentation	658
7.174.1.1 align_address()	658
7.174.1.2 dictionary_m_word_cfa()	658
7.174.1.3 dictionary_m_word_find()	659
7.174.1.4 dictionary_m_word_hidden()	659
7.174.1.5 dictionary_m_word_interpret()	659
7.174.1.6 dictionary_m_word_left_bracket()	659
7.174.1.7 dictionary_m_word_lfa()	659
7.174.1.8 dictionary_m_word_link_from()	659
7.174.1.9 dictionary_m_word_name_to()	659
7.174.1.10 dictionary_m_word_nfa()	660
7.174.1.11 dictionary_m_word_pfa()	660
7.174.1.12 dictionary_m_word_right_bracket()	660
7.174.1.13 dictionary_m_word_smudge()	660
7.174.1.14 dictionary_m_word_state()	660
7.174.1.15 dictionary_m_word_tick()	661
7.174.1.16 dictionary_m_word_to_body()	661
7.174.1.17 dictionary_m_word_to_link()	661
7.174.1.18 dictionary_m_word_to_name()	661
7.174.1.19 dictionary_m_word_traverse()	661
7.174.1.20 find_entry_from_name()	661
7.174.1.21 get_body_address()	662
7.174.1.22 get_name_field()	662
7.174.1.23 register_dictionary_manipulation_words()	662
7.174.1.24 traverse_name_field()	662
7.174.2 Variable Documentation	662
7.174.2.1 state_variable	662
7.175 src/word_source/dictionary_words.c File Reference	662
7.175.1 Function Documentation	663
7.175.1.1 dictionary_word_2comma()	663
7.175.1.2 dictionary_word_allot()	663
7.175.1.3 dictionary_word_c_comma()	664
7.175.1.4 dictionary_word_comma()	664
7.175.1.5 dictionary_word_here()	664
7.175.1.6 dictionary_word_latest()	664
7.175.1.7 dictionary_word_pad()	665
7.175.1.8 dictionary_word_sp_fetch()	665

7.175.1.9 dictionary_word_sp_store()	665
7.175.1.10 register_dictionary_words()	665
7.176 src/word_source/double_words.c File Reference	666
7.176.1 Function Documentation	666
7.176.1.1 d_compare()	666
7.176.1.2 double_word_2drop()	667
7.176.1.3 double_word_2dup()	667
7.176.1.4 double_word_2over()	667
7.176.1.5 double_word_2r_fetch()	667
7.176.1.6 double_word_2r_from()	667
7.176.1.7 double_word_2rot()	667
7.176.1.8 double_word_2swap()	668
7.176.1.9 double_word_2to_r()	668
7.176.1.10 double_word_d_equals()	668
7.176.1.11 double_word_d_less()	668
7.176.1.12 double_word_d_minus()	668
7.176.1.13 double_word_d_plus()	668
7.176.1.14 double_word_d_two_slash()	668
7.176.1.15 double_word_d_two_star()	668
7.176.1.16 double_word_d_zero_equals()	669
7.176.1.17 double_word_d_zero_less()	669
7.176.1.18 double_word_dabs()	669
7.176.1.19 double_word_dmax()	669
7.176.1.20 double_word_dmin()	669
7.176.1.21 double_word_dnegate()	669
7.176.1.22 double_word_s_to_d()	669
7.176.1.23 register_double_words()	670
7.177 src/word_source/editor_words.c File Reference	670
7.177.1 Function Documentation	671
7.177.1.1 current_scr()	671
7.177.1.2 editor_word_edit()	671
7.177.1.3 editor_word_l()	671
7.177.1.4 editor_word_s()	671
7.177.1.5 editor_word_show()	671
7.177.1.6 line_ptr()	672
7.177.1.7 print_line_64()	672
7.177.1.8 register_editor_words()	672
7.177.1.9 set_scr()	672
7.178 src/word_source/format_words.c File Reference	673
7.178.1 Macro Definition Documentation	674
7.178.1.1 CELL_BITS	674
7.178.1.2 CHUNK_BITS	674

7.178.1.3 CHUNK_MASK	674
7.178.2 Function Documentation	674
7.178.2.1 current_base()	674
7.178.2.2 digit_for()	674
7.178.2.3 div_ud_by_base()	674
7.178.2.4 ensure_pn()	675
7.178.2.5 format_word_base()	675
7.178.2.6 format_word_begin_conversion()	675
7.178.2.7 format_word_d_dot()	675
7.178.2.8 format_word_d_dot_r()	675
7.178.2.9 format_word_decimal()	675
7.178.2.10 format_word_dot()	675
7.178.2.11 format_word_dot_r()	676
7.178.2.12 format_word_dot_s()	676
7.178.2.13 format_word_dump()	676
7.178.2.14 format_word_end_conversion()	676
7.178.2.15 format_word_hash()	676
7.178.2.16 format_word_hash_s()	676
7.178.2.17 format_word_hex()	676
7.178.2.18 format_word_hold()	677
7.178.2.19 format_word_octal()	677
7.178.2.20 format_word_question()	677
7.178.2.21 format_word_sign()	677
7.178.2.22 format_word_u_dot()	677
7.178.2.23 format_word_u_dot_r()	677
7.178.2.24 print_number_formatted()	677
7.178.2.25 register_format_words()	678
7.178.3 Variable Documentation	678
7.178.3.1 conversion_pos	678
7.178.3.2 pn_buf	678
7.178.3.3 pn_cap	678
7.179 src/word_source/include/acl_words.h File Reference	678
7.179.1 Function Documentation	679
7.179.1.1 acl_inherit_entry()	679
7.179.1.2 register_acl_words()	679
7.180 src/word_source/include/arithmetic_words.h File Reference	679
7.180.1 Function Documentation	679
7.180.1.1 register_arithmetic_words()	679
7.181 src/word_source/include/block_words.h File Reference	680
7.181.1 Detailed Description	681
7.181.2 Macro Definition Documentation	681
7.181.2.1 BLOCK_SIZE	681

7.181.2.2 BUFFER_COUNT	681
7.181.2.3 MAX_BLOCKS	682
7.181.3 Enumeration Type Documentation	682
7.181.3.1 buffer_type_t	682
7.181.4 Function Documentation	682
7.181.4.1 empty_all_buffers()	682
7.181.4.2 get_block_buffer()	682
7.181.4.3 get_empty_buffer()	684
7.181.4.4 init_block_system()	684
7.181.4.5 mark_buffer_dirty()	684
7.181.4.6 register_block_words()	685
7.181.4.7 save_all_buffers()	685
7.182 src/word_source/include/control_words.h File Reference	686
7.183 src/word_source/include/defining_words.h File Reference	686
7.183.1 Function Documentation	686
7.183.1.1 register_defining_words()	686
7.184 src/word_source/include/dictionary_heat_diagnostic_words.h File Reference	686
7.184.1 Function Documentation	687
7.184.1.1 register_dictionary_heat_diagnostic_words()	687
7.185 src/word_source/include/dictionary_manipulation_words.h File Reference	687
7.185.1 Function Documentation	687
7.185.1.1 register_dictionary_manipulation_words()	687
7.186 src/word_source/include/dictionary_words.h File Reference	688
7.186.1 Function Documentation	688
7.186.1.1 register_dictionary_words()	688
7.187 src/word_source/include/double_words.h File Reference	688
7.187.1 Function Documentation	688
7.187.1.1 register_double_words()	688
7.188 src/word_source/include/editor_words.h File Reference	689
7.188.1 Function Documentation	689
7.188.1.1 register_editor_words()	689
7.189 src/word_source/include/format_words.h File Reference	689
7.189.1 Function Documentation	689
7.189.1.1 register_format_words()	689
7.190 src/word_source/include/io_words.h File Reference	690
7.190.1 Macro Definition Documentation	690
7.190.1.1 IO_WORDS_H	690
7.190.2 Function Documentation	690
7.190.2.1 register_io_words()	690
7.191 src/word_source/include/logical_words.h File Reference	691
7.191.1 Function Documentation	691
7.191.1.1 register_logical_words()	691

7.192 src/word_source/include/memory_words.h File Reference	692
7.192.1 Function Documentation	692
7.192.1.1 register_memory_words()	692
7.193 src/word_source/include/mixed_arithmetic_words.h File Reference	692
7.193.1 Function Documentation	692
7.193.1.1 register_mixed_arithmetic_words()	692
7.194 src/word_source/include/physics_benchmark_words.h File Reference	693
7.194.1 Function Documentation	693
7.194.1.1 register_physics_benchmark_words()	693
7.195 src/word_source/include/physics_diagnostic_words.h File Reference	693
7.195.1 Function Documentation	693
7.195.1.1 forth_PHYSICS_BURN()	693
7.195.1.2 forth_PHYSICS_CALC_KNOBS()	694
7.195.1.3 forth_PHYSICS_SHOW_FEEDBACK()	694
7.195.1.4 forth_PHYSICS_WORD_METRICS()	694
7.195.1.5 register_physics_diagnostic_words()	694
7.196 src/word_source/include/physics_freeze_words.h File Reference	695
7.196.1 Function Documentation	695
7.196.1.1 register_physics_freeze_words()	695
7.197 src/word_source/include/physics_pipelining_diagnostic_words.h File Reference	695
7.197.1 Function Documentation	695
7.197.1.1 register_physics_pipelining_diagnostic_words()	695
7.198 src/word_source/include/return_stack_words.h File Reference	696
7.198.1 Function Documentation	696
7.198.1.1 register_return_stack_words()	696
7.199 src/word_source/include/stack_words.h File Reference	696
7.199.1 Function Documentation	696
7.199.1.1 register_stack_words()	696
7.200 src/word_source/include/starforth_words.h File Reference	697
7.200.1 Function Documentation	697
7.200.1.1 register_starforth_words()	697
7.200.1.2 starforth_word_entropy_fetch()	698
7.200.1.3 starforth_word_entropy_store()	698
7.200.1.4 starforth_word_init()	698
7.200.1.5 starforth_word_paren_dash()	698
7.200.1.6 starforth_word_random()	699
7.200.1.7 starforth_word_reset_entropy()	699
7.200.1.8 starforth_word_seed()	699
7.200.1.9 starforth_word_top_words()	700
7.200.1.10 starforth_word_wait()	700
7.200.1.11 starforth_word_word_entropy()	701
7.201 src/word_source/include/string_words.h File Reference	701

7.201.1 Detailed Description	701
7.201.2 Function Documentation	701
7.201.2.1 register_string_words()	701
7.202 src/word_source/include/system_words.h File Reference	702
7.202.1 Function Documentation	702
7.202.1.1 register_system_words()	702
7.203 src/word_source/include/vocabulary_words.h File Reference	702
7.203.1 Function Documentation	703
7.203.1.1 register_vocabulary_words()	703
7.203.1.2 vocabulary_create_vocabulary_direct()	703
7.203.1.3 vocabulary_word_definitions()	703
7.203.1.4 vocabulary_word_forth()	704
7.204 src/word_source/io_words.c File Reference	704
7.204.1 Function Documentation	705
7.204.1.1 io_word_cr()	705
7.204.1.2 io_word_emit()	705
7.204.1.3 io_word_key()	705
7.204.1.4 io_word_question_terminal()	706
7.204.1.5 io_word_space()	706
7.204.1.6 io_word_spaces()	706
7.204.1.7 io_word_type()	706
7.204.1.8 register_io_words()	707
7.205 src/word_source/logical_words.c File Reference	707
7.205.1 Macro Definition Documentation	708
7.205.1.1 FORTH_FALSE	708
7.205.1.2 FORTH_TRUE	708
7.205.2 Function Documentation	708
7.205.2.1 logical_word_and()	708
7.205.2.2 logical_word_equals()	708
7.205.2.3 logical_word_false()	708
7.205.2.4 logical_word_greater_than()	708
7.205.2.5 logical_word_less_than()	708
7.205.2.6 logical_word_not()	709
7.205.2.7 logical_word_not_equals()	709
7.205.2.8 logical_word_or()	709
7.205.2.9 logical_word_true()	709
7.205.2.10 logical_word_u_greater_than()	709
7.205.2.11 logical_word_u_less_than()	709
7.205.2.12 logical_word_within()	709
7.205.2.13 logical_word_xor()	710
7.205.2.14 logical_word_zero_equals()	710
7.205.2.15 logical_word_zero_greater()	710

7.205.2.16 logical_word_zero_less()	710
7.205.2.17 logical_word_zero_not_equal()	710
7.205.2.18 register_logical_words()	710
7.206 src/word_source/memory_words.c File Reference	711
7.206.1 Function Documentation	711
7.206.1.1 memory_word_2fetch()	711
7.206.1.2 memory_word_2store()	712
7.206.1.3 memory_word_cells()	712
7.206.1.4 memory_word_cfetchn()	712
7.206.1.5 memory_word_cstore()	712
7.206.1.6 memory_word_erase()	713
7.206.1.7 memory_word_fetch()	713
7.206.1.8 memory_word_fill()	713
7.206.1.9 memory_word_minus_store()	713
7.206.1.10 memory_word_move()	714
7.206.1.11 memory_word_plus_store()	714
7.206.1.12 memory_word_store()	714
7.206.1.13 register_memory_words()	715
7.207 src/word_source/mixed_arithmetic_words.c File Reference	715
7.207.1 Function Documentation	715
7.207.1.1 mixed_math_word_m_minus()	715
7.207.1.2 mixed_math_word_m_plus()	715
7.207.1.3 mixed_math_word_m_slash_mod()	716
7.207.1.4 mixed_math_word_m_star()	716
7.207.1.5 mixed_math_word_mod()	716
7.207.1.6 mixed_math_word_slash_mod()	716
7.207.1.7 mixed_math_word_star_slash()	716
7.207.1.8 mixed_math_word_star_slash_mod()	716
7.207.1.9 register_mixed_arithmetic_words()	716
7.208 src/word_source/physics_benchmark_words.c File Reference	717
7.208.1 Function Documentation	717
7.208.1.1 forth_BENCH_DICT_LOOKUP()	717
7.208.1.2 forth_PHYSICS_BAYESIAN_REPORT()	718
7.208.1.3 forth_PHYSICS_BUILD_INFO()	718
7.208.1.4 forth_PHYSICS_CACHE_STATS()	718
7.208.1.5 forth_PHYSICS_RESET_STATS()	718
7.208.1.6 forth_PHYSICS_TOGGLE_CACHE()	718
7.208.1.7 register_physics_benchmark_words()	719
7.208.2 Variable Documentation	719
7.208.2.1 sf_fc_count	719
7.208.2.2 sf_fc_list	719
7.209 src/word_source/physics_diagnostic_words.c File Reference	719

7.209.1 Function Documentation	719
7.209.1.1 forth_PHYSICS_BURN()	719
7.209.1.2 forth_PHYSICS_CALC_KNOBS()	720
7.209.1.3 forth_PHYSICS_SHOW_FEEDBACK()	720
7.209.1.4 forth_PHYSICS_WORD_METRICS()	720
7.209.1.5 register_physics_diagnostic_words()	720
7.210 src/word_source/physics_freeze_words.c File Reference	721
7.210.1 Function Documentation	721
7.210.1.1 forth_ALL_HEATS()	721
7.210.1.2 forth_DECAY_RATE_FETCH()	721
7.210.1.3 forth_FREEZE_CRITICAL()	721
7.210.1.4 forth_FREEZE_WORD()	721
7.210.1.5 forth_FROZEN_QUERY()	722
7.210.1.6 forth_HEAT_FETCH()	722
7.210.1.7 forth_HEAT_STORE()	722
7.210.1.8 forth_SHOW_HEAT()	722
7.210.1.9 forth_UNFREEZE_WORD()	722
7.210.1.10 register_physics_freeze_words()	722
7.211 src/word_source/physics_pipelining_diagnostic_words.c File Reference	722
7.211.1 Function Documentation	723
7.211.1.1 forth_PIPELINING_ANALYZE_WORD()	723
7.211.1.2 forth_PIPELINING_ENABLE()	723
7.211.1.3 forth_PIPELINING_RESET_ALL()	723
7.211.1.4 forth_PIPELINING_SHOW_STATS()	724
7.211.1.5 forth_PIPELINING_SHOW_TOP_TRANSITIONS()	724
7.211.1.6 forth_PIPELINING_STATS()	724
7.211.1.7 register_physics_pipelining_diagnostic_words()	724
7.212 src/word_source/q48_16_words.c File Reference	724
7.212.1 Function Documentation	725
7.212.1.1 q48_div()	725
7.212.1.2 q48_exp_approx()	725
7.212.1.3 q48_from_double()	726
7.212.1.4 q48_is_valid()	726
7.212.1.5 q48_log_approx()	726
7.212.1.6 q48_mul()	727
7.212.1.7 q48_sqrt_approx()	727
7.212.1.8 q48_to_double()	728
7.212.1.9 q48_to_string()	728
7.213 src/word_source/return_stack_words.c File Reference	728
7.213.1 Detailed Description	729
7.213.2 Function Documentation	729
7.213.2.1 register_return_stack_words()	729

7.213.2.2 return_stack_word_r_fetch()	729
7.213.2.3 return_stack_word_r_from()	729
7.213.2.4 return_stack_word_to_r()	730
7.214 src/word_source/stack_words.c File Reference	730
7.214.1 Function Documentation	731
7.214.1.1 register_stack_words()	731
7.214.1.2 stack_word_depth()	731
7.214.1.3 stack_word_drop()	731
7.214.1.4 stack_word_dup()	731
7.214.1.5 stack_word_minus_rot()	732
7.214.1.6 stack_word_over()	732
7.214.1.7 stack_word_pick()	732
7.214.1.8 stack_word_question_dup()	732
7.214.1.9 stack_word_roll()	733
7.214.1.10 stack_word_rot()	733
7.214.1.11 stack_word_swap()	733
7.215 src/word_source/starforth_words.c File Reference	734
7.215.1 Function Documentation	735
7.215.1.1 is_valid_dict_entry()	735
7.215.1.2 prng_next()	735
7.215.1.3 register_starforth_words()	735
7.215.1.4 starforth_word_execution_heat_fetch()	736
7.215.1.5 starforth_word_execution_heat_store()	736
7.215.1.6 starforth_word_init()	736
7.215.1.7 starforth_word_paren_dash()	736
7.215.1.8 starforth_word_random()	737
7.215.1.9 starforth_word_reset_execution_heat()	737
7.215.1.10 starforth_word_seed()	737
7.215.1.11 starforth_word_top_words()	738
7.215.1.12 starforth_word_version()	738
7.215.1.13 starforth_word_wait()	738
7.215.1.14 starforth_word_word_execution_heat()	738
7.215.1.15 starforth_word_zuse_authenticate()	739
7.215.2 Variable Documentation	739
7.215.2.1 g_prng_state	739
7.216 src/word_source/string_words.c File Reference	739
7.216.1 Macro Definition Documentation	740
7.216.1.1 WORD_SCRATCH_CAP	740
7.216.2 Function Documentation	740
7.216.2.1 convert_string_to_number()	740
7.216.2.2 ensure_input()	740
7.216.2.3 register_string_words()	741

7.216.2.4	string_runtime_s_quote()	741
7.216.2.5	string_word_bl()	741
7.216.2.6	string_word_blank()	741
7.216.2.7	string_word_bracket_literal()	742
7.216.2.8	string_word_bracket_tick()	742
7.216.2.9	string_word_cmove()	742
7.216.2.10	string_word_cmove_greater()	742
7.216.2.11	string_word_compare()	742
7.216.2.12	string_word_convert()	742
7.216.2.13	string_word_count()	743
7.216.2.14	string_word_enclose()	743
7.216.2.15	string_word_expect()	743
7.216.2.16	string_word_literal()	743
7.216.2.17	string_word_minus_trailing()	743
7.216.2.18	string_word_number()	743
7.216.2.19	string_word_query()	744
7.216.2.20	string_word_s_quote()	744
7.216.2.21	string_word_scan()	744
7.216.2.22	string_word_search()	744
7.216.2.23	string_word_skip()	744
7.216.2.24	string_word_source()	744
7.216.2.25	string_word_span()	744
7.216.2.26	string_word_tib()	745
7.216.2.27	string_word_tick()	745
7.216.2.28	string_word_to_in()	745
7.216.2.29	string_word_word()	745
7.216.2.30	vaddr_from_ptr()	745
7.216.3	Variable Documentation	746
7.216.3.1	word_scratch_addr	746
7.217	src/word_source/system_words.c File Reference	746
7.217.1	Function Documentation	747
7.217.1.1	forth_paren()	747
7.217.1.2	print_word_name()	747
7.217.1.3	register_system_words()	747
7.217.1.4	reset_vm_state()	747
7.217.1.5	runtime_abortq()	747
7.217.1.6	set_forth_79_compliance()	747
7.217.1.7	system_is_running()	748
7.217.1.8	system_word_79_standard()	748
7.217.1.9	system_word_abort()	748
7.217.1.10	system_word_abort_quote()	748
7.217.1.11	system_word_bye()	748

7.217.1.12 system_word_cold()	748
7.217.1.13 system_word_nop()	748
7.217.1.14 system_word_page()	748
7.217.1.15 system_word_quit()	749
7.217.1.16 system_word_save_system()	749
7.217.1.17 system_word_see()	749
7.217.1.18 system_word_vlist()	749
7.217.1.19 system_word_warm()	749
7.217.1.20 system_word_words()	749
7.217.2 Variable Documentation	749
7.217.2.1 forth_79_standard	749
7.217.2.2 system_running	750
7.218 src/word_source/vocabulary_words.c File Reference	750
7.218.1 Macro Definition Documentation	751
7.218.1.1 SF_FC_BUCKETS	751
7.218.2 Function Documentation	751
7.218.2.1 fc_free()	751
7.218.2.2 fc_rebuild()	751
7.218.2.3 init_vocabulary_system()	751
7.218.2.4 register_vocabulary_words()	752
7.218.2.5 vm_vocabulary_find_word()	752
7.218.2.6 vocab_find_word()	752
7.218.2.7 vocab_safe_len()	752
7.218.2.8 vocab_sync_vm_vars()	753
7.218.2.9 vocabulary_create_vocabulary_direct()	753
7.218.2.10 vocabulary_debug_print_hex()	753
7.218.2.11 vocabulary_select_runtime()	753
7.218.2.12 vocabulary_word_context()	753
7.218.2.13 vocabulary_word_current()	754
7.218.2.14 vocabulary_word_definitions()	754
7.218.2.15 vocabulary_word_forth()	754
7.218.2.16 vocabulary_word_order()	754
7.218.2.17 vocabulary_word_paren_find()	755
7.218.2.18 vocabulary_word_vocabulary()	755
7.218.3 Variable Documentation	755
7.218.3.1 context_var_addr	755
7.218.3.2 context_vocab	755
7.218.3.3 ctx_cached_head	755
7.218.3.4 ctx_fc	755
7.218.3.5 ctx_n	756
7.218.3.6 current_var_addr	756
7.218.3.7 current_vocab	756

7.218.3.8 forth_cached_head	756
7.218.3.9 forth_fc	756
7.218.3.10 forth_n	756
7.218.3.11 forth_vocab	756

Index	757
--------------	------------

Chapter 1

StarForth

<https://github.com/rajames440/StarForth/releases/tag/v1.5.0-RELEASE>

FORTH-79 virtual machine with a physics-grounded adaptive runtime. Written in strict ANSI C99. Runs on Linux, L4Re/Fiasco.OC, and bare metal. Formally verified. Deterministic by proof.

The hosted [VM](#) under StarshipOS — and the foundation for [LithosAnanke](#).

1.1 What makes it different

Seven feedback loops drive self-optimization at runtime while preserving full determinism:

Loop	Mechanism	Role
#1	Execution heat tracking	Identifies hot words
#2	Rolling window of truth	Deterministic history capture
#3	Linear decay	Prevents heat accumulation
#4	Pipelining metrics	Word-transition prediction
#5	Window width inference (Levene's test)	Adaptive window sizing
#6	Decay slope inference (OLS regression)	Slope tuning
#7	Adaptive heartrate	Load-responsive tick timing

Result: ~0% CV algorithmic variance across 90 experimental runs. Adaptive and reproducible.

Formal proof: 2-axiom Isabelle/HOL framework in `proof/` — sorry-free, machine-checkable.

1.2 Quick Start

```
make                # standard build
make fastest        # LTO + direct threading
make test           # 800+ test suite
make smoke          # quick sanity check
./build/amd64/standard/starforth    # interactive REPL
./build/amd64/standard/starforth -c "1 2 + . BYE"
```

1.3 Documentation

Getting Started	Build, install, first steps
Architecture	VM internals, physics engine, heartbeat system
Feedback Loops	All 7 loops in detail

Formal Proofs	Isabelle/HOL — 2 axioms, 21 theory files
Research & DoE	Experimental methodology, results
Quality & Validation	Test coverage, regression tracking
Ontology	Formal definitions — cite this for academic work
Academic Paper	121-page peer-review submission

1.4 License

[Starship License 1.0 (SL-1.0)](LICENSE) — free for personal, research, and educational use. Commercial use requires a separate agreement. Attribution to **R.A. James (Captain Bob)** must be preserved in all distributions.

Patent pending. USPTO provisional filed December 2025 — physics-grounded self-adaptive runtime system. This license does not grant patent rights. Licensing inquiries: rajames440@gmail.com

Robert A. James (Captain Bob) · Systems Engineer · Hacking since 1973

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

Architecture Detection	15
Unified API Wrappers	15
Runtime Architecture Detection	15
Build Information	15
VM Input Operations	15
Stack Operations	20
Fast Primitive Word Implementations	26
Control Flow Stack	31
Runtime Branch Helpers	32
FORTH-79 Control Flow Words	39
FORTH-79 Defining Words	39
FORTH-79 Dictionary Manipulation Words	39
FORTH-79 Dictionary & Compilation Words	40
FORTH-79 Line Editor Words	41
FORTH-79 Logical Operations	41
FORTH-79 Memory Operations	41
FORTH-79 Return Stack Operations	42
FORTH-79 System & Environment Words	42
FORTH-79 Vocabulary System Words	42

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

_EFI_BOOT_SERVICES	45
_EFI_FILE_PROTOCOL	48
_EFI_RUNTIME_SERVICES	49
_EFI_SIMPLE_FILE_SYSTEM_PROTOCOL	50
_EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL	51
_EFI_SYSTEM_TABLE	52
BayesianLatencyPosterior	
Bayesian statistics structure for inference and credible intervals All values stored in Q48.16 fixed-point (64-bit signed integer) representing nanoseconds with $2^{-16} \approx 0.0000153$ ns precision	53
blk_meta_t	54
blk_volume_meta_t	56
blkcfg_t	58
blkio_dev	59
blkio_file_state_t	60
blkio_info_t	60
blkio_params_t	61
blkio_ram_state_t	62
blkio_vtable_t	62
BlockBuffer	
Block buffer structure maintaining buffer state	63
BootInfo	64
cache_slot_t	65
CapsuleDirHeader	66
CapsuleNameEntry	67
CapsuleRunRecord	67
cf_item_t	68
CLIConfig	
Configuration structure from command-line arguments Consolidates all parsed CLI options into a single, cleanly organized struct	69
DictEntry	70
DictPhysics	72
DoeMetrics	
Complete metrics snapshot for a single DoE run	73
EFI_CONFIGURATION_TABLE	80
EFI_FILE_INFO	80
EFI_GUID	81
EFI_LOADED_IMAGE_PROTOCOL	82
EFI_MEMORY_DESCRIPTOR	83
EFI_TABLE_HEADER	83
EFI_TIME	84

Elf64_Ehdr	85
Elf64_Phdr	87
Elf64_Rel	88
Elf64_Rela	88
Elf64_Shdr	88
Elf64_Sym	90
ExecResult	90
FramebufferInfo	91
gdt_entry	
Arch.c (amd64) - Architecture abstractions for StarKernel on x86_64	92
gdt	92
heap_block	93
HeartbeatSnapshot	93
HeartbeatState	94
HeartbeatTickSnapshot	97
HeartbeatWorker	99
HotwordsCache_s	100
HotwordsStats	100
idt_entry	103
idtr	104
InferenceInputs	105
InferenceOutputs	106
kmalloc_prefix_t	107
kmalloc_stats_t	
Kmalloc.h - Kernel heap allocator interface	107
MemoryProfile	108
ParityPacket	
ParityPacket - Summary of VM state for comparison	108
physics_analytics_event_header	
Event header written into the analytics ring buffer	110
physics_analytics_header	
Analytics heap header seen by both producer (VM) and consumer (HOLA)	111
physics_analytics_heap	
Simple view onto the analytics heap layout	113
physics_host_snapshot	
Snapshot of scheduler-adjacent host signals	115
physics_runtime_state	119
physics_seed_definition	120
PhysicsSnapshot	120
PipelineGlobalMetrics	121
pmm_stats_t	122
Profiler	123
ProfileScope	125
ProfileTimer	
Timer for measuring execution duration	125
reloc_block	
Reloc_stub.c - Provide a minimal .reloc section for PE/COFF EFI loader	126
rolling_window_view_t	127
RollingWindowOfTruth	128
sf_alloc_stats	130
sf_mutex	131
sf_time_backend_t	131
SpeedupEstimate	
Calculate speedup estimate with credible interval All values in Q48.16 fixed-point format (64-bit signed integer) Probabilities are in range [0, Q48_SCALE] representing [0%, 100%]	132
ssm_config_t	133
ssm_l8_metrics_t	134
ssm_l8_state_t	134

TestCase	
Structure representing a single test case	135
TestModule	
Structure representing a test module	136
TestStats	
Structure holding test execution statistics	137
time_trust_state	138
time_window	139
timer_calibration_record	139
VM	
Main virtual machine state container	141
VMHostServices	
VMHostServices - pluggable host operations	153
vmm_page_info_t	155
VMRegistryEntry	155
WordContract	156
WordProfile	
Execution statistics for a single word	156
WordStats	
Statistics for individual word execution	157
WordTestSuite	
Collection of test cases for a single Forth word	158
WordTransitionMetrics	159
XXHash64State	
Streaming state structure	162

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/arch_detect.h	
Architecture Detection and Unified API for StarForth	165
include/blkcfg.h	170
include/blkio.h	172
include/blkio_factory.h	174
include/block_subsystem.h	175
include/cli.h	179
include/dictionary_heat_optimization.h	181
include/dictionary_management.h	183
include/doe_metrics.h	
Design of Experiments (DoE) metrics collection API	184
include/inference_engine.h	188
include/io.h	191
include/log.h	194
include/math_portable.h	198
include/memory_management.h	
Memory management functions for the StarForth virtual machine	199
include/physics_execution_hooks.h	200
include/physics_hotwords_cache.h	201
include/physics_metadata.h	205
include/physics_pipelining_metrics.h	207
include/physics_runtime.h	215
include/platform_alloc.h	219
include/platform_lock.h	220
include/platform_time.h	221
include/profiler.h	224
include/q48_16.h	277
include/repl.h	288
include/rolling_window_knobs.h	231
include/rolling_window_of_truth.h	232
include/ssm_jacquard.h	238
include/starforth_config.h	241
include/startup.h	309
include/vm.h	310
include/vm_api.h	
StarForth Virtual Machine Public API	337
include/vm_asm_opt.h	337
include/vm_asm_opt_arm64.h	338
include/vm_asm_opt_riscv64.h	343
include/vm_debug.h	346

include/vm_host.h	348
include/vm_inner_interp_arm64.h	351
include/vm_inner_interp_asm.h	352
include/vm_inner_interp_riscv64.h	360
include/word_registry.h	366
include/starkernel/apic.h	244
include/starkernel/arch.h	246
include/starkernel/capsule.h	247
include/starkernel/capsule_birth.h	253
include/starkernel/capsule_generated.h	258
include/starkernel/capsule_loader.h	258
include/starkernel/capsule_run.h	260
include/starkernel/console.h	264
include/starkernel/elf64.h	266
include/starkernel/elf_loader.h	272
include/starkernel/hal_memory.h	273
include/starkernel/kmalloc.h	274
include/starkernel/pmm.h	275
include/starkernel/q48_16.h	284
include/starkernel/repl.h	288
include/starkernel/timer.h	289
include/starkernel/uefi.h	292
include/starkernel/vmm.h	304
include/starkernel/xxhash64.h	306
include/starkernel/hal/hal.h	273
include/starkernel/vm/arena.h	301
include/starkernel/vm/parity.h	301
include/starkernel/vm/bootstrap/sk_vm_bootstrap.h	301
src/blkio_factory.c	367
src/blkio_file.c	369
src/blkio_ram.c	372
src/block_subsystem.c	374
src/cli.c	387
src/dictionary_heat_optimization.c	388
src/dictionary_management.c	390
src/doe_metrics.c	
Design of Experiments metrics collection implementation	396
src/heartbeat_export.c	400
src/inference_engine.c	401
src/io.c	407
src/log.c	408
src/main.c	412
src/math_portable.c	415
src/memory_management.c	416
src/physics_execution_hooks.c	418
src/physics_hotwords_cache.c	419
src/physics_metadata.c	423
src/physics_pipelining_metrics.c	426
src/physics_runtime.c	432
src/profiler.c	
Performance profiling implementation for StarForth	446
src/repl.c	535
src/rolling_window_of_truth.c	455
src/ssm_jacquard.c	463
src/stack_management.c	
Implementation of stack operations for the Forth virtual machine	464
src/vm.c	610
src/vm_api.c	616

src/vm_bootstrap.c	621
src/vm_debug.c	622
src/vm_internal.h	628
src/vm_time.c	633
src/word_registry.c	637
src/platform/alloc_host.c	438
src/platform/alloc_kernel.c	439
src/platform/platform_init.c	445
src/platform/threading.c	446
src/platform/l4re/time.c	443
src/platform/linux/time.c	444
src/starkernel/kernel_main.c	517
src/starkernel/repl.c	536
src/starkernel/arch/amd64/apic.c	467
src/starkernel/arch/amd64/arch.c	472
src/starkernel/arch/amd64/interrupts.c	475
src/starkernel/arch/amd64/timer.c	478
src/starkernel/boot/elf_loader.c	490
src/starkernel/boot/reloc_stub.c	491
src/starkernel/boot/uefi_loader.c	493
src/starkernel/capsule/capsule_birth.c	494
src/starkernel/capsule/capsule_find.c	499
src/starkernel/capsule/capsule_loader.c	501
src/starkernel/capsule/capsule_run.c	505
src/starkernel/capsule/capsule_validate.c	508
src/starkernel/capsule/capsule_vm_hooks.c	510
src/starkernel/capsule/mama_forth_words.c	
Mama FORTH vocabulary implementation for kernel capsule system (M7.1)	511
src/starkernel/hal/console.c	511
src/starkernel/hal/hal.c	513
src/starkernel/hal/host_services.c	513
src/starkernel/hal/memory.c	513
src/starkernel/hash/xxhash64.c	514
src/starkernel/math/q48_16.c	519
src/starkernel/memory/kmalloc.c	522
src/starkernel/memory/pmm.c	526
src/starkernel/memory/vmm.c	530
src/starkernel/vm/alloc_kernel.c	441
src/starkernel/vm/arena.c	537
src/starkernel/vm/parity.c	537
src/starkernel/vm/q48_stubs.c	542
src/starkernel/vm/vm_bootstrap.c	619
src/starkernel/vm/vm_core.c	542
src/starkernel/vm/vm_internal.h	625
src/starkernel/vm/vm_runtime.c	551
src/starkernel/vm/bootstrap/sk_vm_bootstrap.c	537
src/starkernel/vm/host/shim.c	537
src/test_runner/test_common.c	598
src/test_runner/test_contracts.c	605
src/test_runner/test_runner.c	607
src/test_runner/include/test_common.h	
Common test utilities and structures for StarForth test runner	554
src/test_runner/include/test_contracts.h	563
src/test_runner/include/test_runner.h	566
src/test_runner/modules/acl_words_test.c	576
src/test_runner/modules/arithmetic_words_test.c	578
src/test_runner/modules/block_words_test.c	579
src/test_runner/modules/break_me_tests.c	580

src/test_runner/modules/control_words_test.c	582
src/test_runner/modules/defining_words_tests.c	582
src/test_runner/modules/dictionary_manipulation_words_test.c	583
src/test_runner/modules/dictionary_words_test.c	584
src/test_runner/modules/double_words_test.c	585
src/test_runner/modules/format_words_test.c	585
src/test_runner/modules/integration_tests.c	
Full program integration tests for StarForth	586
src/test_runner/modules/io_words_test.c	588
src/test_runner/modules/logical_words_test.c	589
src/test_runner/modules/mama_forth_words_test.c	
Test module for Mama FORTH vocabulary (M7.1 Capsule System)	589
src/test_runner/modules/memory_words_test.c	591
src/test_runner/modules/mixed_arithmetic_words_test.c	591
src/test_runner/modules/return_stack_words_test.c	592
src/test_runner/modules/stack_words_test.c	
Test module for Forth stack manipulation words	593
src/test_runner/modules/starforth_words_test.c	593
src/test_runner/modules/stress_tests.c	
FORTH-79 Standard Comprehensive Stress Testing Module	594
src/test_runner/modules/string_words_test.c	596
src/test_runner/modules/system_words_test.c	597
src/test_runner/modules/vocabulary_words_test.c	597
src/word_source/acl_words.c	639
src/word_source/arithmetic_words.c	639
src/word_source/block_words.c	643
src/word_source/control_words.c	647
src/word_source/defining_words.c	651
src/word_source/dictionary_heat_diagnostic_words.c	656
src/word_source/dictionary_manipulation_words.c	657
src/word_source/dictionary_words.c	662
src/word_source/double_words.c	666
src/word_source/editor_words.c	670
src/word_source/format_words.c	673
src/word_source/io_words.c	704
src/word_source/logical_words.c	707
src/word_source/memory_words.c	711
src/word_source/mixed_arithmetic_words.c	715
src/word_source/physics_benchmark_words.c	717
src/word_source/physics_diagnostic_words.c	719
src/word_source/physics_freeze_words.c	721
src/word_source/physics_pipelining_diagnostic_words.c	722
src/word_source/q48_16_words.c	724
src/word_source/return_stack_words.c	
Implementation of FORTH-79 standard return stack words	728
src/word_source/stack_words.c	730
src/word_source/starforth_words.c	734
src/word_source/string_words.c	739
src/word_source/system_words.c	746
src/word_source/vocabulary_words.c	750
src/word_source/include/acl_words.h	678
src/word_source/include/arithmetic_words.h	679
src/word_source/include/block_words.h	
FORTH-79 Block and Buffer System Implementation	680
src/word_source/include/control_words.h	686
src/word_source/include/defining_words.h	686
src/word_source/include/dictionary_heat_diagnostic_words.h	686
src/word_source/include/dictionary_manipulation_words.h	687

src/word_source/include/dictionary_words.h	688
src/word_source/include/double_words.h	688
src/word_source/include/editor_words.h	689
src/word_source/include/format_words.h	689
src/word_source/include/io_words.h	690
src/word_source/include/logical_words.h	691
src/word_source/include/memory_words.h	692
src/word_source/include/mixed_arithmetic_words.h	692
src/word_source/include/physics_benchmark_words.h	693
src/word_source/include/physics_diagnostic_words.h	693
src/word_source/include/physics_freeze_words.h	695
src/word_source/include/physics_pipelining_diagnostic_words.h	695
src/word_source/include/return_stack_words.h	696
src/word_source/include/stack_words.h	696
src/word_source/include/starforth_words.h	697
src/word_source/include/string_words.h	
FORTH-79 Standard string and text processing words implementation	701
src/word_source/include/system_words.h	702
src/word_source/include/vocabulary_words.h	702

Chapter 5

Topic Documentation

5.1 Architecture Detection

Automatic detection of target architecture and inclusion of optimization headers.

Automatic detection of target architecture and inclusion of optimization headers.

This module automatically detects the target architecture at compile time and includes the appropriate assembly optimization headers.

Supported architectures:

- x86_64 (AMD64, Intel 64)
- ARM64 (AArch64, ARMv8-A)

Unsupported architectures fall back to pure C implementations.

5.2 Unified API Wrappers

Consistent API macros across all architectures.

Consistent API macros across all architectures.

These macros provide a consistent API across all architectures by automatically dispatching to the appropriate optimized or fallback implementation based on the target architecture and available optimizations.

5.3 Runtime Architecture Detection

Runtime detection of CPU features.

Runtime detection of CPU features.

Functions for detecting available CPU features at runtime, such as SIMD support (SSE4.2, AVX, NEON) and other architecture-specific capabilities.

5.4 Build Information

Functions for querying build configuration.

Functions for querying build configuration.

Provides information about the build configuration, including target architecture, enabled optimizations, and threading model.

5.5 VM Input Operations

Public API for managing [VM](#) input operations including Terminal Input Buffer (TIB), input pointer (>IN), and input span (SPAN) handling.

Functions

- `int vm_input_ensure (VM *vm)`
Ensures input buffer initialization.
- `unsigned char * vm_input_tib (VM *vm)`
Retrieves Terminal Input Buffer address.
- `cell_t * vm_input_in (VM *vm)`
Retrieves input pointer address.
- `cell_t * vm_input_span (VM *vm)`
Retrieves span cell address.
- `void vm_input_load_line (VM *vm, const char *src, size_t n)`
Loads a line into TIB.
- `void vm_input_source (VM *vm, cell_t *out_addr, cell_t *out_len)`
Gets current input source.
- `cell_t vm_addr_from_ptr (VM *vm, void *p)`
Converts host pointer to Forth address.

5.5.1 Detailed Description

Public API for managing [VM](#) input operations including Terminal Input Buffer (TIB), input pointer (>IN), and input span (SPAN) handling.

5.5.2 Function Documentation

5.5.2.1 vm_addr_from_ptr()

```
cell_t vm_addr_from_ptr (
    VM * vm,
    void * p )
```

Converts host pointer to Forth address.

Parameters

<i>vm</i>	Pointer to VM instance
<i>p</i>	Host memory pointer

Returns

Forth memory address (offset into `vm->memory`)

Converts host pointer to Forth address.

Parameters

<i>vm</i>	VM instance
<i>p</i>	Native pointer to convert

Returns

Forth memory address (offset from [VM](#) memory base)

Converts a native C pointer to a Forth memory address by calculating offset from [VM](#) memory base. Returns 0 for invalid pointers.

References [base](#), [VM::memory](#), and [vm](#).

5.5.2.2 vm_input_ensure()

```
int vm_input_ensure (
```

```
VM * vm )
```

Ensures input buffer initialization.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

0 on success, non-zero on failure

Allocates and initializes input buffer if not already done.

Ensures input buffer initialization.

Parameters

<i>vm</i>	VM instance
-----------	-----------------------------

Returns

0 on success, 1 on error

Allocates TIB buffer and input variables if not already allocated

References [VM::error](#), [VM::in_var](#), [INPUT_BUFFER_SIZE](#), [VM::span_var](#), [VM::tib_buf](#), [VM::tib_cap](#), [vm](#), and [vm_allot\(\)](#).

Referenced by [ensure_input\(\)](#), [vm_input_in\(\)](#), [vm_input_load_line\(\)](#), [vm_input_source\(\)](#), [vm_input_span\(\)](#), and [vm_input_tib\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.5.2.3 vm_input_in()

```
cell_t * vm_input_in (
    VM * vm )
```

Retrieves input pointer address.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Pointer to >IN cell

Retrieves input pointer address.

Parameters

<i>vm</i>	VM instance
-----------	-----------------------------

Returns

Pointer to input offset or NULL on error

Returns pointer to [VM](#)'s input offset variable after ensuring it exists

References [VM::in_var](#), [vm](#), and [vm_input_ensure\(\)](#).

Referenced by [string_word_query\(\)](#), [string_word_to_in\(\)](#), and [string_word_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.5.2.4 vm_input_load_line()

```
void vm_input_load_line (
    VM * vm,
    const char * src,
    size_t n )
```

Loads a line into TIB.

Parameters

<i>vm</i>	Pointer to VM instance
<i>src</i>	Source string to copy
<i>n</i>	Number of bytes to copy

Copies data into TIB, updates SPAN, and resets >IN.
Loads a line into TIB.

Parameters

<i>vm</i>	VM instance
<i>src</i>	Source string to load
<i>n</i>	Number of characters to load

Loads up to n characters from src into TIB, zero-pads remainder Resets input offset and updates span. Handles NULL src gracefully.

References [VM::in_var](#), [VM::span_var](#), [VM::tib_buf](#), [VM::tib_cap](#), [vm](#), and [vm_input_ensure\(\)](#).

Here is the call graph for this function:

5.5.2.5 vm_input_source()

```
void vm_input_source (
    VM * vm,
    cell_t * out_addr,
    cell_t * out_len )
```

Gets current input source.

Parameters

<i>vm</i>	Pointer to VM instance
<i>out_addr</i>	Output parameter for source address
<i>out_len</i>	Output parameter for source length

Returns SOURCE pair (address, length) for current input.
Gets current input source.

Parameters

<i>vm</i>	VM instance
<i>out_addr</i>	Output parameter for buffer address
<i>out_len</i>	Output parameter for buffer length

Returns current input buffer address and length through parameters Both parameters are optional (can be NULL)

References [VM::span_var](#), [VM::tib_buf](#), [vm](#), and [vm_input_ensure\(\)](#).

Here is the call graph for this function:

5.5.2.6 vm_input_span()

```
cell_t * vm_input_span (
    VM * vm )
```

Retrieves span cell address.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Pointer to SPAN cell

Retrieves span cell address.

Parameters

<i>vm</i>	VM instance
-----------	-----------------------------

Returns

Pointer to span variable or NULL on error

Returns pointer to [VM](#)'s input span variable after ensuring it exists

References [VM::span_var](#), [vm](#), and [vm_input_ensure\(\)](#).

Referenced by [string_word_expect\(\)](#), [string_word_query\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), and [string_word_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.5.2.7 vm_input_tib()

```
unsigned char * vm_input_tib (
    VM * vm )
```

Retrieves Terminal Input Buffer address.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Pointer to TIB buffer

Retrieves Terminal Input Buffer address.

Parameters

<i>vm</i>	VM instance
-----------	-----------------------------

Returns

Pointer to TIB or NULL on error

Returns pointer to [VM](#)'s terminal input buffer after ensuring it exists

References [VM::tib_buf](#), [vm](#), and [vm_input_ensure\(\)](#).

Referenced by [string_word_query\(\)](#), [string_word_source\(\)](#), [string_word_tib\(\)](#), and [string_word_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.6 Stack Operations

Optimized stack manipulation operations.

Functions

- static void `vm_push_asm` (`VM *vm`, `cell_t` value)
Optimized data stack push operation.
- static `cell_t` `vm_pop_asm` (`VM *vm`)
Optimized data stack pop operation.
- static void `vm_rpush_asm` (`VM *vm`, `cell_t` value)
Optimized return stack push operation.
- static `cell_t` `vm_rpop_asm` (`VM *vm`)
Optimized return stack pop operation.
- static int `vm_add_check_overflow` (`cell_t` a, `cell_t` b, `cell_t *`result)
Performs addition with hardware overflow detection.
- static int `vm_sub_check_overflow` (`cell_t` a, `cell_t` b, `cell_t *`result)
Performs subtraction with hardware overflow detection.
- static void `vm_mul_double` (`cell_t` a, `cell_t` b, `cell_t *`hi, `cell_t *`lo)
Performs double-width multiplication.
- static void `vm_divmod` (`cell_t` dividend, `cell_t` divisor, `cell_t *`quotient, `cell_t *`remainder)
Performs division with remainder.
- static int `vm_strcmp_asm` (const char *s1, const char *s2, size_t len)
Hardware-accelerated string comparison.
- static void `vm_memcpy_asm` (void *dest, const void *src, size_t len)
Hardware-accelerated memory copy.
- static void `vm_memzero_asm` (void *dest, size_t len)
Hardware-accelerated memory zero.
- static `cell_t` `vm_min_asm` (`cell_t` a, `cell_t` b)
Branchless minimum calculation.
- static `cell_t` `vm_max_asm` (`cell_t` a, `cell_t` b)
Branchless maximum calculation.
- static void `vm_cpuid` (uint32_t leaf, uint32_t *eax, uint32_t *ebx, uint32_t *ecx, uint32_t *edx)
Wrapper for CPUID instruction.
- static int `vm_has_sse42` (void)
Checks for SSE4.2 support.

5.6.1 Detailed Description

Optimized stack manipulation operations.

Standard FORTH-79 stack manipulation words.

Critical optimizations for `vm_push`, `vm_pop`, `vm_rpush`, `vm_rpop`

Impact: EXTREME - These are called on nearly every Forth word execution

Benefits:

- Eliminates function call overhead (inline)
- Reduces branch mispredictions with conditional moves
- Keeps frequently-used values in registers
- Uses `lea` for pointer arithmetic (no flags affected)

L4Re Compatibility: YES - Pure computational code, no syscalls

The following FORTH words are implemented for stack manipulation:

- DROP ($n -$) Remove top stack item
- DUP ($n - n n$) Duplicate top stack item
- ?DUP ($n - n n \mid n - 0$) Duplicate if non-zero
- SWAP ($n1 n2 - n2 n1$) Exchange top two stack items
- OVER ($n1 n2 - n1 n2 n1$) Copy second stack item to top
- ROT ($n1 n2 n3 - n2 n3 n1$) Rotate top three stack items
- -ROT ($n1 n2 n3 - n3 n1 n2$) Reverse rotate top three items
- DEPTH ($- n$) Return number of stack items
- PICK ($n - \text{stack}[n]$) Copy nth stack item to top
- ROLL ($n -$) Move nth stack item to top

5.6.2 Function Documentation

5.6.2.1 vm_add_check_overflow()

```
static int vm_add_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

Performs addition with hardware overflow detection.

Parameters

<i>a</i>	First operand
<i>b</i>	Second operand
<i>result</i>	Pointer to store result

Returns

1 if overflow occurred, 0 otherwise

5.6.2.2 vm_cpuid()

```
static void vm_cpuid (
    uint32_t leaf,
    uint32_t * eax,
    uint32_t * ebx,
    uint32_t * ecx,
    uint32_t * edx ) [inline], [static]
```

Wrapper for CPUID instruction.

Parameters

<i>leaf</i>	CPUID leaf number
<i>eax</i>	Pointer to store EAX result
<i>ebx</i>	Pointer to store EBX result
<i>ecx</i>	Pointer to store ECX result
<i>edx</i>	Pointer to store EDX result

Referenced by [vm_has_sse42\(\)](#).

Here is the caller graph for this function:

5.6.2.3 vm_divmod()

```
static void vm_divmod (
    cell_t dividend,
    cell_t divisor,
    cell_t * quotient,
    cell_t * remainder ) [inline], [static]
```

Performs division with remainder.

Parameters

<i>dividend</i>	Dividend value
<i>divisor</i>	Divisor value
<i>quotient</i>	Pointer to store quotient
<i>remainder</i>	Pointer to store remainder

5.6.2.4 vm_has_sse42()

```
static int vm_has_sse42 (
    void ) [inline], [static]
```

Checks for SSE4.2 support.

Returns

1 if SSE4.2 is supported, 0 otherwise

References [vm_cpuid\(\)](#).

Referenced by [vm_has_sse42_opt\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.6.2.5 vm_max_asm()

```
static cell_t vm_max_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

Branchless maximum calculation.

Parameters

<i>a</i>	First value
<i>b</i>	Second value

Returns

Larger of a and b

5.6.2.6 vm_memcpy_asm()

```
static void vm_memcpy_asm (
    void * dest,
    const void * src,
    size_t len ) [inline], [static]
```

Hardware-accelerated memory copy.

Parameters

<i>dest</i>	Destination buffer
<i>src</i>	Source buffer
<i>len</i>	Number of bytes to copy

5.6.2.7 **vm_memzero_asm()**

```
static void vm_memzero_asm (
    void * dest,
    size_t len ) [inline], [static]
```

Hardware-accelerated memory zero.

Parameters

<i>dest</i>	Buffer to zero
<i>len</i>	Number of bytes to zero

5.6.2.8 **vm_min_asm()**

```
static cell_t vm_min_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

Branchless minimum calculation.

Parameters

<i>a</i>	First value
<i>b</i>	Second value

Returns

Smaller of a and b

5.6.2.9 **vm_mul_double()**

```
static void vm_mul_double (
    cell_t a,
    cell_t b,
    cell_t * hi,
    cell_t * lo ) [inline], [static]
```

Performs double-width multiplication.

Parameters

<i>a</i>	First operand
<i>b</i>	Second operand
<i>hi</i>	Pointer to store high 64 bits
<i>lo</i>	Pointer to store low 64 bits

5.6.2.10 **vm_pop_asm()**

```
static cell_t vm_pop_asm (
```

```
VM * vm ) [inline], [static]
```

Optimized data stack pop operation.

Parameters

<i>vm</i>	Pointer to VM context
-----------	---------------------------------------

Returns

Top value from stack, 0 on underflow

Note

Sets `vm->error` on stack underflow

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

Referenced by [vm_pop\(\)](#).

Here is the caller graph for this function:

5.6.2.11 vm_push_asm()

```
static void vm_push_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

Optimized data stack push operation.

Parameters

<i>vm</i>	Pointer to VM context
<i>value</i>	Value to push onto stack

Returns

void

Note

Sets `vm->error` on stack overflow

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

Referenced by [vm_push\(\)](#).

Here is the caller graph for this function:

5.6.2.12 vm_rpop_asm()

```
static cell_t vm_rpop_asm (
    VM * vm ) [inline], [static]
```

Optimized return stack pop operation.

Parameters

<i>vm</i>	Pointer to VM context
-----------	---------------------------------------

Returns

Top value from return stack, 0 on underflow

Note

Sets `vm->error` on stack underflow

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [vm_rpop\(\)](#).

Here is the caller graph for this function:

5.6.2.13 vm_rpush_asm()

```
static void vm_rpush_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

Optimized return stack push operation.

Parameters

<i>vm</i>	Pointer to VM context
<i>value</i>	Value to push onto return stack

Returns

void

Note

Sets `vm->error` on stack overflow

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [vm_rpush\(\)](#).

Here is the caller graph for this function:

5.6.2.14 vm_strcmp_asm()

```
static int vm_strcmp_asm (
    const char * s1,
    const char * s2,
    size_t len ) [inline], [static]
```

Hardware-accelerated string comparison.

Parameters

<i>s1</i>	First string
<i>s2</i>	Second string
<i>len</i>	Length to compare

Returns

0 if equal, non-zero otherwise

5.6.2.15 vm_sub_check_overflow()

```
static int vm_sub_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

Performs subtraction with hardware overflow detection.

Parameters

<i>a</i>	First operand (minuend)
<i>b</i>	Second operand (subtrahend)
<i>result</i>	Pointer to store result

Returns

1 if overflow occurred, 0 otherwise

5.7 Fast Primitive Word Implementations

Register-based stack primitives optimized for ARM64.

Macros

- #define `PRIM_DUP_ARM64()`
DUP primitive - duplicates top of stack item.
- #define `PRIM_DROP_ARM64()`
- #define `PRIM_SWAP_ARM64()`
- #define `PRIM_OVER_ARM64()`
- #define `PRIM_ROT_ARM64()`
- #define `PRIM_PLUS_ARM64()`
- #define `PRIM_MINUS_ARM64()`
- #define `PRIM_STAR_ARM64()`
- #define `PRIM_SLASH_ARM64()`
- #define `PRIM_MOD_ARM64()`
- #define `PRIM_AND_ARM64()`
- #define `PRIM_OR_ARM64()`
- #define `PRIM_XOR_ARM64()`
- #define `PRIM_INVERT_ARM64()`
- #define `PRIM_NEGATE_ARM64()`
- #define `PRIM_FETCH_ARM64()`
- #define `PRIM_STORE_ARM64()`
- #define `PRIM_C_FETCH_ARM64()`
- #define `PRIM_C_STORE_ARM64()`
- #define `PRIM_TO_R_ARM64()`
- #define `PRIM_R_FROM_ARM64()`
- #define `PRIM_R_FETCH_ARM64()`
- #define `PRIM_2DUP_ARM64()`
- #define `PRIM_2DROP_ARM64()`
- #define `PRIM_ZERO_EQUALS_ARM64()`
- #define `PRIM_ZERO_LESS_ARM64()`
- #define `PRIM_EQUALS_ARM64()`
- #define `PRIM_LESS_ARM64()`
- #define `PRIM_GREATER_ARM64()`
- #define `PRIM_BRANCH_ARM64()`
- #define `PRIM_ZBRANCH_ARM64()`
- #define `PRIM_EXECUTE_ARM64()`

5.7.1 Detailed Description

Register-based stack primitives optimized for ARM64.

Key optimization: TOS stays in x23, avoiding memory traffic

5.7.2 Macro Definition Documentation

5.7.2.1 PRIM_2DROP_ARM64

```
#define PRIM_2DROP_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x23, [x21, #-8]\n\t"      /* Load new TOS */ \
    "sub    x21, x21, #16\n\t"      /* DSP -= 2 */ \
    ::: "x21", "x23", "memory" \
)
```

5.7.2.2 PRIM_2DUP_ARM64

```
#define PRIM_2DUP_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21]\n\t"          /* Load NOS */ \
    "stp    x0, x23, [x21, #8]\n\t" /* Store pair */ \
    "add    x21, x21, #16\n\t"      /* DSP += 2 */ \
    ::: "x0", "x21", "memory" \
)
```

5.7.2.3 PRIM_AND_ARM64

```
#define PRIM_AND_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "and    x23, x23, x0\n\t" \
    ::: "x0", "x21", "x23" \
)
```

5.7.2.4 PRIM_BRANCH_ARM64

```
#define PRIM_BRANCH_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x20]\n\t"          /* Load offset */ \
    "add    x20, x20, x0\n\t"      /* IP += offset */ \
    ::: "x0", "x20", "memory" \
)
```

5.7.2.5 PRIM_C_FETCH_ARM64

```
#define PRIM_C_FETCH_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x19]\n\t" \
    "ldrb   w23, [x0, x23]\n\t"      /* Load byte, zero-extend */ \
    ::: "x0", "x23", "memory" \
)
```

5.7.2.6 PRIM_C_STORE_ARM64

```
#define PRIM_C_STORE_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "ldr    x1, [x19]\n\t" \
    "strb   w23, [x1, x0]\n\t"      /* Store byte */ \
    "ldr    x23, [x21]\n\t" \
    ::: "x0", "x1", "x21", "x23", "memory" \
)
```

5.7.2.7 PRIM_DROP_ARM64

```
#define PRIM_DROP_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x23, [x21], #-8\n\t"      /* Load new TOS, decrement DSP */ \
    ::: "x21", "x23", "memory" \
)
```

5.7.2.8 PRIM_DUP_ARM64

```
#define PRIM_DUP_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "str    x23, [x21], #8!\n\t" /* Store TOS, advance DSP */ \
    ::: "x21", "memory" \
)
```

DUP primitive - duplicates top of stack item.

5.7.2.9 PRIM_EQUALS_ARM64

```
#define PRIM_EQUALS_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "cmp    x0, x23\n\t" \
    "cset   x23, eq\n\t" \
    "neg    x23, x23\n\t" \
    ::: "x0", "x21", "x23", "cc" \
)
```

5.7.2.10 PRIM_EXECUTE_ARM64

```
#define PRIM_EXECUTE_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x23, [x21], #-8\n\t" /* Load new TOS */ \
    "br     x0\n\t" /* Branch to xt */ \
    ::: "x0", "x21", "x23", "memory" \
)
```

5.7.2.11 PRIM_FETCH_ARM64

```
#define PRIM_FETCH_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x19]\n\t" /* Load VM->memory */ \
    "ldr    x23, [x0, x23]\n\t" /* TOS = memory[TOS] */ \
    ::: "x0", "x23", "memory" \
)
```

5.7.2.12 PRIM_GREATER_ARM64

```
#define PRIM_GREATER_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "cmp    x0, x23\n\t" \
    "cset   x23, gt\n\t" \
    "neg    x23, x23\n\t" \
    ::: "x0", "x21", "x23", "cc" \
)
```

5.7.2.13 PRIM_INVERT_ARM64

```
#define PRIM_INVERT_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "mvn    x23, x23\n\t" \
    ::: "x23" \
)
```

5.7.2.14 PRIM_LESS_ARM64

```
#define PRIM_LESS_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "cmp    x0, x23\n\t" \
    "cset   x23, lt\n\t" \
    "neg    x23, x23\n\t" \
)
```

```

    ::: "x0", "x21", "x23", "cc" \
)

```

5.7.2.15 PRIM_MINUS_ARM64

```
#define PRIM_MINUS_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"      /* Load NOS, decrement DSP */ \
    "sub    x23, x0, x23\n\t"      /* TOS = NOS - TOS */ \
    ::: "x0", "x21", "x23" \
)

```

5.7.2.16 PRIM_MOD_ARM64

```
#define PRIM_MOD_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"      /* Load NOS, decrement DSP */ \
    "sdiv   x1, x0, x23\n\t"      /* quotient = NOS / TOS */ \
    "msub   x23, x1, x23, x0\n\t"    /* TOS = NOS - quotient * TOS */ \
    ::: "x0", "x1", "x21", "x23" \
)

```

5.7.2.17 PRIM_NEGATE_ARM64

```
#define PRIM_NEGATE_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "neg    x23, x23\n\t" \
    ::: "x23" \
)

```

5.7.2.18 PRIM_OR_ARM64

```
#define PRIM_OR_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "orr    x23, x23, x0\n\t" \
    ::: "x0", "x21", "x23" \
)

```

5.7.2.19 PRIM_OVER_ARM64

```
#define PRIM_OVER_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21]\n\t"          /* Load NOS */ \
    "str    x23, [x21, #8]\n\t"    /* Store TOS, advance DSP */ \
    "mov    x23, x0\n\t"          /* TOS = NOS */ \
    ::: "x0", "x21", "x23", "memory" \
)

```

5.7.2.20 PRIM_PLUS_ARM64

```
#define PRIM_PLUS_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"      /* Load NOS, decrement DSP */ \
    "add    x23, x23, x0\n\t"      /* TOS = TOS + NOS */ \
    ::: "x0", "x21", "x23" \
)

```

5.7.2.21 PRIM_R_FETCH_ARM64

```
#define PRIM_R_FETCH_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "str    x23, [x21, #8]\n\t"      /* Store TOS, advance DSP */ \
    "ldr    x23, [x22, #-8]\n\t"    /* Load from RS (don't pop) */ \
)

```

```

    ::: "x21", "x23", "memory" \
)

```

5.7.2.22 PRIM_R_FROM_ARM64

```
#define PRIM_R_FROM_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "str    x23, [x21, #8]!\n\t"      /* Store TOS, advance DSP */ \
    "ldr    x23, [x22, #-8]!\n\t"    /* Load from RS, decrement RSP */ \
    ::: "x21", "x22", "x23", "memory" \
)

```

5.7.2.23 PRIM_ROT_ARM64

```
#define PRIM_ROT_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21]\n\t"          /* Load item2 */ \
    "ldr    x1, [x21, #-8]\n\t"     /* Load item3 */ \
    "str    x1, [x21]\n\t"          /* item3 -> NOS */ \
    "str    x23, [x21, #-8]\n\t"    /* TOS -> item3 */ \
    "mov    x23, x0\n\t"            /* item2 -> TOS */ \
    ::: "x0", "x1", "x23", "memory" \
)

```

5.7.2.24 PRIM_SLASH_ARM64

```
#define PRIM_SLASH_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"     /* Load NOS (dividend), decrement DSP */ \
    "sdiv   x23, x0, x23\n\t"       /* TOS = NOS / TOS */ \
    ::: "x0", "x21", "x23" \
)

```

5.7.2.25 PRIM_STAR_ARM64

```
#define PRIM_STAR_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"     /* Load NOS, decrement DSP */ \
    "mul    x23, x23, x0\n\t"       /* TOS = TOS * NOS */ \
    ::: "x0", "x21", "x23" \
)

```

5.7.2.26 PRIM_STORE_ARM64

```
#define PRIM_STORE_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t"     /* Load address, decrement DSP */ \
    "ldr    x1, [x19]\n\t"          /* Load VM->memory */ \
    "str    x23, [x1, x0]\n\t"      /* memory[addr] = value */ \
    "ldr    x23, [x21]\n\t"         /* Load new TOS */ \
    ::: "x0", "x1", "x21", "x23", "memory" \
)

```

5.7.2.27 PRIM_SWAP_ARM64

```
#define PRIM_SWAP_ARM64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ldr    x0, [x21]\n\t"          /* Load NOS */ \
    "str    x23, [x21]\n\t"         /* Store TOS to NOS position */ \
    "mov    x23, x0\n\t"            /* TOS = old NOS */ \
    ::: "x0", "x23", "memory" \
)

```


5.7.2.28 PRIM_TO_R_ARM64

```
#define PRIM_TO_R_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "str    x23, [x22], #8\n\t"      /* Store to RS, advance RSP */ \
    "ldr    x23, [x21], #-8\n\t"     /* Load new TOS, decrement DSP */ \
    ::: "x21", "x22", "x23", "memory" \
)
```

5.7.2.29 PRIM_XOR_ARM64

```
#define PRIM_XOR_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x21], #-8\n\t" \
    "eor    x23, x23, x0\n\t" \
    ::: "x0", "x21", "x23" \
)
```

5.7.2.30 PRIM_ZBRANCH_ARM64

```
#define PRIM_ZBRANCH_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x20]\n\t"          /* Load offset */ \
    "cmp    x23, #0\n\t" \
    "add    x1, x20, x0\n\t"        /* Calculate target */ \
    "add    x2, x20, #8\n\t"        /* Calculate skip */ \
    "csel   x20, x1, x2, eq\n\t"    /* Select target if zero */ \
    "ldr    x23, [x21], #-8\n\t"    /* Load new TOS */ \
    ::: "x0", "x1", "x2", "x20", "x21", "x23", "cc", "memory" \
)
```

5.7.2.31 PRIM_ZERO_EQUALS_ARM64

```
#define PRIM_ZERO_EQUALS_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "cmp    x23, #0\n\t" \
    "cset   x23, eq\n\t"          /* Set to 1 if zero, else 0 */ \
    "neg    x23, x23\n\t"        /* Convert to -1 (true flag) */ \
    ::: "x23", "cc" \
)
```

5.7.2.32 PRIM_ZERO_LESS_ARM64

```
#define PRIM_ZERO_LESS_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "cmp    x23, #0\n\t" \
    "cset   x23, lt\n\t" \
    "neg    x23, x23\n\t" \
    ::: "x23", "cc" \
)
```

5.8 Control Flow Stack

Compile-time control flow stack for tracking branch targets and loop structures.

Macros

- `#define CF_STACK_MAX 64`
Maximum depth of control flow stack.

Enumerations

- `enum cf_tag_t {`
CF_BEGIN, CF_IF, CF_ELSE, CF_WHILE,

`CF_DO` , `CF_CASE` , `CF_OF` }

Control flow item types.

5.8.1 Detailed Description

Compile-time control flow stack for tracking branch targets and loop structures.

5.8.2 Macro Definition Documentation

5.8.2.1 CF_STACK_MAX

```
#define CF_STACK_MAX 64
```

Maximum depth of control flow stack.

5.8.3 Enumeration Type Documentation

5.8.3.1 cf_tag_t

```
enum cf_tag_t
```

Control flow item types.

Enumerator

CF_BEGIN	Address of BEGIN target.
CF_IF	Address of IF's 0BRANCH literal.
CF_ELSE	Address of ELSE's BRANCH literal.
CF_WHILE	Address of WHILE's 0BRANCH literal (paired with prior BEGIN)
CF_DO	Address of loop body start (back target for LOOP/+LOOP)
CF_CASE	Marker for CASE statement start.
CF_OF	Address of OF's 0BRANCH literal.

5.9 Runtime Branch Helpers

Runtime support for control flow operations.

Functions

- static void `control_forth_UNLOOP` (VM *vm)
- static void `control_forth_if` (VM *vm)
- static void `control_forth_else` (VM *vm)
- static void `control_forth_then` (VM *vm)
- static void `control_forth_begin` (VM *vm)
- static void `control_forth_until` (VM *vm)
- static void `control_forth_again` (VM *vm)
- static void `control_forth_while` (VM *vm)
- static void `control_forth_repeat` (VM *vm)
- static void `control_forth_qdo` (VM *vm)
- static void `control_forth_do` (VM *vm)
- static void `control_forth_leave` (VM *vm)
- static void `control_forth_loop` (VM *vm)
- static void `control_forth_plus_loop` (VM *vm)
- static void `control_forth_case` (VM *vm)
- static void `control_forth_of` (VM *vm)
- static void `control_forth_endof` (VM *vm)
- static void `control_forth_endcase` (VM *vm)

- void `register_control_words` (VM *vm)
Registers all control flow words with the Forth virtual machine.

Variables

- `size_t cf_item_t::addr`
- `cf_tag_t cf_item_t::tag`

Internal Runtime Functions

Forward declarations for internal runtime helpers and control flow primitives

- static void `control_forth_branch` (VM *vm)
Execute unconditional branch.
- static void `control_forth_0branch` (VM *vm)
Execute conditional branch based on top of stack.
- static void `control_forth_runtime_qdo` (VM *vm)
Runtime handler for ?DO loops.
- static void `control_forth_runtime_do` (VM *vm)
Runtime handler for DO loops.
- static void `control_forth_runtime_loop` (VM *vm)
Runtime handler for LOOP.
- static void `control_forth_runtime_plus_loop` (VM *vm)
Runtime handler for +LOOP.
- static void `control_forth_runtime_leave` (VM *vm)
Runtime handler for LEAVE.
- static void `control_forth_I` (VM *vm)
Get current loop index.
- static void `control_forth_J` (VM *vm)
Get outer loop index.
- static void `control_forth_EXIT` (VM *vm)
Exit from current definition.

5.9.1 Detailed Description

Runtime support for control flow operations.

All functions operate on the per-step return IP (top of RS). All offsets are in BYTES.

5.9.2 Function Documentation

5.9.2.1 `control_forth_0branch()`

```
static void control_forth_0branch (
    VM * vm ) [static]
```

Execute conditional branch based on top of stack.

References `VM::dsp`, `VM::error`, `LOG_DEBUG`, `LOG_ERROR`, `log_message()`, `VM::return_stack`, `VM::rsp`, `vm`, and `vm_pop()`.

Referenced by `control_forth_if()`, `control_forth_of()`, `control_forth_until()`, `control_forth_while()`, and `register_control_words()`.

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.2 control_forth_again()

```
static void control_forth_again (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_BEGIN](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [control_forth_branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.3 control_forth_begin()

```
static void control_forth_begin (
    VM * vm ) [static]
```

References [CF_BEGIN](#), [cf_epoch_sync\(\)](#), [cf_push_item\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), and [vm](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.4 control_forth_branch()

```
static void control_forth_branch (
    VM * vm ) [static]
```

Execute unconditional branch.

Runtime unconditional branch.

Parameters

<i>vm</i>	Virtual machine state
-----------	-----------------------

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [control_forth_again\(\)](#), [control_forth_else\(\)](#), [control_forth_endof\(\)](#), [control_forth_leave\(\)](#), [control_forth_repeat\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.5 control_forth_case()

```
static void control_forth_case (
    VM * vm ) [static]
```

References [CF_CASE](#), [cf_epoch_sync\(\)](#), [cf_push_item\(\)](#), [CF_STACK_MAX](#), [endof_mark_sp](#), [endof_mark_stack](#), [endof_sp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), and [vm](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.6 control_forth_do()

```
static void control_forth_do (
    VM * vm ) [static]
```

References [CF_DO](#), [cf_epoch_sync\(\)](#), [cf_push_item\(\)](#), [control_forth_runtime_do\(\)](#), [VM::error](#), [VM::here](#), [leave_mark_sp](#), [leave_mark_stack](#), [leave_sp](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.7 control_forth_else()

```
static void control_forth_else (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_ELSE](#), [cf_epoch_sync\(\)](#), [CF_IF](#), [cf_peek_item\(\)](#), [cf_pop_item\(\)](#), [cf_push_item\(\)](#), [control_forth_branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.8 control_forth_endcase()

```
static void control_forth_endcase (
    VM * vm ) [static]
```

References [CF_CASE](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [endof_addrs](#), [endof_mark_sp](#), [endof_mark_stack](#), [endof_sp](#), [VM::error](#), [DictEntry::func](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), [vm_compile_call\(\)](#), and [vm_find_word\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.9 control_forth_endof()

```
static void control_forth_endof (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [cf_epoch_sync\(\)](#), [CF_OF](#), [cf_pop_item\(\)](#), [CF_STACK_MAX](#), [control_forth_branch\(\)](#), [emit_cell\(\)](#), [endof_addrs](#), [endof_sp](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.10 control_forth_EXIT()

```
static void control_forth_EXIT (
    VM * vm ) [static]
```

Exit from current definition.

References [VM::error](#), [VM::exit_colon](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::rsp](#), and [vm](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.11 control_forth_I()

```
static void control_forth_I (
    VM * vm ) [static]
```

Get current loop index.

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.12 control_forth_if()

```
static void control_forth_if (
    VM * vm ) [static]
```

References [cf_epoch_sync\(\)](#), [CF_IF](#), [cf_push_item\(\)](#), [control_forth_0branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.13 control_forth_J()

```
static void control_forth_J (
    VM * vm ) [static]
```

Get outer loop index.

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.14 control_forth_leave()

```
static void control_forth_leave (
    VM * vm ) [static]
```

References [CF_DO](#), [cf_epoch_sync\(\)](#), [cf_sp](#), [cf_stack](#), [CF_STACK_MAX](#), [control_forth_branch\(\)](#), [control_forth_runtime_leave\(\)](#), [emit_cell\(\)](#), [VM::error](#), [leave_addrs](#), [leave_mark_sp](#), [leave_sp](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.15 control_forth_loop()

```
static void control_forth_loop (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_DO](#), [cf_epoch_sync\(\)](#), [cf_peek_item\(\)](#), [cf_pop_item\(\)](#), [CF_WHILE](#), [control_forth_runtime_loop\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [leave_addrs](#), [leave_mark_sp](#), [leave_mark_stack](#), [leave_sp](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.16 control_forth_of()

```
static void control_forth_of (
    VM * vm ) [static]
```

References [CF_CASE](#), [cf_epoch_sync\(\)](#), [CF_OF](#), [cf_push_item\(\)](#), [cf_sp](#), [cf_stack](#), [control_forth_0branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [DictEntry::func](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_call\(\)](#), and [vm_find_word\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.17 control_forth_plus_loop()

```
static void control_forth_plus_loop (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_DO](#), [cf_epoch_sync\(\)](#), [cf_peek_item\(\)](#), [cf_pop_item\(\)](#), [CF_WHILE](#), [control_forth_runtime_plus_loop\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [leave_addrs](#), [leave_mark_sp](#), [leave_mark_stack](#), [leave_sp](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.18 control_forth_qdo()

```
static void control_forth_qdo (
    VM * vm ) [static]
```

References [CF_DO](#), [cf_epoch_sync\(\)](#), [cf_push_item\(\)](#), [CF_WHILE](#), [control_forth_runtime_qdo\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [leave_mark_sp](#), [leave_mark_stack](#), [leave_sp](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.19 control_forth_repeat()

```
static void control_forth_repeat (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_BEGIN](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [CF_WHILE](#), [control_forth_branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.20 control_forth_runtime_do()

```
static void control_forth_runtime_do (
    VM * vm ) [static]
```

Runtime handler for DO loops.

References [VM::dsp](#), [VM::error](#), [limit](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [control_forth_do\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.21 control_forth_runtime_leave()

```
static void control_forth_runtime_leave (
    VM * vm ) [static]
```

Runtime handler for LEAVE.

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [control_forth_leave\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.22 control_forth_runtime_loop()

```
static void control_forth_runtime_loop (
    VM * vm ) [static]
```

Runtime handler for LOOP.

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [control_forth_loop\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.23 control_forth_runtime_plus_loop()

```
static void control_forth_runtime_plus_loop (
    VM * vm ) [static]
```

Runtime handler for +LOOP.

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [control_forth_plus_loop\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.24 control_forth_runtime_qdo()

```
static void control_forth_runtime_qdo (
    VM * vm ) [static]
```

Runtime handler for ?DO loops.

References [VM::dsp](#), [VM::error](#), [limit](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [control_forth_qdo\(\)](#), and [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.25 control_forth_then()

```
static void control_forth_then (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_ELSE](#), [cf_epoch_sync\(\)](#), [CF_IF](#), [cf_pop_item\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), and [vm](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.26 control_forth_UNLOOP()

```
static void control_forth_UNLOOP (
    VM * vm ) [static]
```

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.27 control_forth_until()

```
static void control_forth_until (
    VM * vm ) [static]
```

References [cf_item_t::addr](#), [CF_BEGIN](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [control_forth_0branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.28 control_forth_while()

```
static void control_forth_while (
    VM * vm ) [static]
```

References [CF_BEGIN](#), [cf_epoch_sync\(\)](#), [cf_peek_item\(\)](#), [cf_push_item\(\)](#), [CF_WHILE](#), [control_forth_0branch\(\)](#), [emit_cell\(\)](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [cf_item_t::tag](#), [vm](#), and [vm_compile_call\(\)](#).

Referenced by [register_control_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.2.29 register_control_words()

```
void register_control_words (
    VM * vm )
```

Registers all control flow words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [control_forth_0branch\(\)](#), [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_branch\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_EXIT\(\)](#), [control_forth_I\(\)](#), [control_forth_if\(\)](#), [control_forth_J\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_then\(\)](#), [control_forth_UNLOOP\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [register_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.9.3 Variable Documentation

5.9.3.1 addr

```
size_t cf_item_t::addr
```

Referenced by [cf_push_item\(\)](#), [control_forth_again\(\)](#), [control_forth_else\(\)](#), [control_forth_endof\(\)](#), [control_forth_loop\(\)](#),

`control_forth_plus_loop()`, `control_forth_repeat()`, `control_forth_then()`, and `control_forth_until()`.

5.9.3.2 tag

`cf_tag_t cf_item_t::tag`

Referenced by `cf_push_item()`, `control_forth_again()`, `control_forth_else()`, `control_forth_endcase()`, `control_forth_endof()`, `control_forth_loop()`, `control_forth_plus_loop()`, `control_forth_repeat()`, `control_forth_then()`, `control_forth_until()`, and `control_forth_while()`.

5.10 FORTH-79 Control Flow Words

Standard FORTH-79 control flow words implementation.

Standard FORTH-79 control flow words implementation.

5.11 FORTH-79 Defining Words

Standard FORTH-79 defining words.

Standard FORTH-79 defining words.

These words are used for creating new dictionary entries and defining words:

- `:` (`-`) Start colon definition
- `;` (`-`) End colon definition
- `CREATE` (`-`) Create dictionary entry
- `DOES>` (`-`) Define runtime behavior
- `<BUILDS` (`--`) Create word with `<BUILDS/DOES>`
- `CONSTANT` (`n -`) Define constant
- `VARIABLE` (`-`) Define variable
- `2CONSTANT` (`d -`) Define double constant
- `2VARIABLE` (`-`) Define double variable
- `USER` (`n -`) Define user variable
- `IMMEDIATE` (`-`) Make most recent word immediate
- `[COMPILE]` (`-`) Force compilation of immediate word
- `COMPILE` (`-`) Compile inline code
- `FORGET` (`-`) Remove word and all after it

5.12 FORTH-79 Dictionary Manipulation Words

Standard FORTH-79 words for dictionary manipulation.

Standard FORTH-79 words for dictionary manipulation.

`[(-)`

Enter interpretation mode. Switches the system to interpret incoming words.

`]( -)`

Enter compilation mode. Switches the system to compile incoming words.

STATE (– addr)

Variable containing current compilation state. Returns address of STATE variable.

SMUDGE (–)

Toggle smudge bit of latest defined word. Used during word definition.

>BODY (xt – addr)

Convert execution token to parameter field address.

>NAME (xt – addr)

Convert execution token to name field address.

NAME> (addr – xt)

Convert name field address to execution token.

>LINK (addr – addr)

Get link field address from name field address.

LINK> (addr – addr)

Get next word's address from link field.

CFA (addr – xt)

Get code field address from name field address.

LFA (addr – addr)

Get link field address from name field address.

NFA (addr – addr)

Get name field address from any field address.

PFA (addr – addr)

Get parameter field address from name field address.

TRAVERSE (addr n – addr)

Move through name field by n characters.

INTERPRET (–)

Text interpreter - process input stream.

5.13 FORTH-79 Dictionary & Compilation Words

Dictionary and compilation words as defined in FORTH-79 Standard.

Dictionary and compilation words as defined in FORTH-79 Standard.

@word HERE (– addr) Returns address of next free dictionary space @word ALLOT (n –) Allocates n bytes in dictionary @word , (n –) Compiles n into dictionary @word C, (c –) Compiles character into dictionary @word 2, (d –) Compiles double into dictionary @word PAD (– addr) Returns address of temporary text buffer @word SP! (addr –) Sets data stack pointer @word SP@ (– addr) Gets data stack pointer @word LATEST (– addr) Returns address of most recent definition

5.14 FORTH-79 Line Editor Words

Line editor commands implementation.

Line editor commands implementation.

5.15 FORTH-79 Logical Operations

FORTH-79 Standard logical and comparison operations.

FORTH-79 Standard logical and comparison operations.

Implementation of the following FORTH-79 words:

- AND (n1 n2 – n3) Bitwise AND
- OR (n1 n2 – n3) Bitwise OR
- XOR (n1 n2 – n3) Bitwise XOR
- NOT (n1 – n2) Bitwise NOT
- 0= (n – flag) True if n is zero
- 0< (n – flag) True if n is negative
- 0> (n – flag) True if n is positive
- = (n1 n2 – flag) True if n1 equals n2
- < (n1 n2 – flag) True if n1 < n2
- > (n1 n2 – flag) True if n1 > n2
- U< (u1 u2 – flag) True if u1 < u2 (unsigned)
- U> (u1 u2 – flag) True if u1 > u2 (unsigned)
- WITHIN (n low high – flag) True if low <= n < high

5.16 FORTH-79 Memory Operations

Memory operation words defined in FORTH-79 standard.

Memory operation words defined in FORTH-79 standard.

Available operations:

- ! (n addr –) Store n at addr
- @ (addr – n) Fetch n from addr
- C! (c addr –) Store character at addr
- C@ (addr – c) Fetch character from addr
- +! (n addr –) Add n to value at addr
- 2! (d addr –) Store double at addr
- 2@ (addr – d) Fetch double from addr
- FILL (addr u c –) Fill u bytes at addr with c
- MOVE (addr1 addr2 u –) Move u bytes from addr1 to addr2
- CMOVE (addr1 addr2 u –) Move u bytes, low to high addresses
- CMOVE> (addr1 addr2 u –) Move u bytes, high to low addresses
- ERASE (addr u –) Fill u bytes at addr with zeros

5.17 FORTH-79 Return Stack Operations

5.18 FORTH-79 System & Environment Words

System and environment control words for FORTH-79.

System and environment control words for FORTH-79.

The following words provide system and environment control functionality:

COLD (-)

Performs a cold start of the system, reinitializing all system variables

WARM (-)

Performs a warm start of the system, preserving some system state

BYE (-)

Exits the FORTH system and returns to the host operating system

SAVE-SYSTEM (-)

Saves the current system image to disk

WORDS (-)

Lists all words in the current vocabulary

VLIST (-)

Lists all words in all vocabularies

?STACK (-)

Checks the stack depth and reports any stack errors

PAGE (-)

Clears the screen or outputs a form feed

NOP (-)

Performs no operation

79-STANDARD (-)

Ensures FORTH-79 standard compliance

5.19 FORTH-79 Vocabulary System Words

Standard vocabulary manipulation words.

Standard vocabulary manipulation words.

VOCABULARY (-)

Create new vocabulary

DEFINITIONS (-)

Set current vocabulary

CONTEXT (- addr)

Variable: search vocabulary pointer

CURRENT (- addr)

Variable: definition vocabulary pointer

FORTH (-)

Select FORTH vocabulary

ONLY (-)

Set minimal search order

ALSO (-)

Duplicate top of search order

ORDER (-)

Display search order

PREVIOUS (-)

Remove top of search order

(FIND) (addr – addr flag)

Find word in dictionary (primitive)

Chapter 6

Data Structure Documentation

6.1 `_EFI_BOOT_SERVICES` Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for `_EFI_BOOT_SERVICES`:

Data Fields

- [EFI_TABLE_HEADER Hdr](#)
- [EFI_RAISE_TPL RaiseTPL](#)
- [EFI_RESTORE_TPL RestoreTPL](#)
- [EFI_ALLOCATE_PAGES AllocatePages](#)
- [EFI_FREE_PAGES FreePages](#)
- [EFI_GET_MEMORY_MAP GetMemoryMap](#)
- [EFI_ALLOCATE_POOL AllocatePool](#)
- [EFI_FREE_POOL FreePool](#)
- void * [CreateEvent](#)
- void * [SetTimer](#)
- void * [WaitForEvent](#)
- void * [SignalEvent](#)
- void * [CloseEvent](#)
- void * [CheckEvent](#)
- void * [InstallProtocolInterface](#)
- void * [ReinstallProtocolInterface](#)
- void * [UninstallProtocolInterface](#)
- [EFI_HANDLE_PROTOCOL HandleProtocol](#)
- void * [Reserved](#)
- void * [RegisterProtocolNotify](#)
- [EFI_LOCATE_HANDLE LocateHandle](#)
- void * [LocateDevicePath](#)
- void * [InstallConfigurationTable](#)
- void * [LoadImage](#)
- void * [StartImage](#)
- void * [Exit](#)
- void * [UnloadImage](#)
- [EFI_EXIT_BOOT_SERVICES ExitBootServices](#)
- void * [GetNextMonotonicCount](#)
- void * [Stall](#)
- void * [SetWatchdogTimer](#)

6.1.1 Field Documentation

6.1.1.1 AllocatePages

[EFI_ALLOCATE_PAGES](#) `_EFI_BOOT_SERVICES::AllocatePages`

Referenced by [load_kernel_from_esp\(\)](#).

6.1.1.2 AllocatePool

[EFI_ALLOCATE_POOL](#) `_EFI_BOOT_SERVICES::AllocatePool`

6.1.1.3 CheckEvent

`void* _EFI_BOOT_SERVICES::CheckEvent`

6.1.1.4 CloseEvent

`void* _EFI_BOOT_SERVICES::CloseEvent`

6.1.1.5 CreateEvent

`void* _EFI_BOOT_SERVICES::CreateEvent`

6.1.1.6 Exit

`void* _EFI_BOOT_SERVICES::Exit`

6.1.1.7 ExitBootServices

[EFI_EXIT_BOOT_SERVICES](#) `_EFI_BOOT_SERVICES::ExitBootServices`

Referenced by [efi_main\(\)](#).

6.1.1.8 FreePages

[EFI_FREE_PAGES](#) `_EFI_BOOT_SERVICES::FreePages`

6.1.1.9 FreePool

[EFI_FREE_POOL](#) `_EFI_BOOT_SERVICES::FreePool`

6.1.1.10 GetMemoryMap

[EFI_GET_MEMORY_MAP](#) `_EFI_BOOT_SERVICES::GetMemoryMap`

Referenced by [efi_main\(\)](#).

6.1.1.11 GetNextMonotonicCount

`void* _EFI_BOOT_SERVICES::GetNextMonotonicCount`

6.1.1.12 HandleProtocol

[EFI_HANDLE_PROTOCOL](#) `_EFI_BOOT_SERVICES::HandleProtocol`

Referenced by [load_kernel_from_esp\(\)](#).

6.1.1.13 Hdr

[EFI_TABLE_HEADER](#) `_EFI_BOOT_SERVICES::Hdr`

6.1.1.14 InstallConfigurationTable

`void* _EFI_BOOT_SERVICES::InstallConfigurationTable`

6.1.1.15 InstallProtocolInterface

```
void* _EFI_BOOT_SERVICES::InstallProtocolInterface
```

6.1.1.16 LoadImage

```
void* _EFI_BOOT_SERVICES::LoadImage
```

6.1.1.17 LocateDevicePath

```
void* _EFI_BOOT_SERVICES::LocateDevicePath
```

6.1.1.18 LocateHandle

```
EFI\_LOCATE\_HANDLE _EFI_BOOT_SERVICES::LocateHandle
```

6.1.1.19 RaiseTPL

```
EFI\_RAISE\_TPL _EFI_BOOT_SERVICES::RaiseTPL
```

6.1.1.20 RegisterProtocolNotify

```
void* _EFI_BOOT_SERVICES::RegisterProtocolNotify
```

6.1.1.21 ReinstallProtocolInterface

```
void* _EFI_BOOT_SERVICES::ReinstallProtocolInterface
```

6.1.1.22 Reserved

```
void* _EFI_BOOT_SERVICES::Reserved
```

6.1.1.23 RestoreTPL

```
EFI\_RESTORE\_TPL _EFI_BOOT_SERVICES::RestoreTPL
```

6.1.1.24 SetTimer

```
void* _EFI_BOOT_SERVICES::SetTimer
```

6.1.1.25 SetWatchdogTimer

```
void* _EFI_BOOT_SERVICES::SetWatchdogTimer
```

6.1.1.26 SignalEvent

```
void* _EFI_BOOT_SERVICES::SignalEvent
```

6.1.1.27 Stall

```
void* _EFI_BOOT_SERVICES::Stall
```

6.1.1.28 StartImage

```
void* _EFI_BOOT_SERVICES::StartImage
```

6.1.1.29 UninstallProtocolInterface

```
void* _EFI_BOOT_SERVICES::UninstallProtocolInterface
```

6.1.1.30 UnloadImage

```
void* _EFI_BOOT_SERVICES::UnloadImage
```

6.1.1.31 WaitForEvent

```
void* _EFI_BOOT_SERVICES::WaitForEvent
```

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.2 _EFI_FILE_PROTOCOL Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for _EFI_FILE_PROTOCOL:

Data Fields

- [UINT64](#) Revision
- [EFI_FILE_OPEN](#) Open
- [EFI_FILE_CLOSE](#) Close
- [VOID *](#) Delete
- [EFI_FILE_READ](#) Read
- [VOID *](#) Write
- [VOID *](#) GetPosition
- [VOID *](#) SetPosition
- [EFI_FILE_GET_INFO](#) GetInfo
- [VOID *](#) SetInfo
- [VOID *](#) Flush

6.2.1 Field Documentation

6.2.1.1 Close

```
EFI\_FILE\_CLOSE _EFI_FILE_PROTOCOL::Close
```

Referenced by [load_kernel_from_esp\(\)](#).

6.2.1.2 Delete

```
VOID\* _EFI_FILE_PROTOCOL::Delete
```

6.2.1.3 Flush

```
VOID\* _EFI_FILE_PROTOCOL::Flush
```

6.2.1.4 GetInfo

```
EFI\_FILE\_GET\_INFO _EFI_FILE_PROTOCOL::GetInfo
```

Referenced by [load_kernel_from_esp\(\)](#).

6.2.1.5 GetPosition

```
VOID\* _EFI_FILE_PROTOCOL::GetPosition
```

6.2.1.6 Open

```
EFI\_FILE\_OPEN _EFI_FILE_PROTOCOL::Open
```

Referenced by [load_kernel_from_esp\(\)](#).

6.2.1.7 Read

`EFI_FILE_READ` `_EFI_FILE_PROTOCOL::Read`
 Referenced by `load_kernel_from_esp()`.

6.2.1.8 Revision

`UINT64` `_EFI_FILE_PROTOCOL::Revision`

6.2.1.9 SetInfo

`VOID*` `_EFI_FILE_PROTOCOL::SetInfo`

6.2.1.10 SetPosition

`VOID*` `_EFI_FILE_PROTOCOL::SetPosition`

6.2.1.11 Write

`VOID*` `_EFI_FILE_PROTOCOL::Write`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.3 _EFI_RUNTIME_SERVICES Struct Reference

`#include <uefi.h>`

Collaboration diagram for `_EFI_RUNTIME_SERVICES`:

Data Fields

- `EFI_TABLE_HEADER` `Hdr`
- `void *` `GetTime`
- `void *` `SetTime`
- `void *` `GetWakeupTime`
- `void *` `SetWakeupTime`
- `void *` `SetVirtualAddressMap`
- `void *` `ConvertPointer`
- `void *` `GetVariable`
- `void *` `GetNextVariableName`
- `void *` `SetVariable`
- `void *` `GetNextHighMonotonicCount`
- `void *` `ResetSystem`

6.3.1 Field Documentation

6.3.1.1 ConvertPointer

`void*` `_EFI_RUNTIME_SERVICES::ConvertPointer`

6.3.1.2 GetNextHighMonotonicCount

`void*` `_EFI_RUNTIME_SERVICES::GetNextHighMonotonicCount`

6.3.1.3 GetNextVariableName

`void*` `_EFI_RUNTIME_SERVICES::GetNextVariableName`

6.3.1.4 GetTime

```
void* _EFI_RUNTIME_SERVICES::GetTime
```

6.3.1.5 GetVariable

```
void* _EFI_RUNTIME_SERVICES::GetVariable
```

6.3.1.6 GetWakeupTime

```
void* _EFI_RUNTIME_SERVICES::GetWakeupTime
```

6.3.1.7 Hdr

```
EFI\_TABLE\_HEADER _EFI_RUNTIME_SERVICES::Hdr
```

6.3.1.8 ResetSystem

```
void* _EFI_RUNTIME_SERVICES::ResetSystem
```

6.3.1.9 SetTime

```
void* _EFI_RUNTIME_SERVICES::SetTime
```

6.3.1.10 SetVariable

```
void* _EFI_RUNTIME_SERVICES::SetVariable
```

6.3.1.11 SetVirtualAddressMap

```
void* _EFI_RUNTIME_SERVICES::SetVirtualAddressMap
```

6.3.1.12 SetWakeupTime

```
void* _EFI_RUNTIME_SERVICES::SetWakeupTime
```

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.4 _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL:

Data Fields

- [UINT64](#) [Revision](#)
- [EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME](#) [OpenVolume](#)

6.4.1 Field Documentation

6.4.1.1 OpenVolume

```
EFI\_SIMPLE\_FILE\_SYSTEM\_PROTOCOL\_OPEN\_VOLUME _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL::OpenVolume
```

Referenced by [load_kernel_from_esp\(\)](#).

6.4.1.2 Revision

```
UINT64 _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL::Revision
```

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.5 _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL:

Data Fields

- [EFI_TEXT_RESET](#) Reset
- [EFI_TEXT_STRING](#) OutputString
- void * [TestString](#)
- void * [QueryMode](#)
- void * [SetMode](#)
- void * [SetAttribute](#)
- void * [ClearScreen](#)
- void * [SetCursorPosition](#)
- void * [EnableCursor](#)
- void * [Mode](#)

6.5.1 Field Documentation

6.5.1.1 ClearScreen

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::ClearScreen
```

6.5.1.2 EnableCursor

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::EnableCursor
```

6.5.1.3 Mode

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::Mode
```

6.5.1.4 OutputString

```
EFI\_TEXT\_STRING _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::OutputString
```

Referenced by [efi_main\(\)](#).

6.5.1.5 QueryMode

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::QueryMode
```

6.5.1.6 Reset

```
EFI\_TEXT\_RESET _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::Reset
```

Referenced by [efi_main\(\)](#).

6.5.1.7 SetAttribute

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::SetAttribute
```

6.5.1.8 SetCursorPosition

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::SetCursorPosition
```

6.5.1.9 SetMode

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::SetMode
```

6.5.1.10 TestString

```
void* _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::TestString
```

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.6 _EFI_SYSTEM_TABLE Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for _EFI_SYSTEM_TABLE:

Data Fields

- [EFI_TABLE_HEADER](#) Hdr
- [CHAR16](#) * FirmwareVendor
- [UINT32](#) FirmwareRevision
- [EFI_HANDLE](#) ConsoleInHandle
- [void](#) * ConIn
- [EFI_HANDLE](#) ConsoleOutHandle
- [EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL](#) * ConOut
- [EFI_HANDLE](#) StandardErrorHandle
- [EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL](#) * StdErr
- [EFI_RUNTIME_SERVICES](#) * RuntimeServices
- [EFI_BOOT_SERVICES](#) * BootServices
- [UINTN](#) NumberOfTableEntries
- [void](#) * ConfigurationTable

6.6.1 Field Documentation

6.6.1.1 BootServices

```
EFI\_BOOT\_SERVICES* _EFI_SYSTEM_TABLE::BootServices
```

Referenced by [efi_main\(\)](#).

6.6.1.2 ConfigurationTable

```
void* _EFI_SYSTEM_TABLE::ConfigurationTable
```

Referenced by [efi_main\(\)](#).

6.6.1.3 ConIn

```
void* _EFI_SYSTEM_TABLE::ConIn
```

6.6.1.4 ConOut

```
EFI\_SIMPLE\_TEXT\_OUTPUT\_PROTOCOL* _EFI_SYSTEM_TABLE::ConOut
```

Referenced by [efi_main\(\)](#).

6.6.1.5 ConsoleInHandle

```
EFI\_HANDLE _EFI_SYSTEM_TABLE::ConsoleInHandle
```

6.6.1.6 ConsoleOutHandle

```
EFI\_HANDLE _EFI_SYSTEM_TABLE::ConsoleOutHandle
```

6.6.1.7 FirmwareRevision

```
UINT32 _EFI_SYSTEM_TABLE::FirmwareRevision
```

6.6.1.8 FirmwareVendor

`CHAR16* _EFI_SYSTEM_TABLE::FirmwareVendor`

6.6.1.9 Hdr

`EFI_TABLE_HEADER _EFI_SYSTEM_TABLE::Hdr`

6.6.1.10 NumberOfTableEntries

`UINTN _EFI_SYSTEM_TABLE::NumberOfTableEntries`

Referenced by [efi_main\(\)](#).

6.6.1.11 RuntimeServices

`EFI_RUNTIME_SERVICES* _EFI_SYSTEM_TABLE::RuntimeServices`

Referenced by [efi_main\(\)](#).

6.6.1.12 StandardErrorHandle

`EFI_HANDLE _EFI_SYSTEM_TABLE::StandardErrorHandle`

6.6.1.13 StdErr

`EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL* _EFI_SYSTEM_TABLE::StdErr`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.7 BayesianLatencyPosterior Struct Reference

Bayesian statistics structure for inference and credible intervals All values stored in Q48.16 fixed-point (64-bit signed integer) representing nanoseconds with $2^{-16} \approx 0.0000153$ ns precision.

```
#include <physics_hotwords_cache.h>
```

Data Fields

- `int64_t mean_ns_q48`
- `int64_t stddev_ns_q48`
- `int64_t median_ns_q48`
- `int64_t credible_lower_95`
- `int64_t credible_upper_95`
- `int64_t credible_lower_99`
- `int64_t credible_upper_99`
- `uint64_t sample_count`

6.7.1 Detailed Description

Bayesian statistics structure for inference and credible intervals All values stored in Q48.16 fixed-point (64-bit signed integer) representing nanoseconds with $2^{-16} \approx 0.0000153$ ns precision.

6.7.2 Field Documentation

6.7.2.1 credible_lower_95

`int64_t BayesianLatencyPosterior::credible_lower_95`

Referenced by [calculate_posterior\(\)](#), and [hotwords_bayesian_report\(\)](#).

6.7.2.2 credible_lower_99

`int64_t BayesianLatencyPosterior::credible_lower_99`
 Referenced by [calculate_posterior\(\)](#), and [hotwords_bayesian_report\(\)](#).

6.7.2.3 credible_upper_95

`int64_t BayesianLatencyPosterior::credible_upper_95`
 Referenced by [calculate_posterior\(\)](#), and [hotwords_bayesian_report\(\)](#).

6.7.2.4 credible_upper_99

`int64_t BayesianLatencyPosterior::credible_upper_99`
 Referenced by [calculate_posterior\(\)](#), and [hotwords_bayesian_report\(\)](#).

6.7.2.5 mean_ns_q48

`int64_t BayesianLatencyPosterior::mean_ns_q48`
 Referenced by [calculate_posterior\(\)](#), [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

6.7.2.6 median_ns_q48

`int64_t BayesianLatencyPosterior::median_ns_q48`
 Referenced by [calculate_posterior\(\)](#).

6.7.2.7 sample_count

`uint64_t BayesianLatencyPosterior::sample_count`
 Referenced by [calculate_posterior\(\)](#), and [hotwords_bayesian_report\(\)](#).

6.7.2.8 stddev_ns_q48

`int64_t BayesianLatencyPosterior::stddev_ns_q48`
 Referenced by [calculate_posterior\(\)](#), [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).
 The documentation for this struct was generated from the following file:

- `include/physics_hotwords_cache.h`

6.8 blk_meta_t Struct Reference

```
#include <block_subsystem.h>
```

Data Fields

- `uint64_t checksum`
- `uint64_t magic`
- `uint64_t created_time`
- `uint64_t modified_time`
- `uint64_t flags`
- `uint64_t write_count`
- `uint64_t content_type`
- `uint64_t encoding`
- `uint64_t content_length`
- `uint64_t reserved1`
- `uint64_t entropy [4]`
- `uint64_t hash [4]`
- `uint64_t owner_id`
- `uint64_t permissions`

- uint64_t [acl_block](#)
- uint64_t [signature](#) [2]
- uint64_t [prev_block](#)
- uint64_t [next_block](#)
- uint64_t [parent_block](#)
- uint64_t [chain_length](#)
- uint64_t [app_data](#) [15]
- uint8_t [padding](#) [5]

6.8.1 Field Documentation

6.8.1.1 acl_block

uint64_t blk_meta_t::acl_block

6.8.1.2 app_data

uint64_t blk_meta_t::app_data[15]

6.8.1.3 chain_length

uint64_t blk_meta_t::chain_length

6.8.1.4 checksum

uint64_t blk_meta_t::checksum

Referenced by [blk_update\(\)](#).

6.8.1.5 content_length

uint64_t blk_meta_t::content_length

6.8.1.6 content_type

uint64_t blk_meta_t::content_type

6.8.1.7 created_time

uint64_t blk_meta_t::created_time

Referenced by [blk_update\(\)](#).

6.8.1.8 encoding

uint64_t blk_meta_t::encoding

6.8.1.9 entropy

uint64_t blk_meta_t::entropy[4]

6.8.1.10 flags

uint64_t blk_meta_t::flags

6.8.1.11 hash

uint64_t blk_meta_t::hash[4]

6.8.1.12 magic

`uint64_t blk_meta_t::magic`

Referenced by [blk_get_meta\(\)](#), [blk_update\(\)](#), and [cache_load_devblock\(\)](#).

6.8.1.13 modified_time

`uint64_t blk_meta_t::modified_time`

Referenced by [blk_update\(\)](#).

6.8.1.14 next_block

`uint64_t blk_meta_t::next_block`

6.8.1.15 owner_id

`uint64_t blk_meta_t::owner_id`

6.8.1.16 padding

`uint8_t blk_meta_t::padding[5]`

6.8.1.17 parent_block

`uint64_t blk_meta_t::parent_block`

6.8.1.18 permissions

`uint64_t blk_meta_t::permissions`

6.8.1.19 prev_block

`uint64_t blk_meta_t::prev_block`

6.8.1.20 reserved1

`uint64_t blk_meta_t::reserved1`

6.8.1.21 signature

`uint64_t blk_meta_t::signature[2]`

6.8.1.22 write_count

`uint64_t blk_meta_t::write_count`

The documentation for this struct was generated from the following file:

- `include/block_subsystem.h`

6.9 blk_volume_meta_t Struct Reference

```
#include <block_subsystem.h>
```

Data Fields

- `uint32_t` [magic](#)
- `uint32_t` [version](#)
- `uint32_t` [total_volumes](#)
- `uint32_t` [flags](#)
- `char` [label](#) [64]

- uint64_t [total_devblocks](#)
- uint32_t [bam_start](#)
- uint32_t [bam_devblocks](#)
- uint32_t [devblock_base](#)
- uint64_t [tracked_blocks](#)
- uint64_t [total_blocks](#)
- uint64_t [free_blocks](#)
- uint64_t [first_free](#)
- uint64_t [last_allocated](#)
- uint32_t [reserved_disk_lo](#)
- uint32_t [reserved_ram_lo](#)
- uint64_t [created_time](#)
- uint64_t [mounted_time](#)
- uint64_t [hdr_crc](#)
- uint8_t [_pad](#) [4096 - (4+4+ 4+4+64+ 8+ 4+4+4+ 8+8+8+ 8+8+ 4+4+ 8+8+ 8)]

6.9.1 Field Documentation

6.9.1.1 _pad

```
uint8_t blk_volume_meta_t::_pad[4096 - ( 4+4+ 4+4+64+ 8+ 4+4+4+ 8+8+8+ 8+8+ 4+4+ 8+8+ 8)]
```

6.9.1.2 bam_devblocks

```
uint32_t blk_volume_meta_t::bam_devblocks
```

Referenced by [compute_totals_from_B\(\)](#).

6.9.1.3 bam_start

```
uint32_t blk_volume_meta_t::bam_start
```

6.9.1.4 created_time

```
uint64_t blk_volume_meta_t::created_time
```

6.9.1.5 devblock_base

```
uint32_t blk_volume_meta_t::devblock_base
```

6.9.1.6 first_free

```
uint64_t blk_volume_meta_t::first_free
```

Referenced by [compute_totals_from_B\(\)](#).

6.9.1.7 flags

```
uint32_t blk_volume_meta_t::flags
```

6.9.1.8 free_blocks

```
uint64_t blk_volume_meta_t::free_blocks
```

6.9.1.9 hdr_crc

```
uint64_t blk_volume_meta_t::hdr_crc
```

6.9.1.10 label

```
char blk_volume_meta_t::label[64]
```

6.9.1.11 last_allocated

uint64_t blk_volume_meta_t::last_allocated
Referenced by [compute_totals_from_B\(\)](#).

6.9.1.12 magic

uint32_t blk_volume_meta_t::magic

6.9.1.13 mounted_time

uint64_t blk_volume_meta_t::mounted_time

6.9.1.14 reserved_disk_lo

uint32_t blk_volume_meta_t::reserved_disk_lo

6.9.1.15 reserved_ram_lo

uint32_t blk_volume_meta_t::reserved_ram_lo

6.9.1.16 total_blocks

uint64_t blk_volume_meta_t::total_blocks
Referenced by [compute_totals_from_B\(\)](#).

6.9.1.17 total_devblocks

uint64_t blk_volume_meta_t::total_devblocks
Referenced by [compute_totals_from_B\(\)](#).

6.9.1.18 total_volumes

uint32_t blk_volume_meta_t::total_volumes

6.9.1.19 tracked_blocks

uint64_t blk_volume_meta_t::tracked_blocks
Referenced by [compute_totals_from_B\(\)](#).

6.9.1.20 version

uint32_t blk_volume_meta_t::version

The documentation for this struct was generated from the following file:

- [include/block_subsystem.h](#)

6.10 blkcfg_t Struct Reference

```
#include <blkcfg.h>
```

Data Fields

- [blkcfg_backend_t](#) backend
- uint32_t fbs
- uint8_t read_only
- uint64_t size_bytes
- uint32_t blocks
- char file_path [4096]

6.10.1 Field Documentation

6.10.1.1 backend

`blkcfg_backend_t blkcfg_t::backend`

6.10.1.2 blocks

`uint32_t blkcfg_t::blocks`

6.10.1.3 fbs

`uint32_t blkcfg_t::fbs`

6.10.1.4 file_path

`char blkcfg_t::file_path[4096]`

6.10.1.5 read_only

`uint8_t blkcfg_t::read_only`

6.10.1.6 size_bytes

`uint64_t blkcfg_t::size_bytes`

The documentation for this struct was generated from the following file:

- [include/blkcfg.h](#)

6.11 blkio_dev Struct Reference

```
#include <blkio.h>
```

Collaboration diagram for blkio_dev:

Data Fields

- const [blkio_vtable_t](#) * `vt`
- `uint32_t` [forth_block_size](#)
- `uint32_t` [total_blocks](#)
- void * `state`
- `uint8_t` [is_open](#)

6.11.1 Field Documentation

6.11.1.1 forth_block_size

`uint32_t blkio_dev::forth_block_size`

Referenced by [blkio_info\(\)](#), [blkio_open\(\)](#), [file_open\(\)](#), and [ram_open\(\)](#).

6.11.1.2 is_open

`uint8_t blkio_dev::is_open`

Referenced by [blkio_close\(\)](#), [blkio_flush\(\)](#), [blkio_info\(\)](#), [blkio_open\(\)](#), [blkio_read\(\)](#), and [blkio_write\(\)](#).

6.11.1.3 state

`void* blkio_dev::state`

Referenced by [blkio_open\(\)](#), [file_close\(\)](#), [file_flush\(\)](#), [file_info\(\)](#), [file_open\(\)](#), [file_read\(\)](#), [file_write\(\)](#), [ram_close\(\)](#), [ram_info\(\)](#), [ram_open\(\)](#), [ram_read\(\)](#), and [ram_write\(\)](#).

6.11.1.4 total_blocks

```
uint32_t blkio_dev::total_blocks
```

Referenced by [blkio_info\(\)](#), [blkio_open\(\)](#), [blkio_read\(\)](#), [blkio_write\(\)](#), [file_open\(\)](#), [file_read\(\)](#), [file_write\(\)](#), and [ram_open\(\)](#).

6.11.1.5 vt

```
const blkio_vtable_t* blkio_dev::vt
```

Referenced by [blkio_close\(\)](#), [blkio_flush\(\)](#), [blkio_info\(\)](#), [blkio_open\(\)](#), [blkio_read\(\)](#), and [blkio_write\(\)](#).

The documentation for this struct was generated from the following file:

- [include/blkio.h](#)

6.12 blkio_file_state_t Struct Reference

Data Fields

- [int fd](#)
- [char path](#) [4096]
- [uint32_t total_blocks](#)
- [uint32_t fbs](#)
- [uint8_t read_only](#)

6.12.1 Field Documentation

6.12.1.1 fbs

```
uint32_t blkio_file_state_t::fbs
```

Referenced by [blkio_file_init_state\(\)](#), [file_info\(\)](#), [file_open\(\)](#), [file_read\(\)](#), and [file_write\(\)](#).

6.12.1.2 fd

```
int blkio_file_state_t::fd
```

Referenced by [blkio_file_init_state\(\)](#), [file_close\(\)](#), [file_flush\(\)](#), [file_info\(\)](#), [file_open\(\)](#), [file_read\(\)](#), and [file_write\(\)](#).

6.12.1.3 path

```
char blkio_file_state_t::path[4096]
```

Referenced by [blkio_file_init_state\(\)](#), and [file_open\(\)](#).

6.12.1.4 read_only

```
uint8_t blkio_file_state_t::read_only
```

Referenced by [blkio_file_init_state\(\)](#), [file_info\(\)](#), [file_open\(\)](#), and [file_write\(\)](#).

6.12.1.5 total_blocks

```
uint32_t blkio_file_state_t::total_blocks
```

Referenced by [blkio_file_init_state\(\)](#), [file_info\(\)](#), and [file_open\(\)](#).

The documentation for this struct was generated from the following file:

- [src/blkio_file.c](#)

6.13 blkio_info_t Struct Reference

```
#include <blkio.h>
```

Data Fields

- uint32_t [forth_block_size](#)
- uint32_t [total_blocks](#)
- uint32_t [phys_sector_size](#)
- uint64_t [phys_size_bytes](#)
- uint8_t [read_only](#)

6.13.1 Field Documentation**6.13.1.1 forth_block_size**

uint32_t blkio_info_t::forth_block_size

Referenced by [blkio_info\(\)](#), [file_info\(\)](#), [init_blkio\(\)](#), and [ram_info\(\)](#).

6.13.1.2 phys_sector_size

uint32_t blkio_info_t::phys_sector_size

Referenced by [file_info\(\)](#), and [ram_info\(\)](#).

6.13.1.3 phys_size_bytes

uint64_t blkio_info_t::phys_size_bytes

Referenced by [file_info\(\)](#), [init_blkio\(\)](#), and [ram_info\(\)](#).

6.13.1.4 read_only

uint8_t blkio_info_t::read_only

Referenced by [file_info\(\)](#), [init_blkio\(\)](#), and [ram_info\(\)](#).

6.13.1.5 total_blocks

uint32_t blkio_info_t::total_blocks

Referenced by [blk_subsys_attach_device\(\)](#), [blkio_info\(\)](#), [file_info\(\)](#), [init_blkio\(\)](#), and [ram_info\(\)](#).

The documentation for this struct was generated from the following file:

- [include/blkio.h](#)

6.14 blkio_params_t Struct Reference

```
#include <blkio.h>
```

Data Fields

- uint32_t [forth_block_size](#)
- uint32_t [total_blocks](#)
- void * [opaque](#)

6.14.1 Field Documentation**6.14.1.1 forth_block_size**

uint32_t blkio_params_t::forth_block_size

Referenced by [blkio_factory_open\(\)](#), and [blkio_open\(\)](#).

6.14.1.2 opaque

void* blkio_params_t::opaque

Referenced by [file_open\(\)](#), and [ram_open\(\)](#).

6.14.1.3 total_blocks

```
uint32_t blkio_params_t::total_blocks
```

Referenced by [blkio_open\(\)](#).

The documentation for this struct was generated from the following file:

- [include/blkio.h](#)

6.15 blkio_ram_state_t Struct Reference

Data Fields

- `uint8_t * base`
- `uint32_t total_blocks`
- `uint32_t fbs`
- `uint8_t read_only`

6.15.1 Field Documentation

6.15.1.1 base

```
uint8_t* blkio_ram_state_t::base
```

Referenced by [blkio_ram_init_state\(\)](#), [ram_open\(\)](#), [ram_read\(\)](#), and [ram_write\(\)](#).

6.15.1.2 fbs

```
uint32_t blkio_ram_state_t::fbs
```

Referenced by [blkio_ram_init_state\(\)](#), [ram_info\(\)](#), [ram_open\(\)](#), [ram_read\(\)](#), and [ram_write\(\)](#).

6.15.1.3 read_only

```
uint8_t blkio_ram_state_t::read_only
```

Referenced by [blkio_ram_init_state\(\)](#), [ram_info\(\)](#), and [ram_write\(\)](#).

6.15.1.4 total_blocks

```
uint32_t blkio_ram_state_t::total_blocks
```

Referenced by [blkio_ram_init_state\(\)](#), [in_bounds\(\)](#), [ram_info\(\)](#), and [ram_open\(\)](#).

The documentation for this struct was generated from the following file:

- [src/blkio_ram.c](#)

6.16 blkio_vtable_t Struct Reference

```
#include <blkio.h>
```

Data Fields

- `int(* open)(struct blkio_dev *dev, const blkio_params_t *p)`
- `int(* close)(struct blkio_dev *dev)`
- `int(* read)(struct blkio_dev *dev, uint32_t fblock, void *dst)`
- `int(* write)(struct blkio_dev *dev, uint32_t fblock, const void *src)`
- `int(* flush)(struct blkio_dev *dev)`
- `int(* info)(struct blkio_dev *dev, blkio_info_t *out)`

6.16.1 Field Documentation

6.16.1.1 close

```
int(* blkio_vtable_t::close)(struct blkio_dev *dev)
```

Referenced by [blkio_close\(\)](#), and [blkio_open\(\)](#).

6.16.1.2 flush

```
int (* blkio_vtable_t::flush) (struct blkio_dev *dev)
```

Referenced by [blkio_flush\(\)](#), and [blkio_open\(\)](#).

6.16.1.3 info

```
int (* blkio_vtable_t::info) (struct blkio_dev *dev, blkio_info_t *out)
```

Referenced by [blkio_info\(\)](#), and [blkio_open\(\)](#).

6.16.1.4 open

```
int (* blkio_vtable_t::open) (struct blkio_dev *dev, const blkio_params_t *p)
```

Referenced by [blkio_open\(\)](#).

6.16.1.5 read

```
int (* blkio_vtable_t::read) (struct blkio_dev *dev, uint32_t fblock, void *dst)
```

Referenced by [blkio_open\(\)](#), and [blkio_read\(\)](#).

6.16.1.6 write

```
int (* blkio_vtable_t::write) (struct blkio_dev *dev, uint32_t fblock, const void *src)
```

Referenced by [blkio_open\(\)](#), and [blkio_write\(\)](#).

The documentation for this struct was generated from the following file:

- [include/blkio.h](#)

6.17 BlockBuffer Struct Reference

Block buffer structure maintaining buffer state.

```
#include <block_words.h>
```

Data Fields

- unsigned char [data](#) [1024]
Block content buffer.
- int [block_number](#)
Currently loaded block number (-1 = empty)
- int [dirty](#)
Modified flag for write-back.
- int [accessed](#)
LRU counter for buffer management.
- [buffer_type_t](#) type
Buffer type for storage mechanism.
- uintptr_t [hw_address](#)
Future: Hardware backing address.

6.17.1 Detailed Description

Block buffer structure maintaining buffer state.

6.17.2 Field Documentation

6.17.2.1 accessed

```
int BlockBuffer::accessed
```

LRU counter for buffer management.

6.17.2.2 block_number

```
int BlockBuffer::block_number
```

Currently loaded block number (-1 = empty)

6.17.2.3 data

```
unsigned char BlockBuffer::data[1024]
```

Block content buffer.

6.17.2.4 dirty

```
int BlockBuffer::dirty
```

Modified flag for write-back.

6.17.2.5 hw_address

```
uintptr_t BlockBuffer::hw_address
```

Future: Hardware backing address.

6.17.2.6 type

```
buffer_type_t BlockBuffer::type
```

Buffer type for storage mechanism.

The documentation for this struct was generated from the following file:

- [src/word_source/include/block_words.h](#)

6.18 BootInfo Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for BootInfo:

Data Fields

- [EFI_MEMORY_DESCRIPTOR](#) * [memory_map](#)
- [UINTN](#) [memory_map_size](#)
- [UINTN](#) [memory_map_descriptor_size](#)
- [EFI_RUNTIME_SERVICES](#) * [runtime_services](#)
- [void](#) * [acpi_table](#)
- [FramebufferInfo](#) [framebuffer](#)
- [UINT8](#) [uefi_boot_services_exited](#)

6.18.1 Field Documentation

6.18.1.1 acpi_table

```
void* BootInfo::acpi_table
```

Referenced by [efi_main\(\)](#).

6.18.1.2 framebuffer

```
FramebufferInfo BootInfo::framebuffer
```

Referenced by [efi_main\(\)](#).

6.18.1.3 memory_map

```
EFI_MEMORY_DESCRIPTOR* BootInfo::memory_map
```

Referenced by [efi_main\(\)](#), [pmm_init\(\)](#), [print_boot_info\(\)](#), and [vmm_init\(\)](#).

6.18.1.4 memory_map_descriptor_size

`UINTN BootInfo::memory_map_descriptor_size`

Referenced by [efi_main\(\)](#), [pmm_init\(\)](#), [print_boot_info\(\)](#), and [vmm_init\(\)](#).

6.18.1.5 memory_map_size

`UINTN BootInfo::memory_map_size`

Referenced by [efi_main\(\)](#), [pmm_init\(\)](#), [print_boot_info\(\)](#), and [vmm_init\(\)](#).

6.18.1.6 runtime_services

`EFI_RUNTIME_SERVICES* BootInfo::runtime_services`

Referenced by [efi_main\(\)](#).

6.18.1.7 uefi_boot_services_exited

`UINT8 BootInfo::uefi_boot_services_exited`

Referenced by [efi_main\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.19 cache_slot_t Struct Reference

Collaboration diagram for `cache_slot_t`:

Data Fields

- `uint32_t devblock`
- `uint8_t data [BLK_DEVICE_SECTOR]`
- `blk_meta_t meta [BLK_PACK_RATIO]`
- `uint8_t valid`
- `uint8_t loaded`
- `uint8_t dirty`
- `uint8_t meta_dirty`

6.19.1 Field Documentation

6.19.1.1 data

`uint8_t cache_slot_t::data [BLK_DEVICE_SECTOR]`

Referenced by [blk_get_buffer\(\)](#), [blk_update\(\)](#), [cache_load_devblock\(\)](#), and [cache_writeback\(\)](#).

6.19.1.2 devblock

`uint32_t cache_slot_t::devblock`

Referenced by [blk_flush\(\)](#), [cache_get_slot\(\)](#), and [cache_writeback\(\)](#).

6.19.1.3 dirty

`uint8_t cache_slot_t::dirty`

Referenced by [blk_flush\(\)](#), [blk_get_buffer\(\)](#), [blk_update\(\)](#), and [cache_writeback\(\)](#).

6.19.1.4 loaded

`uint8_t cache_slot_t::loaded`

Referenced by [cache_load_devblock\(\)](#).

6.19.1.5 meta

`blk_meta_t cache_slot_t::meta[BLK_PACK_RATIO]`

Referenced by [blk_get_meta\(\)](#), [blk_set_meta\(\)](#), [blk_update\(\)](#), [cache_load_devblock\(\)](#), and [cache_writeback\(\)](#).

6.19.1.6 meta_dirty

`uint8_t cache_slot_t::meta_dirty`

Referenced by [blk_flush\(\)](#), [blk_set_meta\(\)](#), [blk_update\(\)](#), and [cache_writeback\(\)](#).

6.19.1.7 valid

`uint8_t cache_slot_t::valid`

Referenced by [blk_flush\(\)](#), and [cache_get_slot\(\)](#).

The documentation for this struct was generated from the following file:

- [src/block_subsystem.c](#)

6.20 CapsuleDirHeader Struct Reference

```
#include <capsule.h>
```

Data Fields

- `uint64_t` [magic](#)
- `uint64_t` [arena_base](#)
- `uint64_t` [arena_size](#)
- `uint32_t` [desc_count](#)
- `uint32_t` [desc_capacity](#)
- `uint32_t` [name_count](#)
- `uint32_t` [reserved](#)
- `uint64_t` [dir_hash](#)

6.20.1 Field Documentation

6.20.1.1 arena_base

`uint64_t CapsuleDirHeader::arena_base`

Referenced by [kernel_main\(\)](#).

6.20.1.2 arena_size

`uint64_t CapsuleDirHeader::arena_size`

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), [capsule_run_experiment\(\)](#), and [kernel_main\(\)](#).

6.20.1.3 desc_capacity

`uint32_t CapsuleDirHeader::desc_capacity`

Referenced by [kernel_main\(\)](#).

6.20.1.4 desc_count

`uint32_t CapsuleDirHeader::desc_count`

Referenced by [capsule_find_by_id\(\)](#), [capsule_find_by_name\(\)](#), [capsule_find_mama_init\(\)](#), and [kernel_main\(\)](#).

6.20.1.5 dir_hash

`uint64_t CapsuleDirHeader::dir_hash`

Referenced by [kernel_main\(\)](#).

6.20.1.6 magic

uint64_t CapsuleDirHeader::magic
 Referenced by [kernel_main\(\)](#).

6.20.1.7 name_count

uint32_t CapsuleDirHeader::name_count
 Referenced by [kernel_main\(\)](#).

6.20.1.8 reserved

uint32_t CapsuleDirHeader::reserved
 Referenced by [kernel_main\(\)](#).

The documentation for this struct was generated from the following file:

- include/starkernel/[capsule.h](#)

6.21 CapsuleNameEntry Struct Reference

```
#include <capsule.h>
```

Data Fields

- char [name](#) [512]

6.21.1 Field Documentation

6.21.1.1 name

char CapsuleNameEntry::name[512]

The documentation for this struct was generated from the following file:

- include/starkernel/[capsule.h](#)

6.22 CapsuleRunRecord Struct Reference

```
#include <capsule_run.h>
```

Data Fields

- uint64_t [run_id](#)
- uint32_t [vm_id](#)
- uint32_t [reserved](#)
- uint64_t [capsule_id](#)
- uint64_t [capsule_hash](#)
- uint64_t [pre_dict_hash](#)
- uint64_t [post_dict_hash](#)
- uint64_t [started_ns](#)
- uint64_t [ended_ns](#)
- uint32_t [result_code](#)
- uint32_t [flags](#)

6.22.1 Field Documentation

6.22.1.1 capsule_hash

uint64_t CapsuleRunRecord::capsule_hash

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.2 capsule_id

uint64_t CapsuleRunRecord::capsule_id

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.3 ended_ns

uint64_t CapsuleRunRecord::ended_ns

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.4 flags

uint32_t CapsuleRunRecord::flags

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.5 post_dict_hash

uint64_t CapsuleRunRecord::post_dict_hash

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.6 pre_dict_hash

uint64_t CapsuleRunRecord::pre_dict_hash

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.7 reserved

uint32_t CapsuleRunRecord::reserved

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.8 result_code

uint32_t CapsuleRunRecord::result_code

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.9 run_id

uint64_t CapsuleRunRecord::run_id

Referenced by [capsule_run_experiment\(\)](#), [capsule_run_log_init\(\)](#), and [capsule_run_log_record\(\)](#).

6.22.1.10 started_ns

uint64_t CapsuleRunRecord::started_ns

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

6.22.1.11 vm_id

uint32_t CapsuleRunRecord::vm_id

Referenced by [capsule_run_experiment\(\)](#), and [capsule_run_log_init\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/capsule_run.h](#)

6.23 cf_item_t Struct Reference

Data Fields

- [size_t](#) [addr](#)
- [cf_tag_t](#) [tag](#)

The documentation for this struct was generated from the following file:

- `src/word_source/control_words.c`

6.24 CLIConfig Struct Reference

Configuration structure from command-line arguments Consolidates all parsed CLI options into a single, cleanly organized struct.

```
#include <cli.h>
```

Data Fields

- `int run_tests`
- `int break_me`
- `int fail_fast`
- `int benchmark`
- `int benchmark_iterations`
- `int doe_experiment`
- `HeartbeatLogMode heartbeat_log_mode`
- `int script_mode`
- `LogLevel log_level`
- `int log_level_explicitly_set`
- `const char * disk_img_path`
- `uint32_t ram_disk_mb`
- `int run_tests_flag_observed`

6.24.1 Detailed Description

Configuration structure from command-line arguments Consolidates all parsed CLI options into a single, cleanly organized struct.

6.24.2 Field Documentation

6.24.2.1 benchmark

```
int CLIConfig::benchmark
```

Referenced by `cli_parse()`, and `main()`.

6.24.2.2 benchmark_iterations

```
int CLIConfig::benchmark_iterations
```

Referenced by `cli_parse()`, and `run_benchmark_mode()`.

6.24.2.3 break_me

```
int CLIConfig::break_me
```

Referenced by `cli_parse()`, `main()`, and `run_optional_tests()`.

6.24.2.4 disk_img_path

```
const char* CLIConfig::disk_img_path
```

Referenced by `cli_parse()`, and `init_blkio()`.

6.24.2.5 doe_experiment

```
int CLIConfig::doe_experiment
```

Referenced by `cli_parse()`, and `main()`.

6.24.2.6 fail_fast

```
int CLIConfig::fail_fast
```

Referenced by [cli_parse\(\)](#), and [main\(\)](#).

6.24.2.7 heartbeat_log_mode

```
HeartbeatLogMode CLIConfig::heartbeat_log_mode
```

Referenced by [cli_parse\(\)](#).

6.24.2.8 log_level

```
LogLevel CLIConfig::log_level
```

Referenced by [cli_parse\(\)](#), and [main\(\)](#).

6.24.2.9 log_level_explicitly_set

```
int CLIConfig::log_level_explicitly_set
```

Referenced by [cli_parse\(\)](#), and [main\(\)](#).

6.24.2.10 ram_disk_mb

```
uint32_t CLIConfig::ram_disk_mb
```

Referenced by [cli_parse\(\)](#), and [init_blkio\(\)](#).

6.24.2.11 run_tests

```
int CLIConfig::run_tests
```

Referenced by [cli_parse\(\)](#).

6.24.2.12 run_tests_flag_observed

```
int CLIConfig::run_tests_flag_observed
```

Referenced by [cli_parse\(\)](#), and [main\(\)](#).

6.24.2.13 script_mode

```
int CLIConfig::script_mode
```

Referenced by [cli_parse\(\)](#), and [main\(\)](#).

The documentation for this struct was generated from the following file:

- [include/cli.h](#)

6.25 DictEntry Struct Reference

```
#include <vm.h>
```

Collaboration diagram for DictEntry:

Data Fields

- struct [DictEntry](#) * [link](#)
- [word_func_t](#) [func](#)
- [cell_t](#) [flags](#)
- [uint8_t](#) [name_len](#)
- [cell_t](#) [execution_heat](#)
- [uint8_t](#) [acl_default](#)
- [uint32_t](#) [acl_ttl](#)
- [uint8_t](#) [acl_allow](#)
- [uint8_t](#) [acl_mode](#)

- `uint8_t` `acl_pinned`
- `uint32_t` `word_id`
- `DictPhysics` `physics`
- `WordTransitionMetrics` * `transition_metrics`
- `char` `name` []

6.25.1 Field Documentation

6.25.1.1 `acl_allow`

`uint8_t DictEntry::acl_allow`

Referenced by `acl_inherit_entry()`, `execute_colon_word()`, `forth_acl_allow_fetch()`, `forth_acl_allow_store()`, `test_acl_adaptive_ttl()`, `test_acl_deny_aborts()`, `test_acl_inherit()`, `test_acl_pin_immutable()`, `test_acl_strict_mode()`, `test_acl_ttl_hot_path()`, and `test_emergency_console_bypass()`.

6.25.1.2 `acl_default`

`uint8_t DictEntry::acl_default`

Referenced by `sk_dict_canonical_hash()`.

6.25.1.3 `acl_mode`

`uint8_t DictEntry::acl_mode`

Referenced by `acl_inherit_entry()`, `forth_acl_mode_fetch()`, `forth_acl_mode_store()`, `kernel_main()`, `test_acl_inherit()`, `test_acl_pin_immutable()`, `test_acl_strict_mode()`, and `test_acl_ttl_hot_path()`.

6.25.1.4 `acl_pinned`

`uint8_t DictEntry::acl_pinned`

Referenced by `acl_inherit_entry()`, `forth_acl_allow_store()`, `forth_acl_mode_store()`, `forth_acl_pin()`, `forth_acl_pinned_query()`, `forth_acl_ttl_store()`, `kernel_main()`, `test_acl_adaptive_ttl()`, `test_acl_deny_aborts()`, `test_acl_inherit()`, `test_acl_pin_blocks_shadow()`, `test_acl_pin_immutable()`, `test_acl_strict_mode()`, `test_acl_ttl_hot_path()`, and `test_emergency_console_bypass()`.

6.25.1.5 `acl_ttl`

`uint32_t DictEntry::acl_ttl`

Referenced by `acl_inherit_entry()`, `execute_colon_word()`, `forth_acl_ttl_fetch()`, `forth_acl_ttl_store()`, `test_acl_deny_aborts()`, `test_acl_inherit()`, `test_acl_pin_immutable()`, `test_acl_strict_mode()`, `test_acl_ttl_hot_path()`, and `test_emergency_console_bypass()`.

6.25.1.6 `execution_heat`

`cell_t DictEntry::execution_heat`

Referenced by `compare_dict_entry_heat_desc()`, `dict_find_word_heat_aware()`, `execute_colon_word()`, `forth_acl_heat_fetch()`, `forth_ALL_HEATS()`, `forth_PHYSICS_BURN()`, `forth_PHYSICS_CACHE_STATS()`, `forth_PHYSICS_SHOW_FEEDBACK()`, `forth_PHYSICS_WORD_METRICS()`, `hotwords_cache_lookup()`, `physics_on_lookup()`, `physics_post_execute()`, `physics_pre_execute()`, `starforth_word_init()`, `test_acl_adaptive_ttl()`, and `vm_interpret_word()`.

6.25.1.7 `flags`

`cell_t DictEntry::flags`

Referenced by `defining_word_immediate()`, `dictionary_m_word_hidden()`, `forth_ALL_HEATS()`, `sk_dict_canonical_hash()`, `system_word_vlist()`, `vm_enter_compile_mode()`, `vm_exit_compile_mode()`, `vm_find_word()`, `vm_hide_word()`, `vm_interpret_word()`, `vm_make_immediate()`, `vm_pin_execution_heat()`, `vm_smudge_word()`, `vm_unpin_execution_heat()`, `vocab_find_word()`, and `vocabulary_word_paren_find()`.

6.25.1.8 `func`

`word_func_t DictEntry::func`

Referenced by `acl_recheck()`, `control_forth_endcase()`, `control_forth_of()`, `defining_runtime_does_rt()`, `execute_colon_word()`, `forth_PHYSICS_BURN()`, `hash_colon_body()`, `sk_dict_canonical_hash()`, `starforth_word_init()`, and `vm_create_word()`.

6.25.1.9 link

```
struct DictEntry* DictEntry::link
```

Referenced by [dictionary_word_forget\(\)](#), [fc_rebuild\(\)](#), [find_entry_from_name\(\)](#), [sf_try_fast_append\(\)](#), [system_word_words\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

6.25.1.10 name

```
char DictEntry::name[]
```

Referenced by [defining_word_immediate\(\)](#), [dict_find_word_heat_aware\(\)](#), [execute_colon_word\(\)](#), [fc_rebuild\(\)](#), [forth_ALL_HEATS\(\)](#), [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#), [hotwords_cache_lookup\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), [sk_dict_canonical_hash\(\)](#), [system_word_see\(\)](#), [vm_debug_dump_state\(\)](#), [vm_find_word\(\)](#), [vm_make_immediate\(\)](#), [vocab_find_word\(\)](#), and [vocabulary_word_order\(\)](#).

6.25.1.11 name_len

```
uint8_t DictEntry::name_len
```

Referenced by [defining_word_immediate\(\)](#), [dict_find_word_heat_aware\(\)](#), [execute_colon_word\(\)](#), [forth_ALL_HEATS\(\)](#), [hotwords_cache_lookup\(\)](#), [profiler_generate_report\(\)](#), [profiler_print_hotspots\(\)](#), [sk_dict_canonical_hash\(\)](#), [system_word_see\(\)](#), [system_word_vlist\(\)](#), [vm_find_word\(\)](#), [vm_make_immediate\(\)](#), [vocab_find_word\(\)](#), and [vocabulary_word_order\(\)](#).

6.25.1.12 physics

```
DictPhysics DictEntry::physics
```

Referenced by [execute_colon_word\(\)](#), [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

6.25.1.13 transition_metrics

```
WordTransitionMetrics* DictEntry::transition_metrics
```

Referenced by [execute_colon_word\(\)](#), [physics_pre_execute\(\)](#), and [rolling_window_seed_pipelineing_context\(\)](#).

6.25.1.14 word_id

```
uint32_t DictEntry::word_id
```

Referenced by [execute_colon_word\(\)](#), [forth_acl_word_id\(\)](#), [hash_colon_body\(\)](#), [physics_pre_execute\(\)](#), [sk_dict_canonical_hash\(\)](#), [sk_parity_collect\(\)](#), [vm_dictionary_untrack_entry\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.26 DictPhysics Struct Reference

```
#include <vm.h>
```

Data Fields

- [uint16_t temperature_q8](#)
- [uint64_t last_active_ns](#)
- [uint64_t last_decay_ns](#)
- [uint32_t mass_bytes](#)
- [uint32_t avg_latency_ns](#)
- [uint8_t state_flags](#)
- [uint8_t acl_hint](#)
- [uint16_t pubsub_mask](#)

6.26.1 Field Documentation

6.26.1.1 `acl_hint`

`uint8_t DictPhysics::acl_hint`

6.26.1.2 `avg_latency_ns`

`uint32_t DictPhysics::avg_latency_ns`

Referenced by [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), and [forth_PHYSICS_WORD_METRICS\(\)](#).

6.26.1.3 `last_active_ns`

`uint64_t DictPhysics::last_active_ns`

Referenced by [execute_colon_word\(\)](#), [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

6.26.1.4 `last_decay_ns`

`uint64_t DictPhysics::last_decay_ns`

Referenced by [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

6.26.1.5 `mass_bytes`

`uint32_t DictPhysics::mass_bytes`

Referenced by [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), and [forth_PHYSICS_WORD_METRICS\(\)](#).

6.26.1.6 `pubsub_mask`

`uint16_t DictPhysics::pubsub_mask`

6.26.1.7 `state_flags`

`uint8_t DictPhysics::state_flags`

6.26.1.8 `temperature_q8`

`uint16_t DictPhysics::temperature_q8`

Referenced by [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), and [forth_PHYSICS_WORD_METRICS\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.27 DoeMetrics Struct Reference

Complete metrics snapshot for a single DoE run.

```
#include <doe_metrics.h>
```

Data Fields

- `uint32_t` [total_lookups](#)
- `uint64_t` [cache_hits](#)
- `double` [cache_hit_percent](#)
- `uint64_t` [bucket_hits](#)
- `double` [bucket_hit_percent](#)
- `int64_t` [cache_hit_latency_ns](#)

- [int64_t cache_hit_stddev_ns](#)
- [int64_t bucket_search_latency_ns](#)
- [int64_t bucket_search_stddev_ns](#)
- [uint64_t context_predictions_total](#)
- [uint64_t context_correct](#)
- [double context_accuracy_percent](#)
- [uint64_t cache_promotions](#)
- [uint64_t cache_demotions](#)
- [double window_diversity_percent](#)
- [uint32_t window_final_size_bytes](#)
- [uint32_t rolling_window_width](#)
- [uint32_t actual_window_size](#)
- [uint64_t total_executions](#)
- [uint64_t window_variance_q48](#)
- [double decay_slope](#)
- [uint64_t total_heat](#)
- [uint64_t hot_word_count](#)
- [uint64_t stale_word_count](#)
- [double stale_word_ratio](#)
- [double avg_word_heat](#)
- [uint64_t tick_count](#)
- [uint64_t tick_target_ns](#)
- [uint64_t inference_run_count](#)
- [uint64_t early_exit_count](#)
- [double prefetch_accuracy_percent](#)
- [uint64_t prefetch_attempts](#)
- [uint64_t prefetch_hits](#)
- [uint64_t window_tuning_checks](#)
- [uint32_t final_effective_window_size](#)
- [int64_t vm_workload_duration_ns_q48](#)
- [uint64_t total_runtime_ms](#)
- [uint64_t words_executed](#)
- [uint64_t dictionary_lookups](#)
- [uint64_t memory_allocated_bytes](#)
- [double speedup_vs_baseline](#)
- [double ci_lower_95](#)
- [double ci_upper_95](#)
- [int64_t cpu_temp_delta_c_q48](#)
- [int64_t cpu_freq_delta_mhz_q48](#)
- [uint32_t decay_rate_q16](#)
- [uint32_t decay_min_interval_ns](#)
- [uint32_t rolling_window_size](#)
- [uint32_t adaptive_shrink_rate](#)
- [uint32_t heat_cache_demotion_threshold](#)
- [int enable_loop_1_heat_tracking](#)
- [int enable_loop_2_rolling_window](#)
- [int enable_loop_3_linear_decay](#)
- [int enable_loop_4_pipelining](#)
- [int enable_loop_5_window_inference](#)
- [int enable_loop_6_decay_inference](#)
- [int enable_loop_7_adaptive_heartrate](#)
- [int enable_hotwords_cache](#)
- [int enable_pipelining](#)

6.27.1 Detailed Description

Complete metrics snapshot for a single DoE run.

Represents all 35 metrics collected per test iteration, mirroring the CSV schema. All values extracted directly from [VM](#) structures (no string parsing).

6.27.2 Field Documentation

6.27.2.1 actual_window_size

```
uint32_t DoeMetrics::actual_window_size
```

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.2 adaptive_shrink_rate

```
uint32_t DoeMetrics::adaptive_shrink_rate
```

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.3 avg_word_heat

```
double DoeMetrics::avg_word_heat
```

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.4 bucket_hit_percent

```
double DoeMetrics::bucket_hit_percent
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.5 bucket_hits

```
uint64_t DoeMetrics::bucket_hits
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.6 bucket_search_latency_ns

```
int64_t DoeMetrics::bucket_search_latency_ns
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.7 bucket_search_stddev_ns

```
int64_t DoeMetrics::bucket_search_stddev_ns
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.8 cache_demotions

```
uint64_t DoeMetrics::cache_demotions
```

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.9 cache_hit_latency_ns

```
int64_t DoeMetrics::cache_hit_latency_ns
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.10 cache_hit_percent

```
double DoeMetrics::cache_hit_percent
```

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.11 cache_hit_stddev_ns

`int64_t DoeMetrics::cache_hit_stddev_ns`

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.12 cache_hits

`uint64_t DoeMetrics::cache_hits`

Referenced by [extract_cache_metrics\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.13 cache_promotions

`uint64_t DoeMetrics::cache_promotions`

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.14 ci_lower_95

`double DoeMetrics::ci_lower_95`

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.15 ci_upper_95

`double DoeMetrics::ci_upper_95`

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.16 context_accuracy_percent

`double DoeMetrics::context_accuracy_percent`

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.17 context_correct

`uint64_t DoeMetrics::context_correct`

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.18 context_predictions_total

`uint64_t DoeMetrics::context_predictions_total`

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.19 cpu_freq_delta_mhz_q48

`int64_t DoeMetrics::cpu_freq_delta_mhz_q48`

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.20 cpu_temp_delta_c_q48

`int64_t DoeMetrics::cpu_temp_delta_c_q48`

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.21 decay_min_interval_ns

`uint32_t DoeMetrics::decay_min_interval_ns`

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.22 decay_rate_q16

`uint32_t DoeMetrics::decay_rate_q16`

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.23 decay_slope

double DoeMetrics::decay_slope

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.24 dictionary_lookups

uint64_t DoeMetrics::dictionary_lookups

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.25 early_exit_count

uint64_t DoeMetrics::early_exit_count

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.26 enable_hotwords_cache

int DoeMetrics::enable_hotwords_cache

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.27 enable_loop_1_heat_tracking

int DoeMetrics::enable_loop_1_heat_tracking

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.28 enable_loop_2_rolling_window

int DoeMetrics::enable_loop_2_rolling_window

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.29 enable_loop_3_linear_decay

int DoeMetrics::enable_loop_3_linear_decay

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.30 enable_loop_4_pipelining

int DoeMetrics::enable_loop_4_pipelining

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.31 enable_loop_5_window_inference

int DoeMetrics::enable_loop_5_window_inference

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.32 enable_loop_6_decay_inference

int DoeMetrics::enable_loop_6_decay_inference

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.33 enable_loop_7_adaptive_heartrate

int DoeMetrics::enable_loop_7_adaptive_heartrate

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.34 enable_pipelining

int DoeMetrics::enable_pipelining

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.35 final_effective_window_size

uint32_t DoeMetrics::final_effective_window_size

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.36 heat_cache_demotion_threshold

uint32_t DoeMetrics::heat_cache_demotion_threshold

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.37 hot_word_count

uint64_t DoeMetrics::hot_word_count

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.38 inference_run_count

uint64_t DoeMetrics::inference_run_count

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.39 memory_allocated_bytes

uint64_t DoeMetrics::memory_allocated_bytes

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.40 prefetch_accuracy_percent

double DoeMetrics::prefetch_accuracy_percent

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.41 prefetch_attempts

uint64_t DoeMetrics::prefetch_attempts

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.42 prefetch_hits

uint64_t DoeMetrics::prefetch_hits

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.43 rolling_window_size

uint32_t DoeMetrics::rolling_window_size

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.44 rolling_window_width

uint32_t DoeMetrics::rolling_window_width

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.45 speedup_vs_baseline

double DoeMetrics::speedup_vs_baseline

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.46 stale_word_count

uint64_t DoeMetrics::stale_word_count

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.47 stale_word_ratio

double DoeMetrics::stale_word_ratio

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.48 tick_count

uint64_t DoeMetrics::tick_count

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.49 tick_target_ns

uint64_t DoeMetrics::tick_target_ns

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.50 total_executions

uint64_t DoeMetrics::total_executions

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.51 total_heat

uint64_t DoeMetrics::total_heat

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.52 total_lookups

uint32_t DoeMetrics::total_lookups

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.53 total_runtime_ms

uint64_t DoeMetrics::total_runtime_ms

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.54 vm_workload_duration_ns_q48

int64_t DoeMetrics::vm_workload_duration_ns_q48

Referenced by [metrics_from_vm\(\)](#), [metrics_print_text\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.55 window_diversity_percent

double DoeMetrics::window_diversity_percent

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

6.27.2.56 window_final_size_bytes

uint32_t DoeMetrics::window_final_size_bytes

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.57 window_tuning_checks

uint64_t DoeMetrics::window_tuning_checks

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.58 window_variance_q48

uint64_t DoeMetrics::window_variance_q48

Referenced by [metrics_from_vm\(\)](#), and [metrics_write_csv_row\(\)](#).

6.27.2.59 words_executed

```
uint64_t DoeMetrics::words_executed
```

Referenced by [metrics_from_vm\(\)](#), [metrics_write_csv_row\(\)](#), and [metrics_write_james_law_csv_row\(\)](#).

The documentation for this struct was generated from the following file:

- [include/doe_metrics.h](#)

6.28 EFI_CONFIGURATION_TABLE Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for EFI_CONFIGURATION_TABLE:

Data Fields

- [EFI_GUID](#) VendorGuid
- void * [VendorTable](#)

6.28.1 Field Documentation

6.28.1.1 VendorGuid

```
EFI_GUID EFI_CONFIGURATION_TABLE::VendorGuid
```

6.28.1.2 VendorTable

```
void* EFI_CONFIGURATION_TABLE::VendorTable
```

Referenced by [efi_main\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.29 EFI_FILE_INFO Struct Reference

```
#include <uefi.h>
```

Collaboration diagram for EFI_FILE_INFO:

Data Fields

- [UINT64](#) Size
- [UINT64](#) FileSize
- [UINT64](#) PhysicalSize
- [EFI_TIME](#) CreateTime
- [EFI_TIME](#) LastAccessTime
- [EFI_TIME](#) ModificationTime
- [UINT64](#) Attribute
- [CHAR16](#) FileName [256]

6.29.1 Field Documentation

6.29.1.1 Attribute

```
UINT64 EFI_FILE_INFO::Attribute
```

6.29.1.2 CreateTime

```
EFI_TIME EFI_FILE_INFO::CreateTime
```

6.29.1.3 FileName

`CHAR16` `EFI_FILE_INFO::FileName[256]`

6.29.1.4 FileSize

`UINT64` `EFI_FILE_INFO::FileSize`
Referenced by [load_kernel_from_esp\(\)](#).

6.29.1.5 LastAccessTime

`EFI_TIME` `EFI_FILE_INFO::LastAccessTime`

6.29.1.6 ModificationTime

`EFI_TIME` `EFI_FILE_INFO::ModificationTime`

6.29.1.7 PhysicalSize

`UINT64` `EFI_FILE_INFO::PhysicalSize`

6.29.1.8 Size

`UINT64` `EFI_FILE_INFO::Size`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.30 EFI_GUID Struct Reference

```
#include <uefi.h>
```

Data Fields

- `UINT32` `Data1`
- `UINT16` `Data2`
- `UINT16` `Data3`
- `UINT8` `Data4` [8]

6.30.1 Field Documentation

6.30.1.1 Data1

`UINT32` `EFI_GUID::Data1`
Referenced by [guid_equals\(\)](#).

6.30.1.2 Data2

`UINT16` `EFI_GUID::Data2`
Referenced by [guid_equals\(\)](#).

6.30.1.3 Data3

`UINT16` `EFI_GUID::Data3`
Referenced by [guid_equals\(\)](#).

6.30.1.4 Data4

`UINT8 EFI_GUID::Data4[8]`

Referenced by [guid_equals\(\)](#).

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.31 EFI_LOADED_IMAGE_PROTOCOL Struct Reference

```
#include <uefi.h>
```

Data Fields

- [EFI_HANDLE](#) DeviceHandle
- `VOID *` FilePath
- `VOID *` Reserved
- `UINT32` LoadOptionsSize
- `VOID *` LoadOptions
- `VOID *` ImageBase
- `UINT64` ImageSize
- [EFI_MEMORY_TYPE](#) ImageCodeType
- [EFI_MEMORY_TYPE](#) ImageDataType
- `VOID *` Unload

6.31.1 Field Documentation

6.31.1.1 DeviceHandle

[EFI_HANDLE](#) `EFI_LOADED_IMAGE_PROTOCOL::DeviceHandle`

Referenced by [load_kernel_from_esp\(\)](#).

6.31.1.2 FilePath

`VOID*` `EFI_LOADED_IMAGE_PROTOCOL::FilePath`

6.31.1.3 ImageBase

`VOID*` `EFI_LOADED_IMAGE_PROTOCOL::ImageBase`

6.31.1.4 ImageCodeType

[EFI_MEMORY_TYPE](#) `EFI_LOADED_IMAGE_PROTOCOL::ImageCodeType`

6.31.1.5 ImageDataType

[EFI_MEMORY_TYPE](#) `EFI_LOADED_IMAGE_PROTOCOL::ImageDataType`

6.31.1.6 ImageSize

`UINT64` `EFI_LOADED_IMAGE_PROTOCOL::ImageSize`

6.31.1.7 LoadOptions

`VOID*` `EFI_LOADED_IMAGE_PROTOCOL::LoadOptions`

6.31.1.8 LoadOptionsSize

`UINT32` `EFI_LOADED_IMAGE_PROTOCOL::LoadOptionsSize`

6.31.1.9 Reserved

`VOID* EFI_LOADED_IMAGE_PROTOCOL::Reserved`

6.31.1.10 Unload

`VOID* EFI_LOADED_IMAGE_PROTOCOL::Unload`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.32 EFI_MEMORY_DESCRIPTOR Struct Reference

```
#include <uefi.h>
```

Data Fields

- [UINT32 Type](#)
- [UINT64 PhysicalStart](#)
- [UINT64 VirtualStart](#)
- [UINT64 NumberOfPages](#)
- [UINT64 Attribute](#)

6.32.1 Field Documentation

6.32.1.1 Attribute

`UINT64 EFI_MEMORY_DESCRIPTOR::Attribute`

6.32.1.2 NumberOfPages

`UINT64 EFI_MEMORY_DESCRIPTOR::NumberOfPages`

Referenced by [pmm_init\(\)](#), [print_boot_info\(\)](#), and [vmm_init\(\)](#).

6.32.1.3 PhysicalStart

`UINT64 EFI_MEMORY_DESCRIPTOR::PhysicalStart`

Referenced by [pmm_init\(\)](#), and [vmm_init\(\)](#).

6.32.1.4 Type

`UINT32 EFI_MEMORY_DESCRIPTOR::Type`

Referenced by [pmm_init\(\)](#), [print_boot_info\(\)](#), and [vmm_init\(\)](#).

6.32.1.5 VirtualStart

`UINT64 EFI_MEMORY_DESCRIPTOR::VirtualStart`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.33 EFI_TABLE_HEADER Struct Reference

```
#include <uefi.h>
```

Data Fields

- [UINT64 Signature](#)
- [UINT32 Revision](#)
- [UINT32 HeaderSize](#)
- [UINT32 CRC32](#)
- [UINT32 Reserved](#)

6.33.1 Field Documentation

6.33.1.1 CRC32

[UINT32](#) `EFI_TABLE_HEADER::CRC32`

6.33.1.2 HeaderSize

[UINT32](#) `EFI_TABLE_HEADER::HeaderSize`

6.33.1.3 Reserved

[UINT32](#) `EFI_TABLE_HEADER::Reserved`

6.33.1.4 Revision

[UINT32](#) `EFI_TABLE_HEADER::Revision`

6.33.1.5 Signature

[UINT64](#) `EFI_TABLE_HEADER::Signature`

The documentation for this struct was generated from the following file:

- `include/starkernel/uefi.h`

6.34 EFI_TIME Struct Reference

```
#include <uefi.h>
```

Data Fields

- [UINT16 Year](#)
- [UINT8 Month](#)
- [UINT8 Day](#)
- [UINT8 Hour](#)
- [UINT8 Minute](#)
- [UINT8 Second](#)
- [UINT8 Pad1](#)
- [UINT32 Nanosecond](#)
- [INT16 TimeZone](#)
- [UINT8 Daylight](#)
- [UINT8 Pad2](#)

6.34.1 Field Documentation

6.34.1.1 Day

[UINT8](#) `EFI_TIME::Day`

6.34.1.2 Daylight

[UINT8](#) `EFI_TIME::Daylight`

6.34.1.3 Hour

UINT8 EFI_TIME::Hour

6.34.1.4 Minute

UINT8 EFI_TIME::Minute

6.34.1.5 Month

UINT8 EFI_TIME::Month

6.34.1.6 Nanosecond

UINT32 EFI_TIME::Nanosecond

6.34.1.7 Pad1

UINT8 EFI_TIME::Pad1

6.34.1.8 Pad2

UINT8 EFI_TIME::Pad2

6.34.1.9 Second

UINT8 EFI_TIME::Second

6.34.1.10 TimeZone

INT16 EFI_TIME::TimeZone

6.34.1.11 Year

UINT16 EFI_TIME::Year

The documentation for this struct was generated from the following file:

- include/starkernel/[uefi.h](#)

6.35 Elf64_Ehdr Struct Reference

```
#include <elf64.h>
```

Data Fields

- unsigned char [e_ident](#) [16]
- [Elf64_Half e_type](#)
- [Elf64_Half e_machine](#)
- [Elf64_Word e_version](#)
- [Elf64_Addr e_entry](#)
- [Elf64_Off e_phoff](#)
- [Elf64_Off e_shoff](#)
- [Elf64_Word e_flags](#)
- [Elf64_Half e_ehsize](#)
- [Elf64_Half e_phentsize](#)
- [Elf64_Half e_phnum](#)
- [Elf64_Half e_shentsize](#)
- [Elf64_Half e_shnum](#)
- [Elf64_Half e_shstrndx](#)

6.35.1 Field Documentation

6.35.1.1 e_ehsize

`Elf64_Half Elf64_Ehdr::e_ehsize`

6.35.1.2 e_entry

`Elf64_Addr Elf64_Ehdr::e_entry`
Referenced by [elf_load_kernel\(\)](#).

6.35.1.3 e_flags

`Elf64_Word Elf64_Ehdr::e_flags`

6.35.1.4 e_ident

`unsigned char Elf64_Ehdr::e_ident[16]`
Referenced by [elf_validate_header\(\)](#).

6.35.1.5 e_machine

`Elf64_Half Elf64_Ehdr::e_machine`
Referenced by [elf_validate_header\(\)](#).

6.35.1.6 e_phentsize

`Elf64_Half Elf64_Ehdr::e_phentsize`

6.35.1.7 e_phnum

`Elf64_Half Elf64_Ehdr::e_phnum`
Referenced by [elf_load_segments\(\)](#).

6.35.1.8 e_phoff

`Elf64_Off Elf64_Ehdr::e_phoff`
Referenced by [elf_load_segments\(\)](#).

6.35.1.9 e_shentsize

`Elf64_Half Elf64_Ehdr::e_shentsize`

6.35.1.10 e_shnum

`Elf64_Half Elf64_Ehdr::e_shnum`
Referenced by [elf_apply_relocations\(\)](#).

6.35.1.11 e_shoff

`Elf64_Off Elf64_Ehdr::e_shoff`
Referenced by [elf_apply_relocations\(\)](#).

6.35.1.12 e_shstrndx

`Elf64_Half Elf64_Ehdr::e_shstrndx`

6.35.1.13 e_type

`Elf64_Half Elf64_Ehdr::e_type`
Referenced by [elf_load_kernel\(\)](#), and [elf_validate_header\(\)](#).

6.35.1.14 e_version

`Elf64_Word Elf64_Ehdr::e_version`

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.36 Elf64_Phdr Struct Reference

```
#include <elf64.h>
```

Data Fields

- `Elf64_Word p_type`
- `Elf64_Word p_flags`
- `Elf64_Off p_offset`
- `Elf64_Addr p_vaddr`
- `Elf64_Addr p_paddr`
- `Elf64_Xword p_filesz`
- `Elf64_Xword p_memsz`
- `Elf64_Xword p_align`

6.36.1 Field Documentation

6.36.1.1 p_align

`Elf64_Xword Elf64_Phdr::p_align`

6.36.1.2 p_filesz

`Elf64_Xword Elf64_Phdr::p_filesz`

6.36.1.3 p_flags

`Elf64_Word Elf64_Phdr::p_flags`

6.36.1.4 p_memsz

`Elf64_Xword Elf64_Phdr::p_memsz`

6.36.1.5 p_offset

`Elf64_Off Elf64_Phdr::p_offset`

Referenced by `elf_load_segments()`.

6.36.1.6 p_paddr

`Elf64_Addr Elf64_Phdr::p_paddr`

6.36.1.7 p_type

`Elf64_Word Elf64_Phdr::p_type`

6.36.1.8 p_vaddr

`Elf64_Addr Elf64_Phdr::p_vaddr`

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.37 Elf64_Rel Struct Reference

```
#include <elf64.h>
```

Data Fields

- [Elf64_Addr r_offset](#)
- [Elf64_Xword r_info](#)

6.37.1 Field Documentation

6.37.1.1 r_info

[Elf64_Xword](#) `Elf64_Rel::r_info`

6.37.1.2 r_offset

[Elf64_Addr](#) `Elf64_Rel::r_offset`

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.38 Elf64_Rela Struct Reference

```
#include <elf64.h>
```

Data Fields

- [Elf64_Addr r_offset](#)
- [Elf64_Xword r_info](#)
- [Elf64_Sxword r_addend](#)

6.38.1 Field Documentation

6.38.1.1 r_addend

[Elf64_Sxword](#) `Elf64_Rela::r_addend`

Referenced by [elf_apply_relocations\(\)](#).

6.38.1.2 r_info

[Elf64_Xword](#) `Elf64_Rela::r_info`

6.38.1.3 r_offset

[Elf64_Addr](#) `Elf64_Rela::r_offset`

Referenced by [elf_apply_relocations\(\)](#).

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.39 Elf64_Shdr Struct Reference

```
#include <elf64.h>
```

Data Fields

- [Elf64_Word](#) sh_name
- [Elf64_Word](#) sh_type
- [Elf64_Xword](#) sh_flags
- [Elf64_Addr](#) sh_addr
- [Elf64_Off](#) sh_offset
- [Elf64_Xword](#) sh_size
- [Elf64_Word](#) sh_link
- [Elf64_Word](#) sh_info
- [Elf64_Xword](#) sh_addralign
- [Elf64_Xword](#) sh_entsize

6.39.1 Field Documentation

6.39.1.1 sh_addr

[Elf64_Addr](#) Elf64_Shdr::sh_addr

6.39.1.2 sh_addralign

[Elf64_Xword](#) Elf64_Shdr::sh_addralign

6.39.1.3 sh_entsize

[Elf64_Xword](#) Elf64_Shdr::sh_entsize

Referenced by [elf_apply_relocations\(\)](#).

6.39.1.4 sh_flags

[Elf64_Xword](#) Elf64_Shdr::sh_flags

6.39.1.5 sh_info

[Elf64_Word](#) Elf64_Shdr::sh_info

6.39.1.6 sh_link

[Elf64_Word](#) Elf64_Shdr::sh_link

Referenced by [elf_apply_relocations\(\)](#).

6.39.1.7 sh_name

[Elf64_Word](#) Elf64_Shdr::sh_name

6.39.1.8 sh_offset

[Elf64_Off](#) Elf64_Shdr::sh_offset

Referenced by [elf_apply_relocations\(\)](#).

6.39.1.9 sh_size

[Elf64_Xword](#) Elf64_Shdr::sh_size

Referenced by [elf_apply_relocations\(\)](#).

6.39.1.10 sh_type

[Elf64_Word](#) Elf64_Shdr::sh_type

Referenced by [elf_apply_relocations\(\)](#).

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.40 Elf64_Sym Struct Reference

```
#include <elf64.h>
```

Data Fields

- [Elf64_Word](#) `st_name`
- unsigned char `st_info`
- unsigned char `st_other`
- [Elf64_Half](#) `st_shndx`
- [Elf64_Addr](#) `st_value`
- [Elf64_Xword](#) `st_size`

6.40.1 Field Documentation

6.40.1.1 `st_info`

```
unsigned char Elf64_Sym::st_info
```

6.40.1.2 `st_name`

```
Elf64\_Word Elf64_Sym::st_name
```

6.40.1.3 `st_other`

```
unsigned char Elf64_Sym::st_other
```

6.40.1.4 `st_shndx`

```
Elf64\_Half Elf64_Sym::st_shndx
```

6.40.1.5 `st_size`

```
Elf64\_Xword Elf64_Sym::st_size
```

6.40.1.6 `st_value`

```
Elf64\_Addr Elf64_Sym::st_value
```

Referenced by [elf_apply_relocations\(\)](#).

The documentation for this struct was generated from the following file:

- `include/starkernel/elf64.h`

6.41 ExecResult Struct Reference

```
#include <test_contracts.h>
```

Data Fields

- [cell_t](#) `stack` [1024]
- int `dsp`
- int `rsp`
- int `error`

6.41.1 Field Documentation

6.41.1.1 `dsp`

```
int ExecResult::dsp
```

Referenced by [assert_heartbeat_safe\(\)](#), [capture_exec_result\(\)](#), [check_physics_transparent\(\)](#), and [exec_results_equal\(\)](#).

6.41.1.2 error

```
int ExecResult::error
```

Referenced by [capture_exec_result\(\)](#), and [exec_results_equal\(\)](#).

6.41.1.3 rsp

```
int ExecResult::rsp
```

Referenced by [assert_heartbeat_safe\(\)](#), [capture_exec_result\(\)](#), and [exec_results_equal\(\)](#).

6.41.1.4 stack

```
cell_t ExecResult::stack[1024]
```

Referenced by [capture_exec_result\(\)](#), and [exec_results_equal\(\)](#).

The documentation for this struct was generated from the following file:

- [src/test_runner/include/test_contracts.h](#)

6.42 FramebufferInfo Struct Reference

```
#include <uefi.h>
```

Data Fields

- void * [base](#)
- [UINTN](#) size
- [UINT32](#) width
- [UINT32](#) height
- [UINT32](#) pixels_per_scanline

6.42.1 Field Documentation

6.42.1.1 base

```
void* FramebufferInfo::base
```

Referenced by [efi_main\(\)](#).

6.42.1.2 height

```
UINT32 FramebufferInfo::height
```

Referenced by [efi_main\(\)](#).

6.42.1.3 pixels_per_scanline

```
UINT32 FramebufferInfo::pixels_per_scanline
```

Referenced by [efi_main\(\)](#).

6.42.1.4 size

```
UINTN FramebufferInfo::size
```

Referenced by [efi_main\(\)](#).

6.42.1.5 width

```
UINT32 FramebufferInfo::width
```

Referenced by [efi_main\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/uefi.h](#)

6.43 gdt_entry Struct Reference

[arch.c](#) (amd64) - Architecture abstractions for StarKernel on x86_64

Data Fields

- `uint16_t` [limit_low](#)
- `uint16_t` [base_low](#)
- `uint8_t` [base_mid](#)
- `uint8_t` [access](#)
- `uint8_t` [granularity](#)
- `uint8_t` [base_high](#)

6.43.1 Detailed Description

[arch.c](#) (amd64) - Architecture abstractions for StarKernel on x86_64

6.43.2 Field Documentation

6.43.2.1 access

`uint8_t gdt_entry::access`

6.43.2.2 base_high

`uint8_t gdt_entry::base_high`

6.43.2.3 base_low

`uint16_t gdt_entry::base_low`

6.43.2.4 base_mid

`uint8_t gdt_entry::base_mid`

6.43.2.5 granularity

`uint8_t gdt_entry::granularity`

6.43.2.6 limit_low

`uint16_t gdt_entry::limit_low`

Referenced by [gdt_set_entry\(\)](#).

The documentation for this struct was generated from the following file:

- `src/starkernel/arch/amd64/arch.c`

6.44 gdtr Struct Reference

Data Fields

- `uint16_t` [limit](#)
- `uint64_t` [base](#)

6.44.1 Field Documentation

6.44.1.1 base

`uint64_t gdtr::base`

Referenced by [gdt_init\(\)](#).

6.44.1.2 limit

```
uint16_t gdtr::limit
```

Referenced by [gdt_init\(\)](#).

The documentation for this struct was generated from the following file:

- [src/starkernel/arch/amd64/arch.c](#)

6.45 heap_block Struct Reference

Collaboration diagram for heap_block:

Data Fields

- `size_t` [size](#)
- `size_t` [requested](#)
- `int` [free](#)
- `struct heap_block *` [next](#)
- `struct heap_block *` [prev](#)

6.45.1 Field Documentation

6.45.1.1 free

```
int heap_block::free
```

Referenced by [allocate_from_block\(\)](#), [coalesce_neighbors\(\)](#), [kfree\(\)](#), [kmalloc_aligned\(\)](#), and [kmalloc_init\(\)](#).

6.45.1.2 next

```
struct heap_block* heap_block::next
```

Referenced by [allocate_from_block\(\)](#), [coalesce_neighbors\(\)](#), [kmalloc_aligned\(\)](#), and [kmalloc_init\(\)](#).

6.45.1.3 prev

```
struct heap_block* heap_block::prev
```

Referenced by [allocate_from_block\(\)](#), [coalesce_neighbors\(\)](#), and [kmalloc_init\(\)](#).

6.45.1.4 requested

```
size_t heap_block::requested
```

Referenced by [allocate_from_block\(\)](#), [kfree\(\)](#), and [kmalloc_init\(\)](#).

6.45.1.5 size

```
size_t heap_block::size
```

Referenced by [allocate_from_block\(\)](#), [coalesce_neighbors\(\)](#), and [kmalloc_init\(\)](#).

The documentation for this struct was generated from the following file:

- [src/starkernel/memory/kmalloc.c](#)

6.46 HeartbeatSnapshot Struct Reference

```
#include <vm.h>
```

Data Fields

- uint64_t [published_tick](#)
- uint64_t [published_ns](#)
- uint32_t [window_width](#)
- uint32_t [actual_window_size](#)
- uint64_t [decay_slope_q48](#)
- uint64_t [hot_word_count](#)
- uint64_t [stale_word_count](#)
- uint64_t [total_heat](#)

6.46.1 Field Documentation

6.46.1.1 actual_window_size

uint32_t HeartbeatSnapshot::actual_window_size

Referenced by [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.2 decay_slope_q48

uint64_t HeartbeatSnapshot::decay_slope_q48

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.3 hot_word_count

uint64_t HeartbeatSnapshot::hot_word_count

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.4 published_ns

uint64_t HeartbeatSnapshot::published_ns

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.5 published_tick

uint64_t HeartbeatSnapshot::published_tick

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.6 stale_word_count

uint64_t HeartbeatSnapshot::stale_word_count

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.7 total_heat

uint64_t HeartbeatSnapshot::total_heat

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

6.46.1.8 window_width

uint32_t HeartbeatSnapshot::window_width

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.47 HeartbeatState Struct Reference

```
#include <vm.h>
```

Collaboration diagram for HeartbeatState:

Data Fields

- uint64_t [tick_count](#)
- uint64_t [last_inference_tick](#)
- uint32_t [check_counter](#)
- int [heartbeat_enabled](#)
- uint64_t [tick_target_ns](#)
- volatile uint32_t [snapshot_index](#)
- [HeartbeatSnapshot](#) [snapshots](#) [2]
- [HeartbeatWorker](#) * [worker](#)
- uint64_t [inference_run_count](#)
- uint64_t [early_exit_count](#)
- uint64_t [words_executed](#)
- uint64_t [dictionary_lookups](#)
- [HeartbeatTickSnapshot](#) * [tick_buffer](#)
- uint32_t [tick_buffer_size](#)
- uint64_t [tick_buffer_write_index](#)
- uint64_t [tick_count_total](#)
- uint64_t [run_start_ns](#)
- uint32_t [tick_number_offset](#)
- double [bucket_sum_K](#)
- double [bucket_sum_K_squared](#)
- double [bucket_sum_window_variance](#)
- double [bucket_sum_heat_variance](#)
- uint64_t [bucket_mode_transitions](#)
- uint32_t [bucket_collapse_flag](#)
- uint32_t [bucket_tick_count](#)
- uint64_t [m5_time_trust](#)
- uint64_t [m5_variance](#)

6.47.1 Field Documentation**6.47.1.1 bucket_collapse_flag**

uint32_t HeartbeatState::bucket_collapse_flag
 Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.2 bucket_mode_transitions

uint64_t HeartbeatState::bucket_mode_transitions
 Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.3 bucket_sum_heat_variance

double HeartbeatState::bucket_sum_heat_variance
 Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.4 bucket_sum_K

double HeartbeatState::bucket_sum_K
 Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.5 bucket_sum_K_squared

double HeartbeatState::bucket_sum_K_squared
 Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.6 bucket_sum_window_variance

double HeartbeatState::bucket_sum_window_variance
Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.7 bucket_tick_count

uint32_t HeartbeatState::bucket_tick_count
Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.8 check_counter

uint32_t HeartbeatState::check_counter
Referenced by [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_interpret\(\)](#).

6.47.1.9 dictionary_lookups

uint64_t HeartbeatState::dictionary_lookups
Referenced by [metrics_from_vm\(\)](#), and [vm_find_word\(\)](#).

6.47.1.10 early_exit_count

uint64_t HeartbeatState::early_exit_count
Referenced by [metrics_from_vm\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.11 heartbeat_enabled

int HeartbeatState::heartbeat_enabled
Referenced by [vm_heartbeat_run_cycle\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.12 inference_run_count

uint64_t HeartbeatState::inference_run_count
Referenced by [metrics_from_vm\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.13 last_inference_tick

uint64_t HeartbeatState::last_inference_tick
Referenced by [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.14 m5_time_trust

uint64_t HeartbeatState::m5_time_trust

6.47.1.15 m5_variance

uint64_t HeartbeatState::m5_variance

6.47.1.16 run_start_ns

uint64_t HeartbeatState::run_start_ns
Referenced by [heartbeat_capture_tick_snapshot\(\)](#).

6.47.1.17 snapshot_index

volatile uint32_t HeartbeatState::snapshot_index
Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_snapshot_read\(\)](#).

6.47.1.18 snapshots

`HeartbeatSnapshot` `HeartbeatState::snapshots[2]`

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), and [vm_snapshot_read\(\)](#).

6.47.1.19 tick_buffer

`HeartbeatTickSnapshot*` `HeartbeatState::tick_buffer`

Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.20 tick_buffer_size

`uint32_t` `HeartbeatState::tick_buffer_size`

6.47.1.21 tick_buffer_write_index

`uint64_t` `HeartbeatState::tick_buffer_write_index`

Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.47.1.22 tick_count

`uint64_t` `HeartbeatState::tick_count`

Referenced by [heartbeat_publish_snapshot\(\)](#), [metrics_from_vm\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.23 tick_count_total

`uint64_t` `HeartbeatState::tick_count_total`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#).

6.47.1.24 tick_number_offset

`uint32_t` `HeartbeatState::tick_number_offset`

6.47.1.25 tick_target_ns

`uint64_t` `HeartbeatState::tick_target_ns`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), [save_physics\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.47.1.26 words_executed

`uint64_t` `HeartbeatState::words_executed`

Referenced by [execute_colon_word\(\)](#), [metrics_from_vm\(\)](#), [physics_post_execute\(\)](#), and [vm_interpret_word\(\)](#).

6.47.1.27 worker

`HeartbeatWorker*` `HeartbeatState::worker`

Referenced by [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_start_thread\(\)](#), [vm_heartbeat_stop_thread\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_interpret\(\)](#), and [vm_tick_inference_engine\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.48 HeartbeatTickSnapshot Struct Reference

```
#include <vm.h>
```

Data Fields

- [uint32_t](#) `tick_number`
- [uint64_t](#) `elapsed_ns`
- [uint64_t](#) `tick_interval_ns`
- [uint32_t](#) `cache_hits_delta`
- [uint32_t](#) `bucket_hits_delta`
- [uint32_t](#) `word_executions_delta`
- [uint64_t](#) `hot_word_count`
- [double](#) `avg_word_heat`
- [uint32_t](#) `window_width`
- [uint32_t](#) `actual_window_size`
- [uint32_t](#) `predicted_label_hits`
- [double](#) `estimated_jitter_ns`
- [uint8_t](#) `l8_mode`

6.48.1 Field Documentation

6.48.1.1 `actual_window_size`

[uint32_t](#) `HeartbeatTickSnapshot::actual_window_size`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.2 `avg_word_heat`

[double](#) `HeartbeatTickSnapshot::avg_word_heat`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.3 `bucket_hits_delta`

[uint32_t](#) `HeartbeatTickSnapshot::bucket_hits_delta`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.4 `cache_hits_delta`

[uint32_t](#) `HeartbeatTickSnapshot::cache_hits_delta`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.5 `elapsed_ns`

[uint64_t](#) `HeartbeatTickSnapshot::elapsed_ns`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.6 `estimated_jitter_ns`

[double](#) `HeartbeatTickSnapshot::estimated_jitter_ns`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.7 `hot_word_count`

[uint64_t](#) `HeartbeatTickSnapshot::hot_word_count`

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.8 `l8_mode`

[uint8_t](#) `HeartbeatTickSnapshot::l8_mode`

Referenced by [metrics_write_james_law_csv_row\(\)](#).

6.48.1.9 predicted_label_hits

`uint32_t` HeartbeatTickSnapshot::predicted_label_hits

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.10 tick_interval_ns

`uint64_t` HeartbeatTickSnapshot::tick_interval_ns

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.11 tick_number

`uint32_t` HeartbeatTickSnapshot::tick_number

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.12 window_width

`uint32_t` HeartbeatTickSnapshot::window_width

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

6.48.1.13 word_executions_delta

`uint32_t` HeartbeatTickSnapshot::word_executions_delta

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), and [heartbeat_emit_tick_row\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.49 HeartbeatWorker Struct Reference

```
#include <vm_internal.h>
```

Data Fields

- `uint64_t` [tick_ns](#)
- `int` [running](#)
- `int` [stop_requested](#)

6.49.1 Field Documentation**6.49.1.1 running**

`int` HeartbeatWorker::running

6.49.1.2 stop_requested

`int` HeartbeatWorker::stop_requested

Referenced by [vm_cleanup\(\)](#), and [vm_heartbeat_stop_thread\(\)](#).

6.49.1.3 tick_ns

`uint64_t` HeartbeatWorker::tick_ns

Referenced by [vm_heartbeat_start_thread\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_inference_engine\(\)](#).

The documentation for this struct was generated from the following files:

- [src/starkernel/vm/vm_internal.h](#)
- [src/vm_internal.h](#)

6.50 HotwordsCache_s Struct Reference

```
#include <physics_hotwords_cache.h>
```

Collaboration diagram for HotwordsCache_s:

Data Fields

- [DictEntry](#) * [cache](#) [32]
Cache entries (pointers to [DictEntry](#))
- [uint32_t](#) [cache_count](#)
Number of valid entries in cache.
- [uint32_t](#) [lru_index](#)
LRU: last-used index for eviction.
- [HotwordsStats](#) [stats](#)
Performance statistics.
- [int](#) [enabled](#)
Enabled/disabled flag (runtime toggle)

6.50.1 Field Documentation

6.50.1.1 cache

```
DictEntry* HotwordsCache_s::cache[32]
```

Cache entries (pointers to [DictEntry](#))

Referenced by [forth_PHYSICS_CACHE_STATS\(\)](#).

6.50.1.2 cache_count

```
uint32\_t HotwordsCache_s::cache_count
```

Number of valid entries in cache.

Referenced by [forth_PHYSICS_BUILD_INFO\(\)](#), and [forth_PHYSICS_CACHE_STATS\(\)](#).

6.50.1.3 enabled

```
int HotwordsCache_s::enabled
```

Enabled/disabled flag (runtime toggle)

Referenced by [forth_PHYSICS_BUILD_INFO\(\)](#), [forth_PHYSICS_TOGGLE_CACHE\(\)](#), and [metrics_from_vm\(\)](#).

6.50.1.4 lru_index

```
uint32\_t HotwordsCache_s::lru_index
```

LRU: last-used index for eviction.

6.50.1.5 stats

```
HotwordsStats HotwordsCache_s::stats
```

Performance statistics.

Referenced by [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), and [metrics_from_vm\(\)](#).

The documentation for this struct was generated from the following file:

- [include/physics_hotwords_cache.h](#)

6.51 HotwordsStats Struct Reference

```
#include <physics_hotwords_cache.h>
```

Data Fields

- uint64_t [total_lookups](#)
Total lookups performed.
- uint64_t [cache_hits](#)
Lookups that hit the hot-words cache.
- uint64_t [bucket_hits](#)
Lookups that missed cache but found in bucket.
- uint64_t [misses](#)
Lookups that failed (word not found)
- uint64_t [evictions](#)
Words evicted from cache (LRU)
- uint64_t [promotions](#)
Times a word was promoted to hot-words.
- uint64_t [bucket_reorders](#)
Times bucket was reordered.
- int64_t [cache_hit_total_ns_q48](#)
=== 64-bit Fixed-Point Statistics (Q48.16 format) === Q48.16 = 48-bit integer + 16-bit fractional part Precision: 2^{-16} ns ≈ 0.0000153 ns Range: ± 140 trillion ns (≈ 4.4 years) Allows: Bayesian inference, precise averaging, statistical computation
- int64_t [bucket_search_total_ns_q48](#)
Sum of bucket search latencies (64-bit fixed-point, Q48.16)
- uint64_t [cache_hit_samples](#)
Number of cache hit samples (for averaging)
- uint64_t [bucket_search_samples](#)
Number of bucket search samples (for averaging)
- int64_t [min_cache_hit_ns](#)
Minimum cache hit latency (nanoseconds)
- int64_t [max_cache_hit_ns](#)
Maximum cache hit latency (nanoseconds)
- int64_t [min_bucket_search_ns](#)
Minimum bucket search latency (nanoseconds)
- int64_t [max_bucket_search_ns](#)
Maximum bucket search latency (nanoseconds)
- int64_t [cache_hit_variance_sum_q48](#)
Sum of squared cache hit latencies (for variance/stddev)
- int64_t [bucket_search_variance_sum_q48](#)
Sum of squared bucket search latencies (for variance/stddev)

6.51.1 Field Documentation

6.51.1.1 bucket_hits

uint64_t HotwordsStats::bucket_hits

Lookups that missed cache but found in bucket.

Referenced by [extract_cache_metrics\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_bayesian_report\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.2 bucket_reorders

uint64_t HotwordsStats::bucket_reorders

Times bucket was reordered.

Referenced by [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.3 bucket_search_samples

`uint64_t HotwordsStats::bucket_search_samples`

Number of bucket search samples (for averaging)

Referenced by [extract_cache_metrics\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.4 bucket_search_total_ns_q48

`int64_t HotwordsStats::bucket_search_total_ns_q48`

Sum of bucket search latencies (64-bit fixed-point, Q48.16)

Referenced by [extract_cache_metrics\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.5 bucket_search_variance_sum_q48

`int64_t HotwordsStats::bucket_search_variance_sum_q48`

Sum of squared bucket search latencies (for variance/stddev)

Referenced by [extract_cache_metrics\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.6 cache_hit_samples

`uint64_t HotwordsStats::cache_hit_samples`

Number of cache hit samples (for averaging)

Referenced by [extract_cache_metrics\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.7 cache_hit_total_ns_q48

`int64_t HotwordsStats::cache_hit_total_ns_q48`

=== 64-bit Fixed-Point Statistics (Q48.16 format) === Q48.16 = 48-bit integer + 16-bit fractional part Precision↵ : 2^{-16} ns ≈ 0.0000153 ns Range: ± 140 trillion ns (≈ 4.4 years) Allows: Bayesian inference, precise averaging, statistical computation

Sum of cache hit latencies (64-bit fixed-point, Q48.16)

Referenced by [extract_cache_metrics\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.8 cache_hit_variance_sum_q48

`int64_t HotwordsStats::cache_hit_variance_sum_q48`

Sum of squared cache hit latencies (for variance/stddev)

Referenced by [extract_cache_metrics\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.9 cache_hits

`uint64_t HotwordsStats::cache_hits`

Lookups that hit the hot-words cache.

Referenced by [extract_cache_metrics\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_bayesian_report\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.10 evictions

`uint64_t HotwordsStats::evictions`

Words evicted from cache (LRU)

Referenced by [hotwords_stats_print\(\)](#), [hotwords_stats_reset\(\)](#), and [metrics_from_vm\(\)](#).

6.51.1.11 max_bucket_search_ns

`int64_t HotwordsStats::max_bucket_search_ns`

Maximum bucket search latency (nanoseconds)

Referenced by [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.12 max_cache_hit_ns

`int64_t HotwordsStats::max_cache_hit_ns`

Maximum cache hit latency (nanoseconds)

Referenced by [hotwords_posterior_cache_hits\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.13 min_bucket_search_ns

`int64_t HotwordsStats::min_bucket_search_ns`

Minimum bucket search latency (nanoseconds)

Referenced by [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.14 min_cache_hit_ns

`int64_t HotwordsStats::min_cache_hit_ns`

Minimum cache hit latency (nanoseconds)

Referenced by [hotwords_posterior_cache_hits\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.15 misses

`uint64_t HotwordsStats::misses`

Lookups that failed (word not found)

Referenced by [hotwords_bayesian_report\(\)](#), [hotwords_stats_print\(\)](#), and [hotwords_stats_reset\(\)](#).

6.51.1.16 promotions

`uint64_t HotwordsStats::promotions`

Times a word was promoted to hot-words.

Referenced by [hotwords_stats_print\(\)](#), [hotwords_stats_reset\(\)](#), and [metrics_from_vm\(\)](#).

6.51.1.17 total_lookups

`uint64_t HotwordsStats::total_lookups`

Total lookups performed.

Referenced by [extract_cache_metrics\(\)](#), [hotwords_stats_reset\(\)](#), and [metrics_from_vm\(\)](#).

The documentation for this struct was generated from the following file:

- [include/physics_hotwords_cache.h](#)

6.52 idt_entry Struct Reference

Data Fields

- `uint16_t` [offset_low](#)
- `uint16_t` [selector](#)
- `uint8_t` [ist](#)
- `uint8_t` [type_attr](#)
- `uint16_t` [offset_mid](#)
- `uint32_t` [offset_high](#)
- `uint32_t` [zero](#)

6.52.1 Field Documentation

6.52.1.1 ist

`uint8_t idt_entry::ist`

Referenced by [set_idt_entry\(\)](#).

6.52.1.2 offset_high

`uint32_t idt_entry::offset_high`

Referenced by [set_idt_entry\(\)](#).

6.52.1.3 offset_low

`uint16_t idt_entry::offset_low`

Referenced by [set_idt_entry\(\)](#).

6.52.1.4 offset_mid

`uint16_t idt_entry::offset_mid`

Referenced by [set_idt_entry\(\)](#).

6.52.1.5 selector

`uint16_t idt_entry::selector`

Referenced by [set_idt_entry\(\)](#).

6.52.1.6 type_attr

`uint8_t idt_entry::type_attr`

Referenced by [set_idt_entry\(\)](#).

6.52.1.7 zero

`uint32_t idt_entry::zero`

Referenced by [set_idt_entry\(\)](#).

The documentation for this struct was generated from the following file:

- `src/starkernel/arch/amd64/interrupts.c`

6.53 idtr Struct Reference

Data Fields

- `uint16_t limit`
- `uint64_t base`

6.53.1 Field Documentation

6.53.1.1 base

`uint64_t idtr::base`

Referenced by [arch_interrupts_init\(\)](#).

6.53.1.2 limit

`uint16_t idtr::limit`

Referenced by [arch_interrupts_init\(\)](#).

The documentation for this struct was generated from the following file:

- `src/starkernel/arch/amd64/interrupts.c`

6.54 InferenceInputs Struct Reference

```
#include <inference_engine.h>
```

Collaboration diagram for InferenceInputs:

Data Fields

- [VM](#) * [vm](#)
- [RollingWindowOfTruth](#) * [window](#)
- uint64_t [trajectory_length](#)
- uint64_t [prefetch_hits](#)
- uint64_t [prefetch_attempts](#)
- uint64_t [hot_word_count](#)
- uint64_t [stale_word_count](#)
- uint64_t [total_heat](#)
- uint32_t [word_count](#)
- uint64_t [last_total_heat](#)
- uint64_t [last_stale_count](#)

6.54.1 Field Documentation

6.54.1.1 hot_word_count

```
uint64_t InferenceInputs::hot_word_count
```

6.54.1.2 last_stale_count

```
uint64_t InferenceInputs::last_stale_count
```

6.54.1.3 last_total_heat

```
uint64_t InferenceInputs::last_total_heat
```

6.54.1.4 prefetch_attempts

```
uint64_t InferenceInputs::prefetch_attempts
```

6.54.1.5 prefetch_hits

```
uint64_t InferenceInputs::prefetch_hits
```

6.54.1.6 stale_word_count

```
uint64_t InferenceInputs::stale_word_count
```

6.54.1.7 total_heat

```
uint64_t InferenceInputs::total_heat
```

6.54.1.8 trajectory_length

```
uint64_t InferenceInputs::trajectory_length
```

6.54.1.9 vm

```
VM* InferenceInputs::vm
```

Referenced by [inference_engine_run\(\)](#).

6.54.1.10 window

`RollingWindowOfTruth* InferenceInputs::window`

Referenced by [inference_engine_run\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.54.1.11 word_count

`uint32_t InferenceInputs::word_count`

The documentation for this struct was generated from the following file:

- [include/inference_engine.h](#)

6.55 InferenceOutputs Struct Reference

```
#include <inference_engine.h>
```

Data Fields

- `uint32_t` [adaptive_window_width](#)
- `uint64_t` [adaptive_decay_slope](#)
- `uint64_t` [window_variance_q48](#)
- `uint64_t` [slope_fit_quality_q48](#)
- `uint32_t` [early_exited](#)
- `uint64_t` [last_check_tick](#)

6.55.1 Field Documentation

6.55.1.1 adaptive_decay_slope

`uint64_t InferenceOutputs::adaptive_decay_slope`

Referenced by [inference_engine_run\(\)](#), [inference_outputs_to_string\(\)](#), [inference_outputs_validate\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.55.1.2 adaptive_window_width

`uint32_t InferenceOutputs::adaptive_window_width`

Referenced by [inference_engine_run\(\)](#), [inference_outputs_to_string\(\)](#), [inference_outputs_validate\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.55.1.3 early_exited

`uint32_t InferenceOutputs::early_exited`

Referenced by [inference_engine_run\(\)](#), [inference_outputs_to_string\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.55.1.4 last_check_tick

`uint64_t InferenceOutputs::last_check_tick`

6.55.1.5 slope_fit_quality_q48

`uint64_t InferenceOutputs::slope_fit_quality_q48`

Referenced by [inference_engine_run\(\)](#), [inference_outputs_to_string\(\)](#), [inference_outputs_validate\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.55.1.6 window_variance_q48

uint64_t InferenceOutputs::window_variance_q48

Referenced by [inference_engine_run\(\)](#), [inference_outputs_to_string\(\)](#), [metrics_from_vm\(\)](#), and [vm_tick_inference_engine\(\)](#).

The documentation for this struct was generated from the following file:

- [include/inference_engine.h](#)

6.56 kmalloc_prefix_t Struct Reference

Collaboration diagram for kmalloc_prefix_t:

Data Fields

- uint32_t [magic](#)
- [heap_block_t](#) * [block](#)

6.56.1 Field Documentation

6.56.1.1 block

[heap_block_t](#)* kmalloc_prefix_t::block

Referenced by [allocate_from_block\(\)](#), and [kfree\(\)](#).

6.56.1.2 magic

uint32_t kmalloc_prefix_t::magic

Referenced by [allocate_from_block\(\)](#), and [kfree\(\)](#).

The documentation for this struct was generated from the following file:

- [src/starkernel/memory/kmalloc.c](#)

6.57 kmalloc_stats_t Struct Reference

[kmalloc.h](#) - Kernel heap allocator interface

```
#include <kmalloc.h>
```

Data Fields

- uint64_t [total_bytes](#)
- uint64_t [used_bytes](#)
- uint64_t [peak_bytes](#)
- uint64_t [free_bytes](#)

6.57.1 Detailed Description

[kmalloc.h](#) - Kernel heap allocator interface

6.57.2 Field Documentation

6.57.2.1 free_bytes

uint64_t kmalloc_stats_t::free_bytes

Referenced by [kmalloc_init\(\)](#), [print_heap_stats\(\)](#), and [update_free_bytes\(\)](#).

6.57.2.2 peak_bytes

uint64_t kmalloc_stats_t::peak_bytes

Referenced by [account_allocation\(\)](#), [kmalloc_init\(\)](#), and [print_heap_stats\(\)](#).

6.57.2.3 total_bytes

uint64_t kmalloc_stats_t::total_bytes

Referenced by [kmalloc_init\(\)](#), [print_heap_stats\(\)](#), and [update_free_bytes\(\)](#).

6.57.2.4 used_bytes

uint64_t kmalloc_stats_t::used_bytes

Referenced by [account_allocation\(\)](#), [account_free\(\)](#), [kmalloc_init\(\)](#), [print_heap_stats\(\)](#), and [update_free_bytes\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/kmalloc.h](#)

6.58 MemoryProfile Struct Reference

```
#include <profiler.h>
```

Data Fields

- uint64_t [reads](#)
- uint64_t [writes](#)
- uint64_t [cache_hits](#)
- uint64_t [cache_misses](#)
- uint64_t [total_bytes_read](#)
- uint64_t [total_bytes_written](#)

6.58.1 Field Documentation

6.58.1.1 cache_hits

uint64_t MemoryProfile::cache_hits

6.58.1.2 cache_misses

uint64_t MemoryProfile::cache_misses

6.58.1.3 reads

uint64_t MemoryProfile::reads

6.58.1.4 total_bytes_read

uint64_t MemoryProfile::total_bytes_read

6.58.1.5 total_bytes_written

uint64_t MemoryProfile::total_bytes_written

6.58.1.6 writes

uint64_t MemoryProfile::writes

The documentation for this struct was generated from the following file:

- [include/profiler.h](#)

6.59 ParityPacket Struct Reference

[ParityPacket](#) - Summary of [VM](#) state for comparison.

```
#include <parity.h>
```

Data Fields

- uint32_t [word_count](#)
- uint32_t [here_offset](#)
- uint32_t [latest_word_id](#)
- uint64_t [header_hash64](#)
- uint32_t [tests_total](#)
- uint32_t [tests_passed](#)
- uint32_t [tests_failed](#)
- uint32_t [tests_skipped](#)
- uint32_t [tests_errors](#)
- uint64_t [window_hash64](#)
- int [bootstrap_result](#)

6.59.1 Detailed Description

[ParityPacket](#) - Summary of [VM](#) state for comparison.

M7.1a fields are sufficient for bootstrap validation. M7.1b fields are added for POST validation.

6.59.2 Field Documentation

6.59.2.1 bootstrap_result

```
int ParityPacket::bootstrap_result
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.2 header_hash64

```
uint64_t ParityPacket::header_hash64
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.3 here_offset

```
uint32_t ParityPacket::here_offset
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.4 latest_word_id

```
uint32_t ParityPacket::latest_word_id
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.5 tests_errors

```
uint32_t ParityPacket::tests_errors
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.6 tests_failed

```
uint32_t ParityPacket::tests_failed
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.7 tests_passed

```
uint32_t ParityPacket::tests_passed
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.8 tests_skipped

```
uint32_t ParityPacket::tests_skipped
```

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.9 tests_total

uint32_t ParityPacket::tests_total

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

6.59.2.10 window_hash64

uint64_t ParityPacket::window_hash64

Referenced by [sk_parity_collect\(\)](#).

6.59.2.11 word_count

uint32_t ParityPacket::word_count

Referenced by [sk_parity_collect\(\)](#), and [sk_parity_print\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/vm/parity.h](#)

6.60 physics_analytics_event_header Struct Reference

Event header written into the analytics ring buffer.

```
#include <physics_runtime.h>
```

Data Fields

- uint32_t [channel](#)
Logical channel identifier.
- uint16_t [payload_bytes](#)
Size of event payload.
- uint16_t [reserved](#)
Reserved for alignment.
- uint64_t [timestamp_ns](#)
Producer timestamp.

6.60.1 Detailed Description

Event header written into the analytics ring buffer.

6.60.2 Field Documentation

6.60.2.1 channel

uint32_t physics_analytics_event_header::channel

Logical channel identifier.

Referenced by [physics_analytics_publish_event\(\)](#).

6.60.2.2 payload_bytes

uint16_t physics_analytics_event_header::payload_bytes

Size of event payload.

Referenced by [physics_analytics_publish_event\(\)](#).

6.60.2.3 reserved

uint16_t physics_analytics_event_header::reserved

Reserved for alignment.

Referenced by [physics_analytics_publish_event\(\)](#).

6.60.2.4 timestamp_ns

uint64_t physics_analytics_event_header::timestamp_ns

Producer timestamp.

Referenced by [physics_analytics_publish_event\(\)](#).

The documentation for this struct was generated from the following file:

- include/[physics_runtime.h](#)

6.61 physics_analytics_header Struct Reference

Analytics heap header seen by both producer ([VM](#)) and consumer (HOLA).

```
#include <physics_runtime.h>
```

Data Fields

- uint32_t [magic](#)
HOLA_SHARED_MAGIC.
- uint16_t [version_major](#)
Protocol major version.
- uint16_t [version_minor](#)
Protocol minor version.
- uint32_t [heap_bytes](#)
Total heap size.
- uint32_t [ring_offset](#)
Offset to event ring.
- uint32_t [ring_bytes](#)
Event ring length.
- uint32_t [summary_offset](#)
Offset to summary/scratch region.
- uint32_t [summary_bytes](#)
Summary region length.
- uint32_t [scratch_offset](#)
Offset to reserved scratch area.
- uint32_t [scratch_bytes](#)
Scratch area length.
- uint64_t [produce_seq](#)
Monotonic sequence for producers.
- uint64_t [consume_seq](#)
Consumer-visible sequence.
- uint32_t [write_offset](#)
Producer write cursor.
- uint32_t [read_offset](#)
Consumer read cursor.
- uint32_t [dropped_events](#)
Count of dropped events.
- uint32_t [flags](#)
Heap flags (see docs)

6.61.1 Detailed Description

Analytics heap header seen by both producer ([VM](#)) and consumer (HOLA).

6.61.2 Field Documentation

6.61.2.1 consume_seq

`uint64_t physics_analytics_header::consume_seq`
Consumer-visible sequence.
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.2 dropped_events

`uint32_t physics_analytics_header::dropped_events`
Count of dropped events.
Referenced by [analytics_heap_allocate\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.61.2.3 flags

`uint32_t physics_analytics_header::flags`
Heap flags (see docs)
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.4 heap_bytes

`uint32_t physics_analytics_header::heap_bytes`
Total heap size.
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.5 magic

`uint32_t physics_analytics_header::magic`
HOLA_SHARED_MAGIC.
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.6 produce_seq

`uint64_t physics_analytics_header::produce_seq`
Monotonic sequence for producers.
Referenced by [analytics_heap_allocate\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.61.2.7 read_offset

`uint32_t physics_analytics_header::read_offset`
Consumer read cursor.
Referenced by [analytics_heap_allocate\(\)](#), and [analytics_ring_free\(\)](#).

6.61.2.8 ring_bytes

`uint32_t physics_analytics_header::ring_bytes`
Event ring length.
Referenced by [analytics_heap_allocate\(\)](#), [analytics_ring_free\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.61.2.9 ring_offset

`uint32_t physics_analytics_header::ring_offset`
Offset to event ring.
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.10 scratch_bytes

`uint32_t physics_analytics_header::scratch_bytes`
Scratch area length.
Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.11 scratch_offset

```
uint32_t physics_analytics_header::scratch_offset
```

Offset to reserved scratch area.

Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.12 summary_bytes

```
uint32_t physics_analytics_header::summary_bytes
```

Summary region length.

Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.13 summary_offset

```
uint32_t physics_analytics_header::summary_offset
```

Offset to summary/scratch region.

Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.14 version_major

```
uint16_t physics_analytics_header::version_major
```

Protocol major version.

Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.15 version_minor

```
uint16_t physics_analytics_header::version_minor
```

Protocol minor version.

Referenced by [analytics_heap_allocate\(\)](#).

6.61.2.16 write_offset

```
uint32_t physics_analytics_header::write_offset
```

Producer write cursor.

Referenced by [analytics_heap_allocate\(\)](#), [analytics_ring_free\(\)](#), and [physics_analytics_publish_event\(\)](#).

The documentation for this struct was generated from the following file:

- [include/physics_runtime.h](#)

6.62 physics_analytics_heap Struct Reference

Simple view onto the analytics heap layout.

```
#include <physics_runtime.h>
```

Collaboration diagram for physics_analytics_heap:

Data Fields

- `uint8_t * base`
Base pointer (owned by runtime)
- `size_t bytes`
Total bytes reserved.
- `physics\_analytics\_header\_t * header`
Shared header.
- `uint8_t * ring`
Event ring base.
- `size_t ring\_bytes`
Event ring span.
- `uint8_t * summary`

Summary/statistics region.

- `size_t` [summary_bytes](#)

Summary region span.

- `uint8_t *` [scratch](#)

Scratch/reserved region.

- `size_t` [scratch_bytes](#)

Scratch region span.

6.62.1 Detailed Description

Simple view onto the analytics heap layout.

6.62.2 Field Documentation

6.62.2.1 base

```
uint8_t* physics_analytics_heap::base
```

Base pointer (owned by runtime)

Referenced by [analytics_heap_allocate\(\)](#), and [analytics_heap_release\(\)](#).

6.62.2.2 bytes

```
size_t physics_analytics_heap::bytes
```

Total bytes reserved.

Referenced by [analytics_heap_allocate\(\)](#).

6.62.2.3 header

```
physics_analytics_header_t* physics_analytics_heap::header
```

Shared header.

Referenced by [analytics_heap_allocate\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.62.2.4 ring

```
uint8_t* physics_analytics_heap::ring
```

Event ring base.

Referenced by [analytics_heap_allocate\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.62.2.5 ring_bytes

```
size_t physics_analytics_heap::ring_bytes
```

Event ring span.

Referenced by [analytics_heap_allocate\(\)](#).

6.62.2.6 scratch

```
uint8_t* physics_analytics_heap::scratch
```

Scratch/reserved region.

Referenced by [analytics_heap_allocate\(\)](#).

6.62.2.7 scratch_bytes

```
size_t physics_analytics_heap::scratch_bytes
```

Scratch region span.

Referenced by [analytics_heap_allocate\(\)](#).

6.62.2.8 summary

uint8_t* physics_analytics_heap::summary
 Summary/statistics region.
 Referenced by [analytics_heap_allocate\(\)](#).

6.62.2.9 summary_bytes

size_t physics_analytics_heap::summary_bytes
 Summary region span.
 Referenced by [analytics_heap_allocate\(\)](#).
 The documentation for this struct was generated from the following file:

- include/[physics_runtime.h](#)

6.63 physics_host_snapshot Struct Reference

Snapshot of scheduler-adjacent host signals.
 #include <physics_runtime.h>

Data Fields

- [physics_host_backend_t](#) backend
Selected backend (POSIX/L4Re/other)
- uint32_t [scheduler_policy](#)
OS-specific scheduler policy identifier.
- uint32_t [scheduler_priority](#)
Nominal thread priority.
- uint32_t [scheduler_quantum_ns](#)
Reported quantum in nanoseconds (0 if unknown)
- uint32_t [runnable_threads](#)
Runnable entities on this CPU set (best effort)
- uint32_t [cpu_count](#)
Online logical CPU count.
- uint32_t [load_avg_milli](#)
Smoothed load average ($\times 1000$)
- uint32_t [flags](#)
Backend-specific flags.
- uint32_t [psi_cpu_some_avg10_milli](#)
CPU PSI some avg10 $\times 1000$.
- uint32_t [psi_cpu_some_avg60_milli](#)
CPU PSI some avg60 $\times 1000$.
- uint32_t [psi_cpu_some_avg300_milli](#)
CPU PSI some avg300 $\times 1000$.
- uint32_t [psi_cpu_full_avg10_milli](#)
CPU PSI full avg10 $\times 1000$.
- uint32_t [psi_cpu_full_avg60_milli](#)
CPU PSI full avg60 $\times 1000$.
- uint32_t [psi_cpu_full_avg300_milli](#)
CPU PSI full avg300 $\times 1000$.
- uint32_t [psi_io_some_avg10_milli](#)
IO PSI some avg10 $\times 1000$.
- uint32_t [psi_io_some_avg60_milli](#)
IO PSI some avg60 $\times 1000$.

- uint32_t [psi_io_some_avg300_milli](#)
IO PSI some avg300 × 1000.
- uint32_t [psi_io_full_avg10_milli](#)
IO PSI full avg10 × 1000.
- uint32_t [psi_io_full_avg60_milli](#)
IO PSI full avg60 × 1000.
- uint32_t [psi_io_full_avg300_milli](#)
IO PSI full avg300 × 1000.
- uint32_t [psi_mem_some_avg10_milli](#)
Memory PSI some avg10 × 1000.
- uint32_t [psi_mem_some_avg60_milli](#)
Memory PSI some avg60 × 1000.
- uint32_t [psi_mem_some_avg300_milli](#)
Memory PSI some avg300 × 1000.
- uint64_t [monotonic_time_ns](#)
Monotonic clock reading.
- uint64_t [realtime_ns](#)
Wall-clock timestamp (0 if unavailable)
- uint64_t [backend_seq](#)
Backend-provided sequence counter.
- uint64_t [cpu_total_jiffies](#)
/proc/stat total jiffies snapshot
- uint64_t [cpu_idle_jiffies](#)
/proc/stat idle jiffies snapshot
- uint64_t [cgroup_cpu_usage_us](#)
cgroup v2 cpu.stat usage_usec (0 if unavailable)
- uint64_t [cgroup_memory_current_bytes](#)
cgroup v2 memory.current (0 if unavailable)

6.63.1 Detailed Description

Snapshot of scheduler-adjacent host signals.

Fields are intentionally fixed width to maximise portability and allow the governance repository to consume binary dumps without per-platform quirks.

6.63.2 Field Documentation

6.63.2.1 backend

[physics_host_backend_t](#) physics_host_snapshot::backend

Selected backend (POSIX/L4Re/other)

Referenced by [snapshot_posix\(\)](#).

6.63.2.2 backend_seq

uint64_t physics_host_snapshot::backend_seq

Backend-provided sequence counter.

Referenced by [snapshot_posix\(\)](#).

6.63.2.3 cgroup_cpu_usage_us

uint64_t physics_host_snapshot::cgroup_cpu_usage_us

cgroup v2 cpu.stat usage_usec (0 if unavailable)

Referenced by [snapshot_posix\(\)](#).

6.63.2.4 cgroup_memory_current_bytes

uint64_t physics_host_snapshot::cgroup_memory_current_bytes
cgroup v2 memory.current (0 if unavailable)
Referenced by [snapshot_posix\(\)](#).

6.63.2.5 cpu_count

uint32_t physics_host_snapshot::cpu_count
Online logical CPU count.
Referenced by [snapshot_posix\(\)](#).

6.63.2.6 cpu_idle_jiffies

uint64_t physics_host_snapshot::cpu_idle_jiffies
/proc/stat idle jiffies snapshot
Referenced by [snapshot_posix\(\)](#).

6.63.2.7 cpu_total_jiffies

uint64_t physics_host_snapshot::cpu_total_jiffies
/proc/stat total jiffies snapshot
Referenced by [snapshot_posix\(\)](#).

6.63.2.8 flags

uint32_t physics_host_snapshot::flags
Backend-specific flags.
Referenced by [snapshot_posix\(\)](#).

6.63.2.9 load_avg_milli

uint32_t physics_host_snapshot::load_avg_milli
Smoothed load average ($\times 1000$)
Referenced by [snapshot_posix\(\)](#).

6.63.2.10 monotonic_time_ns

uint64_t physics_host_snapshot::monotonic_time_ns
Monotonic clock reading.
Referenced by [snapshot_posix\(\)](#).

6.63.2.11 psi_cpu_full_avg10_milli

uint32_t physics_host_snapshot::psi_cpu_full_avg10_milli
CPU PSI full avg10 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.12 psi_cpu_full_avg300_milli

uint32_t physics_host_snapshot::psi_cpu_full_avg300_milli
CPU PSI full avg300 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.13 psi_cpu_full_avg60_milli

uint32_t physics_host_snapshot::psi_cpu_full_avg60_milli
CPU PSI full avg60 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.14 psi_cpu_some_avg10_milli

uint32_t physics_host_snapshot::psi_cpu_some_avg10_milli
CPU PSI some avg10 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.15 psi_cpu_some_avg300_milli

uint32_t physics_host_snapshot::psi_cpu_some_avg300_milli
CPU PSI some avg300 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.16 psi_cpu_some_avg60_milli

uint32_t physics_host_snapshot::psi_cpu_some_avg60_milli
CPU PSI some avg60 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.17 psi_io_full_avg10_milli

uint32_t physics_host_snapshot::psi_io_full_avg10_milli
IO PSI full avg10 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.18 psi_io_full_avg300_milli

uint32_t physics_host_snapshot::psi_io_full_avg300_milli
IO PSI full avg300 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.19 psi_io_full_avg60_milli

uint32_t physics_host_snapshot::psi_io_full_avg60_milli
IO PSI full avg60 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.20 psi_io_some_avg10_milli

uint32_t physics_host_snapshot::psi_io_some_avg10_milli
IO PSI some avg10 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.21 psi_io_some_avg300_milli

uint32_t physics_host_snapshot::psi_io_some_avg300_milli
IO PSI some avg300 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.22 psi_io_some_avg60_milli

uint32_t physics_host_snapshot::psi_io_some_avg60_milli
IO PSI some avg60 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.23 psi_mem_some_avg10_milli

uint32_t physics_host_snapshot::psi_mem_some_avg10_milli
Memory PSI some avg10 $\times 1000$.
Referenced by [snapshot_posix\(\)](#).

6.63.2.24 psi_mem_some_avg300_milli

uint32_t physics_host_snapshot::psi_mem_some_avg300_milli

Memory PSI some avg300 $\times 1000$.

Referenced by [snapshot_posix\(\)](#).

6.63.2.25 psi_mem_some_avg60_milli

uint32_t physics_host_snapshot::psi_mem_some_avg60_milli

Memory PSI some avg60 $\times 1000$.

Referenced by [snapshot_posix\(\)](#).

6.63.2.26 realtime_ns

uint64_t physics_host_snapshot::realtime_ns

Wall-clock timestamp (0 if unavailable)

Referenced by [snapshot_posix\(\)](#).

6.63.2.27 runnable_threads

uint32_t physics_host_snapshot::runnable_threads

Runnable entities on this CPU set (best effort)

Referenced by [snapshot_posix\(\)](#).

6.63.2.28 scheduler_policy

uint32_t physics_host_snapshot::scheduler_policy

OS-specific scheduler policy identifier.

Referenced by [snapshot_posix\(\)](#).

6.63.2.29 scheduler_priority

uint32_t physics_host_snapshot::scheduler_priority

Nominal thread priority.

Referenced by [snapshot_posix\(\)](#).

6.63.2.30 scheduler_quantum_ns

uint32_t physics_host_snapshot::scheduler_quantum_ns

Reported quantum in nanoseconds (0 if unknown)

Referenced by [snapshot_posix\(\)](#).

The documentation for this struct was generated from the following file:

- [include/physics_runtime.h](#)

6.64 physics_runtime_state Struct Reference

Collaboration diagram for physics_runtime_state:

Data Fields

- [physics_analytics_heap_t heap](#)
- int [heap_ready](#)
- uint64_t [backend_seq](#)

6.64.1 Field Documentation**6.64.1.1 backend_seq**

uint64_t physics_runtime_state::backend_seq

Referenced by [analytics_heap_allocate\(\)](#), [analytics_heap_release\(\)](#), and [snapshot_posix\(\)](#).

6.64.1.2 heap

`physics_analytics_heap_t physics_runtime_state::heap`

Referenced by [analytics_heap_allocate\(\)](#), [analytics_heap_release\(\)](#), [physics_analytics_heap_info\(\)](#), and [physics_analytics_publish_event\(\)](#).

6.64.1.3 heap_ready

`int physics_runtime_state::heap_ready`

Referenced by [analytics_heap_allocate\(\)](#), [analytics_heap_release\(\)](#), [physics_analytics_heap_info\(\)](#), [physics_analytics_publish_event\(\)](#) and [physics_host_snapshot\(\)](#).

The documentation for this struct was generated from the following file:

- [src/physics_runtime.c](#)

6.65 physics_seed_definition Struct Reference

Data Fields

- `const char *` [name](#)
- `uint16_t` [temperature_q8](#)
- `uint32_t` [avg_latency_ns](#)
- `uint8_t` [acl_hint](#)
- `uint8_t` [pubsub_mask](#)

6.65.1 Field Documentation

6.65.1.1 acl_hint

`uint8_t physics_seed_definition::acl_hint`

6.65.1.2 avg_latency_ns

`uint32_t physics_seed_definition::avg_latency_ns`

6.65.1.3 name

`const char* physics_seed_definition::name`

Referenced by [physics_metadata_apply_seed\(\)](#).

6.65.1.4 pubsub_mask

`uint8_t physics_seed_definition::pubsub_mask`

6.65.1.5 temperature_q8

`uint16_t physics_seed_definition::temperature_q8`

Referenced by [physics_metadata_apply_seed\(\)](#).

The documentation for this struct was generated from the following file:

- [src/physics_metadata.c](#)

6.66 PhysicsSnapshot Struct Reference

```
#include <test_contracts.h>
```

Data Fields

- uint64_t [decay_slope_q48](#)
- uint32_t [eff_window_size](#)
- uint64_t [tick_target_ns](#)
- [cell_t](#) [heat_25th](#)
- [cell_t](#) [heat_50th](#)
- [cell_t](#) [heat_75th](#)
- uint64_t [prefetch_attempts](#)
- uint64_t [total_executions](#)

6.66.1 Field Documentation**6.66.1.1 decay_slope_q48**

uint64_t PhysicsSnapshot::decay_slope_q48

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.2 eff_window_size

uint32_t PhysicsSnapshot::eff_window_size

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.3 heat_25th

[cell_t](#) PhysicsSnapshot::heat_25th

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.4 heat_50th

[cell_t](#) PhysicsSnapshot::heat_50th

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.5 heat_75th

[cell_t](#) PhysicsSnapshot::heat_75th

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.6 prefetch_attempts

uint64_t PhysicsSnapshot::prefetch_attempts

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.7 tick_target_ns

uint64_t PhysicsSnapshot::tick_target_ns

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

6.66.1.8 total_executions

uint64_t PhysicsSnapshot::total_executions

Referenced by [restore_physics\(\)](#), and [save_physics\(\)](#).

The documentation for this struct was generated from the following file:

- [src/test_runner/include/test_contracts.h](#)

6.67 PipelineGlobalMetrics Struct Reference

```
#include <vm.h>
```

Data Fields

- uint64_t [prefetch_attempts](#)
- uint64_t [prefetch_hits](#)
- uint64_t [window_tuning_checks](#)
- uint32_t [last_checked_window_size](#)
- double [last_checked_accuracy](#)
- uint32_t [suggested_next_size](#)

6.67.1 Field Documentation

6.67.1.1 last_checked_accuracy

double PipelineGlobalMetrics::last_checked_accuracy

Referenced by [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.67.1.2 last_checked_window_size

uint32_t PipelineGlobalMetrics::last_checked_window_size

Referenced by [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.67.1.3 prefetch_attempts

uint64_t PipelineGlobalMetrics::prefetch_attempts

Referenced by [execute_colon_word\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [physics_pre_execute\(\)](#), [restore_physics\(\)](#), [save_physics\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.67.1.4 prefetch_hits

uint64_t PipelineGlobalMetrics::prefetch_hits

Referenced by [execute_colon_word\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), [metrics_from_vm\(\)](#), [physics_pre_execute\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.67.1.5 suggested_next_size

uint32_t PipelineGlobalMetrics::suggested_next_size

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.67.1.6 window_tuning_checks

uint64_t PipelineGlobalMetrics::window_tuning_checks

Referenced by [forth_PHYSICS_RESET_STATS\(\)](#), [metrics_from_vm\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_window_tuner\(\)](#).

The documentation for this struct was generated from the following file:

- include/[vm.h](#)

6.68 pmm_stats_t Struct Reference

```
#include <pmm.h>
```

Data Fields

- uint64_t [total_pages](#)
- uint64_t [used_pages](#)
- uint64_t [free_pages](#)
- uint64_t [total_bytes](#)

- `uint64_t` [used_bytes](#)
- `uint64_t` [free_bytes](#)

6.68.1 Field Documentation

6.68.1.1 `free_bytes`

`uint64_t pmm_stats_t::free_bytes`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

6.68.1.2 `free_pages`

`uint64_t pmm_stats_t::free_pages`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

6.68.1.3 `total_bytes`

`uint64_t pmm_stats_t::total_bytes`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

6.68.1.4 `total_pages`

`uint64_t pmm_stats_t::total_pages`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

6.68.1.5 `used_bytes`

`uint64_t pmm_stats_t::used_bytes`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

6.68.1.6 `used_pages`

`uint64_t pmm_stats_t::used_pages`

Referenced by [pmm_get_stats\(\)](#), and [print_pmm_stats\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/pmm.h](#)

6.69 Profiler Struct Reference

```
#include <profiler.h>
```

Collaboration diagram for Profiler:

Data Fields

- [ProfileLevel](#) `level`
- `uint64_t` [profiling_overhead](#)
- `uint64_t` [total_execution_time](#)
- [ProfileTimer](#) `category_timers` [[PROF_CATEGORY_COUNT](#)]
- [WordProfile](#) * `word_profiles`
- `size_t` [word_profile_count](#)
- `size_t` [word_profile_capacity](#)
- [MemoryProfile](#) `memory_stats`
- `uint64_t` [instruction_count](#)
- `uint64_t` [branch_count](#)
- `uint64_t` [loop_iterations](#)

- struct {
 - uint64_t vm_cycles
 - uint64_t dict_lookups
 - uint64_t stack_operations
 - uint64_t memory_allocations
 - uint64_t compile_operations
 - } counters

6.69.1 Field Documentation

6.69.1.1 branch_count

uint64_t Profiler::branch_count

6.69.1.2 category_timers

ProfileTimer Profiler::category_timers[PROF_CATEGORY_COUNT]

6.69.1.3 compile_operations

uint64_t Profiler::compile_operations

6.69.1.4 [struct]

struct { ... } Profiler::counters

6.69.1.5 dict_lookups

uint64_t Profiler::dict_lookups

6.69.1.6 instruction_count

uint64_t Profiler::instruction_count

6.69.1.7 level

ProfileLevel Profiler::level

6.69.1.8 loop_iterations

uint64_t Profiler::loop_iterations

6.69.1.9 memory_allocations

uint64_t Profiler::memory_allocations

6.69.1.10 memory_stats

MemoryProfile Profiler::memory_stats

6.69.1.11 profiling_overhead

uint64_t Profiler::profiling_overhead

6.69.1.12 stack_operations

uint64_t Profiler::stack_operations

6.69.1.13 total_execution_time

```
uint64_t Profiler::total_execution_time
```

6.69.1.14 vm_cycles

```
uint64_t Profiler::vm_cycles
```

6.69.1.15 word_profile_capacity

```
size_t Profiler::word_profile_capacity
```

6.69.1.16 word_profile_count

```
size_t Profiler::word_profile_count
```

6.69.1.17 word_profiles

```
WordProfile* Profiler::word_profiles
```

The documentation for this struct was generated from the following file:

- include/[profiler.h](#)

6.70 ProfileScope Struct Reference

```
#include <profiler.h>
```

Data Fields

- [ProfileCategory](#) category
- const char * [word_name](#)

6.70.1 Field Documentation

6.70.1.1 category

```
ProfileCategory ProfileScope::category
```

6.70.1.2 word_name

```
const char* ProfileScope::word_name
```

The documentation for this struct was generated from the following file:

- include/[profiler.h](#)

6.71 ProfileTimer Struct Reference

Timer for measuring execution duration.

```
#include <profiler.h>
```

Data Fields

- uint64_t [start_time](#)
Start timestamp.
- uint64_t [total_time](#)
Accumulated execution time.
- uint64_t [call_count](#)
Number of measurements.

- `uint64_t min_time`
Minimum recorded time.
- `uint64_t max_time`
Maximum recorded time.
- `const char * name`
Timer identifier.

6.71.1 Detailed Description

Timer for measuring execution duration.

6.71.2 Field Documentation

6.71.2.1 `call_count`

`uint64_t ProfileTimer::call_count`
Number of measurements.

6.71.2.2 `max_time`

`uint64_t ProfileTimer::max_time`
Maximum recorded time.

6.71.2.3 `min_time`

`uint64_t ProfileTimer::min_time`
Minimum recorded time.

6.71.2.4 `name`

`const char* ProfileTimer::name`
Timer identifier.

6.71.2.5 `start_time`

`uint64_t ProfileTimer::start_time`
Start timestamp.

6.71.2.6 `total_time`

`uint64_t ProfileTimer::total_time`
Accumulated execution time.

The documentation for this struct was generated from the following file:

- `include/profiler.h`

6.72 `reloc_block` Struct Reference

[reloc_stub.c](#) - Provide a minimal `.reloc` section for PE/COFF EFI loader

Data Fields

- `uint32_t page_rva`
- `uint32_t block_size`
- `uint16_t entry`

6.72.1 Detailed Description

[reloc_stub.c](#) - Provide a minimal .reloc section for PE/COFF EFI loader

Some firmware refuses to load images without a Base Relocation Directory. This stub supplies an empty relocation block to satisfy the loader.

6.72.2 Field Documentation

6.72.2.1 block_size

```
uint32_t reloc_block::block_size
```

6.72.2.2 entry

```
uint16_t reloc_block::entry
```

6.72.2.3 page_rva

```
uint32_t reloc_block::page_rva
```

Referenced by [__attribute__\(\)](#).

The documentation for this struct was generated from the following file:

- [src/starkernel/boot/reloc_stub.c](#)

6.73 rolling_window_view_t Struct Reference

Data Fields

- `const uint32_t *` [history](#)
- `uint32_t` [window_pos](#)
- `uint64_t` [total_executions](#)
- `uint32_t` [effective_window_size](#)
- `int` [is_warm](#)

6.73.1 Field Documentation

6.73.1.1 effective_window_size

```
uint32_t rolling_window_view_t::effective_window_size
```

Referenced by [rolling_window_measure_diversity_view\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.73.1.2 history

```
const uint32_t* rolling_window_view_t::history
```

Referenced by [rolling_window_count_transition\(\)](#), [rolling_window_export_execution_history\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_get_recent_sequence\(\)](#), [rolling_window_measure_diversity_view\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_run_adaptive_pass\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.73.1.3 is_warm

```
int rolling_window_view_t::is_warm
```

Referenced by [rolling_window_count_transition\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_measure_diversity_view\(\)](#), [rolling_window_run_adaptive_pass\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [rolling_window_snapshot_view\(\)](#), and [rolling_window_stats_string\(\)](#).

6.73.1.4 total_executions

uint64_t rolling_window_view_t::total_executions

Referenced by [rolling_window_export_execution_history\(\)](#), [rolling_window_get_recent_sequence\(\)](#), [rolling_window_measure_diversity\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_run_adaptive_pass\(\)](#), [rolling_window_snapshot_view\(\)](#), and [rolling_window_stats_string\(\)](#).

6.73.1.5 window_pos

uint32_t rolling_window_view_t::window_pos

Referenced by [rolling_window_export_execution_history\(\)](#), [rolling_window_get_recent_sequence\(\)](#), [rolling_window_measure_diversity\(\)](#), [rolling_window_snapshot_view\(\)](#), and [rolling_window_stats_string\(\)](#).

The documentation for this struct was generated from the following file:

- [src/rolling_window_of_truth.c](#)

6.74 RollingWindowOfTruth Struct Reference

```
#include <vm.h>
```

Data Fields

- uint32_t * [execution_history](#)
- uint32_t * [snapshot_buffers](#) [2]
- uint32_t [window_pos](#)
- uint64_t [total_executions](#)
- int [is_warm](#)
- uint32_t [effective_window_size](#)
- uint64_t [last_pattern_diversity](#)
- uint64_t [pattern_diversity_check_count](#)
- volatile uint32_t [snapshot_index](#)
- volatile uint32_t [snapshot_pending](#)
- uint32_t [snapshot_window_pos](#) [2]
- uint64_t [snapshot_total_executions](#) [2]
- uint32_t [snapshot_effective_window_size](#) [2]
- int [snapshot_is_warm](#) [2]
- uint32_t [adaptive_check_accumulator](#)
- volatile uint32_t [adaptive_pending](#)

6.74.1 Field Documentation

6.74.1.1 adaptive_check_accumulator

uint32_t RollingWindowOfTruth::adaptive_check_accumulator

Referenced by [rolling_window_init\(\)](#), and [rolling_window_record_execution\(\)](#).

6.74.1.2 adaptive_pending

volatile uint32_t RollingWindowOfTruth::adaptive_pending

Referenced by [rolling_window_check_adaptive_shrink\(\)](#), [rolling_window_init\(\)](#), [rolling_window_record_execution\(\)](#), and [rolling_window_service_internal\(\)](#).

6.74.1.3 effective_window_size

uint32_t RollingWindowOfTruth::effective_window_size

Referenced by [extract_heat_trajectory\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_publish_snapshot\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_run_adaptive_pass\(\)](#), [run_doe_experiment\(\)](#), [save_physics\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.74.1.4 execution_history

uint32_t* RollingWindowOfTruth::execution_history

Referenced by [rolling_window_cleanup\(\)](#), [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_record_execution\(\)](#), and [rolling_window_reset\(\)](#).

6.74.1.5 is_warm

int RollingWindowOfTruth::is_warm

Referenced by [extract_heat_trajectory\(\)](#), [rolling_window_count_transition\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_init\(\)](#), [rolling_window_is_warm\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_record_execution\(\)](#), [rolling_window_reset\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [run_doe_experiment\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.74.1.6 last_pattern_diversity

uint64_t RollingWindowOfTruth::last_pattern_diversity

Referenced by [rolling_window_init\(\)](#), and [rolling_window_run_adaptive_pass\(\)](#).

6.74.1.7 pattern_diversity_check_count

uint64_t RollingWindowOfTruth::pattern_diversity_check_count

Referenced by [rolling_window_init\(\)](#), and [rolling_window_run_adaptive_pass\(\)](#).

6.74.1.8 snapshot_buffers

uint32_t* RollingWindowOfTruth::snapshot_buffers[2]

Referenced by [rolling_window_cleanup\(\)](#), [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.9 snapshot_effective_window_size

uint32_t RollingWindowOfTruth::snapshot_effective_window_size[2]

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.10 snapshot_index

volatile uint32_t RollingWindowOfTruth::snapshot_index

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.11 snapshot_is_warm

int RollingWindowOfTruth::snapshot_is_warm[2]

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.12 snapshot_pending

volatile uint32_t RollingWindowOfTruth::snapshot_pending

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [rolling_window_record_execution\(\)](#), and [rolling_window_reset\(\)](#).

6.74.1.13 snapshot_total_executions

uint64_t RollingWindowOfTruth::snapshot_total_executions[2]

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.14 snapshot_window_pos

uint32_t RollingWindowOfTruth::snapshot_window_pos[2]

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), and [rolling_window_snapshot_view\(\)](#).

6.74.1.15 total_executions

uint64_t RollingWindowOfTruth::total_executions

Referenced by [heartbeat_capture_tick_snapshot\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_record_execution\(\)](#), [rolling_window_reset\(\)](#), [run_doe_experiment\(\)](#), [save_physics\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.74.1.16 window_pos

uint32_t RollingWindowOfTruth::window_pos

Referenced by [rolling_window_init\(\)](#), [rolling_window_publish_snapshot\(\)](#), [rolling_window_record_execution\(\)](#), [rolling_window_reset\(\)](#), and [vm_tick_inference_engine\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.75 sf_alloc_stats Struct Reference

```
#include <platform_alloc.h>
```

Data Fields

- size_t [total_bytes](#)
- size_t [used_bytes](#)
- size_t [peak_bytes](#)
- size_t [alloc_count](#)
- size_t [free_count](#)

6.75.1 Field Documentation

6.75.1.1 alloc_count

size_t sf_alloc_stats::alloc_count

Referenced by [sf_alloc_init\(\)](#), [sf_calloc\(\)](#), [sf_malloc\(\)](#), and [sf_realloc\(\)](#).

6.75.1.2 free_count

size_t sf_alloc_stats::free_count

Referenced by [sf_alloc_init\(\)](#), and [sf_free\(\)](#).

6.75.1.3 peak_bytes

size_t sf_alloc_stats::peak_bytes

Referenced by [sf_alloc_init\(\)](#), [sf_calloc\(\)](#), and [sf_malloc\(\)](#).

6.75.1.4 total_bytes

size_t sf_alloc_stats::total_bytes

Referenced by [sf_alloc_init\(\)](#).

6.75.1.5 used_bytes

size_t sf_alloc_stats::used_bytes

Referenced by [sf_alloc_init\(\)](#), [sf_calloc\(\)](#), and [sf_malloc\(\)](#).

The documentation for this struct was generated from the following file:

- [include/platform_alloc.h](#)

6.76 sf_mutex Struct Reference

```
#include <platform_lock.h>
```

Data Fields

- pthread_mutex_t [handle](#)

6.76.1 Field Documentation

6.76.1.1 handle

```
pthread_mutex_t sf_mutex::handle
```

Referenced by [sf_mutex_destroy\(\)](#), [sf_mutex_init\(\)](#), [sf_mutex_lock\(\)](#), and [sf_mutex_unlock\(\)](#).

The documentation for this struct was generated from the following file:

- include/[platform_lock.h](#)

6.77 sf_time_backend_t Struct Reference

```
#include <platform_time.h>
```

Data Fields

- [sf_time_ns_t](#)(* [get_monotonic_ns](#))(void)
Get monotonic time (for performance measurement)
- [sf_time_ns_t](#)(* [get_realtime_ns](#))(void)
Get wall-clock time.
- int(* [set_realtime_ns](#))([sf_time_ns_t](#) ns_since_epoch)
Set wall-clock time (may require privileges)
- int(* [format_timestamp](#))([sf_time_ns_t](#) ns_since_epoch, char *buf, int format_24h)
Format timestamp as human-readable string.
- int(* [has_rtc](#))(void)
Check if real-time clock is available.

6.77.1 Field Documentation

6.77.1.1 format_timestamp

```
int(* sf_time_backend_t::format_timestamp) (sf\_time\_ns\_t ns_since_epoch, char *buf, int format_↵
_24h)
```

Format timestamp as human-readable string.

Parameters

<i>ns_since_epoch</i>	Nanoseconds since Unix epoch
<i>buf</i>	Output buffer (SF_TIME_STAMP_SIZE bytes)
<i>format_24h</i>	Use 24-hour format (1) or 12-hour (0)

Returns

0 on success, -1 on error

Referenced by [sf_format_timestamp\(\)](#).

6.77.1.2 get_monotonic_ns

```
sf\_time\_ns\_t(* sf_time_backend_t::get_monotonic_ns) (void)
```

Get monotonic time (for performance measurement)

Returns

Nanoseconds since system boot (never decreases)

Referenced by [sf_monotonic_ns\(\)](#).

6.77.1.3 get_realtime_ns

```
sf_time_ns_t (* sf_time_backend_t::get_realtime_ns) (void)
```

Get wall-clock time.

Returns

Nanoseconds since Unix epoch (1970-01-01 00:00:00 UTC) Returns 0 if RTC unavailable

Referenced by [sf_realtime_ns\(\)](#).

6.77.1.4 has_rtc

```
int (* sf_time_backend_t::has_rtc) (void)
```

Check if real-time clock is available.

Returns

1 if RTC available, 0 if not

Referenced by [sf_has_rtc\(\)](#).

6.77.1.5 set_realtime_ns

```
int (* sf_time_backend_t::set_realtime_ns) (sf_time_ns_t ns_since_epoch)
```

Set wall-clock time (may require privileges)

Parameters

<i>ns_since_epoch</i>	Nanoseconds since Unix epoch
-----------------------	------------------------------

Returns

0 on success, -1 on error

Referenced by [sf_set_realtime_ns\(\)](#).

The documentation for this struct was generated from the following file:

- [include/platform_time.h](#)

6.78 SpeedupEstimate Struct Reference

Calculate speedup estimate with credible interval All values in Q48.16 fixed-point format (64-bit signed integer) Probabilities are in range [0, Q48_SCALE] representing [0%, 100%].

```
#include <physics_hotwords_cache.h>
```

Data Fields

- `int64_t` [speedup_factor_q48](#)
- `int64_t` [credible_lower_95_q48](#)
- `int64_t` [credible_upper_95_q48](#)
- `int64_t` [probability_gt_10pct_q48](#)
- `int64_t` [probability_gt_double_q48](#)

6.78.1 Detailed Description

Calculate speedup estimate with credible interval All values in Q48.16 fixed-point format (64-bit signed integer) Probabilities are in range [0, Q48_SCALE] representing [0%, 100%].

6.78.2 Field Documentation

6.78.2.1 credible_lower_95_q48

int64_t SpeedupEstimate::credible_lower_95_q48

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

6.78.2.2 credible_upper_95_q48

int64_t SpeedupEstimate::credible_upper_95_q48

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

6.78.2.3 probability_gt_10pct_q48

int64_t SpeedupEstimate::probability_gt_10pct_q48

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

6.78.2.4 probability_gt_double_q48

int64_t SpeedupEstimate::probability_gt_double_q48

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

6.78.2.5 speedup_factor_q48

int64_t SpeedupEstimate::speedup_factor_q48

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

The documentation for this struct was generated from the following file:

- [include/physics_hotwords_cache.h](#)

6.79 ssm_config_t Struct Reference

```
#include <ssm_jacquard.h>
```

Data Fields

- int [L2_rolling_window](#)
- int [L3_linear_decay](#)
- int [L5_window_inference](#)
- int [L6_decay_inference](#)

6.79.1 Field Documentation

6.79.1.1 L2_rolling_window

int ssm_config_t::L2_rolling_window

Referenced by [ssm_apply_mode\(\)](#).

6.79.1.2 L3_linear_decay

int ssm_config_t::L3_linear_decay

Referenced by [physics_metadata_apply_linear_decay\(\)](#), and [ssm_apply_mode\(\)](#).

6.79.1.3 L5_window_inference

```
int ssm_config_t::L5_window_inference
```

Referenced by [inference_engine_run\(\)](#), and [ssm_apply_mode\(\)](#).

6.79.1.4 L6_decay_inference

```
int ssm_config_t::L6_decay_inference
```

Referenced by [inference_engine_run\(\)](#), and [ssm_apply_mode\(\)](#).

The documentation for this struct was generated from the following file:

- [include/ssm_jacquard.h](#)

6.80 ssm_l8_metrics_t Struct Reference

```
#include <ssm_jacquard.h>
```

Data Fields

- double [entropy](#)
- double [cv](#)
- double [temporal_decay](#)
- double [stability_score](#)

6.80.1 Field Documentation

6.80.1.1 cv

```
double ssm_l8_metrics_t::cv
```

Referenced by [ssm_l8_update\(\)](#), [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

6.80.1.2 entropy

```
double ssm_l8_metrics_t::entropy
```

Referenced by [ssm_l8_update\(\)](#), [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

6.80.1.3 stability_score

```
double ssm_l8_metrics_t::stability_score
```

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

6.80.1.4 temporal_decay

```
double ssm_l8_metrics_t::temporal_decay
```

Referenced by [ssm_l8_update\(\)](#), [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

The documentation for this struct was generated from the following file:

- [include/ssm_jacquard.h](#)

6.81 ssm_l8_state_t Struct Reference

```
#include <ssm_jacquard.h>
```

Data Fields

- [ssm_l8_mode_t](#) [current_mode](#)
- [uint32_t](#) [hysteresis_counter](#)
- [ssm_l8_mode_t](#) [pending_mode](#)

6.81.1 Field Documentation

6.81.1.1 current_mode

`ssm_l8_mode_t ssm_l8_state_t::current_mode`

Referenced by `ssm_apply_mode()`, `ssm_l8_init()`, `ssm_l8_update()`, `vm_heartbeat_update_l8()`, and `vm_heartbeat_update_l8()`.

6.81.1.2 hysteresis_counter

`uint32_t ssm_l8_state_t::hysteresis_counter`

Referenced by `ssm_l8_init()`, and `ssm_l8_update()`.

6.81.1.3 pending_mode

`ssm_l8_mode_t ssm_l8_state_t::pending_mode`

Referenced by `ssm_l8_init()`, and `ssm_l8_update()`.

The documentation for this struct was generated from the following file:

- `include/ssm_jacquard.h`

6.82 TestCase Struct Reference

Structure representing a single test case.

```
#include <test_common.h>
```

Collaboration diagram for TestCase:

Data Fields

- `const char * name`
Name of the test case.
- `const char * input`
Input Forth code.
- `const char * expected`
Expected output.
- `TestType type`
Type of test case.
- `int should_error`
Whether test should produce an error.
- `int implemented`
Whether the functionality is implemented.
- `WordContract contract`
Per-test contract override; zero = inherit from suite/module.

6.82.1 Detailed Description

Structure representing a single test case.

6.82.2 Field Documentation

6.82.2.1 contract

`WordContract TestCase::contract`

Per-test contract override; zero = inherit from suite/module.

Referenced by `run_test_suite_m()`.

6.82.2.2 expected

`const char* TestCase::expected`

Expected output.

Referenced by [run_single_test\(\)](#).

6.82.2.3 implemented

`int TestCase::implemented`

Whether the functionality is implemented.

Referenced by [run_single_test\(\)](#), [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

6.82.2.4 input

`const char* TestCase::input`

Input Forth code.

Referenced by [run_single_test\(\)](#), and [run_test_suite_m\(\)](#).

6.82.2.5 name

`const char* TestCase::name`

Name of the test case.

Referenced by [run_single_test\(\)](#), [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

6.82.2.6 should_error

`int TestCase::should_error`

Whether test should produce an error.

Referenced by [run_single_test\(\)](#), and [run_test_suite_m\(\)](#).

6.82.2.7 type

`TestType TestCase::type`

Type of test case.

Referenced by [run_test_suite_m\(\)](#).

The documentation for this struct was generated from the following file:

- `src/test_runner/include/test_common.h`

6.83 TestModule Struct Reference

Structure representing a test module.

`#include <test_runner.h>`

Data Fields

- `const char * module_name`
Name of the test module.
- `void * reserved`
Reserved for future use.
- `int reserved_int`
Reserved integer value.
- `void(* run_module_tests)(VM *vm)`
Function pointer to run module tests.

6.83.1 Detailed Description

Structure representing a test module.

6.83.2 Field Documentation

6.83.2.1 module_name

```
const char* TestModule::module_name
```

Name of the test module.

Referenced by [run_all_tests\(\)](#), [run_module_tests\(\)](#), and [run_word_tests\(\)](#).

6.83.2.2 reserved

```
void* TestModule::reserved
```

Reserved for future use.

6.83.2.3 reserved_int

```
int TestModule::reserved_int
```

Reserved integer value.

6.83.2.4 run_module_tests

```
void(* TestModule::run_module_tests) (VM *vm)
```

Function pointer to run module tests.

Referenced by [run_module_tests\(\)](#).

The documentation for this struct was generated from the following file:

- [src/test_runner/include/test_runner.h](#)

6.84 TestStats Struct Reference

Structure holding test execution statistics.

```
#include <test_runner.h>
```

Data Fields

- int [total_tests](#)
Total number of tests executed.
- int [total_pass](#)
Number of passed tests.
- int [total_fail](#)
Number of failed tests.
- int [total_skip](#)
Number of skipped tests.
- int [total_error](#)
Number of tests that resulted in errors.

6.84.1 Detailed Description

Structure holding test execution statistics.

6.84.2 Field Documentation

6.84.2.1 total_error

```
int TestStats::total_error
```

Number of tests that resulted in errors.

Referenced by [run_all_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

6.84.2.2 total_fail

```
int TestStats::total_fail
```

Number of failed tests.

Referenced by [run_all_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

6.84.2.3 total_pass

```
int TestStats::total_pass
```

Number of passed tests.

Referenced by [run_all_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

6.84.2.4 total_skip

```
int TestStats::total_skip
```

Number of skipped tests.

Referenced by [run_all_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

6.84.2.5 total_tests

```
int TestStats::total_tests
```

Total number of tests executed.

Referenced by [run_all_tests\(\)](#), [run_break_me_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

The documentation for this struct was generated from the following file:

- [src/test_runner/include/test_runner.h](#)

6.85 time_trust_state Struct Reference

```
#include <timer.h>
```

Collaboration diagram for `time_trust_state`:

Data Fields

- `uint64_t` [ticks](#)
- `uint64_t` [last_tsc](#)
- `uint64_t` [expected_delta](#)
- [TimeWindow](#) [window](#)
- `q48_16_t` [variance](#)
- `q48_16_t` [trust](#)
- `uint64_t` [total_samples](#)

6.85.1 Field Documentation

6.85.1.1 expected_delta

```
uint64_t time_trust_state::expected_delta
```

Referenced by [heartbeat_init\(\)](#), and [heartbeat_tick\(\)](#).

6.85.1.2 last_tsc

```
uint64_t time_trust_state::last_tsc
```

Referenced by [heartbeat_init\(\)](#), and [heartbeat_tick\(\)](#).

6.85.1.3 ticks

```
uint64_t time_trust_state::ticks
```

Referenced by [heartbeat_init\(\)](#), [heartbeat_tick\(\)](#), and [heartbeat_ticks\(\)](#).

6.85.1.4 total_samples

`uint64_t time_trust_state::total_samples`

Referenced by [heartbeat_init\(\)](#), and [heartbeat_tick\(\)](#).

6.85.1.5 trust

`q48_16_t time_trust_state::trust`

Referenced by [heartbeat_init\(\)](#), [heartbeat_tick\(\)](#), and [heartbeat_trust\(\)](#).

6.85.1.6 variance

`q48_16_t time_trust_state::variance`

Referenced by [heartbeat_init\(\)](#), and [heartbeat_tick\(\)](#).

6.85.1.7 window

`TimeWindow time_trust_state::window`

Referenced by [heartbeat_init\(\)](#), and [heartbeat_tick\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/timer.h](#)

6.86 time_window Struct Reference

```
#include <timer.h>
```

Data Fields

- `int64_t` [deltas](#) [64]
- `uint32_t` [pos](#)
- `uint32_t` [count](#)

6.86.1 Field Documentation

6.86.1.1 count

`uint32_t time_window::count`

Referenced by [heartbeat_init\(\)](#), [window_push\(\)](#), and [window_variance_q48\(\)](#).

6.86.1.2 deltas

`int64_t time_window::deltas[64]`

Referenced by [heartbeat_init\(\)](#), [window_push\(\)](#), and [window_variance_q48\(\)](#).

6.86.1.3 pos

`uint32_t time_window::pos`

Referenced by [heartbeat_init\(\)](#), and [window_push\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/timer.h](#)

6.87 timer_calibration_record Struct Reference

```
#include <timer.h>
```

Data Fields

- uint64_t [hpet_hz](#)
- uint64_t [tsc_hz_mean](#)
- uint64_t [pit_hz_mean](#)
- uint64_t [cv_hpet_ppm](#)
- uint64_t [cv_pit_ppm](#)
- uint64_t [diff_ppm](#)
- uint32_t [windows_used](#)
- uint8_t [converged](#)
- uint8_t [vm_mode](#)
- uint8_t [trust](#)
- uint8_t [reserved](#) [1]

6.87.1 Field Documentation

6.87.1.1 converged

uint8_t timer_calibration_record::converged

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), and [timer_init\(\)](#).

6.87.1.2 cv_hpet_ppm

uint64_t timer_calibration_record::cv_hpet_ppm

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.3 cv_pit_ppm

uint64_t timer_calibration_record::cv_pit_ppm

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.4 diff_ppm

uint64_t timer_calibration_record::diff_ppm

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.5 hpet_hz

uint64_t timer_calibration_record::hpet_hz

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.6 pit_hz_mean

uint64_t timer_calibration_record::pit_hz_mean

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.7 reserved

uint8_t timer_calibration_record::reserved[1]

6.87.1.8 trust

uint8_t timer_calibration_record::trust

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), [timer_init\(\)](#), and [timer_now_ns\(\)](#).

6.87.1.9 tsc_hz_mean

uint64_t timer_calibration_record::tsc_hz_mean

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

6.87.1.10 vm_mode

uint8_t timer_calibration_record::vm_mode

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), [timer_check_drift_now\(\)](#), [timer_init\(\)](#), and [timer_now_ns\(\)](#).

6.87.1.11 windows_used

uint32_t timer_calibration_record::windows_used

Referenced by [converge_tsc_bare_metal\(\)](#), and [init_timer_vm_mode\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/timer.h](#)

6.88 VM Struct Reference

Main virtual machine state container.

```
#include <vm.h>
```

Collaboration diagram for VM:

Data Fields

Stack Management

- [cell_t data_stack](#) [1024]
Parameter stack storage.
- [cell_t return_stack](#) [1024]
Return stack storage.
- int [dsp](#)
Data stack pointer.
- int [rsp](#)
Return stack pointer.
- int [exit_colon](#)
Exit flag for colon definitions.
- int [abort_requested](#)
ABORT flag for immediate termination.
- int [ecw_nesting](#)
Depth of execute_colon_word call stack; >0 means inside a colon word.
- volatile int [doe_row_printing](#)
Non-zero while EMIT-ROW assembles a row; heartbeat output must defer.
- uint8_t [emergency_console](#)
1 = fault handler active; bypasses all ACL checks (C-only write)
- uint8_t [zuse_session](#)
1 = zuse authenticated at console; shows zuse)ok> prompt

Dictionary Management

- const [VMHostServices](#) * [host](#)
Host allocator/time services.
- uint8_t * [memory](#)
Unified VM memory buffer.
- uintptr_t [here](#)
Next free memory location (byte offset or absolute pointer if M7)
- DictEntry * [latest](#)
Most recent word.
- DictEntry * [word_id_map](#) [1024]
Stable ID → entry map for speculation.
- uint32_t [recycled_word_ids](#) [1024]
Reusable ID stack for FORGET.
- uint32_t [recycled_word_id_count](#)

- *Depth of recycled stack.*
- `uint32_t next_word_id`
- *Next fresh ID when recycle stack empty.*
- `DictEntry * dict_fence_latest`
- `size_t dict_fence_here`
- `sf_mutex_t dict_lock`
- *Protects dictionary structural mutations.*
- `char input_buffer [256]`
- `size_t input_length`
- `size_t input_pos`
- `vm_mode_t mode`
- `DictEntry * compiling_word`
- `int interpreter_enabled`
- `char current_word_name [31+1]`
- `cell_t * compile_buffer`
- `size_t compile_pos`
- `size_t compile_size`
- `cell_t state_var`
- `DictEntry * current_executing_entry`
- `cell_t * ip`
- `int error`
- `int halted`
- `cell_t base`
- `unsigned char * tib_buf`
- `size_t tib_cap`
- `cell_t * in_var`
- `cell_t * span_var`
- `vaddr_t scr_addr`
- `vaddr_t state_addr`
- `vaddr_t base_addr`

Physics Hot-Words Cache

- `HotwordsCache * hotwords_cache`
- *Frequency-driven dictionary acceleration cache.*

Rolling Window of Truth (Deterministic Metrics)

- `RollingWindowOfTruth rolling_window`
- *Execution history for reproducible seeding.*

Phase 2: Heat-Aware Dictionary Optimization

- `cell_t heat_threshold_25th`
- *25th percentile heat (bucket search priority)*
- `cell_t heat_threshold_50th`
- *50th percentile heat*
- `cell_t heat_threshold_75th`
- *75th percentile heat*
- `uint64_t last_bucket_reorg_ns`
- *When we last reorganized buckets.*
- `int lookup_strategy`
- *0=naive, 1=heat-aware, 2=inference-reorg*

VM Heartbeat (Loop #3 & #5 Coordination)

- `HeartbeatState heartbeat`
- *Centralized time-driven tuning dispatcher.*
- `uint32_t heartbeat_decay_cursor_id`
- *Continuation cursor for background decay sweeps.*

Pipelining Global Metrics (Loop #4 & #5 Feedback)

- [PipelineGlobalMetrics pipeline_metrics](#)

Aggregated prefetch accuracy tracking.

Physics Loop #3: Adaptive Heat Decay Tuning

- [uint64_t decay_slope_q48](#)
Current decay rate (Q48.16 fixed-point, starts at 2:1 = 131072)
- [uint64_t last_decay_check_ns](#)
Timestamp of last decay effectiveness measurement.
- [uint64_t total_heat_at_last_check](#)
Total dictionary heat at last check (for slope inference)
- [uint64_t hot_word_count_at_check](#)
Hot words observed at last inference.
- [uint64_t stale_word_count_at_check](#)
Stale words at last check (for trend analysis)
- [uint32_t word_count_at_check](#)
Total word count at last check (for ratio calculation)
- [int decay_slope_direction](#)
-1 = decrease slope, 0 = stable, +1 = increase slope (inference)
- [sf_mutex_t tuning_lock](#)
Guards shared tuning knobs (slope, snapshots)

Unified Inference Engine (Phase 2: Replacing Loops #3 & #5)

- [InferenceOutputs * last_inference_outputs](#)
Cached inference outputs (for ANOVA early-exit and tuning application)

SSM L8: Jacquard Mode Selector (Steady State Machine)

- [void * ssm_l8_state](#)
Opaque pointer to [ssm_l8_state_t](#) (avoid circular include)
- [void * ssm_config](#)
Opaque pointer to [ssm_config_t](#) (L3/L4 mode bits)

6.88.1 Detailed Description

Main virtual machine state container.

Contains all state information for a StarForth [VM](#) instance including stacks, dictionary, memory, and execution state.

6.88.2 Field Documentation

6.88.2.1 abort_requested

`int VM::abort_requested`

ABORT flag for immediate termination.

Referenced by [check_physics_transparent\(\)](#), [execute_colon_word\(\)](#), [restore_vm_state\(\)](#), [system_word_abort\(\)](#), [test_acl_deny_aborts\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.2 base

`cell_t VM::base`

Referenced by [current_base\(\)](#), [dbg_get_base\(\)](#), [vm_get_base\(\)](#), and [vm_set_base\(\)](#).

6.88.2.3 base_addr

`vaddr_t VM::base_addr`

Referenced by [dbg_get_base\(\)](#), [format_word_base\(\)](#), [format_word_decimal\(\)](#), [format_word_hex\(\)](#), [format_word_octal\(\)](#), [vm_get_base\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_set_base\(\)](#).

6.88.2.4 compile_buffer

`cell_t*` VM::compile_buffer

6.88.2.5 compile_pos

`size_t` VM::compile_pos

6.88.2.6 compile_size

`size_t` VM::compile_size

6.88.2.7 compiling_word

`DictEntry*` VM::compiling_word

Referenced by [restore_vm_state\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.8 current_executing_entry

`DictEntry*` VM::current_executing_entry

Referenced by [acl_recheck\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_variable\(\)](#), [execute_colon_word\(\)](#), [restore_vm_state\(\)](#), [starforth_word_init\(\)](#), [vm_debug_dump_state\(\)](#), [vm_docol_arm64\(\)](#), [vm_docol_asm\(\)](#), [vm_docol_rv64\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_interpret_word\(\)](#).

6.88.2.9 current_word_name

`char` VM::current_word_name[31+1]

Referenced by [vm_enter_compile_mode\(\)](#).

6.88.2.10 data_stack

`cell_t` VM::data_stack[1024]

Parameter stack storage.

Referenced by [assert_stack_top\(\)](#), [capture_exec_result\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_dabs\(\)](#), [format_word_dot_s\(\)](#), [format_word_hash_s\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), [forth_DECAY_RATE_FETCH\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_LOOKUP_STRATEGY_FETCH\(\)](#), [forth_LOOKUP_STRATEGY_STORE\(\)](#), [forth_PHYSICS_BURN\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [io_word_emit\(\)](#), [io_word_key\(\)](#), [io_word_question_terminal\(\)](#), [io_word_spaces\(\)](#), [stack_word_depth\(\)](#), [stack_word_dup\(\)](#), [stack_word_minus_rot\(\)](#), [stack_word_over\(\)](#), [stack_word_pick\(\)](#), [stack_word_question_dup\(\)](#), [stack_word_roll\(\)](#), [stack_word_rot\(\)](#), [stack_word_swap\(\)](#), [string_word_tick\(\)](#), [vm_pop\(\)](#), [vm_pop_asm\(\)](#), [vm_pop_asm\(\)](#), [vm_pop_asm\(\)](#), [vm_pop_fast\(\)](#), [vm_push\(\)](#), [vm_push_asm\(\)](#), [vm_push_asm\(\)](#), [vm_push_asm\(\)](#), [vm_push_fast\(\)](#), [vm_save_registers\(\)](#), [vm_save_registers_arm64\(\)](#), [vm_save_registers_rv64\(\)](#), [vm_setup_registers\(\)](#), [vm_setup_registers_arm64\(\)](#), [vm_setup_registers_rv64\(\)](#), and [vocabulary_word_paren_find\(\)](#).

6.88.2.11 decay_slope_direction

`int` VM::decay_slope_direction

-1 = decrease slope, 0 = stable, +1 = increase slope (inference)

Referenced by [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.12 decay_slope_q48

`uint64_t` VM::decay_slope_q48

Current decay rate (Q48.16 fixed-point, starts at 2:1 = 131072)

Referenced by [heartbeat_publish_snapshot\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [physics_decay_slope_load\(\)](#), [restore_physics\(\)](#), [save_physics\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.13 dict_fence_here

size_t VM::dict_fence_here

Referenced by [dictionary_word_forget\(\)](#), [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.14 dict_fence_latest

DictEntry* VM::dict_fence_latest

Referenced by [dictionary_word_forget\(\)](#), [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.15 dict_lock

sf_mutex_t VM::dict_lock

Protects dictionary structural mutations.

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_word_forget\(\)](#), [extract_heat_trajectory\(\)](#), [is_valid_dict_entry\(\)](#), [metrics_from_vm\(\)](#), [physics_on_lookup\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_cleanup\(\)](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.16 doe_row_printing

volatile int VM::doe_row_printing

Non-zero while EMIT-ROW assembles a row; heartbeat output must defer.

6.88.2.17 dsp

int VM::dsp

Data stack pointer.

Referenced by [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [assert_stack_depth\(\)](#), [assert_stack_top\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_thru\(\)](#), [capture_exec_result\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [defining_word_constant\(\)](#), [defining_word_literal\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_sp_fetch\(\)](#), [dictionary_word_sp_store\(\)](#), [double_word_2drop\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dabs\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), [execute_colon_word\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dot_s\(\)](#), [format_word_dump\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_hold\(\)](#), [format_word_question\(\)](#), [format_word_sign\(\)](#), [format_word_u_dot\(\)](#), [format_word_u_dot_r\(\)](#), [forth_acl_allow_store\(\)](#), [forth_acl_inherit\(\)](#), [forth_acl_mode_store\(\)](#), [forth_acl_ttl_store\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), [forth_DECAY_RATE_FETCH\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_LOOKUP_STRATEGY\(\)](#), [forth_LOOKUP_STRATEGY_STORE\(\)](#), [forth_PHYSICS_BURN\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [io_word_emit\(\)](#), [io_word_key\(\)](#), [io_word_question_terminal\(\)](#), [io_word_spaces\(\)](#), [io_word_type\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), and [mixed_math_word_m_minus\(\)](#).

mixed_math_word_m_plus(), mixed_math_word_m_slash_mod(), mixed_math_word_m_star(), mixed_math_word_mod(), mixed_math_word_mod(), mixed_math_word_slash_mod(), mixed_math_word_star_slash(), mixed_math_word_star_slash_mod(), pop_xt(), reset_vm_state(), restore_vm_state(), return_stack_word_r_fetch(), return_stack_word_r_from(), return_stack_word_to_r(), runtime_abortq(), save_vm_state(), stack_word_depth(), stack_word_drop(), stack_word_dup(), stack_word_minus_rot(), stack_word_over(), stack_word_pick(), stack_word_question_dup(), stack_word_roll(), stack_word_rot(), stack_word_swap(), starforth_word_execution_heat_fetch(), starforth_word_execution_heat_store(), starforth_word_random(), starforth_word_seed(), starforth_word_top_words(), starforth_word_wait(), string_word_blank(), string_word_cmove(), string_word_cmove_greater(), string_word_compare(), string_word_convert(), string_word_count(), string_word_enclose(), string_word_expect(), string_word_minus_trailing(), string_word_number(), string_word_scan(), string_word_search(), string_word_skip(), string_word_tick(), string_word_word(), system_word_abort_quote(), test_acl_adaptive_ttl(), test_acl_deny_aborts(), test_acl_strict_mode(), test_emergency_console_bypass(), vm_debug_dump_state(), vm_init(), vm_init_with_host(), vm_interpret_word(), vm_pop(), vm_pop_asm(), vm_pop_asm(), vm_pop_asm(), vm_pop_fast(), vm_push(), vm_push_asm(), vm_push_asm(), vm_push_asm(), vm_push_fast(), vm_save_registers(), vm_save_registers_arm64(), vm_save_registers_rv64(), vm_setup_registers(), vm_setup_registers_arm64(), vm_setup_registers_rv64(), vocabulary_word_paren_find(), word_divide(), and word_divmod().

6.88.2.18 ecw_nesting

```
int VM::ecw_nesting
```

Depth of execute_colon_word call stack; >0 means inside a colon word.

6.88.2.19 emergency_console

```
uint8_t VM::emergency_console
```

1 = fault handler active; bypasses all ACL checks (C-only write)

Referenced by [acl_recheck\(\)](#), [execute_colon_word\(\)](#), [run_acl_words_tests\(\)](#), [sk_repl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), and [vm_interpret_word\(\)](#).

6.88.2.20 error

```
int VM::error
```

Referenced by [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [assert_vm_error\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), [capsule_exec_hook\(\)](#), [capsule_exec_init\(\)](#), [capsule_vm_alloc_hook\(\)](#), [capture_exec_result\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_branch\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_EXIT\(\)](#), [control_forth_I\(\)](#), [control_forth_if\(\)](#), [control_forth_J\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_then\(\)](#), [control_forth_UNLOOP\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_does\(\)](#), [defining_word_immediate\(\)](#), [defining_word_literal\(\)](#), [defining_word_semicolon\(\)](#), [defining_word_variable\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_hidden\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_smudge\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_forget\(\)](#), [dictionary_word_sp_store\(\)](#), [double_word_2drop\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dabs\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#), [editor_word_I\(\)](#), [editor_word_s\(\)](#), [editor_word_show\(\)](#), [emit_cell\(\)](#), [ensure_input\(\)](#), [ensure_pn\(\)](#), [execute_colon_word\(\)](#), [format_word_begin_conversion\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dump\(\)](#),

format_word_end_conversion(), format_word_hash(), format_word_hash_s(), format_word_hold(), format_word_question(), format_word_sign(), format_word_u_dot(), format_word_u_dot_r(), forth_acl_allow_store(), forth_acl_inherit(), forth_acl_mode_store(), forth_acl_ttl_store(), forth_ALL_HEATS(), forth_BENCH_DICT_LOOKUP(), forth_COMPARE_LOOKUPS(), forth_DECAY_RATE_FETCH(), forth_FREEZE_CRITICAL(), forth_FREEZE_WORD(), forth_FROZEN_QUERY(), forth_HEAT_FETCH(), forth_HEAT_PERCENTILES(), forth_HEAT_STORE(), forth_LOOKUP_STRATEGY_FETCH(), forth_LOOKUP_STRATEGY_STORE(), forth_PHYSICS_BAYESIAN_REPORT(), forth_PHYSICS_BUILD_INFO(), forth_PHYSICS_BURN(), forth_PHYSICS_CACHE_STATS(), forth_PHYSICS_CALC_KNOBS(), forth_PHYSICS_RESET_STATS(), forth_PHYSICS_SHOW_FEEDBACK(), forth_PHYSICS_TOGGLE_CACHE(), forth_PHYSICS_WORD_METRICS(), forth_PIPELINING_ANALYZE_WORD(), forth_PIPELINING_SHOW_STATS(), forth_PIPELINING_SHOW_TOP_TRANSITIONS(), forth_REORG_BUCKETS(), forth_SHOW_HEAT(), forth_SHOW_HEAT_OPTIMIZATION(), forth_UNFREEZE_WORD(), init_vm_and_subsystem(), init_vocabulary_system(), io_word_emit(), io_word_key(), io_word_question_terminal(), io_word_spaces(), io_word_type(), logical_word_and(), logical_word_equals(), logical_word_greater_than(), logical_word_less_than(), logical_word_not(), logical_word_not_equals(), logical_word_or(), logical_word_u_greater_than(), logical_word_u_less_than(), logical_word_within(), logical_word_xor(), logical_word_zero_equals(), logical_word_zero_greater(), logical_word_zero_less(), logical_word_zero_not_equal(), main(), memory_word_2fetch(), memory_word_2store(), memory_word_cells(), memory_word_cfetch(), memory_word_cstore(), memory_word_erase(), memory_word_fetch(), memory_word_fill(), memory_word_minus_store(), memory_word_move(), memory_word_plus_store(), memory_word_store(), mixed_math_word_m_minus(), mixed_math_word_m_plus(), mixed_math_word_m_slash_mod(), mixed_math_word_m_star(), mixed_math_word_mod(), mixed_math_word_mod(), mixed_math_word_slash_mod(), mixed_math_word_star_slash(), mixed_math_word_star_slash_mod(), pop_xt(), reset_vm_state(), restore_vm_state(), return_stack_word_r_fetch(), return_stack_word_r_from(), return_stack_word_to_r(), run_single_test(), runtime_abortq(), save_vm_state(), sk_repl(), stack_word_depth(), stack_word_drop(), stack_word_dup(), stack_word_minus_rot(), stack_word_over(), stack_word_pick(), stack_word_question_dup(), stack_word_roll(), stack_word_rot(), stack_word_swap(), starforth_word_execution_heat_fetch(), starforth_word_execution_heat_store(), starforth_word_init(), starforth_word_random(), starforth_word_seed(), starforth_word_top_words(), starforth_word_wait(), string_runtime_s_quote(), string_word_blank(), string_word_cmove(), string_word_cmove_greater(), string_word_compare(), string_word_convert(), string_word_count(), string_word_enclose(), string_word_expect(), string_word_minus_trailing(), string_word_number(), string_word_query(), string_word_s_quote(), string_word_scan(), string_word_search(), string_word_skip(), string_word_source(), string_word_span(), string_word_tib(), string_word_tick(), string_word_to_in(), string_word_word(), system_word_abort(), system_word_abort_quote(), system_word_quit(), system_word_save_system(), system_word_see(), test_acl_adaptive_ttl(), test_acl_deny_aborts(), test_acl_pin_blocks_shadow(), test_acl_pin_immutable(), test_acl_strict_mode(), test_acl_ttl_hot_path(), test_emergency_console_bypass(), vm_align(), vm_allot(), vm_assert_interpreter_enabled(), vm_bootstrap_root_vocabulary(), vm_bootstrap_scr(), vm_compile_call(), vm_compile_exit(), vm_compile_literal(), vm_compile_word(), vm_create_word(), vm_debug_dump_state(), vm_enter_compile_mode(), vm_exit_compile_mode(), vm_init(), vm_init_with_host(), vm_input_ensure(), vm_interpret(), vm_interpret_word(), vm_load_cell(), vm_load_u8(), vm_make_immediate(), vm_pin_execution_heat(), vm_pop(), vm_pop_asm(), vm_pop_asm(), vm_pop_asm(), vm_push(), vm_push_asm(), vm_push_asm(), vm_push_asm(), vm_repl(), vm_rpop(), vm_rpop_asm(), vm_rpop_asm(), vm_rpop_asm(), vm_rpush(), vm_rpush_asm(), vm_rpush_asm(), vm_rpush_asm(), vm_smudge_word(), vm_store_cell(), vm_store_u8(), vm_unpin_execution_heat(), vocabulary_create_vocabulary_direct(), vocabulary_word_paren_find(), vocabulary_word_vocabulary(), word_divide(), and word_divmod().

6.88.2.21 exit_colon

```
int VM::exit_colon
```

Exit flag for colon definitions.

Referenced by [check_physics_transparent\(\)](#), [control_forth_EXIT\(\)](#), [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [restore_vm_state\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.22 halted

```
int VM::halted
```

Referenced by [__attribute__\(\)](#), [sk_fault_handler\(\)](#), [sk_repl\(\)](#), [starforth_word_init\(\)](#), [system_word_bye\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vm_repl\(\)](#).

6.88.2.23 heartbeat

```
HeartbeatState VM::heartbeat
```

Centralized time-driven tuning dispatcher.

Referenced by [execute_colon_word\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_publish_snapshot\(\)](#), [metrics_from_vm\(\)](#), [metrics_write_james_law_csv_row\(\)](#), [perturb_physics\(\)](#), [physics_post_execute\(\)](#), [restore_physics\(\)](#), [save_physics\(\)](#), [vm_cleanup\(\)](#), [vm_find_word\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_run_cycle\(\)](#), [vm_heartbeat_start_thread\(\)](#), [vm_heartbeat_stop_thread\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_interpret\(\)](#), [vm_interpret_word\(\)](#), [vm_snapshot_read\(\)](#), [vm_tick\(\)](#), and [vm_tick_inference_engine\(\)](#).

6.88.2.24 heartbeat_decay_cursor_id

`uint32_t VM::heartbeat_decay_cursor_id`

Continuation cursor for background decay sweeps.

Referenced by [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

6.88.2.25 heat_threshold_25th

`cell_t VM::heat_threshold_25th`

25th percentile heat (bucket search priority)

Referenced by [dict_find_word_heat_aware\(\)](#), [dict_update_heat_percentiles\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), and [save_physics\(\)](#).

6.88.2.26 heat_threshold_50th

`cell_t VM::heat_threshold_50th`

50th percentile heat

Referenced by [dict_update_heat_percentiles\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), and [save_physics\(\)](#).

6.88.2.27 heat_threshold_75th

`cell_t VM::heat_threshold_75th`

75th percentile heat

Referenced by [dict_find_word_heat_aware\(\)](#), [dict_update_heat_percentiles\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), and [save_physics\(\)](#).

6.88.2.28 here

`uintptr_t VM::here`

Next free memory location (byte offset or absolute pointer if M7)

Referenced by [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_then\(\)](#), [control_forth_until\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_forget\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [main\(\)](#), [reset_vm_state\(\)](#), [restore_dict_state\(\)](#), [save_dict_state\(\)](#), [sk_parity_collect\(\)](#), [string_word_s_quote\(\)](#), [system_word_save_system\(\)](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_cleanup\(\)](#), [vm_debug_dump_state\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

6.88.2.29 host

`const VMHostServices* VM::host`

Host allocator/time services.

Referenced by [vm_host\(\)](#), [vm_init_with_host\(\)](#), and [vm_reset_hosted_fake_ns\(\)](#).

6.88.2.30 hot_word_count_at_check

`uint64_t VM::hot_word_count_at_check`

Hot words observed at last inference.

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.31 hotwords_cache

`HotwordsCache*` VM::hotwords_cache

Frequency-driven dictionary acceleration cache.

Referenced by `execute_colon_word()`, `forth_BENCH_DICT_LOOKUP()`, `forth_PHYSICS_BAYESIAN_REPORT()`, `forth_PHYSICS_BUILD_INFO()`, `forth_PHYSICS_CACHE_STATS()`, `forth_PHYSICS_RESET_STATS()`, `forth_PHYSICS_TOGGLE_C`, `metrics_from_vm()`, `physics_pre_execute()`, `rolling_window_seed_hotwords_cache()`, `vm_cleanup()`, `vm_find_word()`, `vm_init()`, and `vm_init_with_host()`.

6.88.2.32 in_var

`cell_t*` VM::in_var

Referenced by `vm_input_ensure()`, `vm_input_in()`, and `vm_input_load_line()`.

6.88.2.33 input_buffer

`char` VM::input_buffer[256]

Referenced by `starforth_word_paren_dash()`, `string_word_s_quote()`, `system_word_abort_quote()`, `vm_interpret()`, and `vm_parse_word()`.

6.88.2.34 input_length

`size_t` VM::input_length

Referenced by `starforth_word_paren_dash()`, `string_word_s_quote()`, `system_word_abort_quote()`, `vm_init()`, `vm_init_with_host()`, `vm_interpret()`, and `vm_parse_word()`.

6.88.2.35 input_pos

`size_t` VM::input_pos

Referenced by `starforth_word_paren_dash()`, `string_word_s_quote()`, `system_word_abort_quote()`, `vm_init()`, `vm_init_with_host()`, `vm_interpret()`, and `vm_parse_word()`.

6.88.2.36 interpreter_enabled

`int` VM::interpreter_enabled

Referenced by `vm_assert_interpreter_enabled()`, `vm_enable_interpreter()`, and `vm_init_with_host()`.

6.88.2.37 ip

`cell_t*` VM::ip

Referenced by `vm_debug_dump_state()`, `vm_save_registers()`, `vm_save_registers_arm64()`, `vm_save_registers_rv64()`, `vm_setup_registers()`, `vm_setup_registers_arm64()`, and `vm_setup_registers_rv64()`.

6.88.2.38 last_bucket_reorg_ns

`uint64_t` VM::last_bucket_reorg_ns

When we last reorganized buckets.

Referenced by `dict_adaptive_optimization_pass()`, `dict_reorganize_buckets_by_heat()`, `vm_init()`, and `vm_init_with_host()`.

6.88.2.39 last_decay_check_ns

`uint64_t` VM::last_decay_check_ns

Timestamp of last decay effectiveness measurement.

Referenced by `vm_init()`, and `vm_init_with_host()`.

6.88.2.40 last_inference_outputs

`InferenceOutputs*` VM::last_inference_outputs

Cached inference outputs (for ANOVA early-exit and tuning application)

Referenced by `metrics_from_vm()`, `vm_heartbeat_update_l8()`, `vm_heartbeat_update_l8()`, and `vm_tick_inference_engine()`.

6.88.2.41 latest

`DictEntry* VM::latest`

Most recent word.

Referenced by `capsule_dict_hash_hook()`, `defining_runtime_does_rt()`, `defining_word_immediate()`, `dict_update_heat_percentiles()`, `dictionary_m_word_hidden()`, `dictionary_word_forget()`, `find_entry_from_name()`, `forth_acl_init_primitives()`, `forth_ALL_HEATS()`, `forth_PHYSICS_BURN()`, `forth_PHYSICS_CALC_KNOBS()`, `forth_PHYSICS_SHOW_FEEDBACK()`, `forth_PHYSICS_WORD_METRICS()`, `forth_PIPELINING_RESET_ALL()`, `forth_PIPELINING_STATS()`, `heartbeat_capture_tick_snaps`, `init_vocabulary_system()`, `is_valid_dict_entry()`, `main()`, `metrics_from_vm()`, `restore_dict_state()`, `save_dict_state()`, `sf_rebuild_fc_index()`, `sf_try_fast_append()`, `sk_dict_canonical_hash()`, `sk_dict_word_count()`, `sk_parity_collect()`, `starforth_word_reset_execution_heat()`, `starforth_word_top_words()`, `starforth_word_word_execution_heat()`, `string_word_tick()`, `system_word_vlist()`, `system_word_words()`, `vm_create_word()`, `vm_dictionary_find_by_func()`, `vm_find_word()`, `vm_hide_word()`, `vm_init()`, `vm_init_with_host()`, `vm_make_immediate()`, `vm_pin_execution_heat()`, `vm_smudge_word()`, `vm_tick_apply_background_decay()`, `vm_tick_apply_background_decay()`, `vm_tick_inference_engine()`, `vm_tick_slope_validator()`, `vm_unpin_execution_heat()`, `vocabulary_create_vocabulary_direct()`, `vocabulary_select_runtime()`, and `vocabulary_word_vocabulary()`.

6.88.2.42 lookup_strategy

`int VM::lookup_strategy`

0=naive, 1=heat-aware, 2=inference-reorg

Referenced by `dict_adaptive_optimization_pass()`, `forth_COMPARE_LOOKUPS()`, `forth_LOOKUP_STRATEGY_FETCH()`, `forth_LOOKUP_STRATEGY_STORE()`, `forth_SHOW_HEAT_OPTIMIZATION()`, `vm_find_word()`, `vm_init()`, and `vm_init_with_host()`.

6.88.2.43 memory

`uint8_t* VM::memory`

Unified VM memory buffer.

Referenced by `control_forth_else()`, `control_forth_endcase()`, `control_forth_endof()`, `control_forth_loop()`, `control_forth_plus_loop()`, `control_forth_repeat()`, `control_forth_then()`, `dbg_get_base()`, `defining_runtime_does_rt()`, `emit_cell()`, `format_word_end_conversion()`, `hash_colon_body()`, `init_vocabulary_system()`, `io_init()`, `io_word_type()`, `io_write_block()`, `runtime_abortq()`, `system_word_abort_quote()`, `system_word_save_system()`, `system_word_see()`, `vaddr_from_ptr()`, `vm_addr_from_ptr()`, `vm_addr_ok()`, `vm_addr_to_block()`, `vm_allot()`, `vm_bootstrap_scr()`, `vm_cleanup()`, `vm_debug_dump_state()`, `vm_get_block_addr()`, `vm_init()`, `vm_init_with_host()`, `vm_load_cell()`, `vm_ptr()`, `vm_store_cell()`, `vocab_sync_vm_vars()`, `vocabulary_create_vocabulary_direct()`, and `vocabulary_word_vocabulary()`.

6.88.2.44 mode

`vm_mode_t VM::mode`

Referenced by `cf_epoch_sync()`, `control_forth_again()`, `control_forth_begin()`, `control_forth_case()`, `control_forth_do()`, `control_forth_else()`, `control_forth_endcase()`, `control_forth_endof()`, `control_forth_if()`, `control_forth_leave()`, `control_forth_loop()`, `control_forth_of()`, `control_forth_plus_loop()`, `control_forth_qdo()`, `control_forth_repeat()`, `control_forth_then()`, `control_forth_until()`, `control_forth_while()`, `defining_word_colon()`, `defining_word_does()`, `defining_word_left_bracket()`, `defining_word_right_bracket()`, `defining_word_semicolon()`, `dictionary_m_word_hidden()`, `dictionary_m_word_interpret()`, `dictionary_m_word_left_bracket()`, `dictionary_m_word_right_bracket()`, `dictionary_m_word_smudge()`, `dictionary_m_word_tick()`, `emit_cell()`, `reset_vm_state()`, `restore_vm_state()`, `save_vm_state()`, `string_word_s_quote()`, `system_word_abort_quote()`, `system_word_quit()`, `vm_compile_call()`, `vm_compile_exit()`, `vm_compile_literal()`, `vm_compile_word()`, `vm_debug_dump_state()`, `vm_enter_compile_mode()`, `vm_exit_compile_mode()`, `vm_init()`, `vm_init_with_host()`, and `vm_interpret_word()`.

6.88.2.45 next_word_id

`uint32_t VM::next_word_id`

Next fresh ID when recycle stack empty.

Referenced by `vm_dictionary_acquire_word_id()`, `vm_init()`, and `vm_init_with_host()`.

6.88.2.46 pipeline_metrics

`PipelineGlobalMetrics VM::pipeline_metrics`

Aggregated prefetch accuracy tracking.

Referenced by [execute_colon_word\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [physics_pre_execute\(\)](#), [restore_physics\(\)](#), [save_physics\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.88.2.47 recycled_word_id_count

```
uint32_t VM::recycled_word_id_count
```

Depth of recycled stack.

Referenced by [vm_dictionary_acquire_word_id\(\)](#), [vm_dictionary_untrack_entry\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.48 recycled_word_ids

```
uint32_t VM::recycled_word_ids[1024]
```

Reusable ID stack for FORGET.

Referenced by [vm_dictionary_acquire_word_id\(\)](#), and [vm_dictionary_untrack_entry\(\)](#).

6.88.2.49 return_stack

```
cell_t VM::return_stack[1024]
```

Return stack storage.

Referenced by [control_forth_0branch\(\)](#), [control_forth_branch\(\)](#), [control_forth_l\(\)](#), [control_forth_j\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_UNLOOP\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2to_r\(\)](#), [return_stack_word_r_fetch\(\)](#), [string_runtime_s_quote\(\)](#), [vm_rpop\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_fast\(\)](#), [vm_rpush\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_fast\(\)](#), [vm_save_registers\(\)](#), [vm_save_registers_arm64\(\)](#), [vm_save_registers_rv64\(\)](#), [vm_setup_registers\(\)](#), [vm_setup_registers_arm64\(\)](#), and [vm_setup_registers_rv64\(\)](#).

6.88.2.50 rolling_window

```
RollingWindowOfTruth VM::rolling_window
```

Execution history for reproducible seeding.

Referenced by [dict_adaptive_optimization_pass\(\)](#), [execute_colon_word\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_publish_snapshot\(\)](#), [metrics_from_vm\(\)](#), [perturb_physics\(\)](#), [physics_pre_execute\(\)](#), [restore_physics\(\)](#), [run_doe_experiment\(\)](#), [save_physics\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_run_cycle\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_window_tuner\(\)](#).

6.88.2.51 rsp

```
int VM::rsp
```

Return stack pointer.

Referenced by [capture_exec_result\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_branch\(\)](#), [control_forth_EXIT\(\)](#), [control_forth_l\(\)](#), [control_forth_j\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_UNLOOP\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2to_r\(\)](#), [execute_colon_word\(\)](#), [reset_vm_state\(\)](#), [restore_vm_state\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [return_stack_word_to_r\(\)](#), [save_vm_state\(\)](#), [string_runtime_s_quote\(\)](#), [system_word_quit\(\)](#), [vm_debug_dump_state\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_interpret_word\(\)](#), [vm_rpop\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_asm\(\)](#), [vm_rpop_fast\(\)](#), [vm_rpush\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_asm\(\)](#), [vm_rpush_fast\(\)](#), [vm_save_registers\(\)](#), [vm_save_registers_arm64\(\)](#), [vm_save_registers_rv64\(\)](#), [vm_setup_registers\(\)](#), [vm_setup_registers_arm64\(\)](#), and [vm_setup_registers_rv64\(\)](#).

6.88.2.52 scr_addr

```
vaddr_t VM::scr_addr
```

Referenced by [block_word_next_block\(\)](#), [block_word_scr\(\)](#), [block_word_update\(\)](#), [current_scr\(\)](#), [editor_word_edit\(\)](#), [mark_buffer_dirty\(\)](#), [set_scr\(\)](#), [set_scr\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_debug_dump_state\(\)](#), and [vm_init\(\)](#).

6.88.2.53 span_var

`cell_t*` VM::span_var

Referenced by [vm_input_ensure\(\)](#), [vm_input_load_line\(\)](#), [vm_input_source\(\)](#), and [vm_input_span\(\)](#).

6.88.2.54 ssm_config

`void*` VM::ssm_config

Opaque pointer to [ssm_config_t](#) (L3/L4 mode bits)

Referenced by [inference_engine_run\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.55 ssm_l8_state

`void*` VM::ssm_l8_state

Opaque pointer to [ssm_l8_state_t](#) (avoid circular include)

Referenced by [vm_cleanup\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.56 stale_word_count_at_check

`uint64_t` VM::stale_word_count_at_check

Stale words at last check (for trend analysis)

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.57 state_addr

`vaddr_t` VM::state_addr

Referenced by [defining_word_left_bracket\(\)](#), [defining_word_right_bracket\(\)](#), [defining_word_state\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.58 state_var

`cell_t` VM::state_var

Referenced by [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

6.88.2.59 tib_buf

`unsigned char*` VM::tib_buf

Referenced by [vm_input_ensure\(\)](#), [vm_input_load_line\(\)](#), [vm_input_source\(\)](#), and [vm_input_tib\(\)](#).

6.88.2.60 tib_cap

`size_t` VM::tib_cap

Referenced by [vm_input_ensure\(\)](#), and [vm_input_load_line\(\)](#).

6.88.2.61 total_heat_at_last_check

`uint64_t` VM::total_heat_at_last_check

Total dictionary heat at last check (for slope inference)

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.62 tuning_lock

`sf_mutex_t` VM::tuning_lock

Guards shared tuning knobs (slope, snapshots)

Referenced by [metrics_from_vm\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.63 word_count_at_check

```
uint32_t VM::word_count_at_check
```

Total word count at last check (for ratio calculation)

Referenced by [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

6.88.2.64 word_id_map

```
DictEntry* VM::word_id_map[1024]
```

Stable ID → entry map for speculation.

Referenced by [vm_dictionary_lookup_by_word_id\(\)](#), [vm_dictionary_track_entry\(\)](#), and [vm_dictionary_untrack_entry\(\)](#).

6.88.2.65 zuse_session

```
uint8_t VM::zuse_session
```

1 = zuse authenticated at console; shows zuse)ok> prompt

Referenced by [execute_colon_word\(\)](#), [sk_repl\(\)](#), [starforth_word_zuse_authenticate\(\)](#), [vm_interpret_word\(\)](#), and [vm_repl\(\)](#).

The documentation for this struct was generated from the following file:

- [include/vm.h](#)

6.89 VMHostServices Struct Reference

[VMHostServices](#) - pluggable host operations.

```
#include <vm_host.h>
```

Data Fields

- [sk_alloc_fn](#) alloc
- [sk_free_fn](#) free
- [sk_time_fn](#) monotonic_ns
- [sk_mutex_init_fn](#) mutex_init
- [sk_mutex_lock_fn](#) mutex_lock
- [sk_mutex_unlock_fn](#) mutex_unlock
- [sk_mutex_destroy_fn](#) mutex_destroy
- [sk_puts_fn](#) puts
- [sk_putc_fn](#) putc
- [sk_xt_validate_fn](#) is_executable_ptr
- [sk_xt_entry_owns_fn](#) owns_xt_entry
- [sk_panic_fn](#) panic
- int [parity_mode](#)
- int [verbose](#)

6.89.1 Detailed Description

[VMHostServices](#) - pluggable host operations.

Hosted builds point these at libc functions. Kernel builds point these at kernel implementations.

6.89.2 Field Documentation**6.89.2.1 alloc**

```
sk_alloc_fn VMHostServices::alloc
```

Referenced by [vm_host_alloc\(\)](#).

6.89.2.2 free

`sk_free_fn` VMHostServices::free

Referenced by `vm_host_free()`.

6.89.2.3 is_executable_ptr

`sk_xt_validate_fn` VMHostServices::is_executable_ptr

Referenced by `vm_bootstrap_root_vocabulary()`.

6.89.2.4 monotonic_ns

`sk_time_fn` VMHostServices::monotonic_ns

Referenced by `vm_monotonic_ns()`.

6.89.2.5 mutex_destroy

`sk_mutex_destroy_fn` VMHostServices::mutex_destroy

6.89.2.6 mutex_init

`sk_mutex_init_fn` VMHostServices::mutex_init

6.89.2.7 mutex_lock

`sk_mutex_lock_fn` VMHostServices::mutex_lock

6.89.2.8 mutex_unlock

`sk_mutex_unlock_fn` VMHostServices::mutex_unlock

6.89.2.9 owns_xt_entry

`sk_xt_entry_owns_fn` VMHostServices::owns_xt_entry

6.89.2.10 panic

`sk_panic_fn` VMHostServices::panic

Referenced by `vm_bootstrap_root_vocabulary()`.

6.89.2.11 parity_mode

`int` VMHostServices::parity_mode

6.89.2.12 putc

`sk_putc_fn` VMHostServices::putc

6.89.2.13 puts

`sk_puts_fn` VMHostServices::puts

6.89.2.14 verbose

`int` VMHostServices::verbose

The documentation for this struct was generated from the following file:

- `include/vm_host.h`

6.90 vmm_page_info_t Struct Reference

```
#include <vmm.h>
```

Data Fields

- int [present](#)
- int [writable](#)
- int [executable](#)

6.90.1 Field Documentation

6.90.1.1 executable

int vmm_page_info_t::executable
Referenced by [vmm_query_page\(\)](#).

6.90.1.2 present

int vmm_page_info_t::present
Referenced by [vmm_query_page\(\)](#).

6.90.1.3 writable

int vmm_page_info_t::writable
Referenced by [vmm_query_page\(\)](#).

The documentation for this struct was generated from the following file:

- include/starkernel/[vmm.h](#)

6.91 VMRegistryEntry Struct Reference

```
#include <capsule_run.h>
```

Data Fields

- uint32_t [vm_id](#)
- uint32_t [state](#)
- uint64_t [birth_capsule_id](#)
- uint64_t [birth_timestamp_ns](#)
- uint64_t [birth_dict_hash](#)
- uint32_t [flags](#)
- uint32_t [reserved](#)

6.91.1 Field Documentation

6.91.1.1 birth_capsule_id

uint64_t VMRegistryEntry::birth_capsule_id
Referenced by [capsule_birth_mama\(\)](#), and [capsule_vm_registry_init\(\)](#).

6.91.1.2 birth_dict_hash

uint64_t VMRegistryEntry::birth_dict_hash
Referenced by [capsule_birth_mama\(\)](#), and [capsule_vm_registry_init\(\)](#).

6.91.1.3 birth_timestamp_ns

uint64_t VMRegistryEntry::birth_timestamp_ns
Referenced by [capsule_vm_registry_init\(\)](#).

6.91.1.4 flags

uint32_t VMRegistryEntry::flags

Referenced by [capsule_vm_registry_init\(\)](#).

6.91.1.5 reserved

uint32_t VMRegistryEntry::reserved

Referenced by [capsule_vm_registry_init\(\)](#).

6.91.1.6 state

uint32_t VMRegistryEntry::state

Referenced by [capsule_vm_registry_init\(\)](#).

6.91.1.7 vm_id

uint32_t VMRegistryEntry::vm_id

Referenced by [capsule_vm_registry_init\(\)](#).

The documentation for this struct was generated from the following file:

- [include/starkernel/capsule_run.h](#)

6.92 WordContract Struct Reference

```
#include <test_contracts.h>
```

Data Fields

- [contract_flags_t](#) flags
- [int](#) fuzz_rounds

6.92.1 Field Documentation

6.92.1.1 flags

[contract_flags_t](#) WordContract::flags

Referenced by [resolve_contract\(\)](#), and [run_test_suite_m\(\)](#).

6.92.1.2 fuzz_rounds

[int](#) WordContract::fuzz_rounds

Referenced by [run_test_suite_m\(\)](#).

The documentation for this struct was generated from the following file:

- [src/test_runner/include/test_contracts.h](#)

6.93 WordProfile Struct Reference

Execution statistics for a single word.

```
#include <profiler.h>
```

Data Fields

- [const char *](#) [word_name](#)
Name of the profiled word.
- [uint64_t](#) [total_time](#)
Total execution time.
- [uint64_t](#) [call_count](#)

Number of executions.

- uint64_t [avg_time](#)

Average execution time.

- uint64_t [min_time](#)

Minimum execution time.

- uint64_t [max_time](#)

Maximum execution time.

- double [percentage](#)

Percentage of total runtime.

6.93.1 Detailed Description

Execution statistics for a single word.

6.93.2 Field Documentation

6.93.2.1 avg_time

uint64_t WordProfile::avg_time
Average execution time.

6.93.2.2 call_count

uint64_t WordProfile::call_count
Number of executions.

6.93.2.3 max_time

uint64_t WordProfile::max_time
Maximum execution time.

6.93.2.4 min_time

uint64_t WordProfile::min_time
Minimum execution time.

6.93.2.5 percentage

double WordProfile::percentage
Percentage of total runtime.

6.93.2.6 total_time

uint64_t WordProfile::total_time
Total execution time.

6.93.2.7 word_name

const char* WordProfile::word_name
Name of the profiled word.

The documentation for this struct was generated from the following file:

- include/[profiler.h](#)

6.94 WordStats Struct Reference

Statistics for individual word execution.

Collaboration diagram for WordStats:

Data Fields

- const [DictEntry](#) * [entry](#)
Pointer to dictionary entry.
- uint64_t [call_count](#)
Number of times word was called.
- uint64_t [total_time_ns](#)
Total execution time in nanoseconds.
- uint64_t [min_time_ns](#)
Minimum execution time in nanoseconds.
- uint64_t [max_time_ns](#)
Maximum execution time in nanoseconds.

6.94.1 Detailed Description

Statistics for individual word execution.

6.94.2 Field Documentation

6.94.2.1 [call_count](#)

```
uint64_t WordStats::call_count
```

Number of times word was called.

Referenced by [compare_by_calls\(\)](#), [get_word_stats\(\)](#), [profiler_generate_report\(\)](#), [profiler_print_hotspots\(\)](#), [profiler_word_count\(\)](#), and [profiler_word_exit\(\)](#).

6.94.2.2 [entry](#)

```
const DictEntry* WordStats::entry
```

Pointer to dictionary entry.

Referenced by [get_word_stats\(\)](#), [profiler_generate_report\(\)](#), and [profiler_print_hotspots\(\)](#).

6.94.2.3 [max_time_ns](#)

```
uint64_t WordStats::max_time_ns
```

Maximum execution time in nanoseconds.

Referenced by [get_word_stats\(\)](#), [profiler_generate_report\(\)](#), and [profiler_word_exit\(\)](#).

6.94.2.4 [min_time_ns](#)

```
uint64_t WordStats::min_time_ns
```

Minimum execution time in nanoseconds.

Referenced by [get_word_stats\(\)](#), and [profiler_word_exit\(\)](#).

6.94.2.5 [total_time_ns](#)

```
uint64_t WordStats::total_time_ns
```

Total execution time in nanoseconds.

Referenced by [compare_by_time\(\)](#), [get_word_stats\(\)](#), [profiler_generate_report\(\)](#), and [profiler_word_exit\(\)](#).

The documentation for this struct was generated from the following file:

- [src/profiler.c](#)

6.95 WordTestSuite Struct Reference

Collection of test cases for a single Forth word.

```
#include <test_common.h>
```

Collaboration diagram for WordTestSuite:

Data Fields

- const char * [word_name](#)
Name of the Forth word being tested.
- [TestCase](#) [tests](#) [20]
Array of test cases.
- int [test_count](#)
Number of tests in the suite.
- [WordContract](#) [suite_contract](#)
Suite-level contract; zero = inherit from module.

6.95.1 Detailed Description

Collection of test cases for a single Forth word.

6.95.2 Field Documentation**6.95.2.1 suite_contract**

[WordContract](#) WordTestSuite::suite_contract

Suite-level contract; zero = inherit from module.

Referenced by [run_test_suite_m\(\)](#).

6.95.2.2 test_count

int WordTestSuite::test_count

Number of tests in the suite.

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

6.95.2.3 tests

[TestCase](#) WordTestSuite::tests [20]

Array of test cases.

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

6.95.2.4 word_name

const char* WordTestSuite::word_name

Name of the Forth word being tested.

Referenced by [run_arithmetic_words_tests\(\)](#), [run_block_words_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#), [run_dictionary_words_tests\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#), [run_io_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_return_stack_words_tests\(\)](#), [run_stack_words_tests\(\)](#), [run_starforth_words_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [run_vocabulary_words_tests\(\)](#).

The documentation for this struct was generated from the following file:

- src/test_runner/include/test_common.h

6.96 WordTransitionMetrics Struct Reference

```
#include <physics_pipelining_metrics.h>
```

Data Fields

- uint64_t * [transition_heat](#)
transition_heat[i]: Number of times word with index i has been executed immediately after this word.
- uint64_t [total_transitions](#)
Total number of transitions observed from this word.

- uint64_t [prefetch_attempts](#)
Number of speculative prefetch attempts made for this word.
- uint64_t [prefetch_hits](#)
Number of successful prefetch predictions (hits)
- uint64_t [prefetch_misses](#)
Number of failed prefetch predictions (misses)
- int64_t [prefetch_latency_saved_q48](#)
Total latency saved through successful prefetching (Q48.16 nanoseconds).
- int64_t [misprediction_cost_q48](#)
Total latency cost from failed prefetch predictions (Q48.16 nanoseconds).
- int64_t [max_transition_probability_q48](#)
Cached probability of most likely next word (for quick access).
- uint32_t [most_likely_next_word_id](#)
Index (word id) of most likely next word.
- uint32_t * [context_window](#)
Circular buffer of previous word IDs for context tracking.
- uint32_t [context_window_pos](#)
Current position in context_window circular buffer.
- void * [context_transitions](#)
Sparse hash table for context-based transitions.
- uint64_t [total_context_transitions](#)
Total context-based transitions observed.
- uint32_t [actual_window_size](#)
Actual window size for this word's metrics.

6.96.1 Field Documentation

6.96.1.1 actual_window_size

uint32_t WordTransitionMetrics::actual_window_size

Actual window size for this word's metrics.

May differ from TRANSITION_WINDOW_SIZE if memory allocation failed. Phase 2 uses this to validate context data reliability.

Referenced by [transition_metrics_context_accuracy_string\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_update_context_window\(\)](#).

6.96.1.2 context_transitions

void* WordTransitionMetrics::context_transitions

Sparse hash table for context-based transitions.

Maps: hash(context_window) → counts_array[DICTIONARY_SIZE]

Phase 1: Collect only (simple reference implementation) Phase 2: Analyze prediction accuracy by context depth

Phase 3: Use for adaptive prefetching decisions

Opaque pointer - implementation is in transition_metrics.c

Referenced by [transition_metrics_cleanup\(\)](#), [transition_metrics_init\(\)](#), and [transition_metrics_reset\(\)](#).

6.96.1.3 context_window

uint32_t* WordTransitionMetrics::context_window

Circular buffer of previous word IDs for context tracking.

Size = TRANSITION_WINDOW_SIZE (default 2) Used to build multi-word context patterns (e.g., A,B→C)

Example with WINDOW=2: context_window[0] = older word (2 steps back) context_window[1] = recent word (1 step back) Next word completes the pattern: (context_window[0], context_window[1]) → next

Referenced by [transition_metrics_cleanup\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_update_context_window\(\)](#).

6.96.1.4 context_window_pos

```
uint32_t WordTransitionMetrics::context_window_pos
```

Current position in context_window circular buffer.

Points to slot where next word ID will be stored.

Referenced by [transition_metrics_init\(\)](#), and [transition_metrics_reset\(\)](#).

6.96.1.5 max_transition_probability_q48

```
int64_t WordTransitionMetrics::max_transition_probability_q48
```

Cached probability of most likely next word (for quick access).

Updated periodically to avoid recomputation. Q48.16 format (0.0 to 1.0 == 0LL to 1LL<<16)

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_reset\(\)](#), [transition_metrics_stats_string\(\)](#), and [transition_metrics_update_cache\(\)](#).

6.96.1.6 misprediction_cost_q48

```
int64_t WordTransitionMetrics::misprediction_cost_q48
```

Total latency cost from failed prefetch predictions (Q48.16 nanoseconds).

Negative value (cost) to be subtracted from savings.

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record_prefetch_miss\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_stats_string\(\)](#).

6.96.1.7 most_likely_next_word_id

```
uint32_t WordTransitionMetrics::most_likely_next_word_id
```

Index (word id) of most likely next word.

Used for fast prefetch without scanning transition_heat array.

Referenced by [execute_colon_word\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [physics_pre_execute\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_reset\(\)](#), [transition_metrics_stats_string\(\)](#), and [transition_metrics_update_cache\(\)](#).

6.96.1.8 prefetch_attempts

```
uint64_t WordTransitionMetrics::prefetch_attempts
```

Number of speculative prefetch attempts made for this word.

Referenced by [execute_colon_word\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [physics_pre_execute\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_stats_string\(\)](#).

6.96.1.9 prefetch_hits

```
uint64_t WordTransitionMetrics::prefetch_hits
```

Number of successful prefetch predictions (hits)

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record_prefetch_hit\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_stats_string\(\)](#).

6.96.1.10 prefetch_latency_saved_q48

```
int64_t WordTransitionMetrics::prefetch_latency_saved_q48
```

Total latency saved through successful prefetching (Q48.16 nanoseconds).

Positive value indicates net benefit from speculation.

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record_prefetch_hit\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_stats_string\(\)](#).

6.96.1.11 prefetch_misses

```
uint64_t WordTransitionMetrics::prefetch_misses
```

Number of failed prefetch predictions (misses)

Referenced by [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record_prefetch_miss\(\)](#), [transition_metrics_reset\(\)](#), and [transition_metrics_stats_string\(\)](#).

6.96.1.12 total_context_transitions

`uint64_t WordTransitionMetrics::total_context_transitions`

Total context-based transitions observed.

Counter for Phase 2 analysis (accuracy measurement)

Referenced by [transition_metrics_context_accuracy_string\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record_context\(\)](#), and [transition_metrics_reset\(\)](#).

6.96.1.13 total_transitions

`uint64_t WordTransitionMetrics::total_transitions`

Total number of transitions observed from this word.

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [transition_metrics_get_probability_q48\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record\(\)](#), [transition_metrics_reset\(\)](#), [transition_metrics_should_speculate\(\)](#), [transition_metrics_stats_string\(\)](#), and [transition_metrics_update_cache\(\)](#).

6.96.1.14 transition_heat

`uint64_t* WordTransitionMetrics::transition_heat`

`transition_heat[i]`: Number of times word with index `i` has been executed immediately after this word.

Dynamically allocated array of size `DICTIONARY_SIZE`. Initialized to `NULL`; allocated on first use to save memory for words that are never executed.

Referenced by [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [transition_metrics_cleanup\(\)](#), [transition_metrics_get_probability_q48\(\)](#), [transition_metrics_init\(\)](#), [transition_metrics_record\(\)](#), [transition_metrics_reset\(\)](#), [transition_metrics_should_speculate\(\)](#), and [transition_metrics_update_cache\(\)](#).

The documentation for this struct was generated from the following file:

- `include/physics_pipelining_metrics.h`

6.97 XXHash64State Struct Reference

Streaming state structure.

```
#include <xxhash64.h>
```

Data Fields

- `uint64_t total_len`
- `uint64_t v1`
- `uint64_t v2`
- `uint64_t v3`
- `uint64_t v4`
- `uint8_t buffer` [32]
- `uint32_t buffer_size`
- `uint64_t seed`

6.97.1 Detailed Description

Streaming state structure.

6.97.2 Field Documentation

6.97.2.1 buffer

`uint8_t XXHash64State::buffer` [32]

Referenced by [xxhash64_digest\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.2 buffer_size

`uint32_t XXHash64State::buffer_size`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.3 seed

`uint64_t XXHash64State::seed`

Referenced by [xxhash64_digest\(\)](#), and [xxhash64_reset\(\)](#).

6.97.2.4 total_len

`uint64_t XXHash64State::total_len`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.5 v1

`uint64_t XXHash64State::v1`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.6 v2

`uint64_t XXHash64State::v2`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.7 v3

`uint64_t XXHash64State::v3`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

6.97.2.8 v4

`uint64_t XXHash64State::v4`

Referenced by [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

The documentation for this struct was generated from the following file:

- `include/starkernel/xxhash64.h`

Chapter 7

File Documentation

7.1 include/arch_detect.h File Reference

Architecture Detection and Unified API for StarForth.

```
#include "vm_asm_opt.h"
```

```
#include "vm_inner_interp_asm.h"
```

Include dependency graph for arch_detect.h:

Macros

- #define ARCH_X86_64 0
- #define ARCH_ARM64 0
- #define ARCH_RISCV64 0
- #define ARCH_NAME "Unknown"
- #define vm_push_opt(vm, val) vm_push_asm(vm, val)
- #define vm_pop_opt(vm) vm_pop_asm(vm)
- #define vm_rpush_opt(vm, val) vm_rpush_asm(vm, val)
- #define vm_rpop_opt(vm) vm_rpop_asm(vm)
- #define vm_add_overflow_opt(a, b, res) vm_add_check_overflow(a, b, res)
- #define vm_sub_overflow_opt(a, b, res) vm_sub_check_overflow(a, b, res)
- #define vm_mul_double_opt(a, b, hi, lo) vm_mul_double(a, b, hi, lo)
- #define vm_divmod_opt(dividend, divisor, quot, rem) vm_divmod(dividend, divisor, quot, rem)
- #define vm_strcmp_opt(s1, s2, len) vm_strcmp_asm(s1, s2, len)
- #define vm_memcpy_opt(dst, src, len) vm_memcpy_asm(dst, src, len)
- #define vm_memzero_opt(dst, len) vm_memzero_asm(dst, len)
- #define vm_min_opt(a, b) vm_min_asm(a, b)
- #define vm_max_opt(a, b) vm_max_asm(a, b)
- #define vm_abs_opt(a) vm_abs_asm(a)
- #define vm_clz_opt(x) vm_clz(x)
- #define vm_ctz_opt(x) vm_ctz(x)
- #define vm_popcnt_opt(x) vm_popcnt(x)
- #define vm_prefetch_opt(addr) vm_prefetch(addr)
- #define vm_prefetch_stream_opt(addr) vm_prefetch_stream(addr)
- #define NEXT_OPT() NEXT_ASM()
- #define vm_setup_registers_opt(vm) vm_setup_registers(vm)
- #define vm_save_registers_opt(vm) vm_save_registers(vm)
- #define PRIM_DUP_OPT() PRIM_DUP()
- #define PRIM_DROP_OPT() PRIM_DROP()
- #define PRIM_SWAP_OPT() PRIM_SWAP()
- #define PRIM_PLUS_OPT() PRIM_PLUS()
- #define PRIM_MINUS_OPT() PRIM_MINUS()
- #define PRIM_STAR_OPT() PRIM_STAR()

- `#define PRIM_FETCH_OPT() PRIM_FETCH()`
- `#define PRIM_STORE_OPT() PRIM_STORE()`
- `#define PRIM_TO_R_OPT() PRIM_TO_R()`
- `#define PRIM_R_FROM_OPT() PRIM_R_FROM()`
- `#define PRIM_R_FETCH_OPT() PRIM_R_FETCH()`
- `#define PRIM_BRANCH_OPT() PRIM_BRANCH()`
- `#define PRIM_ZBRANCH_OPT() PRIM_ZBRANCH()`

Functions

- static int `vm_has_sse42_opt` (void)
- static const char * `vm_get_arch_name` (void)
- static int `vm_has_asm_opt` (void)
- static int `vm_has_direct_threading` (void)
- static const char * `vm_get_build_info` (void)

7.1.1 Detailed Description

Architecture Detection and Unified API for StarForth.

Automatically selects x86_64 or ARM64 optimizations and provides a unified interface for architecture-specific optimizations.

This work is released under license terms in LICENSE file. No warranty. Use at your own risk.

7.1.2 Macro Definition Documentation

7.1.2.1 ARCH_ARM64

```
#define ARCH_ARM64 0
```

7.1.2.2 ARCH_NAME

```
#define ARCH_NAME "Unknown"
```

7.1.2.3 ARCH_RISCV64

```
#define ARCH_RISCV64 0
```

7.1.2.4 ARCH_X86_64

```
#define ARCH_X86_64 0
```

7.1.2.5 NEXT_OPT

```
#define NEXT_OPT( ) NEXT_ASM()
```

7.1.2.6 PRIM_BRANCH_OPT

```
#define PRIM_BRANCH_OPT( ) PRIM_BRANCH()
```

7.1.2.7 PRIM_DROP_OPT

```
#define PRIM_DROP_OPT( ) PRIM_DROP()
```

7.1.2.8 PRIM_DUP_OPT

```
#define PRIM_DUP_OPT( ) PRIM_DUP()
```

7.1.2.9 PRIM_FETCH_OPT

```
#define PRIM_FETCH_OPT( ) PRIM_FETCH()
```


7.1.2.10 PRIM_MINUS_OPT

```
#define PRIM_MINUS_OPT( ) PRIM_MINUS()
```

7.1.2.11 PRIM_PLUS_OPT

```
#define PRIM_PLUS_OPT( ) PRIM_PLUS()
```

7.1.2.12 PRIM_R_FETCH_OPT

```
#define PRIM_R_FETCH_OPT( ) PRIM_R_FETCH()
```

7.1.2.13 PRIM_R_FROM_OPT

```
#define PRIM_R_FROM_OPT( ) PRIM_R_FROM()
```

7.1.2.14 PRIM_STAR_OPT

```
#define PRIM_STAR_OPT( ) PRIM_STAR()
```

7.1.2.15 PRIM_STORE_OPT

```
#define PRIM_STORE_OPT( ) PRIM_STORE()
```

7.1.2.16 PRIM_SWAP_OPT

```
#define PRIM_SWAP_OPT( ) PRIM_SWAP()
```

7.1.2.17 PRIM_TO_R_OPT

```
#define PRIM_TO_R_OPT( ) PRIM_TO_R()
```

7.1.2.18 PRIM_ZBRANCH_OPT

```
#define PRIM_ZBRANCH_OPT( ) PRIM_ZBRANCH()
```

7.1.2.19 vm_abs_opt

```
#define vm_abs_opt(  
    a ) vm_abs_asm(a)
```

7.1.2.20 vm_add_overflow_opt

```
#define vm_add_overflow_opt(  
    a,  
    b,  
    res ) vm_add_check_overflow(a, b, res)
```

7.1.2.21 vm_clz_opt

```
#define vm_clz_opt(  
    x ) vm_clz(x)
```

7.1.2.22 vm_ctz_opt

```
#define vm_ctz_opt(  
    x ) vm_ctz(x)
```

7.1.2.23 vm_divmod_opt

```
#define vm_divmod_opt(  
    dividend,  
    divisor,  
    quot,  
    rem ) vm_divmod(dividend, divisor, quot, rem)
```

7.1.2.24 vm_max_opt

```
#define vm_max_opt(  
    a,  
    b ) vm_max_asm(a, b)
```

7.1.2.25 vm_memcpy_opt

```
#define vm_memcpy_opt(  
    dst,  
    src,  
    len ) vm_memcpy_asm(dst, src, len)
```

7.1.2.26 vm_memzero_opt

```
#define vm_memzero_opt(  
    dst,  
    len ) vm_memzero_asm(dst, len)
```

7.1.2.27 vm_min_opt

```
#define vm_min_opt(  
    a,  
    b ) vm_min_asm(a, b)
```

7.1.2.28 vm_mul_double_opt

```
#define vm_mul_double_opt(  
    a,  
    b,  
    hi,  
    lo ) vm_mul_double(a, b, hi, lo)
```

7.1.2.29 vm_pop_opt

```
#define vm_pop_opt(  
    vm ) vm_pop_asm(vm)
```

7.1.2.30 vm_popcnt_opt

```
#define vm_popcnt_opt(  
    x ) vm_popcnt(x)
```

7.1.2.31 vm_prefetch_opt

```
#define vm_prefetch_opt(  
    addr ) vm_prefetch(addr)
```

7.1.2.32 vm_prefetch_stream_opt

```
#define vm_prefetch_stream_opt(  
    addr ) vm_prefetch_stream(addr)
```

7.1.2.33 vm_push_opt

```
#define vm_push_opt(
    vm,
    val ) vm_push_asm(vm, val)
```

7.1.2.34 vm_rpop_opt

```
#define vm_rpop_opt(
    vm ) vm_rpop_asm(vm)
```

7.1.2.35 vm_rpush_opt

```
#define vm_rpush_opt(
    vm,
    val ) vm_rpush_asm(vm, val)
```

7.1.2.36 vm_save_registers_opt

```
#define vm_save_registers_opt(
    vm ) vm_save_registers(vm)
```

7.1.2.37 vm_setup_registers_opt

```
#define vm_setup_registers_opt(
    vm ) vm_setup_registers(vm)
```

7.1.2.38 vm_strcmp_opt

```
#define vm_strcmp_opt(
    s1,
    s2,
    len ) vm_strcmp_asm(s1, s2, len)
```

7.1.2.39 vm_sub_overflow_opt

```
#define vm_sub_overflow_opt(
    a,
    b,
    res ) vm_sub_check_overflow(a, b, res)
```

7.1.3 Function Documentation**7.1.3.1 vm_get_arch_name()**

```
static const char * vm_get_arch_name (
    void ) [inline], [static]
```

References [ARCH_NAME](#).

Referenced by [vm_get_build_info\(\)](#).

Here is the caller graph for this function:

7.1.3.2 vm_get_build_info()

```
static const char * vm_get_build_info (
    void ) [inline], [static]
```

References [vm_get_arch_name\(\)](#), [vm_has_asm_opt\(\)](#), and [vm_has_direct_threading\(\)](#).

Here is the call graph for this function:

7.1.3.3 vm_has_asm_opt()

```
static int vm_has_asm_opt (
    void ) [inline], [static]
```

Referenced by [vm_get_build_info\(\)](#).

Here is the caller graph for this function:

7.1.3.4 vm_has_direct_threading()

```
static int vm_has_direct_threading (
    void ) [inline], [static]
```

Referenced by [vm_get_build_info\(\)](#).

Here is the caller graph for this function:

7.1.3.5 vm_has_sse42_opt()

```
static int vm_has_sse42_opt (
    void ) [inline], [static]
```

References [vm_has_sse42\(\)](#).

Here is the call graph for this function:

7.2 include/blkcfg.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for blkcfg.h:

Data Structures

- struct [blkcfg_t](#)

Macros

- #define [BLKCFG_DEFAULT_FBS](#) 1024u /* default Forth Block Size in bytes */
- #define [BLKCFG_PATH_MAX](#) 4096

Enumerations

- enum [blkcfg_backend_t](#) { [BLKCFG_BACKEND_NONE](#) = 0 , [BLKCFG_BACKEND_FILE](#) = 1 , [BLKCFG_BACKEND_RAM](#) = 2 }

Functions

- int [blkcfg_init_from_options](#) ([blkcfg_t](#) *cfg, const char *disk_img_path, uint64_t ram_mb, uint32_t fbs, uint8_t read_only)
Initialize a block configuration from CLI-style options.
- int [blkcfg_format_summary](#) (const [blkcfg_t](#) *cfg, char *dst, size_t dst_cap)
Format a single-line summary into 'dst' (capacity 'dst_cap' bytes).

Variables

- [blkcfg_t g_blkcfg](#)

7.2.1 Macro Definition Documentation

7.2.1.1 BLKCFG_DEFAULT_FBS

```
#define BLKCFG_DEFAULT_FBS 1024u /* default Forth Block Size in bytes */
```

7.2.1.2 BLKCFG_PATH_MAX

```
#define BLKCFG_PATH_MAX 4096
```

7.2.2 Enumeration Type Documentation

7.2.2.1 blkcfg_backend_t

```
enum blkcfg_backend_t
```

Enumerator

BLKCFG_BACKEND_NONE	
BLKCFG_BACKEND_FILE	
BLKCFG_BACKEND_RAM	

7.2.3 Function Documentation

7.2.3.1 blkcfg_format_summary()

```
int blkcfg_format_summary (
    const blkcfg_t * cfg,
    char * dst,
    size_t dst_cap )
```

Format a single-line summary into 'dst' (capacity 'dst_cap' bytes).

Example: "backend=FILE path=./disks/os.img fbs=1024 ro=0" "backend=RAM size=1024MB fbs=1024 blocks=1048576 ro=0"

Returns number of bytes that would have been written (like snprintf).

7.2.3.2 blkcfg_init_from_options()

```
int blkcfg_init_from_options (
    blkcfg_t * cfg,
    const char * disk_img_path,
    uint64_t ram_mb,
    uint32_t fbs,
    uint8_t read_only )
```

Initialize a block configuration from CLI-style options.

Inputs:

- disk_img_path: if non-NULL and non-empty => choose FILE backend
- ram_mb: megabytes for RAM fallback (only used if no disk image)
- fbs: forth block size (bytes). If 0 => BLKCFG_DEFAULT_FBS
- read_only: intended read-only flag (advisory)

Behavior:

- FILE: record path, set blocks/size_bytes = 0 (unknown until attached)
- RAM : compute size_bytes = ram_mb*MiB; blocks = size_bytes/fbs (floored)

Returns: 0 on success; -1 on invalid args.

7.2.4 Variable Documentation

7.2.4.1 g_blkcfg

```
blkcfg_t g_blkcfg [extern]
```

7.3 include/blkio.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for blkio.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [blkio_params_t](#)
- struct [blkio_info_t](#)
- struct [blkio_vtable_t](#)
- struct [blkio_dev](#)

Macros

- #define [BLKIO_FORTH_BLOCK_SIZE](#) 1024u

Typedefs

- typedef struct [blkio_dev](#) [blkio_dev_t](#)

Enumerations

- enum {
[BLKIO_OK](#) = 0 , [BLKIO_EINVAL](#) = -1 , [BLKIO_ENOSUP](#) = -2 , [BLKIO_EIO](#) = -3 ,
[BLKIO_ENOSPACE](#) = -4 , [BLKIO_ECLOSED](#) = -5 }

Functions

- static int [blkio_open](#) ([blkio_dev_t](#) *dev, const [blkio_vtable_t](#) *vt, const [blkio_params_t](#) *p)
- static int [blkio_close](#) ([blkio_dev_t](#) *dev)
- static int [blkio_read](#) ([blkio_dev_t](#) *dev, uint32_t fblock, void *dst)
- static int [blkio_write](#) ([blkio_dev_t](#) *dev, uint32_t fblock, const void *src)
- static int [blkio_flush](#) ([blkio_dev_t](#) *dev)
- static int [blkio_info](#) ([blkio_dev_t](#) *dev, [blkio_info_t](#) *out)
- static void [blkio_bzero](#) (void *p, uint32_t fbs)

7.3.1 Macro Definition Documentation

7.3.1.1 BLKIO_FORTH_BLOCK_SIZE

```
#define BLKIO_FORTH_BLOCK_SIZE 1024u
```

7.3.2 Typedef Documentation

7.3.2.1 blkio_dev_t

```
typedef struct blkio\_dev blkio\_dev\_t
```

7.3.3 Enumeration Type Documentation

7.3.3.1 anonymous enum

```
anonymous enum
```

Enumerator

BLKIO_OK	
BLKIO_EINVAL	
BLKIO_ENOSUP	

Enumerator

BLKIO_EIO	
BLKIO_ENOSPACE	
BLKIO_ECLOSED	

7.3.4 Function Documentation

7.3.4.1 blkio_bzero()

```
static void blkio_bzero (
    void * p,
    uint32_t fbs ) [inline], [static]
```

7.3.4.2 blkio_close()

```
static int blkio_close (
    blkio_dev_t * dev ) [inline], [static]
```

References [BLKIO_ECLOSED](#), [blkio_vtable_t::close](#), [blkio_dev::is_open](#), and [blkio_dev::vt](#).

Referenced by [cleanup_blkio\(\)](#).

Here is the caller graph for this function:

7.3.4.3 blkio_flush()

```
static int blkio_flush (
    blkio_dev_t * dev ) [inline], [static]
```

References [BLKIO_ECLOSED](#), [blkio_vtable_t::flush](#), [blkio_dev::is_open](#), and [blkio_dev::vt](#).

Referenced by [bam_flush_to_disk\(\)](#), [blk_flush\(\)](#), [blk_format_or_load_v2\(\)](#), [blk_subsys_shutdown\(\)](#), and [cleanup_blkio\(\)](#).

Here is the caller graph for this function:

7.3.4.4 blkio_info()

```
static int blkio_info (
    blkio_dev_t * dev,
    blkio_info_t * out ) [inline], [static]
```

References [BLKIO_EINVAL](#), [BLKIO_OK](#), [blkio_info_t::forth_block_size](#), [blkio_dev::forth_block_size](#), [blkio_vtable_t::info](#), [blkio_dev::is_open](#), [blkio_info_t::total_blocks](#), [blkio_dev::total_blocks](#), and [blkio_dev::vt](#).

Referenced by [blk_subsys_attach_device\(\)](#), and [init_blkio\(\)](#).

Here is the caller graph for this function:

7.3.4.5 blkio_open()

```
static int blkio_open (
    blkio_dev_t * dev,
    const blkio_vtable_t * vt,
    const blkio_params_t * p ) [inline], [static]
```

References [BLKIO_EINVAL](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_vtable_t::close](#), [blkio_vtable_t::flush](#), [blkio_params_t::forth_block_size](#), [blkio_dev::forth_block_size](#), [blkio_vtable_t::info](#), [blkio_dev::is_open](#), [blkio_vtable_t::open](#), [blkio_vtable_t::read](#), [blkio_dev::state](#), [blkio_params_t::total_blocks](#), [blkio_dev::total_blocks](#), [blkio_dev::vt](#), and [blkio_vtable_t::write](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.3.4.6 blkio_read()

```
static int blkio_read (
    blkio_dev_t * dev,
```

```
uint32_t fblock,
void * dst ) [inline], [static]
```

References [BLKIO_EINVAL](#), [blkio_dev::is_open](#), [blkio_vtable_t::read](#), [blkio_dev::total_blocks](#), and [blkio_dev::vt](#).
Referenced by [bam_load_from_disk\(\)](#), [read_devblock_4k\(\)](#), and [read_header_4k\(\)](#).

Here is the caller graph for this function:

7.3.4.7 blkio_write()

```
static int blkio_write (
    blkio_dev_t * dev,
    uint32_t fblock,
    const void * src ) [inline], [static]
```

References [BLKIO_EINVAL](#), [blkio_dev::is_open](#), [blkio_dev::total_blocks](#), [blkio_dev::vt](#), and [blkio_vtable_t::write](#).
Referenced by [bam_flush_to_disk\(\)](#), [blk_format_or_load_v2\(\)](#), [write_devblock_4k\(\)](#), and [write_header_4k\(\)](#).

Here is the caller graph for this function:

7.4 include/blkio_factory.h File Reference

```
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for blkio_factory.h: This graph shows which files directly or indirectly include this file:

Functions

- int [blkio_factory_open](#) (struct [blkio_dev](#) *dev, const char *disk_img_path, uint8_t read_only, uint32_t total_blocks, uint32_t fbs, void *state_mem, size_t state_len, uint8_t *ram_base, uint32_t ram_blocks, uint8_t *out_used_file)

Open a block device using either a file-backed image (if disk_img_path is non-empty) or a RAM-backed buffer (fallback).

7.4.1 Function Documentation

7.4.1.1 blkio_factory_open()

```
int blkio_factory_open (
    struct blkio_dev * dev,
    const char * disk_img_path,
    uint8_t read_only,
    uint32_t total_blocks,
    uint32_t fbs,
    void * state_mem,
    size_t state_len,
    uint8_t * ram_base,
    uint32_t ram_blocks,
    uint8_t * out_used_file )
```

Open a block device using either a file-backed image (if disk_img_path is non-empty) or a RAM-backed buffer (fallback).

Parameters:

- dev : output device object (caller-allocated)
- disk_img_path : path to raw image; if NULL/empty => RAM fallback
- read_only : 1 = open read-only; 0 = read/write
- total_blocks : FILE: 0 => derive from file size; >0 => size hint RAM : must match ram_blocks
- fbs : forth block size in bytes (0 => default per [blkio.h](#))
- state_mem : caller-provided backend state buffer (opaque to caller)

- `state_len` : size of `state_mem` in bytes
- `ram_base` : RAM storage base pointer (RAM backend only)
- `ram_blocks` : number of Forth blocks in `ram_base` (RAM backend only)
- `out_used_file` : optional; set to 1 if FILE backend selected, else 0

Returns: 0 on success; negative blkio error code on failure.

References [BLKIO_EINVAL](#), [BLKIO_ENOSPACE](#), [blkio_file_init_state\(\)](#), [blkio_file_state_size\(\)](#), [blkio_file_vtable\(\)](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_open\(\)](#), [blkio_ram_init_state\(\)](#), [blkio_ram_state_size\(\)](#), [blkio_ram_vtable\(\)](#), [blkio_params_t::forth_block_size](#), [nonempty\(\)](#), and [ram_base](#).

Referenced by [init_blkio\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5 include/block_subsystem.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "vm.h"
#include "blkio.h"
```

Include dependency graph for `block_subsystem.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [blk_volume_meta_t](#)
- struct [blk_meta_t](#)

Macros

- `#define BLK_FORTH_SIZE` 1024u /* Forth block size */
- `#define BLK_RAM_BLOCKS` 1024u /* RAM 0..1023 */
- `#define BLK_DISK_START` 1024u /* Disk-backed Forth blocks start */
- `#define BLK_DEVICE_SECTOR` 4096u /* Physical "devblock" size (4×1 KiB blkio units) */
- `#define BLK_PACK_RATIO` 3u /* 3× 1 KiB data per 4 KiB devblock (plus 1 KiB metadata) */
- `#define BLK_META_TOTAL` 1024u /* Last 1 KiB in a 4 KiB devblock is metadata */
- `#define BLK_META_PER_BLOCK` 341u /* 341×3 ≈ 1023, padded to 1024 */
- `#define BLK_FORTH_SYS_RESERVED` 33u /* RAM 0..32 reserved */
- `#define BLK_DISK_SYS_RESERVED` 32u /* Disk 1024..1055 reserved (first N disk blocks) */

Enumerations

- enum {
[BLK_OK](#) = 0 , [BLK_EINVAL](#) = -1 , [BLK_ERANGE](#) = -2 , [BLK_EIO](#) = -3 ,
[BLK_ENODEV](#) = -4 , [BLK_ERESERVED](#) = -5 , [BLK_EDIRTY](#) = -6 , [BLK_ENOMEM](#) = -7 }

Functions

- int [blk_subsys_init](#) (VM *vm, uint8_t *ram_base, size_t ram_size)
- int [blk_subsys_attach_device](#) (struct blkio_dev *dev)
- int [blk_subsys_shutdown](#) (void)
- uint8_t * [blk_get_buffer](#) (uint32_t block_num, int writable)
- uint8_t * [blk_get_empty_buffer](#) (uint32_t block_num)
- int [blk_flush](#) (uint32_t block_num)
- int [blk_update](#) (uint32_t block_num)
- int [blk_get_volume_meta](#) (blk_volume_meta_t *meta)
- int [blk_set_volume_meta](#) (const blk_volume_meta_t *meta)

- int [blk_is_valid](#) (uint32_t block_num)
- uint32_t [blk_get_total_blocks](#) (void)
- int [blk_get_meta](#) (uint32_t block_num, [blk_meta_t](#) *meta)
- int [blk_set_meta](#) (uint32_t block_num, const [blk_meta_t](#) *meta)
- int [blk_is_allocated](#) (uint32_t block_num)
- int [blk_mark_allocated](#) (uint32_t block_num)
- int [blk_mark_free](#) (uint32_t block_num)
- int [blk_allocate](#) (uint32_t *block_num)

7.5.1 Macro Definition Documentation

7.5.1.1 BLK_DEVICE_SECTOR

```
#define BLK_DEVICE_SECTOR 4096u /* Physical "devblock" size (4×1 KiB blkio units) */
```

7.5.1.2 BLK_DISK_START

```
#define BLK_DISK_START 1024u /* Disk-backed Forth blocks start */
```

7.5.1.3 BLK_DISK_SYS_RESERVED

```
#define BLK_DISK_SYS_RESERVED 32u /* Disk 1024..1055 reserved (first N disk blocks) */
```

7.5.1.4 BLK_FORTH_SIZE

```
#define BLK_FORTH_SIZE 1024u /* Forth block size */
```

7.5.1.5 BLK_FORTH_SYS_RESERVED

```
#define BLK_FORTH_SYS_RESERVED 33u /* RAM 0..32 reserved */
```

7.5.1.6 BLK_META_PER_BLOCK

```
#define BLK_META_PER_BLOCK 341u /* 341×3 ~= 1023, padded to 1024 */
```

7.5.1.7 BLK_META_TOTAL

```
#define BLK_META_TOTAL 1024u /* Last 1 KiB in a 4 KiB devblock is metadata */
```

7.5.1.8 BLK_PACK_RATIO

```
#define BLK_PACK_RATIO 3u /* 3× 1 KiB data per 4 KiB devblock (plus 1 KiB metadata) */
```

7.5.1.9 BLK_RAM_BLOCKS

```
#define BLK_RAM_BLOCKS 1024u /* RAM 0..1023 */
```

7.5.2 Enumeration Type Documentation

7.5.2.1 anonymous enum

anonymous enum

Enumerator

BLK_OK	
BLK_EINVAL	
BLK_ERANGE	
BLK_EIO	

Enumerator

BLK_ENODEV	
BLK_ERESERVED	
BLK_EDIRTY	
BLK_ENOMEM	

7.5.3 Function Documentation

7.5.3.1 blk_allocate()

```
int blk_allocate (
    uint32_t * block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [limit](#).

Here is the call graph for this function:

7.5.3.2 blk_flush()

```
int blk_flush (
    uint32_t block_num )
```

References [bam_flush_to_disk\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_FORTH_SYS_RESERVED](#), [BLK_OK](#), [blkio_flush\(\)](#), [cache_writeback\(\)](#), [cache_slot_t::devblock](#), [cache_slot_t::dirty](#), [DISK_CACHE_SLOTS](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [cache_slot_t::meta_dirty](#), and [cache_slot_t::valid](#).

Referenced by [blk_subsys_shutdown\(\)](#), [block_word_flush\(\)](#), and [save_all_buffers\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5.3.3 blk_get_buffer()

```
uint8_t * blk_get_buffer (
    uint32_t block_num,
    int writable )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [cache_load_devblock\(\)](#), [cache_slot_t::data](#), [cache_slot_t::dirty](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), and [lbn_in_range\(\)](#).

Referenced by [blk_get_empty_buffer\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [capsule_clear_blocks\(\)](#), [copy_block_to_ram\(\)](#), [empty_all_buffers\(\)](#), [get_block_buffer\(\)](#), [line_ptr\(\)](#), [starforth_word_init\(\)](#), and [write_ramdrive_block\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5.3.4 blk_get_empty_buffer()

```
uint8_t * blk_get_empty_buffer (
    uint32_t block_num )
```

References [BLK_FORTH_SIZE](#), and [blk_get_buffer\(\)](#).

Referenced by [get_empty_buffer\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5.3.5 blk_get_meta()

```
int blk_get_meta (
    uint32_t block_num,
    blk_meta_t * meta )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [blk_meta_t::magic](#), and [cache_slot_t::meta](#).

Here is the call graph for this function:

7.5.3.6 blk_get_total_blocks()

```
uint32_t blk_get_total_blocks (
    void )
```

References [g](#).

Referenced by [empty_all_buffers\(\)](#).

Here is the caller graph for this function:

7.5.3.7 blk_get_volume_meta()

```
int blk_get_volume_meta (
    blk_volume_meta_t * meta )
```

References [BLK_EINVAL](#), [BLK_OK](#), and [g](#).

7.5.3.8 blk_is_allocated()

```
int blk_is_allocated (
    uint32_t block_num )
```

References [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.5.3.9 blk_is_valid()

```
int blk_is_valid (
    uint32_t block_num )
```

References [g](#), and [lbn_in_range\(\)](#).

Referenced by [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), and [line_ptr\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5.3.10 blk_mark_allocated()

```
int blk_mark_allocated (
    uint32_t block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.5.3.11 blk_mark_free()

```
int blk_mark_free (
    uint32_t block_num )
```

References [bam_clr\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.5.3.12 blk_set_meta()

```
int blk_set_meta (
    uint32_t block_num,
    const blk_meta_t * meta )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [cache_slot_t::meta](#), and [cache_slot_t::meta_dirty](#).

Here is the call graph for this function:

7.5.3.13 blk_set_volume_meta()

```
int blk_set_volume_meta (
    const blk_volume_meta_t * meta )
```

References [BLK_EINVAL](#), [BLK_OK](#), and [g](#).

7.5.3.14 blk_subsys_attach_device()

```
int blk_subsys_attach_device (
    struct blkio_dev * dev )
```

References [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [blk_format_or_load_v2\(\)](#), [blk_get_timestamp\(\)](#), [BLK_OK](#), [blkio_info\(\)](#), [BLKIO_OK](#), [g](#), [LOG_INFO](#), [log_message\(\)](#), and [blkio_info_t::total_blocks](#).

Referenced by [__attribute__\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.5.3.15 blk_subsys_init()

```
int blk_subsys_init (
    VM * vm,
    uint8_t * ram_base,
    size_t ram_size )
```

References [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [BLK_OK](#), [BLK_RAM_BLOCKS](#), [g](#), [ram_base](#), [ram_size](#), and [vm](#).

Referenced by [init_vm_and_subsystem\(\)](#).

Here is the caller graph for this function:

7.5.3.16 blk_subsys_shutdown()

```
int blk_subsys_shutdown (
    void )
```

References [blk_flush\(\)](#), [BLK_OK](#), [blkio_flush\(\)](#), [g](#), [volmeta_to_buf\(\)](#), and [write_header_4k\(\)](#).

Here is the call graph for this function:

7.5.3.17 blk_update()

```
int blk_update (
    uint32_t block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [blk_get_timestamp\(\)](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [blk_meta_t::checksum](#), [compute_crc64\(\)](#), [blk_meta_t::created_time](#), [cache_slot_t::data](#), [cache_slot_t::dirty](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [blk_meta_t::magic](#), [cache_slot_t::meta](#), [cache_slot_t::meta_dirty](#), and [blk_meta_t::modified_time](#).

Referenced by [block_word_update\(\)](#), [capsule_clear_blocks\(\)](#), [copy_block_to_ram\(\)](#), [mark_buffer_dirty\(\)](#), and [write_ramdrive_block\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.6 include/cli.h File Reference

```
#include <stdint.h>
#include "log.h"
```

Include dependency graph for cli.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [CLIConfig](#)

Configuration structure from command-line arguments Consolidates all parsed CLI options into a single, cleanly organized struct.

Enumerations

- enum [HeartbeatLogMode](#) { [HEARTBEAT_LOG_OFF](#) = 0 , [HEARTBEAT_LOG_SUMMARY](#) = 1 , [HEARTBEAT_LOG_FULL](#) = 2 }

Heartbeat logging modes for DoE experiments.

Functions

- [CLIConfig cli_parse](#) (int argc, char *argv[])
Parse command-line arguments into [CLIConfig](#).
- void [cli_print_usage](#) (const char *program_name)
Print usage/help information.

7.6.1 Enumeration Type Documentation

7.6.1.1 HeartbeatLogMode

enum [HeartbeatLogMode](#)

Heartbeat logging modes for DoE experiments.

Enumerator

HEARTBEAT_LOG_OFF	
HEARTBEAT_LOG_SUMMARY	
HEARTBEAT_LOG_FULL	

7.6.2 Function Documentation

7.6.2.1 cli_parse()

```
CLIConfig cli_parse (
    int argc,
    char * argv[] )
```

Parse command-line arguments into [CLIConfig](#).

Parameters

<i>argc</i>	Argument count
<i>argv</i>	Argument vector

Returns

[CLIConfig](#) struct with parsed options, or exits on `-version/-help`

References [CLIConfig::benchmark](#), [CLIConfig::benchmark_iterations](#), [CLIConfig::break_me](#), [cli_print_usage\(\)](#), [CLIConfig::disk_img_path](#), [CLIConfig::doe_experiment](#), [CLIConfig::fail_fast](#), [HEARTBEAT_LOG_FULL](#), [CLIConfig::heartbeat_log_mode](#), [HEARTBEAT_LOG_OFF](#), [HEARTBEAT_LOG_SUMMARY](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [CLIConfig::log_level](#), [CLIConfig::log_level_explicitly_set](#), [LOG_NONE](#), [LOG_TEST](#), [LOG_WARN](#), [mode_str\(\)](#), [parse_u32\(\)](#), [CLIConfig::ram_disk_mb](#), [CLIConfig::run_tests](#), [CLIConfig::run_tests_flag_observed](#), and [CLIConfig::script_mode](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.6.2.2 cli_print_usage()

```
void cli_print_usage (
    const char * program_name )
```

Print usage/help information.

Parameters

<code>program_name</code>	Name of the executable
---------------------------	------------------------

Referenced by [cli_parse\(\)](#).

Here is the caller graph for this function:

7.7 include/dictionary_heat_optimization.h File Reference

```
#include "vm.h"
```

Include dependency graph for dictionary_heat_optimization.h: This graph shows which files directly or indirectly include this file:

Macros

- `#define` [SF_FC_BUCKETS](#) 256

Functions

- void [dict_update_heat_percentiles](#) ([VM](#) *[vm](#))
Calculate heat percentile thresholds for all words in dictionary.
- void [dict_reorganize_buckets_by_heat](#) ([VM](#) *[vm](#))
Reorganize dictionary buckets by inference-driven priorities.
- [DictEntry](#) * [dict_find_word_heat_aware](#) ([VM](#) *[vm](#), const char *[name](#), size_t [len](#))
Find word with heat-aware search strategy.
- void [dict_adaptive_optimization_pass](#) ([VM](#) *[vm](#))
Periodically reoptimize dictionary based on latest heat patterns.

Variables

- [DictEntry](#) ** [sf_fc_list](#) [256]
- size_t [sf_fc_count](#) [256]
- size_t [sf_fc_cap](#) [256]

7.7.1 Macro Definition Documentation

7.7.1.1 SF_FC_BUCKETS

```
#define SF_FC_BUCKETS 256
```

7.7.2 Function Documentation

7.7.2.1 dict_adaptive_optimization_pass()

```
void dict_adaptive_optimization_pass (
    VM * vm )
```

Periodically reoptimize dictionary based on latest heat patterns.

Called every ~1 second or every N executions to adapt lookup strategy to current workload patterns from rolling window.

References [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [VM::last_bucket_reorg_ns](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::lookup_strategy](#), [VM::rolling_window](#), [rolling_window_measure_diversity\(\)](#), [sf_monotonic_ns\(\)](#), and [vm](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.7.2.2 dict_find_word_heat_aware()

```
DictEntry * dict_find_word_heat_aware (
    VM * vm,
    const char * name,
    size_t len )
```

Find word with heat-aware search strategy.

Instead of newest-first, searches buckets in order:

1. Top 25% by heat (likely hits)
2. Middle 50% by heat
3. Bottom 25% (rarely executed)

References [DictEntry::execution_heat](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_75th](#), [name](#), [DictEntry::name](#), [DictEntry::name_len](#), [sf_fc_count](#), [sf_fc_list](#), and [vm](#).

Referenced by [vm_find_word\(\)](#).

Here is the caller graph for this function:

7.7.2.3 dict_reorganize_buckets_by_heat()

```
void dict_reorganize_buckets_by_heat (
    VM * vm )
```

Reorganize dictionary buckets by inference-driven priorities.

Uses rolling window co-execution patterns to reorder words within buckets. Hot/frequently co-executing words are placed first for faster lookup.

References [compare_dict_entry_heat_desc\(\)](#), [VM::dict_lock](#), [VM::last_bucket_reorg_ns](#), [LOG_DEBUG](#), [log_message\(\)](#), [LOG_WARN](#), [sf_fc_cap](#), [sf_fc_count](#), [sf_fc_list](#), [sf_monotonic_ns\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), and [forth_REORG_BUCKETS\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.7.2.4 dict_update_heat_percentiles()

```
void dict_update_heat_percentiles (
    VM * vm )
```

Calculate heat percentile thresholds for all words in dictionary.

Computes 25th, 50th, 75th percentiles of [execution_heat](#) across all dictionary entries. Used to partition lookup search order.

References [compare_heat_ascending\(\)](#), [VM::dict_lock](#), [DICTIONARY_SIZE](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::latest](#), [LOG_DEBUG](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#), [forth_REORG_BUCKETS\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.7.3 Variable Documentation

7.7.3.1 sf_fc_cap

```
size_t sf_fc_cap[256] [extern]
```

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [sf_fc_clear_all\(\)](#), and [sf_fc_reserve\(\)](#).

7.7.3.2 sf_fc_count

```
size_t sf_fc_count[256] [extern]
```

Referenced by [dict_find_word_heat_aware\(\)](#), and [dict_reorganize_buckets_by_heat\(\)](#).

7.7.3.3 sf_fc_list

```
DictEntry** sf_fc_list[256] [extern]
```

Referenced by [dict_find_word_heat_aware\(\)](#), and [dict_reorganize_buckets_by_heat\(\)](#).

7.8 include/dictionary_management.h File Reference

```
#include "vm.h"
```

Include dependency graph for dictionary_management.h:

Functions

- void [dictionary_init](#) (VM *vm)
Initialize the Forth dictionary.
- void [dictionary_register](#) (VM *vm, const char *name, word_func_t func)
Register a new word in the dictionary.
- [word_func_t](#) [dictionary_lookup](#) (VM *vm, const char *name)
Look up a word in the dictionary by name.
- void [dictionary_clear](#) (VM *vm)
Clear the entire dictionary.

7.8.1 Function Documentation

7.8.1.1 dictionary_clear()

```
void dictionary_clear (
    VM * vm )
```

Clear the entire dictionary.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Removes all words from the dictionary and frees associated memory. This is an optional operation that may be used during cleanup.

7.8.1.2 dictionary_init()

```
void dictionary_init (
    VM * vm )
```

Initialize the Forth dictionary.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Initializes all dictionary structures and prepares them for use. Must be called before any other dictionary operations.

7.8.1.3 dictionary_lookup()

```
word_func_t dictionary_lookup (
    VM * vm,
    const char * name )
```

Look up a word in the dictionary by name.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Name of the word to look up

Returns

`word_func_t` Function pointer to the word's implementation or NULL if not found

7.8.1.4 dictionary_register()

```
void dictionary_register (
    VM * vm,
    const char * name,
    word_func_t func )
```

Register a new word in the dictionary.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Name of the word to register
<i>func</i>	Function pointer to the word's implementation

Adds a new word to the dictionary with the specified name and implementation.

7.9 include/doe_metrics.h File Reference

Design of Experiments (DoE) metrics collection API.

```
#include <stdio.h>
#include <stdint.h>
#include <time.h>
#include "vm.h"
```

Include dependency graph for `doe_metrics.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [DoeMetrics](#)
Complete metrics snapshot for a single DoE run.

Functions

- [DoeMetrics metrics_from_vm](#) ([VM](#) *vm, uint64_t workload_duration_ns, int32_t cpu_temp_delta_c, int32_t cpu_freq_delta_mhz)
Extract complete metrics snapshot from VM.
- void [metrics_write_csv_header](#) (FILE *out)
Write CSV header row.
- void [metrics_write_csv_row](#) (FILE *out, const [DoeMetrics](#) *metrics)
Write metrics as CSV row.
- void [metrics_write_james_law_csv_row](#) (FILE *out, const [DoeMetrics](#) *metrics, [VM](#) *vm)
Write reduced James Law CSV row (20 columns)
- void [metrics_print_text](#) (FILE *out, const [DoeMetrics](#) *metrics)
Print metrics as human-readable text.
- int32_t [metrics_get_cpu_temp_c](#) (void)
Get current CPU temperature (Celsius)
- int32_t [metrics_get_cpu_freq_mhz](#) (void)
Get current CPU frequency (MHz)
- void [metrics_get_timestamp](#) (char *buf, size_t bufsize)
Get current timestamp as ISO 8601 string.

7.9.1 Detailed Description

Design of Experiments (DoE) metrics collection API.

Provides C-level programmatic access to [VM](#) metrics for physics-based experiments. Enables efficient in-process data collection without string parsing overhead.

Usage: struct [DoeMetrics](#) metrics = metrics_from_vm(vm); metrics_write_csv_row(stdout, &metrics, run_number);

7.9.2 Function Documentation

7.9.2.1 metrics_from_vm()

```
DoeMetrics metrics_from_vm (
    VM * vm,
    uint64_t workload_duration_ns,
    int32_t cpu_temp_delta_c,
    int32_t cpu_freq_delta_mhz )
```

Extract complete metrics snapshot from [VM](#).

Collects all metrics from current [VM](#) state after a test harness run. Workload duration and CPU deltas must be supplied by caller.

Parameters

<i>vm</i>	Pointer to Forth VM instance
<i>workload_duration_ns</i>	Actual VM workload execution time (nanoseconds)
<i>cpu_temp_delta_c</i>	CPU temperature change during run (°C)
<i>cpu_freq_delta_mhz</i>	CPU frequency change during run (MHz)

Returns

Populated metrics structure (timestamp filled in)

Extract complete metrics snapshot from [VM](#).

References [DoeMetrics::actual_window_size](#), [DoeMetrics::adaptive_shrink_rate](#), [DoeMetrics::avg_word_heat](#), [DoeMetrics::cache_demotions](#), [DoeMetrics::cache_promotions](#), [DoeMetrics::ci_lower_95](#), [DoeMetrics::ci_upper_95](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DECAY_MIN_INTERVAL](#), [DoeMetrics::decay_min_inter](#), [DECAY_RATE_PER_US_Q16](#), [DoeMetrics::decay_rate_q16](#), [DoeMetrics::decay_slope](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [DoeMetrics::dictionary_lookups](#), [HeartbeatState::dictionary_lookups](#), [DoeMetrics::early_exit_count](#), [HeartbeatState::early_exit_count](#), [RollingWindowOfTruth::effective_window_size](#), [DoeMetrics::enable_hotwords_cache](#), [ENABLE_HOTWORDS_CACHE](#), [DoeMetrics::enable_loop_1_heat_tracking](#), [DoeMetrics::enable_loop_2_rolling_window](#), [DoeMetrics::enable_loop_3_linear_decay](#), [DoeMetrics::enable_loop_4_pipelining](#), [DoeMetrics::enable_loop_5_window_inference](#), [DoeMetrics::enable_loop_6_decay_inference](#), [DoeMetrics::enable_loop_7_adaptive_heartate](#), [DoeMetrics::enable_pipelining](#), [ENABLE_PIPELINING](#), [HotwordsCache_s::enabled](#), [HotwordsStats::evictions](#), [extract_cache_metrics\(\)](#), [DoeMetrics::final_effective_v](#), [VM::heartbeat](#), [DoeMetrics::heat_cache_demotion_threshold](#), [DoeMetrics::hot_word_count](#), [VM::hotwords_cache](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [DoeMetrics::inference_run_count](#), [HeartbeatState::inference_run_count](#), [VM::last_inference_outputs](#), [VM::latest](#), [DoeMetrics::memory_allocated_bytes](#), [VM::pipeline_metrics](#), [DoeMetrics::prefetch_accuracy](#), [DoeMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_attempts](#), [DoeMetrics::prefetch_hits](#), [PipelineGlobalMetrics::prefetch](#), [HotwordsStats::promotions](#), [VM::rolling_window](#), [DoeMetrics::rolling_window_size](#), [ROLLING_WINDOW_SIZE](#), [DoeMetrics::rolling_window_width](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [DoeMetrics::speedup_vs_baseline](#), [DoeMetrics::stale_word_count](#), [DoeMetrics::stale_word_ratio](#), [HotwordsCache_s::stats](#), [DoeMetrics::tick_count](#), [HeartbeatState::tick_count](#), [DoeMetrics::tick_target_ns](#), [HeartbeatState::tick_target_ns](#), [DoeMetrics::total_executions](#), [RollingWindowOfTruth::total_executions](#), [DoeMetrics::total_heat](#), [DoeMetrics::total_lookups](#), [HotwordsStats::total_lookups](#), [DoeMetrics::total_runtime_ms](#), [VM::tuning_lock](#), [vm](#), [DoeMetrics::vm_workload_duration_ns_q48](#), [DoeMetrics::window_diversity_per](#), [DoeMetrics::window_final_size_bytes](#), [DoeMetrics::window_tuning_checks](#), [PipelineGlobalMetrics::window_tuning_checks](#), [DoeMetrics::window_variance_q48](#), [InferenceOutputs::window_variance_q48](#), [word_count](#), [DoeMetrics::words_executed](#), and [HeartbeatState::words_executed](#).

Referenced by [run_doe_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.9.2.2 metrics_get_cpu_freq_mhz()

```
int32_t metrics_get_cpu_freq_mhz (
    void )
```

Get current CPU frequency (MHz)

Tries multiple sources:

1. /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq
2. /proc/cpuinfo Returns 0 if unavailable.

Returns

Frequency in MHz, or 0 if unable to read

Get current CPU frequency (MHz)

Referenced by [run_doe_experiment\(\)](#).

Here is the caller graph for this function:

7.9.2.3 metrics_get_cpu_temp_c()

```
int32_t metrics_get_cpu_temp_c (
    void )
```

Get current CPU temperature (Celsius)

Reads /sys/class/thermal/thermal_zone0/temp on Linux, returns 0 if unavailable.

Returns

Temperature in °C, or 0 if unable to read

Get current CPU temperature (Celsius)

Referenced by [run_doe_experiment\(\)](#).

Here is the caller graph for this function:

7.9.2.4 metrics_get_timestamp()

```
void metrics_get_timestamp (
    char * buf,
    size_t bufsize )
```

Get current timestamp as ISO 8601 string.

Format: "YYYY-MM-DDTHH:MM:SS"

Parameters

<i>buf</i>	Buffer to write timestamp (minimum 32 bytes)
<i>bufsize</i>	Size of buffer

7.9.2.5 metrics_print_text()

```
void metrics_print_text (
    FILE * out,
    const DoeMetrics * metrics )
```

Print metrics as human-readable text.

Outputs metrics in formatted text for debugging/verification.

Parameters

<i>out</i>	Output file stream
<i>metrics</i>	Metrics structure to print

References [DoeMetrics::bucket_hit_percent](#), [DoeMetrics::bucket_hits](#), [DoeMetrics::bucket_search_latency_ns](#), [DoeMetrics::bucket_search_stddev_ns](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_percent](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::cache_hits](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DoeMetrics::decay_slope](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::total_lookups](#), and [DoeMetrics::vm_workload_duration_ns](#).

7.9.2.6 metrics_write_csv_header()

```
void metrics_write_csv_header (
    FILE * out )
```

Write CSV header row.

Writes the 35-column CSV header to output stream.

Parameters

<i>out</i>	Output file stream
------------	--------------------

Write CSV header row.

7.9.2.7 metrics_write_csv_row()

```
void metrics_write_csv_row (
    FILE * out,
    const DoeMetrics * metrics )
```

Write metrics as CSV row.

Writes a single run's metrics as comma-separated values.

Parameters

<i>out</i>	Output file stream
<i>metrics</i>	Metrics structure to write

Write metrics as CSV row.

References [DoeMetrics::actual_window_size](#), [DoeMetrics::adaptive_shrink_rate](#), [DoeMetrics::avg_word_heat](#), [DoeMetrics::bucket_hit_percent](#), [DoeMetrics::bucket_hits](#), [DoeMetrics::bucket_search_latency_ns](#), [DoeMetrics::bucket_search_stddev_ns](#), [DoeMetrics::cache_demotions](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_percent](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::cache_hits](#), [DoeMetrics::cache_promotions](#), [DoeMetrics::ci_lower_95](#), [DoeMetrics::ci_upper_95](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DoeMetrics::decay_min_interval_ns](#), [DoeMetrics::decay_rate_q16](#), [DoeMetrics::decay_slope](#), [DoeMetrics::dictionary_lookups](#), [DoeMetrics::early_exit_count](#), [DoeMetrics::enable_hotwords_cache](#), [DoeMetrics::enable_loop_1_heat_tracking](#), [DoeMetrics::enable_loop_2_rolling_window](#), [DoeMetrics::enable_loop_3_linear_decay](#), [DoeMetrics::enable_loop_4_pipelining](#), [DoeMetrics::enable_loop_5_window_inference](#), [DoeMetrics::enable_loop_6_decay_inference](#), [DoeMetrics::enable_loop_7_adaptive_heartrate](#), [DoeMetrics::enable_pipelining](#), [DoeMetrics::final_effective_window_size](#), [DoeMetrics::heat_cache_demotion_threshold](#), [DoeMetrics::hot_word_count](#), [DoeMetrics::inference_run_count](#), [DoeMetrics::memory_allocated_bytes](#), [DoeMetrics::prefetch_accuracy_percent](#), [DoeMetrics::prefetch_attempts](#), [DoeMetrics::prefetch_hits](#), [DoeMetrics::rolling_window_size](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::speedup_vs_baseline](#), [DoeMetrics::stale_word_count](#), [DoeMetrics::stale_word_ratio](#), [DoeMetrics::tick_count](#), [DoeMetrics::tick_target_ns](#), [DoeMetrics::total_executions](#), [DoeMetrics::total_heat](#), [DoeMetrics::total_lookups](#), [DoeMetrics::total_runtime_ms](#), [DoeMetrics::vm_workload_duration_ns_q48](#), [DoeMetrics::window_diversity_percent](#), [DoeMetrics::window_final_size_bytes](#), [DoeMetrics::window_tuning_checks](#), [DoeMetrics::window_variance_q48](#), and [DoeMetrics::words_executed](#).

7.9.2.8 metrics_write_james_law_csv_row()

```
void metrics_write_james_law_csv_row (
    FILE * out,
    const DoeMetrics * metrics,
    VM * vm )
```

Write reduced James Law CSV row (20 columns)

Outputs only the subset of metrics needed for the James Law validation experiment. This matches the header format in `run_window_sweep.sh` (minus the 5 script-prepended columns).

Parameters

<i>out</i>	Output file stream
<i>metrics</i>	Metrics structure to write
<i>vm</i>	VM instance (for heartbeat bucket aggregates and L8 mode)

Write reduced James Law CSV row (20 columns)

This outputs only the subset of metrics needed for the James Law validation experiment. Format matches the header in `run_window_sweep.sh` (minus the 5 script-prepended columns).

Expected columns (20): `workload_duration_ns`, `l8_mode_final`, `lambda_effective`, `K_statistic`, `K_deviation`, `entropy`, `cv`, `temporal_decay`, `stability_score`, `ns_per_word`, `total_runtime_ms`, `window_final`, `hot_word_count`, `avg_heat`, `bucket_mean_K`, `bucket_cv_K`, `bucket_window_var`, `bucket_heat_var`, `bucket_mode_transitions`, `bucket_collapse_flag`

References [DoeMetrics::avg_word_heat](#), [HeartbeatState::bucket_collapse_flag](#), [HeartbeatState::bucket_mode_transitions](#), [HeartbeatState::bucket_sum_heat_variance](#), [HeartbeatState::bucket_sum_K](#), [HeartbeatState::bucket_sum_K_squared](#), [HeartbeatState::bucket_sum_window_variance](#), [HeartbeatState::bucket_tick_count](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::decay_slope](#), [DoeMetrics::final_effective_window_size](#), [VM::heartbeat](#), [HEARTBEAT_TICK_BUFFER_SIZE](#), [DoeMetrics::hot_word_count](#), [isqrt\(\)](#), [HeartbeatTickSnapshot::l8_mode](#), [DoeMetrics::rolling_window_size](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::stale_word_ratio](#), [HeartbeatState::tick_buffer](#), [HeartbeatState::tick_buffer_write_index](#), [DoeMetrics::total_runtime_ms](#), [vm](#), [DoeMetrics::vm_workload_duration_ns_q48](#), [DoeMetrics::window_diversity_percent](#), and [DoeMetrics::words_executed](#).

Here is the call graph for this function:

7.10 include/inference_engine.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "vm.h"
#include "q48_16.h"
```

Include dependency graph for `inference_engine.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [InferenceInputs](#)
- struct [InferenceOutputs](#)

Functions

- void [inference_engine_run](#) ([InferenceInputs](#) *inputs, [InferenceOutputs](#) *outputs)
Run adaptive inference engine on window data.
- const char * [inference_outputs_to_string](#) (const [InferenceOutputs](#) *outputs)
Pretty-print inference outputs for logging.
- int [inference_outputs_validate](#) (const [InferenceOutputs](#) *outputs)
Validate inference outputs for sanity.
- [q48_16_t](#) [compute_variance_q48](#) (const uint64_t *heat_data, uint64_t length)
Compute variance of heat data in Q48.16 format.
- uint32_t [find_variance_inflection](#) (const uint64_t *heat_data, uint64_t trajectory_length, [q48_16_t](#) full_↵ variance)
Find variance inflection point for optimal window width.
- uint64_t [infer_decay_slope_q48](#) (const uint64_t *heat_data, uint64_t length)
Infer decay slope from heat trajectory using linear regression.

7.10.1 Function Documentation

7.10.1.1 compute_variance_q48()

```
q48_16_t compute_variance_q48 (
    const uint64_t * heat_data,
    uint64_t length )
```

Compute variance of heat data in Q48.16 format.

Parameters

<i>heat_data</i>	Array of heat values
<i>length</i>	Number of elements in array

Returns

Variance in Q48.16 format

References [q48_div\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), and [q48_to_u64\(\)](#).

Referenced by [compute_levene_statistic\(\)](#), [find_variance_inflection\(\)](#), and [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.10.1.2 find_variance_inflection()

```
uint32_t find_variance_inflection (
    const uint64_t * heat_data,
    uint64_t trajectory_length,
    q48_16_t full_variance )
```

Find variance inflection point for optimal window width.

Parameters

<i>heat_data</i>	Array of heat values
<i>trajectory_length</i>	Total trajectory length
<i>full_variance</i>	Variance of full trajectory

Returns

Optimal window width

References [ADAPTIVE_MIN_WINDOW_SIZE](#), [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), [q48_from_double\(\)](#), and [ROLLING_WINDOW_SIZE](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.10.1.3 infer_decay_slope_q48()

```
uint64_t infer_decay_slope_q48 (
    const uint64_t * heat_data,
    uint64_t length )
```

Infer decay slope from heat trajectory using linear regression.

Parameters

<i>heat_data</i>	Array of heat values
<i>length</i>	Number of elements

Returns

Decay slope in Q48.16 format

References [q48_add\(\)](#), [q48_from_u64\(\)](#), [q48_log_approx\(\)](#), and [q48_mul\(\)](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.10.1.4 inference_engine_run()

```
void inference_engine_run (
    InferenceInputs * inputs,
    InferenceOutputs * outputs )
```

Run adaptive inference engine on window data.

High-level flow:

1. ANOVA Check: Is variance delta < 5%?
 - YES: Return cached outputs immediately (cost: ~100 cycles)
 - NO: Continue to full inference
2. Extract trajectory: Fresh execution_heat snapshot from dictionary
3. Window Width Inference:
 - Compute variance across entire window
 - Detect inflection point (where $d(\text{variance})/d(\text{size}) \rightarrow 0$)
 - Return optimal window size
4. Decay Slope Inference:
 - Transform trajectory to log space ($\ln(\text{heat}[t])$)
 - Fit linear regression: $\ln(\text{heat}) = a - \text{slope} * t$
 - Extract slope coefficient in Q48.16
5. Diagnostics:
 - Compute fit quality (R^2 or residual)
 - Store variance metric for next early-exit check

Parameters

<i>inputs</i>	Snapshot of window + metrics (caller-provided)
<i>outputs</i>	Tuning parameters + diagnostics (caller-provided, will be updated)

Returns

void (results_run_01_2025_12_08 in outputs struct)

Preconditions:

- inputs != NULL
- outputs != NULL
- inputs->window != NULL
- inputs->window->is_warm == 1 (window has enough data)
- inputs->trajectory_length > 1

Side Effects:

- Modifies outputs struct (caller must preserve if caching)
- Allocates temporary trajectory buffer (freed on return)

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [compute_fit_quality\(\)](#), [compute_variance_q48\(\)](#), [InferenceOutputs::early_exited](#), [extract_heat_trajectory\(\)](#), [find_variance_inflection\(\)](#), [has_variance_stabilized\(\)](#), [infer_decay_slope_q48\(\)](#), [ssm_config_t::L5_window_inference](#), [ssm_config_t::L6_decay_inference](#), [InferenceOutputs::slope_fit_quality_q48](#), [VM::ssm_config](#), [InferenceInputs::vm](#), [InferenceInputs::window](#), and [InferenceOutputs::window_variance_q48](#).

Referenced by [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.10.1.5 inference_outputs_to_string()

```
const char * inference_outputs_to_string (
    const InferenceOutputs * outputs )
```

Pretty-print inference outputs for logging.

Format: "window_width=XXX variance_q48=YYYY slope_q48=ZZZZ fit_quality=WWWW"

Parameters

<i>outputs</i>	Inference outputs struct
----------------	--------------------------

Returns

Pointer to static string (valid until next call)

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [InferenceOutputs::early_exited](#), [q48_to_double\(\)](#), [InferenceOutputs::slope_fit_quality_q48](#), and [InferenceOutputs::window_variance_q48](#).

Here is the call graph for this function:

7.10.1.6 inference_outputs_validate()

```
int inference_outputs_validate (
    const InferenceOutputs * outputs )
```

Validate inference outputs for sanity.

Checks:

- window_width is within [ADAPTIVE_MIN_WINDOW_SIZE, ROLLING_WINDOW_SIZE]
- slope_q48 is reasonable (between 0.001 and 100 in Q48.16)
- variance and fit_quality are between 0.0 and 1.0 in Q48.16

Parameters

<i>outputs</i>	Inference outputs to validate
----------------	-------------------------------

Returns

1 if valid, 0 if suspicious

References [InferenceOutputs::adaptive_decay_slope](#), [ADAPTIVE_MIN_WINDOW_SIZE](#), [InferenceOutputs::adaptive_window_width](#), [q48_from_u64\(\)](#), [ROLLING_WINDOW_SIZE](#), and [InferenceOutputs::slope_fit_quality_q48](#).

Referenced by [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.11 include/io.h File Reference

```
#include <stdint.h>
```

```
#include "vm.h"
```

Include dependency graph for io.h: This graph shows which files directly or indirectly include this file:

Macros

- #define `BLOCK_COUNT` 1024
Total number of blocks available.
- #define `MEMORY_SIZE` (`BLOCK_SIZE` * `BLOCK_COUNT`)
Total memory size in bytes.

Functions

- void `io_init` (`VM` *`vm`)
Initialize the I/O subsystem.
- int `io_read_block` (`VM` *`vm`, int `block_num`, unsigned char *`buffer`)
Read a block from storage.
- int `io_write_block` (`VM` *`vm`, int `block_num`, const unsigned char *`buffer`)
Write a block to storage.

7.11.1 Macro Definition Documentation

7.11.1.1 BLOCK_COUNT

```
#define BLOCK_COUNT 1024
```

Total number of blocks available.

7.11.1.2 MEMORY_SIZE

```
#define MEMORY_SIZE (BLOCK_SIZE * BLOCK_COUNT)
```

Total memory size in bytes.

7.11.2 Function Documentation

7.11.2.1 io_init()

```
void io_init (
    VM * vm )
```

Initialize the I/O subsystem.

Parameters

<code>vm</code>	Pointer to the virtual machine instance
-----------------	---

Initialize the I/O subsystem.

Parameters

<code>vm</code>	Pointer to <code>VM</code> instance
-----------------	-------------------------------------

Initializes the block device view over the unified `VM` memory. Zeros out the entire `VM` memory space. References `BLOCK_SIZE`, `LOG_ERROR`, `LOG_INFO`, `log_message()`, `MAX_BLOCKS`, `VM::memory`, `vm`, and `VM_MEMORY_SIZE`.

Here is the call graph for this function:

7.11.2.2 io_read_block()

```
int io_read_block (
    VM * vm,
```

```
int block_num,
unsigned char * buffer )
```

Read a block from storage.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>block_num</i>	Block number to read
<i>buffer</i>	Buffer to store the read data

Returns

0 on success, non-zero on failure

7.11.2.3 io_write_block()

```
int io_write_block (
    VM * vm,
    int block_num,
    const unsigned char * buffer )
```

Write a block to storage.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>block_num</i>	Block number to write
<i>buffer</i>	Buffer containing data to write

Returns

0 on success, non-zero on failure

Write a block to storage.

Parameters

<i>vm</i>	Pointer to VM instance
<i>block_num</i>	Block number to write (0 to MAX_BLOCKS-1)
<i>buffer</i>	Source buffer containing block data

Returns

0 on success, -1 on error

Write a block to [VM](#) memory

Parameters

<i>vm</i>	Pointer to VM instance
<i>block_num</i>	Block number to write (0 to MAX_BLOCKS-1)
<i>buffer</i>	Source buffer containing block data

Returns

0 on success, -1 on error

Writes a BLOCK_SIZE chunk of data from the provided buffer into the VM's memory at the specified block number offset. The block number must be within valid range. The entire block is written atomically.

Note

The buffer must contain at least BLOCK_SIZE bytes of valid data

References BLOCK_SIZE, LOG_DEBUG, LOG_ERROR, log_message(), MAX_BLOCKS, VM::memory, and vm. Here is the call graph for this function:

7.12 include/log.h File Reference

This graph shows which files directly or indirectly include this file:

Macros

- #define LOG_LINE_MAX 256

Enumerations

- enum LogLevel {
LOG_NONE = -1 , LOG_ERROR = 0 , LOG_WARN , LOG_INFO ,
LOG_TEST , LOG_DEBUG }
Logging level enumeration.
- enum TestResult { TEST_PASS = 0 , TEST_FAIL , TEST_SKIP , TEST_ERROR }
Test result type enumeration.

Functions

- void log_set_level (LogLevel level)
Set the global logging level.
- LogLevel log_get_level (void)
Get the current logging level.
- void log_message (LogLevel level, const char *fmt,...)
Log a formatted message at the specified level.
- void log_test_result (const char *word_name, TestResult result)
Log a test result with colored output.
- void log_set_vm (struct VM *vm)
Set the VM instance for persistent logging.

7.12.1 Macro Definition Documentation

7.12.1.1 LOG_LINE_MAX

```
#define LOG_LINE_MAX 256
```

7.12.2 Enumeration Type Documentation

7.12.2.1 LogLevel

```
enum LogLevel
```

Logging level enumeration.

Defines the available logging levels in order of increasing verbosity.

Enumerator

LOG_NONE	Completely disable all logging.
LOG_ERROR	Error messages only.
LOG_WARN	Warning and error messages.
LOG_INFO	Informational, warning and error messages.
LOG_TEST	Test results_run_01_2025_12_08 and all previous levels.
LOG_DEBUG	Debug messages and all previous levels.

7.12.2.2 TestResult

```
enum TestResult
```

Test result type enumeration.

Defines the possible outcomes of test execution.

Enumerator

TEST_PASS	Test completed successfully.
TEST_FAIL	Test failed.
TEST_SKIP	Test was skipped.
TEST_ERROR	Test encountered an error during execution.

7.12.3 Function Documentation

7.12.3.1 log_get_level()

```
LogLevel log_get_level (
    void )
```

Get the current logging level.

Returns

The current logging level

Get the current logging level.

Returns

LogLevel Current logging level

References [current_level](#).

7.12.3.2 log_message()

```
void log_message (
    LogLevel level,
    const char * fmt,
    ... )
```

Log a formatted message at the specified level.

Parameters

<i>level</i>	The logging level for this message
<i>fmt</i>	Printf-style format string
...	Variable arguments for format string

Log a formatted message at the specified level.

Parameters

<i>level</i>	The LogLevel for this message
<i>fmt</i>	Printf-style format string
...	Variable arguments for format string

References [color_reset](#), [current_level](#), [get_timestamp\(\)](#), [level](#), [level_colors](#), [level_names](#), [LOG_NONE](#), and [SF_TIME_STAMP_SIZE](#).

Referenced by [__attribute__\(\)](#), [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [assert_heartbeat_safe\(\)](#), [assert_stack_depth\(\)](#), [assert_stack_top\(\)](#), [assert_vm_error\(\)](#), [blk_subsys_attach_device\(\)](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [cf_push_item\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_branch\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_EXIT\(\)](#), [control_forth_I\(\)](#), [control_forth_if\(\)](#), [control_forth_J\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_then\(\)](#), [control_forth_UNLOOP\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_does\(\)](#), [defining_word_immediate\(\)](#), [defining_word_left_bracket\(\)](#), [defining_word_literal\(\)](#), [defining_word_right_bracket\(\)](#), [defining_word_variable\(\)](#), [dict_adaptive_optimization_pass\(\)](#), [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_word_forget\(\)](#), [dump_and_die\(\)](#), [editor_word_edit\(\)](#), [editor_word_I\(\)](#), [editor_word_s\(\)](#), [editor_word_show\(\)](#), [emit_cell\(\)](#), [enable_benchmark_mode\(\)](#), [execute_colon_word\(\)](#), [fail_and_exit\(\)](#), [format_word_hold\(\)](#), [forth_acl_allow_store\(\)](#), [forth_acl_inherit\(\)](#), [forth_acl_mode_store\(\)](#), [forth_acl_ttl_store\(\)](#), [forth_paren\(\)](#), [init_blkio\(\)](#), [init_report\(\)](#), [init_vocabulary_system\(\)](#), [io_init\(\)](#), [io_word_type\(\)](#), [io_write_block\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_true\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equals\(\)](#), [main\(\)](#), [memory_word_cells\(\)](#), [mixed_math_word_mod\(\)](#), [pop_xt\(\)](#), [print_module_summary\(\)](#), [read_devblock_4k\(\)](#), [register_arithmetic_words\(\)](#), [register_forth79_words\(\)](#), [register_word\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [return_stack_word_to_r\(\)](#), [rolling_window_run_adaptive_pass\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [run_acl_words_tests\(\)](#), [run_all_tests\(\)](#), [run_arithmetic_words_tests\(\)](#), [run_benchmark_mode\(\)](#), [run_block_words_tests\(\)](#), [run_break_me_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#), [run_dictionary_words_tests\(\)](#), [run_doe_experiment\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#), [run_integration_tests\(\)](#), [run_io_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_module_tests\(\)](#), [run_return_stack_words_tests\(\)](#), [run_single_test\(\)](#), [run_stack_words_tests\(\)](#), [run_starforth_words_tests\(\)](#), [run_stress_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), [run_vocabulary_words_tests\(\)](#), [run_word_tests\(\)](#), [stack_word_depth\(\)](#), [stack_word_drop\(\)](#), [stack_word_dup\(\)](#), [stack_word_minus_rot\(\)](#), [stack_word_over\(\)](#), [stack_word_pick\(\)](#), [stack_word_question_dup\(\)](#), [stack_word_roll\(\)](#), [stack_word_rot\(\)](#), [stack_word_swap\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_execution_heat_store\(\)](#), [starforth_word_init\(\)](#), [starforth_word_paren_dash\(\)](#), [starforth_word_random\(\)](#), [starforth_word_seed\(\)](#), [starforth_word_top_words\(\)](#), [starforth_word_wait\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_count\(\)](#), [string_word_expect\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_strict_mode\(\)](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_assert_interpreter_enabled\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_compile_call\(\)](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_track_entry\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_get_block_addr\(\)](#), [vm_heartbeat_start_thread\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_interpret_word\(\)](#), [vm_make_immediate\(\)](#), [vm_pin_execution_heat\(\)](#), [vm_pop\(\)](#), [vm_push\(\)](#), [vm_repl\(\)](#), [vm_smudge_word\(\)](#), [vm_tick_inference_engine\(\)](#), [vm_tick_slope_validator\(\)](#), [vm_tick_window_tuner\(\)](#), [vm_unpin_execution_heat\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), [vocabulary_select_runtime\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), [vocabulary_word_vocabulary\(\)](#), [word_divide\(\)](#), [word_divmod\(\)](#), and [write_devblock_4k\(\)](#).

Here is the call graph for this function:

7.12.3.3 log_set_level()

```
void log_set_level (
    LogLevel level )
```

Set the global logging level.

Parameters

<i>level</i>	The new logging level to set
--------------	------------------------------

Note

Default level is LOG_INFO

Set the global logging level.

Parameters

<i>level</i>	The LogLevel to set as current logging level
--------------	--

References [current_level](#), and [level](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

7.12.3.4 log_set_vm()

```
void log_set_vm (
    struct VM * vm )
```

Set the [VM](#) instance for persistent logging.

Parameters

<i>vm</i>	VM instance to use for persistent logging (NULL to disable)
-----------	---

7.12.3.5 log_test_result()

```
void log_test_result (
    const char * word_name,
    TestResult result )
```

Log a test result with colored output.

Parameters

<i>word_name</i>	Name of the word being tested
<i>result</i>	Result of the test execution

Log a test result with colored output.

Parameters

<i>word_name</i>	Name of the word being tested
<i>result</i>	Test result (PASS/FAIL/SKIP)

References [color_reset](#), [current_level](#), [get_timestamp\(\)](#), [LOG_NONE](#), [LOG_TEST](#), [SF_TIME_STAMP_SIZE](#), [TEST_FAIL](#), [test_fail_color](#), [TEST_PASS](#), [test_pass_color](#), [TEST_SKIP](#), and [test_skip_color](#).

Referenced by [run_single_test\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.13 include/math_portable.h File Reference

```
#include <stdint.h>
```

Include dependency graph for math_portable.h: This graph shows which files directly or indirectly include this file:

Macros

- `#define Q48_SCALE (1LL << 16) /* 65536 */`

Functions

- `int64_t sqrt_q48 (int64_t x)`
Square root in Q48.16 fixed-point Input: x in Q48.16 format Output: sqrt(x) in Q48.16 format Uses Newton-Raphson: $x_{n+1} = (x_n + a/x_n) / 2$ Converges in ~ 10 -15 iterations Returns 0 for negative inputs (guard against domain error)
- `int64_t erf_q48 (int64_t x)`
Error function in Q48.16 fixed-point Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format (range: -Q48_SCALE to +Q48_SCALE) Uses Abramowitz & Stegun approximation (7.1.26) Maximum error: 1.5e-7 relative to floating-point erf Properties:

7.13.1 Macro Definition Documentation

7.13.1.1 Q48_SCALE

```
#define Q48_SCALE (1LL << 16) /* 65536 */
```

7.13.2 Function Documentation

7.13.2.1 erf_q48()

```
int64_t erf_q48 (
    int64_t x )
```

Error function in Q48.16 fixed-point Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format (range: -Q48_SCALE to +Q48_SCALE) Uses Abramowitz & Stegun approximation (7.1.26) Maximum error: 1.5e-7 relative to floating-point erf Properties:

- $\text{erf}(0) = 0$
- $\text{erf}(-x) = -\text{erf}(x)$ (odd function)
- $\text{erf}(x)$ approaches $\pm Q48_SCALE$ for large $|x|$

Error function in Q48.16 fixed-point Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format (range: -Q48_SCALE to +Q48_SCALE) Uses Abramowitz & Stegun approximation (7.1.26) Maximum error: 1.5e-7 relative to floating-point erf Properties:

Uses Abramowitz & Stegun approximation (7.1.26) Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format, clamped to [-Q48_SCALE, +Q48_SCALE]

Maximum error: $\sim 1.5e-7$ relative to true erf

References [base](#), [exp_q48\(\)](#), and [sqrt_q48\(\)](#).

Referenced by [hotwords_speedup_estimate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.13.2.2 sqrt_q48()

```
int64_t sqrt_q48 (
    int64_t x )
```

Square root in Q48.16 fixed-point Input: x in Q48.16 format Output: sqrt(x) in Q48.16 format Uses Newton-Raphson: $x_{n+1} = (x_n + a/x_n) / 2$ Converges in ~ 10 -15 iterations Returns 0 for negative inputs (guard against domain error)

Square root in Q48.16 fixed-point Input: x in Q48.16 format Output: $\sqrt{x}$ in Q48.16 format Uses Newton-Raphson: $x_{n+1} = (x_n + a/x_n) / 2$ Converges in ~ 10 -15 iterations Returns 0 for negative inputs (guard against domain error)

Input: x in Q48.16 format Output: $\sqrt{x}$ in Q48.16 format

Newton-Raphson: $r_{n+1} = (r_n^2 + x) / (2 * r_n)$ Where multiply/divide preserve Q48.16 scaling. Converges in ~ 10 -15 iterations.

Referenced by [calculate_posterior\(\)](#), [erf_q48\(\)](#), [hotwords_speedup_estimate\(\)](#), and [hotwords_stats_print\(\)](#).

Here is the caller graph for this function:

7.14 include/memory_management.h File Reference

Memory management functions for the StarForth virtual machine.

```
#include "vm.h"
```

Include dependency graph for memory_management.h:

Functions

- void * [vm_allot](#) (VM *[vm](#), size_t bytes)
Allocates memory in the virtual machine's memory space.
- void [vm_align](#) (VM *[vm](#))
Aligns the virtual machine's memory pointer to the next word boundary.

7.14.1 Detailed Description

Memory management functions for the StarForth virtual machine.

This file contains functions for managing memory allocation and alignment in the StarForth virtual machine implementation.

7.14.2 Function Documentation

7.14.2.1 vm_align()

```
void vm_align (
    VM * vm )
```

Aligns the virtual machine's memory pointer to the next word boundary.

Parameters

vm	Pointer to the virtual machine instance
--------------------	---

Aligns the virtual machine's memory pointer to the next word boundary.

Parameters

vm	Pointer to the VM instance
--------------------	--

Note

Adds padding bytes if necessary to ensure proper alignment for `cell_t` type

References [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_allot\(\)](#).

Referenced by [defining_runtime_does_rt\(\)](#), [defining_word_create\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_here\(\)](#), [emit_cell\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_enter_compile_mode\(\)](#), and [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.14.2.2 vm_allot()

```
void * vm_allot (
    VM * vm,
    size_t bytes )
```

Allocates memory in the virtual machine's memory space.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>bytes</i>	Number of bytes to allocate

Returns

Pointer to the allocated memory or NULL if allocation fails

Allocates memory in the virtual machine's memory space.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>bytes</i>	Number of bytes to allocate

Returns

Pointer to the allocated memory or NULL if allocation fails

Note

The allocation is done from the [VM](#)'s dictionary space which is limited to `DICTIONARY_MEMORY_SIZE`

References [BLOCK_SIZE](#), [DICTIONARY_MEMORY_SIZE](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), and [vm](#).

Referenced by [defining_runtime_does_rt\(\)](#), [defining_word_variable\(\)](#), [emit_cell\(\)](#), [ensure_pn\(\)](#), [init_vocabulary_system\(\)](#), [string_word_s_quote\(\)](#), [string_word_word\(\)](#), [system_word_abort_quote\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_input_ensure\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.15 include/physics_execution_hooks.h File Reference

```
#include "vm.h"
```

Include dependency graph for `physics_execution_hooks.h`: This graph shows which files directly or indirectly include this file:

Functions

- void [physics_pre_execute](#) ([VM](#) *vm, [DictEntry](#) *word, [DictEntry](#) *prev)
- void [physics_post_execute](#) ([VM](#) *vm, [DictEntry](#) *word)
- void [physics_on_lookup](#) ([VM](#) *vm, [DictEntry](#) *entry, [DictEntry](#) *canon)

7.15.1 Function Documentation

7.15.1.1 physics_on_lookup()

```
void physics_on_lookup (
    VM * vm,
    DictEntry * entry,
    DictEntry * canon )
```

References [VM::dict_lock](#), [entry](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [DictEntry::physics](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [physics_metadata_touch\(\)](#), [sf_monotonic_ns\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.15.1.2 physics_post_execute()

```
void physics_post_execute (
    VM * vm,
    DictEntry * word )
```

References [HeartbeatState::check_counter](#), [DictEntry::execution_heat](#), [VM::heartbeat](#), [HEARTBEAT_CHECK_FREQUENCY](#), [physics_metadata_touch\(\)](#), [profiler_word_exit\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_heartbeat_run_cycle\(\)](#), [HeartbeatState::words_executed](#) and [HeartbeatState::worker](#).

Referenced by [execute_colon_word\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.15.1.3 physics_pre_execute()

```
void physics_pre_execute (
    VM * vm,
    DictEntry * word,
    DictEntry * prev )
```

References [DICTIONARY_SIZE](#), [ENABLE_HOTWORDS_CACHE](#), [ENABLE_PIPELINING](#), [DictEntry::execution_heat](#), [VM::hotwords_cache](#), [hotwords_cache_promote\(\)](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [WordTransitionMetrics::most_likely_next_word_id](#), [DictEntry::physics](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [VM::pipeline_metrics](#), [WordTransitionMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [rolling_window_record_execution\(\)](#), [sf_monotonic_ns\(\)](#), [DictEntry::transition_metrics](#), [transition_metrics_record\(\)](#), [transition_metrics_record_prefetch_hit\(\)](#), [transition_metrics_should_speculate\(\)](#), [transition_metrics_update_cache\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), [word_id](#), and [DictEntry::word_id](#).

Referenced by [execute_colon_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.16 include/physics_hotwords_cache.h File Reference

```
#include <stdint.h>
```

```
#include "vm.h"
```

Include dependency graph for physics_hotwords_cache.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [HotwordsStats](#)
- struct [HotwordsCache_s](#)
- struct [BayesianLatencyPosterior](#)

Bayesian statistics structure for inference and credible intervals All values stored in Q48.16 fixed-point (64-bit signed integer) representing nanoseconds with $2^{-16} \approx 0.0000153$ ns precision.

- struct [SpeedupEstimate](#)

Calculate speedup estimate with credible interval All values in Q48.16 fixed-point format (64-bit signed integer) Probabilities are in range [0, Q48_SCALE] representing [0%, 100%].

Macros

- `#define HOTWORDS_CACHE_SIZE 32`
Maximum number of hot words to cache (typically 16-32)
- `#define HOTWORDS_EXECUTION_HEAT_THRESHOLD 50`
Execution heat threshold to consider a word "hot" (execution count)
- `#define HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD 100`
Reorder bucket when execution heat delta exceeds this (avoid thrashing)

Typedefs

- `typedef struct HotwordsCache_s HotwordsCache`

Functions

- `void hotwords_cache_init (HotwordsCache *cache)`
Initialize hot-words cache Should be called during VM initialization.
- `void hotwords_cache_cleanup (HotwordsCache *cache)`
Cleanup hot-words cache.
- `DictEntry * hotwords_cache_lookup (HotwordsCache *cache, DictEntry **bucket, size_t bucket_count, const char *name, size_t len)`
Lookup word in cache first, then fall back to bucket search Returns the found word, or NULL if not found.
- `void hotwords_cache_promote (HotwordsCache *cache, DictEntry *word)`
Promote a word to the hot-words cache Called when a word's execution heat exceeds threshold.
- `void hotwords_bucket_reorder (DictEntry **bucket, size_t count)`
Reorder a bucket by execution heat (move hot words forward) Called when execution heat delta exceeds threshold.
- `void hotwords_stats_reset (HotwordsStats *stats)`
Reset statistics (for before/after comparison)
- `void hotwords_stats_print (const HotwordsStats *stats)`
Print statistics to stdout.
- `void hotwords_cache_set_enabled (HotwordsCache *cache, int enabled)`
Enable/disable cache at runtime (for A/B testing)
- `BayesianLatencyPosterior hotwords_posterior_cache_hits (const HotwordsStats *stats)`
Calculate posterior distribution for cache hits Uses accumulated samples to compute credible intervals.
- `BayesianLatencyPosterior hotwords_posterior_bucket_searches (const HotwordsStats *stats)`
Calculate posterior distribution for bucket searches Uses accumulated samples to compute credible intervals.
- `struct SpeedupEstimate hotwords_speedup_estimate (const HotwordsStats *stats)`
Calculate speedup estimate with uncertainty.
- `void hotwords_bayesian_report (const HotwordsStats *stats_with_cache, const HotwordsStats *stats_without_cache)`
Generate detailed Bayesian inference report Compares cache vs bucket performance with confidence measures.

7.16.1 Macro Definition Documentation

7.16.1.1 HOTWORDS_CACHE_SIZE

```
#define HOTWORDS_CACHE_SIZE 32
```

Maximum number of hot words to cache (typically 16-32)

7.16.1.2 HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD

```
#define HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD 100
```

Reorder bucket when execution heat delta exceeds this (avoid thrashing)

7.16.1.3 HOTWORDS_EXECUTION_HEAT_THRESHOLD

```
#define HOTWORDS_EXECUTION_HEAT_THRESHOLD 50
```

Execution heat threshold to consider a word "hot" (execution count)

7.16.2 Typedef Documentation

7.16.2.1 HotwordsCache

```
typedef struct HotwordsCache_s HotwordsCache
```

7.16.3 Function Documentation

7.16.3.1 hotwords_bayesian_report()

```
void hotwords_bayesian_report (
    const HotwordsStats * stats_with_cache,
    const HotwordsStats * stats_without_cache )
```

Generate detailed Bayesian inference report Compares cache vs bucket performance with confidence measures.

Generate detailed Bayesian inference report Compares cache vs bucket performance with confidence measures.

References [HotwordsStats::bucket_hits](#), [HotwordsStats::cache_hits](#), [BayesianLatencyPosterior::credible_lower_95](#), [SpeedupEstimate::credible_lower_95_q48](#), [BayesianLatencyPosterior::credible_lower_99](#), [BayesianLatencyPosterior::credible_upper_95_q48](#), [BayesianLatencyPosterior::credible_upper_99](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [hotwords_speedup_estimate\(\)](#), [BayesianLatencyPosterior::mean_ns_q48](#), [HotwordsStats::misses](#), [SpeedupEstimate::probability_gt_10pct_q48](#), [SpeedupEstimate::probability_gt_double_q48_q48_to_ns_double\(\)](#), [BayesianLatencyPosterior::sample_count](#), [SpeedupEstimate::speedup_factor_q48](#), and [BayesianLatencyPosterior::stddev_ns_q48](#).

Referenced by [forth_PHYSICS_BAYESIAN_REPORT\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.2 hotwords_bucket_reorder()

```
void hotwords_bucket_reorder (
    DictEntry ** bucket,
    size_t count )
```

Reorder a bucket by execution heat (move hot words forward) Called when execution heat delta exceeds threshold.

References [execution_heat](#).

7.16.3.3 hotwords_cache_cleanup()

```
void hotwords_cache_cleanup (
    HotwordsCache * cache )
```

Cleanup hot-words cache.

References [cache](#).

Referenced by [vm_cleanup\(\)](#).

Here is the caller graph for this function:

7.16.3.4 hotwords_cache_init()

```
void hotwords_cache_init (
    HotwordsCache * cache )
```

Initialize hot-words cache Should be called during VM initialization.

References [cache](#), and [ENABLE_HOTWORDS_CACHE](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.16.3.5 hotwords_cache_lookup()

```
DictEntry * hotwords_cache_lookup (
    HotwordsCache * cache,
```

```
DictEntry ** bucket,
size_t bucket_count,
const char * name,
size_t len )
```

Lookup word in cache first, then fall back to bucket search Returns the found word, or NULL if not found.

This is the main function that replaces `vm_find_word()` logic

References `cache`, `DictEntry::execution_heat`, `hotwords_cache_promote()`, `HOTWORDS_EXECUTION_HEAT_THRESHOLD`, `name`, `DictEntry::name`, `DictEntry::name_len`, and `sf_monotonic_ns()`.

Referenced by `vm_find_word()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.6 hotwords_cache_promote()

```
void hotwords_cache_promote (
    HotwordsCache * cache,
    DictEntry * word )
```

Promote a word to the hot-words cache Called when a word's execution heat exceeds threshold.

References `cache`, and `HOTWORDS_CACHE_SIZE`.

Referenced by `execute_colon_word()`, `hotwords_cache_lookup()`, `physics_pre_execute()`, and `rolling_window_seed_hotwords_cache()`.

Here is the caller graph for this function:

7.16.3.7 hotwords_cache_set_enabled()

```
void hotwords_cache_set_enabled (
    HotwordsCache * cache,
    int enabled )
```

Enable/disable cache at runtime (for A/B testing)

References `cache`, and `enabled`.

Referenced by `forth_PHYSICS_TOGGLE_CACHE()`.

Here is the caller graph for this function:

7.16.3.8 hotwords_posterior_bucket_searches()

```
BayesianLatencyPosterior hotwords_posterior_bucket_searches (
    const HotwordsStats * stats )
```

Calculate posterior distribution for bucket searches Uses accumulated samples to compute credible intervals.

Calculate posterior distribution for bucket searches Uses accumulated samples to compute credible intervals.

References `HotwordsStats::bucket_search_samples`, `HotwordsStats::bucket_search_total_ns_q48`, `HotwordsStats::bucket_search_variance_sum_q48`, `calculate_posterior()`, `HotwordsStats::max_bucket_search_ns`, and `HotwordsStats::min_bucket_search_ns`.

Referenced by `hotwords_bayesian_report()`, and `hotwords_speedup_estimate()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.9 hotwords_posterior_cache_hits()

```
BayesianLatencyPosterior hotwords_posterior_cache_hits (
    const HotwordsStats * stats )
```

Calculate posterior distribution for cache hits Uses accumulated samples to compute credible intervals.

Calculate posterior distribution for cache hits Uses accumulated samples to compute credible intervals.

References `HotwordsStats::cache_hit_samples`, `HotwordsStats::cache_hit_total_ns_q48`, `HotwordsStats::cache_hit_variance_sum_q48`, `calculate_posterior()`, `HotwordsStats::max_cache_hit_ns`, and `HotwordsStats::min_cache_hit_ns`.

Referenced by `hotwords_bayesian_report()`, and `hotwords_speedup_estimate()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.10 hotwords_speedup_estimate()

```
struct SpeedupEstimate hotwords_speedup_estimate (
    const HotwordsStats * stats )
```

Calculate speedup estimate with uncertainty.

Calculate speedup estimate with uncertainty.

References [SpeedupEstimate::credible_lower_95_q48](#), [SpeedupEstimate::credible_upper_95_q48](#), [erf_q48\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [BayesianLatencyPosterior::mean_ns_q48](#), [SpeedupEstimate::probability_gt_10pct_q48](#), [SpeedupEstimate::probability_gt_double_q48](#), [SpeedupEstimate::speedup_factor_q48](#), [sqrt_q48\(\)](#), and [BayesianLatencyPosterior::stddev_ns_q48](#).

Referenced by [hotwords_bayesian_report\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.11 hotwords_stats_print()

```
void hotwords_stats_print (
    const HotwordsStats * stats )
```

Print statistics to stdout.

References [HotwordsStats::bucket_hits](#), [HotwordsStats::bucket_reorders](#), [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [HotwordsStats::cache_hit_samples](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [HotwordsStats::cache_hits](#), [HotwordsStats::evictions](#), [HotwordsStats::max_bucket_search_ns](#), [HotwordsStats::max_cache_hit_ns](#), [HotwordsStats::min_bucket_search_ns](#), [HotwordsStats::min_cache_hit_ns](#), [HotwordsStats::misses](#), [HotwordsStats::promotions](#), [q48_to_ns_double\(\)](#), and [sqrt_q48\(\)](#).

Referenced by [forth_BENCH_DICT_LOOKUP\(\)](#), and [forth_PHYSICS_CACHE_STATS\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.16.3.12 hotwords_stats_reset()

```
void hotwords_stats_reset (
    HotwordsStats * stats )
```

Reset statistics (for before/after comparison)

References [HotwordsStats::bucket_hits](#), [HotwordsStats::bucket_reorders](#), [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [HotwordsStats::cache_hit_samples](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [HotwordsStats::cache_hits](#), [HotwordsStats::evictions](#), [HotwordsStats::max_bucket_search_ns](#), [HotwordsStats::max_cache_hit_ns](#), [HotwordsStats::min_bucket_search_ns](#), [HotwordsStats::min_cache_hit_ns](#), [HotwordsStats::misses](#), [HotwordsStats::promotions](#), and [HotwordsStats::total_lookups](#).

Referenced by [forth_BENCH_DICT_LOOKUP\(\)](#), and [forth_PHYSICS_RESET_STATS\(\)](#).

Here is the caller graph for this function:

7.17 include/physics_metadata.h File Reference

```
#include <stdint.h>
```

```
#include "vm.h"
```

Include dependency graph for physics_metadata.h: This graph shows which files directly or indirectly include this file:

Functions

- static void [physics_execution_heat_increment](#) ([DictEntry](#) *entry)
- static [cell_t](#) [physics_execution_heat_load](#) (const [DictEntry](#) *entry)
- static [uint64_t](#) [physics_decay_slope_load](#) (const [VM](#) *vm)
- void [physics_metadata_init](#) ([DictEntry](#) *entry, [uint32_t](#) header_bytes)
- void [physics_metadata_set_mass](#) ([DictEntry](#) *entry, [uint32_t](#) total_bytes)
- void [physics_metadata_touch](#) ([DictEntry](#) *entry, [cell_t](#) execution_heat, [uint64_t](#) now_ns)
- void [physics_metadata_refresh_state](#) ([DictEntry](#) *entry)
- void [physics_metadata_record_latency](#) ([DictEntry](#) *entry, [uint64_t](#) sample_ns)
- void [physics_metadata_apply_seed](#) ([DictEntry](#) *entry)
- void [physics_metadata_apply_linear_decay](#) ([DictEntry](#) *entry, [uint64_t](#) elapsed_ns, [VM](#) *vm)

7.17.1 Function Documentation

7.17.1.1 physics_decay_slope_load()

```
static uint64_t physics_decay_slope_load (
    const VM * vm ) [inline], [static]
```

References [VM::decay_slope_q48](#), and [vm](#).

Referenced by [physics_metadata_apply_linear_decay\(\)](#).

Here is the caller graph for this function:

7.17.1.2 physics_execution_heat_increment()

```
static void physics_execution_heat_increment (
    DictEntry * entry ) [inline], [static]
```

References [entry](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the caller graph for this function:

7.17.1.3 physics_execution_heat_load()

```
static cell_t physics_execution_heat_load (
    const DictEntry * entry ) [inline], [static]
```

References [entry](#).

7.17.1.4 physics_metadata_apply_linear_decay()

```
void physics_metadata_apply_linear_decay (
    DictEntry * entry,
    uint64_t elapsed_ns,
    VM * vm )
```

References [DECAY_MIN_INTERVAL](#), [entry](#), [ssm_config_t::L3_linear_decay](#), [physics_decay_slope_load\(\)](#), [VM::ssm_config](#), [vm](#), and [WORD_FROZEN](#).

Referenced by [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.17.1.5 physics_metadata_apply_seed()

```
void physics_metadata_apply_seed (
    DictEntry * entry )
```

References [entry](#), [physics_seed_definition::name](#), [physics_seed_table](#), and [physics_seed_definition::temperature_q8](#).

Referenced by [vm_create_word\(\)](#).

Here is the caller graph for this function:

7.17.1.6 physics_metadata_init()

```
void physics_metadata_init (
    DictEntry * entry,
    uint32_t header_bytes )
```

References [derive_state_flags\(\)](#), and [entry](#).

Referenced by [vm_create_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.17.1.7 physics_metadata_record_latency()

```
void physics_metadata_record_latency (
    DictEntry * entry,
    uint64_t sample_ns )
```

References [clamp_u32\(\)](#), and [entry](#).

Referenced by [profiler_word_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.17.1.8 physics_metadata_refresh_state()

```
void physics_metadata_refresh_state (
    DictEntry * entry )
```

References [derive_state_flags\(\)](#), and [entry](#).

Referenced by [vm_exit_compile_mode\(\)](#), [vm_hide_word\(\)](#), [vm_make_immediate\(\)](#), [vm_pin_execution_heat\(\)](#), [vm_smudge_word\(\)](#), and [vm_unpin_execution_heat\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.17.1.9 physics_metadata_set_mass()

```
void physics_metadata_set_mass (
    DictEntry * entry,
    uint32_t total_bytes )
```

References [entry](#).

Referenced by [vm_exit_compile_mode\(\)](#).

Here is the caller graph for this function:

7.17.1.10 physics_metadata_touch()

```
void physics_metadata_touch (
    DictEntry * entry,
    cell_t execution_heat,
    uint64_t now_ns )
```

References [entry](#), [execution_heat](#), and [temperature_from_execution_heat\(\)](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.18 include/physics_pipelining_metrics.h File Reference

```
#include <stdint.h>
```

Include dependency graph for physics_pipelining_metrics.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [WordTransitionMetrics](#)

Macros

- #define [ENABLE_PIPELINING](#) 0
- #define [SPECULATION_THRESHOLD_Q48](#) (0x8000LL) /* 0.50 in Q48.16 */
SPECULATION_THRESHOLD_Q48: Minimum probability (Q48.16) to speculate on a word transition.
- #define [SPECULATION_DEPTH](#) 1
SPECULATION_DEPTH: How many words ahead to prefetch speculatively.
- #define [MIN_SAMPLES_FOR_SPECULATION](#) 10
MIN_SAMPLES_FOR_SPECULATION: Minimum number of transitions observed before making speculation decisions.
- #define [MISPREDICTION_COST_Q48](#) (25LL << 16) /* 25 ns in Q48.16 */
MISPREDICTION_COST_Q48: Estimated cost (in nanoseconds, Q48.16 format) of recovering from wrong speculation.
- #define [MINIMUM_PREFETCH_ROI](#) (0x11999AL) /* 1.10 in Q48.16 */
MINIMUM_PREFETCH_ROI: Minimum expected Return-On-Investment ratio for speculation to be worthwhile.

- `#define` [TRANSITION_WINDOW_SIZE](#) 2
TRANSITION_WINDOW_SIZE: Knob #6 - Execution context depth for prediction.

Typedefs

- `typedef struct` [WordTransitionMetrics](#) [WordTransitionMetrics](#)

Functions

- `int` [transition_metrics_init](#) ([WordTransitionMetrics](#) *metrics)
Initialize word transition metrics for a dictionary entry.
- `int` [transition_metrics_record](#) ([WordTransitionMetrics](#) *metrics, `uint32_t` next_word_id, `uint32_t` dict_size)
Record a word transition (word_from -> word_to).
- `int64_t` [transition_metrics_get_probability_q48](#) (const [WordTransitionMetrics](#) *metrics, `uint32_t` target_word_id)
Get the probability (Q48.16 format) of a specific word following this word.
- `void` [transition_metrics_update_cache](#) ([WordTransitionMetrics](#) *metrics, `uint32_t` dict_size)
Update cached most-likely-next-word information.
- `void` [transition_metrics_record_prefetch_hit](#) ([WordTransitionMetrics](#) *metrics, `int64_t` latency_saved_ns)
Record a successful prefetch prediction.
- `void` [transition_metrics_record_prefetch_miss](#) ([WordTransitionMetrics](#) *metrics, `int64_t` recovery_cost_ns)
Record a failed prefetch prediction.
- `int` [transition_metrics_should_speculate](#) (const [WordTransitionMetrics](#) *metrics, `uint32_t` target_word_id)
Check if speculation should be attempted for a given target word.
- `char *` [transition_metrics_stats_string](#) (const [WordTransitionMetrics](#) *metrics)
Get detailed statistics string for diagnostics.
- `void` [transition_metrics_reset](#) ([WordTransitionMetrics](#) *metrics)
Reset all transition metrics (clear counts, free transition_heat array).
- `void` [transition_metrics_cleanup](#) ([WordTransitionMetrics](#) *metrics)
Free all allocated memory for transition metrics.
- `int` [transition_metrics_record_context](#) ([WordTransitionMetrics](#) *metrics, const `uint32_t` *context, `uint32_t` window_size, `uint32_t` next_word_id, `uint32_t` dict_size)
Record a context-based transition (multi-word pattern).
- `int` [transition_metrics_update_context_window](#) ([WordTransitionMetrics](#) *metrics, `uint32_t` word_id)
Update the execution context window with a new word.
- `char *` [transition_metrics_context_accuracy_string](#) (const [WordTransitionMetrics](#) *metrics)
Get context-based prediction accuracy statistics (Phase 2).
- `uint32_t` [transition_metrics_binary_chop_suggest_window](#) (const [WordTransitionMetrics](#) *metrics, `uint32_t` current_window, `double` accuracy_at_current)
Get recommended window size based on collected data.

7.18.1 Macro Definition Documentation

7.18.1.1 ENABLE_PIPELINING

```
#define ENABLE_PIPELINING 0
```

7.18.1.2 MIN_SAMPLES_FOR_SPECULATION

```
#define MIN_SAMPLES_FOR_SPECULATION 10
```

`MIN_SAMPLES_FOR_SPECULATION`: Minimum number of transitions observed before making speculation decisions.

Range: 1 (immediate) to 100 (very conservative) Default: 10 (wait for 10 transitions to gather signal)

7.18.1.3 MINIMUM_PREFETCH_ROI

```
#define MINIMUM_PREFETCH_ROI (0x11999AL) /* 1.10 in Q48.16 */
```

MINIMUM_PREFETCH_ROI: Minimum expected Return-On-Investment ratio for speculation to be worthwhile.

Speculation succeeds only if: (prefetch_latency_saved / total_attempts) > MINIMUM_PREFETCH_ROI Range↔

: 1.0 (always speculate) to 2.0 (very selective) Default: 1.10 (10% expected improvement threshold) Q48.16

encoding: $1.10 = 1.10 * (1 \ll 16) = 0x11999AL$

7.18.1.4 MISPREDICTION_COST_Q48

```
#define MISPREDICTION_COST_Q48 (25LL << 16) /* 25 ns in Q48.16 */
```

MISPREDICTION_COST_Q48: Estimated cost (in nanoseconds, Q48.16 format) of recovering from wrong speculation.

This is calibrated to the actual recovery time on target hardware. Range: 0 to 100 ns Default: 25 ns (typical cache miss penalty) Q48.16 encoding: $25 \text{ ns} = 25 \ll 16 = 0x190000LL$

7.18.1.5 SPECULATION_DEPTH

```
#define SPECULATION_DEPTH 1
```

SPECULATION_DEPTH: How many words ahead to prefetch speculatively.

Range: 1 (next word only) to 4 (very aggressive) Default: 1 (prefetch immediate successor)

7.18.1.6 SPECULATION_THRESHOLD_Q48

```
#define SPECULATION_THRESHOLD_Q48 (0x8000LL) /* 0.50 in Q48.16 */
```

SPECULATION_THRESHOLD_Q48: Minimum probability (Q48.16) to speculate on a word transition.

Range: 0.10 (very aggressive) to 0.95 (very conservative) Default: 0.50 (50% confidence required) Q48.16

encoding: $0.50 = 0x8000LL (1LL \ll 15)$

7.18.1.7 TRANSITION_WINDOW_SIZE

```
#define TRANSITION_WINDOW_SIZE 2
```

TRANSITION_WINDOW_SIZE: Knob #6 - Execution context depth for prediction.

How many previous words to remember when predicting next word. Larger window captures deeper patterns but uses more memory.

Window=1: Only immediate predecessor (A→B predictions) Window=2: Previous 2 words (A,B→C predictions) - DEFAULT, good balance Window=4: Previous 4 words (captures multi-level patterns) Window=8: Very deep (expensive, rarely needed)

Binary Chop Search Space (Phase 2): Iteration 1: Try window=2 (baseline) Iteration 2: Try window=1 and window=4 Iteration 3: Narrow to optimal (1, 2, or 4) Converge: Pick size with highest prediction accuracy

Range: 1 to 8 Default: 2 (empirical sweet spot: captures context without excessive memory) Trade-off: Pattern depth vs. memory usage & complexity

Tuning: make TRANSITION_WINDOW_SIZE=1 (or 2, 4, 8)

7.18.2 Typedef Documentation

7.18.2.1 WordTransitionMetrics

```
typedef struct WordTransitionMetrics WordTransitionMetrics
```

7.18.3 Function Documentation

7.18.3.1 transition_metrics_binary_chop_suggest_window()

```
uint32_t transition_metrics_binary_chop_suggest_window (
    const WordTransitionMetrics * metrics,
    uint32_t current_window,
    double accuracy_at_current )
```

Get recommended window size based on collected data.

Phase 2 will call this as part of binary chop tuning algorithm.

Algorithm:

1. Collect data with current window size
2. Measure prediction accuracy
3. If accuracy improves, try larger window
4. If accuracy declines, try smaller window
5. Converge to optimal size

Parameters

<i>metrics</i>	Metrics to analyze
<i>current_window</i>	Current window size being tested
<i>accuracy_at_current</i>	Measured accuracy with current window

Returns

Suggested next window size to try (1, 2, 4, or 8)

7.18.3.2 transition_metrics_cleanup()

```
void transition_metrics_cleanup (
    WordTransitionMetrics * metrics )
```

Free all allocated memory for transition metrics.

Called during dictionary cleanup.

Parameters

<i>metrics</i>	Metrics to clean up
----------------	---------------------

References [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window](#), and [WordTransitionMetrics::transition](#)
Referenced by [dictionary_word_forget\(\)](#).

Here is the caller graph for this function:

7.18.3.3 transition_metrics_context_accuracy_string()

```
char * transition_metrics_context_accuracy_string (
    const WordTransitionMetrics * metrics )
```

Get context-based prediction accuracy statistics (Phase 2).

Compares prediction success rates across different window sizes.

Phase 2 will use this to run binary chop tuning: Measure accuracy(window=1), accuracy(window=2), accuracy(window=4) Binary search converges to optimal window size

Parameters

<i>metrics</i>	Metrics to analyze
----------------	--------------------

Returns

Formatted string with accuracy statistics (caller must free)

References [WordTransitionMetrics::actual_window_size](#), and [WordTransitionMetrics::total_context_transitions](#).

7.18.3.4 transition_metrics_get_probability_q48()

```
int64_t transition_metrics_get_probability_q48 (
```

```
const WordTransitionMetrics * metrics,
uint32_t target_word_id )
```

Get the probability (Q48.16 format) of a specific word following this word.

Returns: (transition_heat[target] / total_transitions) in Q48.16 format or 0 if total_transitions == 0

Parameters

<i>metrics</i>	Metrics for the current word
<i>target_word_id</i>	Index of potential next word

Returns

Probability in Q48.16 format (0.0 to 1.0)

References [q48_div_u64\(\)](#), [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).

Referenced by [transition_metrics_should_speculate\(\)](#), and [transition_metrics_update_cache\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.18.3.5 transition_metrics_init()

```
int transition_metrics_init (
    WordTransitionMetrics * metrics )
```

Initialize word transition metrics for a dictionary entry.

Called once per word during dictionary entry creation.

Parameters

<i>metrics</i>	Pointer to uninitialized WordTransitionMetrics
----------------	--

Returns

0 on success, -1 on memory allocation failure

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window_pos](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::mispre](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_context](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [TRANSITION_WINDOW_SIZE](#).

Referenced by [vm_create_word\(\)](#).

Here is the caller graph for this function:

7.18.3.6 transition_metrics_record()

```
int transition_metrics_record (
    WordTransitionMetrics * metrics,
    uint32_t next_word_id,
    uint32_t dict_size )
```

Record a word transition (word_from -> word_to).

Called in the inner interpreter after each word execution.

Parameters

<i>metrics</i>	Metrics for the current word
<i>next_word_id</i>	Index of word that executed next
<i>dict_size</i>	Total number of words in dictionary (for array bounds)

Returns

0 on success, -1 on invalid word_id

References [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the caller graph for this function:

7.18.3.7 transition_metrics_record_context()

```
int transition_metrics_record_context (
    WordTransitionMetrics * metrics,
    const uint32_t * context,
    uint32_t window_size,
    uint32_t next_word_id,
    uint32_t dict_size )
```

Record a context-based transition (multi-word pattern).

Called after each word execution with the execution window.

Example with WINDOW=2: context[0] = word_A, context[1] = word_B, next_word = word_C Records: (A,B) → C transition in sparse hash table

Phase 1: Just collects data, no analysis Phase 2: Will measure: "Do patterns with window=2 predict better than window=1?"

Parameters

<i>metrics</i>	Metrics for the current word
<i>context</i>	Array of word IDs forming the context (size = window_size)
<i>window_size</i>	Length of context array
<i>next_word_id</i>	Word that follows the context
<i>dict_size</i>	Total number of words in dictionary

Returns

0 on success, -1 on error

References [WordTransitionMetrics::total_context_transitions](#).

Referenced by [rolling_window_seed_pipelining_context\(\)](#).

Here is the caller graph for this function:

7.18.3.8 transition_metrics_record_prefetch_hit()

```
void transition_metrics_record_prefetch_hit (
    WordTransitionMetrics * metrics,
    int64_t latency_saved_ns )
```

Record a successful prefetch prediction.

Called when speculation correctly predicted the next word.

Parameters

<i>metrics</i>	Metrics for the current word
<i>latency_saved_ns</i>	Nanoseconds saved (will be Q48.16 encoded)

References [ns_to_q48\(\)](#), [WordTransitionMetrics::prefetch_hits](#), and [WordTransitionMetrics::prefetch_latency_saved_q48](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.18.3.9 transition_metrics_record_prefetch_miss()

```
void transition_metrics_record_prefetch_miss (
    WordTransitionMetrics * metrics,
    int64_t recovery_cost_ns )
```

Record a failed prefetch prediction.

Called when speculation incorrectly predicted the next word.

Parameters

<i>metrics</i>	Metrics for the current word
<i>recovery_cost_ns</i>	Nanoseconds lost to misprediction recovery

References [WordTransitionMetrics::misprediction_cost_q48](#), [ns_to_q48\(\)](#), and [WordTransitionMetrics::prefetch_misses](#).

Here is the call graph for this function:

7.18.3.10 transition_metrics_reset()

```
void transition_metrics_reset (
    WordTransitionMetrics * metrics )
```

Reset all transition metrics (clear counts, free transition_heat array).

Called on [VM](#) reset or explicit metrics reset.

Parameters

<i>metrics</i>	Metrics to reset
----------------	------------------

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window_pos](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction_cost_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_context_transitions](#), [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).

Referenced by [forth_PIPELINING_RESET_ALL\(\)](#).

Here is the caller graph for this function:

7.18.3.11 transition_metrics_should_speculate()

```
int transition_metrics_should_speculate (
    const WordTransitionMetrics * metrics,
    uint32_t target_word_id )
```

Check if speculation should be attempted for a given target word.

Applies threshold logic and ROI analysis.

Returns: 1 if speculation should be attempted, 0 otherwise

Parameters

<i>metrics</i>	Metrics for the current word
<i>target_word_id</i>	Index of potential next word

Returns

1 if probability > SPECULATION_THRESHOLD and enough samples, 0 otherwise

References [MIN_SAMPLES_FOR_SPECULATION](#), [SPECULATION_THRESHOLD_Q48](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [transition_metrics_get_probability_q48\(\)](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.18.3.12 transition_metrics_stats_string()

```
char * transition_metrics_stats_string (
    const WordTransitionMetrics * metrics )
```

Get detailed statistics string for diagnostics.
Caller must free the returned string.

Parameters

<i>metrics</i>	Metrics to report on
----------------	----------------------

Returns

Formatted statistics string (must be freed by caller)

References [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction_cost_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [q48_to_ns\(\)](#), and [WordTransitionMetrics::total_transitions](#).

Here is the call graph for this function:

7.18.3.13 transition_metrics_update_cache()

```
void transition_metrics_update_cache (
    WordTransitionMetrics * metrics,
    uint32_t dict_size )
```

Update cached most-likely-next-word information.
Should be called periodically (e.g., after every 100 transitions) to avoid stale cache.

Parameters

<i>metrics</i>	Metrics for the current word
<i>dict_size</i>	Total number of words in dictionary

References [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [transition_metrics_get_probability_q48\(\)](#).
Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.18.3.14 transition_metrics_update_context_window()

```
int transition_metrics_update_context_window (
    WordTransitionMetrics * metrics,
    uint32_t word_id )
```

Update the execution context window with a new word.
Called after each word execution to slide the window forward.
Example with WINDOW=2: Before: [A, B] (two previous words) After calling with C: [B, C] (shift left, add C on right)

Parameters

<i>metrics</i>	Metrics to update
<i>word_id</i>	New word ID to add to window

Returns

0 on success, -1 if context tracking not initialized

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_window](#), and [word_id](#).

7.19 include/physics_runtime.h File Reference

```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for physics_runtime.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [physics_host_snapshot](#)
Snapshot of scheduler-adjacent host signals.
- struct [physics_analytics_event_header](#)
Event header written into the analytics ring buffer.
- struct [physics_analytics_header](#)
Analytics heap header seen by both producer (VM) and consumer (HOLA).
- struct [physics_analytics_heap](#)
Simple view onto the analytics heap layout.

Macros

- #define [PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES](#) (10u * 1024u * 1024u)
Default analytics heap size (10 MiB)
- #define [HOLA_SHARED_MAGIC](#) 0x484f4c41u
Magic tag for HOLA analytics heap headers ('HOLA')
- #define [HOLA_PROTOCOL_MAJOR](#) 0x0001u
HOLA protocol version (major.minor)
- #define [HOLA_PROTOCOL_MINOR](#) 0x0000u
- #define [PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT](#) 0x00000002u
Analytics channel ids.
- #define [PHYSICS_HOST_FLAG_PSI](#) 0x0001u
Host snapshot flag bits.
- #define [PHYSICS_HOST_FLAG_CGROUP](#) 0x0002u
- #define [PHYSICS_HOST_FLAG_CPU_STAT](#) 0x0004u

Typedefs

- typedef enum [physics_host_backend](#) [physics_host_backend_t](#)
Host backend identifier.
- typedef struct [physics_host_snapshot](#) [physics_host_snapshot_t](#)
Snapshot of scheduler-adjacent host signals.
- typedef struct [physics_analytics_event_header](#) [physics_analytics_event_header_t](#)
Event header written into the analytics ring buffer.
- typedef struct [physics_analytics_header](#) [physics_analytics_header_t](#)
Analytics heap header seen by both producer (VM) and consumer (HOLA).
- typedef struct [physics_analytics_heap](#) [physics_analytics_heap_t](#)
Simple view onto the analytics heap layout.

Enumerations

- enum [physics_host_backend](#) { [PHYSICS_HOST_BACKEND_POSIX](#) = 0 , [PHYSICS_HOST_BACKEND_L4RE](#) = 1 , [PHYSICS_HOST_BACKEND_FAKE](#) = 2 }
- Host backend identifier.*

Functions

- int `physics_runtime_init` (size_t analytics_heap_bytes)
Initialise physics runtime services.
- void `physics_runtime_shutdown` (void)
Tear down physics runtime services and release heap storage.
- int `physics_host_snapshot` (physics_host_snapshot_t *out)
Capture a host snapshot.
- int `physics_analytics_publish_event` (uint32_t channel, const void *payload, uint16_t payload_bytes)
Publish an event into the analytics ring buffer.
- const `physics_analytics_heap_t` * `physics_analytics_heap_info` (void)
Retrieve the analytics heap descriptor.

7.19.1 Macro Definition Documentation

7.19.1.1 HOLA_PROTOCOL_MAJOR

```
#define HOLA_PROTOCOL_MAJOR 0x0001u
```

HOLA protocol version (major.minor)

7.19.1.2 HOLA_PROTOCOL_MINOR

```
#define HOLA_PROTOCOL_MINOR 0x0000u
```

7.19.1.3 HOLA_SHARED_MAGIC

```
#define HOLA_SHARED_MAGIC 0x484f4c41u
```

Magic tag for HOLA analytics heap headers ('HOLA')

7.19.1.4 PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT

```
#define PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT 0x00000002u
```

Analytics channel ids.

7.19.1.5 PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES

```
#define PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES (10u * 1024u * 1024u)
```

Default analytics heap size (10 MiB)

7.19.1.6 PHYSICS_HOST_FLAG_CGROUP

```
#define PHYSICS_HOST_FLAG_CGROUP 0x0002u
```

7.19.1.7 PHYSICS_HOST_FLAG_CPU_STAT

```
#define PHYSICS_HOST_FLAG_CPU_STAT 0x0004u
```

7.19.1.8 PHYSICS_HOST_FLAG_PSI

```
#define PHYSICS_HOST_FLAG_PSI 0x0001u
```

Host snapshot flag bits.

7.19.2 Typedef Documentation

7.19.2.1 physics_analytics_event_header_t

```
typedef struct physics_analytics_event_header physics_analytics_event_header_t
```

Event header written into the analytics ring buffer.

7.19.2.2 physics_analytics_header_t

typedef struct [physics_analytics_header](#) [physics_analytics_header_t](#)
Analytics heap header seen by both producer ([VM](#)) and consumer ([HOLA](#)).

7.19.2.3 physics_analytics_heap_t

typedef struct [physics_analytics_heap](#) [physics_analytics_heap_t](#)
Simple view onto the analytics heap layout.

7.19.2.4 physics_host_backend_t

typedef enum [physics_host_backend](#) [physics_host_backend_t](#)
Host backend identifier.

7.19.2.5 physics_host_snapshot_t

typedef struct [physics_host_snapshot](#) [physics_host_snapshot_t](#)
Snapshot of scheduler-adjacent host signals.
Fields are intentionally fixed width to maximise portability and allow the governance repository to consume binary dumps without per-platform quirks.

7.19.3 Enumeration Type Documentation

7.19.3.1 physics_host_backend

enum [physics_host_backend](#)
Host backend identifier.

Enumerator

PHYSICS_HOST_BACKEND_POSIX	
PHYSICS_HOST_BACKEND_L4RE	
PHYSICS_HOST_BACKEND_FAKE	

7.19.4 Function Documentation

7.19.4.1 physics_analytics_heap_info()

const [physics_analytics_heap_t](#) * [physics_analytics_heap_info](#) (
void)

Retrieve the analytics heap descriptor.

Returns

Pointer to heap descriptor (NULL if runtime not initialised)

References [g_runtime](#), [physics_runtime_state::heap](#), and [physics_runtime_state::heap_ready](#).

7.19.4.2 physics_analytics_publish_event()

int [physics_analytics_publish_event](#) (
uint32_t *channel*,
const void * *payload*,
uint16_t *payload_bytes*)

Publish an event into the analytics ring buffer.

Parameters

<i>channel</i>	Logical event channel
----------------	-----------------------

Parameters

<i>payload</i>	Pointer to payload bytes
<i>payload_bytes</i>	Payload length ($\leq$ ring size)

Returns

0 on success, -1 if the payload could not be enqueued

References [align_up\(\)](#), [analytics_ring_free\(\)](#), [physics_analytics_event_header::channel](#), [physics_analytics_header::dropped_events](#), [g_runtime](#), [physics_analytics_heap::header](#), [physics_runtime_state::heap](#), [physics_runtime_state::heap_ready](#), [physics_analytics_event_header::payload_bytes](#), [physics_analytics_header::produce_seq](#), [physics_analytics_event_header::reserved](#), [physics_analytics_heap::ring](#), [physics_analytics_header::ring_bytes](#), [sf_monotonic_ns\(\)](#), [physics_analytics_event_header::timestamp](#) and [physics_analytics_header::write_offset](#).

Referenced by [physics_host_snapshot\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.19.4.3 **physics_host_snapshot()**

```
int physics_host_snapshot (
    physics_host_snapshot_t * out )
```

Capture a host snapshot.

Parameters

<i>out</i>	Output buffer
------------	---------------

Returns

0 on success, -1 on failure

References [g_runtime](#), [physics_runtime_state::heap_ready](#), [PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT](#), [physics_analytics_publish_event\(\)](#), and [snapshot_posix\(\)](#).

Here is the call graph for this function:

7.19.4.4 **physics_runtime_init()**

```
int physics_runtime_init (
    size_t analytics_heap_bytes )
```

Initialise physics runtime services.

Parameters

<i>analytics_heap_bytes</i>	Desired heap size (0 -> default 10 MiB)
-----------------------------	---

Returns

0 on success, -1 on failure

References [analytics_heap_allocate\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.19.4.5 **physics_runtime_shutdown()**

```
void physics_runtime_shutdown (
    void )
```

Tear down physics runtime services and release heap storage.

References [analytics_heap_release\(\)](#).

Referenced by [cleanup_and_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.20 include/platform_alloc.h File Reference

```
#include <stddef.h>
```

Include dependency graph for platform_alloc.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [sf_alloc_stats](#)

Typedefs

- typedef struct [sf_alloc_stats](#) [sf_alloc_stats_t](#)

Functions

- void * [sf_malloc](#) (size_t size)
- void * [sf_calloc](#) (size_t count, size_t size)
- void * [sf_realloc](#) (void *ptr, size_t new_size)
- void [sf_free](#) (void *ptr)
- int [sf_alloc_init](#) (void)
- void [sf_alloc_get_stats](#) ([sf_alloc_stats_t](#) *stats)

7.20.1 Typedef Documentation

7.20.1.1 sf_alloc_stats_t

```
typedef struct sf_alloc_stats sf_alloc_stats_t
```

7.20.2 Function Documentation

7.20.2.1 sf_alloc_get_stats()

```
void sf_alloc_get_stats (
    sf_alloc_stats_t * stats )
```

References [g_stats](#), and [g_stats](#).

7.20.2.2 sf_alloc_init()

```
int sf_alloc_init (
    void )
```

References [sf_alloc_stats::alloc_count](#), [sf_alloc_stats::free_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ARENA_SIZE](#), [sf_alloc_stats::total_bytes](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [sf_malloc\(\)](#).

Here is the caller graph for this function:

7.20.2.3 sf_calloc()

```
void * sf_calloc (
    size_t count,
    size_t size )
```

References [sf_alloc_stats::alloc_count](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [sf_malloc\(\)](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [rolling_window_find_hottest_word\(\)](#), [rolling_window_init\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), and [vm_heartbeat_start_thread\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.20.2.4 sf_free()

```
void sf_free (
    void * ptr )
```

References [sf_alloc_stats::free_count](#), [g_stats](#), and [g_stats](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), [rolling_window_cleanup\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_init\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipeline\(\)](#), [sf_fc_clear_all\(\)](#), [sf_xrealloc\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_start_thread\(\)](#), and [vm_heartbeat_stop_thread\(\)](#).

Here is the caller graph for this function:

7.20.2.5 sf_malloc()

```
void * sf_malloc (
    size_t size )
```

References [sf_alloc_stats::alloc_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ALIGN_UP](#), [sf_alloc_init\(\)](#), [SF_ARENA_SIZE](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_stats_string\(\)](#), [sf_calloc\(\)](#), [sf_realloc\(\)](#), [vm_create_word\(\)](#), [vm_init\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.20.2.6 sf_realloc()

```
void * sf_realloc (
    void * ptr,
    size_t new_size )
```

References [sf_alloc_stats::alloc_count](#), [g_stats](#), and [sf_malloc\(\)](#).

Referenced by [sf_xrealloc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.21 include/platform_lock.h File Reference

```
#include <pthread.h>
```

Include dependency graph for platform_lock.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [sf_mutex](#)

Typedefs

- typedef struct [sf_mutex](#) [sf_mutex_t](#)

Functions

- int [sf_mutex_init](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_destroy](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_lock](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_unlock](#) ([sf_mutex_t](#) *mutex)

7.21.1 Typedef Documentation

7.21.1.1 sf_mutex_t

```
typedef struct sf\_mutex sf\_mutex\_t
```

7.21.2 Function Documentation

7.21.2.1 sf_mutex_destroy()

```
void sf_mutex_destroy (
    sf_mutex_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [hosted_mutex_destroy\(\)](#), [vm_cleanup\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.21.2.2 sf_mutex_init()

```
int sf_mutex_init (
    sf_mutex_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [hosted_mutex_init\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.21.2.3 sf_mutex_lock()

```
void sf_mutex_lock (
    sf_mutex_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_word_forget\(\)](#), [extract_heat_trajectory\(\)](#), [hosted_mutex_lock\(\)](#), [is_valid_dict_entry\(\)](#), [metrics_from_vm\(\)](#), [physics_on_lookup\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

Here is the caller graph for this function:

7.21.2.4 sf_mutex_unlock()

```
void sf_mutex_unlock (
    sf_mutex_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_word_forget\(\)](#), [extract_heat_trajectory\(\)](#), [hosted_mutex_unlock\(\)](#), [is_valid_dict_entry\(\)](#), [metrics_from_vm\(\)](#), [physics_on_lookup\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

Here is the caller graph for this function:

7.22 include/platform_time.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for platform_time.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [sf_time_backend_t](#)

Macros

- #define [SF_TIME_STAMP_SIZE](#) 32

Typedefs

- typedef uint64_t [sf_time_ns_t](#)

Functions

- static [sf_time_ns_t sf_monotonic_ns](#) (void)
- static [sf_time_ns_t sf_realtime_ns](#) (void)
- static int [sf_set_realtime_ns](#) ([sf_time_ns_t](#) ns)
- static int [sf_format_timestamp](#) ([sf_time_ns_t](#) ns, char *buf, int format_24h)
- static int [sf_has_rtc](#) (void)
- static [sf_time_ns_t sf_seconds_to_ns](#) (uint64_t seconds)
- static uint64_t [sf_ns_to_seconds](#) ([sf_time_ns_t](#) ns)
- static uint64_t [sf_ns_to_ms](#) ([sf_time_ns_t](#) ns)
- static uint64_t [sf_ns_to_us](#) ([sf_time_ns_t](#) ns)
- void [sf_time_init](#) (void)

Initialize platform time subsystem.

Variables

- const [sf_time_backend_t](#) * [sf_time_backend](#)
- const [sf_time_backend_t](#) [sf_time_backend_posix](#)
- const [sf_time_backend_t](#) [sf_time_backend_l4re](#)

7.22.1 Macro Definition Documentation

7.22.1.1 SF_TIME_STAMP_SIZE

```
#define SF_TIME_STAMP_SIZE 32
```

7.22.2 Typedef Documentation

7.22.2.1 sf_time_ns_t

```
typedef uint64_t sf_time_ns_t
```

7.22.3 Function Documentation

7.22.3.1 sf_format_timestamp()

```
static int sf_format_timestamp (
    sf_time_ns_t ns,
    char * buf,
    int format_24h ) [inline], [static]
```

References [sf_time_backend_t::format_timestamp](#), and [sf_time_backend](#).

Referenced by [get_timestamp\(\)](#).

Here is the caller graph for this function:

7.22.3.2 sf_has_rtc()

```
static int sf_has_rtc (
    void ) [inline], [static]
```

References [sf_time_backend_t::has_rtc](#), and [sf_time_backend](#).

Referenced by [blk_get_timestamp\(\)](#).

Here is the caller graph for this function:

7.22.3.3 sf_monotonic_ns()

```
static sf_time_ns_t sf_monotonic_ns (
    void ) [inline], [static]
```

References [sf_time_backend_t::get_monotonic_ns](#), and [sf_time_backend](#).

Referenced by [blk_get_timestamp\(\)](#), [defining_runtime_dodoes\(\)](#), [dict_adaptive_optimization_pass\(\)](#), [dict_reorganize_buckets_by_heap\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), [get_time_ns\(\)](#), [get_time_ns\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#),

[hosted_time_ns\(\)](#), [hotwords_cache_lookup\(\)](#), [physics_analytics_publish_event\(\)](#), [physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [physics_pre_execute\(\)](#), [run_doe_experiment\(\)](#), [snapshot_posix\(\)](#), [starforth_word_init\(\)](#), [starforth_word_wait\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_run_cycle\(\)](#).

Here is the caller graph for this function:

7.22.3.4 sf_ns_to_ms()

```
static uint64_t sf_ns_to_ms (
    sf_time_ns_t ns ) [inline], [static]
```

7.22.3.5 sf_ns_to_seconds()

```
static uint64_t sf_ns_to_seconds (
    sf_time_ns_t ns ) [inline], [static]
```

7.22.3.6 sf_ns_to_us()

```
static uint64_t sf_ns_to_us (
    sf_time_ns_t ns ) [inline], [static]
```

7.22.3.7 sf_realtime_ns()

```
static sf_time_ns_t sf_realtime_ns (
    void ) [inline], [static]
```

References [sf_time_backend_t::get_realtime_ns](#), and [sf_time_backend](#).

Referenced by [blk_get_timestamp\(\)](#), [get_timestamp\(\)](#), and [snapshot_posix\(\)](#).

Here is the caller graph for this function:

7.22.3.8 sf_seconds_to_ns()

```
static sf_time_ns_t sf_seconds_to_ns (
    uint64_t seconds ) [inline], [static]
```

7.22.3.9 sf_set_realtime_ns()

```
static int sf_set_realtime_ns (
    sf_time_ns_t ns ) [inline], [static]
```

References [sf_time_backend_t::set_realtime_ns](#), and [sf_time_backend](#).

7.22.3.10 sf_time_init()

```
void sf_time_init (
    void )
```

Initialize platform time subsystem.

Selects appropriate backend based on compile-time platform:

- L4Re/StarshipOS: Uses RTC server if available, falls back to KIP clock
- POSIX: Uses `clock_gettime()`

Call this once at program startup before using any time functions.

References [sf_time_backend](#), [sf_time_backend_l4re](#), [sf_time_backend_posix](#), and [sf_time_init_l4re\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.22.4 Variable Documentation

7.22.4.1 sf_time_backend

```
const sf_time_backend_t* sf_time_backend [extern]
```

Referenced by [sf_format_timestamp\(\)](#), [sf_has_rtc\(\)](#), [sf_monotonic_ns\(\)](#), [sf_realtime_ns\(\)](#), [sf_set_realtime_ns\(\)](#), and [sf_time_init\(\)](#).

7.22.4.2 sf_time_backend_l4re

const [sf_time_backend_t](#) sf_time_backend_l4re [extern]

Referenced by [sf_time_init\(\)](#).

7.22.4.3 sf_time_backend_posix

const [sf_time_backend_t](#) sf_time_backend_posix [extern]

Referenced by [sf_time_init\(\)](#).

7.23 include/profiler.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

```
#include <stdio.h>
```

Include dependency graph for profiler.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [ProfileTimer](#)
Timer for measuring execution duration.
- struct [WordProfile](#)
Execution statistics for a single word.
- struct [MemoryProfile](#)
- struct [Profiler](#)
- struct [ProfileScope](#)

Macros

- #define [PROFILE_ENABLED](#)() (0)
- #define [PROFILE_START](#)(category) ((void)0)
- #define [PROFILE_END](#)(category) ((void)0)
- #define [PROFILE_WORD_ENTER](#)(name) ((void)0)
- #define [PROFILE_WORD_EXIT](#)(name) ((void)0)
- #define [PROFILE_MEMORY_READ](#)(bytes) ((void)0)
- #define [PROFILE_MEMORY_WRITE](#)(bytes) ((void)0)
- #define [PROFILE_INC_CYCLES](#)(n) ((void)0)
- #define [PROFILE_INC_DICT_LOOKUP](#)() ((void)0)
- #define [PROFILE_INC_STACK_OP](#)() ((void)0)

Typedefs

- typedef struct [VM](#) VM
- typedef struct [DictEntry](#) DictEntry

Enumerations

- enum [ProfileLevel](#) {
 [PROFILE_DISABLED](#) = 0 , [PROFILE_BASIC](#) = 1 , [PROFILE_DETAILED](#) = 2 , [PROFILE_VERBOSE](#) = 3 ,
 [PROFILE_FULL](#) = 4 }
Profiling detail levels.
- enum [ProfileCategory](#) {
 [PROF_VM_INIT](#) = 0 , [PROF_VM_EXECUTE](#) , [PROF_VM_COMPILE](#) , [PROF_DICT_LOOKUP](#) ,
 [PROF_DICT_ADD](#) , [PROF_STACK_OPS](#) , [PROF_MEMORY_ACCESS](#) , [PROF_WORD_EXEC](#) ,
 [PROF_IO_OPS](#) , [PROF_CONTROL_FLOW](#) , [PROF_CATEGORY_COUNT](#) }
Profiling measurement categories.

Functions

- int [profiler_init](#) (ProfileLevel level)
Initialize the profiling system.
- void [profiler_shutdown](#) (void)
Shutdown and cleanup the profiling system.
- void [profiler_reset](#) (void)
Reset all profiling statistics.
- void [profiler_set_level](#) (ProfileLevel level)
Change the current profiling detail level.
- uint64_t [profiler_get_time_ns](#) (void)
- void [profiler_start_timer](#) (ProfileCategory category)
- void [profiler_end_timer](#) (ProfileCategory category)
- void [profiler_word_enter](#) (const DictEntry *entry)
Start timing a word execution (PROFILE_DETAILED+)
- void [profiler_word_exit](#) (const DictEntry *entry)
End timing a word execution (PROFILE_DETAILED+)
- void [profiler_word_count](#) (const DictEntry *entry)
Track word execution frequency without timing (PROFILE_BASIC+)
- void [profiler_memory_read](#) (size_t bytes)
- void [profiler_memory_write](#) (size_t bytes)
- void [profiler_memory_alloc](#) (size_t bytes)
- void [profiler_inc_vm_cycles](#) (uint64_t cycles)
- void [profiler_inc_dict_lookup](#) (void)
- void [profiler_inc_stack_op](#) (void)
- void [profiler_inc_branch](#) (void)
- void [profiler_inc_loop_iter](#) (void)
- void [profiler_generate_report](#) (void)
Generate and print detailed profiling report.
- void [profiler_print_summary](#) (void)
- void [profiler_print_word_analysis](#) (void)
- void [profiler_print_memory_analysis](#) (void)
- void [profiler_print_hotspots](#) (void)
Print hot word analysis with optimization recommendations.
- void [profiler_analyze_performance](#) (VM *vm)
- void [profiler_suggest_optimizations](#) (void)

Variables

- void * [g_profiler](#)

7.23.1 Macro Definition Documentation

7.23.1.1 PROFILE_ENABLED

```
#define PROFILE_ENABLED( ) (0)
```

7.23.1.2 PROFILE_END

```
#define PROFILE_END(  
    category ) ((void)0)
```

7.23.1.3 PROFILE_INC_CYCLES

```
#define PROFILE_INC_CYCLES(  
    n ) ((void)0)
```

7.23.1.4 PROFILE_INC_DICT_LOOKUP

```
#define PROFILE_INC_DICT_LOOKUP( ) ((void)0)
```

7.23.1.5 PROFILE_INC_STACK_OP

```
#define PROFILE_INC_STACK_OP( ) ((void)0)
```

7.23.1.6 PROFILE_MEMORY_READ

```
#define PROFILE_MEMORY_READ(  
    bytes ) ((void)0)
```

7.23.1.7 PROFILE_MEMORY_WRITE

```
#define PROFILE_MEMORY_WRITE(  
    bytes ) ((void)0)
```

7.23.1.8 PROFILE_START

```
#define PROFILE_START(  
    category ) ((void)0)
```

7.23.1.9 PROFILE_WORD_ENTER

```
#define PROFILE_WORD_ENTER(  
    name ) ((void)0)
```

7.23.1.10 PROFILE_WORD_EXIT

```
#define PROFILE_WORD_EXIT(  
    name ) ((void)0)
```

7.23.2 Typedef Documentation

7.23.2.1 DictEntry

```
typedef struct DictEntry DictEntry
```

7.23.2.2 VM

```
typedef struct VM VM
```

7.23.3 Enumeration Type Documentation

7.23.3.1 ProfileCategory

```
enum ProfileCategory
```

Profiling measurement categories.

Different aspects of [VM](#) operation that can be profiled

Enumerator

PROF_VM_INIT	VM initialization.
PROF_VM_EXECUTE	VM execution loop.
PROF_VM_COMPILE	Word compilation.
PROF_DICT_LOOKUP	Dictionary searches.
PROF_DICT_ADD	Dictionary additions.
PROF_STACK_OPS	Stack operations.
PROF_MEMORY_ACCESS	Memory access operations.

Enumerator

PROF_WORD_EXEC	Word execution.
PROF_IO_OPS	I/O operations.
PROF_CONTROL_FLOW	Control flow operations.
PROF_CATEGORY_COUNT	Total number of categories.

7.23.3.2 ProfileLevel

enum [ProfileLevel](#)

Profiling detail levels.

Controls the amount of profiling information collected during execution. Higher levels include all features from lower levels.

Enumerator

PROFILE_DISABLED	Profiling disabled (zero overhead)
PROFILE_BASIC	Word execution frequency tracking (minimal overhead)
PROFILE_DETAILED	Basic + word execution timing (5-10% overhead)
PROFILE_VERBOSE	Detailed + stack/memory access patterns (15-20% overhead)
PROFILE_FULL	Reserved for future use (full instrumentation)

7.23.4 Function Documentation

7.23.4.1 profiler_analyze_performance()

```
void profiler_analyze_performance (
    VM * vm )
```

References [vm](#).

7.23.4.2 profiler_end_timer()

```
void profiler_end_timer (
    ProfileCategory category )
```

7.23.4.3 profiler_generate_report()

```
void profiler_generate_report (
    void )
```

Generate and print detailed profiling report.

Generates a report including:

- Global statistics (dictionary lookups, stack ops, memory usage)
- Top words by execution time
- Most frequently called words

References [WordStats::call_count](#), [compare_by_calls\(\)](#), [compare_by_time\(\)](#), [WordStats::entry](#), [limit](#), [WordStats::max_time_ns](#), [name_len](#), [DictEntry::name_len](#), [print_word_name_safe\(\)](#), [PROFILE_BASIC](#), [PROFILE_DETAILED](#), [PROFILE_VERBOSE](#), [profiler_state](#), and [WordStats::total_time_ns](#).

Referenced by [profiler_print_summary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.23.4.4 profiler_get_time_ns()

```
uint64_t profiler_get_time_ns (
    void )
```

References [get_time_ns\(\)](#).

Here is the call graph for this function:

7.23.4.5 profiler_inc_branch()

```
void profiler_inc_branch (
    void )
```

7.23.4.6 profiler_inc_dict_lookup()

```
void profiler_inc_dict_lookup (
    void )
```

References [PROFILE_BASIC](#), and [profiler_state](#).

7.23.4.7 profiler_inc_loop_iter()

```
void profiler_inc_loop_iter (
    void )
```

7.23.4.8 profiler_inc_stack_op()

```
void profiler_inc_stack_op (
    void )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.23.4.9 profiler_inc_vm_cycles()

```
void profiler_inc_vm_cycles (
    uint64_t cycles )
```

7.23.4.10 profiler_init()

```
int profiler_init (
    ProfileLevel level )
```

Initialize the profiling system.

Parameters

<i>level</i>	Initial profiling detail level
--------------	--------------------------------

Returns

0 on success, non-zero on failure

Initialize the profiling system.

Parameters

<i>level</i>	Profiling detail level to enable
--------------	----------------------------------

Returns

0 on success, -1 on failure

References [level](#), [PROFILE_DISABLED](#), and [profiler_state](#).

7.23.4.11 profiler_memory_alloc()

```
void profiler_memory_alloc (
    size_t bytes )
```

7.23.4.12 profiler_memory_read()

```
void profiler_memory_read (
    size_t bytes )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.23.4.13 profiler_memory_write()

```
void profiler_memory_write (
    size_t bytes )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.23.4.14 profiler_print_hotspots()

```
void profiler_print_hotspots (
    void )
```

Print hot word analysis with optimization recommendations.

Analyzes word execution frequency and provides actionable recommendations for performance optimization including:

- Assembly optimization candidates
- Inlining opportunities
- Cache-friendly ordering

References [WordStats::call_count](#), [compare_by_calls\(\)](#), [WordStats::entry](#), [limit](#), [name_len](#), [DictEntry::name_len](#), [print_word_name_safe\(\)](#), and [profiler_state](#).

Referenced by [profiler_print_word_analysis\(\)](#), and [profiler_suggest_optimizations\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.23.4.15 profiler_print_memory_analysis()

```
void profiler_print_memory_analysis (
    void )
```

7.23.4.16 profiler_print_summary()

```
void profiler_print_summary (
    void )
```

References [profiler_generate_report\(\)](#).

Here is the call graph for this function:

7.23.4.17 profiler_print_word_analysis()

```
void profiler_print_word_analysis (
    void )
```

References [profiler_print_hotspots\(\)](#).

Here is the call graph for this function:

7.23.4.18 profiler_reset()

```
void profiler_reset (
    void )
```

Reset all profiling statistics.

References [profiler_state](#).

7.23.4.19 profiler_set_level()

```
void profiler_set_level (
    ProfileLevel level )
```

Change the current profiling detail level.

Parameters

<i>level</i>	New profiling level to set
--------------	----------------------------

References [level](#), and [profiler_state](#).

7.23.4.20 profiler_shutdown()

```
void profiler_shutdown (
    void )
```

Shutdown and cleanup the profiling system.

Shutdown and cleanup the profiling system.

References [profiler_state](#).

7.23.4.21 profiler_start_timer()

```
void profiler_start_timer (
    ProfileCategory category )
```

7.23.4.22 profiler_suggest_optimizations()

```
void profiler_suggest_optimizations (
    void )
```

References [profiler_print_hotspots\(\)](#).

Here is the call graph for this function:

7.23.4.23 profiler_word_count()

```
void profiler_word_count (
    const DictEntry * entry )
```

Track word execution frequency without timing (PROFILE_BASIC+)

Parameters

<i>entry</i>	Dictionary entry of the word being executed
--------------	---

This is a lightweight alternative to `profiler_word_enter/exit` that only increments call counters without timing overhead. Ideal for identifying hot words (frequently executed) for optimization.

Track word execution frequency without timing (PROFILE_BASIC+)

Parameters

<i>entry</i>	Dictionary entry being executed
--------------	---------------------------------

This is a lightweight alternative to `profiler_word_enter/exit` that only tracks execution counts without timing. Use for PROFILE_BASIC level.

References [WordStats::call_count](#), [entry](#), [get_word_stats\(\)](#), [PROFILE_DETAILED](#), and [profiler_state](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.23.4.24 profiler_word_enter()

```
void profiler_word_enter (
```



```
const DictEntry * entry )
```

Start timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry of the word being executed
--------------	---

Start timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry being entered
--------------	--------------------------------

References [entry](#), [get_time_ns\(\)](#), [PROFILE_DETAILED](#), and [profiler_state](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.23.4.25 profiler_word_exit()

```
void profiler_word_exit (
    const DictEntry * entry )
```

End timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry of the word being executed
--------------	---

End timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry being exited
--------------	-------------------------------

References [WordStats::call_count](#), [entry](#), [get_time_ns\(\)](#), [get_word_stats\(\)](#), [WordStats::max_time_ns](#), [WordStats::min_time_ns](#), [physics_metadata_record_latency\(\)](#), [PROFILE_DETAILED](#), [profiler_state](#), and [WordStats::total_time_ns](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.23.5 Variable Documentation

7.23.5.1 g_profiler

```
void* g_profiler [extern]
```

7.24 include/rolling_window_knobs.h File Reference

This graph shows which files directly or indirectly include this file:

Macros

- `#define ADAPTIVE_SHRINK_RATE 75`
- `#define ADAPTIVE_MIN_WINDOW_SIZE 256`
- `#define ADAPTIVE_CHECK_FREQUENCY 256`
- `#define ADAPTIVE_GROWTH_THRESHOLD 1`

7.24.1 Macro Definition Documentation

7.24.1.1 ADAPTIVE_CHECK_FREQUENCY

```
#define ADAPTIVE_CHECK_FREQUENCY 256
```

7.24.1.2 ADAPTIVE_GROWTH_THRESHOLD

```
#define ADAPTIVE_GROWTH_THRESHOLD 1
```

7.24.1.3 ADAPTIVE_MIN_WINDOW_SIZE

```
#define ADAPTIVE_MIN_WINDOW_SIZE 256
```

7.24.1.4 ADAPTIVE_SHRINK_RATE

```
#define ADAPTIVE_SHRINK_RATE 75
```

7.25 include/rolling_window_of_truth.h File Reference

```
#include <stdint.h>
```

```
#include "vm.h"
```

Include dependency graph for rolling_window_of_truth.h: This graph shows which files directly or indirectly include this file:

Functions

- int [rolling_window_init](#) ([RollingWindowOfTruth](#) *window)
Initialize rolling window of truth.
- int [rolling_window_record_execution](#) ([RollingWindowOfTruth](#) *window, uint32_t [word_id](#))
Record a word execution in the rolling window.
- uint32_t [rolling_window_get_recent_sequence](#) (const [RollingWindowOfTruth](#) *window, uint32_t depth, uint32_t *out_sequence)
Get recent execution sequence for context.
- uint32_t [rolling_window_find_hottest_word](#) (const [RollingWindowOfTruth](#) *window, uint32_t dict_size)
Get most frequent word in rolling window.
- uint64_t [rolling_window_count_transition](#) (const [RollingWindowOfTruth](#) *window, uint32_t word_a, uint32_t word_b)
Get transition frequency: how often word_b follows word_a in window.
- int [rolling_window_is_warm](#) (const [RollingWindowOfTruth](#) *window)
Check if window is warm (contains representative data).
- char * [rolling_window_stats_string](#) (const [RollingWindowOfTruth](#) *window)
Get detailed statistics string for diagnostics.
- void [rolling_window_reset](#) ([RollingWindowOfTruth](#) *window)
Reset rolling window (clear all data).
- void [rolling_window_cleanup](#) ([RollingWindowOfTruth](#) *window)
Free all memory allocated for rolling window.
- uint64_t [rolling_window_export_execution_history](#) (const [RollingWindowOfTruth](#) *window, uint32_t *out_
sequence, uint64_t max_count)
Export full execution history for external analysis.
- double [rolling_window_pattern_capture_rate](#) (const [RollingWindowOfTruth](#) *window, uint32_t test_window_
_size)
Analyze pattern capture rate for a given window size.
- uint64_t [rolling_window_measure_diversity](#) (const [RollingWindowOfTruth](#) *window)
Measure pattern diversity in rolling window.

- void [rolling_window_check_adaptive_shrink](#) ([RollingWindowOfTruth](#) *window)
Check and apply adaptive window shrinking.
- void [rolling_window_service](#) ([RollingWindowOfTruth](#) *window)

7.25.1 Function Documentation

7.25.1.1 [rolling_window_check_adaptive_shrink\(\)](#)

```
void rolling_window_check_adaptive_shrink (
    RollingWindowOfTruth * window )
```

Check and apply adaptive window shrinking.

Called periodically by [rolling_window_record_execution](#). Measures pattern diversity and shrinks window if diminishing returns detected.

This function is called automatically - users never invoke it directly. It runs every 256 executions after window becomes warm.

Parameters

<i>window</i>	Rolling window (modified in-place if shrinking occurs)
---------------	--

References [RollingWindowOfTruth::adaptive_pending](#), and [rolling_window_service_internal\(\)](#).

Here is the call graph for this function:

7.25.1.2 [rolling_window_cleanup\(\)](#)

```
void rolling_window_cleanup (
    RollingWindowOfTruth * window )
```

Free all memory allocated for rolling window.

Called during [VM](#) cleanup.

Parameters

<i>window</i>	Rolling window to clean up
---------------	----------------------------

References [RollingWindowOfTruth::execution_history](#), [sf_free\(\)](#), and [RollingWindowOfTruth::snapshot_buffers](#).

Referenced by [vm_cleanup\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.25.1.3 [rolling_window_count_transition\(\)](#)

```
uint64_t rolling_window_count_transition (
    const RollingWindowOfTruth * window,
    uint32_t word_a,
    uint32_t word_b )
```

Get transition frequency: how often word_b follows word_a in window.

Useful for seeding pipelining context predictions.

Parameters

<i>window</i>	Rolling window
<i>word_a</i>	First word in sequence
<i>word_b</i>	Second word in sequence

Returns

Count of (word_a → word_b) transitions in window

References [rolling_window_view_t::history](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), and [rolling_window_snapshot_view\(\)](#).

Here is the call graph for this function:

7.25.1.4 rolling_window_export_execution_history()

```
uint64_t rolling_window_export_execution_history (
    const RollingWindowOfTruth * window,
    uint32_t * out_sequence,
    uint64_t max_count )
```

Export full execution history for external analysis.

Used by bootstrap analyzer to determine optimal window size.

Returns the complete circular buffer linearized in chronological order. Caller allocates output buffer; function fills it with execution sequence.

Parameters

<i>window</i>	Rolling window to export
<i>out_sequence</i>	Output buffer (caller allocates, must be $\geq$ total_executions)
<i>max_count</i>	Maximum entries to write

Returns

Number of word IDs written to out_sequence

References [rolling_window_view_t::history](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Referenced by [extract_heat_trajectory\(\)](#), and [rolling_window_pattern_capture_rate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.25.1.5 rolling_window_find_hottest_word()

```
uint32_t rolling_window_find_hottest_word (
    const RollingWindowOfTruth * window,
    uint32_t dict_size )
```

Get most frequent word in rolling window.

Useful for identifying hot words immediately after POST.

Parameters

<i>window</i>	Rolling window
<i>dict_size</i>	Total number of words in dictionary (for bounds check)

Returns

Word ID of most frequent execution, or 0 if window not warm

References [rolling_window_view_t::history](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_calloc\(\)](#), [sf_free\(\)](#), and [word_id](#).

Here is the call graph for this function:

7.25.1.6 rolling_window_get_recent_sequence()

```
uint32_t rolling_window_get_recent_sequence (
    const RollingWindowOfTruth * window,
```

```
uint32_t depth,
uint32_t * out_sequence )
```

Get recent execution sequence for context.

Returns pointer to last N words (or fewer if not enough executions yet).

Parameters

<i>window</i>	Rolling window
<i>depth</i>	How many previous words to return (1 to ROLLING_WINDOW_SIZE)
<i>out_sequence</i>	Output array (caller allocates, must be $\geq$ depth)

Returns

Number of words written to out_sequence (may be $<$ depth if cold start)

References [rolling_window_view_t::history](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Here is the call graph for this function:

7.25.1.7 rolling_window_init()

```
int rolling_window_init (
    RollingWindowOfTruth * window )
```

Initialize rolling window of truth.

Called during [VM](#) initialization.

Parameters

<i>window</i>	Pointer to uninitialized RollingWindowOfTruth
---------------	---

Returns

0 on success, -1 on malloc failure

References [RollingWindowOfTruth::adaptive_check_accumulator](#), [RollingWindowOfTruth::adaptive_pending](#), [RollingWindowOfTruth::effective_window_size](#), [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [RollingWindowOfTruth::last_pattern_diversity](#), [RollingWindowOfTruth::pattern_diversity_check_count](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [RollingWindowOfTruth::snapshot_buffers](#), [RollingWindowOfTruth::snapshot_effective_index](#), [RollingWindowOfTruth::snapshot_index](#), [RollingWindowOfTruth::snapshot_is_warm](#), [RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::snapshot_total_executions](#), [RollingWindowOfTruth::snapshot_window_pos](#), [RollingWindowOfTruth::total_executions](#), and [RollingWindowOfTruth::window_pos](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.25.1.8 rolling_window_is_warm()

```
int rolling_window_is_warm (
    const RollingWindowOfTruth * window )
```

Check if window is warm (contains representative data).

Seeding functions should check this before using window data.

Parameters

<i>window</i>	Rolling window
---------------	----------------

Returns

1 if window is warm ($\geq$ ROLLING_WINDOW_SIZE executions), 0 otherwise

References [RollingWindowOfTruth::is_warm](#).

7.25.1.9 rolling_window_measure_diversity()

```
uint64_t rolling_window_measure_diversity (
    const RollingWindowOfTruth * window )
```

Measure pattern diversity in rolling window.

Counts unique adjacent transitions (word_a $\rightarrow$ word_b) to indicate workload variability.

Used by dictionary heat optimization to decide lookup strategy:

- High diversity (>70) = use heat-aware lookup (patterns are varied)
- Low diversity (<30) = use naive lookup (patterns are stable)

Parameters

<i>window</i>	Rolling window to analyze
---------------	---------------------------

Returns

Count of unique transitions in window

Measure pattern diversity in rolling window.

Used by adaptive sizing to decide if shrinking is beneficial.

Simple heuristic: count unique word transitions observed. Higher diversity = more patterns = window is capturing something meaningful. If diversity plateaus, shrinking won't lose important data.

References [rolling_window_measure_diversity_view\(\)](#), [rolling_window_publish_snapshot_if_needed\(\)](#), and [rolling_window_snapshot_view\(\)](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.25.1.10 rolling_window_pattern_capture_rate()

```
double rolling_window_pattern_capture_rate (
    const RollingWindowOfTruth * window,
    uint32_t test_window_size )
```

Analyze pattern capture rate for a given window size.

Measures what percentage of unique transitions are captured.

Algorithm:

1. Scan execution history with sliding window of given size
2. Count unique (context, next_word) patterns observed
3. Return: (patterns_captured / total_unique_patterns) * 100

Used for offline analysis to determine reasonable starting window size.

Parameters

<i>window</i>	Rolling window with execution data
<i>test_window_size</i>	Window size to test (e.g., 256, 512, 1024)

Returns

Pattern capture rate (0-100%)

References [rolling_window_view_t::history](#), [rolling_window_export_execution_history\(\)](#), [rolling_window_publish_snapshot_if_needed](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_free\(\)](#), [sf_malloc\(\)](#), and [rolling_window_view_t::total_executions](#).

Here is the call graph for this function:

7.25.1.11 `rolling_window_record_execution()`

```
int rolling_window_record_execution (
    RollingWindowOfTruth * window,
    uint32_t word_id )
```

Record a word execution in the rolling window.

Called after each word executes (in inner interpreter).

THREAD SAFETY: This function is NOT thread-safe. Caller MUST hold `vm->tuning_lock` when calling from multiple threads (main VM + heartbeat).

Called from:

- Main VM thread (inner interpreter loop, [src/vm.c:1162](#))
- Heartbeat worker thread (through `rolling_window_service`, [src/vm.c:743](#))

MUST ALWAYS be protected by: `sf_mutex_lock(&vm->tuning_lock); rolling_window_record_execution(&vm->rolling_window, word_id); sf_mutex_unlock(&vm->tuning_lock);`

Extremely lightweight (constant time):

- Write to circular buffer
- Increment counters
- 2-3 CPU cycles (plus lock overhead ~100 cycles uncontended)

Parameters

<i>window</i>	Rolling window to update
<i>word_id</i>	Word that just executed

Returns

0 on success, -1 on error

References [RollingWindowOfTruth::adaptive_check_accumulator](#), [ADAPTIVE_CHECK_FREQUENCY](#), [RollingWindowOfTruth::adaptive_check_accumulator](#), [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [ROLLING_WINDOW_SIZE](#), [RollingWindowOfTruth::snapshot_pos](#), [RollingWindowOfTruth::total_executions](#), [RollingWindowOfTruth::window_pos](#), and [word_id](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the caller graph for this function:

7.25.1.12 `rolling_window_reset()`

```
void rolling_window_reset (
    RollingWindowOfTruth * window )
```

Reset rolling window (clear all data).

Parameters

<i>window</i>	Rolling window to reset
---------------	-------------------------

References [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [ROLLING_WINDOW_SIZE](#),

[RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::total_executions](#), and [RollingWindowOfTruth::window_pos](#).

7.25.1.13 `rolling_window_service()`

```
void rolling_window_service (
    RollingWindowOfTruth * window )
```

References [rolling_window_service_internal\(\)](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.25.1.14 `rolling_window_stats_string()`

```
char * rolling_window_stats_string (
    const RollingWindowOfTruth * window )
```

Get detailed statistics string for diagnostics.

Parameters

<code>window</code>	Rolling window
---------------------	----------------

Returns

Formatted string (caller must free)

References [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_malloc\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Here is the call graph for this function:

7.26 `include/ssm_jacquard.h` File Reference

```
#include <stdint.h>
```

Include dependency graph for `ssm_jacquard.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [ssm_l8_state_t](#)
- struct [ssm_l8_metrics_t](#)
- struct [ssm_config_t](#)

Macros

- `#define` [SSM_ENTROPY_HIGH_THRESHOLD](#) 0.75 /* Entropy > 0.75 → enable L2 (diversity) */
- `#define` [SSM_CV_HIGH_THRESHOLD](#) 0.15 /* CV > 0.15 → enable L5/L6 (inference) */
- `#define` [SSM_TEMPORAL_DECAY_THRESHOLD](#) 0.5 /* Temporal > 0.5 → enable L3 (decay) */
- `#define` [SSM_TEMPORAL_DECAY_LOW_THRESHOLD](#) 0.3 /* Temporal > 0.3 + CV high → enable L6 */
- `#define` [SSM_HYSTERESIS_TICKS](#) 5 /* Consecutive ticks before mode change */

Enumerations

- enum [ssm_l8_mode_t](#) {
[SSM_MODE_C0](#) = 0x0 , [SSM_MODE_C1](#) = 0x1 , [SSM_MODE_C2](#) = 0x2 , [SSM_MODE_C3](#) = 0x3 ,
[SSM_MODE_C4](#) = 0x4 , [SSM_MODE_C5](#) = 0x5 , [SSM_MODE_C6](#) = 0x6 , [SSM_MODE_C7](#) = 0x7 ,
[SSM_MODE_C8](#) = 0x8 , [SSM_MODE_C9](#) = 0x9 , [SSM_MODE_C10](#) = 0xA , [SSM_MODE_C11](#) = 0xB ,
[SSM_MODE_C12](#) = 0xC , [SSM_MODE_C13](#) = 0xD , [SSM_MODE_C14](#) = 0xE , [SSM_MODE_C15](#) = 0xF }

Functions

- void `ssm_l8_init` (`ssm_l8_state_t` *state, `ssm_l8_mode_t` initial_mode)
Initialize L8 state.
- void `ssm_l8_update` (const `ssm_l8_metrics_t` *metrics, `ssm_l8_state_t` *state)
Update L8 state based on current metrics.
- void `ssm_apply_mode` (const `ssm_l8_state_t` *state, `ssm_config_t` *config)
Apply L8 mode to SSM configuration (set L3/L4 bits)
- const char * `ssm_l8_mode_name` (`ssm_l8_mode_t` mode)
Get human-readable mode name.

7.26.1 Macro Definition Documentation

7.26.1.1 SSM_CV_HIGH_THRESHOLD

```
#define SSM_CV_HIGH_THRESHOLD 0.15 /* CV > 0.15 → enable L5/L6 (inference) */
```

7.26.1.2 SSM_ENTROPY_HIGH_THRESHOLD

```
#define SSM_ENTROPY_HIGH_THRESHOLD 0.75 /* Entropy > 0.75 → enable L2 (diversity) */
```

7.26.1.3 SSM_HYSTERESIS_TICKS

```
#define SSM_HYSTERESIS_TICKS 5 /* Consecutive ticks before mode change */
```

7.26.1.4 SSM_TEMPORAL_DECAY_LOW_THRESHOLD

```
#define SSM_TEMPORAL_DECAY_LOW_THRESHOLD 0.3 /* Temporal > 0.3 + CV high → enable L6 */
```

7.26.1.5 SSM_TEMPORAL_DECAY_THRESHOLD

```
#define SSM_TEMPORAL_DECAY_THRESHOLD 0.5 /* Temporal > 0.5 → enable L3 (decay) */
```

7.26.2 Enumeration Type Documentation

7.26.2.1 ssm_l8_mode_t

```
enum ssm_l8_mode_t
```

Enumerator

SSM_MODE_C0	
SSM_MODE_C1	
SSM_MODE_C2	
SSM_MODE_C3	
SSM_MODE_C4	
SSM_MODE_C5	
SSM_MODE_C6	
SSM_MODE_C7	
SSM_MODE_C8	
SSM_MODE_C9	
SSM_MODE_C10	
SSM_MODE_C11	
SSM_MODE_C12	
SSM_MODE_C13	
SSM_MODE_C14	
SSM_MODE_C15	

7.26.3 Function Documentation

7.26.3.1 `ssm_apply_mode()`

```
void ssm_apply_mode (
    const ssm_l8_state_t * state,
    ssm_config_t * config )
```

Apply L8 mode to SSM configuration (set L3/L4 bits)

Parameters

<i>state</i>	L8 state containing current mode
<i>config</i>	SSM configuration to update (L3/L4 bits)

References [ssm_l8_state_t::current_mode](#), [ssm_config_t::L2_rolling_window](#), [ssm_config_t::L3_linear_decay](#), [ssm_config_t::L5_window_inference](#), and [ssm_config_t::L6_decay_inference](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.26.3.2 `ssm_l8_init()`

```
void ssm_l8_init (
    ssm_l8_state_t * state,
    ssm_l8_mode_t initial_mode )
```

Initialize L8 state.

Parameters

<i>state</i>	L8 state to initialize
<i>initial_mode</i>	Starting mode (default: C0_CRUISE)

References [ssm_l8_state_t::current_mode](#), [ssm_l8_state_t::hysteresis_counter](#), and [ssm_l8_state_t::pending_mode](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.26.3.3 `ssm_l8_mode_name()`

```
const char * ssm_l8_mode_name (
    ssm_l8_mode_t mode )
```

Get human-readable mode name.

Parameters

<i>mode</i>	Mode to describe
-------------	------------------

Returns

Static string describing mode (e.g., "C0_CRUISE")

References [SSM_MODE_C0](#), [SSM_MODE_C1](#), [SSM_MODE_C10](#), [SSM_MODE_C11](#), [SSM_MODE_C12](#), [SSM_MODE_C13](#), [SSM_MODE_C14](#), [SSM_MODE_C15](#), [SSM_MODE_C2](#), [SSM_MODE_C3](#), [SSM_MODE_C4](#), [SSM_MODE_C5](#), [SSM_MODE_C6](#), [SSM_MODE_C7](#), [SSM_MODE_C8](#), and [SSM_MODE_C9](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.26.3.4 `ssm_l8_update()`

```
void ssm_l8_update (
    const ssm_l8_metrics_t * metrics,
    ssm_l8_state_t * state )
```

Update L8 state based on current metrics.

Uses rule-based classification to select mode based on entropy and CV. Includes hysteresis to prevent rapid mode flapping.

Parameters

<i>metrics</i>	Current runtime metrics (ns_ewma, cv, entropy)
<i>state</i>	L8 state (updated in-place)

References [ssm_l8_state_t::current_mode](#), [ssm_l8_metrics_t::cv](#), [ssm_l8_metrics_t::entropy](#), [ssm_l8_state_t::hysteresis_counter](#), [ssm_l8_state_t::pending_mode](#), [SSM_CV_HIGH_THRESHOLD](#), [SSM_ENTROPY_HIGH_THRESHOLD](#), [SSM_HYSTERESIS_TICK](#), [SSM_MODE_C0](#), [SSM_TEMPORAL_DECAY_LOW_THRESHOLD](#), [SSM_TEMPORAL_DECAY_THRESHOLD](#), and [ssm_l8_metrics_t::temporal_decay](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.27 include/starforth_config.h File Reference

This graph shows which files directly or indirectly include this file:

Macros

- [#define STARFORTH_CONFIG_STRICT_PTR_DEFAULT 1](#)
- [#define STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT 0](#)
- [#define STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT 0](#)
- [#define STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT 4096](#)
- [#define STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT 8](#)
- [#define STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT 50](#)
- [#define STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT 256](#)
- [#define STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT 512](#)
- [#define STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT 5](#)
- [#define STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT 21845](#)
- [#define STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT 1000ULL](#)
- [#define STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT 1](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT 1](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT 10000ULL](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT 1000](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT 256](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT 1000](#)
- [#define STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT 5000](#)
- [#define STRICT_PTR STARFORTH_CONFIG_STRICT_PTR_DEFAULT](#)
- [#define VM_STRICT_PTR STRICT_PTR](#)
- [#define ENABLE_HOTWORDS_CACHE STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT](#)
- [#define ENABLE_PIPELINING STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT](#)
- [#define ROLLING_WINDOW_SIZE STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT](#)
- [#define TRANSITION_WINDOW_SIZE STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT](#)
- [#define ADAPTIVE_SHRINK_RATE STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT](#)
- [#define ADAPTIVE_MIN_WINDOW_SIZE STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT](#)
- [#define ADAPTIVE_CHECK_FREQUENCY STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT](#)
- [#define ADAPTIVE_GROWTH_THRESHOLD STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT](#)
- [#define INITIAL_DECAY_SLOPE_Q48 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT](#)
- [#define DECAY_MIN_INTERVAL STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT](#)
- [#define DECAY_RATE_PER_US_Q16 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT](#)
- [#define HEARTBEAT_THREAD_ENABLED STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT](#)
- [#define HEARTBEAT_TICK_NS STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT](#)

- `#define HEARTBEAT_INFERENCE_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT`
- `#define HEARTBEAT_CHECK_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT`
- `#define HEARTBEAT_WINDOW_TUNING_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT`
- `#define HEARTBEAT_SLOPE_VALIDATION_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT`

7.27.1 Macro Definition Documentation

7.27.1.1 ADAPTIVE_CHECK_FREQUENCY

```
#define ADAPTIVE_CHECK_FREQUENCY STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT
```

7.27.1.2 ADAPTIVE_GROWTH_THRESHOLD

```
#define ADAPTIVE_GROWTH_THRESHOLD STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT
```

7.27.1.3 ADAPTIVE_MIN_WINDOW_SIZE

```
#define ADAPTIVE_MIN_WINDOW_SIZE STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT
```

7.27.1.4 ADAPTIVE_SHRINK_RATE

```
#define ADAPTIVE_SHRINK_RATE STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT
```

7.27.1.5 DECAY_MIN_INTERVAL

```
#define DECAY_MIN_INTERVAL STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT
```

7.27.1.6 DECAY_RATE_PER_US_Q16

```
#define DECAY_RATE_PER_US_Q16 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT
```

7.27.1.7 ENABLE_HOTWORDS_CACHE

```
#define ENABLE_HOTWORDS_CACHE STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT
```

7.27.1.8 ENABLE_PIPELINING

```
#define ENABLE_PIPELINING STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT
```

7.27.1.9 HEARTBEAT_CHECK_FREQUENCY

```
#define HEARTBEAT_CHECK_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT
```

7.27.1.10 HEARTBEAT_INFERENCE_FREQUENCY

```
#define HEARTBEAT_INFERENCE_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT
```

7.27.1.11 HEARTBEAT_SLOPE_VALIDATION_FREQUENCY

```
#define HEARTBEAT_SLOPE_VALIDATION_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT
```

7.27.1.12 HEARTBEAT_THREAD_ENABLED

```
#define HEARTBEAT_THREAD_ENABLED STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT
```

7.27.1.13 HEARTBEAT_TICK_NS

```
#define HEARTBEAT_TICK_NS STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT
```

7.27.1.14 HEARTBEAT_WINDOW_TUNING_FREQUENCY

```
#define HEARTBEAT_WINDOW_TUNING_FREQUENCY STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT
```

7.27.1.15 INITIAL_DECAY_SLOPE_Q48

```
#define INITIAL_DECAY_SLOPE_Q48 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT
```

7.27.1.16 ROLLING_WINDOW_SIZE

```
#define ROLLING_WINDOW_SIZE STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT
```

7.27.1.17 STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT

```
#define STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT 512
```

7.27.1.18 STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT

```
#define STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT 5
```

7.27.1.19 STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT

```
#define STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT 256
```

7.27.1.20 STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT

```
#define STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT 50
```

7.27.1.21 STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT

```
#define STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT 1000ULL
```

7.27.1.22 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT

```
#define STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT 1
```

7.27.1.23 STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT

```
#define STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT 0
```

7.27.1.24 STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT

```
#define STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT 0
```

7.27.1.25 STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT 256
```

7.27.1.26 STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT 1000
```

7.27.1.27 STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT 5000
```

7.27.1.28 STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT 1
```

7.27.1.29 STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT 10000ULL
```

7.27.1.30 STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT

```
#define STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT 1000
```

7.27.1.31 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT

```
#define STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT 21845
```

7.27.1.32 STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT

```
#define STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT 4096
```

7.27.1.33 STARFORTH_CONFIG_STRICT_PTR_DEFAULT

```
#define STARFORTH_CONFIG_STRICT_PTR_DEFAULT 1
```

7.27.1.34 STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT

```
#define STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT 8
```

7.27.1.35 STRICT_PTR

```
#define STRICT_PTR STARFORTH\_CONFIG\_STRICT\_PTR\_DEFAULT
```

7.27.1.36 TRANSITION_WINDOW_SIZE

```
#define TRANSITION_WINDOW_SIZE STARFORTH\_CONFIG\_TRANSITION\_WINDOW\_SIZE\_DEFAULT
```

7.27.1.37 VM_STRICT_PTR

```
#define VM_STRICT_PTR STRICT\_PTR
```

7.28 include/starkernel/apic.h File Reference

```
#include "uefi.h"
#include <stdint.h>
```

Include dependency graph for apic.h: This graph shows which files directly or indirectly include this file:

Macros

- #define [APIC_TIMER_VECTOR](#) 0x20
[apic.h](#) - Local APIC interface

Functions

- int [apic_init](#) ([BootInfo](#) *boot_info)
- void [apic_eoi](#) (void)
- int [apic_timer_init](#) (uint64_t tsc_hz, uint32_t tick_hz)
Initialize APIC timer for periodic heartbeat.
- void [apic_timer_start](#) (void)
Start the APIC timer (enables periodic interrupts).
- void [apic_timer_stop](#) (void)
Stop the APIC timer (disables periodic interrupts).
- uint64_t [apic_timer_period_tsc](#) (void)
Get the configured timer period in TSC ticks.

7.28.1 Macro Definition Documentation

7.28.1.1 APIC_TIMER_VECTOR

```
#define APIC_TIMER_VECTOR 0x20
```

[apic.h](#) - Local APIC interface

7.28.2 Function Documentation

7.28.2.1 apic_eoi()

```
void apic_eoi (
    void )
```

References [APIC_REG_EOI](#), and [lapic_write\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.28.2.2 apic_init()

```
int apic_init (
    BootInfo * boot_info )
```

References [APIC_REG_SIVR](#), [console_println\(\)](#), [lapic_base](#), [LAPIC_DEFAULT_PHYS](#), [lapic_phys_base](#), [LAPIC_VIRT_BASE](#), and [lapic_write\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.28.2.3 apic_timer_init()

```
int apic_timer_init (
    uint64_t tsc_hz,
    uint32_t tick_hz )
```

Initialize APIC timer for periodic heartbeat.

Parameters

<i>tsc_hz</i>	TSC frequency in Hz (for calibration)
<i>tick_hz</i>	Desired tick frequency (e.g., 100 = 100 Hz = 10ms period)

Returns

0 on success, -1 on failure

References [APIC_REG_LVT_TIMER](#), [APIC_REG_TIMER_DCR](#), [APIC_REG_TIMER_ICR](#), [APIC_TIMER_VECTOR](#), [calibrate_apic_timer\(\)](#), [console_println\(\)](#), [console_puts\(\)](#), [lapic_write\(\)](#), [LVT_MASKED](#), [LVT_MODE_PERIODIC](#), [TIMER_DIV_1](#), [timer_initial_count](#), [timer_period_tsc_ticks](#), and [timer_tick_hz](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.28.2.4 apic_timer_period_tsc()

```
uint64_t apic_timer_period_tsc (
    void )
```

Get the configured timer period in TSC ticks.

References [timer_period_tsc_ticks](#).

7.28.2.5 apic_timer_start()

```
void apic_timer_start (
    void )
```

Start the APIC timer (enables periodic interrupts).

Call this after IDT and heartbeat handler are set up.

References [APIC_REG_LVT_TIMER](#), [console_puts\(\)](#), [lapic_read\(\)](#), and [lapic_write\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.28.2.6 apic_timer_stop()

```
void apic_timer_stop (
    void )
```

Stop the APIC timer (disables periodic interrupts).

References [APIC_REG_LVT_TIMER](#), [lapic_read\(\)](#), [lapic_write\(\)](#), and [LVT_MASKED](#).

Here is the call graph for this function:

7.29 include/starkernel/arch.h File Reference

```
#include <stdint.h>
```

Include dependency graph for arch.h: This graph shows which files directly or indirectly include this file:

Functions

- void [arch_early_init](#) (void)
[arch.h](#) - Architecture abstraction layer for StarKernel
- void [arch_enable_interrupts](#) (void)
- void [arch_disable_interrupts](#) (void)
- void [arch_interrupts_init](#) (void)
- void [arch_halt](#) (void)
- uint64_t [arch_read_timestamp](#) (void)
- void [arch_mmu_init](#) (void)
- static void [arch_relax](#) (void)

7.29.1 Function Documentation

7.29.1.1 arch_disable_interrupts()

```
void arch_disable_interrupts (
    void )
```

7.29.1.2 arch_early_init()

```
void arch_early_init (
    void )
```

[arch.h](#) - Architecture abstraction layer for StarKernel

Provides a minimal interface used across early boot, interrupt control, low-power halting, timestamp reads, and MMU bring-up.

References [gdt_init\(\)](#).

Here is the call graph for this function:

7.29.1.3 arch_enable_interrupts()

```
void arch_enable_interrupts (
    void )
```

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.29.1.4 arch_halt()

```
void arch_halt (
    void )
```

Referenced by [efi_main\(\)](#), [isr_common_handler\(\)](#), and [kernel_main\(\)](#).

Here is the caller graph for this function:

7.29.1.5 arch_interrupts_init()

```
void arch_interrupts_init (
    void )
```

References [idtr::base](#), [console_println\(\)](#), [idt](#), [IDT_ENTRIES](#), [isr_stub0\(\)](#), [isr_stub_table](#), [lidt\(\)](#), [idtr::limit](#), [pic_disable\(\)](#), and [set_idt_entry\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.29.1.6 arch_mmu_init()

```
void arch_mmu_init (
    void )
```

7.29.1.7 arch_read_timestamp()

```
uint64_t arch_read_timestamp (
    void )
```

Referenced by [calibrate_apic_timer\(\)](#).

Here is the caller graph for this function:

7.29.1.8 arch_relax()

```
static void arch_relax (
    void ) [inline], [static]
```

Referenced by [calibrate_apic_timer\(\)](#), and [console_putc\(\)](#).

Here is the caller graph for this function:

7.30 include/starkernel/capsule.h File Reference

```
#include <stdint.h>
#include <stddef.h>
```

Include dependency graph for capsule.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [CapsuleNameEntry](#)
- struct [CapsuleDirHeader](#)

Macros

- #define [CAPSULE_DESC_MAGIC](#) 0x53504143ULL /* 'CAPS' little-endian */
capsule.h - Init Capsule Architecture (M7.1)
- #define [CAPSULE_DIR_MAGIC](#) 0x44504143ULL /* 'CAPD' little-endian */
- #define [CAPSULE_VERSION_0](#) 0
Version.
- #define [CAPSULE_MAX_COUNT](#) 256
Limits.
- #define [CAPSULE_NAME_MAX](#) 512
Maximum capsule name length (colon-separated path, null-terminated)

- #define CAPSULE_FLAG_ACTIVE 0x00000001 /* Eligible for use */
State flags.
- #define CAPSULE_FLAG_REVOKED 0x00000002 /* Birth-blocked forever */
- #define CAPSULE_FLAG_DEPRECATED 0x00000004 /* Eligible but discouraged */
- #define CAPSULE_FLAG_PINNED 0x00000008 /* Immune to GC */
- #define CAPSULE_FLAG_PRODUCTION 0x00000010 /* (p) truth-bearing */
Mode flags (exactly one must be set for babies)
- #define CAPSULE_FLAG_EXPERIMENT 0x00000020 /* (e) workload only */
- #define CAPSULE_FLAG_MAMA_INIT 0x00000040 /* (m) Mama's init */
Mama init flag (exactly one capsule must have this)
- #define CAPSULE_MODE_VALID(f)
Validate mode flags: exactly one of (p), (e), or (m) must be set.
- #define CAPSULE_IS_MAMA_INIT(f) (((f) & CAPSULE_FLAG_MAMA_INIT) && ((f) & CAPSULE_FLAG_ACTIVE))
Check if capsule is Mama's init.
- #define CAPSULE_BIRTH_ELIGIBLE(f)
Birth eligibility: production, active, not revoked.
- #define CAPSULE_DOE_ELIGIBLE(f)
DoE eligibility: experiment, active, not revoked.
- #define CAPSULE_MAGIC_PACK(ver, alg) (CAPSULE_DESC_MAGIC | ((uint64_t)(ver) << 32) | ((uint64_t)(alg) << 40))
- #define CAPSULE_MAGIC_GET_SIG(m) ((uint32_t)((m) & 0xFFFFFFFFFULL))
- #define CAPSULE_MAGIC_GET_VERSION(m) ((uint8_t)(((m) >> 32) & 0xFF))
- #define CAPSULE_MAGIC_GET_HASHALG(m) ((uint8_t)(((m) >> 40) & 0xFF))

Enumerations

- enum CapsuleHashAlg { CAPSULE_HASH_XXHASH64 = 0 , CAPSULE_HASH_SHA256 = 1 , CAPSULE_HASH_BLAKE3 = 2 }
- enum CapsuleValidateResult { CAPSULE_VALID = 0 , CAPSULE_ERR_BAD_MAGIC , CAPSULE_ERR_BAD_VERSION , CAPSULE_ERR_BAD_HASH_ALG , CAPSULE_ERR_BOUNDS , CAPSULE_ERR_MODE_INVALID , CAPSULE_ERR_REVOKED_ACTIVE , CAPSULE_ERR_HASH_MISMATCH , CAPSULE_ERR_NULL_PTR }

Functions

- struct __attribute__((aligned(64))) CapsuleValidateResult capsule_validate (const CapsuleDesc *desc, const uint8_t *arena_base, uint64_t arena_size, int verify_hash)
capsule_validate - Validate a capsule descriptor
- const char * capsule_validate_result_str (CapsuleValidateResult result)
capsule_validate_result_str - Get string for validation result
- const CapsuleDesc * capsule_find_by_id (const CapsuleDirHeader *dir, const CapsuleDesc *descs, uint64_t id)
capsule_find_by_id - Find capsule by content hash ID
- const CapsuleDesc * capsule_find_by_name (const CapsuleDirHeader *dir, const CapsuleDesc *descs, const CapsuleNameEntry *names, const char *name)
capsule_find_by_name - Find capsule by colon-separated name
- const uint8_t * capsule_get_payload (const CapsuleDesc *desc, const uint8_t *arena_base)
capsule_get_payload - Get pointer to capsule payload bytes
- const CapsuleDesc * capsule_find_mama_init (const CapsuleDirHeader *dir, const CapsuleDesc *descs)
capsule_find_mama_init - Find the Mama init capsule

Variables

- [CapsuleDesc](#)

7.30.1 Macro Definition Documentation**7.30.1.1 CAPSULE_BIRTH_ELIGIBLE**

```
#define CAPSULE_BIRTH_ELIGIBLE(
    f )
```

Value:

```
((f) & CAPSULE_FLAG_PRODUCTION) && \
((f) & CAPSULE_FLAG_ACTIVE) && \
!((f) & CAPSULE_FLAG_REVOKED))
```

Birth eligibility: production, active, not revoked.

7.30.1.2 CAPSULE_DESC_MAGIC

```
#define CAPSULE_DESC_MAGIC 0x53504143ULL /* 'CAPS' little-endian */
```

[capsule.h](#) - Init Capsule Architecture (M7.1)

Content-addressed, immutable init capsules for [VM](#) birth. See docs/lithosananke/M7.1.md for full specification.

Key invariants:

- Exactly ONE production (p) INIT defines a baby [VM](#)
- capsule_id == content_hash (content-addressed)
- No shared/implicit base INITs
- DOMAIN is Mama-only, PERSONALITY is baby-only Magic signatures

7.30.1.3 CAPSULE_DIR_MAGIC

```
#define CAPSULE_DIR_MAGIC 0x44504143ULL /* 'CAPD' little-endian */
```

7.30.1.4 CAPSULE_DOE_ELIGIBLE

```
#define CAPSULE_DOE_ELIGIBLE(
    f )
```

Value:

```
((f) & CAPSULE_FLAG_EXPERIMENT) && \
((f) & CAPSULE_FLAG_ACTIVE) && \
!((f) & CAPSULE_FLAG_REVOKED))
```

DoE eligibility: experiment, active, not revoked.

7.30.1.5 CAPSULE_FLAG_ACTIVE

```
#define CAPSULE_FLAG_ACTIVE 0x00000001 /* Eligible for use */
```

State flags.

7.30.1.6 CAPSULE_FLAG_DEPRECATED

```
#define CAPSULE_FLAG_DEPRECATED 0x00000004 /* Eligible but discouraged */
```

7.30.1.7 CAPSULE_FLAG_EXPERIMENT

```
#define CAPSULE_FLAG_EXPERIMENT 0x00000020 /* (e) workload only */
```

7.30.1.8 CAPSULE_FLAG_MAMA_INIT

```
#define CAPSULE_FLAG_MAMA_INIT 0x00000040 /* (m) Mama's init */
```

Mama init flag (exactly one capsule must have this)

7.30.1.9 CAPSULE_FLAG_PINNED

```
#define CAPSULE_FLAG_PINNED 0x00000008 /* Immune to GC */
```

7.30.1.10 CAPSULE_FLAG_PRODUCTION

```
#define CAPSULE_FLAG_PRODUCTION 0x00000010 /* (p) truth-bearing */
Mode flags (exactly one must be set for babies)
```

7.30.1.11 CAPSULE_FLAG_REVOKED

```
#define CAPSULE_FLAG_REVOKED 0x00000002 /* Birth-blocked forever */
```

7.30.1.12 CAPSULE_IS_MAMA_INIT

```
#define CAPSULE_IS_MAMA_INIT(
    f ) ((f) & CAPSULE_FLAG_MAMA_INIT) && ((f) & CAPSULE_FLAG_ACTIVE))
```

Check if capsule is Mama's init.

7.30.1.13 CAPSULE_MAGIC_GET_HASHALG

```
#define CAPSULE_MAGIC_GET_HASHALG(
    m ) ((uint8_t)((m) >> 40) & 0xFF)
```

7.30.1.14 CAPSULE_MAGIC_GET_SIG

```
#define CAPSULE_MAGIC_GET_SIG(
    m ) ((uint32_t)((m) & 0xFFFFFFFFFULL))
```

7.30.1.15 CAPSULE_MAGIC_GET_VERSION

```
#define CAPSULE_MAGIC_GET_VERSION(
    m ) ((uint8_t)((m) >> 32) & 0xFF)
```

7.30.1.16 CAPSULE_MAGIC_PACK

```
#define CAPSULE_MAGIC_PACK(
    ver,
    alg ) (CAPSULE_DESC_MAGIC | ((uint64_t)(ver) << 32) | ((uint64_t)(alg) << 40))
```

7.30.1.17 CAPSULE_MAX_COUNT

```
#define CAPSULE_MAX_COUNT 256
Limits.
```

7.30.1.18 CAPSULE_MODE_VALID

```
#define CAPSULE_MODE_VALID(
    f )
```

Value:

```
((((f) & CAPSULE_FLAG_MAMA_INIT) != 0) ? \
    (((f) & (CAPSULE_FLAG_PRODUCTION | CAPSULE_FLAG_EXPERIMENT))) : \
    (((f) & CAPSULE_FLAG_PRODUCTION) != 0) ^ \
    (((f) & CAPSULE_FLAG_EXPERIMENT) != 0))
```

Validate mode flags: exactly one of (p), (e), or (m) must be set.

7.30.1.19 CAPSULE_NAME_MAX

```
#define CAPSULE_NAME_MAX 512
Maximum capsule name length (colon-separated path, null-terminated)
```

7.30.1.20 CAPSULE_VERSION_0

```
#define CAPSULE_VERSION_0 0
Version.
```

7.30.2 Enumeration Type Documentation

7.30.2.1 CapsuleHashAlg

```
enum CapsuleHashAlg
```

Enumerator

CAPSULE_HASH_XXHASH64	
CAPSULE_HASH_SHA256	
CAPSULE_HASH_BLAKE3	

7.30.2.2 CapsuleValidateResult

```
enum CapsuleValidateResult
```

Enumerator

CAPSULE_VALID	
CAPSULE_ERR_BAD_MAGIC	
CAPSULE_ERR_BAD_VERSION	
CAPSULE_ERR_BAD_HASH_ALG	
CAPSULE_ERR_BOUNDS	
CAPSULE_ERR_MODE_INVALID	
CAPSULE_ERR_REVOKED_ACTIVE	
CAPSULE_ERR_HASH_MISMATCH	
CAPSULE_ERR_NULL_PTR	

7.30.3 Function Documentation

7.30.3.1 __attribute__()

```
struct __attribute__ (
    (aligned(64)) )
```

7.30.3.2 capsule_find_by_id()

```
const CapsuleDesc * capsule_find_by_id (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * desc,
    uint64_t id )
```

capsule_find_by_id - Find capsule by content hash ID

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array
<i>id</i>	Capsule ID (content hash) to find

Returns

Pointer to descriptor, or NULL if not found

References [CapsuleDirHeader::desc_count](#).

7.30.3.3 capsule_find_by_name()

```
const CapsuleDesc * capsule_find_by_name (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const char * name )
```

capsule_find_by_name - Find capsule by colon-separated name

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array
<i>names</i>	Name entry array (parallel to descs)
<i>name</i>	Colon-separated capsule name, e.g. "core:init.4th"

Returns

Pointer to descriptor, or NULL if not found

References [capsule_streq\(\)](#), [CapsuleDirHeader::desc_count](#), and [name](#).

Referenced by [capsule_birth_baby\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.30.3.4 capsule_find_mama_init()

```
const CapsuleDesc * capsule_find_mama_init (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs )
```

capsule_find_mama_init - Find the Mama init capsule

Searches the descriptor array for the capsule with CAPSULE_FLAG_MAMA_INIT. There must be exactly one such capsule.

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array

Returns

Pointer to Mama's init descriptor, or NULL if not found

References [CAPSULE_IS_MAMA_INIT](#), [CapsuleDirHeader::desc_count](#), and [flags](#).

Referenced by [capsule_birth_mama\(\)](#).

Here is the caller graph for this function:

7.30.3.5 capsule_get_payload()

```
const uint8_t * capsule_get_payload (
    const CapsuleDesc * desc,
    const uint8_t * arena_base )
```

capsule_get_payload - Get pointer to capsule payload bytes

Parameters

<i>desc</i>	Capsule descriptor
<i>arena_base</i>	Base address of payload arena

Returns

Pointer to payload bytes, or NULL on error

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).
Here is the caller graph for this function:

7.30.3.6 capsule_validate()

```
CapsuleValidateResult capsule_validate (
    const CapsuleDesc * desc,
    const uint8_t * arena_base,
    uint64_t arena_size,
    int verify_hash )
```

`capsule_validate` - Validate a capsule descriptor

Parameters

<i>desc</i>	Capsule descriptor to validate
<i>arena_base</i>	Base address of payload arena
<i>arena_size</i>	Size of payload arena in bytes
<i>verify_hash</i>	If true, recompute and compare content hash

Returns

CAPSULE_VALID on success, error code otherwise

References [CAPSULE_DESC_MAGIC](#), [CAPSULE_ERR_BAD_HASH_ALG](#), [CAPSULE_ERR_BAD_MAGIC](#), [CAPSULE_ERR_BAD_VERSION](#), [CAPSULE_ERR_BOUNDS](#), [CAPSULE_ERR_HASH_MISMATCH](#), [CAPSULE_ERR_MODE_INVALID](#), [CAPSULE_ERR_NULL_PTR](#), [CAPSULE_FLAG_ACTIVE](#), [CAPSULE_FLAG_REVOKED](#), [CAPSULE_HASH_BLAKE3](#), [CAPSULE_HASH_XXHASH64](#), [CAPSULE_MAGIC_GET_HASHALG](#), [CAPSULE_MAGIC_GET_SIG](#), [CAPSULE_MAGIC_GET_VERIFY](#), [CAPSULE_MODE_VALID](#), [CAPSULE_VALID](#), [CAPSULE_VERSION_0](#), and [xxhash64_capsule\(\)](#).

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).
Here is the call graph for this function: Here is the caller graph for this function:

7.30.3.7 capsule_validate_result_str()

```
const char * capsule_validate_result_str (
    CapsuleValidateResult result )
```

`capsule_validate_result_str` - Get string for validation result

References [validate_result_strings](#).

7.30.4 Variable Documentation**7.30.4.1 CapsuleDesc**

`CapsuleDesc`

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).

7.31 include/starkernel/capsule_birth.h File Reference

```
#include <stdint.h>
#include "starkernel/capsule.h"
```

```
#include "starkernel/capsule_run.h"
```

Include dependency graph for capsule_birth.h: This graph shows which files directly or indirectly include this file:

Typedefs

- typedef int(* [CapsuleExecFn](#)) (void *vm_ctx, const char *code, uint64_t code_len)
[capsule_birth.h](#) - VM Birth Protocol (M7.1)
- typedef uint64_t(* [CapsuleDictHashFn](#)) (void *vm_ctx)
Dictionary hash function type.
- typedef void (*[CapsuleVMAllocFn](#)) (void)
VM allocation function type (for baby birth)

Functions

- void [capsule_birth_set_hooks](#) ([CapsuleExecFn](#) exec_fn, [CapsuleDictHashFn](#) dict_hash_fn, [CapsuleVMAllocFn](#) vm_alloc_fn)
capsule_birth_set_hooks - Configure VM execution hooks
- [CapsuleRunResult](#) [capsule_birth_mama](#) (void *mama_vm, const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const uint8_t *arena)
capsule_birth_mama - Execute Mama's init capsule
- [CapsuleRunResult](#) [capsule_birth_baby](#) (const char *capsule_name, const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const uint8_t *arena, uint32_t *out_vm_id, void **out_vm_ctx)
capsule_birth_baby - Birth a new VM from a named (p) capsule
- [CapsuleRunResult](#) [capsule_run_experiment](#) (void *mama_vm, const char *capsule_name, const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const uint8_t *arena, uint64_t *out_run_id)
capsule_run_experiment - Execute a named (e) capsule on Mama
- void [capsule_vm_hooks_register](#) (void)
capsule_vm_hooks_register - Wire concrete VM hooks into the capsule subsystem
- void [capsule_vm_registry_init](#) (void)
capsule_vm_registry_init - Initialize VM registry
- int [capsule_vm_registry_get](#) (uint32_t vm_id, [VMRegistryEntry](#) *out)
capsule_vm_registry_get - Get VM registry entry by ID
- uint32_t [capsule_vm_registry_count](#) (void)
capsule_vm_registry_count - Get number of registered VMs

7.31.1 Typedef Documentation

7.31.1.1 CapsuleDictHashFn

```
typedef uint64_t(* CapsuleDictHashFn) (void *vm_ctx)
```

Dictionary hash function type.

Parameters

<code>vm_ctx</code>	Opaque pointer to VM context
---------------------	--

Returns

64-bit hash of dictionary state

7.31.1.2 CapsuleExecFn

```
typedef int(* CapsuleExecFn) (void *vm_ctx, const char *code, uint64_t code_len)
```


[capsule_birth.h](#) - VM Birth Protocol (M7.1)

Functions for birthing VMs from capsules:

- Mama init: Execute core/init.4th to establish Mama's PERSONALITY
- Baby birth: Create new VM from (p) capsule
- Experiment run: Execute (e) capsule on Mama VM execution function type

Parameters

<i>vm_ctx</i>	Opaque pointer to VM context
<i>code</i>	FORTH source code to execute
<i>code_len</i>	Length of code in bytes

Returns

0 on success, non-zero on error

7.31.1.3 CapsuleVMAllocFn

```
typedef void *(* CapsuleVMAllocFn) (void)
```

VM allocation function type (for baby birth)

Returns

Opaque pointer to new VM context, or NULL on failure

7.31.2 Function Documentation

7.31.2.1 capsule_birth_baby()

```
CapsuleRunResult capsule_birth_baby (
    const char * capsule_name,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena,
    uint32_t * out_vm_id,
    void ** out_vm_ctx )
```

capsule_birth_baby - Birth a new VM from a named (p) capsule

Finds the capsule by colon-separated name, validates it, allocates a new VM, executes the capsule payload as IDENTITY, then (if present) executes unit.4th from the baby's block space as PERSONALITY.

Parameters

<i>capsule_name</i>	Colon-separated capsule name, e.g. "production:myvm.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena
<i>out_vm_id</i>	Output: assigned VM ID
<i>out_vm_ctx</i>	Output: new VM context

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [CAPSULE_BIRTH_ELIGIBLE](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [capsule_parity_log_birth\(\)](#), [capsule_parity_log_birth_failed\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_ERR_NOT_ELIGIBLE](#), [CAPSULE_RUN_ERR_STILLBORN](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [dispatch_init_forth\(\)](#), [entry](#), [next_vm_id](#), [vm_alloc_fn](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), [vm_registry_alloc\(\)](#), [VM_STATE_EMBRYO](#), [VM_STATE_LIVE](#), and [VM_STATE_STILLBORN](#).

Here is the call graph for this function:

7.31.2.2 capsule_birth_mama()

```
CapsuleRunResult capsule_birth_mama (
    void * mama_vm,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena )
```

capsule_birth_mama - Execute Mama's init capsule

Finds the MAMA_INIT capsule by flag, validates it, executes it on Mama's [VM](#).

Parameters

<i>mama_vm</i>	Mama's VM context
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [VMRegistryEntry::birth_capsule_id](#), [VMRegistryEntry::birth_dict_hash](#), [capsule_find_mama_init\(\)](#), [capsule_get_payload\(\)](#), [capsule_parity_log_mama_init\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), and [vm_registry](#).

Here is the call graph for this function:

7.31.2.3 capsule_birth_set_hooks()

```
void capsule_birth_set_hooks (
    CapsuleExecFn exec_fn,
    CapsuleDictHashFn dict_hash_fn,
    CapsuleVMAllocFn vm_alloc_fn )
```

capsule_birth_set_hooks - Configure [VM](#) execution hooks

Must be called before any birth operations.

Parameters

<i>exec_fn</i>	Function to execute FORTH code on VM
<i>dict_hash_fn</i>	Function to compute dictionary hash
<i>vm_alloc_fn</i>	Function to allocate new VM (for babies)

References [vm_alloc_fn](#), [vm_dict_hash_fn](#), and [vm_exec_fn](#).

Referenced by [capsule_vm_hooks_register\(\)](#).

Here is the caller graph for this function:

7.31.2.4 capsule_run_experiment()

```
CapsuleRunResult capsule_run_experiment (
    void * mama_vm,
    const char * capsule_name,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena,
    uint64_t * out_run_id )
```

capsule_run_experiment - Execute a named (e) capsule on Mama

Finds the experiment capsule by colon-separated name, validates it, executes it on Mama's [VM](#) without creating a new [VM](#).

Parameters

<i>mama_vm</i>	Mama's VM context
<i>capsule_name</i>	Colon-separated capsule name, e.g. "experiments:doe-l8:init-l8-stable.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena
<i>out_run_id</i>	Output: assigned run ID

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [CAPSULE_DOE_ELIGIBLE](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [CapsuleRunRecord::capsule_hash](#), [CapsuleRunRecord::capsule_id](#), [capsule_parity_log_run\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_ERR_NOT_ELIGIBLE](#), [capsule_run_log_record\(\)](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [CapsuleRunRecord::ended_ns](#), [CapsuleRunRecord::flags](#), [CapsuleRunRecord::post_dict_hash](#), [CapsuleRunRecord::pre_dict_hash](#), [CapsuleRunRecord::reserved](#), [CapsuleRunRecord::result](#), [CapsuleRunRecord::run_id](#), [CapsuleRunRecord::started_ns](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), and [CapsuleRunRecord::vm_id](#).

Here is the call graph for this function:

7.31.2.5 capsule_vm_hooks_register()

```
void capsule_vm_hooks_register (
    void )
```

capsule_vm_hooks_register - Wire concrete [VM](#) hooks into the capsule subsystem

Registers [capsule_exec_hook](#), [capsule_dict_hash_hook](#), and [capsule_vm_alloc_hook](#). Must be called after [vm_init\(\)](#) on Mama's [VM](#) and before any birth operations.

capsule_vm_hooks_register - Wire concrete [VM](#) hooks into the capsule subsystem

Must be called after [vm_init\(\)](#) completes on Mama's [VM](#), before any capsule birth or experiment operations.

References [capsule_birth_set_hooks\(\)](#), [capsule_dict_hash_hook\(\)](#), [capsule_exec_hook\(\)](#), and [capsule_vm_alloc_hook\(\)](#).

Here is the call graph for this function:

7.31.2.6 capsule_vm_registry_count()

```
uint32_t capsule_vm_registry_count (
    void )
```

capsule_vm_registry_count - Get number of registered VMs

References [vm_registry_count](#).

7.31.2.7 capsule_vm_registry_get()

```
int capsule_vm_registry_get (
    uint32_t vm_id,
    VMRegistryEntry * out )
```

capsule_vm_registry_get - Get VM registry entry by ID

Parameters

<i>vm_id</i>	VM ID to look up
<i>out</i>	Output: registry entry

Returns

0 on success, -1 if not found

References [vm_registry](#), and [vm_registry_count](#).

7.31.2.8 capsule_vm_registry_init()

```
void capsule_vm_registry_init (
    void )
```

capsule_vm_registry_init - Initialize VM registry

References [VMRegistryEntry::birth_capsule_id](#), [VMRegistryEntry::birth_dict_hash](#), [VMRegistryEntry::birth_timestamp_ns](#), [VMRegistryEntry::flags](#), [MAX_VMS](#), [next_vm_id](#), [VMRegistryEntry::reserved](#), [VMRegistryEntry::state](#), [VMRegistryEntry::vm_id](#), [vm_registry](#), [vm_registry_count](#), [VM_STATE_EMBRYO](#), and [VM_STATE_LIVE](#).

7.32 include/starkernel/capsule_generated.h File Reference

```
#include <stdint.h>
#include "starkernel/capsule.h"
Include dependency graph for capsule_generated.h:
```

Functions

- [__attribute__](#)((visibility("hidden"))) extern const uint8_t capsule_arena[]
[capsule_generated.h](#) - Declarations for mkcapsule-generated capsule store.

7.32.1 Function Documentation

7.32.1.1 __attribute__()

```
__attribute__ (
    (visibility("hidden")) ) const
```

[capsule_generated.h](#) - Declarations for mkcapsule-generated capsule store.

The matching capsule_generated.c is produced at build time by mkcapsule. These three arrays plus the directory header are compiled into .rodata.

7.33 include/starkernel/capsule_loader.h File Reference

```
#include <stdint.h>
#include "starkernel/capsule.h"
#include "starkernel/capsule_run.h"
```

Include dependency graph for capsule_loader.h: This graph shows which files directly or indirectly include this file:

Functions

- int [capsule_load_blocks](#) (const uint8_t *payload, uint64_t length, uint32_t *out_entry_block)
[capsule_loader.h](#) - Block Capsule Loader and Executor (M7.1)
- void [capsule_clear_blocks](#) (const uint8_t *payload, uint64_t length)
[capsule_clear_blocks](#) - Zero all ramdrive slots referenced in a capsule payload
- [CapsuleRunResult](#) [capsule_exec_init](#) (void *vm, const char *capsule_name, const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const uint8_t *arena)
[capsule_exec_init](#) - Load, execute, and clean up an init capsule

7.33.1 Function Documentation

7.33.1.1 capsule_clear_blocks()

```
void capsule_clear_blocks (
    const uint8_t * payload,
    uint64_t length )
```

[capsule_clear_blocks](#) - Zero all ramdrive slots referenced in a capsule payload

Re-parses the payload for "Block <num>" headers and zeros each corresponding 1024-byte ramdrive slot. Called after execution to reclaim ramdrive space.

Parameters

<i>payload</i>	Raw capsule payload bytes
<i>length</i>	Payload length in bytes

References [blk_get_buffer\(\)](#), [blk_update\(\)](#), [CAPSULE_RAM_OFFSET](#), [is_block_header\(\)](#), and [LOADER_BLOCK_SIZE](#).
Referenced by [capsule_exec_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.33.1.2 capsule_exec_init()

```
CapsuleRunResult capsule_exec_init (
    void * vm,
    const char * capsule_name,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena )
```

[capsule_exec_init](#) - Load, execute, and clean up an init capsule

Full init sequence:

1. Locate capsule by colon-separated name in the capsule directory
2. Validate content hash
3. Write all blocks to the ramdrive ([capsule_load_blocks](#))
4. Execute the entry block via "<num> LOAD" on the [VM](#)
5. Zero ramdrive slots ([capsule_clear_blocks](#))

Parameters

<i>vm</i>	VM to execute on
<i>capsule_name</i>	Colon-separated capsule name, e.g. "init.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [capsule_clear_blocks\(\)](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [capsule_load_blocks\(\)](#), [CAPSULE_RAM_OFFSET](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [copy_device_to_ram\(\)](#), [VM::error](#), [u32_to_dec\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.33.1.3 capsule_load_blocks()

```
int capsule_load_blocks (
    const uint8_t * payload,
    uint64_t length,
    uint32_t * out_entry_block )
```

[capsule_loader.h](#) - Block Capsule Loader and Executor (M7.1)

Parses a capsule payload for "Block <num>" headers and writes each block's content into the correct ramdrive slot, then executes the entry block and zeros the ramdrive slots afterward.

Format: Block <decimal>

<content: up to 1024 bytes, text or binary> Block <decimal>

...

Blocks may appear in any order. Each ramdrive slot is zeroed before content is written. Content exceeding 1024 bytes is silently truncated. `capsule_load_blocks` - Parse a capsule payload and write blocks to the ramdrive

For each "Block <num>" header found in the payload:

1. Zero the 1024-byte ramdrive slot for block <num>
2. Copy content bytes into the slot (excess beyond 1024 silently dropped)
3. Mark the slot dirty via [blk_update\(\)](#)

Content between headers may be text or binary. Anything before the first "Block <num>" header is ignored.

Parameters

<i>payload</i>	Raw capsule payload bytes (not null-terminated)
<i>length</i>	Payload length in bytes
<i>out_entry_block</i>	If non-NULL, receives the first block number found (the execution entry point); set to 0 if no blocks found

Returns

Number of blocks written, or -1 on invalid input

References [is_block_header\(\)](#), and [write_ramdrive_block\(\)](#).

Referenced by [capsule_exec_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.34 include/starkernel/capsule_run.h File Reference

```
#include <stdint.h>
```

Include dependency graph for `capsule_run.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [CapsuleRunRecord](#)
- struct [VMRegistryEntry](#)

Macros

- #define [CAPSULE_MAX_RUN_RECORDS](#) 1024
[capsule_run.h](#) - DoE Run Logging (M7.1)

Enumerations

- enum [CapsuleRunResult](#) {
 [CAPSULE_RUN_OK](#) = 0 , [CAPSULE_RUN_ERR_INVALID](#) , [CAPSULE_RUN_ERR_NOT_ELIGIBLE](#) ,
 [CAPSULE_RUN_ERR_EXEC_FAIL](#) ,
 [CAPSULE_RUN_ERR_HASH_MISMATCH](#) , [CAPSULE_RUN_ERR_STILLBORN](#) }
- enum [VMState](#) { [VM_STATE_EMBRYO](#) = 0 , [VM_STATE_LIVE](#) , [VM_STATE_STILLBORN](#) , [VM_STATE_DEAD](#) }

Functions

- void [capsule_run_log_init](#) (void)
[capsule_run_log_init](#) - Initialize run log
- uint64_t [capsule_run_log_record](#) (const [CapsuleRunRecord](#) *record)
[capsule_run_log_record](#) - Log a run record
- int [capsule_run_log_get](#) (uint64_t run_id, [CapsuleRunRecord](#) *out)
[capsule_run_log_get](#) - Get a run record by ID
- uint32_t [capsule_run_log_count](#) (void)
[capsule_run_log_count](#) - Get number of logged runs
- void [capsule_parity_log_birth](#) (uint32_t vm_id, uint64_t capsule_id, uint64_t capsule_hash, uint64_t dict_hash ↵
[capsule_parity_log_birth](#) - Log VM birth parity record
- void [capsule_parity_log_birth_failed](#) (uint32_t vm_id, uint64_t capsule_id, [CapsuleRunResult](#) error, uint64_t partial_dict_hash)
[capsule_parity_log_birth_failed](#) - Log failed birth
- void [capsule_parity_log_run](#) (uint32_t vm_id, uint64_t run_id, uint64_t capsule_id, uint64_t pre_dict_hash, uint64_t post_dict_hash)
[capsule_parity_log_run](#) - Log DoE run parity record
- void [capsule_parity_log_mama_init](#) (uint64_t capsule_id, uint64_t capsule_hash, uint64_t dict_hash)
[capsule_parity_log_mama_init](#) - Log Mama init parity record
- void [capsule_parity_set_output](#) (void(*putc_fn)(char), void(*puts_fn)(const char *))
[capsule_parity_set_output](#) - Set output hooks for parity logging

7.34.1 Macro Definition Documentation

7.34.1.1 CAPSULE_MAX_RUN_RECORDS

#define [CAPSULE_MAX_RUN_RECORDS](#) 1024

[capsule_run.h](#) - DoE Run Logging (M7.1)

Structures for logging capsule execution runs and VM births. Supports provenance tracking and experiment reproducibility. Phase A: Fixed ring buffer size

7.34.2 Enumeration Type Documentation

7.34.2.1 CapsuleRunResult

enum [CapsuleRunResult](#)

Enumerator

CAPSULE_RUN_OK	
CAPSULE_RUN_ERR_INVALID	

Enumerator

CAPSULE_RUN_ERR_NOT_ELIGIBLE	
CAPSULE_RUN_ERR_EXEC_FAIL	
CAPSULE_RUN_ERR_HASH_MISMATCH	
CAPSULE_RUN_ERR_STILLBORN	

7.34.2.2 VMState

enum [VMState](#)

Enumerator

VM_STATE_EMBRYO	
VM_STATE_LIVE	
VM_STATE_STILLBORN	
VM_STATE_DEAD	

7.34.3 Function Documentation

7.34.3.1 capsule_parity_log_birth()

```
void capsule_parity_log_birth (
    uint32_t vm_id,
    uint64_t capsule_id,
    uint64_t capsule_hash,
    uint64_t dict_hash )
```

capsule_parity_log_birth - Log [VM](#) birth parity record

Output format: PARITY:BIRTH vm_id=N capsule_id=X mode=p capsule_hash=H dict_hash=D

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.34.3.2 capsule_parity_log_birth_failed()

```
void capsule_parity_log_birth_failed (
    uint32_t vm_id,
    uint64_t capsule_id,
    CapsuleRunResult error,
    uint64_t partial_dict_hash )
```

capsule_parity_log_birth_failed - Log failed birth

Output format: PARITY:BIRTH_FAILED vm_id=N capsule_id=X error=E partial_dict_hash=H

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.34.3.3 capsule_parity_log_mama_init()

```
void capsule_parity_log_mama_init (
    uint64_t capsule_id,
    uint64_t capsule_hash,
    uint64_t dict_hash )
```

capsule_parity_log_mama_init - Log Mama init parity record

Output format: PARITY:MAMA_INIT capsule_id=X mode=m capsule_hash=H dict_hash=D

References [parity_put_hex64\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_mama\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.34.3.4 capsule_parity_log_run()

```
void capsule_parity_log_run (
    uint32_t vm_id,
    uint64_t run_id,
    uint64_t capsule_id,
    uint64_t pre_dict_hash,
    uint64_t post_dict_hash )
```

capsule_parity_log_run - Log DoE run parity record

Output format: PARITY:RUN vm_id=N run_id=R capsule_id=X mode=e pre_dict=P post_dict=Q

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), [parity_put_u64\(\)](#), and [parity_puts](#).

Referenced by [capsule_run_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.34.3.5 capsule_parity_set_output()

```
void capsule_parity_set_output (
    void(*) (char) putc_fn,
    void(*) (const char *) puts_fn )
```

capsule_parity_set_output - Set output hooks for parity logging

Parameters

<i>putc</i> ↔ _fn	Function to output a single character
<i>puts</i> ↔ _fn	Function to output a string

References [parity_putc](#), and [parity_puts](#).

7.34.3.6 capsule_run_log_count()

```
uint32_t capsule_run_log_count (
    void )
```

capsule_run_log_count - Get number of logged runs

References [run_log_count](#).

7.34.3.7 capsule_run_log_get()

```
int capsule_run_log_get (
    uint64_t run_id,
    CapsuleRunRecord * out )
```

capsule_run_log_get - Get a run record by ID

Parameters

<i>run</i> ↔ _id	Run ID to retrieve
<i>out</i>	Output record

Returns

0 on success, -1 if not found

References [CAPSULE_MAX_RUN_RECORDS](#), [run_id_counter](#), and [run_log](#).

7.34.3.8 capsule_run_log_init()

```
void capsule_run_log_init (
    void )
```

capsule_run_log_init - Initialize run log

References [CapsuleRunRecord::capsule_hash](#), [CapsuleRunRecord::capsule_id](#), [CAPSULE_MAX_RUN_RECORDS](#), [CapsuleRunRecord::ended_ns](#), [CapsuleRunRecord::flags](#), [CapsuleRunRecord::post_dict_hash](#), [CapsuleRunRecord::pre_dict_hash](#), [CapsuleRunRecord::reserved](#), [CapsuleRunRecord::result_code](#), [CapsuleRunRecord::run_id](#), [run_id_counter](#), [run_log](#), [run_log_count](#), [run_log_head](#), [CapsuleRunRecord::started_ns](#), and [CapsuleRunRecord::vm_id](#).

7.34.3.9 capsule_run_log_record()

```
uint64_t capsule_run_log_record (
    const CapsuleRunRecord * record )
```

capsule_run_log_record - Log a run record

Parameters

<i>record</i>	Record to log
---------------	---------------

Returns

Run ID assigned, or 0 on failure

References [CAPSULE_MAX_RUN_RECORDS](#), [CapsuleRunRecord::run_id](#), [run_id_counter](#), [run_log](#), [run_log_count](#), and [run_log_head](#).

Referenced by [capsule_run_experiment\(\)](#).

Here is the caller graph for this function:

7.35 include/starkernel/console.h File Reference

```
#include <stdint.h>
```

Include dependency graph for console.h: This graph shows which files directly or indirectly include this file:

Functions

- void [console_init](#) (void)
[console.h](#) - Serial console interface for StarKernel
- void [console_putc](#) (char c)
Write a single character to serial console.
- void [console_puts](#) (const char *s)
Write a null-terminated string to serial console.
- void [console_println](#) (const char *s)
Write a string with newline to serial console.
- int [console_getc](#) (void)
Read a single character from serial console (non-blocking) Returns -1 if no character available.
- int [console_poll](#) (void)
Check if character is available for reading.

7.35.1 Function Documentation

7.35.1.1 console_getc()

```
int console_getc (
    void )
```

Read a single character from serial console (non-blocking) Returns -1 if no character available.

References [console_poll\(\)](#), and [inb\(\)](#).

Referenced by [sk_readline\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.35.1.2 console_init()

```
void console_init (
    void )
```

[console.h](#) - Serial console interface for StarKernel

Initialize serial console (UART)

[console.h](#) - Serial console interface for StarKernel

References [inb\(\)](#), [outb\(\)](#), and [SERIAL_SUPPORTED](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.35.1.3 console_poll()

```
int console_poll (
    void )
```

Check if character is available for reading.

Check if character is available for reading.

References [inb\(\)](#).

Referenced by [console_getc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.35.1.4 console_println()

```
void console_println (
    const char * s )
```

Write a string with newline to serial console.

References [console_putc\(\)](#), and [console_puts\(\)](#).

Referenced by [apic_init\(\)](#), [apic_timer_init\(\)](#), [arch_interrupts_init\(\)](#), [converge_tsc_bare_metal\(\)](#), [decode_pf_error\(\)](#), [init_timer_vm_mode\(\)](#), [isr_common_handler\(\)](#), [kernel_main\(\)](#), [print_banner\(\)](#), [print_boot_info\(\)](#), [print_heap_stats\(\)](#), [print_nl\(\)](#), [print_pmm_stats\(\)](#), [print_uint\(\)](#), [sk_fault_handler\(\)](#), [sk_repl\(\)](#), [timer_fatal\(\)](#), [timer_init\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_init_with_host\(\)](#), [vmm_init\(\)](#), [vmm_map_page\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.35.1.5 console_putc()

```
void console_putc (
    char c )
```

Write a single character to serial console.

References [arch_relax\(\)](#), and [outb\(\)](#).

Referenced by [console_println\(\)](#), [console_puts\(\)](#), [converge_tsc_bare_metal\(\)](#), [print_dec\(\)](#), [print_dec_u64\(\)](#), [print_hex64\(\)](#), [print_q48_16\(\)](#), and [sk_readline\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.35.1.6 console_puts()

```
void console_puts (
    const char * s )
```

Write a null-terminated string to serial console.

References [console_putc\(\)](#).

Referenced by [apic_timer_init\(\)](#), [apic_timer_start\(\)](#), [console_println\(\)](#), [converge_tsc_bare_metal\(\)](#), [decode_pf_error\(\)](#), [init_timer_vm_mode\(\)](#), [isr_common_handler\(\)](#), [print_banner\(\)](#), [print_boot_info\(\)](#), [print_hex64\(\)](#), [print_q48_16\(\)](#), [print_str\(\)](#), [print_u32\(\)](#), [print_uint\(\)](#), [sk_repl\(\)](#), [timer_init\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.36 include/starkernel/elf64.h File Reference

```
#include <stdint.h>
```

Include dependency graph for elf64.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [Elf64_Ehdr](#)
- struct [Elf64_Phdr](#)
- struct [Elf64_Shdr](#)
- struct [Elf64_Rela](#)
- struct [Elf64_Sym](#)
- struct [Elf64_Rel](#)

Macros

- #define [EI_MAG0](#) 0
elf64.h - ELF64 structures for kernel loading Minimal ELF64 definitions for the UEFI loader
- #define [EI_MAG1](#) 1
- #define [EI_MAG2](#) 2
- #define [EI_MAG3](#) 3
- #define [EI_CLASS](#) 4
- #define [EI_DATA](#) 5
- #define [EI_VERSION](#) 6
- #define [EI_OSABI](#) 7
- #define [EI_ABIVERSION](#) 8
- #define [EI_PAD](#) 9
- #define [EI_NIDENT](#) 16
- #define [ELFMAG0](#) 0x7F
- #define [ELFMAG1](#) 'E'
- #define [ELFMAG2](#) 'L'
- #define [ELFMAG3](#) 'F'
- #define [ELFCLASS32](#) 1
- #define [ELFCLASS64](#) 2
- #define [ELFDATA2LSB](#) 1 /* Little-endian */
- #define [ELFDATA2MSB](#) 2 /* Big-endian */
- #define [EV_CURRENT](#) 1
- #define [ELFOSABI_NONE](#) 0 /* UNIX System V ABI */
- #define [ET_NONE](#) 0
- #define [ET_REL](#) 1
- #define [ET_EXEC](#) 2
- #define [ET_DYN](#) 3
- #define [ET_CORE](#) 4
- #define [EM_X86_64](#) 62 /* AMD x86-64 */
- #define [EM_AARCH64](#) 183 /* ARM AARCH64 */
- #define [EM_RISCV](#) 243 /* RISC-V */
- #define [PT_NULL](#) 0
- #define [PT_LOAD](#) 1
- #define [PT_DYNAMIC](#) 2

- #define [PT_INTERP](#) 3
- #define [PT_NOTE](#) 4
- #define [PT_SHLIB](#) 5
- #define [PT_PHDR](#) 6
- #define [PT_TLS](#) 7
- #define [PF_X](#) 0x1 /* Execute */
- #define [PF_W](#) 0x2 /* Write */
- #define [PF_R](#) 0x4 /* Read */
- #define [SHT_NULL](#) 0
- #define [SHT_PROGBITS](#) 1
- #define [SHT_SYMTAB](#) 2
- #define [SHT_STRTAB](#) 3
- #define [SHT_RELA](#) 4
- #define [SHT_HASH](#) 5
- #define [SHT_DYNAMIC](#) 6
- #define [SHT_NOTE](#) 7
- #define [SHT_NOBITS](#) 8
- #define [SHT_REL](#) 9
- #define [SHT_SHLIB](#) 10
- #define [SHT_DYNSYM](#) 11
- #define [R_X86_64_NONE](#) 0
- #define [R_X86_64_64](#) 1
- #define [R_X86_64_PC32](#) 2
- #define [R_X86_64_RELATIVE](#) 8
- #define [R_X86_64_32](#) 10
- #define [R_X86_64_32S](#) 11
- #define [R_AARCH64_NONE](#) 0
- #define [R_AARCH64_ABS64](#) 257
- #define [R_AARCH64_RELATIVE](#) 1027
- #define [R_RISCV_NONE](#) 0
- #define [R_RISCV_64](#) 2
- #define [R_RISCV_RELATIVE](#) 3
- #define [ELF64_R_SYM](#)(i) ((i) >> 32)
- #define [ELF64_R_TYPE](#)(i) ((i) & 0xffffffffL)
- #define [ELF64_R_INFO](#)(s, t) ((([Elf64_Xword](#))(s) << 32) + (([Elf64_Xword](#))(t) & 0xffffffffL))

Typedefs

- typedef uint64_t [Elf64_Addr](#)
- typedef uint64_t [Elf64_Off](#)
- typedef uint16_t [Elf64_Half](#)
- typedef uint32_t [Elf64_Word](#)
- typedef int32_t [Elf64_Sword](#)
- typedef uint64_t [Elf64_Xword](#)
- typedef int64_t [Elf64_Sxword](#)

7.36.1 Macro Definition Documentation

7.36.1.1 EI_ABIVERSION

```
#define EI_ABIVERSION 8
```

7.36.1.2 EI_CLASS

```
#define EI_CLASS 4
```

7.36.1.3 EI_DATA

```
#define EI_DATA 5
```

7.36.1.4 EI_MAG0

```
#define EI_MAG0 0
```

[elf64.h](#) - ELF64 structures for kernel loading Minimal ELF64 definitions for the UEFI loader

7.36.1.5 EI_MAG1

```
#define EI_MAG1 1
```

7.36.1.6 EI_MAG2

```
#define EI_MAG2 2
```

7.36.1.7 EI_MAG3

```
#define EI_MAG3 3
```

7.36.1.8 EI_NIDENT

```
#define EI_NIDENT 16
```

7.36.1.9 EI_OSABI

```
#define EI_OSABI 7
```

7.36.1.10 EI_PAD

```
#define EI_PAD 9
```

7.36.1.11 EI_VERSION

```
#define EI_VERSION 6
```

7.36.1.12 ELF64_R_INFO

```
#define ELF64_R_INFO(  
    s,  
    t ) (((Elf64_Xword)(s) << 32) + ((Elf64_Xword)(t) & 0xffffffffL))
```

7.36.1.13 ELF64_R_SYM

```
#define ELF64_R_SYM(  
    i ) ((i) >> 32)
```

7.36.1.14 ELF64_R_TYPE

```
#define ELF64_R_TYPE(  
    i ) ((i) & 0xffffffffL)
```

7.36.1.15 ELFCLASS32

```
#define ELFCLASS32 1
```

7.36.1.16 ELFCLASS64

```
#define ELFCLASS64 2
```

7.36.1.17 ELFDATA2LSB

```
#define ELFDATA2LSB 1 /* Little-endian */
```

7.36.1.18 ELFDATA2MSB

```
#define ELFDATA2MSB 2 /* Big-endian */
```

7.36.1.19 ELFMAG0

```
#define ELFMAG0 0x7F
```

7.36.1.20 ELFMAG1

```
#define ELFMAG1 'E'
```

7.36.1.21 ELFMAG2

```
#define ELFMAG2 'L'
```

7.36.1.22 ELFMAG3

```
#define ELFMAG3 'F'
```

7.36.1.23 ELFOSABI_NONE

```
#define ELFOSABI_NONE 0 /* UNIX System V ABI */
```

7.36.1.24 EM_AARCH64

```
#define EM_AARCH64 183 /* ARM AARCH64 */
```

7.36.1.25 EM_RISCV

```
#define EM_RISCV 243 /* RISC-V */
```

7.36.1.26 EM_X86_64

```
#define EM_X86_64 62 /* AMD x86-64 */
```

7.36.1.27 ET_CORE

```
#define ET_CORE 4
```

7.36.1.28 ET_DYN

```
#define ET_DYN 3
```

7.36.1.29 ET_EXEC

```
#define ET_EXEC 2
```

7.36.1.30 ET_NONE

```
#define ET_NONE 0
```

7.36.1.31 ET_REL

```
#define ET_REL 1
```

7.36.1.32 EV_CURRENT

```
#define EV_CURRENT 1
```

7.36.1.33 PF_R

```
#define PF_R 0x4 /* Read */
```

7.36.1.34 PF_W

```
#define PF_W 0x2 /* Write */
```

7.36.1.35 PF_X

```
#define PF_X 0x1 /* Execute */
```

7.36.1.36 PT_DYNAMIC

```
#define PT_DYNAMIC 2
```

7.36.1.37 PT_INTERP

```
#define PT_INTERP 3
```

7.36.1.38 PT_LOAD

```
#define PT_LOAD 1
```

7.36.1.39 PT_NOTE

```
#define PT_NOTE 4
```

7.36.1.40 PT_NULL

```
#define PT_NULL 0
```

7.36.1.41 PT_PHDR

```
#define PT_PHDR 6
```

7.36.1.42 PT_SHLIB

```
#define PT_SHLIB 5
```

7.36.1.43 PT_TLS

```
#define PT_TLS 7
```

7.36.1.44 R_AARCH64_ABS64

```
#define R_AARCH64_ABS64 257
```

7.36.1.45 R_AARCH64_NONE

```
#define R_AARCH64_NONE 0
```

7.36.1.46 R_AARCH64_RELATIVE

```
#define R_AARCH64_RELATIVE 1027
```


7.36.1.47 R_RISCV_64

```
#define R_RISCV_64 2
```

7.36.1.48 R_RISCV_NONE

```
#define R_RISCV_NONE 0
```

7.36.1.49 R_RISCV_RELATIVE

```
#define R_RISCV_RELATIVE 3
```

7.36.1.50 R_X86_64_32

```
#define R_X86_64_32 10
```

7.36.1.51 R_X86_64_32S

```
#define R_X86_64_32S 11
```

7.36.1.52 R_X86_64_64

```
#define R_X86_64_64 1
```

7.36.1.53 R_X86_64_NONE

```
#define R_X86_64_NONE 0
```

7.36.1.54 R_X86_64_PC32

```
#define R_X86_64_PC32 2
```

7.36.1.55 R_X86_64_RELATIVE

```
#define R_X86_64_RELATIVE 8
```

7.36.1.56 SHT_DYNAMIC

```
#define SHT_DYNAMIC 6
```

7.36.1.57 SHT_DYNSYM

```
#define SHT_DYNSYM 11
```

7.36.1.58 SHT_HASH

```
#define SHT_HASH 5
```

7.36.1.59 SHT_NOBITS

```
#define SHT_NOBITS 8
```

7.36.1.60 SHT_NOTE

```
#define SHT_NOTE 7
```

7.36.1.61 SHT_NULL

```
#define SHT_NULL 0
```

7.36.1.62 SHT_PROGBITS

```
#define SHT_PROGBITS 1
```

7.36.1.63 SHT_REL

```
#define SHT_REL 9
```

7.36.1.64 SHT_RELA

```
#define SHT_RELA 4
```

7.36.1.65 SHT_SHLIB

```
#define SHT_SHLIB 10
```

7.36.1.66 SHT_STRTAB

```
#define SHT_STRTAB 3
```

7.36.1.67 SHT_SYMTAB

```
#define SHT_SYMTAB 2
```

7.36.2 Typedef Documentation**7.36.2.1 Elf64_Addr**

```
typedef uint64_t Elf64_Addr
```

7.36.2.2 Elf64_Half

```
typedef uint16_t Elf64_Half
```

7.36.2.3 Elf64_Off

```
typedef uint64_t Elf64_Off
```

7.36.2.4 Elf64_Sword

```
typedef int32_t Elf64_Sword
```

7.36.2.5 Elf64_Sxword

```
typedef int64_t Elf64_Sxword
```

7.36.2.6 Elf64_Word

```
typedef uint32_t Elf64_Word
```

7.36.2.7 Elf64_Xword

```
typedef uint64_t Elf64_Xword
```

7.37 include/starkernel/elf_loader.h File Reference

```
#include "elf64.h"
```

Include dependency graph for elf_loader.h: This graph shows which files directly or indirectly include this file:

Functions

- int [elf_load_kernel](#) (const uint8_t *elf_data, uint64_t elf_size, [Elf64_Addr](#) *entry_out)
[elf_loader.h](#) - ELF64 kernel loader interface

7.37.1 Function Documentation

7.37.1.1 elf_load_kernel()

```
int elf_load_kernel (
    const uint8_t * elf_data,
    uint64_t elf_size,
    Elf64_Addr * entry_out )
```

[elf_loader.h](#) - ELF64 kernel loader interface
Load and relocate the StarKernel ELF binary

Parameters

<i>elf_data</i>	Pointer to ELF file in memory
<i>elf_size</i>	Size of ELF file in bytes (unused but for future validation)
<i>entry_out</i>	Output: kernel entry point address

Returns

1 on success, 0 on failure

[elf_loader.h](#) - ELF64 kernel loader interface

Parameters

<i>elf_data</i>	Pointer to ELF file in memory
<i>elf_size</i>	Size of ELF file in bytes
<i>entry_out</i>	Output: kernel entry point address

Returns

1 on success, 0 on failure

References [Elf64_Ehdr::e_entry](#), [Elf64_Ehdr::e_type](#), [elf_apply_relocations\(\)](#), [elf_load_segments\(\)](#), [elf_validate_header\(\)](#), [ET_DYN](#), and [ET_EXEC](#).

Referenced by [efi_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.38 include/starkernel/hal/hal.h File Reference

7.39 include/starkernel/hal_memory.h File Reference

```
#include <stddef.h>
```

Include dependency graph for hal_memory.h: This graph shows which files directly or indirectly include this file:

Functions

- void * [hal_mem_alloc](#) (size_t size)
[hal_memory.h](#) - HAL memory allocation wrappers
- void * [hal_mem_alloc_aligned](#) (size_t size, size_t align)
- void [hal_mem_free](#) (void *ptr)

7.39.1 Function Documentation

7.39.1.1 `hal_mem_alloc()`

```
void * hal_mem_alloc (
    size_t size )
```

[hal_memory.h](#) - HAL memory allocation wrappers

[hal_memory.h](#) - HAL memory allocation wrappers

References [kmalloc\(\)](#).

Here is the call graph for this function:

7.39.1.2 `hal_mem_alloc_aligned()`

```
void * hal_mem_alloc_aligned (
    size_t size,
    size_t align )
```

References [kmalloc_aligned\(\)](#).

Here is the call graph for this function:

7.39.1.3 `hal_mem_free()`

```
void hal_mem_free (
    void * ptr )
```

References [kfree\(\)](#).

Here is the call graph for this function:

7.40 `include/starkernel/kmalloc.h` File Reference

```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for `kmalloc.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [kmalloc_stats_t](#)
[kmalloc.h](#) - Kernel heap allocator interface

Functions

- int [kmalloc_init](#) (uint64_t heap_size_bytes)
- int [kmalloc_is_initialized](#) (void)
- void * [kmalloc](#) (size_t size)
- void * [kmalloc_aligned](#) (size_t size, size_t align)
- void [kfree](#) (void *ptr)
- [kmalloc_stats_t](#) [kmalloc_get_stats](#) (void)
- uintptr_t [kmalloc_heap_base_addr](#) (void)
- uintptr_t [kmalloc_heap_end_addr](#) (void)

7.40.1 Function Documentation

7.40.1.1 `kfree()`

```
void kfree (
    void * ptr )
```

References [account_free\(\)](#), [kmalloc_prefix_t::block](#), [coalesce_neighbors\(\)](#), [heap_block::free](#), [heap_initialized](#), [KMALLOC_MAGIC](#), [kmalloc_prefix_t::magic](#), [pointer_within_heap\(\)](#), and [heap_block::requested](#).

Referenced by [hal_mem_free\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.40.1.2 kmalloc()

```
void * kmalloc (
    size_t size )
```

References [kmalloc_aligned\(\)](#), and [KMALLOC_MIN_ALIGN](#).

Referenced by [hal_mem_alloc\(\)](#), and [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.40.1.3 kmalloc_aligned()

```
void * kmalloc_aligned (
    size_t size,
    size_t align )
```

References [allocate_from_block\(\)](#), [heap_block::free](#), [heap_head](#), [heap_initialized](#), [heap_block::next](#), and [normalize_alignment\(\)](#).

Referenced by [hal_mem_alloc_aligned\(\)](#), and [kmalloc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.40.1.4 kmalloc_get_stats()

```
kmalloc_stats_t kmalloc_get_stats (
    void )
```

References [heap_stats](#).

Referenced by [print_heap_stats\(\)](#).

Here is the caller graph for this function:

7.40.1.5 kmalloc_heap_base_addr()

```
uintptr_t kmalloc_heap_base_addr (
    void )
```

References [heap_base](#).

7.40.1.6 kmalloc_heap_end_addr()

```
uintptr_t kmalloc_heap_end_addr (
    void )
```

References [heap_base](#), and [heap_size](#).

7.40.1.7 kmalloc_init()

```
int kmalloc_init (
    uint64_t heap_size_bytes )
```

References [heap_block::free](#), [kmalloc_stats_t::free_bytes](#), [heap_base](#), [heap_head](#), [heap_initialized](#), [heap_size](#), [heap_stats](#), [KMALLOC_DEFAULT_HEAP_SIZE](#), [heap_block::next](#), [kmalloc_stats_t::peak_bytes](#), [pmm_alloc_contiguous\(\)](#), [pmm_is_initialized\(\)](#), [PMM_PAGE_SIZE](#), [heap_block::prev](#), [heap_block::requested](#), [heap_block::size](#), [kmalloc_stats_t::total_bytes](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.40.1.8 kmalloc_is_initialized()

```
int kmalloc_is_initialized (
    void )
```

References [heap_initialized](#).

7.41 include/starkernel/pmm.h File Reference

```
#include <stdint.h>
#include <stddef.h>
```

```
#include "uefi.h"
```

Include dependency graph for pmm.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [pmm_stats_t](#)

Macros

- #define [PMM_PAGE_SIZE](#) 4096u
[pmm.h](#) - Physical Memory Manager interface for StarKernel

Functions

- int [pmm_init](#) ([BootInfo](#) *boot_info)
- int [pmm_is_initialized](#) (void)
- uint64_t [pmm_alloc_page](#) (void)
- uint64_t [pmm_alloc_contiguous](#) (uint64_t num_pages)
- void [pmm_free_page](#) (uint64_t paddr)
- void [pmm_free_contiguous](#) (uint64_t paddr, uint64_t num_pages)
- [pmm_stats_t](#) [pmm_get_stats](#) (void)

7.41.1 Macro Definition Documentation

7.41.1.1 PMM_PAGE_SIZE

```
#define PMM_PAGE_SIZE 4096u
```

[pmm.h](#) - Physical Memory Manager interface for StarKernel

Bitmap-based allocator for 4KB physical pages. Initialization parses the UEFI memory map provided in [BootInfo](#) and exposes simple allocation and statistics helpers for early kernel bring-up.

7.41.2 Function Documentation

7.41.2.1 pmm_alloc_contiguous()

```
uint64_t pmm_alloc_contiguous (
    uint64_t num_pages )
```

References [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Referenced by [kmalloc_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.41.2.2 pmm_alloc_page()

```
uint64_t pmm_alloc_page (
    void )
```

References [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Referenced by [alloc_page_zeroed\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.41.2.3 pmm_free_contiguous()

```
void pmm_free_contiguous (
    uint64_t paddr,
    uint64_t num_pages )
```

References [bitmap_mark_range\(\)](#), [pmm_initialized](#), [PMM_PAGE_SIZE](#), and [pmm_tracked_pages](#).

Referenced by [pmm_free_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.41.2.4 pmm_free_page()

```
void pmm_free_page (
    uint64_t paddr )
```

References [pmm_free_contiguous\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Here is the call graph for this function:

7.41.2.5 pmm_get_stats()

```
pmm_stats_t pmm_get_stats (
    void )
```

References [pmm_stats_t::free_bytes](#), [pmm_stats_t::free_pages](#), [pmm_free_pages](#), [PMM_PAGE_SIZE](#), [pmm_total_pages](#), [pmm_stats_t::total_bytes](#), [pmm_stats_t::total_pages](#), [pmm_stats_t::used_bytes](#), and [pmm_stats_t::used_pages](#).

Referenced by [print_pmm_stats\(\)](#).

Here is the caller graph for this function:

7.41.2.6 pmm_init()

```
int pmm_init (
    BootInfo * boot_info )
```

References [__kernel_end](#), [__kernel_start](#), [bitmap_mark_range\(\)](#), [is_ram_type\(\)](#), [is_usable_memory\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [EFI_MEMORY_DESCRIPTOR::NumberOfPages](#), [EFI_MEMORY_DESCRIPTOR::PhysicalStart](#), [pmm_alloc_floor_page](#), [pmm_bitmap](#), [PMM_BITMAP_BITS](#), [pmm_bytes_to_pages\(\)](#), [pmm_free_pages](#), [pmm_initialized](#), [pmm_memset\(\)](#), [PMM_PAGE_SIZE](#), [pmm_total_pages](#), [pmm_tracked_pages](#), [recompute_free_pages\(\)](#), and [EFI_MEMORY_DESCRIPTOR::Type](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.41.2.7 pmm_is_initialized()

```
int pmm_is_initialized (
    void )
```

References [pmm_initialized](#).

Referenced by [kmallocc_init\(\)](#), and [vmm_init\(\)](#).

Here is the caller graph for this function:

7.42 include/q48_16.h File Reference

```
#include <stdint.h>
```

Include dependency graph for q48_16.h: This graph shows which files directly or indirectly include this file:

Typedefs

- `typedef uint64_t q48_16_t`

Functions

- `q48_16_t q48_mul (q48_16_t a, q48_16_t b)`
Multiply two Q48.16 values, keeping result in Q48.16.
- `q48_16_t q48_div (q48_16_t a, q48_16_t b)`
Divide two Q48.16 values, keeping result in Q48.16.
- `static q48_16_t q48_add (q48_16_t a, q48_16_t b)`
Add two Q48.16 values.
- `static q48_16_t q48_sub (q48_16_t a, q48_16_t b)`
Subtract two Q48.16 values.
- `static q48_16_t q48_abs (q48_16_t a)`
Absolute value of Q48.16.

- static `q48_16_t q48_from_u64` (uint64_t u)
Convert unsigned 64-bit integer to Q48.16.
- static uint64_t `q48_to_u64` (q48_16_t q)
Convert Q48.16 to unsigned 64-bit integer (truncate fractional)
- `q48_16_t q48_from_double` (double d)
Convert double to Q48.16 (for testing/initialization only)
- double `q48_to_double` (q48_16_t q)
Convert Q48.16 to double (for testing/diagnostics only)
- `q48_16_t q48_log_approx` (uint64_t x)
Approximate natural logarithm in Q48.16 (integer-only)
- `q48_16_t q48_exp_approx` (q48_16_t q)
Approximate exponential e^x in Q48.16 (integer-only, future use)
- `q48_16_t q48_sqrt_approx` (q48_16_t q)
Approximate square root in Q48.16 (integer-only)
- const char * `q48_to_string` (q48_16_t q)
Pretty-print Q48.16 value to string for debugging.
- int `q48_is_valid` (q48_16_t q)
Validate Q48.16 value is within safe range.

7.42.1 Typedef Documentation

7.42.1.1 q48_16_t

```
typedef uint64_t q48_16_t
```

7.42.2 Function Documentation

7.42.2.1 q48_abs()

```
static q48_16_t q48_abs (
    q48_16_t a ) [inline], [static]
```

Absolute value of Q48.16.

Parameters

<i>a</i>	Q48.16 value
----------	--------------

Returns

|*a*| in Q48.16 format

7.42.2.2 q48_add()

```
static q48_16_t q48_add (
    q48_16_t a,
    q48_16_t b ) [inline], [static]
```

Add two Q48.16 values.

Math: $(a / 2^{16}) + (b / 2^{16}) = (a + b) / 2^{16}$ Returns: $a + b$ (no shift needed, both have same scaling)

Parameters

<i>a</i>	First Q48.16 operand
<i>b</i>	Second Q48.16 operand

Returns

$a + b$ in Q48.16 format

Referenced by [compute_levene_statistic\(\)](#), [infer_decay_slope_q48\(\)](#), [q48_exp_approx\(\)](#), [q48_log_approx\(\)](#), and [variance_to_trust\(\)](#).

Here is the caller graph for this function:

7.42.2.3 q48_div()

```
q48_16_t q48_div (
    q48_16_t a,
    q48_16_t b )
```

Divide two Q48.16 values, keeping result in Q48.16.

Math: $(a / 2^{16}) / (b / 2^{16}) * 2^{16} = (a \ll 16) / b$ Returns: $(a \ll 16) / b$

Parameters

<i>a</i>	Dividend in Q48.16
<i>b</i>	Divisor in Q48.16 (non-zero)

Returns

$(a \ll 16) / b$ in Q48.16 format

Referenced by [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), [has_variance_stabilized\(\)](#), and [variance_to_trust\(\)](#).

Here is the caller graph for this function:

7.42.2.4 q48_exp_approx()

```
q48_16_t q48_exp_approx (
    q48_16_t q )
```

Approximate exponential e^x in Q48.16 (integer-only, future use)

Purpose: Might be needed for inverse operations or model validation Status: Defined but not required for Phase 1 inference engine Method: Similar to [q48_log_approx](#) (piecewise + Newton)

Parameters

<i>q</i>	Q48.16 exponent
----------	-----------------

Returns

e^q in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), and [Q48_ONE](#).

Here is the call graph for this function:

7.42.2.5 q48_from_double()

```
q48_16_t q48_from_double (
    double d )
```

Convert double to Q48.16 (for testing/initialization only)

NOT used in inference engine (inference is integer-only). Provided for testing and diagnostic tools only.

Parameters

<i>d</i>	Double-precision float
----------	------------------------

Returns

$d * 2^{16}$ as Q48.16

References [zero](#).

Referenced by [find_variance_inflection\(\)](#).

Here is the caller graph for this function:

7.42.2.6 q48_from_u64()

```
static q48_16_t q48_from_u64 (
    uint64_t u ) [inline], [static]
```

Convert unsigned 64-bit integer to Q48.16.

Math: $u * 2^{16}$ (shift left by 16 bits) Example: `q48_from_u64(1) = 0x10000` (1.0 in Q48.16)

Parameters

<i>u</i>	Unsigned 64-bit integer (max $2^{48}-1$ to avoid overflow)
----------	--

Returns

$u \ll 16$ in Q48.16 format

Referenced by [compute_fit_quality\(\)](#), [compute_levene_statistic\(\)](#), [compute_mean_q48\(\)](#), [compute_variance_q48\(\)](#), [infer_decay_slope_q48\(\)](#), [inference_outputs_validate\(\)](#), [q48_exp_approx\(\)](#), and [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.42.2.7 q48_is_valid()

```
int q48_is_valid (
    q48_16_t q )
```

Validate Q48.16 value is within safe range.

Checks: Does not overflow on mul/div operations Returns: 1 if valid, 0 if risky

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

1 if safe, 0 if risky

7.42.2.8 q48_log_approx()

```
q48_16_t q48_log_approx (
    uint64_t x )
```

Approximate natural logarithm in Q48.16 (integer-only)

Purpose: Required for decay slope inference via exponential fitting Model: $\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} * t$

Method: Piecewise linear approximation or Newton-Raphson iteration

- Input x must be > 0 (assertion check)
- Output is $\ln(x)$ in Q48.16 format
- Precision: $\sim 5\%$ error acceptable (tuning parameter)
- No floating-point operations

Implementation Strategy:

1. Use bit position ($\text{floor}(\log_2(x))$) as coarse estimate

2. Refine with Newton iteration (3-4 cycles for Q48.16 precision)
3. Keep all arithmetic in integer space

Example: $\text{q48_log_approx}(65536) = \log(1.0)$ in Q48.16 ≈ 0 $\text{q48_log_approx}(131072) = \log(2.0)$ in Q48.16 $\approx 0x0000B172$

Parameters

<i>x</i>	Value to take logarithm of ($x > 0$)
----------	--

Returns

$\ln(x)$ in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_exp_approx\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), [Q48_ONE](#), and [q48_sub\(\)](#).
Referenced by [infer_decay_slope_q48\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.42.2.9 q48_mul()

```
q48_16_t q48_mul (
    q48_16_t a,
    q48_16_t b )
```

Multiply two Q48.16 values, keeping result in Q48.16.

Math: $(a / 2^{16}) * (b / 2^{16}) * 2^{16} = (a * b) / 2^{16}$ Returns: $(a * b) >> 16$

Parameters

<i>a</i>	First Q48.16 operand
<i>b</i>	Second Q48.16 operand

Returns

$(a * b) >> 16$ in Q48.16 format

Multiply two Q48.16 values, keeping result in Q48.16.

Freestanding implementation: no libc, no floating-point.

Referenced by [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), and [infer_decay_slope_q48\(\)](#).

Here is the caller graph for this function:

7.42.2.10 q48_sqrt_approx()

```
q48_16_t q48_sqrt_approx (
    q48_16_t q )
```

Approximate square root in Q48.16 (integer-only)

Purpose: Compute R^2 (fit quality) for regression diagnostics Method: Newton-Raphson for integer-only sqrt

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

$\text{sqrt}(q)$ in Q48.16 format

References [q48_div\(\)](#), and [Q48_ONE](#).

Here is the call graph for this function:

7.42.2.11 q48_sub()

```
static q48_16_t q48_sub (
    q48_16_t a,
    q48_16_t b ) [inline], [static]
```

Subtract two Q48.16 values.

Math: $(a / 2^{16}) - (b / 2^{16}) = (a - b) / 2^{16}$ Returns: $a - b$ (no shift needed)

Parameters

<i>a</i>	Minuend in Q48.16
<i>b</i>	Subtrahend in Q48.16

Returns

$a - b$ in Q48.16 format

Referenced by [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.42.2.12 q48_to_double()

```
double q48_to_double (
    q48_16_t q )
```

Convert Q48.16 to double (for testing/diagnostics only)

NOT used in inference engine (inference is integer-only). Provided for logging and RStudio dashboard output.

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

$q / 2^{16}$ as double

Referenced by [inference_outputs_to_string\(\)](#).

Here is the caller graph for this function:

7.42.2.13 q48_to_string()

```
const char * q48_to_string (
    q48_16_t q )
```

Pretty-print Q48.16 value to string for debugging.

Format: "integer.fractional" (e.g., "1.50000") Uses internal static buffer - NOT thread-safe

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

Pointer to static string (valid until next call)

7.42.2.14 q48_to_u64()

```
static uint64_t q48_to_u64 (
    q48_16_t q ) [inline], [static]
```

Convert Q48.16 to unsigned 64-bit integer (truncate fractional)

Math: $q / 2^{16}$ (shift right by 16 bits) Example: `q48_to_u64(0x10000) = 1`

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

q >> 16 as unsigned 64-bit integer

Referenced by [compute_variance_q48\(\)](#).

Here is the caller graph for this function:

7.43 include/starkernel/q48_16.h File Reference

```
#include <stdint.h>
```

Include dependency graph for q48_16.h: This graph shows which files directly or indirectly include this file:

Macros

- `#define Q48_ONE ((q48_16_t)0x10000ULL)`

Typedefs

- `typedef uint64_t q48_16_t`
q48_16.h - Q48.16 Fixed-Point Arithmetic for StarkKernel

Functions

- `q48_16_t q48_mul (q48_16_t a, q48_16_t b)`
*Multiply two Q48.16 values: (a * b) >> 16.*
- `q48_16_t q48_div (q48_16_t a, q48_16_t b)`
Divide two Q48.16 values: (a << 16) / b Returns 0 if b == 0.
- `static q48_16_t q48_add (q48_16_t a, q48_16_t b)`
Add two Q48.16 values.
- `static q48_16_t q48_sub (q48_16_t a, q48_16_t b)`
Subtract two Q48.16 values.
- `static q48_16_t q48_abs (q48_16_t a)`
Absolute value of Q48.16 (treats as unsigned, so just returns a)
- `static q48_16_t q48_from_u64 (uint64_t u)`
Convert unsigned 64-bit integer to Q48.16: u << 16.
- `static uint64_t q48_to_u64 (q48_16_t q)`
Convert Q48.16 to unsigned 64-bit integer (truncate fractional): q >> 16.
- `q48_16_t q48_log_approx (q48_16_t x)`
Approximate natural logarithm in Q48.16 (integer-only) Input: x in Q48.16 format (x > 0) Output: ln(x) in Q48.16 format.
- `q48_16_t q48_exp_approx (q48_16_t q)`
Approximate exponential e^x in Q48.16 (integer-only) Input: q in Q48.16 format Output: e^q in Q48.16 format.
- `q48_16_t q48_sqrt_approx (q48_16_t q)`
Approximate square root in Q48.16 (integer-only, Newton-Raphson) Input: q in Q48.16 format Output: sqrt(q) in Q48.16 format.

7.43.1 Macro Definition Documentation

7.43.1.1 Q48_ONE

```
#define Q48_ONE ((q48_16_t)0x10000ULL)
```

7.43.2 Typedef Documentation

7.43.2.1 q48_16_t

```
typedef uint64_t q48_16_t
```

q48_16.h - Q48.16 Fixed-Point Arithmetic for StarKernel

Format: uint64_t with fixed decimal point after bit 15

- Bits 0-15: Fractional part (1/65536 resolution)
- Bits 16-63: Integer part (up to $2^{48}-1$)

Example: 0x00010000 = 1.0, 0x00018000 = 1.5

All operations are integer-only. NO FLOATING-POINT.

7.43.3 Function Documentation

7.43.3.1 q48_abs()

```
static q48_16_t q48_abs (
    q48_16_t a ) [inline], [static]
```

Absolute value of Q48.16 (treats as unsigned, so just returns a)

7.43.3.2 q48_add()

```
static q48_16_t q48_add (
    q48_16_t a,
    q48_16_t b ) [inline], [static]
```

Add two Q48.16 values.

7.43.3.3 q48_div()

```
q48_16_t q48_div (
    q48_16_t a,
    q48_16_t b )
```

Divide two Q48.16 values: $(a \ll 16) / b$ Returns 0 if $b == 0$.

Divide two Q48.16 values: $(a \ll 16) / b$ Returns 0 if $b == 0$.

Math: $(a / 2^{16}) / (b / 2^{16}) * 2^{16} = (a \ll 16) / b$ Returns: $(a \ll 16) / b$

Parameters

<i>a</i>	Dividend in Q48.16
<i>b</i>	Divisor in Q48.16 (non-zero)

Returns

$(a \ll 16) / b$ in Q48.16 format

Referenced by [q48_exp_approx\(\)](#), [q48_log_approx\(\)](#), and [q48_sqrt_approx\(\)](#).

Here is the caller graph for this function:

7.43.3.4 q48_exp_approx()

```
q48_16_t q48_exp_approx (
    q48_16_t q )
```

Approximate exponential e^x in Q48.16 (integer-only) Input: q in Q48.16 format Output: e^q in Q48.16 format.

Approximate exponential e^x in Q48.16 (integer-only) Input: q in Q48.16 format Output: e^q in Q48.16 format.

Purpose: Might be needed for inverse operations or model validation Status: Defined but not required for Phase 1 inference engine Method: Similar to [q48_log_approx](#) (piecewise + Newton)

Parameters

q	Q48.16 exponent
-----	-----------------

Returns

e^q in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_from_u64\(\)](#), and [q48_mul\(\)](#).

Referenced by [q48_log_approx\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.43.3.5 q48_from_u64()

```
static q48_16_t q48_from_u64 (
    uint64_t u ) [inline], [static]
```

Convert unsigned 64-bit integer to Q48.16: $u \ll 16$.

7.43.3.6 q48_log_approx()

```
q48_16_t q48_log_approx (
    q48_16_t x )
```

Approximate natural logarithm in Q48.16 (integer-only) Input: x in Q48.16 format ($x > 0$) Output: $\ln(x)$ in Q48.16 format.

Approximate natural logarithm in Q48.16 (integer-only) Input: x in Q48.16 format ($x > 0$) Output: $\ln(x)$ in Q48.16 format.

Purpose: Required for decay slope inference via exponential fitting Model: $\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} * t$

Method: Piecewise linear approximation or Newton-Raphson iteration

- Input x must be > 0 (assertion check)
- Output is $\ln(x)$ in Q48.16 format
- Precision: $\sim 5\%$ error acceptable (tuning parameter)
- No floating-point operations

Implementation Strategy:

1. Use bit position ($\text{floor}(\log_2(x))$) as coarse estimate
2. Refine with Newton iteration (3-4 cycles for Q48.16 precision)
3. Keep all arithmetic in integer space

Example: $\text{q48_log_approx}(65536) = \log(1.0)$ in Q48.16 ≈ 0 $\text{q48_log_approx}(131072) = \log(2.0)$ in Q48.16 $\approx 0x0000B172$

Parameters

x	Value to take logarithm of ($x > 0$)
-----	--

Returns

$\ln(x)$ in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_exp_approx\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), and [q48_sub\(\)](#).

Here is the call graph for this function:

7.43.3.7 q48_mul()

```
q48_16_t q48_mul (
```



```

    q48_16_t a,
    q48_16_t b )

```

Multiply two Q48.16 values: $(a * b) \gg 16$.

Multiply two Q48.16 values: $(a * b) \gg 16$.

Multiply two Q48.16 values, keeping result in Q48.16.

Freestanding implementation: no libc, no floating-point.

Multiply two Q48.16 values: $(a * b) \gg 16$.

Math: $(a / 2^{16}) * (b / 2^{16}) * 2^{16} = (a * b) / 2^{16}$ Returns: $(a * b) \gg 16$

Parameters

<i>a</i>	First Q48.16 operand
<i>b</i>	Second Q48.16 operand

Returns

$(a * b) \gg 16$ in Q48.16 format

Multiply two Q48.16 values, keeping result in Q48.16.

Freestanding implementation: no libc, no floating-point.

Referenced by [q48_exp_approx\(\)](#), and [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.43.3.8 q48_sqrt_approx()

```

q48_16_t q48_sqrt_approx (
    q48_16_t q )

```

Approximate square root in Q48.16 (integer-only, Newton-Raphson) Input: q in Q48.16 format Output: sqrt(q) in Q48.16 format.

Approximate square root in Q48.16 (integer-only, Newton-Raphson) Input: q in Q48.16 format Output: sqrt(q) in Q48.16 format.

Purpose: Compute R^2 (fit quality) for regression diagnostics Method: Newton-Raphson for integer-only sqrt

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

sqrt(q) in Q48.16 format

References [q48_div\(\)](#).

Here is the call graph for this function:

7.43.3.9 q48_sub()

```

static q48_16_t q48_sub (
    q48_16_t a,
    q48_16_t b ) [inline], [static]

```

Subtract two Q48.16 values.

7.43.3.10 q48_to_u64()

```

static uint64_t q48_to_u64 (
    q48_16_t q ) [inline], [static]

```

Convert Q48.16 to unsigned 64-bit integer (truncate fractional): $q \gg 16$.

7.44 include/repl.h File Reference

```
#include "vm.h"
```

Include dependency graph for repl.h: This graph shows which files directly or indirectly include this file:

Functions

- void [vm_repl](#) ([VM](#) *vm, int script_mode)
Starts the REPL (Read-Eval-Print Loop) for the Forth virtual machine.

7.44.1 Function Documentation

7.44.1.1 vm_repl()

```
void vm_repl (
    VM * vm,
    int script_mode )
```

Starts the REPL (Read-Eval-Print Loop) for the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the VM instance to run the REPL on
<i>script_mode</i>	If 1, suppress prompts and "ok" output (for piped input)

Starts the REPL (Read-Eval-Print Loop) for the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>script_mode</i>	If 1, suppress prompts and "ok" output (for piped input)

This function implements an interactive REPL for the Forth interpreter. It continuously reads input from the user, interprets it, and prints the result until the [VM](#) is halted or an error occurs. In interactive mode (script_mode=0):

- Prints colored prompt "ok> "
- Acknowledges successful commands with " ok"
- Reports errors with " ERROR"

In script mode (script_mode=1):

- Suppresses all prompts and status messages
- Suitable for piped input (here doc, pipes, redirects)

References [VM::error](#), [VM::halted](#), [LOG_INFO](#), [log_message\(\)](#), [vm](#), [vm_interpret\(\)](#), and [VM::zuse_session](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.45 include/starkernel/repl.h File Reference

```
#include "vm.h"
```

Include dependency graph for repl.h: This graph shows which files directly or indirectly include this file:

Functions

- void [sk_repl](#) ([VM](#) *vm)
repl.h - Emergency FORTH REPL for LithosAnanke kernel

7.45.1 Function Documentation

7.45.1.1 sk_repl()

```
void sk_repl (
    VM * vm )
```

repl.h - Emergency FORTH REPL for LithosAnanke kernel

sk_repl - Run the emergency FORTH REPL on the serial console.

Blocks until vm->halted is set (BYE word) or the VM encounters a halt. Runs with interrupts enabled; the APIC heartbeat continues to fire.

Parameters

vm	Mama VM instance (must be fully initialised)
----	--

References [console_println\(\)](#), [console_puts\(\)](#), [VM::emergency_console](#), [VM::error](#), [VM::halted](#), [sk_fault_handler\(\)](#), [sk_readline\(\)](#), [vm](#), [vm_interpret\(\)](#), and [VM::zuse_session](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.46 include/starkernel/timer.h File Reference

```
#include <stdint.h>
#include "uefi.h"
#include "q48_16.h"
```

Include dependency graph for timer.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [time_window](#)
- struct [time_trust_state](#)
- struct [timer_calibration_record](#)

Macros

- #define [TIME_WINDOW_SIZE](#) 64
- #define [TIME_TRUST_HIGH](#) [Q48_ONE](#) /* 1.0 = full confidence */
- #define [TIME_TRUST_LOW](#) ([Q48_ONE](#) >> 2) /* 0.25 = low confidence */

Typedefs

- typedef [q48_16_t](#) [time_trust_t](#)
timer.h - Timer and Heartbeat Interface
- typedef struct [time_window](#) [TimeWindow](#)
- typedef struct [time_trust_state](#) [TimeTrustState](#)
- typedef enum [timer_trust_level](#) [timer_trust_level_t](#)
- typedef struct [timer_calibration_record](#) [timer_calibration_record_t](#)

Enumerations

- enum [timer_trust_level](#) { [TIMER_TRUST_NONE](#) = 0 , [TIMER_TRUST_RELATIVE](#) = 1 , [TIMER_TRUST_ABSOLUTE](#) = 2 }

Functions

- int [timer_init](#) ([BootInfo](#) *boot_info)
- uint64_t [timer_tsc_hz](#) (void)
- uint64_t [timer_now_ns](#) (void)

- int [timer_check_drift_now](#) (void)
- const [timer_calibration_record_t](#) * [timer_calibration_record](#) (void)
- void [heartbeat_init](#) (uint64_t tsc_hz, uint64_t tick_hz)
Initialize the heartbeat subsystem.
- void [heartbeat_tick](#) (void)
Called by heartbeat ISR on each tick.
- uint64_t [heartbeat_ticks](#) (void)
Get current TIME-TICKS (monotonic heartbeat count).
- [time_trust_t](#) [heartbeat_trust](#) (void)
Get current TIME-TRUST (Q48.16 confidence metric).
- const [TimeTrustState](#) * [heartbeat_state](#) (void)
Get pointer to full heartbeat state (for diagnostics).

7.46.1 Macro Definition Documentation

7.46.1.1 TIME_TRUST_HIGH

```
#define TIME_TRUST_HIGH Q48_ONE /* 1.0 = full confidence */
```

7.46.1.2 TIME_TRUST_LOW

```
#define TIME_TRUST_LOW (Q48_ONE >> 2) /* 0.25 = low confidence */
```

7.46.1.3 TIME_WINDOW_SIZE

```
#define TIME_WINDOW_SIZE 64
```

7.46.2 Typedef Documentation

7.46.2.1 time_trust_t

```
typedef q48_16_t time_trust_t
```

[timer.h](#) - Timer and Heartbeat Interface
M5 Time Model:

- TIME-TICKS (Q64.0): Monotonic heartbeat counter, never decreases
- TIME-TRUST (Q48.16): Continuous confidence metric [0.0, 1.0]
- No discrete modes (NONE/REL/ABS are legacy, being phased out)
- Trust is a measurement, never gates execution

7.46.2.2 timer_calibration_record_t

```
typedef struct timer_calibration_record timer_calibration_record_t
```

7.46.2.3 timer_trust_level_t

```
typedef enum timer_trust_level timer_trust_level_t
```

7.46.2.4 TimeTrustState

```
typedef struct time_trust_state TimeTrustState
```

7.46.2.5 TimeWindow

```
typedef struct time_window TimeWindow
```

7.46.3 Enumeration Type Documentation

7.46.3.1 timer_trust_level

```
enum timer_trust_level
```

Enumerator

TIMER_TRUST_NONE	
TIMER_TRUST_RELATIVE	
TIMER_TRUST_ABSOLUTE	

7.46.4 Function Documentation

7.46.4.1 heartbeat_init()

```
void heartbeat_init (
    uint64_t tsc_hz,
    uint64_t tick_hz )
```

Initialize the heartbeat subsystem.

Called after [timer_init\(\)](#), before enabling APIC timer.

References [time_window::count](#), [time_window::deltas](#), [time_trust_state::expected_delta](#), [g_heartbeat](#), [time_trust_state::last_tsc](#), [time_window::pos](#), [Q48_ONE](#), [time_trust_state::ticks](#), [TIME_WINDOW_SIZE](#), [time_trust_state::total_samples](#), [time_trust_state::trust](#), [time_trust_state::variance](#), and [time_trust_state::window](#).

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.46.4.2 heartbeat_state()

```
const TimeTrustState * heartbeat_state (
    void )
```

Get pointer to full heartbeat state (for diagnostics).

References [g_heartbeat](#).

7.46.4.3 heartbeat_tick()

```
void heartbeat_tick (
    void )
```

Called by heartbeat ISR on each tick.

Updates TIME-TICKS, samples TSC, updates rolling window and TIME-TRUST.

References [time_trust_state::expected_delta](#), [g_heartbeat](#), [time_trust_state::last_tsc](#), [rdtsc\(\)](#), [time_trust_state::ticks](#), [time_trust_state::total_samples](#), [time_trust_state::trust](#), [time_trust_state::variance](#), [variance_to_trust\(\)](#), [time_trust_state::window](#), [window_push\(\)](#), and [window_variance_q48\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.46.4.4 heartbeat_ticks()

```
uint64_t heartbeat_ticks (
    void )
```

Get current TIME-TICKS (monotonic heartbeat count).

References [g_heartbeat](#), and [time_trust_state::ticks](#).

Referenced by [isr_common_handler\(\)](#).

Here is the caller graph for this function:

7.46.4.5 heartbeat_trust()

```
time_trust_t heartbeat_trust (
    void )
```

Get current TIME-TRUST (Q48.16 confidence metric).
 References [g_heartbeat](#), and [time_trust_state::trust](#).
 Referenced by [isr_common_handler\(\)](#).
 Here is the caller graph for this function:

7.46.4.6 timer_calibration_record()

```
const timer_calibration_record_t * timer_calibration_record (
    void )
```

References [calib_record](#).

7.46.4.7 timer_check_drift_now()

```
int timer_check_drift_now (
    void )
```

References [calib_record](#), [timer_drift_check_bare_metal\(\)](#), and [timer_calibration_record::vm_mode](#).

Here is the call graph for this function:

7.46.4.8 timer_init()

```
int timer_init (
    BootInfo * boot_info )
```

References [calib_record](#), [console_println\(\)](#), [console_puts\(\)](#), [converge_tsc_bare_metal\(\)](#), [timer_calibration_record::converged](#), [enable_hpet\(\)](#), [hpet_freq_hz](#), [HPET_GEN_CAP_ID](#), [HPET_PHYS_BASE](#), [hpet_read\(\)](#), [hpet_regs](#), [HPET_VIRT_BASE](#), [init_timer_vm_mode\(\)](#), [pit_available](#), [pit_probe_running\(\)](#), [pit_tsc_hz_mean](#), [print_dec\(\)](#), [running_under_hypervisor\(\)](#), [sanity_check_hpet_running_quick\(\)](#), [timer_fatal\(\)](#), [TIMER_REQUIRE_PIT](#), [TIMER_TRUST_NONE](#), [timer_calibration_record::trust](#), [tsc_hz_locked](#), [timer_calibration_record::vm_mode](#), [VMM_FLAG_WRITABLE](#), and [vmm_map_range\(\)](#).
 Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.46.4.9 timer_now_ns()

```
uint64_t timer_now_ns (
    void )
```

References [calib_record](#), [cpu_has_rdtscp\(\)](#), [muldiv64\(\)](#), [rdtsc\(\)](#), [rdtscp_aux\(\)](#), [timer_now_ns_vm_relative\(\)](#), [TIMER_TRUST_ABSOLUTE](#), [TIMER_TRUST_NONE](#), [timer_calibration_record::trust](#), [tsc_hz_locked](#), and [timer_calibration_record::vm_mode](#).

Here is the call graph for this function:

7.46.4.10 timer_tsc_hz()

```
uint64_t timer_tsc_hz (
    void )
```

References [tsc_hz_locked](#).

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.47 include/starkernel/uefi.h File Reference

```
#include <stdint.h>
#include <stddef.h>
```

Include dependency graph for uefi.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [EFI_GUID](#)
- struct [EFI_MEMORY_DESCRIPTOR](#)
- struct [EFI_TABLE_HEADER](#)

- struct [_EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL](#)
- struct [_EFI_BOOT_SERVICES](#)
- struct [_EFI_RUNTIME_SERVICES](#)
- struct [_EFI_SYSTEM_TABLE](#)
- struct [EFI_CONFIGURATION_TABLE](#)
- struct [FramebufferInfo](#)
- struct [BootInfo](#)
- struct [_EFI_FILE_PROTOCOL](#)
- struct [_EFI_SIMPLE_FILE_SYSTEM_PROTOCOL](#)
- struct [EFI_TIME](#)
- struct [EFI_FILE_INFO](#)
- struct [EFI_LOADED_IMAGE_PROTOCOL](#)

Macros

- #define [TRUE](#) ((BOOLEAN)1)
- #define [FALSE](#) ((BOOLEAN)0)
- #define [TPL_APPLICATION](#) 4
- #define [TPL_CALLBACK](#) 8
- #define [TPL_NOTIFY](#) 16
- #define [TPL_HIGH_LEVEL](#) 31
- #define [EFIAPI](#)
- #define [EFI_SUCCESS](#) 0
- #define [EFI_LOAD_ERROR](#) (1 | (1UL << 63))
- #define [EFI_INVALID_PARAMETER](#) (2 | (1UL << 63))
- #define [EFI_UNSUPPORTED](#) (3 | (1UL << 63))
- #define [EFI_BAD_BUFFER_SIZE](#) (4 | (1UL << 63))
- #define [EFI_BUFFER_TOO_SMALL](#) (5 | (1UL << 63))
- #define [EFI_NOT_READY](#) (6 | (1UL << 63))
- #define [EFI_DEVICE_ERROR](#) (7 | (1UL << 63))
- #define [EFI_WRITE_PROTECTED](#) (8 | (1UL << 63))
- #define [EFI_OUT_OF_RESOURCES](#) (9 | (1UL << 63))
- #define [EFI_NOT_FOUND](#) (14 | (1UL << 63))
- #define [EFI_ABORTED](#) (21 | (1UL << 63))
- #define [EFI_MEMORY_UC](#) 0x0000000000000001ULL
- #define [EFI_MEMORY_WC](#) 0x0000000000000002ULL
- #define [EFI_MEMORY_WT](#) 0x0000000000000004ULL
- #define [EFI_MEMORY_WB](#) 0x0000000000000008ULL
- #define [EFI_MEMORY_UCE](#) 0x0000000000000010ULL
- #define [EFI_MEMORY_WP](#) 0x0000000000001000ULL
- #define [EFI_MEMORY_RP](#) 0x0000000000002000ULL
- #define [EFI_MEMORY_XP](#) 0x0000000000004000ULL
- #define [EFI_MEMORY_RUNTIME](#) 0x8000000000000000ULL
- #define [EFI_FILE_MODE_READ](#) 0x0000000000000001ULL

Typedefs

- typedef uint64_t [UINTN](#)
[uefi.h](#) - UEFI definitions for StarKernel Minimal UEFI interface for bare-metal boot
- typedef int64_t [INTN](#)
- typedef uint8_t [UINT8](#)
- typedef uint16_t [UINT16](#)
- typedef uint32_t [UINT32](#)
- typedef uint64_t [UINT64](#)
- typedef int8_t [INT8](#)

- typedef int16_t INT16
- typedef int32_t INT32
- typedef int64_t INT64
- typedef uint8_t BOOLEAN
- typedef void VOID
- typedef uint16_t CHAR16
- typedef UINTN EFI_TPL
- typedef UINTN EFI_STATUS
- typedef void * EFI_HANDLE
- typedef UINT64 EFI_PHYSICAL_ADDRESS
- typedef EFI_STATUS(* EFI_TEXT_STRING) (struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *This, CHAR16 *String)
- typedef EFI_STATUS(* EFI_TEXT_RESET) (struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *This, BOOLEAN ExtendedVerification)
- typedef struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
- typedef EFI_STATUS(* EFI_GET_MEMORY_MAP) (UINTN *MemoryMapSize, EFI_MEMORY_DESCRIPTOR *MemoryMap, UINTN *MapKey, UINTN *DescriptorSize, UINT32 *DescriptorVersion)
- typedef EFI_STATUS(* EFI_EXIT_BOOT_SERVICES) (EFI_HANDLE ImageHandle, UINTN MapKey)
- typedef EFI_STATUS(* EFI_ALLOCATE_POOL) (EFI_MEMORY_TYPE PoolType, UINTN Size, void **Buffer)
- typedef EFI_STATUS(* EFI_FREE_POOL) (void *Buffer)
- typedef EFI_STATUS(* EFI_ALLOCATE_PAGES) (EFI_ALLOCATE_TYPE Type, EFI_MEMORY_TYPE MemoryType, UINTN Pages, EFI_PHYSICAL_ADDRESS *Memory)
- typedef EFI_STATUS(* EFI_FREE_PAGES) (EFI_PHYSICAL_ADDRESS Memory, UINTN Pages)
- typedef EFI_STATUS(* EFI_HANDLE_PROTOCOL) (EFI_HANDLE Handle, EFI_GUID *Protocol, void **Interface)
- typedef EFI_STATUS(* EFI_LOCATE_HANDLE) (EFI_LOCATE_SEARCH_TYPE SearchType, EFI_GUID *Protocol, void *SearchKey, UINTN *BufferSize, EFI_HANDLE *Buffer)
- typedef EFI_TPL(* EFI_RAISE_TPL) (EFI_TPL NewTpl)
- typedef void(* EFI_RESTORE_TPL) (EFI_TPL OldTpl)
- typedef struct _EFI_BOOT_SERVICES EFI_BOOT_SERVICES
- typedef struct _EFI_RUNTIME_SERVICES EFI_RUNTIME_SERVICES
- typedef struct _EFI_SYSTEM_TABLE EFI_SYSTEM_TABLE
- typedef EFI_STATUS(* EFI_FILE_OPEN) (struct _EFI_FILE_PROTOCOL *This, struct _EFI_FILE_PROTOCOL **NewHandle, CHAR16 *FileName, UINT64 OpenMode, UINT64 Attributes)
- typedef EFI_STATUS(* EFI_FILE_CLOSE) (struct _EFI_FILE_PROTOCOL *This)
- typedef EFI_STATUS(* EFI_FILE_READ) (struct _EFI_FILE_PROTOCOL *This, UINTN *BufferSize, VOID *Buffer)
- typedef EFI_STATUS(* EFI_FILE_GET_INFO) (struct _EFI_FILE_PROTOCOL *This, EFI_GUID *InformationType, UINTN *BufferSize, VOID *Buffer)
- typedef struct _EFI_FILE_PROTOCOL EFI_FILE_PROTOCOL
- typedef EFI_STATUS(* EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME) (struct _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL *This, EFI_FILE_PROTOCOL **Root)
- typedef struct _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL EFI_SIMPLE_FILE_SYSTEM_PROTOCOL

Enumerations

- enum EFI_ALLOCATE_TYPE { AllocateAnyPages , AllocateMaxAddress , AllocateAddress , MaxAllocateType }
- enum EFI_MEMORY_TYPE { EfiReservedMemoryType , EfiLoaderCode , EfiLoaderData , EfiBootServicesCode , EfiBootServicesData , EfiRuntimeServicesCode , EfiRuntimeServicesData , EfiConventionalMemory , EfiUnusableMemory , EfiACPIReclaimMemory , EfiACPIMemoryNVS , EfiMemoryMappedIO , EfiMemoryMappedIOPortSpace , EfiPalCode , EfiPersistentMemory , EfiMaxMemoryType }
- enum EFI_LOCATE_SEARCH_TYPE { AllHandles , ByRegisterNotify , ByProtocol }

Variables

- static const [EFI_GUID EFI_ACPI_20_TABLE_GUID](#) = {0x8868e871,0xe4f1,0x11d3,{0xbc,0x22,0x00,0x80,0xc7,0x3c,0x88,0x8}}
- static const [EFI_GUID EFI_ACPI_TABLE_GUID](#) = {0xeb9d2d30,0x2d88,0x11d3,{0x9a,0x16,0x00,0x90,0x27,0x3f,0xc1,0x4d}}
- static const [EFI_GUID EFI_LOADED_IMAGE_PROTOCOL_GUID](#)
- static const [EFI_GUID EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID](#)
- static const [EFI_GUID EFI_FILE_INFO_GUID](#)

7.47.1 Macro Definition Documentation

7.47.1.1 EFI_ABORTED

```
#define EFI_ABORTED (21 | (1UL << 63))
```

7.47.1.2 EFI_BAD_BUFFER_SIZE

```
#define EFI_BAD_BUFFER_SIZE (4 | (1UL << 63))
```

7.47.1.3 EFI_BUFFER_TOO_SMALL

```
#define EFI_BUFFER_TOO_SMALL (5 | (1UL << 63))
```

7.47.1.4 EFI_DEVICE_ERROR

```
#define EFI_DEVICE_ERROR (7 | (1UL << 63))
```

7.47.1.5 EFI_FILE_MODE_READ

```
#define EFI_FILE_MODE_READ 0x0000000000000001ULL
```

7.47.1.6 EFI_INVALID_PARAMETER

```
#define EFI_INVALID_PARAMETER (2 | (1UL << 63))
```

7.47.1.7 EFI_LOAD_ERROR

```
#define EFI_LOAD_ERROR (1 | (1UL << 63))
```

7.47.1.8 EFI_MEMORY_MP

```
#define EFI_MEMORY_MP 0x0000000000000200ULL
```

7.47.1.9 EFI_MEMORY_RUNTIME

```
#define EFI_MEMORY_RUNTIME 0x8000000000000000ULL
```

7.47.1.10 EFI_MEMORY_UC

```
#define EFI_MEMORY_UC 0x0000000000000001ULL
```

7.47.1.11 EFI_MEMORY_UCE

```
#define EFI_MEMORY_UCE 0x000000000000010ULL
```

7.47.1.12 EFI_MEMORY_WB

```
#define EFI_MEMORY_WB 0x0000000000000008ULL
```

7.47.1.13 EFI_MEMORY_WC

```
#define EFI_MEMORY_WC 0x0000000000000002ULL
```

7.47.1.14 EFI_MEMORY_WP

```
#define EFI_MEMORY_WP 0x0000000000000100UULL
```

7.47.1.15 EFI_MEMORY_WT

```
#define EFI_MEMORY_WT 0x0000000000000004UULL
```

7.47.1.16 EFI_MEMORY_XP

```
#define EFI_MEMORY_XP 0x0000000000000400UULL
```

7.47.1.17 EFI_NOT_FOUND

```
#define EFI_NOT_FOUND (14 | (1UL << 63))
```

7.47.1.18 EFI_NOT_READY

```
#define EFI_NOT_READY (6 | (1UL << 63))
```

7.47.1.19 EFI_OUT_OF_RESOURCES

```
#define EFI_OUT_OF_RESOURCES (9 | (1UL << 63))
```

7.47.1.20 EFI_SUCCESS

```
#define EFI_SUCCESS 0
```

7.47.1.21 EFI_UNSUPPORTED

```
#define EFI_UNSUPPORTED (3 | (1UL << 63))
```

7.47.1.22 EFI_WRITE_PROTECTED

```
#define EFI_WRITE_PROTECTED (8 | (1UL << 63))
```

7.47.1.23 EFIAPI

```
#define EFIAPI
```

7.47.1.24 FALSE

```
#define FALSE ((BOOLEAN)0)
```

7.47.1.25 TPL_APPLICATION

```
#define TPL_APPLICATION 4
```

7.47.1.26 TPL_CALLBACK

```
#define TPL_CALLBACK 8
```

7.47.1.27 TPL_HIGH_LEVEL

```
#define TPL_HIGH_LEVEL 31
```

7.47.1.28 TPL_NOTIFY

```
#define TPL_NOTIFY 16
```

7.47.1.29 TRUE

```
#define TRUE ((BOOLEAN)1)
```

7.47.2 Typedef Documentation

7.47.2.1 BOOLEAN

```
typedef uint8_t BOOLEAN
```

7.47.2.2 CHAR16

```
typedef uint16_t CHAR16
```

7.47.2.3 EFI_ALLOCATE_PAGES

```
typedef EFI_STATUS( * EFI_ALLOCATE_PAGES) (EFI_ALLOCATE_TYPE Type, EFI_MEMORY_TYPE MemoryType,
UINTN Pages, EFI_PHYSICAL_ADDRESS *Memory)
```

7.47.2.4 EFI_ALLOCATE_POOL

```
typedef EFI_STATUS( * EFI_ALLOCATE_POOL) (EFI_MEMORY_TYPE PoolType, UINTN Size, void **Buffer)
```

7.47.2.5 EFI_BOOT_SERVICES

```
typedef struct _EFI_BOOT_SERVICES EFI_BOOT_SERVICES
```

7.47.2.6 EFI_EXIT_BOOT_SERVICES

```
typedef EFI_STATUS( * EFI_EXIT_BOOT_SERVICES) (EFI_HANDLE ImageHandle, UINTN MapKey)
```

7.47.2.7 EFI_FILE_CLOSE

```
typedef EFI_STATUS( * EFI_FILE_CLOSE) (struct _EFI_FILE_PROTOCOL *This)
```

7.47.2.8 EFI_FILE_GET_INFO

```
typedef EFI_STATUS( * EFI_FILE_GET_INFO) (struct _EFI_FILE_PROTOCOL *This, EFI_GUID *Information←
Type, UINTN *BufferSize, VOID *Buffer)
```

7.47.2.9 EFI_FILE_OPEN

```
typedef EFI_STATUS( * EFI_FILE_OPEN) (struct _EFI_FILE_PROTOCOL *This, struct _EFI_FILE_PROTOCOL
**NewHandle, CHAR16 *FileName, UINT64 OpenMode, UINT64 Attributes)
```

7.47.2.10 EFI_FILE_PROTOCOL

```
typedef struct _EFI_FILE_PROTOCOL EFI_FILE_PROTOCOL
```

7.47.2.11 EFI_FILE_READ

```
typedef EFI_STATUS( * EFI_FILE_READ) (struct _EFI_FILE_PROTOCOL *This, UINTN *BufferSize, VOID
*Buffer)
```

7.47.2.12 EFI_FREE_PAGES

```
typedef EFI_STATUS( * EFI_FREE_PAGES) (EFI_PHYSICAL_ADDRESS Memory, UINTN Pages)
```

7.47.2.13 EFI_FREE_POOL

```
typedef EFI_STATUS( * EFI_FREE_POOL) (void *Buffer)
```

7.47.2.14 EFI_GET_MEMORY_MAP

```
typedef EFI_STATUS( * EFI_GET_MEMORY_MAP) (UINTN *MemoryMapSize, EFI_MEMORY_DESCRIPTOR *MemoryMap,
UINTN *MapKey, UINTN *DescriptorSize, UINT32 *DescriptorVersion)
```

7.47.2.15 EFI_HANDLE

```
typedef void* EFI_HANDLE
```

7.47.2.16 EFI_HANDLE_PROTOCOL

```
typedef EFI_STATUS( * EFI_HANDLE_PROTOCOL) (EFI_HANDLE Handle, EFI_GUID *Protocol, void **Interface)
```

7.47.2.17 EFI_LOCATE_HANDLE

```
typedef EFI_STATUS( * EFI_LOCATE_HANDLE) (EFI_LOCATE_SEARCH_TYPE SearchType, EFI_GUID *Protocol,
void *SearchKey, UINTN *BufferSize, EFI_HANDLE *Buffer)
```

7.47.2.18 EFI_PHYSICAL_ADDRESS

```
typedef UINT64 EFI_PHYSICAL_ADDRESS
```

7.47.2.19 EFI_RAISE_TPL

```
typedef EFI_TPL( * EFI_RAISE_TPL) (EFI_TPL NewTpl)
```

7.47.2.20 EFI_RESTORE_TPL

```
typedef void( * EFI_RESTORE_TPL) (EFI_TPL OldTpl)
```

7.47.2.21 EFI_RUNTIME_SERVICES

```
typedef struct _EFI_RUNTIME_SERVICES EFI_RUNTIME_SERVICES
```

7.47.2.22 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL

```
typedef struct _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL EFI_SIMPLE_FILE_SYSTEM_PROTOCOL
```

7.47.2.23 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME

```
typedef EFI_STATUS( * EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME) (struct _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL
*This, EFI_FILE_PROTOCOL **Root)
```

7.47.2.24 EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL

```
typedef struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
```

7.47.2.25 EFI_STATUS

```
typedef UINTN EFI_STATUS
```

7.47.2.26 EFI_SYSTEM_TABLE

```
typedef struct _EFI_SYSTEM_TABLE EFI_SYSTEM_TABLE
```

7.47.2.27 EFI_TEXT_RESET

```
typedef EFI_STATUS( * EFI_TEXT_RESET) (struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *This, BOOLEAN
ExtendedVerification)
```

7.47.2.28 EFI_TEXT_STRING

```
typedef EFI_STATUS ( * EFI_TEXT_STRING) (struct _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *This, CHAR16 *String)
```

7.47.2.29 EFI_TPL

```
typedef UINTN EFI_TPL
```

7.47.2.30 INT16

```
typedef int16_t INT16
```

7.47.2.31 INT32

```
typedef int32_t INT32
```

7.47.2.32 INT64

```
typedef int64_t INT64
```

7.47.2.33 INT8

```
typedef int8_t INT8
```

7.47.2.34 INTN

```
typedef int64_t INTN
```

7.47.2.35 UINT16

```
typedef uint16_t UINT16
```

7.47.2.36 UINT32

```
typedef uint32_t UINT32
```

7.47.2.37 UINT64

```
typedef uint64_t UINT64
```

7.47.2.38 UINT8

```
typedef uint8_t UINT8
```

7.47.2.39 UINTN

```
typedef uint64_t UINTN
```

[uefi.h](#) - UEFI definitions for StarKernel Minimal UEFI interface for bare-metal boot

7.47.2.40 VOID

```
typedef void VOID
```

7.47.3 Enumeration Type Documentation

7.47.3.1 EFI_ALLOCATE_TYPE

```
enum EFI_ALLOCATE_TYPE
```

Enumerator

AllocateAnyPages	
AllocateMaxAddress	
AllocateAddress	
MaxAllocateType	

7.47.3.2 EFI_LOCATE_SEARCH_TYPE

enum [EFI_LOCATE_SEARCH_TYPE](#)

Enumerator

AllHandles	
ByRegisterNotify	
ByProtocol	

7.47.3.3 EFI_MEMORY_TYPE

enum [EFI_MEMORY_TYPE](#)

Enumerator

EfiReservedMemoryType	
EfiLoaderCode	
EfiLoaderData	
EfiBootServicesCode	
EfiBootServicesData	
EfiRuntimeServicesCode	
EfiRuntimeServicesData	
EfiConventionalMemory	
EfiUnusableMemory	
EfiACPIReclaimMemory	
EfiACPIMemoryNVS	
EfiMemoryMappedIO	
EfiMemoryMappedIOPortSpace	
EfiPalCode	
EfiPersistentMemory	
EfiMaxMemoryType	

7.47.4 Variable Documentation**7.47.4.1 EFI_ACPI_20_TABLE_GUID**

```
const EFI\_GUID EFI_ACPI_20_TABLE_GUID = {0x8868e871, 0xe4f1, 0x11d3, {0xbc, 0x22, 0x00, 0x80, 0xc7, 0x3c, 0x88, 0x81}}
```

[static]

Referenced by [efi_main\(\)](#).

7.47.4.2 EFI_ACPI_TABLE_GUID

```
const EFI\_GUID EFI_ACPI_TABLE_GUID = {0xeb9d2d30, 0x2d88, 0x11d3, {0x9a, 0x16, 0x00, 0x90, 0x27, 0x3f, 0xc1, 0x4d}}
```

[static]

Referenced by [efi_main\(\)](#).

7.47.4.3 EFI_FILE_INFO_GUID

```
const EFI_GUID EFI_FILE_INFO_GUID [static]
```

Initial value:

```
=
{0x09576e92,0x6d3f,0x11d2, {0x8e,0x39,0x00,0xa0,0xc9,0x69,0x72,0x3b}}
```

Referenced by [load_kernel_from_esp\(\)](#).

7.47.4.4 EFI_LOADED_IMAGE_PROTOCOL_GUID

```
const EFI_GUID EFI_LOADED_IMAGE_PROTOCOL_GUID [static]
```

Initial value:

```
=
{0x5B1B31A1,0x9562,0x11d2, {0x8E,0x3F,0x00,0xA0,0xC9,0x69,0x72,0x3B}}
```

Referenced by [load_kernel_from_esp\(\)](#).

7.47.4.5 EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID

```
const EFI_GUID EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID [static]
```

Initial value:

```
=
{0x0964e5b22,0x6459,0x11d2, {0x8e,0x39,0x00,0xa0,0xc9,0x69,0x72,0x3b}}
```

Referenced by [load_kernel_from_esp\(\)](#).

7.48 include/starkernel/vm/arena.h File Reference

7.49 include/starkernel/vm/bootstrap/sk_vm_bootstrap.h File Reference

7.50 include/starkernel/vm/parity.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for parity.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [ParityPacket](#)
ParityPacket - Summary of [VM](#) state for comparison.

Macros

- #define [SK_BOOTSTRAP_OK](#) 0
parity.h - Parity packet for [VM](#) validation
- #define [SK_BOOTSTRAP_ARENA_FAIL](#) 1
- #define [SK_BOOTSTRAP_INIT_FAIL](#) 2
- #define [SK_BOOTSTRAP_DICT_FAIL](#) 3
- #define [FNV1A_64_OFFSET_BASIS](#) 0xCBF29CE484222325ULL
FNV-1a constants.
- #define [FNV1A_64_PRIME](#) 0x100000001B3ULL

Functions

- void [sk_parity_collect](#) (struct [VM](#) *vm, [ParityPacket](#) *out)
sk_parity_collect - Collect parity data from [VM](#)
- void [sk_parity_print](#) (const [ParityPacket](#) *pkt)
sk_parity_print - Print parity packet to console

- uint64_t [sk_dict_canonical_hash](#) (struct [VM](#) *vm)
sk_dict_canonical_hash - Compute canonical dictionary hash
- uint32_t [sk_dict_word_count](#) (struct [VM](#) *vm)
sk_dict_word_count - Count dictionary entries
- uint64_t [fnv1a_64](#) (const uint8_t *data, size_t len, uint64_t hash)
fnv1a_64 - FNV-1a 64-bit hash function

7.50.1 Macro Definition Documentation

7.50.1.1 FNV1A_64_OFFSET_BASIS

```
#define FNV1A_64_OFFSET_BASIS 0xCBF29CE484222325ULL
```

FNV-1a constants.

7.50.1.2 FNV1A_64_PRIME

```
#define FNV1A_64_PRIME 0x100000001B3ULL
```

7.50.1.3 SK_BOOTSTRAP_ARENA_FAIL

```
#define SK_BOOTSTRAP_ARENA_FAIL 1
```

7.50.1.4 SK_BOOTSTRAP_DICT_FAIL

```
#define SK_BOOTSTRAP_DICT_FAIL 3
```

7.50.1.5 SK_BOOTSTRAP_INIT_FAIL

```
#define SK_BOOTSTRAP_INIT_FAIL 2
```

7.50.1.6 SK_BOOTSTRAP_OK

```
#define SK_BOOTSTRAP_OK 0
```

[parity.h](#) - Parity packet for [VM](#) validation

Defines structures and functions for comparing hosted vs kernel [VM](#) state. Parity is verified via canonical dictionary hash, not raw memory comparison.

M7 Normative Rules:

- word_id is monotonic creation index starting at 0
- Dictionary traversal is creation order (oldest to newest)
- Hash excludes pointers, padding, runtime fields
- Colon bodies are hashed as word_id sequences, not addresses Bootstrap result codes

7.50.2 Function Documentation

7.50.2.1 fnv1a_64()

```
uint64_t fnv1a_64 (
    const uint8_t * data,
    size_t len,
    uint64_t hash )
```

fnv1a_64 - FNV-1a 64-bit hash function

Parameters

<i>data</i>	Pointer to data to hash
<i>len</i>	Length of data
<i>hash</i>	Current hash value (use FNV1A_64_OFFSET_BASIS for initial)

Returns

Updated hash value

References [FNV1A_64_PRIME](#).

Referenced by [sk_dict_canonical_hash\(\)](#).

Here is the caller graph for this function:

7.50.2.2 sk_dict_canonical_hash()

```
uint64_t sk_dict_canonical_hash (
    struct VM * vm )
```

sk_dict_canonical_hash - Compute canonical dictionary hash

Hashes structural fields only:

- flags, name_len, name[], acl_default, word_id
- For colon words: body as word_id sequence

Excludes:

- link (pointer)
- func (function pointer)
- execution_heat (runtime)
- physics (runtime)
- transition_metrics (pointer)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

64-bit FNV-1a hash

Traverses dictionary in creation order, hashes structural fields only.

References [DictEntry::acl_default](#), [DictEntry::flags](#), [fnv1a_64\(\)](#), [FNV1A_64_OFFSET_BASIS](#), [fnv1a_64_u32\(\)](#), [fnv1a_64_u8\(\)](#), [DictEntry::func](#), [hash_colon_body\(\)](#), [VM::latest](#), [MAX_DICT_ENTRIES](#), [DictEntry::name](#), [DictEntry::name_len](#), [sk_parity_debug_check_ptr](#), [sk_parity_debug_log_msg](#), [vm](#), [WORD_COMPILED](#), and [DictEntry::word_id](#).

Referenced by [sk_parity_collect\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.50.2.3 sk_dict_word_count()

```
uint32_t sk_dict_word_count (
    struct VM * vm )
```

sk_dict_word_count - Count dictionary entries

Traverses from vm->latest to NULL.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Number of dictionary entries

References [VM::latest](#), [MAX_DICT_ENTRIES](#), and [vm](#).

Referenced by [sk_parity_collect\(\)](#).

Here is the caller graph for this function:

7.50.2.4 sk_parity_collect()

```
void sk_parity_collect (
    struct VM * vm,
    ParityPacket * out )
```

sk_parity_collect - Collect parity data from [VM](#)

Traverses dictionary in creation order, computes canonical hash. Does NOT include runtime fields (execution_heat, physics, etc.).

Parameters

<i>vm</i>	Pointer to VM instance
<i>out</i>	Pointer to ParityPacket to fill

References [ParityPacket::bootstrap_result](#), [global_test_stats](#), [ParityPacket::header_hash64](#), [VM::here](#), [ParityPacket::here_offset](#), [VM::latest](#), [ParityPacket::latest_word_id](#), [SK_BOOTSTRAP_INIT_FAIL](#), [SK_BOOTSTRAP_OK](#), [sk_dict_canonical_hash\(\)](#), [sk_dict_word_count\(\)](#), [sk_parity_debug_log_msg](#), [ParityPacket::tests_errors](#), [ParityPacket::tests_failed](#), [ParityPacket::tests_passed](#), [ParityPacket::tests_skipped](#), [ParityPacket::tests_total](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), [vm](#), [ParityPacket::window_hash64](#), [ParityPacket::word_count](#), and [DictEntry::word_id](#).

Here is the call graph for this function:

7.50.2.5 sk_parity_print()

```
void sk_parity_print (
    const ParityPacket * pkt )
```

sk_parity_print - Print parity packet to console

Output format: PARITY:M7.1a word_count=N here=0xHHHH latest_id=N hash=0xHHHHHHHHHHHHHHHHHH

PARITY:M7.1b tests=N pass=N fail=N skip=N err=N

Parameters

<i>pkt</i>	Pointer to ParityPacket to print
------------	--

References [ParityPacket::bootstrap_result](#), [ParityPacket::header_hash64](#), [ParityPacket::here_offset](#), [ParityPacket::latest_word_id](#), [print_hex64\(\)](#), [print_nl\(\)](#), [print_str\(\)](#), [print_u32\(\)](#), [SK_BOOTSTRAP_OK](#), [ParityPacket::tests_errors](#), [ParityPacket::tests_failed](#), [ParityPacket::tests_passed](#), [ParityPacket::tests_skipped](#), [ParityPacket::tests_total](#), and [ParityPacket::word_count](#).

Here is the call graph for this function:

7.51 include/starkernel/vmm.h File Reference

```
#include <stdint.h>
#include "uefi.h"
```

Include dependency graph for vmm.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [vmm_page_info_t](#)

Macros

- `#define VMM_PAGE_SIZE 4096ull`
vmm.h - Virtual Memory Manager interface (x86_64 4-level paging)
- `#define VMM_FLAG_PRESENT (1ull << 0)`
- `#define VMM_FLAG_WRITABLE (1ull << 1)`
- `#define VMM_FLAG_USER (1ull << 2)`
- `#define VMM_FLAG_NX (1ull << 63)`

Functions

- `int vmm_init (BootInfo *boot_info)`
- `int vmm_map_page (uint64_t vaddr, uint64_t paddr, uint64_t flags)`
- `int vmm_unmap_page (uint64_t vaddr)`
- `uint64_t vmm_get_paddr (uint64_t vaddr)`
- `int vmm_map_range (uint64_t vaddr, uint64_t paddr, uint64_t size, uint64_t flags)`
- `int vmm_query_page (uint64_t vaddr, vmm_page_info_t *info)`

7.51.1 Macro Definition Documentation

7.51.1.1 VMM_FLAG_NX

```
#define VMM_FLAG_NX (1ull << 63)
```

7.51.1.2 VMM_FLAG_PRESENT

```
#define VMM_FLAG_PRESENT (1ull << 0)
```

7.51.1.3 VMM_FLAG_USER

```
#define VMM_FLAG_USER (1ull << 2)
```

7.51.1.4 VMM_FLAG_WRITABLE

```
#define VMM_FLAG_WRITABLE (1ull << 1)
```

7.51.1.5 VMM_PAGE_SIZE

```
#define VMM_PAGE_SIZE 4096ull
```

vmm.h - Virtual Memory Manager interface (x86_64 4-level paging)

7.51.2 Function Documentation

7.51.2.1 vmm_get_paddr()

```
uint64_t vmm_get_paddr (
    uint64_t vaddr )
```

References [ADDR_MASK](#), [get_table\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), and [vmm_root_pml4_phys](#).

Here is the call graph for this function:

7.51.2.2 vmm_init()

```
int vmm_init (
    BootInfo * boot_info )
```

References [alloc_page_zeroed\(\)](#), [console_println\(\)](#), [hpet_phys_base](#), [is_ram_type\(\)](#), [lapic_phys_base](#), [load_cr3\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [EFI_MEMORY_DESCRIPTOR::Num](#), [EFI_MEMORY_DESCRIPTOR::PhysicalStart](#), [pmm_is_initialized\(\)](#), [EFI_MEMORY_DESCRIPTOR::Type](#), [VMM_FLAG_WRITABLE](#), [vmm_map_range\(\)](#), [VMM_PAGE_SIZE](#), and [vmm_root_pml4_phys](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.51.2.3 vmm_map_page()

```
int vmm_map_page (
    uint64_t vaddr,
    uint64_t paddr,
    uint64_t flags )
```

References [ADDR_MASK](#), [console_println\(\)](#), [flags](#), [get_or_alloc_table\(\)](#), [invlpg\(\)](#), [make_pte\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), [PTE_WRITABLE](#), [VMM_FLAG_PRESENT](#), and [vmm_root_pml4_phys](#).

Referenced by [vmm_map_range\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.51.2.4 vmm_map_range()

```
int vmm_map_range (
    uint64_t vaddr,
    uint64_t paddr,
    uint64_t size,
    uint64_t flags )
```

References [flags](#), [vmm_map_page\(\)](#), and [VMM_PAGE_SIZE](#).

Referenced by [timer_init\(\)](#), and [vmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.51.2.5 vmm_query_page()

```
int vmm_query_page (
    uint64_t vaddr,
    vmm_page_info_t * info )
```

References [entry](#), [vmm_page_info_t::executable](#), [vmm_page_info_t::present](#), [PTE_NX](#), [PTE_PRESENT](#), [PTE_WRITABLE](#), [resolve_pt\(\)](#), and [vmm_page_info_t::writable](#).

Here is the call graph for this function:

7.51.2.6 vmm_unmap_page()

```
int vmm_unmap_page (
    uint64_t vaddr )
```

References [get_table\(\)](#), [invlpg\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), and [vmm_root_pml4_phys](#).

Here is the call graph for this function:

7.52 include/starkernel/xxhash64.h File Reference

```
#include <stdint.h>
```

```
#include <stddef.h>
```

Include dependency graph for xxhash64.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [XXHash64State](#)
Streaming state structure.

Macros

- #define [XXHASH64_CAPSULE_SEED](#) 0ULL
[xxhash64.h](#) - xxHash64 Implementation (Freestanding)

Functions

- `uint64_t xxhash64` (const void *data, size_t len, uint64_t seed)
xxhash64 - Compute xxHash64 of a buffer
- `static uint64_t xxhash64_capsule` (const void *data, size_t len)
xxhash64_capsule - Hash capsule payload with standard seed
- `void xxhash64_reset` (XXHash64State *state, uint64_t seed)
xxhash64_reset - Initialize streaming state
- `void xxhash64_update` (XXHash64State *state, const void *data, size_t len)
xxhash64_update - Feed data into streaming hash
- `uint64_t xxhash64_digest` (const XXHash64State *state)
xxhash64_digest - Finalize and get hash value

7.52.1 Macro Definition Documentation

7.52.1.1 XXHASH64_CAPSULE_SEED

```
#define XXHASH64_CAPSULE_SEED 0ULL
```

[xxhash64.h](#) - xxHash64 Implementation (Freestanding)

Fast, deterministic 64-bit hash function for capsule content addressing. No libc dependency - works in kernel context.

Based on xxHash by Yann Collet (BSD-2-Clause license).

<https://github.com/Cyan4973/xxHash>

Default seed for capsule hashing (deterministic)

7.52.2 Function Documentation

7.52.2.1 xxhash64()

```
uint64_t xxhash64 (
    const void * data,
    size_t len,
    uint64_t seed )
```

xxhash64 - Compute xxHash64 of a buffer

Parameters

<i>data</i>	Input data
<i>len</i>	Length in bytes
<i>seed</i>	Hash seed (use XXHASH64_CAPSULE_SEED for capsules)

Returns

64-bit hash value

References [limit](#), [xxh64_avalanche\(\)](#), [xxh64_merge_round\(\)](#), [xxh64_read32\(\)](#), [xxh64_read64\(\)](#), [xxh64_rotl\(\)](#), [xxh64_round\(\)](#), [XXHASH64_PRIME1](#), [XXHASH64_PRIME2](#), [XXHASH64_PRIME3](#), [XXHASH64_PRIME4](#), and [XXHASH64_PRIME5](#).

Referenced by [xxhash64_capsule\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.52.2.2 xxhash64_capsule()

```
static uint64_t xxhash64_capsule (
    const void * data,
    size_t len ) [inline], [static]
```

xxhash64_capsule - Hash capsule payload with standard seed

Convenience wrapper using XXHASH64_CAPSULE_SEED.

Parameters

<i>data</i>	Capsule payload bytes
<i>len</i>	Payload length

Returns

Content hash (capsule_id)

References [xxhash64\(\)](#), and [XXHASH64_CAPSULE_SEED](#).

Referenced by [capsule_validate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.52.2.3 xxhash64_digest()

```
uint64_t xxhash64_digest (
    const XXHash64State * state )
```

xxhash64_digest - Finalize and get hash value

Parameters

<i>state</i>	Streaming state
--------------	-----------------

Returns

64-bit hash value

References [XXHash64State::buffer](#), [XXHash64State::buffer_size](#), [XXHash64State::seed](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [xxh64_avalanche\(\)](#), [xxh64_merge_round\(\)](#), [xxh64_read32\(\)](#), [xxh64_read64\(\)](#), [xxh64_rotl\(\)](#), [xxh64_round\(\)](#), [XXHASH64_PRIME1](#), [XXHASH64_PRIME2](#), [XXHASH64_PRIME3](#), [XXHASH64_PRIME4](#), and [XXHASH64_PRIME5](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.52.2.4 xxhash64_reset()

```
void xxhash64_reset (
    XXHash64State * state,
    uint64_t seed )
```

xxhash64_reset - Initialize streaming state

Parameters

<i>state</i>	State to initialize
<i>seed</i>	Hash seed

References [XXHash64State::buffer_size](#), [XXHash64State::seed](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [XXHASH64_PRIME1](#), and [XXHASH64_PRIME2](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the caller graph for this function:

7.52.2.5 xxhash64_update()

```
void xxhash64_update (
    XXHash64State * state,
    const void * data,
    size_t len )
```

xxhash64_update - Feed data into streaming hash

Parameters

<i>state</i>	Streaming state
<i>data</i>	Input data
<i>len</i>	Length in bytes

References [XXHash64State::buffer](#), [XXHash64State::buffer_size](#), [limit](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [xxh64_read64\(\)](#), and [xxh64_round\(\)](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.53 include/startup.h File Reference

```
#include "vm.h"
```

```
#include "cli.h"
```

Include dependency graph for startup.h:

Functions

- int [startup_blkio_init](#) (const [CLIConfig](#) *config)
Initialize block I/O device from CLI configuration.
- void [startup_blkio_cleanup](#) (void)
Clean up block I/O device.
- int [startup_vm_init](#) ([VM](#) *vm)
Initialize VM with defaults.
- int [startup_block_subsys_init](#) ([VM](#) *vm)
Initialize block subsystem.

7.53.1 Function Documentation

7.53.1.1 startup_blkio_cleanup()

```
void startup_blkio_cleanup (
    void )
```

Clean up block I/O device.

Flushes and closes block device, frees RAM disk if allocated. Called automatically via [atexit\(\)](#).

7.53.1.2 startup_blkio_init()

```
int startup_blkio_init (
    const CLIConfig * config )
```

Initialize block I/O device from CLI configuration.

Opens disk image or allocates RAM disk based on [CLIConfig](#). Must be called before [vm_init\(\)](#).

Parameters

<i>config</i>	CLI configuration
---------------	-------------------

Returns

0 on success, 1 on error (logs messages internally)

7.53.1.3 startup_block_subsys_init()

```
int startup_block_subsys_init (
    VM * vm )
```

Initialize block subsystem.
Sets up block buffering and attaches blkio device.

Parameters

<i>vm</i>	Pointer to initialized VM
-----------	---

Returns

0 on success, 1 on error

7.53.1.4 startup_vm_init()

```
int startup_vm_init (
    VM * vm )
```

Initialize [VM](#) with defaults.
Initializes the [VM](#) state after block device is ready.

Parameters

<i>vm</i>	Pointer to VM to initialize
-----------	---

Returns

0 on success, 1 on error

7.54 include/vm.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include <stdio.h>
#include "platform_lock.h"
#include "starforth_config.h"
Include dependency graph for vm.h:
```

Data Structures

- struct [RollingWindowOfTruth](#)
- struct [HeartbeatSnapshot](#)
- struct [HeartbeatTickSnapshot](#)
- struct [DictPhysics](#)
- struct [HeartbeatState](#)
- struct [PipelineGlobalMetrics](#)
- struct [DictEntry](#)
- struct [VM](#)

Main virtual machine state container.

Macros

- #define [STACK_SIZE](#) 1024
- #define [DICTIONARY_SIZE](#) 1024
- #define [WORD_ID_INVALID](#) UINT32_MAX
- #define [VM_MEMORY_SIZE](#) (5 * 1024 * 1024) /* 5 MB total [VM](#) memory */
- #define [INPUT_BUFFER_SIZE](#) 256
- #define [WORD_NAME_MAX](#) 31
- #define [COMPILE_BUFFER_SIZE](#) 1024

- `#define BLOCK_SIZE 1024 /* 1KB per block */`
- `#define MAX_BLOCKS (VM_MEMORY_SIZE / BLOCK_SIZE) /* 5120 blocks from 5MB */`
- `#define DICTIONARY_BLOCKS 2048 /* First 2048 blocks (2MB) for dictionary */`
- `#define DICTIONARY_MEMORY_SIZE (DICTIONARY_BLOCKS * BLOCK_SIZE)`
- `#define USER_BLOCKS_START DICTIONARY_BLOCKS /* User blocks start at block 2048 */`
- `#define LOG_LINE_MAX 64 /* Each log line is 64 bytes */`
- `#define LOG_LINES_PER_BLOCK (BLOCK_SIZE / LOG_LINE_MAX) /* 16 lines per 1KB block */`
- `#define LOG_BLOCKS_START 3072 /* Log starts at block 3072 */`
- `#define LOG_BLOCKS_END 5120 /* Log ends at block 5120 (exclusive) */`
- `#define LOG_BLOCKS (LOG_BLOCKS_END - LOG_BLOCKS_START) /* 2048 blocks (2MB) for logs */`
- `#define LOG_LAYER1_MAX_LINES (LOG_BLOCKS * LOG_LINES_PER_BLOCK) /* 32768 max lines */`
- `#define WORD_IMMEDIATE 0x80 /* Word executes immediately even in compile mode */`
- `#define WORD_HIDDEN 0x40 /* Word is hidden from dictionary searches */`
- `#define WORD_SMUDGED 0x20 /* Word is smudged (being defined) - FORTH-79 */`
- `#define WORD_COMPILED 0x10 /* Word is user-compiled (not built-in) */`
- `#define WORD_PINNED 0x08 /* Word's execution heat is pinned (cannot decay to zero) */`
- `#define WORD_FROZEN 0x04 /* Word is frozen: execution heat does not decay (Phase 2) */`
- `#define ACL_USER_DEFAULT 0x01 /* Default access: users can execute and compile */`
- `#define ACL_MODE_TTL 0 /* Adaptive TTL countdown; 0 → calls ACL-RECHECK */`
- `#define ACL_MODE_STRICT 1 /* Re-check every execution (STRICT mode) */`
- `#define ACL_TTL_OPEN UINT32_MAX /* Permissive TTL before ACL.4th loads */`
- `#define SPIN_IDLE 0 /* Particle spin state: idle */`
- `#define CHARGE_NEUTRAL 0 /* Particle charge state: neutral (execution heat acts as charge) */`
- `#define PHYSICS_STATE_IMMEDIATE 0x01`
- `#define PHYSICS_STATE_PINNED 0x02`
- `#define PHYSICS_STATE_HIDDEN 0x04`
- `#define PHYSICS_STATE_COMPILED 0x08`
- `#define HEAT_CACHE_DEMOTION_THRESHOLD 10 /* Demotion from hotwords cache threshold */`
- `#define HEARTBEAT_TICK_BUFFER_SIZE 100000 /* ~100K ticks = ~100 seconds at 1kHz */`
- `#define VM_PUSH(vm, val) vm_push_fast(vm, val)`
- `#define VM_POP(vm) vm_pop_fast(vm)`
- `#define VM_RPUSH(vm, val) vm_rpush_fast(vm, val)`
- `#define VM_RPOP(vm) vm_rpop_fast(vm)`
- `#define LIKELY(x) __builtin_expect(!!(x), 1)`
- `#define UNLIKELY(x) __builtin_expect(!!(x), 0)`

Typedefs

- `typedef struct HotwordsCache_s HotwordsCache`
- `typedef struct WordTransitionMetrics WordTransitionMetrics`
- `typedef struct VMHostServices VMHostServices`
- `typedef struct InferenceOutputs InferenceOutputs`
- `typedef struct HeartbeatWorker HeartbeatWorker`
- `typedef signed long cell_t`
- `typedef unsigned long ucell_t`
- `typedef void(* word_func_t) (struct VM *vm)`
- `typedef uint64_t vaddr_t`
- `typedef struct VM VM`

Enumerations

- `enum vm_mode_t { MODE_INTERPRET = 0, MODE_COMPILE = 1 }`

Functions

- `int vm_addr_ok (struct VM *vm, vaddr_t addr, size_t len)`
Check if memory address range is valid.
- `uint8_t * vm_ptr (struct VM *vm, vaddr_t addr)`
- `uint8_t vm_load_u8 (struct VM *vm, vaddr_t addr)`
- `void vm_store_u8 (struct VM *vm, vaddr_t addr, uint8_t v)`
- `cell_t vm_load_cell (struct VM *vm, vaddr_t addr)`
- `void vm_store_cell (struct VM *vm, vaddr_t addr, cell_t v)`
- `static vaddr_t VM_ADDR (cell_t c)`
- `static cell_t CELL (vaddr_t a)`
- `struct DictEntry __attribute__((aligned(8))) DictEntry`
- `void vm_push (VM *vm, cell_t value)`
Push a value onto the data stack.
- `cell_t vm_pop (VM *vm)`
Pop a value from the data stack.
- `void vm_rpush (VM *vm, cell_t value)`
Push a value onto the return stack.
- `cell_t vm_rpop (VM *vm)`
Pop a value from the return stack.
- `DictEntry * vm_find_word (VM *vm, const char *name, size_t len)`
Finds a word in the dictionary by name.
- `DictEntry * vm_create_word (VM *vm, const char *name, size_t len, word_func_t func)`
Creates a new word in the dictionary.
- `void vm_make_immediate (VM *vm)`
- `void vm_bootstrap_root_vocabulary (VM *vm, const char *name)`
- `void vm_hide_word (VM *vm)`
Marks the latest word as hidden.
- `void vm_smudge_word (VM *vm)`
- `void vm_pin_execution_heat (VM *vm)`
- `void vm_unpin_execution_heat (VM *vm)`
- `DictEntry * vm_dictionary_find_by_func (VM *vm, word_func_t func)`
Finds a word by its function pointer.
- `DictEntry * vm_dictionary_find_latest_by_func (VM *vm, word_func_t func)`
- `DictEntry * vm_dictionary_lookup_by_word_id (VM *vm, uint32_t word_id)`
- `void vm_dictionary_track_entry (VM *vm, DictEntry *entry)`
- `void vm_dictionary_untrack_entry (VM *vm, DictEntry *entry)`
- `cell_t * vm_dictionary_get_data_field (DictEntry *entry)`
Gets pointer to a word's data field.
- `void vm_compile_word (VM *vm, DictEntry *entry)`
- `void * vm_allot (VM *vm, size_t bytes)`
Allocates memory from the VM's dictionary space.
- `void vm_align (VM *vm)`
Aligns the VM's dictionary pointer (here) to the cell boundary.
- `int vm_parse_word (VM *vm, char *word, size_t max_len)`
Parse next word from input buffer.
- `int vm_parse_number (VM *vm, const char *str, cell_t *value)`
Parse string as number in current base.
- `void vm_enter_compile_mode (VM *vm, const char *name, size_t len)`
- `void vm_exit_compile_mode (VM *vm)`
- `void execute_colon_word (VM *vm)`
- `void vm_compile_call (VM *vm, word_func_t func)`
- `void vm_compile_literal (VM *vm, cell_t value)`

- void [vm_compile_exit](#) (VM *vm)
- void [vm_interpret_word](#) (VM *vm, const char *word_str, size_t len)
- void * [vm_get_block_addr](#) (VM *vm, int block_num)
Gets the memory address for a specified block number.
- int [vm_addr_to_block](#) (VM *vm, void *addr)
Converts a memory address to its corresponding block number.
- void [vm_run_smoke_tests](#) (VM *vm)
- void [vm_cleanup](#) (VM *vm)
Clean up and free VM resources.
- void [vm_tick](#) (VM *vm)
Central heartbeat dispatcher for time-driven tuning operations.
- void [vm_tick_inference_engine](#) (VM *vm)
Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.
- void [vm_tick_window_tuner](#) (VM *vm)
Loop #5: Context-aware window tuning via binary chop search.
- void [vm_tick_slope_validator](#) (VM *vm)
Loop #3: Heat decay slope validation via periodic measurement.
- void [vm_snapshot_read](#) (const VM *vm, HeartbeatSnapshot *out_snapshot)
- void [heartbeat_capture_tick_snapshot](#) (VM *vm, HeartbeatTickSnapshot *snapshot)
Capture current VM metrics into a heartbeat tick snapshot.
- void [heartbeat_emit_tick_row](#) (VM *vm, HeartbeatTickSnapshot *snapshot)
Emit a heartbeat tick snapshot as CSV row to stderr.
- void [vm_heartbeat_start](#) (VM *vm)
- static void [vm_push_fast](#) (VM *vm, cell_t value)
- static cell_t [vm_pop_fast](#) (VM *vm)
- static void [vm_rpush_fast](#) (VM *vm, cell_t value)
- static cell_t [vm_rpop_fast](#) (VM *vm)

Core VM Functions

- void [vm_init](#) (VM *vm)
Initialize a new VM instance.
- void [vm_init_with_host](#) (VM *vm, const VMHostServices *host)
Initialize VM with explicit host services.
- void [vm_interpret](#) (VM *vm, const char *input)
Interpret a string of Forth code.
- void [vm_enable_interpreter](#) (VM *vm)
- void [vm_repl](#) (VM *vm, int script_mode)
Start the VM's read-eval-print loop.

Variables

- typedef [__attribute__](#)
- struct [DictEntry](#) * [link](#)
- [word_func_t](#) [func](#)
- [cell_t](#) [flags](#)
- [uint8_t](#) [name_len](#)
- [cell_t](#) [execution_heat](#)
- [uint8_t](#) [acl_default](#)
- [uint32_t](#) [acl_ttl](#)
- [uint8_t](#) [acl_allow](#)
- [uint8_t](#) [acl_mode](#)
- [uint8_t](#) [acl_pinned](#)
- [uint32_t](#) [word_id](#)
- [DictPhysics](#) [physics](#)
- [WordTransitionMetrics](#) * [transition_metrics](#)
- char [name](#) []

7.54.1 Macro Definition Documentation

7.54.1.1 ACL_MODE_STRICT

```
#define ACL_MODE_STRICT 1 /* Re-check every execution (STRICT mode) */
```

7.54.1.2 ACL_MODE_TTL

```
#define ACL_MODE_TTL 0 /* Adaptive TTL countdown; 0 → calls ACL-RECHECK */
```

7.54.1.3 ACL_TTL_OPEN

```
#define ACL_TTL_OPEN UINT32_MAX /* Permissive TTL before ACL.4th loads */
```

7.54.1.4 ACL_USER_DEFAULT

```
#define ACL_USER_DEFAULT 0x01 /* Default access: users can execute and compile */
```

7.54.1.5 BLOCK_SIZE

```
#define BLOCK_SIZE 1024 /* 1KB per block */
```

7.54.1.6 CHARGE_NEUTRAL

```
#define CHARGE_NEUTRAL 0 /* Particle charge state: neutral (execution heat acts as charge) */
```

7.54.1.7 COMPILE_BUFFER_SIZE

```
#define COMPILE_BUFFER_SIZE 1024
```

7.54.1.8 DICTIONARY_BLOCKS

```
#define DICTIONARY_BLOCKS 2048 /* First 2048 blocks (2MB) for dictionary */
```

7.54.1.9 DICTIONARY_MEMORY_SIZE

```
#define DICTIONARY_MEMORY_SIZE (DICTIONARY_BLOCKS * BLOCK_SIZE)
```

7.54.1.10 DICTIONARY_SIZE

```
#define DICTIONARY_SIZE 1024
```

7.54.1.11 HEARTBEAT_TICK_BUFFER_SIZE

```
#define HEARTBEAT_TICK_BUFFER_SIZE 100000 /* ~100K ticks = ~100 seconds at 1kHz */
```

7.54.1.12 HEAT_CACHE_DEMOTION_THRESHOLD

```
#define HEAT_CACHE_DEMOTION_THRESHOLD 10 /* Demotion from hotwords cache threshold */
```

7.54.1.13 INPUT_BUFFER_SIZE

```
#define INPUT_BUFFER_SIZE 256
```

7.54.1.14 LIKELY

```
#define LIKELY(  
    x ) __builtin_expect(!!(x), 1)
```

7.54.1.15 LOG_BLOCKS

```
#define LOG_BLOCKS (LOG_BLOCKS_END - LOG_BLOCKS_START) /* 2048 blocks (2MB) for logs */
```

7.54.1.16 LOG_BLOCKS_END

```
#define LOG_BLOCKS_END 5120 /* Log ends at block 5120 (exclusive) */
```

7.54.1.17 LOG_BLOCKS_START

```
#define LOG_BLOCKS_START 3072 /* Log starts at block 3072 */
```

7.54.1.18 LOG_LAYER1_MAX_LINES

```
#define LOG_LAYER1_MAX_LINES (LOG_BLOCKS * LOG_LINES_PER_BLOCK) /* 32768 max lines */
```

7.54.1.19 LOG_LINE_MAX

```
#define LOG_LINE_MAX 64 /* Each log line is 64 bytes */
```

7.54.1.20 LOG_LINES_PER_BLOCK

```
#define LOG_LINES_PER_BLOCK (BLOCK_SIZE / LOG_LINE_MAX) /* 16 lines per 1KB block */
```

7.54.1.21 MAX_BLOCKS

```
#define MAX_BLOCKS (VM_MEMORY_SIZE / BLOCK_SIZE) /* 5120 blocks from 5MB */
```

7.54.1.22 PHYSICS_STATE_COMPILED

```
#define PHYSICS_STATE_COMPILED 0x08
```

7.54.1.23 PHYSICS_STATE_HIDDEN

```
#define PHYSICS_STATE_HIDDEN 0x04
```

7.54.1.24 PHYSICS_STATE_IMMEDIATE

```
#define PHYSICS_STATE_IMMEDIATE 0x01
```

7.54.1.25 PHYSICS_STATE_PINNED

```
#define PHYSICS_STATE_PINNED 0x02
```

7.54.1.26 SPIN_IDLE

```
#define SPIN_IDLE 0 /* Particle spin state: idle */
```

7.54.1.27 STACK_SIZE

```
#define STACK_SIZE 1024
```

7.54.1.28 UNLIKELY

```
#define UNLIKELY(  
    x ) __builtin_expect(!!(x), 0)
```

7.54.1.29 USER_BLOCKS_START

```
#define USER_BLOCKS_START DICTIONARY_BLOCKS /* User blocks start at block 2048 */
```

7.54.1.30 VM_MEMORY_SIZE

```
#define VM_MEMORY_SIZE (5 * 1024 * 1024) /* 5 MB total VM memory */
```

7.54.1.31 VM_POP

```
#define VM_POP(
    vm ) vm_pop_fast(vm)
```

7.54.1.32 VM_PUSH

```
#define VM_PUSH(
    vm,
    val ) vm_push_fast(vm, val)
```

7.54.1.33 VM_RPOP

```
#define VM_RPOP(
    vm ) vm_rpop_fast(vm)
```

7.54.1.34 VM_RPUSH

```
#define VM_RPUSH(
    vm,
    val ) vm_rpush_fast(vm, val)
```

7.54.1.35 WORD_COMPILED

```
#define WORD_COMPILED 0x10 /* Word is user-compiled (not built-in) */
```

7.54.1.36 WORD_FROZEN

```
#define WORD_FROZEN 0x04 /* Word is frozen: execution heat does not decay (Phase 2) */
```

7.54.1.37 WORD_HIDDEN

```
#define WORD_HIDDEN 0x40 /* Word is hidden from dictionary searches */
```

7.54.1.38 WORD_ID_INVALID

```
#define WORD_ID_INVALID UINT32_MAX
```

7.54.1.39 WORD_IMMEDIATE

```
#define WORD_IMMEDIATE 0x80 /* Word executes immediately even in compile mode */
```

7.54.1.40 WORD_NAME_MAX

```
#define WORD_NAME_MAX 31
```

7.54.1.41 WORD_PINNED

```
#define WORD_PINNED 0x08 /* Word's execution heat is pinned (cannot decay to zero) */
```

7.54.1.42 WORD_SMUDGED

```
#define WORD_SMUDGED 0x20 /* Word is smudged (being defined) - FORTH-79 */
```

7.54.2 Typedef Documentation**7.54.2.1 cell_t**

```
typedef signed long cell_t
```

7.54.2.2 HeartbeatWorker

```
typedef struct HeartbeatWorker HeartbeatWorker
```

7.54.2.3 HotwordsCache

```
typedef struct HotwordsCache_s HotwordsCache
```

7.54.2.4 InferenceOutputs

```
typedef struct InferenceOutputs InferenceOutputs
```

7.54.2.5 ucell_t

```
typedef unsigned long ucell_t
```

7.54.2.6 vaddr_t

```
typedef uint64_t vaddr_t
```

7.54.2.7 VM

```
typedef struct VM VM
```

7.54.2.8 VMHostServices

```
typedef struct VMHostServices VMHostServices
```

7.54.2.9 word_func_t

```
typedef void(* word_func_t) (struct VM *vm)
```

7.54.2.10 WordTransitionMetrics

```
typedef struct WordTransitionMetrics WordTransitionMetrics
```

7.54.3 Enumeration Type Documentation

7.54.3.1 vm_mode_t

```
enum vm_mode_t
```

Enumerator

MODE_INTERPRET	
MODE_COMPILE	

7.54.4 Function Documentation

7.54.4.1 __attribute__()

```
struct DictEntry __attribute__ (
    (aligned(8)) )
```

7.54.4.2 CELL()

```
static cell_t CELL (
    vaddr_t a ) [inline], [static]
```

Referenced by [block_word_scr\(\)](#), [defining_runtime_create\(\)](#), [format_word_base\(\)](#), [format_word_end_conversion\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#),

[string_word_skip\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_to_in\(\)](#), [string_word_word\(\)](#), [vocabulary_word_context\(\)](#), and [vocabulary_word_current\(\)](#).

Here is the caller graph for this function:

7.54.4.3 execute_colon_word()

```
void execute_colon_word (
    VM * vm )
```

References [VM::abort_requested](#), [HeartbeatState::check_counter](#), [VM::current_executing_entry](#), [DICTIONARY_SIZE](#), [ENABLE_HOTWORDS_CACHE](#), [ENABLE_PIPELINING](#), [entry](#), [VM::error](#), [DictEntry::execution_heat](#), [VM::exit_colon](#), [DictEntry::func](#), [VM::heartbeat](#), [HEARTBEAT_CHECK_FREQUENCY](#), [VM::hotwords_cache](#), [hotwords_cache_promote\(\)](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [LOG_ERROR](#), [log_message\(\)](#), [WordTransitionMetrics::most_likely_next_word_id](#), [DictEntry::physics](#), [physics_execution_heat_increment](#), [physics_metadata_apply_linear_decay\(\)](#), [physics_metadata_touch\(\)](#), [VM::pipeline_metrics](#), [WordTransitionMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [profiler_word_exit\(\)](#), [VM::rolling_window](#), [rolling_window_record_execution\(\)](#), [DictEntry::transition_metrics](#), [transition_metrics_record\(\)](#), [transition_metrics_record_prefetch_hit\(\)](#), [transition_metrics_should_speculate\(\)](#), [transition_metrics_update_cache\(\)](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_dictionary_lookup_by_word_id\(\)](#), [vm_heartbeat_run_cycle\(\)](#), [vm_monotonic_ns\(\)](#), [vm_ptr\(\)](#), [vm_rpop\(\)](#), [vm_rpush\(\)](#), [word_id](#), [DictEntry::word_id](#), [HeartbeatState::words_executed](#), and [HeartbeatState::worker](#).

Referenced by [system_word_see\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.4 heartbeat_capture_tick_snapshot()

```
void heartbeat_capture_tick_snapshot (
    VM * vm,
    HeartbeatTickSnapshot * snapshot )
```

Capture current [VM](#) metrics into a heartbeat tick snapshot.

Collects 11 metrics per tick for multivariate dynamics analysis. Called every heartbeat tick to record [VM](#) state.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>snapshot</i>	Pointer to snapshot structure to populate

References [HeartbeatTickSnapshot::actual_window_size](#), [HeartbeatTickSnapshot::avg_word_heat](#), [HeartbeatTickSnapshot::bucket](#), [HeartbeatTickSnapshot::cache_hits_delta](#), [RollingWindowOfTruth::effective_window_size](#), [HeartbeatTickSnapshot::elapsed_ns](#), [entry](#), [HeartbeatTickSnapshot::estimated_jitter_ns](#), [VM::heartbeat](#), [HEARTBEAT_TICK_NS](#), [HeartbeatTickSnapshot::hot_word_count](#), [VM::latest](#), [HeartbeatTickSnapshot::predicted_label_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [HeartbeatState::run_start_ns](#), [sf_monotonic_ns\(\)](#), [HeartbeatState::tick_count_total](#), [HeartbeatTickSnapshot::tick_interval_ns](#), [HeartbeatTickSnapshot::tick_number](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [vm](#), [HeartbeatTickSnapshot::window_width](#), [word_count](#), and [HeartbeatTickSnapshot::word_executions_delta](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.5 heartbeat_emit_tick_row()

```
void heartbeat_emit_tick_row (
    VM * vm,
    HeartbeatTickSnapshot * snapshot )
```

Emit a heartbeat tick snapshot as CSV row to stderr.

Outputs 11 comma-separated values (no header) for real-time streaming. Called every heartbeat tick to emit metrics immediately.

Format: tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta, bucket_hits_delta,word_executions_delta,hot_word_count, avg_word_heat>window_width,actual_window_size,predicted_label_hits,estimated_jitter_ns

Parameters

<i>vm</i>	Pointer to the VM instance (unused, for API consistency)
<i>snapshot</i>	Pointer to the snapshot to emit

References [HeartbeatTickSnapshot::actual_window_size](#), [HeartbeatTickSnapshot::avg_word_heat](#), [HeartbeatTickSnapshot::bucket](#), [HeartbeatTickSnapshot::cache_hits_delta](#), [HeartbeatTickSnapshot::elapsed_ns](#), [HeartbeatTickSnapshot::estimated_jitter_ns](#), [HeartbeatTickSnapshot::hot_word_count](#), [HeartbeatTickSnapshot::predicted_label_hits](#), [HeartbeatTickSnapshot::tick_interval_ns](#), [HeartbeatTickSnapshot::tick_number](#), [vm](#), [HeartbeatTickSnapshot::window_width](#), and [HeartbeatTickSnapshot::word_executions_de](#).
Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the caller graph for this function:

7.54.4.6 VM_ADDR()

```
static vaddr\_t VM_ADDR (
    cell\_t c ) [inline], [static]
```

Referenced by [dictionary_word_2comma\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_tick\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the caller graph for this function:

7.54.4.7 vm_addr_ok()

```
int vm_addr_ok (
    struct VM * vm,
    vaddr\_t addr,
    size\_t len )
```

Check if memory address range is valid.

Validates that an address range falls within [VM](#) memory bounds.

Parameters

<i>vm</i>	Pointer to VM instance
<i>addr</i>	Virtual address to check
<i>len</i>	Length of memory range

Returns

1 if range is valid, 0 if invalid

References [VM::memory](#), [vm](#), and [VM_MEMORY_SIZE](#).

Referenced by [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_forget\(\)](#), [dump_block_preview\(\)](#), [editor_word_s\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [runtime_abortq\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_tick\(\)](#), [system_word_see\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_load_cell\(\)](#), [vm_ptr\(\)](#), [vm_store_cell\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the caller graph for this function:

7.54.4.8 vm_addr_to_block()

```
int vm_addr_to_block (
```

```

    VM * vm,
    void * addr )

```

Converts a memory address to its corresponding block number.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>addr</i>	Memory address to convert

Returns

Block number (0 to MAX_BLOCKS-1) or -1 if address is outside [VM](#) memory

Note

Used for determining which block a memory address belongs to

References [BLOCK_SIZE](#), [VM::memory](#), [vm](#), and [VM_MEMORY_SIZE](#).

7.54.4.9 vm_align()

```

void vm_align (
    VM * vm )

```

Aligns the [VM](#)'s dictionary pointer (here) to the cell boundary.

Aligns the virtual machine's memory pointer to the next word boundary.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Note

Adds padding bytes if necessary to ensure proper alignment for cell_t type

7.54.4.10 vm_allot()

```

void * vm_allot (
    VM * vm,
    size_t bytes )

```

Allocates memory from the [VM](#)'s dictionary space.

Allocates memory in the virtual machine's memory space.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>bytes</i>	Number of bytes to allocate

Returns

Pointer to the allocated memory or NULL if allocation fails

Note

The allocation is done from the [VM](#)'s dictionary space which is limited to [DICTIONARY_MEMORY_SIZE](#)

Referenced by [vm_align\(\)](#).

Here is the caller graph for this function:

7.54.4.11 vm_bootstrap_root_vocabulary()

```
void vm_bootstrap_root_vocabulary (
    VM * vm,
    const char * name )
```

References [console_println\(\)](#), [console_puts\(\)](#), [entry](#), [VM::error](#), [VMHostServices::is_executable_ptr](#), [LOG_ERROR](#), [log_message\(\)](#), [name](#), [VMHostServices::panic](#), [vm](#), [vm_find_word\(\)](#), [vm_host\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_definitions\(\)](#).

Here is the call graph for this function:

7.54.4.12 vm_cleanup()

```
void vm_cleanup (
    VM * vm )
```

Clean up and free [VM](#) resources.

Frees allocated memory and resets [VM](#) state.

Parameters

<i>vm</i>	Pointer to VM structure to clean up
-----------	---

References [VM::dict_lock](#), [VM::heartbeat](#), [VM::here](#), [VM::hotwords_cache](#), [hotwords_cache_cleanup\(\)](#), [VM::memory](#), [VM::rolling_window](#), [rolling_window_cleanup\(\)](#), [sf_mutex_destroy\(\)](#), [VM::ssm_config](#), [VM::ssm_l8_state](#), [HeartbeatWorker::stop_requested](#), [VM::tuning_lock](#), [vm](#), [vm_host_free\(\)](#), and [HeartbeatState::worker](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), and [cleanup_and_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.13 vm_compile_call()

```
void vm_compile_call (
    VM * vm,
    word_func_t func )
```

References [entry](#), [VM::error](#), [func](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_word\(\)](#), and [vm_dictionary_find_by_func\(\)](#).

Referenced by [control_forth_again\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_if\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [defining_word_does\(\)](#), and [system_word_abort_quote\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.14 vm_compile_exit()

```
void vm_compile_exit (
    VM * vm )
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_word\(\)](#), and [vm_find_word\(\)](#).

Here is the call graph for this function:

7.54.4.15 vm_compile_literal()

```
void vm_compile_literal (
    VM * vm,
    cell_t value )
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_word_literal\(\)](#), [dictionary_m_word_tick\(\)](#), [system_word_abort_quote\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.16 vm_compile_word()

```
void vm_compile_word (
    VM * vm,
    DictEntry * entry )
```

References [entry](#), [VM::error](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_align\(\)](#), and [vm_allot\(\)](#).

Referenced by [defining_word_compile\(\)](#), [defining_word_does\(\)](#), [string_word_s_quote\(\)](#), [vm_compile_call\(\)](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.17 vm_create_word()

```
DictEntry * vm_create_word (
    VM * vm,
    const char * name,
    size_t len,
    word_func_t func )
```

Creates a new word in the dictionary.

Allocates and initializes a new dictionary entry with the given name and function. The word is added to the dictionary but index update is deferred until next lookup.

Parameters

<i>vm</i>	The virtual machine context
<i>name</i>	Name for the new word
<i>len</i>	Length of the name string
<i>func</i>	Function pointer for the word's behavior

Returns

DictEntry* Pointer to new dictionary entry or NULL on failure

References [ACL_MODE_TTL](#), [ACL_USER_DEFAULT](#), [base](#), [VM::dict_lock](#), [entry](#), [VM::error](#), [func](#), [DictEntry::func](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [name](#), [physics_metadata_apply_seed\(\)](#), [physics_metadata_init\(\)](#), [sf_malloc\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [transition_metrics_init\(\)](#), [vm](#), [vm_dictionary_track_entry\(\)](#), [WORD_ID_INVALID](#), and [WORD_NAME_MAX](#).

Referenced by [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [register_physics_pipelining_diagnostic_w](#), [register_word\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_inherit\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_pin_immutable\(\)](#), [vm_api_create_word\(\)](#), [vm_enter_compile_mode\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.18 vm_dictionary_find_by_func()

```
DictEntry * vm_dictionary_find_by_func (
    VM * vm,
    word_func_t func )
```

Finds a word by its function pointer.

Performs a linear scan of the dictionary to find an entry with matching function.

Parameters

<i>vm</i>	The virtual machine context
<i>func</i>	Function pointer to search for

Returns

DictEntry* Pointer to found dictionary entry or NULL if not found

References [func](#), [VM::latest](#), and [vm](#).

Referenced by [vm_compile_call\(\)](#), and [vm_dictionary_find_latest_by_func\(\)](#).

Here is the caller graph for this function:

7.54.4.19 vm_dictionary_find_latest_by_func()

```
DictEntry * vm_dictionary_find_latest_by_func (
    VM * vm,
    word_func_t func )
```

References [func](#), [vm](#), and [vm_dictionary_find_by_func\(\)](#).

Here is the call graph for this function:

7.54.4.20 vm_dictionary_get_data_field()

```
cell_t * vm_dictionary_get_data_field (
    DictEntry * entry )
```

Gets pointer to a word's data field.

Calculates and returns pointer to the aligned data field following the name field.

Parameters

<i>entry</i>	Dictionary entry to get data field from
--------------	---

Returns

cell_t* Pointer to data field or NULL if entry is NULL

References [base](#), [entry](#), and [name](#).

Referenced by [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_word_forget\(\)](#), [execute_colon_word\(\)](#), [hash_colon_body\(\)](#), [system_word_see\(\)](#), [vm_enter_compile_mode\(\)](#), and [vm_exit_compile_mode\(\)](#).

Here is the caller graph for this function:

7.54.4.21 vm_dictionary_lookup_by_word_id()

```
DictEntry * vm_dictionary_lookup_by_word_id (
    VM * vm,
    uint32_t word_id )
```

References [DICTIONARY_SIZE](#), [vm](#), [word_id](#), and [VM::word_id_map](#).

Referenced by [execute_colon_word\(\)](#), [extract_heat_trajectory\(\)](#), [physics_pre_execute\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Here is the caller graph for this function:

7.54.4.22 vm_dictionary_track_entry()

```
void vm_dictionary_track_entry (
    VM * vm,
    DictEntry * entry )
```

References [DICTIONARY_SIZE](#), [entry](#), [log_message\(\)](#), [LOG_WARN](#), [vm](#), [vm_dictionary_acquire_word_id\(\)](#), [word_id](#), [WORD_ID_INVALID](#), and [VM::word_id_map](#).

Referenced by [vm_create_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.23 vm_dictionary_untrack_entry()

```
void vm_dictionary_untrack_entry (
```

```

    VM * vm,
    DictEntry * entry )

```

References [DICTIONARY_SIZE](#), [entry](#), [VM::recycled_word_id_count](#), [VM::recycled_word_ids](#), [vm](#), [word_id](#), [DictEntry::word_id](#), [WORD_ID_INVALID](#), and [VM::word_id_map](#).

Referenced by [dictionary_word_forget\(\)](#).

Here is the caller graph for this function:

7.54.4.24 vm_enable_interpreter()

```

void vm_enable_interpreter (
    VM * vm )

```

References [VM::interpreter_enabled](#), and [vm](#).

7.54.4.25 vm_enter_compile_mode()

```

void vm_enter_compile_mode (
    VM * vm,
    const char * name,
    size_t len )

```

References [VM::compiling_word](#), [VM::current_word_name](#), [VM::error](#), [execute_colon_word\(\)](#), [DictEntry::flags](#), [VM::here](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [name](#), [VM::state_addr](#), [VM::state_var](#), [vm](#), [vm_align\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_store_cell\(\)](#), [WORD_NAME_MAX](#), and [WORD_SMUDGED](#).

Referenced by [defining_word_colon\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.26 vm_exit_compile_mode()

```

void vm_exit_compile_mode (
    VM * vm )

```

References [VM::compiling_word](#), [VM::error](#), [DictEntry::flags](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_INTERPRET](#), [physics_metadata_refresh_state\(\)](#), [physics_metadata_set_mass\(\)](#), [VM::state_addr](#), [VM::state_var](#), [vm](#), [vm_compile_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_find_word\(\)](#), [vm_store_cell\(\)](#), and [WORD_COMPILED](#).

Referenced by [defining_word_semicolon\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.27 vm_find_word()

```

DictEntry * vm_find_word (
    VM * vm,
    const char * name,
    size_t len )

```

Finds a word in the dictionary by name.

Searches for a word in the dictionary using an optimized first-character index. The search is performed newest-first within the matching first-character bucket.

Parameters

<i>vm</i>	The virtual machine context
<i>name</i>	The name to search for
<i>len</i>	Length of the name string

Returns

DictEntry* Pointer to found dictionary entry or NULL if not found

References [dict_find_word_heat_aware\(\)](#), [VM::dict_lock](#), [HeartbeatState::dictionary_lookups](#), [DictEntry::flags](#), [VM::heartbeat](#), [VM::hotwords_cache](#), [hotwords_cache_lookup\(\)](#), [VM::latest](#), [VM::lookup_strategy](#), [name](#),

[DictEntry::name](#), [DictEntry::name_len](#), [sf_cached_latest](#), [sf_fc_count](#), [sf_fc_list](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [sf_try_fast_append\(\)](#), [UNLIKELY](#), [vm](#), [WORD_HIDDEN](#), and [WORD_SMUDGED](#).

Referenced by [acl_recheck\(\)](#), [control_forth_endcase\(\)](#), [control_forth_of\(\)](#), [defining_word_compile\(\)](#), [defining_word_does\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_tick\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TO](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [hash_colon_body\(\)](#), [kernel_main\(\)](#), [starforth_word_init\(\)](#), [string_word_s_quote\(\)](#), [system_word_see\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), [vm_api_find_word\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.28 vm_get_block_addr()

```
void * vm_get_block_addr (
    VM * vm,
    int block_num )
```

Gets the memory address for a specified block number.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to get the address for

Returns

Pointer to the start of the block or NULL if block number is invalid

Note

Blocks are fixed-size memory regions of [BLOCK_SIZE](#) bytes

References [BLOCK_SIZE](#), [LOG_ERROR](#), [log_message\(\)](#), [MAX_BLOCKS](#), [VM::memory](#), and [vm](#).

Here is the call graph for this function:

7.54.4.29 vm_heartbeat_start()

```
void vm_heartbeat_start (
    VM * vm )
```

7.54.4.30 vm_hide_word()

```
void vm_hide_word (
    VM * vm )
```

Marks the latest word as hidden.

Sets the [WORD_HIDDEN](#) flag on the most recently defined word.

Parameters

<i>vm</i>	The virtual machine context
-----------	-----------------------------

References [DictEntry::flags](#), [VM::latest](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_HIDDEN](#).

Referenced by [vm_api_hide_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.31 vm_init()

```
void vm_init (
    VM * vm )
```

Initialize a new [VM](#) instance.

Parameters

<i>vm</i>	Pointer to VM structure to initialize
-----------	---

Initialize a new [VM](#) instance.

Allocates memory and initializes all [VM](#) structures including:

- Memory array
- Data and return stacks
- System variables (SCR, STATE, BASE)
- Dictionary
- Forth-79 wordset

Parameters

<i>vm</i>	Pointer to VM structure to initialize
-----------	---

References [vm](#), and [vm_init_with_host\(\)](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), and [init_vm_and_subsystem\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.32 vm_init_with_host()

```
void vm_init_with_host (
    VM * vm,
    const VMHostServices * host )
```

Initialize [VM](#) with explicit host services.

Parameters

<i>vm</i>	Pointer to VM structure to initialize
<i>host</i>	Host services (allocator/time). If NULL, uses build default.

References [VM::abort_requested](#), [VM::base_addr](#), [HeartbeatState::check_counter](#), [VM::compiling_word](#), [console_println\(\)](#), [VM::current_executing_entry](#), [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [VM::dict_lock](#), [dict_update_heat_percentiles\(\)](#), [VM::dsp](#), [RollingWindowOfTruth::effective_window_size](#), [VM::error](#), [VM::exit_colon](#), [VM::halted](#), [VM::heartbeat](#), [VM::heartbeat_decay_cursor_id](#), [HeartbeatState::heartbeat_enabled](#), [heartbeat_publish_snapshot\(\)](#), [HEARTBEAT_TICK_NS](#), [VM::here](#), [VM::host](#), [VM::hotwords_cache](#), [hotwords_cache_init\(\)](#), [VM::input_length](#), [VM::input_pos](#), [VM::interpreter_enabled](#), [VM::last_bucket_reorg_ns](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [VM::last_decay_check_ns](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [VM::lookup_strategy](#), [VM::memory](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::next_word_id](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::recycled_word_id_count](#), [register_forth79_words\(\)](#), [VM::rolling_window](#), [rolling_window_init\(\)](#), [VM::rsp](#), [sf_mutex_destroy\(\)](#), [sf_mutex_init\(\)](#), [HeartbeatState::snapshot_index](#), [VM::ssm_config](#), [ssm_l8_init\(\)](#), [VM::ssm_l8_state](#), [SSM_MODE_C0](#), [VM::stale_word_count_at_check](#), [VM::state_addr](#), [VM::state_var](#), [PipelineGlobalMetrics::suggested_next_size](#), [HeartbeatState::tick_count](#), [HeartbeatWorker::tick_ns](#), [HeartbeatState::tick_target_ns](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_allot\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_debug_install_signal_handlers\(\)](#), [vm_debug_set_current_vm\(\)](#), [vm_default_host_services\(\)](#), [vm_host_alloc\(\)](#), [vm_host_calloc\(\)](#), [vm_host_free\(\)](#), [VM_MEMORY_SIZE](#), [vm_reset_hosted_fake_ns\(\)](#), [vm_set_base\(\)](#), [vm_store_cell\(\)](#), [PipelineGlobalMetrics::window_tuning_checks](#), [WORD_ID_INVALID](#), and [HeartbeatState::worker](#).

Referenced by [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.33 vm_interpret()

```
void vm_interpret (
    VM * vm,
    const char * input )
```

Interpret a string of Forth code.

Parameters

<i>vm</i>	VM instance to use
<i>input</i>	String containing Forth code to interpret

Main interpretation loop that:

- Loads input into VM buffer
- Parses words
- Executes or compiles each word
- Handles numbers

Parameters

<i>vm</i>	Pointer to VM instance
<i>input</i>	String containing Forth code to interpret

References [HeartbeatState::check_counter](#), [VM::error](#), [VM::heartbeat](#), [HEARTBEAT_CHECK_FREQUENCY](#), [VM::input_buffer](#), [INPUT_BUFFER_SIZE](#), [VM::input_length](#), [VM::input_pos](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), [vm_heartbeat_run_cycle\(\)](#), [vm_interpret_word\(\)](#), [vm_parse_word\(\)](#), and [HeartbeatState::worker](#).

Referenced by [block_word_load\(\)](#), [block_word_next_block\(\)](#), [capsule_exec_hook\(\)](#), [capsule_exec_init\(\)](#), [check_physics_transparent\(\)](#), [main\(\)](#), [register_starforth_words\(\)](#), [run_doe_experiment\(\)](#), [run_single_test\(\)](#), [sk_repl\(\)](#), [starforth_word_init\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), and [vm_repl\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.34 vm_interpret_word()

```
void vm_interpret_word (
    VM * vm,
    const char * word_str,
    size_t len )
```

References [acl_recheck\(\)](#), [VM::current_executing_entry](#), [VM::dict_lock](#), [VM::dsp](#), [VM::emergency_console](#), [entry](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::flags](#), [VM::heartbeat](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [VM::mode](#), [MODE_COMPILE](#), [name](#), [DictEntry::physics](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [physics_metadata_touch\(\)](#), [physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [profiler_word_exit\(\)](#), [VM::rsp](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), [vm_monotonic_ns\(\)](#), [vm_parse_number\(\)](#), [vm_push\(\)](#), [vm_vocabulary_find_word\(\)](#), [WORD_IMMEDIATE](#), [HeartbeatState::words_executed](#), and [VM::zuse_session](#).

Referenced by [vm_interpret\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.35 vm_load_cell()

```
cell_t vm_load_cell (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::error](#), [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [block_word_next_block\(\)](#), [block_word_update\(\)](#), [current_scr\(\)](#), [editor_word_edit\(\)](#), [mark_buffer_dirty\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_fetch\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_plus_store\(\)](#), and [vm_get_base\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.36 vm_load_u8()

```
uint8_t vm_load_u8 (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::error](#), [vm](#), and [vm_ptr\(\)](#).

Referenced by [memory_word_cfetch\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_count\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), and [string_word_skip\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.37 vm_make_immediate()

```
void vm_make_immediate (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [DictEntry::name](#), [DictEntry::name_len](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_IMMEDIATE](#).

Referenced by [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_string_words\(\)](#), and [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.38 vm_parse_number()

```
int vm_parse_number (
    VM * vm,
    const char * s,
    cell_t * out )
```

Parse string as number in current base.

Attempts to parse string as number using [VM](#)'s current number base. Handles optional sign prefix.

Parameters

<i>vm</i>	Pointer to VM instance
<i>s</i>	String to parse
<i>out</i>	Pointer to store parsed value

Returns

1 on success, 0 on parse failure

References [base](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), and [vm_get_base\(\)](#).

Referenced by [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.39 vm_parse_word()

```
int vm_parse_word (
    VM * vm,
    char * word,
    size_t max_len )
```

Parse next word from input buffer.

Skips leading whitespace and extracts next word delimited by whitespace.

Parameters

<i>vm</i>	Pointer to VM instance
<i>word</i>	Buffer to store parsed word
<i>max_len</i>	Maximum length of word buffer

Returns

Length of parsed word or 0 if no word found

References [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [vm](#), and [vm_assert_interpreter_enabled\(\)](#).

Referenced by [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_word_forget\(\)](#), [forth_paren\(\)](#), [system_word_see\(\)](#), [vm_interpret\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.40 [vm_pin_execution_heat\(\)](#)

```
void vm_pin_execution_heat (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_PINNED](#).

Here is the call graph for this function:

7.54.4.41 [vm_pop\(\)](#)

```
cell_t vm_pop (
    VM * vm )
```

Pop a value from the data stack.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Returns

The popped value

Note

Returns 0 and sets error flag if stack underflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [PROFILE_INC_STACK_OP](#), [vm](#), and [vm_pop_asm\(\)](#).

Referenced by [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_thru\(\)](#), [control_forth_0branch\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [defining_word_constant\(\)](#), [defining_word_literal\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_sp_store\(\)](#), [double_word_2drop\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#),

editor_word_l(), editor_word_s(), format_word_d_dot(), format_word_d_dot_r(), format_word_dot(), format_word_dot_r(), format_word_dump(), format_word_end_conversion(), format_word_hash(), format_word_hash_s(), format_word_hold(), format_word_question(), format_word_sign(), format_word_u_dot(), format_word_u_dot_r(), forth_acl_allow_store(), forth_acl_mode_store(), forth_acl_ttl_store(), forth_PIPELINING_ANALYZE_WORD(), forth_PIPELINING_SHOW_STATS(), forth_PIPELINING_SHOW_TOP_TRANSITIONS(), io_word_type(), logical_word_and(), logical_word_equals(), logical_word_greater_than(), logical_word_less_than(), logical_word_not(), logical_word_not_equals(), logical_word_or(), logical_word_u_greater_than(), logical_word_u_less_than(), logical_word_within(), logical_word_xor(), logical_word_zero_equals(), logical_word_zero_greater(), logical_word_zero_less(), logical_word_zero_not_equal(), memory_word_2fetch(), memory_word_2store(), memory_word_cells(), memory_word_cfetch(), memory_word_cstore(), memory_word_erase(), memory_word_fetch(), memory_word_fill(), memory_word_minus_store(), memory_word_move(), memory_word_plus_store(), memory_word_store(), mixed_math_word_m_minus(), mixed_math_word_m_plus(), mixed_math_word_m_slash_mod(), mixed_math_word_m_star(), mixed_math_word_mod(), mixed_math_word_mod(), mixed_math_word_slash_mod(), mixed_math_word_star_slash(), mixed_math_word_star_slash_mod(), pop_xt(), return_stack_word_to_r(), runtime_abortq(), starforth_word_execution_heat_fetch(), starforth_word_execution_heat_store(), starforth_word_random(), starforth_word_seed(), starforth_word_top_words(), starforth_word_wait(), string_word_blank(), string_word_cmove(), string_word_cmove_greater(), string_word_compare(), string_word_convert(), string_word_count(), string_word_enclose(), string_word_expect(), string_word_minus_trailing(), string_word_number(), string_word_scan(), string_word_search(), string_word_skip(), string_word_word(), system_word_abort_quote(), test_acl_adaptive_ttl(), vm_api_pop(), word_divide(), and word_divmod().

Here is the call graph for this function:

7.54.4.42 vm_pop_fast()

```
static cell_t vm_pop_fast (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), and [vm](#).

7.54.4.43 vm_ptr()

```
uint8_t * vm_ptr (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [dump_block_preview\(\)](#), [editor_word_s\(\)](#), [execute_colon_word\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TO](#), [hash_colon_body\(\)](#), [memory_word_erase\(\)](#), [memory_word_fill\(\)](#), [memory_word_move\(\)](#), [string_word_convert\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_number\(\)](#), [string_word_tick\(\)](#), [string_word_word\(\)](#), [system_word_see\(\)](#), [vm_load_u8\(\)](#), [vm_store_u8\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.44 vm_push()

```
void vm_push (
    VM * vm,
    cell_t value )
```

Push a value onto the data stack.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>value</i>	Value to push onto the stack

Note

Performs bounds checking to prevent stack overflow

Uses assembly optimization if available for x86_64 or ARM64

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [PROFILE_INC_STACK_OP](#), [STACK_SIZE](#), [vm](#), and [vm_push_asm\(\)](#).

Referenced by [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_scr\(\)](#), [block_word_thru\(\)](#), [control_forth_I\(\)](#), [control_forth_J\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_lit\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_state\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_state\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [dictionary_word_pad\(\)](#), [dictionary_word_sp_fetch\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#), [emit_cell\(\)](#), [format_word_base\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_sign\(\)](#), [forth_acl_allow_fetch\(\)](#), [forth_acl_heat_fetch\(\)](#), [forth_acl_mode_fetch\(\)](#), [forth_acl_pinned_query\(\)](#), [forth_acl_ttl_fetch\(\)](#), [forth_acl_word_id\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_true\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetchn\(\)](#), [memory_word_fetch\(\)](#), [mixed_math_word_m_minus\(\)](#), [mixed_math_word_m_plus\(\)](#), [mixed_math_word_m_slash_mod\(\)](#), [mixed_math_word_m_star\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_slash_mod\(\)](#), [mixed_math_word_star_slash\(\)](#), [mixed_math_word_star_slash_mod\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_init\(\)](#), [starforth_word_random\(\)](#), [string_runtime_s_quote\(\)](#), [string_word_bl\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_tick\(\)](#), [string_word_to_in\(\)](#), [string_word_word\(\)](#), [system_word_79_standard\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [vm_api_push\(\)](#), [vm_compile_literal\(\)](#), [vm_interpret_word\(\)](#), [vocabulary_word_context\(\)](#), [vocabulary_word_current\(\)](#), [vocabulary_word_paren_find\(\)](#), [word_divide\(\)](#), and [word_divmod\(\)](#).

Here is the call graph for this function:

7.54.4.45 vm_push_fast()

```
static void vm_push_fast (
    VM * vm,
    cell_t value ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), and [vm](#).

7.54.4.46 vm_repl()

```
void vm_repl (
    VM * vm,
    int script_mode )
```

Start the [VM](#)'s read-eval-print loop.

Parameters

<i>vm</i>	VM instance to run
<i>script_mode</i>	If 1, suppress prompts and "ok" output (for piped input)

Start the [VM](#)'s read-eval-print loop.

Starts the REPL (Read-Eval-Print Loop) for the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>script_mode</i>	If 1, suppress prompts and "ok" output (for piped input)

This function implements an interactive REPL for the Forth interpreter. It continuously reads input from the user, interprets it, and prints the result until the [VM](#) is halted or an error occurs.

In interactive mode (`script_mode=0`):

- Prints colored prompt "ok> "
- Acknowledges successful commands with " ok"
- Reports errors with " ERROR"

In script mode (`script_mode=1`):

- Suppresses all prompts and status messages
- Suitable for piped input (heredoc, pipes, redirects)

7.54.4.47 `vm_rpop()`

```
cell_t vm_rpop (
    VM * vm )
```

Pop a value from the return stack.

Parameters

<code>vm</code>	Pointer to the VM structure
-----------------	---

Returns

The popped value

Note

Returns 0 and sets error flag if stack underflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_rpop_asm\(\)](#).

Referenced by [execute_colon_word\(\)](#), [return_stack_word_r_from\(\)](#), and [vm_api_rpop\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.48 `vm_rpop_fast()`

```
static cell_t vm_rpop_fast (
    VM * vm ) [inline], [static]
```

References [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.54.4.49 `vm_rpush()`

```
void vm_rpush (
    VM * vm,
    cell_t value )
```

Push a value onto the return stack.

Parameters

<code>vm</code>	Pointer to the VM structure
<code>value</code>	Value to push onto the return stack

Note

Sets error flag if stack overflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_rpush_asm\(\)](#).

Referenced by [execute_colon_word\(\)](#), [return_stack_word_to_r\(\)](#), and [vm_api_rpush\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.50 vm_rpush_fast()

```
static void vm_rpush_fast (
    VM * vm,
    cell_t value ) [inline], [static]
```

References [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.54.4.51 vm_run_smoke_tests()

```
void vm_run_smoke_tests (
    VM * vm )
```

7.54.4.52 vm_smudge_word()

```
void vm_smudge_word (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_SMUDGED](#).

Referenced by [vm_api_smudge_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.53 vm_snapshot_read()

```
void vm_snapshot_read (
    const VM * vm,
    HeartbeatSnapshot * out_snapshot )
```

References [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), and [vm](#).

Here is the call graph for this function:

7.54.4.54 vm_store_cell()

```
void vm_store_cell (
    struct VM * vm,
    vaddr_t addr,
    cell_t v )
```

References [VM::error](#), [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [defining_word_left_bracket\(\)](#), [defining_word_right_bracket\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_comma\(\)](#), [editor_word_edit\(\)](#), [format_word_decimal\(\)](#), [format_word_hex\(\)](#), [format_word_octal\(\)](#), [memory_word_2store\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [set_scr\(\)](#), [set_scr\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_set_base\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.55 vm_store_u8()

```
void vm_store_u8 (
    struct VM * vm,
    vaddr_t addr,
    uint8_t v )
```

References [VM::error](#), [vm](#), and [vm_ptr\(\)](#).

Referenced by [dictionary_word_c_comma\(\)](#), [memory_word_cstore\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), and [string_word_s_quote\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.56 vm_tick()

```
void vm_tick (
    VM * vm )
```

Central heartbeat dispatcher for time-driven tuning operations.

Aggregates all periodic optimization tasks (Loop #3 and Loop #5) into one place. Designed as plugin architecture - new tuning operations can be added as plugins.

Options for integration:

- Synchronous (now): Called from main execution loop, every N executions
- Background thread (future): Runs in separate thread, decoupled from [VM](#) execution

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_INFERENCE_FREQUENCY](#), [HeartbeatState::last_inference_tick](#), [HeartbeatState::tick_count](#), [vm](#), and [vm_tick_inference_engine\(\)](#).

Referenced by [assert_heartbeat_safe\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.54.4.57 vm_tick_inference_engine()

```
void vm_tick_inference_engine (
    VM * vm )
```

Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.

Coordinates inference on rolling window of truth to determine:

- Optimal adaptive window width (via variance inflection detection)
- Optimal decay slope (via exponential regression on heat trajectory)

Uses ANOVA early-exit to skip full inference when variance is stable (<5% change). All math uses Q48.16 fixed-point (integer-only, no floating-point).

Replaces legacy [vm_tick_window_tuner\(\)](#) and [vm_tick_slope_validator\(\)](#).

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [HeartbeatState::early_exit_count](#), [InferenceOutputs::early_exited](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_TICK_NS](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [inference_engine_run\(\)](#), [inference_outputs_validate\(\)](#), [HeartbeatState::inference_run_count](#), [RollingWindowOfTruth::is_warm](#), [VM::last_inference_outputs](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [InferenceOutputs::slope_fit_quality_q48](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [HeartbeatWorker::tick_ns](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_host_alloc\(\)](#), [InferenceInputs::window](#), [RollingWindowOfTruth::window_pos](#), [InferenceOutputs::window_variance_q48](#), [word_count](#), [VM::word_count_at_check](#), and [HeartbeatState::worker](#).

Here is the call graph for this function:

7.54.4.58 vm_tick_slope_validator()

```
void vm_tick_slope_validator (
    VM * vm )
```

Loop #3: Heat decay slope validation via periodic measurement.

Validates that linear decay is actually helping optimize dictionary caching. Measures stale word ratio, hot word count, and average heat distribution.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [VM::hot_word_count_at_check](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [VM::latest](#), [LOG_INFO](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [VM::stale_word_count_at_check](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [word_count](#), and [VM::word_count_at_check](#).

Here is the call graph for this function:

7.54.4.59 vm_tick_window_tuner()

```
void vm_tick_window_tuner (
    VM * vm )
```

Loop #5: Context-aware window tuning via binary chop search.

Uses prefetch accuracy to guide window size adaptation. Binary search converges on optimal effective_window_size for current workload.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [ADAPTIVE_MIN_WINDOW_SIZE](#), [RollingWindowOfTruth::effective_window_size](#), [ENABLE_PIPELINING](#), [RollingWindowOfTruth::is_warm](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [LOG_INFO](#), [log_message\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [vm](#), and [PipelineGlobalMetrics::window_tuning_checks](#).

Here is the call graph for this function:

7.54.4.60 vm_unpin_execution_heat()

```
void vm_unpin_execution_heat (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), and [vm](#).

Here is the call graph for this function:

7.54.5 Variable Documentation**7.54.5.1 __attribute__**

```
struct idtr __attribute__
```

7.54.5.2 acl_allow

```
uint8_t acl_allow
```

7.54.5.3 acl_default

```
uint8_t acl_default
```

7.54.5.4 acl_mode

```
uint8_t acl_mode
```

7.54.5.5 acl_pinned

```
uint8_t acl_pinned
```

7.54.5.6 acl_ttl

```
uint32_t acl_ttl
```

7.54.5.7 execution_heat

```
cell_t execution_heat
```

Referenced by [forth_ALL_HEATS\(\)](#), [hotwords_bucket_reorder\(\)](#), [physics_metadata_touch\(\)](#), [starforth_word_top_words\(\)](#), and [temperature_from_execution_heat\(\)](#).

7.54.5.8 flags

```
cell_t flags
```

Referenced by [capsule_find_mama_init\(\)](#), [derive_state_flags\(\)](#), [make_pte\(\)](#), [vmm_map_page\(\)](#), and [vmm_map_range\(\)](#).

7.54.5.9 func

```
word_func_t func
```

Referenced by [register_word\(\)](#), [vm_compile_call\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_find_by_func\(\)](#), and [vm_dictionary_find_latest_by_func\(\)](#).

7.54.5.10 link

```
struct DictEntry* link
```

7.54.5.11 name

```
char name[]
```

Referenced by [capsule_find_by_name\(\)](#), [dict_find_word_heat_aware\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [hotwords_cache_lookup\(\)](#), [register_word\(\)](#), [sig_handler\(\)](#), [string_word_tick\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_find_word\(\)](#), [vm_interpret_word\(\)](#), [vm_vocabulary_find_word\(\)](#), [vocab_find_word\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), [vocabulary_word_paren_find\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

7.54.5.12 name_len

```
uint8_t name_len
```

Referenced by [profiler_generate_report\(\)](#), [profiler_print_hotspots\(\)](#), [string_word_tick\(\)](#), and [traverse_name_field\(\)](#).

7.54.5.13 physics

```
DictPhysics physics
```

7.54.5.14 transition_metrics

```
WordTransitionMetrics* transition_metrics
```

7.54.5.15 word_id

```
uint32_t word_id
```

Referenced by [execute_colon_word\(\)](#), [extract_heat_trajectory\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [physics_pre_execute\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_record_execution\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [transition_metrics_update_context_window\(\)](#), [vm_dictionary_lookup_by_word_id\(\)](#), [vm_dictionary_track_entry\(\)](#), and [vm_dictionary_untrack_entry\(\)](#).

7.55 include/vm_api.h File Reference

StarForth Virtual Machine Public API.

```
#include "vm.h"
```

Include dependency graph for vm_api.h: This graph shows which files directly or indirectly include this file:

Functions

- [int vm_input_ensure](#) ([VM *vm](#))
Ensures input buffer initialization.
- [unsigned char * vm_input_tib](#) ([VM *vm](#))
Retrieves Terminal Input Buffer address.
- [cell_t * vm_input_in](#) ([VM *vm](#))
Retrieves input pointer address.
- [cell_t * vm_input_span](#) ([VM *vm](#))
Retrieves span cell address.
- [void vm_input_load_line](#) ([VM *vm](#), [const char *src](#), [size_t n](#))
Loads a line into TIB.
- [void vm_input_source](#) ([VM *vm](#), [cell_t *out_addr](#), [cell_t *out_len](#))
Gets current input source.
- [cell_t vm_addr_from_ptr](#) ([VM *vm](#), [void *p](#))
Converts host pointer to Forth address.

7.55.1 Detailed Description

StarForth Virtual Machine Public API.

This header provides the public interface for interacting with the StarForth Virtual Machine, particularly focusing on input handling and memory management.

7.56 include/vm_asm_opt.h File Reference

```
#include "vm.h"
```

```
#include <stdint.h>
```

Include dependency graph for vm_asm_opt.h: This graph shows which files directly or indirectly include this file:

Functions

- [static void vm_push_asm](#) ([VM *vm](#), [cell_t value](#))
Optimized data stack push operation.
- [static cell_t vm_pop_asm](#) ([VM *vm](#))
Optimized data stack pop operation.
- [static void vm_rpush_asm](#) ([VM *vm](#), [cell_t value](#))
Optimized return stack push operation.
- [static cell_t vm_rpop_asm](#) ([VM *vm](#))
Optimized return stack pop operation.
- [static int vm_add_check_overflow](#) ([cell_t a](#), [cell_t b](#), [cell_t *result](#))

- Performs addition with hardware overflow detection.*
- static int `vm_sub_check_overflow` (`cell_t` a, `cell_t` b, `cell_t` *result)
- Performs subtraction with hardware overflow detection.*
- static void `vm_mul_double` (`cell_t` a, `cell_t` b, `cell_t` *hi, `cell_t` *lo)
- Performs double-width multiplication.*
- static void `vm_divmod` (`cell_t` dividend, `cell_t` divisor, `cell_t` *quotient, `cell_t` *remainder)
- Performs division with remainder.*
- static int `vm_strcmp_asm` (const char *s1, const char *s2, size_t len)
- Hardware-accelerated string comparison.*
- static void `vm_memcpy_asm` (void *dest, const void *src, size_t len)
- Hardware-accelerated memory copy.*
- static void `vm_memzero_asm` (void *dest, size_t len)
- Hardware-accelerated memory zero.*
- static `cell_t` `vm_min_asm` (`cell_t` a, `cell_t` b)
- Branchless minimum calculation.*
- static `cell_t` `vm_max_asm` (`cell_t` a, `cell_t` b)
- Branchless maximum calculation.*
- static void `vm_cpuid` (uint32_t leaf, uint32_t *eax, uint32_t *ebx, uint32_t *ecx, uint32_t *edx)
- Wrapper for CPUID instruction.*
- static int `vm_has_sse42` (void)
- Checks for SSE4.2 support.*

7.57 include/vm_asm_opt_arm64.h File Reference

```
#include "vm.h"
#include <stdint.h>
Include dependency graph for vm_asm_opt_arm64.h:
```

Functions

- static void `vm_push_asm` (`VM` *vm, `cell_t` value)
- Optimized assembly implementation of data stack push operation.*
- static `cell_t` `vm_pop_asm` (`VM` *vm)
- Optimized assembly implementation of data stack pop operation.*
- static void `vm_rpush_asm` (`VM` *vm, `cell_t` value)
- Optimized assembly implementation of return stack push operation.*
- static `cell_t` `vm_rpop_asm` (`VM` *vm)
- Optimized assembly implementation of return stack pop operation.*
- static int `vm_add_check_overflow` (`cell_t` a, `cell_t` b, `cell_t` *result)
- Fast addition with overflow detection using ARM64 ADDS instruction.*
- static int `vm_sub_check_overflow` (`cell_t` a, `cell_t` b, `cell_t` *result)
- Fast subtraction with overflow detection using ARM64 SUBS instruction.*
- static void `vm_mul_double` (`cell_t` a, `cell_t` b, `cell_t` *hi, `cell_t` *lo)
- Fast 128-bit multiplication using ARM64 SMULH instruction.*
- static void `vm_divmod` (`cell_t` dividend, `cell_t` divisor, `cell_t` *quotient, `cell_t` *remainder)
- Fast division with remainder using ARM64 SDIV and MSUB instructions.*
- static int `vm_strcmp_short_asm` (const char *s1, const char *s2, size_t len)
- Optimized string comparison for short strings (<=32 bytes)*
- static int `vm_strcmp_asm` (const char *s1, const char *s2, size_t len)
- static void `vm_memcpy_asm` (void *dest, const void *src, size_t len)
- Optimized memory copy using ARM64 load/store pair instructions.*

- static void `vm_memzero_asm` (void *dest, size_t len)
Fast memory zeroing using ARM64 store pair instructions.
- static `cell_t` `vm_min_asm` (`cell_t` a, `cell_t` b)
- static `cell_t` `vm_max_asm` (`cell_t` a, `cell_t` b)
- static `cell_t` `vm_abs_asm` (`cell_t` a)
- static int `vm_clz` (`cell_t` x)
- static int `vm_ctz` (`cell_t` x)
- static int `vm_popcnt` (`cell_t` x)
- static uint64_t `vm_read_midr` (void)
- static int `vm_has_neon` (void)
- static int `vm_is_cortex_a72` (void)
- static void `vm_prefetch` (const void *addr)
- static void `vm_prefetch_stream` (const void *addr)
- static void `vm_dc_zva` (void *addr)

7.57.1 Function Documentation

7.57.1.1 `vm_abs_asm()`

```
static cell_t vm_abs_asm (
    cell_t a ) [inline], [static]
```

7.57.1.2 `vm_add_check_overflow()`

```
static int vm_add_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

Fast addition with overflow detection using ARM64 ADDS instruction.

Parameters

<i>a</i>	First operand
<i>b</i>	Second operand
<i>result</i>	Pointer to store result

Returns

1 if overflow occurred, 0 otherwise

7.57.1.3 `vm_clz()`

```
static int vm_clz (
    cell_t x ) [inline], [static]
```

7.57.1.4 `vm_ctz()`

```
static int vm_ctz (
    cell_t x ) [inline], [static]
```

7.57.1.5 `vm_dc_zva()`

```
static void vm_dc_zva (
    void * addr ) [inline], [static]
```

7.57.1.6 vm_divmod()

```
static void vm_divmod (
    cell_t dividend,
    cell_t divisor,
    cell_t * quotient,
    cell_t * remainder ) [inline], [static]
```

Fast division with remainder using ARM64 SDIV and MSUB instructions.

Parameters

<i>dividend</i>	Dividend value
<i>divisor</i>	Divisor value
<i>quotient</i>	Pointer to store quotient
<i>remainder</i>	Pointer to store remainder

7.57.1.7 vm_has_neon()

```
static int vm_has_neon (
    void ) [inline], [static]
```

7.57.1.8 vm_is_cortex_a72()

```
static int vm_is_cortex_a72 (
    void ) [inline], [static]
```

7.57.1.9 vm_max_asm()

```
static cell_t vm_max_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

7.57.1.10 vm_memcpy_asm()

```
static void vm_memcpy_asm (
    void * dest,
    const void * src,
    size_t len ) [inline], [static]
```

Optimized memory copy using ARM64 load/store pair instructions.

Parameters

<i>dest</i>	Destination buffer
<i>src</i>	Source buffer
<i>len</i>	Number of bytes to copy

7.57.1.11 vm_memzero_asm()

```
static void vm_memzero_asm (
    void * dest,
    size_t len ) [inline], [static]
```

Fast memory zeroing using ARM64 store pair instructions.

Parameters

<i>dest</i>	Buffer to zero
<i>len</i>	Number of bytes to zero

7.57.1.12 vm_min_asm()

```
static cell_t vm_min_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

7.57.1.13 vm_mul_double()

```
static void vm_mul_double (
    cell_t a,
    cell_t b,
    cell_t * hi,
    cell_t * lo ) [inline], [static]
```

Fast 128-bit multiplication using ARM64 SMULH instruction.

Parameters

<i>a</i>	First operand
<i>b</i>	Second operand
<i>hi</i>	Pointer to store high 64 bits of result
<i>lo</i>	Pointer to store low 64 bits of result

7.57.1.14 vm_pop_asm()

```
static cell_t vm_pop_asm (
    VM * vm ) [inline], [static]
```

Optimized assembly implementation of data stack pop operation.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Popped value, or 0 on underflow

Note

Sets `vm->error` on stack underflow

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

7.57.1.15 vm_popcnt()

```
static int vm_popcnt (
    cell_t x ) [inline], [static]
```

7.57.1.16 vm_prefetch()

```
static void vm_prefetch (
    const void * addr ) [inline], [static]
```

7.57.1.17 vm_prefetch_stream()

```
static void vm_prefetch_stream (
    const void * addr ) [inline], [static]
```

7.57.1.18 vm_push_asm()

```
static void vm_push_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

Optimized assembly implementation of data stack push operation.

Parameters

<i>vm</i>	Pointer to VM instance
<i>value</i>	Value to push onto data stack

Note

Sets `vm->error` on stack overflow

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

7.57.1.19 vm_read_midr()

```
static uint64_t vm_read_midr (
    void ) [inline], [static]
```

7.57.1.20 vm_rpop_asm()

```
static cell_t vm_rpop_asm (
    VM * vm ) [inline], [static]
```

Optimized assembly implementation of return stack pop operation.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Popped value, or 0 on underflow

Note

Sets `vm->error` on stack underflow

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.57.1.21 vm_rpush_asm()

```
static void vm_rpush_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

Optimized assembly implementation of return stack push operation.

Parameters

<i>vm</i>	Pointer to VM instance
<i>value</i>	Value to push onto return stack

Note

Sets `vm->error` on stack overflow

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.57.1.22 vm_strcmp_asm()

```
static int vm_strcmp_asm (
    const char * s1,
    const char * s2,
    size_t len ) [inline], [static]
```

References [vm_strcmp_short_asm\(\)](#).

Here is the call graph for this function:

7.57.1.23 vm_strcmp_short_asm()

```
static int vm_strcmp_short_asm (
    const char * s1,
    const char * s2,
    size_t len ) [inline], [static]
```

Optimized string comparison for short strings (<=32 bytes)

Parameters

<i>s1</i>	First string
<i>s2</i>	Second string
<i>len</i>	Length to compare

Returns

0 if equal, 1 if different

Referenced by [vm_strcmp_asm\(\)](#).

Here is the caller graph for this function:

7.57.1.24 vm_sub_check_overflow()

```
static int vm_sub_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

Fast subtraction with overflow detection using ARM64 SUBS instruction.

Parameters

<i>a</i>	First operand
<i>b</i>	Second operand
<i>result</i>	Pointer to store result

Returns

1 if overflow occurred, 0 otherwise

7.58 include/vm_asm_opt_riscv64.h File Reference

```
#include "vm.h"
#include <stdint.h>
```

Include dependency graph for vm_asm_opt_riscv64.h:

Functions

- static void [vm_push_asm](#) (VM *vm, cell_t value)

- static `cell_t vm_pop_asm (VM *vm)`
- static void `vm_rpush_asm (VM *vm, cell_t value)`
- static `cell_t vm_rpop_asm (VM *vm)`
- static int `vm_add_check_overflow (cell_t a, cell_t b, cell_t *result)`
- static int `vm_sub_check_overflow (cell_t a, cell_t b, cell_t *result)`
- static void `vm_mul_double (cell_t a, cell_t b, cell_t *hi, cell_t *lo)`
- static void `vm_divmod (cell_t dividend, cell_t divisor, cell_t *quotient, cell_t *remainder)`
- static int `vm_strcmp_asm (const char *s1, const char *s2, size_t len)`
- static void `vm_memcpy_asm (void *dest, const void *src, size_t len)`
- static void `vm_memzero_asm (void *dest, size_t len)`
- static `cell_t vm_min_asm (cell_t a, cell_t b)`
- static `cell_t vm_max_asm (cell_t a, cell_t b)`
- static `cell_t vm_abs_asm (cell_t a)`
- static int `vm_clz (cell_t x)`
- static int `vm_ctz (cell_t x)`
- static int `vm_popcnt (cell_t x)`
- static void `vm_prefetch (const void *addr)`
- static void `vm_prefetch_stream (const void *addr)`
- static int `vm_has_riscv_m_ext (void)`
- static int `vm_has_riscv_b_ext (void)`

7.58.1 Function Documentation

7.58.1.1 vm_abs_asm()

```
static cell_t vm_abs_asm (
    cell_t a ) [inline], [static]
```

7.58.1.2 vm_add_check_overflow()

```
static int vm_add_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

7.58.1.3 vm_clz()

```
static int vm_clz (
    cell_t x ) [inline], [static]
```

7.58.1.4 vm_ctz()

```
static int vm_ctz (
    cell_t x ) [inline], [static]
```

7.58.1.5 vm_divmod()

```
static void vm_divmod (
    cell_t dividend,
    cell_t divisor,
    cell_t * quotient,
    cell_t * remainder ) [inline], [static]
```

7.58.1.6 vm_has_riscv_b_ext()

```
static int vm_has_riscv_b_ext (
    void ) [inline], [static]
```

7.58.1.7 vm_has_riscv_m_ext()

```
static int vm_has_riscv_m_ext (
    void ) [inline], [static]
```

7.58.1.8 vm_max_asm()

```
static cell_t vm_max_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

7.58.1.9 vm_memcpy_asm()

```
static void vm_memcpy_asm (
    void * dest,
    const void * src,
    size_t len ) [inline], [static]
```

7.58.1.10 vm_memzero_asm()

```
static void vm_memzero_asm (
    void * dest,
    size_t len ) [inline], [static]
```

7.58.1.11 vm_min_asm()

```
static cell_t vm_min_asm (
    cell_t a,
    cell_t b ) [inline], [static]
```

7.58.1.12 vm_mul_double()

```
static void vm_mul_double (
    cell_t a,
    cell_t b,
    cell_t * hi,
    cell_t * lo ) [inline], [static]
```

7.58.1.13 vm_pop_asm()

```
static cell_t vm_pop_asm (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

7.58.1.14 vm_popcnt()

```
static int vm_popcnt (
    cell_t x ) [inline], [static]
```

7.58.1.15 vm_prefetch()

```
static void vm_prefetch (
    const void * addr ) [inline], [static]
```

7.58.1.16 vm_prefetch_stream()

```
static void vm_prefetch_stream (
    const void * addr ) [inline], [static]
```

7.58.1.17 vm_push_asm()

```
static void vm_push_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

7.58.1.18 vm_rpop_asm()

```
static cell_t vm_rpop_asm (
    VM * vm ) [inline], [static]
```

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.58.1.19 vm_rpush_asm()

```
static void vm_rpush_asm (
    VM * vm,
    cell_t value ) [inline], [static]
```

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.58.1.20 vm_strcmp_asm()

```
static int vm_strcmp_asm (
    const char * s1,
    const char * s2,
    size_t len ) [inline], [static]
```

7.58.1.21 vm_sub_check_overflow()

```
static int vm_sub_check_overflow (
    cell_t a,
    cell_t b,
    cell_t * result ) [inline], [static]
```

7.59 include/vm_debug.h File Reference

```
#include "vm.h"
```

Include dependency graph for `vm_debug.h`: This graph shows which files directly or indirectly include this file:

Functions

- void [vm_debug_set_current_vm](#) (VM *vm)
Sets the VM instance to dump state for when signals occur.
- void [vm_debug_install_signal_handlers](#) (void)
Installs signal handlers for SIGSEGV and SIGABRT.
- void [vm_debug_dump_state](#) (const VM *vm, const char *reason)
Manually dumps the state of a VM instance.

7.59.1 Function Documentation

7.59.1.1 vm_debug_dump_state()

```
void vm_debug_dump_state (
    const VM * vm,
    const char * reason )
```

Manually dumps the state of a VM instance.

Parameters

<i>vm</i>	The VM instance to dump state for
<i>reason</i>	Description of why the dump was triggered

Note

Typically called before `exit(1)` on test failure

Manually dumps the state of a [VM](#) instance.

Parameters

<i>vm</i>	Pointer to the VM instance to dump
<i>reason</i>	Optional string describing the reason for the dump

References [base](#), [VM::current_executing_entry](#), [dbg_get_base\(\)](#), [VM::dsp](#), [dump_block_preview\(\)](#), [VM::error](#), [VM::here](#), [VM::ip](#), [VM::memory](#), [VM::mode](#), [mode_str\(\)](#), [DictEntry::name](#), [VM::rsp](#), [VM::scr_addr](#), [vm](#), and [VM_MEMORY_SIZE](#).

Referenced by [dump_and_die\(\)](#), and [sig_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.59.1.2 vm_debug_install_signal_handlers()

```
void vm_debug_install_signal_handlers (
    void )
```

Installs signal handlers for SIGSEGV and SIGABRT.

The handlers will print [VM](#) state before re-raising the signal

Installs signal handlers for SIGSEGV and SIGABRT.

Sets up handlers for SIGSEGV and SIGABRT to dump [VM](#) state on crash

References [sig_handler\(\)](#).

Referenced by [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.59.1.3 vm_debug_set_current_vm()

```
void vm_debug_set_current_vm (
    VM * vm )
```

Sets the [VM](#) instance to dump state for when signals occur.

Parameters

<i>vm</i>	Pointer to the VM instance to monitor
-----------	---

Note

Should be called once after `vm_init`

Sets the [VM](#) instance to dump state for when signals occur.

Parameters

<i>vm</i>	Pointer to the VM instance to be monitored
-----------	--

References [g_vm](#), and [vm](#).

Referenced by [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.60 include/vm_host.h File Reference

```
#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>
```

Include dependency graph for vm_host.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [VMHostServices](#)
VMHostServices - pluggable host operations.

Macros

- #define [SK_ALLOC](#)(size) ([sk_host_services](#)()->alloc((size), sizeof(void*)))
Initialize host services for kernel build.
- #define [SK_ALLOC_ALIGNED](#)(s, a) ([sk_host_services](#)()->alloc((s), (a)))
- #define [SK_FREE](#)(ptr) ([sk_host_services](#)()->free(ptr))
- #define [SK_TIME_NS](#)() ([sk_host_services](#)()->monotonic_ns())
- #define [SK_PUTS](#)(str) ([sk_host_services](#)()->puts(str))
- #define [SK_PUTC](#)(c) ([sk_host_services](#)()->putc(c))
- #define [SK_MUTEX_INIT](#)(m) ([sk_host_services](#)()->mutex_init(m))
Mutex wrapper macros.
- #define [SK_MUTEX_LOCK](#)(m) ([sk_host_services](#)()->mutex_lock(m))
- #define [SK_MUTEX_UNLOCK](#)(m) ([sk_host_services](#)()->mutex_unlock(m))
- #define [SK_MUTEX_DESTROY](#)(m) ([sk_host_services](#)()->mutex_destroy(m))

Typedefs

- typedef void *(* [sk_alloc_fn](#)) (size_t size, size_t align)
Host allocator function signature Returns NULL on failure.
- typedef void(* [sk_free_fn](#)) (void *ptr)
Host free function signature.
- typedef uint64_t(* [sk_time_fn](#)) (void)
Host monotonic time function signature Returns nanoseconds since arbitrary epoch In PARITY_MODE, returns deterministic fake time.
- typedef int(* [sk_mutex_init_fn](#)) (void *mutex)
Host mutex operations (no-op in kernel single-threaded mode)
- typedef int(* [sk_mutex_lock_fn](#)) (void *mutex)
- typedef int(* [sk_mutex_unlock_fn](#)) (void *mutex)
- typedef void(* [sk_mutex_destroy_fn](#)) (void *mutex)
- typedef int(* [sk_puts_fn](#)) (const char *str)
Host console output (for words that print) Returns number of bytes written, or -1 on error.
- typedef int(* [sk_putc_fn](#)) (int c)
- typedef bool(* [sk_xt_validate_fn](#)) (const void *ptr)
Host execution-token validation Returns 1 if pointer is executable, 0 otherwise.
- typedef bool(* [sk_xt_entry_owns_fn](#)) (const void *entry, size_t bytes)
Host dictionary entry ownership check.
- typedef void(* [sk_panic_fn](#)) (const char *message)
Host fatal handler (panic)
- typedef struct [VMHostServices](#) [VMHostServices](#)
VMHostServices - pluggable host operations.

Functions

- const [VMHostServices](#) * [sk_host_services](#) (void)

Get the current host services instance.

7.60.1 Macro Definition Documentation

7.60.1.1 SK_ALLOC

```
#define SK_ALLOC(  
    size ) (sk_host_services()->alloc((size), sizeof(void*)))
```

Initialize host services for kernel build.

Must be called before any [VM](#) operations. Convenience macros for common operations Use these instead of direct malloc/free/etc.

7.60.1.2 SK_ALLOC_ALIGNED

```
#define SK_ALLOC_ALIGNED(  
    s,  
    a ) (sk_host_services()->alloc((s), (a)))
```

7.60.1.3 SK_FREE

```
#define SK_FREE(  
    ptr ) (sk_host_services()->free(ptr))
```

7.60.1.4 SK_MUTEX_DESTROY

```
#define SK_MUTEX_DESTROY(  
    m ) (sk_host_services()->mutex_destroy(m))
```

7.60.1.5 SK_MUTEX_INIT

```
#define SK_MUTEX_INIT(  
    m ) (sk_host_services()->mutex_init(m))
```

Mutex wrapper macros.

7.60.1.6 SK_MUTEX_LOCK

```
#define SK_MUTEX_LOCK(  
    m ) (sk_host_services()->mutex_lock(m))
```

7.60.1.7 SK_MUTEX_UNLOCK

```
#define SK_MUTEX_UNLOCK(  
    m ) (sk_host_services()->mutex_unlock(m))
```

7.60.1.8 SK_PUTC

```
#define SK_PUTC(  
    c ) (sk_host_services()->putc(c))
```

7.60.1.9 SK_PUTS

```
#define SK_PUTS(  
    str ) (sk_host_services()->puts(str))
```

7.60.1.10 SK_TIME_NS

```
#define SK_TIME_NS( ) (sk_host_services()->monotonic_ns())
```

7.60.2 Typedef Documentation

7.60.2.1 sk_alloc_fn

```
typedef void>(* sk_alloc_fn) (size_t size, size_t align)
```

Host allocator function signature Returns NULL on failure.

7.60.2.2 sk_free_fn

```
typedef void(* sk_free_fn) (void *ptr)
```

Host free function signature.

7.60.2.3 sk_mutex_destroy_fn

```
typedef void(* sk_mutex_destroy_fn) (void *mutex)
```

7.60.2.4 sk_mutex_init_fn

```
typedef int(* sk_mutex_init_fn) (void *mutex)
```

Host mutex operations (no-op in kernel single-threaded mode)

7.60.2.5 sk_mutex_lock_fn

```
typedef int(* sk_mutex_lock_fn) (void *mutex)
```

7.60.2.6 sk_mutex_unlock_fn

```
typedef int(* sk_mutex_unlock_fn) (void *mutex)
```

7.60.2.7 sk_panic_fn

```
typedef void(* sk_panic_fn) (const char *message)
```

Host fatal handler (panic)

7.60.2.8 sk_putc_fn

```
typedef int(* sk_putc_fn) (int c)
```

7.60.2.9 sk_puts_fn

```
typedef int(* sk_puts_fn) (const char *str)
```

Host console output (for words that print) Returns number of bytes written, or -1 on error.

7.60.2.10 sk_time_fn

```
typedef uint64_t(* sk_time_fn) (void)
```

Host monotonic time function signature Returns nanoseconds since arbitrary epoch In PARITY_MODE, returns deterministic fake time.

7.60.2.11 sk_xt_entry_owns_fn

```
typedef bool(* sk_xt_entry_owns_fn) (const void *entry, size_t bytes)
```

Host dictionary entry ownership check.

7.60.2.12 sk_xt_validate_fn

```
typedef bool(* sk_xt_validate_fn) (const void *ptr)
```

Host execution-token validation Returns 1 if pointer is executable, 0 otherwise.

7.60.2.13 VMHostServices

typedef struct VMHostServices VMHostServices

VMHostServices - pluggable host operations.

Hosted builds point these at libc functions. Kernel builds point these at kernel implementations.

7.60.3 Function Documentation

7.60.3.1 sk_host_services()

```
const VMHostServices * sk_host_services (
    void )
```

Get the current host services instance.

Returns hosted or kernel services depending on build.

Referenced by [vm_default_host_services\(\)](#).

Here is the caller graph for this function:

7.61 include/vm_inner_interp_arm64.h File Reference

```
#include "vm.h"
```

Include dependency graph for vm_inner_interp_arm64.h:

Macros

- #define NEXT_ARM64()
- #define PRIM_DUP_ARM64()
 - DUP primitive - duplicates top of stack item.*
- #define PRIM_DROP_ARM64()
- #define PRIM_SWAP_ARM64()
- #define PRIM_OVER_ARM64()
- #define PRIM_ROT_ARM64()
- #define PRIM_PLUS_ARM64()
- #define PRIM_MINUS_ARM64()
- #define PRIM_STAR_ARM64()
- #define PRIM_SLASH_ARM64()
- #define PRIM_MOD_ARM64()
- #define PRIM_AND_ARM64()
- #define PRIM_OR_ARM64()
- #define PRIM_XOR_ARM64()
- #define PRIM_INVERT_ARM64()
- #define PRIM_NEGATE_ARM64()
- #define PRIM_FETCH_ARM64()
- #define PRIM_STORE_ARM64()
- #define PRIM_C_FETCH_ARM64()
- #define PRIM_C_STORE_ARM64()
- #define PRIM_TO_R_ARM64()
- #define PRIM_R_FROM_ARM64()
- #define PRIM_R_FETCH_ARM64()
- #define PRIM_2DUP_ARM64()
- #define PRIM_2DROP_ARM64()
- #define PRIM_ZERO_EQUALS_ARM64()
- #define PRIM_ZERO_LESS_ARM64()
- #define PRIM_EQUALS_ARM64()
- #define PRIM_LESS_ARM64()
- #define PRIM_GREATER_ARM64()
- #define PRIM_BRANCH_ARM64()
- #define PRIM_ZBRANCH_ARM64()
- #define PRIM_EXECUTE_ARM64()

Functions

- static void [vm_docol_arm64](#) (VM *vm)
- static void [vm_exit_arm64](#) (void)
- static void [vm_setup_registers_arm64](#) (VM *vm)
- static void [vm_save_registers_arm64](#) (VM *vm)

7.61.1 Macro Definition Documentation

7.61.1.1 NEXT_ARM64

```
#define NEXT_ARM64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ldr    x0, [x20], #8\n\t"      /* Load word ptr from [IP], IP += 8 */ \
    "br     x0\n\t"                /* Branch to word */ \
    ::: "x0", "memory" \
)
```

7.61.2 Function Documentation

7.61.2.1 vm_docol_arm64()

```
static void vm_docol_arm64 (
    VM * vm ) [inline], [static]
```

References [VM::current_executing_entry](#), [entry](#), and [vm](#).

7.61.2.2 vm_exit_arm64()

```
static void vm_exit_arm64 (
    void ) [inline], [static]
```

7.61.2.3 vm_save_registers_arm64()

```
static void vm_save_registers_arm64 (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.61.2.4 vm_setup_registers_arm64()

```
static void vm_setup_registers_arm64 (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.62 include/vm_inner_interp_asm.h File Reference

```
#include "vm.h"
```

Include dependency graph for vm_inner_interp_asm.h: This graph shows which files directly or indirectly include this file:

Macros

- #define [NEXT_ASM](#)()
Register allocation strategy for optimized execution.
- #define [PRIM_DUP](#)()
- #define [PRIM_DROP](#)()
- #define [PRIM_SWAP](#)()
- #define [PRIM_PLUS](#)()
- #define [PRIM_MINUS](#)()
- #define [PRIM_STAR](#)()

- #define `PRIM_FETCH()`
- #define `PRIM_STORE()`
- #define `PRIM_TO_R()`
- #define `PRIM_R_FROM()`
- #define `PRIM_R_FETCH()`
- #define `PRIM_BRANCH()`
- #define `PRIM_ZBRANCH()`
- #define `PRIM_OVER()`
- #define `PRIM_ROT()`
- #define `PRIM_SLASH()`
- #define `PRIM_MOD()`
- #define `PRIM_AND()`
- #define `PRIM_OR()`
- #define `PRIM_XOR()`
- #define `PRIM_INVERT()`
- #define `PRIM_NEGATE()`
- #define `PRIM_C_FETCH()`
- #define `PRIM_C_STORE()`
- #define `PRIM_2DUP()`
- #define `PRIM_2DROP()`
- #define `PRIM_ZERO_EQUALS()`
- #define `PRIM_ZERO_LESS()`
- #define `PRIM_EQUALS()`
- #define `PRIM_LESS()`
- #define `PRIM_GREATER()`
- #define `PRIM_EXECUTE()`

Functions

- static void `vm_docol_asm (VM *vm)`
Enter a colon definition (DOCOL)
- static void `vm_exit_asm (void)`
Return from a colon definition (EXIT)
- static void `vm_setup_registers (VM *vm)`
Main interpreter loop implementation.
- static void `vm_save_registers (VM *vm)`

7.62.1 Macro Definition Documentation

7.62.1.1 NEXT_ASM

```
#define NEXT_ASM( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%%r13), %%rax\n\t"      /* Load word pointer from [IP] */ \
    "addq    $8, %%r13\n\t"          /* IP++ */ \
    "jmpq    *%%rax\n\t"              /* Jump to word's code */ \
    ::: "rax", "memory" \
)
```

Register allocation strategy for optimized execution.

Register usage:

- r12 `VM` pointer (callee-saved, preserved across calls)
- r13 Instruction Pointer (IP) - points to current cell
- r14 Data Stack Pointer (DSP) - points to `vm->data_stack[vm->dsp]`
- r15 Return Stack Pointer (RSP) - points to `vm->return_stack[vm->rsp]`

Callee-saved registers are used because:

- They're preserved across function calls
- Allows mixing C and assembly code
- No need to save/restore on each word execution

Core inner interpreter macro - executes next word

Fetches next word pointer from IP, increments IP, and jumps to word

7.62.1.2 PRIM_2DROP

```
#define PRIM_2DROP( )
```

Value:

```
__asm__ __volatile__ ( \
    "subq    $16, %%r14\n\t"      /* DSP -= 2 */ \
    ::: "r14" \
)
```

7.62.1.3 PRIM_2DUP

```
#define PRIM_2DUP( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    -8(%%r14), %%rax\n\t" /* Load NOS (a) */ \
    "movq    (%%r14), %%rcx\n\t"  /* Load TOS (b) */ \
    "movq    %%rax, 8(%%r14)\n\t" /* Store a above TOS */ \
    "movq    %%rcx, 16(%%r14)\n\t" /* Store b above that */ \
    "addq    $16, %%r14\n\t"      /* DSP += 2 */ \
    ::: "rax", "rcx", "r14", "memory" \
)
```

7.62.1.4 PRIM_AND

```
#define PRIM_AND( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%%r14), %%rax\n\t" /* Load TOS */ \
    "andq    %%rax, -8(%%r14)\n\t" /* NOS &= TOS */ \
    "subq    $8, %%r14\n\t"      /* DSP-- */ \
    ::: "rax", "r14", "cc", "memory" \
)
```

7.62.1.5 PRIM_BRANCH

```
#define PRIM_BRANCH( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%%r13), %%rax\n\t" /* Load offset from IP */ \
    "addq    %%rax, %%r13\n\t"   /* IP += offset */ \
    ::: "rax", "r13", "memory" \
)
```

7.62.1.6 PRIM_C_FETCH

```
#define PRIM_C_FETCH( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%%r14), %%rax\n\t" /* Load address from TOS */ \
    "movq    (%%r12), %%rcx\n\t" /* Load VM->memory */ \
    "movzbl    (%%rcx, %%rax, 1), %%rax\n\t" /* Load byte, zero-extend */ \
    "movq    %%rax, (%%r14)\n\t" /* Store byte value to TOS */ \
    ::: "rax", "rcx", "memory" \
)
```

7.62.1.7 PRIM_C_STORE

```
#define PRIM_C_STORE( )
```

Value:

```
__asm__ __volatile__ ( \
```

```

    "movq    (%r14), %%rax\n\t"      /* Load address from TOS */ \
    "movq    -8(%%r14), %%rcx\n\t"  /* Load byte value from NOS */ \
    "movq    (%%r12), %%rdx\n\t"    /* Load VM->memory */ \
    "movb     %%cl, (%%rdx, %%rax, 1)\n\t" /* Store byte */ \
    "subq     $16, %%r14\n\t"        /* DSP -= 2 */ \
    ::: "rax", "rcx", "rdx", "r14", "memory" \
)

```

7.62.1.8 PRIM_DROP

```
#define PRIM_DROP( )
```

Value:

```

__asm__ __volatile__ ( \
    "subq     $8, %%r14\n\t"        /* DSP-- */ \
    ::: "r14" \
)

```

7.62.1.9 PRIM_DUP

```
#define PRIM_DUP( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "addq     $8, %%r14\n\t"        /* DSP++ */ \
    "movq     %%rax, (%%r14)\n\t"    /* Store duplicate */ \
    ::: "rax", "r14", "memory" \
)

```

7.62.1.10 PRIM_EQUALS

```
#define PRIM_EQUALS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS (b) */ \
    "cmpq     %%rax, -8(%%r14)\n\t"  /* Compare NOS (a) with TOS (b) */ \
    "sete     %%al\n\t"              /* AL = 1 if equal */ \
    "movzbq   %%al, %%rax\n\t"        /* Zero-extend */ \
    "negq     %%rax\n\t"              /* -1 if true, 0 if false */ \
    "movq     %%rax, -8(%%r14)\n\t"  /* Store result to NOS slot */ \
    "subq     $8, %%r14\n\t"        /* DSP-- */ \
    ::: "rax", "r14", "cc", "memory" \
)

```

7.62.1.11 PRIM_EXECUTE

```
#define PRIM_EXECUTE( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load xt from TOS */ \
    "subq     $8, %%r14\n\t"        /* DSP-- */ \
    "jmpq     *%%rax\n\t"           /* Jump to execution token */ \
    ::: "rax", "r14", "memory" \
)

```

7.62.1.12 PRIM_FETCH

```
#define PRIM_FETCH( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load address from TOS */ \
    "movq    (%%r12), %%rcx\n\t"    /* Load VM->memory */ \
    "movq    (%%rcx, %%rax, 1), %%rax\n\t" /* Load cell from memory */ \
    "movq     %%rax, (%%r14)\n\t"    /* Store to TOS */ \
    ::: "rax", "rcx", "memory" \
)

```

7.62.1.13 PRIM_GREATER

```
#define PRIM_GREATER( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rcx\n\t"      /* Load TOS (b) */ \
    "movq    -8(%%r14), %%rax\n\t"  /* Load NOS (a) */ \
    "cmpq     %%rcx, %%rax\n\t"      /* rax - rcx: signed compare a vs b */ \
)

```

```

    "setg    %al\n\t"                /* AL = 1 if a > b */ \
    "movzbq  %al, %%rax\n\t"        /* Zero-extend */ \
    "negq    %%rax\n\t"             /* -1 if true, 0 if false */ \
    "movq    %%rax, -8(%%r14)\n\t"   /* Store result to NOS slot */ \
    "subq    $8, %%r14\n\t"         /* DSP-- */ \
    ::: "rax", "rcx", "r14", "cc", "memory" \
)

```

7.62.1.14 PRIM_INVERT

```
#define PRIM_INVERT( )
```

Value:

```

__asm__ __volatile__ ( \
    "notq    (%%r14)\n\t"           /* TOS = ~TOS */ \
    ::: "memory" \
)

```

7.62.1.15 PRIM_LESS

```
#define PRIM_LESS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%%r14), %%rcx\n\t"    /* Load TOS (b) */ \
    "movq    -8(%%r14), %%rax\n\t" /* Load NOS (a) */ \
    "cmpq    %%rcx, %%rax\n\t"      /* rax - rcx: signed compare a vs b */ \
    "setl    %al\n\t"              /* AL = 1 if a < b */ \
    "movzbq  %al, %%rax\n\t"        /* Zero-extend */ \
    "negq    %%rax\n\t"            /* -1 if true, 0 if false */ \
    "movq    %%rax, -8(%%r14)\n\t" /* Store result to NOS slot */ \
    "subq    $8, %%r14\n\t"        /* DSP-- */ \
    ::: "rax", "rcx", "r14", "cc", "memory" \
)

```

7.62.1.16 PRIM_MINUS

```
#define PRIM_MINUS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%%r14), %%rax\n\t"    /* Load TOS */ \
    "subq    %%rax, -8(%%r14)\n\t"   /* Subtract from NOS */ \
    "subq    $8, %%r14\n\t"         /* DSP-- */ \
    ::: "rax", "r14", "cc", "memory" \
)

```

7.62.1.17 PRIM_MOD

```
#define PRIM_MOD( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    -8(%%r14), %%rax\n\t" /* Load dividend (NOS) */ \
    "cqo\n\t"                      /* Sign-extend */ \
    "idivq   (%%r14)\n\t"          /* rdx = remainder */ \
    "movq    %%rdx, -8(%%r14)\n\t" /* Store remainder to NOS slot */ \
    "subq    $8, %%r14\n\t"        /* DSP-- */ \
    ::: "rax", "rdx", "r14", "cc", "memory" \
)

```

7.62.1.18 PRIM_NEGATE

```
#define PRIM_NEGATE( )
```

Value:

```

__asm__ __volatile__ ( \
    "negq    (%%r14)\n\t"           /* TOS = -TOS */ \
    ::: "cc", "memory" \
)

```

7.62.1.19 PRIM_OR

```
#define PRIM_OR( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%%r14), %%rax\n\t"    /* Load TOS */ \
    "orq     %%rax, -8(%%r14)\n\t"   /* NOS |= TOS */ \
    "subq    $8, %%r14\n\t"         /* DSP-- */ \
)

```

```

    ::: "rax", "r14", "cc", "memory" \
)

```

7.62.1.20 PRIM_OVER

```
#define PRIM_OVER( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    -8(%r14), %%rax\n\t"    /* Load NOS */ \
    "addq    $8, %%r14\n\t"         /* DSP++ */ \
    "movq    %%rax, (%r14)\n\t"     /* Store NOS as new TOS */ \
    ::: "rax", "r14", "memory" \
)

```

7.62.1.21 PRIM_PLUS

```
#define PRIM_PLUS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"     /* Load TOS */ \
    "addq    %%rax, -8(%r14)\n\t"   /* Add to NOS */ \
    "subq    $8, %%r14\n\t"         /* DSP-- */ \
    ::: "rax", "r14", "cc", "memory" \
)

```

7.62.1.22 PRIM_R_FETCH

```
#define PRIM_R_FETCH( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    -8(%r15), %%rax\n\t"   /* Load from top of return stack */ \
    "addq    $8, %%r14\n\t"         /* DSP++ */ \
    "movq    %%rax, (%r14)\n\t"     /* Store to TOS */ \
    ::: "rax", "r14", "memory" \
)

```

7.62.1.23 PRIM_R_FROM

```
#define PRIM_R_FROM( )
```

Value:

```

__asm__ __volatile__ ( \
    "subq    $8, %%r15\n\t"         /* RSP-- */ \
    "movq    (%r15), %%rax\n\t"     /* Load from return stack */ \
    "addq    $8, %%r14\n\t"         /* DSP++ */ \
    "movq    %%rax, (%r14)\n\t"     /* Store to TOS */ \
    ::: "rax", "r14", "r15", "memory" \
)

```

7.62.1.24 PRIM_ROT

```
#define PRIM_ROT( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"     /* c = TOS */ \
    "movq    -8(%r14), %%rcx\n\t"   /* b = NOS */ \
    "movq    -16(%r14), %%rdx\n\t"  /* a = third */ \
    "movq    %%rdx, -8(%r14)\n\t"   /* a -> NOS */ \
    "movq    %%rax, -16(%r14)\n\t"  /* c -> third */ \
    "movq    %%rcx, (%r14)\n\t"     /* b -> TOS */ \
    ::: "rax", "rcx", "rdx", "memory" \
)

```

7.62.1.25 PRIM_SLASH

```
#define PRIM_SLASH( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    -8(%r14), %%rax\n\t"   /* Load dividend (NOS) */ \
    "cqo\n\t"                        /* Sign-extend rax to rdx:rax */ \
    "idivq   (%r14)\n\t"             /* Divide by TOS */ \
    "movq    %%rax, -8(%r14)\n\t"   /* Store quotient to NOS slot */ \
    "subq    $8, %%r14\n\t"         /* DSP-- */ \
    ::: "rax", "rdx", "r14", "cc", "memory" \
)

```

7.62.1.26 PRIM_STAR

```
#define PRIM_STAR( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "imulq   -8(%r14), %%rax\n\t"    /* Multiply with NOS */ \
    "movq    %%rax, -8(%r14)\n\t"    /* Store result */ \
    "subq    $8, %%r14\n\t"          /* DSP-- */ \
    ::: "rax", "rdx", "r14", "cc", "memory" \
)
```

7.62.1.27 PRIM_STORE

```
#define PRIM_STORE( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load address from TOS */ \
    "movq    -8(%r14), %%rcx\n\t"    /* Load value from NOS */ \
    "movq    (%r12), %%rdx\n\t"      /* Load VM->memory */ \
    "movq    %%rcx, (%%rdx, %%rax, 1)\n\t" /* Store to memory */ \
    "subq    $16, %%r14\n\t"          /* DSP -= 2 */ \
    ::: "rax", "rcx", "rdx", "r14", "memory" \
)
```

7.62.1.28 PRIM_SWAP

```
#define PRIM_SWAP( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "movq    -8(%r14), %%rcx\n\t"    /* Load NOS */ \
    "movq    %%rcx, (%r14)\n\t"      /* Store NOS to TOS */ \
    "movq    %%rax, -8(%r14)\n\t"    /* Store TOS to NOS */ \
    ::: "rax", "rcx", "memory" \
)
```

7.62.1.29 PRIM_TO_R

```
#define PRIM_TO_R( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load from TOS */ \
    "subq    $8, %%r14\n\t"          /* DSP-- */ \
    "movq    %%rax, (%r15)\n\t"      /* Store to return stack */ \
    "addq    $8, %%r15\n\t"          /* RSP++ */ \
    ::: "rax", "r14", "r15", "memory" \
)
```

7.62.1.30 PRIM_XOR

```
#define PRIM_XOR( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "xorq    %%rax, -8(%r14)\n\t"    /* NOS ^= TOS */ \
    "subq    $8, %%r14\n\t"          /* DSP-- */ \
    ::: "rax", "r14", "cc", "memory" \
)
```

7.62.1.31 PRIM_ZBRANCH

```
#define PRIM_ZBRANCH( )
```

Value:

```
__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "subq    $8, %%r14\n\t"          /* DSP-- */ \
    "testq   %%rax, %%rax\n\t"      /* Test if zero */ \
    "jnz     1f\n\t"                /* Skip if non-zero */ \
    "movq    (%r13), %%rax\n\t"      /* Load offset */ \
    "addq    %%rax, %%r13\n\t"      /* IP += offset */ \
    "jmp     2f\n\t" \
    "1:\n\t" \
    "addq    $8, %%r13\n\t"          /* Skip offset */ \
    "2:\n\t" \
)
```



```

    ::: "rax", "r13", "r14", "cc", "memory" \
)

```

7.62.1.32 PRIM_ZERO_EQUALS

```
#define PRIM_ZERO_EQUALS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "testq   %%rax, %%rax\n\t"      /* Test for zero */ \
    "sete    %%al\n\t"              /* AL = 1 if zero */ \
    "movzbq   %%al, %%rax\n\t"      /* Zero-extend */ \
    "negq     %%rax\n\t"            /* -1 if true, 0 if false */ \
    "movq     %%rax, (%r14)\n\t"    /* Store result */ \
    ::: "rax", "cc", "memory" \
)

```

7.62.1.33 PRIM_ZERO_LESS

```
#define PRIM_ZERO_LESS( )
```

Value:

```

__asm__ __volatile__ ( \
    "movq    (%r14), %%rax\n\t"      /* Load TOS */ \
    "sarq    $63, %%rax\n\t"        /* Arithmetic shift: -1 if negative, 0 otherwise */ \
    "movq     %%rax, (%r14)\n\t"    /* Store result */ \
    ::: "rax", "cc", "memory" \
)

```

7.62.2 Function Documentation

7.62.2.1 vm_docol_asm()

```
static void vm_docol_asm (
    VM * vm ) [inline], [static]
```

Enter a colon definition (DOCOL)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Called when executing a user-defined word

References [VM::current_executing_entry](#), [entry](#), and [vm](#).

7.62.2.2 vm_exit_asm()

```
static void vm_exit_asm (
    void ) [inline], [static]
```

Return from a colon definition (EXIT)

Pops IP from return stack and continues execution

7.62.2.3 vm_save_registers()

```
static void vm_save_registers (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.62.2.4 vm_setup_registers()

```
static void vm_setup_registers (
    VM * vm ) [inline], [static]
```

Main interpreter loop implementation.

Replaces execute_colon_word function with optimized assembly

Usage example:

```
vm_setup_registers(vm);
VM_INNER_LOOP();
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.63 include/vm_inner_interp_riscv64.h File Reference

```
#include "vm.h"
```

Include dependency graph for vm_inner_interp_riscv64.h:

Macros

- #define [NEXT_RV64\(\)](#)
- #define [PRIM_DUP_RV64\(\)](#)
- #define [PRIM_DROP_RV64\(\)](#)
- #define [PRIM_SWAP_RV64\(\)](#)
- #define [PRIM_OVER_RV64\(\)](#)
- #define [PRIM_ROT_RV64\(\)](#)
- #define [PRIM_PLUS_RV64\(\)](#)
- #define [PRIM_MINUS_RV64\(\)](#)
- #define [PRIM_STAR_RV64\(\)](#)
- #define [PRIM_SLASH_RV64\(\)](#)
- #define [PRIM_MOD_RV64\(\)](#)
- #define [PRIM_AND_RV64\(\)](#)
- #define [PRIM_OR_RV64\(\)](#)
- #define [PRIM_XOR_RV64\(\)](#)
- #define [PRIM_INVERT_RV64\(\)](#)
- #define [PRIM_NEGATE_RV64\(\)](#)
- #define [PRIM_FETCH_RV64\(\)](#)
- #define [PRIM_STORE_RV64\(\)](#)
- #define [PRIM_C_FETCH_RV64\(\)](#)
- #define [PRIM_C_STORE_RV64\(\)](#)
- #define [PRIM_TO_R_RV64\(\)](#)
- #define [PRIM_R_FROM_RV64\(\)](#)
- #define [PRIM_R_FETCH_RV64\(\)](#)
- #define [PRIM_2DUP_RV64\(\)](#)
- #define [PRIM_2DROP_RV64\(\)](#)
- #define [PRIM_ZERO_EQUALS_RV64\(\)](#)
- #define [PRIM_ZERO_LESS_RV64\(\)](#)
- #define [PRIM_EQUALS_RV64\(\)](#)
- #define [PRIM_LESS_RV64\(\)](#)
- #define [PRIM_GREATER_RV64\(\)](#)
- #define [PRIM_BRANCH_RV64\(\)](#)
- #define [PRIM_ZBRANCH_RV64\(\)](#)
- #define [PRIM_EXECUTE_RV64\(\)](#)

Functions

- static void [vm_docol_rv64](#) (VM *vm)
- static void [vm_exit_rv64](#) (void)
- static void [vm_setup_registers_rv64](#) (VM *vm)
- static void [vm_save_registers_rv64](#) (VM *vm)

7.63.1 Macro Definition Documentation

7.63.1.1 NEXT_RV64

```
#define NEXT_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s3)\n\t"      /* a0 = *IP          */ \
    "addi    s3, s3, 8\n\t"      /* IP++              */ \
    "jr      a0\n\t"            /* jump to code field */ \
    ::: "a0", "memory" \
)
```

7.63.1.2 PRIM_2DROP_RV64

```
#define PRIM_2DROP_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      s6, -8(s4)\n\t" /* new TOS = item3      */ \
    "addi    s4, s4, -16\n\t" \
    ::: "s4", "s6", "memory" \
)
```

7.63.1.3 PRIM_2DUP_RV64

```
#define PRIM_2DUP_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = NOS      */ \
    "addi    s4, s4, 8\n\t" \
    "sd      a0, 0(s4)\n\t" /* push NOS      */ \
    "addi    s4, s4, 8\n\t" \
    "sd      s6, 0(s4)\n\t" /* push TOS copy */ \
    ::: "a0", "s4", "memory" \
)
```

7.63.1.4 PRIM_AND_RV64

```
#define PRIM_AND_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "and     s6, s6, a0\n\t" \
    ::: "a0", "s4", "s6" \
)
```

7.63.1.5 PRIM_BRANCH_RV64

```
#define PRIM_BRANCH_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s3)\n\t" /* offset      */ \
    "add     s3, s3, a0\n\t" /* IP += offset */ \
    ::: "a0", "s3", "memory" \
)
```

7.63.1.6 PRIM_C_FETCH_RV64

```
#define PRIM_C_FETCH_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s2)\n\t" \
    "add     a0, a0, s6\n\t" \
    "lbu     s6, 0(a0)\n\t" /* zero-extend byte */ \
    ::: "a0", "s6", "memory" \
)
```

7.63.1.7 PRIM_C_STORE_RV64

```
#define PRIM_C_STORE_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = addr      */ \
    "addi    s4, s4, -8\n\t" \
    "ld      a1, 0(s2)\n\t" \
    "add     a1, a1, a0\n\t" \
    "sb      s6, 0(a1)\n\t" \
    "ld      s6, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    ::: "a0", "a1", "s4", "s6", "memory" \
)
```

7.63.1.8 PRIM_DROP_RV64

```
#define PRIM_DROP_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      s6, -8(s4)\n\t" /* new TOS = NOS          */ \
    "addi    s4, s4, -8\n\t" /* DSP--              */ \
    ::: "s4", "s6", "memory" \
)

```

7.63.1.9 PRIM_DUP_RV64

```
#define PRIM_DUP_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "addi    s4, s4, 8\n\t" /* DSP++              */ \
    "sd      s6, 0(s4)\n\t" /* push TOS copy      */ \
    ::: "s4", "memory" \
)

```

7.63.1.10 PRIM_EQUALS_RV64

```
#define PRIM_EQUALS_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "sub     s6, a0, s6\n\t" \
    "seqz    s6, s6\n\t" \
    "neg     s6, s6\n\t" \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.11 PRIM_EXECUTE_RV64

```
#define PRIM_EXECUTE_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "mv      a0, s6\n\t" /* a0 = xt              */ \
    "ld      s6, 0(s4)\n\t" /* new TOS              */ \
    "addi    s4, s4, -8\n\t" \
    "jrr     a0\n\t" /* jump to xt           */ \
    ::: "a0", "s4", "s6", "memory" \
)

```

7.63.1.12 PRIM_FETCH_RV64

```
#define PRIM_FETCH_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s2)\n\t" /* a0 = vm->memory      */ \
    "add     a0, a0, s6\n\t" /* a0 = &memory[TOS]    */ \
    "ld      s6, 0(a0)\n\t" /* TOS = memory[TOS]    */ \
    ::: "a0", "s6", "memory" \
)

```

7.63.1.13 PRIM_GREATER_RV64

```
#define PRIM_GREATER_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "slt     s6, s6, a0\n\t" /* 1 if TOS < NOS, i.e. NOS > TOS */ \
    "neg     s6, s6\n\t" \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.14 PRIM_INVERT_RV64

```
#define PRIM_INVERT_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "not     s6, s6\n\t" \
    ::: "s6" \
)

```

7.63.1.15 PRIM_LESS_RV64

```
#define PRIM_LESS_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = NOS */ \
    "addi    s4, s4, -8\n\t" \
    "slt     s6, a0, s6\n\t" /* 1 if NOS < TOS */ \
    "neg     s6, s6\n\t" \
    ::: "a0", "s4", "s6" \
)
```

7.63.1.16 PRIM_MINUS_RV64

```
#define PRIM_MINUS_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = NOS */ \
    "addi    s4, s4, -8\n\t" \
    "sub     s6, a0, s6\n\t" /* TOS = NOS - TOS */ \
    ::: "a0", "s4", "s6" \
)
```

7.63.1.17 PRIM_MOD_RV64

```
#define PRIM_MOD_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "rem     s6, a0, s6\n\t" \
    ::: "a0", "s4", "s6" \
)
```

7.63.1.18 PRIM_NEGATE_RV64

```
#define PRIM_NEGATE_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "neg     s6, s6\n\t" \
    ::: "s6" \
)
```

7.63.1.19 PRIM_OR_RV64

```
#define PRIM_OR_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "or      s6, s6, a0\n\t" \
    ::: "a0", "s4", "s6" \
)
```

7.63.1.20 PRIM_OVER_RV64

```
#define PRIM_OVER_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = NOS */ \
    "addi    s4, s4, 8\n\t" /* DSP++ */ \
    "sd      s6, 0(s4)\n\t" /* push old TOS */ \
    "mv      s6, a0\n\t" /* TOS = NOS */ \
    ::: "a0", "s4", "s6", "memory" \
)
```

7.63.1.21 PRIM_PLUS_RV64

```
#define PRIM_PLUS_RV64( )
```

Value:

```
__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
)
```

```

    "add    s6, s6, a0\n\t" \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.22 PRIM_R_FETCH_RV64

```
#define PRIM_R_FETCH_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "addi    s4, s4, 8\n\t" \
    "sd      s6, 0(s4)\n\t" \
    "ld      s6, 0(s5)\n\t" /* TOS = RS top (no pop) */ \
    ::: "s4", "s6", "memory" \
)

```

7.63.1.23 PRIM_R_FROM_RV64

```
#define PRIM_R_FROM_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "addi    s4, s4, 8\n\t" \
    "sd      s6, 0(s4)\n\t" /* push old TOS */ \
    "ld      s6, 0(s5)\n\t" /* TOS = RS top */ \
    "addi    s5, s5, -8\n\t" \
    ::: "s4", "s5", "s6", "memory" \
)

```

7.63.1.24 PRIM_ROT_RV64

```
#define PRIM_ROT_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = item2 (NOS) */ \
    "ld      a1, -8(s4)\n\t" /* a1 = item3 */ \
    "sd      a1, 0(s4)\n\t" /* item3 -> NOS position */ \
    "sd      s6, -8(s4)\n\t" /* TOS -> item3 position */ \
    "mv      s6, a0\n\t" /* item2 -> TOS */ \
    ::: "a0", "a1", "s6", "memory" \
)

```

7.63.1.25 PRIM_SLASH_RV64

```
#define PRIM_SLASH_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = NOS (dividend) */ \
    "addi    s4, s4, -8\n\t" \
    "div     s6, a0, s6\n\t" /* TOS = NOS / TOS */ \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.26 PRIM_STAR_RV64

```
#define PRIM_STAR_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" \
    "addi    s4, s4, -8\n\t" \
    "mul     s6, s6, a0\n\t" \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.27 PRIM_STORE_RV64

```
#define PRIM_STORE_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld      a0, 0(s4)\n\t" /* a0 = addr (NOS) */ \
    "addi    s4, s4, -8\n\t" \
    "ld      a1, 0(s2)\n\t" /* a1 = vm->memory */ \
    "add     a1, a1, a0\n\t" \
    "sd      s6, 0(a1)\n\t" /* memory[addr] = TOS */ \
    "ld      s6, 0(s4)\n\t" /* load new TOS */ \
    "addi    s4, s4, -8\n\t" \
)

```

```

    ::: "a0", "a1", "s4", "s6", "memory" \
)

```

7.63.1.28 PRIM_SWAP_RV64

```
#define PRIM_SWAP_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld    a0, 0(s4)\n\t"      /* a0 = NOS          */ \
    "sd    s6, 0(s4)\n\t"      /* NOS = old TOS     */ \
    "mv    s6, a0\n\t"         /* TOS = old NOS     */ \
    ::: "a0", "s6", "memory" \
)

```

7.63.1.29 PRIM_TO_R_RV64

```
#define PRIM_TO_R_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "addi   s5, s5, 8\n\t" \
    "sd     s6, 0(s5)\n\t" /* RS[top] = TOS      */ \
    "ld     s6, 0(s4)\n\t" /* new TOS = NOS      */ \
    "addi   s4, s4, -8\n\t" \
    ::: "s4", "s5", "s6", "memory" \
)

```

7.63.1.30 PRIM_XOR_RV64

```
#define PRIM_XOR_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld     a0, 0(s4)\n\t" \
    "addi   s4, s4, -8\n\t" \
    "xor    s6, s6, a0\n\t" \
    ::: "a0", "s4", "s6" \
)

```

7.63.1.31 PRIM_ZBRANCH_RV64

```
#define PRIM_ZBRANCH_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "ld     a0, 0(s3)\n\t" /* offset          */ \
    "ld     s6, 0(s4)\n\t" /* new TOS         */ \
    "addi   s4, s4, -8\n\t" \
    /* if old TOS was zero: IP += offset; else IP += 8 */ \
    "bnez   s6, 1f\n\t" /* skip branch if nonzero */ \
    "add    s3, s3, a0\n\t" \
    "j      2f\n\t" \
    "1:\n\t" \
    "addi   s3, s3, 8\n\t" \
    "2:\n\t" \
    ::: "a0", "s3", "s4", "s6", "memory" \
)

```

7.63.1.32 PRIM_ZERO_EQUALS_RV64

```
#define PRIM_ZERO_EQUALS_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "seqz   s6, s6\n\t" /* 1 if zero, else 0 */ \
    "neg    s6, s6\n\t" /* convert to -1 (true) */ \
    ::: "s6" \
)

```

7.63.1.33 PRIM_ZERO_LESS_RV64

```
#define PRIM_ZERO_LESS_RV64( )
```

Value:

```

__asm__ __volatile__ ( \
    "srai   s6, s6, 63\n\t" /* -1 if negative, 0 if >= 0 */ \
    ::: "s6" \
)

```

7.63.2 Function Documentation

7.63.2.1 vm_docol_rv64()

```
static void vm_docol_rv64 (
    VM * vm ) [inline], [static]
```

References [VM::current_executing_entry](#), [entry](#), and [vm](#).

7.63.2.2 vm_exit_rv64()

```
static void vm_exit_rv64 (
    void ) [inline], [static]
```

7.63.2.3 vm_save_registers_rv64()

```
static void vm_save_registers_rv64 (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.63.2.4 vm_setup_registers_rv64()

```
static void vm_setup_registers_rv64 (
    VM * vm ) [inline], [static]
```

References [VM::data_stack](#), [VM::dsp](#), [VM::ip](#), [VM::return_stack](#), [VM::rsp](#), and [vm](#).

7.64 include/word_registry.h File Reference

```
#include "vm.h"
```

Include dependency graph for word_registry.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_word](#) ([VM *vm](#), const char *[name](#), [word_func_t](#) [func](#))
Registers a single FORTH word in the virtual machine.
- void [register_forth79_words](#) ([VM *vm](#))
Registers all standard FORTH-79 words in the virtual machine.

7.64.1 Function Documentation

7.64.1.1 register_forth79_words()

```
void register_forth79_words (
    VM * vm )
```

Registers all standard FORTH-79 words in the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>vm</i>	Pointer to the virtual machine instance

Registers all 18 FORTH-79 word modules in dependency order, starting with fundamental stack operations and building up to more complex functionality like control flow and editing.

References [LOG_INFO](#), [log_message\(\)](#), [register_acl_words\(\)](#), [register_arithmetic_words\(\)](#), [register_block_words\(\)](#), [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_heat_diagnostic_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_dictionary_words\(\)](#), [register_double_words\(\)](#), [register_editor_words\(\)](#), [register_format_words\(\)](#), [register_io_words\(\)](#), [register_logical_words\(\)](#), [register_memory_words\(\)](#), [register_mixed_arithmetic_words\(\)](#), [register_physics_benchmark_words\(\)](#), [register_physics_freeze_words\(\)](#), [register_physics_pipelining_diagnostic_words\(\)](#), [register_return_stack_words\(\)](#), [register_stack_words\(\)](#), [register_starforth_words\(\)](#), [register_string_words\(\)](#), [register_system_words\(\)](#), [register_vocabulary_words\(\)](#), and [vm](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.64.1.2 register_word()

```
void register_word (
    VM * vm,
    const char * name,
    word_func_t func )
```

Registers a single FORTH word in the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Name of the FORTH word to register
<i>func</i>	Function pointer to the word's implementation

References [entry](#), [func](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [name](#), [vm](#), and [vm_create_word\(\)](#).

Referenced by [register_acl_words\(\)](#), [register_arithmetic_words\(\)](#), [register_block_words\(\)](#), [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_heat_diagnostic_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_dictionary_words\(\)](#), [register_double_words\(\)](#), [register_editor_words\(\)](#), [register_format_words\(\)](#), [register_io_words\(\)](#), [register_logical_words\(\)](#), [register_memory_words\(\)](#), [register_mixed_arithmetic_words\(\)](#), [register_physics_benchmark_words\(\)](#), [register_physics_diagnostic_words\(\)](#), [register_physics_freeze_words\(\)](#), [register_return_stack_words\(\)](#), [register_stack_words\(\)](#), [register_starforth_words\(\)](#), [register_string_words\(\)](#), [register_system_words\(\)](#), and [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.65 README.md File Reference

7.66 src/blkio_factory.c File Reference

```
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include "blkio.h"
#include "blkio_factory.h"
Include dependency graph for blkio_factory.c:
```

Functions

- [size_t blkio_file_state_size](#) (void)
- [const blkio_vtable_t * blkio_file_vtable](#) (void)
- [int blkio_file_init_state](#) (void *state_mem, size_t state_len, const char *path, uint32_t total_blocks, uint32_t fbs, uint8_t read_only, uint8_t create_if_missing, uint8_t truncate_to_size, void **out_opaque)
- [size_t blkio_ram_state_size](#) (void)
- [const blkio_vtable_t * blkio_ram_vtable](#) (void)
- [int blkio_ram_init_state](#) (void *state_mem, size_t state_len, uint8_t *base, uint32_t total_blocks, uint32_t fbs, uint8_t read_only, void **out_opaque)
- [static int nonempty](#) (const char *s)
- [int blkio_factory_open](#) ([blkio_dev_t](#) *dev, const char *disk_img_path, uint8_t read_only, uint32_t total_blocks, uint32_t fbs, void *state_mem, size_t state_len, uint8_t *ram_base, uint32_t ram_blocks, uint8_t *out_used_file)

Open a block device using either a file-backed image (if disk_img_path is non-empty) or a RAM-backed buffer (fallback).

7.66.1 Function Documentation

7.66.1.1 blkio_factory_open()

```
int blkio_factory_open (
    struct blkio_dev * dev,
    const char * disk_img_path,
    uint8_t read_only,
    uint32_t total_blocks,
    uint32_t fbs,
    void * state_mem,
    size_t state_len,
    uint8_t * ram_base,
    uint32_t ram_blocks,
    uint8_t * out_used_file )
```

Open a block device using either a file-backed image (if `disk_img_path` is non-empty) or a RAM-backed buffer (fallback).

Parameters:

- `dev` : output device object (caller-allocated)
- `disk_img_path` : path to raw image; if NULL/empty => RAM fallback
- `read_only` : 1 = open read-only; 0 = read/write
- `total_blocks` : FILE: 0 => derive from file size; >0 => size hint RAM : must match `ram_blocks`
- `fbs` : forth block size in bytes (0 => default per [blkio.h](#))
- `state_mem` : caller-provided backend state buffer (opaque to caller)
- `state_len` : size of `state_mem` in bytes
- `ram_base` : RAM storage base pointer (RAM backend only)
- `ram_blocks` : number of Forth blocks in `ram_base` (RAM backend only)
- `out_used_file` : optional; set to 1 if FILE backend selected, else 0

Returns: 0 on success; negative blkio error code on failure.

References [BLKIO_EINVAL](#), [BLKIO_ENOSPACE](#), [blkio_file_init_state\(\)](#), [blkio_file_state_size\(\)](#), [blkio_file_vtable\(\)](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_open\(\)](#), [blkio_ram_init_state\(\)](#), [blkio_ram_state_size\(\)](#), [blkio_ram_vtable\(\)](#), [blkio_params_t::forth_block_size](#), [nonempty\(\)](#), and [ram_base](#).

Referenced by [init_blkio\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.66.1.2 blkio_file_init_state()

```
int blkio_file_init_state (
    void * state_mem,
    size_t state_len,
    const char * path,
    uint32_t total_blocks,
    uint32_t fbs,
    uint8_t read_only,
    uint8_t create_if_missing,
    uint8_t truncate_to_size,
    void ** out_opaque ) [extern]
```

References [BLKIO_EINVAL](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [blkio_file_state_t::path](#), [PATH_MAX](#), [blkio_file_state_t::read_only](#), and [blkio_file_state_t::total_blocks](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.3 blkio_file_state_size()

```
size_t blkio_file_state_size (
    void ) [extern]
```

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.4 blkio_file_vtable()

```
const blkio_vtable_t * blkio_file_vtable (
    void ) [extern]
```

References [BLKIO_FILE_VT](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.5 blkio_ram_init_state()

```
int blkio_ram_init_state (
    void * state_mem,
    size_t state_len,
    uint8_t * base,
    uint32_t total_blocks,
    uint32_t fbs,
    uint8_t read_only,
    void ** out_opaque ) [extern]
```

References [blkio_ram_state_t::base](#), [base](#), [BLKIO_EINVAL](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [blkio_ram_state_t::read_only](#), and [blkio_ram_state_t::total_blocks](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.6 blkio_ram_state_size()

```
size_t blkio_ram_state_size (
    void ) [extern]
```

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.7 blkio_ram_vtable()

```
const blkio_vtable_t * blkio_ram_vtable (
    void ) [extern]
```

References [BLKIO_RAM_VT](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.66.1.8 nonempty()

```
static int nonempty (
    const char * s ) [static]
```

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.67 src/blkio_file.c File Reference

```
#include "blkio.h"
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
```

```
#include <errno.h>
#include <string.h>
#include <stdint.h>
Include dependency graph for blkio_file.c:
```

Data Structures

- struct [blkio_file_state_t](#)

Macros

- #define [PATH_MAX](#) 4096

Functions

- [size_t blkio_file_state_size](#) (void)
- [int blkio_file_init_state](#) (void *state_mem, [size_t](#) state_len, const char *path, [uint32_t](#) total_blocks, [uint32_t](#) fbs, [uint8_t](#) read_only, [uint8_t](#) create_if_missing, [uint8_t](#) truncate_to_size, void **out_opaque)
- static [int full_read_at](#) (int fd, void *buf, [size_t](#) len, [off_t](#) off_abs)
- static [int full_write_at](#) (int fd, const void *buf, [size_t](#) len, [off_t](#) off_abs)
- static [int file_open](#) ([blkio_dev_t](#) *dev, const [blkio_params_t](#) *p)
- static [int file_close](#) ([blkio_dev_t](#) *dev)
- static [int file_read](#) ([blkio_dev_t](#) *dev, [uint32_t](#) fblock, void *dst)
- static [int file_write](#) ([blkio_dev_t](#) *dev, [uint32_t](#) fblock, const void *src)
- static [int file_flush](#) ([blkio_dev_t](#) *dev)
- static [int file_info](#) ([blkio_dev_t](#) *dev, [blkio_info_t](#) *out)
- const [blkio_vtable_t](#) * [blkio_file_vtable](#) (void)

Variables

- static const [blkio_vtable_t](#) [BLKIO_FILE_VT](#)

7.67.1 Macro Definition Documentation

7.67.1.1 [PATH_MAX](#)

```
#define PATH_MAX 4096
```

7.67.2 Function Documentation

7.67.2.1 [blkio_file_init_state\(\)](#)

```
int blkio_file_init_state (
    void * state_mem,
    size_t state_len,
    const char * path,
    uint32_t total_blocks,
    uint32_t fbs,
    uint8_t read_only,
    uint8_t create_if_missing,
    uint8_t truncate_to_size,
    void ** out_opaque )
```

References [BLKIO_EINVAL](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [blkio_file_state_t::path](#), [PATH_MAX](#), [blkio_file_state_t::read_only](#), and [blkio_file_state_t::total_blocks](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.67.2.2 blkio_file_state_size()

```
size_t blkio_file_state_size (
    void )
```

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.67.2.3 blkio_file_vtable()

```
const blkio_vtable_t * blkio_file_vtable (
    void )
```

References [BLKIO_FILE_VT](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.67.2.4 file_close()

```
static int file_close (
    blkio_dev_t * dev ) [static]
```

References [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_OK](#), [blkio_file_state_t::fd](#), and [blkio_dev::state](#).

7.67.2.5 file_flush()

```
static int file_flush (
    blkio_dev_t * dev ) [static]
```

References [BLKIO_ECLOSED](#), [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_OK](#), [blkio_file_state_t::fd](#), and [blkio_dev::state](#).

7.67.2.6 file_info()

```
static int file_info (
    blkio_dev_t * dev,
    blkio_info_t * out ) [static]
```

References [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [blkio_info_t::forth_block_size](#), [blkio_info_t::phys_sector_size](#), [blkio_info_t::phys_size_bytes](#), [blkio_info_t::read_only](#), [blkio_file_state_t::read_only](#), [blkio_dev::state](#), [blkio_info_t::total_blocks](#), and [blkio_file_state_t::total_blocks](#).

Referenced by [load_kernel_from_esp\(\)](#).

Here is the caller graph for this function:

7.67.2.7 file_open()

```
static int file_open (
    blkio_dev_t * dev,
    const blkio_params_t * p ) [static]
```

References [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [blkio_dev::forth_block_size](#), [blkio_params_t::opaque](#), [blkio_file_state_t::path](#), [blkio_file_state_t::read_only](#), [blkio_dev::state](#), [blkio_dev::total_blocks](#), and [blkio_file_state_t::total_blocks](#).

7.67.2.8 file_read()

```
static int file_read (
    blkio_dev_t * dev,
    uint32_t fblock,
    void * dst ) [static]
```

References [BLKIO_ECLOSED](#), [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [full_read_at\(\)](#), [blkio_dev::state](#), and [blkio_dev::total_blocks](#).

Here is the call graph for this function:

7.67.2.9 file_write()

```
static int file_write (
    blkio_dev_t * dev,
    uint32_t fblock,
    const void * src ) [static]
```

References [BLKIO_ECLOSED](#), [BLKIO_EINVAL](#), [BLKIO_EIO](#), [BLKIO_ENOSUP](#), [BLKIO_OK](#), [blkio_file_state_t::fbs](#), [blkio_file_state_t::fd](#), [full_write_at\(\)](#), [blkio_file_state_t::read_only](#), [blkio_dev::state](#), and [blkio_dev::total_blocks](#).

Here is the call graph for this function:

7.67.2.10 full_read_at()

```
static int full_read_at (
    int fd,
    void * buf,
    size_t len,
    off_t off_abs ) [static]
```

Referenced by [file_read\(\)](#).

Here is the caller graph for this function:

7.67.2.11 full_write_at()

```
static int full_write_at (
    int fd,
    const void * buf,
    size_t len,
    off_t off_abs ) [static]
```

Referenced by [file_write\(\)](#).

Here is the caller graph for this function:

7.67.3 Variable Documentation

7.67.3.1 BLKIO_FILE_VT

```
const blkio_vtable_t BLKIO_FILE_VT [static]
```

Initial value:

```
= {
    .open = file_open,
    .close = file_close,
    .read = file_read,
    .write = file_write,
    .flush = file_flush,
    .info = file_info
}
```

Referenced by [blkio_file_vtable\(\)](#).

7.68 src/blkio_ram.c File Reference

```
#include "blkio.h"
#include <string.h>
```

Include dependency graph for blkio_ram.c:

Data Structures

- struct [blkio_ram_state_t](#)

Functions

- [size_t blkio_ram_state_size](#) (void)
- [int blkio_ram_init_state](#) (void *state_mem, size_t state_len, uint8_t *base, uint32_t total_blocks, uint32_t fbs, uint8_t read_only, void **out_opaque)

- static int [ram_open](#) ([blkio_dev_t](#) *dev, const [blkio_params_t](#) *p)
- static int [ram_close](#) ([blkio_dev_t](#) *dev)
- static int [ram_read](#) ([blkio_dev_t](#) *dev, uint32_t fblock, void *dst)
- static int [ram_write](#) ([blkio_dev_t](#) *dev, uint32_t fblock, const void *src)
- static int [ram_flush](#) ([blkio_dev_t](#) *dev)
- static int [ram_info](#) ([blkio_dev_t](#) *dev, [blkio_info_t](#) *out)
- const [blkio_vtable_t](#) * [blkio_ram_vtable](#) (void)
- static int [in_bounds](#) (const [blkio_ram_state_t](#) *st, uint32_t fb)

Variables

- static const [blkio_vtable_t](#) [BLKIO_RAM_VT](#)

7.68.1 Function Documentation

7.68.1.1 blkio_ram_init_state()

```
int blkio_ram_init_state (
    void * state_mem,
    size_t state_len,
    uint8_t * base,
    uint32_t total_blocks,
    uint32_t fbs,
    uint8_t read_only,
    void ** out_opaque )
```

References [blkio_ram_state_t::base](#), [base](#), [BLKIO_EINVAL](#), [BLKIO_FORTH_BLOCK_SIZE](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [blkio_ram_state_t::read_only](#), and [blkio_ram_state_t::total_blocks](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.68.1.2 blkio_ram_state_size()

```
size_t blkio_ram_state_size (
    void )
```

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.68.1.3 blkio_ram_vtable()

```
const blkio\_vtable\_t * blkio_ram_vtable (
    void )
```

References [BLKIO_RAM_VT](#).

Referenced by [blkio_factory_open\(\)](#).

Here is the caller graph for this function:

7.68.1.4 in_bounds()

```
static int in_bounds (
    const blkio\_ram\_state\_t * st,
    uint32_t fb ) [inline], [static]
```

References [blkio_ram_state_t::total_blocks](#).

Referenced by [ram_read\(\)](#), and [ram_write\(\)](#).

Here is the caller graph for this function:

7.68.1.5 ram_close()

```
static int ram_close (
    blkio\_dev\_t * dev ) [static]
```

References [BLKIO_EINVAL](#), [BLKIO_OK](#), and [blkio_dev::state](#).

7.68.1.6 ram_flush()

```
static int ram_flush (
    blkio_dev_t * dev ) [static]
```

References [BLKIO_OK](#).

7.68.1.7 ram_info()

```
static int ram_info (
    blkio_dev_t * dev,
    blkio_info_t * out ) [static]
```

References [BLKIO_EINVAL](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [blkio_info_t::forth_block_size](#), [blkio_info_t::phys_sector_size](#), [blkio_info_t::phys_size_bytes](#), [blkio_info_t::read_only](#), [blkio_ram_state_t::read_only](#), [blkio_dev::state](#), [blkio_info_t::total_blocks](#), and [blkio_ram_state_t::total_blocks](#).

7.68.1.8 ram_open()

```
static int ram_open (
    blkio_dev_t * dev,
    const blkio_params_t * p ) [static]
```

References [blkio_ram_state_t::base](#), [BLKIO_EINVAL](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [blkio_dev::forth_block_size](#), [blkio_params_t::opaque](#), [blkio_dev::state](#), [blkio_dev::total_blocks](#), and [blkio_ram_state_t::total_blocks](#).

7.68.1.9 ram_read()

```
static int ram_read (
    blkio_dev_t * dev,
    uint32_t fblock,
    void * dst ) [static]
```

References [blkio_ram_state_t::base](#), [BLKIO_EINVAL](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [in_bounds\(\)](#), and [blkio_dev::state](#).

Here is the call graph for this function:

7.68.1.10 ram_write()

```
static int ram_write (
    blkio_dev_t * dev,
    uint32_t fblock,
    const void * src ) [static]
```

References [blkio_ram_state_t::base](#), [BLKIO_EINVAL](#), [BLKIO_ENOSUP](#), [BLKIO_OK](#), [blkio_ram_state_t::fbs](#), [in_bounds\(\)](#), [blkio_ram_state_t::read_only](#), and [blkio_dev::state](#).

Here is the call graph for this function:

7.68.2 Variable Documentation

7.68.2.1 BLKIO_RAM_VT

```
const blkio_vtable_t BLKIO_RAM_VT [static]
```

Initial value:

```
= {
    .open = ram_open,
    .close = ram_close,
    .read = ram_read,
    .write = ram_write,
    .flush = ram_flush,
    .info = ram_info
}
```

Referenced by [blkio_ram_vtable\(\)](#).

7.69 src/block_subsystem.c File Reference

```
#include "../include/blkio.h"
```



```
#include "../include/block_subsystem.h"
#include "../include/log.h"
#include "../include/platform_time.h"
#include <string.h>
#include <stdlib.h>
#include <stdint.h>
```

Include dependency graph for block_subsystem.c:

Data Structures

- struct [cache_slot_t](#)

Macros

- #define [BLK_FORTH_SYS_RESERVED](#) 32 /* RAM: hide 0..31; user RAM LBN 0..991 */
- #define [BLK_DISK_SYS_RESERVED](#) 32 /* DISK: hide 1024..1055 */
- #define [META_REGION_OFFSET](#) (BLK_PACK_RATIO * BLK_FORTH_SIZE) /* 3072 */
- #define [META_REGION_SIZE](#) (BLK_DEVICE_SECTOR - META_REGION_OFFSET) /* 1024 */
- #define [META_PER_BLOCK](#) (META_REGION_SIZE / BLK_PACK_RATIO) /* 341 */
- #define [DISK_CACHE_SLOTS](#) 8

Functions

- static size_t [minzu](#) (size_t a, size_t b)
 - static uint32_t [udiv_floor](#) (uint32_t a, uint32_t b)
 - static void [crc64_init](#) (void)
 - static uint64_t [compute_crc64](#) (const uint8_t *data, size_t len)
 - static int [bam_test](#) (const uint8_t *bm, uint64_t idx)
 - static void [bam_set](#) (uint8_t *bm, uint64_t idx)
 - static void [bam_clr](#) (uint8_t *bm, uint64_t idx)
 - static uint32_t [fblock_to_devblock](#) (uint32_t phys_block)
 - static uint32_t [fblock_pack_offset](#) (uint32_t phys_block)
 - static uint32_t [devblock4k_to_lba1k](#) (uint32_t dev4k)
 - static int [read_devblock_4k](#) (uint32_t dev4k, uint8_t *buf4k)
 - static int [write_devblock_4k](#) (uint32_t dev4k, const uint8_t *buf4k)
 - static void [meta_from_slice](#) (blk_meta_t *dst, const uint8_t *slice)
 - static void [meta_to_slice](#) (const blk_meta_t *src, uint8_t *slice)
 - static int [cache_writeback](#) (cache_slot_t *s)
 - static cache_slot_t * [cache_get_slot](#) (uint32_t dev4k)
 - static cache_slot_t * [cache_load_devblock](#) (uint32_t dev4k)
 - static void [volmeta_from_buf](#) (blk_volume_meta_t *out, const uint8_t *buf)
 - static void [volmeta_to_buf](#) (const blk_volume_meta_t *in, uint8_t *buf)
 - static int [read_header_4k](#) (uint8_t *buf4k)
 - static int [write_header_4k](#) (const uint8_t *buf4k)
 - static uint32_t [choose_B](#) (uint64_t total_devblocks_4k)
 - static void [compute_totals_from_B](#) (blk_volume_meta_t *m)
 - static int [bam_load_from_disk](#) (void)
 - static int [bam_flush_to_disk](#) (void)
 - static int [lbn_in_range](#) (uint32_t lbn)
 - static uint32_t [lbn_to_pbn](#) (uint32_t lbn)
 - static uint64_t [blk_get_timestamp](#) (void)
- Get current timestamp with RTC fallback to monotonic clock.*
- int [blk_subsys_init](#) (VM *vm, uint8_t *ram_base, size_t ram_size)
 - static int [blk_format_or_load_v2](#) (void)
 - int [blk_subsys_attach_device](#) (struct blkio_dev *dev)
 - int [blk_subsys_shutdown](#) (void)

- uint8_t * [blk_get_buffer](#) (uint32_t block_num, int writable)
- uint8_t * [blk_get_empty_buffer](#) (uint32_t block_num)
- int [blk_update](#) (uint32_t block_num)
- int [blk_flush](#) (uint32_t block_num)
- int [blk_is_allocated](#) (uint32_t block_num)
- int [blk_mark_allocated](#) (uint32_t block_num)
- int [blk_mark_free](#) (uint32_t block_num)
- int [blk_allocate](#) (uint32_t *block_num)
- int [blk_get_volume_meta](#) ([blk_volume_meta_t](#) *meta)
- int [blk_set_volume_meta](#) (const [blk_volume_meta_t](#) *meta)
- int [blk_is_valid](#) (uint32_t block_num)
- uint32_t [blk_get_total_blocks](#) (void)
- int [blk_get_meta](#) (uint32_t block_num, [blk_meta_t](#) *meta)
- int [blk_set_meta](#) (uint32_t block_num, const [blk_meta_t](#) *meta)
- [__attribute__](#) ((weak))

Variables

- static uint64_t [crc64_table](#) [256]
- static int [crc64_initd](#) = 0
- struct {
 - [VM](#) * [vm](#)
 - struct [blkio_dev](#) * [disk_dev](#)
 - uint8_t * [ram_base](#)
 - size_t [ram_size](#)
 - uint8_t [dirty_ram](#) [[BLK_RAM_BLOCKS](#)]
 - [blk_volume_meta_t](#) [vol_meta](#)
 - uint8_t [vol_meta_dirty](#)
 - uint32_t [total_blkio_blocks_1k](#)
 - uint8_t * [bam](#)
 - size_t [bam_bytes](#)
 - uint8_t [bam_dirty](#)
 - uint32_t [devblock_base_4k](#)
 - [cache_slot_t](#) [cache](#) [8]
 - uint32_t [ram_user](#)
 - uint64_t [total_user_lbn](#)
 - int [initialized](#)
- } [g](#) = {0}

7.69.1 Macro Definition Documentation

7.69.1.1 BLK_DISK_SYS_RESERVED

```
#define BLK_DISK_SYS_RESERVED 32 /* DISK: hide 1024..1055 */
```

7.69.1.2 BLK_FORTH_SYS_RESERVED

```
#define BLK_FORTH_SYS_RESERVED 32 /* RAM: hide 0..31; user RAM LBN 0..991 */
```

7.69.1.3 DISK_CACHE_SLOTS

```
#define DISK_CACHE_SLOTS 8
```

7.69.1.4 META_PER_BLOCK

```
#define META_PER_BLOCK (META_REGION_SIZE / BLK_PACK_RATIO) /* 341 */
```

7.69.1.5 META_REGION_OFFSET

```
#define META_REGION_OFFSET (BLK_PACK_RATIO * BLK_FORTH_SIZE) /* 3072 */
```

7.69.1.6 META_REGION_SIZE

```
#define META_REGION_SIZE (BLK_DEVICE_SECTOR - META_REGION_OFFSET) /* 1024 */
```

7.69.2 Function Documentation

7.69.2.1 __attribute__()

```
__attribute__ (
    (weak) )
```

References [blk_subsys_attach_device\(\)](#).

Here is the call graph for this function:

7.69.2.2 bam_clr()

```
static void bam_clr (
    uint8_t * bm,
    uint64_t idx ) [inline], [static]
```

Referenced by [blk_mark_free\(\)](#).

Here is the caller graph for this function:

7.69.2.3 bam_flush_to_disk()

```
static int bam_flush_to_disk (
    void ) [static]
```

References [BLK_EIO](#), [BLK_OK](#), [blkio_flush\(\)](#), [BLKIO_OK](#), [blkio_write\(\)](#), and [g](#).

Referenced by [blk_flush\(\)](#), and [blk_format_or_load_v2\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.4 bam_load_from_disk()

```
static int bam_load_from_disk (
    void ) [static]
```

References [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_OK](#), [BLKIO_OK](#), [blkio_read\(\)](#), and [g](#).

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.5 bam_set()

```
static void bam_set (
    uint8_t * bm,
    uint64_t idx ) [inline], [static]
```

Referenced by [blk_allocate\(\)](#), [blk_format_or_load_v2\(\)](#), [blk_mark_allocated\(\)](#), and [blk_update\(\)](#).

Here is the caller graph for this function:

7.69.2.6 bam_test()

```
static int bam_test (
    const uint8_t * bm,
    uint64_t idx ) [inline], [static]
```

Referenced by [blk_allocate\(\)](#), [blk_is_allocated\(\)](#), [blk_mark_allocated\(\)](#), [blk_mark_free\(\)](#), and [blk_update\(\)](#).

Here is the caller graph for this function:

7.69.2.7 blk_allocate()

```
int blk_allocate (
    uint32_t * block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [limit](#).

Here is the call graph for this function:

7.69.2.8 blk_flush()

```
int blk_flush (
    uint32_t block_num )
```

References [bam_flush_to_disk\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_FORTH_SYS_RESERVED](#), [BLK_OK](#), [blkio_flush\(\)](#), [cache_writeback\(\)](#), [cache_slot_t::devblock](#), [cache_slot_t::dirty](#), [DISK_CACHE_SLOTS](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [cache_slot_t::meta_dirty](#), and [cache_slot_t::valid](#).

Referenced by [blk_subsys_shutdown\(\)](#), [block_word_flush\(\)](#), and [save_all_buffers\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.9 blk_format_or_load_v2()

```
static int blk_format_or_load_v2 (
    void ) [static]
```

References [bam_flush_to_disk\(\)](#), [bam_load_from_disk\(\)](#), [bam_set\(\)](#), [BLK_DEVICE_SECTOR](#), [BLK_DISK_SYS_RESERVED](#), [BLK_ENODEV](#), [BLK_FORTH_SYS_RESERVED](#), [blk_get_timestamp\(\)](#), [BLK_OK](#), [blkio_flush\(\)](#), [blkio_write\(\)](#), [choose_B\(\)](#), [compute_totals_from_B\(\)](#), [g](#), [read_header_4k\(\)](#), [udiv_floor\(\)](#), [volmeta_from_buf\(\)](#), [volmeta_to_buf\(\)](#), and [write_header_4k\(\)](#).

Referenced by [blk_subsys_attach_device\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.10 blk_get_buffer()

```
uint8_t * blk_get_buffer (
    uint32_t block_num,
    int writable )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [cache_load_devblock\(\)](#), [cache_slot_t::data](#), [cache_slot_t::dirty](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), and [lbn_in_range\(\)](#).

Referenced by [blk_get_empty_buffer\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [capsule_clear_blocks\(\)](#), [copy_block_to_ram\(\)](#), [empty_all_buffers\(\)](#), [get_block_buffer\(\)](#), [line_ptr\(\)](#), [starforth_word_init\(\)](#), and [write_ramdrive_block\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.11 blk_get_empty_buffer()

```
uint8_t * blk_get_empty_buffer (
    uint32_t block_num )
```

References [BLK_FORTH_SIZE](#), and [blk_get_buffer\(\)](#).

Referenced by [get_empty_buffer\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.12 blk_get_meta()

```
int blk_get_meta (
    uint32_t block_num,
    blk_meta_t * meta )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [blk_meta_t::magic](#), and [cache_slot_t::meta](#).

Here is the call graph for this function:

7.69.2.13 blk_get_timestamp()

```
static uint64_t blk_get_timestamp (
    void ) [inline], [static]
```

Get current timestamp with RTC fallback to monotonic clock.

Uses real-time clock (wall-clock) if available, otherwise falls back to monotonic clock. This ensures timestamps work on both POSIX and L4Re platforms, even when RTC capability is unavailable.

Returns

Nanoseconds since epoch (realtime) or boot (monotonic fallback)

References [sf_has_rtc\(\)](#), [sf_monotonic_ns\(\)](#), and [sf_realtime_ns\(\)](#).

Referenced by [blk_format_or_load_v2\(\)](#), [blk_subsys_attach_device\(\)](#), and [blk_update\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.14 blk_get_total_blocks()

```
uint32_t blk_get_total_blocks (
    void )
```

References [g](#).

Referenced by [empty_all_buffers\(\)](#).

Here is the caller graph for this function:

7.69.2.15 blk_get_volume_meta()

```
int blk_get_volume_meta (
    blk_volume_meta_t * meta )
```

References [BLK_EINVAL](#), [BLK_OK](#), and [g](#).

7.69.2.16 blk_is_allocated()

```
int blk_is_allocated (
    uint32_t block_num )
```

References [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.69.2.17 blk_is_valid()

```
int blk_is_valid (
    uint32_t block_num )
```

References [g](#), and [lbn_in_range\(\)](#).

Referenced by [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), and [line_ptr\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.18 blk_mark_allocated()

```
int blk_mark_allocated (
    uint32_t block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.69.2.19 blk_mark_free()

```
int blk_mark_free (
    uint32_t block_num )
```

References [bam_clr\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [g](#), and [lbn_in_range\(\)](#).

Here is the call graph for this function:

7.69.2.20 blk_set_meta()

```
int blk_set_meta (
    uint32_t block_num,
    const blk_meta_t * meta )
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [cache_slot_t::meta](#), and [cache_slot_t::meta_dirty](#).

Here is the call graph for this function:

7.69.2.21 blk_set_volume_meta()

```
int blk_set_volume_meta (
    const blk_volume_meta_t * meta )
```

References [BLK_EINVAL](#), [BLK_OK](#), and [g](#).

7.69.2.22 blk_subsys_attach_device()

```
int blk_subsys_attach_device (
    struct blkio_dev * dev )
```

References [BLK_EINVAL](#), [BLK_EIO](#), [BLK_ENODEV](#), [blk_format_or_load_v2\(\)](#), [blk_get_timestamp\(\)](#), [BLK_OK](#), [blkio_info\(\)](#), [BLKIO_OK](#), [g](#), [LOG_INFO](#), [log_message\(\)](#), and [blkio_info_t::total_blocks](#).

Referenced by [__attribute__\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.23 blk_subsys_init()

```
int blk_subsys_init (
    VM * vm,
    uint8_t * ram_base,
    size_t ram_size )
```

References [BLK_DISK_SYS_RESERVED](#), [BLK_EINVAL](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [BLK_OK](#), [BLK_RAM_BLOCKS](#), [g](#), [ram_base](#), [ram_size](#), and [vm](#).

Referenced by [init_vm_and_subsystem\(\)](#).

Here is the caller graph for this function:

7.69.2.24 blk_subsys_shutdown()

```
int blk_subsys_shutdown (
    void )
```

References [blk_flush\(\)](#), [BLK_OK](#), [blkio_flush\(\)](#), [g](#), [volmeta_to_buf\(\)](#), and [write_header_4k\(\)](#).

Here is the call graph for this function:

7.69.2.25 blk_update()

```
int blk_update (
    uint32_t block_num )
```

References [bam_set\(\)](#), [bam_test\(\)](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_EIO](#), [BLK_ENODEV](#), [BLK_ERANGE](#), [BLK_FORTH_SIZE](#), [BLK_FORTH_SYS_RESERVED](#), [blk_get_timestamp\(\)](#), [BLK_OK](#), [cache_load_devblock\(\)](#), [blk_meta_t::checksum](#), [compute_crc64\(\)](#), [blk_meta_t::created_time](#), [cache_slot_t::data](#), [cache_slot_t::dirty](#), [fblock_pack_offset\(\)](#), [fblock_to_devblock\(\)](#), [g](#), [lbn_in_range\(\)](#), [blk_meta_t::magic](#), [cache_slot_t::meta](#), [cache_slot_t::meta_dirty](#), and [blk_meta_t::modified_time](#).

Referenced by [block_word_update\(\)](#), [capsule_clear_blocks\(\)](#), [copy_block_to_ram\(\)](#), [mark_buffer_dirty\(\)](#), and [write_ramdrive_block\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.26 cache_get_slot()

```
static cache_slot_t * cache_get_slot (
    uint32_t dev4k ) [static]
```

References [cache_writeback\(\)](#), [cache_slot_t::devblock](#), [DISK_CACHE_SLOTS](#), [g](#), and [cache_slot_t::valid](#).
Referenced by [cache_load_devblock\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.27 cache_load_devblock()

```
static cache_slot_t * cache_load_devblock (
    uint32_t dev4k ) [static]
```

References [BLK_PACK_RATIO](#), [cache_get_slot\(\)](#), [cache_slot_t::data](#), [cache_slot_t::loaded](#), [blk_meta_t::magic](#), [cache_slot_t::meta](#), [meta_from_slice\(\)](#), [META_PER_BLOCK](#), [META_REGION_OFFSET](#), and [read_devblock_4k\(\)](#).

Referenced by [blk_get_buffer\(\)](#), [blk_get_meta\(\)](#), [blk_set_meta\(\)](#), and [blk_update\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.28 cache_writeback()

```
static int cache_writeback (
    cache_slot_t * s ) [static]
```

References [BLK_OK](#), [BLK_PACK_RATIO](#), [cache_slot_t::data](#), [cache_slot_t::devblock](#), [cache_slot_t::dirty](#), [g](#), [cache_slot_t::meta](#), [cache_slot_t::meta_dirty](#), [META_PER_BLOCK](#), [META_REGION_OFFSET](#), [meta_to_slice\(\)](#), and [write_devblock_4k\(\)](#).

Referenced by [blk_flush\(\)](#), and [cache_get_slot\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.29 choose_B()

```
static uint32_t choose_B (
    uint64_t total_devblocks_4k ) [static]
```

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the caller graph for this function:

7.69.2.30 compute_crc64()

```
static uint64_t compute_crc64 (
    const uint8_t * data,
    size_t len ) [inline], [static]
```

References [crc64_init\(\)](#), [crc64_inited](#), and [crc64_table](#).

Referenced by [blk_update\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.31 compute_totals_from_B()

```
static void compute_totals_from_B (
    blk_volume_meta_t * m ) [static]
```

References [blk_volume_meta_t::bam_devblocks](#), [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [blk_volume_meta_t::first_free](#), [blk_volume_meta_t::last_allocated](#), [blk_volume_meta_t::total_blocks](#), [blk_volume_meta_t::total_devblocks](#), and [blk_volume_meta_t::tracked_blocks](#).

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the caller graph for this function:

7.69.2.32 crc64_init()

```
static void crc64_init (
    void ) [static]
```

References [crc64_inited](#), and [crc64_table](#).

Referenced by [compute_crc64\(\)](#).

Here is the caller graph for this function:

7.69.2.33 devblock4k_to_lba1k()

```
static uint32_t devblock4k_to_lba1k (
    uint32_t dev4k ) [inline], [static]
```

References [g](#).

Referenced by [read_devblock_4k\(\)](#), and [write_devblock_4k\(\)](#).

Here is the caller graph for this function:

7.69.2.34 fblock_pack_offset()

```
static uint32_t fblock_pack_offset (
    uint32_t phys_block ) [inline], [static]
```

References [BLK_DISK_START](#), and [BLK_PACK_RATIO](#).

Referenced by [blk_get_buffer\(\)](#), [blk_get_meta\(\)](#), [blk_set_meta\(\)](#), and [blk_update\(\)](#).

Here is the caller graph for this function:

7.69.2.35 fblock_to_devblock()

```
static uint32_t fblock_to_devblock (
    uint32_t phys_block ) [inline], [static]
```

References [BLK_DISK_START](#), and [BLK_PACK_RATIO](#).

Referenced by [blk_flush\(\)](#), [blk_get_buffer\(\)](#), [blk_get_meta\(\)](#), [blk_set_meta\(\)](#), and [blk_update\(\)](#).

Here is the caller graph for this function:

7.69.2.36 lbn_in_range()

```
static int lbn_in_range (
    uint32_t lbn ) [inline], [static]
```

References [g](#).

Referenced by [blk_flush\(\)](#), [blk_get_buffer\(\)](#), [blk_get_meta\(\)](#), [blk_is_allocated\(\)](#), [blk_is_valid\(\)](#), [blk_mark_allocated\(\)](#), [blk_mark_free\(\)](#), [blk_set_meta\(\)](#), and [blk_update\(\)](#).

Here is the caller graph for this function:

7.69.2.37 lbn_to_pbn()

```
static uint32_t lbn_to_pbn (
    uint32_t lbn ) [inline], [static]
```

References [BLK_DISK_START](#), [BLK_DISK_SYS_RESERVED](#), [BLK_FORTH_SYS_RESERVED](#), and [g](#).

7.69.2.38 meta_from_slice()

```
static void meta_from_slice (
    blk_meta_t * dst,
    const uint8_t * slice ) [static]
```

References [META_PER_BLOCK](#), and [minzu\(\)](#).

Referenced by [cache_load_devblock\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.39 meta_to_slice()

```
static void meta_to_slice (
    const blk_meta_t * src,
    uint8_t * slice ) [static]
```

References [META_PER_BLOCK](#), and [minzu\(\)](#).

Referenced by [cache_writeback\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.40 minzu()

```
static size_t minzu (
    size_t a,
    size_t b ) [inline], [static]
```

Referenced by [meta_from_slice\(\)](#), [meta_to_slice\(\)](#), [volmeta_from_buf\(\)](#), and [volmeta_to_buf\(\)](#).

Here is the caller graph for this function:

7.69.2.41 read_devblock_4k()

```
static int read_devblock_4k (
    uint32_t dev4k,
    uint8_t * buf4k ) [static]
```

References [base](#), [BLK_OK](#), [BLKIO_OK](#), [blkio_read\(\)](#), [devblock4k_to_lba1k\(\)](#), [g](#), [log_message\(\)](#), and [LOG_WARN](#).

Referenced by [cache_load_devblock\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.42 read_header_4k()

```
static int read_header_4k (
    uint8_t * buf4k ) [static]
```

References [BLK_OK](#), [BLKIO_OK](#), [blkio_read\(\)](#), and [g](#).

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.43 udiv_floor()

```
static uint32_t udiv_floor (
    uint32_t a,
    uint32_t b ) [inline], [static]
```

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the caller graph for this function:

7.69.2.44 volmeta_from_buf()

```
static void volmeta_from_buf (
    blk_volume_meta_t * out,
    const uint8_t * buf ) [inline], [static]
```

References [BLK_DEVICE_SECTOR](#), and [minzu\(\)](#).

Referenced by [blk_format_or_load_v2\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.45 volmeta_to_buf()

```
static void volmeta_to_buf (
    const blk_volume_meta_t * in,
    uint8_t * buf ) [inline], [static]
```

References [BLK_DEVICE_SECTOR](#), and [minzu\(\)](#).

Referenced by [blk_format_or_load_v2\(\)](#), and [blk_subsys_shutdown\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.46 write_devblock_4k()

```
static int write_devblock_4k (
    uint32_t dev4k,
    const uint8_t * buf4k ) [static]
```

References [base](#), [BLK_EIO](#), [BLK_OK](#), [BLKIO_OK](#), [blkio_write\(\)](#), [devblock4k_to_lba1k\(\)](#), [g](#), [LOG_ERROR](#), and [log_message\(\)](#).

Referenced by [cache_writeback\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.2.47 write_header_4k()

```
static int write_header_4k (
    const uint8_t * buf4k ) [static]
```

References [BLK_EIO](#), [BLK_OK](#), [BLKIO_OK](#), [blkio_write\(\)](#), and [g](#).

Referenced by [blk_format_or_load_v2\(\)](#), and [blk_subsys_shutdown\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.69.3 Variable Documentation

7.69.3.1 bam

```
uint8_t* bam
```

7.69.3.2 bam_bytes

```
size_t bam_bytes
```

7.69.3.3 bam_dirty

```
uint8_t bam_dirty
```

7.69.3.4 cache

```
cache_slot_t cache[8]
```

Referenced by [extract_cache_metrics\(\)](#), [hotwords_cache_cleanup\(\)](#), [hotwords_cache_init\(\)](#), [hotwords_cache_lookup\(\)](#), [hotwords_cache_promote\(\)](#), and [hotwords_cache_set_enabled\(\)](#).

7.69.3.5 crc64_init

```
int crc64_init = 0 [static]
```

Referenced by [compute_crc64\(\)](#), and [crc64_init\(\)](#).

7.69.3.6 crc64_table

```
uint64_t crc64_table[256] [static]
```

Referenced by [compute_crc64\(\)](#), and [crc64_init\(\)](#).

7.69.3.7 devblock_base_4k

```
uint32_t devblock_base_4k
```

7.69.3.8 dirty_ram

```
uint8_t dirty_ram[BLK_RAM_BLOCKS]
```

7.69.3.9 disk_dev

```
struct blkio_dev* disk_dev
```

7.69.3.10 [struct]

```
struct { ... } g [static]
```

Referenced by [bam_flush_to_disk\(\)](#), [bam_load_from_disk\(\)](#), [blk_allocate\(\)](#), [blk_flush\(\)](#), [blk_format_or_load_v2\(\)](#), [blk_get_buffer\(\)](#), [blk_get_meta\(\)](#), [blk_get_total_blocks\(\)](#), [blk_get_volume_meta\(\)](#), [blk_is_allocated\(\)](#), [blk_is_valid\(\)](#), [blk_mark_allocated\(\)](#), [blk_mark_free\(\)](#), [blk_set_meta\(\)](#), [blk_set_volume_meta\(\)](#), [blk_subsys_attach_device\(\)](#), [blk_subsys_init\(\)](#), [blk_subsys_shutdown\(\)](#), [blk_update\(\)](#), [cache_get_slot\(\)](#), [cache_writeback\(\)](#), [devblock4k_to_lba1k\(\)](#), [lbn_in_range\(\)](#), [lbn_to_pbn\(\)](#), [read_devblock_4k\(\)](#), [read_header_4k\(\)](#), [write_devblock_4k\(\)](#), and [write_header_4k\(\)](#).

7.69.3.11 initialized

`int initialized`

Referenced by [init_vocabulary_system\(\)](#).

7.69.3.12 ram_base

`uint8_t* ram_base`

Referenced by [blk_subsys_init\(\)](#), [blkio_factory_open\(\)](#), and [init_vm_and_subsystem\(\)](#).

7.69.3.13 ram_size

`size_t ram_size`

Referenced by [blk_subsys_init\(\)](#), and [init_vm_and_subsystem\(\)](#).

7.69.3.14 ram_user

`uint32_t ram_user`

7.69.3.15 total_blkio_blocks_1k

`uint32_t total_blkio_blocks_1k`

7.69.3.16 total_user_lbn

`uint64_t total_user_lbn`

7.69.3.17 vm

`VM* vm`

Referenced by [__attribute__\(\)](#), [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [assert_heartbeat_safe\(\)](#), [assert_stack_depth\(\)](#), [assert_stack_top\(\)](#), [assert_vm_error\(\)](#), [blk_subsys_init\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_empty_buffers\(\)](#), [block_word_flush\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_save_buffers\(\)](#), [block_word_scr\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), [capsule_dict_hash_hook\(\)](#), [capsule_exec_hook\(\)](#), [capsule_exec_init\(\)](#), [capsule_vm_alloc_hook\(\)](#), [capture_exec_result\(\)](#), [cf_epoch_sync\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_branch\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_EXIT\(\)](#), [control_forth_I\(\)](#), [control_forth_if\(\)](#), [control_forth_J\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_leave\(\)](#), [control_forth_runtime_loop\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [control_forth_then\(\)](#), [control_forth_UNLOOP\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [current_base\(\)](#), [current_scr\(\)](#), [dbg_get_base\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_bracket_compile\(\)](#), [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_does\(\)](#), [defining_word_immediate\(\)](#), [defining_word_left_bracket\(\)](#), [defining_word_literal\(\)](#), [defining_word_right_bracket\(\)](#), [defining_word_semicolon\(\)](#), [defining_word_state\(\)](#), [defining_word_variable\(\)](#), [dict_adaptive_optimization_pass\(\)](#), [dict_find_word_heat_aware\(\)](#), [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_hidden\(\)](#), [dictionary_m_word_interpret\(\)](#), [dictionary_m_word_left_bracket\(\)](#), [dictionary_m_word_lfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_right_bracket\(\)](#), [dictionary_m_word_smudge\(\)](#), [dictionary_m_word_state\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_forget\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [dictionary_word_pad\(\)](#), [dictionary_word_sp_fetch\(\)](#), [dictionary_word_sp_store\(\)](#), [double_word_2drop\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#),

[double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dabs\(\)](#), [double_word_dmax\(\)](#),
[double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [dump_and_die\(\)](#), [dump_block_preview\(\)](#),
[editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), [editor_word_show\(\)](#), [emit_cell\(\)](#), [empty_all_buffers\(\)](#),
[ensure_input\(\)](#), [ensure_pn\(\)](#), [execute_colon_word\(\)](#), [extract_heat_trajectory\(\)](#), [fail_and_exit\(\)](#), [find_entry_from_name\(\)](#),
[format_word_base\(\)](#), [format_word_begin_conversion\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_decimal\(\)](#),
[format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dot_s\(\)](#), [format_word_dump\(\)](#), [format_word_end_conversion\(\)](#),
[format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_hex\(\)](#), [format_word_hold\(\)](#), [format_word_octal\(\)](#),
[format_word_question\(\)](#), [format_word_sign\(\)](#), [format_word_u_dot\(\)](#), [format_word_u_dot_r\(\)](#), [forth_acl_allow_fetch\(\)](#),
[forth_acl_allow_store\(\)](#), [forth_acl_heat_fetch\(\)](#), [forth_acl_inherit\(\)](#), [forth_acl_init_primitives\(\)](#), [forth_acl_mode_fetch\(\)](#),
[forth_acl_mode_store\(\)](#), [forth_acl_pin\(\)](#), [forth_acl_pinned_query\(\)](#), [forth_acl_ttl_fetch\(\)](#), [forth_acl_ttl_store\(\)](#),
[forth_acl_word_id\(\)](#), [forth_ALL_HEATS\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#),
[forth_DECAY_RATE_FETCH\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#),
[forth_HEAT_FETCH\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_LOOKUP_STRATEGY_FETCH\(\)](#),
[forth_LOOKUP_STRATEGY_STORE\(\)](#), [forth_paren\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [forth_PHYSICS_BUILD_INFO\(\)](#),
[forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#),
[forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_TOGGLE_CACHE\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#),
[forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_ENABLE\(\)](#), [forth_PIPELINING_RESET_ALL\(\)](#),
[forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [forth_PIPELINING_STATS\(\)](#),
[forth_REORG_BUCKETS\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [forth_UNFREEZE_WORD\(\)](#),
[get_block_buffer\(\)](#), [get_empty_buffer\(\)](#), [hash_colon_body\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_emit_tick_row\(\)](#),
[heartbeat_publish_snapshot\(\)](#), [init_block_system\(\)](#), [init_vm_and_subsystem\(\)](#), [init_vocabulary_system\(\)](#),
[io_init\(\)](#), [io_word_emit\(\)](#), [io_word_key\(\)](#), [io_word_question_terminal\(\)](#), [io_word_spaces\(\)](#), [io_word_type\(\)](#),
[io_write_block\(\)](#), [is_valid_dict_entry\(\)](#), [line_ptr\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#),
[logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#),
[logical_word_true\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#),
[logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#),
[main\(\)](#), [mark_buffer_dirty\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetchn\(\)](#),
[memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#),
[memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [metrics_from_vm\(\)](#), [metrics_write_james_law_csv_row\(\)](#),
[mixed_math_word_m_minus\(\)](#), [mixed_math_word_m_plus\(\)](#), [mixed_math_word_m_slash_mod\(\)](#), [mixed_math_word_m_star\(\)](#),
[mixed_math_word_mod\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_slash_mod\(\)](#), [mixed_math_word_star_slash\(\)](#),
[mixed_math_word_star_slash_mod\(\)](#), [perturb_physics\(\)](#), [physics_decay_slope_load\(\)](#), [physics_metadata_apply_linear_decay\(\)](#),
[physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [physics_pre_execute\(\)](#), [pop_xt\(\)](#), [print_number_formatted\(\)](#),
[profiler_analyze_performance\(\)](#), [register_acl_words\(\)](#), [register_arithmetic_words\(\)](#), [register_block_words\(\)](#),
[register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_heat_diagnostic_words\(\)](#), [register_dictionary_manipulation_v](#),
[register_dictionary_words\(\)](#), [register_double_words\(\)](#), [register_editor_words\(\)](#), [register_format_words\(\)](#), [register_forth79_words\(\)](#),
[register_io_words\(\)](#), [register_logical_words\(\)](#), [register_memory_words\(\)](#), [register_mixed_arithmetic_words\(\)](#),
[register_physics_benchmark_words\(\)](#), [register_physics_diagnostic_words\(\)](#), [register_physics_freeze_words\(\)](#),
[register_physics_pipelining_diagnostic_words\(\)](#), [register_return_stack_words\(\)](#), [register_stack_words\(\)](#), [register_starforth_words\(\)](#),
[register_string_words\(\)](#), [register_system_words\(\)](#), [register_vocabulary_words\(\)](#), [register_word\(\)](#), [reset_vm_state\(\)](#),
[restore_dict_state\(\)](#), [restore_physics\(\)](#), [restore_vm_state\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#),
[return_stack_word_to_r\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#),
[run_acl_words_tests\(\)](#), [run_arithmetic_words_tests\(\)](#), [run_benchmark_mode\(\)](#), [run_block_words_tests\(\)](#),
[run_break_me_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#),
[run_dictionary_words_tests\(\)](#), [run_doe_experiment\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#),
[run_integration_tests\(\)](#), [run_io_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#),
[run_memory_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_module_tests\(\)](#), [run_optional_tests\(\)](#),
[run_return_stack_words_tests\(\)](#), [run_single_test\(\)](#), [run_stack_words_tests\(\)](#), [run_starforth_words_tests\(\)](#),
[run_stress_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#),
[run_vocabulary_words_tests\(\)](#), [runtime_abortq\(\)](#), [save_all_buffers\(\)](#), [save_dict_state\(\)](#), [save_physics\(\)](#), [save_vm_state\(\)](#),
[set_scr\(\)](#), [set_scr\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), [sk_dict_canonical_hash\(\)](#), [sk_dict_word_count\(\)](#),
[sk_fault_handler\(\)](#), [sk_parity_collect\(\)](#), [sk_repl\(\)](#), [stack_word_depth\(\)](#), [stack_word_drop\(\)](#), [stack_word_dup\(\)](#),
[stack_word_minus_rot\(\)](#), [stack_word_over\(\)](#), [stack_word_pick\(\)](#), [stack_word_question_dup\(\)](#), [stack_word_roll\(\)](#),
[stack_word_rot\(\)](#), [stack_word_swap\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_execution_heat_store\(\)](#),
[starforth_word_init\(\)](#), [starforth_word_paren_dash\(\)](#), [starforth_word_random\(\)](#), [starforth_word_reset_execution_heat\(\)](#),
[starforth_word_seed\(\)](#), [starforth_word_top_words\(\)](#), [starforth_word_version\(\)](#), [starforth_word_wait\(\)](#), [starforth_word_word_execution](#),
[starforth_word_zuse_authenticate\(\)](#), [string_runtime_s_quote\(\)](#), [string_word_bl\(\)](#), [string_word_blank\(\)](#), [string_word_bracket_literal\(\)](#),
[string_word_bracket_tick\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#),

string_word_convert(), string_word_count(), string_word_enclose(), string_word_expect(), string_word_minus_trailing(), string_word_number(), string_word_query(), string_word_s_quote(), string_word_scan(), string_word_search(), string_word_skip(), string_word_source(), string_word_span(), string_word_tib(), string_word_tick(), string_word_to_in(), string_word_word(), system_word_79_standard(), system_word_abort(), system_word_abort_quote(), system_word_bye(), system_word_cold(), system_word_nop(), system_word_page(), system_word_quit(), system_word_save_system(), system_word_see(), system_word_vlist(), system_word_warm(), system_word_words(), test_acl_adaptive_ttl(), test_acl_deny_aborts(), test_acl_inherit(), test_acl_pin_blocks_shadow(), test_acl_pin_immutable(), test_acl_strict_mode(), test_acl_ttl_hot_path(), test_emergency_console_bypass(), vaddr_from_ptr(), vm_addr_from_ptr(), vm_addr_ok(), vm_addr_to_block(), vm_align(), vm_allot(), vm_api_create_word(), vm_api_find_word(), vm_api_hide_word(), vm_api_pop(), vm_api_push(), vm_api_rpop(), vm_api_rpush(), vm_api_smudge_word(), vm_assert_interpreter_enabled(), vm_bootstrap_root_vocabulary(), vm_bootstrap_scr(), vm_cleanup(), vm_compile_call(), vm_compile_exit(), vm_compile_literal(), vm_compile_word(), vm_create_word(), vm_debug_dump_state(), vm_debug_set_current_vm(), vm_dictionary_acquire_word_id(), vm_dictionary_find_by_func(), vm_dictionary_find_latest_by_func(), vm_dictionary_lookup_by_word(), vm_dictionary_track_entry(), vm_dictionary_untrack_entry(), vm_docol_arm64(), vm_docol_asm(), vm_docol_rv64(), vm_enable_interpreter(), vm_enter_compile_mode(), vm_exit_compile_mode(), vm_find_word(), vm_get_base(), vm_get_block_addr(), vm_heartbeat_publish_snapshot(), vm_heartbeat_run_cycle(), vm_heartbeat_start_thread(), vm_heartbeat_stop_thread(), vm_heartbeat_update_l8(), vm_heartbeat_update_l8(), vm_hide_word(), vm_host(), vm_host_alloc(), vm_host_calloc(), vm_host_free(), vm_init(), vm_init_with_host(), vm_input_ensure(), vm_input_in(), vm_input_load_line(), vm_input_source(), vm_input_span(), vm_input_tib(), vm_interpret(), vm_interpret_word(), vm_load_cell(), vm_load_u8(), vm_make_immediate(), vm_monotonic_ns(), vm_parse_number(), vm_parse_number(), vm_parse_word(), vm_pin_execution_heat(), vm_pop(), vm_pop_asm(), vm_pop_asm(), vm_pop_asm(), vm_pop_fast(), vm_ptr(), vm_push(), vm_push_asm(), vm_push_asm(), vm_push_asm(), vm_push_fast(), vm_repl(), vm_reset_hosted_fake_ns(), vm_rpop(), vm_rpop_asm(), vm_rpop_asm(), vm_rpop_asm(), vm_rpop_fast(), vm_rpush(), vm_rpush_asm(), vm_rpush_asm(), vm_rpush_asm(), vm_rpush_asm(), vm_rpush_fast(), vm_save_registers(), vm_save_registers_arm64(), vm_save_registers_rv64(), vm_set_base(), vm_setup_registers(), vm_setup_registers_arm64(), vm_setup_registers_rv64(), vm_smudge_word(), vm_snapshot_read(), vm_store_cell(), vm_store_u8(), vm_tick(), vm_tick_apply_background_decay(), vm_tick_apply_background_decay(), vm_tick_inference_engine(), vm_tick_slope_validator(), vm_tick_window_tuner(), vm_unpin_execution_heat(), vm_vocabulary_find_word(), vocab_find_word(), vocab_sync_vm_vars(), vocabulary_create_vocabulary_direct(), vocabulary_select_runtime(), vocabulary_word_context(), vocabulary_word_current(), vocabulary_word_definitions(), vocabulary_word_forth(), vocabulary_word_order(), vocabulary_word_paren_find(), vocabulary_word_vocabulary(), word_divide(), and word_divmod().

7.69.3.18 vol_meta

```
blk_volume_meta_t vol_meta
```

7.69.3.19 vol_meta_dirty

```
uint8_t vol_meta_dirty
```

7.70 src/cli.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "cli.h"
#include "version.h"
```

Include dependency graph for cli.c:

Functions

- static int [parse_u32](#) (const char *s, uint32_t *out)
- void [cli_print_usage](#) (const char *program_name)

Print usage/help information.
- [CLConfig cli_parse](#) (int argc, char *argv[])

Parse command-line arguments into [CLConfig](#).

7.70.1 Function Documentation

7.70.1.1 cli_parse()

```
CLConfig cli_parse (
    int argc,
    char * argv[] )
```

Parse command-line arguments into [CLConfig](#).

Parameters

<i>argc</i>	Argument count
<i>argv</i>	Argument vector

Returns

[CLConfig](#) struct with parsed options, or exits on `--version/--help`

References [CLConfig::benchmark](#), [CLConfig::benchmark_iterations](#), [CLConfig::break_me](#), [cli_print_usage\(\)](#), [CLConfig::disk_img_path](#), [CLConfig::doe_experiment](#), [CLConfig::fail_fast](#), [HEARTBEAT_LOG_FULL](#), [CLConfig::heartbeat_log_mo](#), [HEARTBEAT_LOG_OFF](#), [HEARTBEAT_LOG_SUMMARY](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [CLConfig::log_level](#), [CLConfig::log_level_explicitly_set](#), [LOG_NONE](#), [LOG_TEST](#), [LOG_WARN](#), [mode_str\(\)](#), [parse_u32\(\)](#), [CLConfig::ram_disk_mb](#), [CLConfig::run_tests](#), [CLConfig::run_tests_flag_observed](#), and [CLConfig::script_mode](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.70.1.2 cli_print_usage()

```
void cli_print_usage (
    const char * program_name )
```

Print usage/help information.

Parameters

<i>program_name</i>	Name of the executable
---------------------	------------------------

Referenced by [cli_parse\(\)](#).

Here is the caller graph for this function:

7.70.1.3 parse_u32()

```
static int parse_u32 (
    const char * s,
    uint32_t * out ) [static]
```

Referenced by [cli_parse\(\)](#).

Here is the caller graph for this function:

7.71 src/dictionary_heat_optimization.c File Reference

```
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include "vm.h"
#include "rolling_window_of_truth.h"
#include "dictionary_heat_optimization.h"
#include "log.h"
#include "platform_time.h"
```

Include dependency graph for `dictionary_heat_optimization.c`:

Functions

- static int [compare_heat_ascending](#) (const void *a, const void *b)
Compare function for qsort (ascending order)
- void [dict_update_heat_percentiles](#) (VM *vm)
Calculate heat percentile thresholds for all words in dictionary.
- DictEntry * [dict_find_word_heat_aware](#) (VM *vm, const char *name, size_t len)
Find word with heat-aware search strategy.
- static int [compare_dict_entry_heat_desc](#) (const void *a, const void *b)
Compare function for bucket reordering (descending heat order)
- void [dict_reorganize_buckets_by_heat](#) (VM *vm)
Reorganize dictionary buckets by inference-driven priorities.
- void [dict_adaptive_optimization_pass](#) (VM *vm)
Periodically reoptimize dictionary based on latest heat patterns.

7.71.1 Function Documentation

7.71.1.1 [compare_dict_entry_heat_desc\(\)](#)

```
static int compare_dict_entry_heat_desc (
    const void * a,
    const void * b ) [static]
```

Compare function for bucket reordering (descending heat order)

References [DictEntry::execution_heat](#).

Referenced by [dict_reorganize_buckets_by_heat\(\)](#).

Here is the caller graph for this function:

7.71.1.2 [compare_heat_ascending\(\)](#)

```
static int compare_heat_ascending (
    const void * a,
    const void * b ) [static]
```

Compare function for qsort (ascending order)

Referenced by [dict_update_heat_percentiles\(\)](#).

Here is the caller graph for this function:

7.71.1.3 [dict_adaptive_optimization_pass\(\)](#)

```
void dict_adaptive_optimization_pass (
    VM * vm )
```

Periodically reoptimize dictionary based on latest heat patterns.

Called every ~1 second or every N executions to adapt lookup strategy to current workload patterns from rolling window.

References [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [VM::last_bucket_reorg_ns](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::lookup_strategy](#), [VM::rolling_window](#), [rolling_window_measure_diversity\(\)](#), [sf_monotonic_ns\(\)](#), and [vm](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.71.1.4 [dict_find_word_heat_aware\(\)](#)

```
DictEntry * dict_find_word_heat_aware (
    VM * vm,
    const char * name,
    size_t len )
```

Find word with heat-aware search strategy.

Instead of newest-first, searches buckets in order:

1. Top 25% by heat (likely hits)
2. Middle 50% by heat
3. Bottom 25% (rarely executed)

References [DictEntry::execution_heat](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_75th](#), [name](#), [DictEntry::name](#), [DictEntry::name_len](#), [sf_fc_count](#), [sf_fc_list](#), and [vm](#).

Referenced by [vm_find_word\(\)](#).

Here is the caller graph for this function:

7.71.1.5 dict_reorganize_buckets_by_heat()

```
void dict_reorganize_buckets_by_heat (
    VM * vm )
```

Reorganize dictionary buckets by inference-driven priorities.

Uses rolling window co-execution patterns to reorder words within buckets. Hot/frequently co-executing words are placed first for faster lookup.

References [compare_dict_entry_heat_desc\(\)](#), [VM::dict_lock](#), [VM::last_bucket_reorg_ns](#), [LOG_DEBUG](#), [log_message\(\)](#), [LOG_WARN](#), [sf_fc_cap](#), [sf_fc_count](#), [sf_fc_list](#), [sf_monotonic_ns\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), and [forth_REORG_BUCKETS\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.71.1.6 dict_update_heat_percentiles()

```
void dict_update_heat_percentiles (
    VM * vm )
```

Calculate heat percentile thresholds for all words in dictionary.

Computes 25th, 50th, 75th percentiles of execution_heat across all dictionary entries. Used to partition lookup search order.

References [compare_heat_ascending\(\)](#), [VM::dict_lock](#), [DICTIONARY_SIZE](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::latest](#), [LOG_DEBUG](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#), [forth_REORG_BUCKETS\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72 src/dictionary_management.c File Reference

```
#include "../include/vm.h"
#include "../include/io.h"
#include "../include/log.h"
#include "../include/physics_metadata.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/physics_pipelining_metrics.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/platform_alloc.h"
#include <string.h>
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for dictionary_management.c:

Macros

- [#define SF_FC_BUCKETS](#) 256

Functions

- static uint32_t [vm_dictionary_acquire_word_id](#) (VM *vm)
- void [vm_dictionary_track_entry](#) (VM *vm, DictEntry *entry)
- void [vm_dictionary_untrack_entry](#) (VM *vm, DictEntry *entry)

- [DictEntry](#) * [vm_dictionary_lookup_by_word_id](#) ([VM](#) *[vm](#), [uint32_t](#) [word_id](#))
- static void * [sf_xrealloc](#) (void *[p](#), [size_t](#) [nbytes](#))
- static void [sf_fc_clear_all](#) (void)
- static int [sf_fc_reserve](#) ([size_t](#) [i](#), [size_t](#) [need](#))
- static void [sf_rebuild_fc_index](#) ([VM](#) *[vm](#))
- static void [sf_try_fast_append](#) ([VM](#) *[vm](#))
- [DictEntry](#) * [vm_find_word](#) ([VM](#) *[vm](#), const char *[name](#), [size_t](#) [len](#))
Finds a word in the dictionary by name.
- [DictEntry](#) * [vm_create_word](#) ([VM](#) *[vm](#), const char *[name](#), [size_t](#) [len](#), [word_func_t](#) [func](#))
Creates a new word in the dictionary.
- [DictEntry](#) * [vm_dictionary_find_by_func](#) ([VM](#) *[vm](#), [word_func_t](#) [func](#))
Finds a word by its function pointer.
- [DictEntry](#) * [vm_dictionary_find_latest_by_func](#) ([VM](#) *[vm](#), [word_func_t](#) [func](#))
- [cell_t](#) * [vm_dictionary_get_data_field](#) ([DictEntry](#) *[entry](#))
Gets pointer to a word's data field.
- void [vm_hide_word](#) ([VM](#) *[vm](#))
Marks the latest word as hidden.
- void [vm_smudge_word](#) ([VM](#) *[vm](#))
- void [vm_pin_execution_heat](#) ([VM](#) *[vm](#))
- void [vm_unpin_execution_heat](#) ([VM](#) *[vm](#))
- void [vm_dictionary_index_reset](#) (void)
Resets the dictionary index.

Variables

- [DictEntry](#) ** [sf_fc_list](#) [256]
- [size_t](#) [sf_fc_count](#) [256]
- [size_t](#) [sf_fc_cap](#) [256]
- static [DictEntry](#) * [sf_cached_latest](#) = NULL

7.72.1 Macro Definition Documentation

7.72.1.1 SF_FC_BUCKETS

```
#define SF_FC_BUCKETS 256
```

7.72.2 Function Documentation

7.72.2.1 sf_fc_clear_all()

```
static void sf_fc_clear_all (
    void ) [static]
```

References [SF_FC_BUCKETS](#), [sf_fc_cap](#), [sf_fc_count](#), [sf_fc_list](#), and [sf_free\(\)](#).

Referenced by [vm_dictionary_index_reset\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.2 sf_fc_reserve()

```
static int sf_fc_reserve (
    size_t i,
    size_t need ) [static]
```

References [sf_fc_cap](#), [sf_fc_list](#), and [sf_xrealloc\(\)](#).

Referenced by [sf_rebuild_fc_index\(\)](#), and [sf_try_fast_append\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.3 sf_rebuild_fc_index()

```
static void sf_rebuild_fc_index (
    VM * vm ) [static]
```

References [VM::latest](#), [DictEntry::name](#), [sf_cached_latest](#), [SF_FC_BUCKETS](#), [sf_fc_count](#), [sf_fc_list](#), [sf_fc_reserve\(\)](#), and [vm](#).

Referenced by [sf_try_fast_append\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.4 sf_try_fast_append()

```
static void sf_try_fast_append (
    VM * vm ) [static]
```

References [VM::latest](#), [DictEntry::link](#), [DictEntry::name](#), [sf_cached_latest](#), [sf_fc_count](#), [sf_fc_list](#), [sf_fc_reserve\(\)](#), [sf_rebuild_fc_index\(\)](#), and [vm](#).

Referenced by [vm_find_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.5 sf_xrealloc()

```
static void * sf_xrealloc (
    void * p,
    size_t nbytes ) [inline], [static]
```

References [sf_free\(\)](#), and [sf_realloc\(\)](#).

Referenced by [sf_fc_reserve\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.6 vm_create_word()

```
DictEntry * vm_create_word (
    VM * vm,
    const char * name,
    size_t len,
    word_func_t func )
```

Creates a new word in the dictionary.

Allocates and initializes a new dictionary entry with the given name and function. The word is added to the dictionary but index update is deferred until next lookup.

Parameters

<i>vm</i>	The virtual machine context
<i>name</i>	Name for the new word
<i>len</i>	Length of the name string
<i>func</i>	Function pointer for the word's behavior

Returns

DictEntry* Pointer to new dictionary entry or NULL on failure

References [ACL_MODE_TTL](#), [ACL_USER_DEFAULT](#), [base](#), [VM::dict_lock](#), [entry](#), [VM::error](#), [func](#), [DictEntry::func](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [name](#), [physics_metadata_apply_seed\(\)](#), [physics_metadata_init\(\)](#), [sf_malloc\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [transition_metrics_init\(\)](#), [vm](#), [vm_dictionary_track_entry\(\)](#), [WORD_ID_INVALID](#), and [WORD_NAME_MAX](#).

Referenced by [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [register_physics_pipelining_diagnostic_w](#), [register_word\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_inherit\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_pin_immutable\(\)](#), [vm_api_create_word\(\)](#), [vm_enter_compile_mode\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.7 vm_dictionary_acquire_word_id()

```
static uint32_t vm_dictionary_acquire_word_id (
    VM * vm ) [static]
```

References [DICTIONARY_SIZE](#), [VM::next_word_id](#), [VM::recycled_word_id_count](#), [VM::recycled_word_ids](#), [vm](#), and [WORD_ID_INVALID](#).

Referenced by [vm_dictionary_track_entry\(\)](#).

Here is the caller graph for this function:

7.72.2.8 vm_dictionary_find_by_func()

```
DictEntry * vm_dictionary_find_by_func (
    VM * vm,
    word_func_t func )
```

Finds a word by its function pointer.

Performs a linear scan of the dictionary to find an entry with matching function.

Parameters

<i>vm</i>	The virtual machine context
<i>func</i>	Function pointer to search for

Returns

DictEntry* Pointer to found dictionary entry or NULL if not found

References [func](#), [VM::latest](#), and [vm](#).

Referenced by [vm_compile_call\(\)](#), and [vm_dictionary_find_latest_by_func\(\)](#).

Here is the caller graph for this function:

7.72.2.9 vm_dictionary_find_latest_by_func()

```
DictEntry * vm_dictionary_find_latest_by_func (
    VM * vm,
    word_func_t func )
```

References [func](#), [vm](#), and [vm_dictionary_find_by_func\(\)](#).

Here is the call graph for this function:

7.72.2.10 vm_dictionary_get_data_field()

```
cell_t * vm_dictionary_get_data_field (
    DictEntry * entry )
```

Gets pointer to a word's data field.

Calculates and returns pointer to the aligned data field following the name field.

Parameters

<i>entry</i>	Dictionary entry to get data field from
--------------	---

Returns

cell_t* Pointer to data field or NULL if entry is NULL

References [base](#), [entry](#), and [name](#).

Referenced by [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_word_forget\(\)](#), [execute_colon_word\(\)](#), [hash_colon_body\(\)](#), [system_word_see\(\)](#), [vm_enter_compile_mode\(\)](#), and [vm_exit_compile_mode\(\)](#).

Here is the caller graph for this function:

7.72.2.11 `vm_dictionary_index_reset()`

```
void vm_dictionary_index_reset (
    void )
```

Resets the dictionary index.

Clears all index data and cached state. Used when switching vocabularies.

References [sf_cached_latest](#), and [sf_fc_clear_all\(\)](#).

Here is the call graph for this function:

7.72.2.12 `vm_dictionary_lookup_by_word_id()`

```
DictEntry * vm_dictionary_lookup_by_word_id (
    VM * vm,
    uint32_t word_id )
```

References [DICTIONARY_SIZE](#), [vm](#), [word_id](#), and [VM::word_id_map](#).

Referenced by [execute_colon_word\(\)](#), [extract_heat_trajectory\(\)](#), [physics_pre_execute\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Here is the caller graph for this function:

7.72.2.13 `vm_dictionary_track_entry()`

```
void vm_dictionary_track_entry (
    VM * vm,
    DictEntry * entry )
```

References [DICTIONARY_SIZE](#), [entry](#), [log_message\(\)](#), [LOG_WARN](#), [vm](#), [vm_dictionary_acquire_word_id\(\)](#), [word_id](#), [WORD_ID_INVALID](#), and [VM::word_id_map](#).

Referenced by [vm_create_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.14 `vm_dictionary_untrack_entry()`

```
void vm_dictionary_untrack_entry (
    VM * vm,
    DictEntry * entry )
```

References [DICTIONARY_SIZE](#), [entry](#), [VM::recycled_word_id_count](#), [VM::recycled_word_ids](#), [vm](#), [word_id](#), [DictEntry::word_id](#), [WORD_ID_INVALID](#), and [VM::word_id_map](#).

Referenced by [dictionary_word_forget\(\)](#).

Here is the caller graph for this function:

7.72.2.15 `vm_find_word()`

```
DictEntry * vm_find_word (
    VM * vm,
    const char * name,
    size_t len )
```

Finds a word in the dictionary by name.

Searches for a word in the dictionary using an optimized first-character index. The search is performed newest-first within the matching first-character bucket.

Parameters

<i>vm</i>	The virtual machine context
<i>name</i>	The name to search for
<i>len</i>	Length of the name string

Returns

DictEntry* Pointer to found dictionary entry or NULL if not found

References [dict_find_word_heat_aware\(\)](#), [VM::dict_lock](#), [HeartbeatState::dictionary_lookups](#), [DictEntry::flags](#), [VM::heartbeat](#), [VM::hotwords_cache](#), [hotwords_cache_lookup\(\)](#), [VM::latest](#), [VM::lookup_strategy](#), [name](#), [DictEntry::name](#), [DictEntry::name_len](#), [sf_cached_latest](#), [sf_fc_count](#), [sf_fc_list](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [sf_try_fast_append\(\)](#), [UNLIKELY](#), [vm](#), [WORD_HIDDEN](#), and [WORD_SMUDGED](#).

Referenced by [acl_recheck\(\)](#), [control_forth_endcase\(\)](#), [control_forth_of\(\)](#), [defining_word_compile\(\)](#), [defining_word_does\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_tick\(\)](#), [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_COMPARE_LOOKUPS\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TO](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [hash_colon_body\(\)](#), [kernel_main\(\)](#), [starforth_word_init\(\)](#), [string_word_s_quote\(\)](#), [system_word_see\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), [vm_api_find_word\(\)](#), [vm_bootstrap_root_vocabulary](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.16 vm_hide_word()

```
void vm_hide_word (
    VM * vm )
```

Marks the latest word as hidden.

Sets the WORD_HIDDEN flag on the most recently defined word.

Parameters

<i>vm</i>	The virtual machine context
-----------	-----------------------------

References [DictEntry::flags](#), [VM::latest](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_HIDDEN](#).

Referenced by [vm_api_hide_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.17 vm_pin_execution_heat()

```
void vm_pin_execution_heat (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_PINNED](#).

Here is the call graph for this function:

7.72.2.18 vm_smudge_word()

```
void vm_smudge_word (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_SMUDGED](#).

Referenced by [vm_api_smudge_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.72.2.19 vm_unpin_execution_heat()

```
void vm_unpin_execution_heat (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_metadata_refresh_state\(\)](#), and [vm](#).

Here is the call graph for this function:

7.72.3 Variable Documentation

7.72.3.1 sf_cached_latest

`DictEntry* sf_cached_latest = NULL [static]`

Referenced by [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), [vm_dictionary_index_reset\(\)](#), and [vm_find_word\(\)](#).

7.72.3.2 sf_fc_cap

`size_t sf_fc_cap[256]`

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [sf_fc_clear_all\(\)](#), and [sf_fc_reserve\(\)](#).

7.72.3.3 sf_fc_count

`size_t sf_fc_count[256]`

Referenced by [dict_find_word_heat_aware\(\)](#), [dict_reorganize_buckets_by_heat\(\)](#), [sf_fc_clear_all\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), and [vm_find_word\(\)](#).

7.72.3.4 sf_fc_list

`DictEntry** sf_fc_list[256]`

Referenced by [dict_find_word_heat_aware\(\)](#), [dict_reorganize_buckets_by_heat\(\)](#), [sf_fc_clear_all\(\)](#), [sf_fc_reserve\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), and [vm_find_word\(\)](#).

7.73 src/doe_metrics.c File Reference

Design of Experiments metrics collection implementation.

```
#include "doe_metrics.h"
#include "physics_hotwords_cache.h"
#include "rolling_window_of_truth.h"
#include "rolling_window_knobs.h"
#include "inference_engine.h"
#include "platform_time.h"
#include <time.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for `doe_metrics.c`:

Functions

- `int32_t metrics_get_cpu_temp_c (void)`
Get current CPU temperature in Celsius.
- `int32_t metrics_get_cpu_freq_mhz (void)`
Get current CPU frequency in MHz.
- `void metrics_get_timestamp (char *buf, size_t bufsize)`
Get current timestamp as ISO 8601 string.
- `static void extract_cache_metrics (const HotwordsCache *cache, DoeMetrics *metrics)`
Extract metrics from VM hotwords cache stats.
- `DoeMetrics metrics_from_vm (VM *vm, uint64_t workload_duration_ns, int32_t cpu_temp_delta_c, int32_t cpu_freq_delta_mhz)`
Extract metrics from entire VM.
- `void metrics_write_csv_header (FILE *out)`
Write CSV header.
- `void metrics_write_csv_row (FILE *out, const DoeMetrics *metrics)`
Write CSV row - MUST match header column order exactly.
- `void metrics_print_text (FILE *out, const DoeMetrics *metrics)`

Print metrics as human-readable text.

- static uint64_t [isqrt](#) (uint64_t n)
Integer square root using Newton's method (pure C99, no math.h)
- void [metrics_write_james_law_csv_row](#) (FILE *out, const [DoeMetrics](#) *metrics, [VM](#) *vm)
Write reduced James Law CSV row (20 columns for window scaling experiment)

Variables

- struct {
 uint64_t [total_lookups](#)
 uint64_t [cache_hits](#)
 uint64_t [bucket_hits](#)
} [physics_global_stats](#)

7.73.1 Detailed Description

Design of Experiments metrics collection implementation.

7.73.2 Function Documentation

7.73.2.1 [extract_cache_metrics\(\)](#)

```
static void extract_cache_metrics (
    const HotwordsCache * cache,
    DoeMetrics * metrics ) [static]
```

Extract metrics from [VM](#) hotwords cache stats.

References [DoeMetrics::bucket_hit_percent](#), [DoeMetrics::bucket_hits](#), [HotwordsStats::bucket_hits](#), [DoeMetrics::bucket_search_latency](#), [HotwordsStats::bucket_search_samples](#), [DoeMetrics::bucket_search_stddev_ns](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [cache](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_percent](#), [HotwordsStats::cache_hit_samples](#), [DoeMetrics::cache_hit_stddev_ns](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [DoeMetrics::cache_hits](#), [HotwordsStats::cache_hits](#), and [HotwordsStats::total_lookups](#).

Referenced by [metrics_from_vm\(\)](#).

Here is the caller graph for this function:

7.73.2.2 [isqrt\(\)](#)

```
static uint64_t isqrt (
    uint64_t n ) [static]
```

Integer square root using Newton's method (pure C99, no math.h)

Referenced by [metrics_write_james_law_csv_row\(\)](#).

Here is the caller graph for this function:

7.73.2.3 [metrics_from_vm\(\)](#)

```
DoeMetrics metrics_from_vm (
    VM * vm,
    uint64_t workload_duration_ns,
    int32_t cpu_temp_delta_c,
    int32_t cpu_freq_delta_mhz )
```

Extract metrics from entire [VM](#).

Extract complete metrics snapshot from [VM](#).

References [DoeMetrics::actual_window_size](#), [DoeMetrics::adaptive_shrink_rate](#), [DoeMetrics::avg_word_heat](#), [DoeMetrics::cache_demotions](#), [DoeMetrics::cache_promotions](#), [DoeMetrics::ci_lower_95](#), [DoeMetrics::ci_upper_95](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DECAY_MIN_INTERVAL](#), [DoeMetrics::decay_min_inter](#), [DECAY_RATE_PER_US_Q16](#), [DoeMetrics::decay_rate_q16](#), [DoeMetrics::decay_slope](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [DoeMetrics::dictionary_lookups](#), [HeartbeatState::dictionary_lookups](#), [DoeMetrics::early_exit_count](#), [HeartbeatState::early_exit_count](#), [RollingWindowOfTruth::effective_window_size](#), [DoeMetrics::enable_hotwords_cache](#),

ENABLE_HOTWORDS_CACHE, DoeMetrics::enable_loop_1_heat_tracking, DoeMetrics::enable_loop_2_rolling_window, DoeMetrics::enable_loop_3_linear_decay, DoeMetrics::enable_loop_4_pipelining, DoeMetrics::enable_loop_5_window_inference, DoeMetrics::enable_loop_6_decay_inference, DoeMetrics::enable_loop_7_adaptive_heartrate, DoeMetrics::enable_pipelining, ENABLE_PIPELINING, HotwordsCache_s::enabled, HotwordsStats::evictions, extract_cache_metrics(), DoeMetrics::final_effective_v, VM::heartbeat, DoeMetrics::heat_cache_demotion_threshold, DoeMetrics::hot_word_count, VM::hotwords_cache, HOTWORDS_EXECUTION_HEAT_THRESHOLD, DoeMetrics::inference_run_count, HeartbeatState::inference_run_count, VM::last_inference_outputs, VM::latest, DoeMetrics::memory_allocated_bytes, VM::pipeline_metrics, DoeMetrics::prefetch_accuracy, DoeMetrics::prefetch_attempts, PipelineGlobalMetrics::prefetch_attempts, DoeMetrics::prefetch_hits, PipelineGlobalMetrics::prefetch, HotwordsStats::promotions, VM::rolling_window, DoeMetrics::rolling_window_size, ROLLING_WINDOW_SIZE, DoeMetrics::rolling_window_width, sf_mutex_lock(), sf_mutex_unlock(), DoeMetrics::speedup_vs_baseline, DoeMetrics::stale_word_count, DoeMetrics::stale_word_ratio, HotwordsCache_s::stats, DoeMetrics::tick_count, HeartbeatState::tick_count, DoeMetrics::tick_target_ns, HeartbeatState::tick_target_ns, DoeMetrics::total_executions, RollingWindowOfTruth::total_executions, DoeMetrics::total_heat, DoeMetrics::total_lookups, HotwordsStats::total_lookups, DoeMetrics::total_runtime_ms, VM::tuning_lock, vm, DoeMetrics::vm_workload_duration_ns_q48, DoeMetrics::window_diversity_per, DoeMetrics::window_final_size_bytes, DoeMetrics::window_tuning_checks, PipelineGlobalMetrics::window_tuning_checks, DoeMetrics::window_variance_q48, InferenceOutputs::window_variance_q48, word_count, DoeMetrics::words_executed, and HeartbeatState::words_executed.

Referenced by [run_doe_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.73.2.4 metrics_get_cpu_freq_mhz()

```
int32_t metrics_get_cpu_freq_mhz (
    void )
```

Get current CPU frequency in MHz.

Get current CPU frequency (MHz)

Referenced by [run_doe_experiment\(\)](#).

Here is the caller graph for this function:

7.73.2.5 metrics_get_cpu_temp_c()

```
int32_t metrics_get_cpu_temp_c (
    void )
```

Get current CPU temperature in Celsius.

Get current CPU temperature (Celsius)

Referenced by [run_doe_experiment\(\)](#).

Here is the caller graph for this function:

7.73.2.6 metrics_get_timestamp()

```
void metrics_get_timestamp (
    char * buf,
    size_t bufsize )
```

Get current timestamp as ISO 8601 string.

Format: "YYYY-MM-DDTHH:MM:SS"

Parameters

<i>buf</i>	Buffer to write timestamp (minimum 32 bytes)
<i>bufsize</i>	Size of buffer

7.73.2.7 metrics_print_text()

```
void metrics_print_text (
    FILE * out,
    const DoeMetrics * metrics )
```

Print metrics as human-readable text.

Outputs metrics in formatted text for debugging/verification.

Parameters

<i>out</i>	Output file stream
<i>metrics</i>	Metrics structure to print

References [DoeMetrics::bucket_hit_percent](#), [DoeMetrics::bucket_hits](#), [DoeMetrics::bucket_search_latency_ns](#), [DoeMetrics::bucket_search_stddev_ns](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_percent](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::cache_hits](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DoeMetrics::decay_slope](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::total_lookups](#), and [DoeMetrics::vm_workload_duration_ns](#).

7.73.2.8 metrics_write_csv_header()

```
void metrics_write_csv_header (
    FILE * out )
```

Write CSV header.

Write CSV header row.

7.73.2.9 metrics_write_csv_row()

```
void metrics_write_csv_row (
    FILE * out,
    const DoeMetrics * metrics )
```

Write CSV row - MUST match header column order exactly.

Write metrics as CSV row.

References [DoeMetrics::actual_window_size](#), [DoeMetrics::adaptive_shrink_rate](#), [DoeMetrics::avg_word_heat](#), [DoeMetrics::bucket_hit_percent](#), [DoeMetrics::bucket_hits](#), [DoeMetrics::bucket_search_latency_ns](#), [DoeMetrics::bucket_search_stddev_ns](#), [DoeMetrics::cache_demotions](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_percent](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::cache_hits](#), [DoeMetrics::cache_promotions](#), [DoeMetrics::ci_lower_95](#), [DoeMetrics::ci_upper_95](#), [DoeMetrics::context_accuracy_percent](#), [DoeMetrics::context_correct](#), [DoeMetrics::context_predictions_total](#), [DoeMetrics::cpu_freq_delta_mhz_q48](#), [DoeMetrics::cpu_temp_delta_c_q48](#), [DoeMetrics::decay_min_interval_ns](#), [DoeMetrics::decay_rate_q16](#), [DoeMetrics::decay_slope](#), [DoeMetrics::dictionary_lookups](#), [DoeMetrics::early_exit_count](#), [DoeMetrics::enable_hotwords_cache](#), [DoeMetrics::enable_loop_1_heat_tracking](#), [DoeMetrics::enable_loop_2_rolling_window](#), [DoeMetrics::enable_loop_3_linear_decay](#), [DoeMetrics::enable_loop_4_pipelining](#), [DoeMetrics::enable_loop_5_window_inference](#), [DoeMetrics::enable_loop_6_decay_inference](#), [DoeMetrics::enable_loop_7_adaptive_heartrate](#), [DoeMetrics::enable_pipelining](#), [DoeMetrics::final_effective_window_size](#), [DoeMetrics::heat_cache_demotion_threshold](#), [DoeMetrics::hot_word_count](#), [DoeMetrics::inference_run_count](#), [DoeMetrics::memory_allocated_bytes](#), [DoeMetrics::prefetch_accuracy_percent](#), [DoeMetrics::prefetch_attempts](#), [DoeMetrics::prefetch_hits](#), [DoeMetrics::rolling_window_size](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::speedup_vs_baseline](#), [DoeMetrics::stale_word_count](#), [DoeMetrics::stale_word_ratio](#), [DoeMetrics::tick_count](#), [DoeMetrics::tick_target_ns](#), [DoeMetrics::total_executions](#), [DoeMetrics::total_heat](#), [DoeMetrics::total_lookups](#), [DoeMetrics::total_runtime_ms](#), [DoeMetrics::vm_workload_duration_ns_q48](#), [DoeMetrics::window_diversity_percent](#), [DoeMetrics::window_final_size_bytes](#), [DoeMetrics::window_tuning_checks](#), [DoeMetrics::window_variance_q48](#), and [DoeMetrics::words_executed](#).

7.73.2.10 metrics_write_james_law_csv_row()

```
void metrics_write_james_law_csv_row (
    FILE * out,
    const DoeMetrics * metrics,
    VM * vm )
```

Write reduced James Law CSV row (20 columns for window scaling experiment)

Write reduced James Law CSV row (20 columns)

This outputs only the subset of metrics needed for the James Law validation experiment. Format matches the header in `run_window_sweep.sh` (minus the 5 script-prepended columns).

Expected columns (20): `workload_duration_ns`, `l8_mode_final`, `lambda_effective`, `K_statistic`, `K_deviation`, `entropy`, `cv`, `temporal_decay`, `stability_score`, `ns_per_word`, `total_runtime_ms`, `window_final`, `hot_word_count`, `avg_heat`, `bucket_mean_K`, `bucket_cv_K`, `bucket_window_var`, `bucket_heat_var`, `bucket_mode_transitions`, `bucket_collapse_flag`

References [DoeMetrics::avg_word_heat](#), [HeartbeatState::bucket_collapse_flag](#), [HeartbeatState::bucket_mode_transitions](#), [HeartbeatState::bucket_sum_heat_variance](#), [HeartbeatState::bucket_sum_K](#), [HeartbeatState::bucket_sum_K_squared](#), [HeartbeatState::bucket_sum_window_variance](#), [HeartbeatState::bucket_tick_count](#), [DoeMetrics::cache_hit_latency_ns](#), [DoeMetrics::cache_hit_stddev_ns](#), [DoeMetrics::decay_slope](#), [DoeMetrics::final_effective_window_size](#), [VM::heartbeat](#), [HEARTBEAT_TICK_BUFFER_SIZE](#), [DoeMetrics::hot_word_count](#), [isqrt\(\)](#), [HeartbeatTickSnapshot::l8_mode](#), [DoeMetrics::rolling_window_size](#), [DoeMetrics::rolling_window_width](#), [DoeMetrics::stale_word_ratio](#), [HeartbeatState::tick_buffer](#), [HeartbeatState::tick_buffer_write_index](#), [DoeMetrics::total_runtime_ms](#), [vm](#), [DoeMetrics::vm_workload_duration_ns_q48](#), [DoeMetrics::window_diversity_percent](#), and [DoeMetrics::words_executed](#).

Here is the call graph for this function:

7.73.3 Variable Documentation

7.73.3.1 bucket_hits

```
uint64_t bucket_hits
```

7.73.3.2 cache_hits

```
uint64_t cache_hits
```

7.73.3.3 [struct]

```
struct { ... } physics_global_stats
```

7.73.3.4 total_lookups

```
uint64_t total_lookups
```

7.74 src/heartbeat_export.c File Reference

```
#include "vm.h"
#include "log.h"
#include "platform_time.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for heartbeat_export.c:

Functions

- void [heartbeat_capture_tick_snapshot](#) ([VM *vm](#), [HeartbeatTickSnapshot *snapshot](#))
Capture current VM metrics into a heartbeat tick snapshot.
- void [heartbeat_emit_tick_row](#) ([VM *vm](#), [HeartbeatTickSnapshot *snapshot](#))
Emit a heartbeat tick snapshot as CSV row to stderr.

7.74.1 Function Documentation

7.74.1.1 heartbeat_capture_tick_snapshot()

```
void heartbeat_capture_tick_snapshot (
    VM * vm,
    HeartbeatTickSnapshot * snapshot )
```

Capture current VM metrics into a heartbeat tick snapshot.

Collects 11 metrics per tick for multivariate dynamics analysis. Called every heartbeat tick to record VM state.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>snapshot</i>	Pointer to snapshot structure to populate

References [HeartbeatTickSnapshot::actual_window_size](#), [HeartbeatTickSnapshot::avg_word_heat](#), [HeartbeatTickSnapshot::bucket](#), [HeartbeatTickSnapshot::cache_hits_delta](#), [RollingWindowOfTruth::effective_window_size](#), [HeartbeatTickSnapshot::elapsed_ns](#), [entry](#), [HeartbeatTickSnapshot::estimated_jitter_ns](#), [VM::heartbeat](#), [HEARTBEAT_TICK_NS](#), [HeartbeatTickSnapshot::hot_word_count](#), [VM::latest](#), [HeartbeatTickSnapshot::predicted_label_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [HeartbeatState::run_start_ns](#), [sf_monotonic_ns\(\)](#), [HeartbeatState::tick_count_total](#), [HeartbeatTickSnapshot::tick_interval_ns](#), [HeartbeatTickSnapshot::tick_number](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [vm](#), [HeartbeatTickSnapshot::window_width](#), [word_count](#), and [HeartbeatTickSnapshot::word_executions_delta](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.74.1.2 heartbeat_emit_tick_row()

```
void heartbeat_emit_tick_row (
    VM * vm,
    HeartbeatTickSnapshot * snapshot )
```

Emit a heartbeat tick snapshot as CSV row to stderr.

Outputs 11 comma-separated values (no header) for real-time streaming. Called every heartbeat tick to emit metrics immediately.

Format: tick_number,elapsed_ns,tick_interval_ns,cache_hits_delta, bucket_hits_delta,word_executions_delta,hot_word_count, avg_word_heat>window_width,actual_window_size,predicted_label_hits,estimated_jitter_ns

Parameters

<i>vm</i>	Pointer to the VM instance (unused, for API consistency)
<i>snapshot</i>	Pointer to the snapshot to emit

References [HeartbeatTickSnapshot::actual_window_size](#), [HeartbeatTickSnapshot::avg_word_heat](#), [HeartbeatTickSnapshot::bucket](#), [HeartbeatTickSnapshot::cache_hits_delta](#), [HeartbeatTickSnapshot::elapsed_ns](#), [HeartbeatTickSnapshot::estimated_jitter_ns](#), [HeartbeatTickSnapshot::hot_word_count](#), [HeartbeatTickSnapshot::predicted_label_hits](#), [HeartbeatTickSnapshot::tick_interval_ns](#), [HeartbeatTickSnapshot::tick_number](#), [vm](#), [HeartbeatTickSnapshot::window_width](#), and [HeartbeatTickSnapshot::word_executions_delta](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the caller graph for this function:

7.75 src/inference_engine.c File Reference

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <stdint.h>
#include <assert.h>
#include "inference_engine.h"
#include "q48_16.h"
#include "vm.h"
#include "rolling_window_of_truth.h"
#include "ssm_jacquard.h"
```

Include dependency graph for inference_engine.c:

Functions

- static int [has_variance_stabilized](#) ([q48_16_t](#) current_variance, [q48_16_t](#) last_variance)
- static uint64_t * [extract_heat_trajectory](#) ([RollingWindowOfTruth](#) *window, [VM](#) *vm, uint64_t *out_length)
- [q48_16_t compute_variance_q48](#) (const uint64_t *heat_data, uint64_t length)

Compute variance of heat data in Q48.16 format.
- static [q48_16_t compute_median_q48](#) (const [q48_16_t](#) *data, uint32_t length)
- static [q48_16_t compute_mean_q48](#) (const [q48_16_t](#) *data, uint32_t length)
- static [q48_16_t compute_levene_statistic](#) (const [q48_16_t](#) *chunk_variances, uint32_t num_chunks, uint32_t chunk_size)

- uint32_t [find_variance_inflection](#) (const uint64_t *heat_data, uint64_t trajectory_length, [q48_16_t](#) full_↵ variance)
Find variance inflection point for optimal window width.
- uint64_t [infer_decay_slope_q48](#) (const uint64_t *heat_data, uint64_t length)
Infer decay slope from heat trajectory using linear regression.
- static uint64_t [compute_fit_quality](#) (const uint64_t *heat_data, uint64_t length, uint64_t slope_q48)
- void [inference_engine_run](#) ([InferenceInputs](#) *inputs, [InferenceOutputs](#) *outputs)
Run adaptive inference engine on window data.
- const char * [inference_outputs_to_string](#) (const [InferenceOutputs](#) *outputs)
Pretty-print inference outputs for logging.
- int [inference_outputs_validate](#) (const [InferenceOutputs](#) *outputs)
Validate inference outputs for sanity.

7.75.1 Function Documentation

7.75.1.1 [compute_fit_quality\(\)](#)

```
static uint64_t compute_fit_quality (
    const uint64_t * heat_data,
    uint64_t length,
    uint64_t slope_q48 ) [static]
```

References [q48_from_u64\(\)](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.2 [compute_levene_statistic\(\)](#)

```
static q48\_16\_t compute_levene_statistic (
    const q48\_16\_t * chunk_variances,
    uint32_t num_chunks,
    uint32_t chunk_size ) [static]
```

References [compute_mean_q48\(\)](#), [compute_median_q48\(\)](#), [compute_variance_q48\(\)](#), [q48_add\(\)](#), [q48_div\(\)](#), [q48_from_u64\(\)](#), and [q48_mul\(\)](#).

Referenced by [find_variance_inflection\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.3 [compute_mean_q48\(\)](#)

```
static q48\_16\_t compute_mean_q48 (
    const q48\_16\_t * data,
    uint32_t length ) [static]
```

References [q48_from_u64\(\)](#).

Referenced by [compute_levene_statistic\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.4 [compute_median_q48\(\)](#)

```
static q48\_16\_t compute_median_q48 (
    const q48\_16\_t * data,
    uint32_t length ) [static]
```

Referenced by [compute_levene_statistic\(\)](#).

Here is the caller graph for this function:

7.75.1.5 [compute_variance_q48\(\)](#)

```
q48\_16\_t compute_variance_q48 (
    const uint64_t * heat_data,
    uint64_t length )
```

Compute variance of heat data in Q48.16 format.

Parameters

<i>heat_data</i>	Array of heat values
<i>length</i>	Number of elements in array

Returns

Variance in Q48.16 format

References [q48_div\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), and [q48_to_u64\(\)](#).

Referenced by [compute_levene_statistic\(\)](#), [find_variance_inflection\(\)](#), and [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.6 extract_heat_trajectory()

```
static uint64_t * extract_heat_trajectory (
    RollingWindowOfTruth * window,
    VM * vm,
    uint64_t * out_length ) [static]
```

References [VM::dict_lock](#), [DICTIONARY_SIZE](#), [RollingWindowOfTruth::effective_window_size](#), [entry](#), [RollingWindowOfTruth::is_warn](#), [rolling_window_export_execution_history\(\)](#), [ROLLING_WINDOW_SIZE](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), and [word_id](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.7 find_variance_inflection()

```
uint32_t find_variance_inflection (
    const uint64_t * heat_data,
    uint64_t trajectory_length,
    q48_16_t full_variance )
```

Find variance inflection point for optimal window width.

Parameters

<i>heat_data</i>	Array of heat values
<i>trajectory_length</i>	Total trajectory length
<i>full_variance</i>	Variance of full trajectory

Returns

Optimal window width

References [ADAPTIVE_MIN_WINDOW_SIZE](#), [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), [q48_from_double\(\)](#), and [ROLLING_WINDOW_SIZE](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.8 has_variance_stabilized()

```
static int has_variance_stabilized (
    q48_16_t current_variance,
    q48_16_t last_variance ) [static]
```

References [q48_div\(\)](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.9 infer_decay_slope_q48()

```
uint64_t infer_decay_slope_q48 (
    const uint64_t * heat_data,
    uint64_t length )
```

Infer decay slope from heat trajectory using linear regression.

Parameters

<i>heat_data</i>	Array of heat values
<i>length</i>	Number of elements

Returns

Decay slope in Q48.16 format

References [q48_add\(\)](#), [q48_from_u64\(\)](#), [q48_log_approx\(\)](#), and [q48_mul\(\)](#).

Referenced by [inference_engine_run\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.10 inference_engine_run()

```
void inference_engine_run (
    InferenceInputs * inputs,
    InferenceOutputs * outputs )
```

Run adaptive inference engine on window data.

High-level flow:

1. ANOVA Check: Is variance delta < 5%?
 - YES: Return cached outputs immediately (cost: ~100 cycles)
 - NO: Continue to full inference
2. Extract trajectory: Fresh execution_heat snapshot from dictionary
3. Window Width Inference:
 - Compute variance across entire window
 - Detect inflection point (where $d(\text{variance})/d(\text{size}) \rightarrow 0$)
 - Return optimal window size
4. Decay Slope Inference:
 - Transform trajectory to log space ($\ln(\text{heat}[t])$)
 - Fit linear regression: $\ln(\text{heat}) = a - \text{slope} * t$
 - Extract slope coefficient in Q48.16
5. Diagnostics:
 - Compute fit quality (R^2 or residual)
 - Store variance metric for next early-exit check

Parameters

<i>inputs</i>	Snapshot of window + metrics (caller-provided)
<i>outputs</i>	Tuning parameters + diagnostics (caller-provided, will be updated)

Returns

void (results_run_01_2025_12_08 in outputs struct)

Preconditions:

- inputs != NULL
- outputs != NULL
- inputs->window != NULL
- inputs->window->is_warm == 1 (window has enough data)
- inputs->trajectory_length > 1

Side Effects:

- Modifies outputs struct (caller must preserve if caching)
- Allocates temporary trajectory buffer (freed on return)

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [compute_fit_quality\(\)](#), [compute_variance_q48\(\)](#), [InferenceOutputs::early_exited](#), [extract_heat_trajectory\(\)](#), [find_variance_inflection\(\)](#), [has_variance_stabilized\(\)](#), [infer_decay_slope_q48\(\)](#), [ssm_config_t::L5_window_inference](#), [ssm_config_t::L6_decay_inference](#), [InferenceOutputs::slope_fit_quality_q48](#), [VM::ssm_config](#), [InferenceInputs::vm](#), [InferenceInputs::window](#), and [InferenceOutputs::window_variance_q48](#).

Referenced by [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.75.1.11 inference_outputs_to_string()

```
const char * inference_outputs_to_string (
    const InferenceOutputs * outputs )
```

Pretty-print inference outputs for logging.

Format: "window_width=XXX variance_q48=YYYY slope_q48=ZZZZ fit_quality=WWWW"

Parameters

<i>outputs</i>	Inference outputs struct
----------------	--------------------------

Returns

Pointer to static string (valid until next call)

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [InferenceOutputs::early_exited](#), [q48_to_double\(\)](#), [InferenceOutputs::slope_fit_quality_q48](#), and [InferenceOutputs::window_variance_q48](#).

Here is the call graph for this function:

7.75.1.12 inference_outputs_validate()

```
int inference_outputs_validate (
    const InferenceOutputs * outputs )
```

Validate inference outputs for sanity.

Checks:

- window_width is within [ADAPTIVE_MIN_WINDOW_SIZE, ROLLING_WINDOW_SIZE]
- slope_q48 is reasonable (between 0.001 and 100 in Q48.16)
- variance and fit_quality are between 0.0 and 1.0 in Q48.16

Parameters

<i>outputs</i>	Inference outputs to validate
----------------	-------------------------------

Returns

1 if valid, 0 if suspicious

References [InferenceOutputs::adaptive_decay_slope](#), [ADAPTIVE_MIN_WINDOW_SIZE](#), [InferenceOutputs::adaptive_window_width](#), [q48_from_u64\(\)](#), [ROLLING_WINDOW_SIZE](#), and [InferenceOutputs::slope_fit_quality_q48](#).

Referenced by [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.76 src/io.c File Reference

```
#include "io.h"
#include "log.h"
#include <string.h>
Include dependency graph for io.c:
```

Functions

- void [io_init](#) ([VM](#) **vm*)
Initialize the IO subsystem.
- int [io_write_block](#) ([VM](#) **vm*, int *block_num*, const unsigned char **buffer*)
Write a block to [VM](#) memory.

7.76.1 Function Documentation

7.76.1.1 io_init()

```
void io_init (
    VM * vm )
```

Initialize the IO subsystem.

Initialize the I/O subsystem.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Initializes the block device view over the unified [VM](#) memory. Zeros out the entire [VM](#) memory space.

References [BLOCK_SIZE](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [MAX_BLOCKS](#), [VM::memory](#), [vm](#), and [VM_MEMORY_SIZE](#).

Here is the call graph for this function:

7.76.1.2 io_write_block()

```
int io_write_block (
    VM * vm,
    int block_num,
    const unsigned char * buffer )
```

Write a block to [VM](#) memory.

Write a block to storage.

Parameters

<i>vm</i>	Pointer to VM instance
<i>block_num</i>	Block number to write (0 to MAX_BLOCKS -1)

Parameters

<i>buffer</i>	Source buffer containing block data
---------------	-------------------------------------

Returns

0 on success, -1 on error

Write a block to [VM](#) memory

Parameters

<i>vm</i>	Pointer to VM instance
<i>block_num</i>	Block number to write (0 to MAX_BLOCKS-1)
<i>buffer</i>	Source buffer containing block data

Returns

0 on success, -1 on error

Writes a BLOCK_SIZE chunk of data from the provided buffer into the [VM](#)'s memory at the specified block number offset. The block number must be within valid range. The entire block is written atomically.

Note

The buffer must contain at least BLOCK_SIZE bytes of valid data

References [BLOCK_SIZE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [MAX_BLOCKS](#), [VM::memory](#), and [vm](#).
Here is the call graph for this function:

7.77 src/log.c File Reference

```
#include "../include/log.h"
#include "../include/platform_time.h"
#include <stdio.h>
#include <stdarg.h>
#include <time.h>
```

Include dependency graph for log.c:

Functions

- void [log_set_level](#) (LogLevel level)
Sets the global logging level.
- LogLevel [log_get_level](#) (void)
Gets the current global logging level.
- static void [get_timestamp](#) (char *buffer, size_t size)
Helper function to generate timestamp string.
- void [log_message](#) (LogLevel level, const char *fmt,...)
Logs a message with the specified level and format.
- void [log_test_result](#) (const char *word_name, [TestResult](#) result)
Logs test results_run_01_2025_12_08 with appropriate formatting and colors.

Variables

- static LogLevel [current_level](#) = LOG_INFO
- static const char * [level_names](#) []
- static const char * [level_colors](#) []

- static const char * [test_pass_color](#) = "\x1b[32m"
- static const char * [test_fail_color](#) = "\x1b[31m"
- static const char * [test_skip_color](#) = "\x1b[33m"
- static const char * [color_reset](#) = "\x1b[0m"

7.77.1 Function Documentation

7.77.1.1 [get_timestamp\(\)](#)

```
static void get_timestamp (
    char * buffer,
    size_t size ) [static]
```

Helper function to generate timestamp string.

Parameters

<i>buffer</i>	Output buffer for the timestamp string
<i>size</i>	Size of the output buffer

References [sf_format_timestamp\(\)](#), and [sf_realtime_ns\(\)](#).

Referenced by [log_message\(\)](#), and [log_test_result\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.77.1.2 [log_get_level\(\)](#)

```
LogLevel log_get_level (
    void )
```

Gets the current global logging level.

Get the current logging level.

Returns

LogLevel Current logging level

References [current_level](#).

7.77.1.3 [log_message\(\)](#)

```
void log_message (
    LogLevel level,
    const char * fmt,
    ... )
```

Logs a message with the specified level and format.

Log a formatted message at the specified level.

Parameters

<i>level</i>	The LogLevel for this message
<i>fmt</i>	Printf-style format string
...	Variable arguments for format string

References [color_reset](#), [current_level](#), [get_timestamp\(\)](#), [level](#), [level_colors](#), [level_names](#), [LOG_NONE](#), and [SF_TIME_STAMP_SIZE](#).

Referenced by [__attribute__\(\)](#), [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [assert_heartbeat_safe\(\)](#), [assert_stack_depth\(\)](#), [assert_stack_top\(\)](#), [assert_vm_error\(\)](#), [blk_subsys_attach_device\(\)](#), [cf_epoch_sync\(\)](#), [cf_pop_item\(\)](#), [cf_push_item\(\)](#), [check_physics_transparent\(\)](#), [control_forth_0branch\(\)](#), [control_forth_again\(\)](#),

control_forth_begin(), control_forth_branch(), control_forth_case(), control_forth_do(), control_forth_else(), control_forth_endcase(), control_forth_endof(), control_forth_EXIT(), control_forth_l(), control_forth_if(), control_forth_J(), control_forth_leave(), control_forth_loop(), control_forth_of(), control_forth_plus_loop(), control_forth_qdo(), control_forth_repeat(), control_forth_runtime_do(), control_forth_runtime_leave(), control_forth_runtime_loop(), control_forth_runtime_plus_loop(), control_forth_runtime_qdo(), control_forth_then(), control_forth_UNLOOP(), control_forth_until(), control_forth_while(), defining_runtime_constant(), defining_runtime_create(), defining_runtime_dodoes(), defining_runtime_does_rt(), defining_runtime_lit(), defining_runtime_variable(), defining_word_colon(), defining_word_compile(), defining_word_constant(), defining_word_create(), defining_word_does(), defining_word_immediate(), defining_word_left_bracket(), defining_word_literal(), defining_word_right_bracket(), defining_word_variable(), dict_adaptive_optimization_pass(), dict_reorganize_buckets_by_heat(), dict_update_heat_percentiles(), dictionary_m_word_tick(), dictionary_m_word_to_body(), dictionary_word_forget(), dump_and_die(), editor_word_edit(), editor_word_l(), editor_word_s(), editor_word_show(), emit_cell(), enable_benchmark_mode(), execute_colon_word(), fail_and_exit(), format_word_hold(), forth_acl_allow_store(), forth_acl_inherit(), forth_acl_mode_store(), forth_acl_ttl_store(), forth_paren(), init_blkio(), init_report(), init_vocabulary_system(), io_init(), io_word_type(), io_write_block(), logical_word_and(), logical_word_equals(), logical_word_false(), logical_word_greater_than(), logical_word_less_than(), logical_word_not(), logical_word_not_equals(), logical_word_or(), logical_word_true(), logical_word_u_greater_than(), logical_word_u_less_than(), logical_word_within(), logical_word_xor(), logical_word_zero_equals(), logical_word_zero_greater(), logical_word_zero_less(), logical_word_zero_not_equals(), main(), memory_word_cells(), mixed_math_word_mod(), pop_xt(), print_module_summary(), read_devblock_4k(), register_arithmetic_words(), register_forth79_words(), register_word(), return_stack_word_r_fetch(), return_stack_word_r_from(), return_stack_word_to_r(), rolling_window_run_adaptive_pass(), rolling_window_seed_hotwords_cache(), rolling_window_seed_pipelining_context(), run_acl_words_tests(), run_all_tests(), run_arithmetic_words_tests(), run_benchmark_mode(), run_block_words_tests(), run_break_me_tests(), run_control_words_tests(), run_defining_words_tests(), run_dictionary_manipulation_words_tests(), run_dictionary_words_tests(), run_doe_experiment(), run_double_words_tests(), run_format_words_tests(), run_integration_tests(), run_io_words_tests(), run_logical_words_tests(), run_mama_forth_words_tests(), run_memory_words_tests(), run_mixed_arithmetic_words_tests(), run_module_tests(), run_return_stack_words_tests(), run_single_test(), run_stack_words_tests(), run_starforth_words_tests(), run_stress_tests(), run_string_words_tests(), run_system_words_tests(), run_test_suite(), run_test_suite_m(), run_vocabulary_words_tests(), run_word_tests(), stack_word_depth(), stack_word_drop(), stack_word_dup(), stack_word_minus_rot(), stack_word_over(), stack_word_pick(), stack_word_question_dup(), stack_word_roll(), stack_word_rot(), stack_word_swap(), starforth_word_execution_heat_fetch(), starforth_word_execution_heat_store(), starforth_word_init(), starforth_word_paren_dash(), starforth_word_random(), starforth_word_seed(), starforth_word_top_words(), starforth_word_wait(), string_word_blank(), string_word_cmove(), string_word_cmove_greater(), string_word_compare(), string_word_count(), string_word_expect(), string_word_minus_trailing(), string_word_s_quote(), string_word_scan(), string_word_search(), string_word_skip(), test_acl_adaptive_ttl(), test_acl_strict_mode(), vm_align(), vm_allot(), vm_assert_interpreter_enabled(), vm_bootstrap_root_vocabulary(), vm_bootstrap_scr(), vm_compile_call(), vm_compile_exit(), vm_compile_literal(), vm_create_word(), vm_dictionary_track_entry(), vm_enter_compile_mode(), vm_exit_compile_mode(), vm_get_block_addr(), vm_heartbeat_start_thread(), vm_heartbeat_update_l8(), vm_heartbeat_update_l8(), vm_init(), vm_init_with_host(), vm_interpret_word(), vm_make_immediate(), vm_pin_execution_heat(), vm_pop(), vm_push(), vm_repl(), vm_smudge_word(), vm_tick_inference_engine(), vm_tick_slope_validator(), vm_tick_window_tuner(), vm_unpin_execution_heat(), vocabulary_create_vocabulary_direct(), vocabulary_select_runtime(), vocabulary_word_definitions(), vocabulary_word_forth(), vocabulary_word_vocabulary(), word_divide(), word_divmod(), and write_devblock_4k().

Here is the call graph for this function:

7.77.1.4 log_set_level()

```
void log_set_level (
    LogLevel level )
```

Sets the global logging level.

Set the global logging level.

Parameters

<i>level</i>	The LogLevel to set as current logging level
--------------	--

References [current_level](#), and [level](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

7.77.1.5 log_test_result()

```
void log_test_result (
    const char * word_name,
    TestResult result )
```

Logs test results `_run_01_2025_12_08` with appropriate formatting and colors.

Log a test result with colored output.

Parameters

<i>word_name</i>	Name of the word being tested
<i>result</i>	Test result (PASS/FAIL/SKIP)

References [color_reset](#), [current_level](#), [get_timestamp\(\)](#), [LOG_NONE](#), [LOG_TEST](#), [SF_TIME_STAMP_SIZE](#), [TEST_FAIL](#), [test_fail_color](#), [TEST_PASS](#), [test_pass_color](#), [TEST_SKIP](#), and [test_skip_color](#).

Referenced by [run_single_test\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.77.2 Variable Documentation

7.77.2.1 color_reset

```
const char* color_reset = "\x1b[0m" [static]
```

Referenced by [log_message\(\)](#), and [log_test_result\(\)](#).

7.77.2.2 current_level

```
LogLevel current_level = LOG_INFO [static]
```

Referenced by [log_get_level\(\)](#), [log_message\(\)](#), [log_set_level\(\)](#), and [log_test_result\(\)](#).

7.77.2.3 level_colors

```
const char* level_colors[] [static]
```

Initial value:

```
= {
    "\x1b[31m",
    "\x1b[33m",
    "\x1b[32m",
    "\x1b[35m",
    "\x1b[34m"
}
```

Referenced by [log_message\(\)](#).

7.77.2.4 level_names

```
const char* level_names[] [static]
```

Initial value:

```
= {
    "ERROR",
    "WARN",
    "INFO",
    "TEST",
    "DEBUG"
}
```

Referenced by [log_message\(\)](#).

7.77.2.5 test_fail_color

```
const char* test_fail_color = "\x1b[31m" [static]
```

Referenced by [log_test_result\(\)](#).

7.77.2.6 test_pass_color

```
const char* test_pass_color = "\x1b[32m" [static]
```

Referenced by [log_test_result\(\)](#).

7.77.2.7 test_skip_color

const char* test_skip_color = "\x1b[33m" [static]
 Referenced by [log_test_result\(\)](#).

7.78 src/main.c File Reference

```
#include "vm.h"
#include "vm_debug.h"
#include "log.h"
#include "profiler.h"
#include "physics_runtime.h"
#include "platform_time.h"
#include "version.h"
#include "cli.h"
#include "test_runner/include/test_runner.h"
#include "test_runner/include/test_common.h"
#include "doe_metrics.h"
#include "blkio.h"
#include "blkio_factory.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <ctype.h>
#include <stdint.h>
```

Include dependency graph for main.c:

Macros

- #define [STARFORTH_STATE_BYTES](#) 8192u

Functions

- [__attribute__](#) ((weak))
- static void [cleanup_blkio](#) (void)
- void [cleanup_and_exit](#) (void)
Performs cleanup operations before program termination.
- void [signal_handler](#) (int sig)
- void [print_usage](#) (const char *program_name)
Prints program usage information.
- static int [init_blkio](#) (const [CLIConfig](#) *config)
Initialize block I/O from CLI configuration.
- static int [init_vm_and_subsystem](#) ([VM](#) *vm)
Initialize VM core and block subsystem.
- static void [run_benchmark_mode](#) ([VM](#) *vm, const [CLIConfig](#) *config)
Run benchmark mode and exit.
- static void [run_optional_tests](#) ([VM](#) *vm, const [CLIConfig](#) *config)
Run optional test modes after system initialization.
- static void [run_doe_experiment](#) ([VM](#) *vm)
Run Design of Experiments experiment.
- int [main](#) (int argc, char *argv[])
Main entry point for StarForth.

Variables

- static int [g_blkio_opened](#) = 0
- static uint8_t * [g_ram_base](#) = NULL
- static size_t [g_ram_bytes](#) = 0
- static int [g_ram_is_mmap](#) = 0

7.78.1 Macro Definition Documentation

7.78.1.1 STARFORTH_STATE_BYTES

```
#define STARFORTH_STATE_BYTES 8192u
```

7.78.2 Function Documentation

7.78.2.1 __attribute__()

```
__attribute__ (
    (weak) )
```

7.78.2.2 cleanup_and_exit()

```
void cleanup_and_exit (
    void )
```

Performs cleanup operations before program termination.

References [cleanup_blkio\(\)](#), [physics_runtime_shutdown\(\)](#), and [vm_cleanup\(\)](#).

Referenced by [main\(\)](#), and [signal_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.3 cleanup_blkio()

```
static void cleanup_blkio (
    void ) [static]
```

References [blkio_close\(\)](#), [blkio_flush\(\)](#), [g_blkio_opened](#), [g_ram_base](#), [g_ram_bytes](#), and [g_ram_is_mmap](#).

Referenced by [cleanup_and_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.4 init_blkio()

```
static int init_blkio (
    const CLIConfig * config ) [static]
```

Initialize block I/O from CLI configuration.

References [blkio_factory_open\(\)](#), [BLKIO_FORTH_BLOCK_SIZE](#), [blkio_info\(\)](#), [BLKIO_OK](#), [CLIConfig::disk_img_path](#), [blkio_info_t::forth_block_size](#), [g_blkio_opened](#), [g_ram_base](#), [g_ram_bytes](#), [g_ram_is_mmap](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [blkio_info_t::phys_size_bytes](#), [CLIConfig::ram_disk_mb](#), [blkio_info_t::read_only](#), [STARFORTH_STATE_BYTES](#), and [blkio_info_t::total_blocks](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.5 init_vm_and_subsystem()

```
static int init_vm_and_subsystem (
    VM * vm ) [static]
```

Initialize [VM](#) core and block subsystem.

References [blk_subsys_init\(\)](#), [VM::error](#), [g_blkio_opened](#), [ram_base](#), [ram_size](#), [vm](#), and [vm_init\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.6 main()

```
int main (
    int argc,
    char * argv[] )
```

Main entry point for StarForth.

References [CLConfig::benchmark](#), [CLConfig::break_me](#), [cleanup_and_exit\(\)](#), [cli_parse\(\)](#), [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [CLConfig::doe_experiment](#), [VM::error](#), [CLConfig::fail_fast](#), [fail_fast](#), [VM::here](#), [init_blkio\(\)](#), [init_vm_and_subsystem\(\)](#), [VM::latest](#), [LOG_ERROR](#), [LOG_INFO](#), [CLConfig::log_level](#), [CLConfig::log_level_explicitly_set](#), [log_message\(\)](#), [LOG_NONE](#), [log_set_level\(\)](#), [LOG_TEST](#), [LOG_WARN](#), [physics_runtime_init\(\)](#), [run_all_tests\(\)](#), [run_benchmark_mode\(\)](#), [run_doe_experiment\(\)](#), [run_optional_tests\(\)](#), [CLConfig::run_tests_flag_observed](#), [CLConfig::script_mode](#), [sf_time_init\(\)](#), [signal_handler\(\)](#), [vm](#), [vm_debug_install_signal_handlers\(\)](#), [vm_debug_set_current_vm\(\)](#), [vm_interpret\(\)](#), and [vm_repl\(\)](#).

Here is the call graph for this function:

7.78.2.7 print_usage()

```
void print_usage (
    const char * program_name )
```

Prints program usage information.

Parameters

<i>program_name</i>	Name of the executable
---------------------	------------------------

7.78.2.8 run_benchmark_mode()

```
static void run_benchmark_mode (
    VM * vm,
    const CLConfig * config ) [static]
```

Run benchmark mode and exit.

References [CLConfig::benchmark_iterations](#), [enable_benchmark_mode\(\)](#), [LOG_INFO](#), [log_message\(\)](#), [run_all_tests\(\)](#), and [vm](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.9 run_doe_experiment()

```
static void run_doe_experiment (
    VM * vm ) [static]
```

Run Design of Experiments experiment.

Executes the full test harness N times, collecting metrics after each run. Writes results to CSV file for statistical analysis.

References [RollingWindowOfTruth::effective_window_size](#), [RollingWindowOfTruth::is_warm](#), [LOG_INFO](#), [log_message\(\)](#), [metrics_from_vm\(\)](#), [metrics_get_cpu_freq_mhz\(\)](#), [metrics_get_cpu_temp_c\(\)](#), [VM::rolling_window](#), [run_all_tests\(\)](#), [sf_monotonic_ns\(\)](#), [RollingWindowOfTruth::total_executions](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.10 run_optional_tests()

```
static void run_optional_tests (
    VM * vm,
    const CLConfig * config ) [static]
```

Run optional test modes after system initialization.

References [CLConfig::break_me](#), [run_break_me_tests\(\)](#), and [vm](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.2.11 signal_handler()

```
void signal_handler (
    int sig )
```

References [cleanup_and_exit\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.78.3 Variable Documentation

7.78.3.1 g_blkio_opened

```
int g_blkio_opened = 0 [static]
```

Referenced by [cleanup_blkio\(\)](#), [init_blkio\(\)](#), and [init_vm_and_subsystem\(\)](#).

7.78.3.2 g_ram_base

```
uint8_t* g_ram_base = NULL [static]
```

Referenced by [cleanup_blkio\(\)](#), and [init_blkio\(\)](#).

7.78.3.3 g_ram_bytes

```
size_t g_ram_bytes = 0 [static]
```

Referenced by [cleanup_blkio\(\)](#), and [init_blkio\(\)](#).

7.78.3.4 g_ram_is_mmap

```
int g_ram_is_mmap = 0 [static]
```

Referenced by [cleanup_blkio\(\)](#), and [init_blkio\(\)](#).

7.79 src/math_portable.c File Reference

```
#include <stdint.h>
```

```
#include "math_portable.h"
```

Include dependency graph for math_portable.c:

Functions

- static int64_t [exp_q48](#) (int64_t x)
Portable exponential function via Taylor series in Q48.16 fixed-point $\exp(x) = 1 + x + x^2/2! + x^3/3! + \dots$
- int64_t [sqrt_q48](#) (int64_t x)
Square root in Q48.16 fixed-point via Newton-Raphson.
- int64_t [erf_q48](#) (int64_t x)
*Error function in Q48.16 fixed-point $\operatorname{erf}(x) \approx \operatorname{sign}(x) * \sqrt{1 - \exp(-x^2 * (4/\pi + ax^2) / (1 + ax^2))}$*

7.79.1 Function Documentation

7.79.1.1 erf_q48()

```
int64_t erf_q48 (
    int64_t x )
```

Error function in Q48.16 fixed-point $\operatorname{erf}(x) \approx \operatorname{sign}(x) * \sqrt{1 - \exp(-x^2 * (4/\pi + ax^2) / (1 + ax^2))}$

Error function in Q48.16 fixed-point Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format (range: -Q48_SCALE to +Q48_SCALE) Uses Abramowitz & Stegun approximation (7.1.26) Maximum error: 1.5e-7 relative to floating-point erf Properties:

Uses Abramowitz & Stegun approximation (7.1.26) Input: x in Q48.16 format (dimensionless Z-score) Output: erf(x) in Q48.16 format, clamped to [-Q48_SCALE, +Q48_SCALE]

Maximum error: ~1.5e-7 relative to true erf

References [base](#), [exp_q48\(\)](#), and [sqrt_q48\(\)](#).

Referenced by [hotwords_speedup_estimate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.79.1.2 exp_q48()

```
static int64_t exp_q48 (
    int64_t x ) [static]
```

Portable exponential function via Taylor series in Q48.16 fixed-point $\exp(x) = 1 + x + x^2/2! + x^3/3! + \dots$

Input: x in Q48.16 format Output: exp(x) in Q48.16 format

Handles only reasonable inputs (-20 to 0) for our use case ($\exp(-Z^2)$ in erf). Returns Q48_SCALE (65536) for x=0, approaches 0 for x<-20, saturates for x>0.

Referenced by [erf_q48\(\)](#).

Here is the caller graph for this function:

7.79.1.3 sqrt_q48()

```
int64_t sqrt_q48 (
    int64_t x )
```

Square root in Q48.16 fixed-point via Newton-Raphson.

Square root in Q48.16 fixed-point Input: x in Q48.16 format Output: sqrt(x) in Q48.16 format Uses Newton-Raphson: $x_{n+1} = (x_n + a/x_n) / 2$ Converges in ~10-15 iterations Returns 0 for negative inputs (guard against domain error)

Input: x in Q48.16 format Output: sqrt(x) in Q48.16 format

Newton-Raphson: $r_{n+1} = (r_n^2 + x) / (2 * r_n)$ Where multiply/divide preserve Q48.16 scaling. Converges in ~10-15 iterations.

Referenced by [calculate_posterior\(\)](#), [erf_q48\(\)](#), [hotwords_speedup_estimate\(\)](#), and [hotwords_stats_print\(\)](#).

Here is the caller graph for this function:

7.80 src/memory_management.c File Reference

```
#include "../include/vm.h"
```

```
#include "../include/log.h"
```

Include dependency graph for memory_management.c:

Functions

- void * [vm_allot](#) (VM *vm, size_t bytes)
Allocates memory from the VM's dictionary space.
- void [vm_align](#) (VM *vm)
Aligns the VM's dictionary pointer (here) to the cell boundary.
- void * [vm_get_block_addr](#) (VM *vm, int block_num)
Gets the memory address for a specified block number.
- int [vm_addr_to_block](#) (VM *vm, void *addr)
Converts a memory address to its corresponding block number.

7.80.1 Function Documentation

7.80.1.1 vm_addr_to_block()

```
int vm_addr_to_block (
    VM * vm,
    void * addr )
```

Converts a memory address to its corresponding block number.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>addr</i>	Memory address to convert

Returns

Block number (0 to MAX_BLOCKS-1) or -1 if address is outside [VM](#) memory

Note

Used for determining which block a memory address belongs to

References [BLOCK_SIZE](#), [VM::memory](#), [vm](#), and [VM_MEMORY_SIZE](#).

7.80.1.2 vm_align()

```
void vm_align (
    VM * vm )
```

Aligns the [VM](#)'s dictionary pointer (here) to the cell boundary.

Aligns the virtual machine's memory pointer to the next word boundary.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Note

Adds padding bytes if necessary to ensure proper alignment for cell_t type

References [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_allot\(\)](#).

Referenced by [defining_runtime_does_rt\(\)](#), [defining_word_create\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_here\(\)](#), [emit_cell\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_enter_compile_mode\(\)](#), and [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.80.1.3 vm_allot()

```
void * vm_allot (
    VM * vm,
    size_t bytes )
```

Allocates memory from the [VM](#)'s dictionary space.

Allocates memory in the virtual machine's memory space.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>bytes</i>	Number of bytes to allocate

Returns

Pointer to the allocated memory or NULL if allocation fails

Note

The allocation is done from the [VM](#)'s dictionary space which is limited to [DICTIONARY_MEMORY_SIZE](#)

References [BLOCK_SIZE](#), [DICTIONARY_MEMORY_SIZE](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), and [vm](#).

Referenced by [defining_runtime_does_rt\(\)](#), [defining_word_variable\(\)](#), [emit_cell\(\)](#), [ensure_pn\(\)](#), [init_vocabulary_system\(\)](#), [string_word_s_quote\(\)](#), [string_word_word\(\)](#), [system_word_abort_quote\(\)](#), [vm_align\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_input_ensure\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.80.1.4 vm_get_block_addr()

```
void * vm_get_block_addr (
    VM * vm,
    int block_num )
```

Gets the memory address for a specified block number.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to get the address for

Returns

Pointer to the start of the block or NULL if block number is invalid

Note

Blocks are fixed-size memory regions of BLOCK_SIZE bytes

References [BLOCK_SIZE](#), [LOG_ERROR](#), [log_message\(\)](#), [MAX_BLOCKS](#), [VM::memory](#), and [vm](#).

Here is the call graph for this function:

7.81 src/physics_execution_hooks.c File Reference

```
#include "../include/physics_execution_hooks.h"
#include "../include/platform_time.h"
#include "../include/profiler.h"
#include "../include/physics_metadata.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/physics_pipelining_metrics.h"
#include "../include/rolling_window_of_truth.h"
```

Include dependency graph for physics_execution_hooks.c:

Functions

- void [physics_pre_execute](#) ([VM](#) *vm, [DictEntry](#) *word, [DictEntry](#) *prev)
- void [physics_post_execute](#) ([VM](#) *vm, [DictEntry](#) *word)
- void [physics_on_lookup](#) ([VM](#) *vm, [DictEntry](#) *entry, [DictEntry](#) *canon)

7.81.1 Function Documentation

7.81.1.1 physics_on_lookup()

```
void physics_on_lookup (
    VM * vm,
    DictEntry * entry,
    DictEntry * canon )
```

References [VM::dict_lock](#), [entry](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [DictEntry::physics](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [physics_metadata_touch\(\)](#), [sf_monotonic_ns\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.81.1.2 physics_post_execute()

```
void physics_post_execute (
    VM * vm,
    DictEntry * word )
```

References [HeartbeatState::check_counter](#), [DictEntry::execution_heat](#), [VM::heartbeat](#), [HEARTBEAT_CHECK_FREQUENCY](#), [physics_metadata_touch\(\)](#), [profiler_word_exit\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_heartbeat_run_cycle\(\)](#), [HeartbeatState::words_executed](#) and [HeartbeatState::worker](#).

Referenced by [execute_colon_word\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.81.1.3 physics_pre_execute()

```
void physics_pre_execute (
    VM * vm,
    DictEntry * word,
    DictEntry * prev )
```

References [DICTIONARY_SIZE](#), [ENABLE_HOTWORDS_CACHE](#), [ENABLE_PIPELINING](#), [DictEntry::execution_heat](#), [VM::hotwords_cache](#), [hotwords_cache_promote\(\)](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [WordTransitionMetrics::most_likely_next_word_id](#), [DictEntry::physics](#), [physics_execution_heat_increment](#), [physics_metadata_apply_linear_decay\(\)](#), [VM::pipeline_metrics](#), [WordTransitionMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [rolling_window_record_execution](#), [sf_monotonic_ns\(\)](#), [DictEntry::transition_metrics](#), [transition_metrics_record\(\)](#), [transition_metrics_record_prefetch_hit\(\)](#), [transition_metrics_should_speculate\(\)](#), [transition_metrics_update_cache\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), [word_id](#), and [DictEntry::word_id](#).

Referenced by [execute_colon_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82 src/physics_hotwords_cache.c File Reference

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "physics_hotwords_cache.h"
#include "platform_time.h"
#include "math_portable.h"
Include dependency graph for physics_hotwords_cache.c:
```

Functions

- static double [q48_to_ns_double](#) (int64_t q48)

Helper: Convert Q48.16 fixed-point value to nanoseconds (double) for display only NOT used in calculations - all calculations stay in fixed-point.
- void [hotwords_cache_init](#) ([HotwordsCache](#) *cache)

Initialize hot-words cache Should be called during VM initialization.
- void [hotwords_cache_cleanup](#) ([HotwordsCache](#) *cache)

Cleanup hot-words cache.
- [DictEntry](#) * [hotwords_cache_lookup](#) ([HotwordsCache](#) *cache, [DictEntry](#) **bucket, size_t bucket_count, const char *name, size_t len)

Lookup word in cache first, then fall back to bucket search Returns the found word, or NULL if not found.
- void [hotwords_cache_promote](#) ([HotwordsCache](#) *cache, [DictEntry](#) *word)

Promote a word to the hot-words cache Called when a word's execution heat exceeds threshold.
- void [hotwords_bucket_reorder](#) ([DictEntry](#) **bucket, size_t count)

Reorder a bucket by execution heat (move hot words forward) Called when execution heat delta exceeds threshold.
- void [hotwords_stats_reset](#) ([HotwordsStats](#) *stats)

Reset statistics (for before/after comparison)
- void [hotwords_stats_print](#) (const [HotwordsStats](#) *stats)

Print statistics to stdout.
- void [hotwords_cache_set_enabled](#) ([HotwordsCache](#) *cache, int enabled)

Enable/disable cache at runtime (for A/B testing)

- static [BayesianLatencyPosterior calculate_posterior](#) (int64_t total_ns_q48, int64_t variance_sum_q48, int64_t min_ns, int64_t max_ns, uint64_t sample_count)
Calculate statistics for a single distribution (cache hits or bucket searches) Returns posterior with all values in Q48.16 fixed-point format Pure fixed-point arithmetic - no floating-point operations.
- [BayesianLatencyPosterior hotwords_posterior_cache_hits](#) (const [HotwordsStats](#) *stats)
Calculate posterior for cache hits.
- [BayesianLatencyPosterior hotwords_posterior_bucket_searches](#) (const [HotwordsStats](#) *stats)
Calculate posterior for bucket searches.
- struct [SpeedupEstimate hotwords_speedup_estimate](#) (const [HotwordsStats](#) *stats)
Calculate speedup estimate with credible intervals Pure 64-bit fixed-point Q48.16 arithmetic throughout All results↔_run_01_2025_12_08 FORTH-compatible: can be pushed to stack and manipulated by FORTH.
- void [hotwords_bayesian_report](#) (const [HotwordsStats](#) *stats_with_cache, const [HotwordsStats](#) *stats_↔without_cache)
Generate detailed Bayesian inference report comparing with/without cache.

Variables

- [HotwordsCache g_hotwords_cache](#) = {0}

7.82.1 Function Documentation

7.82.1.1 calculate_posterior()

```
static BayesianLatencyPosterior calculate_posterior (
    int64_t total_ns_q48,
    int64_t variance_sum_q48,
    int64_t min_ns,
    int64_t max_ns,
    uint64_t sample_count ) [static]
```

Calculate statistics for a single distribution (cache hits or bucket searches) Returns posterior with all values in Q48.16 fixed-point format Pure fixed-point arithmetic - no floating-point operations.

References [BayesianLatencyPosterior::credible_lower_95](#), [BayesianLatencyPosterior::credible_lower_99](#), [BayesianLatencyPosterior::credible_upper_99](#), [BayesianLatencyPosterior::mean_ns_q48](#), [BayesianLatencyPosterior::median_ns_q48](#), [BayesianLatencyPosterior::sample_count](#), [sqrt_q48\(\)](#), and [BayesianLatencyPosterior::stddev_ns_q48](#).

Referenced by [hotwords_posterior_bucket_searches\(\)](#), and [hotwords_posterior_cache_hits\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.2 hotwords_bayesian_report()

```
void hotwords_bayesian_report (
    const HotwordsStats * stats_with_cache,
    const HotwordsStats * stats_without_cache )
```

Generate detailed Bayesian inference report comparing with/without cache.

Generate detailed Bayesian inference report Compares cache vs bucket performance with confidence measures.

References [HotwordsStats::bucket_hits](#), [HotwordsStats::cache_hits](#), [BayesianLatencyPosterior::credible_lower_95](#), [SpeedupEstimate::credible_lower_95_q48](#), [BayesianLatencyPosterior::credible_lower_99](#), [BayesianLatencyPosterior::credible_upper_99](#), [SpeedupEstimate::credible_upper_95_q48](#), [BayesianLatencyPosterior::credible_upper_99](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [hotwords_speedup_estimate\(\)](#), [BayesianLatencyPosterior::mean_ns_q48](#), [HotwordsStats::misses](#), [SpeedupEstimate::probability_gt_10pct_q48](#), [SpeedupEstimate::probability_gt_double_q48](#), [q48_to_ns_double\(\)](#), [BayesianLatencyPosterior::sample_count](#), [SpeedupEstimate::speedup_factor_q48](#), and [BayesianLatencyPosterior::stddev_ns_q48](#).

Referenced by [forth_PHYSICS_BAYESIAN_REPORT\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.3 hotwords_bucket_reorder()

```
void hotwords_bucket_reorder (
    DictEntry ** bucket,
    size_t count )
```

Reorder a bucket by execution heat (move hot words forward) Called when execution heat delta exceeds threshold.
References [execution_heat](#).

7.82.1.4 hotwords_cache_cleanup()

```
void hotwords_cache_cleanup (
    HotwordsCache * cache )
```

Cleanup hot-words cache.

References [cache](#).

Referenced by [vm_cleanup\(\)](#).

Here is the caller graph for this function:

7.82.1.5 hotwords_cache_init()

```
void hotwords_cache_init (
    HotwordsCache * cache )
```

Initialize hot-words cache Should be called during VM initialization.

References [cache](#), and [ENABLE_HOTWORDS_CACHE](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.82.1.6 hotwords_cache_lookup()

```
DictEntry * hotwords_cache_lookup (
    HotwordsCache * cache,
    DictEntry ** bucket,
    size_t bucket_count,
    const char * name,
    size_t len )
```

Lookup word in cache first, then fall back to bucket search Returns the found word, or NULL if not found.

This is the main function that replaces [vm_find_word\(\)](#) logic

References [cache](#), [DictEntry::execution_heat](#), [hotwords_cache_promote\(\)](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [name](#), [DictEntry::name](#), [DictEntry::name_len](#), and [sf_monotonic_ns\(\)](#).

Referenced by [vm_find_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.7 hotwords_cache_promote()

```
void hotwords_cache_promote (
    HotwordsCache * cache,
    DictEntry * word )
```

Promote a word to the hot-words cache Called when a word's execution heat exceeds threshold.

References [cache](#), and [HOTWORDS_CACHE_SIZE](#).

Referenced by [execute_colon_word\(\)](#), [hotwords_cache_lookup\(\)](#), [physics_pre_execute\(\)](#), and [rolling_window_seed_hotwords_cache\(\)](#).

Here is the caller graph for this function:

7.82.1.8 hotwords_cache_set_enabled()

```
void hotwords_cache_set_enabled (
    HotwordsCache * cache,
    int enabled )
```

Enable/disable cache at runtime (for A/B testing)

References [cache](#), and [enabled](#).

Referenced by [forth_PHYSICS_TOGGLE_CACHE\(\)](#).

Here is the caller graph for this function:

7.82.1.9 hotwords_posterior_bucket_searches()

```
BayesianLatencyPosterior hotwords_posterior_bucket_searches (
    const HotwordsStats * stats )
```

Calculate posterior for bucket searches.

Calculate posterior distribution for bucket searches Uses accumulated samples to compute credible intervals.

References [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [calculate_posterior\(\)](#), [HotwordsStats::max_bucket_search_ns](#), and [HotwordsStats::min_bucket_search_ns](#).

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.10 hotwords_posterior_cache_hits()

```
BayesianLatencyPosterior hotwords_posterior_cache_hits (
    const HotwordsStats * stats )
```

Calculate posterior for cache hits.

Calculate posterior distribution for cache hits Uses accumulated samples to compute credible intervals.

References [HotwordsStats::cache_hit_samples](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [calculate_posterior\(\)](#), [HotwordsStats::max_cache_hit_ns](#), and [HotwordsStats::min_cache_hit_ns](#).

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_speedup_estimate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.11 hotwords_speedup_estimate()

```
struct SpeedupEstimate hotwords_speedup_estimate (
    const HotwordsStats * stats )
```

Calculate speedup estimate with credible intervals Pure 64-bit fixed-point Q48.16 arithmetic throughout All results↵

_run_01_2025_12_08 FORTH-compatible: can be pushed to stack and manipulated by FORTH.

Calculate speedup estimate with uncertainty.

References [SpeedupEstimate::credible_lower_95_q48](#), [SpeedupEstimate::credible_upper_95_q48](#), [erf_q48\(\)](#), [hotwords_posterior_bucket_searches\(\)](#), [hotwords_posterior_cache_hits\(\)](#), [BayesianLatencyPosterior::mean_ns_q48](#), [SpeedupEstimate::probability_gt_10pct_q48](#), [SpeedupEstimate::probability_gt_double_q48](#), [SpeedupEstimate::speedup_factor_q48](#), [sqrt_q48\(\)](#), and [BayesianLatencyPosterior::stddev_ns_q48](#).

Referenced by [hotwords_bayesian_report\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.12 hotwords_stats_print()

```
void hotwords_stats_print (
    const HotwordsStats * stats )
```

Print statistics to stdout.

References [HotwordsStats::bucket_hits](#), [HotwordsStats::bucket_reorders](#), [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [HotwordsStats::cache_hit_samples](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [HotwordsStats::cache_hits](#), [HotwordsStats::evictions](#), [HotwordsStats::max_bucket_search_ns](#), [HotwordsStats::max_cache_hit_ns](#), [HotwordsStats::min_bucket_search_ns](#), [HotwordsStats::min_cache_hit_ns](#), [HotwordsStats::misses](#), [HotwordsStats::promotions](#), [q48_to_ns_double\(\)](#), and [sqrt_q48\(\)](#).

Referenced by [forth_BENCH_DICT_LOOKUP\(\)](#), and [forth_PHYSICS_CACHE_STATS\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.82.1.13 hotwords_stats_reset()

```
void hotwords_stats_reset (
    HotwordsStats * stats )
```

Reset statistics (for before/after comparison)

References [HotwordsStats::bucket_hits](#), [HotwordsStats::bucket_reorders](#), [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [HotwordsStats::cache_hit_samples](#), [HotwordsStats::cache_hit_total_ns_q48](#), [HotwordsStats::cache_hit_variance_sum_q48](#), [HotwordsStats::cache_hits](#), [HotwordsStats::evictions](#), [HotwordsStats::max_bucket_search_ns](#), [HotwordsStats::max_cache_hit_ns](#), [HotwordsStats::min_bucket_search_ns](#), [HotwordsStats::min_cache_hit_ns](#), [HotwordsStats::misses](#), [HotwordsStats::promotions](#), and [HotwordsStats::total_lookups](#).

Referenced by [forth_BENCH_DICT_LOOKUP\(\)](#), and [forth_PHYSICS_RESET_STATS\(\)](#).

Here is the caller graph for this function:

7.82.1.14 q48_to_ns_double()

```
static double q48_to_ns_double (
    int64_t q48 ) [inline], [static]
```

Helper: Convert Q48.16 fixed-point value to nanoseconds (double) for display only NOT used in calculations - all calculations stay in fixed-point.

Referenced by [hotwords_bayesian_report\(\)](#), and [hotwords_stats_print\(\)](#).

Here is the caller graph for this function:

7.82.2 Variable Documentation

7.82.2.1 g_hotwords_cache

```
HotwordsCache g_hotwords_cache = {0}
```

7.83 src/physics_metadata.c File Reference

```
#include "../include/physics_metadata.h"
#include "../include/ssm_jacquard.h"
#include <stddef.h>
#include <stdint.h>
#include <string.h>
```

Include dependency graph for physics_metadata.c:

Data Structures

- struct [physics_seed_definition](#)

Typedefs

- typedef struct [physics_seed_definition](#) [physics_seed_definition_t](#)

Functions

- static uint32_t [clamp_u32](#) (uint64_t value)
- static uint16_t [temperature_from_execution_heat](#) (cell_t [execution_heat](#), uint16_t prior_q8)
- static uint8_t [derive_state_flags](#) (const DictEntry *entry)
- void [physics_metadata_init](#) (DictEntry *entry, uint32_t header_bytes)
- void [physics_metadata_set_mass](#) (DictEntry *entry, uint32_t total_bytes)
- void [physics_metadata_touch](#) (DictEntry *entry, cell_t [execution_heat](#), uint64_t now_ns)
- void [physics_metadata_refresh_state](#) (DictEntry *entry)
- void [physics_metadata_record_latency](#) (DictEntry *entry, uint64_t sample_ns)
- void [physics_metadata_apply_seed](#) (DictEntry *entry)
- void [physics_metadata_apply_linear_decay](#) (DictEntry *entry, uint64_t elapsed_ns, VM *vm)

Variables

- static const [physics_seed_definition_t](#) [physics_seed_table](#) []

7.83.1 Typedef Documentation

7.83.1.1 physics_seed_definition_t

```
typedef struct physics\_seed\_definition physics\_seed\_definition\_t
```

7.83.2 Function Documentation

7.83.2.1 clamp_u32()

```
static uint32_t clamp_u32 (
    uint64_t value ) [static]
```

Referenced by [physics_metadata_record_latency\(\)](#).

Here is the caller graph for this function:

7.83.2.2 derive_state_flags()

```
static uint8_t derive_state_flags (
    const DictEntry * entry ) [static]
```

References [entry](#), [flags](#), [PHYSICS_STATE_COMPILED](#), [PHYSICS_STATE_HIDDEN](#), [PHYSICS_STATE_IMMEDIATE](#), [PHYSICS_STATE_PINNED](#), [WORD_COMPILED](#), [WORD_HIDDEN](#), [WORD_IMMEDIATE](#), and [WORD_PINNED](#).

Referenced by [physics_metadata_init\(\)](#), and [physics_metadata_refresh_state\(\)](#).

Here is the caller graph for this function:

7.83.2.3 physics_metadata_apply_linear_decay()

```
void physics_metadata_apply_linear_decay (
    DictEntry * entry,
    uint64_t elapsed_ns,
    VM * vm )
```

References [DECAY_MIN_INTERVAL](#), [entry](#), [ssm_config_t::L3_linear_decay](#), [physics_decay_slope_load\(\)](#), [VM::ssm_config](#), [vm](#), and [WORD_FROZEN](#).

Referenced by [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_pre_execute\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.83.2.4 physics_metadata_apply_seed()

```
void physics_metadata_apply_seed (
    DictEntry * entry )
```

References [entry](#), [physics_seed_definition::name](#), [physics_seed_table](#), and [physics_seed_definition::temperature_q8](#).

Referenced by [vm_create_word\(\)](#).

Here is the caller graph for this function:

7.83.2.5 physics_metadata_init()

```
void physics_metadata_init (
    DictEntry * entry,
    uint32_t header_bytes )
```

References [derive_state_flags\(\)](#), and [entry](#).

Referenced by [vm_create_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.83.2.6 physics_metadata_record_latency()

```
void physics_metadata_record_latency (
    DictEntry * entry,
    uint64_t sample_ns )
```

References [clamp_u32\(\)](#), and [entry](#).

Referenced by [profiler_word_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.83.2.7 physics_metadata_refresh_state()

```
void physics_metadata_refresh_state (
    DictEntry * entry )
```

References [derive_state_flags\(\)](#), and [entry](#).

Referenced by [vm_exit_compile_mode\(\)](#), [vm_hide_word\(\)](#), [vm_make_immediate\(\)](#), [vm_pin_execution_heat\(\)](#), [vm_smudge_word\(\)](#), and [vm_unpin_execution_heat\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.83.2.8 physics_metadata_set_mass()

```
void physics_metadata_set_mass (
    DictEntry * entry,
    uint32_t total_bytes )
```

References [entry](#).

Referenced by [vm_exit_compile_mode\(\)](#).

Here is the caller graph for this function:

7.83.2.9 physics_metadata_touch()

```
void physics_metadata_touch (
    DictEntry * entry,
    cell_t execution_heat,
    uint64_t now_ns )
```

References [entry](#), [execution_heat](#), and [temperature_from_execution_heat\(\)](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.83.2.10 temperature_from_execution_heat()

```
static uint16_t temperature_from_execution_heat (
    cell_t execution_heat,
    uint16_t prior_q8 ) [static]
```

References [execution_heat](#).

Referenced by [physics_metadata_touch\(\)](#).

Here is the caller graph for this function:

7.83.3 Variable Documentation

7.83.3.1 physics_seed_table

```
const physics_seed_definition_t physics_seed_table[] [static]
```

Initial value:

```
= {
    {"IF", 0x0100u, 400u, 0, 0},
    {"ELSE", 0x0100u, 400u, 0, 0},
    {"THEN", 0x0100u, 350u, 0, 0},
    {"BEGIN", 0x0120u, 450u, 0, 0},
    {"WHILE", 0x0120u, 450u, 0, 0},
    {"REPEAT", 0x0120u, 450u, 0, 0},
    {"DO", 0x0180u, 500u, 0, 0},
    {"LOOP", 0x0180u, 500u, 0, 0},
    {"+LOOP", 0x0200u, 520u, 0, 0},
    {"LEAVE", 0x0200u, 520u, 0, 0},
    {".", 0x0080u, 300u, 0, 0},
    {"EMIT", 0x0180u, 800u, 0, 0x02u},
    {"TYPE", 0x0180u, 900u, 0, 0x02u},
    {"BLOCK", 0x0200u, 950u, 0, 0x04u},
    {"BUFFER", 0x01C0u, 900u, 0, 0x04u},
    {"SAVE-BUFFERS", 0x0260u, 1400u, 0, 0x04u},
    {"FLUSH", 0x0240u, 1200u, 0, 0x04u},
    {"LOAD", 0x0220u, 1100u, 0, 0x04u},
    {"LIST", 0x0200u, 1000u, 0, 0x04u},
    {"SAVE-SYSTEM", 0x0280u, 2000u, 0x80u, 0x08u},
    {NULL, 0, 0, 0, 0}
}
```

Referenced by [physics_metadata_apply_seed\(\)](#).

7.84 src/physics_pipelining_metrics.c File Reference

```
#include "physics_pipelining_metrics.h"
#include "vm.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
```

Include dependency graph for physics_pipelining_metrics.c:

Functions

- static int64_t [ns_to_q48](#) (int64_t ns)
Convert nanoseconds to Q48.16 format.
- static int64_t [q48_to_ns](#) (int64_t q48)
Convert Q48.16 to nanoseconds (integer part only, truncated).
- static int64_t [q48_div_u64](#) (int64_t a_q48, uint64_t b)
Divide Q48.16 by unsigned integer, result in Q48.16.
- static int64_t [q48_mul_q48](#) (int64_t a, int64_t b)
Multiply Q48.16 by Q48.16, result in Q48.16.
- int [transition_metrics_init](#) ([WordTransitionMetrics](#) *metrics)
Initialize word transition metrics for a dictionary entry.
- int [transition_metrics_record](#) ([WordTransitionMetrics](#) *metrics, uint32_t next_word_id, uint32_t dict_size)
Record a word transition (word_from -> word_to).
- int64_t [transition_metrics_get_probability_q48](#) (const [WordTransitionMetrics](#) *metrics, uint32_t target_word_id)
Get the probability (Q48.16 format) of a specific word following this word.
- void [transition_metrics_update_cache](#) ([WordTransitionMetrics](#) *metrics, uint32_t dict_size)
Update cached most-likely-next-word information.
- void [transition_metrics_record_prefetch_hit](#) ([WordTransitionMetrics](#) *metrics, int64_t latency_saved_ns)
Record a successful prefetch prediction.
- void [transition_metrics_record_prefetch_miss](#) ([WordTransitionMetrics](#) *metrics, int64_t recovery_cost_ns)
Record a failed prefetch prediction.
- int [transition_metrics_should_speculate](#) (const [WordTransitionMetrics](#) *metrics, uint32_t target_word_id)
Check if speculation should be attempted for a given target word.
- char * [transition_metrics_stats_string](#) (const [WordTransitionMetrics](#) *metrics)
Get detailed statistics string for diagnostics.
- void [transition_metrics_reset](#) ([WordTransitionMetrics](#) *metrics)
Reset all transition metrics (clear counts, free transition_heat array).
- void [transition_metrics_cleanup](#) ([WordTransitionMetrics](#) *metrics)
Free all allocated memory for transition metrics.
- int [transition_metrics_update_context_window](#) ([WordTransitionMetrics](#) *metrics, uint32_t word_id)
Update the execution context window with a new word.
- int [transition_metrics_record_context](#) ([WordTransitionMetrics](#) *metrics, const uint32_t *context, uint32_t window_size, uint32_t next_word_id, uint32_t dict_size)
Record a context-based transition (multi-word pattern).
- char * [transition_metrics_context_accuracy_string](#) (const [WordTransitionMetrics](#) *metrics)
Get context-based prediction accuracy statistics (Phase 2).
- uint32_t [transition_metrics_binary_chop_suggest_window](#) (const [WordTransitionMetrics](#) *metrics, uint32_t current_window, double accuracy_at_current)
Get recommended window size based on collected data.

7.84.1 Function Documentation

7.84.1.1 ns_to_q48()

```
static int64_t ns_to_q48 (
    int64_t ns ) [inline], [static]
```

Convert nanoseconds to Q48.16 format.

ns -> ns_q48 where ns_q48 = ns * (1LL << 16)

Referenced by [transition_metrics_record_prefetch_hit\(\)](#), and [transition_metrics_record_prefetch_miss\(\)](#).

Here is the caller graph for this function:

7.84.1.2 q48_div_u64()

```
static int64_t q48_div_u64 (
    int64_t a_q48,
    uint64_t b ) [inline], [static]
```

Divide Q48.16 by unsigned integer, result in Q48.16.

(a_q48 / b) where a is Q48.16, b is uint64_t

Referenced by [transition_metrics_get_probability_q48\(\)](#).

Here is the caller graph for this function:

7.84.1.3 q48_mul_q48()

```
static int64_t q48_mul_q48 (
    int64_t a,
    int64_t b ) [inline], [static]
```

Multiply Q48.16 by Q48.16, result in Q48.16.

Maintains precision by dividing by (1LL << 16).

7.84.1.4 q48_to_ns()

```
static int64_t q48_to_ns (
    int64_t q48 ) [inline], [static]
```

Convert Q48.16 to nanoseconds (integer part only, truncated).

ns_q48 -> ns where ns = ns_q48 >> 16

Referenced by [transition_metrics_stats_string\(\)](#).

Here is the caller graph for this function:

7.84.1.5 transition_metrics_binary_chop_suggest_window()

```
uint32_t transition_metrics_binary_chop_suggest_window (
    const WordTransitionMetrics * metrics,
    uint32_t current_window,
    double accuracy_at_current )
```

Get recommended window size based on collected data.

Phase 2 will call this as part of binary chop tuning algorithm.

Algorithm:

1. Collect data with current window size
2. Measure prediction accuracy
3. If accuracy improves, try larger window
4. If accuracy declines, try smaller window
5. Converge to optimal size

Parameters

<i>metrics</i>	Metrics to analyze
----------------	--------------------

Parameters

<i>current_window</i>	Current window size being tested
<i>accuracy_at_current</i>	Measured accuracy with current window

Returns

Suggested next window size to try (1, 2, 4, or 8)

7.84.1.6 transition_metrics_cleanup()

```
void transition_metrics_cleanup (
    WordTransitionMetrics * metrics )
```

Free all allocated memory for transition metrics.
Called during dictionary cleanup.

Parameters

<i>metrics</i>	Metrics to clean up
----------------	---------------------

References [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window](#), and [WordTransitionMetrics::transition](#)
Referenced by [dictionary_word_forget\(\)](#).

Here is the caller graph for this function:

7.84.1.7 transition_metrics_context_accuracy_string()

```
char * transition_metrics_context_accuracy_string (
    const WordTransitionMetrics * metrics )
```

Get context-based prediction accuracy statistics (Phase 2).

Compares prediction success rates across different window sizes.

Phase 2 will use this to run binary chop tuning: Measure accuracy(window=1), accuracy(window=2), accuracy(window=4) Binary search converges to optimal window size

Parameters

<i>metrics</i>	Metrics to analyze
----------------	--------------------

Returns

Formatted string with accuracy statistics (caller must free)

References [WordTransitionMetrics::actual_window_size](#), and [WordTransitionMetrics::total_context_transitions](#).

7.84.1.8 transition_metrics_get_probability_q48()

```
int64_t transition_metrics_get_probability_q48 (
    const WordTransitionMetrics * metrics,
    uint32_t target_word_id )
```

Get the probability (Q48.16 format) of a specific word following this word.

Returns: (transition_heat[target] / total_transitions) in Q48.16 format or 0 if total_transitions == 0

Parameters

<i>metrics</i>	Metrics for the current word
<i>target_word_id</i>	Index of potential next word

Returns

Probability in Q48.16 format (0.0 to 1.0)

References [q48_div_u64\(\)](#), [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).

Referenced by [transition_metrics_should_speculate\(\)](#), and [transition_metrics_update_cache\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.84.1.9 transition_metrics_init()

```
int transition_metrics_init (
    WordTransitionMetrics * metrics )
```

Initialize word transition metrics for a dictionary entry.

Called once per word during dictionary entry creation.

Parameters

<i>metrics</i>	Pointer to uninitialized WordTransitionMetrics
----------------	--

Returns

0 on success, -1 on memory allocation failure

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window_pos](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::mispredictions](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_context_transitions](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [TRANSITION_WINDOW_SIZE](#).

Referenced by [vm_create_word\(\)](#).

Here is the caller graph for this function:

7.84.1.10 transition_metrics_record()

```
int transition_metrics_record (
    WordTransitionMetrics * metrics,
    uint32_t next_word_id,
    uint32_t dict_size )
```

Record a word transition (word_from -> word_to).

Called in the inner interpreter after each word execution.

Parameters

<i>metrics</i>	Metrics for the current word
<i>next_word_id</i>	Index of word that executed next
<i>dict_size</i>	Total number of words in dictionary (for array bounds)

Returns

0 on success, -1 on invalid word_id

References [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the caller graph for this function:

7.84.1.11 transition_metrics_record_context()

```
int transition_metrics_record_context (
    WordTransitionMetrics * metrics,
    const uint32_t * context,
```

```
uint32_t window_size,
uint32_t next_word_id,
uint32_t dict_size )
```

Record a context-based transition (multi-word pattern).

Called after each word execution with the execution window.

Example with WINDOW=2: context[0] = word_A, context[1] = word_B, next_word = word_C Records: (A,B) → C transition in sparse hash table

Phase 1: Just collects data, no analysis Phase 2: Will measure: "Do patterns with window=2 predict better than window=1?"

Parameters

<i>metrics</i>	Metrics for the current word
<i>context</i>	Array of word IDs forming the context (size = window_size)
<i>window_size</i>	Length of context array
<i>next_word_id</i>	Word that follows the context
<i>dict_size</i>	Total number of words in dictionary

Returns

0 on success, -1 on error

References [WordTransitionMetrics::total_context_transitions](#).

Referenced by [rolling_window_seed_pipelineing_context\(\)](#).

Here is the caller graph for this function:

7.84.1.12 transition_metrics_record_prefetch_hit()

```
void transition_metrics_record_prefetch_hit (
    WordTransitionMetrics * metrics,
    int64_t latency_saved_ns )
```

Record a successful prefetch prediction.

Called when speculation correctly predicted the next word.

Parameters

<i>metrics</i>	Metrics for the current word
<i>latency_saved_ns</i>	Nanoseconds saved (will be Q48.16 encoded)

References [ns_to_q48\(\)](#), [WordTransitionMetrics::prefetch_hits](#), and [WordTransitionMetrics::prefetch_latency_saved_q48](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.84.1.13 transition_metrics_record_prefetch_miss()

```
void transition_metrics_record_prefetch_miss (
    WordTransitionMetrics * metrics,
    int64_t recovery_cost_ns )
```

Record a failed prefetch prediction.

Called when speculation incorrectly predicted the next word.

Parameters

<i>metrics</i>	Metrics for the current word
<i>recovery_cost_ns</i>	Nanoseconds lost to misprediction recovery

References [WordTransitionMetrics::misprediction_cost_q48](#), [ns_to_q48\(\)](#), and [WordTransitionMetrics::prefetch_misses](#).
Here is the call graph for this function:

7.84.1.14 transition_metrics_reset()

```
void transition_metrics_reset (
    WordTransitionMetrics * metrics )
```

Reset all transition metrics (clear counts, free transition_heat array).
Called on [VM](#) reset or explicit metrics reset.

Parameters

<i>metrics</i>	Metrics to reset
----------------	------------------

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_transitions](#), [WordTransitionMetrics::context_window_pos](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction_cost_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_context_transitions](#), [WordTransitionMetrics::total_transitions](#), and [WordTransitionMetrics::transition_heat](#).
Referenced by [forth_PIPELINING_RESET_ALL\(\)](#).

Here is the caller graph for this function:

7.84.1.15 transition_metrics_should_speculate()

```
int transition_metrics_should_speculate (
    const WordTransitionMetrics * metrics,
    uint32_t target_word_id )
```

Check if speculation should be attempted for a given target word.
Applies threshold logic and ROI analysis.
Returns: 1 if speculation should be attempted, 0 otherwise

Parameters

<i>metrics</i>	Metrics for the current word
<i>target_word_id</i>	Index of potential next word

Returns

1 if probability > SPECULATION_THRESHOLD and enough samples, 0 otherwise

References [MIN_SAMPLES_FOR_SPECULATION](#), [SPECULATION_THRESHOLD_Q48](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [transition_metrics_get_probability_q48\(\)](#).
Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.84.1.16 transition_metrics_stats_string()

```
char * transition_metrics_stats_string (
    const WordTransitionMetrics * metrics )
```

Get detailed statistics string for diagnostics.
Caller must free the returned string.

Parameters

<i>metrics</i>	Metrics to report on
----------------	----------------------

Returns

Formatted statistics string (must be freed by caller)

References [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction_cost_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [q48_to_ns\(\)](#), and [WordTransitionMetrics::total_transitions](#).

Here is the call graph for this function:

7.84.1.17 transition_metrics_update_cache()

```
void transition_metrics_update_cache (
    WordTransitionMetrics * metrics,
    uint32_t dict_size )
```

Update cached most-likely-next-word information.

Should be called periodically (e.g., after every 100 transitions) to avoid stale cache.

Parameters

<i>metrics</i>	Metrics for the current word
<i>dict_size</i>	Total number of words in dictionary

References [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), and [transition_metrics_get_probability_q48\(\)](#).
Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.84.1.18 transition_metrics_update_context_window()

```
int transition_metrics_update_context_window (
    WordTransitionMetrics * metrics,
    uint32_t word_id )
```

Update the execution context window with a new word.

Called after each word execution to slide the window forward.

Example with WINDOW=2: Before: [A, B] (two previous words) After calling with C: [B, C] (shift left, add C on right)

Parameters

<i>metrics</i>	Metrics to update
<i>word_id</i>	New word ID to add to window

Returns

0 on success, -1 if context tracking not initialized

References [WordTransitionMetrics::actual_window_size](#), [WordTransitionMetrics::context_window](#), and [word_id](#).

7.85 src/physics_runtime.c File Reference

```
#include "physics_runtime.h"
#include "platform_time.h"
#include <errno.h>
#include <inttypes.h>
#include <limits.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <stdio.h>
#include <sched.h>
#include <sys/time.h>
#include <sys/types.h>
#include <time.h>
#include <unistd.h>
#include <ctype.h>
Include dependency graph for physics_runtime.c:
```

Data Structures

- struct [physics_runtime_state](#)

Macros

- #define [PATH_MAX](#) 4096

Typedefs

- typedef struct [physics_runtime_state](#) [physics_runtime_state_t](#)

Functions

- static size_t [align_up](#) (size_t value, size_t alignment)
- static uint32_t [parse_avg_milli](#) (const char *line, const char *tag)
- static int [read_psi_file](#) (const char *path, uint32_t *some10, uint32_t *some60, uint32_t *some300, uint32_t *full10, uint32_t *full60, uint32_t *full300)
- static int [read_proc_stat_totals](#) (uint64_t *total, uint64_t *idle)
- static int [resolve_cgroup_path](#) (char *out, size_t out_sz)
- static uint64_t [read_cgroup_cpu_usage_us](#) (const char *cgpath)
- static uint64_t [read_cgroup_memory_current](#) (const char *cgpath)
- static int [analytics_heap_allocate](#) (size_t requested_bytes)
- static void [analytics_heap_release](#) (void)
- static size_t [analytics_ring_free](#) (const [physics_analytics_header_t](#) *hdr)
- int [physics_runtime_init](#) (size_t analytics_heap_bytes)
Initialise physics runtime services.
- void [physics_runtime_shutdown](#) (void)
Tear down physics runtime services and release heap storage.
- const [physics_analytics_heap_t](#) * [physics_analytics_heap_info](#) (void)
Retrieve the analytics heap descriptor.
- int [physics_analytics_publish_event](#) (uint32_t channel, const void *payload, uint16_t payload_bytes)
Publish an event into the analytics ring buffer.
- static int [snapshot_posix](#) ([physics_host_snapshot_t](#) *out)
- int [physics_host_snapshot](#) ([physics_host_snapshot_t](#) *out)
Capture a host snapshot.

Variables

- static [physics_runtime_state_t](#) [g_runtime](#) = {0}

7.85.1 Macro Definition Documentation

7.85.1.1 PATH_MAX

```
#define PATH_MAX 4096
```

7.85.2 Typedef Documentation

7.85.2.1 physics_runtime_state_t

```
typedef struct physics_runtime_state physics_runtime_state_t
```

7.85.3 Function Documentation

7.85.3.1 align_up()

```
static size_t align_up (
    size_t value,
    size_t alignment ) [static]
```

Referenced by [analytics_heap_allocate\(\)](#), and [physics_analytics_publish_event\(\)](#).

Here is the caller graph for this function:

7.85.3.2 analytics_heap_allocate()

```
static int analytics_heap_allocate (
    size_t requested_bytes ) [static]
```

References [align_up\(\)](#), [physics_runtime_state::backend_seq](#), [physics_analytics_heap::base](#), [base](#), [physics_analytics_heap::bytes](#), [physics_analytics_header::consume_seq](#), [physics_analytics_header::dropped_events](#), [physics_analytics_header::flags](#), [g_runtime](#), [physics_analytics_heap::header](#), [physics_runtime_state::heap](#), [physics_analytics_header::heap_bytes](#), [physics_runtime_state::heap_ready](#), [HOLA_PROTOCOL_MAJOR](#), [HOLA_PROTOCOL_MINOR](#), [HOLA_SHARED_MAGIC](#), [physics_analytics_header::magic](#), [PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES](#), [physics_analytics_header::produce_seq](#), [physics_analytics_header::read_offset](#), [physics_analytics_heap::ring](#), [physics_analytics_header::ring_bytes](#), [physics_analytics_heap::ring_bytes](#), [physics_analytics_header::ring_offset](#), [physics_analytics_heap::scratch](#), [physics_analytics_header::scratch_bytes](#), [physics_analytics_heap::scratch_bytes](#), [physics_analytics_header::scratch_offset](#), [physics_analytics_heap::summary](#), [physics_analytics_header::summary_bytes](#), [physics_analytics_heap::summary_bytes](#), [physics_analytics_header::summary_offset](#), [physics_analytics_header::version_major](#), [physics_analytics_header::version_minor](#), and [physics_analytics_header::write_offset](#).

Referenced by [physics_runtime_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.3.3 analytics_heap_release()

```
static void analytics_heap_release (
    void ) [static]
```

References [physics_runtime_state::backend_seq](#), [physics_analytics_heap::base](#), [g_runtime](#), [physics_runtime_state::heap](#), and [physics_runtime_state::heap_ready](#).

Referenced by [physics_runtime_shutdown\(\)](#).

Here is the caller graph for this function:

7.85.3.4 analytics_ring_free()

```
static size_t analytics_ring_free (
    const physics_analytics_header_t * hdr ) [static]
```

References [physics_analytics_header::read_offset](#), [physics_analytics_header::ring_bytes](#), and [physics_analytics_header::write_offset](#).

Referenced by [physics_analytics_publish_event\(\)](#).

Here is the caller graph for this function:

7.85.3.5 parse_avg_milli()

```
static uint32_t parse_avg_milli (
    const char * line,
    const char * tag ) [static]
```

Referenced by [read_psi_file\(\)](#).

Here is the caller graph for this function:

7.85.3.6 physics_analytics_heap_info()

```
const physics_analytics_heap_t * physics_analytics_heap_info (
    void )
```

Retrieve the analytics heap descriptor.

Returns

Pointer to heap descriptor (NULL if runtime not initialised)

References [g_runtime](#), [physics_runtime_state::heap](#), and [physics_runtime_state::heap_ready](#).

7.85.3.7 physics_analytics_publish_event()

```
int physics_analytics_publish_event (
    uint32_t channel,
    const void * payload,
    uint16_t payload_bytes )
```

Publish an event into the analytics ring buffer.

Parameters

<i>channel</i>	Logical event channel
<i>payload</i>	Pointer to payload bytes
<i>payload_bytes</i>	Payload length (<= ring size)

Returns

0 on success, -1 if the payload could not be enqueued

References [align_up\(\)](#), [analytics_ring_free\(\)](#), [physics_analytics_event_header::channel](#), [physics_analytics_header::dropped_events](#), [g_runtime](#), [physics_analytics_heap::header](#), [physics_runtime_state::heap](#), [physics_runtime_state::heap_ready](#), [physics_analytics_event_header::payload_bytes](#), [physics_analytics_header::produce_seq](#), [physics_analytics_event_header::reserved](#), [physics_analytics_heap::ring](#), [physics_analytics_header::ring_bytes](#), [sf_monotonic_ns\(\)](#), [physics_analytics_event_header::timestamp](#) and [physics_analytics_header::write_offset](#).

Referenced by [physics_host_snapshot\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.3.8 physics_host_snapshot()

```
int physics_host_snapshot (
    physics_host_snapshot_t * out )
```

Capture a host snapshot.

Parameters

<i>out</i>	Output buffer
------------	---------------

Returns

0 on success, -1 on failure

References [g_runtime](#), [physics_runtime_state::heap_ready](#), [PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT](#), [physics_analytics_publish_event\(\)](#), and [snapshot_posix\(\)](#).

Here is the call graph for this function:

7.85.3.9 physics_runtime_init()

```
int physics_runtime_init (
    size_t analytics_heap_bytes )
```

Initialise physics runtime services.

Parameters

<i>analytics_heap_bytes</i>	Desired heap size (0 -> default 10 MiB)
-----------------------------	---

Returns

0 on success, -1 on failure

References [analytics_heap_allocate\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.3.10 physics_runtime_shutdown()

```
void physics_runtime_shutdown (
    void )
```

Tear down physics runtime services and release heap storage.

References [analytics_heap_release\(\)](#).

Referenced by [cleanup_and_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.3.11 read_cgroup_cpu_usage_us()

```
static uint64_t read_cgroup_cpu_usage_us (
    const char * cgroup ) [static]
```

References [PATH_MAX](#).

Referenced by [snapshot_posix\(\)](#).

Here is the caller graph for this function:

7.85.3.12 read_cgroup_memory_current()

```
static uint64_t read_cgroup_memory_current (
    const char * cgroup ) [static]
```

References [PATH_MAX](#).

Referenced by [snapshot_posix\(\)](#).

Here is the caller graph for this function:

7.85.3.13 read_proc_stat_totals()

```
static int read_proc_stat_totals (
    uint64_t * total,
    uint64_t * idle ) [static]
```

Referenced by [snapshot_posix\(\)](#).

Here is the caller graph for this function:

7.85.3.14 read_psi_file()

```
static int read_psi_file (
    const char * path,
    uint32_t * some10,
    uint32_t * some60,
    uint32_t * some300,
    uint32_t * full10,
    uint32_t * full60,
    uint32_t * full300 ) [static]
```

References [parse_avg_milli\(\)](#).

Referenced by [snapshot_posix\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.3.15 resolve_cgroup_path()

```
static int resolve_cgroup_path (
    char * out,
    size_t out_sz ) [static]
```

Referenced by [snapshot_posix\(\)](#).

Here is the caller graph for this function:

7.85.3.16 snapshot_posix()

```
static int snapshot_posix (
    physics_host_snapshot_t * out ) [static]
```

References [physics_host_snapshot::backend](#), [physics_host_snapshot::backend_seq](#), [physics_runtime_state::backend_seq](#), [physics_host_snapshot::cgroup_cpu_usage_us](#), [physics_host_snapshot::cgroup_memory_current_bytes](#), [physics_host_snapshot::cpu_idle_jiffies](#), [physics_host_snapshot::cpu_total_jiffies](#), [physics_host_snapshot::flags](#), [g_runtime](#), [physics_host_snapshot::load_avg_milli](#), [physics_host_snapshot::monotonic_time_ns](#), [PATH_MAX](#), [PHYSICS_HOST_BACKEND_POSIX](#), [PHYSICS_HOST_FLAG_CGROUP](#), [PHYSICS_HOST_FLAG_CPU_STAT](#), [PHYSICS_HOST_FLAG_PSI](#), [physics_host_snapshot::psi_cpu_full_avg10_milli](#), [physics_host_snapshot::psi_cpu_full_avg300_milli](#), [physics_host_snapshot::psi_cpu_full_avg60_milli](#), [physics_host_snapshot::psi_cpu_some_avg10_milli](#), [physics_host_snapshot::psi_io_full_avg10_milli](#), [physics_host_snapshot::psi_io_full_avg60_milli](#), [physics_host_snapshot::psi_io_some_avg10_milli](#), [physics_host_snapshot::psi_io_some_avg60_milli](#), [physics_host_snapshot::psi_mem_some_avg10_milli](#), [physics_host_snapshot::psi_mem_some_avg60_milli](#), [read_cgroup_cpu_usage_us\(\)](#), [read_cgroup_memory_current\(\)](#), [read_proc_stat_totals\(\)](#), [read_psi_file\(\)](#), [physics_host_snapshot::realtime_ns](#), [resolve_cgroup_path\(\)](#), [physics_host_snapshot::runnable](#), [physics_host_snapshot::scheduler_policy](#), [physics_host_snapshot::scheduler_priority](#), [physics_host_snapshot::scheduler_quantum](#), [sf_monotonic_ns\(\)](#), and [sf_realtime_ns\(\)](#).

Referenced by [physics_host_snapshot\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.85.4 Variable Documentation

7.85.4.1 g_runtime

```
physics_runtime_state_t g_runtime = {0} [static]
```

Referenced by [analytics_heap_allocate\(\)](#), [analytics_heap_release\(\)](#), [physics_analytics_heap_info\(\)](#), [physics_analytics_publish_event\(\)](#), [physics_host_snapshot\(\)](#), and [snapshot_posix\(\)](#).

7.86 src/platform/alloc_host.c File Reference

```
#include "../include/platform_alloc.h"
#include <stdlib.h>
#include <string.h>
```

Include dependency graph for alloc_host.c:

Functions

- int [sf_alloc_init](#) (void)
- void * [sf_malloc](#) (size_t size)
- void * [sf_calloc](#) (size_t count, size_t size)
- void * [sf_realloc](#) (void *ptr, size_t new_size)
- void [sf_free](#) (void *ptr)
- void [sf_alloc_get_stats](#) ([sf_alloc_stats_t](#) *stats)

Variables

- static [sf_alloc_stats_t](#) [g_stats](#) = {0}

7.86.1 Function Documentation

7.86.1.1 sf_alloc_get_stats()

```
void sf_alloc_get_stats (
    sf_alloc_stats_t * stats )
```

7.86.1.2 sf_alloc_init()

```
int sf_alloc_init (
    void )
```

7.86.1.3 sf_calloc()

```
void * sf_calloc (
    size_t count,
    size_t size )
```

7.86.1.4 sf_free()

```
void sf_free (
    void * ptr )
```

7.86.1.5 sf_malloc()

```
void * sf_malloc (
    size_t size )
```

7.86.1.6 sf_realloc()

```
void * sf_realloc (
    void * ptr,
    size_t new_size )
```

7.86.2 Variable Documentation

7.86.2.1 g_stats

```
sf_alloc_stats_t g_stats = {0} [static]
```

Referenced by [sf_alloc_get_stats\(\)](#), [sf_alloc_init\(\)](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [sf_malloc\(\)](#), and [sf_realloc\(\)](#).

7.87 src/platform/alloc_kernel.c File Reference

```
#include "../include/platform_alloc.h"
```

```
#include <stdint.h>
```

Include dependency graph for alloc_kernel.c:

Macros

- #define [SF_ARENA_SIZE](#) (4 * 1024 * 1024) /* 4MB default */
- #define [SF_ALIGN](#) 8
- #define [SF_ALIGN_UP](#)(x) (((x) + (SF_ALIGN - 1)) & ~(SF_ALIGN - 1))

Functions

- int [sf_alloc_init](#) (void)
- void * [sf_malloc](#) (size_t size)
- void * [sf_calloc](#) (size_t count, size_t size)
- void * [sf_realloc](#) (void *ptr, size_t new_size)

- void [sf_free](#) (void *ptr)
- void [sf_alloc_get_stats](#) ([sf_alloc_stats_t](#) *stats)

Variables

- static uint8_t [g_arena](#)[(4 * 1024 * 1024) [__attribute__](#) ((aligned(8)))]
- static size_t [g_arena_offset](#) = 0
- static int [g_initialized](#) = 0
- static [sf_alloc_stats_t](#) [g_stats](#) = {0}

7.87.1 Macro Definition Documentation

7.87.1.1 SF_ALIGN

```
#define SF_ALIGN 8
```

7.87.1.2 SF_ALIGN_UP

```
#define SF_ALIGN_UP(
    x ) ((x) + (SF_ALIGN - 1)) & ~(SF_ALIGN - 1))
```

7.87.1.3 SF_ARENA_SIZE

```
#define SF_ARENA_SIZE (4 * 1024 * 1024) /* 4MB default */
```

7.87.2 Function Documentation

7.87.2.1 sf_alloc_get_stats()

```
void sf_alloc_get_stats (
    sf\_alloc\_stats\_t * stats )
```

References [g_stats](#).

7.87.2.2 sf_alloc_init()

```
int sf_alloc_init (
    void )
```

References [sf_alloc_stats::alloc_count](#), [sf_alloc_stats::free_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ARENA_SIZE](#), [sf_alloc_stats::total_bytes](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [sf_malloc\(\)](#).

Here is the caller graph for this function:

7.87.2.3 sf_calloc()

```
void * sf_calloc (
    size_t count,
    size_t size )
```

References [sf_malloc\(\)](#).

Here is the call graph for this function:

7.87.2.4 sf_free()

```
void sf_free (
    void * ptr )
```

References [sf_alloc_stats::free_count](#), and [g_stats](#).

7.87.2.5 sf_malloc()

```
void * sf_malloc (
    size_t size )
```

References [sf_alloc_stats::alloc_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ALIGN_UP](#), [sf_alloc_init\(\)](#), [SF_ARENA_SIZE](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [sf_calloc\(\)](#), and [sf_realloc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.87.2.6 sf_realloc()

```
void * sf_realloc (
    void * ptr,
    size_t new_size )
```

References [sf_malloc\(\)](#).

Here is the call graph for this function:

7.87.3 Variable Documentation

7.87.3.1 __attribute__

```
uint8_t g_arena[(4 * 1024 * 1024) __attribute__ ((aligned(8))) (
    (aligned(8)) ) [static]
```

7.87.3.2 g_arena_offset

```
size_t g_arena_offset = 0 [static]
```

Referenced by [sf_alloc_init\(\)](#), and [sf_malloc\(\)](#).

7.87.3.3 g_initialized

```
int g_initialized = 0 [static]
```

Referenced by [sf_alloc_init\(\)](#), and [sf_malloc\(\)](#).

7.87.3.4 g_stats

```
sf_alloc_stats_t g_stats = {0} [static]
```

Referenced by [sf_alloc_get_stats\(\)](#), [sf_alloc_init\(\)](#), [sf_free\(\)](#), and [sf_malloc\(\)](#).

7.88 src/starkernel/vm/alloc_kernel.c File Reference

```
#include "platform_alloc.h"
```

```
#include <stdint.h>
```

Include dependency graph for alloc_kernel.c:

Macros

- `#define SF_ARENA_SIZE (4 * 1024 * 1024) /* 4MB default */`
- `#define SF_ALIGN 8`
- `#define SF_ALIGN_UP(x) (((x) + (SF_ALIGN - 1)) & ~(SF_ALIGN - 1))`

Functions

- `int sf_alloc_init (void)`
- `void * sf_malloc (size_t size)`
- `void * sf_calloc (size_t count, size_t size)`
- `void * sf_realloc (void *ptr, size_t new_size)`
- `void sf_free (void *ptr)`
- `void sf_alloc_get_stats (sf_alloc_stats_t *stats)`

Variables

- static uint8_t g_arena[(4 * 1024 * 1024) [__attribute__](#)__((aligned(8)))
- static size_t g_arena_offset = 0
- static int g_initialized = 0
- static [sf_alloc_stats_t](#) g_stats = {0}

7.88.1 Macro Definition Documentation

7.88.1.1 SF_ALIGN

```
#define SF_ALIGN 8
```

7.88.1.2 SF_ALIGN_UP

```
#define SF_ALIGN_UP(
    x ) ((x) + (SF_ALIGN - 1)) & ~(SF_ALIGN - 1))
```

7.88.1.3 SF_ARENA_SIZE

```
#define SF_ARENA_SIZE (4 * 1024 * 1024) /* 4MB default */
```

7.88.2 Function Documentation

7.88.2.1 sf_alloc_get_stats()

```
void sf_alloc_get_stats (
    sf\_alloc\_stats\_t * stats )
```

References [g_stats](#), and [g_stats](#).

7.88.2.2 sf_alloc_init()

```
int sf_alloc_init (
    void )
```

References [sf_alloc_stats::alloc_count](#), [sf_alloc_stats::free_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ARENA_SIZE](#), [sf_alloc_stats::total_bytes](#), and [sf_alloc_stats::used_bytes](#).

7.88.2.3 sf_calloc()

```
void * sf_calloc (
    size_t count,
    size_t size )
```

References [sf_alloc_stats::alloc_count](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [sf_malloc\(\)](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [rolling_window_find_hottest_word\(\)](#), [rolling_window_init\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelineing_context\(\)](#), and [vm_heartbeat_start_thread\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.88.2.4 sf_free()

```
void sf_free (
    void * ptr )
```

References [sf_alloc_stats::free_count](#), [g_stats](#), and [g_stats](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), [rolling_window_cleanup\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_init\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelineing_context\(\)](#), [sf_fc_clear_all\(\)](#), [sf_xrealloc\(\)](#), [vm_cleanup\(\)](#), [vm_heartbeat_start_thread\(\)](#), and [vm_heartbeat_stop_thread\(\)](#).

Here is the caller graph for this function:

7.88.2.5 sf_malloc()

```
void * sf_malloc (
    size_t size )
```

References [sf_alloc_stats::alloc_count](#), [g_arena_offset](#), [g_initialized](#), [g_stats](#), [sf_alloc_stats::peak_bytes](#), [SF_ALIGN_UP](#), [sf_alloc_init\(\)](#), [SF_ARENA_SIZE](#), and [sf_alloc_stats::used_bytes](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_stats_string\(\)](#), [vm_create_word\(\)](#), [vm_init\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.88.2.6 sf_realloc()

```
void * sf_realloc (
    void * ptr,
    size_t new_size )
```

References [sf_alloc_stats::alloc_count](#), [g_stats](#), and [sf_malloc\(\)](#).

Referenced by [sf_xrealloc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.88.3 Variable Documentation

7.88.3.1 __attribute__

```
uint8_t g_arena[(4 *1024 *1024) __attribute__ ((aligned(8))) (
    (aligned(8)) ) [static]
```

7.88.3.2 g_arena_offset

```
size_t g_arena_offset = 0 [static]
```

7.88.3.3 g_initialized

```
int g_initialized = 0 [static]
```

7.88.3.4 g_stats

```
sf_alloc_stats_t g_stats = {0} [static]
```

7.89 src/platform/l4re/time.c File Reference

```
#include "../.../include/platform_time.h"
```

Include dependency graph for time.c:

Functions

- void [sf_time_init_l4re](#) (void)

Variables

- const [sf_time_backend_t sf_time_backend_l4re](#) = {0}

7.89.1 Function Documentation

7.89.1.1 sf_time_init_l4re()

```
void sf_time_init_l4re (
    void )
```

Referenced by [sf_time_init\(\)](#).

Here is the caller graph for this function:

7.89.2 Variable Documentation

7.89.2.1 sf_time_backend_l4re

const [sf_time_backend_t](#) sf_time_backend_l4re = {0}

Referenced by [sf_time_init\(\)](#).

7.90 src/platform/linux/time.c File Reference

```
#include "../.../include/platform_time.h"
#include <stdio.h>
#include <time.h>
#include <string.h>
Include dependency graph for time.c:
```

Macros

- `#define \_POSIX\_C\_SOURCE 199309L`

Functions

- static [sf_time_ns_t](#) [posix_get_monotonic_ns](#) (void)
- static [sf_time_ns_t](#) [posix_get_realtime_ns](#) (void)
- static int [posix_set_realtime_ns](#) ([sf_time_ns_t](#) ns_since_epoch)
- static int [posix_format_timestamp](#) ([sf_time_ns_t](#) ns_since_epoch, char *buf, int format_24h)
- static int [posix_has_rtc](#) (void)

Variables

- const [sf_time_backend_t](#) [sf_time_backend_posix](#)

7.90.1 Macro Definition Documentation

7.90.1.1 _POSIX_C_SOURCE

```
#define \_POSIX\_C\_SOURCE 199309L
```

7.90.2 Function Documentation

7.90.2.1 posix_format_timestamp()

```
static int posix_format_timestamp (
    sf\_time\_ns\_t ns_since_epoch,
    char * buf,
    int format_24h ) [static]
```

References [SF_TIME_STAMP_SIZE](#).

7.90.2.2 posix_get_monotonic_ns()

```
static sf\_time\_ns\_t posix_get_monotonic_ns (
    void ) [static]
```

7.90.2.3 posix_get_realtime_ns()

```
static sf\_time\_ns\_t posix_get_realtime_ns (
    void ) [static]
```

7.90.2.4 posix_has_rtc()

```
static int posix_has_rtc (
    void ) [static]
```

7.90.2.5 posix_set_realtime_ns()

```
static int posix_set_realtime_ns (
    sf_time_ns_t ns_since_epoch ) [static]
```

7.90.3 Variable Documentation

7.90.3.1 sf_time_backend_posix

```
const sf_time_backend_t sf_time_backend_posix
```

Initial value:

```
= {
    .get_monotonic_ns = posix_get_monotonic_ns,
    .get_realtime_ns = posix_get_realtime_ns,
    .set_realtime_ns = posix_set_realtime_ns,
    .format_timestamp = posix_format_timestamp,
    .has_rtc = posix_has_rtc,
}
```

Referenced by [sf_time_init\(\)](#).

7.91 src/platform/platform_init.c File Reference

```
#include "../include/platform_time.h"
```

Include dependency graph for platform_init.c:

Functions

- void [sf_time_init](#) (void)
Initialize platform time subsystem.

Variables

- const [sf_time_backend_t](#) * [sf_time_backend](#) = NULL

7.91.1 Function Documentation

7.91.1.1 sf_time_init()

```
void sf_time_init (
    void )
```

Initialize platform time subsystem.

Selects appropriate backend based on compile-time platform:

- L4Re/StarshipOS: Uses RTC server if available, falls back to KIP clock
- POSIX: Uses `clock_gettime()`

Call this once at program startup before using any time functions.

References [sf_time_backend](#), [sf_time_backend_l4re](#), [sf_time_backend_posix](#), and [sf_time_init_l4re\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.91.2 Variable Documentation

7.91.2.1 sf_time_backend

```
const sf_time_backend_t* sf_time_backend = NULL
```

Referenced by [sf_format_timestamp\(\)](#), [sf_has_rtc\(\)](#), [sf_monotonic_ns\(\)](#), [sf_realtime_ns\(\)](#), [sf_set_realtime_ns\(\)](#), and [sf_time_init\(\)](#).

7.92 src/platform/threading.c File Reference

```
#include "../include/platform_lock.h"
#include <pthread.h>
Include dependency graph for threading.c:
```

Functions

- int [sf_mutex_init](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_destroy](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_lock](#) ([sf_mutex_t](#) *mutex)
- void [sf_mutex_unlock](#) ([sf_mutex_t](#) *mutex)

7.92.1 Function Documentation

7.92.1.1 [sf_mutex_destroy\(\)](#)

```
void sf_mutex_destroy (
    sf\_mutex\_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [hosted_mutex_destroy\(\)](#), [vm_cleanup\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.92.1.2 [sf_mutex_init\(\)](#)

```
int sf_mutex_init (
    sf\_mutex\_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [hosted_mutex_init\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.92.1.3 [sf_mutex_lock\(\)](#)

```
void sf_mutex_lock (
    sf\_mutex\_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_word_forget\(\)](#), [extract_heat_trajectory\(\)](#), [hosted_mutex_lock\(\)](#), [is_valid_dict_entry\(\)](#), [metrics_from_vm\(\)](#), [physics_on_lookup\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

Here is the caller graph for this function:

7.92.1.4 [sf_mutex_unlock\(\)](#)

```
void sf_mutex_unlock (
    sf\_mutex\_t * mutex )
```

References [sf_mutex::handle](#).

Referenced by [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [dictionary_word_forget\(\)](#), [extract_heat_trajectory\(\)](#), [hosted_mutex_unlock\(\)](#), [is_valid_dict_entry\(\)](#), [metrics_from_vm\(\)](#), [physics_on_lookup\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_interpret_word\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_apply_background_decay\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

Here is the caller graph for this function:

7.93 src/profiler.c File Reference

Performance profiling implementation for StarForth.


```
#include "../include/profiler.h"
#include "../include/platform_time.h"
#include "../include/vm.h"
#include "../include/physics_metadata.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

Include dependency graph for profiler.c:

Data Structures

- struct [WordStats](#)
Statistics for individual word execution.

Macros

- #define [_POSIX_C_SOURCE](#) 199309L

Typedefs

- typedef struct [DictEntry](#) [DictEntry](#)

Functions

- static uint64_t [get_time_ns](#) (void)
Get current time in nanoseconds.
- static void [print_word_name_safe](#) (const [DictEntry](#) *entry)
Safely print a word name, handling non-null-terminated names.
- int [profiler_init](#) ([ProfileLevel](#) level)
Initialize the profiler subsystem.
- void [profiler_shutdown](#) (void)
Shutdown the profiler and free resources.
- static [WordStats](#) * [get_word_stats](#) (const [DictEntry](#) *entry)
Find or create statistics entry for a word.
- void [profiler_word_enter](#) (const [DictEntry](#) *entry)
Record entry into a word for profiling.
- void [profiler_word_exit](#) (const [DictEntry](#) *entry)
Record exit from a word for profiling.
- void [profiler_word_count](#) (const [DictEntry](#) *entry)
Track word execution frequency without timing overhead.
- void [profiler_inc_dict_lookup](#) (void)
- void [profiler_inc_stack_op](#) (void)
- void [profiler_memory_read](#) (size_t bytes)
- void [profiler_memory_write](#) (size_t bytes)
- static int [compare_by_time](#) (const void *a, const void *b)
Compare two [WordStats](#) entries by total execution time.
- static int [compare_by_calls](#) (const void *a, const void *b)
Compare two [WordStats](#) entries by call count.
- void [profiler_generate_report](#) (void)
Generate and print detailed profiling report.
- void [profiler_reset](#) (void)
Reset all profiling statistics.
- void [profiler_set_level](#) ([ProfileLevel](#) level)

Change the current profiling detail level.

- uint64_t `profiler_get_time_ns` (void)
- void `profiler_start_timer` (ProfileCategory category)
- void `profiler_end_timer` (ProfileCategory category)
- void `profiler_memory_alloc` (size_t bytes)
- void `profiler_inc_vm_cycles` (uint64_t cycles)
- void `profiler_inc_branch` (void)
- void `profiler_inc_loop_iter` (void)
- void `profiler_print_summary` (void)
- void `profiler_print_hotspots` (void)

Print hot word analysis with optimization recommendations.

- void `profiler_print_word_analysis` (void)
- void `profiler_print_memory_analysis` (void)
- void `profiler_analyze_performance` (VM *vm)
- void `profiler_suggest_optimizations` (void)

Variables

- struct {
 - `ProfileLevel` level
 - int enabled
 - `WordStats` * word_stats
 - size_t word_count
 - size_t word_capacity
 - const char * current_word
 - uint64_t current_start_time
 - uint64_t total_dict_lookups
 - uint64_t total_stack_ops
 - uint64_t total_memory_reads
 - uint64_t total_memory_writes
 } `profiler_state` = {0}
- void * `g_profiler` = &`profiler_state`

7.93.1 Detailed Description

Performance profiling implementation for StarForth.

This module implements performance profiling functionality for the StarForth virtual machine. It tracks execution statistics including:

- Word execution counts and timing
- Dictionary lookup frequency
- Stack operations
- Memory usage patterns

The profiler supports multiple detail levels from basic to verbose profiling.

7.93.2 Macro Definition Documentation

7.93.2.1 _POSIX_C_SOURCE

```
#define _POSIX_C_SOURCE 199309L
```

7.93.3 Typedef Documentation

7.93.3.1 DictEntry

```
typedef struct DictEntry DictEntry
```

7.93.4 Function Documentation

7.93.4.1 `compare_by_calls()`

```
static int compare_by_calls (
    const void * a,
    const void * b ) [static]
```

Compare two [WordStats](#) entries by call count.

Parameters

<i>a</i>	First WordStats entry to compare
<i>b</i>	Second WordStats entry to compare

Returns

-1, 0, or 1 for qsort ordering

References [WordStats::call_count](#).

Referenced by [profiler_generate_report\(\)](#), and [profiler_print_hotspots\(\)](#).

Here is the caller graph for this function:

7.93.4.2 `compare_by_time()`

```
static int compare_by_time (
    const void * a,
    const void * b ) [static]
```

Compare two [WordStats](#) entries by total execution time.

Parameters

<i>a</i>	First WordStats entry to compare
<i>b</i>	Second WordStats entry to compare

Returns

-1, 0, or 1 for qsort ordering

References [WordStats::total_time_ns](#).

Referenced by [profiler_generate_report\(\)](#).

Here is the caller graph for this function:

7.93.4.3 `get_time_ns()`

```
static uint64_t get_time_ns (
    void ) [static]
```

Get current time in nanoseconds.

Returns

Current time in nanoseconds from monotonic clock

References [sf_monotonic_ns\(\)](#).

Referenced by [profiler_get_time_ns\(\)](#), [profiler_word_enter\(\)](#), and [profiler_word_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.93.4.4 `get_word_stats()`

```
static WordStats * get_word_stats (
    const DictEntry * entry ) [static]
```

Find or create statistics entry for a word.

Parameters

<i>entry</i>	Dictionary entry to find/create stats for
--------------	---

Returns

Pointer to word stats or NULL if not found/created

References [WordStats::call_count](#), [WordStats::entry](#), [entry](#), [WordStats::max_time_ns](#), [WordStats::min_time_ns](#), [profiler_state](#), and [WordStats::total_time_ns](#).

Referenced by [profiler_word_count\(\)](#), and [profiler_word_exit\(\)](#).

Here is the caller graph for this function:

7.93.4.5 print_word_name_safe()

```
static void print_word_name_safe (
    const DictEntry * entry ) [static]
```

Safely print a word name, handling non-null-terminated names.

Parameters

<i>entry</i>	Dictionary entry whose name to print
--------------	--------------------------------------

References [entry](#).

Referenced by [profiler_generate_report\(\)](#), and [profiler_print_hotspots\(\)](#).

Here is the caller graph for this function:

7.93.4.6 profiler_analyze_performance()

```
void profiler_analyze_performance (
    VM * vm )
```

References [vm](#).

7.93.4.7 profiler_end_timer()

```
void profiler_end_timer (
    ProfileCategory category )
```

7.93.4.8 profiler_generate_report()

```
void profiler_generate_report (
    void )
```

Generate and print detailed profiling report.

Generates a report including:

- Global statistics (dictionary lookups, stack ops, memory usage)
- Top words by execution time
- Most frequently called words

References [WordStats::call_count](#), [compare_by_calls\(\)](#), [compare_by_time\(\)](#), [WordStats::entry](#), [limit](#), [WordStats::max_time_ns](#), [name_len](#), [DictEntry::name_len](#), [print_word_name_safe\(\)](#), [PROFILE_BASIC](#), [PROFILE_DETAILED](#), [PROFILE_VERBOSE](#), [profiler_state](#), and [WordStats::total_time_ns](#).

Referenced by [profiler_print_summary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.93.4.9 profiler_get_time_ns()

```
uint64_t profiler_get_time_ns (
    void )
```

References [get_time_ns\(\)](#).

Here is the call graph for this function:

7.93.4.10 profiler_inc_branch()

```
void profiler_inc_branch (
    void )
```

7.93.4.11 profiler_inc_dict_lookup()

```
void profiler_inc_dict_lookup (
    void )
```

References [PROFILE_BASIC](#), and [profiler_state](#).

7.93.4.12 profiler_inc_loop_iter()

```
void profiler_inc_loop_iter (
    void )
```

7.93.4.13 profiler_inc_stack_op()

```
void profiler_inc_stack_op (
    void )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.93.4.14 profiler_inc_vm_cycles()

```
void profiler_inc_vm_cycles (
    uint64_t cycles )
```

7.93.4.15 profiler_init()

```
int profiler_init (
    ProfileLevel level )
```

Initialize the profiler subsystem.

Initialize the profiling system.

Parameters

<i>level</i>	Profiling detail level to enable
--------------	----------------------------------

Returns

0 on success, -1 on failure

References [level](#), [PROFILE_DISABLED](#), and [profiler_state](#).

7.93.4.16 profiler_memory_alloc()

```
void profiler_memory_alloc (
    size_t bytes )
```

7.93.4.17 profiler_memory_read()

```
void profiler_memory_read (
    size_t bytes )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.93.4.18 profiler_memory_write()

```
void profiler_memory_write (
    size_t bytes )
```

References [PROFILE_VERBOSE](#), and [profiler_state](#).

7.93.4.19 profiler_print_hotspots()

```
void profiler_print_hotspots (
    void )
```

Print hot word analysis with optimization recommendations.

Analyzes word execution frequency and provides actionable recommendations for performance optimization including:

- Assembly optimization candidates
- Inlining opportunities
- Cache-friendly ordering

References [WordStats::call_count](#), [compare_by_calls\(\)](#), [WordStats::entry](#), [limit](#), [name_len](#), [DictEntry::name_len](#), [print_word_name_safe\(\)](#), and [profiler_state](#).

Referenced by [profiler_print_word_analysis\(\)](#), and [profiler_suggest_optimizations\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.93.4.20 profiler_print_memory_analysis()

```
void profiler_print_memory_analysis (
    void )
```

7.93.4.21 profiler_print_summary()

```
void profiler_print_summary (
    void )
```

References [profiler_generate_report\(\)](#).

Here is the call graph for this function:

7.93.4.22 profiler_print_word_analysis()

```
void profiler_print_word_analysis (
    void )
```

References [profiler_print_hotspots\(\)](#).

Here is the call graph for this function:

7.93.4.23 profiler_reset()

```
void profiler_reset (
    void )
```

Reset all profiling statistics.

References [profiler_state](#).

7.93.4.24 profiler_set_level()

```
void profiler_set_level (
    ProfileLevel level )
```

Change the current profiling detail level.

Parameters

<i>level</i>	New profiling level to set
--------------	----------------------------

References [level](#), and [profiler_state](#).

7.93.4.25 profiler_shutdown()

```
void profiler_shutdown (
    void )
```

Shutdown the profiler and free resources.
Shutdown and cleanup the profiling system.
References [profiler_state](#).

7.93.4.26 profiler_start_timer()

```
void profiler_start_timer (
    ProfileCategory category )
```

7.93.4.27 profiler_suggest_optimizations()

```
void profiler_suggest_optimizations (
    void )
```

References [profiler_print_hotspots\(\)](#).
Here is the call graph for this function:

7.93.4.28 profiler_word_count()

```
void profiler_word_count (
    const DictEntry * entry )
```

Track word execution frequency without timing overhead.
Track word execution frequency without timing (PROFILE_BASIC+)

Parameters

<i>entry</i>	Dictionary entry being executed
--------------	---------------------------------

This is a lightweight alternative to `profiler_word_enter/exit` that only tracks execution counts without timing. Use for PROFILE_BASIC level.

References [WordStats::call_count](#), [entry](#), [get_word_stats\(\)](#), [PROFILE_DETAILED](#), and [profiler_state](#).
Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).
Here is the call graph for this function: Here is the caller graph for this function:

7.93.4.29 profiler_word_enter()

```
void profiler_word_enter (
    const DictEntry * entry )
```

Record entry into a word for profiling.
Start timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry being entered
--------------	--------------------------------

References [entry](#), [get_time_ns\(\)](#), [PROFILE_DETAILED](#), and [profiler_state](#).
Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).
Here is the call graph for this function: Here is the caller graph for this function:

7.93.4.30 profiler_word_exit()

```
void profiler_word_exit (
    const DictEntry * entry )
```

Record exit from a word for profiling.

End timing a word execution (PROFILE_DETAILED+)

Parameters

<i>entry</i>	Dictionary entry being exited
--------------	-------------------------------

References [WordStats::call_count](#), [entry](#), [get_time_ns\(\)](#), [get_word_stats\(\)](#), [WordStats::max_time_ns](#), [WordStats::min_time_ns](#), [physics_metadata_record_latency\(\)](#), [PROFILE_DETAILED](#), [profiler_state](#), and [WordStats::total_time_ns](#).

Referenced by [defining_runtime_dodoes\(\)](#), [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), [starforth_word_init\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.93.5 Variable Documentation

7.93.5.1 current_start_time

```
uint64_t current_start_time
```

7.93.5.2 current_word

```
const char* current_word
```

7.93.5.3 enabled

```
int enabled
```

Referenced by [hotwords_cache_set_enabled\(\)](#), and [set_forth_79_compliance\(\)](#).

7.93.5.4 g_profiler

```
void* g_profiler = &profiler_state
```

7.93.5.5 level

```
ProfileLevel level
```

Referenced by [log_message\(\)](#), [log_set_level\(\)](#), [profiler_init\(\)](#), and [profiler_set_level\(\)](#).

7.93.5.6 [struct]

```
struct { ... } profiler_state [static]
```

Referenced by [get_word_stats\(\)](#), [profiler_generate_report\(\)](#), [profiler_inc_dict_lookup\(\)](#), [profiler_inc_stack_op\(\)](#), [profiler_init\(\)](#), [profiler_memory_read\(\)](#), [profiler_memory_write\(\)](#), [profiler_print_hotspots\(\)](#), [profiler_reset\(\)](#), [profiler_set_level\(\)](#), [profiler_shutdown\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), and [profiler_word_exit\(\)](#).

7.93.5.7 total_dict_lookups

```
uint64_t total_dict_lookups
```

7.93.5.8 total_memory_reads

```
uint64_t total_memory_reads
```

7.93.5.9 total_memory_writes

```
uint64_t total_memory_writes
```

7.93.5.10 total_stack_ops

```
uint64_t total_stack_ops
```


7.93.5.11 word_capacity

```
size_t word_capacity
```

7.93.5.12 word_count

```
size_t word_count
```

Referenced by [forth_PIPELINING_STATS\(\)](#), [heartbeat_capture_tick_snapshot\(\)](#), [metrics_from_vm\(\)](#), [starforth_word_top_words\(\)](#), [starforth_word_word_execution_heat\(\)](#), [system_word_see\(\)](#), [vm_tick_inference_engine\(\)](#), and [vm_tick_slope_validator\(\)](#).

7.93.5.13 word_stats

```
WordStats* word_stats
```

7.94 src/rolling_window_of_truth.c File Reference

```
#include "rolling_window_of_truth.h"
#include "../include/vm.h"
#include "../include/rolling_window_knobs.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/physics_pipelining_metrics.h"
#include "../include/log.h"
#include "../include/platform_alloc.h"
#include <string.h>
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
```

Include dependency graph for `rolling_window_of_truth.c`:

Data Structures

- struct [rolling_window_view_t](#)

Functions

- static uint64_t [rolling_window_measure_diversity_view](#) (const [rolling_window_view_t](#) *view)
- static void [rolling_window_snapshot_view](#) (const [RollingWindowOfTruth](#) *window, [rolling_window_view_t](#) *view)
- static void [rolling_window_publish_snapshot](#) ([RollingWindowOfTruth](#) *window)
- static void [rolling_window_publish_snapshot_if_needed](#) ([RollingWindowOfTruth](#) *window)
- int [rolling_window_init](#) ([RollingWindowOfTruth](#) *window)

Initialize rolling window of truth.
- int [rolling_window_record_execution](#) ([RollingWindowOfTruth](#) *window, uint32_t word_id)

Record a word execution in the rolling window.
- uint32_t [rolling_window_get_recent_sequence](#) (const [RollingWindowOfTruth](#) *window, uint32_t depth, uint32_t *out_sequence)

Get recent execution sequence for context.
- uint32_t [rolling_window_find_hottest_word](#) (const [RollingWindowOfTruth](#) *window, uint32_t dict_size)

Get most frequent word in rolling window.
- uint64_t [rolling_window_count_transition](#) (const [RollingWindowOfTruth](#) *window, uint32_t word_a, uint32_t word_b)

Get transition frequency: how often word_b follows word_a in window.
- int [rolling_window_is_warm](#) (const [RollingWindowOfTruth](#) *window)

Check if window is warm (contains representative data).
- char * [rolling_window_stats_string](#) (const [RollingWindowOfTruth](#) *window)

Get detailed statistics string for diagnostics.

- void [rolling_window_reset](#) ([RollingWindowOfTruth](#) *window)
Reset rolling window (clear all data).
- void [rolling_window_cleanup](#) ([RollingWindowOfTruth](#) *window)
Free all memory allocated for rolling window.
- void [rolling_window_seed_hotwords_cache](#) (const [RollingWindowOfTruth](#) *window, struct [VM](#) *vm)
Seed the hot-words cache from rolling window execution history.
- void [rolling_window_seed_pipelining_context](#) (const [RollingWindowOfTruth](#) *window, struct [VM](#) *vm)
Seed pipelining context windows from rolling window.
- uint64_t [rolling_window_export_execution_history](#) (const [RollingWindowOfTruth](#) *window, uint32_t *out_↵ sequence, uint64_t max_count)
Export full execution history for external analysis.
- double [rolling_window_pattern_capture_rate](#) (const [RollingWindowOfTruth](#) *window, uint32_t test_window_↵ _size)
Analyze pattern capture rate for a given window size.
- uint64_t [rolling_window_measure_diversity](#) (const [RollingWindowOfTruth](#) *window)
Measure pattern diversity in current window.
- static void [rolling_window_run_adaptive_pass](#) ([RollingWindowOfTruth](#) *window, const [rolling_window_view_t](#) *view)
- static void [rolling_window_service_internal](#) ([RollingWindowOfTruth](#) *window)
- void [rolling_window_check_adaptive_shrink](#) ([RollingWindowOfTruth](#) *window)
Check and apply adaptive window shrinking.
- void [rolling_window_service](#) ([RollingWindowOfTruth](#) *window)

7.94.1 Function Documentation

7.94.1.1 [rolling_window_check_adaptive_shrink\(\)](#)

```
void rolling_window_check_adaptive_shrink (
    RollingWindowOfTruth * window )
```

Check and apply adaptive window shrinking.

Called periodically by [rolling_window_record_execution](#). Measures pattern diversity and shrinks window if diminishing returns detected.

This function is called automatically - users never invoke it directly. It runs every 256 executions after window becomes warm.

Parameters

<i>window</i>	Rolling window (modified in-place if shrinking occurs)
---------------	--

References [RollingWindowOfTruth::adaptive_pending](#), and [rolling_window_service_internal\(\)](#).

Here is the call graph for this function:

7.94.1.2 [rolling_window_cleanup\(\)](#)

```
void rolling_window_cleanup (
    RollingWindowOfTruth * window )
```

Free all memory allocated for rolling window.

Called during [VM](#) cleanup.

Parameters

<i>window</i>	Rolling window to clean up
---------------	----------------------------

References [RollingWindowOfTruth::execution_history](#), [sf_free\(\)](#), and [RollingWindowOfTruth::snapshot_buffers](#).

Referenced by [vm_cleanup\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.3 rolling_window_count_transition()

```
uint64_t rolling_window_count_transition (
    const RollingWindowOfTruth * window,
    uint32_t word_a,
    uint32_t word_b )
```

Get transition frequency: how often word_b follows word_a in window.
Useful for seeding pipelining context predictions.

Parameters

<i>window</i>	Rolling window
<i>word_a</i>	First word in sequence
<i>word_b</i>	Second word in sequence

Returns

Count of (word_a → word_b) transitions in window

References [rolling_window_view_t::history](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), and [rolling_window_snapshot_view\(\)](#).
Here is the call graph for this function:

7.94.1.4 rolling_window_export_execution_history()

```
uint64_t rolling_window_export_execution_history (
    const RollingWindowOfTruth * window,
    uint32_t * out_sequence,
    uint64_t max_count )
```

Export full execution history for external analysis.

Used by bootstrap analyzer to determine optimal window size.

Returns the complete circular buffer linearized in chronological order. Caller allocates output buffer; function fills it with execution sequence.

Parameters

<i>window</i>	Rolling window to export
<i>out_sequence</i>	Output buffer (caller allocates, must be >= total_executions)
<i>max_count</i>	Maximum entries to write

Returns

Number of word IDs written to out_sequence

References [rolling_window_view_t::history](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).
Referenced by [extract_heat_trajectory\(\)](#), and [rolling_window_pattern_capture_rate\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.5 rolling_window_find_hottest_word()

```
uint32_t rolling_window_find_hottest_word (
    const RollingWindowOfTruth * window,
    uint32_t dict_size )
```

Get most frequent word in rolling window.

Useful for identifying hot words immediately after POST.

Parameters

<i>window</i>	Rolling window
<i>dict_size</i>	Total number of words in dictionary (for bounds check)

Returns

Word ID of most frequent execution, or 0 if window not warm

References [rolling_window_view_t::history](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_calloc\(\)](#), [sf_free\(\)](#), and [word_id](#).

Here is the call graph for this function:

7.94.1.6 rolling_window_get_recent_sequence()

```
uint32_t rolling_window_get_recent_sequence (
    const RollingWindowOfTruth * window,
    uint32_t depth,
    uint32_t * out_sequence )
```

Get recent execution sequence for context.

Returns pointer to last N words (or fewer if not enough executions yet).

Parameters

<i>window</i>	Rolling window
<i>depth</i>	How many previous words to return (1 to ROLLING_WINDOW_SIZE)
<i>out_sequence</i>	Output array (caller allocates, must be >= depth)

Returns

Number of words written to out_sequence (may be < depth if cold start)

References [rolling_window_view_t::history](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Here is the call graph for this function:

7.94.1.7 rolling_window_init()

```
int rolling_window_init (
    RollingWindowOfTruth * window )
```

Initialize rolling window of truth.

Called during VM initialization.

Parameters

<i>window</i>	Pointer to uninitialized RollingWindowOfTruth
---------------	---

Returns

0 on success, -1 on malloc failure

References [RollingWindowOfTruth::adaptive_check_accumulator](#), [RollingWindowOfTruth::adaptive_pending](#), [RollingWindowOfTruth::effective_window_size](#), [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [RollingWindowOfTruth::last_pattern_diversity](#), [RollingWindowOfTruth::pattern_diversity_check_count](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [RollingWindowOfTruth::snapshot_buffers](#), [RollingWindowOfTruth::snapshot_effective_index](#), [RollingWindowOfTruth::snapshot_index](#), [RollingWindowOfTruth::snapshot_is_warm](#), [RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::snapshot_total_executions](#), [RollingWindowOfTruth::snapshot_window_pos](#), [RollingWindowOfTruth::total_executions](#), and [RollingWindowOfTruth::window_pos](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.8 rolling_window_is_warm()

```
int rolling_window_is_warm (
    const RollingWindowOfTruth * window )
```

Check if window is warm (contains representative data).

Seeding functions should check this before using window data.

Parameters

<i>window</i>	Rolling window
---------------	----------------

Returns

1 if window is warm ($\geq$ ROLLING_WINDOW_SIZE executions), 0 otherwise

References [RollingWindowOfTruth::is_warm](#).

7.94.1.9 rolling_window_measure_diversity()

```
uint64_t rolling_window_measure_diversity (
    const RollingWindowOfTruth * window )
```

Measure pattern diversity in current window.

Measure pattern diversity in rolling window.

Used by adaptive sizing to decide if shrinking is beneficial.

Simple heuristic: count unique word transitions observed. Higher diversity = more patterns = window is capturing something meaningful. If diversity plateaus, shrinking won't lose important data.

References [rolling_window_measure_diversity_view\(\)](#), [rolling_window_publish_snapshot_if_needed\(\)](#), and [rolling_window_snapshot_view\(\)](#).

Referenced by [dict_adaptive_optimization_pass\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.10 rolling_window_measure_diversity_view()

```
static uint64_t rolling_window_measure_diversity_view (
    const rolling_window_view_t * view ) [static]
```

References [rolling_window_view_t::effective_window_size](#), [rolling_window_view_t::history](#), [rolling_window_view_t::is_warm](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Referenced by [rolling_window_measure_diversity\(\)](#), and [rolling_window_run_adaptive_pass\(\)](#).

Here is the caller graph for this function:

7.94.1.11 rolling_window_pattern_capture_rate()

```
double rolling_window_pattern_capture_rate (
    const RollingWindowOfTruth * window,
    uint32_t test_window_size )
```

Analyze pattern capture rate for a given window size.

Measures what percentage of unique transitions are captured.

Algorithm:

1. Scan execution history with sliding window of given size
2. Count unique (context, next_word) patterns observed
3. Return: (patterns_captured / total_unique_patterns) * 100

Used for offline analysis to determine reasonable starting window size.

Parameters

<i>window</i>	Rolling window with execution data
<i>test_window_size</i>	Window size to test (e.g., 256, 512, 1024)

Returns

Pattern capture rate (0-100%)

References [rolling_window_view_t::history](#), [rolling_window_export_execution_history\(\)](#), [rolling_window_publish_snapshot_if_needed](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_free\(\)](#), [sf_malloc\(\)](#), and [rolling_window_view_t::total_executions](#).

Here is the call graph for this function:

7.94.1.12 rolling_window_publish_snapshot()

```
static void rolling_window_publish_snapshot (
    RollingWindowOfTruth * window ) [static]
```

References [RollingWindowOfTruth::effective_window_size](#), [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [ROLLING_WINDOW_SIZE](#), [RollingWindowOfTruth::snapshot_buffers](#), [RollingWindowOfTruth::snapshot_effective_window_size](#), [RollingWindowOfTruth::snapshot_index](#), [RollingWindowOfTruth::snapshot_is_warm](#), [RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::snapshot_total_executions](#), [RollingWindowOfTruth::snapshot_window_pos](#), [RollingWindowOfTruth::total_executions](#), and [RollingWindowOfTruth::window_pos](#).

Referenced by [rolling_window_init\(\)](#), and [rolling_window_publish_snapshot_if_needed\(\)](#).

Here is the caller graph for this function:

7.94.1.13 rolling_window_publish_snapshot_if_needed()

```
static void rolling_window_publish_snapshot_if_needed (
    RollingWindowOfTruth * window ) [static]
```

References [rolling_window_publish_snapshot\(\)](#), and [RollingWindowOfTruth::snapshot_pending](#).

Referenced by [rolling_window_count_transition\(\)](#), [rolling_window_export_execution_history\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_get_recent_sequence\(\)](#), [rolling_window_measure_diversity\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [rolling_window_service_internal\(\)](#), and [rolling_window_stats_string\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.14 rolling_window_record_execution()

```
int rolling_window_record_execution (
    RollingWindowOfTruth * window,
    uint32_t word_id )
```

Record a word execution in the rolling window.

Called after each word executes (in inner interpreter).

THREAD SAFETY: This function is NOT thread-safe. Caller MUST hold `vm->tuning_lock` when calling from multiple threads (main VM + heartbeat).

Called from:

- Main VM thread (inner interpreter loop, [src/vm.c:1162](#))
- Heartbeat worker thread (through [rolling_window_service](#), [src/vm.c:743](#))

MUST ALWAYS be protected by: `sf_mutex_lock(&vm->tuning_lock); rolling_window_record_execution(&vm->rolling_window, word_id); sf_mutex_unlock(&vm->tuning_lock);`

Extremely lightweight (constant time):

- Write to circular buffer
- Increment counters
- 2-3 CPU cycles (plus lock overhead ~100 cycles uncontended)

Parameters

<i>window</i>	Rolling window to update
<i>word_id</i>	Word that just executed

Returns

0 on success, -1 on error

References [RollingWindowOfTruth::adaptive_check_accumulator](#), [ADAPTIVE_CHECK_FREQUENCY](#), [RollingWindowOfTruth::adaptive_check_accumulator](#), [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [ROLLING_WINDOW_SIZE](#), [RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::total_executions](#), [RollingWindowOfTruth::window_pos](#), and [word_id](#).

Referenced by [execute_colon_word\(\)](#), and [physics_pre_execute\(\)](#).

Here is the caller graph for this function:

7.94.1.15 rolling_window_reset()

```
void rolling_window_reset (
    RollingWindowOfTruth * window )
```

Reset rolling window (clear all data).

Parameters

<i>window</i>	Rolling window to reset
---------------	-------------------------

References [RollingWindowOfTruth::execution_history](#), [RollingWindowOfTruth::is_warm](#), [ROLLING_WINDOW_SIZE](#), [RollingWindowOfTruth::snapshot_pending](#), [RollingWindowOfTruth::total_executions](#), and [RollingWindowOfTruth::window_pos](#).

7.94.1.16 rolling_window_run_adaptive_pass()

```
static void rolling_window_run_adaptive_pass (
    RollingWindowOfTruth * window,
    const rolling_window_view_t * view ) [static]
```

References [ADAPTIVE_GROWTH_THRESHOLD](#), [ADAPTIVE_MIN_WINDOW_SIZE](#), [ADAPTIVE_SHRINK_RATE](#), [RollingWindowOfTruth::effective_window_size](#), [rolling_window_view_t::history](#), [rolling_window_view_t::is_warm](#), [RollingWindowOfTruth::last_pattern_diversity](#), [LOG_DEBUG](#), [log_message\(\)](#), [RollingWindowOfTruth::pattern_diversity_check_count](#), [rolling_window_measure_diversity_view\(\)](#), [ROLLING_WINDOW_SIZE](#), and [rolling_window_view_t::total_executions](#).

Referenced by [rolling_window_service_internal\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.17 rolling_window_seed_hotwords_cache()

```
void rolling_window_seed_hotwords_cache (
    const RollingWindowOfTruth * window,
    struct VM * vm )
```

Seed the hot-words cache from rolling window execution history.

Called after POST completes, before REPL starts. Analyzes which words appeared most frequently in POST and promotes them to the cache immediately (warm-start optimization).

This ensures that the first user command benefits from knowing which words are "hot" based on representative execution data.

References [VM::dict_lock](#), [DICTIONARY_SIZE](#), [entry](#), [rolling_window_view_t::history](#), [VM::hotwords_cache](#), [hotwords_cache_promote\(\)](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [LOG_INFO](#), [log_message\(\)](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), and [word_id](#).

Here is the call graph for this function:

7.94.1.18 rolling_window_seed_pipelining_context()

```
void rolling_window_seed_pipelining_context (
    const RollingWindowOfTruth * window,
    struct VM * vm )
```

Seed pipelining context windows from rolling window.

Called after POST completes, before REPL starts. Populates the context_window for each word with actual predecessor data from POST execution.

This ensures that the first user command has contextual history for transition prediction.

References [VM::dict_lock](#), [DICTIONARY_SIZE](#), [ENABLE_PIPELINING](#), [rolling_window_view_t::history](#), [RollingWindowOfTruth::is_warm](#), [rolling_window_view_t::is_warm](#), [LOG_INFO](#), [log_message\(\)](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [DictEntry::transition_metrics](#), [transition_metrics_record_context\(\)](#), [TRANSITION_WINDOW_SIZE](#), [vm](#), and [vm_dictionary_lookup_by_word_id\(\)](#).

Here is the call graph for this function:

7.94.1.19 rolling_window_service()

```
void rolling_window_service (
    RollingWindowOfTruth * window )
```

References [rolling_window_service_internal\(\)](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.20 rolling_window_service_internal()

```
static void rolling_window_service_internal (
    RollingWindowOfTruth * window ) [static]
```

References [RollingWindowOfTruth::adaptive_pending](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [rolling_window_run_adaptive](#), and [rolling_window_snapshot_view\(\)](#).

Referenced by [rolling_window_check_adaptive_shrink\(\)](#), and [rolling_window_service\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.94.1.21 rolling_window_snapshot_view()

```
static void rolling_window_snapshot_view (
    const RollingWindowOfTruth * window,
    rolling_window_view_t * view ) [static]
```

References [rolling_window_view_t::effective_window_size](#), [rolling_window_view_t::history](#), [rolling_window_view_t::is_warm](#), [RollingWindowOfTruth::snapshot_buffers](#), [RollingWindowOfTruth::snapshot_effective_window_size](#), [RollingWindowOfTruth::snapshot_is_warm](#), [RollingWindowOfTruth::snapshot_total_executions](#), [RollingWindowOfTruth::snapshot_window_pos](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).

Referenced by [rolling_window_count_transition\(\)](#), [rolling_window_export_execution_history\(\)](#), [rolling_window_find_hottest_word\(\)](#), [rolling_window_get_recent_sequence\(\)](#), [rolling_window_measure_diversity\(\)](#), [rolling_window_pattern_capture_rate\(\)](#), [rolling_window_seed_hotwords_cache\(\)](#), [rolling_window_seed_pipelining_context\(\)](#), [rolling_window_service_internal\(\)](#), and [rolling_window_stats_string\(\)](#).

Here is the caller graph for this function:

7.94.1.22 rolling_window_stats_string()

```
char * rolling_window_stats_string (
    const RollingWindowOfTruth * window )
```

Get detailed statistics string for diagnostics.

Parameters

<i>window</i>	Rolling window
---------------	----------------

Returns

Formatted string (caller must free)

References [rolling_window_view_t::is_warm](#), [rolling_window_publish_snapshot_if_needed\(\)](#), [ROLLING_WINDOW_SIZE](#), [rolling_window_snapshot_view\(\)](#), [sf_malloc\(\)](#), [rolling_window_view_t::total_executions](#), and [rolling_window_view_t::window_pos](#).
Here is the call graph for this function:

7.95 src/ssm_jacquard.c File Reference

```
#include "ssm_jacquard.h"
#include <string.h>
Include dependency graph for ssm_jacquard.c:
```

Functions

- void [ssm_l8_init](#) ([ssm_l8_state_t](#) *state, [ssm_l8_mode_t](#) initial_mode)
Initialize L8 state.
- void [ssm_l8_update](#) (const [ssm_l8_metrics_t](#) *metrics, [ssm_l8_state_t](#) *state)
Update L8 state based on current metrics.
- void [ssm_apply_mode](#) (const [ssm_l8_state_t](#) *state, [ssm_config_t](#) *config)
Apply L8 mode to SSM configuration (set L3/L4 bits)
- const char * [ssm_l8_mode_name](#) ([ssm_l8_mode_t](#) mode)
Get human-readable mode name.

7.95.1 Function Documentation

7.95.1.1 ssm_apply_mode()

```
void ssm_apply_mode (
    const ssm\_l8\_state\_t * state,
    ssm\_config\_t * config )
```

Apply L8 mode to SSM configuration (set L3/L4 bits)

Parameters

<i>state</i>	L8 state containing current mode
<i>config</i>	SSM configuration to update (L3/L4 bits)

References [ssm_l8_state_t::current_mode](#), [ssm_config_t::L2_rolling_window](#), [ssm_config_t::L3_linear_decay](#), [ssm_config_t::L5_window_inference](#), and [ssm_config_t::L6_decay_inference](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.95.1.2 ssm_l8_init()

```
void ssm_l8_init (
    ssm\_l8\_state\_t * state,
    ssm\_l8\_mode\_t initial_mode )
```

Initialize L8 state.

Parameters

<i>state</i>	L8 state to initialize
<i>initial_mode</i>	Starting mode (default: C0_CRUISE)

References [ssm_l8_state_t::current_mode](#), [ssm_l8_state_t::hysteresis_counter](#), and [ssm_l8_state_t::pending_mode](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).
Here is the caller graph for this function:

7.95.1.3 ssm_l8_mode_name()

```
const char * ssm_l8_mode_name (
    ssm_l8_mode_t mode )
```

Get human-readable mode name.

Parameters

<i>mode</i>	Mode to describe
-------------	------------------

Returns

Static string describing mode (e.g., "C0_CRUISE")

References [SSM_MODE_C0](#), [SSM_MODE_C1](#), [SSM_MODE_C10](#), [SSM_MODE_C11](#), [SSM_MODE_C12](#), [SSM_MODE_C13](#), [SSM_MODE_C14](#), [SSM_MODE_C15](#), [SSM_MODE_C2](#), [SSM_MODE_C3](#), [SSM_MODE_C4](#), [SSM_MODE_C5](#), [SSM_MODE_C6](#), [SSM_MODE_C7](#), [SSM_MODE_C8](#), and [SSM_MODE_C9](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.95.1.4 ssm_l8_update()

```
void ssm_l8_update (
    const ssm_l8_metrics_t * metrics,
    ssm_l8_state_t * state )
```

Update L8 state based on current metrics.

Uses rule-based classification to select mode based on entropy and CV. Includes hysteresis to prevent rapid mode flapping.

Parameters

<i>metrics</i>	Current runtime metrics (ns_ewma, cv, entropy)
<i>state</i>	L8 state (updated in-place)

References [ssm_l8_state_t::current_mode](#), [ssm_l8_metrics_t::cv](#), [ssm_l8_metrics_t::entropy](#), [ssm_l8_state_t::hysteresis_counter](#), [ssm_l8_state_t::pending_mode](#), [SSM_CV_HIGH_THRESHOLD](#), [SSM_ENTROPY_HIGH_THRESHOLD](#), [SSM_HYSTERESIS_TICK](#), [SSM_MODE_C0](#), [SSM_TEMPORAL_DECAY_LOW_THRESHOLD](#), [SSM_TEMPORAL_DECAY_THRESHOLD](#), and [ssm_l8_metrics_t::temporal_decay](#).

Referenced by [vm_heartbeat_update_l8\(\)](#), and [vm_heartbeat_update_l8\(\)](#).

Here is the caller graph for this function:

7.96 src/stack_management.c File Reference

Implementation of stack operations for the Forth virtual machine.

```
#include "vm.h"
#include "log.h"
#include "profiler.h"
#include "vm_asm_opt.h"
```

Include dependency graph for stack_management.c:

Functions

- void [vm_push](#) ([VM](#) *[vm](#), [cell_t](#) value)
Push a value onto the data stack.

- [cell_t vm_pop](#) ([VM *vm](#))
Pop a value from the data stack.
- void [vm_rpush](#) ([VM *vm](#), [cell_t](#) value)
Push a value onto the return stack.
- [cell_t vm_rpop](#) ([VM *vm](#))
Pop a value from the return stack.

7.96.1 Detailed Description

Implementation of stack operations for the Forth virtual machine.

This module provides the core stack manipulation functions for both the data stack and return stack of the Forth virtual machine. It includes optimized assembly implementations for x86_64 and ARM64 architectures when available.

7.96.2 Function Documentation

7.96.2.1 vm_pop()

```
cell_t vm_pop (
    VM * vm )
```

Pop a value from the data stack.

Parameters

vm	Pointer to the VM structure
--------------------	---

Returns

The popped value

Note

Returns 0 and sets error flag if stack underflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [PROFILE_INC_STACK_OP](#), [vm](#), and [vm_pop_asm\(\)](#).

Referenced by [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_thru\(\)](#), [control_forth_0branch\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_plus_loop\(\)](#), [control_forth_runtime_qdo\(\)](#), [defining_word_constant\(\)](#), [defining_word_literal\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_sp_store\(\)](#), [double_word_2drop\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dump\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_hold\(\)](#), [format_word_question\(\)](#), [format_word_sign\(\)](#), [format_word_u_dot\(\)](#), [format_word_u_dot_r\(\)](#), [forth_acl_allow_store\(\)](#), [forth_acl_mode_store\(\)](#), [forth_acl_ttl_store\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [io_word_type\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [memory_word_2fetch\(\)](#),

memory_word_2store(), memory_word_cells(), memory_word_cfetch(), memory_word_cstore(), memory_word_erase(), memory_word_fetch(), memory_word_fill(), memory_word_minus_store(), memory_word_move(), memory_word_plus_store(), memory_word_store(), mixed_math_word_m_minus(), mixed_math_word_m_plus(), mixed_math_word_m_slash_mod(), mixed_math_word_m_star(), mixed_math_word_mod(), mixed_math_word_mod(), mixed_math_word_slash_mod(), mixed_math_word_star_slash(), mixed_math_word_star_slash_mod(), pop_xt(), return_stack_word_to_r(), runtime_abortq(), starforth_word_execution_heat_fetch(), starforth_word_execution_heat_store(), starforth_word_random(), starforth_word_seed(), starforth_word_top_words(), starforth_word_wait(), string_word_blank(), string_word_cmove(), string_word_cmove_greater(), string_word_compare(), string_word_convert(), string_word_count(), string_word_enclose(), string_word_expect(), string_word_minus_trailing(), string_word_number(), string_word_scan(), string_word_search(), string_word_skip(), string_word_word(), system_word_abort_quote(), test_acl_adaptive_ttl(), vm_api_pop(), word_divide(), and word_divmod().

Here is the call graph for this function:

7.96.2.2 vm_push()

```
void vm_push (
    VM * vm,
    cell_t value )
```

Push a value onto the data stack.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>value</i>	Value to push onto the stack

Note

Performs bounds checking to prevent stack overflow

Uses assembly optimization if available for x86_64 or ARM64

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [PROFILE_INC_STACK_OP](#), [STACK_SIZE](#), [vm](#), and [vm_push_asm\(\)](#).

Referenced by [acl_recheck\(\)](#), [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_scr\(\)](#), [block_word_thru\(\)](#), [control_forth_I\(\)](#), [control_forth_J\(\)](#), [defining_runtime_constant\(\)](#), [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_lit\(\)](#), [defining_runtime_variable\(\)](#), [defining_word_state\(\)](#), [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_state\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [dictionary_word_pad\(\)](#), [dictionary_word_sp_fetch\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [editor_word_edit\(\)](#), [emit_cell\(\)](#), [format_word_base\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_sign\(\)](#), [forth_acl_allow_fetch\(\)](#), [forth_acl_heat_fetch\(\)](#), [forth_acl_mode_fetch\(\)](#), [forth_acl_pinned_query\(\)](#), [forth_acl_ttl_fetch\(\)](#), [forth_acl_word_id\(\)](#), [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_true\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_fetch\(\)](#), [mixed_math_word_m_minus\(\)](#), [mixed_math_word_m_plus\(\)](#), [mixed_math_word_m_slash_mod\(\)](#), [mixed_math_word_m_star\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_slash_mod\(\)](#), [mixed_math_word_star_slash\(\)](#), [mixed_math_word_star_slash_mod\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_init\(\)](#), [starforth_word_random\(\)](#), [string_runtime_s_quote\(\)](#), [string_word_bl\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_s_quote\(\)](#),

[string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_tick\(\)](#), [string_word_to_in\(\)](#), [string_word_word\(\)](#), [system_word_79_standard\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [vm_api_push\(\)](#), [vm_compile_literal\(\)](#), [vm_interpret_word\(\)](#), [vocabulary_word_context\(\)](#), [vocabulary_word_current\(\)](#), [vocabulary_word_paren_find\(\)](#), [word_divide\(\)](#), and [word_divmod\(\)](#).

Here is the call graph for this function:

7.96.2.3 vm_rpop()

```
cell_t vm_rpop (
    VM * vm )
```

Pop a value from the return stack.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Returns

The popped value

Note

Returns 0 and sets error flag if stack underflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_rpop_asm\(\)](#).

Referenced by [execute_colon_word\(\)](#), [return_stack_word_r_from\(\)](#), and [vm_api_rpop\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.96.2.4 vm_rpush()

```
void vm_rpush (
    VM * vm,
    cell_t value )
```

Push a value onto the return stack.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>value</i>	Value to push onto the return stack

Note

Sets error flag if stack overflow occurs

Uses assembly optimization if available for x86_64 or ARM64

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_rpush_asm\(\)](#).

Referenced by [execute_colon_word\(\)](#), [return_stack_word_to_r\(\)](#), and [vm_api_rpush\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97 src/starkernel/arch/amd64/apic.c File Reference

```
#include <stdint.h>
#include "apic.h"
#include "console.h"
#include "vmm.h"
```

```
#include "arch.h"
Include dependency graph for apic.c:
```

Macros

- #define [LAPIC_DEFAULT_PHYS](#) 0xFEE00000u
apic.c - Local APIC and Timer (amd64)
- #define [LAPIC_VIRT_BASE](#) 0xFEE00000u /* identity-mapped */
- #define [APIC_REG_ID](#) 0x020 /* APIC ID */
- #define [APIC_REG_EOI](#) 0x0B0 /* End of Interrupt */
- #define [APIC_REG_SIVR](#) 0x0F0 /* Spurious Interrupt Vector Register */
- #define [APIC_REG_LVT_TIMER](#) 0x320 /* LVT Timer Register */
- #define [APIC_REG_TIMER_ICR](#) 0x380 /* Timer Initial Count Register */
- #define [APIC_REG_TIMER_CCR](#) 0x390 /* Timer Current Count Register */
- #define [APIC_REG_TIMER_DCR](#) 0x3E0 /* Timer Divide Configuration Register */
- #define [LVT_MASKED](#) (1u << 16) /* Interrupt masked */
- #define [LVT_MODE_PERIODIC](#) (1u << 17) /* Periodic mode */
- #define [LVT_MODE_ONESHOT](#) 0 /* One-shot mode (default) */
- #define [TIMER_DIV_1](#) 0x0B /* Divide by 1 */
- #define [TIMER_DIV_2](#) 0x00 /* Divide by 2 */
- #define [TIMER_DIV_4](#) 0x01 /* Divide by 4 */
- #define [TIMER_DIV_8](#) 0x02 /* Divide by 8 */
- #define [TIMER_DIV_16](#) 0x03 /* Divide by 16 */
- #define [TIMER_DIV_32](#) 0x08 /* Divide by 32 */
- #define [TIMER_DIV_64](#) 0x09 /* Divide by 64 */
- #define [TIMER_DIV_128](#) 0x0A /* Divide by 128 */

Functions

- static void [lapic_write](#) (uint32_t reg, uint32_t val)
- static uint32_t [lapic_read](#) (uint32_t reg)
- int [apic_init](#) (BootInfo *boot_info)
- void [apic_eoi](#) (void)
- static uint32_t [calibrate_apic_timer](#) (uint64_t tsc_hz)
Calibrate APIC timer using TSC.
- int [apic_timer_init](#) (uint64_t tsc_hz, uint32_t tick_hz)
Initialize APIC timer for periodic heartbeat.
- void [apic_timer_start](#) (void)
Start the APIC timer (enables periodic interrupts).
- void [apic_timer_stop](#) (void)
Stop the APIC timer (disables periodic interrupts).
- uint64_t [apic_timer_period_tsc](#) (void)
Get the configured timer period in TSC ticks.

Variables

- static volatile uint32_t * [lapic_base](#) = (volatile uint32_t *) (uintptr_t) 0xFEE00000u
- static uint64_t [lapic_phys_base](#) = 0xFEE00000u
- static uint32_t [timer_initial_count](#) = 0
- static uint64_t [timer_period_tsc_ticks](#) = 0
- static uint32_t [timer_tick_hz](#) = 0

7.97.1 Macro Definition Documentation

7.97.1.1 APIC_REG_EOI

```
#define APIC_REG_EOI 0x0B0 /* End of Interrupt */
```

7.97.1.2 APIC_REG_ID

```
#define APIC_REG_ID 0x020 /* APIC ID */
```

7.97.1.3 APIC_REG_LVT_TIMER

```
#define APIC_REG_LVT_TIMER 0x320 /* LVT Timer Register */
```

7.97.1.4 APIC_REG_SIVR

```
#define APIC_REG_SIVR 0x0F0 /* Spurious Interrupt Vector Register */
```

7.97.1.5 APIC_REG_TIMER_CCR

```
#define APIC_REG_TIMER_CCR 0x390 /* Timer Current Count Register */
```

7.97.1.6 APIC_REG_TIMER_DCR

```
#define APIC_REG_TIMER_DCR 0x3E0 /* Timer Divide Configuration Register */
```

7.97.1.7 APIC_REG_TIMER_ICR

```
#define APIC_REG_TIMER_ICR 0x380 /* Timer Initial Count Register */
```

7.97.1.8 LAPIC_DEFAULT_PHYS

```
#define LAPIC_DEFAULT_PHYS 0xFEE00000u
```

[apic.c](#) - Local APIC and Timer (amd64)

Provides:

- APIC initialization
- APIC timer for periodic heartbeat interrupts

7.97.1.9 LAPIC_VIRT_BASE

```
#define LAPIC_VIRT_BASE 0xFEE00000u /* identity-mapped */
```

7.97.1.10 LVT_MASKED

```
#define LVT_MASKED (1u << 16) /* Interrupt masked */
```

7.97.1.11 LVT_MODE_ONESHOT

```
#define LVT_MODE_ONESHOT 0 /* One-shot mode (default) */
```

7.97.1.12 LVT_MODE_PERIODIC

```
#define LVT_MODE_PERIODIC (1u << 17) /* Periodic mode */
```

7.97.1.13 TIMER_DIV_1

```
#define TIMER_DIV_1 0x0B /* Divide by 1 */
```

7.97.1.14 TIMER_DIV_128

```
#define TIMER_DIV_128 0x0A /* Divide by 128 */
```

7.97.1.15 TIMER_DIV_16

```
#define TIMER_DIV_16 0x03 /* Divide by 16 */
```

7.97.1.16 TIMER_DIV_2

```
#define TIMER_DIV_2 0x00 /* Divide by 2 */
```

7.97.1.17 TIMER_DIV_32

```
#define TIMER_DIV_32 0x08 /* Divide by 32 */
```

7.97.1.18 TIMER_DIV_4

```
#define TIMER_DIV_4 0x01 /* Divide by 4 */
```

7.97.1.19 TIMER_DIV_64

```
#define TIMER_DIV_64 0x09 /* Divide by 64 */
```

7.97.1.20 TIMER_DIV_8

```
#define TIMER_DIV_8 0x02 /* Divide by 8 */
```

7.97.2 Function Documentation

7.97.2.1 apic_eoi()

```
void apic_eoi (
    void )
```

References [APIC_REG_EOI](#), and [lapic_write\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97.2.2 apic_init()

```
int apic_init (
    BootInfo * boot_info )
```

References [APIC_REG_SIVR](#), [console_println\(\)](#), [lapic_base](#), [LAPIC_DEFAULT_PHYS](#), [lapic_phys_base](#), [LAPIC_VIRT_BASE](#), and [lapic_write\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97.2.3 apic_timer_init()

```
int apic_timer_init (
    uint64_t tsc_hz,
    uint32_t tick_hz )
```

Initialize APIC timer for periodic heartbeat.

Parameters

<i>tsc_hz</i>	TSC frequency in Hz (for calibration)
<i>tick_hz</i>	Desired tick frequency (e.g., 100 = 100 Hz = 10ms period)

Returns

0 on success, -1 on failure

References [APIC_REG_LVT_TIMER](#), [APIC_REG_TIMER_DCR](#), [APIC_REG_TIMER_ICR](#), [APIC_TIMER_VECTOR](#), [calibrate_apic_timer\(\)](#), [console_println\(\)](#), [console_puts\(\)](#), [lapic_write\(\)](#), [LVT_MASKED](#), [LVT_MODE_PERIODIC](#), [TIMER_DIV_1](#), [timer_initial_count](#), [timer_period_tsc_ticks](#), and [timer_tick_hz](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97.2.4 apic_timer_period_tsc()

```
uint64_t apic_timer_period_tsc (
    void )
```

Get the configured timer period in TSC ticks.

References [timer_period_tsc_ticks](#).

7.97.2.5 apic_timer_start()

```
void apic_timer_start (
    void )
```

Start the APIC timer (enables periodic interrupts).

Call this after IDT and heartbeat handler are set up.

References [APIC_REG_LVT_TIMER](#), [console_puts\(\)](#), [lapic_read\(\)](#), and [lapic_write\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97.2.6 apic_timer_stop()

```
void apic_timer_stop (
    void )
```

Stop the APIC timer (disables periodic interrupts).

References [APIC_REG_LVT_TIMER](#), [lapic_read\(\)](#), [lapic_write\(\)](#), and [LVT_MASKED](#).

Here is the call graph for this function:

7.97.2.7 calibrate_apic_timer()

```
static uint32_t calibrate_apic_timer (
    uint64_t tsc_hz ) [static]
```

Calibrate APIC timer using TSC.

Returns APIC timer ticks per TSC tick (approximate).

Strategy:

1. Set APIC timer to max count, divisor = 1
2. Wait for a fixed TSC interval
3. Read how many APIC timer ticks elapsed
4. Compute ratio

References [APIC_REG_LVT_TIMER](#), [APIC_REG_TIMER_CCR](#), [APIC_REG_TIMER_DCR](#), [APIC_REG_TIMER_ICR](#), [arch_read_timestamp\(\)](#), [arch_relax\(\)](#), [lapic_read\(\)](#), [lapic_write\(\)](#), [LVT_MASKED](#), and [TIMER_DIV_1](#).

Referenced by [apic_timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.97.2.8 lapic_read()

```
static uint32_t lapic_read (
    uint32_t reg ) [static]
```

References [lapic_base](#).

Referenced by [apic_timer_start\(\)](#), [apic_timer_stop\(\)](#), and [calibrate_apic_timer\(\)](#).

Here is the caller graph for this function:

7.97.2.9 lapic_write()

```
static void lapic_write (
    uint32_t reg,
    uint32_t val ) [static]
```

References [lapic_base](#).

Referenced by [apic_eoi\(\)](#), [apic_init\(\)](#), [apic_timer_init\(\)](#), [apic_timer_start\(\)](#), [apic_timer_stop\(\)](#), and [calibrate_apic_timer\(\)](#).

Here is the caller graph for this function:

7.97.3 Variable Documentation

7.97.3.1 `lapic_base`

```
volatile uint32_t* lapic_base = (volatile uint32_t *) (uintptr_t) 0xFEE00000u [static]
```

Referenced by [apic_init\(\)](#), [lapic_read\(\)](#), and [lapic_write\(\)](#).

7.97.3.2 `lapic_phys_base`

```
uint64_t lapic_phys_base = 0xFEE00000u [static]
```

Referenced by [apic_init\(\)](#).

7.97.3.3 `timer_initial_count`

```
uint32_t timer_initial_count = 0 [static]
```

Referenced by [apic_timer_init\(\)](#).

7.97.3.4 `timer_period_tsc_ticks`

```
uint64_t timer_period_tsc_ticks = 0 [static]
```

Referenced by [apic_timer_init\(\)](#), and [apic_timer_period_tsc\(\)](#).

7.97.3.5 `timer_tick_hz`

```
uint32_t timer_tick_hz = 0 [static]
```

Referenced by [apic_timer_init\(\)](#).

7.98 `src/starkernel/arch/amd64/arch.c` File Reference

```
#include "arch.h"
```

```
#include <stdint.h>
```

Include dependency graph for `arch.c`:

Data Structures

- struct [gdt_entry](#)
[arch.c](#) (amd64) - Architecture abstractions for StarKernel on x86_64
- struct [gdt_r](#)

Functions

- struct [gdt_entry](#) [__attribute__\(\(packed\)\)](#)
- static struct [gdt_entry](#) [gdt\[3\]](#) [__attribute__\(\(aligned\(16\)\)\)](#)
- static void [gdt_set_entry](#) (int index, uint8_t [access](#), uint8_t [granularity](#))
- static void [gdt_init](#) (void)
- void [arch_early_init](#) (void)
[arch.h](#) - Architecture abstraction layer for StarKernel
- void [arch_enable_interrupts](#) (void)
- void [arch_disable_interrupts](#) (void)
- void [arch_halt](#) (void)
- uint64_t [arch_read_timestamp](#) (void)
- void [arch_mmu_init](#) (void)

Variables

- [uint16_t limit_low](#)
- [uint16_t base_low](#)
- [uint8_t base_mid](#)
- [uint8_t access](#)
- [uint8_t granularity](#)
- [uint8_t base_high](#)
- [uint16_t limit](#)
- [uint64_t base](#)

7.98.1 Function Documentation

7.98.1.1 `__attribute__()` [1/2]

```
static struct gdt\_entry gdt[3] __attribute__ (
    (aligned(16)) ) [static]
```

7.98.1.2 `__attribute__()` [2/2]

```
struct gdt\_entry __attribute__ (
    (packed) )
```

7.98.1.3 `arch_disable_interrupts()`

```
void arch_disable_interrupts (
    void )
```

7.98.1.4 `arch_early_init()`

```
void arch_early_init (
    void )
```

[arch.h](#) - Architecture abstraction layer for StarKernel

Provides a minimal interface used across early boot, interrupt control, low-power halting, timestamp reads, and MMU bring-up.

References [gdt_init\(\)](#).

Here is the call graph for this function:

7.98.1.5 `arch_enable_interrupts()`

```
void arch_enable_interrupts (
    void )
```

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.98.1.6 `arch_halt()`

```
void arch_halt (
    void )
```

Referenced by [efi_main\(\)](#), [isr_common_handler\(\)](#), and [kernel_main\(\)](#).

Here is the caller graph for this function:

7.98.1.7 `arch_mmu_init()`

```
void arch_mmu_init (
    void )
```

7.98.1.8 arch_read_timestamp()

```
uint64_t arch_read_timestamp (
    void )
```

Referenced by [calibrate_apic_timer\(\)](#).

Here is the caller graph for this function:

7.98.1.9 gdt_init()

```
static void gdt_init (
    void ) [static]
```

References [gdt::base](#), [gdt_set_entry\(\)](#), and [gdt::limit](#).

Referenced by [arch_early_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.98.1.10 gdt_set_entry()

```
static void gdt_set_entry (
    int index,
    uint8_t access,
    uint8_t granularity ) [static]
```

References [access](#), [granularity](#), and [gdt_entry::limit_low](#).

Referenced by [gdt_init\(\)](#).

Here is the caller graph for this function:

7.98.2 Variable Documentation

7.98.2.1 access

```
uint8_t access
```

Referenced by [gdt_set_entry\(\)](#).

7.98.2.2 base

```
uint64_t base
```

Referenced by [analytics_heap_allocate\(\)](#), [blkio_ram_init_state\(\)](#), [div_ud_by_base\(\)](#), [erf_q48\(\)](#), [format_word_hash\(\)](#), [itoa_simple\(\)](#), [print_number_formatted\(\)](#), [read_devblock_4k\(\)](#), [vm_addr_from_ptr\(\)](#), [vm_create_word\(\)](#), [vm_debug_dump_state\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_parse_number\(\)](#), [vm_parse_number\(\)](#), [vocab_sync_vm_vars\(\)](#), and [write_devblock_4k\(\)](#).

7.98.2.3 base_high

```
uint8_t base_high
```

7.98.2.4 base_low

```
uint16_t base_low
```

7.98.2.5 base_mid

```
uint8_t base_mid
```

7.98.2.6 granularity

```
uint8_t granularity
```

Referenced by [gdt_set_entry\(\)](#).

7.98.2.7 limit

uint16_t limit

Referenced by [blk_allocate\(\)](#), [control_forth_runtime_do\(\)](#), [control_forth_runtime_qdo\(\)](#), [profiler_generate_report\(\)](#), [profiler_print_hotspots\(\)](#), [string_word_search\(\)](#), [xxhash64\(\)](#), and [xxhash64_update\(\)](#).

7.98.2.8 limit_low

uint16_t limit_low

7.99 src/starkernel/arch/amd64/interrupts.c File Reference

```
#include <stdint.h>
#include "arch.h"
#include "console.h"
#include "apic.h"
#include "timer.h"
#include "q48_16.h"
```

Include dependency graph for interrupts.c:

Data Structures

- struct [idt_entry](#)
- struct [idtr](#)

Macros

- #define [IDT_ENTRIES](#) 256
interrupts.c (amd64) - Interrupt setup and exception/IRQ handling
- #define [INTERRUPT_GATE](#) 0x8E

Functions

- static void [print_q48_16](#) ([q48_16_t](#) q)
- static void [outb](#) (uint16_t port, uint8_t val)
- struct [idt_entry](#) [__attribute__](#) ((packed))
- void [isr_common_handler](#) (uint64_t vector, uint64_t error_code, uint64_t rip, uint64_t cs, uint64_t rflags, uint64_t cr2, uint64_t stack_frame_ptr)
- static void [set_idt_entry](#) (int vector, void *isr)
- static void [lidt](#) (struct [idtr](#) *idtr_desc)
- static void [print_hex64](#) (uint64_t value)
- static void [print_dec_u64](#) (uint64_t v)
- static void [pic_disable](#) (void)
- static void [decode_pf_error](#) (uint64_t ec)
- void [isr_stub0](#) (void)
- void [arch_interrupts_init](#) (void)

Variables

- uint16_t [offset_low](#)
- uint16_t [selector](#)
- uint8_t [ist](#)
- uint8_t [type_attr](#)
- uint16_t [offset_mid](#)
- uint32_t [offset_high](#)
- uint32_t [zero](#)
- uint16_t [limit](#)

- `uint64_t base`
- `void * isr_stub_table [256]`
- `static struct idt_entry idt [256]`

7.99.1 Macro Definition Documentation

7.99.1.1 IDT_ENTRIES

```
#define IDT_ENTRIES 256
```

[interrupts.c](#) (amd64) - Interrupt setup and exception/IRQ handling

7.99.1.2 INTERRUPT_GATE

```
#define INTERRUPT_GATE 0x8E
```

7.99.2 Function Documentation

7.99.2.1 __attribute__()

```
struct idt_entry __attribute__ (
    (packed) )
```

7.99.2.2 arch_interrupts_init()

```
void arch_interrupts_init (
    void )
```

References [idt::base](#), [console_println\(\)](#), [idt](#), [IDT_ENTRIES](#), [isr_stub0\(\)](#), [isr_stub_table](#), [lidt\(\)](#), [idt::limit](#), [pic_disable\(\)](#), and [set_idt_entry\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.3 decode_pf_error()

```
static void decode_pf_error (
    uint64_t ec ) [static]
```

References [console_println\(\)](#), and [console_puts\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.4 isr_common_handler()

```
void isr_common_handler (
    uint64_t vector,
    uint64_t error_code,
    uint64_t rip,
    uint64_t cs,
    uint64_t rflags,
    uint64_t cr2,
    uint64_t stack_frame_ptr ) [extern]
```

References [apic_eoi\(\)](#), [APIC_TIMER_VECTOR](#), [arch_halt\(\)](#), [console_println\(\)](#), [console_puts\(\)](#), [decode_pf_error\(\)](#), [heartbeat_tick\(\)](#), [heartbeat_ticks\(\)](#), [heartbeat_trust\(\)](#), [print_dec_u64\(\)](#), [print_hex64\(\)](#), and [print_q48_16\(\)](#).

Here is the call graph for this function:

7.99.2.5 isr_stub0()

```
void isr_stub0 (
    void ) [extern]
```

Referenced by [arch_interrupts_init\(\)](#).

Here is the caller graph for this function:

7.99.2.6 lidt()

```
static void lidt (
    struct idtr * idtr_desc ) [static]
```

Referenced by [arch_interrupts_init\(\)](#).

Here is the caller graph for this function:

7.99.2.7 outb()

```
static void outb (
    uint16_t port,
    uint8_t val ) [inline], [static]
```

Referenced by [pic_disable\(\)](#).

Here is the caller graph for this function:

7.99.2.8 pic_disable()

```
static void pic_disable (
    void ) [static]
```

References [outb\(\)](#).

Referenced by [arch_interrupts_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.9 print_dec_u64()

```
static void print_dec_u64 (
    uint64_t v ) [static]
```

References [console_putc\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.10 print_hex64()

```
static void print_hex64 (
    uint64_t value ) [static]
```

References [console_putc\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.11 print_q48_16()

```
static void print_q48_16 (
    q48_16_t q ) [static]
```

References [console_putc\(\)](#), and [console_puts\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.99.2.12 set_idt_entry()

```
static void set_idt_entry (
    int vector,
    void * isr ) [static]
```

References [idt](#), [INTERRUPT_GATE](#), [idt_entry::ist](#), [idt_entry::offset_high](#), [idt_entry::offset_low](#), [idt_entry::offset_mid](#), [idt_entry::selector](#), [idt_entry::type_attr](#), and [idt_entry::zero](#).

Referenced by [arch_interrupts_init\(\)](#).

Here is the caller graph for this function:

7.99.3 Variable Documentation

7.99.3.1 base

`uint64_t base`

7.99.3.2 idt

`struct idt\_entry idt[256] [static]`

Referenced by [arch_interrupts_init\(\)](#), and [set_idt_entry\(\)](#).

7.99.3.3 isr_stub_table

`void* isr_stub_table[256] [extern]`

Referenced by [arch_interrupts_init\(\)](#).

7.99.3.4 ist

`uint8_t ist`

7.99.3.5 limit

`uint16_t limit`

7.99.3.6 offset_high

`uint32_t offset_high`

7.99.3.7 offset_low

`uint16_t offset_low`

7.99.3.8 offset_mid

`uint16_t offset_mid`

7.99.3.9 selector

`uint16_t selector`

7.99.3.10 type_attr

`uint8_t type_attr`

7.99.3.11 zero

`uint32_t zero`

Referenced by [q48_from_double\(\)](#).

7.100 src/starkernel/arch/amd64/timer.c File Reference

```
#include <stdint.h>
#include "timer.h"
#include "console.h"
#include "vmm.h"
```

Include dependency graph for timer.c:

Macros

- #define [HPET_GEN_CAP_ID](#) 0x000
timer.c - Timer policy + TSC calibration (amd64)
- #define [HPET_GEN_CONFIG](#) 0x010
- #define [HPET_MAIN_COUNTER](#) 0x0F0
- #define [HPET_PHYS_BASE](#) 0xFED00000ull
- #define [HPET_VIRT_BASE](#) 0xFED00000ull /* identity-mapped in this setup */
- #define [PIT_CHANNEL0_DATA](#) 0x40
- #define [PIT_COMMAND](#) 0x43
- #define [PIT_FREQ_HZ](#) 1193182u
- #define [PMTIMER_IO_PORT](#) 0x408
- #define [PMTIMER_FREQ_HZ](#) 3579545ull
- #define [PMTIMER_WIDTH_BITS](#) 24
- #define [PMTIMER_MASK](#) ((1ull << [PMTIMER_WIDTH_BITS](#)) - 1ull)
- #define [WARMUP_WINDOWS](#) 2
- #define [SAMPLE_WINDOWS](#) 8
- #define [WINDOW_TICKS_HPET](#) 10000ull /* ~1ms at 10MHz nominal */
- #define [WINDOW_TICKS_PIT](#) 4000u /* ~3.3ms nominal */
- #define [CV_EPSILON_PPM](#) 1000ull /* 0.1% */
- #define [CV_STABLE_COUNT](#) 6
- #define [DRIFT_EPSILON_PPM](#) 2000ull
- #define [TIMER_REQUIRE_PIT](#) 0
- #define [TIMER_VM_STRICT_INVARIANT_TSC](#) 0

Functions

- static uint64_t [muldiv64](#) (uint64_t a, uint64_t b, uint64_t c)
*muldiv64 - compute (a * b) / c without overflow using x86_64 mul/div*
- static void [print_dec](#) (uint64_t val)
- static uint64_t [rdtsc](#) (void)
- static uint64_t [rdtscp_aux](#) (uint32_t *aux)
- static void [cpuid](#) (uint32_t leaf, uint32_t subleaf, uint32_t *a, uint32_t *b, uint32_t *c, uint32_t *d)
- static void [outb](#) (uint16_t port, uint8_t val)
- static uint8_t [inb](#) (uint16_t port)
- static uint32_t [inl](#) (uint16_t port)
- static uint64_t [hpet_read](#) (uint32_t offset)
- static void [hpet_write](#) (uint32_t offset, uint64_t val)
- static void [timer_fatal](#) (const char *msg)
- static int [cpu_has_invariant_tsc](#) (void)
- static int [cpu_has_rdtscp](#) (void)
- static int [running_under_hypervisor](#) (void)
- static uint32_t [pmtimer_read](#) (void)
- static uint64_t [pmtimer_delta](#) (uint32_t start, uint32_t cur)
- static uint64_t [calibrate_tsc_with_pmtimer](#) (void)
- static void [vm_timebase_init_from_pmtimer](#) (void)
- static void [enable_hpet](#) (void)
- static void [sanity_check_hpet_running_quick](#) (void)
- static void [pit_program_reload](#) (uint16_t reload)
- static uint16_t [pit_read_counter16](#) (void)
- static int [pit_probe_running](#) (void)
- static void [update_stats](#) (const uint64_t *samples, uint32_t count, uint64_t *mean, uint64_t *stddev)
- static uint64_t [calibrate_window_hpet](#) (uint64_t target_ticks)
- static uint64_t [calibrate_window_pit](#) (uint32_t target_ticks)
- static int [converge_tsc_bare_metal](#) (uint64_t *locked_hz, uint64_t *pit_mean_out)

- static uint64_t [derive_tsc_hz_from_cpuid](#) (void)
- static void [init_timer_vm_mode](#) (void)
- int [timer_init](#) ([BootInfo](#) *boot_info)
- uint64_t [timer_tsc_hz](#) (void)
- static uint64_t [timer_now_ns_vm_relative](#) (void)
- uint64_t [timer_now_ns](#) (void)
- static int [timer_drift_check_bare_metal](#) (void)
- int [timer_check_drift_now](#) (void)
- const [timer_calibration_record_t](#) * [timer_calibration_record](#) (void)
- static void [window_push](#) ([TimeWindow](#) *w, int64_t delta)
 - Push a delta into the rolling window.*
- static [q48_16_t](#) [window_variance_q48](#) (const [TimeWindow](#) *w, uint64_t expected_delta)
 - Compute variance of window deltas.*
- static [q48_16_t](#) [variance_to_trust](#) ([q48_16_t](#) variance)
 - Derive TIME-TRUST from relative variance.*
- void [heartbeat_init](#) (uint64_t tsc_hz, uint64_t tick_hz)
 - Initialize the heartbeat subsystem.*
- void [heartbeat_tick](#) (void)
 - Called by heartbeat ISR on each tick.*
- uint64_t [heartbeat_ticks](#) (void)
 - Get current TIME-TICKS (monotonic heartbeat count).*
- [time_trust_t](#) [heartbeat_trust](#) (void)
 - Get current TIME-TRUST (Q48.16 confidence metric).*
- const [TimeTrustState](#) * [heartbeat_state](#) (void)
 - Get pointer to full heartbeat state (for diagnostics).*

Variables

- static volatile uint64_t * [hpet_regs](#) = 0
- static uint64_t [hpet_freq_hz](#) = 0
- static uint64_t [tsc_hz_locked](#) = 0
- static uint64_t [pit_tsc_hz_mean](#) = 0
- static uint32_t [windows_used](#) = 0
- static [timer_calibration_record_t](#) [calib_record](#) = {0}
- static int [pit_available](#) = 0
- static uint32_t [vm_pm_start](#) = 0
- static uint64_t [vm_ns_base](#) = 0
- static [TimeTrustState](#) [g_heartbeat](#)

7.100.1 Macro Definition Documentation

7.100.1.1 CV_EPSILON_PPM

```
#define CV_EPSILON_PPM 1000ull /* 0.1% */
```

7.100.1.2 CV_STABLE_COUNT

```
#define CV_STABLE_COUNT 6
```

7.100.1.3 DRIFT_EPSILON_PPM

```
#define DRIFT_EPSILON_PPM 2000ull
```

7.100.1.4 HPET_GEN_CAP_ID

```
#define HPET_GEN_CAP_ID 0x000
```

[timer.c](#) - Timer policy + TSC calibration (amd64)

The kernel will always run with the best time source available, but will only make determinism claims when the hardware contract is provably satisfied.

Policy split:

- [VM](#) path: avoid HPET MMIO. Prefer invariant TSC for ABSOLUTE time. Derive TSC Hz from CPUID leaves (0x15/0x16) when available; otherwise fall back to ACPI PM Timer (port I/O, stable under QEMU/KVM) to obtain TSC Hz. If invariant TSC is absent, kernel continues in RELATIVE time mode (PM Timer-based), unless strict mode is enabled.
- Bare metal path: calibrate TSC using HPET as primary reference, with optional PIT cross-check. Fail hard on non-convergence.

Failure mode: PANIC/HALT when invariants are not met.

Strict [VM](#) policy: -DTIMER_VM_STRICT_INVARIANT_TSC=1 enforces invariant TSC in [VM](#) mode (fail-hard).

7.100.1.5 HPET_GEN_CONFIG

```
#define HPET_GEN_CONFIG 0x010
```

7.100.1.6 HPET_MAIN_COUNTER

```
#define HPET_MAIN_COUNTER 0x0F0
```

7.100.1.7 HPET_PHYS_BASE

```
#define HPET_PHYS_BASE 0xFED00000ull
```

7.100.1.8 HPET_VIRT_BASE

```
#define HPET_VIRT_BASE 0xFED00000ull /* identity-mapped in this setup */
```

7.100.1.9 PIT_CHANNEL0_DATA

```
#define PIT_CHANNEL0_DATA 0x40
```

7.100.1.10 PIT_COMMAND

```
#define PIT_COMMAND 0x43
```

7.100.1.11 PIT_FREQ_HZ

```
#define PIT_FREQ_HZ 1193182u
```

7.100.1.12 PMTIMER_FREQ_HZ

```
#define PMTIMER_FREQ_HZ 3579545ull
```

7.100.1.13 PMTIMER_IO_PORT

```
#define PMTIMER_IO_PORT 0x408
```

7.100.1.14 PMTIMER_MASK

```
#define PMTIMER_MASK ((1ull << PMTIMER_WIDTH_BITS) - 1ull)
```

7.100.1.15 PMTIMER_WIDTH_BITS

```
#define PMTIMER_WIDTH_BITS 24
```

7.100.1.16 SAMPLE_WINDOWS

```
#define SAMPLE_WINDOWS 8
```

7.100.1.17 TIMER_REQUIRE_PIT

```
#define TIMER_REQUIRE_PIT 0
```

7.100.1.18 TIMER_VM_STRICT_INVARIANT_TSC

```
#define TIMER_VM_STRICT_INVARIANT_TSC 0
```

7.100.1.19 WARMUP_WINDOWS

```
#define WARMUP_WINDOWS 2
```

7.100.1.20 WINDOW_TICKS_HPET

```
#define WINDOW_TICKS_HPET 10000ull /* ~1ms at 10MHz nominal */
```

7.100.1.21 WINDOW_TICKS_PIT

```
#define WINDOW_TICKS_PIT 4000u /* ~3.3ms nominal */
```

7.100.2 Function Documentation**7.100.2.1 calibrate_tsc_with_pmtimer()**

```
static uint64_t calibrate_tsc_with_pmtimer (
    void ) [static]
```

References [muldiv64\(\)](#), [pmtimer_delta\(\)](#), [PMTIMER_FREQ_HZ](#), [PMTIMER_MASK](#), [pmtimer_read\(\)](#), and [rdtsc\(\)](#).

Referenced by [init_timer_vm_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.2 calibrate_window_hpet()

```
static uint64_t calibrate_window_hpet (
    uint64_t target_ticks ) [static]
```

References [hpet_freq_hz](#), [HPET_MAIN_COUNTER](#), [hpet_read\(\)](#), [muldiv64\(\)](#), and [rdtsc\(\)](#).

Referenced by [converge_tsc_bare_metal\(\)](#), and [timer_drift_check_bare_metal\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.3 calibrate_window_pit()

```
static uint64_t calibrate_window_pit (
    uint32_t target_ticks ) [static]
```

References [muldiv64\(\)](#), [pit_available](#), [PIT_FREQ_HZ](#), [pit_program_reload\(\)](#), [pit_read_counter16\(\)](#), and [rdtsc\(\)](#).

Referenced by [converge_tsc_bare_metal\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.4 converge_tsc_bare_metal()

```
static int converge_tsc_bare_metal (
    uint64_t * locked_hz,
    uint64_t * pit_mean_out ) [static]
```

References [calib_record](#), [calibrate_window_hpet\(\)](#), [calibrate_window_pit\(\)](#), [console_println\(\)](#), [console_putc\(\)](#), [console_puts\(\)](#), [timer_calibration_record::converged](#), [CV_EPSILON_PPM](#), [timer_calibration_record::cv_hpet_ppm](#), [timer_calibration_record::cv_pit_ppm](#), [CV_STABLE_COUNT](#), [timer_calibration_record::diff_ppm](#), [hpet_freq_hz](#), [timer_calibration_record::hpet_hz](#), [muldiv64\(\)](#), [pit_available](#), [timer_calibration_record::pit_hz_mean](#), [print_dec\(\)](#), [SAMPLE_WINDOWS](#), [timer_fatal\(\)](#), [TIMER_REQUIRE_PIT](#), [TIMER_TRUST_ABSOLUTE](#), [timer_calibration_record::trust](#),

[timer_calibration_record::tsc_hz_mean](#), [update_stats\(\)](#), [timer_calibration_record::vm_mode](#), [WARMUP_WINDOWS](#), [WINDOW_TICKS_HPET](#), [WINDOW_TICKS_PIT](#), [timer_calibration_record::windows_used](#), and [windows_used](#).
Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.5 [cpu_has_invariant_tsc\(\)](#)

```
static int cpu_has_invariant_tsc (
    void ) [static]
```

References [cpuid\(\)](#).

Referenced by [init_timer_vm_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.6 [cpu_has_rdtscp\(\)](#)

```
static int cpu_has_rdtscp (
    void ) [static]
```

References [cpuid\(\)](#).

Referenced by [init_timer_vm_mode\(\)](#), and [timer_now_ns\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.7 [cpuid\(\)](#)

```
static void cpuid (
    uint32_t leaf,
    uint32_t subleaf,
    uint32_t * a,
    uint32_t * b,
    uint32_t * c,
    uint32_t * d ) [inline], [static]
```

Referenced by [cpu_has_invariant_tsc\(\)](#), [cpu_has_rdtscp\(\)](#), [derive_tsc_hz_from_cpuid\(\)](#), and [running_under_hypervisor\(\)](#).

Here is the caller graph for this function:

7.100.2.8 [derive_tsc_hz_from_cpuid\(\)](#)

```
static uint64_t derive_tsc_hz_from_cpuid (
    void ) [static]
```

References [cpuid\(\)](#), and [muldiv64\(\)](#).

Referenced by [init_timer_vm_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.9 [enable_hpet\(\)](#)

```
static void enable_hpet (
    void ) [static]
```

References [HPET_GEN_CONFIG](#), [hpet_read\(\)](#), and [hpet_write\(\)](#).

Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.10 [heartbeat_init\(\)](#)

```
void heartbeat_init (
    uint64_t tsc_hz,
    uint64_t tick_hz )
```

Initialize the heartbeat subsystem.

Called after [timer_init\(\)](#), before enabling APIC timer.

References [time_window::count](#), [time_window::deltas](#), [time_trust_state::expected_delta](#), [g_heartbeat](#), [time_trust_state::last_tsc](#), [time_window::pos](#), [Q48_ONE](#), [time_trust_state::ticks](#), [TIME_WINDOW_SIZE](#), [time_trust_state::total_samples](#), [time_trust_state::trust](#), [time_trust_state::variance](#), and [time_trust_state::window](#).

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.100.2.11 heartbeat_state()

```
const TimeTrustState * heartbeat_state (
    void )
```

Get pointer to full heartbeat state (for diagnostics).

References [g_heartbeat](#).

7.100.2.12 heartbeat_tick()

```
void heartbeat_tick (
    void )
```

Called by heartbeat ISR on each tick.

Updates TIME-TICKS, samples TSC, updates rolling window and TIME-TRUST.

References [time_trust_state::expected_delta](#), [g_heartbeat](#), [time_trust_state::last_tsc](#), [rdtsc\(\)](#), [time_trust_state::ticks](#), [time_trust_state::total_samples](#), [time_trust_state::trust](#), [time_trust_state::variance](#), [variance_to_trust\(\)](#), [time_trust_state::window](#), [window_push\(\)](#), and [window_variance_q48\(\)](#).

Referenced by [isr_common_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.13 heartbeat_ticks()

```
uint64_t heartbeat_ticks (
    void )
```

Get current TIME-TICKS (monotonic heartbeat count).

References [g_heartbeat](#), and [time_trust_state::ticks](#).

Referenced by [isr_common_handler\(\)](#).

Here is the caller graph for this function:

7.100.2.14 heartbeat_trust()

```
time_trust_t heartbeat_trust (
    void )
```

Get current TIME-TRUST (Q48.16 confidence metric).

References [g_heartbeat](#), and [time_trust_state::trust](#).

Referenced by [isr_common_handler\(\)](#).

Here is the caller graph for this function:

7.100.2.15 hpet_read()

```
static uint64_t hpet_read (
    uint32_t offset ) [inline], [static]
```

References [hpet_regs](#).

Referenced by [calibrate_window_hpet\(\)](#), [enable_hpet\(\)](#), [sanity_check_hpet_running_quick\(\)](#), and [timer_init\(\)](#).

Here is the caller graph for this function:

7.100.2.16 hpet_write()

```
static void hpet_write (
    uint32_t offset,
    uint64_t val ) [inline], [static]
```

References [hpet_regs](#).

Referenced by [enable_hpet\(\)](#).

Here is the caller graph for this function:

7.100.2.17 inb()

```
static uint8_t inb (
    uint16_t port ) [inline], [static]
```

Referenced by [pit_read_counter16\(\)](#).

Here is the caller graph for this function:

7.100.2.18 init_timer_vm_mode()

```
static void init_timer_vm_mode (
    void ) [static]
```

References [calib_record](#), [calibrate_tsc_with_pmtimer\(\)](#), [console_println\(\)](#), [console_puts\(\)](#), [timer_calibration_record::converged](#), [cpu_has_invariant_tsc\(\)](#), [cpu_has_rdtscp\(\)](#), [timer_calibration_record::cv_hpet_ppm](#), [timer_calibration_record::cv_pit_ppm](#), [derive_tsc_hz_from_cpuid\(\)](#), [timer_calibration_record::diff_ppm](#), [timer_calibration_record::hpet_hz](#), [timer_calibration_record::pit_hz_r](#), [print_dec\(\)](#), [timer_fatal\(\)](#), [TIMER_TRUST_ABSOLUTE](#), [TIMER_TRUST_RELATIVE](#), [TIMER_VM_STRICT_INVARIANT_TSC](#), [timer_calibration_record::trust](#), [tsc_hz_locked](#), [timer_calibration_record::tsc_hz_mean](#), [timer_calibration_record::vm_mode](#), [vm_timebase_init_from_pmtimer\(\)](#), and [timer_calibration_record::windows_used](#).

Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.19 inl()

```
static uint32_t inl (
    uint16_t port ) [inline], [static]
```

Referenced by [pmtimer_read\(\)](#).

Here is the caller graph for this function:

7.100.2.20 muldiv64()

```
static uint64_t muldiv64 (
    uint64_t a,
    uint64_t b,
    uint64_t c ) [inline], [static]
```

muldiv64 - compute (a * b) / c without overflow using x86_64 mul/div

Uses 64x64->128 multiplication followed by 128/64->64 division. This is pure inline assembly, no libgcc required.

Referenced by [calibrate_tsc_with_pmtimer\(\)](#), [calibrate_window_hpet\(\)](#), [calibrate_window_pit\(\)](#), [converge_tsc_bare_metal\(\)](#), [derive_tsc_hz_from_cpuid\(\)](#), [timer_drift_check_bare_metal\(\)](#), [timer_now_ns\(\)](#), and [timer_now_ns_vm_relative\(\)](#).

Here is the caller graph for this function:

7.100.2.21 outb()

```
static void outb (
    uint16_t port,
    uint8_t val ) [inline], [static]
```

Referenced by [pit_program_reload\(\)](#), and [pit_read_counter16\(\)](#).

Here is the caller graph for this function:

7.100.2.22 pit_probe_running()

```
static int pit_probe_running (
    void ) [static]
```

References [pit_program_reload\(\)](#), and [pit_read_counter16\(\)](#).

Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.23 pit_program_reload()

```
static void pit_program_reload (
    uint16_t reload ) [static]
```

References [outb\(\)](#), [PIT_CHANNEL0_DATA](#), and [PIT_COMMAND](#).

Referenced by [calibrate_window_pit\(\)](#), and [pit_probe_running\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.24 pit_read_counter16()

```
static uint16_t pit_read_counter16 (
    void ) [static]
```

References [inb\(\)](#), [outb\(\)](#), [PIT_CHANNEL0_DATA](#), and [PIT_COMMAND](#).

Referenced by [calibrate_window_pit\(\)](#), and [pit_probe_running\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.25 pmtimer_delta()

```
static uint64_t pmtimer_delta (
    uint32_t start,
    uint32_t cur ) [static]
```

References [PMTIMER_MASK](#).

Referenced by [calibrate_tsc_with_pmtimer\(\)](#), and [timer_now_ns_vm_relative\(\)](#).

Here is the caller graph for this function:

7.100.2.26 pmtimer_read()

```
static uint32_t pmtimer_read (
    void ) [inline], [static]
```

References [inl\(\)](#), and [PMTIMER_IO_PORT](#).

Referenced by [calibrate_tsc_with_pmtimer\(\)](#), [timer_now_ns_vm_relative\(\)](#), and [vm_timebase_init_from_pmtimer\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.27 print_dec()

```
static void print_dec (
    uint64_t val ) [static]
```

References [console_putc\(\)](#).

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), and [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.28 rdtsc()

```
static uint64_t rdtsc (
    void ) [inline], [static]
```

Referenced by [calibrate_tsc_with_pmtimer\(\)](#), [calibrate_window_hpet\(\)](#), [calibrate_window_pit\(\)](#), [heartbeat_tick\(\)](#), and [timer_now_ns\(\)](#).

Here is the caller graph for this function:

7.100.2.29 rdtscp_aux()

```
static uint64_t rdtscp_aux (
    uint32_t * aux ) [inline], [static]
```

Referenced by [timer_now_ns\(\)](#).

Here is the caller graph for this function:

7.100.2.30 running_under_hypervisor()

```
static int running_under_hypervisor (
    void ) [static]
```

References [cpuid\(\)](#).

Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.31 sanity_check_hpet_running_quick()

```
static void sanity_check_hpet_running_quick (
    void ) [static]
```

References [HPET_MAIN_COUNTER](#), [hpet_read\(\)](#), and [timer_fatal\(\)](#).

Referenced by [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.32 timer_calibration_record()

```
const timer_calibration_record_t * timer_calibration_record (
    void )
```

References [calib_record](#).

7.100.2.33 timer_check_drift_now()

```
int timer_check_drift_now (
    void )
```

References [calib_record](#), [timer_drift_check_bare_metal\(\)](#), and [timer_calibration_record::vm_mode](#).

Here is the call graph for this function:

7.100.2.34 timer_drift_check_bare_metal()

```
static int timer_drift_check_bare_metal (
    void ) [static]
```

References [calibrate_window_hpet\(\)](#), [DRIFT_EPSILON_PPM](#), [muldiv64\(\)](#), [timer_fatal\(\)](#), [tsc_hz_locked](#), and [WINDOW_TICKS_HPET](#).

Referenced by [timer_check_drift_now\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.35 timer_fatal()

```
static void timer_fatal (
    const char * msg ) [static]
```

References [console_println\(\)](#).

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), [sanity_check_hpet_running_quick\(\)](#), [timer_drift_check_bare_metal\(\)](#), and [timer_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.36 timer_init()

```
int timer_init (
    BootInfo * boot_info )
```

References [calib_record](#), [console_println\(\)](#), [console_puts\(\)](#), [converge_tsc_bare_metal\(\)](#), [timer_calibration_record::converged](#), [enable_hpet\(\)](#), [hpet_freq_hz](#), [HPET_GEN_CAP_ID](#), [HPET_PHYS_BASE](#), [hpet_read\(\)](#), [hpet_regs](#), [HPET_VIRT_BASE](#), [init_timer_vm_mode\(\)](#), [pit_available](#), [pit_probe_running\(\)](#), [pit_tsc_hz_mean](#), [print_dec\(\)](#), [running_under_hypervisor\(\)](#), [sanity_check_hpet_running_quick\(\)](#), [timer_fatal\(\)](#), [TIMER_REQUIRE_PIT](#), [TIMER_TRUST_NONE](#), [timer_calibration_record::trust](#), [tsc_hz_locked](#), [timer_calibration_record::vm_mode](#), [VMM_FLAG_WRITABLE](#), and [vmm_map_range\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.37 timer_now_ns()

```
uint64_t timer_now_ns (
    void )
```

References [calib_record](#), [cpu_has_rdtscp\(\)](#), [muldiv64\(\)](#), [rdtsc\(\)](#), [rdtscp_aux\(\)](#), [timer_now_ns_vm_relative\(\)](#), [TIMER_TRUST_ABSOLUTE](#), [TIMER_TRUST_NONE](#), [timer_calibration_record::trust](#), [tsc_hz_locked](#), and [timer_calibration_record::vm_mode](#).

Here is the call graph for this function:

7.100.2.38 timer_now_ns_vm_relative()

```
static uint64_t timer_now_ns_vm_relative (
    void ) [static]
```

References [muldiv64\(\)](#), [pmtimer_delta\(\)](#), [PMTIMER_FREQ_HZ](#), [PMTIMER_MASK](#), [pmtimer_read\(\)](#), [vm_ns_base](#), and [vm_pm_start](#).

Referenced by [timer_now_ns\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.39 timer_tsc_hz()

```
uint64_t timer_tsc_hz (
    void )
```

References [tsc_hz_locked](#).

Referenced by [kernel_main\(\)](#).

Here is the caller graph for this function:

7.100.2.40 update_stats()

```
static void update_stats (
    const uint64_t * samples,
    uint32_t count,
    uint64_t * mean,
    uint64_t * stddev ) [static]
```

Referenced by [converge_tsc_bare_metal\(\)](#).

Here is the caller graph for this function:

7.100.2.41 variance_to_trust()

```
static q48_16_t variance_to_trust (
    q48_16_t variance ) [static]
```

Derive TIME-TRUST from relative variance.

Formula: $\text{trust} = 1.0 / (1.0 + \text{variance})$

References [q48_add\(\)](#), [q48_div\(\)](#), and [Q48_ONE](#).

Referenced by [heartbeat_tick\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.42 vm_timebase_init_from_pmtimer()

```
static void vm_timebase_init_from_pmtimer (
    void ) [static]
```

References [PMTIMER_MASK](#), [pmtimer_read\(\)](#), [vm_ns_base](#), and [vm_pm_start](#).

Referenced by [init_timer_vm_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.100.2.43 window_push()

```
static void window_push (
    TimeWindow * w,
    int64_t delta ) [static]
```

Push a delta into the rolling window.

References [time_window::count](#), [time_window::deltas](#), [time_window::pos](#), and [TIME_WINDOW_SIZE](#).

Referenced by [heartbeat_tick\(\)](#).

Here is the caller graph for this function:

7.100.2.44 window_variance_q48()

```
static q48_16_t window_variance_q48 (
    const TimeWindow * w,
    uint64_t expected_delta ) [static]
```

Compute variance of window deltas.

Returns as Q48.16 relative variance (normalized by expected_delta^2).

References [time_window::count](#), and [time_window::deltas](#).

Referenced by [heartbeat_tick\(\)](#).

Here is the caller graph for this function:

7.100.3 Variable Documentation

7.100.3.1 calib_record

```
timer_calibration_record_t calib_record = {0} [static]
```

Referenced by [converge_tsc_bare_metal\(\)](#), [init_timer_vm_mode\(\)](#), [timer_calibration_record\(\)](#), [timer_check_drift_now\(\)](#), [timer_init\(\)](#), and [timer_now_ns\(\)](#).

7.100.3.2 g_heartbeat

```
TimeTrustState g_heartbeat [static]
```

Referenced by [heartbeat_init\(\)](#), [heartbeat_state\(\)](#), [heartbeat_tick\(\)](#), [heartbeat_ticks\(\)](#), and [heartbeat_trust\(\)](#).

7.100.3.3 hpet_freq_hz

```
uint64_t hpet_freq_hz = 0 [static]
```

Referenced by [calibrate_window_hpet\(\)](#), [converge_tsc_bare_metal\(\)](#), and [timer_init\(\)](#).

7.100.3.4 hpet_regs

```
volatile uint64_t* hpet_regs = 0 [static]
```

Referenced by [hpet_read\(\)](#), [hpet_write\(\)](#), and [timer_init\(\)](#).

7.100.3.5 pit_available

```
int pit_available = 0 [static]
```

Referenced by [calibrate_window_pit\(\)](#), [converge_tsc_bare_metal\(\)](#), and [timer_init\(\)](#).

7.100.3.6 pit_tsc_hz_mean

```
uint64_t pit_tsc_hz_mean = 0 [static]
```

Referenced by [timer_init\(\)](#).

7.100.3.7 tsc_hz_locked

```
uint64_t tsc_hz_locked = 0 [static]
```

Referenced by [init_timer_vm_mode\(\)](#), [timer_drift_check_bare_metal\(\)](#), [timer_init\(\)](#), [timer_now_ns\(\)](#), and [timer_tsc_hz\(\)](#).

7.100.3.8 vm_ns_base

```
uint64_t vm_ns_base = 0 [static]
```

Referenced by [timer_now_ns_vm_relative\(\)](#), and [vm_timebase_init_from_pmtimer\(\)](#).

7.100.3.9 vm_pm_start

```
uint32_t vm_pm_start = 0 [static]
```

Referenced by [timer_now_ns_vm_relative\(\)](#), and [vm_timebase_init_from_pmtimer\(\)](#).

7.100.3.10 windows_used

```
uint32_t windows_used = 0 [static]
```

Referenced by [converge_tsc_bare_metal\(\)](#).

7.101 src/starkernel/boot/elf_loader.c File Reference

```
#include "uefi.h"
#include "elf64.h"
#include "arch.h"
Include dependency graph for elf_loader.c:
```

Functions

- static void * [elf_memset](#) (void *s, int c, uint64_t n)
[elf_loader.c](#) - ELF64 kernel loader for StarKernel
- static void * [elf_memcpy](#) (void *dest, const void *src, uint64_t n)
- static int [elf_validate_header](#) (const [Elf64_Ehdr](#) *ehdr)
- static int [elf_load_segments](#) (const uint8_t *elf_data, [Elf64_Addr](#) load_base)
- static int [elf_apply_relocations](#) (const uint8_t *elf_data, [Elf64_Addr](#) load_base)
- int [elf_load_kernel](#) (const uint8_t *elf_data, uint64_t elf_size, [Elf64_Addr](#) *entry_out)
Load and relocate the StarKernel ELF binary.

7.101.1 Function Documentation

7.101.1.1 elf_apply_relocations()

```
static int elf_apply_relocations (
    const uint8_t * elf_data,
    Elf64_Addr load_base ) [static]
```

References [Elf64_Ehdr::e_shnum](#), [Elf64_Ehdr::e_shoff](#), [ELF64_R_SYM](#), [ELF64_R_TYPE](#), [R_AARCH64_ABS64](#), [R_AARCH64_NONE](#), [R_AARCH64_RELATIVE](#), [Elf64_Rela::r_addend](#), [Elf64_Rela::r_offset](#), [R_RISCV_64](#), [R_RISCV_NONE](#), [R_RISCV_RELATIVE](#), [R_X86_64_32](#), [R_X86_64_32S](#), [R_X86_64_64](#), [R_X86_64_NONE](#), [R_X86_64_RELATIVE](#), [Elf64_Shdr::sh_entsize](#), [Elf64_Shdr::sh_link](#), [Elf64_Shdr::sh_offset](#), [Elf64_Shdr::sh_size](#), [Elf64_Shdr::sh_type](#), [SHT_REL](#), and [Elf64_Sym::st_value](#).

Referenced by [elf_load_kernel](#)().

Here is the caller graph for this function:

7.101.1.2 elf_load_kernel()

```
int elf_load_kernel (
    const uint8_t * elf_data,
    uint64_t elf_size,
    Elf64_Addr * entry_out )
```

Load and relocate the StarKernel ELF binary.

[elf_loader.h](#) - ELF64 kernel loader interface

Parameters

<i>elf_data</i>	Pointer to ELF file in memory
<i>elf_size</i>	Size of ELF file in bytes
<i>entry_out</i>	Output: kernel entry point address

Returns

1 on success, 0 on failure

References [Elf64_Ehdr::e_entry](#), [Elf64_Ehdr::e_type](#), [elf_apply_relocations\(\)](#), [elf_load_segments\(\)](#), [elf_validate_header\(\)](#), [ET_DYN](#), and [ET_EXEC](#).

Referenced by [efi_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.101.1.3 elf_load_segments()

```
static int elf_load_segments (
    const uint8_t * elf_data,
    Elf64_Addr load_base ) [static]
```

References [Elf64_Ehdr::e_phnum](#), [Elf64_Ehdr::e_phoff](#), [elf_memcpy\(\)](#), [elf_memset\(\)](#), [Elf64_Phdr::p_offset](#), and [PT_LOAD](#).

Referenced by [elf_load_kernel\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.101.1.4 elf_memcpy()

```
static void * elf_memcpy (
    void * dest,
    const void * src,
    uint64_t n ) [static]
```

Referenced by [elf_load_segments\(\)](#).

Here is the caller graph for this function:

7.101.1.5 elf_memset()

```
static void * elf_memset (
    void * s,
    int c,
    uint64_t n ) [static]
```

[elf_loader.c](#) - ELF64 kernel loader for StarKernel

Loads the StarKernel ELF binary from ESP, parses segments, applies relocations, and jumps to entry point.

Referenced by [elf_load_segments\(\)](#).

Here is the caller graph for this function:

7.101.1.6 elf_validate_header()

```
static int elf_validate_header (
    const Elf64_Ehdr * ehdr ) [static]
```

References [Elf64_Ehdr::e_ident](#), [Elf64_Ehdr::e_machine](#), [Elf64_Ehdr::e_type](#), [EI_CLASS](#), [EI_DATA](#), [EI_MAG0](#), [EI_MAG1](#), [EI_MAG2](#), [EI_MAG3](#), [EI_VERSION](#), [ELFCLASS64](#), [ELFDATA2LSB](#), [ELFMAG0](#), [ELFMAG1](#), [ELFMAG2](#), [ELFMAG3](#), [EM_AARCH64](#), [EM_RISCV](#), [EM_X86_64](#), [ET_DYN](#), [ET_EXEC](#), and [EV_CURRENT](#).

Referenced by [elf_load_kernel\(\)](#).

Here is the caller graph for this function:

7.102 src/starkernel/boot/reloc_stub.c File Reference

```
#include <stdint.h>
```

Include dependency graph for [reloc_stub.c](#):

Data Structures

- struct [reloc_block](#)

[reloc_stub.c](#) - Provide a minimal .reloc section for PE/COFF EFI loader

Functions

- struct [reloc_block](#) [__attribute__](#) ((packed, aligned(4)))
- [__attribute__](#) ((section(".reloc"), used))

Variables

- uint32_t [page_rva](#)
- uint32_t [block_size](#)
- uint16_t [entry](#)

7.102.1 Function Documentation

7.102.1.1 [__attribute__](#)() [1/2]

```
struct reloc\_block \_\_attribute\_\_ (
    (packed, aligned(4)) )
```

7.102.1.2 [__attribute__](#)() [2/2]

```
\_\_attribute\_\_ (
    (section(".reloc"), used) )
```

References [entry](#), and [reloc_block::page_rva](#).

7.102.2 Variable Documentation

7.102.2.1 [block_size](#)

```
uint32_t block\_size
```

7.102.2.2 [entry](#)

```
uint16_t entry
```

Referenced by [__attribute__](#)(), [acl_recheck](#)(), [capsule_birth_baby](#)(), [capsule_dict_hash_hook](#)(), [defining_runtime_constant](#)(), [defining_runtime_create](#)(), [defining_runtime_dodoes](#)(), [defining_runtime_variable](#)(), [defining_word_compile](#)(), [defining_word_constant](#)(), [defining_word_create](#)(), [defining_word_variable](#)(), [derive_state_flags](#)(), [dictionary_m_word_cfa](#)(), [dictionary_m_word_name_to](#)(), [dictionary_m_word_nfa](#)(), [dictionary_m_word_pfa](#)(), [dictionary_m_word_to_body](#)(), [dictionary_m_word_to_link](#)(), [dictionary_m_word_to_name](#)(), [execute_colon_word](#)(), [extract_heat_trajectory](#)(), [find_entry_from_name](#)(), [forth_FREEZE_CRITICAL](#)(), [forth_FREEZE_WORD](#)(), [forth_FROZEN_QUERY](#)(), [forth_HEAT_FETCH](#)(), [forth_HEAT_STORE](#)(), [forth_PIPELINING_ANALYZE_WORD](#)(), [forth_PIPELINING_RESET_ALL](#)(), [forth_PIPELINING_SHOW_STATS](#)(), [forth_PIPELINING_SHOW_TOP_TRANSITIONS](#)(), [forth_PIPELINING_STATS](#)(), [forth_SHOW_HEAT](#)(), [forth_UNFREEZE_WORD](#)(), [get_body_address](#)(), [get_name_field](#)(), [get_or_alloc_table](#)(), [get_table](#)(), [get_word_stats](#)(), [hash_colon_body](#)(), [heartbeat_capture_tick_snapshot](#)(), [physics_execution_heat_increment](#)(), [physics_execution_heat_load](#)(), [physics_metadata_apply_linear_decay](#)(), [physics_metadata_apply_seed](#)(), [physics_metadata_init](#)(), [physics_metadata_record_latency](#)(), [physics_metadata_refresh_state](#)(), [physics_metadata_set_mass](#)(), [physics_metadata_touch](#)(), [physics_on_lookup](#)(), [pop_xt](#)(), [print_word_name](#)(), [print_word_name_safe](#)(), [profiler_word_count](#)(), [profiler_word_enter](#)(), [profiler_word_exit](#)(), [register_word](#)(), [rolling_window_seed_hotwords_cache](#)(), [starforth_word_execution_heat_fetch](#)(), [starforth_word_execution_heat_store](#)(), [starforth_word_reset_execution_heat](#)(), [starforth_word_top_words](#)(), [starforth_word_word_execution_heat](#)(), [string_word_tick](#)(), [system_word_see](#)(), [vm_bootstrap_root_vocabulary](#)(), [vm_compile_call](#)(), [vm_compile_word](#)(), [vm_create_word](#)(), [vm_dictionary_get_data_field](#)(), [vm_dictionary_track_entry](#)(), [vm_dictionary_untrack_entry](#)(), [vm_docol_arm64](#)(), [vm_docol_asm](#)(), [vm_docol_rv64](#)(), [vm_interpret_word](#)(), [vmm_query_page](#)(), [vocabulary_create_vocabulary_direct](#)(), and [vocabulary_word_vocabulary](#)()

7.102.2.3 [page_rva](#)

```
uint32_t page\_rva
```

7.103 src/starkernel/boot/uefi_loader.c File Reference

```
#include "uefi.h"
#include "arch.h"
#include "elf_loader.h"
#include "elf64.h"
Include dependency graph for uefi_loader.c:
```

Macros

- #define [RAW_LOG](#)(str) ((void)0)
uefi_loader.c - UEFI boot loader for Starkernel
- #define [KERNEL_MAX_SIZE](#) (8 * 1024 * 1024)

Typedefs

- typedef void(* [KernelEntry](#)) ([BootInfo](#) *boot_info)

Functions

- static int [guid_equals](#) (const [EFI_GUID](#) *a, const [EFI_GUID](#) *b)
- static [EFI_STATUS](#) [load_kernel_from_esp](#) ([EFI_HANDLE](#) ImageHandle, [EFI_BOOT_SERVICES](#) *BS)
- [EFI_STATUS](#) [EFIAPI efi_main](#) ([EFI_HANDLE](#) ImageHandle, [EFI_SYSTEM_TABLE](#) *SystemTable)

Variables

- static [BootInfo](#) [g_boot_info](#) = {0}
- static uint8_t * [kernel_elf_buffer](#) = NULL
- static uint64_t [kernel_elf_size](#) = 0

7.103.1 Macro Definition Documentation

7.103.1.1 KERNEL_MAX_SIZE

```
#define KERNEL_MAX_SIZE (8 * 1024 * 1024)
```

7.103.1.2 RAW_LOG

```
#define RAW_LOG(  
    str ) ((void)0)
```

[uefi_loader.c](#) - UEFI boot loader for Starkernel

This loader loads the Starkernel ELF binary from the ESP, parses it, loads segments, applies relocations, and jumps to entry.

7.103.2 Typedef Documentation

7.103.2.1 KernelEntry

```
typedef void(* KernelEntry) (BootInfo *boot_info)
```

7.103.3 Function Documentation

7.103.3.1 efi_main()

```
EFI\_STATUS EFIAPI efi_main (  
    EFI\_HANDLE ImageHandle,  
    EFI\_SYSTEM\_TABLE * SystemTable )
```

References [BootInfo::acpi_table](#), [arch_halt\(\)](#), [FramebufferInfo::base](#), [_EFI_SYSTEM_TABLE::BootServices](#), [_EFI_SYSTEM_TABLE::ConfigurationTable](#), [_EFI_SYSTEM_TABLE::ConOut](#), [EFI_ACPI_20_TABLE_GUID](#), [EFI_ACPI_TABLE_GUID](#), [EFI_BUFFER_TOO_SMALL](#), [EFI_INVALID_PARAMETER](#), [EFI_SUCCESS](#), [elf_load_kernel\(\)](#),

[_EFI_BOOT_SERVICES::ExitBootServices](#), [FALSE](#), [BootInfo::framebuffer](#), [g_boot_info](#), [_EFI_BOOT_SERVICES::GetMemoryMap](#), [guid_equals\(\)](#), [FramebufferInfo::height](#), [kernel_elf_buffer](#), [kernel_elf_size](#), [kernel_main\(\)](#), [load_kernel_from_esp\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [_EFI_SYSTEM_TABLE::NumberOfT](#), [_EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::OutputString](#), [FramebufferInfo::pixels_per_scanline](#), [RAW_LOG](#), [_EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL::Reset](#), [BootInfo::runtime_services](#), [_EFI_SYSTEM_TABLE::RuntimeServices](#), [FramebufferInfo::size](#), [BootInfo::uefi_boot_services_exited](#), [EFI_CONFIGURATION_TABLE::VendorTable](#), and [FramebufferInfo::width](#).

Here is the call graph for this function:

7.103.3.2 guid_equals()

```
static int guid_equals (
    const EFI_GUID * a,
    const EFI_GUID * b ) [static]
```

References [EFI_GUID::Data1](#), [EFI_GUID::Data2](#), [EFI_GUID::Data3](#), and [EFI_GUID::Data4](#).

Referenced by [efi_main\(\)](#).

Here is the caller graph for this function:

7.103.3.3 load_kernel_from_esp()

```
static EFI_STATUS load_kernel_from_esp (
    EFI_HANDLE ImageHandle,
    EFI_BOOT_SERVICES * BS ) [static]
```

References [AllocateAnyPages](#), [_EFI_BOOT_SERVICES::AllocatePages](#), [_EFI_FILE_PROTOCOL::Close](#), [EFI_LOADED_IMAGE_PROTOCOL::DeviceHandle](#), [EFI_ABORTED](#), [EFI_BUFFER_TOO_SMALL](#), [EFI_FILE_INFO_GUID](#), [EFI_FILE_MODE_READ](#), [EFI_LOADED_IMAGE_PROTOCOL_GUID](#), [EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID](#), [EFI_SUCCESS](#), [EfiLoaderData](#), [file_info\(\)](#), [EFI_FILE_INFO::FileSize](#), [_EFI_FILE_PROTOCOL::GetInfo](#), [_EFI_BOOT_SERVICES::HandleProtocol](#), [kernel_elf_buffer](#), [kernel_elf_size](#), [KERNEL_MAX_SIZE](#), [_EFI_FILE_PROTOCOL::Open](#), [_EFI_SIMPLE_FILE_SYSTEM_PROTOCOL::OpenVolume](#), [RAW_LOG](#), and [_EFI_FILE_PROTOCOL::Read](#).

Referenced by [efi_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.103.4 Variable Documentation

7.103.4.1 g_boot_info

```
BootInfo g_boot_info = {0} [static]
```

Referenced by [efi_main\(\)](#).

7.103.4.2 kernel_elf_buffer

```
uint8_t* kernel_elf_buffer = NULL [static]
```

Referenced by [efi_main\(\)](#), and [load_kernel_from_esp\(\)](#).

7.103.4.3 kernel_elf_size

```
uint64_t kernel_elf_size = 0 [static]
```

Referenced by [efi_main\(\)](#), and [load_kernel_from_esp\(\)](#).

7.104 src/starkernel/capsule/capsule_birth.c File Reference

```
#include "starkernel/capsule_birth.h"
#include "starkernel/capsule.h"
#include "starkernel/capsule_run.h"
```

Include dependency graph for capsule_birth.c:

Macros

- `#define MAX_VMS 64`

Functions

- void `capsule_birth_set_hooks` (`CapsuleExecFn` `exec_fn`, `CapsuleDictHashFn` `dict_hash_fn`, `CapsuleVMAllocFn` `vm_alloc_fn_arg`)
capsule_birth_set_hooks - Configure [VM](#) execution hooks
- void `capsule_vm_registry_init` (void)
capsule_vm_registry_init - Initialize [VM](#) registry
- static `VMRegistryEntry` * `vm_registry_alloc` (void)
- int `capsule_vm_registry_get` (uint32_t `vm_id`, `VMRegistryEntry` *`out`)
capsule_vm_registry_get - Get [VM](#) registry entry by ID
- uint32_t `capsule_vm_registry_count` (void)
capsule_vm_registry_count - Get number of registered VMs
- static void `dispatch_init_forth` (void *`vm_ctx`)
- `CapsuleRunResult` `capsule_birth_mama` (void *`mama_vm`, const `CapsuleDirHeader` *`dir`, const `CapsuleDesc` *`descs`, const `CapsuleNameEntry` *`names`, const uint8_t *`arena`)
capsule_birth_mama - Execute Mama's init capsule
- `CapsuleRunResult` `capsule_birth_baby` (const char *`capsule_name`, const `CapsuleDirHeader` *`dir`, const `CapsuleDesc` *`descs`, const `CapsuleNameEntry` *`names`, const uint8_t *`arena`, uint32_t *`out_vm_id`, void **`out_vm_ctx`)
capsule_birth_baby - Birth a new [VM](#) from a named (p) capsule
- `CapsuleRunResult` `capsule_run_experiment` (void *`mama_vm`, const char *`capsule_name`, const `CapsuleDirHeader` *`dir`, const `CapsuleDesc` *`descs`, const `CapsuleNameEntry` *`names`, const uint8_t *`arena`, uint64_t *`out_run_id`)
capsule_run_experiment - Execute a named (e) capsule on Mama

Variables

- static `CapsuleExecFn` `vm_exec_fn` = 0
capsule_birth.c - [VM Birth Protocol Implementation \(M7.1\)](#)
- static `CapsuleDictHashFn` `vm_dict_hash_fn` = 0
- static `CapsuleVMAllocFn` `vm_alloc_fn` = 0
- static `VMRegistryEntry` `vm_registry` [64]
- static uint32_t `vm_registry_count` = 0
- static uint32_t `next_vm_id` = 1

7.104.1 Macro Definition Documentation

7.104.1.1 MAX_VMS

```
#define MAX_VMS 64
```

7.104.2 Function Documentation

7.104.2.1 capsule_birth_baby()

```
CapsuleRunResult capsule_birth_baby (
    const char * capsule_name,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena,
    uint32_t * out_vm_id,
    void ** out_vm_ctx )
```

`capsule_birth_baby` - Birth a new [VM](#) from a named (p) capsule

Finds the capsule by colon-separated name, validates it, allocates a new [VM](#), executes the capsule payload as IDENTITY, then (if present) executes unit.4th from the baby's block space as PERSONALITY.

Parameters

<i>capsule_name</i>	Colon-separated capsule name, e.g. "production:myvm.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to <i>descs</i>)
<i>arena</i>	Capsule payload arena
<i>out_vm_id</i>	Output: assigned VM ID
<i>out_vm_ctx</i>	Output: new VM context

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [CAPSULE_BIRTH_ELIGIBLE](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [capsule_parity_log_birth\(\)](#), [capsule_parity_log_birth_failed\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_ERR_NOT_ELIGIBLE](#), [CAPSULE_RUN_ERR_STILLBORN](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [dispatch_init_forth\(\)](#), [entry](#), [next_vm_id](#), [vm_alloc_fn](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), [vm_registry_alloc\(\)](#), [VM_STATE_EMBRYO](#), [VM_STATE_LIVE](#), and [VM_STATE_STILLBORN](#).

Here is the call graph for this function:

7.104.2.2 capsule_birth_mama()

```
CapsuleRunResult capsule_birth_mama (
    void * mama_vm,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena )
```

capsule_birth_mama - Execute Mama's init capsule

Finds the MAMA_INIT capsule by flag, validates it, executes it on Mama's VM.

Parameters

<i>mama_vm</i>	Mama's VM context
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to <i>descs</i>)
<i>arena</i>	Capsule payload arena

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [VMRegistryEntry::birth_capsule_id](#), [VMRegistryEntry::birth_dict_hash](#), [capsule_find_mama_init\(\)](#), [capsule_get_payload\(\)](#), [capsule_parity_log_mama_init\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), and [vm_registry](#).

Here is the call graph for this function:

7.104.2.3 capsule_birth_set_hooks()

```
void capsule_birth_set_hooks (
    CapsuleExecFn exec_fn,
    CapsuleDictHashFn dict_hash_fn,
    CapsuleVMAllocFn vm_alloc_fn )
```

capsule_birth_set_hooks - Configure VM execution hooks

Must be called before any birth operations.

Parameters

<i>exec_fn</i>	Function to execute FORTH code on VM
<i>dict_hash_fn</i>	Function to compute dictionary hash
<i>vm_alloc_fn</i>	Function to allocate new VM (for babies)

References [vm_alloc_fn](#), [vm_dict_hash_fn](#), and [vm_exec_fn](#).

Referenced by [capsule_vm_hooks_register\(\)](#).

Here is the caller graph for this function:

7.104.2.4 capsule_run_experiment()

```
CapsuleRunResult capsule_run_experiment (
    void * mama_vm,
    const char * capsule_name,
    const CapsuleDirHeader * dir,
    const CapsuleDesc * descs,
    const CapsuleNameEntry * names,
    const uint8_t * arena,
    uint64_t * out_run_id )
```

`capsule_run_experiment` - Execute a named (e) capsule on Mama

Finds the experiment capsule by colon-separated name, validates it, executes it on Mama's [VM](#) without creating a new [VM](#).

Parameters

<i>mama_vm</i>	Mama's VM context
<i>capsule_name</i>	Colon-separated capsule name, e.g. "experiments:doe-l8:init-l8-stable.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena
<i>out_run_id</i>	Output: assigned run ID

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [CAPSULE_DOE_ELIGIBLE](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [CapsuleRunRecord::capsule_hash](#), [CapsuleRunRecord::capsule_id](#), [capsule_parity_log_run\(\)](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_ERR_NOT_ELIGIBLE](#), [capsule_run_log_record\(\)](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [CapsuleRunRecord::ended_ns](#), [CapsuleRunRecord::flags](#), [CapsuleRunRecord::post_dict_hash](#), [CapsuleRunRecord::pre_dict_hash](#), [CapsuleRunRecord::reserved](#), [CapsuleRunRecord::result](#), [CapsuleRunRecord::run_id](#), [CapsuleRunRecord::started_ns](#), [vm_dict_hash_fn](#), [vm_exec_fn](#), and [CapsuleRunRecord::vm_id](#).

Here is the call graph for this function:

7.104.2.5 capsule_vm_registry_count()

```
uint32_t capsule_vm_registry_count (
    void )
```

`capsule_vm_registry_count` - Get number of registered VMs

References [vm_registry_count](#).

7.104.2.6 capsule_vm_registry_get()

```
int capsule_vm_registry_get (
    uint32_t vm_id,
    VMRegistryEntry * out )
```

capsule_vm_registry_get - Get VM registry entry by ID

Parameters

<i>vm_id</i>	VM ID to look up
<i>out</i>	Output: registry entry

Returns

0 on success, -1 if not found

References [vm_registry](#), and [vm_registry_count](#).

7.104.2.7 capsule_vm_registry_init()

```
void capsule_vm_registry_init (
    void )
```

capsule_vm_registry_init - Initialize VM registry

References [VMRegistryEntry::birth_capsule_id](#), [VMRegistryEntry::birth_dict_hash](#), [VMRegistryEntry::birth_timestamp_ns](#), [VMRegistryEntry::flags](#), [MAX_VMS](#), [next_vm_id](#), [VMRegistryEntry::reserved](#), [VMRegistryEntry::state](#), [VMRegistryEntry::vm_id](#), [vm_registry](#), [vm_registry_count](#), [VM_STATE_EMBRYO](#), and [VM_STATE_LIVE](#).

7.104.2.8 dispatch_init_forth()

```
static void dispatch_init_forth (
    void * vm_ctx ) [static]
```

References [vm_exec_fn](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the caller graph for this function:

7.104.2.9 vm_registry_alloc()

```
static VMRegistryEntry * vm_registry_alloc (
    void ) [static]
```

References [MAX_VMS](#), [vm_registry](#), and [vm_registry_count](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the caller graph for this function:

7.104.3 Variable Documentation

7.104.3.1 next_vm_id

```
uint32_t next_vm_id = 1 [static]
```

Referenced by [capsule_birth_baby\(\)](#), and [capsule_vm_registry_init\(\)](#).

7.104.3.2 vm_alloc_fn

```
CapsuleVMAllocFn vm_alloc_fn = 0 [static]
```

Referenced by [capsule_birth_baby\(\)](#), and [capsule_birth_set_hooks\(\)](#).

7.104.3.3 vm_dict_hash_fn

```
CapsuleDictHashFn vm_dict_hash_fn = 0 [static]
```

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_birth_set_hooks\(\)](#), and [capsule_run_experiment\(\)](#).

7.104.3.4 vm_exec_fn

`CapsuleExecFn vm_exec_fn = 0 [static]`

[capsule_birth.c](#) - VM Birth Protocol Implementation (M7.1)

Mama init, baby birth, and experiment execution. Freestanding - no libc dependency.

Birth sequence for a baby VM:

1. Find capsule by name (`capsule_find_by_name`)
2. Assert CAPSULE_BIRTH_ELIGIBLE
3. Validate content hash
4. `vm_alloc_hook()` — fresh VM
5. `vm_exec_hook(payload)` — IDENTITY (init capsule from Hera's store)
6. `vm_exec_hook("1 LOAD")` — PERSONALITY (baby's personal init.4th from block 1, if present)
7. Log parity record

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_birth_set_hooks\(\)](#), [capsule_run_experiment\(\)](#), and [dispatch_init_forth\(\)](#).

7.104.3.5 vm_registry

`VMRegistryEntry vm_registry[64] [static]`

Referenced by [capsule_birth_mama\(\)](#), [capsule_vm_registry_get\(\)](#), [capsule_vm_registry_init\(\)](#), and [vm_registry_alloc\(\)](#).

7.104.3.6 vm_registry_count

`uint32_t vm_registry_count = 0 [static]`

Referenced by [capsule_vm_registry_count\(\)](#), [capsule_vm_registry_get\(\)](#), [capsule_vm_registry_init\(\)](#), and [vm_registry_alloc\(\)](#).

7.105 src/starkernel/capsule/capsule_find.c File Reference

`#include "starkernel/capsule.h"`

Include dependency graph for `capsule_find.c`:

Functions

- static int [capsule_streq](#) (const char *a, const char *b)
[capsule_find.c](#) - Capsule Lookup (M7.1)
- const [CapsuleDesc](#) * [capsule_find_by_id](#) (const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, uint64_t id)
[capsule_find_by_id](#) - Find capsule by content hash ID
- const [CapsuleDesc](#) * [capsule_find_by_name](#) (const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const char *name)
[capsule_find_by_name](#) - Find capsule by colon-separated name
- const [CapsuleDesc](#) * [capsule_find_mama_init](#) (const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs)
[capsule_find_mama_init](#) - Find the Mama init capsule
- const uint8_t * [capsule_get_payload](#) (const [CapsuleDesc](#) *desc, const uint8_t *arena_base)
[capsule_get_payload](#) - Get pointer to capsule payload bytes

7.105.1 Function Documentation

7.105.1.1 capsule_find_by_id()

```
const CapsuleDesc * capsule_find_by_id (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * desc,
    uint64_t id )
```

capsule_find_by_id - Find capsule by content hash ID

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array
<i>id</i>	Capsule ID (content hash) to find

Returns

Pointer to descriptor, or NULL if not found

References [CapsuleDirHeader::desc_count](#).

7.105.1.2 capsule_find_by_name()

```
const CapsuleDesc * capsule_find_by_name (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * desc,
    const CapsuleNameEntry * names,
    const char * name )
```

capsule_find_by_name - Find capsule by colon-separated name

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array
<i>names</i>	Name entry array (parallel to desc)
<i>name</i>	Colon-separated capsule name, e.g. "core:init.4th"

Returns

Pointer to descriptor, or NULL if not found

References [capsule_streq\(\)](#), [CapsuleDirHeader::desc_count](#), and [name](#).

Referenced by [capsule_birth_baby\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.105.1.3 capsule_find_mama_init()

```
const CapsuleDesc * capsule_find_mama_init (
    const CapsuleDirHeader * dir,
    const CapsuleDesc * desc )
```

capsule_find_mama_init - Find the Mama init capsule

Searches the descriptor array for the capsule with CAPSULE_FLAG_MAMA_INIT. There must be exactly one such capsule.

Parameters

<i>dir</i>	Directory header
<i>descs</i>	Descriptor array

Returns

Pointer to Mama's init descriptor, or NULL if not found

References [CAPSULE_IS_MAMA_INIT](#), [CapsuleDirHeader::desc_count](#), and [flags](#).

Referenced by [capsule_birth_mama\(\)](#).

Here is the caller graph for this function:

7.105.1.4 capsule_get_payload()

```
const uint8_t * capsule_get_payload (
    const CapsuleDesc * desc,
    const uint8_t * arena_base )
```

[capsule_get_payload](#) - Get pointer to capsule payload bytes

Parameters

<i>desc</i>	Capsule descriptor
<i>arena_base</i>	Base address of payload arena

Returns

Pointer to payload bytes, or NULL on error

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).

Here is the caller graph for this function:

7.105.1.5 capsule_streq()

```
static int capsule_streq (
    const char * a,
    const char * b ) [static]
```

[capsule_find.c](#) - Capsule Lookup (M7.1)

Lookup functions for capsule descriptors by ID, name, and role. Freestanding - no libc dependency.

Referenced by [capsule_find_by_name\(\)](#).

Here is the caller graph for this function:

7.106 src/starkernel/capsule/capsule_loader.c File Reference

```
#include "starkernel/capsule_loader.h"
#include "starkernel/capsule.h"
#include "starkernel/capsule_run.h"
#include "block_subsystem.h"
#include "vm.h"
```

Include dependency graph for [capsule_loader.c](#):

Macros

- #define [CAPSULE_RAM_OFFSET](#) 2048u
[capsule_loader.c](#) - Block Capsule Loader and Executor (M7.1)
- #define [LOADER_BLOCK_SIZE](#) 1024u

Functions

- static int [is_block_header](#) (const uint8_t *p, const uint8_t *end, uint32_t *out_num, const uint8_t **out_after)
- static void [write_ramdrive_block](#) (uint32_t block_num, const uint8_t *content, uint32_t content_len)
- static void [copy_block_to_ram](#) (uint32_t source_block_num)
- static size_t [u32_to_dec](#) (uint32_t val, char *buf, size_t size)

- int [capsule_load_blocks](#) (const uint8_t *payload, uint64_t length, uint32_t *out_entry_block)
[capsule_loader.h](#) - Block Capsule Loader and Executor (M7.1)
- static void [copy_device_to_ram](#) (const uint8_t *payload, uint64_t length)
- void [capsule_clear_blocks](#) (const uint8_t *payload, uint64_t length)
[capsule_clear_blocks](#) - Zero all ramdrive slots referenced in a capsule payload
- [CapsuleRunResult capsule_exec_init](#) (void *vm_opaque, const char *capsule_name, const [CapsuleDirHeader](#) *dir, const [CapsuleDesc](#) *descs, const [CapsuleNameEntry](#) *names, const uint8_t *arena)
[capsule_exec_init](#) - Load, execute, and clean up an init capsule

7.106.1 Macro Definition Documentation

7.106.1.1 CAPSULE_RAM_OFFSET

```
#define CAPSULE_RAM_OFFSET 2048u
```

[capsule_loader.c](#) - Block Capsule Loader and Executor (M7.1)

Parses a capsule payload for "Block <num>" headers and implements the full init sequence:

1. Write blocks to ramdrive (2048+) — establishes device content
2. Copy from ramdrive N to dedicated RAM N-2048
3. Execute entry block from dedicated RAM via LOAD
4. Zero dedicated RAM blocks to free them for userspace

The ramdrive (2048+) is the fallback device when no physical block device is mounted. Execution always runs from the dedicated RAM region (0–2047).

Block number mapping: dest = source - CAPSULE_RAM_OFFSET (2048). Conflicts and out-of-range sources are silently swallowed. TODO: proper conflict and bounds handling in a future enhancement.

7.106.1.2 LOADER_BLOCK_SIZE

```
#define LOADER_BLOCK_SIZE 1024u
```

7.106.2 Function Documentation

7.106.2.1 capsule_clear_blocks()

```
void capsule_clear_blocks (
    const uint8_t * payload,
    uint64_t length )
```

[capsule_clear_blocks](#) - Zero all ramdrive slots referenced in a capsule payload

Re-parses the payload for "Block <num>" headers and zeros each corresponding 1024-byte ramdrive slot. Called after execution to reclaim ramdrive space.

Parameters

<i>payload</i>	Raw capsule payload bytes
<i>length</i>	Payload length in bytes

References [blk_get_buffer\(\)](#), [blk_update\(\)](#), [CAPSULE_RAM_OFFSET](#), [is_block_header\(\)](#), and [LOADER_BLOCK_SIZE](#).
Referenced by [capsule_exec_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.106.2.2 capsule_exec_init()

```
CapsuleRunResult capsule_exec_init (
    void * vm,
    const char * capsule_name,
```



```

const CapsuleDirHeader * dir,
const CapsuleDesc * descs,
const CapsuleNameEntry * names,
const uint8_t * arena )

```

capsule_exec_init - Load, execute, and clean up an init capsule

Full init sequence:

1. Locate capsule by colon-separated name in the capsule directory
2. Validate content hash
3. Write all blocks to the ramdrive (capsule_load_blocks)
4. Execute the entry block via "<num> LOAD" on the [VM](#)
5. Zero ramdrive slots (capsule_clear_blocks)

Parameters

<i>vm</i>	VM to execute on
<i>capsule_name</i>	Colon-separated capsule name, e.g. "init.4th"
<i>dir</i>	Capsule directory header
<i>descs</i>	Capsule descriptor array
<i>names</i>	Capsule name entry array (parallel to descs)
<i>arena</i>	Capsule payload arena

Returns

CAPSULE_RUN_OK on success, error code otherwise

References [CapsuleDirHeader::arena_size](#), [capsule_clear_blocks\(\)](#), [capsule_find_by_name\(\)](#), [capsule_get_payload\(\)](#), [capsule_load_blocks\(\)](#), [CAPSULE_RAM_OFFSET](#), [CAPSULE_RUN_ERR_EXEC_FAIL](#), [CAPSULE_RUN_ERR_INVALID](#), [CAPSULE_RUN_OK](#), [CAPSULE_VALID](#), [capsule_validate\(\)](#), [CapsuleDesc](#), [copy_device_to_ram\(\)](#), [VM::error](#), [u32_to_dec\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.106.2.3 capsule_load_blocks()

```

int capsule_load_blocks (
    const uint8_t * payload,
    uint64_t length,
    uint32_t * out_entry_block )

```

[capsule_loader.h](#) - Block Capsule Loader and Executor (M7.1)

Parses a capsule payload for "Block <num>" headers and writes each block's content into the correct ramdrive slot, then executes the entry block and zeros the ramdrive slots afterward.

Format: Block <decimal>

<content: up to 1024 bytes, text or binary> Block <decimal>

...

Blocks may appear in any order. Each ramdrive slot is zeroed before content is written. Content exceeding 1024 bytes is silently truncated. capsule_load_blocks - Parse a capsule payload and write blocks to the ramdrive

For each "Block <num>" header found in the payload:

1. Zero the 1024-byte ramdrive slot for block <num>
2. Copy content bytes into the slot (excess beyond 1024 silently dropped)
3. Mark the slot dirty via [blk_update\(\)](#)

Content between headers may be text or binary. Anything before the first "Block <num>" header is ignored.

Parameters

<i>payload</i>	Raw capsule payload bytes (not null-terminated)
<i>length</i>	Payload length in bytes
<i>out_entry_block</i>	If non-NULL, receives the first block number found (the execution entry point); set to 0 if no blocks found

Returns

Number of blocks written, or -1 on invalid input

References [is_block_header\(\)](#), and [write_ramdrive_block\(\)](#).

Referenced by [capsule_exec_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.106.2.4 copy_block_to_ram()

```
static void copy_block_to_ram (
    uint32_t source_block_num ) [static]
```

References [blk_get_buffer\(\)](#), [blk_update\(\)](#), [CAPSULE_RAM_OFFSET](#), and [LOADER_BLOCK_SIZE](#).

Referenced by [copy_device_to_ram\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.106.2.5 copy_device_to_ram()

```
static void copy_device_to_ram (
    const uint8_t * payload,
    uint64_t length ) [static]
```

References [copy_block_to_ram\(\)](#), and [is_block_header\(\)](#).

Referenced by [capsule_exec_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.106.2.6 is_block_header()

```
static int is_block_header (
    const uint8_t * p,
    const uint8_t * end,
    uint32_t * out_num,
    const uint8_t ** out_after ) [static]
```

Referenced by [capsule_clear_blocks\(\)](#), [capsule_load_blocks\(\)](#), and [copy_device_to_ram\(\)](#).

Here is the caller graph for this function:

7.106.2.7 u32_to_dec()

```
static size_t u32_to_dec (
    uint32_t val,
    char * buf,
    size_t size ) [static]
```

Referenced by [capsule_exec_init\(\)](#).

Here is the caller graph for this function:

7.106.2.8 write_ramdrive_block()

```
static void write_ramdrive_block (
    uint32_t block_num,
    const uint8_t * content,
    uint32_t content_len ) [static]
```

References [blk_get_buffer\(\)](#), [blk_update\(\)](#), and [LOADER_BLOCK_SIZE](#).

Referenced by [capsule_load_blocks\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.107 src/starkernel/capsule/capsule_run.c File Reference

```
#include "starkernel/capsule_run.h"
#include "starkernel/capsule.h"
Include dependency graph for capsule_run.c:
```

Functions

- void [capsule_run_log_init](#) (void)
capsule_run_log_init - Initialize run log
- uint64_t [capsule_run_log_record](#) (const [CapsuleRunRecord](#) *record)
capsule_run_log_record - Log a run record
- int [capsule_run_log_get](#) (uint64_t run_id, [CapsuleRunRecord](#) *out)
capsule_run_log_get - Get a run record by ID
- uint32_t [capsule_run_log_count](#) (void)
capsule_run_log_count - Get number of logged runs
- void [capsule_parity_set_output](#) (void(*putc_fn)(char), void(*puts_fn)(const char *))
capsule_parity_set_output - Set output hooks for parity logging
- static void [parity_put_hex64](#) (uint64_t val)
- static void [parity_put_u32](#) (uint32_t val)
- static void [parity_put_u64](#) (uint64_t val)
- void [capsule_parity_log_birth](#) (uint32_t vm_id, uint64_t capsule_id, uint64_t capsule_hash, uint64_t dict_hash)
capsule_parity_log_birth - Log VM birth parity record
- void [capsule_parity_log_birth_failed](#) (uint32_t vm_id, uint64_t capsule_id, [CapsuleRunResult](#) error, uint64_t partial_dict_hash)
capsule_parity_log_birth_failed - Log failed birth
- void [capsule_parity_log_run](#) (uint32_t vm_id, uint64_t run_id, uint64_t capsule_id, uint64_t pre_dict_hash, uint64_t post_dict_hash)
capsule_parity_log_run - Log DoE run parity record
- void [capsule_parity_log_mama_init](#) (uint64_t capsule_id, uint64_t capsule_hash, uint64_t dict_hash)
capsule_parity_log_mama_init - Log Mama init parity record

Variables

- static [CapsuleRunRecord](#) [run_log](#) [1024]
capsule_run.c - DoE Run Logging and Parity (M7.1)
- static uint32_t [run_log_head](#) = 0
- static uint32_t [run_log_count](#) = 0
- static uint64_t [run_id_counter](#) = 0
- static void(* [parity_putc](#))(char c)=0
- static void(* [parity_puts](#))(const char *s)=0

7.107.1 Function Documentation

7.107.1.1 capsule_parity_log_birth()

```
void capsule_parity_log_birth (
    uint32_t vm_id,
    uint64_t capsule_id,
    uint64_t capsule_hash,
    uint64_t dict_hash )
```

capsule_parity_log_birth - Log VM birth parity record

Output format: PARITY:BIRTH vm_id=N capsule_id=X mode=p capsule_hash=H dict_hash=D

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.107.1.2 capsule_parity_log_birth_failed()

```
void capsule_parity_log_birth_failed (
    uint32_t vm_id,
    uint64_t capsule_id,
    CapsuleRunResult error,
    uint64_t partial_dict_hash )
```

capsule_parity_log_birth_failed - Log failed birth

Output format: PARITY:BIRTH_FAILED vm_id=N capsule_id=X error=E partial_dict_hash=H

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_baby\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.107.1.3 capsule_parity_log_mama_init()

```
void capsule_parity_log_mama_init (
    uint64_t capsule_id,
    uint64_t capsule_hash,
    uint64_t dict_hash )
```

capsule_parity_log_mama_init - Log Mama init parity record

Output format: PARITY:MAMA_INIT capsule_id=X mode=m capsule_hash=H dict_hash=D

References [parity_put_hex64\(\)](#), and [parity_puts](#).

Referenced by [capsule_birth_mama\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.107.1.4 capsule_parity_log_run()

```
void capsule_parity_log_run (
    uint32_t vm_id,
    uint64_t run_id,
    uint64_t capsule_id,
    uint64_t pre_dict_hash,
    uint64_t post_dict_hash )
```

capsule_parity_log_run - Log DoE run parity record

Output format: PARITY:RUN vm_id=N run_id=R capsule_id=X mode=e pre_dict=P post_dict=Q

References [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), [parity_put_u64\(\)](#), and [parity_puts](#).

Referenced by [capsule_run_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.107.1.5 capsule_parity_set_output()

```
void capsule_parity_set_output (
    void(*) (char) putc_fn,
    void(*) (const char *) puts_fn )
```

capsule_parity_set_output - Set output hooks for parity logging

Parameters

<i>putc</i> ↔ <i>_fn</i>	Function to output a single character
<i>puts</i> ↔ <i>_fn</i>	Function to output a string

References [parity_putc](#), and [parity_puts](#).

7.107.1.6 capsule_run_log_count()

```
uint32_t capsule_run_log_count (
    void )
```

`capsule_run_log_count` - Get number of logged runs
References [run_log_count](#).

7.107.1.7 capsule_run_log_get()

```
int capsule_run_log_get (
    uint64_t run_id,
    CapsuleRunRecord * out )
```

`capsule_run_log_get` - Get a run record by ID

Parameters

<i>run_id</i>	Run ID to retrieve
<i>out</i>	Output record

Returns

0 on success, -1 if not found

References [CAPSULE_MAX_RUN_RECORDS](#), [run_id_counter](#), and [run_log](#).

7.107.1.8 capsule_run_log_init()

```
void capsule_run_log_init (
    void )
```

`capsule_run_log_init` - Initialize run log

References [CapsuleRunRecord::capsule_hash](#), [CapsuleRunRecord::capsule_id](#), [CAPSULE_MAX_RUN_RECORDS](#), [CapsuleRunRecord::ended_ns](#), [CapsuleRunRecord::flags](#), [CapsuleRunRecord::post_dict_hash](#), [CapsuleRunRecord::pre_dict_hash](#), [CapsuleRunRecord::reserved](#), [CapsuleRunRecord::result_code](#), [CapsuleRunRecord::run_id](#), [run_id_counter](#), [run_log](#), [run_log_count](#), [run_log_head](#), [CapsuleRunRecord::started_ns](#), and [CapsuleRunRecord::vm_id](#).

7.107.1.9 capsule_run_log_record()

```
uint64_t capsule_run_log_record (
    const CapsuleRunRecord * record )
```

`capsule_run_log_record` - Log a run record

Parameters

<i>record</i>	Record to log
---------------	---------------

Returns

Run ID assigned, or 0 on failure

References [CAPSULE_MAX_RUN_RECORDS](#), [CapsuleRunRecord::run_id](#), [run_id_counter](#), [run_log](#), [run_log_count](#), and [run_log_head](#).

Referenced by [capsule_run_experiment\(\)](#).

Here is the caller graph for this function:

7.107.1.10 parity_put_hex64()

```
static void parity_put_hex64 (
    uint64_t val ) [static]
```

References [parity_puts](#).

Referenced by [capsule_parity_log_birth\(\)](#), [capsule_parity_log_birth_failed\(\)](#), [capsule_parity_log_mama_init\(\)](#), and [capsule_parity_log_run\(\)](#).

Here is the caller graph for this function:

7.107.1.11 parity_put_u32()

```
static void parity_put_u32 (
    uint32_t val ) [static]
```

References [parity_puts](#).

Referenced by [capsule_parity_log_birth\(\)](#), [capsule_parity_log_birth_failed\(\)](#), and [capsule_parity_log_run\(\)](#).

Here is the caller graph for this function:

7.107.1.12 parity_put_u64()

```
static void parity_put_u64 (
    uint64_t val ) [static]
```

References [parity_puts](#).

Referenced by [capsule_parity_log_run\(\)](#).

Here is the caller graph for this function:

7.107.2 Variable Documentation**7.107.2.1 parity_putc**

```
void(* parity_putc) (char c)=0 (
    char c ) [static]
```

Referenced by [capsule_parity_set_output\(\)](#).

7.107.2.2 parity_puts

```
void(* parity_puts) (const char *s)=0 (
    const char * s ) [static]
```

Referenced by [capsule_parity_log_birth\(\)](#), [capsule_parity_log_birth_failed\(\)](#), [capsule_parity_log_mama_init\(\)](#), [capsule_parity_log_run\(\)](#), [capsule_parity_set_output\(\)](#), [parity_put_hex64\(\)](#), [parity_put_u32\(\)](#), and [parity_put_u64\(\)](#).

7.107.2.3 run_id_counter

```
uint64_t run_id_counter = 0 [static]
```

Referenced by [capsule_run_log_get\(\)](#), [capsule_run_log_init\(\)](#), and [capsule_run_log_record\(\)](#).

7.107.2.4 run_log

```
CapsuleRunRecord run_log[1024] [static]
```

[capsule_run.c](#) - DoE Run Logging and Parity (M7.1)

Ring buffer for capsule execution logging and parity record emission. Freestanding - no libc dependency except for console output hooks.

Referenced by [capsule_run_log_get\(\)](#), [capsule_run_log_init\(\)](#), and [capsule_run_log_record\(\)](#).

7.107.2.5 run_log_count

```
uint32_t run_log_count = 0 [static]
```

Referenced by [capsule_run_log_count\(\)](#), [capsule_run_log_init\(\)](#), and [capsule_run_log_record\(\)](#).

7.107.2.6 run_log_head

```
uint32_t run_log_head = 0 [static]
```

Referenced by [capsule_run_log_init\(\)](#), and [capsule_run_log_record\(\)](#).

7.108 src/starkernel/capsule/capsule_validate.c File Reference

```
#include "starkernel/capsule.h"
```

```
#include "starkernel/xxhash64.h"
```

Include dependency graph for capsule_validate.c:

Functions

- const char * [capsule_validate_result_str](#) ([CapsuleValidateResult](#) result)
capsule_validate_result_str - Get string for validation result
- [CapsuleValidateResult](#) [capsule_validate](#) (const [CapsuleDesc](#) *desc, const uint8_t *arena_base, uint64_t arena_size, int verify_hash)
capsule_validate - Validate a capsule descriptor

Variables

- static const char * [validate_result_strings](#) []
capsule_validate.c - Capsule Validation (M7.1)

7.108.1 Function Documentation

7.108.1.1 capsule_validate()

```
CapsuleValidateResult capsule_validate (
    const CapsuleDesc * desc,
    const uint8_t * arena_base,
    uint64_t arena_size,
    int verify_hash )
```

capsule_validate - Validate a capsule descriptor

Parameters

<i>desc</i>	Capsule descriptor to validate
<i>arena_base</i>	Base address of payload arena
<i>arena_size</i>	Size of payload arena in bytes
<i>verify_hash</i>	If true, recompute and compare content hash

Returns

CAPSULE_VALID on success, error code otherwise

References [CAPSULE_DESC_MAGIC](#), [CAPSULE_ERR_BAD_HASH_ALG](#), [CAPSULE_ERR_BAD_MAGIC](#), [CAPSULE_ERR_BAD_VERSION](#), [CAPSULE_ERR_BOUNDS](#), [CAPSULE_ERR_HASH_MISMATCH](#), [CAPSULE_ERR_MODE_INVALID](#), [CAPSULE_ERR_NULL_PTR](#), [CAPSULE_FLAG_ACTIVE](#), [CAPSULE_FLAG_REVOKED](#), [CAPSULE_HASH_BLAKE3](#), [CAPSULE_HASH_XXHASH64](#), [CAPSULE_MAGIC_GET_HASHALG](#), [CAPSULE_MAGIC_GET_SIG](#), [CAPSULE_MAGIC_GET_VERIFY](#), [CAPSULE_MODE_VALID](#), [CAPSULE_VALID](#), [CAPSULE_VERSION_0](#), and [xxhash64_capsule\(\)](#).

Referenced by [capsule_birth_baby\(\)](#), [capsule_birth_mama\(\)](#), [capsule_exec_init\(\)](#), and [capsule_run_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.108.1.2 capsule_validate_result_str()

```
const char * capsule_validate_result_str (
    CapsuleValidateResult result )
```

capsule_validate_result_str - Get string for validation result

References [validate_result_strings](#).

7.108.2 Variable Documentation

7.108.2.1 validate_result_strings

```
const char* validate_result_strings[] [static]
```

Initial value:

```
= {
    "valid",
    "bad magic",
    "bad version",
```

```

    "bad hash algorithm",
    "bounds error",
    "mode invalid",
    "revoked but active",
    "hash mismatch",
    "null pointer",
}

```

[capsule_validate.c](#) - Capsule Validation (M7.1)

Validates capsule descriptors for structural integrity and content hash. Freestanding - no libc dependency.

Referenced by [capsule_validate_result_str\(\)](#).

7.109 src/starkernel/capsule/capsule_vm_hooks.c File Reference

```

#include "starkernel/capsule_birth.h"
#include "starkernel/xxhash64.h"
#include "vm.h"
#include "platform_alloc.h"
Include dependency graph for capsule_vm_hooks.c:

```

Functions

- static int [capsule_exec_hook](#) (void *vm_ctx, const char *code, uint64_t code_len)
[capsule_vm_hooks.c](#) - VM Hook Implementations for Capsule Birth (M7.1)
- static uint64_t [capsule_dict_hash_hook](#) (void *vm_ctx)
- static void * [capsule_vm_alloc_hook](#) (void)
- void [capsule_vm_hooks_register](#) (void)

[capsule_vm_hooks_register](#) - Wire the three hooks into the capsule subsystem.

7.109.1 Function Documentation

7.109.1.1 capsule_dict_hash_hook()

```

static uint64_t capsule_dict_hash_hook (
    void * vm_ctx ) [static]

```

References [entry](#), [VM::latest](#), [vm](#), [xxhash64_digest\(\)](#), [xxhash64_reset\(\)](#), and [xxhash64_update\(\)](#).

Referenced by [capsule_vm_hooks_register\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.109.1.2 capsule_exec_hook()

```

static int capsule_exec_hook (
    void * vm_ctx,
    const char * code,
    uint64_t code_len ) [static]

```

[capsule_vm_hooks.c](#) - VM Hook Implementations for Capsule Birth (M7.1)

Concrete implementations of the three function pointers required by [capsule_birth_set_hooks\(\)](#):

[capsule_exec_hook](#) – wraps [vm_interpret\(\)](#) [capsule_dict_hash_hook](#) – xxhash64 walk of the live dictionary

[capsule_vm_alloc_hook](#) – allocates and initialises a fresh VM

Call [capsule_vm_hooks_register\(\)](#) once during VM bootstrap (after [register_forth79_words](#)) to wire everything up.

References [VM::error](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [capsule_vm_hooks_register\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.109.1.3 capsule_vm_alloc_hook()

```

static void * capsule_vm_alloc_hook (
    void ) [static]

```

References [VM::error](#), [sf_free\(\)](#), [sf_malloc\(\)](#), [vm](#), [vm_cleanup\(\)](#), and [vm_init\(\)](#).

Referenced by [capsule_vm_hooks_register\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.109.1.4 capsule_vm_hooks_register()

```
void capsule_vm_hooks_register (
    void )
```

capsule_vm_hooks_register - Wire the three hooks into the capsule subsystem.

capsule_vm_hooks_register - Wire concrete VM hooks into the capsule subsystem

Must be called after `vm_init()` completes on Mama's VM, before any capsule birth or experiment operations.

References `capsule_birth_set_hooks()`, `capsule_dict_hash_hook()`, `capsule_exec_hook()`, and `capsule_vm_alloc_hook()`.

Here is the call graph for this function:

7.110 src/starkernel/capsule/mama_forth_words.c File Reference

Mama FORTH vocabulary implementation for kernel capsule system (M7.1)

7.110.1 Detailed Description

Mama FORTH vocabulary implementation for kernel capsule system (M7.1)

Implements the MAMA vocabulary - kernel-only words for the LithosAnanke capsule birth protocol. This file is ONLY compiled for **STARKERNEL** builds.

The MAMA vocabulary provides:

- Capsule directory enumeration
- Baby VM birth from production capsules
- Experiment execution on Mama
- VM registry queries

7.111 src/starkernel/hal/console.c File Reference

```
#include "console.h"
```

```
#include "arch.h"
```

Include dependency graph for console.c:

Macros

- #define `SERIAL_SUPPORTED` 0
console.c - Serial console implementation (UART 16550) Supports amd64 via I/O ports

Functions

- static void `outb` (uint16_t port, uint8_t val)
- static uint8_t `inb` (uint16_t port)
- void `console_init` (void)
Initialize serial console Sets up COM1 at 115200 baud, 8N1.
- void `console_putc` (char c)
Write a single character to serial console.
- void `console_puts` (const char *s)
Write a null-terminated string to serial console.
- void `console_println` (const char *s)
Write a string with newline to serial console.
- int `console_poll` (void)
Check if data is available to read.
- int `console_getc` (void)
Read a single character from serial console (non-blocking) Returns -1 if no character available.

7.111.1 Macro Definition Documentation

7.111.1.1 SERIAL_SUPPORTED

```
#define SERIAL_SUPPORTED 0
```

[console.c](#) - Serial console implementation (UART 16550) Supports amd64 via I/O ports

7.111.2 Function Documentation

7.111.2.1 console_getc()

```
int console_getc (
    void )
```

Read a single character from serial console (non-blocking) Returns -1 if no character available.

References [console_poll\(\)](#), and [inb\(\)](#).

Referenced by [sk_readline\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.2 console_init()

```
void console_init (
    void )
```

Initialize serial console Sets up COM1 at 115200 baud, 8N1.

[console.h](#) - Serial console interface for StarKernel

References [inb\(\)](#), [outb\(\)](#), and [SERIAL_SUPPORTED](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.3 console_poll()

```
int console_poll (
    void )
```

Check if data is available to read.

Check if character is available for reading.

References [inb\(\)](#).

Referenced by [console_getc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.4 console_println()

```
void console_println (
    const char * s )
```

Write a string with newline to serial console.

References [console_putc\(\)](#), and [console_puts\(\)](#).

Referenced by [apic_init\(\)](#), [apic_timer_init\(\)](#), [arch_interrupts_init\(\)](#), [converge_tsc_bare_metal\(\)](#), [decode_pf_error\(\)](#), [init_timer_vm_mode\(\)](#), [isr_common_handler\(\)](#), [kernel_main\(\)](#), [print_banner\(\)](#), [print_boot_info\(\)](#), [print_heap_stats\(\)](#), [print_nl\(\)](#), [print_pmm_stats\(\)](#), [print_uint\(\)](#), [sk_fault_handler\(\)](#), [sk_repl\(\)](#), [timer_fatal\(\)](#), [timer_init\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_init_with_host\(\)](#), [vmm_init\(\)](#), [vmm_map_page\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.5 console_putc()

```
void console_putc (
    char c )
```

Write a single character to serial console.

References [arch_relax\(\)](#), and [outb\(\)](#).

Referenced by [console_println\(\)](#), [console_puts\(\)](#), [converge_tsc_bare_metal\(\)](#), [print_dec\(\)](#), [print_dec_u64\(\)](#), [print_hex64\(\)](#), [print_q48_16\(\)](#), and [sk_readline\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.6 console_puts()

```
void console_puts (
    const char * s )
```

Write a null-terminated string to serial console.

References [console_putc\(\)](#).

Referenced by [apic_timer_init\(\)](#), [apic_timer_start\(\)](#), [console_println\(\)](#), [converge_tsc_bare_metal\(\)](#), [decode_pf_error\(\)](#), [init_timer_vm_mode\(\)](#), [isr_common_handler\(\)](#), [print_banner\(\)](#), [print_boot_info\(\)](#), [print_hex64\(\)](#), [print_q48_16\(\)](#), [print_str\(\)](#), [print_u32\(\)](#), [print_uint\(\)](#), [sk_repl\(\)](#), [timer_init\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.111.2.7 inb()

```
static uint8_t inb (
    uint16_t port ) [inline], [static]
```

Referenced by [console_getc\(\)](#), [console_init\(\)](#), and [console_poll\(\)](#).

Here is the caller graph for this function:

7.111.2.8 outb()

```
static void outb (
    uint16_t port,
    uint8_t val ) [inline], [static]
```

Referenced by [console_init\(\)](#), and [console_putc\(\)](#).

Here is the caller graph for this function:

7.112 src/starkernel/hal/hal.c File Reference

7.113 src/starkernel/hal/host_services.c File Reference

7.114 src/starkernel/hal/memory.c File Reference

```
#include <stddef.h>
#include "hal_memory.h"
#include "kmalloc.h"
Include dependency graph for memory.c:
```

Functions

- void * [hal_mem_alloc](#) (size_t size)
memory.c - HAL memory wrappers
- void * [hal_mem_alloc_aligned](#) (size_t size, size_t align)
- void [hal_mem_free](#) (void *ptr)

7.114.1 Function Documentation

7.114.1.1 hal_mem_alloc()

```
void * hal_mem_alloc (
    size_t size )
```

[memory.c](#) - HAL memory wrappers

[hal_memory.h](#) - HAL memory allocation wrappers

References [kmalloc\(\)](#).

Here is the call graph for this function:

7.114.1.2 `hal_mem_alloc_aligned()`

```
void * hal_mem_alloc_aligned (
    size_t size,
    size_t align )
```

References [kmalloc_aligned\(\)](#).

Here is the call graph for this function:

7.114.1.3 `hal_mem_free()`

```
void hal_mem_free (
    void * ptr )
```

References [kfree\(\)](#).

Here is the call graph for this function:

7.115 `src/starkernel/hash/xxhash64.c` File Reference

```
#include "starkernel/xxhash64.h"
```

Include dependency graph for `xxhash64.c`:

Macros

- `#define XXHASH64_PRIME1` 0x9E3779B185EBCA87ULL
xxhash64.c - Freestanding xxHash64 Implementation
- `#define XXHASH64_PRIME2` 0xC2B2AE3D27D4EB4FULL
- `#define XXHASH64_PRIME3` 0x165667B19E3779F9ULL
- `#define XXHASH64_PRIME4` 0x85EBCA77C2B2AE63ULL
- `#define XXHASH64_PRIME5` 0x27D4EB2F165667C5ULL

Functions

- static uint64_t `xxh64_rotl` (uint64_t x, int r)
Rotate left 64-bit.
- static uint64_t `xxh64_read64` (const uint8_t *p)
Read 64-bit little-endian (unaligned safe)
- static uint32_t `xxh64_read32` (const uint8_t *p)
Read 32-bit little-endian (unaligned safe)
- static uint64_t `xxh64_round` (uint64_t acc, uint64_t input)
Round function.
- static uint64_t `xxh64_merge_round` (uint64_t acc, uint64_t val)
Merge accumulator.
- static uint64_t `xxh64_avalanche` (uint64_t h)
Avalanche finalization.
- uint64_t `xxhash64` (const void *data, size_t len, uint64_t seed)
xxhash64 - Compute xxHash64 of a buffer
- void `xxhash64_reset` (XXHash64State *state, uint64_t seed)
xxhash64_reset - Initialize streaming state
- void `xxhash64_update` (XXHash64State *state, const void *data, size_t len)
xxhash64_update - Feed data into streaming hash
- uint64_t `xxhash64_digest` (const XXHash64State *state)
xxhash64_digest - Finalize and get hash value

7.115.1 Macro Definition Documentation

7.115.1.1 XXHASH64_PRIME1

```
#define XXHASH64_PRIME1 0x9E3779B185EBCA87ULL
```

[xxhash64.c](#) - Freestanding xxHash64 Implementation

Based on xxHash by Yann Collet (BSD-2-Clause). Adapted for freestanding kernel use - no libc dependency.

xxHash is an extremely fast non-cryptographic hash algorithm. <https://github.com/Cyan4973/xxHash>

7.115.1.2 XXHASH64_PRIME2

```
#define XXHASH64_PRIME2 0xC2B2AE3D27D4EB4FULL
```

7.115.1.3 XXHASH64_PRIME3

```
#define XXHASH64_PRIME3 0x165667B19E3779F9ULL
```

7.115.1.4 XXHASH64_PRIME4

```
#define XXHASH64_PRIME4 0x85EBCA77C2B2AE63ULL
```

7.115.1.5 XXHASH64_PRIME5

```
#define XXHASH64_PRIME5 0x27D4EB2F165667C5ULL
```

7.115.2 Function Documentation

7.115.2.1 xxh64_avalanche()

```
static uint64_t xxh64_avalanche (
    uint64_t h ) [inline], [static]
```

Avalanche finalization.

References [XXHASH64_PRIME2](#), and [XXHASH64_PRIME3](#).

Referenced by [xxhash64\(\)](#), and [xxhash64_digest\(\)](#).

Here is the caller graph for this function:

7.115.2.2 xxh64_merge_round()

```
static uint64_t xxh64_merge_round (
    uint64_t acc,
    uint64_t val ) [inline], [static]
```

Merge accumulator.

References [xxh64_round\(\)](#), [XXHASH64_PRIME1](#), and [XXHASH64_PRIME4](#).

Referenced by [xxhash64\(\)](#), and [xxhash64_digest\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.115.2.3 xxh64_read32()

```
static uint32_t xxh64_read32 (
    const uint8_t * p ) [inline], [static]
```

Read 32-bit little-endian (unaligned safe)

Referenced by [xxhash64\(\)](#), and [xxhash64_digest\(\)](#).

Here is the caller graph for this function:

7.115.2.4 xxh64_read64()

```
static uint64_t xxh64_read64 (
    const uint8_t * p ) [inline], [static]
```

Read 64-bit little-endian (unaligned safe)

Referenced by [xxhash64\(\)](#), [xxhash64_digest\(\)](#), and [xxhash64_update\(\)](#).

Here is the caller graph for this function:

7.115.2.5 xxh64_rotl()

```
static uint64_t xxh64_rotl (
    uint64_t x,
    int r ) [inline], [static]
```

Rotate left 64-bit.

Referenced by [xxh64_round\(\)](#), [xxhash64\(\)](#), and [xxhash64_digest\(\)](#).

Here is the caller graph for this function:

7.115.2.6 xxh64_round()

```
static uint64_t xxh64_round (
    uint64_t acc,
    uint64_t input ) [inline], [static]
```

Round function.

References [xxh64_rotl\(\)](#), [XXHASH64_PRIME1](#), and [XXHASH64_PRIME2](#).

Referenced by [xxh64_merge_round\(\)](#), [xxhash64\(\)](#), [xxhash64_digest\(\)](#), and [xxhash64_update\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.115.2.7 xxhash64()

```
uint64_t xxhash64 (
    const void * data,
    size_t len,
    uint64_t seed )
```

xxhash64 - Compute xxHash64 of a buffer

Parameters

<i>data</i>	Input data
<i>len</i>	Length in bytes
<i>seed</i>	Hash seed (use XXHASH64_CAPSULE_SEED for capsules)

Returns

64-bit hash value

References [limit](#), [xxh64_avalanche\(\)](#), [xxh64_merge_round\(\)](#), [xxh64_read32\(\)](#), [xxh64_read64\(\)](#), [xxh64_rotl\(\)](#), [xxh64_round\(\)](#), [XXHASH64_PRIME1](#), [XXHASH64_PRIME2](#), [XXHASH64_PRIME3](#), [XXHASH64_PRIME4](#), and [XXHASH64_PRIME5](#).

Referenced by [xxhash64_capsule\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.115.2.8 xxhash64_digest()

```
uint64_t xxhash64_digest (
    const XXHash64State * state )
```

xxhash64_digest - Finalize and get hash value

Parameters

<i>state</i>	Streaming state
--------------	-----------------

Returns

64-bit hash value

References [XXHash64State::buffer](#), [XXHash64State::buffer_size](#), [XXHash64State::seed](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [xxh64_avalanche\(\)](#), [xxh64_merge_round\(\)](#), [xxh64_read32\(\)](#), [xxh64_read64\(\)](#), [xxh64_rotl\(\)](#), [xxh64_round\(\)](#), [XXHASH64_PRIME1](#), [XXHASH64_PRIME2](#), [XXHASH64_PRIME3](#), [XXHASH64_PRIME4](#), and [XXHASH64_PRIME5](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.115.2.9 xxhash64_reset()

```
void xxhash64_reset (
    XXHash64State * state,
    uint64_t seed )
```

xxhash64_reset - Initialize streaming state

Parameters

<i>state</i>	State to initialize
<i>seed</i>	Hash seed

References [XXHash64State::buffer_size](#), [XXHash64State::seed](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [XXHASH64_PRIME1](#), and [XXHASH64_PRIME2](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the caller graph for this function:

7.115.2.10 xxhash64_update()

```
void xxhash64_update (
    XXHash64State * state,
    const void * data,
    size_t len )
```

xxhash64_update - Feed data into streaming hash

Parameters

<i>state</i>	Streaming state
<i>data</i>	Input data
<i>len</i>	Length in bytes

References [XXHash64State::buffer](#), [XXHash64State::buffer_size](#), [limit](#), [XXHash64State::total_len](#), [XXHash64State::v1](#), [XXHash64State::v2](#), [XXHash64State::v3](#), [XXHash64State::v4](#), [xxh64_read64\(\)](#), and [xxh64_round\(\)](#).

Referenced by [capsule_dict_hash_hook\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116 src/starkernel/kernel_main.c File Reference

```
#include "uefi.h"
#include "console.h"
#include "arch.h"
#include "pmm.h"
#include "vmm.h"
#include "apic.h"
#include "timer.h"
#include "kmalloc.h"
```

Include dependency graph for kernel_main.c:

Macros

- `#define KERNEL\_HEAP\_SIZE (16 * 1024 * 1024) /* 16 MB */`

Functions

- static int [is_ram_type](#) (uint32_t type)
[kernel_main.c](#) - StarKernel main entry point (LithosAnanke branch)
- static void [itoa_simple](#) (uint64_t value, char *buf, int [base](#))
- static void [print_uint](#) (const char *label, uint64_t value)
- static void [print_boot_info](#) ([BootInfo](#) *boot_info)
- static void [print_pmm_stats](#) (void)
- static void [print_heap_stats](#) (void)
- static void [print_banner](#) (void)
- void [kernel_main](#) ([BootInfo](#) *boot_info)
[kernel_main](#) - Main entry point after UEFI handoff

7.116.1 Macro Definition Documentation

7.116.1.1 [KERNEL_HEAP_SIZE](#)

```
#define KERNEL\_HEAP\_SIZE (16 * 1024 * 1024) /* 16 MB */
```

7.116.2 Function Documentation

7.116.2.1 [is_ram_type\(\)](#)

```
static int is\_ram\_type (  
    uint32_t type ) [static]
```

[kernel_main.c](#) - StarKernel main entry point (LithosAnanke branch)

Milestone status: M0-M5: Complete (build, boot, PMM, VMM, interrupts, timer) M6: Infrastructure present (kmmalloc exists, validation deferred) M7: Not started ([VM](#) integration pending)

References [EfiACPIMemoryNVS](#), [EfiACPIReclaimMemory](#), [EfiBootServicesCode](#), [EfiBootServicesData](#), [EfiConventionalMemory](#), [EfiLoaderCode](#), [EfiLoaderData](#), [EfiRuntimeServicesCode](#), and [EfiRuntimeServicesData](#).

Referenced by [print_boot_info\(\)](#).

Here is the caller graph for this function:

7.116.2.2 [itoa_simple\(\)](#)

```
static void itoa\_simple (  
    uint64_t value,  
    char * buf,  
    int base ) [static]
```

References [base](#).

Referenced by [print_boot_info\(\)](#), and [print_uint\(\)](#).

Here is the caller graph for this function:

7.116.2.3 [kernel_main\(\)](#)

```
void kernel\_main (  
    BootInfo * boot_info )
```

[kernel_main](#) - Main entry point after UEFI handoff

This is called by [uefi_loader.c](#) after [ExitBootServices\(\)](#). At this point we own the machine - no UEFI services available.

References [DictEntry::acl_mode](#), [ACL_MODE_STRICT](#), [DictEntry::acl_pinned](#), [apic_init\(\)](#), [apic_timer_init\(\)](#), [apic_timer_start\(\)](#), [arch_enable_interrupts\(\)](#), [arch_halt\(\)](#), [arch_interrupts_init\(\)](#), [CapsuleDirHeader::arena_base](#), [CapsuleDirHeader::arena_size](#), [capsule_exec_init\(\)](#), [CAPSULE_RUN_OK](#), [console_init\(\)](#), [console_println\(\)](#), [CapsuleDirHeader::desc_capacity](#), [CapsuleDirHeader::desc_count](#), [CapsuleDirHeader::dir_hash](#), [heartbeat_init\(\)](#), [KERNEL_HEAP_SIZE](#), [kmmalloc\(\)](#), [kmmalloc_init\(\)](#), [CapsuleDirHeader::magic](#), [CapsuleDirHeader::name_count](#),

[pmm_init\(\)](#), [print_banner\(\)](#), [print_boot_info\(\)](#), [print_heap_stats\(\)](#), [print_pmm_stats\(\)](#), [CapsuleDirHeader::reserved](#), [sk_repl\(\)](#), [timer_init\(\)](#), [timer_tsc_hz\(\)](#), [vm_find_word\(\)](#), and [vmm_init\(\)](#).

Referenced by [efi_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116.2.4 [print_banner\(\)](#)

```
static void print_banner (
    void ) [static]
```

References [console_println\(\)](#), and [console_puts\(\)](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116.2.5 [print_boot_info\(\)](#)

```
static void print_boot_info (
    BootInfo * boot_info ) [static]
```

References [console_println\(\)](#), [console_puts\(\)](#), [EfiConventionalMemory](#), [is_ram_type\(\)](#), [itoa_simple\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [EFI_MEMORY_DESCRIPTOR::NumberOfPages](#), and [EFI_MEMORY_DESCRIPTOR::Type](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116.2.6 [print_heap_stats\(\)](#)

```
static void print_heap_stats (
    void ) [static]
```

References [console_println\(\)](#), [kmalloc_stats_t::free_bytes](#), [kmalloc_get_stats\(\)](#), [kmalloc_stats_t::peak_bytes](#), [print_uint\(\)](#), [kmalloc_stats_t::total_bytes](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116.2.7 [print_pmm_stats\(\)](#)

```
static void print_pmm_stats (
    void ) [static]
```

References [console_println\(\)](#), [pmm_stats_t::free_bytes](#), [pmm_stats_t::free_pages](#), [pmm_get_stats\(\)](#), [print_uint\(\)](#), [pmm_stats_t::total_bytes](#), [pmm_stats_t::total_pages](#), [pmm_stats_t::used_bytes](#), and [pmm_stats_t::used_pages](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.116.2.8 [print_uint\(\)](#)

```
static void print_uint (
    const char * label,
    uint64_t value ) [static]
```

References [console_println\(\)](#), [console_puts\(\)](#), and [itoa_simple\(\)](#).

Referenced by [print_heap_stats\(\)](#), and [print_pmm_stats\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.117 src/starkernel/math/q48_16.c File Reference

```
#include "q48_16.h"
```

Include dependency graph for q48_16.c:

Functions

- [q48_16_t q48_mul \(q48_16_t a, q48_16_t b\)](#)

[q48_16.c](#) - Q48.16 Fixed-Point Arithmetic for StarKernel

- [q48_16_t q48_div \(q48_16_t a, q48_16_t b\)](#)
Divide two Q48.16 values, keeping result in Q48.16.
- [q48_16_t q48_exp_approx \(q48_16_t q\)](#)
Approximate exponential e^x in Q48.16 (integer-only, future use)
- [q48_16_t q48_log_approx \(q48_16_t x\)](#)
Approximate natural logarithm in Q48.16 (integer-only)
- [q48_16_t q48_sqrt_approx \(q48_16_t q\)](#)
Approximate square root in Q48.16 (integer-only)

7.117.1 Function Documentation

7.117.1.1 q48_div()

```
q48_16_t q48_div (
    q48_16_t a,
    q48_16_t b )
```

Divide two Q48.16 values, keeping result in Q48.16.

Divide two Q48.16 values: $(a \ll 16) / b$ Returns 0 if $b == 0$.

Math: $(a / 2^{16}) / (b / 2^{16}) * 2^{16} = (a \ll 16) / b$ Returns: $(a \ll 16) / b$

Parameters

<i>a</i>	Dividend in Q48.16
<i>b</i>	Divisor in Q48.16 (non-zero)

Returns

$(a \ll 16) / b$ in Q48.16 format

Referenced by [q48_exp_approx\(\)](#), [q48_log_approx\(\)](#), and [q48_sqrt_approx\(\)](#).

Here is the caller graph for this function:

7.117.1.2 q48_exp_approx()

```
q48_16_t q48_exp_approx (
    q48_16_t q )
```

Approximate exponential e^x in Q48.16 (integer-only, future use)

Approximate exponential e^x in Q48.16 (integer-only) Input: q in Q48.16 format Output: e^q in Q48.16 format.

Purpose: Might be needed for inverse operations or model validation Status: Defined but not required for Phase 1 inference engine Method: Similar to [q48_log_approx](#) (piecewise + Newton)

Parameters

<i>q</i>	Q48.16 exponent
----------	-----------------

Returns

e^q in Q48.16 format

Referenced by [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.117.1.3 q48_log_approx()

```
q48_16_t q48_log_approx (
    uint64_t x )
```

Approximate natural logarithm in Q48.16 (integer-only)

Approximate natural logarithm in Q48.16 (integer-only) Input: x in Q48.16 format ($x > 0$) Output: $\ln(x)$ in Q48.16 format.

Purpose: Required for decay slope inference via exponential fitting Model: $\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} * t$

Method: Piecewise linear approximation or Newton-Raphson iteration

- Input x must be > 0 (assertion check)
- Output is $\ln(x)$ in Q48.16 format
- Precision: $\sim 5\%$ error acceptable (tuning parameter)
- No floating-point operations

Implementation Strategy:

1. Use bit position ($\text{floor}(\log_2(x))$) as coarse estimate
2. Refine with Newton iteration (3-4 cycles for Q48.16 precision)
3. Keep all arithmetic in integer space

Example: $\text{q48_log_approx}(65536) = \log(1.0)$ in Q48.16 ≈ 0 $\text{q48_log_approx}(131072) = \log(2.0)$ in Q48.16 $\approx 0x0000B172$

Parameters

x	Value to take logarithm of ($x > 0$)
---	--

Returns

$\ln(x)$ in Q48.16 format

7.117.1.4 q48_mul()

```
q48_16_t q48_mul (
    q48_16_t a,
    q48_16_t b )
```

q48_16.c - Q48.16 Fixed-Point Arithmetic for StarKernel

Multiply two Q48.16 values: $(a * b) \gg 16$.

Multiply two Q48.16 values, keeping result in Q48.16.

Freestanding implementation: no libc, no floating-point.

Referenced by [q48_exp_approx\(\)](#), and [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.117.1.5 q48_sqrt_approx()

```
q48_16_t q48_sqrt_approx (
    q48_16_t q )
```

Approximate square root in Q48.16 (integer-only)

Approximate square root in Q48.16 (integer-only, Newton-Raphson) Input: q in Q48.16 format Output: $\sqrt{q}$ in Q48.16 format.

Purpose: Compute R^2 (fit quality) for regression diagnostics Method: Newton-Raphson for integer-only sqrt

Parameters

q	Q48.16 value
---	--------------

Returns

`sqrt(q)` in Q48.16 format

7.118 `src/starkernel/memory/kmalloc.c` File Reference

```
#include <stdint.h>
#include <stddef.h>
#include "kmalloc.h"
#include "pmm.h"
Include dependency graph for kmalloc.c:
```

Data Structures

- struct [heap_block](#)
- struct [kmalloc_prefix_t](#)

Macros

- `#define KMALLOC_DEFAULT_HEAP_SIZE (16u * 1024u * 1024u) /* 16 MiB */`
kmalloc.c - Simple kernel heap allocator
- `#define KMALLOC_MIN_ALIGN 16u`
- `#define KMALLOC_MAGIC 0x4B4D414Cu /* 'KMAL' */`

Typedefs

- typedef struct [heap_block](#) [heap_block_t](#)

Functions

- `uintptr_t kmalloc_heap_base_addr (void)`
- `uintptr_t kmalloc_heap_end_addr (void)`
- `static size_t align_up_size (size_t value, size_t align)`
- `static size_t normalize_alignment (size_t align)`
- `static void update_free_bytes (void)`
- `static void account_allocation (size_t requested)`
- `static void account_free (size_t requested)`
- `static int pointer_within_heap (const void *ptr)`
- `static void coalesce_neighbors (heap_block_t *block)`
- `static size_t minSplittablePayload (void)`
- `static int can_split (size_t remainder)`
- `static void * allocate_from_block (heap_block_t *block, size_t requested, size_t align)`
- `int kmalloc_init (uint64_t heap_size_bytes)`
- `int kmalloc_is_initialized (void)`
- `void * kmalloc (size_t size)`
- `void * kmalloc_aligned (size_t size, size_t align)`
- `void kfree (void *ptr)`
- `kmalloc_stats_t kmalloc_get_stats (void)`

Variables

- `static uint8_t * heap_base = NULL`
- `static uint64_t heap_size = 0`
- `static heap_block_t * heap_head = NULL`
- `static kmalloc_stats_t heap_stats = {0}`
- `static int heap_initialized = 0`

7.118.1 Macro Definition Documentation

7.118.1.1 KMALLOC_DEFAULT_HEAP_SIZE

```
#define KMALLOC_DEFAULT_HEAP_SIZE (16u * 1024u * 1024u) /* 16 MiB */
```

[kmalloc.c](#) - Simple kernel heap allocator

Reserves a fixed heap from PMM and serves aligned allocations with a doubly-linked free list plus coalescing. The allocator avoids libc helpers and keeps metadata inside the heap itself.

7.118.1.2 KMALLOC_MAGIC

```
#define KMALLOC_MAGIC 0x4B4D414Cu /* 'KMAL' */
```

7.118.1.3 KMALLOC_MIN_ALIGN

```
#define KMALLOC_MIN_ALIGN 16u
```

7.118.2 Typedef Documentation

7.118.2.1 heap_block_t

```
typedef struct heap_block heap_block_t
```

7.118.3 Function Documentation

7.118.3.1 account_allocation()

```
static void account_allocation (
    size_t requested ) [static]
```

References [heap_stats](#), [kmalloc_stats_t::peak_bytes](#), [update_free_bytes\(\)](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [allocate_from_block\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.2 account_free()

```
static void account_free (
    size_t requested ) [static]
```

References [heap_stats](#), [update_free_bytes\(\)](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [kfree\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.3 align_up_size()

```
static size_t align_up_size (
    size_t value,
    size_t align ) [static]
```

Referenced by [allocate_from_block\(\)](#).

Here is the caller graph for this function:

7.118.3.4 allocate_from_block()

```
static void * allocate_from_block (
    heap_block_t * block,
    size_t requested,
    size_t align ) [static]
```

References [account_allocation\(\)](#), [align_up_size\(\)](#), [kmalloc_prefix_t::block](#), [can_split\(\)](#), [heap_block::free](#), [KMALLOC_MAGIC](#), [KMALLOC_MIN_ALIGN](#), [kmalloc_prefix_t::magic](#), [heap_block::next](#), [normalize_alignment\(\)](#), [heap_block::prev](#), [heap_block::requested](#), and [heap_block::size](#).

Referenced by [kmalloc_aligned\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.5 can_split()

```
static int can_split (
    size_t remainder ) [static]
```

References [minSplittablePayload\(\)](#).

Referenced by [allocateFromBlock\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.6 coalesce_neighbors()

```
static void coalesce_neighbors (
    heap_block_t * block ) [static]
```

References [heap_block::free](#), [heap_head](#), [heap_block::next](#), [heap_block::prev](#), and [heap_block::size](#).

Referenced by [kfree\(\)](#).

Here is the caller graph for this function:

7.118.3.7 kfree()

```
void kfree (
    void * ptr )
```

References [account_free\(\)](#), [kmalloc_prefix_t::block](#), [coalesce_neighbors\(\)](#), [heap_block::free](#), [heap_initialized](#), [KMALLOC_MAGIC](#), [kmalloc_prefix_t::magic](#), [pointer_within_heap\(\)](#), and [heap_block::requested](#).

Referenced by [hal_mem_free\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.8 kmalloc()

```
void * kmalloc (
    size_t size )
```

References [kmalloc_aligned\(\)](#), and [KMALLOC_MIN_ALIGN](#).

Referenced by [hal_mem_alloc\(\)](#), and [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.9 kmalloc_aligned()

```
void * kmalloc_aligned (
    size_t size,
    size_t align )
```

References [allocateFromBlock\(\)](#), [heap_block::free](#), [heap_head](#), [heap_initialized](#), [heap_block::next](#), and [normalize_alignment\(\)](#).

Referenced by [hal_mem_alloc_aligned\(\)](#), and [kmalloc\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.10 kmalloc_get_stats()

```
kmalloc_stats_t kmalloc_get_stats (
    void )
```

References [heap_stats](#).

Referenced by [print_heap_stats\(\)](#).

Here is the caller graph for this function:

7.118.3.11 kmalloc_heap_base_addr()

```
uintptr_t kmalloc_heap_base_addr (
    void )
```

References [heap_base](#).

7.118.3.12 kmalloc_heap_end_addr()

```
uintptr_t kmalloc_heap_end_addr (
    void )
```

References [heap_base](#), and [heap_size](#).

7.118.3.13 kmalloc_init()

```
int kmalloc_init (
    uint64_t heap_size_bytes )
```

References [heap_block::free](#), [kmalloc_stats_t::free_bytes](#), [heap_base](#), [heap_head](#), [heap_initialized](#), [heap_size](#), [heap_stats](#), [KMALLOC_DEFAULT_HEAP_SIZE](#), [heap_block::next](#), [kmalloc_stats_t::peak_bytes](#), [pmm_alloc_contiguous\(\)](#), [pmm_is_initialized\(\)](#), [PMM_PAGE_SIZE](#), [heap_block::prev](#), [heap_block::requested](#), [heap_block::size](#), [kmalloc_stats_t::total_bytes](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.118.3.14 kmalloc_is_initialized()

```
int kmalloc_is_initialized (
    void )
```

References [heap_initialized](#).

7.118.3.15 minSplittablePayload()

```
static size_t minSplittablePayload (
    void ) [static]
```

References [KMALLOC_MIN_ALIGN](#).

Referenced by [can_split\(\)](#).

Here is the caller graph for this function:

7.118.3.16 normalize_alignment()

```
static size_t normalize_alignment (
    size_t align ) [static]
```

References [KMALLOC_MIN_ALIGN](#).

Referenced by [allocate_from_block\(\)](#), and [kmalloc_aligned\(\)](#).

Here is the caller graph for this function:

7.118.3.17 pointer_within_heap()

```
static int pointer_within_heap (
    const void * ptr ) [static]
```

References [heap_base](#), and [heap_size](#).

Referenced by [kfree\(\)](#).

Here is the caller graph for this function:

7.118.3.18 update_free_bytes()

```
static void update_free_bytes (
    void ) [static]
```

References [kmalloc_stats_t::free_bytes](#), [heap_stats](#), [kmalloc_stats_t::total_bytes](#), and [kmalloc_stats_t::used_bytes](#).

Referenced by [account_allocation\(\)](#), and [account_free\(\)](#).

Here is the caller graph for this function:

7.118.4 Variable Documentation**7.118.4.1 heap_base**

```
uint8_t* heap_base = NULL [static]
```

Referenced by [kmalloc_heap_base_addr\(\)](#), [kmalloc_heap_end_addr\(\)](#), [kmalloc_init\(\)](#), and [pointer_within_heap\(\)](#).

7.118.4.2 heap_head

```
heap_block_t* heap_head = NULL [static]
```

Referenced by [coalesce_neighbors\(\)](#), [kmalloc_aligned\(\)](#), and [kmalloc_init\(\)](#).

7.118.4.3 heap_initialized

```
int heap_initialized = 0 [static]
```

Referenced by [kfree\(\)](#), [kmalloc_aligned\(\)](#), [kmalloc_init\(\)](#), and [kmalloc_is_initialized\(\)](#).

7.118.4.4 heap_size

```
uint64_t heap_size = 0 [static]
```

Referenced by [kmalloc_heap_end_addr\(\)](#), [kmalloc_init\(\)](#), and [pointer_within_heap\(\)](#).

7.118.4.5 heap_stats

```
kmalloc_stats_t heap_stats = {0} [static]
```

Referenced by [account_allocation\(\)](#), [account_free\(\)](#), [kmalloc_get_stats\(\)](#), [kmalloc_init\(\)](#), and [update_free_bytes\(\)](#).

7.119 src/starkernel/memory/pmm.c File Reference

```
#include "pmm.h"
```

```
#include "console.h"
```

Include dependency graph for pmm.c:

Macros

- #define [PMM_MAX_BITMAP_BYTES](#) (2u * 1024u * 1024u) /* Tracks up to ~64 GiB */
pmm.c - Physical Memory Manager (bitmap allocator)
- #define [PMM_BITMAP_BITS](#) ([PMM_MAX_BITMAP_BYTES](#) * 8ull)

Functions

- static void [pmm_memset](#) (void *dst, uint8_t value, size_t size)
- static uint64_t [pmm_bytes_to_pages](#) (uint64_t bytes)
- static void [bitmap_set](#) (uint64_t bit)
- static void [bitmap_clear](#) (uint64_t bit)
- static int [bitmap_test](#) (uint64_t bit)
- static void [bitmap_mark_range](#) (uint64_t start_page, uint64_t page_count, int used)
- static uint64_t [find_contiguous_free](#) (uint64_t pages_needed)
- static int [is_usable_memory](#) (uint32_t type)
- static int [is_ram_type](#) (uint32_t type)
- static void [recompute_free_pages](#) (void)
- int [pmm_init](#) (BootInfo *boot_info)
- int [pmm_is_initialized](#) (void)
- uint64_t [pmm_alloc_page](#) (void)
- uint64_t [pmm_alloc_contiguous](#) (uint64_t num_pages)
- void [pmm_free_page](#) (uint64_t paddr)
- void [pmm_free_contiguous](#) (uint64_t paddr, uint64_t num_pages)
- [pmm_stats_t](#) [pmm_get_stats](#) (void)

Variables

- static const uint64_t [pmm_alloc_floor_page](#) = 0x100000u / [PMM_PAGE_SIZE](#)
- static uint8_t [pmm_bitmap](#) [(2u * 1024u * 1024u)]
- static uint64_t [pmm_total_pages](#) = 0
- static uint64_t [pmm_free_pages](#) = 0
- static int [pmm_initialized](#) = 0
- static uint64_t [pmm_tracked_pages](#) = 0
- char [__kernel_start](#) []
- char [__kernel_end](#) []

7.119.1 Macro Definition Documentation

7.119.1.1 PMM_BITMAP_BITS

```
#define PMM_BITMAP_BITS (PMM_MAX_BITMAP_BYTES * 8ull)
```

7.119.1.2 PMM_MAX_BITMAP_BYTES

```
#define PMM_MAX_BITMAP_BYTES (2u * 1024u * 1024u) /* Tracks up to ~64 GiB */
```

[pmm.c](#) - Physical Memory Manager (bitmap allocator)

Builds a 1-bit-per-page bitmap from the UEFI memory map and provides primitive page allocation for early kernel bring-up. The allocator favors simplicity and debuggability; performance tuning can happen once paging and interrupts are in place.

7.119.2 Function Documentation

7.119.2.1 bitmap_clear()

```
static void bitmap_clear (
    uint64_t bit ) [inline], [static]
```

References [pmm_bitmap](#).

Referenced by [bitmap_mark_range\(\)](#).

Here is the caller graph for this function:

7.119.2.2 bitmap_mark_range()

```
static void bitmap_mark_range (
    uint64_t start_page,
    uint64_t page_count,
    int used ) [static]
```

References [bitmap_clear\(\)](#), [bitmap_set\(\)](#), [bitmap_test\(\)](#), [pmm_free_pages](#), and [pmm_tracked_pages](#).

Referenced by [pmm_alloc_contiguous\(\)](#), [pmm_alloc_page\(\)](#), [pmm_free_contiguous\(\)](#), and [pmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.3 bitmap_set()

```
static void bitmap_set (
    uint64_t bit ) [inline], [static]
```

References [pmm_bitmap](#).

Referenced by [bitmap_mark_range\(\)](#).

Here is the caller graph for this function:

7.119.2.4 bitmap_test()

```
static int bitmap_test (
    uint64_t bit ) [inline], [static]
```

References [pmm_bitmap](#).

Referenced by [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), and [recompute_free_pages\(\)](#).

Here is the caller graph for this function:

7.119.2.5 find_contiguous_free()

```
static uint64_t find_contiguous_free (
    uint64_t pages_needed ) [static]
```

References [bitmap_test\(\)](#), [pmm_alloc_floor_page](#), and [pmm_tracked_pages](#).

Referenced by [pmm_alloc_contiguous\(\)](#), and [pmm_alloc_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.6 is_ram_type()

```
static int is_ram_type (
    uint32_t type ) [static]
```

References [EfiACPIMemoryNVS](#), [EfiACPIReclaimMemory](#), [EfiRuntimeServicesCode](#), [EfiRuntimeServicesData](#), and [is_usable_memory\(\)](#).

Referenced by [pmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.7 is_usable_memory()

```
static int is_usable_memory (
    uint32_t type ) [static]
```

References [EfiConventionalMemory](#).

Referenced by [is_ram_type\(\)](#), and [pmm_init\(\)](#).

Here is the caller graph for this function:

7.119.2.8 pmm_alloc_contiguous()

```
uint64_t pmm_alloc_contiguous (
    uint64_t num_pages )
```

References [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Referenced by [kmallocc_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.9 pmm_alloc_page()

```
uint64_t pmm_alloc_page (
    void )
```

References [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Referenced by [alloc_page_zeroed\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.10 pmm_bytes_to_pages()

```
static uint64_t pmm_bytes_to_pages (
    uint64_t bytes ) [inline], [static]
```

References [PMM_PAGE_SIZE](#).

Referenced by [pmm_init\(\)](#).

Here is the caller graph for this function:

7.119.2.11 pmm_free_contiguous()

```
void pmm_free_contiguous (
    uint64_t paddr,
    uint64_t num_pages )
```

References [bitmap_mark_range\(\)](#), [pmm_initialized](#), [PMM_PAGE_SIZE](#), and [pmm_tracked_pages](#).

Referenced by [pmm_free_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.12 pmm_free_page()

```
void pmm_free_page (
    uint64_t paddr )
```

References [pmm_free_contiguous\(\)](#), [pmm_initialized](#), and [PMM_PAGE_SIZE](#).

Here is the call graph for this function:

7.119.2.13 pmm_get_stats()

```
pmm_stats_t pmm_get_stats (
    void )
```

References [pmm_stats_t::free_bytes](#), [pmm_stats_t::free_pages](#), [pmm_free_pages](#), [PMM_PAGE_SIZE](#), [pmm_total_pages](#), [pmm_stats_t::total_bytes](#), [pmm_stats_t::total_pages](#), [pmm_stats_t::used_bytes](#), and [pmm_stats_t::used_pages](#).

Referenced by [print_pmm_stats\(\)](#).

Here is the caller graph for this function:

7.119.2.14 pmm_init()

```
int pmm_init (
    BootInfo * boot_info )
```

References [__kernel_end](#), [__kernel_start](#), [bitmap_mark_range\(\)](#), [is_ram_type\(\)](#), [is_usable_memory\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [EFI_MEMORY_DESCRIPTOR::NumberOfPages](#), [EFI_MEMORY_DESCRIPTOR::PhysicalStart](#), [pmm_alloc_floor_page](#), [pmm_bitmap](#), [PMM_BITMAP_BITS](#), [pmm_bytes_to_pages\(\)](#), [pmm_free_pages](#), [pmm_initialized](#), [pmm_memset\(\)](#), [PMM_PAGE_SIZE](#), [pmm_total_pages](#), [pmm_tracked_pages](#), [recompute_free_pages\(\)](#), and [EFI_MEMORY_DESCRIPTOR::Type](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.2.15 pmm_is_initialized()

```
int pmm_is_initialized (
    void )
```

References [pmm_initialized](#).

Referenced by [kmallocc_init\(\)](#), and [vmm_init\(\)](#).

Here is the caller graph for this function:

7.119.2.16 pmm_memset()

```
static void pmm_memset (
    void * dst,
    uint8_t value,
    size_t size ) [static]
```

Referenced by [pmm_init\(\)](#).

Here is the caller graph for this function:

7.119.2.17 recompute_free_pages()

```
static void recompute_free_pages (
    void ) [static]
```

References [bitmap_test\(\)](#), [pmm_free_pages](#), and [pmm_tracked_pages](#).

Referenced by [pmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.119.3 Variable Documentation

7.119.3.1 __kernel_end

```
char __kernel_end[] [extern]
```

Referenced by [pmm_init\(\)](#).

7.119.3.2 `__kernel_start`

`char __kernel_start[] [extern]`
 Referenced by [pmm_init\(\)](#).

7.119.3.3 `pmm_alloc_floor_page`

`const uint64_t pmm_alloc_floor_page = 0x100000u / PMM_PAGE_SIZE [static]`
 Referenced by [find_contiguous_free\(\)](#), and [pmm_init\(\)](#).

7.119.3.4 `pmm_bitmap`

`uint8_t pmm_bitmap[(2u * 1024u * 1024u)] [static]`
 Referenced by [bitmap_clear\(\)](#), [bitmap_set\(\)](#), [bitmap_test\(\)](#), and [pmm_init\(\)](#).

7.119.3.5 `pmm_free_pages`

`uint64_t pmm_free_pages = 0 [static]`
 Referenced by [bitmap_mark_range\(\)](#), [pmm_get_stats\(\)](#), [pmm_init\(\)](#), and [recompute_free_pages\(\)](#).

7.119.3.6 `pmm_initialized`

`int pmm_initialized = 0 [static]`
 Referenced by [pmm_alloc_contiguous\(\)](#), [pmm_alloc_page\(\)](#), [pmm_free_contiguous\(\)](#), [pmm_free_page\(\)](#), [pmm_init\(\)](#), and [pmm_is_initialized\(\)](#).

7.119.3.7 `pmm_total_pages`

`uint64_t pmm_total_pages = 0 [static]`
 Referenced by [pmm_get_stats\(\)](#), and [pmm_init\(\)](#).

7.119.3.8 `pmm_tracked_pages`

`uint64_t pmm_tracked_pages = 0 [static]`
 Referenced by [bitmap_mark_range\(\)](#), [find_contiguous_free\(\)](#), [pmm_free_contiguous\(\)](#), [pmm_init\(\)](#), and [recompute_free_pages\(\)](#).

7.120 `src/starkernel/memory/vmm.c` File Reference

```
#include <stddef.h>
#include <stdint.h>
#include "vmm.h"
#include "pmm.h"
#include "console.h"
#include "uefi.h"
Include dependency graph for vmm.c:
```

Macros

- `#define PTE_PRESENT (1ull << 0)`
 vmm.c - 4-level paging Virtual Memory Manager (x86_64)
- `#define PTE_WRITABLE (1ull << 1)`
- `#define PTE_USER (1ull << 2)`
- `#define PTE_PWT (1ull << 3)`
- `#define PTE_PCD (1ull << 4)`
- `#define PTE_ACCESSED (1ull << 5)`
- `#define PTE_DIRTY (1ull << 6)`
- `#define PTE_PS (1ull << 7)`

- `#define PTE_GLOBAL (1ull << 8)`
- `#define PTE_NX (1ull << 63)`
- `#define ADDR_MASK 0x000FFFFFFFFFFFF000ull`

Functions

- static int `is_ram_type` (uint32_t type)
- static uint64_t * `paddr_to_virt` (uint64_t paddr)
- static uint64_t `alloc_page_zeroed` (void)
- static uint16_t `pml4_index` (uint64_t vaddr)
- static uint16_t `pdpt_index` (uint64_t vaddr)
- static uint16_t `pd_index` (uint64_t vaddr)
- static uint16_t `pt_index` (uint64_t vaddr)
- static uint64_t * `get_or_alloc_table` (uint64_t *parent, uint16_t idx)
- static uint64_t * `get_table` (uint64_t *parent, uint16_t idx)
- static uint64_t `make_pte` (uint64_t paddr, uint64_t flags)
- static void `invlpg` (uint64_t vaddr)
- static void `load_cr3` (uint64_t pml4_phys)
- static uint64_t * `resolve_pt` (uint64_t vaddr, uint16_t *idx_out)
- int `vmm_map_page` (uint64_t vaddr, uint64_t paddr, uint64_t flags)
- int `vmm_unmap_page` (uint64_t vaddr)
- uint64_t `vmm_get_paddr` (uint64_t vaddr)
- int `vmm_map_range` (uint64_t vaddr, uint64_t paddr, uint64_t size, uint64_t flags)
- int `vmm_query_page` (uint64_t vaddr, vmm_page_info_t *info)
- int `vmm_init` (BootInfo *boot_info)

Variables

- static uint64_t `vmm_root_pml4_phys` = 0
- static const uint64_t `lapic_phys_base` = 0xFEE00000ull
- static const uint64_t `hpet_phys_base` = 0xFED00000ull

7.120.1 Macro Definition Documentation

7.120.1.1 ADDR_MASK

```
#define ADDR_MASK 0x000FFFFFFFFFFFF000ull
```

7.120.1.2 PTE_ACCESSED

```
#define PTE_ACCESSED (1ull << 5)
```

7.120.1.3 PTE_DIRTY

```
#define PTE_DIRTY (1ull << 6)
```

7.120.1.4 PTE_GLOBAL

```
#define PTE_GLOBAL (1ull << 8)
```

7.120.1.5 PTE_NX

```
#define PTE_NX (1ull << 63)
```

7.120.1.6 PTE_PCD

```
#define PTE_PCD (1ull << 4)
```

7.120.1.7 PTE_PRESENT

```
#define PTE_PRESENT (1ull << 0)
```

[vmm.c](#) - 4-level paging Virtual Memory Manager (x86_64)

Minimal bring-up: identity maps the first 16 MiB, supports single-page map/unmap/query, and switches to the new page table root.

7.120.1.8 PTE_PS

```
#define PTE_PS (1ull << 7)
```

7.120.1.9 PTE_PWT

```
#define PTE_PWT (1ull << 3)
```

7.120.1.10 PTE_USER

```
#define PTE_USER (1ull << 2)
```

7.120.1.11 PTE_WRITABLE

```
#define PTE_WRITABLE (1ull << 1)
```

7.120.2 Function Documentation

7.120.2.1 alloc_page_zeroed()

```
static uint64_t alloc_page_zeroed (
    void ) [static]
```

References [paddr_to_virt\(\)](#), [pmm_alloc_page\(\)](#), and [VMM_PAGE_SIZE](#).

Referenced by [get_or_alloc_table\(\)](#), and [vmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.2 get_or_alloc_table()

```
static uint64_t * get_or_alloc_table (
    uint64_t * parent,
    uint16_t idx ) [static]
```

References [ADDR_MASK](#), [alloc_page_zeroed\(\)](#), [entry](#), [paddr_to_virt\(\)](#), [PTE_PRESENT](#), and [PTE_WRITABLE](#).

Referenced by [vmm_map_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.3 get_table()

```
static uint64_t * get_table (
    uint64_t * parent,
    uint16_t idx ) [static]
```

References [ADDR_MASK](#), [entry](#), [paddr_to_virt\(\)](#), and [PTE_PRESENT](#).

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.4 invlpg()

```
static void invlpg (
    uint64_t vaddr ) [inline], [static]
```

Referenced by [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.5 is_ram_type()

```
static int is_ram_type (
    uint32_t type ) [static]
```

References [EfiACPIMemoryNVS](#), [EfiACPIReclaimMemory](#), [EfiBootServicesCode](#), [EfiBootServicesData](#), [EfiConventionalMemory](#), [EfiLoaderCode](#), [EfiLoaderData](#), [EfiRuntimeServicesCode](#), and [EfiRuntimeServicesData](#).

Referenced by [vmm_init\(\)](#).

Here is the caller graph for this function:

7.120.2.6 load_cr3()

```
static void load_cr3 (
    uint64_t pml4_phys ) [inline], [static]
```

Referenced by [vmm_init\(\)](#).

Here is the caller graph for this function:

7.120.2.7 make_pte()

```
static uint64_t make_pte (
    uint64_t paddr,
    uint64_t flags ) [static]
```

References [ADDR_MASK](#), [flags](#), [PTE_NX](#), [PTE_PRESENT](#), [PTE_USER](#), [PTE_WRITABLE](#), [VMM_FLAG_NX](#), [VMM_FLAG_PRESENT](#), [VMM_FLAG_USER](#), and [VMM_FLAG_WRITABLE](#).

Referenced by [vmm_map_page\(\)](#).

Here is the caller graph for this function:

7.120.2.8 paddr_to_virt()

```
static uint64_t * paddr_to_virt (
    uint64_t paddr ) [inline], [static]
```

Referenced by [alloc_page_zeroed\(\)](#), [get_or_alloc_table\(\)](#), [get_table\(\)](#), [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.9 pd_index()

```
static uint16_t pd_index (
    uint64_t vaddr ) [inline], [static]
```

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.10 pdpt_index()

```
static uint16_t pdpt_index (
    uint64_t vaddr ) [inline], [static]
```

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.11 pml4_index()

```
static uint16_t pml4_index (
    uint64_t vaddr ) [inline], [static]
```

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.12 pt_index()

```
static uint16_t pt_index (
    uint64_t vaddr ) [inline], [static]
```

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

Here is the caller graph for this function:

7.120.2.13 `resolve_pt()`

```
static uint64_t * resolve_pt (
    uint64_t vaddr,
    uint16_t * idx_out ) [static]
```

References [get_table\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), and [vmm_root_pml4_phys](#).

Referenced by [vmm_query_page\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.14 `vmm_get_paddr()`

```
uint64_t vmm_get_paddr (
    uint64_t vaddr )
```

References [ADDR_MASK](#), [get_table\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), and [vmm_root_pml4_phys](#).

Here is the call graph for this function:

7.120.2.15 `vmm_init()`

```
int vmm_init (
    BootInfo * boot_info )
```

References [alloc_page_zeroed\(\)](#), [console_println\(\)](#), [hpet_phys_base](#), [is_ram_type\(\)](#), [lapic_phys_base](#), [load_cr3\(\)](#), [BootInfo::memory_map](#), [BootInfo::memory_map_descriptor_size](#), [BootInfo::memory_map_size](#), [EFI_MEMORY_DESCRIPTOR::Num](#), [EFI_MEMORY_DESCRIPTOR::PhysicalStart](#), [pmm_is_initialized\(\)](#), [EFI_MEMORY_DESCRIPTOR::Type](#), [VMM_FLAG_WRITABLE](#), [vmm_map_range\(\)](#), [VMM_PAGE_SIZE](#), and [vmm_root_pml4_phys](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.16 `vmm_map_page()`

```
int vmm_map_page (
    uint64_t vaddr,
    uint64_t paddr,
    uint64_t flags )
```

References [ADDR_MASK](#), [console_println\(\)](#), [flags](#), [get_or_alloc_table\(\)](#), [invlpg\(\)](#), [make_pte\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), [PTE_WRITABLE](#), [VMM_FLAG_PRESENT](#), and [vmm_root_pml4_phys](#).

Referenced by [vmm_map_range\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.17 `vmm_map_range()`

```
int vmm_map_range (
    uint64_t vaddr,
    uint64_t paddr,
    uint64_t size,
    uint64_t flags )
```

References [flags](#), [vmm_map_page\(\)](#), and [VMM_PAGE_SIZE](#).

Referenced by [timer_init\(\)](#), and [vmm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.120.2.18 `vmm_query_page()`

```
int vmm_query_page (
    uint64_t vaddr,
    vmm_page_info_t * info )
```


References [entry](#), [vmm_page_info_t::executable](#), [vmm_page_info_t::present](#), [PTE_NX](#), [PTE_PRESENT](#), [PTE_WRITABLE](#), [resolve_pt\(\)](#), and [vmm_page_info_t::writable](#).

Here is the call graph for this function:

7.120.2.19 vmm_unmap_page()

```
int vmm_unmap_page (
    uint64_t vaddr )
```

References [get_table\(\)](#), [invlpg\(\)](#), [paddr_to_virt\(\)](#), [pd_index\(\)](#), [pdpt_index\(\)](#), [pml4_index\(\)](#), [pt_index\(\)](#), [PTE_PRESENT](#), and [vmm_root_pml4_phys](#).

Here is the call graph for this function:

7.120.3 Variable Documentation

7.120.3.1 hpet_phys_base

```
const uint64_t hpet_phys_base = 0xFED00000ull [static]
```

Referenced by [vmm_init\(\)](#).

7.120.3.2 lapic_phys_base

```
const uint64_t lapic_phys_base = 0xFEE00000ull [static]
```

Referenced by [vmm_init\(\)](#).

7.120.3.3 vmm_root_pml4_phys

```
uint64_t vmm_root_pml4_phys = 0 [static]
```

Referenced by [resolve_pt\(\)](#), [vmm_get_paddr\(\)](#), [vmm_init\(\)](#), [vmm_map_page\(\)](#), and [vmm_unmap_page\(\)](#).

7.121 src/repl.c File Reference

```
#include "../include/repl.h"
#include "../include/log.h"
#include "../include/vm.h"
#include <stdio.h>
```

Include dependency graph for repl.c:

Functions

- [__attribute__](#) ((weak))
- void [vm_repl](#) (VM *vm, int script_mode)
Starts the Forth REPL (Read-Eval-Print Loop)

7.121.1 Function Documentation

7.121.1.1 __attribute__()

```
__attribute__ (
    (weak) )
```

References [VM::halted](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.121.1.2 vm_repl()

```
void vm_repl (
    VM * vm,
    int script_mode )
```

Starts the Forth REPL (Read-Eval-Print Loop)

Start the [VM](#)'s read-eval-print loop.

Starts the REPL (Read-Eval-Print Loop) for the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the VM structure
<i>script_mode</i>	If 1, suppress prompts and "ok" output (for piped input)

This function implements an interactive REPL for the Forth interpreter. It continuously reads input from the user, interprets it, and prints the result until the [VM](#) is halted or an error occurs.

In interactive mode (*script_mode*=0):

- Prints colored prompt "ok> "
- Acknowledges successful commands with " ok"
- Reports errors with " ERROR"

In script mode (*script_mode*=1):

- Suppresses all prompts and status messages
- Suitable for piped input (here doc, pipes, redirects)

References [VM::error](#), [VM::halted](#), [LOG_INFO](#), [log_message\(\)](#), [vm](#), [vm_interpret\(\)](#), and [VM::zuse_session](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.122 src/starkernel/repl.c File Reference

```
#include "starkernel/repl.h"
#include "console.h"
#include "vm.h"
```

Include dependency graph for repl.c:

Functions

- static int [sk_readline](#) (char *buf, int size)
repl.c - Emergency FORTH REPL for LithosAnanke kernel
- static void [sk_fault_handler](#) (VM *vm)
- void [sk_repl](#) (VM *vm)
repl.h - Emergency FORTH REPL for LithosAnanke kernel

7.122.1 Function Documentation

7.122.1.1 sk_fault_handler()

```
static void sk_fault_handler (
    VM * vm ) [static]
```

References [console_println\(\)](#), [VM::halted](#), and [vm](#).

Referenced by [sk_repl\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.122.1.2 sk_readline()

```
static int sk_readline (
    char * buf,
    int size ) [static]
```

repl.c - Emergency FORTH REPL for LithosAnanke kernel

Direct adaptation of [src/repl.c](#) for the freestanding kernel context. Replaces libc stdio (fgets/printf/fflush) with HAL serial I/O:

- Input: [console_getc\(\)](#) busy-wait with local echo and backspace support
- Output: [console_puts\(\)](#) / [console_putc\(\)](#)

Runs with interrupts enabled so the APIC heartbeat fires normally. Designed as the last thing kernel_main does before the idle loop.

References [console_getc\(\)](#), and [console_putc\(\)](#).

Referenced by [sk_repl\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.122.1.3 sk_repl()

```
void sk_repl (
    VM * vm )
```

repl.h - Emergency FORTH REPL for LithosAnanke kernel

sk_repl - Run the emergency FORTH REPL on the serial console.

Blocks until vm->halted is set (BYE word) or the VM encounters a halt. Runs with interrupts enabled; the APIC heartbeat continues to fire.

Parameters

vm	Mama VM instance (must be fully initialised)
----	--

References [console_println\(\)](#), [console_puts\(\)](#), [VM::emergency_console](#), [VM::error](#), [VM::halted](#), [sk_fault_handler\(\)](#), [sk_readline\(\)](#), [vm](#), [vm_interpret\(\)](#), and [VM::zuse_session](#).

Referenced by [kernel_main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.123 src/starkernel/vm/arena.c File Reference

7.124 src/starkernel/vm/bootstrap/sk_vm_bootstrap.c File Reference

7.125 src/starkernel/vm/host/shim.c File Reference

7.126 src/starkernel/vm/parity.c File Reference

```
#include "starkernel/vm/parity.h"
#include "vm.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
```

Include dependency graph for parity.c:

Macros

- #define [SK_PARITY_DEBUG](#) 0
parity.c - Parity packet collection and canonical hash
- #define [sk_parity_debug_log_msg](#)(msg) do { (void)(msg); } while (0)
- #define [sk_parity_debug_check_ptr](#)(vm, label, [entry](#), idx, hpctr, xt, ptr) do { (void)(vm); (void)(label); (void)([entry](#)); (void)(idx); (void)(hpctr); (void)(xt); (void)(ptr); } while (0)
- #define [MAX_DICT_ENTRIES](#) 2048

Functions

- static void [print_u32](#) (uint32_t val)
Helper: print uint32 as decimal.

- static void [print_hex64](#) (uint64_t val)
Helper: print uint64 as hex (with 0x prefix)
- static void [print_str](#) (const char *s)
Helper: print string.
- static void [print_nl](#) (void)
Helper: print newline.
- uint64_t [fnv1a_64](#) (const uint8_t *data, size_t len, uint64_t hash)
fnv1a_64 - FNV-1a 64-bit hash function
- static uint64_t [fnv1a_64_u8](#) (uint8_t byte, uint64_t hash)
fnv1a_64_u8 - Hash a single byte
- static uint64_t [fnv1a_64_u32](#) (uint32_t val, uint64_t hash)
fnv1a_64_u32 - Hash a 32-bit value (little-endian)
- uint32_t [sk_dict_word_count](#) (struct VM *vm)
sk_dict_word_count - Count dictionary entries
- static uint64_t [hash_colon_body](#) (struct VM *vm, DictEntry *entry, uint32_t word_index, uint64_t hash)
hash_colon_body - Hash compiled word body as word_id sequence
- uint64_t [sk_dict_canonical_hash](#) (struct VM *vm)
sk_dict_canonical_hash - Compute canonical dictionary hash
- void [sk_parity_collect](#) (struct VM *vm, ParityPacket *out)
sk_parity_collect - Collect parity data from VM
- void [sk_parity_print](#) (const ParityPacket *pkt)
sk_parity_print - Print parity packet to console

7.126.1 Macro Definition Documentation

7.126.1.1 MAX_DICT_ENTRIES

```
#define MAX_DICT_ENTRIES 2048
```

7.126.1.2 SK_PARITY_DEBUG

```
#define SK_PARITY_DEBUG 0
```

[parity.c](#) - Parity packet collection and canonical hash

Implements M7 parity validation for hosted vs kernel comparison.

M7 Normative Rules enforced here:

- Rule 1: word_id is monotonic creation index
- Rule 2: Colon bodies hashed as word_id sequence
- Rule 3: Dictionary traversal in creation order

7.126.1.3 sk_parity_debug_check_ptr

```
#define sk_parity_debug_check_ptr(
    vm,
    label,
    entry,
    idx,
    hptr,
    xt,
    ptr ) do { (void) (vm); (void) (label); (void) (entry); (void) (idx); (void) (hptr);
(void) (xt); (void) (ptr); } while (0)
```

7.126.1.4 sk_parity_debug_log_msg

```
#define sk_parity_debug_log_msg(
    msg ) do { (void)(msg); } while (0)
```

7.126.2 Function Documentation

7.126.2.1 fnv1a_64()

```
uint64_t fnv1a_64 (
    const uint8_t * data,
    size_t len,
    uint64_t hash )
```

fnv1a_64 - FNV-1a 64-bit hash function

Parameters

<i>data</i>	Pointer to data to hash
<i>len</i>	Length of data
<i>hash</i>	Current hash value (use FNV1A_64_OFFSET_BASIS for initial)

Returns

Updated hash value

References [FNV1A_64_PRIME](#).

Referenced by [sk_dict_canonical_hash\(\)](#).

Here is the caller graph for this function:

7.126.2.2 fnv1a_64_u32()

```
static uint64_t fnv1a_64_u32 (
    uint32_t val,
    uint64_t hash ) [static]
```

fnv1a_64_u32 - Hash a 32-bit value (little-endian)

References [fnv1a_64_u8\(\)](#).

Referenced by [hash_colon_body\(\)](#), and [sk_dict_canonical_hash\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.3 fnv1a_64_u8()

```
static uint64_t fnv1a_64_u8 (
    uint8_t byte,
    uint64_t hash ) [static]
```

fnv1a_64_u8 - Hash a single byte

References [FNV1A_64_PRIME](#).

Referenced by [fnv1a_64_u32\(\)](#), [hash_colon_body\(\)](#), and [sk_dict_canonical_hash\(\)](#).

Here is the caller graph for this function:

7.126.2.4 hash_colon_body()

```
static uint64_t hash_colon_body (
    struct VM * vm,
    DictEntry * entry,
    uint32_t word_index,
    uint64_t hash ) [static]
```

hash_colon_body - Hash compiled word body as word_id sequence

Traverses threaded code, hashes word_id of each called word. Literals are hashed as their value (not address).

Parameters

<i>vm</i>	VM instance
<i>entry</i>	Dictionary entry (must have WORD_COMPILED flag)
<i>word_index</i>	Dictionary ordinal (creation order)
<i>hash</i>	Current hash value

Returns

Updated hash value

References [entry](#), [fnv1a_64_u32\(\)](#), [fnv1a_64_u8\(\)](#), [DictEntry::func](#), [VM::memory](#), [sk_parity_debug_check_ptr](#), [sk_parity_debug_log_msg](#), [vm](#), [vm_dictionary_get_data_field\(\)](#), [vm_find_word\(\)](#), [VM_MEMORY_SIZE](#), [vm_ptr\(\)](#), [WORD_COMPILED](#), and [DictEntry::word_id](#).

Referenced by [sk_dict_canonical_hash\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.5 print_hex64()

```
static void print_hex64 (
    uint64_t val ) [static]
```

Helper: print uint64 as hex (with 0x prefix)

References [console_puts\(\)](#).

Referenced by [sk_parity_print\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.6 print_nl()

```
static void print_nl (
    void ) [static]
```

Helper: print newline.

References [console_println\(\)](#).

Referenced by [sk_parity_print\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.7 print_str()

```
static void print_str (
    const char * s ) [static]
```

Helper: print string.

References [console_puts\(\)](#).

Referenced by [sk_parity_print\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.8 print_u32()

```
static void print_u32 (
    uint32_t val ) [static]
```

Helper: print uint32 as decimal.

References [console_puts\(\)](#).

Referenced by [sk_parity_print\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.9 sk_dict_canonical_hash()

```
uint64_t sk_dict_canonical_hash (
    struct VM * vm )
```

`sk_dict_canonical_hash` - Compute canonical dictionary hash

Traverses dictionary in creation order, hashes structural fields only.

References [DictEntry::acl_default](#), [DictEntry::flags](#), [fnv1a_64\(\)](#), [FNV1A_64_OFFSET_BASIS](#), [fnv1a_64_u32\(\)](#), [fnv1a_64_u8\(\)](#), [DictEntry::func](#), [hash_colon_body\(\)](#), [VM::latest](#), [MAX_DICT_ENTRIES](#), [DictEntry::name](#), [DictEntry::name_len](#), [sk_parity_debug_check_ptr](#), [sk_parity_debug_log_msg](#), [vm](#), [WORD_COMPILED](#), and [DictEntry::word_id](#).

Referenced by [sk_parity_collect\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.126.2.10 sk_dict_word_count()

```
uint32_t sk_dict_word_count (
    struct VM * vm )
```

sk_dict_word_count - Count dictionary entries

Traverses from vm->latest to NULL.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Returns

Number of dictionary entries

References [VM::latest](#), [MAX_DICT_ENTRIES](#), and [vm](#).

Referenced by [sk_parity_collect\(\)](#).

Here is the caller graph for this function:

7.126.2.11 sk_parity_collect()

```
void sk_parity_collect (
    struct VM * vm,
    ParityPacket * out )
```

sk_parity_collect - Collect parity data from [VM](#)

Traverses dictionary in creation order, computes canonical hash. Does NOT include runtime fields (execution_heat, physics, etc.).

Parameters

<i>vm</i>	Pointer to VM instance
<i>out</i>	Pointer to ParityPacket to fill

References [ParityPacket::bootstrap_result](#), [global_test_stats](#), [ParityPacket::header_hash64](#), [VM::here](#), [ParityPacket::here_offset](#), [VM::latest](#), [ParityPacket::latest_word_id](#), [SK_BOOTSTRAP_INIT_FAIL](#), [SK_BOOTSTRAP_OK](#), [sk_dict_canonical_hash\(\)](#), [sk_dict_word_count\(\)](#), [sk_parity_debug_log_msg](#), [ParityPacket::tests_errors](#), [ParityPacket::tests_failed](#), [ParityPacket::tests_passed](#), [ParityPacket::tests_skipped](#), [ParityPacket::tests_total](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), [vm](#), [ParityPacket::window_hash64](#), [ParityPacket::word_count](#), and [DictEntry::word_id](#).

Here is the call graph for this function:

7.126.2.12 sk_parity_print()

```
void sk_parity_print (
    const ParityPacket * pkt )
```

sk_parity_print - Print parity packet to console

Output format: PARITY:M7.1a word_count=N here=0xHHHH latest_id=N hash=0xHHHHHHHHHHHHHHHHHH
PARITY:M7.1b tests=N pass=N fail=N skip=N err=N

Parameters

<i>pkt</i>	Pointer to ParityPacket to print
------------	--

References [ParityPacket::bootstrap_result](#), [ParityPacket::header_hash64](#), [ParityPacket::here_offset](#), [ParityPacket::latest_word_id](#), [print_hex64\(\)](#), [print_nl\(\)](#), [print_str\(\)](#), [print_u32\(\)](#), [SK_BOOTSTRAP_OK](#), [ParityPacket::tests_errors](#), [ParityPacket::tests_failed](#), [ParityPacket::tests_passed](#), [ParityPacket::tests_skipped](#), [ParityPacket::tests_total](#), and [ParityPacket::word_count](#).

Here is the call graph for this function:

7.127 src/starkernel/vm/q48_stubs.c File Reference

```
#include "q48_16.h"
```

Include dependency graph for q48_stubs.c:

Functions

- [q48_16_t q48_from_double](#) (double d)
Convert double to Q48.16 (for testing/initialization only)
- double [q48_to_double](#) ([q48_16_t](#) q)
Convert Q48.16 to double (for testing/diagnostics only)

7.127.1 Function Documentation

7.127.1.1 q48_from_double()

```
q48_16_t q48_from_double (
    double d )
```

Convert double to Q48.16 (for testing/initialization only)

NOT used in inference engine (inference is integer-only). Provided for testing and diagnostic tools only.

Parameters

<i>d</i>	Double-precision float
----------	------------------------

Returns

$d * 2^{16}$ as Q48.16

7.127.1.2 q48_to_double()

```
double q48_to_double (
    q48_16_t q )
```

Convert Q48.16 to double (for testing/diagnostics only)

NOT used in inference engine (inference is integer-only). Provided for logging and RStudio dashboard output.

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

$q / 2^{16}$ as double

7.128 src/starkernel/vm/vm_core.c File Reference

```
#include "../include/vm.h"
#include "../include/inference_engine.h"
#include "../include/log.h"
#include "../include/word_registry.h"
#include "../include/vm_debug.h"
```



```

#include "../include/profiler.h"
#include "../include/platform_time.h"
#include "../include/physics_metadata.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/physics_pipelining_metrics.h"
#include "../include/rolling_window_of_truth.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/ssm_jacquard.h"
#include "../include/vm_host.h"
#include "word_source/include/vocabulary_words.h"
#include "vm_internal.h"
#include <stdlib.h>
#include <errno.h>
#include <time.h>
#include <stdio.h>
#include <string.h>
#include <stdint.h>

```

Include dependency graph for vm_core.c:

Macros

- #define [vm_check_arena](#)(tag) ((void)0)

Functions

- static void * [hosted_alloc](#) (size_t size, size_t align)
- static void [hosted_free](#) (void *ptr)
- static uint64_t [hosted_time_ns](#) (void)
- static int [hosted_mutex_init](#) (void *mutex)
- static int [hosted_mutex_lock](#) (void *mutex)
- static int [hosted_mutex_unlock](#) (void *mutex)
- static void [hosted_mutex_destroy](#) (void *mutex)
- static int [hosted_puts](#) (const char *str)
- static int [hosted_putc](#) (int c)
- static void [hosted_panic](#) (const char *message)
- static bool [hosted_xt_is_executable](#) (const void *ptr)
- static bool [hosted_xt_entry_owned](#) (const void *ptr, size_t bytes)
- void [vm_reset_hosted_fake_ns](#) (VM *vm)
- static void [vm_assert_interpreter_enabled](#) (VM *vm, const char *caller)
- void [vm_enable_interpreter](#) (VM *vm)
- const VMHostServices * [vm_default_host_services](#) (void)
- const VMHostServices * [vm_host](#) (const VM *vm)
- void * [vm_host_alloc](#) (VM *vm, size_t size, size_t align)
- void * [vm_host_calloc](#) (VM *vm, size_t n, size_t size)
- void [vm_host_free](#) (VM *vm, void *ptr)
- uint64_t [vm_monotonic_ns](#) (VM *vm)
- unsigned [vm_get_base](#) (const VM *vm)
- void [vm_set_base](#) (VM *vm, unsigned b)
- void [vm_cleanup](#) (VM *vm)
- int [vm_parse_word](#) (VM *vm, char *word, size_t max_len)
 - Parse next word from input buffer.
- int [vm_parse_number](#) (VM *vm, const char *s, cell_t *out)
 - Parse string as number in current base.
- void [vm_enter_compile_mode](#) (VM *vm, const char *name, size_t len)
- void [vm_compile_word](#) (VM *vm, DictEntry *entry)
- void [vm_compile_literal](#) (VM *vm, cell_t value)

- void [vm_compile_call](#) (VM *vm, word_func_t func)
- void [vm_compile_exit](#) (VM *vm)
- void [vm_exit_compile_mode](#) (VM *vm)
- void [execute_colon_word](#) (VM *vm)
- void [vm_interpret_word](#) (VM *vm, const char *word_str, size_t len)
- void [vm_interpret](#) (VM *vm, const char *input)
Interpret a string of Forth code.
- int [vm_addr_ok](#) (struct VM *vm, vaddr_t addr, size_t len)
Check if memory address range is valid.
- uint8_t * [vm_ptr](#) (struct VM *vm, vaddr_t addr)
- uint8_t [vm_load_u8](#) (struct VM *vm, vaddr_t addr)
- void [vm_store_u8](#) (struct VM *vm, vaddr_t addr, uint8_t v)
- cell_t [vm_load_cell](#) (struct VM *vm, vaddr_t addr)
- void [vm_store_cell](#) (struct VM *vm, vaddr_t addr, cell_t v)
- void [vm_make_immediate](#) (VM *vm)

Variables

- static uint64_t [hosted_fake_ns](#) = 0
- static const VMHostServices [hosted_services](#)

7.128.1 Macro Definition Documentation

7.128.1.1 vm_check_arena

```
#define vm_check_arena(  
    tag ) ((void)0)
```

7.128.2 Function Documentation

7.128.2.1 execute_colon_word()

```
void execute_colon_word (  
    VM * vm )
```

Referenced by [vm_enter_compile_mode\(\)](#).

Here is the caller graph for this function:

7.128.2.2 hosted_alloc()

```
static void * hosted_alloc (  
    size_t size,  
    size_t align ) [static]
```

7.128.2.3 hosted_free()

```
static void hosted_free (  
    void * ptr ) [static]
```

7.128.2.4 hosted_mutex_destroy()

```
static void hosted_mutex_destroy (  
    void * mutex ) [static]
```

References [sf_mutex_destroy\(\)](#).

Here is the call graph for this function:

7.128.2.5 hosted_mutex_init()

```
static int hosted_mutex_init (
    void * mutex ) [static]
```

References [sf_mutex_init\(\)](#).

Here is the call graph for this function:

7.128.2.6 hosted_mutex_lock()

```
static int hosted_mutex_lock (
    void * mutex ) [static]
```

References [sf_mutex_lock\(\)](#).

Here is the call graph for this function:

7.128.2.7 hosted_mutex_unlock()

```
static int hosted_mutex_unlock (
    void * mutex ) [static]
```

References [sf_mutex_unlock\(\)](#).

Here is the call graph for this function:

7.128.2.8 hosted_panic()

```
static void hosted_panic (
    const char * message ) [static]
```

7.128.2.9 hosted_putc()

```
static int hosted_putc (
    int c ) [static]
```

7.128.2.10 hosted_puts()

```
static int hosted_puts (
    const char * str ) [static]
```

7.128.2.11 hosted_time_ns()

```
static uint64_t hosted_time_ns (
    void ) [static]
```

References [hosted_fake_ns](#), and [sf_monotonic_ns\(\)](#).

Here is the call graph for this function:

7.128.2.12 hosted_xt_entry_owned()

```
static bool hosted_xt_entry_owned (
    const void * ptr,
    size_t bytes ) [static]
```

7.128.2.13 hosted_xt_is_executable()

```
static bool hosted_xt_is_executable (
    const void * ptr ) [static]
```

7.128.2.14 vm_addr_ok()

```
int vm_addr_ok (
    struct VM * vm,
    vaddr_t addr,
    size_t len )
```

Check if memory address range is valid.
Validates that an address range falls within [VM](#) memory bounds.

Parameters

<i>vm</i>	Pointer to VM instance
<i>addr</i>	Virtual address to check
<i>len</i>	Length of memory range

Returns

1 if range is valid, 0 if invalid

Referenced by [vm_load_cell\(\)](#), [vm_ptr\(\)](#), and [vm_store_cell\(\)](#).
Here is the caller graph for this function:

7.128.2.15 vm_assert_interpreter_enabled()

```
static void vm_assert_interpreter_enabled (
    VM * vm,
    const char * caller ) [static]
```

References [VM::error](#), [VM::interpreter_enabled](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_host\(\)](#).
Referenced by [execute_colon_word\(\)](#), [vm_interpret\(\)](#), [vm_interpret_word\(\)](#), [vm_parse_number\(\)](#), and [vm_parse_word\(\)](#).
Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.16 vm_cleanup()

```
void vm_cleanup (
    VM * vm )
```

7.128.2.17 vm_compile_call()

```
void vm_compile_call (
    VM * vm,
    word_func_t func )
```

7.128.2.18 vm_compile_exit()

```
void vm_compile_exit (
    VM * vm )
```

7.128.2.19 vm_compile_literal()

```
void vm_compile_literal (
    VM * vm,
    cell_t value )
```

Referenced by [vm_interpret_word\(\)](#).
Here is the caller graph for this function:

7.128.2.20 vm_compile_word()

```
void vm_compile_word (
    VM * vm,
    DictEntry * entry )
```

Referenced by [vm_compile_call\(\)](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_interpret_word\(\)](#).
Here is the caller graph for this function:

7.128.2.21 vm_default_host_services()

```
const VMHostServices * vm_default_host_services (
    void )
```

References [hosted_services](#), and [sk_host_services\(\)](#).

Referenced by [vm_host\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.22 vm_enable_interpreter()

```
void vm_enable_interpreter (
    VM * vm )
```

References [VM::interpreter_enabled](#), and [vm](#).

7.128.2.23 vm_enter_compile_mode()

```
void vm_enter_compile_mode (
    VM * vm,
    const char * name,
    size_t len )
```

7.128.2.24 vm_exit_compile_mode()

```
void vm_exit_compile_mode (
    VM * vm )
```

7.128.2.25 vm_get_base()

```
unsigned vm_get_base (
    const VM * vm )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), [vm_load_cell\(\)](#), and [VM_MEMORY_SIZE](#).

Referenced by [vm_parse_number\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.26 vm_host()

```
const VMHostServices * vm_host (
    const VM * vm )
```

References [VM::host](#), [vm](#), and [vm_default_host_services\(\)](#).

Referenced by [vm_assert_interpreter_enabled\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_host_alloc\(\)](#), [vm_host_free\(\)](#), and [vm_monotonic_ns\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.27 vm_host_alloc()

```
void * vm_host_alloc (
    VM * vm,
    size_t size,
    size_t align )
```

References [VMHostServices::alloc](#), [vm](#), and [vm_host\(\)](#).

Referenced by [vm_host_calloc\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.28 vm_host_calloc()

```
void * vm_host_calloc (
    VM * vm,
    size_t n,
    size_t size )
```

References [vm](#), and [vm_host_alloc\(\)](#).

Referenced by [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.29 vm_host_free()

```
void vm_host_free (
    VM * vm,
    void * ptr )
```

References [VMHostServices::free](#), [vm](#), and [vm_host\(\)](#).

Referenced by [vm_cleanup\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.30 vm_interpret()

```
void vm_interpret (
    VM * vm,
    const char * input )
```

Interpret a string of Forth code.

Main interpretation loop that:

- Loads input into [VM](#) buffer
- Parses words
- Executes or compiles each word
- Handles numbers

Parameters

<i>vm</i>	Pointer to VM instance
<i>input</i>	String containing Forth code to interpret

7.128.2.31 vm_interpret_word()

```
void vm_interpret_word (
    VM * vm,
    const char * word_str,
    size_t len )
```

Referenced by [vm_interpret\(\)](#).

Here is the caller graph for this function:

7.128.2.32 vm_load_cell()

```
cell_t vm_load_cell (
    struct VM * vm,
    vaddr_t addr )
```

Referenced by [vm_get_base\(\)](#).

Here is the caller graph for this function:

7.128.2.33 vm_load_u8()

```
uint8_t vm_load_u8 (
    struct VM * vm,
    vaddr_t addr )
```

7.128.2.34 vm_make_immediate()

```
void vm_make_immediate (
```

```
VM * vm )
```

7.128.2.35 vm_monotonic_ns()

```
uint64_t vm_monotonic_ns (
    VM * vm )
```

References [VMHostServices::monotonic_ns](#), [vm](#), and [vm_host\(\)](#).

Referenced by [execute_colon_word\(\)](#), [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_run_cycle\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.36 vm_parse_number()

```
int vm_parse_number (
    VM * vm,
    const char * s,
    cell_t * out )
```

Parse string as number in current base.

Attempts to parse string as number using [VM](#)'s current number base. Handles optional sign prefix.

Parameters

<i>vm</i>	Pointer to VM instance
<i>s</i>	String to parse
<i>out</i>	Pointer to store parsed value

Returns

1 on success, 0 on parse failure

Referenced by [vm_interpret_word\(\)](#).

Here is the caller graph for this function:

7.128.2.37 vm_parse_word()

```
int vm_parse_word (
    VM * vm,
    char * word,
    size_t max_len )
```

Parse next word from input buffer.

Skips leading whitespace and extracts next word delimited by whitespace.

Parameters

<i>vm</i>	Pointer to VM instance
<i>word</i>	Buffer to store parsed word
<i>max_len</i>	Maximum length of word buffer

Returns

Length of parsed word or 0 if no word found

Referenced by [vm_interpret\(\)](#).

Here is the caller graph for this function:

7.128.2.38 vm_ptr()

```
uint8_t * vm_ptr (
    struct VM * vm,
    vaddr_t addr )
```

Referenced by [execute_colon_word\(\)](#), [vm_load_u8\(\)](#), and [vm_store_u8\(\)](#).
Here is the caller graph for this function:

7.128.2.39 vm_reset_hosted_fake_ns()

```
void vm_reset_hosted_fake_ns (
    VM * vm )
```

References [VM::host](#), [hosted_fake_ns](#), [hosted_services](#), [PARITY_MODE](#), and [vm](#).
Referenced by [vm_init_with_host\(\)](#).
Here is the caller graph for this function:

7.128.2.40 vm_set_base()

```
void vm_set_base (
    VM * vm,
    unsigned b )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).
Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).
Here is the call graph for this function: Here is the caller graph for this function:

7.128.2.41 vm_store_cell()

```
void vm_store_cell (
    struct VM * vm,
    vaddr_t addr,
    cell_t v )
```

Referenced by [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_set_base\(\)](#).
Here is the caller graph for this function:

7.128.2.42 vm_store_u8()

```
void vm_store_u8 (
    struct VM * vm,
    vaddr_t addr,
    uint8_t v )
```

7.128.3 Variable Documentation

7.128.3.1 hosted_fake_ns

```
uint64_t hosted_fake_ns = 0 [static]
```

Referenced by [hosted_time_ns\(\)](#), and [vm_reset_hosted_fake_ns\(\)](#).

7.128.3.2 hosted_services

```
const VMHostServices hosted_services [static]
```

Initial value:

```
= {
    .alloc = hosted_alloc,
    .free = hosted_free,
    .monotonic_ns = hosted_time_ns,
    .mutex_init = hosted_mutex_init,
    .mutex_lock = hosted_mutex_lock,
    .mutex_unlock = hosted_mutex_unlock,
    .mutex_destroy = hosted_mutex_destroy,
    .puts = hosted_puts,
    .putc = hosted_putc,
    .is_executable_ptr = hosted_xt_is_executable,
    .owns_xt_entry = hosted_xt_entry_owned,
    .panic = hosted_panic,
    .parity_mode = 0 ,
    .verbose = 0
}
```

Referenced by [vm_default_host_services\(\)](#), and [vm_reset_hosted_fake_ns\(\)](#).

7.129 src/starkernel/vm/vm_runtime.c File Reference

```
#include "../include/vm.h"
#include "../include/log.h"
#include "../include/rolling_window_of_truth.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/inference_engine.h"
#include "../include/physics_metadata.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/ssm_jacquard.h"
#include "vm_internal.h"
#include <errno.h>
#include <string.h>
#include <time.h>
```

Include dependency graph for vm_runtime.c:

Functions

- `uint32_t heartbeat_snapshot_index_load` (const volatile uint32_t *ptr)
- `void heartbeat_snapshot_index_store` (volatile uint32_t *ptr, uint32_t value)
- `void heartbeat_publish_snapshot` (VM *vm)
- `void vm_tick` (VM *vm)
Central heartbeat dispatcher for time-driven tuning operations.
- `void vm_tick_window_tuner` (VM *vm)
Loop #5: Context-aware window tuning via binary chop search.
- `void vm_tick_slope_validator` (VM *vm)
Loop #3: Heat decay slope validation via periodic measurement.
- `static void vm_tick_apply_background_decay` (VM *vm, uint64_t now_ns)
- `static void vm_heartbeat_update_l8` (VM *vm)
L8 FINAL INTEGRATION: Jacquard mode selector heartbeat update.
- `void vm_heartbeat_run_cycle` (VM *vm)
- `void vm_snapshot_read` (const VM *vm, HeartbeatSnapshot *out_snapshot)
- `void vm_tick_inference_engine` (VM *vm)
Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.

7.129.1 Function Documentation

7.129.1.1 heartbeat_publish_snapshot()

```
void heartbeat_publish_snapshot (
    VM * vm )
```

References [HeartbeatSnapshot::decay_slope_q48](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [heartbeat_snapshot_index_store\(\)](#), [HeartbeatSnapshot::hot_word_count](#), [VM::hot_word_count_at_check](#), [HeartbeatSnapshot::published_ns](#), [HeartbeatSnapshot::published_tick](#), [VM::rolling_window](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), [HeartbeatSnapshot::stale_word_count](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [HeartbeatSnapshot::total_heat](#), [VM::total_heat_at_last_check](#), [vm](#), [vm_monotonic_ns\(\)](#), and [HeartbeatSnapshot::window_width](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.129.1.2 heartbeat_snapshot_index_load()

```
uint32_t heartbeat_snapshot_index_load (
    const volatile uint32_t * ptr )
```

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), and [vm_snapshot_read\(\)](#).

Here is the caller graph for this function:

7.129.1.3 heartbeat_snapshot_index_store()

```
void heartbeat_snapshot_index_store (
    volatile uint32_t * ptr,
    uint32_t value )
```

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

Here is the caller graph for this function:

7.129.1.4 vm_heartbeat_run_cycle()

```
void vm_heartbeat_run_cycle (
    VM * vm )
```

7.129.1.5 vm_heartbeat_update_l8()

```
static void vm_heartbeat_update_l8 (
    VM * vm ) [static]
```

L8 FINAL INTEGRATION: Jacquard mode selector heartbeat update.

Collects metrics from L1-L7 physics layers and feeds them to the L8 Jacquard mode selector. L8 then chooses the optimal configuration mode based on workload characteristics and applies it to the runtime.

This is the SOLE policy engine - all adaptive decisions flow through L8.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [ssm_l8_state_t::current_mode](#), [ssm_l8_metrics_t::cv](#), [VM::decay_slope_q48](#), [InferenceOutputs::early_exited](#), [RollingWindowOfTruth::effective_window_size](#), [ssm_l8_metrics_t::entropy](#), [VM::last_inference_outputs](#), [LOG_INFO](#), [log_message\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [ssm_apply_mode\(\)](#), [VM::ssm_config](#), [ssm_l8_mode_name\(\)](#), [VM::ssm_l8_state](#), [ssm_l8_update\(\)](#), [ssm_l8_metrics_t::stability_score](#), [ssm_l8_metrics_t::temporal_decay](#), [RollingWindowOfTruth::total_executions](#), [VM::tuning_lock](#), and [vm](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.129.1.6 vm_snapshot_read()

```
void vm_snapshot_read (
    const VM * vm,
    HeartbeatSnapshot * out_snapshot )
```

7.129.1.7 vm_tick()

```
void vm_tick (
    VM * vm )
```

Central heartbeat dispatcher for time-driven tuning operations.

Aggregates all periodic optimization tasks (Loop #3 and Loop #5) into one place. Designed as plugin architecture - new tuning operations can be added as plugins.

Options for integration:

- Synchronous (now): Called from main execution loop, every N executions
- Background thread (future): Runs in separate thread, decoupled from [VM](#) execution

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the caller graph for this function:

7.129.1.8 vm_tick_apply_background_decay()

```
static void vm_tick_apply_background_decay (
    VM * vm,
    uint64_t now_ns ) [static]
```

References [VM::dict_lock](#), [HEARTBEAT_DECAY_BATCH](#), [VM::heartbeat_decay_cursor_id](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [VM::latest](#), [DictEntry::link](#), [DictEntry::physics](#), [physics_metadata_apply_linear_decay\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), [DictEntry::word_id](#), and [WORD_ID_INVALID](#).
Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.129.1.9 vm_tick_inference_engine()

```
void vm_tick_inference_engine (
    VM * vm )
```

Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.

Coordinates inference on rolling window of truth to determine:

- Optimal adaptive window width (via variance inflection detection)
- Optimal decay slope (via exponential regression on heat trajectory)

Uses ANOVA early-exit to skip full inference when variance is stable (<5% change). All math uses Q48.16 fixed-point (integer-only, no floating-point).

Replaces legacy [vm_tick_window_tuner\(\)](#) and [vm_tick_slope_validator\(\)](#).

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Referenced by [vm_tick\(\)](#).

Here is the caller graph for this function:

7.129.1.10 vm_tick_slope_validator()

```
void vm_tick_slope_validator (
    VM * vm )
```

Loop #3: Heat decay slope validation via periodic measurement.

Validates that linear decay is actually helping optimize dictionary caching. Measures stale word ratio, hot word count, and average heat distribution.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

7.129.1.11 vm_tick_window_tuner()

```
void vm_tick_window_tuner (
    VM * vm )
```

Loop #5: Context-aware window tuning via binary chop search.

Uses prefetch accuracy to guide window size adaptation. Binary search converges on optimal effective_window_↔ size for current workload.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

7.130 src/test_runner/include/test_common.h File Reference

Common test utilities and structures for StarForth test runner.

```
#include "../include/vm.h"
#include "../include/log.h"
#include "test_contracts.h"
```

Include dependency graph for test_common.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [TestCase](#)
Structure representing a single test case.
- struct [WordTestSuite](#)
Collection of test cases for a single Forth word.

Macros

Test Configuration

- #define [MAX_TEST_OUTPUT](#) 1024
Maximum length of test output buffer.
- #define [MAX_TEST_INPUT](#) 256
Maximum length of test input buffer.
- #define [MAX_TESTS_PER_WORD](#) 20
Maximum number of test cases per word.

Enumerations

- enum [TestType](#) { [TEST_NORMAL](#) = 0 , [TEST_EDGE_CASE](#) = 1 , [TEST_ERROR_CASE](#) = 2 }
Types of test cases.

Functions

Core Test Execution Functions

- [TestResult](#) [run_single_test](#) (VM *vm, const char *word_name, const [TestCase](#) *test)
Executes a single test case.
- void [run_test_suite](#) (VM *vm, const [WordTestSuite](#) *suite)
Runs all tests in a test suite.
- void [run_test_suite_m](#) (VM *vm, const [WordTestSuite](#) *suite, [WordContract](#) module_contract)
Runs all tests in a suite with an inherited module-level contract.
- void [print_module_summary](#) (const char *module_name, int pass, int fail, int skip, int error)
Prints test results_run_01_2025_12_08 summary for a module.

Test Assertion Functions

- int [assert_stack_depth](#) (VM *vm, int expected_depth)
Verifies the VM stack depth.
- int [assert_stack_top](#) (VM *vm, int expected_value)
Verifies the value at the top of the stack.
- int [assert_vm_error](#) (VM *vm, int should_have_error)
Verifies VM error state.

VM State Management Functions

- void [save_vm_state](#) (VM *vm, int *dsp, int *rsp, int *error, [vm_mode_t](#) *mode)
Saves current VM state.
- void [restore_vm_state](#) (VM *vm, int dsp, int rsp, int error, [vm_mode_t](#) mode)
Restores VM state from saved values.
- void [save_dict_state](#) (VM *vm, [DictEntry](#) **latest, [size_t](#) *here)
Saves dictionary state before test execution.
- void [restore_dict_state](#) (VM *vm, [DictEntry](#) *latest, [size_t](#) here)
Restores dictionary state after test execution.

Variables

- int `fail_fast`

Global flag to control test execution behavior.

7.130.1 Detailed Description

Common test utilities and structures for StarForth test runner.

This file contains test configuration, structures, and utility functions used throughout the StarForth test framework.

7.130.2 Macro Definition Documentation

7.130.2.1 MAX_TEST_INPUT

```
#define MAX_TEST_INPUT 256
```

Maximum length of test input buffer.

7.130.2.2 MAX_TEST_OUTPUT

```
#define MAX_TEST_OUTPUT 1024
```

Maximum length of test output buffer.

7.130.2.3 MAX_TESTS_PER_WORD

```
#define MAX_TESTS_PER_WORD 20
```

Maximum number of test cases per word.

7.130.3 Enumeration Type Documentation

7.130.3.1 TestType

```
enum TestType
```

Types of test cases.

Enumerator

TEST_NORMAL	Standard test case.
TEST_EDGE_CASE	Edge case test.
TEST_ERROR_CASE	Error condition test.

7.130.4 Function Documentation

7.130.4.1 assert_stack_depth()

```
int assert_stack_depth (
    VM * vm,
    int expected_depth )
```

Verifies the VM stack depth.

Parameters

<code>vm</code>	Pointer to the VM instance
<code>expected_depth</code>	Expected stack depth

Returns

1 if assertion passes, 0 otherwise

Verifies the [VM](#) stack depth.

Parameters

<i>vm</i>	Pointer to VM instance
<i>expected_depth</i>	Expected stack depth

Returns

1 if assertion passes, 0 if it fails

References [VM::dsp](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.130.4.2 assert_stack_top()

```
int assert_stack_top (
    VM * vm,
    int expected_value )
```

Verifies the value at the top of the stack.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>expected_value</i>	Expected value

Returns

1 if assertion passes, 0 otherwise

Verifies the value at the top of the stack.

Parameters

<i>vm</i>	Pointer to VM instance
<i>expected_value</i>	Expected value on top of stack

Returns

1 if assertion passes, 0 if it fails

References [VM::data_stack](#), [VM::dsp](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.130.4.3 assert_vm_error()

```
int assert_vm_error (
    VM * vm,
    int should_have_error )
```

Verifies [VM](#) error state.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>should_have_error</i>	Expected error state

Returns

1 if assertion passes, 0 otherwise

Verifies [VM](#) error state.

Parameters

<i>vm</i>	Pointer to VM instance
<i>should_have_error</i>	Expected error state (1 for error, 0 for no error)

Returns

1 if assertion passes, 0 if it fails

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.130.4.4 print_module_summary()

```
void print_module_summary (
    const char * module_name,
    int pass,
    int fail,
    int skip,
    int error )
```

Prints test results_run_01_2025_12_08 summary for a module.

Parameters

<i>module_name</i>	Name of the module
<i>pass</i>	Number of passed tests
<i>fail</i>	Number of failed tests
<i>skip</i>	Number of skipped tests
<i>error</i>	Number of errors

Prints test results_run_01_2025_12_08 summary for a module.

Parameters

<i>module_name</i>	Name of the test module
<i>pass</i>	Number of passed tests
<i>fail</i>	Number of failed tests
<i>skip</i>	Number of skipped tests
<i>error</i>	Number of tests with errors

References [log_message\(\)](#), and [LOG_TEST](#).

Referenced by [run_acl_words_tests\(\)](#), [run_arithmetic_words_tests\(\)](#), [run_block_words_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#), [run_dictionary_words_tests\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#), [run_io_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_return_stack_words_tests\(\)](#), [run_stack_words_tests\(\)](#), [run_starforth_words_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), and [run_vocabulary_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.130.4.5 restore_dict_state()

```
void restore_dict_state (
    VM * vm,
    DictEntry * latest,
    size_t here )
```

Restores dictionary state after test execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Latest dictionary entry to restore
<i>here</i>	HERE pointer to restore

Removes all words added during test by restoring dictionary pointers

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Latest dictionary entry to restore
<i>here</i>	HERE pointer to restore

Removes all words added during test by restoring dictionary pointers. This prevents test pollution of the dictionary and ensures each test starts with a clean slate.

References [VM::here](#), [VM::latest](#), and [vm](#).

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

Here is the caller graph for this function:

7.130.4.6 [restore_vm_state\(\)](#)

```
void restore_vm_state (
    VM * vm,
    int dsp,
    int rsp,
    int error,
    vm_mode_t mode )
```

Restores [VM](#) state from saved values.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>dsp</i>	Data stack pointer to restore
<i>rsp</i>	Return stack pointer to restore
<i>error</i>	Error state to restore
<i>mode</i>	VM mode to restore

Restores [VM](#) state from saved values.

Parameters

<i>vm</i>	Pointer to VM instance
<i>dsp</i>	Data stack pointer to restore
<i>rsp</i>	Return stack pointer to restore
<i>error</i>	Error state to restore
<i>mode</i>	VM mode to restore

References [VM::abort_requested](#), [VM::compiling_word](#), [VM::current_executing_entry](#), [VM::dsp](#), [VM::error](#), [VM::exit_colon](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::rsp](#), and [vm](#).

Referenced by [run_acl_words_tests\(\)](#), and [run_single_test\(\)](#).

Here is the caller graph for this function:

7.130.4.7 [run_single_test\(\)](#)

```
TestResult run_single_test (
```

```

    VM * vm,
    const char * word_name,
    const TestCase * test )

```

Executes a single test case.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>word_name</i>	Name of the word being tested
<i>test</i>	Test case to execute

Returns

Test execution result

Parameters

<i>vm</i>	Pointer to VM instance
<i>word_name</i>	Name of the word being tested
<i>test</i>	Pointer to test case definition

Returns

Test result (PASS, FAIL, SKIP, or ERROR)

References [VM::error](#), [TestCase::expected](#), [fail_and_exit\(\)](#), [fail_fast](#), [TestCase::implemented](#), [TestCase::input](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_TEST](#), [log_test_result\(\)](#), [TestCase::name](#), [restore_vm_state\(\)](#), [save_vm_state\(\)](#), [TestCase::should_error](#), [TEST_FAIL](#), [TEST_PASS](#), [TEST_SKIP](#), [vm](#), and [vm_interpret\(\)](#).
Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.130.4.8 run_test_suite()

```

void run_test_suite (
    VM * vm,
    const WordTestSuite * suite )

```

Runs all tests in a test suite.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>suite</i>	Test suite to execute

Runs all tests in a test suite.

Parameters

<i>vm</i>	Pointer to VM instance
<i>suite</i>	Pointer to test suite definition

References [global_test_stats](#), [TestCase::implemented](#), [log_message\(\)](#), [LOG_TEST](#), [TestCase::name](#), [restore_dict_state\(\)](#), [run_single_test\(\)](#), [save_dict_state\(\)](#), [WordTestSuite::test_count](#), [TEST_ERROR](#), [TEST_FAIL](#), [TEST_PASS](#), [TEST_SKIP](#), [WordTestSuite::tests](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), [vm](#), and [WordTestSuite::word_name](#).

Referenced by [run_block_words_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#), [run_dictionary_words_tests\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#), [run_integration_tests\(\)](#), [run_io_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_starforth_words_tests\(\)](#),

[run_stress_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), and [run_vocabulary_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.130.4.9 run_test_suite_m()

```
void run_test_suite_m (
    VM * vm,
    const WordTestSuite * suite,
    WordContract module_contract )
```

Runs all tests in a suite with an inherited module-level contract.

Contract resolution order (first non-NONE wins): test->contract > suite->suite_contract > module_contract

When CONTRACTS_ENABLED=1, each passing TEST_NORMAL test is re-run under physics perturbation to witness A4' (word_physics_transparent).

Parameters

<i>vm</i>	Pointer to the VM instance
<i>suite</i>	Test suite to execute
<i>module_contract</i>	Default contract for tests that don't specify one

References [check_physics_transparent\(\)](#), [TestCase::contract](#), [CONTRACT_PHYSICS_TRANSPARENT](#), [WordContract::flags](#), [WordContract::fuzz_rounds](#), [global_test_stats](#), [TestCase::implemented](#), [TestCase::input](#), [log_message\(\)](#), [LOG_TEST](#), [TestCase::name](#), [resolve_contract\(\)](#), [restore_dict_state\(\)](#), [run_single_test\(\)](#), [save_dict_state\(\)](#), [TestCase::should_error](#), [WordTestSuite::suite_contract](#), [WordTestSuite::test_count](#), [TEST_ERROR](#), [TEST_ERROR_CASE](#), [TEST_FAIL](#), [TEST_PASS](#), [TEST_SKIP](#), [WordTestSuite::tests](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), [TestCase::type](#), *vm*, and [WordTestSuite::word_name](#).

Referenced by [run_arithmetic_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_return_stack_words_tests\(\)](#) and [run_stack_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.130.4.10 save_dict_state()

```
void save_dict_state (
    VM * vm,
    DictEntry ** latest,
    size_t * here )
```

Saves dictionary state before test execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Pointer to store latest dictionary entry
<i>here</i>	Pointer to store HERE pointer

Saves dictionary state before test execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Pointer to store latest dictionary entry
<i>here</i>	Pointer to store HERE pointer

Captures dictionary state so tests can be cleaned up after execution

References [VM::here](#), [VM::latest](#), and [vm](#).

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

Here is the caller graph for this function:

7.130.4.11 save_vm_state()

```
void save_vm_state (
    VM * vm,
    int * dsp,
    int * rsp,
    int * error,
    vm_mode_t * mode )
```

Saves current VM state.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>dsp</i>	Pointer to store data stack pointer
<i>rsp</i>	Pointer to store return stack pointer
<i>error</i>	Pointer to store error state
<i>mode</i>	Pointer to store VM mode
<i>vm</i>	Pointer to VM instance
<i>dsp</i>	Pointer to store data stack pointer
<i>rsp</i>	Pointer to store return stack pointer
<i>error</i>	Pointer to store error state
<i>mode</i>	Pointer to store VM mode

References [VM::dsp](#), [VM::error](#), [VM::mode](#), [VM::rsp](#), and [vm](#).
Referenced by [run_acl_words_tests\(\)](#), and [run_single_test\(\)](#).
Here is the caller graph for this function:

7.130.5 Variable Documentation

7.130.5.1 fail_fast

```
int fail_fast [extern]
```

Global flag to control test execution behavior.

When set to non-zero, causes test runner to exit immediately on first failure

Referenced by [main\(\)](#), and [run_single_test\(\)](#).

7.131 src/test_runner/include/test_contracts.h File Reference

```
#include "vm.h"
#include <stdint.h>
#include <stddef.h>
```

Include dependency graph for test_contracts.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [WordContract](#)
- struct [ExecResult](#)
- struct [PhysicsSnapshot](#)

Macros

- #define [CONTRACT_NONE](#) 0u
- #define [CONTRACT_PHYSICS_TRANSPARENT](#) (1u << 0)
- #define [CONTRACT_HEARTBEAT_SAFE](#) (1u << 1)
- #define [CONTRACT_DEFAULT_FUZZ_ROUNDS](#) 4

Typedefs

- typedef uint32_t [contract_flags_t](#)

Functions

- void [capture_exec_result](#) (VM *vm, ExecResult *r)
- int [exec_results_equal](#) (const ExecResult *a, const ExecResult *b)
- void [save_physics](#) (VM *vm, PhysicsSnapshot *s)
- void [restore_physics](#) (VM *vm, const PhysicsSnapshot *s)
- void [perturb_physics](#) (VM *vm, int round)
- int [check_physics_transparent](#) (VM *vm, const char *word_name, const char *input, int should_error, int implemented, int fuzz_rounds)
- int [assert_heartbeat_safe](#) (VM *vm)

Variables

- int [global_contract_violations](#)

7.131.1 Macro Definition Documentation

7.131.1.1 CONTRACT_DEFAULT_FUZZ_ROUNDS

```
#define CONTRACT_DEFAULT_FUZZ_ROUNDS 4
```

7.131.1.2 CONTRACT_HEARTBEAT_SAFE

```
#define CONTRACT_HEARTBEAT_SAFE (1u << 1)
```

7.131.1.3 CONTRACT_NONE

```
#define CONTRACT_NONE 0u
```

7.131.1.4 CONTRACT_PHYSICS_TRANSPARENT

```
#define CONTRACT_PHYSICS_TRANSPARENT (1u << 0)
```

7.131.2 Typedef Documentation

7.131.2.1 contract_flags_t

```
typedef uint32_t contract\_flags\_t
```

7.131.3 Function Documentation

7.131.3.1 assert_heartbeat_safe()

```
int assert_heartbeat_safe (
    VM * vm )
```

References [capture_exec_result\(\)](#), [ExecResult::dsp](#), [exec_results_equal\(\)](#), [global_contract_violations](#), [LOG_ERROR](#), [log_message\(\)](#), [ExecResult::rsp](#), [vm](#), and [vm_tick\(\)](#).

Here is the call graph for this function:

7.131.3.2 capture_exec_result()

```
void capture_exec_result (
    VM * vm,
    ExecResult * r )
```

References [VM::data_stack](#), [VM::dsp](#), [ExecResult::dsp](#), [VM::error](#), [ExecResult::error](#), [VM::rsp](#), [ExecResult::rsp](#), [ExecResult::stack](#), [STACK_SIZE](#), and [vm](#).

Referenced by [assert_heartbeat_safe\(\)](#), and [check_physics_transparent\(\)](#).
Here is the caller graph for this function:

7.131.3.3 check_physics_transparent()

```
int check_physics_transparent (
    VM * vm,
    const char * word_name,
    const char * input,
    int should_error,
    int implemented,
    int fuzz_rounds )
```

References [VM::abort_requested](#), [capture_exec_result\(\)](#), [CONTRACT_DEFAULT_FUZZ_ROUNDS](#), [VM::dsp](#), [ExecResult::dsp](#), [VM::error](#), [exec_results_equal\(\)](#), [VM::exit_colon](#), [global_contract_violations](#), [LOG_ERROR](#), [log_message\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), [VM::rsp](#), [save_physics\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [run_test_suite_m\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.131.3.4 exec_results_equal()

```
int exec_results_equal (
    const ExecResult * a,
    const ExecResult * b )
```

References [ExecResult::dsp](#), [ExecResult::error](#), [ExecResult::rsp](#), and [ExecResult::stack](#).

Referenced by [assert_heartbeat_safe\(\)](#), and [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.131.3.5 perturb_physics()

```
void perturb_physics (
    VM * vm,
    int round )
```

References [contract_rand\(\)](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_rng_seed\(\)](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.131.3.6 restore_physics()

```
void restore_physics (
    VM * vm,
    const PhysicsSnapshot * s )
```

References [VM::decay_slope_q48](#), [PhysicsSnapshot::decay_slope_q48](#), [PhysicsSnapshot::eff_window_size](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [PhysicsSnapshot::heat_25th](#), [PhysicsSnapshot::heat_50th](#), [PhysicsSnapshot::heat_75th](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PhysicsSnapshot::prefetch_attempts](#), [VM::rolling_window](#), [HeartbeatState::tick_target_ns](#), [PhysicsSnapshot::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [PhysicsSnapshot::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.131.3.7 save_physics()

```
void save_physics (
    VM * vm,
    PhysicsSnapshot * s )
```

References [VM::decay_slope_q48](#), [PhysicsSnapshot::decay_slope_q48](#), [PhysicsSnapshot::eff_window_size](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [PhysicsSnapshot::heat_25th](#), [PhysicsSnapshot::heat_50th](#), [PhysicsSnapshot::heat_75th](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PhysicsSnapshot::prefetch_attempts](#), [VM::rolling_window](#), [HeartbeatState::tick_target_ns](#), [PhysicsSnapshot::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [PhysicsSnapshot::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.131.4 Variable Documentation

7.131.4.1 global_contract_violations

```
int global_contract_violations [extern]
```

Referenced by [assert_heartbeat_safe\(\)](#), [check_physics_transparent\(\)](#), and [run_all_tests\(\)](#).

7.132 src/test_runner/include/test_runner.h File Reference

```
#include "../.../include/vm.h"
#include <stdint.h>
```

Include dependency graph for test_runner.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [TestStats](#)
Structure holding test execution statistics.
- struct [TestModule](#)
Structure representing a test module.

Functions

- void [run_stack_words_tests](#) ([VM *vm](#))
Run tests for stack manipulation words.
- void [run_return_stack_words_tests](#) ([VM *vm](#))
Executes all test suites for return stack manipulation words.
- void [run_memory_words_tests](#) ([VM *vm](#))
Executes the test suite for memory manipulation words.
- void [run_arithmetic_words_tests](#) ([VM *vm](#))
Executes all arithmetic word test suites.
- void [run_logical_words_tests](#) ([VM *vm](#))
Executes all logical words test suites.
- void [run_mixed_arithmetic_words_tests](#) ([VM *vm](#))
- void [run_double_words_tests](#) ([VM *vm](#))
Executes all test suites for double number words.
- void [run_dictionary_words_tests](#) ([VM *vm](#))
Executes all dictionary word test suites.
- void [run_io_words_tests](#) ([VM *vm](#))
Executes all I/O words test suites.
- void [run_format_words_tests](#) ([VM *vm](#))
Executes all format words test suites.
- void [run_string_words_tests](#) ([VM *vm](#))
Executes all string manipulation word tests.
- void [run_dictionary_manipulation_words_tests](#) ([VM *vm](#))
Executes all dictionary manipulation words test suites.

- void [run_vocabulary_words_tests](#) (VM *vm)
Executes all vocabulary word test suites.
- void [run_block_words_tests](#) (VM *vm)
Executes all block words test suites.
- void [run_system_words_tests](#) (VM *vm)
Executes all system word tests.
- void [run_defining_words_tests](#) (VM *vm)
Executes all test suites for Forth defining words.
- void [run_control_words_tests](#) (VM *vm)
Executes all control words test suites.
- void [run_starforth_words_tests](#) (VM *vm)
Executes all StarForth words test suites.
- void [run_acl_words_tests](#) (VM *vm)
- void [run_mama_forth_words_tests](#) (VM *vm)
Run tests for Mama FORTH vocabulary (capsule system M7.1)
- void [enable_benchmark_mode](#) (int iterations)
Enable benchmark mode for performance testing.
- void [run_stress_tests](#) (VM *vm)
Execute stress tests.
- void [run_integration_tests](#) (VM *vm)
Execute integration tests.
- void [run_all_tests](#) (VM *vm)
Run all available tests.
- void [run_module_tests](#) (VM *vm, const char *module_name)
Run tests for a specific module.
- void [run_word_tests](#) (VM *vm, const char *word_name)
Run tests for a specific word.
- void [run_break_me_tests](#) (VM *vm)
Run ultra-comprehensive break-me diagnostic suite.

Variables

- [TestStats global_test_stats](#)
Global statistics for test execution.

7.132.1 Function Documentation

7.132.1.1 [enable_benchmark_mode\(\)](#)

```
void enable_benchmark_mode (
    int iterations )
```

Enable benchmark mode for performance testing.

Parameters

<i>iterations</i>	Number of iterations for benchmark tests
-------------------	--

Enable benchmark mode for performance testing.

Parameters

<i>iterations</i>	Number of times each test should be repeated
-------------------	--

References [benchmark_iterations](#), [benchmark_mode](#), [LOG_INFO](#), and [log_message\(\)](#).

Referenced by [run_benchmark_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.132.1.2 run_acl_words_tests()

```
void run_acl_words_tests (
    VM * vm )
```

References [VM::emergency_console](#), [LOG_INFO](#), [log_message\(\)](#), [print_module_summary\(\)](#), [restore_vm_state\(\)](#), [save_vm_state\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_inherit\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), [tests_failed](#), [tests_passed](#), and [vm](#).

Here is the call graph for this function:

7.132.1.3 run_all_tests()

```
void run_all_tests (
    VM * vm )
```

Run all available tests.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Run all available tests.

Executes all registered test modules sequentially, collecting statistics and/or benchmark data depending on the current mode.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [global_contract_violations](#), [global_test_stats](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), [test_modules](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), and [TestStats::total_tests](#).

Referenced by [main\(\)](#), [run_benchmark_mode\(\)](#), [run_break_me_tests\(\)](#), and [run_doe_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.132.1.4 run_arithmetic_words_tests()

```
void run_arithmetic_words_tests (
    VM * vm )
```

Executes all arithmetic word test suites.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all test cases for arithmetic operations including basic operations (+, -, *, /), modulo operations (MOD, /MOD), and numeric functions (ABS, NEGATE, MIN, MAX)

References [arithmetic_word_suites](#), [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.5 run_block_words_tests()

```
void run_block_words_tests (
    VM * vm )
```

Executes all block words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all test cases for block manipulation words, including block access, buffer management, and block I/O operations. Results are logged and summarized after completion.

References [block_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.6 run_break_me_tests()

```
void run_break_me_tests (
    VM * vm )
```

Run ultra-comprehensive break-me diagnostic suite.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Executes the most exhaustive test battery ever designed for StarForth, generates a detailed markdown report in docs/BREAK_ME_REPORT.md, and includes a surprise easter egg at the end.

Run ultra-comprehensive break-me diagnostic suite.

References [finalize_report\(\)](#), [global_test_stats](#), [init_report\(\)](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [run_all_tests\(\)](#), [TestStats::total_tests](#), [total_tests_run](#), and [vm](#).

Referenced by [run_optional_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.132.1.7 run_control_words_tests()

```
void run_control_words_tests (
    VM * vm )
```

Executes all control words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all defined test cases for control words, testing conditional branches and loop constructs

References [control_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.8 run_defining_words_tests()

```
void run_defining_words_tests (
    VM * vm )
```

Executes all test suites for Forth defining words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through each test suite in [defining_word_suites](#) array, executing individual test cases and logging results ↩
run_01_2025_12_08

References [defining_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.9 run_dictionary_manipulation_words_tests()

```
void run_dictionary_manipulation_words_tests (
    VM * vm )
```

Executes all dictionary manipulation words test suites.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all test cases for CREATE, FORGET, IMMEDIATE, FIND, DEFINITIONS, SMUDGE, LATEST, >BODY, and HIDDEN words

References [dict_manip_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.10 run_dictionary_words_tests()

```
void run_dictionary_words_tests (
    VM * vm )
```

Executes all dictionary word test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

This function runs through all dictionary word test suites, including tests for:

- HERE (dictionary pointer)
- ALLOT (memory allocation)
- Comma operations (, C, 2,)
- PAD (scratch pad area)
- Stack pointer operations (SP@ SP!)
- LATEST (most recent dictionary entry)
- Compilation sequences
- Stack management operations

References [dictionary_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.11 run_double_words_tests()

```
void run_double_words_tests (
    VM * vm )
```

Executes all test suites for double number words.

This function runs comprehensive tests for all double number word operations, including stack manipulation (2DROP, 2DUP, etc.), arithmetic operations (D+, D-, etc.), and comparison operations (D=, D<, etc.).

Parameters

<i>vm</i>	Pointer to the virtual machine instance where tests will be executed
-----------	--

References [double_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#),

[vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.12 run_format_words_tests()

```
void run_format_words_tests (
    VM * vm )
```

Executes all format words test suites.

Parameters

vm	Pointer to the Forth virtual machine instance
--------------------	---

Runs through all test cases for number base manipulation and formatting words

References [format_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.13 run_integration_tests()

```
void run_integration_tests (
    VM * vm )
```

Execute integration tests.

Parameters

vm	Pointer to the VM instance
--------------------	--

Execute integration tests.

Parameters

vm	Pointer to the Forth virtual machine instance
--------------------	---

This function runs through all defined integration test suites, including:

- Complete Programs suite (prime checker, array operations, etc)
- Data Structures suite (linked list, min/max tracker)
- Algorithms suite (sorting, GCD)

References [integration_suites](#), [LOG_INFO](#), [log_message\(\)](#), [run_test_suite\(\)](#), and [vm](#).

Here is the call graph for this function:

7.132.1.14 run_io_words_tests()

```
void run_io_words_tests (
    VM * vm )
```

Executes all I/O words test suites.

Parameters

vm	Pointer to the FORTH virtual machine instance
--------------------	---

Runs through all defined test suites for I/O words, logging results `_run_01_2025_12_08` and providing a summary of the test execution

References [io_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.15 run_logical_words_tests()

```
void run_logical_words_tests (
    VM * vm )
```

Executes all logical words test suites.

This function runs comprehensive tests for all FORTH logical and comparison operators, including stack underflow checks and edge cases.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [logical_word_suites](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.16 run_mama_forth_words_tests()

```
void run_mama_forth_words_tests (
    VM * vm )
```

Run tests for Mama FORTH vocabulary (capsule system M7.1)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

In hosted builds, all tests are skipped (Mama words are kernel-only). In kernel builds, tests validate capsule enumeration and [VM](#) birth.

Run tests for Mama FORTH vocabulary (capsule system M7.1)

In hosted builds, all tests will be skipped since `MAMA_IMPLEMENTED=0`. In kernel builds, tests will execute against the capsule system.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [LOG_INFO](#), [log_message\(\)](#), [mama_word_suites](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.17 run_memory_words_tests()

```
void run_memory_words_tests (
    VM * vm )
```

Executes the test suite for memory manipulation words.

Runs through a series of test cases for Forth memory operations including store, fetch, allot, and other memory-related words

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [memory_word_suites](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.18 run_mixed_arithmetic_words_tests()

```
void run_mixed_arithmetic_words_tests (
```

```
VM * vm )
```

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [mixed_arithmetic_word_suites](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.19 run_module_tests()

```
void run_module_tests (
    VM * vm,
    const char * module_name )
```

Run tests for a specific module.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>module_name</i>	Name of the module to test
<i>vm</i>	Pointer to the Forth virtual machine instance
<i>module_name</i>	Name of the module to test

References [benchmark_iterations](#), [benchmark_mode](#), [get_time_ns\(\)](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), [run_module_tests\(\)](#), [TestModule::run_module_tests](#), [test_modules](#), and [vm](#).

Referenced by [run_module_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.132.1.20 run_return_stack_words_tests()

```
void run_return_stack_words_tests (
    VM * vm )
```

Executes all test suites for return stack manipulation words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all defined test cases for >R, R>, and R@ words and reports results_run_01_2025_12_08 through the logging system

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [return_stack_word_suites](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.21 run_stack_words_tests()

```
void run_stack_words_tests (
    VM * vm )
```

Run tests for stack manipulation words.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Run tests for stack manipulation words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all defined test cases for stack manipulation words and reports results_run_01_2025_12_08 through

the logging system

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [stack_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.22 run_starforth_words_tests()

```
void run_starforth_words_tests (
    VM * vm )
```

Executes all StarForth words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all test cases for StarForth-specific implementation words, including entropy tracking, comments, and initialization.

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [starforth_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.23 run_stress_tests()

```
void run_stress_tests (
    VM * vm )
```

Execute stress tests.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Execute stress tests.

Runs an extensive set of stress tests to verify the [VM](#)'s behavior under extreme conditions. Tests include:

- Stack operations and boundaries
- Deep nesting and control flow
- Numeric boundaries and overflow
- Dictionary pressure
- Memory allocation patterns
- Error recovery
- Mixed complexity scenarios

All tests are FORTH-79 compliant.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [global_test_stats](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [run_test_suite\(\)](#), [stress_suites](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), and [vm](#).

Here is the call graph for this function:

7.132.1.24 run_string_words_tests()

```
void run_string_words_tests (
    VM * vm )
```

Executes all string manipulation word tests.

Runs test suites for FORTH string operations including COUNT, -TRAILING, CMOVE, CMOVE>, COMPARE, SEARCH, SCAN, SKIP, and BLANK

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [string_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.25 run_system_words_tests()

```
void run_system_words_tests (
    VM * vm )
```

Executes all system word tests.

Runs test suites for FORTH system control operations including QUIT, ABORT, ABORT", BYE, COLD, and WARM. Some tests are marked as unimplemented due to their system-level effects

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [LOG_WARN](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [system_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.26 run_vocabulary_words_tests()

```
void run_vocabulary_words_tests (
    VM * vm )
```

Executes all vocabulary word test suites.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all vocabulary-related test suites, executing each test case and logging the results_run_01_2025_↵
12_08. Tests vocabulary creation, selection, and manipulation.

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), [vocabulary_word_suites](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.132.1.27 run_word_tests()

```
void run_word_tests (
    VM * vm,
    const char * word_name )
```

Run tests for a specific word.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>word_name</i>	Name of the word to test

Run tests for a specific word.
Searches all modules for tests related to the specified word and executes them.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>word_name</i>	Name of the Forth word to test

References [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), and [test_modules](#).
Here is the call graph for this function:

7.132.2 Variable Documentation

7.132.2.1 global_test_stats

`TestStats global_test_stats [extern]`

Global statistics for test execution.

Referenced by [run_all_tests\(\)](#), [run_break_me_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

7.133 src/test_runner/modules/acl_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
#include "../../include/vm.h"
#include "../../word_source/include/acl_words.h"
#include <stdio.h>
```

Include dependency graph for `acl_words_test.c`:

Macros

- `#define` [ACL_ASSERT](#)(cond, msg)

Functions

- static void [test_acl_pin_immutable](#) (VM *vm)
- static void [test_acl_inherit](#) (VM *vm)
- static void [test_emergency_console_bypass](#) (VM *vm)
- static void [test_acl_deny_aborts](#) (VM *vm)
- static void [test_acl_ttl_hot_path](#) (VM *vm)
- static void [test_acl_strict_mode](#) (VM *vm)
- static void [test_acl_adaptive_ttl](#) (VM *vm)
- static void [test_acl_pin_blocks_shadow](#) (VM *vm)
- void [run_acl_words_tests](#) (VM *vm)

Variables

- static int [tests_passed](#) = 0
- static int [tests_failed](#) = 0

7.133.1 Macro Definition Documentation

7.133.1.1 ACL_ASSERT

```
#define ACL_ASSERT(  
    cond,  
    msg )
```

Value:

```

do { \
  if (cond) { \
    tests_passed++; \
    log_message(LOG_DEBUG, " PASS: %s", msg); \
  } else { \
    tests_failed++; \
    log_message(LOG_ERROR, " FAIL: %s", msg); \
  } \
} while (0)

```

7.133.2 Function Documentation

7.133.2.1 run_acl_words_tests()

```

void run_acl_words_tests (
    VM * vm )

```

References [VM::emergency_console](#), [LOG_INFO](#), [log_message\(\)](#), [print_module_summary\(\)](#), [restore_vm_state\(\)](#), [save_vm_state\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_inherit\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), [tests_failed](#), [tests_passed](#), and [vm](#).

Here is the call graph for this function:

7.133.2.2 test_acl_adaptive_ttl()

```

static void test_acl_adaptive_ttl (
    VM * vm ) [static]

```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_pinned](#), [VM::dsp](#), [VM::error](#), [DictEntry::execution_heat](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [tests_failed](#), [tests_passed](#), [vm](#), [vm_create_word\(\)](#), [vm_find_word\(\)](#), [vm_interpret\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.3 test_acl_deny_aborts()

```

static void test_acl_deny_aborts (
    VM * vm ) [static]

```

References [VM::abort_requested](#), [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [VM::dsp](#), [VM::emergency_console](#), [VM::error](#), [tests_failed](#), [vm](#), [vm_find_word\(\)](#), and [vm_interpret\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.4 test_acl_inherit()

```

static void test_acl_inherit (
    VM * vm ) [static]

```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [acl_inherit_entry\(\)](#), [DictEntry::acl_mode](#), [ACL_MODE_STRICT](#), [ACL_MODE_TTL](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [tests_failed](#), [vm](#), and [vm_create_word\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.5 test_acl_pin_blocks_shadow()

```

static void test_acl_pin_blocks_shadow (
    VM * vm ) [static]

```

References [ACL_ASSERT](#), [DictEntry::acl_pinned](#), [VM::error](#), [tests_failed](#), [vm](#), [vm_create_word\(\)](#), and [vm_find_word\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.6 test_acl_pin_immutable()

```
static void test_acl_pin_immutable (
    VM * vm ) [static]
```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_mode](#), [ACL_MODE_TTL](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [VM::error](#), [tests_failed](#), [vm](#), [vm_create_word\(\)](#), [vm_interpret\(\)](#), and [vm_push\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.7 test_acl_strict_mode()

```
static void test_acl_strict_mode (
    VM * vm ) [static]
```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_mode](#), [ACL_MODE_STRICT](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [VM::dsp](#), [VM::error](#), [LOG_INFO](#), [log_message\(\)](#), [tests_failed](#), [tests_passed](#), [vm](#), [vm_find_word\(\)](#), [vm_interpret\(\)](#), and [vm_push\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.8 test_acl_ttl_hot_path()

```
static void test_acl_ttl_hot_path (
    VM * vm ) [static]
```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_mode](#), [ACL_MODE_TTL](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [VM::emergency_console](#), [VM::error](#), [tests_failed](#), [vm](#), [vm_find_word\(\)](#), and [vm_interpret\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.2.9 test_emergency_console_bypass()

```
static void test_emergency_console_bypass (
    VM * vm ) [static]
```

References [DictEntry::acl_allow](#), [ACL_ASSERT](#), [DictEntry::acl_pinned](#), [DictEntry::acl_ttl](#), [VM::dsp](#), [VM::emergency_console](#), [VM::error](#), [tests_failed](#), [vm](#), [vm_find_word\(\)](#), and [vm_interpret\(\)](#).

Referenced by [run_acl_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.133.3 Variable Documentation

7.133.3.1 tests_failed

```
int tests_failed = 0 [static]
```

Referenced by [run_acl_words_tests\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_inherit\(\)](#), [test_acl_pin_blocks_shadow\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), and [test_emergency_console_bypass\(\)](#).

7.133.3.2 tests_passed

```
int tests_passed = 0 [static]
```

Referenced by [run_acl_words_tests\(\)](#), [test_acl_adaptive_ttl\(\)](#), and [test_acl_strict_mode\(\)](#).

7.134 src/test_runner/modules/arithmetic_words_test.c File Reference

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for arithmetic_words_test.c:

Functions

- void [run_arithmetic_words_tests](#) (VM *vm)
Executes all arithmetic word test suites.

Variables

- static [WordTestSuite arithmetic_word_suites](#) []

7.134.1 Function Documentation

7.134.1.1 run_arithmetic_words_tests()

```
void run_arithmetic_words_tests (
    VM * vm )
```

Executes all arithmetic word test suites.

Parameters

<code>vm</code>	Pointer to the Forth virtual machine instance
-----------------	---

Runs through all test cases for arithmetic operations including basic operations (+, -, *, /), modulo operations (MOD, /MOD), and numeric functions (ABS, NEGATE, MIN, MAX)

References [arithmetic_word_suites](#), [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.134.2 Variable Documentation

7.134.2.1 arithmetic_word_suites

```
WordTestSuite arithmetic_word_suites[] [static]
```

Referenced by [run_arithmetic_words_tests\(\)](#).

7.135 src/test_runner/modules/block_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for block_words_test.c:
```

Functions

- void [run_block_words_tests](#) (VM *vm)
Executes all block words test suites.

Variables

- static [WordTestSuite block_word_suites](#) []
Test suites for FORTH block words (Module 16)

7.135.1 Function Documentation

7.135.1.1 run_block_words_tests()

```
void run_block_words_tests (
    VM * vm )
```

Executes all block words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all test cases for block manipulation words, including block access, buffer management, and block I/O operations. Results are logged and summarized after completion.

References [block_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.135.2 Variable Documentation

7.135.2.1 block_word_suites

[WordTestSuite](#) `block_word_suites[ ]` `[static]`

Test suites for FORTH block words (Module 16)

Contains test cases for the following FORTH words:

- BLOCK: Block access
- BUFFER: Buffer management
- UPDATE: Block modification marking
- SAVE-BUFFERS: Buffer flushing
- EMPTY-BUFFERS: Buffer invalidation
- FLUSH: Buffer saving and invalidation
- LOAD: Block interpretation
- LIST: Block display
- SCR: Current block number
- THRU: Load block range
- -->: Continue to next block

Referenced by [run_block_words_tests\(\)](#).

7.136 src/test_runner/modules/break_me_tests.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include "test_runner.h"
#include "../../include/log.h"
#include "../../include/vm.h"
Include dependency graph for break_me_tests.c:
```

Functions

- static int [init_report](#) (void)
Initialize markdown report file.
- static void [finalize_report](#) (VM *vm)
Finalize report with system info and easter egg.
- void [run_break_me_tests](#) (VM *vm)
Main entry point for -break-me mode.

Variables

- static FILE * [report_file](#) = NULL
- static time_t [test_start_time](#)
- static time_t [test_end_time](#)
- static int [total_tests_run](#) = 0

7.136.1 Function Documentation

7.136.1.1 finalize_report()

```
static void finalize_report (
    VM * vm ) [static]
```

Finalize report with system info and easter egg.

References [report_file](#), [STACK_SIZE](#), [test_end_time](#), [test_start_time](#), [total_tests_run](#), and [VM_MEMORY_SIZE](#).

Referenced by [run_break_me_tests\(\)](#).

Here is the caller graph for this function:

7.136.1.2 init_report()

```
static int init_report (
    void ) [static]
```

Initialize markdown report file.

References [LOG_ERROR](#), [log_message\(\)](#), [report_file](#), and [test_start_time](#).

Referenced by [run_break_me_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.136.1.3 run_break_me_tests()

```
void run_break_me_tests (
    VM * vm )
```

Main entry point for `-break-me` mode.

Run ultra-comprehensive break-me diagnostic suite.

References [finalize_report\(\)](#), [global_test_stats](#), [init_report\(\)](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [run_all_tests\(\)](#), [TestStats::total_tests](#), [total_tests_run](#), and [vm](#).

Referenced by [run_optional_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.136.2 Variable Documentation

7.136.2.1 report_file

```
FILE* report_file = NULL [static]
```

Referenced by [finalize_report\(\)](#), and [init_report\(\)](#).

7.136.2.2 test_end_time

```
time_t test_end_time [static]
```

Referenced by [finalize_report\(\)](#).

7.136.2.3 test_start_time

```
time_t test_start_time [static]
```

Referenced by [finalize_report\(\)](#), and [init_report\(\)](#).

7.136.2.4 total_tests_run

```
int total_tests_run = 0 [static]
```

Referenced by [finalize_report\(\)](#), and [run_break_me_tests\(\)](#).

7.137 src/test_runner/modules/control_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for control_words_test.c:
```

Functions

- void [run_control_words_tests](#) (VM *vm)
Executes all control words test suites.

Variables

- static [WordTestSuite control_word_suites](#) []
Test suites for FORTH control words.

7.137.1 Function Documentation

7.137.1.1 run_control_words_tests()

```
void run_control_words_tests (
    VM * vm )
```

Executes all control words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all defined test cases for control words, testing conditional branches and loop constructs
References [control_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).
Here is the call graph for this function:

7.137.2 Variable Documentation

7.137.2.1 control_word_suites

```
WordTestSuite control_word_suites[] [static]
```

Test suites for FORTH control words.

Contains test cases for all control flow words in Module 12, including conditional branches and loop structures
Referenced by [run_control_words_tests\(\)](#).

7.138 src/test_runner/modules/defining_words_tests.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for defining_words_tests.c:
```

Functions

- void [run_defining_words_tests](#) (VM *vm)
Executes all test suites for Forth defining words.

Variables

- static [WordTestSuite defining_word_suites](#) []
Test suites for Forth defining words (Module 13)

7.138.1 Function Documentation

7.138.1.1 run_defining_words_tests()

```
void run_defining_words_tests (
    VM * vm )
```

Executes all test suites for Forth defining words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through each test suite in defining_word_suites array, executing individual test cases and logging results ↵
run_01_2025_12_08

References [defining_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.138.2 Variable Documentation

7.138.2.1 defining_word_suites

```
WordTestSuite defining_word_suites[] [static]
```

Test suites for Forth defining words (Module 13)

Contains test cases for core defining words including: :, ;, CONSTANT, VARIABLE, CREATE, DOES>, [, and]

Referenced by [run_defining_words_tests\(\)](#).

7.139 src/test_runner/modules/dictionary_manipulation_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
```

Include dependency graph for dictionary_manipulation_words_test.c:

Functions

- void [run_dictionary_manipulation_words_tests](#) (VM *vm)
Executes all dictionary manipulation words test suites.

Variables

- static [WordTestSuite dict_manip_word_suites](#) []

7.139.1 Function Documentation

7.139.1.1 run_dictionary_manipulation_words_tests()

```
void run_dictionary_manipulation_words_tests (
    VM * vm )
```

Executes all dictionary manipulation words test suites.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all test cases for CREATE, FORGET, IMMEDIATE, FIND, DEFINITIONS, SMUDGE, LATEST, >BODY, and HIDDEN words

References [dict_manip_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#),

[run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.139.2 Variable Documentation

7.139.2.1 dict_manip_word_suites

[WordTestSuite](#) dict_manip_word_suites[] [static]

Referenced by [run_dictionary_manipulation_words_tests\(\)](#).

7.140 src/test_runner/modules/dictionary_words_test.c File Reference

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for dictionary_words_test.c:

Functions

- void [run_dictionary_words_tests](#) (VM *vm)
Executes all dictionary word test suites.

Variables

- static [WordTestSuite](#) dictionary_word_suites []

7.140.1 Function Documentation

7.140.1.1 run_dictionary_words_tests()

```
void run_dictionary_words_tests (
    VM * vm )
```

Executes all dictionary word test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

This function runs through all dictionary word test suites, including tests for:

- HERE (dictionary pointer)
- ALLOT (memory allocation)
- Comma operations (, C, 2,)
- PAD (scratch pad area)
- Stack pointer operations (SP@ SP!)
- LATEST (most recent dictionary entry)
- Compilation sequences
- Stack management operations

References [dictionary_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.140.2 Variable Documentation

7.140.2.1 dictionary_word_suites

`WordTestSuite` dictionary_word_suites[] [static]

Referenced by [run_dictionary_words_tests\(\)](#).

7.141 src/test_runner/modules/double_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for double_words_test.c:
```

Functions

- void [run_double_words_tests](#) (VM *vm)
Executes all test suites for double number words.

Variables

- static `WordTestSuite` double_word_suites []

7.141.1 Function Documentation

7.141.1.1 run_double_words_tests()

```
void run_double_words_tests (
    VM * vm )
```

Executes all test suites for double number words.

This function runs comprehensive tests for all double number word operations, including stack manipulation (2DROP, 2DUP, etc.), arithmetic operations (D+, D-, etc.), and comparison operations (D=, D<, etc.).

Parameters

<code>vm</code>	Pointer to the virtual machine instance where tests will be executed
-----------------	--

References [double_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.141.2 Variable Documentation

7.141.2.1 double_word_suites

`WordTestSuite` double_word_suites[] [static]

Referenced by [run_double_words_tests\(\)](#).

7.142 src/test_runner/modules/format_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for format_words_test.c:
```

Functions

- void [run_format_words_tests](#) (VM *vm)
Executes all format words test suites.

Variables

- static [WordTestSuite format_word_suites](#) []

Test suites for format words (Module 10) Contains test cases for number base conversion and formatting operations.

7.142.1 Function Documentation**7.142.1.1 run_format_words_tests()**

```
void run_format_words_tests (
    VM * vm )
```

Executes all format words test suites.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all test cases for number base manipulation and formatting words

References [format_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.142.2 Variable Documentation**7.142.2.1 format_word_suites**

```
WordTestSuite format_word_suites[] [static]
```

Test suites for format words (Module 10) Contains test cases for number base conversion and formatting operations.

Referenced by [run_format_words_tests\(\)](#).

7.143 src/test_runner/modules/integration_tests.c File Reference

Full program integration tests for StarForth.

```
#include "test_common.h"
```

```
#include "test_runner.h"
```

Include dependency graph for integration_tests.c:

Functions

- void [run_integration_tests](#) (VM *vm)

Executes all integration test suites for StarForth.

Variables

- static [WordTestSuite integration_suites](#) []

7.143.1 Detailed Description

Full program integration tests for StarForth.

This file contains integration tests that verify complete Forth programs as regression tests. It includes test suites for complete programs, data structures, and algorithms implementations.

7.143.2 Function Documentation**7.143.2.1 run_integration_tests()**

```
void run_integration_tests (
    VM * vm )
```

Executes all integration test suites for StarForth.

Execute integration tests.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

This function runs through all defined integration test suites, including:

- Complete Programs suite (prime checker, array operations, etc)
- Data Structures suite (linked list, min/max tracker)
- Algorithms suite (sorting, GCD)

References [integration_suites](#), [LOG_INFO](#), [log_message\(\)](#), [run_test_suite\(\)](#), and [vm](#).

Here is the call graph for this function:

7.143.3 Variable Documentation**7.143.3.1 integration_suites**

[WordTestSuite](#) integration_suites[] [static]

Referenced by [run_integration_tests\(\)](#).

7.144 src/test_runner/modules/io_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
```

Include dependency graph for io_words_test.c:

Functions

- void [run_io_words_tests](#) (VM *vm)
Executes all I/O words test suites.

Variables

- static [WordTestSuite](#) io_word_suites []
Test suites for I/O words (Module 9)

7.144.1 Function Documentation**7.144.1.1 run_io_words_tests()**

```
void run_io_words_tests (
    VM * vm )
```

Executes all I/O words test suites.

Parameters

<i>vm</i>	Pointer to the FORTH virtual machine instance
-----------	---

Runs through all defined test suites for I/O words, logging results_run_01_2025_12_08 and providing a summary of the test execution

References [io_word_suites](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.144.2 Variable Documentation

7.144.2.1 io_word_suites

`WordTestSuite io_word_suites[] [static]`

Test suites for I/O words (Module 9)

Contains comprehensive test cases for all I/O related FORTH words including output formatting, character I/O, and base conversion operations

Referenced by [run_io_words_tests\(\)](#).

7.145 src/test_runner/modules/logical_words_test.c File Reference

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for `logical_words_test.c`:

Functions

- void [run_logical_words_tests](#) (VM *vm)

Executes all logical words test suites.

Variables

- static `WordTestSuite logical_word_suites []`

7.145.1 Function Documentation

7.145.1.1 run_logical_words_tests()

```
void run_logical_words_tests (
    VM * vm )
```

Executes all logical words test suites.

This function runs comprehensive tests for all FORTH logical and comparison operators, including stack underflow checks and edge cases.

Parameters

<code>vm</code>	Pointer to the FORTH virtual machine instance
-----------------	---

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [logical_word_suites](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), `vm`, and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.145.2 Variable Documentation

7.145.2.1 logical_word_suites

`WordTestSuite logical_word_suites[] [static]`

Referenced by [run_logical_words_tests\(\)](#).

7.146 src/test_runner/modules/mama_forth_words_test.c File Reference

Test module for Mama FORTH vocabulary (M7.1 Capsule System)

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for `mama_forth_words_test.c`:

Macros

- `#define MAMA_IMPLEMENTED 0`

Functions

- void `run_mama_forth_words_tests (VM *vm)`
Executes all Mama FORTH vocabulary test suites.

Variables

- static `WordTestSuite mama_word_suites []`

7.146.1 Detailed Description

Test module for Mama FORTH vocabulary (M7.1 Capsule System)

Tests for the MAMA vocabulary words. These words are kernel-only (**STARKERNEL**), so in hosted builds all tests are marked as not implemented and will be skipped.

In kernel builds, these tests validate:

- Capsule directory enumeration
- Descriptor field access
- `VM` birth protocol
- Experiment execution

7.146.2 Macro Definition Documentation

7.146.2.1 MAMA_IMPLEMENTED

```
#define MAMA_IMPLEMENTED 0
```

7.146.3 Function Documentation

7.146.3.1 run_mama_forth_words_tests()

```
void run_mama_forth_words_tests (
    VM * vm )
```

Executes all Mama FORTH vocabulary test suites.

Run tests for Mama FORTH vocabulary (capsule system M7.1)

In hosted builds, all tests will be skipped since `MAMA_IMPLEMENTED=0`. In kernel builds, tests will execute against the capsule system.

Parameters

<code>vm</code>	Pointer to the Forth virtual machine instance
-----------------	---

References `LOG_INFO`, `log_message()`, `mama_word_suites`, `print_module_summary()`, `run_test_suite()`, `vm`, and `WordTestSuite::word_name`.

Here is the call graph for this function:

7.146.4 Variable Documentation

7.146.4.1 mama_word_suites

```
WordTestSuite mama_word_suites[] [static]
```

Referenced by `run_mama_forth_words_tests()`.

7.147 src/test_runner/modules/memory_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for memory_words_test.c:
```

Functions

- void [run_memory_words_tests](#) (VM *vm)
Executes the test suite for memory manipulation words.

Variables

- static [WordTestSuite](#) [memory_word_suites](#) []

7.147.1 Function Documentation

7.147.1.1 run_memory_words_tests()

```
void run_memory_words_tests (
    VM * vm )
```

Executes the test suite for memory manipulation words.

Runs through a series of test cases for Forth memory operations including store, fetch, allot, and other memory-related words

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [memory_word_suites](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.147.2 Variable Documentation

7.147.2.1 memory_word_suites

```
WordTestSuite memory\_word\_suites[] [static]
```

Referenced by [run_memory_words_tests\(\)](#).

7.148 src/test_runner/modules/mixed_arithmetic_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for mixed_arithmetic_words_test.c:
```

Functions

- void [run_mixed_arithmetic_words_tests](#) (VM *vm)

Variables

- static [WordTestSuite](#) [mixed_arithmetic_word_suites](#) []

7.148.1 Function Documentation

7.148.1.1 run_mixed_arithmetic_words_tests()

```
void run_mixed_arithmetic_words_tests (
    VM * vm )
```

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [mixed_arithmetic_word_suites](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.148.2 Variable Documentation

7.148.2.1 mixed_arithmetic_word_suites

```
WordTestSuite mixed_arithmetic_word_suites[] [static]
```

Referenced by [run_mixed_arithmetic_words_tests\(\)](#).

7.149 src/test_runner/modules/return_stack_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for return_stack_words_test.c:
```

Functions

- void [run_return_stack_words_tests](#) (VM *vm)
Executes all test suites for return stack manipulation words.

Variables

- static [WordTestSuite return_stack_word_suites](#) []
Test suites for return stack manipulation words.

7.149.1 Function Documentation

7.149.1.1 run_return_stack_words_tests()

```
void run_return_stack_words_tests (
    VM * vm )
```

Executes all test suites for return stack manipulation words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all defined test cases for >R, R>, and R@ words and reports results_run_01_2025_12_08 through the logging system

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [return_stack_word_suites](#), [run_test_suite_m\(\)](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.149.2 Variable Documentation

7.149.2.1 return_stack_word_suites

```
WordTestSuite return_stack_word_suites[] [static]
```

Test suites for return stack manipulation words.

Contains test cases for >R (push to return stack), R> (pop from return stack), and R@ (copy top of return stack)

Referenced by [run_return_stack_words_tests\(\)](#).

7.150 src/test_runner/modules/stack_words_test.c File Reference

Test module for Forth stack manipulation words.

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for stack_words_test.c:

Functions

- void [run_stack_words_tests](#) (VM *vm)
Executes all test suites for stack manipulation words.

Variables

- static [WordTestSuite](#) [stack_word_suites](#) []
Test suites for stack manipulation words.

7.150.1 Detailed Description

Test module for Forth stack manipulation words.

Contains test suites for basic stack operations including DUP, DROP, SWAP, OVER, ROT, DEPTH, PICK, and ROLL

7.150.2 Function Documentation

7.150.2.1 run_stack_words_tests()

```
void run_stack_words_tests (
    VM * vm )
```

Executes all test suites for stack manipulation words.

Run tests for stack manipulation words.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Runs through all defined test cases for stack manipulation words and reports results_run_01_2025_12_08 through the logging system

References [CONTRACT_PHYSICS_TRANSPARENT](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite_m\(\)](#), [stack_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.150.3 Variable Documentation

7.150.3.1 stack_word_suites

```
WordTestSuite stack_word_suites[] [static]
```

Test suites for stack manipulation words.

Contains comprehensive test cases for all standard stack operations, including normal cases, edge cases, and error conditions

Referenced by [run_stack_words_tests\(\)](#).

7.151 src/test_runner/modules/starforth_words_test.c File Reference

```
#include "../include/test_runner.h"
```

```
#include "../include/test_common.h"
```

Include dependency graph for starforth_words_test.c:

Functions

- void [run_starforth_words_tests](#) ([VM](#) *vm)
Executes all StarForth words test suites.

Variables

- static [WordTestSuite](#) [starforth_word_suites](#) []
Test suites for StarForth-specific words.

7.151.1 Function Documentation

7.151.1.1 run_starforth_words_tests()

```
void run_starforth_words_tests (
    VM * vm )
```

Executes all StarForth words test suites.

Parameters

vm	Pointer to the FORTH virtual machine instance
--------------------	---

Runs through all test cases for StarForth-specific implementation words, including entropy tracking, comments, and initialization.

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [starforth_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.151.2 Variable Documentation

7.151.2.1 starforth_word_suites

```
WordTestSuite starforth\_word\_suites[] [static]
```

Test suites for StarForth-specific words.

Contains test cases for the following StarForth implementation words:

- ENTROPY@: Fetch word execution count
- ENTROPY!: Store word execution count
- WORD-ENTROPY: Get execution count for a word by name
- RESET-ENTROPY: Reset all word execution counts
- TOP-WORDS: Display most frequently executed words
- (-: Comment to end of line
- INIT: System initialization

Referenced by [run_starforth_words_tests\(\)](#).

7.152 src/test_runner/modules/stress_tests.c File Reference

FORTH-79 Standard Comprehensive Stress Testing Module.

```
#include <stdio.h>
```

```
#include "test_common.h"
```

```
#include "test_runner.h"
```

Include dependency graph for stress_tests.c:

Functions

- void [run_stress_tests](#) ([VM](#) *vm)
Execute comprehensive stress test suites.

Variables

- static [WordTestSuite](#) [stress_suites](#) []

7.152.1 Detailed Description

FORTH-79 Standard Comprehensive Stress Testing Module.

This module contains extensive stress tests that verify the interpreter's behavior under extreme conditions while remaining within FORTH-79 standard compliance.

Test categories (expanded):

- Deep nesting of control structures (up to 5 levels)
- Stack exhaustion and boundary tests (near-limit operations)
- Numeric boundary conditions (cell_t min/max, overflow)
- Dictionary pressure (many definitions, long names)
- Word execution chains (deep call stacks)
- Control flow edge cases (empty loops, large counts, EXIT)
- Memory allocation patterns (fragmentation, large blocks)
- Error recovery (ABORT, stack underflow recovery)
- Large definitions and word chains

IMPORTANT: These tests only use FORTH-79 standard words. Features from later standards (ANS Forth 1994, Forth 2012) are NOT tested here:

- RECURSE (ANS Forth) - not in FORTH-79
- Self-referential definitions - not supported in FORTH-79
- Extended VARIABLE usage - only standard semantics tested

For testing extended/non-standard features, see [integration_tests.c](#)

7.152.2 Function Documentation

7.152.2.1 run_stress_tests()

```
void run_stress_tests (
    VM * vm )
```

Execute comprehensive stress test suites.

Execute stress tests.

Runs an extensive set of stress tests to verify the [VM](#)'s behavior under extreme conditions. Tests include:

- Stack operations and boundaries
- Deep nesting and control flow
- Numeric boundaries and overflow
- Dictionary pressure
- Memory allocation patterns
- Error recovery
- Mixed complexity scenarios

All tests are FORTH-79 compliant.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [global_test_stats](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [run_test_suite\(\)](#), [stress_suites](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), and [vm](#).

Here is the call graph for this function:

7.152.3 Variable Documentation

7.152.3.1 stress_suites

[WordTestSuite](#) stress_suites[] [static]

Referenced by [run_stress_tests\(\)](#).

7.153 src/test_runner/modules/string_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for string_words_test.c:
```

Functions

- void [run_string_words_tests](#) ([VM](#) **vm*)
Executes all string manipulation word tests.

Variables

- static [WordTestSuite](#) string_word_suites []

7.153.1 Function Documentation

7.153.1.1 run_string_words_tests()

```
void run_string_words_tests (
    VM * vm )
```

Executes all string manipulation word tests.

Runs test suites for FORTH string operations including COUNT, -TRAILING, CMOVE, CMOVE>, COMPARE, SEARCH, SCAN, SKIP, and BLANK

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [string_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.153.2 Variable Documentation

7.153.2.1 string_word_suites

[WordTestSuite](#) string_word_suites[] [static]

Referenced by [run_string_words_tests\(\)](#).

7.154 src/test_runner/modules/system_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for system_words_test.c:
```

Functions

- void [run_system_words_tests](#) (VM *vm)
Executes all system word tests.

Variables

- static [WordTestSuite](#) [system_word_suites](#) []
Test suite definitions for system control words.

7.154.1 Function Documentation

7.154.1.1 run_system_words_tests()

```
void run_system_words_tests (
    VM * vm )
```

Executes all system word tests.

Runs test suites for FORTH system control operations including QUIT, ABORT, ABORT", BYE, COLD, and WARM. Some tests are marked as unimplemented due to their system-level effects

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [LOG_WARN](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [system_word_suites](#), [vm](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.154.2 Variable Documentation

7.154.2.1 system_word_suites

```
WordTestSuite system_word_suites[] [static]
```

Test suite definitions for system control words.

Contains test cases for each system word including normal operations and error conditions

Referenced by [run_system_words_tests\(\)](#).

7.155 src/test_runner/modules/vocabulary_words_test.c File Reference

```
#include "../include/test_runner.h"
#include "../include/test_common.h"
Include dependency graph for vocabulary_words_test.c:
```

Functions

- void [run_vocabulary_words_tests](#) (VM *vm)
Executes all vocabulary word test suites.

Variables

- static [WordTestSuite](#) [vocabulary_word_suites](#) []
Test suites for vocabulary-related FORTH words (Module 15)

7.155.1 Function Documentation

7.155.1.1 run_vocabulary_words_tests()

```
void run_vocabulary_words_tests (
    VM * vm )
```

Executes all vocabulary word test suites.

Parameters

<code>vm</code>	Pointer to the Forth virtual machine instance
-----------------	---

Runs through all vocabulary-related test suites, executing each test case and logging the results_run_01_2025_↵12_08. Tests vocabulary creation, selection, and manipulation.

References [LOG_INFO](#), [log_message\(\)](#), [LOG_TEST](#), [print_module_summary\(\)](#), [run_test_suite\(\)](#), [vm](#), [vocabulary_word_suites](#), and [WordTestSuite::word_name](#).

Here is the call graph for this function:

7.155.2 Variable Documentation

7.155.2.1 vocabulary_word_suites

```
WordTestSuite vocabulary_word_suites[] [static]
```

Test suites for vocabulary-related FORTH words (Module 15)

Contains test cases for each vocabulary word, including normal operations and error cases. Each suite tests specific functionality of the vocabulary system.

Referenced by [run_vocabulary_words_tests\(\)](#).

7.156 src/test_runner/test_common.c File Reference

```
#include "../include/vm.h"
#include "../include/vm_debug.h"
#include "include/test_common.h"
#include "include/test_contracts.h"
#include "../include/log.h"
#include "test_runner.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for test_common.c:

Macros

- #define [test_fatal_exit\(\)](#) exit(1)
- #define [DIE\(vm, why\)](#) [dump_and_die\(\(vm\), \(why\), __FILE__, __LINE__\)](#)

Functions

- static void [dump_and_die](#) (VM *vm, const char *reason, const char *file, int line)
Dumps VM state and terminates execution.
- static void [fail_and_exit](#) (VM *vm, const char *module, const char *case_name, const char *input, const char *reason)
Handles test failure and exits.
- void [save_vm_state](#) (VM *vm, int *dsp, int *rsp, int *error, vm_mode_t *mode)
Saves current VM state.
- void [restore_vm_state](#) (VM *vm, int dsp, int rsp, int error, vm_mode_t mode)
Restores previously saved VM state.
- int [assert_stack_depth](#) (VM *vm, int expected_depth)

- Verifies [VM](#) stack depth matches expected value.
- int [assert_stack_top](#) ([VM](#) *vm, int expected_value)
Verifies top value on [VM](#) stack matches expected value.
- int [assert_vm_error](#) ([VM](#) *vm, int should_have_error)
Verifies [VM](#) error state matches expectation.
- [TestResult](#) [run_single_test](#) ([VM](#) *vm, const char *word_name, const [TestCase](#) *test)
Executes a single test case.
- void [run_test_suite](#) ([VM](#) *vm, const [WordTestSuite](#) *suite)
Executes a complete test suite for a Forth word.
- static [WordContract](#) [resolve_contract](#) ([WordContract](#) test_c, [WordContract](#) suite_c, [WordContract](#) module_c)
Resolves the effective contract for a test case.
- void [run_test_suite_m](#) ([VM](#) *vm, const [WordTestSuite](#) *suite, [WordContract](#) module_contract)
Runs all tests in a suite with an inherited module-level contract.
- void [print_module_summary](#) (const char *module_name, int pass, int fail, int skip, int error)
Prints test module execution summary.
- void [save_dict_state](#) ([VM](#) *vm, [DictEntry](#) **latest, size_t *here)
Saves current dictionary state before test execution.
- void [restore_dict_state](#) ([VM](#) *vm, [DictEntry](#) *latest, size_t here)
Restores dictionary state after test execution.

Variables

- int [fail_fast](#) = 0
Global flag to control test execution behavior.

7.156.1 Macro Definition Documentation

7.156.1.1 DIE

```
#define DIE(  
    vm,  
    why ) dump_and_die((vm), (why), __FILE__, __LINE__)
```

7.156.1.2 test_fatal_exit

```
#define test_fatal_exit( ) exit(1)
```

7.156.2 Function Documentation

7.156.2.1 assert_stack_depth()

```
int assert_stack_depth (  
    VM * vm,  
    int expected_depth )
```

Verifies [VM](#) stack depth matches expected value.

Verifies the [VM](#) stack depth.

Parameters

<i>vm</i>	Pointer to VM instance
<i>expected_depth</i>	Expected stack depth

Returns

1 if assertion passes, 0 if it fails

References [VM::dsp](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.156.2.2 assert_stack_top()

```
int assert_stack_top (
    VM * vm,
    int expected_value )
```

Verifies top value on [VM](#) stack matches expected value.

Verifies the value at the top of the stack.

Parameters

<i>vm</i>	Pointer to VM instance
<i>expected_value</i>	Expected value on top of stack

Returns

1 if assertion passes, 0 if it fails

References [VM::data_stack](#), [VM::dsp](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.156.2.3 assert_vm_error()

```
int assert_vm_error (
    VM * vm,
    int should_have_error )
```

Verifies [VM](#) error state matches expectation.

Verifies [VM](#) error state.

Parameters

<i>vm</i>	Pointer to VM instance
<i>should_have_error</i>	Expected error state (1 for error, 0 for no error)

Returns

1 if assertion passes, 0 if it fails

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Here is the call graph for this function:

7.156.2.4 dump_and_die()

```
static void dump_and_die (
    VM * vm,
    const char * reason,
    const char * file,
    int line ) [static]
```

Dumps [VM](#) state and terminates execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>reason</i>	String describing the failure reason

Parameters

<i>file</i>	Source file where failure occurred
<i>line</i>	Line number where failure occurred

References [LOG_ERROR](#), [log_message\(\)](#), [test_fatal_exit](#), [vm](#), and [vm_debug_dump_state\(\)](#).

Here is the call graph for this function:

7.156.2.5 fail_and_exit()

```
static void fail_and_exit (
    VM * vm,
    const char * module,
    const char * case_name,
    const char * input,
    const char * reason ) [static]
```

Handles test failure and exits.

Parameters

<i>vm</i>	Pointer to VM instance
<i>module</i>	Name of the test module
<i>case_name</i>	Name of the test case
<i>input</i>	Test input that caused the failure
<i>reason</i>	Description of failure reason

References [DIE](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [run_single_test\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.156.2.6 print_module_summary()

```
void print_module_summary (
    const char * module_name,
    int pass,
    int fail,
    int skip,
    int error )
```

Prints test module execution summary.

Prints test results_run_01_2025_12_08 summary for a module.

Parameters

<i>module_name</i>	Name of the test module
<i>pass</i>	Number of passed tests
<i>fail</i>	Number of failed tests
<i>skip</i>	Number of skipped tests
<i>error</i>	Number of tests with errors

References [log_message\(\)](#), and [LOG_TEST](#).

Referenced by [run_acl_words_tests\(\)](#), [run_arithmetic_words_tests\(\)](#), [run_block_words_tests\(\)](#), [run_control_words_tests\(\)](#), [run_defining_words_tests\(\)](#), [run_dictionary_manipulation_words_tests\(\)](#), [run_dictionary_words_tests\(\)](#), [run_double_words_tests\(\)](#), [run_format_words_tests\(\)](#), [run_io_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_mama_forth_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_mixed_arithmetic_words_tests\(\)](#), [run_return_stack_words_tests\(\)](#), [run_stack_words_tests\(\)](#), [run_starforth_words_tests\(\)](#), [run_string_words_tests\(\)](#), [run_system_words_tests\(\)](#), and [run_vocabulary_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.156.2.7 resolve_contract()

```
static WordContract resolve_contract (
    WordContract test_c,
    WordContract suite_c,
    WordContract module_c ) [static]
```

Resolves the effective contract for a test case.

First non-NONE source wins: per-test > per-suite > module.

References [WordContract::flags](#).

Referenced by [run_test_suite_m\(\)](#).

Here is the caller graph for this function:

7.156.2.8 restore_dict_state()

```
void restore_dict_state (
    VM * vm,
    DictEntry * latest,
    size_t here )
```

Restores dictionary state after test execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Latest dictionary entry to restore
<i>here</i>	HERE pointer to restore

Removes all words added during test by restoring dictionary pointers. This prevents test pollution of the dictionary and ensures each test starts with a clean slate.

References [VM::here](#), [VM::latest](#), and [vm](#).

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

Here is the caller graph for this function:

7.156.2.9 restore_vm_state()

```
void restore_vm_state (
    VM * vm,
    int dsp,
    int rsp,
    int error,
    vm_mode_t mode )
```

Restores previously saved [VM](#) state.

Restores [VM](#) state from saved values.

Parameters

<i>vm</i>	Pointer to VM instance
<i>dsp</i>	Data stack pointer to restore
<i>rsp</i>	Return stack pointer to restore
<i>error</i>	Error state to restore
<i>mode</i>	VM mode to restore

References [VM::abort_requested](#), [VM::compiling_word](#), [VM::current_executing_entry](#), [VM::dsp](#), [VM::error](#), [VM::exit_colon](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::rsp](#), and [vm](#).

Referenced by [run_acl_words_tests\(\)](#), and [run_single_test\(\)](#).

Here is the caller graph for this function:

7.156.2.10 run_single_test()

```

TestResult run_single_test (
    VM * vm,
    const char * word_name,
    const TestCase * test )

```

Executes a single test case.

Parameters

<i>vm</i>	Pointer to VM instance
<i>word_name</i>	Name of the word being tested
<i>test</i>	Pointer to test case definition

Returns

Test result (PASS, FAIL, SKIP, or ERROR)

References **VM::error**, **TestCase::expected**, **fail_and_exit()**, **fail_fast**, **TestCase::implemented**, **TestCase::input**, **LOG_ERROR**, **log_message()**, **LOG_TEST**, **log_test_result()**, **TestCase::name**, **restore_vm_state()**, **save_vm_state()**, **TestCase::should_error**, **TEST_FAIL**, **TEST_PASS**, **TEST_SKIP**, **vm**, and **vm_interpret()**.

Referenced by **run_test_suite()**, and **run_test_suite_m()**.

Here is the call graph for this function: Here is the caller graph for this function:

7.156.2.11 run_test_suite()

```

void run_test_suite (
    VM * vm,
    const WordTestSuite * suite )

```

Executes a complete test suite for a Forth word.

Runs all tests in a test suite.

Parameters

<i>vm</i>	Pointer to VM instance
<i>suite</i>	Pointer to test suite definition

References **global_test_stats**, **TestCase::implemented**, **log_message()**, **LOG_TEST**, **TestCase::name**, **restore_dict_state()**, **run_single_test()**, **save_dict_state()**, **WordTestSuite::test_count**, **TEST_ERROR**, **TEST_FAIL**, **TEST_PASS**, **TEST_SKIP**, **WordTestSuite::tests**, **TestStats::total_error**, **TestStats::total_fail**, **TestStats::total_pass**, **TestStats::total_skip**, **TestStats::total_tests**, **vm**, and **WordTestSuite::word_name**.

Referenced by **run_block_words_tests()**, **run_control_words_tests()**, **run_defining_words_tests()**, **run_dictionary_manipulation_words_tests()**, **run_dictionary_words_tests()**, **run_double_words_tests()**, **run_format_words_tests()**, **run_integration_tests()**, **run_io_words_tests()**, **run_mama_forth_words_tests()**, **run_mixed_arithmetic_words_tests()**, **run_starforth_words_tests()**, **run_stress_tests()**, **run_string_words_tests()**, **run_system_words_tests()**, and **run_vocabulary_words_tests()**.

Here is the call graph for this function: Here is the caller graph for this function:

7.156.2.12 run_test_suite_m()

```

void run_test_suite_m (
    VM * vm,
    const WordTestSuite * suite,
    WordContract module_contract )

```

Runs all tests in a suite with an inherited module-level contract.

Contract resolution order (first non-NONE wins): test->contract > suite->suite_contract > module_contract

When **CONTRACTS_ENABLED=1**, each passing **TEST_NORMAL** test is re-run under physics perturbation to witness A4' (word_physics_transparent).

Parameters

<i>vm</i>	Pointer to the VM instance
<i>suite</i>	Test suite to execute
<i>module_contract</i>	Default contract for tests that don't specify one

References [check_physics_transparent\(\)](#), [TestCase::contract](#), [CONTRACT_PHYSICS_TRANSPARENT](#), [WordContract::flags](#), [WordContract::fuzz_rounds](#), [global_test_stats](#), [TestCase::implemented](#), [TestCase::input](#), [log_message\(\)](#), [LOG_TEST](#), [TestCase::name](#), [resolve_contract\(\)](#), [restore_dict_state\(\)](#), [run_single_test\(\)](#), [save_dict_state\(\)](#), [TestCase::should_error](#), [WordTestSuite::suite_contract](#), [WordTestSuite::test_count](#), [TEST_ERROR](#), [TEST_ERROR_CASE](#), [TEST_FAIL](#), [TEST_PASS](#), [TEST_SKIP](#), [WordTestSuite::tests](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), [TestStats::total_tests](#), [TestCase::type](#), [vm](#), and [WordTestSuite::word_name](#).
Referenced by [run_arithmetic_words_tests\(\)](#), [run_logical_words_tests\(\)](#), [run_memory_words_tests\(\)](#), [run_return_stack_words_tests\(\)](#), and [run_stack_words_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.156.2.13 save_dict_state()

```
void save_dict_state (
    VM * vm,
    DictEntry ** latest,
    size_t * here )
```

Saves current dictionary state before test execution.

Saves dictionary state before test execution.

Parameters

<i>vm</i>	Pointer to VM instance
<i>latest</i>	Pointer to store latest dictionary entry
<i>here</i>	Pointer to store HERE pointer

Captures dictionary state so tests can be cleaned up after execution

References [VM::here](#), [VM::latest](#), and [vm](#).

Referenced by [run_test_suite\(\)](#), and [run_test_suite_m\(\)](#).

Here is the caller graph for this function:

7.156.2.14 save_vm_state()

```
void save_vm_state (
    VM * vm,
    int * dsp,
    int * rsp,
    int * error,
    vm_mode_t * mode )
```

Saves current [VM](#) state.

Parameters

<i>vm</i>	Pointer to VM instance
<i>dsp</i>	Pointer to store data stack pointer
<i>rsp</i>	Pointer to store return stack pointer
<i>error</i>	Pointer to store error state
<i>mode</i>	Pointer to store VM mode

References [VM::dsp](#), [VM::error](#), [VM::mode](#), [VM::rsp](#), and [vm](#).

Referenced by [run_acl_words_tests\(\)](#), and [run_single_test\(\)](#).

Here is the caller graph for this function:

7.156.3 Variable Documentation

7.156.3.1 fail_fast

```
int fail_fast = 0
```

Global flag to control test execution behavior.

When set to non-zero, causes test runner to exit immediately on first failure

Referenced by [main\(\)](#), and [run_single_test\(\)](#).

7.157 src/test_runner/test_contracts.c File Reference

```
#include "test_contracts.h"
#include "log.h"
#include "vm.h"
#include <string.h>
```

Include dependency graph for test_contracts.c:

Functions

- static uint64_t [contract_rand](#) (void)
- static void [rng_seed](#) (int round)
- void [capture_exec_result](#) (VM *vm, ExecResult *r)
- int [exec_results_equal](#) (const ExecResult *a, const ExecResult *b)
- void [save_physics](#) (VM *vm, PhysicsSnapshot *s)
- void [restore_physics](#) (VM *vm, const PhysicsSnapshot *s)
- void [perturb_physics](#) (VM *vm, int round)
- int [check_physics_transparent](#) (VM *vm, const char *word_name, const char *input, int should_error, int implemented, int fuzz_rounds)
- int [assert_heartbeat_safe](#) (VM *vm)

Variables

- int [global_contract_violations](#) = 0
- static uint64_t [rng_state](#) = 0xDEADBEEFCAFEBAEULL

7.157.1 Function Documentation

7.157.1.1 assert_heartbeat_safe()

```
int assert_heartbeat_safe (
    VM * vm )
```

References [capture_exec_result\(\)](#), [ExecResult::dsp](#), [exec_results_equal\(\)](#), [global_contract_violations](#), [LOG_ERROR](#), [log_message\(\)](#), [ExecResult::rsp](#), [vm](#), and [vm_tick\(\)](#).

Here is the call graph for this function:

7.157.1.2 capture_exec_result()

```
void capture_exec_result (
    VM * vm,
    ExecResult * r )
```

References [VM::data_stack](#), [VM::dsp](#), [ExecResult::dsp](#), [VM::error](#), [ExecResult::error](#), [VM::rsp](#), [ExecResult::rsp](#), [ExecResult::stack](#), [STACK_SIZE](#), and [vm](#).

Referenced by [assert_heartbeat_safe\(\)](#), and [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.157.1.3 check_physics_transparent()

```
int check_physics_transparent (
    VM * vm,
    const char * word_name,
    const char * input,
    int should_error,
    int implemented,
    int fuzz_rounds )
```

References [VM::abort_requested](#), [capture_exec_result\(\)](#), [CONTRACT_DEFAULT_FUZZ_ROUNDS](#), [VM::dsp](#), [ExecResult::dsp](#), [VM::error](#), [exec_results_equal\(\)](#), [VM::exit_colon](#), [global_contract_violations](#), [LOG_ERROR](#), [log_message\(\)](#), [perturb_physics\(\)](#), [restore_physics\(\)](#), [VM::rsp](#), [save_physics\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [run_test_suite_m\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.157.1.4 contract_rand()

```
static uint64_t contract_rand (
    void ) [static]
```

References [rng_state](#).

Referenced by [perturb_physics\(\)](#).

Here is the caller graph for this function:

7.157.1.5 exec_results_equal()

```
int exec_results_equal (
    const ExecResult * a,
    const ExecResult * b )
```

References [ExecResult::dsp](#), [ExecResult::error](#), [ExecResult::rsp](#), and [ExecResult::stack](#).

Referenced by [assert_heartbeat_safe\(\)](#), and [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.157.1.6 perturb_physics()

```
void perturb_physics (
    VM * vm,
    int round )
```

References [contract_rand\(\)](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_rng_seed\(\)](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.157.1.7 restore_physics()

```
void restore_physics (
    VM * vm,
    const PhysicsSnapshot * s )
```

References [VM::decay_slope_q48](#), [PhysicsSnapshot::decay_slope_q48](#), [PhysicsSnapshot::eff_window_size](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [PhysicsSnapshot::heat_25th](#), [PhysicsSnapshot::heat_50th](#), [PhysicsSnapshot::heat_75th](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PhysicsSnapshot::prefetch_attempts](#), [VM::rolling_window](#), [HeartbeatState::tick_target_ns](#), [PhysicsSnapshot::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [PhysicsSnapshot::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.157.1.8 rng_seed()

```
static void rng_seed (
    int round ) [static]
```

References [rng_state](#).

Referenced by [perturb_physics\(\)](#).

Here is the caller graph for this function:

7.157.1.9 save_physics()

```
void save_physics (
    VM * vm,
    PhysicsSnapshot * s )
```

References [VM::decay_slope_q48](#), [PhysicsSnapshot::decay_slope_q48](#), [PhysicsSnapshot::eff_window_size](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [PhysicsSnapshot::heat_25th](#), [PhysicsSnapshot::heat_50th](#), [PhysicsSnapshot::heat_75th](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PhysicsSnapshot::prefetch_attempts](#), [VM::rolling_window](#), [HeartbeatState::tick_target_ns](#), [PhysicsSnapshot::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [PhysicsSnapshot::total_executions](#), and [vm](#).

Referenced by [check_physics_transparent\(\)](#).

Here is the caller graph for this function:

7.157.2 Variable Documentation

7.157.2.1 global_contract_violations

```
int global_contract_violations = 0
```

Referenced by [assert_heartbeat_safe\(\)](#), [check_physics_transparent\(\)](#), and [run_all_tests\(\)](#).

7.157.2.2 rng_state

```
uint64_t rng_state = 0xDEADBEEFCAFEBABEULL [static]
```

Referenced by [contract_rand\(\)](#), and [rng_seed\(\)](#).

7.158 src/test_runner/test_runner.c File Reference

```
#include "include/test_runner.h"
#include "include/test_contracts.h"
#include "../include/log.h"
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
```

Include dependency graph for test_runner.c:

Functions

- static uint64_t [get_time_ns](#) (void)
Get current time in nanoseconds.
- void [enable_benchmark_mode](#) (int iterations)
Enable benchmark mode for test execution.
- void [run_all_tests](#) (VM *vm)
Run all test modules with optional benchmarking.
- void [run_module_tests](#) (VM *vm, const char *module_name)
Run tests for a specific module.
- void [run_word_tests](#) (VM *vm, const char *word_name)
Run tests for a specific Forth word.

Variables

- `TestStats global_test_stats = {0, 0, 0, 0, 0}`
Global statistics for test execution.
- static int `benchmark_mode = 0`
Flag indicating if benchmark mode is enabled.
- static int `benchmark_iterations = 1000`
Number of iterations for benchmark tests.
- static `TestModule test_modules []`

7.158.1 Function Documentation

7.158.1.1 `enable_benchmark_mode()`

```
void enable_benchmark_mode (
    int iterations )
```

Enable benchmark mode for test execution.

Enable benchmark mode for performance testing.

Parameters

<i>iterations</i>	Number of times each test should be repeated
-------------------	--

References [benchmark_iterations](#), [benchmark_mode](#), [LOG_INFO](#), and [log_message\(\)](#).

Referenced by [run_benchmark_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.158.1.2 `get_time_ns()`

```
static uint64_t get_time_ns (
    void ) [static]
```

Get current time in nanoseconds.

Uses platform-specific timing: [sf_monotonic_ns\(\)](#) on kernel, `clock()` on hosted.

Returns

Current time in nanoseconds, or 0 on error

References [sf_monotonic_ns\(\)](#).

Referenced by [run_module_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.158.1.3 `run_all_tests()`

```
void run_all_tests (
    VM * vm )
```

Run all test modules with optional benchmarking.

Run all available tests.

Executes all registered test modules sequentially, collecting statistics and/or benchmark data depending on the current mode.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [global_contract_violations](#), [global_test_stats](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), [test_modules](#), [TestStats::total_error](#), [TestStats::total_fail](#), [TestStats::total_pass](#), [TestStats::total_skip](#), and [TestStats::total_tests](#).

Referenced by [main\(\)](#), [run_benchmark_mode\(\)](#), [run_break_me_tests\(\)](#), and [run_doe_experiment\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.158.1.4 run_module_tests()

```
void run_module_tests (
    VM * vm,
    const char * module_name )
```

Run tests for a specific module.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>module_name</i>	Name of the module to test

References [benchmark_iterations](#), [benchmark_mode](#), [get_time_ns\(\)](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), [run_module_tests\(\)](#), [TestModule::run_module_tests](#), [test_modules](#), and [vm](#).

Referenced by [run_module_tests\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.158.1.5 run_word_tests()

```
void run_word_tests (
    VM * vm,
    const char * word_name )
```

Run tests for a specific Forth word.

Run tests for a specific word.

Searches all modules for tests related to the specified word and executes them.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>word_name</i>	Name of the Forth word to test

References [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [TestModule::module_name](#), and [test_modules](#).

Here is the call graph for this function:

7.158.2 Variable Documentation

7.158.2.1 benchmark_iterations

```
int benchmark_iterations = 1000 [static]
```

Number of iterations for benchmark tests.

Referenced by [enable_benchmark_mode\(\)](#), and [run_module_tests\(\)](#).

7.158.2.2 benchmark_mode

```
int benchmark_mode = 0 [static]
```

Flag indicating if benchmark mode is enabled.

Referenced by [enable_benchmark_mode\(\)](#), and [run_module_tests\(\)](#).

7.158.2.3 global_test_stats

```
TestStats global_test_stats = {0, 0, 0, 0, 0}
```

Global statistics for test execution.

Referenced by [run_all_tests\(\)](#), [run_break_me_tests\(\)](#), [run_stress_tests\(\)](#), [run_test_suite\(\)](#), [run_test_suite_m\(\)](#), and [sk_parity_collect\(\)](#).

7.158.2.4 test_modules

`TestModule test_modules[] [static]`

Initial value:

```
= {
    {"Stack Words", NULL, 0, run_stack_words_tests},
    {"Return Stack Words", NULL, 0, run_return_stack_words_tests},
    {"Memory Words", NULL, 0, run_memory_words_tests},
    {"Arithmetic Words", NULL, 0, run_arithmetic_words_tests},
    {"Logical Words", NULL, 0, run_logical_words_tests},
    {"Mixed Arithmetic Words", NULL, 0, run_mixed_arithmetic_words_tests},
    {"Double Words", NULL, 0, run_double_words_tests},
    {"Format Words", NULL, 0, run_format_words_tests},
    {"String Words", NULL, 0, run_string_words_tests},
    {"I/O Words", NULL, 0, run_io_words_tests},
    {"Block Words", NULL, 0, run_block_words_tests},
    {"Dictionary Words", NULL, 0, run_dictionary_words_tests},
    {"Dictionary Manipulation Words", NULL, 0, run_dictionary_manipulation_words_tests},
    {"Vocabulary Words", NULL, 0, run_vocabulary_words_tests},
    {"System Words", NULL, 0, run_system_words_tests},
    {"Defining Words", NULL, 0, run_defining_words_tests},
    {"Control Words", NULL, 0, run_control_words_tests},
    {"StarForth Words", NULL, 0, run_starforth_words_tests},
    {"ACL Words", NULL, 0, run_acl_words_tests},
    {"Mama FORTH Words", NULL, 0, run_mama_forth_words_tests},
    {NULL, NULL, 0, NULL}
}
```

Referenced by [run_all_tests\(\)](#), [run_module_tests\(\)](#), and [run_word_tests\(\)](#).

7.159 src/vm.c File Reference

```
#include "vm_internal.h"
#include "../include/log.h"
#include "../include/profiler.h"
#include "../include/physics_metadata.h"
#include "../include/physics_execution_hooks.h"
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
```

Include dependency graph for vm.c:

Functions

- unsigned [vm_get_base](#) (const [VM](#) *vm)
- void [vm_set_base](#) ([VM](#) *vm, unsigned b)
- int [vm_parse_word](#) ([VM](#) *vm, char *word, size_t max_len)

Parse next word from input buffer.
- int [vm_parse_number](#) ([VM](#) *vm, const char *s, [cell_t](#) *out)

Parse string as number in current base.
- void [vm_enter_compile_mode](#) ([VM](#) *vm, const char *name, size_t len)
- void [vm_compile_word](#) ([VM](#) *vm, [DictEntry](#) *entry)
- void [vm_compile_literal](#) ([VM](#) *vm, [cell_t](#) value)
- void [vm_compile_call](#) ([VM](#) *vm, [word_func_t](#) func)
- void [vm_compile_exit](#) ([VM](#) *vm)
- void [vm_exit_compile_mode](#) ([VM](#) *vm)
- static void [acl_recheck](#) ([VM](#) *vm, [DictEntry](#) *entry)
- void [execute_colon_word](#) ([VM](#) *vm)
- void [vm_interpret_word](#) ([VM](#) *vm, const char *word_str, size_t len)
- void [vm_interpret](#) ([VM](#) *vm, const char *input)

Interpret a string of Forth code.
- int [vm_addr_ok](#) (struct [VM](#) *vm, [vaddr_t](#) addr, size_t len)

Check if memory address range is valid.
- uint8_t * [vm_ptr](#) (struct [VM](#) *vm, [vaddr_t](#) addr)

- `uint8_t vm_load_u8` (struct `VM *vm`, `vaddr_t` `addr`)
- `void vm_store_u8` (struct `VM *vm`, `vaddr_t` `addr`, `uint8_t` `v`)
- `cell_t vm_load_cell` (struct `VM *vm`, `vaddr_t` `addr`)
- `void vm_store_cell` (struct `VM *vm`, `vaddr_t` `addr`, `cell_t` `v`)
- `void vm_make_immediate` (`VM *vm`)

7.159.1 Function Documentation

7.159.1.1 `acl_recheck()`

```
static void acl_recheck (
    VM * vm,
    DictEntry * entry ) [static]
```

References `ACL_TTL_OPEN`, `VM::current_executing_entry`, `VM::emergency_console`, `entry`, `VM::error`, `DictEntry::func`, `log_message()`, `LOG_WARN`, `vm`, `vm_find_word()`, and `vm_push()`.

Referenced by `execute_colon_word()`, and `vm_interpret_word()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.2 `execute_colon_word()`

```
void execute_colon_word (
    VM * vm )
```

References `VM::abort_requested`, `DictEntry::acl_allow`, `acl_recheck()`, `DictEntry::acl_ttl`, `HeartbeatState::check_counter`, `VM::current_executing_entry`, `DICTIONARY_SIZE`, `VM::dsp`, `VM::emergency_console`, `ENABLE_HOTWORDS_CACHE`, `ENABLE_PIPELINING`, `entry`, `VM::error`, `DictEntry::execution_heat`, `VM::exit_colon`, `DictEntry::func`, `VM::heartbeat`, `HEARTBEAT_CHECK_FREQUENCY`, `VM::hotwords_cache`, `hotwords_cache_promote()`, `HOTWORDS_EXECUTION_HEAT_THRESHOLD`, `DictPhysics::last_active_ns`, `DictPhysics::last_decay_ns`, `LOG_ERROR`, `log_message()`, `LOG_WARN`, `WordTransitionMetrics::most_likely`, `DictEntry::name`, `DictEntry::name_len`, `DictEntry::physics`, `physics_execution_heat_increment()`, `physics_metadata_apply_linear_decay()`, `physics_metadata_touch()`, `physics_post_execute()`, `physics_pre_execute()`, `VM::pipeline_metrics`, `WordTransitionMetrics::prefetch_attempts`, `PipelineGlobalMetrics::prefetch_attempts`, `PipelineGlobalMetrics::prefetch_hits`, `profiler_word_count()`, `profiler_word_enter()`, `profiler_word_exit()`, `VM::rolling_window`, `rolling_window_record_execution()`, `VM::rsp`, `DictEntry::transition_metrics`, `transition_metrics_record()`, `transition_metrics_record_prefetch_hit()`, `transition_metrics_should_speculate()`, `transition_metrics_update_cache()`, `vm`, `vm_assert_interpreter_enabled()`, `vm_dictionary_get_data_field()`, `vm_dictionary_lookup_by_word_id()`, `vm_heartbeat_run_cycle()`, `vm_monotonic_ns()`, `vm_ptr()`, `vm_rpop()`, `vm_rpush()`, `word_id`, `DictEntry::word_id`, `HeartbeatState::words_executed`, `HeartbeatState::worker`, and `VM::zuse_session`.

Referenced by `system_word_see()`, and `vm_enter_compile_mode()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.3 `vm_addr_ok()`

```
int vm_addr_ok (
    struct VM * vm,
    vaddr_t addr,
    size_t len )
```

Check if memory address range is valid.

Validates that an address range falls within `VM` memory bounds.

Parameters

<i>vm</i>	Pointer to <code>VM</code> instance
<i>addr</i>	Virtual address to check
<i>len</i>	Length of memory range

Returns

1 if range is valid, 0 if invalid

References `VM::memory`, `vm`, and `VM_MEMORY_SIZE`.

Referenced by [defining_runtime_create\(\)](#), [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_forget\(\)](#), [dump_block_preview\(\)](#), [editor_word_s\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cfetchn\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [runtime_abortq\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_tick\(\)](#), [system_word_see\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_load_cell\(\)](#), [vm_ptr\(\)](#), [vm_store_cell\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the caller graph for this function:

7.159.1.4 vm_compile_call()

```
void vm_compile_call (
    VM * vm,
    word_func_t func )
```

References [entry](#), [VM::error](#), [func](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_word\(\)](#), and [vm_dictionary_find_by_func\(\)](#).

Referenced by [control_forth_again\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_if\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_until\(\)](#), [control_forth_while\(\)](#), [defining_word_does\(\)](#), and [system_word_abort_quote\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.5 vm_compile_exit()

```
void vm_compile_exit (
    VM * vm )
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_word\(\)](#), and [vm_find_word\(\)](#).

Here is the call graph for this function:

7.159.1.6 vm_compile_literal()

```
void vm_compile_literal (
    VM * vm,
    cell_t value )
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_word_literal\(\)](#), [dictionary_m_word_tick\(\)](#), [system_word_abort_quote\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.7 vm_compile_word()

```
void vm_compile_word (
    VM * vm,
    DictEntry * entry )
```

References [entry](#), [VM::error](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_align\(\)](#), and [vm_allot\(\)](#).

Referenced by [defining_word_compile\(\)](#), [defining_word_does\(\)](#), [string_word_s_quote\(\)](#), [vm_compile_call\(\)](#), [vm_compile_exit\(\)](#), [vm_compile_literal\(\)](#), [vm_exit_compile_mode\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.8 vm_enter_compile_mode()

```
void vm_enter_compile_mode (
    VM * vm,
    const char * name,
    size_t len )
```

References [VM::compiling_word](#), [VM::current_word_name](#), [VM::error](#), [execute_colon_word\(\)](#), [DictEntry::flags](#), [VM::here](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [name](#), [VM::state_addr](#), [VM::state_var](#), [vm](#), [vm_align\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_store_cell\(\)](#), [WORD_NAME_MAX](#), and [WORD_SMUDGED](#).

Referenced by [defining_word_colon\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.9 vm_exit_compile_mode()

```
void vm_exit_compile_mode (
    VM * vm )
```

References [VM::compiling_word](#), [VM::error](#), [DictEntry::flags](#), [VM::here](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_INTERPRET](#), [physics_metadata_refresh_state\(\)](#), [physics_metadata_set_mass\(\)](#), [VM::state_addr](#), [VM::state_var](#), [vm](#), [vm_compile_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_find_word\(\)](#), [vm_store_cell\(\)](#), and [WORD_COMPILED](#).

Referenced by [defining_word_semicolon\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.10 vm_get_base()

```
unsigned vm_get_base (
    const VM * vm )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), [vm_load_cell\(\)](#), and [VM_MEMORY_SIZE](#).

Referenced by [vm_parse_number\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.11 vm_interpret()

```
void vm_interpret (
    VM * vm,
    const char * input )
```

Interpret a string of Forth code.

Main interpretation loop that:

- Loads input into [VM](#) buffer
- Parses words
- Executes or compiles each word
- Handles numbers

Parameters

<i>vm</i>	Pointer to VM instance
<i>input</i>	String containing Forth code to interpret

References [HeartbeatState::check_counter](#), [VM::error](#), [VM::heartbeat](#), [HEARTBEAT_CHECK_FREQUENCY](#), [VM::input_buffer](#), [INPUT_BUFFER_SIZE](#), [VM::input_length](#), [VM::input_pos](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), [vm_heartbeat_run_cycle\(\)](#), [vm_interpret_word\(\)](#), [vm_parse_word\(\)](#), and [HeartbeatState::worker](#).

Referenced by [block_word_load\(\)](#), [block_word_next_block\(\)](#), [capsule_exec_hook\(\)](#), [capsule_exec_init\(\)](#), [check_physics_transparent\(\)](#), [main\(\)](#), [register_starforth_words\(\)](#), [run_doe_experiment\(\)](#), [run_single_test\(\)](#), [sk_repl\(\)](#), [starforth_word_init\(\)](#), [test_acl_adaptive_ttl\(\)](#), [test_acl_deny_aborts\(\)](#), [test_acl_pin_immutable\(\)](#), [test_acl_strict_mode\(\)](#), [test_acl_ttl_hot_path\(\)](#), [test_emergency_console_bypass\(\)](#), and [vm_repl\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.12 vm_interpret_word()

```
void vm_interpret_word (
    VM * vm,
```

```
const char * word_str,
size_t len )
```

References [acl_recheck\(\)](#), [VM::current_executing_entry](#), [VM::dict_lock](#), [VM::dsp](#), [VM::emergency_console](#), [entry](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::flags](#), [VM::heartbeat](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [VM::mode](#), [MODE_COMPILE](#), [name](#), [DictEntry::physics](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_apply_linear_decay\(\)](#), [physics_metadata_touch\(\)](#), [physics_on_lookup\(\)](#), [physics_post_execute\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [profiler_word_exit\(\)](#), [VM::rsp](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), [vm_compile_literal\(\)](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), [vm_monotonic_ns\(\)](#), [vm_parse_number\(\)](#), [vm_push\(\)](#), [vm_vocabulary_find_word\(\)](#), [WORD_IMMEDIATE](#), [HeartbeatState::words_executed](#), and [VM::zuse_session](#).

Referenced by [vm_interpret\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.13 vm_load_cell()

```
cell_t vm_load_cell (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::error](#), [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [block_word_next_block\(\)](#), [block_word_update\(\)](#), [current_scr\(\)](#), [editor_word_edit\(\)](#), [mark_buffer_dirty\(\)](#), [memory_word_2fetch\(\)](#), [memory_word_fetch\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_plus_store\(\)](#), and [vm_get_base\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.14 vm_load_u8()

```
uint8_t vm_load_u8 (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::error](#), [vm](#), and [vm_ptr\(\)](#).

Referenced by [memory_word_cfetch\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_count\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), and [string_word_skip\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.15 vm_make_immediate()

```
void vm_make_immediate (
    VM * vm )
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [DictEntry::name](#), [DictEntry::name_len](#), [physics_metadata_refresh_state\(\)](#), [vm](#), and [WORD_IMMEDIATE](#).

Referenced by [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_string_words\(\)](#), and [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.16 vm_parse_number()

```
int vm_parse_number (
    VM * vm,
    const char * s,
    cell_t * out )
```

Parse string as number in current base.

Attempts to parse string as number using [VM](#)'s current number base. Handles optional sign prefix.

Parameters

<i>vm</i>	Pointer to VM instance
<i>s</i>	String to parse
<i>out</i>	Pointer to store parsed value

Returns

1 on success, 0 on parse failure

References [base](#), [vm](#), [vm_assert_interpreter_enabled\(\)](#), and [vm_get_base\(\)](#).

Referenced by [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.17 vm_parse_word()

```
int vm_parse_word (
    VM * vm,
    char * word,
    size_t max_len )
```

Parse next word from input buffer.

Skips leading whitespace and extracts next word delimited by whitespace.

Parameters

<i>vm</i>	Pointer to VM instance
<i>word</i>	Buffer to store parsed word
<i>max_len</i>	Maximum length of word buffer

Returns

Length of parsed word or 0 if no word found

References [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [vm](#), and [vm_assert_interpreter_enabled\(\)](#).

Referenced by [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_variable\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_word_forget\(\)](#), [forth_paren\(\)](#), [system_word_see\(\)](#), [vm_interpret\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.18 vm_ptr()

```
uint8_t * vm_ptr (
    struct VM * vm,
    vaddr_t addr )
```

References [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [defining_runtime_dodoes\(\)](#), [defining_runtime_does_rt\(\)](#), [dump_block_preview\(\)](#), [editor_word_s\(\)](#), [execute_colon_word\(\)](#), [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TO](#), [hash_colon_body\(\)](#), [memory_word_erase\(\)](#), [memory_word_fill\(\)](#), [memory_word_move\(\)](#), [string_word_convert\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_number\(\)](#), [string_word_tick\(\)](#), [string_word_word\(\)](#), [system_word_see\(\)](#), [vm_load_u8\(\)](#), [vm_store_u8\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.19 vm_set_base()

```
void vm_set_base (
    VM * vm,
    unsigned b )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Here is the call graph for this function:

7.159.1.20 vm_store_cell()

```
void vm_store_cell (
    struct VM * vm,
    vaddr_t addr,
    cell_t v )
```

References [VM::error](#), [VM::memory](#), [vm](#), and [vm_addr_ok\(\)](#).

Referenced by [defining_word_left_bracket\(\)](#), [defining_word_right_bracket\(\)](#), [dictionary_word_2comma\(\)](#), [dictionary_word_comma\(\)](#), [editor_word_edit\(\)](#), [format_word_decimal\(\)](#), [format_word_hex\(\)](#), [format_word_octal\(\)](#), [memory_word_2store\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [set_scr\(\)](#), [set_scr\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_enter_compile_mode\(\)](#), [vm_exit_compile_mode\(\)](#), [vm_init\(\)](#), [vm_init_with_host\(\)](#), [vm_set_base\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.159.1.21 [vm_store_u8\(\)](#)

```
void vm_store_u8 (
    struct VM * vm,
    vaddr_t addr,
    uint8_t v )
```

References [VM::error](#), [vm](#), and [vm_ptr\(\)](#).

Referenced by [dictionary_word_c_comma\(\)](#), [memory_word_cstore\(\)](#), [string_word_blank\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), and [string_word_s_quote\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.160 [src/vm_api.c](#) File Reference

```
#include "../include/vm_api.h"
#include "../include/vm.h"
#include <string.h>
```

Include dependency graph for [vm_api.c](#):

Functions

- int [vm_input_ensure](#) ([VM](#) *[vm](#))
Ensure input buffers are allocated.
- unsigned char * [vm_input_tib](#) ([VM](#) *[vm](#))
Get pointer to terminal input buffer (TIB)
- [cell_t](#) * [vm_input_in](#) ([VM](#) *[vm](#))
Get pointer to input offset variable.
- [cell_t](#) * [vm_input_span](#) ([VM](#) *[vm](#))
Get pointer to input span variable.
- void [vm_input_load_line](#) ([VM](#) *[vm](#), const char *src, size_t n)
Load input line into TIB.
- void [vm_input_source](#) ([VM](#) *[vm](#), [cell_t](#) *out_addr, [cell_t](#) *out_len)
Get current input source buffer info.

Stack Operations

- void [vm_api_push](#) ([VM](#) *[vm](#), intptr_t v)
Push value onto data stack.
- intptr_t [vm_api_pop](#) ([VM](#) *[vm](#))
Pop value from data stack.
- void [vm_api_rpush](#) ([VM](#) *[vm](#), intptr_t v)
Push value onto return stack.
- intptr_t [vm_api_rpop](#) ([VM](#) *[vm](#))
Pop value from return stack.

Dictionary Operations

- void * [vm_api_create_word](#) ([VM](#) *[vm](#), const char *n, size_t len, [word_func_t](#) fn)
Create new word in dictionary.

- void * [vm_api_find_word](#) (VM *vm, const char *n, size_t len)
Find word in dictionary.
- void [vm_api_hide_word](#) (VM *vm)
Hide last defined word.
- void [vm_api_smudge_word](#) (VM *vm)
Toggle smudge bit of last word.

Memory Operations

- [cell_t vm_addr_from_ptr](#) (VM *vm, void *p)
Convert native pointer to Forth memory address.

7.160.1 Function Documentation

7.160.1.1 vm_api_create_word()

```
void * vm_api_create_word (
    VM * vm,
    const char * n,
    size_t len,
    word_func_t fn )
```

Create new word in dictionary.

Parameters

<i>vm</i>	VM instance
<i>n</i>	Word name
<i>len</i>	Name length
<i>fn</i>	Word implementation function

Returns

Pointer to created dictionary entry

References [vm](#), and [vm_create_word\(\)](#).
Here is the call graph for this function:

7.160.1.2 vm_api_find_word()

```
void * vm_api_find_word (
    VM * vm,
    const char * n,
    size_t len )
```

Find word in dictionary.

Parameters

<i>vm</i>	VM instance
<i>n</i>	Word name to find
<i>len</i>	Name length

Returns

Pointer to found dictionary entry or NULL

References [vm](#), and [vm_find_word\(\)](#).
Here is the call graph for this function:

7.160.1.3 vm_api_hide_word()

```
void vm_api_hide_word (
    VM * vm )
```

Hide last defined word.

Parameters

<i>vm</i>	VM instance
-----------	-------------

References [vm](#), and [vm_hide_word\(\)](#).
Here is the call graph for this function:

7.160.1.4 vm_api_pop()

```
intptr_t vm_api_pop (
    VM * vm )
```

Pop value from data stack.

Parameters

<i>vm</i>	VM instance
-----------	-------------

Returns

Popped value

References [vm](#), and [vm_pop\(\)](#).
Here is the call graph for this function:

7.160.1.5 vm_api_push()

```
void vm_api_push (
    VM * vm,
    intptr_t v )
```

Push value onto data stack.

Parameters

<i>vm</i>	VM instance
<i>v</i>	Value to push

References [vm](#), and [vm_push\(\)](#).
Here is the call graph for this function:

7.160.1.6 vm_api_rpop()

```
intptr_t vm_api_rpop (
    VM * vm )
```

Pop value from return stack.

Parameters

<i>vm</i>	VM instance
-----------	-------------

Returns

Popped value

References [vm](#), and [vm_rpop\(\)](#).

Here is the call graph for this function:

7.160.1.7 vm_api_rpush()

```
void vm_api_rpush (
    VM * vm,
    intptr_t v )
```

Push value onto return stack.

Parameters

<i>vm</i>	VM instance
<i>v</i>	Value to push

References [vm](#), and [vm_rpush\(\)](#).

Here is the call graph for this function:

7.160.1.8 vm_api_smudge_word()

```
void vm_api_smudge_word (
    VM * vm )
```

Toggle smudge bit of last word.

Parameters

<i>vm</i>	VM instance
-----------	-------------

References [vm](#), and [vm_smudge_word\(\)](#).

Here is the call graph for this function:

7.161 src/starkernel/vm/vm_bootstrap.c File Reference

```
#include "../include/vm.h"
#include "../include/log.h"
#include "../include/word_registry.h"
#include "../include/vm_debug.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/rolling_window_of_truth.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/ssm_jacquard.h"
#include "word_source/include/vocabulary_words.h"
#include "vm_internal.h"
#include <string.h>
#include <errno.h>
```

Include dependency graph for vm_bootstrap.c:

Functions

- static void [vm_bootstrap_scr](#) (VM *vm)
- void [vm_bootstrap_root_vocabulary](#) (VM *vm, const char *name)
- void [vm_init](#) (VM *vm)

Initialize a new VM instance.
- void [vm_init_with_host](#) (VM *vm, const VMHostServices *host)

Initialize [VM](#) with explicit host services.

7.161.1 Function Documentation

7.161.1.1 [vm_bootstrap_root_vocabulary\(\)](#)

```
void vm_bootstrap_root_vocabulary (
    VM * vm,
    const char * name )
```

References [console_println\(\)](#), [console_puts\(\)](#), [entry](#), [VM::error](#), [VMHostServices::is_executable_ptr](#), [LOG_ERROR](#), [log_message\(\)](#), [name](#), [VMHostServices::panic](#), [vm](#), [vm_find_word\(\)](#), [vm_host\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_definitions\(\)](#).

Here is the call graph for this function:

7.161.1.2 [vm_bootstrap_scr\(\)](#)

```
static void vm_bootstrap_scr (
    VM * vm ) [static]
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::scr_addr](#), [vm](#), [vm_align\(\)](#), [vm_allot\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.161.1.3 [vm_init\(\)](#)

```
void vm_init (
    VM * vm )
```

Initialize a new [VM](#) instance.

Parameters

<i>vm</i>	Pointer to VM structure to initialize
-----------	---

7.161.1.4 [vm_init_with_host\(\)](#)

```
void vm_init_with_host (
    VM * vm,
    const VMHostServices * host )
```

Initialize [VM](#) with explicit host services.

Parameters

<i>vm</i>	Pointer to VM structure to initialize
<i>host</i>	Host services (allocator/time). If NULL, uses build default.

References [VM::abort_requested](#), [VM::base_addr](#), [HeartbeatState::check_counter](#), [VM::compiling_word](#), [console_println\(\)](#), [VM::current_executing_entry](#), [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [VM::dict_lock](#), [dict_update_heat_percentiles\(\)](#), [VM::dsp](#), [RollingWindowOfTruth::effective_window_size](#), [VM::error](#), [VM::exit_colon](#), [VM::halted](#), [VM::heartbeat](#), [VM::heartbeat_decay_cursor_id](#), [HeartbeatState::heartbeat_enabled](#), [heartbeat_publish_snapshot\(\)](#), [HEARTBEAT_TICK_NS](#), [VM::here](#), [VM::host](#), [VM::hotwords_cache](#), [hotwords_cache_init\(\)](#), [VM::input_length](#), [VM::input_pos](#), [VM::interpreter_enabled](#), [VM::last_bucket_reorg_ns](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [VM::last_decay_check_ns](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [VM::lookup_strategy](#), [VM::memory](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::next_word_id](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::recycled_word_id_count](#), [register_forth79_words\(\)](#), [VM::rolling_window](#), [rolling_window_init\(\)](#), [VM::rsp](#), [sf_mutex_destroy\(\)](#), [sf_mutex_init\(\)](#), [HeartbeatState::snapshot_index](#), [VM::ssm_config](#), [ssm_l8_init\(\)](#), [VM::ssm_l8_state](#), [SSM_MODE_C0](#), [VM::stale_word_count_at_check](#), [VM::state_addr](#), [VM::state_var](#), [PipelineGlobalMetrics::suggested_next_size](#), [HeartbeatState::tick_count](#), [HeartbeatWorker::tick_ns](#), [HeartbeatState::tick_target_ns](#),

[VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_allot\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_debug_install_signal_handlers\(\)](#), [vm_debug_set_current_vm\(\)](#), [vm_default_host_services\(\)](#), [vm_host_alloc\(\)](#), [vm_host_calloc\(\)](#), [vm_host_free\(\)](#), [VM_MEMORY_SIZE](#), [vm_reset_hosted_fake_ns\(\)](#), [vm_set_base\(\)](#), [vm_store_cell\(\)](#), [PipelineGlobalMetrics::window_tuning_checks](#), [WORD_ID_INVALID](#), and [HeartbeatState::worker](#).

Referenced by [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.162 src/vm_bootstrap.c File Reference

```
#include "vm_internal.h"
#include "../include/log.h"
#include "../include/word_registry.h"
#include "../include/vm_debug.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/rolling_window_of_truth.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/ssm_jacquard.h"
#include "../include/platform_alloc.h"
#include <string.h>
```

Include dependency graph for [vm_bootstrap.c](#):

Functions

- static void [vm_bootstrap_scr](#) ([VM](#) *[vm](#))
- void [vm_init](#) ([VM](#) *[vm](#))
Initialize a new virtual machine instance.
- void [vm_cleanup](#) ([VM](#) *[vm](#))
Clean up and free [VM](#) resources.

7.162.1 Function Documentation

7.162.1.1 [vm_bootstrap_scr\(\)](#)

```
static void vm_bootstrap_scr (
    VM * vm ) [static]
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::scr_addr](#), [vm](#), [vm_addr_ok\(\)](#), [vm_align\(\)](#), [vm_allot\(\)](#), and [vm_store_cell\(\)](#).

Here is the call graph for this function:

7.162.1.2 [vm_cleanup\(\)](#)

```
void vm_cleanup (
    VM * vm )
```

Clean up and free [VM](#) resources.

Frees allocated memory and resets [VM](#) state.

Parameters

vm	Pointer to VM structure to clean up
--------------------	---

References [VM::dict_lock](#), [VM::heartbeat](#), [VM::here](#), [VM::hotwords_cache](#), [hotwords_cache_cleanup\(\)](#), [VM::memory](#), [VM::rolling_window](#), [rolling_window_cleanup\(\)](#), [sf_free\(\)](#), [sf_mutex_destroy\(\)](#), [VM::ssm_config](#), [VM::ssm_l8_state](#), [HeartbeatWorker::stop_requested](#), [VM::tuning_lock](#), [vm](#), [vm_heartbeat_stop_thread\(\)](#), [vm_host_free\(\)](#), and [HeartbeatState::worker](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), and [cleanup_and_exit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.162.1.3 vm_init()

```
void vm_init (
    VM * vm )
```

Initialize a new virtual machine instance.

Initialize a new [VM](#) instance.

Allocates memory and initializes all [VM](#) structures including:

- Memory array
- Data and return stacks
- System variables (SCR, STATE, BASE)
- Dictionary
- Forth-79 wordset

Parameters

<i>vm</i>	Pointer to VM structure to initialize
-----------	---

References [VM::abort_requested](#), [VM::base_addr](#), [HeartbeatState::check_counter](#), [VM::compiling_word](#), [VM::current_executing_entry](#), [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [VM::dict_lock](#), [dict_update_heat_percentiles\(\)](#), [VM::dsp](#), [RollingWindowOfTruth::effective_window_size](#), [VM::error](#), [VM::exit_colon](#), [VM::halted](#), [VM::heartbeat](#), [VM::heartbeat_decay_cursor_id](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_TICK_NS](#), [VM::here](#), [VM::hotwords_cache](#), [hotwords_cache_init\(\)](#), [VM::input_length](#), [VM::input_pos](#), [VM::last_bucket_reorg_ns](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [VM::last_decay_check_ns](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::lookup_strategy](#), [VM::memory](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::next_word_id](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::recycled_word_id_count](#), [register_forth79_words\(\)](#), [VM::rolling_window](#), [rolling_window_init\(\)](#), [VM::rsp](#), [VM::scr_addr](#), [sf_malloc\(\)](#), [sf_mutex_destroy\(\)](#), [sf_mutex_init\(\)](#), [HeartbeatState::snapshot_index](#), [VM::ssm_config](#), [ssm_l8_init\(\)](#), [VM::ssm_l8_state](#), [SSM_MODE_C0](#), [VM::stale_word_count_at_check](#), [VM::state_addr](#), [VM::state_var](#), [PipelineGlobalMetrics::suggested_next_size](#), [HeartbeatState::tick_count](#), [HeartbeatState::tick_target_ns](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_debug_install_signal_handlers\(\)](#), [vm_debug_set_current_vm\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_start_thread\(\)](#), [vm_init_with_host\(\)](#), [VM_MEMORY_SIZE](#), [vm_set_base\(\)](#), [vm_store_cell\(\)](#), [PipelineGlobalMetrics::window_tuning_checks](#), [WORD_ID_INVALID](#), and [HeartbeatState::worker](#).

Referenced by [capsule_vm_alloc_hook\(\)](#), and [init_vm_and_subsystem\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.163 src/vm_debug.c File Reference

```
#include <signal.h>
#include "../include/vm.h"
#include "../include/vm_debug.h"
#include <stdio.h>
#include <string.h>
```

Include dependency graph for [vm_debug.c](#):

Functions

- void [vm_debug_set_current_vm](#) ([VM](#) *[vm](#))
Sets the current [VM](#) instance for debug operations.
- static unsigned [dbg_get_base](#) (const [VM](#) *[vm](#))
Safely retrieves the current number base from [VM](#).
- static const char * [mode_str](#) (int m)

Converts VM mode to string representation.

- static void `dump_block_preview` (const VM *vm, cell_t scr)
Dumps preview of a block's contents.
- void `vm_debug_dump_state` (const VM *vm, const char *reason)
Dumps the current state of the VM for debugging.
- static void `sig_handler` (int sig)
Signal handler for crash debugging.
- void `vm_debug_install_signal_handlers` (void)
Installs signal handlers for crash debugging.

Variables

- static VM * `g_vm` = NULL

7.163.1 Function Documentation

7.163.1.1 dbg_get_base()

```
static unsigned dbg_get_base (
    const VM * vm ) [static]
```

Safely retrieves the current number base from VM.

Parameters

<code>vm</code>	Pointer to the VM instance
-----------------	----------------------------

Returns

Current number base (2-36) or 10 if invalid

References `VM::base`, `VM::base_addr`, `VM::memory`, `vm`, and `VM_MEMORY_SIZE`.

Referenced by `vm_debug_dump_state()`.

Here is the caller graph for this function:

7.163.1.2 dump_block_preview()

```
static void dump_block_preview (
    const VM * vm,
    cell_t scr ) [static]
```

Dumps preview of a block's contents.

Parameters

<code>vm</code>	Pointer to the VM instance
<code>scr</code>	Block number to preview

References `BLOCK_SIZE`, `MAX_BLOCKS`, `vm`, `vm_addr_ok()`, and `vm_ptr()`.

Referenced by `vm_debug_dump_state()`.

Here is the call graph for this function: Here is the caller graph for this function:

7.163.1.3 mode_str()

```
static const char * mode_str (
    int m ) [static]
```

Converts VM mode to string representation.

Parameters

<i>m</i>	The VM mode value
----------	-----------------------------------

Returns

String representing the mode

References [MODE_COMPILE](#), and [MODE_INTERPRET](#).

Referenced by [cli_parse\(\)](#), and [vm_debug_dump_state\(\)](#).

Here is the caller graph for this function:

7.163.1.4 sig_handler()

```
static void sig_handler (
    int sig ) [static]
```

Signal handler for crash debugging.

Parameters

<i>sig</i>	Signal number that was caught
------------	-------------------------------

References [g_vm](#), [name](#), and [vm_debug_dump_state\(\)](#).

Referenced by [vm_debug_install_signal_handlers\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.163.1.5 vm_debug_dump_state()

```
void vm_debug_dump_state (
    const VM * vm,
    const char * reason )
```

Dumps the current state of the [VM](#) for debugging.

Manually dumps the state of a [VM](#) instance.

Parameters

<i>vm</i>	Pointer to the VM instance to dump
<i>reason</i>	Optional string describing the reason for the dump

References [base](#), [VM::current_executing_entry](#), [dbg_get_base\(\)](#), [VM::dsp](#), [dump_block_preview\(\)](#), [VM::error](#), [VM::here](#), [VM::ip](#), [VM::memory](#), [VM::mode](#), [mode_str\(\)](#), [DictEntry::name](#), [VM::rsp](#), [VM::scr_addr](#), [vm](#), and [VM_MEMORY_SIZE](#).

Referenced by [dump_and_die\(\)](#), and [sig_handler\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.163.1.6 vm_debug_install_signal_handlers()

```
void vm_debug_install_signal_handlers (
    void )
```

Installs signal handlers for crash debugging.

Installs signal handlers for SIGSEGV and SIGABRT.

Sets up handlers for SIGSEGV and SIGABRT to dump [VM](#) state on crash

References [sig_handler\(\)](#).

Referenced by [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.163.1.7 vm_debug_set_current_vm()

```
void vm_debug_set_current_vm (
    VM * vm )
```

Sets the current [VM](#) instance for debug operations.
Sets the [VM](#) instance to dump state for when signals occur.

Parameters

vm	Pointer to the VM instance to be monitored
--------------------	--

References [g_vm](#), and [vm](#).
Referenced by [main\(\)](#), [vm_init\(\)](#), and [vm_init_with_host\(\)](#).
Here is the caller graph for this function:

7.163.2 Variable Documentation

7.163.2.1 g_vm

```
VM* g_vm = NULL [static]
```

Referenced by [sig_handler\(\)](#), and [vm_debug_set_current_vm\(\)](#).

7.164 src/starkernel/vm/vm_internal.h File Reference

```
#include "../include/vm.h"
#include "../include/vm_host.h"
```

Include dependency graph for vm_internal.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [HeartbeatWorker](#)

Macros

- #define [PARITY_MODE](#) 0
- #define [SK_PARITY_DEBUG](#) 0
- #define [HEARTBEAT_HAS_THREADS](#) 0
- #define [HEARTBEAT_DECAY_BATCH](#) 64u

Typedefs

- typedef struct [HeartbeatWorker](#) [HeartbeatWorker](#)

Functions

- const [VMHostServices](#) * [vm_default_host_services](#) (void)
- const [VMHostServices](#) * [vm_host](#) (const [VM](#) *vm)
- void * [vm_host_alloc](#) ([VM](#) *vm, size_t size, size_t align)
- void * [vm_host_calloc](#) ([VM](#) *vm, size_t n, size_t size)
- void [vm_host_free](#) ([VM](#) *vm, void *ptr)
- void [vm_reset_hosted_fake_ns](#) ([VM](#) *vm)
- uint64_t [vm_monotonic_ns](#) ([VM](#) *vm)
- unsigned [vm_get_base](#) (const [VM](#) *vm)
- void [vm_set_base](#) ([VM](#) *vm, unsigned b)
- uint32_t [heartbeat_snapshot_index_load](#) (const volatile uint32_t *ptr)
- void [heartbeat_snapshot_index_store](#) (volatile uint32_t *ptr, uint32_t value)
- void [heartbeat_publish_snapshot](#) ([VM](#) *vm)
- void [vm_heartbeat_run_cycle](#) ([VM](#) *vm)

7.164.1 Macro Definition Documentation

7.164.1.1 HEARTBEAT_DECAY_BATCH

```
#define HEARTBEAT_DECAY_BATCH 64u
```

7.164.1.2 HEARTBEAT_HAS_THREADS

```
#define HEARTBEAT_HAS_THREADS 0
```

7.164.1.3 PARITY_MODE

```
#define PARITY_MODE 0
```

7.164.1.4 SK_PARITY_DEBUG

```
#define SK_PARITY_DEBUG 0
```

7.164.2 Typedef Documentation

7.164.2.1 HeartbeatWorker

```
typedef struct HeartbeatWorker HeartbeatWorker
```

7.164.3 Function Documentation

7.164.3.1 heartbeat_publish_snapshot()

```
void heartbeat_publish_snapshot (
    VM * vm )
```

References [HeartbeatSnapshot::decay_slope_q48](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [heartbeat_snapshot_index_store\(\)](#), [HeartbeatSnapshot::hot_word_count](#), [VM::hot_word_count_at_check](#), [HeartbeatSnapshot::published_ns](#), [HeartbeatSnapshot::published_tick](#), [VM::rolling_window](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), [HeartbeatSnapshot::stale_word_count](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [HeartbeatSnapshot::total_heat](#), [VM::total_heat_at_last_check](#), [vm](#), [vm_monotonic_ns\(\)](#), and [HeartbeatSnapshot::window_width](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.2 heartbeat_snapshot_index_load()

```
uint32_t heartbeat_snapshot_index_load (
    const volatile uint32_t * ptr )
```

Referenced by [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), and [vm_snapshot_read\(\)](#).

Here is the caller graph for this function:

7.164.3.3 heartbeat_snapshot_index_store()

```
void heartbeat_snapshot_index_store (
    volatile uint32_t * ptr,
    uint32_t value )
```

Referenced by [heartbeat_publish_snapshot\(\)](#), and [vm_heartbeat_publish_snapshot\(\)](#).

Here is the caller graph for this function:

7.164.3.4 vm_default_host_services()

```
const VMHostServices * vm_default_host_services (
    void )
```

References [hosted_services](#), and [sk_host_services\(\)](#).

Referenced by [vm_host\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.5 vm_get_base()

```
unsigned vm_get_base (
    const VM * vm )
```

7.164.3.6 vm_heartbeat_run_cycle()

```
void vm_heartbeat_run_cycle (
    VM * vm )
```

References [dict_adaptive_optimization_pass\(\)](#), [VM::heartbeat](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_emit_tick_row\(\)](#), [HeartbeatState::heartbeat_enabled](#), [heartbeat_publish_snapshot\(\)](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [vm](#), [vm_heartbeat_update_l8\(\)](#), [vm_monotonic_ns\(\)](#), [vm_tick\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Referenced by [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), and [vm_interpret\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.7 vm_host()

```
const VMHostServices * vm_host (
    const VM * vm )
```

References [VM::host](#), [vm](#), and [vm_default_host_services\(\)](#).

Referenced by [vm_assert_interpreter_enabled\(\)](#), [vm_bootstrap_root_vocabulary\(\)](#), [vm_host_alloc\(\)](#), [vm_host_free\(\)](#), and [vm_monotonic_ns\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.8 vm_host_alloc()

```
void * vm_host_alloc (
    VM * vm,
    size_t size,
    size_t align )
```

References [VMHostServices::alloc](#), [vm](#), and [vm_host\(\)](#).

Referenced by [vm_host_calloc\(\)](#), [vm_init_with_host\(\)](#), and [vm_tick_inference_engine\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.9 vm_host_calloc()

```
void * vm_host_calloc (
    VM * vm,
    size_t n,
    size_t size )
```

References [vm](#), and [vm_host_alloc\(\)](#).

Referenced by [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.10 vm_host_free()

```
void vm_host_free (
    VM * vm,
    void * ptr )
```

References [VMHostServices::free](#), [vm](#), and [vm_host\(\)](#).

Referenced by [vm_cleanup\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.11 vm_monotonic_ns()

```
uint64_t vm_monotonic_ns (
    VM * vm )
```

References [VMHostServices::monotonic_ns](#), [vm](#), and [vm_host\(\)](#).

Referenced by [execute_colon_word\(\)](#), [heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_run_cycle\(\)](#), and [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.164.3.12 vm_reset_hosted_fake_ns()

```
void vm_reset_hosted_fake_ns (
    VM * vm )
```

References [VM::host](#), [hosted_fake_ns](#), [hosted_services](#), [PARITY_MODE](#), and [vm](#).

Referenced by [vm_init_with_host\(\)](#).

Here is the caller graph for this function:

7.164.3.13 vm_set_base()

```
void vm_set_base (
    VM * vm,
    unsigned b )
```

7.165 src/vm_internal.h File Reference

```
#include "../include/vm.h"
#include "../include/platform_time.h"
#include <stdint.h>
```

Include dependency graph for vm_internal.h: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [HeartbeatWorker](#)

Macros

- #define [HEARTBEAT_HAS_THREADS](#) 0
- #define [HEARTBEAT_DECAY_BATCH](#) 64u

Typedefs

- typedef struct [HeartbeatWorker](#) [HeartbeatWorker](#)

Functions

- static uint32_t [heartbeat_snapshot_index_load](#) (const volatile uint32_t *ptr)
- static void [heartbeat_snapshot_index_store](#) (volatile uint32_t *ptr, uint32_t value)
- void [execute_colon_word](#) (VM *vm)
- void [vm_tick](#) (VM *vm)
 - Central heartbeat dispatcher for time-driven tuning operations.*
- void [vm_tick_window_tuner](#) (VM *vm)
 - Loop #5: Context-aware window tuning via binary chop search.*
- void [vm_tick_slope_validator](#) (VM *vm)
 - Loop #3: Heat decay slope validation via periodic measurement.*
- void [vm_tick_inference_engine](#) (VM *vm)
 - Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.*
- void [vm_heartbeat_run_cycle](#) (VM *vm)
- void [vm_snapshot_read](#) (const VM *vm, [HeartbeatSnapshot](#) *out_snapshot)
- void [vm_heartbeat_start_thread](#) (VM *vm)
- void [vm_heartbeat_stop_thread](#) (VM *vm)
- void [vm_heartbeat_publish_snapshot](#) (VM *vm)
- void [vm_init](#) (VM *vm)
 - Initialize a new VM instance.*
- void [vm_cleanup](#) (VM *vm)
 - Clean up and free VM resources.*
- unsigned [vm_get_base](#) (const VM *vm)
- void [vm_set_base](#) (VM *vm, unsigned b)

7.165.1 Macro Definition Documentation

7.165.1.1 HEARTBEAT_DECAY_BATCH

```
#define HEARTBEAT_DECAY_BATCH 64u
```

7.165.1.2 HEARTBEAT_HAS_THREADS

```
#define HEARTBEAT_HAS_THREADS 0
```

7.165.2 Typedef Documentation

7.165.2.1 HeartbeatWorker

```
typedef struct HeartbeatWorker HeartbeatWorker
```

7.165.3 Function Documentation

7.165.3.1 execute_colon_word()

```
void execute_colon_word (
    VM * vm )
```

References [VM::abort_requested](#), [DictEntry::acl_allow](#), [acl_recheck\(\)](#), [DictEntry::acl_ttl](#), [VM::current_executing_entry](#), [VM::dsp](#), [VM::emergency_console](#), [entry](#), [VM::error](#), [VM::exit_colon](#), [DictEntry::func](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [DictEntry::name](#), [DictEntry::name_len](#), [physics_post_execute\(\)](#), [physics_pre_execute\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [VM::rsp](#), [vm](#), [vm_dictionary_get_data_field\(\)](#), [vm_ptr\(\)](#), [vm_rpop\(\)](#), [vm_rpush\(\)](#), and [VM::zuse_session](#).

Referenced by [vm_enter_compile_mode\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.2 heartbeat_snapshot_index_load()

```
static uint32_t heartbeat_snapshot_index_load (
    const volatile uint32_t * ptr ) [inline], [static]
```

7.165.3.3 heartbeat_snapshot_index_store()

```
static void heartbeat_snapshot_index_store (
    volatile uint32_t * ptr,
    uint32_t value ) [inline], [static]
```

7.165.3.4 vm_cleanup()

```
void vm_cleanup (
    VM * vm )
```

Clean up and free [VM](#) resources.

Frees allocated memory and resets [VM](#) state.

Parameters

vm	Pointer to VM structure to clean up
--------------------	---

References [VM::dict_lock](#), [VM::here](#), [VM::hotwords_cache](#), [hotwords_cache_cleanup\(\)](#), [VM::memory](#), [VM::rolling_window](#), [rolling_window_cleanup\(\)](#), [sf_free\(\)](#), [sf_mutex_destroy\(\)](#), [VM::ssm_config](#), [VM::ssm_l8_state](#), [VM::tuning_lock](#), [vm](#), and [vm_heartbeat_stop_thread\(\)](#).

Here is the call graph for this function:

7.165.3.5 vm_get_base()

```
unsigned vm_get_base (
    const VM * vm )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), [vm_load_cell\(\)](#), and [VM_MEMORY_SIZE](#).

Referenced by [vm_parse_number\(\)](#), and [vm_parse_number\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.6 vm_heartbeat_publish_snapshot()

```
void vm_heartbeat_publish_snapshot (
    VM * vm )
```

References [HeartbeatSnapshot::actual_window_size](#), [HeartbeatSnapshot::decay_slope_q48](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [heartbeat_snapshot_index_store\(\)](#), [HeartbeatSnapshot::hot_word_count](#), [VM::hot_word_count_at_check](#), [HeartbeatSnapshot::published_ns](#), [HeartbeatSnapshot::publish](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [sf_monotonic_ns\(\)](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), [HeartbeatSnapshot::stale_word_count](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [RollingWindowOfTruth::total_ex](#), [HeartbeatSnapshot::total_heat](#), [VM::total_heat_at_last_check](#), [vm](#), and [HeartbeatSnapshot::window_width](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.7 vm_heartbeat_run_cycle()

```
void vm_heartbeat_run_cycle (
    VM * vm )
```

References [dict_adaptive_optimization_pass\(\)](#), [VM::heartbeat](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_emit_tick_row\(\)](#), [HeartbeatState::heartbeat_enabled](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_tick\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Here is the call graph for this function:

7.165.3.8 vm_heartbeat_start_thread()

```
void vm_heartbeat_start_thread (
    VM * vm )
```

References [VM::heartbeat](#), [HEARTBEAT_TICK_NS](#), [log_message\(\)](#), [LOG_WARN](#), [sf_calloc\(\)](#), [sf_free\(\)](#), [HeartbeatWorker::tick_ns](#), [vm](#), and [HeartbeatState::worker](#).

Referenced by [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.9 vm_heartbeat_stop_thread()

```
void vm_heartbeat_stop_thread (
    VM * vm )
```

References [VM::heartbeat](#), [sf_free\(\)](#), [HeartbeatWorker::stop_requested](#), [vm](#), and [HeartbeatState::worker](#).

Referenced by [vm_cleanup\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.10 vm_init()

```
void vm_init (
    VM * vm )
```

Initialize a new [VM](#) instance.

Parameters

vm	Pointer to VM structure to initialize
--------------------	---

Initialize a new [VM](#) instance.

Allocates memory and initializes all [VM](#) structures including:

- Memory array
- Data and return stacks
- System variables (SCR, STATE, BASE)

- Dictionary
- Forth-79 wordset

Parameters

<code>vm</code>	Pointer to VM structure to initialize
-----------------	---

References [VM::abort_requested](#), [VM::base_addr](#), [HeartbeatState::check_counter](#), [VM::compiling_word](#), [VM::current_executing_entry](#), [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [VM::dict_lock](#), [dict_update_heat_percentiles\(\)](#), [VM::dsp](#), [RollingWindowOfTruth::effective_window_size](#), [VM::error](#), [VM::exit_colon](#), [VM::halted](#), [VM::heartbeat](#), [VM::heartbeat_decay_cursor_id](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_TICK_NS](#), [VM::here](#), [VM::hotwords_cache](#), [hotwords_cache_init\(\)](#), [VM::input_length](#), [VM::input_pos](#), [VM::last_bucket_reorg_ns](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [VM::last_decay_check_ns](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::lookup_strategy](#), [VM::memory](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::next_word_id](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::recycled_word_id_count](#), [register_forth79_words\(\)](#), [VM::rolling_window](#), [rolling_window_init\(\)](#), [VM::rsp](#), [VM::scr_addr](#), [sf_malloc\(\)](#), [sf_mutex_destroy\(\)](#), [sf_mutex_init\(\)](#), [HeartbeatState::snapshot_index](#), [VM::ssm_config](#), [ssm_l8_init\(\)](#), [VM::ssm_l8_state](#), [SSM_MODE_C0](#), [VM::stale_word_count_at_check](#), [VM::state_addr](#), [VM::state_var](#), [PipelineGlobalMetrics::suggested_next_size](#), [HeartbeatState::tick_count](#), [HeartbeatState::tick_target_ns](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_bootstrap_scr\(\)](#), [vm_debug_install_signal_handlers\(\)](#), [vm_debug_set_current_vm\(\)](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_start_thread\(\)](#), [VM_MEMORY_SIZE](#), [vm_set_base\(\)](#), [vm_store_cell\(\)](#), [PipelineGlobalMetrics::window_tuning_checks](#), [WORD_ID_INVALID](#), and [HeartbeatState::worker](#).

Here is the call graph for this function:

7.165.3.11 `vm_set_base()`

```
void vm_set_base (
    VM * vm,
    unsigned b )
```

References [VM::base](#), [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.12 `vm_snapshot_read()`

```
void vm_snapshot_read (
    const VM * vm,
    HeartbeatSnapshot * out_snapshot )
```

References [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), and [vm](#).

Here is the call graph for this function:

7.165.3.13 `vm_tick()`

```
void vm_tick (
    VM * vm )
```

Central heartbeat dispatcher for time-driven tuning operations.

Aggregates all periodic optimization tasks (Loop #3 and Loop #5) into one place. Designed as plugin architecture - new tuning operations can be added as plugins.

Options for integration:

- Synchronous (now): Called from main execution loop, every N executions
- Background thread (future): Runs in separate thread, decoupled from [VM](#) execution

Parameters

<code>vm</code>	Pointer to VM instance
-----------------	--

References [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_INFERENCE_FREQUENCY](#), [HeartbeatState::last_inference_tick](#), [HeartbeatState::tick_count](#), [vm](#), and [vm_tick_inference_engine\(\)](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.14 vm_tick_inference_engine()

```
void vm_tick_inference_engine (
    VM * vm )
```

Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.

Coordinates inference on rolling window of truth to determine:

- Optimal adaptive window width (via variance inflection detection)
- Optimal decay slope (via exponential regression on heat trajectory)

Uses ANOVA early-exit to skip full inference when variance is stable (<5% change). All math uses Q48.16 fixed-point (integer-only, no floating-point).

Replaces legacy [vm_tick_window_tuner\(\)](#) and [vm_tick_slope_validator\(\)](#).

Parameters

<code>vm</code>	Pointer to VM instance
-----------------	--

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [HeartbeatState::early_exit_count](#), [InferenceOutputs::early_exited](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_TICK_NS](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [inference_engine_run\(\)](#), [inference_outputs_validate\(\)](#), [HeartbeatState::inference_run_count](#), [RollingWindowOfTruth::is_warm](#), [VM::last_inference_outputs](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [LOG_WARN](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [sf_malloc\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [InferenceOutputs::slope_fit_quality_q48](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [HeartbeatWorker::tick_ns](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [InferenceInputs::window](#), [RollingWindowOfTruth::window_pos](#), [InferenceOutputs::window_variance_q48](#), [word_count](#), [VM::word_count_at_check](#), and [HeartbeatState::worker](#).

Referenced by [vm_tick\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.165.3.15 vm_tick_slope_validator()

```
void vm_tick_slope_validator (
    VM * vm )
```

Loop #3: Heat decay slope validation via periodic measurement.

Validates that linear decay is actually helping optimize dictionary caching. Measures stale word ratio, hot word count, and average heat distribution.

Parameters

<code>vm</code>	Pointer to VM instance
-----------------	--

References [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [VM::hot_word_count_at_check](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [VM::latest](#), [LOG_DEBUG](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [VM::stale_word_count_at_check](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [word_count](#), and [VM::word_count_at_check](#).

Here is the call graph for this function:

7.165.3.16 vm_tick_window_tuner()

```
void vm_tick_window_tuner (
    VM * vm )
```

Loop #5: Context-aware window tuning via binary chop search.

Uses prefetch accuracy to guide window size adaptation. Binary search converges on optimal effective_window_size↵ size for current workload.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [ADAPTIVE_MIN_WINDOW_SIZE](#), [RollingWindowOfTruth::effective_window_size](#), [ENABLE_PIPELINING](#), [RollingWindowOfTruth::is_warm](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [vm](#), and [PipelineGlobalMetrics::window_tuning_checks](#).

Here is the call graph for this function:

7.166 src/vm_time.c File Reference

```
#include "vm_internal.h"
#include "../include/inference_engine.h"
#include "../include/log.h"
#include "../include/physics_metadata.h"
#include "../include/physics_hotwords_cache.h"
#include "../include/rolling_window_of_truth.h"
#include "../include/dictionary_heat_optimization.h"
#include "../include/ssm_jacquard.h"
#include "../include/platform_alloc.h"
#include <string.h>
#include <errno.h>
#include <time.h>
```

Include dependency graph for vm_time.c:

Functions

- static void [vm_tick_apply_background_decay](#) (VM *vm, uint64_t now_ns)
- static void [vm_heartbeat_update_l8](#) (VM *vm)
 - L8 FINAL INTEGRATION: Jacquard mode selector heartbeat update.*
- void [vm_heartbeat_publish_snapshot](#) (VM *vm)
- void [vm_tick](#) (VM *vm)
 - Central heartbeat dispatcher for time-driven tuning operations.*
- void [vm_tick_window_tuner](#) (VM *vm)
 - Loop #5: Context-aware window tuning via binary chop search.*
- void [vm_tick_slope_validator](#) (VM *vm)
 - Loop #3: Heat decay slope validation via periodic measurement.*
- void [vm_heartbeat_run_cycle](#) (VM *vm)
- void [vm_snapshot_read](#) (const VM *vm, HeartbeatSnapshot *out_snapshot)
- void [vm_tick_inference_engine](#) (VM *vm)
 - Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.*
- void [vm_heartbeat_start_thread](#) (VM *vm)
- void [vm_heartbeat_stop_thread](#) (VM *vm)

7.166.1 Function Documentation

7.166.1.1 vm_heartbeat_publish_snapshot()

```
void vm_heartbeat_publish_snapshot (
    VM * vm )
```

References [HeartbeatSnapshot::actual_window_size](#), [HeartbeatSnapshot::decay_slope_q48](#), [VM::decay_slope_q48](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [heartbeat_snapshot_index_store\(\)](#), [HeartbeatSnapshot::hot_word_count](#), [VM::hot_word_count_at_check](#), [HeartbeatSnapshot::published_ns](#), [HeartbeatSnapshot::published_ns](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [sf_monotonic_ns\(\)](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), [HeartbeatSnapshot::stale_word_count](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [RollingWindowOfTruth::total_ex](#), [HeartbeatSnapshot::total_heat](#), [VM::total_heat_at_last_check](#), [vm](#), and [HeartbeatSnapshot::window_width](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#), and [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.2 vm_heartbeat_run_cycle()

```
void vm_heartbeat_run_cycle (
    VM * vm )
```

References [dict_adaptive_optimization_pass\(\)](#), [VM::heartbeat](#), [heartbeat_capture_tick_snapshot\(\)](#), [heartbeat_emit_tick_row\(\)](#), [HeartbeatState::heartbeat_enabled](#), [heartbeat_publish_snapshot\(\)](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_heartbeat_publish_snapshot\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_heartbeat_update_l8\(\)](#), [vm_monotonic_ns\(\)](#), [vm_tick\(\)](#), [vm_tick_apply_background_decay\(\)](#), and [vm_tick_apply_background_decay\(\)](#).

Referenced by [execute_colon_word\(\)](#), [physics_post_execute\(\)](#), and [vm_interpret\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.3 vm_heartbeat_start_thread()

```
void vm_heartbeat_start_thread (
    VM * vm )
```

References [VM::heartbeat](#), [HEARTBEAT_TICK_NS](#), [log_message\(\)](#), [LOG_WARN](#), [sf_malloc\(\)](#), [sf_free\(\)](#), [HeartbeatWorker::tick_ns](#), [vm](#), and [HeartbeatState::worker](#).

Referenced by [vm_init\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.4 vm_heartbeat_stop_thread()

```
void vm_heartbeat_stop_thread (
    VM * vm )
```

References [VM::heartbeat](#), [sf_free\(\)](#), [HeartbeatWorker::stop_requested](#), [vm](#), and [HeartbeatState::worker](#).

Referenced by [vm_cleanup\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.5 vm_heartbeat_update_l8()

```
static void vm_heartbeat_update_l8 (
    VM * vm ) [static]
```

L8 FINAL INTEGRATION: Jacquard mode selector heartbeat update.

Collects metrics from L1-L7 physics layers and feeds them to the L8 Jacquard mode selector. L8 then chooses the optimal configuration mode based on workload characteristics and applies it to the runtime.

This is the SOLE policy engine - all adaptive decisions flow through L8.

Parameters

<code>vm</code>	Pointer to VM instance
-----------------	--

References [ssm_l8_state_t::current_mode](#), [ssm_l8_metrics_t::cv](#), [VM::decay_slope_q48](#), [InferenceOutputs::early_exited](#), [RollingWindowOfTruth::effective_window_size](#), [ssm_l8_metrics_t::entropy](#), [VM::last_inference_outputs](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [ssm_apply_mode\(\)](#),

[VM::ssm_config](#), [ssm_l8_mode_name\(\)](#), [VM::ssm_l8_state](#), [ssm_l8_update\(\)](#), [ssm_l8_metrics_t::stability_score](#), [ssm_l8_metrics_t::temporal_decay](#), [RollingWindowOfTruth::total_executions](#), [VM::tuning_lock](#), and [vm](#).
Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.6 vm_snapshot_read()

```
void vm_snapshot_read (
    const VM * vm,
    HeartbeatSnapshot * out_snapshot )
```

References [VM::heartbeat](#), [heartbeat_snapshot_index_load\(\)](#), [HeartbeatState::snapshot_index](#), [HeartbeatState::snapshots](#), and [vm](#).

Here is the call graph for this function:

7.166.1.7 vm_tick()

```
void vm_tick (
    VM * vm )
```

Central heartbeat dispatcher for time-driven tuning operations.

Aggregates all periodic optimization tasks (Loop #3 and Loop #5) into one place. Designed as plugin architecture - new tuning operations can be added as plugins.

Options for integration:

- Synchronous (now): Called from main execution loop, every N executions
- Background thread (future): Runs in separate thread, decoupled from [VM](#) execution

Parameters

vm	Pointer to VM instance
--------------------	--

References [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_INFERENCE_FREQUENCY](#), [HeartbeatState::last_inference_tick](#), [HeartbeatState::tick_count](#), [vm](#), and [vm_tick_inference_engine\(\)](#).

Referenced by [assert_heartbeat_safe\(\)](#), and [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.8 vm_tick_apply_background_decay()

```
static void vm_tick_apply_background_decay (
    VM * vm,
    uint64_t now_ns ) [static]
```

References [VM::dict_lock](#), [HEARTBEAT_DECAY_BATCH](#), [VM::heartbeat_decay_cursor_id](#), [DictPhysics::last_active_ns](#), [DictPhysics::last_decay_ns](#), [VM::latest](#), [DictEntry::link](#), [DictEntry::physics](#), [physics_metadata_apply_linear_decay\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [vm](#), [vm_dictionary_lookup_by_word_id\(\)](#), [DictEntry::word_id](#), and [WORD_ID_INVALID](#).

Referenced by [vm_heartbeat_run_cycle\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.9 vm_tick_inference_engine()

```
void vm_tick_inference_engine (
    VM * vm )
```

Phase 2: Unified Inference Engine - Adaptive Window & Decay Slope Tuning.

Coordinates inference on rolling window of truth to determine:

- Optimal adaptive window width (via variance inflection detection)
- Optimal decay slope (via exponential regression on heat trajectory)

Uses ANOVA early-exit to skip full inference when variance is stable (<5% change). All math uses Q48.16 fixed-point (integer-only, no floating-point).

Replaces legacy [vm_tick_window_tuner\(\)](#) and [vm_tick_slope_validator\(\)](#).

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [InferenceOutputs::adaptive_decay_slope](#), [InferenceOutputs::adaptive_window_width](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [HeartbeatState::early_exit_count](#), [InferenceOutputs::early_exited](#), [RollingWindowOfTruth::effective_window_size](#), [VM::heartbeat](#), [HeartbeatState::heartbeat_enabled](#), [HEARTBEAT_TICK_NS](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [inference_engine_run\(\)](#), [inference_outputs_validate\(\)](#), [HeartbeatState::inference_run_count](#), [RollingWindowOfTruth::is_warm](#), [VM::last_inference_outputs](#), [HeartbeatState::last_inference_tick](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [rolling_window_service\(\)](#), [sf_malloc\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [InferenceOutputs::slope_fit_quality_q48](#), [VM::stale_word_count_at_check](#), [HeartbeatState::tick_count](#), [HeartbeatWorker::tick_ns](#), [HeartbeatState::tick_target_ns](#), [RollingWindowOfTruth::total_executions](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [vm_host_alloc\(\)](#), [InferenceInputs::window](#), [RollingWindowOfTruth::window_pos](#), [InferenceOutputs::window_vari](#), [word_count](#), [VM::word_count_at_check](#), and [HeartbeatState::worker](#).

Referenced by [vm_tick\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.166.1.10 vm_tick_slope_validator()

```
void vm_tick_slope_validator (
    VM * vm )
```

Loop #3: Heat decay slope validation via periodic measurement.

Validates that linear decay is actually helping optimize dictionary caching. Measures stale word ratio, hot word count, and average heat distribution.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [VM::decay_slope_direction](#), [VM::decay_slope_q48](#), [VM::dict_lock](#), [VM::hot_word_count_at_check](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_INFO](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [VM::stale_word_count_at_check](#), [VM::total_heat_at_last_check](#), [VM::tuning_lock](#), [vm](#), [word_count](#), and [VM::word_count_at_check](#).

Here is the call graph for this function:

7.166.1.11 vm_tick_window_tuner()

```
void vm_tick_window_tuner (
    VM * vm )
```

Loop #5: Context-aware window tuning via binary chop search.

Uses prefetch accuracy to guide window size adaptation. Binary search converges on optimal effective_window_↔ size for current workload.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [ADAPTIVE_MIN_WINDOW_SIZE](#), [RollingWindowOfTruth::effective_window_size](#), [ENABLE_PIPELINING](#), [RollingWindowOfTruth::is_warm](#), [PipelineGlobalMetrics::last_checked_accuracy](#), [PipelineGlobalMetrics::last_checked_window_size](#), [LOG_DEBUG](#), [LOG_INFO](#), [log_message\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [VM::rolling_window](#), [ROLLING_WINDOW_SIZE](#), [vm](#), and [PipelineGlobalMetrics::window_tuning](#).

Here is the call graph for this function:

7.167 src/word_registry.c File Reference

```
#include "../include/word_registry.h"
#include "../include/log.h"
```

```
#include <string.h>
#include "word_source/include/acl_words.h"
#include "word_source/include/stack_words.h"
#include "word_source/include/return_stack_words.h"
#include "word_source/include/memory_words.h"
#include "word_source/include/dictionary_words.h"
#include "word_source/include/arithmetic_words.h"
#include "word_source/include/double_words.h"
#include "word_source/include/mixed_arithmetic_words.h"
#include "word_source/include/logical_words.h"
#include "word_source/include/io_words.h"
#include "word_source/include/format_words.h"
#include "word_source/include/string_words.h"
#include "word_source/include/control_words.h"
#include "word_source/include/defining_words.h"
#include "word_source/include/dictionary_manipulation_words.h"
#include "word_source/include/vocabulary_words.h"
#include "word_source/include/block_words.h"
#include "word_source/include/editor_words.h"
#include "word_source/include/system_words.h"
#include "word_source/include/starforth_words.h"
#include "word_source/include/physics_benchmark_words.h"
#include "word_source/include/physics_pipelining_diagnostic_words.h"
#include "word_source/include/physics_freeze_words.h"
#include "word_source/include/dictionary_heat_diagnostic_words.h"
Include dependency graph for word_registry.c:
```

Functions

- void [register_word](#) (VM *vm, const char *name, word_func_t func)
Registers a single FORTH word in the virtual machine.
- void [register_forth79_words](#) (VM *vm)
Registers all standard FORTH-79 words in the virtual machine.

7.167.1 Function Documentation

7.167.1.1 register_forth79_words()

```
void register_forth79_words (
    VM * vm )
```

Registers all standard FORTH-79 words in the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers all 18 FORTH-79 word modules in dependency order, starting with fundamental stack operations and building up to more complex functionality like control flow and editing.

References [LOG_INFO](#), [log_message\(\)](#), [register_acl_words\(\)](#), [register_arithmetic_words\(\)](#), [register_block_words\(\)](#), [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_heat_diagnostic_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_dictionary_words\(\)](#), [register_double_words\(\)](#), [register_editor_words\(\)](#), [register_format_words\(\)](#), [register_io_words\(\)](#), [register_logical_words\(\)](#), [register_memory_words\(\)](#), [register_mixed_arithmetic_words\(\)](#), [register_physics_benchmark_words\(\)](#), [register_physics_freeze_words\(\)](#), [register_physics_pipelining_diagnostic_words\(\)](#), [register_return_stack_words\(\)](#), [register_stack_words\(\)](#), [register_starforth_words\(\)](#), [register_string_words\(\)](#), [register_system_words\(\)](#), [register_vocabulary_words\(\)](#), and [vm](#).

Referenced by [vm_init\(\)](#), and [vm_init_with_host\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.167.1.2 register_word()

```
void register_word (
    VM * vm,
    const char * name,
    word_func_t func )
```

Registers a single FORTH word in the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Name of the FORTH word to register
<i>func</i>	Function pointer to the word's implementation

References [entry](#), [func](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [name](#), [vm](#), and [vm_create_word\(\)](#).

Referenced by [register_acl_words\(\)](#), [register_arithmetic_words\(\)](#), [register_block_words\(\)](#), [register_control_words\(\)](#), [register_defining_words\(\)](#), [register_dictionary_heat_diagnostic_words\(\)](#), [register_dictionary_manipulation_words\(\)](#), [register_dictionary_words\(\)](#), [register_double_words\(\)](#), [register_editor_words\(\)](#), [register_format_words\(\)](#), [register_io_words\(\)](#), [register_logical_words\(\)](#), [register_memory_words\(\)](#), [register_mixed_arithmetic_words\(\)](#), [register_physics_benchmark_words\(\)](#), [register_physics_diagnostic_words\(\)](#), [register_physics_freeze_words\(\)](#), [register_return_stack_words\(\)](#), [register_stack_words\(\)](#), [register_starforth_words\(\)](#), [register_string_words\(\)](#), [register_system_words\(\)](#), and [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.168 src/word_source/acl_words.c File Reference

```
#include "include/acl_words.h"
#include "../include/vm.h"
#include "../include/word_registry.h"
#include "../include/log.h"
```

Include dependency graph for `acl_words.c`:

7.169 src/word_source/arithmetic_words.c File Reference

```
#include "include/arithmetic_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
```

Include dependency graph for `arithmetic_words.c`:

Functions

- static void [arithmetic_word_star_slash](#) (VM *vm)
- static void [arithmetic_word_star_slash_mod](#) (VM *vm)
- void [arithmetic_word_plus](#) (VM *vm)
Implements FORTH word "+" that adds two numbers.
- static void [arithmetic_word_minus](#) (VM *vm)
Implements FORTH word "-" that subtracts two numbers.
- static void [arithmetic_word_multiply](#) (VM *vm)
Implements FORTH word "" that multiplies two numbers.*
- static void [word_divide](#) (VM *vm)
- static void [mixed_math_word_mod](#) (VM *vm)
- static void [word_divmod](#) (VM *vm)
- static void [arithmetic_word_one_plus](#) (VM *vm)
- static void [arithmetic_word_one_minus](#) (VM *vm)
- static void [arithmetic_word_two_plus](#) (VM *vm)
- static void [arithmetic_word_two_minus](#) (VM *vm)

- static void [arithmetic_word_two_multiply](#) (VM *vm)
- static void [arithmetic_word_two_divide](#) (VM *vm)
- static void [arithmetic_word_abs](#) (VM *vm)
- static void [arithmetic_word_negate](#) (VM *vm)
- static void [arithmetic_word_min](#) (VM *vm)
- static void [arithmetic_word_max](#) (VM *vm)
- void [register_arithmetic_words](#) (VM *vm)

Registers all FORTH-79 arithmetic words with the VM.

7.169.1 Function Documentation

7.169.1.1 arithmetic_word_abs()

```
static void arithmetic_word_abs (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.2 arithmetic_word_max()

```
static void arithmetic_word_max (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.3 arithmetic_word_min()

```
static void arithmetic_word_min (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.4 arithmetic_word_minus()

```
static void arithmetic_word_minus (
    VM * vm ) [static]
```

Implements FORTH word "-" that subtracts two numbers.

Stack effect: (n1 n2 – n3) Subtracts n2 from n1, pushing their difference n3 onto the stack.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.5 arithmetic_word_multiply()

```
static void arithmetic_word_multiply (
    VM * vm ) [static]
```

Implements FORTH word "*" that multiplies two numbers.

Stack effect: (n1 n2 – n3) Multiplies n1 by n2, pushing their product n3 onto the stack.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.6 arithmetic_word_negate()

```
static void arithmetic_word_negate (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.7 arithmetic_word_one_minus()

```
static void arithmetic_word_one_minus (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.8 arithmetic_word_one_plus()

```
static void arithmetic_word_one_plus (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.9 arithmetic_word_plus()

```
void arithmetic_word_plus (
    VM * vm )
```

Implements FORTH word "+" that adds two numbers.

Stack effect: (n1 n2 – n3) Adds n1 and n2, pushing their sum n3 onto the stack.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.10 arithmetic_word_star_slash()

```
static void arithmetic_word_star_slash (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.11 arithmetic_word_star_slash_mod()

```
static void arithmetic_word_star_slash_mod (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.12 arithmetic_word_two_divide()

```
static void arithmetic_word_two_divide (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.13 arithmetic_word_two_minus()

```
static void arithmetic_word_two_minus (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.14 arithmetic_word_two_multiply()

```
static void arithmetic_word_two_multiply (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.15 arithmetic_word_two_plus()

```
static void arithmetic_word_two_plus (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.16 mixed_math_word_mod()

```
static void mixed_math_word_mod (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.17 register_arithmetic_words()

```
void register_arithmetic_words (
    VM * vm )
```

Registers all FORTH-79 arithmetic words with the [VM](#).

Registers all FORTH-79 arithmetic words with the virtual machine.

This function registers all the standard FORTH-79 arithmetic operations including basic arithmetic, advanced math operations, increment/decrement operations, and comparison functions.

Parameters

<i>vm</i>	Pointer to the VM structure where words will be registered
-----------	--

References [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#),

[arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [LOG_INFO](#), [log_message\(\)](#), [mixed_math_word_mod\(\)](#), [register_word\(\)](#), [vm](#), [word_divide\(\)](#), and [word_divmod\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.18 word_divide()

```
static void word_divide (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.169.1.19 word_divmod()

```
static void word_divmod (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170 src/word_source/block_words.c File Reference

```
#include "include/block_words.h"
#include "../include/word_registry.h"
#include "../include/vm.h"
#include "../include/block_subsystem.h"
#include <string.h>
#include <stdio.h>
```

Include dependency graph for block_words.c:

Functions

- static void [set_scr](#) (VM *vm, cell_t blk)
- void [init_block_system](#) (VM *vm)

Initializes the block system.
- unsigned char * [get_block_buffer](#) (VM *vm, int block_num)

Gets a buffer for the specified block.
- unsigned char * [get_empty_buffer](#) (VM *vm, int block_num)

Gets an empty buffer for the specified block.
- void [mark_buffer_dirty](#) (VM *vm)

Marks the current block buffer as dirty.
- void [save_all_buffers](#) (VM *vm)

Saves all dirty buffers.
- void [empty_all_buffers](#) (VM *vm)

Empties all user block buffers.
- void [block_word_block](#) (VM *vm)
- void [block_word_buffer](#) (VM *vm)
- void [block_word_update](#) (VM *vm)
- void [block_word_save_buffers](#) (VM *vm)
- void [block_word_empty_buffers](#) (VM *vm)
- void [block_word_flush](#) (VM *vm)
- void [block_word_load](#) (VM *vm)
- void [block_word_list](#) (VM *vm)

- void [block_word_thru](#) (VM *vm)
- void [block_word_scr](#) (VM *vm)
- void [block_word_next_block](#) (VM *vm)
- void [register_block_words](#) (VM *vm)

Registers all block-related FORTH words.

7.170.1 Function Documentation

7.170.1.1 [block_word_block\(\)](#)

```
void block_word_block (
    VM * vm )
```

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [VM::dsp](#), [VM::error](#), [set_scr\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.2 [block_word_buffer\(\)](#)

```
void block_word_buffer (
    VM * vm )
```

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [VM::dsp](#), [VM::error](#), [set_scr\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.3 [block_word_empty_buffers\(\)](#)

```
void block_word_empty_buffers (
    VM * vm )
```

References [empty_all_buffers\(\)](#), and [vm](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.4 [block_word_flush\(\)](#)

```
void block_word_flush (
    VM * vm )
```

References [blk_flush\(\)](#), and [vm](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.5 [block_word_list\(\)](#)

```
void block_word_list (
    VM * vm )
```

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [VM::dsp](#), [VM::error](#), [set_scr\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.6 [block_word_load\(\)](#)

```
void block_word_load (
    VM * vm )
```

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [VM::dsp](#), [VM::error](#), [set_scr\(\)](#), [vm](#), [vm_interpret\(\)](#), and [vm_pop\(\)](#).

Referenced by [block_word_thru\(\)](#), and [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.7 block_word_next_block()

```
void block_word_next_block (
    VM * vm )
```

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [current_scr\(\)](#), [VM::error](#), [VM::scr_addr](#), [set_scr\(\)](#), [vm](#), [vm_interpret\(\)](#), and [vm_load_cell\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.8 block_word_save_buffers()

```
void block_word_save_buffers (
    VM * vm )
```

References [save_all_buffers\(\)](#), and [vm](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.9 block_word_scr()

```
void block_word_scr (
    VM * vm )
```

References [CELL\(\)](#), [VM::scr_addr](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.10 block_word_thru()

```
void block_word_thru (
    VM * vm )
```

References [blk_is_valid\(\)](#), [block_word_load\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.11 block_word_update()

```
void block_word_update (
    VM * vm )
```

References [blk_is_valid\(\)](#), [blk_update\(\)](#), [VM::error](#), [VM::scr_addr](#), [vm](#), and [vm_load_cell\(\)](#).

Referenced by [register_block_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.12 empty_all_buffers()

```
void empty_all_buffers (
    VM * vm )
```

Empties all user block buffers.

Marks all buffers as empty.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_get_buffer\(\)](#), [blk_get_total_blocks\(\)](#), [BLOCK_SIZE](#), [USER_BLOCKS_START](#), and [vm](#).

Referenced by [block_word_empty_buffers\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.13 get_block_buffer()

```
unsigned char * get_block_buffer (
    VM * vm,
```

```
int block_num )
```

Gets a buffer for the specified block.

Gets a buffer containing the specified block.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to retrieve

Returns

Pointer to block buffer or NULL if invalid

References [blk_get_buffer\(\)](#), [set_scr\(\)](#), and [vm](#).

Here is the call graph for this function:

7.170.1.14 get_empty_buffer()

```
unsigned char * get_empty_buffer (
    VM * vm,
    int block_num )
```

Gets an empty buffer for the specified block.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to create

Returns

Pointer to empty block buffer or NULL if invalid

References [blk_get_empty_buffer\(\)](#), [set_scr\(\)](#), and [vm](#).

Here is the call graph for this function:

7.170.1.15 init_block_system()

```
void init_block_system (
    VM * vm )
```

Initializes the block system.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [vm](#).

7.170.1.16 mark_buffer_dirty()

```
void mark_buffer_dirty (
    VM * vm )
```

Marks the current block buffer as dirty.

Marks the current buffer as modified.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_update\(\)](#), [VM::scr_addr](#), [vm](#), and [vm_load_cell\(\)](#).

Referenced by [editor_word_edit\(\)](#), and [editor_word_s\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.17 register_block_words()

```
void register_block_words (
    VM * vm )
```

Registers all block-related FORTH words.

Registers all block and buffer words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_empty_buffers\(\)](#), [block_word_flush\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_save_buffers\(\)](#), [block_word_scr\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.18 save_all_buffers()

```
void save_all_buffers (
    VM * vm )
```

Saves all dirty buffers.

Saves all modified buffers to storage.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_flush\(\)](#), and [vm](#).

Referenced by [block_word_save_buffers\(\)](#), and [editor_word_edit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.170.1.19 set_scr()

```
static void set_scr (
    VM * vm,
    cell_t blk ) [static]
```

References [VM::scr_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [get_block_buffer\(\)](#), and [get_empty_buffer\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.171 src/word_source/control_words.c File Reference

```
#include "include/control_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
#include <stddef.h>
#include <stdint.h>
```

Include dependency graph for control_words.c:

Data Structures

- struct [cf_item_t](#)

Macros

- #define [CF_STACK_MAX](#) 64
Maximum depth of control flow stack.

Enumerations

- enum [cf_tag_t](#) {
 [CF_BEGIN](#) , [CF_IF](#) , [CF_ELSE](#) , [CF_WHILE](#) ,
 [CF_DO](#) , [CF_CASE](#) , [CF_OF](#) }
Control flow item types.

Functions

- static void [cf_epoch_sync](#) (VM *vm)
- static int [cf_push_item](#) (cf_tag_t tag, size_t mark)
- static int [cf_pop_item](#) (cf_item_t *out)
- static int [cf_peek_item](#) (cf_item_t *out)
- static size_t [emit_cell](#) (VM *vm, cell_t value)
- static void [control_forth_UNLOOP](#) (VM *vm)
- static void [control_forth_if](#) (VM *vm)
- static void [control_forth_else](#) (VM *vm)
- static void [control_forth_then](#) (VM *vm)
- static void [control_forth_begin](#) (VM *vm)
- static void [control_forth_until](#) (VM *vm)
- static void [control_forth_again](#) (VM *vm)
- static void [control_forth_while](#) (VM *vm)
- static void [control_forth_repeat](#) (VM *vm)
- static void [control_forth_qdo](#) (VM *vm)
- static void [control_forth_do](#) (VM *vm)
- static void [control_forth_leave](#) (VM *vm)
- static void [control_forth_loop](#) (VM *vm)
- static void [control_forth_plus_loop](#) (VM *vm)
- static void [control_forth_case](#) (VM *vm)
- static void [control_forth_of](#) (VM *vm)
- static void [control_forth_endof](#) (VM *vm)
- static void [control_forth_endcase](#) (VM *vm)
- void [register_control_words](#) (VM *vm)
Registers all control flow words with the Forth virtual machine.

Internal Runtime Functions

Forward declarations for internal runtime helpers and control flow primitives

- static void [control_forth_branch](#) (VM *vm)
Execute unconditional branch.
- static void [control_forth_0branch](#) (VM *vm)
Execute conditional branch based on top of stack.
- static void [control_forth_runtime_qdo](#) (VM *vm)
Runtime handler for ?DO loops.
- static void [control_forth_runtime_do](#) (VM *vm)
Runtime handler for DO loops.
- static void [control_forth_runtime_loop](#) (VM *vm)
Runtime handler for LOOP.

- static void [control_forth_runtime_plus_loop](#) (VM *vm)
Runtime handler for +LOOP.
- static void [control_forth_runtime_leave](#) (VM *vm)
Runtime handler for LEAVE.
- static void [control_forth_I](#) (VM *vm)
Get current loop index.
- static void [control_forth_J](#) (VM *vm)
Get outer loop index.
- static void [control_forth_EXIT](#) (VM *vm)
Exit from current definition.

Variables

- static [cf_item_t](#) [cf_stack](#) [64]
- static int [cf_sp](#) = -1
- static int [cf_last_mode](#) = -999
- static [size_t](#) [leave_addrs](#) [64]
- static int [leave_sp](#) = -1
- static int [leave_mark_stack](#) [64]
- static int [leave_mark_sp](#) = -1
- static [size_t](#) [endof_addrs](#) [64]
- static int [endof_sp](#) = -1
- static int [endof_mark_stack](#) [64]
- static int [endof_mark_sp](#) = -1

7.171.1 Function Documentation

7.171.1.1 [cf_epoch_sync\(\)](#)

```
static void cf_epoch_sync (
    VM * vm ) [inline], [static]
```

References [cf_last_mode](#), [cf_sp](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::mode](#), and [vm](#).

Referenced by [control_forth_again\(\)](#), [control_forth_begin\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_if\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#), [control_forth_then\(\)](#), [control_forth_until\(\)](#), and [control_forth_while\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.171.1.2 [cf_peek_item\(\)](#)

```
static int cf_peek_item (
    cf_item_t * out ) [inline], [static]
```

References [cf_sp](#), and [cf_stack](#).

Referenced by [control_forth_else\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), and [control_forth_while\(\)](#).

Here is the caller graph for this function:

7.171.1.3 [cf_pop_item\(\)](#)

```
static int cf_pop_item (
    cf_item_t * out ) [inline], [static]
```

References [cf_sp](#), [cf_stack](#), [LOG_ERROR](#), and [log_message\(\)](#).

Referenced by [control_forth_again\(\)](#), [control_forth_else\(\)](#), [control_forth_endcase\(\)](#), [control_forth_endof\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_repeat\(\)](#), [control_forth_then\(\)](#), and [control_forth_until\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.171.1.4 cf_push_item()

```
static int cf_push_item (
    cf_tag_t tag,
    size_t mark ) [inline], [static]
```

References [cf_item_t::addr](#), [cf_sp](#), [cf_stack](#), [CF_STACK_MAX](#), [LOG_ERROR](#), [log_message\(\)](#), and [cf_item_t::tag](#).
 Referenced by [control_forth_begin\(\)](#), [control_forth_case\(\)](#), [control_forth_do\(\)](#), [control_forth_else\(\)](#), [control_forth_if\(\)](#),
[control_forth_of\(\)](#), [control_forth_qdo\(\)](#), and [control_forth_while\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.171.1.5 emit_cell()

```
static size_t emit_cell (
    VM * vm,
    cell_t value ) [inline], [static]
```

References [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_align\(\)](#),
[vm_allot\(\)](#), and [vm_push\(\)](#).

Referenced by [control_forth_again\(\)](#), [control_forth_else\(\)](#), [control_forth_endof\(\)](#), [control_forth_if\(\)](#), [control_forth_leave\(\)](#),
[control_forth_loop\(\)](#), [control_forth_of\(\)](#), [control_forth_plus_loop\(\)](#), [control_forth_qdo\(\)](#), [control_forth_repeat\(\)](#),
[control_forth_until\(\)](#), and [control_forth_while\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.171.2 Variable Documentation

7.171.2.1 cf_last_mode

```
int cf_last_mode = -999 [static]
```

Referenced by [cf_epoch_sync\(\)](#).

7.171.2.2 cf_sp

```
int cf_sp = -1 [static]
```

Referenced by [cf_epoch_sync\(\)](#), [cf_peek_item\(\)](#), [cf_pop_item\(\)](#), [cf_push_item\(\)](#), [control_forth_leave\(\)](#), and
[control_forth_of\(\)](#).

7.171.2.3 cf_stack

```
cf_item_t cf_stack[64] [static]
```

Referenced by [cf_peek_item\(\)](#), [cf_pop_item\(\)](#), [cf_push_item\(\)](#), [control_forth_leave\(\)](#), and [control_forth_of\(\)](#).

7.171.2.4 endof_addrs

```
size_t endof_addrs[64] [static]
```

Referenced by [control_forth_endcase\(\)](#), and [control_forth_endof\(\)](#).

7.171.2.5 endof_mark_sp

```
int endof_mark_sp = -1 [static]
```

Referenced by [control_forth_case\(\)](#), and [control_forth_endcase\(\)](#).

7.171.2.6 endof_mark_stack

```
int endof_mark_stack[64] [static]
```

Referenced by [control_forth_case\(\)](#), and [control_forth_endcase\(\)](#).

7.171.2.7 endof_sp

```
int endof_sp = -1 [static]
```

Referenced by [control_forth_case\(\)](#), [control_forth_endcase\(\)](#), and [control_forth_endof\(\)](#).

7.171.2.8 leave_addr

```
size_t leave_addr[64] [static]
```

Referenced by [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), and [control_forth_plus_loop\(\)](#).

7.171.2.9 leave_mark_sp

```
int leave_mark_sp = -1 [static]
```

Referenced by [control_forth_do\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), and [control_forth_qdo\(\)](#).

7.171.2.10 leave_mark_stack

```
int leave_mark_stack[64] [static]
```

Referenced by [control_forth_do\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), and [control_forth_qdo\(\)](#).

7.171.2.11 leave_sp

```
int leave_sp = -1 [static]
```

Referenced by [control_forth_do\(\)](#), [control_forth_leave\(\)](#), [control_forth_loop\(\)](#), [control_forth_plus_loop\(\)](#), and [control_forth_qdo\(\)](#).

7.172 src/word_source/defining_words.c File Reference

```
#include "include/defining_words.h"
#include "../include/vm.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/profiler.h"
#include "../include/physics_metadata.h"
#include "../include/platform_time.h"
#include "../include/physics_pipelining_metrics.h"
#include <string.h>
#include <stdlib.h>
```

Include dependency graph for defining_words.c:

Functions

- static void [defining_runtime_create](#) (VM *vm)
Runtime behavior for words defined by CREATE.
- static void [defining_runtime_constant](#) (VM *vm)
Runtime behavior for CONSTANT words.
- static void [defining_runtime_variable](#) (VM *vm)
Runtime behavior for VARIABLE words.
- static void [defining_runtime_lit](#) (VM *vm)
Runtime behavior for literal values in threaded code.
- static void [defining_runtime_dodoes](#) (VM *vm)
Runtime behavior for words defined with DOES>
- static void [defining_runtime_does_rt](#) (VM *vm)
Helper routine executed at DOES> time.
- static void [defining_word_colon](#) (VM *vm)
- static void [defining_word_semicolon](#) (VM *vm)
- static void [defining_word_create](#) (VM *vm)
- static void [defining_word_constant](#) (VM *vm)
- static void [defining_word_variable](#) (VM *vm)
- static void [defining_word_left_bracket](#) (VM *vm)

- static void [defining_word_right_bracket](#) (VM *vm)
- static void [defining_word_state](#) (VM *vm)
- static void [defining_word_compile](#) (VM *vm)
- static void [defining_word_bracket_compile](#) (VM *vm)
- static void [defining_word_literal](#) (VM *vm)
- static void [defining_word_does](#) (VM *vm)
- static void [defining_word_immediate](#) (VM *vm)
- static void [dictionary_word_forget](#) (VM *vm)
- void [register_defining_words](#) (VM *vm)

Register all defining words in the dictionary.

7.172.1 Function Documentation

7.172.1.1 [defining_runtime_constant\(\)](#)

```
static void defining_runtime_constant (
    VM * vm ) [static]
```

Runtime behavior for CONSTANT words.

Pushes the constant value stored in the word's data field onto the stack. Stack effect: (– n)

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [VM::current_executing_entry](#), [entry](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_dictionary_get_data_field\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_word_constant\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.2 [defining_runtime_create\(\)](#)

```
static void defining_runtime_create (
    VM * vm ) [static]
```

Runtime behavior for words defined by CREATE.

Pushes the word's data field address (DFA) captured at CREATE time. Stack effect: (– addr)

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [CELL\(\)](#), [VM::current_executing_entry](#), [entry](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_addr_ok\(\)](#), [vm_dictionary_get_data_field\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_word_create\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.3 [defining_runtime_dodoes\(\)](#)

```
static void defining_runtime_dodoes (
    VM * vm ) [static]
```

Runtime behavior for words defined with DOES>

Pushes the address of user data area then executes the DOES> body code. PFA layout: [body_vaddr][user_data...]

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [VM::current_executing_entry](#), [entry](#), [VM::error](#), [VM::exit_colon](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_touch\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#),

[profiler_word_exit\(\)](#), [VM::return_stack](#), [VM::rsp](#), [sf_monotonic_ns\(\)](#), [STACK_SIZE](#), [vm](#), [vm_addr_ok\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_ptr\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_runtime_does_rt\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.4 defining_runtime_does_rt()

```
static void defining_runtime_does_rt (
    VM * vm ) [static]
```

Helper routine executed at DOES> time.

Converts the just-created child word to use DODOES runtime behavior and records the DOES> body address in the child's PFA.

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [defining_runtime_dodoes\(\)](#), [VM::error](#), [DictEntry::func](#), [VM::here](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), [vm_addr_ok\(\)](#), [vm_align\(\)](#), [vm_allot\(\)](#), [vm_dictionary_get_data_field\(\)](#), and [vm_ptr\(\)](#).

Referenced by [defining_word_does\(\)](#), and [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.5 defining_runtime_lit()

```
static void defining_runtime_lit (
    VM * vm ) [static]
```

Runtime behavior for literal values in threaded code.

Fetches the next cell from threaded code via IP and pushes it. Stack effect: (– n)

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.6 defining_runtime_variable()

```
static void defining_runtime_variable (
    VM * vm ) [static]
```

Runtime behavior for VARIABLE words.

Pushes the variable's address ([VM](#) offset) stored in the word's data field. Stack effect: (– addr)

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [VM::current_executing_entry](#), [entry](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_dictionary_get_data_field\(\)](#), and [vm_push\(\)](#).

Referenced by [defining_word_variable\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.7 defining_word_bracket_compile()

```
static void defining_word_bracket_compile (
    VM * vm ) [static]
```

References [defining_word_compile\(\)](#), and [vm](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.8 defining_word_colon()

```
static void defining_word_colon (
    VM * vm ) [static]
```

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_enter_compile_mode\(\)](#), [vm_parse_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.9 defining_word_compile()

```
static void defining_word_compile (
    VM * vm ) [static]
```

References [entry](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), [vm_parse_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [defining_word_bracket_compile\(\)](#), and [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.10 defining_word_constant()

```
static void defining_word_constant (
    VM * vm ) [static]
```

References [defining_runtime_constant\(\)](#), [VM::dsp](#), [entry](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_parse_word\(\)](#), [vm_pop\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.11 defining_word_create()

```
static void defining_word_create (
    VM * vm ) [static]
```

References [defining_runtime_create\(\)](#), [entry](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_align\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_parse_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.12 defining_word_does()

```
static void defining_word_does (
    VM * vm ) [static]
```

References [defining_runtime_does_rt\(\)](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_call\(\)](#), [vm_compile_word\(\)](#), and [vm_find_word\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.13 defining_word_immediate()

```
static void defining_word_immediate (
    VM * vm ) [static]
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [LOG_DEBUG](#), [log_message\(\)](#), [DictEntry::name](#), [DictEntry::name_len](#), [vm](#), and [WORD_IMMEDIATE](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.14 defining_word_left_bracket()

```
static void defining_word_left_bracket (
    VM * vm ) [static]
```

References [LOG_DEBUG](#), [log_message\(\)](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::state_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.15 defining_word_literal()

```
static void defining_word_literal (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_compile_literal\(\)](#), and [vm_pop\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.16 defining_word_right_bracket()

```
static void defining_word_right_bracket (
    VM * vm ) [static]
```

References [LOG_DEBUG](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [VM::state_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.17 defining_word_semicolon()

```
static void defining_word_semicolon (
    VM * vm ) [static]
```

References [VM::error](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), and [vm_exit_compile_mode\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.18 defining_word_state()

```
static void defining_word_state (
    VM * vm ) [static]
```

References [VM::state_addr](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.19 defining_word_variable()

```
static void defining_word_variable (
    VM * vm ) [static]
```

References [defining_runtime_variable\(\)](#), [entry](#), [VM::error](#), [VM::here](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), [vm_allot\(\)](#), [vm_create_word\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_parse_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.20 dictionary_word_forget()

```
static void dictionary_word_forget (
    VM * vm ) [static]
```

References [VM::dict_fence_here](#), [VM::dict_fence_latest](#), [VM::dict_lock](#), [VM::error](#), [VM::here](#), [VM::latest](#), [DictEntry::link](#), [LOG_DEBUG](#), [log_message\(\)](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), [transition_metrics_cleanup\(\)](#), [vm](#), [vm_addr_ok\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_dictionary_untrack_entry\(\)](#), [vm_parse_word\(\)](#), [WORD_HIDDEN](#), and [WORD_NAME_MAX](#).

Referenced by [register_defining_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.172.1.21 register_defining_words()

```
void register_defining_words (
    VM * vm )
```

Register all defining words in the dictionary.

Registers all FORTH-79 defining words with the virtual machine.

Registers core defining words like :, ::, CREATE, CONSTANT, VARIABLE, and their support words like IMMEDIATE, STATE, etc.

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [defining_word_bracket_compile\(\)](#), [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_does\(\)](#), [defining_word_immediate\(\)](#), [defining_word_left_bracket\(\)](#), [defining_word_literal\(\)](#), [defining_word_right_bracket\(\)](#), [defining_word_semicolon\(\)](#), [defining_word_state\(\)](#), [defining_word_variable\(\)](#), [dictionary_word_forget\(\)](#), [register_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).
Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.173 src/word_source/dictionary_heat_diagnostic_words.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "vm.h"
#include "word_registry.h"
#include "platform_time.h"
#include "dictionary_heat_optimization.h"
Include dependency graph for dictionary_heat_diagnostic_words.c:
```

Functions

- void [forth_HEAT_PERCENTILES](#) (VM *vm)
- void [forth_LOOKUP_STRATEGY_FETCH](#) (VM *vm)
- void [forth_LOOKUP_STRATEGY_STORE](#) (VM *vm)
- void [forth_REORG_BUCKETS](#) (VM *vm)
- void [forth_SHOW_HEAT_OPTIMIZATION](#) (VM *vm)
- void [forth_COMPARE_LOOKUPS](#) (VM *vm)
- void [register_dictionary_heat_diagnostic_words](#) (VM *vm)

Register all dictionary heat diagnostic words with the [VM](#).

7.173.1 Function Documentation

7.173.1.1 forth_COMPARE_LOOKUPS()

```
void forth_COMPARE_LOOKUPS (
    VM * vm )
```

References [VM::data_stack](#), [dict_reorganize_buckets_by_heat\(\)](#), [VM::dsp](#), [VM::error](#), [VM::lookup_strategy](#), [sf_monotonic_ns\(\)](#), [vm](#), and [vm_find_word\(\)](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.173.1.2 forth_HEAT_PERCENTILES()

```
void forth_HEAT_PERCENTILES (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.173.1.3 forth_LOOKUP_STRATEGY_FETCH()

```
void forth_LOOKUP_STRATEGY_FETCH (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [VM::lookup_strategy](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.173.1.4 forth_LOOKUP_STRATEGY_STORE()

```
void forth_LOOKUP_STRATEGY_STORE (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [VM::lookup_strategy](#), and [vm](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.173.1.5 forth_REORG_BUCKETS()

```
void forth_REORG_BUCKETS (
    VM * vm )
```

References [dict_reorganize_buckets_by_heat\(\)](#), [dict_update_heat_percentiles\(\)](#), [VM::error](#), and [vm](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.173.1.6 forth_SHOW_HEAT_OPTIMIZATION()

```
void forth_SHOW_HEAT_OPTIMIZATION (
    VM * vm )
```

References [VM::error](#), [VM::heat_threshold_25th](#), [VM::heat_threshold_50th](#), [VM::heat_threshold_75th](#), [VM::lookup_strategy](#), and [vm](#).

Referenced by [register_dictionary_heat_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.173.1.7 register_dictionary_heat_diagnostic_words()

```
void register_dictionary_heat_diagnostic_words (
    VM * vm )
```

Register all dictionary heat diagnostic words with the [VM](#).

References [forth_COMPARE_LOOKUPS\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_LOOKUP_STRATEGY_FETCH\(\)](#), [forth_LOOKUP_STRATEGY_STORE\(\)](#), [forth_REORG_BUCKETS\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174 src/word_source/dictionary_manipulation_words.c File Reference

```
#include "include/dictionary_manipulation_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
```

Include dependency graph for dictionary_manipulation_words.c:

Functions

- static uintptr_t [align_address](#) (uintptr_t addr)
- static char * [get_name_field](#) (DictEntry *entry)
- static void * [get_body_address](#) (DictEntry *entry)
- static DictEntry * [find_entry_from_name](#) (VM *vm, char *name_field)
- static char * [traverse_name_field](#) (char *name_addr, int direction)
- void [dictionary_m_word_left_bracket](#) (VM *vm)
FORTH word [- Enter interpretation mode.
- void [dictionary_m_word_right_bracket](#) (VM *vm)
FORTH word] - Enter compilation mode.
- void [dictionary_m_word_state](#) (VM *vm)
FORTH word STATE - Get compilation state variable address.
- static void [dictionary_m_word_smudge](#) (VM *vm)
FORTH word SMUDGE - Toggle smudge bit of latest word.
- void [dictionary_m_word_to_body](#) (VM *vm)
- void [dictionary_m_word_to_name](#) (VM *vm)
- void [dictionary_m_word_name_to](#) (VM *vm)
- void [dictionary_m_word_to_link](#) (VM *vm)
- void [dictionary_m_word_link_from](#) (VM *vm)
- void [dictionary_m_word_cfa](#) (VM *vm)
- void [dictionary_m_word_lfa](#) (VM *vm)
- void [dictionary_m_word_nfa](#) (VM *vm)
- void [dictionary_m_word_pfa](#) (VM *vm)
- void [dictionary_m_word_traverse](#) (VM *vm)
- void [dictionary_m_word_interpret](#) (VM *vm)
- static void [dictionary_m_word_find](#) (VM *vm)
- static void [dictionary_m_word_tick](#) (VM *vm)
- static void [dictionary_m_word_hidden](#) (VM *vm)
- void [register_dictionary_manipulation_words](#) (VM *vm)
Register all dictionary manipulation words with the VM.

Variables

- static cell_t [state_variable](#) = 0

7.174.1 Function Documentation

7.174.1.1 align_address()

```
static uintptr_t align_address (
    uintptr_t addr ) [static]
```

Referenced by [get_body_address\(\)](#).

Here is the caller graph for this function:

7.174.1.2 dictionary_m_word_cfa()

```
void dictionary_m_word_cfa (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.3 dictionary_m_word_find()

```
static void dictionary_m_word_find (
    VM * vm ) [static]
```

References [VM::error](#), [vm](#), [vm_find_word\(\)](#), [vm_parse_word\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.4 dictionary_m_word_hidden()

```
static void dictionary_m_word_hidden (
    VM * vm ) [static]
```

References [VM::error](#), [DictEntry::flags](#), [VM::latest](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [WORD_HIDDEN](#), and [WORD_SMUDGED](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the caller graph for this function:

7.174.1.5 dictionary_m_word_interpret()

```
void dictionary_m_word_interpret (
    VM * vm )
```

References [VM::mode](#), [MODE_INTERPRET](#), [state_variable](#), and [vm](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the caller graph for this function:

7.174.1.6 dictionary_m_word_left_bracket()

```
void dictionary_m_word_left_bracket (
    VM * vm )
```

FORTH word [- Enter interpretation mode.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

References [VM::mode](#), [MODE_INTERPRET](#), [state_variable](#), and [vm](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the caller graph for this function:

7.174.1.7 dictionary_m_word_lfa()

```
void dictionary_m_word_lfa (
    VM * vm )
```

References [dictionary_m_word_to_link\(\)](#), and [vm](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.8 dictionary_m_word_link_from()

```
void dictionary_m_word_link_from (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.9 dictionary_m_word_name_to()

```
void dictionary_m_word_name_to (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [find_entry_from_name\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.10 dictionary_m_word_nfa()

```
void dictionary_m_word_nfa (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [get_name_field\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.11 dictionary_m_word_pfa()

```
void dictionary_m_word_pfa (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [get_body_address\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.12 dictionary_m_word_right_bracket()

```
void dictionary_m_word_right_bracket (
    VM * vm )
```

FORTH word] - Enter compilation mode.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

References [VM::mode](#), [MODE_COMPILE](#), [state_variable](#), and [vm](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the caller graph for this function:

7.174.1.13 dictionary_m_word_smudge()

```
static void dictionary_m_word_smudge (
    VM * vm ) [static]
```

FORTH word SMUDGE - Toggle smudge bit of latest word.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

Note

Compile-only word

References [VM::error](#), [VM::mode](#), [MODE_COMPILE](#), and [vm](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the caller graph for this function:

7.174.1.14 dictionary_m_word_state()

```
void dictionary_m_word_state (
    VM * vm )
```

FORTH word STATE - Get compilation state variable address.

Parameters

<code>vm</code>	Pointer to VM instance @stack (– addr)
-----------------	--

References [state_variable](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.15 dictionary_m_word_tick()

```
static void dictionary_m_word_tick (
    VM * vm ) [static]
```

References [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_compile_literal\(\)](#), [vm_find_word\(\)](#), [vm_parse_word\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.16 dictionary_m_word_to_body()

```
void dictionary_m_word_to_body (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_dictionary_get_data_field\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.17 dictionary_m_word_to_link()

```
void dictionary_m_word_to_link (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [dictionary_m_word_lfa\(\)](#), and [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.18 dictionary_m_word_to_name()

```
void dictionary_m_word_to_name (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [get_name_field\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.19 dictionary_m_word_traverse()

```
void dictionary_m_word_traverse (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [traverse_name_field\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_manipulation_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.20 find_entry_from_name()

```
static DictEntry * find_entry_from_name (
    VM * vm,
    char * name_field ) [static]
```

References [entry](#), [VM::latest](#), [DictEntry::link](#), and [vm](#).

Referenced by [dictionary_m_word_name_to\(\)](#).

Here is the caller graph for this function:

7.174.1.21 get_body_address()

```
static void * get_body_address (
    DictEntry * entry ) [static]
```

References [align_address\(\)](#), and [entry](#).

Referenced by [dictionary_m_word_pfa\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.22 get_name_field()

```
static char * get_name_field (
    DictEntry * entry ) [static]
```

References [entry](#).

Referenced by [dictionary_m_word_nfa\(\)](#), and [dictionary_m_word_to_name\(\)](#).

Here is the caller graph for this function:

7.174.1.23 register_dictionary_manipulation_words()

```
void register_dictionary_manipulation_words (
    VM * vm )
```

Register all dictionary manipulation words with the [VM](#).

Register all dictionary manipulation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Registers standard FORTH-79 dictionary manipulation words

References [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_hidden\(\)](#), [dictionary_m_word_interpret\(\)](#), [dictionary_m_word_left_bracket\(\)](#), [dictionary_m_word_lfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_right_bracket\(\)](#), [dictionary_m_word_smudge\(\)](#), [dictionary_m_word_state\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [register_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.174.1.24 traverse_name_field()

```
static char * traverse_name_field (
    char * name_addr,
    int direction ) [static]
```

References [name_len](#).

Referenced by [dictionary_m_word_traverse\(\)](#).

Here is the caller graph for this function:

7.174.2 Variable Documentation**7.174.2.1 state_variable**

```
cell_t state_variable = 0 [static]
```

Referenced by [dictionary_m_word_interpret\(\)](#), [dictionary_m_word_left_bracket\(\)](#), [dictionary_m_word_right_bracket\(\)](#), and [dictionary_m_word_state\(\)](#).

7.175 src/word_source/dictionary_words.c File Reference

```
#include "include/dictionary_words.h"
#include "../include/word_registry.h"
#include "vm.h"
```



```
#include "../include/log.h"
Include dependency graph for dictionary_words.c:
```

Functions

- void [dictionary_word_here](#) (VM *vm)
HERE (- addr) Returns the dictionary pointer.
- void [dictionary_word_allot](#) (VM *vm)
ALLOT (n -) Allocate n bytes in dictionary.
- void [dictionary_word_comma](#) (VM *vm)
, (n -) Compile cell into dictionary
- void [dictionary_word_c_comma](#) (VM *vm)
C, (c -) Compile byte into dictionary.
- void [dictionary_word_2comma](#) (VM *vm)
2, (d -) Compile double-cell into dictionary
- void [dictionary_word_pad](#) (VM *vm)
PAD (- addr) Get scratch pad buffer address.
- void [dictionary_word_sp_fetch](#) (VM *vm)
SP@ (- sp) Get current stack pointer.
- void [dictionary_word_sp_store](#) (VM *vm)
SP! (sp -) Set stack pointer.
- void [dictionary_word_latest](#) (VM *vm)
LATEST (- addr) Get latest definition address.
- void [register_dictionary_words](#) (VM *vm)
Registers all dictionary and compilation words with the Forth VM.

7.175.1 Function Documentation

7.175.1.1 dictionary_word_2comma()

```
void dictionary_word_2comma (
    VM * vm )
```

2, (d -) Compile double-cell into dictionary

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Compiles a double cell value into the dictionary at HERE, low cell first

References [VM::dsp](#), [VM::error](#), [VM::here](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_align\(\)](#), [VM_MEMORY_SIZE](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.2 dictionary_word_allot()

```
void dictionary_word_allot (
    VM * vm )
```

ALLOT (n -) Allocate n bytes in dictionary.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Allocates n bytes in the dictionary. n can be negative to deallocate space

References [VM::dsp](#), [VM::error](#), [VM::here](#), [vm](#), [VM_MEMORY_SIZE](#), and [vm_pop\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.3 dictionary_word_c_comma()

```
void dictionary_word_c_comma (
    VM * vm )
```

C, (c –) Compile byte into dictionary.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Compiles a single byte value into the dictionary at HERE

References [VM::dsp](#), [VM::error](#), [VM::here](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [VM_MEMORY_SIZE](#), [vm_pop\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.4 dictionary_word_comma()

```
void dictionary_word_comma (
    VM * vm )
```

, (n –) Compile cell into dictionary

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Compiles a single cell value into the dictionary at HERE

References [VM::dsp](#), [VM::error](#), [VM::here](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_align\(\)](#), [VM_MEMORY_SIZE](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.5 dictionary_word_here()

```
void dictionary_word_here (
    VM * vm )
```

HERE (– addr) Returns the dictionary pointer.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::here](#), [vm](#), [vm_align\(\)](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.6 dictionary_word_latest()

```
void dictionary_word_latest (
    VM * vm )
```

LATEST (– addr) Get latest definition address.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Returns [VM](#) address near most recent compiled definition (end of dictionary)

References [VM::here](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.7 dictionary_word_pad()

```
void dictionary_word_pad (
    VM * vm )
```

PAD (– addr) Get scratch pad buffer address.

Parameters

vm	Pointer to the VM structure
--------------------	---

Returns [VM](#) address of 512-byte temporary text buffer at top of memory

References [vm](#), [VM_MEMORY_SIZE](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.8 dictionary_word_sp_fetch()

```
void dictionary_word_sp_fetch (
    VM * vm )
```

SP@ (– sp) Get current stack pointer.

Parameters

vm	Pointer to the VM structure
--------------------	---

Returns the current stack-pointer index (top is 0)

References [VM::dsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.9 dictionary_word_sp_store()

```
void dictionary_word_sp_store (
    VM * vm )
```

SP! (sp –) Set stack pointer.

Parameters

vm	Pointer to the VM structure
--------------------	---

Restores the stack-pointer, but never allows growing the stack

References [VM::dsp](#), [VM::error](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_dictionary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.175.1.10 register_dictionary_words()

```
void register_dictionary_words (
    VM * vm )
```

Registers all dictionary and compilation words with the Forth [VM](#).

Parameters

vm	Pointer to the Forth virtual machine instance
--------------------	---

References [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [dictionary_word_pad\(\)](#), [dictionary_word_sp_fetch\(\)](#), [dictionary_word_sp_store\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176 src/word_source/double_words.c File Reference

```
#include "include/double_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
Include dependency graph for double_words.c:
```

Functions

- void [double_word_s_to_d](#) (VM *vm)
Convert single number to double (n – d)
- void [double_word_d_plus](#) (VM *vm)
- void [double_word_d_minus](#) (VM *vm)
- void [double_word_dnegate](#) (VM *vm)
- void [double_word_dabs](#) (VM *vm)
- static int [d_compare](#) (cell_t d1h, cell_t d1l, cell_t d2h, cell_t d2l)
Compare two double numbers.
- void [double_word_dmax](#) (VM *vm)
- void [double_word_dmin](#) (VM *vm)
- void [double_word_d_less](#) (VM *vm)
- void [double_word_d_equals](#) (VM *vm)
- void [double_word_2drop](#) (VM *vm)
- void [double_word_2dup](#) (VM *vm)
- void [double_word_2swap](#) (VM *vm)
- void [double_word_2over](#) (VM *vm)
- void [double_word_2rot](#) (VM *vm)
- void [double_word_2to_r](#) (VM *vm)
- void [double_word_2r_from](#) (VM *vm)
- void [double_word_2r_fetch](#) (VM *vm)
- void [double_word_d_zero_equals](#) (VM *vm)
- void [double_word_d_zero_less](#) (VM *vm)
- void [double_word_d_two_star](#) (VM *vm)
- void [double_word_d_two_slash](#) (VM *vm)
- void [register_double_words](#) (VM *vm)
Register all double number words.

7.176.1 Function Documentation

7.176.1.1 d_compare()

```
static int d_compare (
    cell_t d1h,
    cell_t d1l,
    cell_t d2h,
    cell_t d2l ) [static]
```

Compare two double numbers.

Parameters

<i>d1h</i>	High cell of first double number
------------	----------------------------------

Parameters

<i>d1l</i>	Low cell of first double number
<i>d2h</i>	High cell of second double number
<i>d2l</i>	Low cell of second double number

Returns

int -1 if $d1 < d2$, 0 if $d1 = d2$, 1 if $d1 > d2$

Referenced by [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_dmax\(\)](#), and [double_word_dmin\(\)](#).
Here is the caller graph for this function:

7.176.1.2 double_word_2drop()

```
void double_word_2drop (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.3 double_word_2dup()

```
void double_word_2dup (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.4 double_word_2over()

```
void double_word_2over (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.5 double_word_2r_fetch()

```
void double_word_2r_fetch (
    VM * vm )
```

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.6 double_word_2r_from()

```
void double_word_2r_from (
    VM * vm )
```

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.7 double_word_2rot()

```
void double_word_2rot (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.8 double_word_2swap()

```
void double_word_2swap (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.9 double_word_2to_r()

```
void double_word_2to_r (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.10 double_word_d_equals()

```
void double_word_d_equals (
    VM * vm )
```

References [d_compare\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.11 double_word_d_less()

```
void double_word_d_less (
    VM * vm )
```

References [d_compare\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.12 double_word_d_minus()

```
void double_word_d_minus (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.13 double_word_d_plus()

```
void double_word_d_plus (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.14 double_word_d_two_slash()

```
void double_word_d_two_slash (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.15 double_word_d_two_star()

```
void double_word_d_two_star (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.16 double_word_d_zero_equals()

```
void double_word_d_zero_equals (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.17 double_word_d_zero_less()

```
void double_word_d_zero_less (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.18 double_word_dabs()

```
void double_word_dabs (
    VM * vm )
```

References [VM::data_stack](#), [double_word_dnegate\(\)](#), [VM::dsp](#), [VM::error](#), and [vm](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.19 double_word_dmax()

```
void double_word_dmax (
    VM * vm )
```

References [d_compare\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.20 double_word_dmin()

```
void double_word_dmin (
    VM * vm )
```

References [d_compare\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.21 double_word_dnegate()

```
void double_word_dnegate (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [double_word_dabs\(\)](#), and [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.22 double_word_s_to_d()

```
void double_word_s_to_d (
    VM * vm )
```

Convert single number to double (n - d)

Converts a single-precision number to double-precision format. Result is pushed as (dlow dhigh) with high cell on top.

Parameters

<code>vm</code>	Pointer to the virtual machine state
-----------------	--------------------------------------

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_double_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.176.1.23 register_double_words()

```
void register_double_words (
    VM * vm )
```

Register all double number words.

Registers all double precision number manipulation words with the Forth virtual machine.

Registers all double-precision number operations with the virtual machine.

Parameters

<code>vm</code>	Pointer to the virtual machine state
-----------------	--------------------------------------

Note

This function must be called during [VM](#) initialization.

References [double_word_2drop\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dabs\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177 src/word_source/editor_words.c File Reference

```
#include "include/editor_words.h"
#include "include/block_words.h"
#include "../include/vm.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

Include dependency graph for editor_words.c:

Functions

- static [cell_t](#) [current_scr](#) ([VM](#) *[vm](#))
Get current screen number from [VM](#).
- static void [set_scr](#) ([VM](#) *[vm](#), [cell_t](#) blk)
Set current screen number in [VM](#).
- static int [line_ptr](#) ([VM](#) *[vm](#), [cell_t](#) scr, [cell_t](#) line, unsigned char **out)
Get pointer to specific line in a screen.
- static void [print_line_64](#) (const unsigned char *p)
Print 64-character line with character filtering.
- static void [editor_word_l](#) ([VM](#) *[vm](#))
- static void [editor_word_s](#) ([VM](#) *[vm](#))

- static void [editor_word_show](#) (VM *vm)
- static void [editor_word_edit](#) (VM *vm)
- void [register_editor_words](#) (VM *vm)

Register all editor words with the Forth virtual machine.

7.177.1 Function Documentation

7.177.1.1 [current_scr\(\)](#)

```
static cell\_t current\_scr (
    VM * vm ) [inline], [static]
```

Get current screen number from [VM](#).

Parameters

vm	Pointer to VM instance
--------------------	--

Returns

Current screen number

References [VM::scr_addr](#), [vm](#), and [vm_load_cell\(\)](#).

Referenced by [block_word_next_block\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), and [editor_word_show\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.2 [editor_word_edit\(\)](#)

```
static void editor\_word\_edit (
    VM * vm ) [static]
```

References [VM::dsp](#), [editor_word_l\(\)](#), [editor_word_show\(\)](#), [VM::error](#), [line_ptr\(\)](#), [LOG_ERROR](#), [log_message\(\)](#), [mark_buffer_dirty\(\)](#), [MAX_BLOCKS](#), [save_all_buffers\(\)](#), [VM::scr_addr](#), [vm](#), [vm_load_cell\(\)](#), [vm_pop\(\)](#), [vm_push\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_editor_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.3 [editor_word_l\(\)](#)

```
static void editor\_word\_l (
    VM * vm ) [static]
```

References [current_scr\(\)](#), [VM::dsp](#), [VM::error](#), [line_ptr\(\)](#), [LOG_ERROR](#), [log_message\(\)](#), [MAX_BLOCKS](#), [print_line_64\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [editor_word_edit\(\)](#), and [register_editor_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.4 [editor_word_s\(\)](#)

```
static void editor\_word\_s (
    VM * vm ) [static]
```

References [current_scr\(\)](#), [VM::dsp](#), [VM::error](#), [line_ptr\(\)](#), [LOG_ERROR](#), [log_message\(\)](#), [mark_buffer_dirty\(\)](#), [MAX_BLOCKS](#), [vm](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_editor_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.5 [editor_word_show\(\)](#)

```
static void editor\_word\_show (
    VM * vm ) [static]
```

References [current_scr\(\)](#), [VM::error](#), [line_ptr\(\)](#), [LOG_ERROR](#), [log_message\(\)](#), [MAX_BLOCKS](#), [print_line_64\(\)](#), and [vm](#).

Referenced by [editor_word_edit\(\)](#), and [register_editor_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.6 line_ptr()

```
static int line_ptr (
    VM * vm,
    cell_t scr,
    cell_t line,
    unsigned char ** out ) [static]
```

Get pointer to specific line in a screen.

Parameters

<i>vm</i>	Pointer to VM instance
<i>scr</i>	Screen number (1-based)
<i>line</i>	Line number (0-15)
<i>out</i>	Output parameter for line pointer

Returns

1 if successful, 0 if parameters invalid

References [blk_get_buffer\(\)](#), [blk_is_valid\(\)](#), [MAX_BLOCKS](#), and [vm](#).

Referenced by [editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), and [editor_word_show\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.7 print_line_64()

```
static void print_line_64 (
    const unsigned char * p ) [static]
```

Print 64-character line with character filtering.

Parameters

<i>p</i>	Pointer to 64-byte line buffer
----------	--------------------------------

Referenced by [editor_word_l\(\)](#), and [editor_word_show\(\)](#).

Here is the caller graph for this function:

7.177.1.8 register_editor_words()

```
void register_editor_words (
    VM * vm )
```

Register all editor words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), [editor_word_show\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.177.1.9 set_scr()

```
static void set_scr (
    VM * vm,
```

```
cell_t blk ) [inline], [static]
```

Set current screen number in [VM](#).

Parameters

<i>vm</i>	Pointer to VM instance
<i>blk</i>	Screen number to set

References [VM::scr_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Here is the call graph for this function:

7.178 src/word_source/format_words.c File Reference

```
#include "include/format_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
#include <stdint.h>
#include <limits.h>
#include <string.h>
#include <stdio.h>
```

Include dependency graph for format_words.c:

Macros

- #define [CELL_BITS](#) ((int)(sizeof([cell_t](#)) * CHAR_BIT))
- #define [CHUNK_BITS](#) 16
- #define [CHUNK_MASK](#) ((unsigned long)((1UL << [CHUNK_BITS](#)) - 1))

Functions

- static void [ensure_pn](#) ([VM](#) *[vm](#))
Ensures picture number buffer is allocated.
- static unsigned [current_base](#) ([VM](#) *[vm](#))
Returns current numeric conversion base, defaulting to 10 if invalid.
- static char [digit_for](#) (unsigned v)
Converts value 0-35 to character '0'-'9' or 'A'-'Z'.
- static void [div_ud_by_base](#) ([cell_t](#) dhigh_in, [cell_t](#) dlow_in, unsigned base, [cell_t](#) *qhigh_out, [cell_t](#) *qlow_out, unsigned *rem_out)
Divides unsigned double-cell number by base using portable C99 code.
- static void [print_number_formatted](#) ([VM](#) *[vm](#), [cell_t](#) n, int width, int is_unsigned)
Prints number with optional width field and sign handling.
- void [format_word_base](#) ([VM](#) *[vm](#))
- void [format_word_decimal](#) ([VM](#) *[vm](#))
- void [format_word_hex](#) ([VM](#) *[vm](#))
- void [format_word_octal](#) ([VM](#) *[vm](#))
- void [format_word_begin_conversion](#) ([VM](#) *[vm](#))
- void [format_word_hold](#) ([VM](#) *[vm](#))
- void [format_word_sign](#) ([VM](#) *[vm](#))
- void [format_word_hash](#) ([VM](#) *[vm](#))
- void [format_word_hash_s](#) ([VM](#) *[vm](#))
- void [format_word_end_conversion](#) ([VM](#) *[vm](#))
- void [format_word_dot](#) ([VM](#) *[vm](#))
- void [format_word_dot_r](#) ([VM](#) *[vm](#))
- void [format_word_u_dot](#) ([VM](#) *[vm](#))

- void [format_word_u_dot_r](#) (VM *vm)
- void [format_word_d_dot](#) (VM *vm)
- void [format_word_d_dot_r](#) (VM *vm)
- void [format_word_dot_s](#) (VM *vm)
- void [format_word_question](#) (VM *vm)
- void [format_word_dump](#) (VM *vm)
- void [register_format_words](#) (VM *vm)

Registers all formatting and numeric conversion words with the [VM](#).

Variables

- static unsigned char * [pn_buf](#) = NULL
Picture number buffer for numeric conversion.
- static size_t [pn_cap](#) = 0
Capacity of picture number buffer.
- static int [conversion_pos](#) = 0
Current position in picture number buffer.

7.178.1 Macro Definition Documentation

7.178.1.1 CELL_BITS

```
#define CELL_BITS ((int)(sizeof(cell\_t) * CHAR_BIT))
```

7.178.1.2 CHUNK_BITS

```
#define CHUNK_BITS 16
```

7.178.1.3 CHUNK_MASK

```
#define CHUNK_MASK ((unsigned long)((1UL << CHUNK\_BITS) - 1))
```

7.178.2 Function Documentation

7.178.2.1 current_base()

```
static unsigned current_base (  
    VM * vm ) [static]
```

Returns current numeric conversion base, defaulting to 10 if invalid.

References [VM::base](#), and [vm](#).

Referenced by [format_word_hash\(\)](#), and [print_number_formatted\(\)](#).

Here is the caller graph for this function:

7.178.2.2 digit_for()

```
static char digit_for (  
    unsigned v ) [static]
```

Converts value 0-35 to character '0'-'9' or 'A'-'Z'.

Referenced by [format_word_hash\(\)](#), and [print_number_formatted\(\)](#).

Here is the caller graph for this function:

7.178.2.3 div_ud_by_base()

```
static void div_ud_by_base (  
    cell\_t dhigh_in,  
    cell\_t dlow_in,  
    unsigned base,  
    cell\_t * qhigh_out,
```

```

    cell_t * glow_out,
    unsigned * rem_out ) [static]

```

Divides unsigned double-cell number by base using portable C99 code.

References [base](#), [CELL_BITS](#), [CHUNK_BITS](#), and [CHUNK_MASK](#).

Referenced by [format_word_hash\(\)](#).

Here is the caller graph for this function:

7.178.2.4 ensure_pn()

```

static void ensure_pn (
    VM * vm ) [static]

```

Ensures picture number buffer is allocated.

References [VM::error](#), [pn_buf](#), [pn_cap](#), [vm](#), and [vm_allot\(\)](#).

Referenced by [format_word_begin_conversion\(\)](#), [format_word_end_conversion\(\)](#), and [format_word_hold\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.5 format_word_base()

```

void format_word_base (
    VM * vm )

```

References [VM::base_addr](#), [CELL\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.6 format_word_begin_conversion()

```

void format_word_begin_conversion (
    VM * vm )

```

References [conversion_pos](#), [ensure_pn\(\)](#), [VM::error](#), [pn_buf](#), [pn_cap](#), and [vm](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.7 format_word_d_dot()

```

void format_word_d_dot (
    VM * vm )

```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.8 format_word_d_dot_r()

```

void format_word_d_dot_r (
    VM * vm )

```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.9 format_word_decimal()

```

void format_word_decimal (
    VM * vm )

```

References [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.10 format_word_dot()

```

void format_word_dot (
    VM * vm )

```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.11 `format_word_dot_r()`

```
void format_word_dot_r (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.12 `format_word_dot_s()`

```
void format_word_dot_s (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [print_number_formatted\(\)](#), and [vm](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.13 `format_word_dump()`

```
void format_word_dump (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), and [vm_pop\(\)](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.14 `format_word_end_conversion()`

```
void format_word_end_conversion (
    VM * vm )
```

References [CELL\(\)](#), [conversion_pos](#), [VM::dsp](#), [ensure_pn\(\)](#), [VM::error](#), [VM::memory](#), [pn_buf](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.15 `format_word_hash()`

```
void format_word_hash (
    VM * vm )
```

References [base](#), [current_base\(\)](#), [digit_for\(\)](#), [div_ud_by_base\(\)](#), [VM::dsp](#), [VM::error](#), [format_word_hold\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [format_word_hash_s\(\)](#), and [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.16 `format_word_hash_s()`

```
void format_word_hash_s (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [format_word_hash\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [register_format_words\(\)](#).
 Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.17 `format_word_hex()`

```
void format_word_hex (
    VM * vm )
```

References [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.18 format_word_hold()

```
void format_word_hold (
    VM * vm )
```

References [conversion_pos](#), [VM::dsp](#), [ensure_pn\(\)](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [pn_buf](#), [pn_cap](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [format_word_hash\(\)](#), [format_word_sign\(\)](#), and [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.19 format_word_octal()

```
void format_word_octal (
    VM * vm )
```

References [VM::base_addr](#), [vm](#), and [vm_store_cell\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.20 format_word_question()

```
void format_word_question (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.21 format_word_sign()

```
void format_word_sign (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [format_word_hold\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.22 format_word_u_dot()

```
void format_word_u_dot (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.23 format_word_u_dot_r()

```
void format_word_u_dot_r (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [print_number_formatted\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_format_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.24 print_number_formatted()

```
static void print_number_formatted (
    VM * vm,
    cell_t n,
    int width,
    int is_unsigned ) [static]
```

Prints number with optional width field and sign handling.

References [base](#), [current_base\(\)](#), [digit_for\(\)](#), and [vm](#).

Referenced by [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dot_s\(\)](#), [format_word_question\(\)](#), [format_word_u_dot\(\)](#), and [format_word_u_dot_r\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.2.25 register_format_words()

```
void register_format_words (
    VM * vm )
```

Registers all formatting and numeric conversion words with the [VM](#).

Registers all formatting and conversion words with the Forth virtual machine.

This function registers the standard FORTH formatting words including:

- Number output words (., .R, U., U.R, D., D.R, .S, ?)
- Picture number formatting (<#, #, #S, #>, HOLD, SIGN)
- Base conversion (BASE, DECIMAL, HEX, OCTAL)
- Memory dump (DUMP)

Parameters

<code>vm</code>	Pointer to the virtual machine instance
-----------------	---

References [format_word_base\(\)](#), [format_word_begin_conversion\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_decimal\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dot_s\(\)](#), [format_word_dump\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_hex\(\)](#), [format_word_hold\(\)](#), [format_word_octal\(\)](#), [format_word_question\(\)](#), [format_word_sign\(\)](#), [format_word_u_dot\(\)](#), [format_word_u_dot_r\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.178.3 Variable Documentation

7.178.3.1 conversion_pos

```
int conversion_pos = 0 [static]
```

Current position in picture number buffer.

Referenced by [format_word_begin_conversion\(\)](#), [format_word_end_conversion\(\)](#), and [format_word_hold\(\)](#).

7.178.3.2 pn_buf

```
unsigned char* pn_buf = NULL [static]
```

Picture number buffer for numeric conversion.

Referenced by [ensure_pn\(\)](#), [format_word_begin_conversion\(\)](#), [format_word_end_conversion\(\)](#), and [format_word_hold\(\)](#).

7.178.3.3 pn_cap

```
size_t pn_cap = 0 [static]
```

Capacity of picture number buffer.

Referenced by [ensure_pn\(\)](#), [format_word_begin_conversion\(\)](#), and [format_word_hold\(\)](#).

7.179 src/word_source/include/acl_words.h File Reference

```
#include "vm.h"
```

Include dependency graph for `acl_words.h`: This graph shows which files directly or indirectly include this file:

Functions

- void [register_acl_words](#) (VM *vm)
- void [acl_inherit_entry](#) (DictEntry *src, DictEntry *dst)

7.179.1 Function Documentation

7.179.1.1 [acl_inherit_entry](#)()

```
void acl_inherit_entry (
    DictEntry * src,
    DictEntry * dst )
```

References [DictEntry::acl_allow](#), [DictEntry::acl_mode](#), [DictEntry::acl_pinned](#), and [DictEntry::acl_ttl](#).

Referenced by [forth_acl_inherit\(\)](#), and [test_acl_inherit\(\)](#).

Here is the caller graph for this function:

7.179.1.2 [register_acl_words](#)()

```
void register_acl_words (
    VM * vm )
```

References [forth_acl_allow_fetch\(\)](#), [forth_acl_allow_store\(\)](#), [forth_acl_heat_fetch\(\)](#), [forth_acl_inherit\(\)](#), [forth_acl_init_primitives\(\)](#), [forth_acl_mode_fetch\(\)](#), [forth_acl_mode_store\(\)](#), [forth_acl_pin\(\)](#), [forth_acl_pinned_query\(\)](#), [forth_acl_ttl_fetch\(\)](#), [forth_acl_ttl_store\(\)](#), [forth_acl_word_id\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.180 src/word_source/include/arithmetic_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for arithmetic_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_arithmetic_words](#) (VM *vm)
Registers all FORTH-79 arithmetic words with the virtual machine.

7.180.1 Function Documentation

7.180.1.1 [register_arithmetic_words](#)()

```
void register_arithmetic_words (
    VM * vm )
```

Registers all FORTH-79 arithmetic words with the virtual machine.

Registers the following words:

- + (add)
- - (subtract)
- * (multiply)
- / (divide)
- MOD (modulo)
- /MOD (divide with remainder)
- NEGATE
- ABS (absolute)

- MIN
- MAX
- 1+ (increment)
- 1- (decrement)
- 2* (multiply by 2)
- 2/ (divide by 2)
- STAR-SLASH (multiply then divide)

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Registers all FORTH-79 arithmetic words with the virtual machine.

This function registers all the standard FORTH-79 arithmetic operations including basic arithmetic, advanced math operations, increment/decrement operations, and comparison functions.

Parameters

<i>vm</i>	Pointer to the VM structure where words will be registered
-----------	--

References [arithmetic_word_abs\(\)](#), [arithmetic_word_max\(\)](#), [arithmetic_word_min\(\)](#), [arithmetic_word_minus\(\)](#), [arithmetic_word_multiply\(\)](#), [arithmetic_word_negate\(\)](#), [arithmetic_word_one_minus\(\)](#), [arithmetic_word_one_plus\(\)](#), [arithmetic_word_plus\(\)](#), [arithmetic_word_star_slash\(\)](#), [arithmetic_word_star_slash_mod\(\)](#), [arithmetic_word_two_divide\(\)](#), [arithmetic_word_two_minus\(\)](#), [arithmetic_word_two_multiply\(\)](#), [arithmetic_word_two_plus\(\)](#), [LOG_INFO](#), [log_message\(\)](#), [mixed_math_word_mod\(\)](#), [register_word\(\)](#), [vm](#), [word_divide\(\)](#), and [word_divmod\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.181 `src/word_source/include/block_words.h` File Reference

FORTH-79 Block and Buffer System Implementation.

```
#include "../..../include/vm.h"
```

Include dependency graph for `block_words.h`: This graph shows which files directly or indirectly include this file:

Data Structures

- struct [BlockBuffer](#)
Block buffer structure maintaining buffer state.

Macros

Block System Configuration

- `#define BLOCK_SIZE 1024`
Size of each block in bytes (1KB)
- `#define MAX_BLOCKS (VM_MEMORY_SIZE / BLOCK_SIZE)`
Maximum number of blocks (1024 blocks from 1MB memory)
- `#define BUFFER_COUNT 4`
Number of block buffers in memory.

Enumerations

- enum `buffer_type_t` { `BUFFER_TYPE_MEMORY` , `BUFFER_TYPE_HARDWARE` , `BUFFER_TYPE_STORAGE` , `BUFFER_TYPE_NETWORK` }

Block buffer types for different storage mechanisms.

Functions

- void `register_block_words` (VM *vm)
Registers all block and buffer words with the virtual machine.
- void `init_block_system` (VM *vm)
Initializes the block system.
- unsigned char * `get_block_buffer` (VM *vm, int block_num)
Gets a buffer containing the specified block.
- unsigned char * `get_empty_buffer` (VM *vm, int block_num)
Gets an empty buffer for the specified block.
- void `mark_buffer_dirty` (VM *vm)
Marks the current buffer as modified.
- void `save_all_buffers` (VM *vm)
Saves all modified buffers to storage.
- void `empty_all_buffers` (VM *vm)
Marks all buffers as empty.

7.181.1 Detailed Description

FORTH-79 Block and Buffer System Implementation.

This file implements the standard FORTH-79 block and buffer words:

- BLOCK (u – addr) Get block buffer address
- BUFFER (u – addr) Get empty buffer for block
- UPDATE (–) Mark current block as modified
- SAVE-BUFFERS (–) Save all modified blocks
- EMPTY-BUFFERS (–) Mark all buffers as empty
- FLUSH (–) Save buffers and empty them
- LIST (u –) List block contents
- LOAD (u –) Load and interpret block
- SCR (– addr) Variable: screen/block being listed
- THRU (u1 u2 –) Load blocks from u1 to u2
- > or → (–) Continue to next block

7.181.2 Macro Definition Documentation

7.181.2.1 BLOCK_SIZE

```
#define BLOCK_SIZE 1024
```

Size of each block in bytes (1KB)

7.181.2.2 BUFFER_COUNT

```
#define BUFFER_COUNT 4
```

Number of block buffers in memory.

7.181.2.3 MAX_BLOCKS

```
#define MAX_BLOCKS (VM_MEMORY_SIZE / BLOCK_SIZE)
```

Maximum number of blocks (1024 blocks from 1MB memory)

7.181.3 Enumeration Type Documentation

7.181.3.1 buffer_type_t

```
enum buffer_type_t
```

Block buffer types for different storage mechanisms.

Enumerator

BUFFER_TYPE_MEMORY	Direct memory access (current implementation)
BUFFER_TYPE_HARDWARE	Future: Hardware register blocks.
BUFFER_TYPE_STORAGE	Future: Persistent storage blocks.
BUFFER_TYPE_NETWORK	Future: Network packet buffers.

7.181.4 Function Documentation

7.181.4.1 empty_all_buffers()

```
void empty_all_buffers (
    VM * vm )
```

Marks all buffers as empty.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Marks all buffers as empty.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_get_buffer\(\)](#), [blk_get_total_blocks\(\)](#), [BLOCK_SIZE](#), [USER_BLOCKS_START](#), and [vm](#).
Referenced by [block_word_empty_buffers\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.181.4.2 get_block_buffer()

```
unsigned char * get_block_buffer (
    VM * vm,
    int block_num )
```

Gets a buffer containing the specified block.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>block_num</i>	Block number to retrieve

Returns

Pointer to the block buffer data

Gets a buffer containing the specified block.

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to retrieve

Returns

Pointer to block buffer or NULL if invalid

References [blk_get_buffer\(\)](#), [set_scr\(\)](#), and [vm](#).
Here is the call graph for this function:

7.181.4.3 get_empty_buffer()

```
unsigned char * get_empty_buffer (
    VM * vm,
    int block_num )
```

Gets an empty buffer for the specified block.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>block_num</i>	Block number to allocate

Returns

Pointer to the empty buffer data

Parameters

<i>vm</i>	Pointer to the VM instance
<i>block_num</i>	Block number to create

Returns

Pointer to empty block buffer or NULL if invalid

References [blk_get_empty_buffer\(\)](#), [set_scr\(\)](#), and [vm](#).
Here is the call graph for this function:

7.181.4.4 init_block_system()

```
void init_block_system (
    VM * vm )
```

Initializes the block system.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
<i>vm</i>	Pointer to the VM instance

References [vm](#).

7.181.4.5 mark_buffer_dirty()

```
void mark_buffer_dirty (
    VM * vm )
```

Marks the current buffer as modified.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Marks the current buffer as modified.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_update\(\)](#), [VM::scr_addr](#), [vm](#), and [vm_load_cell\(\)](#).

Referenced by [editor_word_edit\(\)](#), and [editor_word_s\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.181.4.6 register_block_words()

```
void register_block_words (
    VM * vm )
```

Registers all block and buffer words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Registers all block and buffer words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [block_word_block\(\)](#), [block_word_buffer\(\)](#), [block_word_empty_buffers\(\)](#), [block_word_flush\(\)](#), [block_word_list\(\)](#), [block_word_load\(\)](#), [block_word_next_block\(\)](#), [block_word_save_buffers\(\)](#), [block_word_scr\(\)](#), [block_word_thru\(\)](#), [block_word_update\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.181.4.7 save_all_buffers()

```
void save_all_buffers (
    VM * vm )
```

Saves all modified buffers to storage.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Saves all modified buffers to storage.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_flush\(\)](#), and [vm](#).

Referenced by [block_word_save_buffers\(\)](#), and [editor_word_edit\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.182 src/word_source/include/control_words.h File Reference

```
#include "../../include/vm.h"
```

Include dependency graph for control_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_control_words](#) (VM *vm)
Registers all control flow words with the Forth virtual machine.

7.183 src/word_source/include/defining_words.h File Reference

```
#include "../../include/vm.h"
```

Include dependency graph for defining_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_defining_words](#) (VM *vm)
Registers all FORTH-79 defining words with the virtual machine.

7.183.1 Function Documentation

7.183.1.1 register_defining_words()

```
void register_defining_words (
    VM * vm )
```

Registers all FORTH-79 defining words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers all FORTH-79 defining words with the virtual machine.

Registers core defining words like :, ;, CREATE, CONSTANT, VARIABLE, and their support words like IMMEDIATE, STATE, etc.

Parameters

<i>vm</i>	Pointer to the VM context
-----------	---

References [defining_runtime_does_rt\(\)](#), [defining_runtime_lit\(\)](#), [defining_word_bracket_compile\(\)](#), [defining_word_colon\(\)](#), [defining_word_compile\(\)](#), [defining_word_constant\(\)](#), [defining_word_create\(\)](#), [defining_word_does\(\)](#), [defining_word_immediate\(\)](#), [defining_word_left_bracket\(\)](#), [defining_word_literal\(\)](#), [defining_word_right_bracket\(\)](#), [defining_word_semicolon\(\)](#), [defining_word_state\(\)](#), [defining_word_variable\(\)](#), [dictionary_word_forget\(\)](#), [register_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).
Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.184 src/word_source/include/dictionary_heat_diagnostic_words.h File Reference

```
#include "../../include/vm.h"
```

Include dependency graph for dictionary_heat_diagnostic_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_dictionary_heat_diagnostic_words](#) (VM *vm)

Register all dictionary heat diagnostic words with the [VM](#).

7.184.1 Function Documentation

7.184.1.1 register_dictionary_heat_diagnostic_words()

```
void register_dictionary_heat_diagnostic_words (
    VM * vm )
```

Register all dictionary heat diagnostic words with the [VM](#).

References [forth_COMPARE_LOOKUPS\(\)](#), [forth_HEAT_PERCENTILES\(\)](#), [forth_LOOKUP_STRATEGY_FETCH\(\)](#), [forth_LOOKUP_STRATEGY_STORE\(\)](#), [forth_REORG_BUCKETS\(\)](#), [forth_SHOW_HEAT_OPTIMIZATION\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.185 src/word_source/include/dictionary_manipulation_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for dictionary_manipulation_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_dictionary_manipulation_words](#) ([VM](#) *[vm](#))

Register all dictionary manipulation words with the virtual machine.

7.185.1 Function Documentation

7.185.1.1 register_dictionary_manipulation_words()

```
void register_dictionary_manipulation_words (
    VM * vm )
```

Register all dictionary manipulation words with the virtual machine.

Parameters

vm	Pointer to the virtual machine instance
--------------------	---

This function registers all FORTH-79 standard dictionary manipulation words with the provided virtual machine instance. These words are essential for dictionary management and word definition functionality.

Register all dictionary manipulation words with the virtual machine.

Parameters

vm	Pointer to VM instance
--------------------	--

Registers standard FORTH-79 dictionary manipulation words

References [dictionary_m_word_cfa\(\)](#), [dictionary_m_word_find\(\)](#), [dictionary_m_word_hidden\(\)](#), [dictionary_m_word_interpret\(\)](#), [dictionary_m_word_left_bracket\(\)](#), [dictionary_m_word_lfa\(\)](#), [dictionary_m_word_link_from\(\)](#), [dictionary_m_word_name_to\(\)](#), [dictionary_m_word_nfa\(\)](#), [dictionary_m_word_pfa\(\)](#), [dictionary_m_word_right_bracket\(\)](#), [dictionary_m_word_smudge\(\)](#), [dictionary_m_word_state\(\)](#), [dictionary_m_word_tick\(\)](#), [dictionary_m_word_to_body\(\)](#), [dictionary_m_word_to_link\(\)](#), [dictionary_m_word_to_name\(\)](#), [dictionary_m_word_traverse\(\)](#), [register_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.186 src/word_source/include/dictionary_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for dictionary_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_dictionary_words](#) (VM *vm)
Registers all dictionary and compilation words with the Forth VM.

7.186.1 Function Documentation

7.186.1.1 register_dictionary_words()

```
void register_dictionary_words (
    VM * vm )
```

Registers all dictionary and compilation words with the Forth VM.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

References [dictionary_word_2comma\(\)](#), [dictionary_word_allot\(\)](#), [dictionary_word_c_comma\(\)](#), [dictionary_word_comma\(\)](#), [dictionary_word_here\(\)](#), [dictionary_word_latest\(\)](#), [dictionary_word_pad\(\)](#), [dictionary_word_sp_fetch\(\)](#), [dictionary_word_sp_store\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.187 src/word_source/include/double_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for double_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_double_words](#) (VM *vm)
Registers all double precision number manipulation words with the Forth virtual machine.

7.187.1 Function Documentation

7.187.1.1 register_double_words()

```
void register_double_words (
    VM * vm )
```

Registers all double precision number manipulation words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Registers all double precision number manipulation words with the Forth virtual machine.

Registers all double-precision number operations with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Note

This function must be called during [VM](#) initialization.

References [double_word_2drop\(\)](#), [double_word_2dup\(\)](#), [double_word_2over\(\)](#), [double_word_2r_fetch\(\)](#), [double_word_2r_from\(\)](#), [double_word_2rot\(\)](#), [double_word_2swap\(\)](#), [double_word_2to_r\(\)](#), [double_word_d_equals\(\)](#), [double_word_d_less\(\)](#), [double_word_d_minus\(\)](#), [double_word_d_plus\(\)](#), [double_word_d_two_slash\(\)](#), [double_word_d_two_star\(\)](#), [double_word_d_zero_equals\(\)](#), [double_word_d_zero_less\(\)](#), [double_word_dabs\(\)](#), [double_word_dmax\(\)](#), [double_word_dmin\(\)](#), [double_word_dnegate\(\)](#), [double_word_s_to_d\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.188 src/word_source/include/editor_words.h File Reference

```
#include ".../.../include/vm.h"
```

Include dependency graph for editor_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_editor_words](#) ([VM](#) *[vm](#))
Register all editor words with the Forth virtual machine.

7.188.1 Function Documentation

7.188.1.1 register_editor_words()

```
void register_editor_words (
    VM * vm )
```

Register all editor words with the Forth virtual machine.

Parameters

vm	Pointer to the Forth virtual machine instance
--------------------	---

References [editor_word_edit\(\)](#), [editor_word_l\(\)](#), [editor_word_s\(\)](#), [editor_word_show\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.189 src/word_source/include/format_words.h File Reference

```
#include ".../.../include/vm.h"
```

Include dependency graph for format_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_format_words](#) ([VM](#) *[vm](#))
Registers all formatting and conversion words with the Forth virtual machine.

7.189.1 Function Documentation

7.189.1.1 register_format_words()

```
void register_format_words (
    VM * vm )
```

Registers all formatting and conversion words with the Forth virtual machine.

Parameters

vm	Pointer to the Forth virtual machine instance
--------------------	---

Registers all formatting and conversion words with the Forth virtual machine.
This function registers the standard FORTH formatting words including:

- Number output words (., .R, U., U.R, D., D.R, .S, ?)
- Picture number formatting (<#, #, #S, #>, HOLD, SIGN)
- Base conversion (BASE, DECIMAL, HEX, OCTAL)
- Memory dump (DUMP)

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

References [format_word_base\(\)](#), [format_word_begin_conversion\(\)](#), [format_word_d_dot\(\)](#), [format_word_d_dot_r\(\)](#), [format_word_decimal\(\)](#), [format_word_dot\(\)](#), [format_word_dot_r\(\)](#), [format_word_dot_s\(\)](#), [format_word_dump\(\)](#), [format_word_end_conversion\(\)](#), [format_word_hash\(\)](#), [format_word_hash_s\(\)](#), [format_word_hex\(\)](#), [format_word_hold\(\)](#), [format_word_octal\(\)](#), [format_word_question\(\)](#), [format_word_sign\(\)](#), [format_word_u_dot\(\)](#), [format_word_u_dot_r\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.190 src/word_source/include/io_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for io_words.h: This graph shows which files directly or indirectly include this file:

Macros

- [#define IO_WORDS_H](#)
 < Include guard

Functions

- void [register_io_words](#) ([VM](#) **vm*)
 Registers all I/O related FORTH words with the virtual machine.

7.190.1 Macro Definition Documentation

7.190.1.1 IO_WORDS_H

```
#define IO_WORDS_H
< Include guard
Include guard
```

7.190.2 Function Documentation

7.190.2.1 register_io_words()

```
void register_io_words (
    VM * vm )
```

Registers all I/O related FORTH words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers the following FORTH-79 I/O & Terminal Words:

- EMIT (c –) Output character c
- CR (–) Output carriage return
- KEY (– c) Input character from keyboard
- ?TERMINAL (– flag) True if key pressed
- TYPE (addr u –) Output u characters from addr
- SPACE (–) Output one space
- SPACES (n –) Output n spaces

Registers all I/O related FORTH words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Registers all FORTH-79 I/O and terminal words with the virtual machine

References [io_word_cr\(\)](#), [io_word_emit\(\)](#), [io_word_key\(\)](#), [io_word_question_terminal\(\)](#), [io_word_space\(\)](#), [io_word_spaces\(\)](#), [io_word_type\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.191 src/word_source/include/logical_words.h File Reference

```
#include ".../.../include/vm.h"
```

Include dependency graph for logical_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_logical_words](#) (VM *vm)
Registers all logical and comparison words with the Forth virtual machine.

7.191.1 Function Documentation

7.191.1.1 register_logical_words()

```
void register_logical_words (
    VM * vm )
```

Registers all logical and comparison words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the Forth virtual machine instance
-----------	---

Registers all logical and comparison words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers logical operations (AND, OR, XOR, NOT), comparison operations (=, <>, <, >, etc.), and constant words (TRUE, FALSE) with the virtual machine.

References [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_true\(\)](#),

[logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.192 src/word_source/include/memory_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for `memory_words.h`: This graph shows which files directly or indirectly include this file:

Functions

- void [register_memory_words](#) (VM *vm)

Registers all memory operation words with the Forth virtual machine.

7.192.1 Function Documentation

7.192.1.1 register_memory_words()

```
void register_memory_words (
    VM * vm )
```

Registers all memory operation words with the Forth virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers all memory operation words with the Forth virtual machine.

Registers all standard memory words (@, !, C@, C!, etc.) with the [VM](#)'s word dictionary.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.193 src/word_source/include/mixed_arithmetic_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for `mixed_arithmetic_words.h`: This graph shows which files directly or indirectly include this file:

Functions

- void [register_mixed_arithmetic_words](#) (VM *vm)

7.193.1 Function Documentation

7.193.1.1 register_mixed_arithmetic_words()

```
void register_mixed_arithmetic_words (
    VM * vm )
```

References [mixed_math_word_m_minus\(\)](#), [mixed_math_word_m_plus\(\)](#), [mixed_math_word_m_slash_mod\(\)](#), [mixed_math_word_m_star\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_slash_mod\(\)](#), [mixed_math_word_star_slash\(\)](#), [mixed_math_word_star_slash_mod\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.194 src/word_source/include/physics_benchmark_words.h File Reference

```
#include "../..../include/vm.h"
```

Include dependency graph for physics_benchmark_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_physics_benchmark_words](#) ([VM](#) *vm)
Register all physics benchmark words with the [VM](#).

7.194.1 Function Documentation

7.194.1.1 register_physics_benchmark_words()

```
void register_physics_benchmark_words (
    VM * vm )
```

Register all physics benchmark words with the [VM](#).

References [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [forth_PHYSICS_BUILD_INFO\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), [forth_PHYSICS_TOGGLE_CACHE\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.195 src/word_source/include/physics_diagnostic_words.h File Reference

```
#include "../..../include/vm.h"
```

Include dependency graph for physics_diagnostic_words.h:

Functions

- void [forth_PHYSICS_WORD_METRICS](#) ([VM](#) *vm)
Display detailed physics metrics for the most recently executed word.
- void [forth_PHYSICS_CALC_KNOBS](#) ([VM](#) *vm)
Calculate and show knob adjustments based on current thermal metrics.
- void [forth_PHYSICS_BURN](#) ([VM](#) *vm)
Execute a word repeatedly (burn) and show thermal feedback.
- void [forth_PHYSICS_SHOW_FEEDBACK](#) ([VM](#) *vm)
Display the complete feedback loop: inputs → math → knobs → effect.
- void [register_physics_diagnostic_words](#) ([VM](#) *vm)
Register all physics diagnostic words with the [VM](#).

7.195.1 Function Documentation

7.195.1.1 forth_PHYSICS_BURN()

```
void forth_PHYSICS_BURN (
    VM * vm )
```

Execute a word repeatedly (burn) and show thermal feedback.

Execute a word repeatedly (burn) and show thermal feedback.

Execute the top-of-stack word n times and show thermal feedback.

Usage: : HOTTEST 100 0 DO 1 2 + 3 * LOOP ; HOTTEST 10 PHYSICS-BURN (execute 10 times) PHYSICS-↔
SHOW-FEEDBACK

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::func](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.195.1.2 forth_PHYSICS_CALC_KNOBS()

```
void forth_PHYSICS_CALC_KNOBS (
    VM * vm )
```

Calculate and show knob adjustments based on current thermal metrics.

Calculate and show knob adjustments based on current thermal metrics.

Calculate what knobs should be adjusted based on current word metrics. Shows:

- Thermal pressure (0.0 to 1.0)
- Recommended priority adjustment
- Recommended sampling rate
- Stack depth limit
- Core affinity preference

References [VM::error](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.195.1.3 forth_PHYSICS_SHOW_FEEDBACK()

```
void forth_PHYSICS_SHOW_FEEDBACK (
    VM * vm )
```

Display the complete feedback loop: inputs → math → knobs → effect.

Display the complete feedback loop: inputs → math → knobs → effect.

Display the complete feedback loop for the most recent word. Shows: Metrics → Calculations → Knobs → Effect

References [DictPhysics::avg_latency_ns](#), [VM::error](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictPhysics::mass_bytes](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.195.1.4 forth_PHYSICS_WORD_METRICS()

```
void forth_PHYSICS_WORD_METRICS (
    VM * vm )
```

Display detailed physics metrics for the most recently executed word.

Display detailed physics metrics for the most recently executed word.

Display detailed physics metrics for a word. Looks up the most recently executed word and shows its state.

References [DictPhysics::avg_latency_ns](#), [VM::error](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictPhysics::mass_bytes](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.195.1.5 register_physics_diagnostic_words()

```
void register_physics_diagnostic_words (
    VM * vm )
```


Register all physics diagnostic words with the [VM](#).

References [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#), [register_word\(\)](#), and [vm](#).

Here is the call graph for this function:

7.196 src/word_source/include/physics_freeze_words.h File Reference

```
#include "../include/vm.h"
```

Include dependency graph for physics_freeze_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_physics_freeze_words](#) ([VM](#) *[vm](#))

Register all Phase 2 freeze/decay control words with the [VM](#).

7.196.1 Function Documentation

7.196.1.1 register_physics_freeze_words()

```
void register_physics_freeze_words (
    VM * vm )
```

Register all Phase 2 freeze/decay control words with the [VM](#).

References [forth_ALL_HEATS\(\)](#), [forth_DECAY_RATE_FETCH\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.197 src/word_source/include/physics_pipelining_diagnostic_words.h File Reference

```
#include "../include/vm.h"
```

Include dependency graph for physics_pipelining_diagnostic_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_physics_pipelining_diagnostic_words](#) ([VM](#) *[vm](#))

Register all pipelining diagnostic words Called during [VM](#) initialization.

7.197.1 Function Documentation

7.197.1.1 register_physics_pipelining_diagnostic_words()

```
void register_physics_pipelining_diagnostic_words (
    VM * vm )
```

Register all pipelining diagnostic words Called during [VM](#) initialization.

Register all pipelining diagnostic words Called during [VM](#) initialization.

References [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_ENABLE\(\)](#), [forth_PIPELINING_RESET_ALL\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [vm](#), and [vm_create_word\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.198 src/word_source/include/return_stack_words.h File Reference

```
#include "../../include/vm.h"
```

Include dependency graph for return_stack_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_return_stack_words](#) (VM *vm)
Registers all return stack manipulation words with the virtual machine.

7.198.1 Function Documentation

7.198.1.1 register_return_stack_words()

```
void register_return_stack_words (
    VM * vm )
```

Registers all return stack manipulation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers all return stack manipulation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Registers >R, R>, and R@ words. Direct stack addressing words (RP!, RP@) are deliberately omitted for FORTH-79 compliance.

References [register_word\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [return_stack_word_to_r\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.199 src/word_source/include/stack_words.h File Reference

```
#include "../../include/vm.h"
```

Include dependency graph for stack_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_stack_words](#) (VM *vm)
Registers all stack operation words with the virtual machine.

7.199.1 Function Documentation

7.199.1.1 register_stack_words()

```
void register_stack_words (
    VM * vm )
```

Registers all stack operation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>vm</i>	Pointer to the virtual machine state

Registers all FORTH-79 standard stack operation words with the word registry.

References [register_word\(\)](#), [stack_word_depth\(\)](#), [stack_word_drop\(\)](#), [stack_word_dup\(\)](#), [stack_word_minus_rot\(\)](#), [stack_word_over\(\)](#), [stack_word_pick\(\)](#), [stack_word_question_dup\(\)](#), [stack_word_roll\(\)](#), [stack_word_rot\(\)](#), [stack_word_swap\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200 src/word_source/include/starforth_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for starforth_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_starforth_words](#) (VM *vm)

Register all StarForth words with VM.

StarForth Implementation Vocabulary Words

- void [starforth_word_entropy_fetch](#) (VM *vm)
Fetch entropy value from address.
- void [starforth_word_entropy_store](#) (VM *vm)
Store entropy value to address.
- void [starforth_word_word_entropy](#) (VM *vm)
Calculate word entropy.
- void [starforth_word_reset_entropy](#) (VM *vm)
Reset entropy values.
- void [starforth_word_top_words](#) (VM *vm)
Display top N words by entropy.
- void [starforth_word_paren_dash](#) (VM *vm)
Shebang-style comment for init-4.4th metadata.
- void [starforth_word_init](#) (VM *vm)
Initialize system from init-4.4th.
- void [starforth_word_seed](#) (VM *vm)
Set PRNG seed for reproducible random sequences.
- void [starforth_word_random](#) (VM *vm)
Generate bounded random number.
- void [starforth_word_wait](#) (VM *vm)
Wait/sleep for specified milliseconds.

7.200.1 Function Documentation

7.200.1.1 register_starforth_words()

```
void register_starforth_words (
    VM * vm )
```

Register all StarForth words with [VM](#).

Parameters

<i>vm</i>	Pointer to VM instance where words will be registered
-----------	---

Register all StarForth words with [VM](#).

Registers all StarForth-specific words and creates the STARFORTH vocabulary

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [register_word\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_execution_heat_store\(\)](#), [starforth_word_init\(\)](#), [starforth_word_paren_dash\(\)](#), [starforth_word_random\(\)](#), [starforth_word_reset_execution_heat\(\)](#), [starforth_word_seed\(\)](#), [starforth_word_top_words\(\)](#), [starforth_word_version\(\)](#), [starforth_word_wait\(\)](#), [starforth_word_word_execution_starforth_word_zuse_authenticate\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.2 starforth_word_entropy_fetch()

```
void starforth_word_entropy_fetch (
    VM * vm )
```

Fetch entropy value from address.

Parameters

<i>vm</i>	Pointer to VM instance @stack (addr – n)
-----------	--

7.200.1.3 starforth_word_entropy_store()

```
void starforth_word_entropy_store (
    VM * vm )
```

Store entropy value to address.

Parameters

<i>vm</i>	Pointer to VM instance @stack (n addr –)
-----------	--

7.200.1.4 starforth_word_init()

```
void starforth_word_init (
    VM * vm )
```

Initialize system from init-4.4th.

Parameters

<i>vm</i>	Pointer to VM instance @stack (–)
-----------	---

Initialize system from init-4.4th.

Reads ./capsules/init.4th (or ROMFS in L4Re), parses Block headers, copies blocks sequentially starting at block 1, then executes them. Stack effect: (–)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [blk_get_buffer\(\)](#), [VM::current_executing_entry](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::func](#), [VM::halted](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_touch\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [profiler_word_exit\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_find_word\(\)](#), [vm_interpret\(\)](#), and [vm_push\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.5 starforth_word_paren_dash()

```
void starforth_word_paren_dash (
    VM * vm )
```

Shebang-style comment for init-4.4th metadata.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

Shebang-style comment for init-4.4th metadata.

Starts with "(- " and consumes input until closing ")" Used for marking blocks that should be extracted to init.4th
Stack effect: (-)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [LOG_DEBUG](#), [log_message\(\)](#), [LOG_WARN](#), and [vm](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.6 starforth_word_random()

```
void starforth_word_random (
    VM * vm )
```

Generate bounded random number.

Parameters

<i>vm</i>	Pointer to VM instance @stack (lo hi - n)
-----------	---

Note

Returns random value in inclusive range [lo, hi]

Stack effect: (lo hi - n) Returns a pseudo-random number in the inclusive range [lo, hi].

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [prng_next\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.7 starforth_word_reset_entropy()

```
void starforth_word_reset_entropy (
    VM * vm )
```

Reset entropy values.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

7.200.1.8 starforth_word_seed()

```
void starforth_word_seed (
```

```
VM * vm )
```

Set PRNG seed for reproducible random sequences.

Parameters

<i>vm</i>	Pointer to VM instance @stack (n –)
-----------	---

Set PRNG seed for reproducible random sequences.

Stack effect: (n –) Sets the internal PRNG state for reproducible random sequences.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [g_prng_state](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.9 starforth_word_top_words()

```
void starforth_word_top_words (
    VM * vm )
```

Display top N words by entropy.

Parameters

<i>vm</i>	Pointer to VM instance @stack (n –)
-----------	---

Display top N words by entropy.

Stack effect: (n –)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [entry](#), [VM::error](#), [execution_heat](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [word_count](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.10 starforth_word_wait()

```
void starforth_word_wait (
    VM * vm )
```

Wait/sleep for specified milliseconds.

Parameters

<i>vm</i>	Pointer to VM instance @stack (n –)
-----------	---

Note

General-purpose delay, also useful for [VM](#) cooling between benchmarks

Stack effect: (n –) Pauses execution for n milliseconds. Useful for:

- Allowing VM/system to cool between benchmark phases
- General-purpose timing in FORTH programs

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.200.1.11 starforth_word_word_entropy()

```
void starforth_word_word_entropy (
    VM * vm )
```

Calculate word entropy.

Parameters

<i>vm</i>	Pointer to VM instance @stack (-)
-----------	---

7.201 src/word_source/include/string_words.h File Reference

FORTH-79 Standard string and text processing words implementation.

```
#include "../.../include/vm.h"
```

Include dependency graph for string_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_string_words](#) ([VM](#) **vm*)

COUNT (addr1 – addr2 u) Get string length and address Returns the character count and address of a counted string.

7.201.1 Detailed Description

FORTH-79 Standard string and text processing words implementation.

This file contains the implementation of FORTH-79 standard string and text processing words. These words provide functionality for string manipulation, input handling, and parsing.

7.201.2 Function Documentation**7.201.2.1 register_string_words()**

```
void register_string_words (
    VM * vm )
```

COUNT (addr1 – addr2 u) Get string length and address Returns the character count and address of a counted string.

EXPECT (addr u –) Accept input line Accepts up to u characters of input and stores them at addr

SPAN (– addr) Variable: number of characters input Contains the count of characters actually received by EXPECT

QUERY (–) Accept input into TIB Accepts a line of input into the Terminal Input Buffer

TIB (– addr) Terminal input buffer address Returns the address of the Terminal Input Buffer

WORD (c – addr) Parse next word, delimited by c Parses the next word from input stream using c as delimiter

>IN (– addr) Input stream pointer Contains the offset into the input buffer

SOURCE (– addr u) Input source specification Returns current input buffer address and length

BL (– c) ASCII space character (32) Pushes ASCII space character value onto stack

FIND (addr – addr flag) Find word in dictionary Searches dictionary for word, returns status flag

' (– xt) Get execution token of next word Returns execution token of next word in input stream

[] (– xt) Compile execution token Compiles execution token as literal

LITERAL (n –) Compile literal number Compiles top stack value as literal
 [LITERAL] (n –) Compile literal at compile time Compiles literal during compilation
 CONVERT (d1 addr1 – d2 addr2) Convert string to number Converts string starting at addr1 to number
 NUMBER (addr – n flag) Convert string to single number Converts string to single precision number
 ENCLOSE (addr c – addr1 n1 n2 n3) Parse for delimiter Parses string for delimiter character
 Register all string processing words with [VM](#)

Parameters

<i>vm</i>	Pointer to VM instance where words will be registered
-----------	---

References [register_word\(\)](#), [string_runtime_s_quote\(\)](#), [string_word_bl\(\)](#), [string_word_blank\(\)](#), [string_word_bracket_literal\(\)](#), [string_word_bracket_tick\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_literal\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_query\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_to_in\(\)](#), [string_word_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.202 src/word_source/include/system_words.h File Reference

```
#include "../.../include/vm.h"
```

Include dependency graph for system_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_system_words](#) ([VM](#) **vm*)
Registers all system words with the virtual machine.

7.202.1 Function Documentation

7.202.1.1 register_system_words()

```
void register_system_words (
    VM * vm )
```

Registers all system words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

This function registers all FORTH-79 system and environment words with the specified virtual machine instance, making them available for use in the FORTH environment.

References [forth_paren\(\)](#), [register_word\(\)](#), [runtime_abortq\(\)](#), [system_word_79_standard\(\)](#), [system_word_abort\(\)](#), [system_word_abort_quote\(\)](#), [system_word_bye\(\)](#), [system_word_cold\(\)](#), [system_word_nop\(\)](#), [system_word_page\(\)](#), [system_word_quit\(\)](#), [system_word_save_system\(\)](#), [system_word_see\(\)](#), [system_word_vlist\(\)](#), [system_word_warm\(\)](#), [system_word_words\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.203 src/word_source/include/vocabulary_words.h File Reference

```
#include "vm.h"
```

Include dependency graph for vocabulary_words.h: This graph shows which files directly or indirectly include this file:

Functions

- void [register_vocabulary_words](#) (VM *vm)
Registers all vocabulary-related words in the Forth system.
- void [vocabulary_create_vocabulary_direct](#) (VM *vm, const char *name)
Direct vocabulary creation (kernel bootstrap)
- void [vocabulary_word_definitions](#) (VM *vm)
DEFINITIONS word implementation (kernel bootstrap)
- void [vocabulary_word_forth](#) (VM *vm)
FORTH word implementation - selects FORTH vocabulary.

7.203.1 Function Documentation

7.203.1.1 register_vocabulary_words()

```
void register_vocabulary_words (
    VM * vm )
```

Registers all vocabulary-related words in the Forth system.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Registers all vocabulary-related words in the Forth system.

Registers VOCABULARY, DEFINITIONS, CONTEXT, CURRENT, FORTH, ORDER, and (FIND)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [init_vocabulary_system\(\)](#), [register_word\(\)](#), [vm](#), [vocabulary_word_context\(\)](#), [vocabulary_word_current\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), [vocabulary_word_order\(\)](#), [vocabulary_word_paren_find\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.203.1.2 vocabulary_create_vocabulary_direct()

```
void vocabulary_create_vocabulary_direct (
    VM * vm,
    const char * name )
```

Direct vocabulary creation (kernel bootstrap)

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Vocabulary name

References [console_println\(\)](#), [console_puts\(\)](#), [entry](#), [VM::error](#), [VM::here](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [name](#), [vm](#), [vm_allot\(\)](#), [vm_create_word\(\)](#), [vm_store_cell\(\)](#), [vocab_safe_len\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_debug_print_hex\(\)](#), and [vocabulary_select_runtime\(\)](#).

Referenced by [vm_bootstrap_root_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.203.1.3 vocabulary_word_definitions()

```
void vocabulary_word_definitions (
```

```
VM * vm )
```

DEFINITIONS word implementation (kernel bootstrap)

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

DEFINITIONS word implementation (kernel bootstrap)

Makes new words be defined in the CONTEXT vocabulary

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [context_vocab](#), [current_vocab](#), [init_vocabulary_system\(\)](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [register_vocabulary_words\(\)](#), and [vm_bootstrap_root_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.203.1.4 vocabulary_word_forth()

```
void vocabulary_word_forth (
    VM * vm )
```

FORTH word implementation - selects FORTH vocabulary.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

FORTH word implementation - selects FORTH vocabulary.

Makes FORTH the CONTEXT vocabulary

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [context_vocab](#), [forth_vocab](#), [init_vocabulary_system\(\)](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.204 src/word_source/io_words.c File Reference

```
#include "include/io_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include <stdio.h>
```

Include dependency graph for io_words.c:

Functions

- static void [io_word_emit](#) ([VM](#) *[vm](#))
FORTH word EMIT - Output character to terminal.
- static void [io_word_cr](#) ([VM](#) *[vm](#))
FORTH word CR - Output carriage return.
- static void [io_word_key](#) ([VM](#) *[vm](#))

- FORTH word KEY - Input character from terminal.*
- static void `io_word_question_terminal` (`VM *vm`)
FORTH word ?TERMINAL - Check if input is available.
- static void `io_word_type` (`VM *vm`)
FORTH word TYPE - Output string of characters.
- static void `io_word_space` (`VM *vm`)
FORTH word SPACE - Output one space character.
- static void `io_word_spaces` (`VM *vm`)
FORTH word SPACES - Output multiple spaces.
- void `register_io_words` (`VM *vm`)
Register all I/O words with the VM.

7.204.1 Function Documentation

7.204.1.1 `io_word_cr()`

```
static void io_word_cr (
    VM * vm ) [static]
```

FORTH word CR - Output carriage return.

Parameters

<code>vm</code>	Pointer to the <code>VM</code> structure
-----------------	--

Stack effect: (–) Outputs a newline character to the terminal

Referenced by `register_io_words()`.

Here is the caller graph for this function:

7.204.1.2 `io_word_emit()`

```
static void io_word_emit (
    VM * vm ) [static]
```

FORTH word EMIT - Output character to terminal.

Parameters

<code>vm</code>	Pointer to the <code>VM</code> structure
-----------------	--

Stack effect: (c –) Outputs the character from top of stack to the terminal

References `VM::data_stack`, `VM::dsp`, `VM::error`, and `vm`.

Referenced by `register_io_words()`.

Here is the caller graph for this function:

7.204.1.3 `io_word_key()`

```
static void io_word_key (
    VM * vm ) [static]
```

FORTH word KEY - Input character from terminal.

Parameters

<code>vm</code>	Pointer to the <code>VM</code> structure
-----------------	--

Stack effect: (– c) Reads one character from terminal and pushes it to stack

References `VM::data_stack`, `VM::dsp`, `VM::error`, `STACK_SIZE`, and `vm`.

Referenced by `register_io_words()`.

Here is the caller graph for this function:

7.204.1.4 io_word_question_terminal()

```
static void io_word_question_terminal (
    VM * vm ) [static]
```

FORTH word ?TERMINAL - Check if input is available.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Stack effect: (– flag) Pushes true if input is available, false otherwise

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_io_words\(\)](#).

Here is the caller graph for this function:

7.204.1.5 io_word_space()

```
static void io_word_space (
    VM * vm ) [static]
```

FORTH word SPACE - Output one space character.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Stack effect: (–) Outputs a single space character to terminal

Referenced by [register_io_words\(\)](#).

Here is the caller graph for this function:

7.204.1.6 io_word_spaces()

```
static void io_word_spaces (
    VM * vm ) [static]
```

FORTH word SPACES - Output multiple spaces.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Stack effect: (n –) Outputs n space characters to terminal

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), and [vm](#).

Referenced by [register_io_words\(\)](#).

Here is the caller graph for this function:

7.204.1.7 io_word_type()

```
static void io_word_type (
    VM * vm ) [static]
```

FORTH word TYPE - Output string of characters.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

Stack effect: (addr u –) Outputs u characters from memory starting at addr

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [vm](#), [VM_MEMORY_SIZE](#), and [vm_pop\(\)](#).

Referenced by [register_io_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.204.1.8 register_io_words()

```
void register_io_words (
    VM * vm )
```

Register all I/O words with the VM.

Registers all I/O related FORTH words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	-----------------------------

Registers all FORTH-79 I/O and terminal words with the virtual machine

References [io_word_cr\(\)](#), [io_word_emit\(\)](#), [io_word_key\(\)](#), [io_word_question_terminal\(\)](#), [io_word_space\(\)](#), [io_word_spaces\(\)](#), [io_word_type\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205 src/word_source/logical_words.c File Reference

```
#include "include/logical_words.h"
#include "../include/log.h"
#include "../include/word_registry.h"
Include dependency graph for logical_words.c:
```

Macros

- `#define FORTH_TRUE ((cell_t)-1)`
FORTH-79 TRUE value (-1, all bits set)
- `#define FORTH_FALSE ((cell_t)0)`
FORTH-79 FALSE value (0)

Functions

- static void [logical_word_and](#) (VM *vm)
FORTH word: AND (n1 n2 – n3)
- static void [logical_word_or](#) (VM *vm)
- static void [logical_word_xor](#) (VM *vm)
- static void [logical_word_not](#) (VM *vm)
- static void [logical_word_zero_equals](#) (VM *vm)
- static void [logical_word_zero_less](#) (VM *vm)
- static void [logical_word_zero_greater](#) (VM *vm)
- static void [logical_word_equals](#) (VM *vm)
- static void [logical_word_not_equals](#) (VM *vm)
- static void [logical_word_less_than](#) (VM *vm)
- static void [logical_word_greater_than](#) (VM *vm)
- static void [logical_word_u_less_than](#) (VM *vm)
- static void [logical_word_u_greater_than](#) (VM *vm)
- static void [logical_word_within](#) (VM *vm)
- static void [logical_word_true](#) (VM *vm)
- static void [logical_word_false](#) (VM *vm)
- void [logical_word_zero_not_equal](#) (VM *vm)
- void [register_logical_words](#) (VM *vm)
Registers all FORTH-79 logical and comparison words with the VM.

7.205.1 Macro Definition Documentation

7.205.1.1 FORTH_FALSE

```
#define FORTH_FALSE ((cell_t)0)
FORTH-79 FALSE value (0)
```

7.205.1.2 FORTH_TRUE

```
#define FORTH_TRUE ((cell_t)-1)
FORTH-79 TRUE value (-1, all bits set)
```

7.205.2 Function Documentation

7.205.2.1 logical_word_and()

```
static void logical_word_and (
    VM * vm ) [static]
FORTH word: AND ( n1 n2 – n3 )
```

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

Performs bitwise AND operation on top two stack values. Stack effect: (n1 n2 – n3) where n3 = n1 AND n2
References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.2 logical_word_equals()

```
static void logical_word_equals (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.3 logical_word_false()

```
static void logical_word_false (
    VM * vm ) [static]
```

References [FORTH_FALSE](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.4 logical_word_greater_than()

```
static void logical_word_greater_than (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.5 logical_word_less_than()

```
static void logical_word_less_than (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.6 logical_word_not()

```
static void logical_word_not (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.7 logical_word_not_equals()

```
static void logical_word_not_equals (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.8 logical_word_or()

```
static void logical_word_or (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.9 logical_word_true()

```
static void logical_word_true (
    VM * vm ) [static]
```

References [FORTH_TRUE](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.10 logical_word_u_greater_than()

```
static void logical_word_u_greater_than (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.11 logical_word_u_less_than()

```
static void logical_word_u_less_than (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.12 logical_word_within()

```
static void logical_word_within (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.13 `logical_word_xor()`

```
static void logical_word_xor (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.14 `logical_word_zero_equals()`

```
static void logical_word_zero_equals (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.15 `logical_word_zero_greater()`

```
static void logical_word_zero_greater (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.16 `logical_word_zero_less()`

```
static void logical_word_zero_less (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [FORTH_FALSE](#), [FORTH_TRUE](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.17 `logical_word_zero_not_equal()`

```
void logical_word_zero_not_equal (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_logical_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.205.2.18 `register_logical_words()`

```
void register_logical_words (
    VM * vm )
```

Registers all FORTH-79 logical and comparison words with the [VM](#).

Registers all logical and comparison words with the Forth virtual machine.

Parameters

<code>vm</code>	Pointer to the virtual machine instance
-----------------	---

Registers logical operations (AND, OR, XOR, NOT), comparison operations (=, <=>, <, >, etc.), and constant words (TRUE, FALSE) with the virtual machine.

References [logical_word_and\(\)](#), [logical_word_equals\(\)](#), [logical_word_false\(\)](#), [logical_word_greater_than\(\)](#), [logical_word_less_than\(\)](#), [logical_word_not\(\)](#), [logical_word_not_equals\(\)](#), [logical_word_or\(\)](#), [logical_word_true\(\)](#), [logical_word_u_greater_than\(\)](#), [logical_word_u_less_than\(\)](#), [logical_word_within\(\)](#), [logical_word_xor\(\)](#), [logical_word_zero_equals\(\)](#), [logical_word_zero_greater\(\)](#), [logical_word_zero_less\(\)](#), [logical_word_zero_not_equal\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206 src/word_source/memory_words.c File Reference

```
#include "include/memory_words.h"
#include "../include/word_registry.h"
#include "vm.h"
#include "../include/log.h"
#include <string.h>
```

Include dependency graph for memory_words.c:

Functions

- void [memory_word_fetch](#) (VM *vm)
Implements FORTH word '@' - Fetch cell from VM memory.
- void [memory_word_store](#) (VM *vm)
Implements FORTH word '!' - Store cell into VM memory.
- void [memory_word_cfetch](#) (VM *vm)
Implements FORTH word 'C@' - Fetch byte from VM memory.
- void [memory_word_cstore](#) (VM *vm)
Implements FORTH word 'C!' - Store byte into VM memory.
- void [memory_word_plus_store](#) (VM *vm)
Implements FORTH word '+!' - Add to contents of cell.
- void [memory_word_minus_store](#) (VM *vm)
Implements FORTH word '-!' - Subtract from contents of cell.
- void [memory_word_fill](#) (VM *vm)
Implements FORTH word 'FILL' - Fill memory with byte value.
- void [memory_word_move](#) (VM *vm)
Implements FORTH word 'MOVE' - Copy memory region.
- void [memory_word_erase](#) (VM *vm)
Implements FORTH word 'ERASE' - Zero memory region.
- void [memory_word_2fetch](#) (VM *vm)
Implements FORTH word '2@' - Fetch double cell.
- void [memory_word_2store](#) (VM *vm)
Implements FORTH word '2!' - Store double cell.
- static void [memory_word_cells](#) (VM *vm)
Implements FORTH word 'CELLS' - Scale by cell size.
- void [register_memory_words](#) (VM *vm)
Register all memory manipulation words with the VM.

7.206.1 Function Documentation

7.206.1.1 memory_word_2fetch()

```
void memory_word_2fetch (
    VM * vm )
```

Implements FORTH word '2@' - Fetch double cell.

Stack effect: (addr – x_low x_high) Fetches two consecutive cells from memory, pushing low cell then high cell.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_cell\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.2 memory_word_2store()

```
void memory_word_2store (
    VM * vm )
```

Implements FORTH word '2!' - Store double cell.

Stack effect: (*x_low* *x_high* *addr* -) Stores two consecutive cells to memory, low cell at *addr*, high cell at *addr*+cell.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).
 Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.3 memory_word_cells()

```
static void memory_word_cells (
    VM * vm ) [static]
```

Implements FORTH word 'CELLS' - Scale by cell size.

Stack effect: (*n* - *n'*) Multiplies top of stack by cell size in bytes.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.4 memory_word_cfetch()

```
void memory_word_cfetch (
    VM * vm )
```

Implements FORTH word 'C@' - Fetch byte from [VM](#) memory.

Stack effect: (*addr* - *c*) Fetches a byte value from the given memory address in [VM](#) memory.

Parameters

<i>vm</i>	Pointer to the VM structure
-----------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).
 Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.5 memory_word_cstore()

```
void memory_word_cstore (
    VM * vm )
```

Implements FORTH word 'C!' - Store byte into [VM](#) memory.

Stack effect: (c addr –) Stores a byte value at the given memory address in [VM](#) memory.

Parameters

vm	Pointer to the VM structure
--------------------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.6 memory_word_erase()

```
void memory_word_erase (
    VM * vm )
```

Implements FORTH word 'ERASE' - Zero memory region.

Stack effect: (addr len –) Sets len bytes to zero starting at addr.

Parameters

vm	Pointer to the VM structure
--------------------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.7 memory_word_fetch()

```
void memory_word_fetch (
    VM * vm )
```

Implements FORTH word '@' - Fetch cell from [VM](#) memory.

Stack effect: (addr – n) Fetches a cell value from the given memory address in [VM](#) memory.

Parameters

vm	Pointer to the VM structure
--------------------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_cell\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.8 memory_word_fill()

```
void memory_word_fill (
    VM * vm )
```

Implements FORTH word 'FILL' - Fill memory with byte value.

Stack effect: (addr len c –) Fills len bytes of memory starting at addr with the value c.

Parameters

vm	Pointer to the VM structure
--------------------	---

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.9 memory_word_minus_store()

```
void memory_word_minus_store (
```

```
VM * vm )
```

Implements FORTH word '-' - Subtract from contents of cell.

Stack effect: (n addr –) Subtracts n from the contents of the cell at the given address.

Parameters

vm	Pointer to the VM structure
----	-----------------------------

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_cell\(\)](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.10 memory_word_move()

```
void memory_word_move (
```

```
VM * vm )
```

Implements FORTH word 'MOVE' - Copy memory region.

Stack effect: (addr1 addr2 len –) Copies len bytes from addr1 to addr2, handling overlapping regions correctly.

Parameters

vm	Pointer to the VM structure
----	-----------------------------

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.11 memory_word_plus_store()

```
void memory_word_plus_store (
```

```
VM * vm )
```

Implements FORTH word '+' - Add to contents of cell.

Stack effect: (n addr –) Adds n to the contents of the cell at the given address.

Parameters

vm	Pointer to the VM structure
----	-----------------------------

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_cell\(\)](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.12 memory_word_store()

```
void memory_word_store (
```

```
VM * vm )
```

Implements FORTH word '!' - Store cell into VM memory.

Stack effect: (n addr –) Stores a cell value at the given memory address in VM memory.

Parameters

vm	Pointer to the VM structure
----	-----------------------------

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), and [vm_store_cell\(\)](#).

Referenced by [register_memory_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.206.1.13 register_memory_words()

```
void register_memory_words (
    VM * vm )
```

Register all memory manipulation words with the [VM](#).

Registers all memory operation words with the Forth virtual machine.

Registers all standard memory words (@, !, C@, C!, etc.) with the [VM](#)'s word dictionary.

Parameters

<code>vm</code>	Pointer to the VM structure
-----------------	---

References [memory_word_2fetch\(\)](#), [memory_word_2store\(\)](#), [memory_word_cells\(\)](#), [memory_word_cfetch\(\)](#), [memory_word_cstore\(\)](#), [memory_word_erase\(\)](#), [memory_word_fetch\(\)](#), [memory_word_fill\(\)](#), [memory_word_minus_store\(\)](#), [memory_word_move\(\)](#), [memory_word_plus_store\(\)](#), [memory_word_store\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207 src/word_source/mixed_arithmetic_words.c File Reference

```
#include "include/mixed_arithmetic_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
Include dependency graph for mixed_arithmetic_words.c:
```

Functions

- void [mixed_math_word_m_plus](#) ([VM](#) *[vm](#))
- void [mixed_math_word_m_minus](#) ([VM](#) *[vm](#))
- void [mixed_math_word_m_star](#) ([VM](#) *[vm](#))
- void [mixed_math_word_m_slash_mod](#) ([VM](#) *[vm](#))
- void [mixed_math_word_mod](#) ([VM](#) *[vm](#))
- void [mixed_math_word_slash_mod](#) ([VM](#) *[vm](#))
- void [mixed_math_word_star_slash](#) ([VM](#) *[vm](#))
- void [mixed_math_word_star_slash_mod](#) ([VM](#) *[vm](#))
- void [register_mixed_arithmetic_words](#) ([VM](#) *[vm](#))

7.207.1 Function Documentation**7.207.1.1 mixed_math_word_m_minus()**

```
void mixed_math_word_m_minus (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.2 mixed_math_word_m_plus()

```
void mixed_math_word_m_plus (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.3 mixed_math_word_m_slash_mod()

```
void mixed_math_word_m_slash_mod (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.4 mixed_math_word_m_star()

```
void mixed_math_word_m_star (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.5 mixed_math_word_mod()

```
void mixed_math_word_mod (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.6 mixed_math_word_slash_mod()

```
void mixed_math_word_slash_mod (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.7 mixed_math_word_star_slash()

```
void mixed_math_word_star_slash (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.8 mixed_math_word_star_slash_mod()

```
void mixed_math_word_star_slash_mod (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_mixed_arithmetic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.207.1.9 register_mixed_arithmetic_words()

```
void register_mixed_arithmetic_words (
    VM * vm )
```

References [mixed_math_word_m_minus\(\)](#), [mixed_math_word_m_plus\(\)](#), [mixed_math_word_m_slash_mod\(\)](#), [mixed_math_word_m_star\(\)](#), [mixed_math_word_mod\(\)](#), [mixed_math_word_slash_mod\(\)](#), [mixed_math_word_star_slash\(\)](#), [mixed_math_word_star_slash_mod\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208 src/word_source/physics_benchmark_words.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "vm.h"
#include "physics_hotwords_cache.h"
#include "platform_time.h"
#include "word_registry.h"
Include dependency graph for physics_benchmark_words.c:
```

Functions

- void [forth_BENCH_DICT_LOOKUP](#) (VM *vm)
BENCH-DICT-LOOKUP (iterations –)
- void [forth_PHYSICS_CACHE_STATS](#) (VM *vm)
PHYSICS-CACHE-STATS (–)
- void [forth_PHYSICS_TOGGLE_CACHE](#) (VM *vm)
PHYSICS-TOGGLE-CACHE (–)
- void [forth_PHYSICS_RESET_STATS](#) (VM *vm)
PHYSICS-RESET-STATS (–)
- void [forth_PHYSICS_BUILD_INFO](#) (VM *vm)
PHYSICS-BUILD-INFO (–)
- void [forth_PHYSICS_BAYESIAN_REPORT](#) (VM *vm)
PHYSICS-BAYESIAN-REPORT (addr_baseline –)
- void [register_physics_benchmark_words](#) (VM *vm)
Register all physics benchmark words with the VM.

Variables

- DictEntry ** [sf_fc_list](#) [256]
- size_t [sf_fc_count](#) [256]

7.208.1 Function Documentation

7.208.1.1 forth_BENCH_DICT_LOOKUP()

```
void forth_BENCH_DICT_LOOKUP (
    VM * vm )
```

BENCH-DICT-LOOKUP (iterations –)

Benchmark dictionary lookup performance with statistically valid sample sizes.

STATISTICAL VALIDITY:

- Minimum recommended: 10,000 iterations (95% confidence interval)
- Standard test: 100,000 iterations (tight confidence intervals)
- Stress test: 1,000,000 iterations (ultra-precise measurements)

FOR BAYESIAN INFERENCE:

- All latencies recorded in 64-bit Q48.16 fixed-point format
- Variance accumulators enable prior/posterior calculations
- Sample sizes affect confidence in ML decisions

USAGE: 100000 BENCH-DICT-LOOKUP (statistically valid baseline) 1000000 BENCH-DICT-LOOKUP (stress test, ~30sec on modern CPUs)

References [VM::data_stack](#), [VM::dsp](#), [ENABLE_HOTWORDS_CACHE](#), [VM::error](#), [VM::hotwords_cache](#), [hotwords_stats_print\(\)](#), [hotwords_stats_reset\(\)](#), [sf_monotonic_ns\(\)](#), [HotwordsCache_s::stats](#), [vm](#), and [vm_find_word\(\)](#).
Referenced by [register_physics_benchmark_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.1.2 forth_PHYSICS_BAYESIAN_REPORT()

```
void forth_PHYSICS_BAYESIAN_REPORT (
    VM * vm )
```

PHYSICS-BAYESIAN-REPORT (addr_baseline -)

Generate Bayesian inference report comparing current cache stats against baseline stats stored at given address

Usage: PHYSICS-RESET-STATS [run baseline benchmark without cache] [save stats somewhere] PHYSICS-RESET-STATS [run benchmark with cache] PHYSICS-BAYESIAN-REPORT

References [HotwordsStats::bucket_hits](#), [HotwordsStats::bucket_search_samples](#), [HotwordsStats::bucket_search_total_ns_q48](#), [HotwordsStats::bucket_search_variance_sum_q48](#), [HotwordsStats::cache_hits](#), [VM::error](#), [hotwords_bayesian_report\(\)](#), [VM::hotwords_cache](#), [HotwordsStats::max_bucket_search_ns](#), [HotwordsStats::min_bucket_search_ns](#), [HotwordsCache_s::stats](#), and [vm](#).

Referenced by [register_physics_benchmark_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.1.3 forth_PHYSICS_BUILD_INFO()

```
void forth_PHYSICS_BUILD_INFO (
    VM * vm )
```

PHYSICS-BUILD-INFO (-)

Show current build configuration for this variant

References [HotwordsCache_s::cache_count](#), [ENABLE_HOTWORDS_CACHE](#), [HotwordsCache_s::enabled](#), [VM::error](#), [VM::hotwords_cache](#), [HOTWORDS_CACHE_SIZE](#), [HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD](#), [HOTWORDS_EXECUTION_HEAT_THRESHOLD](#), and [vm](#).

Referenced by [register_physics_benchmark_words\(\)](#).

Here is the caller graph for this function:

7.208.1.4 forth_PHYSICS_CACHE_STATS()

```
void forth_PHYSICS_CACHE_STATS (
    VM * vm )
```

PHYSICS-CACHE-STATS (-)

Display detailed hot-words cache statistics

References [HotwordsCache_s::cache](#), [HotwordsCache_s::cache_count](#), [ENABLE_HOTWORDS_CACHE](#), [VM::error](#), [DictEntry::execution_heat](#), [VM::hotwords_cache](#), [hotwords_stats_print\(\)](#), [DictEntry::name](#), [DictEntry::physics](#), [HotwordsCache_s::stats](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_benchmark_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.1.5 forth_PHYSICS_RESET_STATS()

```
void forth_PHYSICS_RESET_STATS (
    VM * vm )
```

PHYSICS-RESET-STATS (-)

Reset cache statistics for clean before/after comparison

References [VM::error](#), [VM::hotwords_cache](#), [hotwords_stats_reset\(\)](#), [VM::pipeline_metrics](#), [PipelineGlobalMetrics::prefetch_attempts](#), [PipelineGlobalMetrics::prefetch_hits](#), [HotwordsCache_s::stats](#), [vm](#), and [PipelineGlobalMetrics::window_tuning_checks](#).

Referenced by [register_physics_benchmark_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.1.6 forth_PHYSICS_TOGGLE_CACHE()

```
void forth_PHYSICS_TOGGLE_CACHE (
    VM * vm )
```

PHYSICS-TOGGLE-CACHE (-)

Enable/disable hot-words cache at runtime (for A/B testing)

References [ENABLE_HOTWORDS_CACHE](#), [HotwordsCache_s::enabled](#), [VM::error](#), [VM::hotwords_cache](#), [hotwords_cache_set_enabled\(\)](#), and [vm](#).

Referenced by [register_physics_benchmark_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.1.7 register_physics_benchmark_words()

```
void register_physics_benchmark_words (
    VM * vm )
```

Register all physics benchmark words with the VM.

References [forth_BENCH_DICT_LOOKUP\(\)](#), [forth_PHYSICS_BAYESIAN_REPORT\(\)](#), [forth_PHYSICS_BUILD_INFO\(\)](#), [forth_PHYSICS_CACHE_STATS\(\)](#), [forth_PHYSICS_RESET_STATS\(\)](#), [forth_PHYSICS_TOGGLE_CACHE\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.208.2 Variable Documentation

7.208.2.1 sf_fc_count

```
size_t sf_fc_count[256] [extern]
```

Referenced by [sf_fc_clear_all\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), and [vm_find_word\(\)](#).

7.208.2.2 sf_fc_list

```
DictEntry** sf_fc_list[256] [extern]
```

Referenced by [sf_fc_clear_all\(\)](#), [sf_fc_reserve\(\)](#), [sf_rebuild_fc_index\(\)](#), [sf_try_fast_append\(\)](#), and [vm_find_word\(\)](#).

7.209 src/word_source/physics_diagnostic_words.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include "vm.h"
#include "physics_metadata.h"
#include "word_registry.h"
Include dependency graph for physics_diagnostic_words.c:
```

Functions

- void [forth_PHYSICS_WORD_METRICS](#) (VM *vm)
PHYSICS-WORD-METRICS (-)
- void [forth_PHYSICS_CALC_KNOBS](#) (VM *vm)
PHYSICS-CALC-KNOBS (-)
- void [forth_PHYSICS_BURN](#) (VM *vm)
PHYSICS-BURN (n -)
- void [forth_PHYSICS_SHOW_FEEDBACK](#) (VM *vm)
PHYSICS-SHOW-FEEDBACK (-)
- void [register_physics_diagnostic_words](#) (VM *vm)
Register all physics diagnostic words with the VM.

7.209.1 Function Documentation

7.209.1.1 forth_PHYSICS_BURN()

```
void forth_PHYSICS_BURN (
    VM * vm )
```

PHYSICS-BURN (n -)

Execute a word repeatedly (burn) and show thermal feedback.

Execute the top-of-stack word n times and show thermal feedback.

Usage: : HOTTEST 100 0 DO 1 2 + 3 * LOOP ; HOTTEST 10 PHYSICS-BURN (execute 10 times) PHYSICS-↵
SHOW-FEEDBACK

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::func](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.209.1.2 forth_PHYSICS_CALC_KNOBS()

```
void forth_PHYSICS_CALC_KNOBS (
    VM * vm )
```

PHYSICS-CALC-KNOBS (-)

Calculate and show knob adjustments based on current thermal metrics.

Calculate what knobs should be adjusted based on current word metrics. Shows:

- Thermal pressure (0.0 to 1.0)
- Recommended priority adjustment
- Recommended sampling rate
- Stack depth limit
- Core affinity preference

References [VM::error](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.209.1.3 forth_PHYSICS_SHOW_FEEDBACK()

```
void forth_PHYSICS_SHOW_FEEDBACK (
    VM * vm )
```

PHYSICS-SHOW-FEEDBACK (-)

Display the complete feedback loop: inputs → math → knobs → effect.

Display the complete feedback loop for the most recent word. Shows: Metrics → Calculations → Knobs → Effect

References [DictPhysics::avg_latency_ns](#), [VM::error](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictPhysics::mass_bytes](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.209.1.4 forth_PHYSICS_WORD_METRICS()

```
void forth_PHYSICS_WORD_METRICS (
    VM * vm )
```

PHYSICS-WORD-METRICS (-)

Display detailed physics metrics for the most recently executed word.

Display detailed physics metrics for a word. Looks up the most recently executed word and shows its state.

References [DictPhysics::avg_latency_ns](#), [VM::error](#), [DictEntry::execution_heat](#), [DictPhysics::last_active_ns](#), [VM::latest](#), [DictPhysics::mass_bytes](#), [DictEntry::name](#), [DictEntry::physics](#), [DictPhysics::temperature_q8](#), and [vm](#).

Referenced by [register_physics_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.209.1.5 register_physics_diagnostic_words()

```
void register_physics_diagnostic_words (
    VM * vm )
```

Register all physics diagnostic words with the [VM](#).

References [forth_PHYSICS_BURN\(\)](#), [forth_PHYSICS_CALC_KNOBS\(\)](#), [forth_PHYSICS_SHOW_FEEDBACK\(\)](#), [forth_PHYSICS_WORD_METRICS\(\)](#), [register_word\(\)](#), and [vm](#).

Here is the call graph for this function:

7.210 src/word_source/physics_freeze_words.c File Reference

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
#include "vm.h"
#include "physics_metadata.h"
#include "word_registry.h"
Include dependency graph for physics_freeze_words.c:
```

Functions

- void [forth_FREEZE_WORD](#) (VM *vm)
- void [forth_UNFREEZE_WORD](#) (VM *vm)
- void [forth_FROZEN_QUERY](#) (VM *vm)
- void [forth_HEAT_STORE](#) (VM *vm)
- void [forth_HEAT_FETCH](#) (VM *vm)
- void [forth_SHOW_HEAT](#) (VM *vm)
- void [forth_ALL_HEATS](#) (VM *vm)
- void [forth_DECAY_RATE_FETCH](#) (VM *vm)
- void [forth_FREEZE_CRITICAL](#) (VM *vm)
- void [register_physics_freeze_words](#) (VM *vm)

Register all Phase 2 freeze/decay control words with the VM.

7.210.1 Function Documentation

7.210.1.1 forth_ALL_HEATS()

```
void forth_ALL_HEATS (
    VM * vm )
```

References [VM::error](#), [execution_heat](#), [DictEntry::execution_heat](#), [DictEntry::flags](#), [VM::latest](#), [DictEntry::name](#), [DictEntry::name_len](#), [vm](#), [WORD_FROZEN](#), and [WORD_PINNED](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the caller graph for this function:

7.210.1.2 forth_DECAY_RATE_FETCH()

```
void forth_DECAY_RATE_FETCH (
    VM * vm )
```

References [VM::data_stack](#), [DECAY_RATE_PER_US_Q16](#), [VM::dsp](#), [VM::error](#), and [vm](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the caller graph for this function:

7.210.1.3 forth_FREEZE_CRITICAL()

```
void forth_FREEZE_CRITICAL (
    VM * vm )
```

References [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), and [WORD_FROZEN](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.4 forth_FREEZE_WORD()

```
void forth_FREEZE_WORD (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), [WORD_FROZEN](#), and [WORD_NAME_MAX](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.5 forth_FROZEN_QUERY()

```
void forth_FROZEN_QUERY (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), [WORD_FROZEN](#), and [WORD_NAME_MAX](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.6 forth_HEAT_FETCH()

```
void forth_HEAT_FETCH (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.7 forth_HEAT_STORE()

```
void forth_HEAT_STORE (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.8 forth_SHOW_HEAT()

```
void forth_SHOW_HEAT (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), [WORD_FROZEN](#), [WORD_NAME_MAX](#), and [WORD_PINNED](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.9 forth_UNFREEZE_WORD()

```
void forth_UNFREEZE_WORD (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [name](#), [vm](#), [vm_find_word\(\)](#), and [WORD_NAME_MAX](#).

Referenced by [register_physics_freeze_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.210.1.10 register_physics_freeze_words()

```
void register_physics_freeze_words (
    VM * vm )
```

Register all Phase 2 freeze/decay control words with the [VM](#).

References [forth_ALL_HEATS\(\)](#), [forth_DECAY_RATE_FETCH\(\)](#), [forth_FREEZE_CRITICAL\(\)](#), [forth_FREEZE_WORD\(\)](#), [forth_FROZEN_QUERY\(\)](#), [forth_HEAT_FETCH\(\)](#), [forth_HEAT_STORE\(\)](#), [forth_SHOW_HEAT\(\)](#), [forth_UNFREEZE_WORD\(\)](#), [register_word\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.211 src/word_source/physics_pipelining_diagnostic_words.c File Reference

```
#include "../include/vm.h"
#include "../include/physics_pipelining_metrics.h"
```

```
#include "../include/word_registry.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
Include dependency graph for physics_pipelining_diagnostic_words.c:
```

Functions

- static void [forth_PIPELINING_SHOW_STATS](#) (VM *vm)
PIPELINING-SHOW-STATS (<wordname> -)
- static void [forth_PIPELINING_SHOW_TOP_TRANSITIONS](#) (VM *vm)
PIPELINING-SHOW-TOP-TRANSITIONS (<wordname> N -)
- static void [forth_PIPELINING_RESET_ALL](#) (VM *vm)
PIPELINING-RESET-ALL (-)
- static void [forth_PIPELINING_ENABLE](#) (VM *vm)
PIPELINING-ENABLE (-)
- static void [forth_PIPELINING_STATS](#) (VM *vm)
PIPELINING-STATS (-)
- static void [forth_PIPELINING_ANALYZE_WORD](#) (VM *vm)
PIPELINING-ANALYZE-WORD (<wordname> -)
- void [register_physics_pipelining_diagnostic_words](#) (VM *vm)
Register all pipelining diagnostic words.

7.211.1 Function Documentation

7.211.1.1 forth_PIPELINING_ANALYZE_WORD()

```
static void forth_PIPELINING_ANALYZE_WORD (
    VM * vm ) [static]
```

PIPELINING-ANALYZE-WORD (<wordname> -)

Comprehensive analysis of a single word's transition pattern. Shows raw data and interpretation hints.

References [DICTIONARY_SIZE](#), [VM::dsp](#), [entry](#), [VM::error](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction_cost_q48](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_a](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition_heat](#), [vm](#), [vm_find_word\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.211.1.2 forth_PIPELINING_ENABLE()

```
static void forth_PIPELINING_ENABLE (
    VM * vm ) [static]
```

PIPELINING-ENABLE (-)

Enable pipelining metrics collection (sets ENABLE_PIPELINING flag). Note: This is compile-time on/off for now; this word is a placeholder.

References [vm](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.211.1.3 forth_PIPELINING_RESET_ALL()

```
static void forth_PIPELINING_RESET_ALL (
    VM * vm ) [static]
```

PIPELINING-RESET-ALL (-)

Clear all transition metrics from the dictionary. Useful for starting fresh measurements.

References [entry](#), [VM::latest](#), [transition_metrics_reset\(\)](#), and [vm](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.211.1.4 forth_PIPELINING_SHOW_STATS()

```
static void forth_PIPELINING_SHOW_STATS (
    VM * vm ) [static]
```

PIPELINING-SHOW-STATS (<wordname> -)

Display transition metrics for a given word. Typical usage: PIPELINING-SHOW-STATS EXIT (show where EXIT typically goes) PIPELINING-SHOW-STATS BENCH-DICT (show where BENCH-DICT transitions)

Requires: word must exist in dictionary Effect: Prints transition statistics to stdout

References [VM::dsp](#), [entry](#), [VM::error](#), [WordTransitionMetrics::max_transition_probability_q48](#), [WordTransitionMetrics::misprediction](#), [WordTransitionMetrics::most_likely_next_word_id](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_latency_saved_q48](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_transition](#), [vm](#), [vm_find_word\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.211.1.5 forth_PIPELINING_SHOW_TOP_TRANSITIONS()

```
static void forth_PIPELINING_SHOW_TOP_TRANSITIONS (
    VM * vm ) [static]
```

PIPELINING-SHOW-TOP-TRANSITIONS (<wordname> N -)

Display the top N words that typically follow a given word. Useful for understanding hot execution paths.

Stack: <wordname> N – Example: " EXIT" 5 PIPELINING-SHOW-TOP-TRANSITIONS (Show top 5 words that usually follow EXIT)

References [DICTIONARY_SIZE](#), [VM::dsp](#), [entry](#), [VM::error](#), [WordTransitionMetrics::total_transitions](#), [WordTransitionMetrics::transition](#), [vm](#), [vm_find_word\(\)](#), [vm_pop\(\)](#), [vm_ptr\(\)](#), and [word_id](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.211.1.6 forth_PIPELINING_STATS()

```
static void forth_PIPELINING_STATS (
    VM * vm ) [static]
```

PIPELINING-STATS (-)

Display aggregate pipelining statistics across entire dictionary.

References [entry](#), [VM::latest](#), [WordTransitionMetrics::prefetch_attempts](#), [WordTransitionMetrics::prefetch_hits](#), [WordTransitionMetrics::prefetch_misses](#), [WordTransitionMetrics::total_transitions](#), [vm](#), and [word_count](#).

Referenced by [register_physics_pipelining_diagnostic_words\(\)](#).

Here is the caller graph for this function:

7.211.1.7 register_physics_pipelining_diagnostic_words()

```
void register_physics_pipelining_diagnostic_words (
    VM * vm )
```

Register all pipelining diagnostic words.

Register all pipelining diagnostic words Called during [VM](#) initialization.

References [forth_PIPELINING_ANALYZE_WORD\(\)](#), [forth_PIPELINING_ENABLE\(\)](#), [forth_PIPELINING_RESET_ALL\(\)](#), [forth_PIPELINING_SHOW_STATS\(\)](#), [forth_PIPELINING_SHOW_TOP_TRANSITIONS\(\)](#), [forth_PIPELINING_STATS\(\)](#), [vm](#), and [vm_create_word\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.212 src/word_source/q48_16_words.c File Reference

```
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <assert.h>
#include "../include/q48_16.h"
Include dependency graph for q48_16_words.c:
```

Functions

- [q48_16_t q48_mul](#) ([q48_16_t](#) a, [q48_16_t](#) b)
Multiply two Q48.16 values, keeping result in Q48.16.
- [q48_16_t q48_div](#) ([q48_16_t](#) a, [q48_16_t](#) b)
Divide two Q48.16 values, keeping result in Q48.16.
- [q48_16_t q48_from_double](#) (double d)
Convert double to Q48.16 (for testing/initialization only)
- double [q48_to_double](#) ([q48_16_t](#) q)
Convert Q48.16 to double (for testing/diagnostics only)
- [q48_16_t q48_log_approx](#) (uint64_t x)
Approximate natural logarithm in Q48.16 (integer-only)
- [q48_16_t q48_exp_approx](#) ([q48_16_t](#) q)
Approximate exponential e^x in Q48.16 (integer-only, future use)
- [q48_16_t q48_sqrt_approx](#) ([q48_16_t](#) q)
Approximate square root in Q48.16 (integer-only)
- const char * [q48_to_string](#) ([q48_16_t](#) q)
Pretty-print Q48.16 value to string for debugging.
- int [q48_is_valid](#) ([q48_16_t](#) q)
Validate Q48.16 value is within safe range.

7.212.1 Function Documentation

7.212.1.1 q48_div()

```
q48_16_t q48_div (
    q48_16_t a,
    q48_16_t b )
```

Divide two Q48.16 values, keeping result in Q48.16.

Divide two Q48.16 values: (a << 16) / b Returns 0 if b == 0.

Math: $(a / 2^{16}) / (b / 2^{16}) * 2^{16} = (a << 16) / b$ Returns: $(a << 16) / b$

Parameters

<i>a</i>	Dividend in Q48.16
<i>b</i>	Divisor in Q48.16 (non-zero)

Returns

$(a << 16) / b$ in Q48.16 format

Referenced by [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), [has_variance_stabilized\(\)](#), [q48_exp_approx\(\)](#), [q48_log_approx\(\)](#), [q48_sqrt_approx\(\)](#), and [variance_to_trust\(\)](#).

Here is the caller graph for this function:

7.212.1.2 q48_exp_approx()

```
q48_16_t q48_exp_approx (
    q48_16_t q )
```

Approximate exponential e^x in Q48.16 (integer-only, future use)

Approximate exponential e^x in Q48.16 (integer-only) Input: q in Q48.16 format Output: e^q in Q48.16 format.

Purpose: Might be needed for inverse operations or model validation Status: Defined but not required for Phase 1 inference engine Method: Similar to [q48_log_approx](#) (piecewise + Newton)

Parameters

<i>q</i>	Q48.16 exponent
----------	-----------------

Returns

e^q in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), and [Q48_ONE](#).

Referenced by [q48_log_approx\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.212.1.3 q48_from_double()

```
q48_16_t q48_from_double (
    double d )
```

Convert double to Q48.16 (for testing/initialization only)

NOT used in inference engine (inference is integer-only). Provided for testing and diagnostic tools only.

Parameters

<i>d</i>	Double-precision float
----------	------------------------

Returns

$d * 2^{16}$ as Q48.16

References [zero](#).

Referenced by [find_variance_inflection\(\)](#).

Here is the caller graph for this function:

7.212.1.4 q48_is_valid()

```
int q48_is_valid (
    q48_16_t q )
```

Validate Q48.16 value is within safe range.

Checks: Does not overflow on mul/div operations Returns: 1 if valid, 0 if risky

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

1 if safe, 0 if risky

7.212.1.5 q48_log_approx()

```
q48_16_t q48_log_approx (
    uint64_t x )
```

Approximate natural logarithm in Q48.16 (integer-only)

Approximate natural logarithm in Q48.16 (integer-only) Input: x in Q48.16 format ($x > 0$) Output: $\ln(x)$ in Q48.16 format.

Purpose: Required for decay slope inference via exponential fitting Model: $\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} * t$

Method: Piecewise linear approximation or Newton-Raphson iteration

- Input x must be > 0 (assertion check)
- Output is $\ln(x)$ in Q48.16 format
- Precision: $\sim 5\%$ error acceptable (tuning parameter)
- No floating-point operations

Implementation Strategy:

1. Use bit position ($\text{floor}(\log_2(x))$) as coarse estimate
2. Refine with Newton iteration (3-4 cycles for Q48.16 precision)
3. Keep all arithmetic in integer space

Example: $\text{q48_log_approx}(65536) = \log(1.0)$ in Q48.16 ≈ 0 $\text{q48_log_approx}(131072) = \log(2.0)$ in Q48.16 $\approx 0x0000B172$

Parameters

<i>x</i>	Value to take logarithm of ($x > 0$)
----------	--

Returns

$\ln(x)$ in Q48.16 format

References [q48_add\(\)](#), [q48_div\(\)](#), [q48_exp_approx\(\)](#), [q48_from_u64\(\)](#), [q48_mul\(\)](#), [Q48_ONE](#), and [q48_sub\(\)](#).

Referenced by [infer_decay_slope_q48\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.212.1.6 q48_mul()

```
q48_16_t q48_mul (
    q48_16_t a,
    q48_16_t b )
```

Multiply two Q48.16 values, keeping result in Q48.16.

Multiply two Q48.16 values: $(a * b) >> 16$.

Math: $(a / 2^{16}) * (b / 2^{16}) * 2^{16} = (a * b) / 2^{16}$ Returns: $(a * b) >> 16$

Parameters

<i>a</i>	First Q48.16 operand
<i>b</i>	Second Q48.16 operand

Returns

$(a * b) >> 16$ in Q48.16 format

Multiply two Q48.16 values, keeping result in Q48.16.

Freestanding implementation: no libc, no floating-point.

Referenced by [compute_levene_statistic\(\)](#), [compute_variance_q48\(\)](#), [infer_decay_slope_q48\(\)](#), [q48_exp_approx\(\)](#), and [q48_log_approx\(\)](#).

Here is the caller graph for this function:

7.212.1.7 q48_sqrt_approx()

```
q48_16_t q48_sqrt_approx (
    q48_16_t q )
```

Approximate square root in Q48.16 (integer-only)

Approximate square root in Q48.16 (integer-only, Newton-Raphson) Input: *q* in Q48.16 format Output: $\text{sqrt}(q)$ in Q48.16 format.

Purpose: Compute R^2 (fit quality) for regression diagnostics Method: Newton-Raphson for integer-only sqrt

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

`sqrt(q)` in Q48.16 format

References [q48_div\(\)](#), and [Q48_ONE](#).

Here is the call graph for this function:

7.212.1.8 q48_to_double()

```
double q48_to_double (
    q48_16_t q )
```

Convert Q48.16 to double (for testing/diagnostics only)

NOT used in inference engine (inference is integer-only). Provided for logging and RStudio dashboard output.

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

$q / 2^{16}$ as double

Referenced by [inference_outputs_to_string\(\)](#).

Here is the caller graph for this function:

7.212.1.9 q48_to_string()

```
const char * q48_to_string (
    q48_16_t q )
```

Pretty-print Q48.16 value to string for debugging.

Format: "integer.fractional" (e.g., "1.50000") Uses internal static buffer - NOT thread-safe

Parameters

<i>q</i>	Q48.16 value
----------	--------------

Returns

Pointer to static string (valid until next call)

7.213 src/word_source/return_stack_words.c File Reference

Implementation of FORTH-79 standard return stack words.

```
#include "include/return_stack_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
```

Include dependency graph for `return_stack_words.c`:

Functions

- static void [return_stack_word_to_r](#) (VM *vm)
Move TOS from data stack to return stack (>R word)
- static void [return_stack_word_r_from](#) (VM *vm)
Move TOS from return stack back to data stack (R> word)
- static void [return_stack_word_r_fetch](#) (VM *vm)
Copy TOS of return stack to data stack (R@ word)
- void [register_return_stack_words](#) (VM *vm)

Register all FORTH-79 return stack manipulation words.

7.213.1 Detailed Description

Implementation of FORTH-79 standard return stack words.

StarForth - FORTH-79 Standard and ANSI C99 ONLY Last modified - 2025-09-10 See LICENSE file — No warranty.

FORTH-79 compliance:

- Provide only >R, R>, R@ for return stack transfer.
- No direct stack addressing (no RP!, RP@) — forbidden by FORTH-79.

7.213.2 Function Documentation

7.213.2.1 register_return_stack_words()

```
void register_return_stack_words (
    VM * vm )
```

Register all FORTH-79 return stack manipulation words.

Registers all return stack manipulation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Registers >R, R>, and R@ words. Direct stack addressing words (RP!, RP@) are deliberately omitted for FORTH-79 compliance.

References [register_word\(\)](#), [return_stack_word_r_fetch\(\)](#), [return_stack_word_r_from\(\)](#), [return_stack_word_to_r\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.213.2.2 return_stack_word_r_fetch()

```
static void return_stack_word_r_fetch (
    VM * vm ) [static]
```

Copy TOS of return stack to data stack (R@ word)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Stack effect: (− x)

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::return_stack](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_return_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.213.2.3 return_stack_word_r_from()

```
static void return_stack_word_r_from (
    VM * vm ) [static]
```

Move TOS from return stack back to data stack (R> word)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Stack effect: (− x)

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), [vm_push\(\)](#), and [vm_rpop\(\)](#).

Referenced by [register_return_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.213.2.4 return_stack_word_to_r()

```
static void return_stack_word_to_r (
    VM * vm ) [static]
```

Move TOS from data stack to return stack (>R word)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

Stack effect: (x -)

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::rsp](#), [STACK_SIZE](#), [vm](#), [vm_pop\(\)](#), and [vm_rpush\(\)](#).

Referenced by [register_return_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214 src/word_source/stack_words.c File Reference

```
#include "include/stack_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
Include dependency graph for stack_words.c:
```

Functions

- static void [stack_word_drop](#) ([VM](#) *[vm](#))
Implements DROP (n -) - Removes top stack item.
- static void [stack_word_dup](#) ([VM](#) *[vm](#))
Implements DUP (n - n n) - Duplicates top stack item.
- static void [stack_word_question_dup](#) ([VM](#) *[vm](#))
Implements ?DUP (n - n n | n - 0) - Conditionally duplicates top stack item.
- static void [stack_word_swap](#) ([VM](#) *[vm](#))
Implements SWAP (n1 n2 - n2 n1) - Exchanges top two stack items.
- static void [stack_word_over](#) ([VM](#) *[vm](#))
Implements OVER (n1 n2 - n1 n2 n1) - Copies second stack item to top.
- static void [stack_word_rot](#) ([VM](#) *[vm](#))
Implements ROT (n1 n2 n3 - n2 n3 n1) - Rotates top three stack items.
- static void [stack_word_minus_rot](#) ([VM](#) *[vm](#))
Implements -ROT (n1 n2 n3 - n3 n1 n2) - Reverse rotates top three items.
- static void [stack_word_depth](#) ([VM](#) *[vm](#))
Implements DEPTH (- n) - Returns number of stack items.
- static void [stack_word_pick](#) ([VM](#) *[vm](#))
Implements PICK (n - stack[n]) - Copies nth stack item to top.
- static void [stack_word_roll](#) ([VM](#) *[vm](#))
Implements ROLL (n -) - Moves nth stack item to top.
- void [register_stack_words](#) ([VM](#) *[vm](#))
Registers all stack operation words with the virtual machine.

7.214.1 Function Documentation

7.214.1.1 register_stack_words()

```
void register_stack_words (
    VM * vm )
```

Registers all stack operation words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Registers all FORTH-79 standard stack operation words with the word registry.

References [register_word\(\)](#), [stack_word_depth\(\)](#), [stack_word_drop\(\)](#), [stack_word_dup\(\)](#), [stack_word_minus_rot\(\)](#), [stack_word_over\(\)](#), [stack_word_pick\(\)](#), [stack_word_question_dup\(\)](#), [stack_word_roll\(\)](#), [stack_word_rot\(\)](#), [stack_word_swap\(\)](#), and [vm](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.2 stack_word_depth()

```
static void stack_word_depth (
    VM * vm ) [static]
```

Implements DEPTH ($- n$) - Returns number of stack items.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Pushes the current number of items on the data stack onto the stack.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.3 stack_word_drop()

```
static void stack_word_drop (
    VM * vm ) [static]
```

Implements DROP ($n -$) - Removes top stack item.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Removes the top item from the data stack. Generates an error on stack underflow.

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.4 stack_word_dup()

```
static void stack_word_dup (
    VM * vm ) [static]
```

Implements DUP ($n - n$) - Duplicates top stack item.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Duplicates the top item on the data stack. Generates an error on stack underflow/overflow.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [STACK_SIZE](#), [UNLIKELY](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.5 stack_word_minus_rot()

```
static void stack_word_minus_rot (
    VM * vm ) [static]
```

Implements -ROT (n1 n2 n3 – n3 n1 n2) - Reverse rotates top three items.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Performs the reverse rotation of the top three stack items.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.6 stack_word_over()

```
static void stack_word_over (
    VM * vm ) [static]
```

Implements OVER (n1 n2 – n1 n2 n1) - Copies second stack item to top.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Copies the second item from the top of the stack to the top.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.7 stack_word_pick()

```
static void stack_word_pick (
    VM * vm ) [static]
```

Implements PICK (n – stack[n]) - Copies nth stack item to top.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Copies the nth item from the stack to the top, where n is popped from the stack.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.8 stack_word_question_dup()

```
static void stack_word_question_dup (
    VM * vm ) [static]
```

Implements ?DUP (n – n n | n – 0) - Conditionally duplicates top stack item.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Duplicates top stack item if it's non-zero, otherwise leaves it unchanged.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [STACK_SIZE](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.9 stack_word_roll()

```
static void stack_word_roll (
    VM * vm ) [static]
```

Implements ROLL (*n*) - Moves *n*th stack item to top.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Moves the *n*th item from the stack to the top, shifting intermediate items down.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.10 stack_word_rot()

```
static void stack_word_rot (
    VM * vm ) [static]
```

Implements ROT (*n1 n2 n3*) - Rotates top three stack items.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Rotates the top three items on the stack, moving the third item to the top.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.214.1.11 stack_word_swap()

```
static void stack_word_swap (
    VM * vm ) [static]
```

Implements SWAP (*n1 n2*) - Exchanges top two stack items.

Parameters

<i>vm</i>	Pointer to the virtual machine state
-----------	--------------------------------------

Exchanges the positions of the top two items on the data stack.

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [UNLIKELY](#), and [vm](#).

Referenced by [register_stack_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215 src/word_source/starforth_words.c File Reference

```
#include "include/starforth_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
#include "../include/version.h"
#include "../include/physics_metadata.h"
#include "../include/platform_time.h"
#include "../include/profiler.h"
#include "../include/block_subsystem.h"
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for starforth_words.c:

Functions

- static int [is_valid_dict_entry](#) (VM *vm, DictEntry *candidate)
Get execution heat count for word at dictionary address.
- void [starforth_word_execution_heat_fetch](#) (VM *vm)
- void [starforth_word_execution_heat_store](#) (VM *vm)
Set execution heat count for word at dictionary address.
- void [starforth_word_word_execution_heat](#) (VM *vm)
Display execution heat statistics for all words in dictionary.
- void [starforth_word_reset_execution_heat](#) (VM *vm)
Reset all execution heat counters to zero.
- void [starforth_word_top_words](#) (VM *vm)
Display the N most frequently used words.
- void [starforth_word_paren_dash](#) (VM *vm)
Shebang-style comment word for init.4th metadata.
- void [starforth_word_init](#) (VM *vm)
Initialize system from init.4th configuration file.
- static uint64_t [prng_next](#) (void)
Internal PRNG step using Linear Congruential Generator.
- void [starforth_word_seed](#) (VM *vm)
Set the PRNG seed.
- void [starforth_word_random](#) (VM *vm)
Generate bounded random number.
- void [starforth_word_wait](#) (VM *vm)
Wait/sleep for specified milliseconds.
- void [starforth_word_version](#) (VM *vm)
Print StarForth version information (compliance)
- static void [starforth_word_zuse_authenticate](#) (VM *vm)
- void [register_starforth_words](#) (VM *vm)
Register StarForth vocabulary words with the VM.

Variables

- static uint64_t [g_prng_state](#) = 1

7.215.1 Function Documentation

7.215.1.1 is_valid_dict_entry()

```
static int is_valid_dict_entry (
    VM * vm,
    DictEntry * candidate ) [static]
```

Get execution heat count for word at dictionary address.

Stack effect: (addr – n) Returns the execution frequency (execution_heat counter) for a word. Note: Exposed as ENTROPY@ for FORTH compatibility, but measures execution heat.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

Validate that an address is a valid [DictEntry](#) pointer

Parameters

<i>vm</i>	The VM instance
<i>candidate</i>	The address to validate

Returns

1 if valid, 0 if not

References [VM::dict_lock](#), [VM::latest](#), [sf_mutex_lock\(\)](#), [sf_mutex_unlock\(\)](#), and [vm](#).

Referenced by [starforth_word_execution_heat_fetch\(\)](#), and [starforth_word_execution_heat_store\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.2 prng_next()

```
static uint64_t prng_next (
    void ) [static]
```

Internal PRNG step using Linear Congruential Generator.

Uses Numerical Recipes LCG constants for good statistical properties.

Returns

Next pseudo-random 64-bit value

References [g_prng_state](#).

Referenced by [starforth_word_random\(\)](#).

Here is the caller graph for this function:

7.215.1.3 register_starforth_words()

```
void register_starforth_words (
    VM * vm )
```

Register StarForth vocabulary words with the [VM](#).

Register all StarForth words with [VM](#).

Registers all StarForth-specific words and creates the STARFORTH vocabulary

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [register_word\(\)](#), [starforth_word_execution_heat_fetch\(\)](#), [starforth_word_execution_heat_store\(\)](#), [starforth_word_init\(\)](#), [starforth_word_paren_dash\(\)](#), [starforth_word_random\(\)](#), [starforth_word_reset_execution_heat\(\)](#), [starforth_word_seed\(\)](#), [starforth_word_top_words\(\)](#), [starforth_word_version\(\)](#), [starforth_word_wait\(\)](#), [starforth_word_word_execution](#)

[starforth_word_zuse_authenticate\(\)](#), [vm](#), and [vm_interpret\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.4 starforth_word_execution_heat_fetch()

```
void starforth_word_execution_heat_fetch (
    VM * vm )
```

References [VM::dsp](#), [entry](#), [VM::error](#), [is_valid_dict_entry\(\)](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.5 starforth_word_execution_heat_store()

```
void starforth_word_execution_heat_store (
    VM * vm )
```

Set execution heat count for word at dictionary address.

Stack effect: (n addr –) Note: Exposed as ENTROPY! for FORTH compatibility, but sets execution heat.

Parameters

vm	Pointer to the VM instance
--------------------	--

References [VM::dsp](#), [entry](#), [VM::error](#), [is_valid_dict_entry\(\)](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.6 starforth_word_init()

```
void starforth_word_init (
    VM * vm )
```

Initialize system from init.4th configuration file.

Initialize system from init-4.4th.

Reads ./capsules/init.4th (or ROMFS in L4Re), parses Block headers, copies blocks sequentially starting at block 1, then executes them. Stack effect: (–)

Parameters

vm	Pointer to the VM instance
--------------------	--

References [blk_get_buffer\(\)](#), [VM::current_executing_entry](#), [VM::error](#), [DictEntry::execution_heat](#), [DictEntry::func](#), [VM::halted](#), [LOG_DEBUG](#), [LOG_ERROR](#), [LOG_INFO](#), [log_message\(\)](#), [LOG_WARN](#), [physics_execution_heat_increment\(\)](#), [physics_metadata_touch\(\)](#), [profiler_word_count\(\)](#), [profiler_word_enter\(\)](#), [profiler_word_exit\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), [vm_find_word\(\)](#), [vm_interpret\(\)](#), and [vm_push\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.7 starforth_word_paren_dash()

```
void starforth_word_paren_dash (
    VM * vm )
```

Shebang-style comment word for init.4th metadata.

Shebang-style comment for init-4.4th metadata.

Starts with "(- " and consumes input until closing ")" Used for marking blocks that should be extracted to init.4th

Stack effect: (–)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [LOG_DEBUG](#), [log_message\(\)](#), [LOG_WARN](#), and [vm](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.8 starforth_word_random()

```
void starforth_word_random (
    VM * vm )
```

Generate bounded random number.

Stack effect: (lo hi – n) Returns a pseudo-random number in the inclusive range [lo, hi].

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [prng_next\(\)](#), [vm](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.9 starforth_word_reset_execution_heat()

```
void starforth_word_reset_execution_heat (
    VM * vm )
```

Reset all execution heat counters to zero.

Stack effect: (–)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [entry](#), [VM::latest](#), and [vm](#).

Referenced by [register_starforth_words\(\)](#).

Here is the caller graph for this function:

7.215.1.10 starforth_word_seed()

```
void starforth_word_seed (
    VM * vm )
```

Set the PRNG seed.

Set PRNG seed for reproducible random sequences.

Stack effect: (n –) Sets the internal PRNG state for reproducible random sequences.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [g_prng_state](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.11 starforth_word_top_words()

```
void starforth_word_top_words (
    VM * vm )
```

Display the N most frequently used words.

Display top N words by entropy.

Stack effect: (n –)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [entry](#), [VM::error](#), [execution_heat](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [vm_pop\(\)](#), and [word_count](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.12 starforth_word_version()

```
void starforth_word_version (
    VM * vm )
```

Print StarForth version information (compliance)

Prints: StarForth v<version> <architecture> <variant> <timestamp> Stack effect: (–)

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [vm](#).

Referenced by [register_starforth_words\(\)](#).

Here is the caller graph for this function:

7.215.1.13 starforth_word_wait()

```
void starforth_word_wait (
    VM * vm )
```

Wait/sleep for specified milliseconds.

Stack effect: (n –) Pauses execution for n milliseconds. Useful for:

- Allowing VM/system to cool between benchmark phases
- General-purpose timing in FORTH programs

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [VM::dsp](#), [VM::error](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [sf_monotonic_ns\(\)](#), [vm](#), and [vm_pop\(\)](#).

Referenced by [register_starforth_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.215.1.14 starforth_word_word_execution_heat()

```
void starforth_word_word_execution_heat (
    VM * vm )
```

Display execution heat statistics for all words in dictionary.

Stack effect: (–) Shows execution frequency ([execution_heat](#)) for each word.

Parameters

<i>vm</i>	Pointer to the VM instance
-----------	--

References [entry](#), [VM::latest](#), [vm](#), and [word_count](#).

Referenced by [register_starforth_words\(\)](#).

Here is the caller graph for this function:

7.215.1.15 starforth_word_zuse_authenticate()

```
static void starforth_word_zuse_authenticate (
    VM * vm ) [static]
```

References [vm](#), and [VM::zuse_session](#).

Referenced by [register_starforth_words\(\)](#).

Here is the caller graph for this function:

7.215.2 Variable Documentation

7.215.2.1 g_prng_state

```
uint64_t g_prng_state = 1 [static]
```

Referenced by [prng_next\(\)](#), and [starforth_word_seed\(\)](#).

7.216 src/word_source/string_words.c File Reference

```
#include "include/string_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
#include "../include/vm_api.h"
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
```

Include dependency graph for string_words.c:

Macros

- `#define WORD_SCRATCH_CAP 64`

Functions

- static `vaddr_t vaddr_from_ptr (VM *vm, const void *p)`
Convert a C pointer that points into vm->memory into a VM address (offset)
- static int `ensure_input (VM *vm)`
Ensure input buffers and variables are available.
- static int `convert_string_to_number (const char *str, size_t len, cell_t *result)`
- void `string_word_count (VM *vm)`
COUNT (addr1 – addr2 u)
- void `string_word_expect (VM *vm)`
- void `string_word_span (VM *vm)`
- void `string_word_query (VM *vm)`
- void `string_word_tib (VM *vm)`
- void `string_word_word (VM *vm)`
- void `string_word_to_in (VM *vm)`
- void `string_word_source (VM *vm)`
- void `string_word_bl (VM *vm)`

- void [string_word_tick](#) (VM *vm)
- void [string_word_bracket_tick](#) (VM *vm)
- void [string_word_literal](#) (VM *vm)
- void [string_word_bracket_literal](#) (VM *vm)
- void [string_word_convert](#) (VM *vm)
- void [string_word_number](#) (VM *vm)
- void [string_word_enclose](#) (VM *vm)
- static void [string_runtime_s_quote](#) (VM *vm)
- static void [string_word_s_quote](#) (VM *vm)
- static void [string_word_minus_trailing](#) (VM *vm)
- static void [string_word_cmove](#) (VM *vm)
- static void [string_word_cmove_greater](#) (VM *vm)
- static void [string_word_compare](#) (VM *vm)
- static void [string_word_search](#) (VM *vm)
- static void [string_word_scan](#) (VM *vm)
- static void [string_word_skip](#) (VM *vm)
- static void [string_word_blank](#) (VM *vm)
- void [register_string_words](#) (VM *vm)

COUNT (addr1 – addr2 u) Get string length and address Returns the character count and address of a counted string.

Variables

- static [vaddr_t](#) [word_scratch_addr](#) = 0

7.216.1 Macro Definition Documentation

7.216.1.1 WORD_SCRATCH_CAP

```
#define WORD_SCRATCH_CAP 64
```

7.216.2 Function Documentation

7.216.2.1 convert_string_to_number()

```
static int convert_string_to_number (
    const char * str,
    size_t len,
    cell_t * result ) [static]
```

Referenced by [string_word_number\(\)](#).

Here is the caller graph for this function:

7.216.2.2 ensure_input()

```
static int ensure_input (
    VM * vm ) [inline], [static]
```

Ensure input buffers and variables are available.

Parameters

vm	Pointer to VM instance
--------------------	--

Returns

0 on success, non-zero on error

References [VM::error](#), [vm](#), and [vm_input_ensure\(\)](#).

Referenced by [string_word_expect\(\)](#), [string_word_query\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_to_in\(\)](#), and [string_word_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.3 register_string_words()

```
void register_string_words (
    VM * vm )
```

COUNT (addr1 – addr2 u) Get string length and address Returns the character count and address of a counted string.

EXPECT (addr u –) Accept input line Accepts up to u characters of input and stores them at addr

SPAN (– addr) Variable: number of characters input Contains the count of characters actually received by EXPECT

QUERY (–) Accept input into TIB Accepts a line of input into the Terminal Input Buffer

TIB (– addr) Terminal input buffer address Returns the address of the Terminal Input Buffer

WORD (c – addr) Parse next word, delimited by c Parses the next word from input stream using c as delimiter

>IN (– addr) Input stream pointer Contains the offset into the input buffer

SOURCE (– addr u) Input source specification Returns current input buffer address and length

BL (– c) ASCII space character (32) Pushes ASCII space character value onto stack

FIND (addr – addr flag) Find word in dictionary Searches dictionary for word, returns status flag

' (– xt) Get execution token of next word Returns execution token of next word in input stream

[] (– xt) Compile execution token Compiles execution token as literal

LITERAL (n –) Compile literal number Compiles top stack value as literal

[LITERAL] (n –) Compile literal at compile time Compiles literal during compilation

CONVERT (d1 addr1 – d2 addr2) Convert string to number Converts string starting at addr1 to number

NUMBER (addr – n flag) Convert string to single number Converts string to single precision number

ENCLOSE (addr c – addr1 n1 n2 n3) Parse for delimiter Parses string for delimiter character

Register all string processing words with VM

Parameters

vm	Pointer to VM instance where words will be registered
----	---

References [register_word\(\)](#), [string_runtime_s_quote\(\)](#), [string_word_bl\(\)](#), [string_word_blank\(\)](#), [string_word_bracket_literal\(\)](#), [string_word_bracket_tick\(\)](#), [string_word_cmove\(\)](#), [string_word_cmove_greater\(\)](#), [string_word_compare\(\)](#), [string_word_convert\(\)](#), [string_word_count\(\)](#), [string_word_enclose\(\)](#), [string_word_expect\(\)](#), [string_word_literal\(\)](#), [string_word_minus_trailing\(\)](#), [string_word_number\(\)](#), [string_word_query\(\)](#), [string_word_s_quote\(\)](#), [string_word_scan\(\)](#), [string_word_search\(\)](#), [string_word_skip\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_to_in\(\)](#), [string_word_word\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.4 string_runtime_s_quote()

```
static void string_runtime_s_quote (
    VM * vm ) [static]
```

References [VM::error](#), [VM::return_stack](#), [VM::rsp](#), [vaddr_from_ptr\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.5 string_word_bl()

```
void string_word_bl (
    VM * vm )
```

References [vm](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.6 string_word_blank()

```
static void string_word_blank (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.7 string_word_bracket_literal()

```
void string_word_bracket_literal (
    VM * vm )
```

References [string_word_literal\(\)](#), and [vm](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.8 string_word_bracket_tick()

```
void string_word_bracket_tick (
    VM * vm )
```

References [string_word_tick\(\)](#), and [vm](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.9 string_word_cmove()

```
static void string_word_cmove (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.10 string_word_cmove_greater()

```
static void string_word_cmove_greater (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.11 string_word_compare()

```
static void string_word_compare (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.12 string_word_convert()

```
void string_word_convert (
    VM * vm )
```

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), [vm_ptr\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.13 string_word_count()

```
void string_word_count (
    VM * vm )
```

COUNT (addr1 – addr2 u)

Get string length and address. Converts a counted string at addr1 into an address (addr2) and length (u) pair.

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.14 string_word_enclose()

```
void string_word_enclose (
    VM * vm )
```

References [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), [vm_ptr\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.15 string_word_expect()

```
void string_word_expect (
    VM * vm )
```

References [VM::dsp](#), [ensure_input\(\)](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_input_span\(\)](#), [vm_pop\(\)](#), and [vm_ptr\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.16 string_word_literal()

```
void string_word_literal (
    VM * vm )
```

Referenced by [register_string_words\(\)](#), and [string_word_bracket_literal\(\)](#).

Here is the caller graph for this function:

7.216.2.17 string_word_minus_trailing()

```
static void string_word_minus_trailing (
    VM * vm ) [static]
```

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.18 string_word_number()

```
void string_word_number (
    VM * vm )
```

References [convert_string_to_number\(\)](#), [VM::dsp](#), [VM::error](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_pop\(\)](#), [vm_ptr\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.19 string_word_query()

```
void string_word_query (
    VM * vm )
```

References [ensure_input\(\)](#), [VM::error](#), [INPUT_BUFFER_SIZE](#), [vm](#), [vm_input_in\(\)](#), [vm_input_span\(\)](#), and [vm_input_tib\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.20 string_word_s_quote()

```
static void string_word_s_quote (
    VM * vm ) [static]
```

References [VM::error](#), [VM::here](#), [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::mode](#), [MODE_COMPILE](#), [vm](#), [vm_addr_ok\(\)](#), [vm_allot\(\)](#), [vm_compile_word\(\)](#), [vm_find_word\(\)](#), [vm_push\(\)](#), and [vm_store_u8\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.21 string_word_scan()

```
static void string_word_scan (
    VM * vm ) [static]
```

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.22 string_word_search()

```
static void string_word_search (
    VM * vm ) [static]
```

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [limit](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.23 string_word_skip()

```
static void string_word_skip (
    VM * vm ) [static]
```

References [CELL\(\)](#), [VM::dsp](#), [VM::error](#), [LOG_ERROR](#), [log_message\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_load_u8\(\)](#), [vm_pop\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.24 string_word_source()

```
void string_word_source (
    VM * vm )
```

References [CELL\(\)](#), [ensure_input\(\)](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [vm_input_span\(\)](#), [vm_input_tib\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.25 string_word_span()

```
void string_word_span (
    VM * vm )
```

References [CELL\(\)](#), [ensure_input\(\)](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [vm_input_span\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.26 string_word_tib()

```
void string_word_tib (
    VM * vm )
```

References [CELL\(\)](#), [ensure_input\(\)](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [vm_input_tib\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.27 string_word_tick()

```
void string_word_tick (
    VM * vm )
```

References [VM::data_stack](#), [VM::dsp](#), [entry](#), [VM::error](#), [VM::latest](#), [name](#), [name_len](#), [string_word_word\(\)](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_ptr\(\)](#), and [vm_push\(\)](#).

Referenced by [string_word_bracket_tick\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.28 string_word_to_in()

```
void string_word_to_in (
    VM * vm )
```

References [CELL\(\)](#), [ensure_input\(\)](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [vm_input_in\(\)](#), and [vm_push\(\)](#).

Referenced by [register_string_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.29 string_word_word()

```
void string_word_word (
    VM * vm )
```

References [CELL\(\)](#), [VM::dsp](#), [ensure_input\(\)](#), [VM::error](#), [vaddr_from_ptr\(\)](#), [vm](#), [vm_allot\(\)](#), [vm_input_in\(\)](#), [vm_input_span\(\)](#), [vm_input_tib\(\)](#), [vm_pop\(\)](#), [vm_ptr\(\)](#), [vm_push\(\)](#), [word_scratch_addr](#), and [WORD_SCRATCH_CAP](#).

Referenced by [register_string_words\(\)](#), and [string_word_tick\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.216.2.30 vaddr_from_ptr()

```
static vaddr_t vaddr_from_ptr (
    VM * vm,
    const void * p ) [inline], [static]
```

Convert a C pointer that points into `vm->memory` into a [VM](#) address (offset)

Parameters

<i>vm</i>	Pointer to VM instance
<i>p</i>	Pointer into VM memory space

Returns

Virtual address (offset) corresponding to the pointer

References [VM::memory](#), and [vm](#).

Referenced by [string_runtime_s_quote\(\)](#), [string_word_convert\(\)](#), [string_word_source\(\)](#), [string_word_span\(\)](#), [string_word_tib\(\)](#), [string_word_to_in\(\)](#), and [string_word_word\(\)](#).

Here is the caller graph for this function:

7.216.3 Variable Documentation

7.216.3.1 word_scratch_addr

```
vaddr_t word_scratch_addr = 0 [static]
```

Referenced by [string_word_word\(\)](#).

7.217 src/word_source/system_words.c File Reference

```
#include "include/system_words.h"
#include "../include/vm.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
```

Include dependency graph for system_words.c:

Functions

- static void [print_word_name](#) (const [DictEntry](#) *entry)
- static void [reset_vm_state](#) ([VM](#) *vm, int cold_start)
- static void [forth_paren](#) ([VM](#) *vm)
- void [system_word_cold](#) ([VM](#) *vm)
- void [system_word_warm](#) ([VM](#) *vm)
- void [system_word_bye](#) ([VM](#) *vm)
- void [system_word_save_system](#) ([VM](#) *vm)
- void [system_word_words](#) ([VM](#) *vm)
- void [system_word_vlist](#) ([VM](#) *vm)
- void [system_word_page](#) ([VM](#) *vm)
- void [system_word_nop](#) ([VM](#) *vm)
- void [system_word_79_standard](#) ([VM](#) *vm)
- static void [system_word_quit](#) ([VM](#) *vm)
- static void [system_word_abort](#) ([VM](#) *vm)
- static void [runtime_abortq](#) ([VM](#) *vm)
- static void [system_word_abort_quote](#) ([VM](#) *vm)
- static void [system_word_see](#) ([VM](#) *vm)
- int [system_is_running](#) (void)
- void [set_forth_79_compliance](#) (int enabled)
- void [register_system_words](#) ([VM](#) *vm)

Registers all system words with the virtual machine.

Variables

- static int [system_running](#) = 1
Flag indicating if the Forth system is currently running.
- static int [forth_79_standard](#) = 1
Flag controlling FORTH-79 standard compliance mode.

7.217.1 Function Documentation

7.217.1.1 forth_paren()

```
static void forth_paren (
    VM * vm ) [static]
```

References [log_message\(\)](#), [LOG_WARN](#), [vm](#), and [vm_parse_word\(\)](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.2 print_word_name()

```
static void print_word_name (
    const DictEntry * entry ) [static]
```

References [entry](#).

Referenced by [system_word_see\(\)](#), [system_word_vlist\(\)](#), and [system_word_words\(\)](#).

Here is the caller graph for this function:

7.217.1.3 register_system_words()

```
void register_system_words (
    VM * vm )
```

Registers all system words with the virtual machine.

Parameters

<i>vm</i>	Pointer to the virtual machine instance
-----------	---

This function registers all FORTH-79 system and environment words with the specified virtual machine instance, making them available for use in the FORTH environment.

References [forth_paren\(\)](#), [register_word\(\)](#), [runtime_abortq\(\)](#), [system_word_79_standard\(\)](#), [system_word_abort\(\)](#), [system_word_abort_quote\(\)](#), [system_word_bye\(\)](#), [system_word_cold\(\)](#), [system_word_nop\(\)](#), [system_word_page\(\)](#), [system_word_quit\(\)](#), [system_word_save_system\(\)](#), [system_word_see\(\)](#), [system_word_vlist\(\)](#), [system_word_warm\(\)](#), [system_word_words\(\)](#), [vm](#), and [vm_make_immediate\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.4 reset_vm_state()

```
static void reset_vm_state (
    VM * vm,
    int cold_start ) [static]
```

References [VM::dsp](#), [VM::error](#), [VM::here](#), [VM::mode](#), [MODE_INTERPRET](#), [VM::rsp](#), [system_running](#), and [vm](#).

Referenced by [runtime_abortq\(\)](#), [system_word_abort\(\)](#), [system_word_abort_quote\(\)](#), [system_word_cold\(\)](#), and [system_word_warm\(\)](#).

Here is the caller graph for this function:

7.217.1.5 runtime_abortq()

```
static void runtime_abortq (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [VM::memory](#), [reset_vm_state\(\)](#), [vm](#), [vm_addr_ok\(\)](#), and [vm_pop\(\)](#).

Referenced by [register_system_words\(\)](#), and [system_word_abort_quote\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.6 set_forth_79_compliance()

```
void set_forth_79_compliance (
    int enabled )
```

References [enabled](#), and [forth_79_standard](#).

7.217.1.7 system_is_running()

```
int system_is_running (
    void )
```

References [system_running](#).

7.217.1.8 system_word_79_standard()

```
void system_word_79_standard (
    VM * vm )
```

References [forth_79_standard](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.9 system_word_abort()

```
static void system_word_abort (
    VM * vm ) [static]
```

References [VM::abort_requested](#), [VM::error](#), [reset_vm_state\(\)](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.10 system_word_abort_quote()

```
static void system_word_abort_quote (
    VM * vm ) [static]
```

References [VM::dsp](#), [VM::error](#), [VM::input_buffer](#), [VM::input_length](#), [VM::input_pos](#), [VM::memory](#), [VM::mode](#), [MODE_INTERPRET](#), [reset_vm_state\(\)](#), [runtime_abortq\(\)](#), [vm](#), [vm_allot\(\)](#), [vm_compile_call\(\)](#), [vm_compile_literal\(\)](#), and [vm_pop\(\)](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.11 system_word_bye()

```
void system_word_bye (
    VM * vm )
```

References [VM::halted](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the caller graph for this function:

7.217.1.12 system_word_cold()

```
void system_word_cold (
    VM * vm )
```

References [reset_vm_state\(\)](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.13 system_word_nop()

```
void system_word_nop (
    VM * vm )
```

References [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the caller graph for this function:

7.217.1.14 system_word_page()

```
void system_word_page (
    VM * vm )
```

References [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the caller graph for this function:

7.217.1.15 system_word_quit()

```
static void system_word_quit (
    VM * vm ) [static]
```

References [VM::error](#), [VM::mode](#), [MODE_COMPILE](#), [MODE_INTERPRET](#), [VM::rsp](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the caller graph for this function:

7.217.1.16 system_word_save_system()

```
void system_word_save_system (
    VM * vm )
```

References [VM::error](#), [VM::here](#), [VM::memory](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the caller graph for this function:

7.217.1.17 system_word_see()

```
static void system_word_see (
    VM * vm ) [static]
```

References [entry](#), [VM::error](#), [execute_colon_word\(\)](#), [VM::memory](#), [DictEntry::name](#), [DictEntry::name_len](#), [print_word_name\(\)](#), [vm](#), [vm_addr_ok\(\)](#), [vm_dictionary_get_data_field\(\)](#), [vm_find_word\(\)](#), [vm_parse_word\(\)](#), [vm_ptr\(\)](#), [word_count](#), and [WORD_NAME_MAX](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.18 system_word_vlist()

```
void system_word_vlist (
    VM * vm )
```

References [DictEntry::flags](#), [VM::latest](#), [DictEntry::name_len](#), [print_word_name\(\)](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.19 system_word_warm()

```
void system_word_warm (
    VM * vm )
```

References [reset_vm_state\(\)](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.1.20 system_word_words()

```
void system_word_words (
    VM * vm )
```

References [VM::latest](#), [DictEntry::link](#), [print_word_name\(\)](#), and [vm](#).

Referenced by [register_system_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.217.2 Variable Documentation

7.217.2.1 forth_79_standard

```
int forth_79_standard = 1 [static]
```

Flag controlling FORTH-79 standard compliance mode.

Referenced by [set_forth_79_compliance\(\)](#), and [system_word_79_standard\(\)](#).

7.217.2.2 system_running

```
int system_running = 1 [static]
```

Flag indicating if the Forth system is currently running.

Referenced by [reset_vm_state\(\)](#), and [system_is_running\(\)](#).

7.218 src/word_source/vocabulary_words.c File Reference

```
#include "include/vocabulary_words.h"
#include "../include/word_registry.h"
#include "../include/log.h"
#include "../include/vm.h"
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for vocabulary_words.c:

Macros

- #define [SF_FC_BUCKETS](#) 256

Functions

- static void [vocabulary_debug_print_hex](#) (uint64_t value) [__attribute__\(\(unused\)\)](#)
- static size_t [vocab_safe_len](#) (const char *text, size_t max_len)
- static void [fc_free](#) (DictEntry ***lists, size_t *counts)
- static void [fc_rebuild](#) (DictEntry *head, DictEntry ***lists, size_t *counts, DictEntry **cached_head)
- static void [vocab_sync_vm_vars](#) (VM *vm)
- static void [init_vocabulary_system](#) (VM *vm)
Initialize the FORTH vocabulary system.
- static DictEntry * [vocab_find_word](#) (VM *vm, const char *name, size_t len)
- static void [vocabulary_select_runtime](#) (VM *vm)
- void [vocabulary_word_vocabulary](#) (VM *vm)
VOCABULARY (-) Create a new vocabulary.
- void [vocabulary_create_vocabulary_direct](#) (VM *vm, const char *name)
Direct vocabulary creation (kernel bootstrap)
- void [vocabulary_word_definitions](#) (VM *vm)
DEFINITIONS (-) Set CURRENT to CONTEXT.
- void [vocabulary_word_context](#) (VM *vm)
CONTEXT (- addr) Get CONTEXT variable address.
- void [vocabulary_word_current](#) (VM *vm)
CURRENT (- addr) Get CURRENT variable address.
- void [vocabulary_word_forth](#) (VM *vm)
FORTH (-) Select FORTH vocabulary.
- void [vocabulary_word_paren_find](#) (VM *vm)
(FIND) (addr - addr flag) Find word in dictionary
- void [vocabulary_word_order](#) (VM *vm)
ORDER (-) Display search order.
- DictEntry * [vm_vocabulary_find_word](#) (VM *vm, const char *name, size_t len)
Find word in vocabulary-aware dictionary.
- void [register_vocabulary_words](#) (VM *vm)
Register all FORTH-79 vocabulary words.

Variables

- static [DictEntry](#) ** [ctx_fc](#) [256]
- static [size_t](#) [ctx_n](#) [256]
- static [DictEntry](#) * [ctx_cached_head](#) = NULL
- static [DictEntry](#) ** [forth_fc](#) [256]
- static [size_t](#) [forth_n](#) [256]
- static [DictEntry](#) * [forth_cached_head](#) = NULL
- static [DictEntry](#) * [forth_vocab](#) = NULL
- static [DictEntry](#) * [context_vocab](#) = NULL
- static [DictEntry](#) * [current_vocab](#) = NULL
- static [vaddr_t](#) [context_var_addr](#) = 0
- static [vaddr_t](#) [current_var_addr](#) = 0

7.218.1 Macro Definition Documentation

7.218.1.1 SF_FC_BUCKETS

```
#define SF_FC_BUCKETS 256
```

7.218.2 Function Documentation

7.218.2.1 fc_free()

```
static void fc_free (
    DictEntry *** lists,
    size_t * counts ) [static]
```

References [SF_FC_BUCKETS](#).

Referenced by [fc_rebuild\(\)](#).

Here is the caller graph for this function:

7.218.2.2 fc_rebuild()

```
static void fc_rebuild (
    DictEntry * head,
    DictEntry *** lists,
    size_t * counts,
    DictEntry ** cached_head ) [static]
```

References [fc_free\(\)](#), [DictEntry::link](#), [DictEntry::name](#), and [SF_FC_BUCKETS](#).

Referenced by [vocab_find_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.3 init_vocabulary_system()

```
static void init_vocabulary_system (
    VM * vm ) [static]
```

Initialize the FORTH vocabulary system.

Sets up FORTH as root vocabulary and allocates [VM](#) cells for CONTEXT/CURRENT

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [context_var_addr](#), [context_vocab](#), [current_var_addr](#), [current_vocab](#), [VM::error](#), [forth_vocab](#), [initialized](#), [VM::latest](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [vm](#), [vm_allot\(\)](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [register_vocabulary_words\(\)](#), [vocab_find_word\(\)](#), [vocabulary_select_runtime\(\)](#), [vocabulary_word_context\(\)](#), [vocabulary_word_current\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), and [vocabulary_word_order\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.4 register_vocabulary_words()

```
void register_vocabulary_words (
    VM * vm )
```

Register all FORTH-79 vocabulary words.

Registers all vocabulary-related words in the Forth system.

Registers VOCABULARY, DEFINITIONS, CONTEXT, CURRENT, FORTH, ORDER, and (FIND)

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [init_vocabulary_system\(\)](#), [register_word\(\)](#), [vm](#), [vocabulary_word_context\(\)](#), [vocabulary_word_current\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), [vocabulary_word_order\(\)](#), [vocabulary_word_paren_find\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Referenced by [register_forth79_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.5 vm_vocabulary_find_word()

```
DictEntry * vm_vocabulary_find_word (
    VM * vm,
    const char * name,
    size_t len )
```

Find word in vocabulary-aware dictionary.

Searches CONTEXT then FORTH vocabularies

Parameters

<i>vm</i>	Pointer to VM instance
<i>name</i>	Word name to find
<i>len</i>	Length of word name

Returns

Pointer to dictionary entry if found, NULL if not found

References [name](#), [vm](#), and [vocab_find_word\(\)](#).

Referenced by [vm_interpret_word\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.6 vocab_find_word()

```
static DictEntry * vocab_find_word (
    VM * vm,
    const char * name,
    size_t len ) [static]
```

References [context_vocab](#), [ctx_cached_head](#), [ctx_fc](#), [ctx_n](#), [fc_rebuild\(\)](#), [DictEntry::flags](#), [forth_cached_head](#), [forth_fc](#), [forth_n](#), [forth_vocab](#), [init_vocabulary_system\(\)](#), [name](#), [DictEntry::name](#), [DictEntry::name_len](#), [vm](#), [WORD_HIDDEN](#), and [WORD_SMUDGED](#).

Referenced by [vm_vocabulary_find_word\(\)](#), and [vocabulary_word_paren_find\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.7 vocab_safe_len()

```
static size_t vocab_safe_len (
    const char * text,
    size_t max_len ) [static]
```

Referenced by [vocabulary_create_vocabulary_direct\(\)](#).

Here is the caller graph for this function:

7.218.2.8 vocab_sync_vm_vars()

```
static void vocab_sync_vm_vars (
    VM * vm ) [inline], [static]
```

References [base](#), [console_println\(\)](#), [console_puts\(\)](#), [context_var_addr](#), [context_vocab](#), [current_var_addr](#), [current_vocab](#), [VM::memory](#), [vm](#), [VM_MEMORY_SIZE](#), [vm_store_cell\(\)](#), and [vocabulary_debug_print_hex\(\)](#).

Referenced by [init_vocabulary_system\(\)](#), [vocabulary_create_vocabulary_direct\(\)](#), [vocabulary_select_runtime\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.9 vocabulary_create_vocabulary_direct()

```
void vocabulary_create_vocabulary_direct (
    VM * vm,
    const char * name )
```

Direct vocabulary creation (kernel bootstrap)

Parameters

<i>vm</i>	Pointer to the virtual machine instance
<i>name</i>	Vocabulary name

References [console_println\(\)](#), [console_puts\(\)](#), [entry](#), [VM::error](#), [VM::here](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [name](#), [vm](#), [vm_allot\(\)](#), [vm_create_word\(\)](#), [vm_store_cell\(\)](#), [vocab_safe_len\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_debug_print_hex\(\)](#), and [vocabulary_select_runtime\(\)](#).

Referenced by [vm_bootstrap_root_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.10 vocabulary_debug_print_hex()

```
static void vocabulary_debug_print_hex (
    uint64_t value ) [static]
```

Referenced by [vocab_sync_vm_vars\(\)](#), and [vocabulary_create_vocabulary_direct\(\)](#).

Here is the caller graph for this function:

7.218.2.11 vocabulary_select_runtime()

```
static void vocabulary_select_runtime (
    VM * vm ) [static]
```

References [context_vocab](#), [init_vocabulary_system\(\)](#), [VM::latest](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [vocabulary_create_vocabulary_direct\(\)](#), and [vocabulary_word_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.12 vocabulary_word_context()

```
void vocabulary_word_context (
    VM * vm )
```

CONTEXT (– addr) Get CONTEXT variable address.

Returns [VM](#) address of cell containing CONTEXT vocabulary pointer

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [CELL\(\)](#), [context_var_addr](#), [init_vocabulary_system\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.13 vocabulary_word_current()

```
void vocabulary_word_current (
    VM * vm )
```

CURRENT (– addr) Get CURRENT variable address.

Returns VM address of cell containing CURRENT vocabulary pointer

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [CELL\(\)](#), [current_var_addr](#), [init_vocabulary_system\(\)](#), [vm](#), and [vm_push\(\)](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.14 vocabulary_word_definitions()

```
void vocabulary_word_definitions (
    VM * vm )
```

DEFINITIONS (–) Set CURRENT to CONTEXT.

DEFINITIONS word implementation (kernel bootstrap)

Makes new words be defined in the CONTEXT vocabulary

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [context_vocab](#), [current_vocab](#), [init_vocabulary_system\(\)](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [register_vocabulary_words\(\)](#), and [vm_bootstrap_root_vocabulary\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.15 vocabulary_word_forth()

```
void vocabulary_word_forth (
    VM * vm )
```

FORTH (–) Select FORTH vocabulary.

FORTH word implementation - selects FORTH vocabulary.

Makes FORTH the CONTEXT vocabulary

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [context_vocab](#), [forth_vocab](#), [init_vocabulary_system\(\)](#), [LOG_DEBUG](#), [log_message\(\)](#), [vm](#), and [vocab_sync_vm_vars\(\)](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.16 vocabulary_word_order()

```
void vocabulary_word_order (
    VM * vm )
```

ORDER (–) Display search order.

Shows CONTEXT and FORTH vocabularies plus CURRENT

Parameters

<i>vm</i>	Pointer to VM instance
-----------	------------------------

References [context_vocab](#), [current_vocab](#), [forth_vocab](#), [init_vocabulary_system\(\)](#), [DictEntry::name](#), [DictEntry::name_len](#), and [vm](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.17 vocabulary_word_paren_find()

```
void vocabulary_word_paren_find (
    VM * vm )
```

(FIND) (addr – addr flag) Find word in dictionary

Searches for counted string at addr in CONTEXT then FORTH vocabularies

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [VM::data_stack](#), [VM::dsp](#), [VM::error](#), [DictEntry::flags](#), [name](#), [vm](#), [VM_ADDR\(\)](#), [vm_addr_ok\(\)](#), [vm_ptr\(\)](#), [vm_push\(\)](#), [vocab_find_word\(\)](#), and [WORD_IMMEDIATE](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.2.18 vocabulary_word_vocabulary()

```
void vocabulary_word_vocabulary (
    VM * vm )
```

VOCABULARY (–) Create a new vocabulary.

Creates a vocabulary that when executed makes itself the CONTEXT

Parameters

<i>vm</i>	Pointer to VM instance
-----------	--

References [entry](#), [VM::error](#), [VM::latest](#), [LOG_DEBUG](#), [LOG_ERROR](#), [log_message\(\)](#), [VM::memory](#), [name](#), [vm](#), [vm_allot\(\)](#), [vm_create_word\(\)](#), [vm_parse_word\(\)](#), [vm_store_cell\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_select_runtime\(\)](#).

Referenced by [register_vocabulary_words\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

7.218.3 Variable Documentation

7.218.3.1 context_var_addr

```
vaddr_t context_var_addr = 0 [static]
```

Referenced by [init_vocabulary_system\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_word_context\(\)](#).

7.218.3.2 context_vocab

```
DictEntry* context_vocab = NULL [static]
```

Referenced by [init_vocabulary_system\(\)](#), [vocab_find_word\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_select_runtime\(\)](#), [vocabulary_word_definitions\(\)](#), [vocabulary_word_forth\(\)](#), and [vocabulary_word_order\(\)](#).

7.218.3.3 ctx_cached_head

```
DictEntry* ctx_cached_head = NULL [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.4 ctx_fc

```
DictEntry** ctx_fc[256] [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.5 ctx_n

```
size_t ctx_n[256] [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.6 current_var_addr

```
vaddr_t current_var_addr = 0 [static]
```

Referenced by [init_vocabulary_system\(\)](#), [vocab_sync_vm_vars\(\)](#), and [vocabulary_word_current\(\)](#).

7.218.3.7 current_vocab

```
DictEntry* current_vocab = NULL [static]
```

Referenced by [init_vocabulary_system\(\)](#), [vocab_sync_vm_vars\(\)](#), [vocabulary_word_definitions\(\)](#), and [vocabulary_word_order\(\)](#).

7.218.3.8 forth_cached_head

```
DictEntry* forth_cached_head = NULL [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.9 forth_fc

```
DictEntry** forth_fc[256] [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.10 forth_n

```
size_t forth_n[256] [static]
```

Referenced by [vocab_find_word\(\)](#).

7.218.3.11 forth_vocab

```
DictEntry* forth_vocab = NULL [static]
```

Referenced by [init_vocabulary_system\(\)](#), [vocab_find_word\(\)](#), [vocabulary_word_forth\(\)](#), and [vocabulary_word_order\(\)](#).

Index

- [_EFI_BOOT_SERVICES, 45](#)
 - [AllocatePages, 46](#)
 - [AllocatePool, 46](#)
 - [CheckEvent, 46](#)
 - [CloseEvent, 46](#)
 - [CreateEvent, 46](#)
 - [Exit, 46](#)
 - [ExitBootServices, 46](#)
 - [FreePages, 46](#)
 - [FreePool, 46](#)
 - [GetMemoryMap, 46](#)
 - [GetNextMonotonicCount, 46](#)
 - [HandleProtocol, 46](#)
 - [Hdr, 46](#)
 - [InstallConfigurationTable, 46](#)
 - [InstallProtocolInterface, 46](#)
 - [LoadImage, 47](#)
 - [LocateDevicePath, 47](#)
 - [LocateHandle, 47](#)
 - [RaiseTPL, 47](#)
 - [RegisterProtocolNotify, 47](#)
 - [ReinstallProtocolInterface, 47](#)
 - [Reserved, 47](#)
 - [RestoreTPL, 47](#)
 - [SetTimer, 47](#)
 - [SetWatchdogTimer, 47](#)
 - [SignalEvent, 47](#)
 - [Stall, 47](#)
 - [StartImage, 47](#)
 - [UninstallProtocolInterface, 47](#)
 - [UnloadImage, 47](#)
 - [WaitForEvent, 48](#)
- [_EFI_FILE_PROTOCOL, 48](#)
 - [Close, 48](#)
 - [Delete, 48](#)
 - [Flush, 48](#)
 - [GetInfo, 48](#)
 - [GetPosition, 48](#)
 - [Open, 48](#)
 - [Read, 48](#)
 - [Revision, 49](#)
 - [SetInfo, 49](#)
 - [SetPosition, 49](#)
 - [Write, 49](#)
- [_EFI_RUNTIME_SERVICES, 49](#)
 - [ConvertPointer, 49](#)
 - [GetNextHighMonotonicCount, 49](#)
 - [GetNextVariableName, 49](#)
 - [GetTime, 49](#)
 - [GetVariable, 50](#)
 - [GetWakeupTime, 50](#)
 - [Hdr, 50](#)
 - [ResetSystem, 50](#)
 - [SetTime, 50](#)
 - [SetVariable, 50](#)
 - [SetVirtualAddressMap, 50](#)
 - [SetWakeupTime, 50](#)
- [_EFI_SIMPLE_FILE_SYSTEM_PROTOCOL, 50](#)
 - [OpenVolume, 50](#)
 - [Revision, 50](#)
- [_EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51](#)
 - [ClearScreen, 51](#)
 - [EnableCursor, 51](#)
 - [Mode, 51](#)
 - [OutputString, 51](#)
 - [QueryMode, 51](#)
 - [Reset, 51](#)
 - [SetAttribute, 51](#)
 - [SetCursorPosition, 51](#)
 - [SetMode, 51](#)
 - [TestString, 51](#)
- [_EFI_SYSTEM_TABLE, 52](#)
 - [BootServices, 52](#)
 - [ConfigurationTable, 52](#)
 - [ConIn, 52](#)
 - [ConOut, 52](#)
 - [ConsoleInHandle, 52](#)
 - [ConsoleOutHandle, 52](#)
 - [FirmwareRevision, 52](#)
 - [FirmwareVendor, 52](#)
 - [Hdr, 53](#)
 - [NumberOfTableEntries, 53](#)
 - [RuntimeServices, 53](#)
 - [StandardErrorHandle, 53](#)
 - [StdErr, 53](#)
- [_POSIX_C_SOURCE](#)
 - [profiler.c, 448](#)
 - [time.c, 444](#)
- [__attribute__](#)
 - [alloc_kernel.c, 441, 443](#)
 - [arch.c, 473](#)
 - [block_subsystem.c, 377](#)
 - [capsule.h, 251](#)
 - [capsule_generated.h, 258](#)
 - [interrupts.c, 476](#)
 - [main.c, 413](#)
 - [reloc_stub.c, 492](#)
 - [repl.c, 535](#)

- vm.h, [317](#), [335](#)
- __kernel_end
 - pmm.c, [529](#)
- __kernel_start
 - pmm.c, [529](#)
- _pad
 - blk_volume_meta_t, [57](#)
- abort_requested
 - VM, [143](#)
- access
 - arch.c, [474](#)
 - gdt_entry, [92](#)
- accessed
 - BlockBuffer, [63](#)
- account_allocation
 - kmalloc.c, [523](#)
- account_free
 - kmalloc.c, [523](#)
- acl_allow
 - DictEntry, [71](#)
 - vm.h, [335](#)
- ACL_ASSERT
 - acl_words_test.c, [576](#)
- acl_block
 - blk_meta_t, [55](#)
- acl_default
 - DictEntry, [71](#)
 - vm.h, [335](#)
- acl_hint
 - DictPhysics, [73](#)
 - physics_seed_definition, [120](#)
- acl_inherit_entry
 - acl_words.h, [679](#)
- acl_mode
 - DictEntry, [71](#)
 - vm.h, [335](#)
- ACL_MODE_STRICT
 - vm.h, [314](#)
- ACL_MODE_TTL
 - vm.h, [314](#)
- acl_pinned
 - DictEntry, [71](#)
 - vm.h, [336](#)
- acl_recheck
 - vm.c, [611](#)
- acl_ttl
 - DictEntry, [71](#)
 - vm.h, [336](#)
- ACL_TTL_OPEN
 - vm.h, [314](#)
- ACL_USER_DEFAULT
 - vm.h, [314](#)
- acl_words.h
 - acl_inherit_entry, [679](#)
 - register_acl_words, [679](#)
- acl_words_test.c
 - ACL_ASSERT, [576](#)
 - run_acl_words_tests, [577](#)
- test_acl_adaptive_ttl, [577](#)
- test_acl_deny_aborts, [577](#)
- test_acl_inherit, [577](#)
- test_acl_pin_blocks_shadow, [577](#)
- test_acl_pin_immutable, [577](#)
- test_acl_strict_mode, [578](#)
- test_acl_ttl_hot_path, [578](#)
- test_emergency_console_bypass, [578](#)
- tests_failed, [578](#)
- tests_passed, [578](#)
- acpi_table
 - BootInfo, [64](#)
- actual_window_size
 - DoeMetrics, [75](#)
 - HeartbeatSnapshot, [94](#)
 - HeartbeatTickSnapshot, [98](#)
 - WordTransitionMetrics, [160](#)
- adaptive_check_accumulator
 - RollingWindowOfTruth, [128](#)
- ADAPTIVE_CHECK_FREQUENCY
 - rolling_window_knobs.h, [232](#)
 - starforth_config.h, [242](#)
- adaptive_decay_slope
 - InferenceOutputs, [106](#)
- ADAPTIVE_GROWTH_THRESHOLD
 - rolling_window_knobs.h, [232](#)
 - starforth_config.h, [242](#)
- ADAPTIVE_MIN_WINDOW_SIZE
 - rolling_window_knobs.h, [232](#)
 - starforth_config.h, [242](#)
- adaptive_pending
 - RollingWindowOfTruth, [128](#)
- ADAPTIVE_SHRINK_RATE
 - rolling_window_knobs.h, [232](#)
 - starforth_config.h, [242](#)
- adaptive_shrink_rate
 - DoeMetrics, [75](#)
- adaptive_window_width
 - InferenceOutputs, [106](#)
- addr
 - Runtime Branch Helpers, [38](#)
- ADDR_MASK
 - vmm.c, [531](#)
- align_address
 - dictionary_manipulation_words.c, [658](#)
- align_up
 - physics_runtime.c, [434](#)
- align_up_size
 - kmalloc.c, [523](#)
- AllHandles
 - uefi.h, [300](#)
- alloc
 - VMHostServices, [153](#)
- alloc_count
 - sf_alloc_stats, [130](#)
- alloc_host.c
 - g_stats, [439](#)
 - sf_alloc_get_stats, [439](#)

- sf_alloc_init, [439](#)
- sf_calloc, [439](#)
- sf_free, [439](#)
- sf_malloc, [439](#)
- sf_realloc, [439](#)
- alloc_kernel.c
 - __attribute__, [441](#), [443](#)
 - g_arena_offset, [441](#), [443](#)
 - g_initialized, [441](#), [443](#)
 - g_stats, [441](#), [443](#)
 - SF_ALIGN, [440](#), [442](#)
 - SF_ALIGN_UP, [440](#), [442](#)
 - sf_alloc_get_stats, [440](#), [442](#)
 - sf_alloc_init, [440](#), [442](#)
 - SF_ARENA_SIZE, [440](#), [442](#)
 - sf_calloc, [440](#), [442](#)
 - sf_free, [440](#), [442](#)
 - sf_malloc, [440](#), [442](#)
 - sf_realloc, [441](#), [443](#)
- alloc_page_zeroed
 - vmm.c, [532](#)
- allocate_from_block
 - kmalloc.c, [523](#)
- AllocateAddress
 - uefi.h, [300](#)
- AllocateAnyPages
 - uefi.h, [300](#)
- AllocateMaxAddress
 - uefi.h, [300](#)
- AllocatePages
 - _EFI_BOOT_SERVICES, [46](#)
- AllocatePool
 - _EFI_BOOT_SERVICES, [46](#)
- analytics_heap_allocate
 - physics_runtime.c, [434](#)
- analytics_heap_release
 - physics_runtime.c, [434](#)
- analytics_ring_free
 - physics_runtime.c, [434](#)
- apic.c
 - apic_eoi, [470](#)
 - apic_init, [470](#)
 - APIC_REG_EOI, [468](#)
 - APIC_REG_ID, [468](#)
 - APIC_REG_LVT_TIMER, [469](#)
 - APIC_REG_SIVR, [469](#)
 - APIC_REG_TIMER_CCR, [469](#)
 - APIC_REG_TIMER_DCR, [469](#)
 - APIC_REG_TIMER_ICR, [469](#)
 - apic_timer_init, [470](#)
 - apic_timer_period_tsc, [470](#)
 - apic_timer_start, [471](#)
 - apic_timer_stop, [471](#)
 - calibrate_apic_timer, [471](#)
 - lapic_base, [472](#)
 - LAPIC_DEFAULT_PHYS, [469](#)
 - lapic_phys_base, [472](#)
 - lapic_read, [471](#)
 - LAPIC_VIRT_BASE, [469](#)
 - lapic_write, [471](#)
 - LVT_MASKED, [469](#)
 - LVT_MODE_ONESHOT, [469](#)
 - LVT_MODE_PERIODIC, [469](#)
 - TIMER_DIV_1, [469](#)
 - TIMER_DIV_128, [469](#)
 - TIMER_DIV_16, [469](#)
 - TIMER_DIV_2, [469](#)
 - TIMER_DIV_32, [470](#)
 - TIMER_DIV_4, [470](#)
 - TIMER_DIV_64, [470](#)
 - TIMER_DIV_8, [470](#)
 - timer_initial_count, [472](#)
 - timer_period_tsc_ticks, [472](#)
 - timer_tick_hz, [472](#)
- apic.h
 - apic_eoi, [245](#)
 - apic_init, [245](#)
 - apic_timer_init, [245](#)
 - apic_timer_period_tsc, [245](#)
 - apic_timer_start, [245](#)
 - apic_timer_stop, [246](#)
 - APIC_TIMER_VECTOR, [245](#)
- apic_eoi
 - apic.c, [470](#)
 - apic.h, [245](#)
- apic_init
 - apic.c, [470](#)
 - apic.h, [245](#)
- APIC_REG_EOI
 - apic.c, [468](#)
- APIC_REG_ID
 - apic.c, [468](#)
- APIC_REG_LVT_TIMER
 - apic.c, [469](#)
- APIC_REG_SIVR
 - apic.c, [469](#)
- APIC_REG_TIMER_CCR
 - apic.c, [469](#)
- APIC_REG_TIMER_DCR
 - apic.c, [469](#)
- APIC_REG_TIMER_ICR
 - apic.c, [469](#)
- apic_timer_init
 - apic.c, [470](#)
 - apic.h, [245](#)
- apic_timer_period_tsc
 - apic.c, [470](#)
 - apic.h, [245](#)
- apic_timer_start
 - apic.c, [471](#)
 - apic.h, [245](#)
- apic_timer_stop
 - apic.c, [471](#)
 - apic.h, [246](#)
- APIC_TIMER_VECTOR
 - apic.h, [245](#)

- app_data
 - blk_meta_t, 55
- arch.c
 - __attribute__, 473
 - access, 474
 - arch_disable_interrupts, 473
 - arch_early_init, 473
 - arch_enable_interrupts, 473
 - arch_halt, 473
 - arch_mmu_init, 473
 - arch_read_timestamp, 473
 - base, 474
 - base_high, 474
 - base_low, 474
 - base_mid, 474
 - gdt_init, 474
 - gdt_set_entry, 474
 - granularity, 474
 - limit, 474
 - limit_low, 475
- arch.h
 - arch_disable_interrupts, 246
 - arch_early_init, 246
 - arch_enable_interrupts, 246
 - arch_halt, 246
 - arch_interrupts_init, 247
 - arch_mmu_init, 247
 - arch_read_timestamp, 247
 - arch_relax, 247
- ARCH_ARM64
 - arch_detect.h, 166
- arch_detect.h
 - ARCH_ARM64, 166
 - ARCH_NAME, 166
 - ARCH_RISCV64, 166
 - ARCH_X86_64, 166
 - NEXT_OPT, 166
 - PRIM_BRANCH_OPT, 166
 - PRIM_DROP_OPT, 166
 - PRIM_DUP_OPT, 166
 - PRIM_FETCH_OPT, 166
 - PRIM_MINUS_OPT, 166
 - PRIM_PLUS_OPT, 167
 - PRIM_R_FETCH_OPT, 167
 - PRIM_R_FROM_OPT, 167
 - PRIM_STAR_OPT, 167
 - PRIM_STORE_OPT, 167
 - PRIM_SWAP_OPT, 167
 - PRIM_TO_R_OPT, 167
 - PRIM_ZBRANCH_OPT, 167
 - vm_abs_opt, 167
 - vm_add_overflow_opt, 167
 - vm_clz_opt, 167
 - vm_ctz_opt, 167
 - vm_divmod_opt, 167
 - vm_get_arch_name, 169
 - vm_get_build_info, 169
 - vm_has_asm_opt, 169
 - vm_has_direct_threading, 170
 - vm_has_sse42_opt, 170
 - vm_max_opt, 168
 - vm_memcpy_opt, 168
 - vm_memzero_opt, 168
 - vm_min_opt, 168
 - vm_mul_double_opt, 168
 - vm_pop_opt, 168
 - vm_popcnt_opt, 168
 - vm_prefetch_opt, 168
 - vm_prefetch_stream_opt, 168
 - vm_push_opt, 168
 - vm_rpop_opt, 169
 - vm_rpush_opt, 169
 - vm_save_registers_opt, 169
 - vm_setup_registers_opt, 169
 - vm_strcmp_opt, 169
 - vm_sub_overflow_opt, 169
- arch_disable_interrupts
 - arch.c, 473
 - arch.h, 246
- arch_early_init
 - arch.c, 473
 - arch.h, 246
- arch_enable_interrupts
 - arch.c, 473
 - arch.h, 246
- arch_halt
 - arch.c, 473
 - arch.h, 246
- arch_interrupts_init
 - arch.h, 247
 - interrupts.c, 476
- arch_mmu_init
 - arch.c, 473
 - arch.h, 247
- ARCH_NAME
 - arch_detect.h, 166
- arch_read_timestamp
 - arch.c, 473
 - arch.h, 247
- arch_relax
 - arch.h, 247
- ARCH_RISCV64
 - arch_detect.h, 166
- ARCH_X86_64
 - arch_detect.h, 166
- Architecture Detection, 15
- arena_base
 - CapsuleDirHeader, 66
- arena_size
 - CapsuleDirHeader, 66
- arithmetic_word_abs
 - arithmetic_words.c, 640
- arithmetic_word_max
 - arithmetic_words.c, 640
- arithmetic_word_min
 - arithmetic_words.c, 640

- arithmetic_word_minus
 - arithmetic_words.c, [640](#)
- arithmetic_word_multiply
 - arithmetic_words.c, [640](#)
- arithmetic_word_negate
 - arithmetic_words.c, [641](#)
- arithmetic_word_one_minus
 - arithmetic_words.c, [641](#)
- arithmetic_word_one_plus
 - arithmetic_words.c, [641](#)
- arithmetic_word_plus
 - arithmetic_words.c, [641](#)
- arithmetic_word_star_slash
 - arithmetic_words.c, [641](#)
- arithmetic_word_star_slash_mod
 - arithmetic_words.c, [641](#)
- arithmetic_word_suites
 - arithmetic_words_test.c, [579](#)
- arithmetic_word_two_divide
 - arithmetic_words.c, [642](#)
- arithmetic_word_two_minus
 - arithmetic_words.c, [642](#)
- arithmetic_word_two_multiply
 - arithmetic_words.c, [642](#)
- arithmetic_word_two_plus
 - arithmetic_words.c, [642](#)
- arithmetic_words.c
 - arithmetic_word_abs, [640](#)
 - arithmetic_word_max, [640](#)
 - arithmetic_word_min, [640](#)
 - arithmetic_word_minus, [640](#)
 - arithmetic_word_multiply, [640](#)
 - arithmetic_word_negate, [641](#)
 - arithmetic_word_one_minus, [641](#)
 - arithmetic_word_one_plus, [641](#)
 - arithmetic_word_plus, [641](#)
 - arithmetic_word_star_slash, [641](#)
 - arithmetic_word_star_slash_mod, [641](#)
 - arithmetic_word_two_divide, [642](#)
 - arithmetic_word_two_minus, [642](#)
 - arithmetic_word_two_multiply, [642](#)
 - arithmetic_word_two_plus, [642](#)
 - mixed_math_word_mod, [642](#)
 - register_arithmetic_words, [642](#)
 - word_divide, [643](#)
 - word_divmod, [643](#)
- arithmetic_words.h
 - register_arithmetic_words, [679](#)
- arithmetic_words_test.c
 - arithmetic_word_suites, [579](#)
 - run_arithmetic_words_tests, [579](#)
- assert_heartbeat_safe
 - test_contracts.c, [605](#)
 - test_contracts.h, [564](#)
- assert_stack_depth
 - test_common.c, [599](#)
 - test_common.h, [555](#)
- assert_stack_top
 - test_common.c, [600](#)
 - test_common.h, [557](#)
- assert_vm_error
 - test_common.c, [600](#)
 - test_common.h, [557](#)
- Attribute
 - EFI_FILE_INFO, [80](#)
 - EFI_MEMORY_DESCRIPTOR, [83](#)
- avg_latency_ns
 - DictPhysics, [73](#)
 - physics_seed_definition, [120](#)
- avg_time
 - WordProfile, [157](#)
- avg_word_heat
 - DoeMetrics, [75](#)
 - HeartbeatTickSnapshot, [98](#)
- backend
 - blkcfg_t, [59](#)
 - physics_host_snapshot, [116](#)
- backend_seq
 - physics_host_snapshot, [116](#)
 - physics_runtime_state, [119](#)
- bam
 - block_subsystem.c, [384](#)
- bam_bytes
 - block_subsystem.c, [384](#)
- bam_clr
 - block_subsystem.c, [377](#)
- bam_devblocks
 - blk_volume_meta_t, [57](#)
- bam_dirty
 - block_subsystem.c, [384](#)
- bam_flush_to_disk
 - block_subsystem.c, [377](#)
- bam_load_from_disk
 - block_subsystem.c, [377](#)
- bam_set
 - block_subsystem.c, [377](#)
- bam_start
 - blk_volume_meta_t, [57](#)
- bam_test
 - block_subsystem.c, [377](#)
- base
 - arch.c, [474](#)
 - blkio_ram_state_t, [62](#)
 - FramebufferInfo, [91](#)
 - gdt, [92](#)
 - idtr, [104](#)
 - interrupts.c, [478](#)
 - physics_analytics_heap, [114](#)
 - VM, [143](#)
- base_addr
 - VM, [143](#)
- base_high
 - arch.c, [474](#)
 - gdt_entry, [92](#)
- base_low
 - arch.c, [474](#)

- gdt_entry, [92](#)
- base_mid
 - arch.c, [474](#)
 - gdt_entry, [92](#)
- BayesianLatencyPosterior, [53](#)
 - credible_lower_95, [53](#)
 - credible_lower_99, [53](#)
 - credible_upper_95, [54](#)
 - credible_upper_99, [54](#)
 - mean_ns_q48, [54](#)
 - median_ns_q48, [54](#)
 - sample_count, [54](#)
 - stddev_ns_q48, [54](#)
- benchmark
 - CLIConfig, [69](#)
- benchmark_iterations
 - CLIConfig, [69](#)
 - test_runner.c, [609](#)
- benchmark_mode
 - test_runner.c, [609](#)
- birth_capsule_id
 - VMRegistryEntry, [155](#)
- birth_dict_hash
 - VMRegistryEntry, [155](#)
- birth_timestamp_ns
 - VMRegistryEntry, [155](#)
- bitmap_clear
 - pmm.c, [527](#)
- bitmap_mark_range
 - pmm.c, [527](#)
- bitmap_set
 - pmm.c, [527](#)
- bitmap_test
 - pmm.c, [527](#)
- blk_allocate
 - block_subsystem.c, [377](#)
 - block_subsystem.h, [177](#)
- BLK_DEVICE_SECTOR
 - block_subsystem.h, [176](#)
- BLK_DISK_START
 - block_subsystem.h, [176](#)
- BLK_DISK_SYS_RESERVED
 - block_subsystem.c, [376](#)
 - block_subsystem.h, [176](#)
- BLK_DIRTY
 - block_subsystem.h, [177](#)
- BLK_EINVAL
 - block_subsystem.h, [176](#)
- BLK_EIO
 - block_subsystem.h, [176](#)
- BLK_ENODEV
 - block_subsystem.h, [177](#)
- BLK_ENOMEM
 - block_subsystem.h, [177](#)
- BLK_ERANGE
 - block_subsystem.h, [176](#)
- BLK_ERESERVED
 - block_subsystem.h, [177](#)
- blk_flush
 - block_subsystem.c, [378](#)
 - block_subsystem.h, [177](#)
- blk_format_or_load_v2
 - block_subsystem.c, [378](#)
- BLK_FORTH_SIZE
 - block_subsystem.h, [176](#)
- BLK_FORTH_SYS_RESERVED
 - block_subsystem.c, [376](#)
 - block_subsystem.h, [176](#)
- blk_get_buffer
 - block_subsystem.c, [378](#)
 - block_subsystem.h, [177](#)
- blk_get_empty_buffer
 - block_subsystem.c, [378](#)
 - block_subsystem.h, [177](#)
- blk_get_meta
 - block_subsystem.c, [378](#)
 - block_subsystem.h, [177](#)
- blk_get_timestamp
 - block_subsystem.c, [378](#)
- blk_get_total_blocks
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [177](#)
- blk_get_volume_meta
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- blk_is_allocated
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- blk_is_valid
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- blk_mark_allocated
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- blk_mark_free
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- BLK_META_PER_BLOCK
 - block_subsystem.h, [176](#)
- blk_meta_t, [54](#)
 - acl_block, [55](#)
 - app_data, [55](#)
 - chain_length, [55](#)
 - checksum, [55](#)
 - content_length, [55](#)
 - content_type, [55](#)
 - created_time, [55](#)
 - encoding, [55](#)
 - entropy, [55](#)
 - flags, [55](#)
 - hash, [55](#)
 - magic, [55](#)
 - modified_time, [56](#)
 - next_block, [56](#)
 - owner_id, [56](#)
 - padding, [56](#)

- parent_block, [56](#)
- permissions, [56](#)
- prev_block, [56](#)
- reserved1, [56](#)
- signature, [56](#)
- write_count, [56](#)
- BLK_META_TOTAL
 - block_subsystem.h, [176](#)
- BLK_OK
 - block_subsystem.h, [176](#)
- BLK_PACK_RATIO
 - block_subsystem.h, [176](#)
- BLK_RAM_BLOCKS
 - block_subsystem.h, [176](#)
- blk_set_meta
 - block_subsystem.c, [379](#)
 - block_subsystem.h, [178](#)
- blk_set_volume_meta
 - block_subsystem.c, [380](#)
 - block_subsystem.h, [178](#)
- blk_subsys_attach_device
 - block_subsystem.c, [380](#)
 - block_subsystem.h, [179](#)
- blk_subsys_init
 - block_subsystem.c, [380](#)
 - block_subsystem.h, [179](#)
- blk_subsys_shutdown
 - block_subsystem.c, [380](#)
 - block_subsystem.h, [179](#)
- blk_update
 - block_subsystem.c, [380](#)
 - block_subsystem.h, [179](#)
- blk_volume_meta_t, [56](#)
 - _pad, [57](#)
 - bam_devblocks, [57](#)
 - bam_start, [57](#)
 - created_time, [57](#)
 - devblock_base, [57](#)
 - first_free, [57](#)
 - flags, [57](#)
 - free_blocks, [57](#)
 - hdr_crc, [57](#)
 - label, [57](#)
 - last_allocated, [57](#)
 - magic, [58](#)
 - mounted_time, [58](#)
 - reserved_disk_lo, [58](#)
 - reserved_ram_lo, [58](#)
 - total_blocks, [58](#)
 - total_devblocks, [58](#)
 - total_volumes, [58](#)
 - tracked_blocks, [58](#)
 - version, [58](#)
- blkcfg.h
 - BLKCFG_BACKEND_FILE, [171](#)
 - BLKCFG_BACKEND_NONE, [171](#)
 - BLKCFG_BACKEND_RAM, [171](#)
 - blkcfg_backend_t, [171](#)
 - BLKCFG_DEFAULT_FBS, [170](#)
 - blkcfg_format_summary, [171](#)
 - blkcfg_init_from_options, [171](#)
 - BLKCFG_PATH_MAX, [170](#)
 - g_blkcfg, [171](#)
- BLKCFG_BACKEND_FILE
 - blkcfg.h, [171](#)
- BLKCFG_BACKEND_NONE
 - blkcfg.h, [171](#)
- BLKCFG_BACKEND_RAM
 - blkcfg.h, [171](#)
- blkcfg_backend_t
 - blkcfg.h, [171](#)
- BLKCFG_DEFAULT_FBS
 - blkcfg.h, [170](#)
- blkcfg_format_summary
 - blkcfg.h, [171](#)
- blkcfg_init_from_options
 - blkcfg.h, [171](#)
- BLKCFG_PATH_MAX
 - blkcfg.h, [170](#)
- blkcfg_t, [58](#)
 - backend, [59](#)
 - blocks, [59](#)
 - fbs, [59](#)
 - file_path, [59](#)
 - read_only, [59](#)
 - size_bytes, [59](#)
- blkio.h
 - blkio_bzero, [173](#)
 - blkio_close, [173](#)
 - blkio_dev_t, [172](#)
 - BLKIO_ECLOSED, [173](#)
 - BLKIO_EINVAL, [172](#)
 - BLKIO_EIO, [173](#)
 - BLKIO_ENOSPACE, [173](#)
 - BLKIO_ENOSUP, [172](#)
 - blkio_flush, [173](#)
 - BLKIO_FORTH_BLOCK_SIZE, [172](#)
 - blkio_info, [173](#)
 - BLKIO_OK, [172](#)
 - blkio_open, [173](#)
 - blkio_read, [173](#)
 - blkio_write, [174](#)
- blkio_bzero
 - blkio.h, [173](#)
- blkio_close
 - blkio.h, [173](#)
- blkio_dev, [59](#)
 - forth_block_size, [59](#)
 - is_open, [59](#)
 - state, [59](#)
 - total_blocks, [59](#)
 - vt, [60](#)
- blkio_dev_t
 - blkio.h, [172](#)
- BLKIO_ECLOSED
 - blkio.h, [173](#)

- BLKIO_EINVAL
 - blkio.h, 172
- BLKIO_EIO
 - blkio.h, 173
- BLKIO_ENOSPACE
 - blkio.h, 173
- BLKIO_ENOSUP
 - blkio.h, 172
- blkio_factory.c
 - blkio_factory_open, 368
 - blkio_file_init_state, 368
 - blkio_file_state_size, 368
 - blkio_file_vtable, 369
 - blkio_ram_init_state, 369
 - blkio_ram_state_size, 369
 - blkio_ram_vtable, 369
 - nonempty, 369
- blkio_factory.h
 - blkio_factory_open, 174
- blkio_factory_open
 - blkio_factory.c, 368
 - blkio_factory.h, 174
- blkio_file.c
 - blkio_file_init_state, 370
 - blkio_file_state_size, 370
 - BLKIO_FILE_VT, 372
 - blkio_file_vtable, 371
 - file_close, 371
 - file_flush, 371
 - file_info, 371
 - file_open, 371
 - file_read, 371
 - file_write, 371
 - full_read_at, 372
 - full_write_at, 372
 - PATH_MAX, 370
- blkio_file_init_state
 - blkio_factory.c, 368
 - blkio_file.c, 370
- blkio_file_state_size
 - blkio_factory.c, 368
 - blkio_file.c, 370
- blkio_file_state_t, 60
 - fbs, 60
 - fd, 60
 - path, 60
 - read_only, 60
 - total_blocks, 60
- BLKIO_FILE_VT
 - blkio_file.c, 372
- blkio_file_vtable
 - blkio_factory.c, 369
 - blkio_file.c, 371
- blkio_flush
 - blkio.h, 173
- BLKIO_FORTH_BLOCK_SIZE
 - blkio.h, 172
- blkio_info
 - blkio.h, 173
- blkio_info_t, 60
 - forth_block_size, 61
 - phys_sector_size, 61
 - phys_size_bytes, 61
 - read_only, 61
 - total_blocks, 61
- BLKIO_OK
 - blkio.h, 172
- blkio_open
 - blkio.h, 173
- blkio_params_t, 61
 - forth_block_size, 61
 - opaque, 61
 - total_blocks, 61
- blkio_ram.c
 - blkio_ram_init_state, 373
 - blkio_ram_state_size, 373
 - BLKIO_RAM_VT, 374
 - blkio_ram_vtable, 373
 - in_bounds, 373
 - ram_close, 373
 - ram_flush, 373
 - ram_info, 374
 - ram_open, 374
 - ram_read, 374
 - ram_write, 374
- blkio_ram_init_state
 - blkio_factory.c, 369
 - blkio_ram.c, 373
- blkio_ram_state_size
 - blkio_factory.c, 369
 - blkio_ram.c, 373
- blkio_ram_state_t, 62
 - base, 62
 - fbs, 62
 - read_only, 62
 - total_blocks, 62
- BLKIO_RAM_VT
 - blkio_ram.c, 374
- blkio_ram_vtable
 - blkio_factory.c, 369
 - blkio_ram.c, 373
- blkio_read
 - blkio.h, 173
- blkio_vtable_t, 62
 - close, 62
 - flush, 62
 - info, 63
 - open, 63
 - read, 63
 - write, 63
- blkio_write
 - blkio.h, 174
- block
 - kmalloc_prefix_t, 107
- BLOCK_COUNT
 - io.h, 192

block_number
 BlockBuffer, 63
BLOCK_SIZE
 block_words.h, 681
 vm.h, 314
block_size
 reloc_block, 127
 reloc_stub.c, 492
block_subsystem.c
 __attribute__, 377
 bam, 384
 bam_bytes, 384
 bam_clr, 377
 bam_dirty, 384
 bam_flush_to_disk, 377
 bam_load_from_disk, 377
 bam_set, 377
 bam_test, 377
 blk_allocate, 377
 BLK_DISK_SYS_RESERVED, 376
 blk_flush, 378
 blk_format_or_load_v2, 378
 BLK_FORTH_SYS_RESERVED, 376
 blk_get_buffer, 378
 blk_get_empty_buffer, 378
 blk_get_meta, 378
 blk_get_timestamp, 378
 blk_get_total_blocks, 379
 blk_get_volume_meta, 379
 blk_is_allocated, 379
 blk_is_valid, 379
 blk_mark_allocated, 379
 blk_mark_free, 379
 blk_set_meta, 379
 blk_set_volume_meta, 380
 blk_subsys_attach_device, 380
 blk_subsys_init, 380
 blk_subsys_shutdown, 380
 blk_update, 380
 cache, 384
 cache_get_slot, 380
 cache_load_devblock, 381
 cache_writeback, 381
 choose_B, 381
 compute_crc64, 381
 compute_totals_from_B, 381
 crc64_init, 381
 crc64_inited, 384
 crc64_table, 384
 devblock4k_to_lba1k, 381
 devblock_base_4k, 384
 dirty_ram, 384
 DISK_CACHE_SLOTS, 376
 disk_dev, 384
 fblock_pack_offset, 382
 fblock_to_devblock, 382
 g, 384
 initialized, 384
 lbn_in_range, 382
 lbn_to_pbn, 382
 meta_from_slice, 382
 META_PER_BLOCK, 376
 META_REGION_OFFSET, 376
 META_REGION_SIZE, 377
 meta_to_slice, 382
 minzu, 382
 ram_base, 385
 ram_size, 385
 ram_user, 385
 read_devblock_4k, 383
 read_header_4k, 383
 total_blkio_blocks_1k, 385
 total_user_lbn, 385
 udiv_floor, 383
 vm, 385
 vol_meta, 387
 vol_meta_dirty, 387
 volmeta_from_buf, 383
 volmeta_to_buf, 383
 write_devblock_4k, 383
 write_header_4k, 383
block_subsystem.h
 blk_allocate, 177
 BLK_DEVICE_SECTOR, 176
 BLK_DISK_START, 176
 BLK_DISK_SYS_RESERVED, 176
 BLK_EDIRTY, 177
 BLK_EINVAL, 176
 BLK_EIO, 176
 BLK_ENODEV, 177
 BLK_ENOMEM, 177
 BLK_ERANGE, 176
 BLK_ERESERVED, 177
 blk_flush, 177
 BLK_FORTH_SIZE, 176
 BLK_FORTH_SYS_RESERVED, 176
 blk_get_buffer, 177
 blk_get_empty_buffer, 177
 blk_get_meta, 177
 blk_get_total_blocks, 177
 blk_get_volume_meta, 178
 blk_is_allocated, 178
 blk_is_valid, 178
 blk_mark_allocated, 178
 blk_mark_free, 178
 BLK_META_PER_BLOCK, 176
 BLK_META_TOTAL, 176
 BLK_OK, 176
 BLK_PACK_RATIO, 176
 BLK_RAM_BLOCKS, 176
 blk_set_meta, 178
 blk_set_volume_meta, 178
 blk_subsys_attach_device, 179
 blk_subsys_init, 179
 blk_subsys_shutdown, 179
 blk_update, 179

- block_word_block
 - block_words.c, [644](#)
- block_word_buffer
 - block_words.c, [644](#)
- block_word_empty_buffers
 - block_words.c, [644](#)
- block_word_flush
 - block_words.c, [644](#)
- block_word_list
 - block_words.c, [644](#)
- block_word_load
 - block_words.c, [644](#)
- block_word_next_block
 - block_words.c, [644](#)
- block_word_save_buffers
 - block_words.c, [645](#)
- block_word_scr
 - block_words.c, [645](#)
- block_word_suites
 - block_words_test.c, [580](#)
- block_word_thru
 - block_words.c, [645](#)
- block_word_update
 - block_words.c, [645](#)
- block_words.c
 - block_word_block, [644](#)
 - block_word_buffer, [644](#)
 - block_word_empty_buffers, [644](#)
 - block_word_flush, [644](#)
 - block_word_list, [644](#)
 - block_word_load, [644](#)
 - block_word_next_block, [644](#)
 - block_word_save_buffers, [645](#)
 - block_word_scr, [645](#)
 - block_word_thru, [645](#)
 - block_word_update, [645](#)
 - empty_all_buffers, [645](#)
 - get_block_buffer, [645](#)
 - get_empty_buffer, [646](#)
 - init_block_system, [646](#)
 - mark_buffer_dirty, [646](#)
 - register_block_words, [647](#)
 - save_all_buffers, [647](#)
 - set_scr, [647](#)
- block_words.h
 - BLOCK_SIZE, [681](#)
 - BUFFER_COUNT, [681](#)
 - BUFFER_TYPE_HARDWARE, [682](#)
 - BUFFER_TYPE_MEMORY, [682](#)
 - BUFFER_TYPE_NETWORK, [682](#)
 - BUFFER_TYPE_STORAGE, [682](#)
 - buffer_type_t, [682](#)
 - empty_all_buffers, [682](#)
 - get_block_buffer, [682](#)
 - get_empty_buffer, [684](#)
 - init_block_system, [684](#)
 - mark_buffer_dirty, [684](#)
 - MAX_BLOCKS, [681](#)
 - register_block_words, [685](#)
 - save_all_buffers, [685](#)
- block_words_test.c
 - block_word_suites, [580](#)
 - run_block_words_tests, [579](#)
- BlockBuffer, [63](#)
 - accessed, [63](#)
 - block_number, [63](#)
 - data, [64](#)
 - dirty, [64](#)
 - hw_address, [64](#)
 - type, [64](#)
- blocks
 - blkcfg_t, [59](#)
- BOOLEAN
 - uefi.h, [297](#)
- BootInfo, [64](#)
 - acpi_table, [64](#)
 - framebuffer, [64](#)
 - memory_map, [64](#)
 - memory_map_descriptor_size, [64](#)
 - memory_map_size, [65](#)
 - runtime_services, [65](#)
 - uefi_boot_services_exited, [65](#)
- BootServices
 - _EFI_SYSTEM_TABLE, [52](#)
- bootstrap_result
 - ParityPacket, [109](#)
- branch_count
 - Profiler, [124](#)
- break_me
 - CLICfg, [69](#)
- break_me_tests.c
 - finalize_report, [581](#)
 - init_report, [581](#)
 - report_file, [581](#)
 - run_break_me_tests, [581](#)
 - test_end_time, [581](#)
 - test_start_time, [581](#)
 - total_tests_run, [581](#)
- bucket_collapse_flag
 - HeartbeatState, [95](#)
- bucket_hit_percent
 - DoeMetrics, [75](#)
- bucket_hits
 - doe_metrics.c, [400](#)
 - DoeMetrics, [75](#)
 - HotwordsStats, [101](#)
- bucket_hits_delta
 - HeartbeatTickSnapshot, [98](#)
- bucket_mode_transitions
 - HeartbeatState, [95](#)
- bucket_reorders
 - HotwordsStats, [101](#)
- bucket_search_latency_ns
 - DoeMetrics, [75](#)
- bucket_search_samples
 - HotwordsStats, [101](#)

- bucket_search_stddev_ns
 - DoeMetrics, [75](#)
- bucket_search_total_ns_q48
 - HotwordsStats, [102](#)
- bucket_search_variance_sum_q48
 - HotwordsStats, [102](#)
- bucket_sum_heat_variance
 - HeartbeatState, [95](#)
- bucket_sum_K
 - HeartbeatState, [95](#)
- bucket_sum_K_squared
 - HeartbeatState, [95](#)
- bucket_sum_window_variance
 - HeartbeatState, [95](#)
- bucket_tick_count
 - HeartbeatState, [96](#)
- buffer
 - XXHash64State, [162](#)
- BUFFER_COUNT
 - block_words.h, [681](#)
- buffer_size
 - XXHash64State, [162](#)
- BUFFER_TYPE_HARDWARE
 - block_words.h, [682](#)
- BUFFER_TYPE_MEMORY
 - block_words.h, [682](#)
- BUFFER_TYPE_NETWORK
 - block_words.h, [682](#)
- BUFFER_TYPE_STORAGE
 - block_words.h, [682](#)
- buffer_type_t
 - block_words.h, [682](#)
- Build Information, [15](#)
- ByProtocol
 - uefi.h, [300](#)
- ByRegisterNotify
 - uefi.h, [300](#)
- bytes
 - physics_analytics_heap, [114](#)
- cache
 - block_subsystem.c, [384](#)
 - HotwordsCache_s, [100](#)
- cache_count
 - HotwordsCache_s, [100](#)
- cache_demotions
 - DoeMetrics, [75](#)
- cache_get_slot
 - block_subsystem.c, [380](#)
- cache_hit_latency_ns
 - DoeMetrics, [75](#)
- cache_hit_percent
 - DoeMetrics, [75](#)
- cache_hit_samples
 - HotwordsStats, [102](#)
- cache_hit_stddev_ns
 - DoeMetrics, [75](#)
- cache_hit_total_ns_q48
 - HotwordsStats, [102](#)
- cache_hit_variance_sum_q48
 - HotwordsStats, [102](#)
- cache_hits
 - doe_metrics.c, [400](#)
 - DoeMetrics, [76](#)
 - HotwordsStats, [102](#)
 - MemoryProfile, [108](#)
- cache_hits_delta
 - HeartbeatTickSnapshot, [98](#)
- cache_load_devblock
 - block_subsystem.c, [381](#)
- cache_misses
 - MemoryProfile, [108](#)
- cache_promotions
 - DoeMetrics, [76](#)
- cache_slot_t, [65](#)
 - data, [65](#)
 - devblock, [65](#)
 - dirty, [65](#)
 - loaded, [65](#)
 - meta, [65](#)
 - meta_dirty, [66](#)
 - valid, [66](#)
- cache_writeback
 - block_subsystem.c, [381](#)
- calculate_posterior
 - physics_hotwords_cache.c, [420](#)
- calib_record
 - timer.c, [489](#)
- calibrate_apic_timer
 - apic.c, [471](#)
- calibrate_tsc_with_pmtimer
 - timer.c, [482](#)
- calibrate_window_hpet
 - timer.c, [482](#)
- calibrate_window_pit
 - timer.c, [482](#)
- call_count
 - ProfileTimer, [126](#)
 - WordProfile, [157](#)
 - WordStats, [158](#)
- can_split
 - kmalloc.c, [523](#)
- capsule.h
 - __attribute__, [251](#)
 - CAPSULE_BIRTH_ELIGIBLE, [249](#)
 - CAPSULE_DESC_MAGIC, [249](#)
 - CAPSULE_DIR_MAGIC, [249](#)
 - CAPSULE_DOE_ELIGIBLE, [249](#)
 - CAPSULE_ERR_BAD_HASH_ALG, [251](#)
 - CAPSULE_ERR_BAD_MAGIC, [251](#)
 - CAPSULE_ERR_BAD_VERSION, [251](#)
 - CAPSULE_ERR_BOUNDS, [251](#)
 - CAPSULE_ERR_HASH_MISMATCH, [251](#)
 - CAPSULE_ERR_MODE_INVALID, [251](#)
 - CAPSULE_ERR_NULL_PTR, [251](#)
 - CAPSULE_ERR_REVOKED_ACTIVE, [251](#)
 - capsule_find_by_id, [251](#)

- capsule_find_by_name, 252
- capsule_find_mama_init, 252
- CAPSULE_FLAG_ACTIVE, 249
- CAPSULE_FLAG_DEPRECATED, 249
- CAPSULE_FLAG_EXPERIMENT, 249
- CAPSULE_FLAG_MAMA_INIT, 249
- CAPSULE_FLAG_PINNED, 249
- CAPSULE_FLAG_PRODUCTION, 250
- CAPSULE_FLAG_REVOKED, 250
- capsule_get_payload, 252
- CAPSULE_HASH_BLAKE3, 251
- CAPSULE_HASH_SHA256, 251
- CAPSULE_HASH_XXHASH64, 251
- CAPSULE_IS_MAMA_INIT, 250
- CAPSULE_MAGIC_GET_HASHALG, 250
- CAPSULE_MAGIC_GET_SIG, 250
- CAPSULE_MAGIC_GET_VERSION, 250
- CAPSULE_MAGIC_PACK, 250
- CAPSULE_MAX_COUNT, 250
- CAPSULE_MODE_VALID, 250
- CAPSULE_NAME_MAX, 250
- CAPSULE_VALID, 251
- capsule_validate, 253
- capsule_validate_result_str, 253
- CAPSULE_VERSION_0, 250
- CapsuleDesc, 253
- CapsuleHashAlg, 251
- CapsuleValidateResult, 251
- capsule_birth.c
 - capsule_birth_baby, 495
 - capsule_birth_mama, 496
 - capsule_birth_set_hooks, 496
 - capsule_run_experiment, 497
 - capsule_vm_registry_count, 497
 - capsule_vm_registry_get, 497
 - capsule_vm_registry_init, 498
 - dispatch_init_forth, 498
 - MAX_VMS, 495
 - next_vm_id, 498
 - vm_alloc_fn, 498
 - vm_dict_hash_fn, 498
 - vm_exec_fn, 498
 - vm_registry, 499
 - vm_registry_alloc, 498
 - vm_registry_count, 499
- capsule_birth.h
 - capsule_birth_baby, 255
 - capsule_birth_mama, 256
 - capsule_birth_set_hooks, 256
 - capsule_run_experiment, 257
 - capsule_vm_hooks_register, 257
 - capsule_vm_registry_count, 257
 - capsule_vm_registry_get, 257
 - capsule_vm_registry_init, 258
 - CapsuleDictHashFn, 254
 - CapsuleExecFn, 254
 - CapsuleVMAllocFn, 255
- capsule_birth_baby
 - capsule_birth.c, 495
 - capsule_birth.h, 255
- CAPSULE_BIRTH_ELIGIBLE
 - capsule.h, 249
- capsule_birth_mama
 - capsule_birth.c, 496
 - capsule_birth.h, 256
- capsule_birth_set_hooks
 - capsule_birth.c, 496
 - capsule_birth.h, 256
- capsule_clear_blocks
 - capsule_loader.c, 502
 - capsule_loader.h, 259
- CAPSULE_DESC_MAGIC
 - capsule.h, 249
- capsule_dict_hash_hook
 - capsule_vm_hooks.c, 510
- CAPSULE_DIR_MAGIC
 - capsule.h, 249
- CAPSULE_DOE_ELIGIBLE
 - capsule.h, 249
- CAPSULE_ERR_BAD_HASH_ALG
 - capsule.h, 251
- CAPSULE_ERR_BAD_MAGIC
 - capsule.h, 251
- CAPSULE_ERR_BAD_VERSION
 - capsule.h, 251
- CAPSULE_ERR_BOUNDS
 - capsule.h, 251
- CAPSULE_ERR_HASH_MISMATCH
 - capsule.h, 251
- CAPSULE_ERR_MODE_INVALID
 - capsule.h, 251
- CAPSULE_ERR_NULL_PTR
 - capsule.h, 251
- CAPSULE_ERR_REVOKED_ACTIVE
 - capsule.h, 251
- capsule_exec_hook
 - capsule_vm_hooks.c, 510
- capsule_exec_init
 - capsule_loader.c, 502
 - capsule_loader.h, 259
- capsule_find.c
 - capsule_find_by_id, 500
 - capsule_find_by_name, 500
 - capsule_find_mama_init, 500
 - capsule_get_payload, 501
 - capsule_streq, 501
- capsule_find_by_id
 - capsule.h, 251
 - capsule_find.c, 500
- capsule_find_by_name
 - capsule.h, 252
 - capsule_find.c, 500
- capsule_find_mama_init
 - capsule.h, 252
 - capsule_find.c, 500
- CAPSULE_FLAG_ACTIVE

- capsule.h, [249](#)
- CAPSULE_FLAG_DEPRECATED
 - capsule.h, [249](#)
- CAPSULE_FLAG_EXPERIMENT
 - capsule.h, [249](#)
- CAPSULE_FLAG_MAMA_INIT
 - capsule.h, [249](#)
- CAPSULE_FLAG_PINNED
 - capsule.h, [249](#)
- CAPSULE_FLAG_PRODUCTION
 - capsule.h, [250](#)
- CAPSULE_FLAG_REVOKED
 - capsule.h, [250](#)
- capsule_generated.h
 - __attribute__, [258](#)
- capsule_get_payload
 - capsule.h, [252](#)
 - capsule_find.c, [501](#)
- capsule_hash
 - CapsuleRunRecord, [67](#)
- CAPSULE_HASH_BLAKE3
 - capsule.h, [251](#)
- CAPSULE_HASH_SHA256
 - capsule.h, [251](#)
- CAPSULE_HASH_XXHASH64
 - capsule.h, [251](#)
- capsule_id
 - CapsuleRunRecord, [67](#)
- CAPSULE_IS_MAMA_INIT
 - capsule.h, [250](#)
- capsule_load_blocks
 - capsule_loader.c, [503](#)
 - capsule_loader.h, [260](#)
- capsule_loader.c
 - capsule_clear_blocks, [502](#)
 - capsule_exec_init, [502](#)
 - capsule_load_blocks, [503](#)
 - CAPSULE_RAM_OFFSET, [502](#)
 - copy_block_to_ram, [504](#)
 - copy_device_to_ram, [504](#)
 - is_block_header, [504](#)
 - LOADER_BLOCK_SIZE, [502](#)
 - u32_to_dec, [504](#)
 - write_ramdrive_block, [504](#)
- capsule_loader.h
 - capsule_clear_blocks, [259](#)
 - capsule_exec_init, [259](#)
 - capsule_load_blocks, [260](#)
- CAPSULE_MAGIC_GET_HASHALG
 - capsule.h, [250](#)
- CAPSULE_MAGIC_GET_SIG
 - capsule.h, [250](#)
- CAPSULE_MAGIC_GET_VERSION
 - capsule.h, [250](#)
- CAPSULE_MAGIC_PACK
 - capsule.h, [250](#)
- CAPSULE_MAX_COUNT
 - capsule.h, [250](#)
- CAPSULE_MAX_RUN_RECORDS
 - capsule_run.h, [261](#)
- CAPSULE_MODE_VALID
 - capsule.h, [250](#)
- CAPSULE_NAME_MAX
 - capsule.h, [250](#)
- capsule_parity_log_birth
 - capsule_run.c, [505](#)
 - capsule_run.h, [262](#)
- capsule_parity_log_birth_failed
 - capsule_run.c, [505](#)
 - capsule_run.h, [262](#)
- capsule_parity_log_mama_init
 - capsule_run.c, [506](#)
 - capsule_run.h, [262](#)
- capsule_parity_log_run
 - capsule_run.c, [506](#)
 - capsule_run.h, [263](#)
- capsule_parity_set_output
 - capsule_run.c, [506](#)
 - capsule_run.h, [263](#)
- CAPSULE_RAM_OFFSET
 - capsule_loader.c, [502](#)
- capsule_run.c
 - capsule_parity_log_birth, [505](#)
 - capsule_parity_log_birth_failed, [505](#)
 - capsule_parity_log_mama_init, [506](#)
 - capsule_parity_log_run, [506](#)
 - capsule_parity_set_output, [506](#)
 - capsule_run_log_count, [506](#)
 - capsule_run_log_get, [507](#)
 - capsule_run_log_init, [507](#)
 - capsule_run_log_record, [507](#)
 - parity_put_hex64, [507](#)
 - parity_put_u32, [507](#)
 - parity_put_u64, [508](#)
 - parity_putc, [508](#)
 - parity_puts, [508](#)
 - run_id_counter, [508](#)
 - run_log, [508](#)
 - run_log_count, [508](#)
 - run_log_head, [508](#)
- capsule_run.h
 - CAPSULE_MAX_RUN_RECORDS, [261](#)
 - capsule_parity_log_birth, [262](#)
 - capsule_parity_log_birth_failed, [262](#)
 - capsule_parity_log_mama_init, [262](#)
 - capsule_parity_log_run, [263](#)
 - capsule_parity_set_output, [263](#)
 - CAPSULE_RUN_ERR_EXEC_FAIL, [262](#)
 - CAPSULE_RUN_ERR_HASH_MISMATCH, [262](#)
 - CAPSULE_RUN_ERR_INVALID, [261](#)
 - CAPSULE_RUN_ERR_NOT_ELIGIBLE, [262](#)
 - CAPSULE_RUN_ERR_STILLBORN, [262](#)
 - capsule_run_log_count, [263](#)
 - capsule_run_log_get, [263](#)
 - capsule_run_log_init, [264](#)
 - capsule_run_log_record, [264](#)

- CAPSULE_RUN_OK, [261](#)
- CapsuleRunResult, [261](#)
- VM_STATE_DEAD, [262](#)
- VM_STATE_EMBRYO, [262](#)
- VM_STATE_LIVE, [262](#)
- VM_STATE_STILLBORN, [262](#)
- VMState, [262](#)
- CAPSULE_RUN_ERR_EXEC_FAIL
 - capsule_run.h, [262](#)
- CAPSULE_RUN_ERR_HASH_MISMATCH
 - capsule_run.h, [262](#)
- CAPSULE_RUN_ERR_INVALID
 - capsule_run.h, [261](#)
- CAPSULE_RUN_ERR_NOT_ELIGIBLE
 - capsule_run.h, [262](#)
- CAPSULE_RUN_ERR_STILLBORN
 - capsule_run.h, [262](#)
- capsule_run_experiment
 - capsule_birth.c, [497](#)
 - capsule_birth.h, [257](#)
- capsule_run_log_count
 - capsule_run.c, [506](#)
 - capsule_run.h, [263](#)
- capsule_run_log_get
 - capsule_run.c, [507](#)
 - capsule_run.h, [263](#)
- capsule_run_log_init
 - capsule_run.c, [507](#)
 - capsule_run.h, [264](#)
- capsule_run_log_record
 - capsule_run.c, [507](#)
 - capsule_run.h, [264](#)
- CAPSULE_RUN_OK
 - capsule_run.h, [261](#)
- capsule_streq
 - capsule_find.c, [501](#)
- CAPSULE_VALID
 - capsule.h, [251](#)
- capsule_validate
 - capsule.h, [253](#)
 - capsule_validate.c, [509](#)
- capsule_validate.c
 - capsule_validate, [509](#)
 - capsule_validate_result_str, [509](#)
 - validate_result_strings, [509](#)
- capsule_validate_result_str
 - capsule.h, [253](#)
 - capsule_validate.c, [509](#)
- CAPSULE_VERSION_0
 - capsule.h, [250](#)
- capsule_vm_alloc_hook
 - capsule_vm_hooks.c, [510](#)
- capsule_vm_hooks.c
 - capsule_dict_hash_hook, [510](#)
 - capsule_exec_hook, [510](#)
 - capsule_vm_alloc_hook, [510](#)
 - capsule_vm_hooks_register, [510](#)
- capsule_vm_hooks_register
 - capsule_birth.h, [257](#)
 - capsule_vm_hooks.c, [510](#)
- capsule_vm_registry_count
 - capsule_birth.c, [497](#)
 - capsule_birth.h, [257](#)
- capsule_vm_registry_get
 - capsule_birth.c, [497](#)
 - capsule_birth.h, [257](#)
- capsule_vm_registry_init
 - capsule_birth.c, [498](#)
 - capsule_birth.h, [258](#)
- CapsuleDesc
 - capsule.h, [253](#)
- CapsuleDictHashFn
 - capsule_birth.h, [254](#)
- CapsuleDirHeader, [66](#)
 - arena_base, [66](#)
 - arena_size, [66](#)
 - desc_capacity, [66](#)
 - desc_count, [66](#)
 - dir_hash, [66](#)
 - magic, [66](#)
 - name_count, [67](#)
 - reserved, [67](#)
- CapsuleExecFn
 - capsule_birth.h, [254](#)
- CapsuleHashAlg
 - capsule.h, [251](#)
- CapsuleNameEntry, [67](#)
 - name, [67](#)
- CapsuleRunRecord, [67](#)
 - capsule_hash, [67](#)
 - capsule_id, [67](#)
 - ended_ns, [68](#)
 - flags, [68](#)
 - post_dict_hash, [68](#)
 - pre_dict_hash, [68](#)
 - reserved, [68](#)
 - result_code, [68](#)
 - run_id, [68](#)
 - started_ns, [68](#)
 - vm_id, [68](#)
- CapsuleRunResult
 - capsule_run.h, [261](#)
- CapsuleValidateResult
 - capsule.h, [251](#)
- CapsuleVMAllocFn
 - capsule_birth.h, [255](#)
- capture_exec_result
 - test_contracts.c, [605](#)
 - test_contracts.h, [564](#)
- category
 - ProfileScope, [125](#)
- category_timers
 - Profiler, [124](#)
- CELL
 - vm.h, [317](#)
- CELL_BITS

- format_words.c, [674](#)
- cell_t
 - vm.h, [316](#)
- CF_BEGIN
 - Control Flow Stack, [32](#)
- CF_CASE
 - Control Flow Stack, [32](#)
- CF_DO
 - Control Flow Stack, [32](#)
- CF_ELSE
 - Control Flow Stack, [32](#)
- cf_epoch_sync
 - control_words.c, [649](#)
- CF_IF
 - Control Flow Stack, [32](#)
- cf_item_t, [68](#)
- cf_last_mode
 - control_words.c, [650](#)
- CF_OF
 - Control Flow Stack, [32](#)
- cf_peek_item
 - control_words.c, [649](#)
- cf_pop_item
 - control_words.c, [649](#)
- cf_push_item
 - control_words.c, [649](#)
- cf_sp
 - control_words.c, [650](#)
- cf_stack
 - control_words.c, [650](#)
- CF_STACK_MAX
 - Control Flow Stack, [32](#)
- cf_tag_t
 - Control Flow Stack, [32](#)
- CF_WHILE
 - Control Flow Stack, [32](#)
- cgroup_cpu_usage_us
 - physics_host_snapshot, [116](#)
- cgroup_memory_current_bytes
 - physics_host_snapshot, [116](#)
- chain_length
 - blk_meta_t, [55](#)
- channel
 - physics_analytics_event_header, [110](#)
- CHAR16
 - uefi.h, [297](#)
- CHARGE_NEUTRAL
 - vm.h, [314](#)
- check_counter
 - HeartbeatState, [96](#)
- check_physics_transparent
 - test_contracts.c, [605](#)
 - test_contracts.h, [565](#)
- CheckEvent
 - _EFI_BOOT_SERVICES, [46](#)
- checksum
 - blk_meta_t, [55](#)
- choose_B
 - block_subsystem.c, [381](#)
- CHUNK_BITS
 - format_words.c, [674](#)
- CHUNK_MASK
 - format_words.c, [674](#)
- ci_lower_95
 - DoeMetrics, [76](#)
- ci_upper_95
 - DoeMetrics, [76](#)
- clamp_u32
 - physics_metadata.c, [424](#)
- cleanup_and_exit
 - main.c, [413](#)
- cleanup_blkio
 - main.c, [413](#)
- ClearScreen
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, [51](#)
- cli.c
 - cli_parse, [388](#)
 - cli_print_usage, [388](#)
 - parse_u32, [388](#)
- cli.h
 - cli_parse, [180](#)
 - cli_print_usage, [180](#)
 - HEARTBEAT_LOG_FULL, [180](#)
 - HEARTBEAT_LOG_OFF, [180](#)
 - HEARTBEAT_LOG_SUMMARY, [180](#)
 - HeartbeatLogMode, [180](#)
- cli_parse
 - cli.c, [388](#)
 - cli.h, [180](#)
- cli_print_usage
 - cli.c, [388](#)
 - cli.h, [180](#)
- CLIConfig, [69](#)
 - benchmark, [69](#)
 - benchmark_iterations, [69](#)
 - break_me, [69](#)
 - disk_img_path, [69](#)
 - doe_experiment, [69](#)
 - fail_fast, [69](#)
 - heartbeat_log_mode, [70](#)
 - log_level, [70](#)
 - log_level_explicitly_set, [70](#)
 - ram_disk_mb, [70](#)
 - run_tests, [70](#)
 - run_tests_flag_observed, [70](#)
 - script_mode, [70](#)
- Close
 - _EFI_FILE_PROTOCOL, [48](#)
- close
 - blkio_vtable_t, [62](#)
- CloseEvent
 - _EFI_BOOT_SERVICES, [46](#)
- coalesce_neighbors
 - kmallocc.c, [524](#)
- color_reset
 - log.c, [411](#)

- compare_by_calls
 - profiler.c, [449](#)
- compare_by_time
 - profiler.c, [449](#)
- compare_dict_entry_heat_desc
 - dictionary_heat_optimization.c, [389](#)
- compare_heat_ascending
 - dictionary_heat_optimization.c, [389](#)
- compile_buffer
 - VM, [143](#)
- COMPILE_BUFFER_SIZE
 - vm.h, [314](#)
- compile_operations
 - Profiler, [124](#)
- compile_pos
 - VM, [144](#)
- compile_size
 - VM, [144](#)
- compiling_word
 - VM, [144](#)
- compute_crc64
 - block_subsystem.c, [381](#)
- compute_fit_quality
 - inference_engine.c, [402](#)
- compute_levene_statistic
 - inference_engine.c, [402](#)
- compute_mean_q48
 - inference_engine.c, [402](#)
- compute_median_q48
 - inference_engine.c, [402](#)
- compute_totals_from_B
 - block_subsystem.c, [381](#)
- compute_variance_q48
 - inference_engine.c, [402](#)
 - inference_engine.h, [189](#)
- ConfigurationTable
 - _EFI_SYSTEM_TABLE, [52](#)
- ConIn
 - _EFI_SYSTEM_TABLE, [52](#)
- ConOut
 - _EFI_SYSTEM_TABLE, [52](#)
- console.c
 - console_getc, [512](#)
 - console_init, [512](#)
 - console_poll, [512](#)
 - console_println, [512](#)
 - console_putc, [512](#)
 - console_puts, [512](#)
 - inb, [513](#)
 - outb, [513](#)
 - SERIAL_SUPPORTED, [512](#)
- console.h
 - console_getc, [265](#)
 - console_init, [265](#)
 - console_poll, [265](#)
 - console_println, [265](#)
 - console_putc, [265](#)
 - console_puts, [265](#)
- console_getc
 - console.c, [512](#)
 - console.h, [265](#)
- console_init
 - console.c, [512](#)
 - console.h, [265](#)
- console_poll
 - console.c, [512](#)
 - console.h, [265](#)
- console_println
 - console.c, [512](#)
 - console.h, [265](#)
- console_putc
 - console.c, [512](#)
 - console.h, [265](#)
- console_puts
 - console.c, [512](#)
 - console.h, [265](#)
- ConsoleInHandle
 - _EFI_SYSTEM_TABLE, [52](#)
- ConsoleOutHandle
 - _EFI_SYSTEM_TABLE, [52](#)
- consume_seq
 - physics_analytics_header, [112](#)
- content_length
 - blk_meta_t, [55](#)
- content_type
 - blk_meta_t, [55](#)
- context_accuracy_percent
 - DoeMetrics, [76](#)
- context_correct
 - DoeMetrics, [76](#)
- context_predictions_total
 - DoeMetrics, [76](#)
- context_transitions
 - WordTransitionMetrics, [160](#)
- context_var_addr
 - vocabulary_words.c, [755](#)
- context_vocab
 - vocabulary_words.c, [755](#)
- context_window
 - WordTransitionMetrics, [160](#)
- context_window_pos
 - WordTransitionMetrics, [160](#)
- contract
 - TestCase, [135](#)
- CONTRACT_DEFAULT_FUZZ_ROUNDS
 - test_contracts.h, [564](#)
- contract_flags_t
 - test_contracts.h, [564](#)
- CONTRACT_HEARTBEAT_SAFE
 - test_contracts.h, [564](#)
- CONTRACT_NONE
 - test_contracts.h, [564](#)
- CONTRACT_PHYSICS_TRANSPARENT
 - test_contracts.h, [564](#)
- contract_rand
 - test_contracts.c, [606](#)

- Control Flow Stack, [31](#)
 - CF_BEGIN, [32](#)
 - CF_CASE, [32](#)
 - CF_DO, [32](#)
 - CF_ELSE, [32](#)
 - CF_IF, [32](#)
 - CF_OF, [32](#)
 - CF_STACK_MAX, [32](#)
 - cf_tag_t, [32](#)
 - CF_WHILE, [32](#)
- control_forth_0branch
 - Runtime Branch Helpers, [33](#)
- control_forth_again
 - Runtime Branch Helpers, [33](#)
- control_forth_begin
 - Runtime Branch Helpers, [34](#)
- control_forth_branch
 - Runtime Branch Helpers, [34](#)
- control_forth_case
 - Runtime Branch Helpers, [34](#)
- control_forth_do
 - Runtime Branch Helpers, [34](#)
- control_forth_else
 - Runtime Branch Helpers, [34](#)
- control_forth_endcase
 - Runtime Branch Helpers, [35](#)
- control_forth_endof
 - Runtime Branch Helpers, [35](#)
- control_forth_EXIT
 - Runtime Branch Helpers, [35](#)
- control_forth_I
 - Runtime Branch Helpers, [35](#)
- control_forth_if
 - Runtime Branch Helpers, [35](#)
- control_forth_J
 - Runtime Branch Helpers, [35](#)
- control_forth_leave
 - Runtime Branch Helpers, [36](#)
- control_forth_loop
 - Runtime Branch Helpers, [36](#)
- control_forth_of
 - Runtime Branch Helpers, [36](#)
- control_forth_plus_loop
 - Runtime Branch Helpers, [36](#)
- control_forth_qdo
 - Runtime Branch Helpers, [36](#)
- control_forth_repeat
 - Runtime Branch Helpers, [36](#)
- control_forth_runtime_do
 - Runtime Branch Helpers, [37](#)
- control_forth_runtime_leave
 - Runtime Branch Helpers, [37](#)
- control_forth_runtime_loop
 - Runtime Branch Helpers, [37](#)
- control_forth_runtime_plus_loop
 - Runtime Branch Helpers, [37](#)
- control_forth_runtime_qdo
 - Runtime Branch Helpers, [37](#)
- control_forth_then
 - Runtime Branch Helpers, [37](#)
- control_forth_UNLOOP
 - Runtime Branch Helpers, [38](#)
- control_forth_until
 - Runtime Branch Helpers, [38](#)
- control_forth_while
 - Runtime Branch Helpers, [38](#)
- control_word_suites
 - control_words_test.c, [582](#)
- control_words.c
 - cf_epoch_sync, [649](#)
 - cf_last_mode, [650](#)
 - cf_peek_item, [649](#)
 - cf_pop_item, [649](#)
 - cf_push_item, [649](#)
 - cf_sp, [650](#)
 - cf_stack, [650](#)
 - emit_cell, [650](#)
 - endof_addrs, [650](#)
 - endof_mark_sp, [650](#)
 - endof_mark_stack, [650](#)
 - endof_sp, [650](#)
 - leave_addrs, [650](#)
 - leave_mark_sp, [651](#)
 - leave_mark_stack, [651](#)
 - leave_sp, [651](#)
- control_words_test.c
 - control_word_suites, [582](#)
 - run_control_words_tests, [582](#)
- converge_tsc_bare_metal
 - timer.c, [482](#)
- converged
 - timer_calibration_record, [140](#)
- conversion_pos
 - format_words.c, [678](#)
- convert_string_to_number
 - string_words.c, [740](#)
- ConvertPointer
 - _EFI_RUNTIME_SERVICES, [49](#)
- copy_block_to_ram
 - capsule_loader.c, [504](#)
- copy_device_to_ram
 - capsule_loader.c, [504](#)
- count
 - time_window, [139](#)
- counters
 - Profiler, [124](#)
- cpu_count
 - physics_host_snapshot, [117](#)
- cpu_freq_delta_mhz_q48
 - DoeMetrics, [76](#)
- cpu_has_invariant_tsc
 - timer.c, [483](#)
- cpu_has_rdtscp
 - timer.c, [483](#)
- cpu_idle_jiffies
 - physics_host_snapshot, [117](#)

- cpu_temp_delta_c_q48
 - DoeMetrics, 76
- cpu_total_jiffies
 - physics_host_snapshot, 117
- cpuid
 - timer.c, 483
- CRC32
 - EFI_TABLE_HEADER, 84
- crc64_init
 - block_subsystem.c, 381
- crc64_inited
 - block_subsystem.c, 384
- crc64_table
 - block_subsystem.c, 384
- created_time
 - blk_meta_t, 55
 - blk_volume_meta_t, 57
- CreateEvent
 - _EFI_BOOT_SERVICES, 46
- CreateTime
 - EFI_FILE_INFO, 80
- credible_lower_95
 - BayesianLatencyPosterior, 53
- credible_lower_95_q48
 - SpeedupEstimate, 133
- credible_lower_99
 - BayesianLatencyPosterior, 53
- credible_upper_95
 - BayesianLatencyPosterior, 54
- credible_upper_95_q48
 - SpeedupEstimate, 133
- credible_upper_99
 - BayesianLatencyPosterior, 54
- ctx_cached_head
 - vocabulary_words.c, 755
- ctx_fc
 - vocabulary_words.c, 755
- ctx_n
 - vocabulary_words.c, 756
- current_base
 - format_words.c, 674
- current_executing_entry
 - VM, 144
- current_level
 - log.c, 411
- current_mode
 - ssm_l8_state_t, 135
- current_scr
 - editor_words.c, 671
- current_start_time
 - profiler.c, 454
- current_var_addr
 - vocabulary_words.c, 756
- current_vocab
 - vocabulary_words.c, 756
- current_word
 - profiler.c, 454
- current_word_name
 - VM, 144
- cv
 - ssm_l8_metrics_t, 134
- CV_EPSILON_PPM
 - timer.c, 480
- cv_hpet_ppm
 - timer_calibration_record, 140
- cv_pit_ppm
 - timer_calibration_record, 140
- CV_STABLE_COUNT
 - timer.c, 480
- d_compare
 - double_words.c, 666
- data
 - BlockBuffer, 64
 - cache_slot_t, 65
- Data1
 - EFI_GUID, 81
- Data2
 - EFI_GUID, 81
- Data3
 - EFI_GUID, 81
- Data4
 - EFI_GUID, 81
- data_stack
 - VM, 144
- Day
 - EFI_TIME, 84
- Daylight
 - EFI_TIME, 84
- dbg_get_base
 - vm_debug.c, 623
- DECAY_MIN_INTERVAL
 - starforth_config.h, 242
- decay_min_interval_ns
 - DoeMetrics, 76
- DECAY_RATE_PER_US_Q16
 - starforth_config.h, 242
- decay_rate_q16
 - DoeMetrics, 76
- decay_slope
 - DoeMetrics, 76
- decay_slope_direction
 - VM, 144
- decay_slope_q48
 - HeartbeatSnapshot, 94
 - PhysicsSnapshot, 121
 - VM, 144
- decode_pf_error
 - interrupts.c, 476
- defining_runtime_constant
 - defining_words.c, 652
- defining_runtime_create
 - defining_words.c, 652
- defining_runtime_dodoes
 - defining_words.c, 652
- defining_runtime_does_rt
 - defining_words.c, 653

- defining_runtime_lit
 - defining_words.c, [653](#)
- defining_runtime_variable
 - defining_words.c, [653](#)
- defining_word_bracket_compile
 - defining_words.c, [653](#)
- defining_word_colon
 - defining_words.c, [654](#)
- defining_word_compile
 - defining_words.c, [654](#)
- defining_word_constant
 - defining_words.c, [654](#)
- defining_word_create
 - defining_words.c, [654](#)
- defining_word_does
 - defining_words.c, [654](#)
- defining_word_immediate
 - defining_words.c, [654](#)
- defining_word_left_bracket
 - defining_words.c, [654](#)
- defining_word_literal
 - defining_words.c, [655](#)
- defining_word_right_bracket
 - defining_words.c, [655](#)
- defining_word_semicolon
 - defining_words.c, [655](#)
- defining_word_state
 - defining_words.c, [655](#)
- defining_word_suites
 - defining_words_tests.c, [583](#)
- defining_word_variable
 - defining_words.c, [655](#)
- defining_words.c
 - defining_runtime_constant, [652](#)
 - defining_runtime_create, [652](#)
 - defining_runtime_dodoes, [652](#)
 - defining_runtime_does_rt, [653](#)
 - defining_runtime_lit, [653](#)
 - defining_runtime_variable, [653](#)
 - defining_word_bracket_compile, [653](#)
 - defining_word_colon, [654](#)
 - defining_word_compile, [654](#)
 - defining_word_constant, [654](#)
 - defining_word_create, [654](#)
 - defining_word_does, [654](#)
 - defining_word_immediate, [654](#)
 - defining_word_left_bracket, [654](#)
 - defining_word_literal, [655](#)
 - defining_word_right_bracket, [655](#)
 - defining_word_semicolon, [655](#)
 - defining_word_state, [655](#)
 - defining_word_variable, [655](#)
 - dictionary_word_forget, [655](#)
 - register_defining_words, [655](#)
- defining_words.h
 - register_defining_words, [686](#)
- defining_words_tests.c
 - defining_word_suites, [583](#)
 - run_defining_words_tests, [583](#)
- Delete
 - _EFI_FILE_PROTOCOL, [48](#)
- deltas
 - time_window, [139](#)
- derive_state_flags
 - physics_metadata.c, [424](#)
- derive_tsc_hz_from_cpuid
 - timer.c, [483](#)
- desc_capacity
 - CapsuleDirHeader, [66](#)
- desc_count
 - CapsuleDirHeader, [66](#)
- devblock
 - cache_slot_t, [65](#)
- devblock4k_to_lba1k
 - block_subsystem.c, [381](#)
- devblock_base
 - blk_volume_meta_t, [57](#)
- devblock_base_4k
 - block_subsystem.c, [384](#)
- DeviceHandle
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- dict_adaptive_optimization_pass
 - dictionary_heat_optimization.c, [389](#)
 - dictionary_heat_optimization.h, [181](#)
- dict_fence_here
 - VM, [144](#)
- dict_fence_latest
 - VM, [145](#)
- dict_find_word_heat_aware
 - dictionary_heat_optimization.c, [389](#)
 - dictionary_heat_optimization.h, [181](#)
- dict_lock
 - VM, [145](#)
- dict_lookups
 - Profiler, [124](#)
- dict_manip_word_suites
 - dictionary_manipulation_words_test.c, [584](#)
- dict_reorganize_buckets_by_heat
 - dictionary_heat_optimization.c, [390](#)
 - dictionary_heat_optimization.h, [182](#)
- dict_update_heat_percentiles
 - dictionary_heat_optimization.c, [390](#)
 - dictionary_heat_optimization.h, [182](#)
- DictEntry, [70](#)
 - acl_allow, [71](#)
 - acl_default, [71](#)
 - acl_mode, [71](#)
 - acl_pinned, [71](#)
 - acl_ttl, [71](#)
 - execution_heat, [71](#)
 - flags, [71](#)
 - func, [71](#)
 - link, [71](#)
 - name, [72](#)
 - name_len, [72](#)
 - physics, [72](#)

- profiler.c, 448
- profiler.h, 226
- transition_metrics, 72
- word_id, 72
- DICTIONARY_BLOCKS
 - vm.h, 314
- dictionary_clear
 - dictionary_management.h, 183
- dictionary_heat_diagnostic_words.c
 - forth_COMPARE_LOOKUPS, 656
 - forth_HEAT_PERCENTILES, 656
 - forth_LOOKUP_STRATEGY_FETCH, 657
 - forth_LOOKUP_STRATEGY_STORE, 657
 - forth_REORG_BUCKETS, 657
 - forth_SHOW_HEAT_OPTIMIZATION, 657
 - register_dictionary_heat_diagnostic_words, 657
- dictionary_heat_diagnostic_words.h
 - register_dictionary_heat_diagnostic_words, 687
- dictionary_heat_optimization.c
 - compare_dict_entry_heat_desc, 389
 - compare_heat_ascending, 389
 - dict_adaptive_optimization_pass, 389
 - dict_find_word_heat_aware, 389
 - dict_reorganize_buckets_by_heat, 390
 - dict_update_heat_percentiles, 390
- dictionary_heat_optimization.h
 - dict_adaptive_optimization_pass, 181
 - dict_find_word_heat_aware, 181
 - dict_reorganize_buckets_by_heat, 182
 - dict_update_heat_percentiles, 182
 - SF_FC_BUCKETS, 181
 - sf_fc_cap, 182
 - sf_fc_count, 182
 - sf_fc_list, 182
- dictionary_init
 - dictionary_management.h, 183
- dictionary_lookup
 - dictionary_management.h, 183
- dictionary_lookups
 - DoeMetrics, 77
 - HeartbeatState, 96
- dictionary_m_word_cfa
 - dictionary_manipulation_words.c, 658
- dictionary_m_word_find
 - dictionary_manipulation_words.c, 658
- dictionary_m_word_hidden
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_interpret
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_left_bracket
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_lfa
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_link_from
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_name_to
 - dictionary_manipulation_words.c, 659
- dictionary_m_word_nfa
 - dictionary_manipulation_words.c, 660
- dictionary_m_word_pfa
 - dictionary_manipulation_words.c, 660
- dictionary_m_word_right_bracket
 - dictionary_manipulation_words.c, 660
- dictionary_m_word_smudge
 - dictionary_manipulation_words.c, 660
- dictionary_m_word_state
 - dictionary_manipulation_words.c, 660
- dictionary_m_word_tick
 - dictionary_manipulation_words.c, 661
- dictionary_m_word_to_body
 - dictionary_manipulation_words.c, 661
- dictionary_m_word_to_link
 - dictionary_manipulation_words.c, 661
- dictionary_m_word_to_name
 - dictionary_manipulation_words.c, 661
- dictionary_m_word_traverse
 - dictionary_manipulation_words.c, 661
- dictionary_management.c
 - sf_cached_latest, 396
 - SF_FC_BUCKETS, 391
 - sf_fc_cap, 396
 - sf_fc_clear_all, 391
 - sf_fc_count, 396
 - sf_fc_list, 396
 - sf_fc_reserve, 391
 - sf_rebuild_fc_index, 391
 - sf_try_fast_append, 392
 - sf_xrealloc, 392
 - vm_create_word, 392
 - vm_dictionary_acquire_word_id, 392
 - vm_dictionary_find_by_func, 393
 - vm_dictionary_find_latest_by_func, 393
 - vm_dictionary_get_data_field, 393
 - vm_dictionary_index_reset, 393
 - vm_dictionary_lookup_by_word_id, 394
 - vm_dictionary_track_entry, 394
 - vm_dictionary_untrack_entry, 394
 - vm_find_word, 394
 - vm_hide_word, 395
 - vm_pin_execution_heat, 395
 - vm_smudge_word, 395
 - vm_unpin_execution_heat, 395
- dictionary_management.h
 - dictionary_clear, 183
 - dictionary_init, 183
 - dictionary_lookup, 183
 - dictionary_register, 184
- dictionary_manipulation_words.c
 - align_address, 658
 - dictionary_m_word_cfa, 658
 - dictionary_m_word_find, 658
 - dictionary_m_word_hidden, 659
 - dictionary_m_word_interpret, 659
 - dictionary_m_word_left_bracket, 659
 - dictionary_m_word_lfa, 659
 - dictionary_m_word_link_from, 659

- dictionary_m_word_name_to, [659](#)
- dictionary_m_word_nfa, [660](#)
- dictionary_m_word_pfa, [660](#)
- dictionary_m_word_right_bracket, [660](#)
- dictionary_m_word_smudge, [660](#)
- dictionary_m_word_state, [660](#)
- dictionary_m_word_tick, [661](#)
- dictionary_m_word_to_body, [661](#)
- dictionary_m_word_to_link, [661](#)
- dictionary_m_word_to_name, [661](#)
- dictionary_m_word_traverse, [661](#)
- find_entry_from_name, [661](#)
- get_body_address, [661](#)
- get_name_field, [662](#)
- register_dictionary_manipulation_words, [662](#)
- state_variable, [662](#)
- traverse_name_field, [662](#)
- dictionary_manipulation_words.h
 - register_dictionary_manipulation_words, [687](#)
- dictionary_manipulation_words_test.c
 - dict_manip_word_suites, [584](#)
 - run_dictionary_manipulation_words_tests, [583](#)
- DICTIONARY_MEMORY_SIZE
 - vm.h, [314](#)
- dictionary_register
 - dictionary_management.h, [184](#)
- DICTIONARY_SIZE
 - vm.h, [314](#)
- dictionary_word_2comma
 - dictionary_words.c, [663](#)
- dictionary_word_allot
 - dictionary_words.c, [663](#)
- dictionary_word_c_comma
 - dictionary_words.c, [664](#)
- dictionary_word_comma
 - dictionary_words.c, [664](#)
- dictionary_word_forget
 - defining_words.c, [655](#)
- dictionary_word_here
 - dictionary_words.c, [664](#)
- dictionary_word_latest
 - dictionary_words.c, [664](#)
- dictionary_word_pad
 - dictionary_words.c, [665](#)
- dictionary_word_sp_fetch
 - dictionary_words.c, [665](#)
- dictionary_word_sp_store
 - dictionary_words.c, [665](#)
- dictionary_word_suites
 - dictionary_words_test.c, [585](#)
- dictionary_words.c
 - dictionary_word_2comma, [663](#)
 - dictionary_word_allot, [663](#)
 - dictionary_word_c_comma, [664](#)
 - dictionary_word_comma, [664](#)
 - dictionary_word_here, [664](#)
 - dictionary_word_latest, [664](#)
 - dictionary_word_pad, [665](#)
 - dictionary_word_sp_fetch, [665](#)
 - dictionary_word_sp_store, [665](#)
 - register_dictionary_words, [665](#)
- dictionary_words.h
 - register_dictionary_words, [688](#)
- dictionary_words_test.c
 - dictionary_word_suites, [585](#)
 - run_dictionary_words_tests, [584](#)
- DictPhysics, [72](#)
 - acl_hint, [73](#)
 - avg_latency_ns, [73](#)
 - last_active_ns, [73](#)
 - last_decay_ns, [73](#)
 - mass_bytes, [73](#)
 - pubsub_mask, [73](#)
 - state_flags, [73](#)
 - temperature_q8, [73](#)
- DIE
 - test_common.c, [599](#)
- diff_ppm
 - timer_calibration_record, [140](#)
- digit_for
 - format_words.c, [674](#)
- dir_hash
 - CapsuleDirHeader, [66](#)
- dirty
 - BlockBuffer, [64](#)
 - cache_slot_t, [65](#)
- dirty_ram
 - block_subsystem.c, [384](#)
- DISK_CACHE_SLOTS
 - block_subsystem.c, [376](#)
- disk_dev
 - block_subsystem.c, [384](#)
- disk_img_path
 - CLConfig, [69](#)
- dispatch_init_forth
 - capsule_birth.c, [498](#)
- div_ud_by_base
 - format_words.c, [674](#)
- doe_experiment
 - CLConfig, [69](#)
- doe_metrics.c
 - bucket_hits, [400](#)
 - cache_hits, [400](#)
 - extract_cache_metrics, [397](#)
 - isqrt, [397](#)
 - metrics_from_vm, [397](#)
 - metrics_get_cpu_freq_mhz, [398](#)
 - metrics_get_cpu_temp_c, [398](#)
 - metrics_get_timestamp, [398](#)
 - metrics_print_text, [398](#)
 - metrics_write_csv_header, [399](#)
 - metrics_write_csv_row, [399](#)
 - metrics_write_james_law_csv_row, [399](#)
 - physics_global_stats, [400](#)
 - total_lookups, [400](#)
- doe_metrics.h

- metrics_from_vm, 185
- metrics_get_cpu_freq_mhz, 185
- metrics_get_cpu_temp_c, 186
- metrics_get_timestamp, 186
- metrics_print_text, 186
- metrics_write_csv_header, 187
- metrics_write_csv_row, 187
- metrics_write_james_law_csv_row, 187
- doe_row_printing
 - VM, 145
- DoeMetrics, 73
 - actual_window_size, 75
 - adaptive_shrink_rate, 75
 - avg_word_heat, 75
 - bucket_hit_percent, 75
 - bucket_hits, 75
 - bucket_search_latency_ns, 75
 - bucket_search_stddev_ns, 75
 - cache_demotions, 75
 - cache_hit_latency_ns, 75
 - cache_hit_percent, 75
 - cache_hit_stddev_ns, 75
 - cache_hits, 76
 - cache_promotions, 76
 - ci_lower_95, 76
 - ci_upper_95, 76
 - context_accuracy_percent, 76
 - context_correct, 76
 - context_predictions_total, 76
 - cpu_freq_delta_mhz_q48, 76
 - cpu_temp_delta_c_q48, 76
 - decay_min_interval_ns, 76
 - decay_rate_q16, 76
 - decay_slope, 76
 - dictionary_lookups, 77
 - early_exit_count, 77
 - enable_hotwords_cache, 77
 - enable_loop_1_heat_tracking, 77
 - enable_loop_2_rolling_window, 77
 - enable_loop_3_linear_decay, 77
 - enable_loop_4_pipelining, 77
 - enable_loop_5_window_inference, 77
 - enable_loop_6_decay_inference, 77
 - enable_loop_7_adaptive_hearttrate, 77
 - enable_pipelining, 77
 - final_effective_window_size, 77
 - heat_cache_demotion_threshold, 78
 - hot_word_count, 78
 - inference_run_count, 78
 - memory_allocated_bytes, 78
 - prefetch_accuracy_percent, 78
 - prefetch_attempts, 78
 - prefetch_hits, 78
 - rolling_window_size, 78
 - rolling_window_width, 78
 - speedup_vs_baseline, 78
 - stale_word_count, 78
 - stale_word_ratio, 78
 - tick_count, 79
 - tick_target_ns, 79
 - total_executions, 79
 - total_heat, 79
 - total_lookups, 79
 - total_runtime_ms, 79
 - vm_workload_duration_ns_q48, 79
 - window_diversity_percent, 79
 - window_final_size_bytes, 79
 - window_tuning_checks, 79
 - window_variance_q48, 79
 - words_executed, 79
- double_word_2drop
 - double_words.c, 667
- double_word_2dup
 - double_words.c, 667
- double_word_2over
 - double_words.c, 667
- double_word_2r_fetch
 - double_words.c, 667
- double_word_2r_from
 - double_words.c, 667
- double_word_2rot
 - double_words.c, 667
- double_word_2swap
 - double_words.c, 667
- double_word_2to_r
 - double_words.c, 668
- double_word_d_equals
 - double_words.c, 668
- double_word_d_less
 - double_words.c, 668
- double_word_d_minus
 - double_words.c, 668
- double_word_d_plus
 - double_words.c, 668
- double_word_d_two_slash
 - double_words.c, 668
- double_word_d_two_star
 - double_words.c, 668
- double_word_d_zero_equals
 - double_words.c, 669
- double_word_d_zero_less
 - double_words.c, 669
- double_word_dabs
 - double_words.c, 669
- double_word_dmax
 - double_words.c, 669
- double_word_dmin
 - double_words.c, 669
- double_word_dnegate
 - double_words.c, 669
- double_word_s_to_d
 - double_words.c, 669
- double_word_suites
 - double_words_test.c, 585
- double_words.c
 - d_compare, 666

- double_word_2drop, [667](#)
- double_word_2dup, [667](#)
- double_word_2over, [667](#)
- double_word_2r_fetch, [667](#)
- double_word_2r_from, [667](#)
- double_word_2rot, [667](#)
- double_word_2swap, [667](#)
- double_word_2to_r, [668](#)
- double_word_d_equals, [668](#)
- double_word_d_less, [668](#)
- double_word_d_minus, [668](#)
- double_word_d_plus, [668](#)
- double_word_d_two_slash, [668](#)
- double_word_d_two_star, [668](#)
- double_word_d_zero_equals, [669](#)
- double_word_d_zero_less, [669](#)
- double_word_dabs, [669](#)
- double_word_dmax, [669](#)
- double_word_dmin, [669](#)
- double_word_dnegate, [669](#)
- double_word_s_to_d, [669](#)
- register_double_words, [670](#)
- double_words.h
 - register_double_words, [688](#)
- double_words_test.c
 - double_word_suites, [585](#)
 - run_double_words_tests, [585](#)
- DRIFT_EPSILON_PPM
 - timer.c, [480](#)
- dropped_events
 - physics_analytics_header, [112](#)
- dsp
 - ExecResult, [90](#)
 - VM, [145](#)
- dump_and_die
 - test_common.c, [600](#)
- dump_block_preview
 - vm_debug.c, [623](#)
- e_ehsize
 - Elf64_Ehdr, [86](#)
- e_entry
 - Elf64_Ehdr, [86](#)
- e_flags
 - Elf64_Ehdr, [86](#)
- e_ident
 - Elf64_Ehdr, [86](#)
- e_machine
 - Elf64_Ehdr, [86](#)
- e_phentsize
 - Elf64_Ehdr, [86](#)
- e_phnum
 - Elf64_Ehdr, [86](#)
- e_phoff
 - Elf64_Ehdr, [86](#)
- e_shentsize
 - Elf64_Ehdr, [86](#)
- e_shnum
 - Elf64_Ehdr, [86](#)
- e_shoff
 - Elf64_Ehdr, [86](#)
- e_shstrndx
 - Elf64_Ehdr, [86](#)
- e_type
 - Elf64_Ehdr, [86](#)
- e_version
 - Elf64_Ehdr, [86](#)
- early_exit_count
 - DoeMetrics, [77](#)
 - HeartbeatState, [96](#)
- early_exited
 - InferenceOutputs, [106](#)
- ecw_nesting
 - VM, [146](#)
- editor_word_edit
 - editor_words.c, [671](#)
- editor_word_l
 - editor_words.c, [671](#)
- editor_word_s
 - editor_words.c, [671](#)
- editor_word_show
 - editor_words.c, [671](#)
- editor_words.c
 - current_scr, [671](#)
 - editor_word_edit, [671](#)
 - editor_word_l, [671](#)
 - editor_word_s, [671](#)
 - editor_word_show, [671](#)
 - line_ptr, [672](#)
 - print_line_64, [672](#)
 - register_editor_words, [672](#)
 - set_scr, [672](#)
- editor_words.h
 - register_editor_words, [689](#)
- eff_window_size
 - PhysicsSnapshot, [121](#)
- effective_window_size
 - rolling_window_view_t, [127](#)
 - RollingWindowOfTruth, [128](#)
- EFI_ABORTED
 - uefi.h, [295](#)
- EFI_ACPI_20_TABLE_GUID
 - uefi.h, [300](#)
- EFI_ACPI_TABLE_GUID
 - uefi.h, [300](#)
- EFI_ALLOCATE_PAGES
 - uefi.h, [297](#)
- EFI_ALLOCATE_POOL
 - uefi.h, [297](#)
- EFI_ALLOCATE_TYPE
 - uefi.h, [299](#)
- EFI_BAD_BUFFER_SIZE
 - uefi.h, [295](#)
- EFI_BOOT_SERVICES
 - uefi.h, [297](#)
- EFI_BUFFER_TOO_SMALL
 - uefi.h, [295](#)

- EFI_CONFIGURATION_TABLE, 80
 - VendorGuid, 80
 - VendorTable, 80
- EFI_DEVICE_ERROR
 - uefi.h, 295
- EFI_EXIT_BOOT_SERVICES
 - uefi.h, 297
- EFI_FILE_CLOSE
 - uefi.h, 297
- EFI_FILE_GET_INFO
 - uefi.h, 297
- EFI_FILE_INFO, 80
 - Attribute, 80
 - CreateTime, 80
 - FileName, 80
 - FileSize, 81
 - LastAccessTime, 81
 - ModificationTime, 81
 - PhysicalSize, 81
 - Size, 81
- EFI_FILE_INFO_GUID
 - uefi.h, 301
- EFI_FILE_MODE_READ
 - uefi.h, 295
- EFI_FILE_OPEN
 - uefi.h, 297
- EFI_FILE_PROTOCOL
 - uefi.h, 297
- EFI_FILE_READ
 - uefi.h, 297
- EFI_FREE_PAGES
 - uefi.h, 297
- EFI_FREE_POOL
 - uefi.h, 297
- EFI_GET_MEMORY_MAP
 - uefi.h, 297
- EFI_GUID, 81
 - Data1, 81
 - Data2, 81
 - Data3, 81
 - Data4, 81
- EFI_HANDLE
 - uefi.h, 298
- EFI_HANDLE_PROTOCOL
 - uefi.h, 298
- EFI_INVALID_PARAMETER
 - uefi.h, 295
- EFI_LOAD_ERROR
 - uefi.h, 295
- EFI_LOADED_IMAGE_PROTOCOL, 82
 - DeviceHandle, 82
 - FilePath, 82
 - ImageBase, 82
 - ImageCodeType, 82
 - ImageDataType, 82
 - ImageSize, 82
 - LoadOptions, 82
 - LoadOptionsSize, 82
 - Reserved, 82
 - Unload, 83
- EFI_LOADED_IMAGE_PROTOCOL_GUID
 - uefi.h, 301
- EFI_LOCATE_HANDLE
 - uefi.h, 298
- EFI_LOCATE_SEARCH_TYPE
 - uefi.h, 300
- efi_main
 - uefi_loader.c, 493
- EFI_MEMORY_DESCRIPTOR, 83
 - Attribute, 83
 - NumberOfPages, 83
 - PhysicalStart, 83
 - Type, 83
 - VirtualStart, 83
- EFI_MEMORY_RP
 - uefi.h, 295
- EFI_MEMORY_RUNTIME
 - uefi.h, 295
- EFI_MEMORY_TYPE
 - uefi.h, 300
- EFI_MEMORY_UC
 - uefi.h, 295
- EFI_MEMORY_UCE
 - uefi.h, 295
- EFI_MEMORY_WB
 - uefi.h, 295
- EFI_MEMORY_WC
 - uefi.h, 295
- EFI_MEMORY_WP
 - uefi.h, 295
- EFI_MEMORY_WT
 - uefi.h, 296
- EFI_MEMORY_XP
 - uefi.h, 296
- EFI_NOT_FOUND
 - uefi.h, 296
- EFI_NOT_READY
 - uefi.h, 296
- EFI_OUT_OF_RESOURCES
 - uefi.h, 296
- EFI_PHYSICAL_ADDRESS
 - uefi.h, 298
- EFI_RAISE_TPL
 - uefi.h, 298
- EFI_RESTORE_TPL
 - uefi.h, 298
- EFI_RUNTIME_SERVICES
 - uefi.h, 298
- EFI_SIMPLE_FILE_SYSTEM_PROTOCOL
 - uefi.h, 298
- EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID
 - uefi.h, 301
- EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME
 - uefi.h, 298
- EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
 - uefi.h, 298

- EFI_STATUS
 - uefi.h, [298](#)
- EFI_SUCCESS
 - uefi.h, [296](#)
- EFI_SYSTEM_TABLE
 - uefi.h, [298](#)
- EFI_TABLE_HEADER, [83](#)
 - CRC32, [84](#)
 - HeaderSize, [84](#)
 - Reserved, [84](#)
 - Revision, [84](#)
 - Signature, [84](#)
- EFI_TEXT_RESET
 - uefi.h, [298](#)
- EFI_TEXT_STRING
 - uefi.h, [298](#)
- EFI_TIME, [84](#)
 - Day, [84](#)
 - Daylight, [84](#)
 - Hour, [84](#)
 - Minute, [85](#)
 - Month, [85](#)
 - Nanosecond, [85](#)
 - Pad1, [85](#)
 - Pad2, [85](#)
 - Second, [85](#)
 - TimeZone, [85](#)
 - Year, [85](#)
- EFI_TPL
 - uefi.h, [299](#)
- EFI_UNSUPPORTED
 - uefi.h, [296](#)
- EFI_WRITE_PROTECTED
 - uefi.h, [296](#)
- EfiACPIMemoryNVS
 - uefi.h, [300](#)
- EfiACPIReclaimMemory
 - uefi.h, [300](#)
- EFIAPI
 - uefi.h, [296](#)
- EfiBootServicesCode
 - uefi.h, [300](#)
- EfiBootServicesData
 - uefi.h, [300](#)
- EfiConventionalMemory
 - uefi.h, [300](#)
- EfiLoaderCode
 - uefi.h, [300](#)
- EfiLoaderData
 - uefi.h, [300](#)
- EfiMaxMemoryType
 - uefi.h, [300](#)
- EfiMemoryMappedIO
 - uefi.h, [300](#)
- EfiMemoryMappedIOPortSpace
 - uefi.h, [300](#)
- EfiPalCode
 - uefi.h, [300](#)
- EfiPersistentMemory
 - uefi.h, [300](#)
- EfiReservedMemoryType
 - uefi.h, [300](#)
- EfiRuntimeServicesCode
 - uefi.h, [300](#)
- EfiRuntimeServicesData
 - uefi.h, [300](#)
- EfiUnusableMemory
 - uefi.h, [300](#)
- EI_ABIVERSION
 - elf64.h, [267](#)
- EI_CLASS
 - elf64.h, [267](#)
- EI_DATA
 - elf64.h, [267](#)
- EI_MAG0
 - elf64.h, [268](#)
- EI_MAG1
 - elf64.h, [268](#)
- EI_MAG2
 - elf64.h, [268](#)
- EI_MAG3
 - elf64.h, [268](#)
- EI_NIDENT
 - elf64.h, [268](#)
- EI_OSABI
 - elf64.h, [268](#)
- EI_PAD
 - elf64.h, [268](#)
- EI_VERSION
 - elf64.h, [268](#)
- elapsed_ns
 - HeartbeatTickSnapshot, [98](#)
- elf64.h
 - EI_ABIVERSION, [267](#)
 - EI_CLASS, [267](#)
 - EI_DATA, [267](#)
 - EI_MAG0, [268](#)
 - EI_MAG1, [268](#)
 - EI_MAG2, [268](#)
 - EI_MAG3, [268](#)
 - EI_NIDENT, [268](#)
 - EI_OSABI, [268](#)
 - EI_PAD, [268](#)
 - EI_VERSION, [268](#)
 - Elf64_Addr, [272](#)
 - Elf64_Half, [272](#)
 - Elf64_Off, [272](#)
 - ELF64_R_INFO, [268](#)
 - ELF64_R_SYM, [268](#)
 - ELF64_R_TYPE, [268](#)
 - Elf64_Sword, [272](#)
 - Elf64_Sxword, [272](#)
 - Elf64_Word, [272](#)
 - Elf64_Xword, [272](#)
 - ELFCLASS32, [268](#)
 - ELFCLASS64, [268](#)

- ELFDATA2LSB, 268
- ELFDATA2MSB, 269
- ELFMAG0, 269
- ELFMAG1, 269
- ELFMAG2, 269
- ELFMAG3, 269
- ELFOSABI_NONE, 269
- EM_AARCH64, 269
- EM_RISCV, 269
- EM_X86_64, 269
- ET_CORE, 269
- ET_DYN, 269
- ET_EXEC, 269
- ET_NONE, 269
- ET_REL, 269
- EV_CURRENT, 269
- PF_R, 270
- PF_W, 270
- PF_X, 270
- PT_DYNAMIC, 270
- PT_INTERP, 270
- PT_LOAD, 270
- PT_NOTE, 270
- PT_NULL, 270
- PT_PHDR, 270
- PT_SHLIB, 270
- PT_TLS, 270
- R_AARCH64_ABS64, 270
- R_AARCH64_NONE, 270
- R_AARCH64_RELATIVE, 270
- R_RISCV_64, 270
- R_RISCV_NONE, 271
- R_RISCV_RELATIVE, 271
- R_X86_64_32, 271
- R_X86_64_32S, 271
- R_X86_64_64, 271
- R_X86_64_NONE, 271
- R_X86_64_PC32, 271
- R_X86_64_RELATIVE, 271
- SHT_DYNAMIC, 271
- SHT_DYNSYM, 271
- SHT_HASH, 271
- SHT_NOBITS, 271
- SHT_NOTE, 271
- SHT_NULL, 271
- SHT_PROGBITS, 271
- SHT_REL, 272
- SHT_RELA, 272
- SHT_SHLIB, 272
- SHT_STRTAB, 272
- SHT_SYMTAB, 272
- Elf64_Addr
 - elf64.h, 272
- Elf64_Ehdr, 85
 - e_ehsize, 86
 - e_entry, 86
 - e_flags, 86
 - e_ident, 86
 - e_machine, 86
 - e_phentsize, 86
 - e_phnum, 86
 - e_phoff, 86
 - e_shentsize, 86
 - e_shnum, 86
 - e_shoff, 86
 - e_shstrndx, 86
 - e_type, 86
 - e_version, 86
- Elf64_Half
 - elf64.h, 272
- Elf64_Off
 - elf64.h, 272
- Elf64_Phdr, 87
 - p_align, 87
 - p_filesz, 87
 - p_flags, 87
 - p_memsz, 87
 - p_offset, 87
 - p_paddr, 87
 - p_type, 87
 - p_vaddr, 87
- ELF64_R_INFO
 - elf64.h, 268
- ELF64_R_SYM
 - elf64.h, 268
- ELF64_R_TYPE
 - elf64.h, 268
- Elf64_Rel, 88
 - r_info, 88
 - r_offset, 88
- Elf64_Rela, 88
 - r_addend, 88
 - r_info, 88
 - r_offset, 88
- Elf64_Shdr, 88
 - sh_addr, 89
 - sh_addralign, 89
 - sh_entsize, 89
 - sh_flags, 89
 - sh_info, 89
 - sh_link, 89
 - sh_name, 89
 - sh_offset, 89
 - sh_size, 89
 - sh_type, 89
- Elf64_Sword
 - elf64.h, 272
- Elf64_Sxword
 - elf64.h, 272
- Elf64_Sym, 90
 - st_info, 90
 - st_name, 90
 - st_other, 90
 - st_shndx, 90
 - st_size, 90
 - st_value, 90

- Elf64_Word
 - elf64.h, [272](#)
- Elf64_Xword
 - elf64.h, [272](#)
- elf_apply_relocations
 - elf_loader.c, [490](#)
- elf_load_kernel
 - elf_loader.c, [490](#)
 - elf_loader.h, [273](#)
- elf_load_segments
 - elf_loader.c, [491](#)
- elf_loader.c
 - elf_apply_relocations, [490](#)
 - elf_load_kernel, [490](#)
 - elf_load_segments, [491](#)
 - elf_memcpy, [491](#)
 - elf_memset, [491](#)
 - elf_validate_header, [491](#)
- elf_loader.h
 - elf_load_kernel, [273](#)
- elf_memcpy
 - elf_loader.c, [491](#)
- elf_memset
 - elf_loader.c, [491](#)
- elf_validate_header
 - elf_loader.c, [491](#)
- ELFCLASS32
 - elf64.h, [268](#)
- ELFCLASS64
 - elf64.h, [268](#)
- ELFDATA2LSB
 - elf64.h, [268](#)
- ELFDATA2MSB
 - elf64.h, [269](#)
- ELFMAG0
 - elf64.h, [269](#)
- ELFMAG1
 - elf64.h, [269](#)
- ELFMAG2
 - elf64.h, [269](#)
- ELFMAG3
 - elf64.h, [269](#)
- ELFOSABI_NONE
 - elf64.h, [269](#)
- EM_AARCH64
 - elf64.h, [269](#)
- EM_RISCV
 - elf64.h, [269](#)
- EM_X86_64
 - elf64.h, [269](#)
- emergency_console
 - VM, [146](#)
- emit_cell
 - control_words.c, [650](#)
- empty_all_buffers
 - block_words.c, [645](#)
 - block_words.h, [682](#)
- enable_benchmark_mode
 - test_runner.c, [608](#)
 - test_runner.h, [567](#)
- ENABLE_HOTWORDS_CACHE
 - starforth_config.h, [242](#)
- enable_hotwords_cache
 - DoeMetrics, [77](#)
- enable_hpet
 - timer.c, [483](#)
- enable_loop_1_heat_tracking
 - DoeMetrics, [77](#)
- enable_loop_2_rolling_window
 - DoeMetrics, [77](#)
- enable_loop_3_linear_decay
 - DoeMetrics, [77](#)
- enable_loop_4_pipelining
 - DoeMetrics, [77](#)
- enable_loop_5_window_inference
 - DoeMetrics, [77](#)
- enable_loop_6_decay_inference
 - DoeMetrics, [77](#)
- enable_loop_7_adaptive_heartrate
 - DoeMetrics, [77](#)
- ENABLE_PIPELINING
 - physics_pipelining_metrics.h, [208](#)
 - starforth_config.h, [242](#)
- enable_pipelining
 - DoeMetrics, [77](#)
- EnableCursor
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, [51](#)
- enabled
 - HotwordsCache_s, [100](#)
 - profiler.c, [454](#)
- encoding
 - blk_meta_t, [55](#)
- ended_ns
 - CapsuleRunRecord, [68](#)
- endof_addr
 - control_words.c, [650](#)
- endof_mark_sp
 - control_words.c, [650](#)
- endof_mark_stack
 - control_words.c, [650](#)
- endof_sp
 - control_words.c, [650](#)
- ensure_input
 - string_words.c, [740](#)
- ensure_pn
 - format_words.c, [675](#)
- entropy
 - blk_meta_t, [55](#)
 - ssm_l8_metrics_t, [134](#)
- entry
 - reloc_block, [127](#)
 - reloc_stub.c, [492](#)
 - WordStats, [158](#)
- erf_q48
 - math_portable.c, [415](#)
 - math_portable.h, [198](#)

- error
 - ExecResult, [90](#)
 - VM, [146](#)
- estimated_jitter_ns
 - HeartbeatTickSnapshot, [98](#)
- ET_CORE
 - elf64.h, [269](#)
- ET_DYN
 - elf64.h, [269](#)
- ET_EXEC
 - elf64.h, [269](#)
- ET_NONE
 - elf64.h, [269](#)
- ET_REL
 - elf64.h, [269](#)
- EV_CURRENT
 - elf64.h, [269](#)
- evictions
 - HotwordsStats, [102](#)
- exec_results_equal
 - test_contracts.c, [606](#)
 - test_contracts.h, [565](#)
- ExecResult, [90](#)
 - dsp, [90](#)
 - error, [90](#)
 - rsp, [91](#)
 - stack, [91](#)
- executable
 - vmm_page_info_t, [155](#)
- execute_colon_word
 - vm.c, [611](#)
 - vm.h, [318](#)
 - vm_core.c, [544](#)
 - vm_internal.h, [629](#)
- execution_heat
 - DictEntry, [71](#)
 - vm.h, [336](#)
- execution_history
 - RollingWindowOfTruth, [128](#)
- Exit
 - _EFI_BOOT_SERVICES, [46](#)
- exit_colon
 - VM, [147](#)
- ExitBootServices
 - _EFI_BOOT_SERVICES, [46](#)
- exp_q48
 - math_portable.c, [416](#)
- expected
 - TestCase, [135](#)
- expected_delta
 - time_trust_state, [138](#)
- extract_cache_metrics
 - doe_metrics.c, [397](#)
- extract_heat_trajectory
 - inference_engine.c, [404](#)
- fail_and_exit
 - test_common.c, [601](#)
- fail_fast
 - CLConfig, [69](#)
 - test_common.c, [605](#)
 - test_common.h, [563](#)
- FALSE
 - uefi.h, [296](#)
- Fast Primitive Word Implementations, [26](#)
 - PRIM_2DROP_ARM64, [27](#)
 - PRIM_2DUP_ARM64, [27](#)
 - PRIM_AND_ARM64, [27](#)
 - PRIM_BRANCH_ARM64, [27](#)
 - PRIM_C_FETCH_ARM64, [27](#)
 - PRIM_C_STORE_ARM64, [27](#)
 - PRIM_DROP_ARM64, [27](#)
 - PRIM_DUP_ARM64, [27](#)
 - PRIM_EQUALS_ARM64, [28](#)
 - PRIM_EXECUTE_ARM64, [28](#)
 - PRIM_FETCH_ARM64, [28](#)
 - PRIM_GREATER_ARM64, [28](#)
 - PRIM_INVERT_ARM64, [28](#)
 - PRIM_LESS_ARM64, [28](#)
 - PRIM_MINUS_ARM64, [29](#)
 - PRIM_MOD_ARM64, [29](#)
 - PRIM_NEGATE_ARM64, [29](#)
 - PRIM_OR_ARM64, [29](#)
 - PRIM_OVER_ARM64, [29](#)
 - PRIM_PLUS_ARM64, [29](#)
 - PRIM_R_FETCH_ARM64, [29](#)
 - PRIM_R_FROM_ARM64, [30](#)
 - PRIM_ROT_ARM64, [30](#)
 - PRIM_SLASH_ARM64, [30](#)
 - PRIM_STAR_ARM64, [30](#)
 - PRIM_STORE_ARM64, [30](#)
 - PRIM_SWAP_ARM64, [30](#)
 - PRIM_TO_R_ARM64, [30](#)
 - PRIM_XOR_ARM64, [31](#)
 - PRIM_ZBRANCH_ARM64, [31](#)
 - PRIM_ZERO_EQUALS_ARM64, [31](#)
 - PRIM_ZERO_LESS_ARM64, [31](#)
- fblock_pack_offset
 - block_subsystem.c, [382](#)
- fblock_to_devblock
 - block_subsystem.c, [382](#)
- fbs
 - blkcfg_t, [59](#)
 - blkio_file_state_t, [60](#)
 - blkio_ram_state_t, [62](#)
- fc_free
 - vocabulary_words.c, [751](#)
- fc_rebuild
 - vocabulary_words.c, [751](#)
- fd
 - blkio_file_state_t, [60](#)
- file_close
 - blkio_file.c, [371](#)
- file_flush
 - blkio_file.c, [371](#)
- file_info
 - blkio_file.c, [371](#)

file_open
 blkio_file.c, 371
file_path
 blkcfg_t, 59
file_read
 blkio_file.c, 371
file_write
 blkio_file.c, 371
FileName
 EFI_FILE_INFO, 80
FilePath
 EFI_LOADED_IMAGE_PROTOCOL, 82
FileSize
 EFI_FILE_INFO, 81
final_effective_window_size
 DoeMetrics, 77
finalize_report
 break_me_tests.c, 581
find_contiguous_free
 pmm.c, 527
find_entry_from_name
 dictionary_manipulation_words.c, 661
find_variance_inflection
 inference_engine.c, 404
 inference_engine.h, 189
FirmwareRevision
 _EFI_SYSTEM_TABLE, 52
FirmwareVendor
 _EFI_SYSTEM_TABLE, 52
first_free
 blk_volume_meta_t, 57
flags
 blk_meta_t, 55
 blk_volume_meta_t, 57
 CapsuleRunRecord, 68
 DictEntry, 71
 physics_analytics_header, 112
 physics_host_snapshot, 117
 vm.h, 336
 VMRegistryEntry, 155
 WordContract, 156
Flush
 _EFI_FILE_PROTOCOL, 48
flush
 blkio_vtable_t, 62
fnv1a_64
 parity.c, 539
 parity.h, 302
FNV1A_64_OFFSET_BASIS
 parity.h, 302
FNV1A_64_PRIME
 parity.h, 302
fnv1a_64_u32
 parity.c, 539
fnv1a_64_u8
 parity.c, 539
format_timestamp
 sf_time_backend_t, 131
format_word_base
 format_words.c, 675
format_word_begin_conversion
 format_words.c, 675
format_word_d_dot
 format_words.c, 675
format_word_d_dot_r
 format_words.c, 675
format_word_decimal
 format_words.c, 675
format_word_dot
 format_words.c, 675
format_word_dot_r
 format_words.c, 676
format_word_dot_s
 format_words.c, 676
format_word_dump
 format_words.c, 676
format_word_end_conversion
 format_words.c, 676
format_word_hash
 format_words.c, 676
format_word_hash_s
 format_words.c, 676
format_word_hex
 format_words.c, 676
format_word_hold
 format_words.c, 677
format_word_octal
 format_words.c, 677
format_word_question
 format_words.c, 677
format_word_sign
 format_words.c, 677
format_word_suites
 format_words_test.c, 586
format_word_u_dot
 format_words.c, 677
format_word_u_dot_r
 format_words.c, 677
format_words.c
 CELL_BITS, 674
 CHUNK_BITS, 674
 CHUNK_MASK, 674
 conversion_pos, 678
 current_base, 674
 digit_for, 674
 div_ud_by_base, 674
 ensure_pn, 675
 format_word_base, 675
 format_word_begin_conversion, 675
 format_word_d_dot, 675
 format_word_d_dot_r, 675
 format_word_decimal, 675
 format_word_dot, 675
 format_word_dot_r, 676
 format_word_dot_s, 676
 format_word_dump, 676

- format_word_end_conversion, [676](#)
- format_word_hash, [676](#)
- format_word_hash_s, [676](#)
- format_word_hex, [676](#)
- format_word_hold, [677](#)
- format_word_octal, [677](#)
- format_word_question, [677](#)
- format_word_sign, [677](#)
- format_word_u_dot, [677](#)
- format_word_u_dot_r, [677](#)
- pn_buf, [678](#)
- pn_cap, [678](#)
- print_number_formatted, [677](#)
- register_format_words, [678](#)
- format_words.h
 - register_format_words, [689](#)
- format_words_test.c
 - format_word_suites, [586](#)
 - run_format_words_tests, [586](#)
- FORTH-79 Control Flow Words, [39](#)
- FORTH-79 Defining Words, [39](#)
- FORTH-79 Dictionary & Compilation Words, [40](#)
- FORTH-79 Dictionary Manipulation Words, [39](#)
- FORTH-79 Line Editor Words, [41](#)
- FORTH-79 Logical Operations, [41](#)
- FORTH-79 Memory Operations, [41](#)
- FORTH-79 Return Stack Operations, [42](#)
- FORTH-79 System & Environment Words, [42](#)
- FORTH-79 Vocabulary System Words, [42](#)
- forth_79_standard
 - system_words.c, [749](#)
- forth_ALL_HEATS
 - physics_freeze_words.c, [721](#)
- forth_BENCH_DICT_LOOKUP
 - physics_benchmark_words.c, [717](#)
- forth_block_size
 - blkio_dev, [59](#)
 - blkio_info_t, [61](#)
 - blkio_params_t, [61](#)
- forth_cached_head
 - vocabulary_words.c, [756](#)
- forth_COMPARE_LOOKUPS
 - dictionary_heat_diagnostic_words.c, [656](#)
- forth_DECAY_RATE_FETCH
 - physics_freeze_words.c, [721](#)
- FORTH_FALSE
 - logical_words.c, [708](#)
- forth_fc
 - vocabulary_words.c, [756](#)
- forth_FREEZE_CRITICAL
 - physics_freeze_words.c, [721](#)
- forth_FREEZE_WORD
 - physics_freeze_words.c, [721](#)
- forth_FROZEN_QUERY
 - physics_freeze_words.c, [721](#)
- forth_HEAT_FETCH
 - physics_freeze_words.c, [722](#)
- forth_HEAT_PERCENTILES
 - dictionary_heat_diagnostic_words.c, [656](#)
- forth_HEAT_STORE
 - physics_freeze_words.c, [722](#)
- forth_LOOKUP_STRATEGY_FETCH
 - dictionary_heat_diagnostic_words.c, [657](#)
- forth_LOOKUP_STRATEGY_STORE
 - dictionary_heat_diagnostic_words.c, [657](#)
- forth_n
 - vocabulary_words.c, [756](#)
- forth_paren
 - system_words.c, [747](#)
- forth_PHYSICS_BAYESIAN_REPORT
 - physics_benchmark_words.c, [717](#)
- forth_PHYSICS_BUILD_INFO
 - physics_benchmark_words.c, [718](#)
- forth_PHYSICS_BURN
 - physics_diagnostic_words.c, [719](#)
 - physics_diagnostic_words.h, [693](#)
- forth_PHYSICS_CACHE_STATS
 - physics_benchmark_words.c, [718](#)
- forth_PHYSICS_CALC_KNOBS
 - physics_diagnostic_words.c, [720](#)
 - physics_diagnostic_words.h, [694](#)
- forth_PHYSICS_RESET_STATS
 - physics_benchmark_words.c, [718](#)
- forth_PHYSICS_SHOW_FEEDBACK
 - physics_diagnostic_words.c, [720](#)
 - physics_diagnostic_words.h, [694](#)
- forth_PHYSICS_TOGGLE_CACHE
 - physics_benchmark_words.c, [718](#)
- forth_PHYSICS_WORD_METRICS
 - physics_diagnostic_words.c, [720](#)
 - physics_diagnostic_words.h, [694](#)
- forth_PIPELINING_ANALYZE_WORD
 - physics_pipelining_diagnostic_words.c, [723](#)
- forth_PIPELINING_ENABLE
 - physics_pipelining_diagnostic_words.c, [723](#)
- forth_PIPELINING_RESET_ALL
 - physics_pipelining_diagnostic_words.c, [723](#)
- forth_PIPELINING_SHOW_STATS
 - physics_pipelining_diagnostic_words.c, [723](#)
- forth_PIPELINING_SHOW_TOP_TRANSITIONS
 - physics_pipelining_diagnostic_words.c, [724](#)
- forth_PIPELINING_STATS
 - physics_pipelining_diagnostic_words.c, [724](#)
- forth_REORG_BUCKETS
 - dictionary_heat_diagnostic_words.c, [657](#)
- forth_SHOW_HEAT
 - physics_freeze_words.c, [722](#)
- forth_SHOW_HEAT_OPTIMIZATION
 - dictionary_heat_diagnostic_words.c, [657](#)
- FORTH_TRUE
 - logical_words.c, [708](#)
- forth_UNFREEZE_WORD
 - physics_freeze_words.c, [722](#)
- forth_vocab
 - vocabulary_words.c, [756](#)
- framebuffer

- BootInfo, [64](#)
- FramebufferInfo, [91](#)
 - base, [91](#)
 - height, [91](#)
 - pixels_per_scanline, [91](#)
 - size, [91](#)
 - width, [91](#)
- free
 - heap_block, [93](#)
 - VMHostServices, [153](#)
- free_blocks
 - blk_volume_meta_t, [57](#)
- free_bytes
 - kmalloc_stats_t, [107](#)
 - pmm_stats_t, [123](#)
- free_count
 - sf_alloc_stats, [130](#)
- free_pages
 - pmm_stats_t, [123](#)
- FreePages
 - _EFI_BOOT_SERVICES, [46](#)
- FreePool
 - _EFI_BOOT_SERVICES, [46](#)
- full_read_at
 - blkio_file.c, [372](#)
- full_write_at
 - blkio_file.c, [372](#)
- func
 - DictEntry, [71](#)
 - vm.h, [336](#)
- fuzz_rounds
 - WordContract, [156](#)
- g
 - block_subsystem.c, [384](#)
- g_arena_offset
 - alloc_kernel.c, [441](#), [443](#)
- g_blkcfg
 - blkcfg.h, [171](#)
- g_blkio_opened
 - main.c, [415](#)
- g_boot_info
 - uefi_loader.c, [494](#)
- g_heartbeat
 - timer.c, [489](#)
- g_hotwords_cache
 - physics_hotwords_cache.c, [423](#)
- g_initialized
 - alloc_kernel.c, [441](#), [443](#)
- g_prng_state
 - starforth_words.c, [739](#)
- g_profiler
 - profiler.c, [454](#)
 - profiler.h, [231](#)
- g_ram_base
 - main.c, [415](#)
- g_ram_bytes
 - main.c, [415](#)
- g_ram_is_mmap
 - main.c, [415](#)
- g_runtime
 - physics_runtime.c, [438](#)
- g_stats
 - alloc_host.c, [439](#)
 - alloc_kernel.c, [441](#), [443](#)
- g_vm
 - vm_debug.c, [625](#)
- gdt_entry, [92](#)
 - access, [92](#)
 - base_high, [92](#)
 - base_low, [92](#)
 - base_mid, [92](#)
 - granularity, [92](#)
 - limit_low, [92](#)
- gdt_init
 - arch.c, [474](#)
- gdt_set_entry
 - arch.c, [474](#)
- gdtr, [92](#)
 - base, [92](#)
 - limit, [92](#)
- get_block_buffer
 - block_words.c, [645](#)
 - block_words.h, [682](#)
- get_body_address
 - dictionary_manipulation_words.c, [661](#)
- get_empty_buffer
 - block_words.c, [646](#)
 - block_words.h, [684](#)
- get_monotonic_ns
 - sf_time_backend_t, [131](#)
- get_name_field
 - dictionary_manipulation_words.c, [662](#)
- get_or_alloc_table
 - vmm.c, [532](#)
- get_realtime_ns
 - sf_time_backend_t, [132](#)
- get_table
 - vmm.c, [532](#)
- get_time_ns
 - profiler.c, [449](#)
 - test_runner.c, [608](#)
- get_timestamp
 - log.c, [409](#)
- get_word_stats
 - profiler.c, [449](#)
- GetInfo
 - _EFI_FILE_PROTOCOL, [48](#)
- GetMemoryMap
 - _EFI_BOOT_SERVICES, [46](#)
- GetNextHighMonotonicCount
 - _EFI_RUNTIME_SERVICES, [49](#)
- GetNextMonotonicCount
 - _EFI_BOOT_SERVICES, [46](#)
- GetNextVariableName
 - _EFI_RUNTIME_SERVICES, [49](#)
- GetPosition

- [_EFI_FILE_PROTOCOL](#), 48
- [GetTime](#)
 - [_EFI_RUNTIME_SERVICES](#), 49
- [GetVariable](#)
 - [_EFI_RUNTIME_SERVICES](#), 50
- [GetWakeupTime](#)
 - [_EFI_RUNTIME_SERVICES](#), 50
- [global_contract_violations](#)
 - [test_contracts.c](#), 607
 - [test_contracts.h](#), 566
- [global_test_stats](#)
 - [test_runner.c](#), 609
 - [test_runner.h](#), 576
- [granularity](#)
 - [arch.c](#), 474
 - [gdt_entry](#), 92
- [guid_equals](#)
 - [uefi_loader.c](#), 494
- [hal_mem_alloc](#)
 - [hal_memory.h](#), 274
 - [memory.c](#), 513
- [hal_mem_alloc_aligned](#)
 - [hal_memory.h](#), 274
 - [memory.c](#), 513
- [hal_mem_free](#)
 - [hal_memory.h](#), 274
 - [memory.c](#), 514
- [hal_memory.h](#)
 - [hal_mem_alloc](#), 274
 - [hal_mem_alloc_aligned](#), 274
 - [hal_mem_free](#), 274
- [halted](#)
 - [VM](#), 147
- [handle](#)
 - [sf_mutex](#), 131
- [HandleProtocol](#)
 - [_EFI_BOOT_SERVICES](#), 46
- [has_rtc](#)
 - [sf_time_backend_t](#), 132
- [has_variance_stabilized](#)
 - [inference_engine.c](#), 404
- [hash](#)
 - [blk_meta_t](#), 55
- [hash_colon_body](#)
 - [parity.c](#), 539
- [Hdr](#)
 - [_EFI_BOOT_SERVICES](#), 46
 - [_EFI_RUNTIME_SERVICES](#), 50
 - [_EFI_SYSTEM_TABLE](#), 53
- [hdr_crc](#)
 - [blk_volume_meta_t](#), 57
- [header](#)
 - [physics_analytics_heap](#), 114
- [header_hash64](#)
 - [ParityPacket](#), 109
- [HeaderSize](#)
 - [EFI_TABLE_HEADER](#), 84
- [heap](#)
 - [physics_runtime_state](#), 119
- [heap_base](#)
 - [kmallocc](#), 525
- [heap_block](#), 93
 - [free](#), 93
 - [next](#), 93
 - [prev](#), 93
 - [requested](#), 93
 - [size](#), 93
- [heap_block_t](#)
 - [kmallocc](#), 523
- [heap_bytes](#)
 - [physics_analytics_header](#), 112
- [heap_head](#)
 - [kmallocc](#), 526
- [heap_initialized](#)
 - [kmallocc](#), 526
- [heap_ready](#)
 - [physics_runtime_state](#), 120
- [heap_size](#)
 - [kmallocc](#), 526
- [heap_stats](#)
 - [kmallocc](#), 526
- [heartbeat](#)
 - [VM](#), 147
- [heartbeat_capture_tick_snapshot](#)
 - [heartbeat_export.c](#), 400
 - [vm.h](#), 318
- [HEARTBEAT_CHECK_FREQUENCY](#)
 - [starforth_config.h](#), 242
- [HEARTBEAT_DECAY_BATCH](#)
 - [vm_internal.h](#), 626, 629
- [heartbeat_decay_cursor_id](#)
 - [VM](#), 148
- [heartbeat_emit_tick_row](#)
 - [heartbeat_export.c](#), 401
 - [vm.h](#), 318
- [heartbeat_enabled](#)
 - [HeartbeatState](#), 96
- [heartbeat_export.c](#)
 - [heartbeat_capture_tick_snapshot](#), 400
 - [heartbeat_emit_tick_row](#), 401
- [HEARTBEAT_HAS_THREADS](#)
 - [vm_internal.h](#), 626, 629
- [HEARTBEAT_INFERENCE_FREQUENCY](#)
 - [starforth_config.h](#), 242
- [heartbeat_init](#)
 - [timer.c](#), 483
 - [timer.h](#), 291
- [HEARTBEAT_LOG_FULL](#)
 - [cli.h](#), 180
- [heartbeat_log_mode](#)
 - [CLIConfig](#), 70
- [HEARTBEAT_LOG_OFF](#)
 - [cli.h](#), 180
- [HEARTBEAT_LOG_SUMMARY](#)
 - [cli.h](#), 180
- [heartbeat_publish_snapshot](#)

- vm_internal.h, 626
- vm_runtime.c, 551
- HEARTBEAT_SLOPE_VALIDATION_FREQUENCY
 - starforth_config.h, 242
- heartbeat_snapshot_index_load
 - vm_internal.h, 626, 629
 - vm_runtime.c, 551
- heartbeat_snapshot_index_store
 - vm_internal.h, 626, 629
 - vm_runtime.c, 551
- heartbeat_state
 - timer.c, 484
 - timer.h, 291
- HEARTBEAT_THREAD_ENABLED
 - starforth_config.h, 242
- heartbeat_tick
 - timer.c, 484
 - timer.h, 291
- HEARTBEAT_TICK_BUFFER_SIZE
 - vm.h, 314
- HEARTBEAT_TICK_NS
 - starforth_config.h, 242
- heartbeat_ticks
 - timer.c, 484
 - timer.h, 291
- heartbeat_trust
 - timer.c, 484
 - timer.h, 291
- HEARTBEAT_WINDOW_TUNING_FREQUENCY
 - starforth_config.h, 242
- HeartbeatLogMode
 - cli.h, 180
- HeartbeatSnapshot, 93
 - actual_window_size, 94
 - decay_slope_q48, 94
 - hot_word_count, 94
 - published_ns, 94
 - published_tick, 94
 - stale_word_count, 94
 - total_heat, 94
 - window_width, 94
- HeartbeatState, 94
 - bucket_collapse_flag, 95
 - bucket_mode_transitions, 95
 - bucket_sum_heat_variance, 95
 - bucket_sum_K, 95
 - bucket_sum_K_squared, 95
 - bucket_sum_window_variance, 95
 - bucket_tick_count, 96
 - check_counter, 96
 - dictionary_lookups, 96
 - early_exit_count, 96
 - heartbeat_enabled, 96
 - inference_run_count, 96
 - last_inference_tick, 96
 - m5_time_trust, 96
 - m5_variance, 96
 - run_start_ns, 96
 - snapshot_index, 96
 - snapshots, 96
 - tick_buffer, 97
 - tick_buffer_size, 97
 - tick_buffer_write_index, 97
 - tick_count, 97
 - tick_count_total, 97
 - tick_number_offset, 97
 - tick_target_ns, 97
 - words_executed, 97
 - worker, 97
- HeartbeatTickSnapshot, 97
 - actual_window_size, 98
 - avg_word_heat, 98
 - bucket_hits_delta, 98
 - cache_hits_delta, 98
 - elapsed_ns, 98
 - estimated_jitter_ns, 98
 - hot_word_count, 98
 - l8_mode, 98
 - predicted_label_hits, 98
 - tick_interval_ns, 99
 - tick_number, 99
 - window_width, 99
 - word_executions_delta, 99
- HeartbeatWorker, 99
 - running, 99
 - stop_requested, 99
 - tick_ns, 99
 - vm.h, 316
 - vm_internal.h, 626, 629
- heat_25th
 - PhysicsSnapshot, 121
- heat_50th
 - PhysicsSnapshot, 121
- heat_75th
 - PhysicsSnapshot, 121
- HEAT_CACHE_DEMOTION_THRESHOLD
 - vm.h, 314
- heat_cache_demotion_threshold
 - DoeMetrics, 78
- heat_threshold_25th
 - VM, 148
- heat_threshold_50th
 - VM, 148
- heat_threshold_75th
 - VM, 148
- height
 - FramebufferInfo, 91
- here
 - VM, 148
- here_offset
 - ParityPacket, 109
- history
 - rolling_window_view_t, 127
- HOLA_PROTOCOL_MAJOR
 - physics_runtime.h, 216
- HOLA_PROTOCOL_MINOR

- physics_runtime.h, 216
- HOLA_SHARED_MAGIC
 - physics_runtime.h, 216
- host
 - VM, 148
- hosted_alloc
 - vm_core.c, 544
- hosted_fake_ns
 - vm_core.c, 550
- hosted_free
 - vm_core.c, 544
- hosted_mutex_destroy
 - vm_core.c, 544
- hosted_mutex_init
 - vm_core.c, 544
- hosted_mutex_lock
 - vm_core.c, 545
- hosted_mutex_unlock
 - vm_core.c, 545
- hosted_panic
 - vm_core.c, 545
- hosted_putc
 - vm_core.c, 545
- hosted_puts
 - vm_core.c, 545
- hosted_services
 - vm_core.c, 550
- hosted_time_ns
 - vm_core.c, 545
- hosted_xt_entry_owned
 - vm_core.c, 545
- hosted_xt_is_executable
 - vm_core.c, 545
- hot_word_count
 - DoeMetrics, 78
 - HeartbeatSnapshot, 94
 - HeartbeatTickSnapshot, 98
 - InferenceInputs, 105
- hot_word_count_at_check
 - VM, 148
- hotwords_bayesian_report
 - physics_hotwords_cache.c, 420
 - physics_hotwords_cache.h, 203
- hotwords_bucket_reorder
 - physics_hotwords_cache.c, 420
 - physics_hotwords_cache.h, 203
- hotwords_cache
 - VM, 148
- hotwords_cache_cleanup
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 203
- hotwords_cache_init
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 203
- hotwords_cache_lookup
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 203
- hotwords_cache_promote
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 204
- hotwords_cache_set_enabled
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 204
- HOTWORDS_CACHE_SIZE
 - physics_hotwords_cache.h, 202
- HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD
 - physics_hotwords_cache.h, 202
- HOTWORDS_EXECUTION_HEAT_THRESHOLD
 - physics_hotwords_cache.h, 202
- hotwords_posterior_bucket_searches
 - physics_hotwords_cache.c, 421
 - physics_hotwords_cache.h, 204
- hotwords_posterior_cache_hits
 - physics_hotwords_cache.c, 422
 - physics_hotwords_cache.h, 204
- hotwords_speedup_estimate
 - physics_hotwords_cache.c, 422
 - physics_hotwords_cache.h, 204
- hotwords_stats_print
 - physics_hotwords_cache.c, 422
 - physics_hotwords_cache.h, 205
- hotwords_stats_reset
 - physics_hotwords_cache.c, 422
 - physics_hotwords_cache.h, 205
- HotwordsCache
 - physics_hotwords_cache.h, 203
 - vm.h, 317
- HotwordsCache_s, 100
 - cache, 100
 - cache_count, 100
 - enabled, 100
 - lru_index, 100
 - stats, 100
- HotwordsStats, 100
 - bucket_hits, 101
 - bucket_reorders, 101
 - bucket_search_samples, 101
 - bucket_search_total_ns_q48, 102
 - bucket_search_variance_sum_q48, 102
 - cache_hit_samples, 102
 - cache_hit_total_ns_q48, 102
 - cache_hit_variance_sum_q48, 102
 - cache_hits, 102
 - evictions, 102
 - max_bucket_search_ns, 102
 - max_cache_hit_ns, 103
 - min_bucket_search_ns, 103
 - min_cache_hit_ns, 103
 - misses, 103
 - promotions, 103
 - total_lookups, 103
- Hour
 - EFI_TIME, 84
- hpet_freq_hz
 - timer.c, 489
- HPET_GEN_CAP_ID

- timer.c, [480](#)
- HPET_GEN_CONFIG
 - timer.c, [481](#)
- hpet_hz
 - timer_calibration_record, [140](#)
- HPET_MAIN_COUNTER
 - timer.c, [481](#)
- HPET_PHYS_BASE
 - timer.c, [481](#)
- hpet_phys_base
 - vmm.c, [535](#)
- hpet_read
 - timer.c, [484](#)
- hpet_regs
 - timer.c, [489](#)
- HPET_VIRT_BASE
 - timer.c, [481](#)
- hpet_write
 - timer.c, [484](#)
- hw_address
 - BlockBuffer, [64](#)
- hysteresis_counter
 - ssm_l8_state_t, [135](#)
- idt
 - interrupts.c, [478](#)
- IDT_ENTRIES
 - interrupts.c, [476](#)
- idt_entry, [103](#)
 - ist, [104](#)
 - offset_high, [104](#)
 - offset_low, [104](#)
 - offset_mid, [104](#)
 - selector, [104](#)
 - type_attr, [104](#)
 - zero, [104](#)
- idtr, [104](#)
 - base, [104](#)
 - limit, [104](#)
- ImageBase
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- ImageCodeType
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- ImageDataType
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- ImageSize
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- implemented
 - TestCase, [136](#)
- in_bounds
 - blkio_ram.c, [373](#)
- in_var
 - VM, [149](#)
- inb
 - console.c, [513](#)
 - timer.c, [484](#)
- include/arch_detect.h, [165](#)
- include/blkcfg.h, [170](#)
- include/blkio.h, [172](#)
- include/blkio_factory.h, [174](#)
- include/block_subsystem.h, [175](#)
- include/cli.h, [179](#)
- include/dictionary_heat_optimization.h, [181](#)
- include/dictionary_management.h, [183](#)
- include/doe_metrics.h, [184](#)
- include/inference_engine.h, [188](#)
- include/io.h, [191](#)
- include/log.h, [194](#)
- include/math_portable.h, [198](#)
- include/memory_management.h, [199](#)
- include/physics_execution_hooks.h, [200](#)
- include/physics_hotwords_cache.h, [201](#)
- include/physics_metadata.h, [205](#)
- include/physics_pipelining_metrics.h, [207](#)
- include/physics_runtime.h, [215](#)
- include/platform_alloc.h, [219](#)
- include/platform_lock.h, [220](#)
- include/platform_time.h, [221](#)
- include/profiler.h, [224](#)
- include/q48_16.h, [277](#)
- include/repl.h, [288](#)
- include/rolling_window_knobs.h, [231](#)
- include/rolling_window_of_truth.h, [232](#)
- include/ssm_jacquard.h, [238](#)
- include/starforth_config.h, [241](#)
- include/starkernel/apic.h, [244](#)
- include/starkernel/arch.h, [246](#)
- include/starkernel/capsule.h, [247](#)
- include/starkernel/capsule_birth.h, [253](#)
- include/starkernel/capsule_generated.h, [258](#)
- include/starkernel/capsule_loader.h, [258](#)
- include/starkernel/capsule_run.h, [260](#)
- include/starkernel/console.h, [264](#)
- include/starkernel/elf64.h, [266](#)
- include/starkernel/elf_loader.h, [272](#)
- include/starkernel/hal/hal.h, [273](#)
- include/starkernel/hal_memory.h, [273](#)
- include/starkernel/kmalloc.h, [274](#)
- include/starkernel/pmm.h, [275](#)
- include/starkernel/q48_16.h, [284](#)
- include/starkernel/repl.h, [288](#)
- include/starkernel/timer.h, [289](#)
- include/starkernel/uefi.h, [292](#)
- include/starkernel/vm/arena.h, [301](#)
- include/starkernel/vm/bootstrap/sk_vm_bootstrap.h, [301](#)
- include/starkernel/vm/parity.h, [301](#)
- include/starkernel/vmm.h, [304](#)
- include/starkernel/xxhash64.h, [306](#)
- include/startup.h, [309](#)
- include/vm.h, [310](#)
- include/vm_api.h, [337](#)
- include/vm_asm_opt.h, [337](#)
- include/vm_asm_opt_arm64.h, [338](#)
- include/vm_asm_opt_riscv64.h, [343](#)
- include/vm_debug.h, [346](#)
- include/vm_host.h, [348](#)

- include/vm_inner_interp_arm64.h, 351
- include/vm_inner_interp_asm.h, 352
- include/vm_inner_interp_riscv64.h, 360
- include/word_registry.h, 366
- infer_decay_slope_q48
 - inference_engine.c, 404
 - inference_engine.h, 189
- inference_engine.c
 - compute_fit_quality, 402
 - compute_levene_statistic, 402
 - compute_mean_q48, 402
 - compute_median_q48, 402
 - compute_variance_q48, 402
 - extract_heat_trajectory, 404
 - find_variance_inflection, 404
 - has_variance_stabilized, 404
 - infer_decay_slope_q48, 404
 - inference_engine_run, 405
 - inference_outputs_to_string, 406
 - inference_outputs_validate, 406
- inference_engine.h
 - compute_variance_q48, 189
 - find_variance_inflection, 189
 - infer_decay_slope_q48, 189
 - inference_engine_run, 190
 - inference_outputs_to_string, 191
 - inference_outputs_validate, 191
- inference_engine_run
 - inference_engine.c, 405
 - inference_engine.h, 190
- inference_outputs_to_string
 - inference_engine.c, 406
 - inference_engine.h, 191
- inference_outputs_validate
 - inference_engine.c, 406
 - inference_engine.h, 191
- inference_run_count
 - DoeMetrics, 78
 - HeartbeatState, 96
- InferenceInputs, 105
 - hot_word_count, 105
 - last_stale_count, 105
 - last_total_heat, 105
 - prefetch_attempts, 105
 - prefetch_hits, 105
 - stale_word_count, 105
 - total_heat, 105
 - trajectory_length, 105
 - vm, 105
 - window, 105
 - word_count, 106
- InferenceOutputs, 106
 - adaptive_decay_slope, 106
 - adaptive_window_width, 106
 - early_exited, 106
 - last_check_tick, 106
 - slope_fit_quality_q48, 106
 - vm.h, 317
 - window_variance_q48, 106
- info
 - blkio_vtable_t, 63
- init_blkio
 - main.c, 413
- init_block_system
 - block_words.c, 646
 - block_words.h, 684
- init_report
 - break_me_tests.c, 581
- init_timer_vm_mode
 - timer.c, 485
- init_vm_and_subsystem
 - main.c, 413
- init_vocabulary_system
 - vocabulary_words.c, 751
- INITIAL_DECAY_SLOPE_Q48
 - starforth_config.h, 243
- initialized
 - block_subsystem.c, 384
- inl
 - timer.c, 485
- input
 - TestCase, 136
- input_buffer
 - VM, 149
- INPUT_BUFFER_SIZE
 - vm.h, 314
- input_length
 - VM, 149
- input_pos
 - VM, 149
- InstallConfigurationTable
 - _EFI_BOOT_SERVICES, 46
- InstallProtocolInterface
 - _EFI_BOOT_SERVICES, 46
- instruction_count
 - Profiler, 124
- INT16
 - uefi.h, 299
- INT32
 - uefi.h, 299
- INT64
 - uefi.h, 299
- INT8
 - uefi.h, 299
- integration_suites
 - integration_tests.c, 588
- integration_tests.c
 - integration_suites, 588
 - run_integration_tests, 586
- interpreter_enabled
 - VM, 149
- INTERRUPT_GATE
 - interrupts.c, 476
- interrupts.c
 - __attribute__, 476
 - arch_interrupts_init, 476

- base, [478](#)
- decode_pf_error, [476](#)
- idt, [478](#)
- IDT_ENTRIES, [476](#)
- INTERRUPT_GATE, [476](#)
- isr_common_handler, [476](#)
- isr_stub0, [476](#)
- isr_stub_table, [478](#)
- ist, [478](#)
- lidt, [476](#)
- limit, [478](#)
- offset_high, [478](#)
- offset_low, [478](#)
- offset_mid, [478](#)
- outb, [477](#)
- pic_disable, [477](#)
- print_dec_u64, [477](#)
- print_hex64, [477](#)
- print_q48_16, [477](#)
- selector, [478](#)
- set_idt_entry, [477](#)
- type_attr, [478](#)
- zero, [478](#)
- INTN
 - uefi.h, [299](#)
- invlpg
 - vmm.c, [532](#)
- io.c
 - io_init, [407](#)
 - io_write_block, [407](#)
- io.h
 - BLOCK_COUNT, [192](#)
 - io_init, [192](#)
 - io_read_block, [192](#)
 - io_write_block, [193](#)
 - MEMORY_SIZE, [192](#)
- io_init
 - io.c, [407](#)
 - io.h, [192](#)
- io_read_block
 - io.h, [192](#)
- io_word_cr
 - io_words.c, [705](#)
- io_word_emit
 - io_words.c, [705](#)
- io_word_key
 - io_words.c, [705](#)
- io_word_question_terminal
 - io_words.c, [705](#)
- io_word_space
 - io_words.c, [706](#)
- io_word_spaces
 - io_words.c, [706](#)
- io_word_suites
 - io_words_test.c, [589](#)
- io_word_type
 - io_words.c, [706](#)
- io_words.c
 - io_word_cr, [705](#)
 - io_word_emit, [705](#)
 - io_word_key, [705](#)
 - io_word_question_terminal, [705](#)
 - io_word_space, [706](#)
 - io_word_spaces, [706](#)
 - io_word_type, [706](#)
 - register_io_words, [707](#)
- io_words.h
 - IO_WORDS_H, [690](#)
 - register_io_words, [690](#)
- IO_WORDS_H
 - io_words.h, [690](#)
- io_words_test.c
 - io_word_suites, [589](#)
 - run_io_words_tests, [588](#)
- io_write_block
 - io.c, [407](#)
 - io.h, [193](#)
- ip
 - VM, [149](#)
- is_block_header
 - capsule_loader.c, [504](#)
- is_executable_ptr
 - VMHostServices, [154](#)
- is_open
 - blkio_dev, [59](#)
- is_ram_type
 - kernel_main.c, [518](#)
 - pmm.c, [528](#)
 - vmm.c, [532](#)
- is_usable_memory
 - pmm.c, [528](#)
- is_valid_dict_entry
 - starforth_words.c, [735](#)
- is_warm
 - rolling_window_view_t, [127](#)
 - RollingWindowOfTruth, [129](#)
- isqrt
 - doe_metrics.c, [397](#)
- isr_common_handler
 - interrupts.c, [476](#)
- isr_stub0
 - interrupts.c, [476](#)
- isr_stub_table
 - interrupts.c, [478](#)
- ist
 - idt_entry, [104](#)
 - interrupts.c, [478](#)
- itoa_simple
 - kernel_main.c, [518](#)
- kernel_elf_buffer
 - uefi_loader.c, [494](#)
- kernel_elf_size
 - uefi_loader.c, [494](#)
- KERNEL_HEAP_SIZE
 - kernel_main.c, [518](#)
- kernel_main

- kernel_main.c, 518
- kernel_main.c
 - is_ram_type, 518
 - itoa_simple, 518
 - KERNEL_HEAP_SIZE, 518
 - kernel_main, 518
 - print_banner, 519
 - print_boot_info, 519
 - print_heap_stats, 519
 - print_pmm_stats, 519
 - print_uint, 519
- KERNEL_MAX_SIZE
 - uefi_loader.c, 493
- KernelEntry
 - uefi_loader.c, 493
- kfree
 - kmalloc.c, 524
 - kmalloc.h, 274
- kmalloc
 - kmalloc.c, 524
 - kmalloc.h, 274
- kmalloc.c
 - account_allocation, 523
 - account_free, 523
 - align_up_size, 523
 - allocate_from_block, 523
 - can_split, 523
 - coalesce_neighbors, 524
 - heap_base, 525
 - heap_block_t, 523
 - heap_head, 526
 - heap_initialized, 526
 - heap_size, 526
 - heap_stats, 526
 - kfree, 524
 - kmalloc, 524
 - kmalloc_aligned, 524
 - KMALLOC_DEFAULT_HEAP_SIZE, 523
 - kmalloc_get_stats, 524
 - kmalloc_heap_base_addr, 524
 - kmalloc_heap_end_addr, 524
 - kmalloc_init, 525
 - kmalloc_is_initialized, 525
 - KMALLOC_MAGIC, 523
 - KMALLOC_MIN_ALIGN, 523
 - minSplittable_payload, 525
 - normalize_alignment, 525
 - pointer_within_heap, 525
 - update_free_bytes, 525
- kmalloc.h
 - kfree, 274
 - kmalloc, 274
 - kmalloc_aligned, 275
 - kmalloc_get_stats, 275
 - kmalloc_heap_base_addr, 275
 - kmalloc_heap_end_addr, 275
 - kmalloc_init, 275
 - kmalloc_is_initialized, 275
- kmalloc_aligned
 - kmalloc.c, 524
 - kmalloc.h, 275
- KMALLOC_DEFAULT_HEAP_SIZE
 - kmalloc.c, 523
- kmalloc_get_stats
 - kmalloc.c, 524
 - kmalloc.h, 275
- kmalloc_heap_base_addr
 - kmalloc.c, 524
 - kmalloc.h, 275
- kmalloc_heap_end_addr
 - kmalloc.c, 524
 - kmalloc.h, 275
- kmalloc_init
 - kmalloc.c, 525
 - kmalloc.h, 275
- kmalloc_is_initialized
 - kmalloc.c, 525
 - kmalloc.h, 275
- KMALLOC_MAGIC
 - kmalloc.c, 523
- KMALLOC_MIN_ALIGN
 - kmalloc.c, 523
- kmalloc_prefix_t, 107
 - block, 107
 - magic, 107
- kmalloc_stats_t, 107
 - free_bytes, 107
 - peak_bytes, 107
 - total_bytes, 107
 - used_bytes, 108
- L2_rolling_window
 - ssm_config_t, 133
- L3_linear_decay
 - ssm_config_t, 133
- L5_window_inference
 - ssm_config_t, 133
- L6_decay_inference
 - ssm_config_t, 134
- I8_mode
 - HeartbeatTickSnapshot, 98
- label
 - blk_volume_meta_t, 57
- lapic_base
 - apic.c, 472
- LAPIC_DEFAULT_PHYS
 - apic.c, 469
- lapic_phys_base
 - apic.c, 472
 - vmm.c, 535
- lapic_read
 - apic.c, 471
- LAPIC_VIRT_BASE
 - apic.c, 469
- lapic_write
 - apic.c, 471
- last_active_ns

- DictPhysics, [73](#)
- last_allocated
 - blk_volume_meta_t, [57](#)
- last_bucket_reorg_ns
 - VM, [149](#)
- last_check_tick
 - InferenceOutputs, [106](#)
- last_checked_accuracy
 - PipelineGlobalMetrics, [122](#)
- last_checked_window_size
 - PipelineGlobalMetrics, [122](#)
- last_decay_check_ns
 - VM, [149](#)
- last_decay_ns
 - DictPhysics, [73](#)
- last_inference_outputs
 - VM, [149](#)
- last_inference_tick
 - HeartbeatState, [96](#)
- last_pattern_diversity
 - RollingWindowOfTruth, [129](#)
- last_stale_count
 - InferenceInputs, [105](#)
- last_total_heat
 - InferenceInputs, [105](#)
- last_tsc
 - time_trust_state, [138](#)
- LastAccessTime
 - EFI_FILE_INFO, [81](#)
- latest
 - VM, [149](#)
- latest_word_id
 - ParityPacket, [109](#)
- lbn_in_range
 - block_subsystem.c, [382](#)
- lbn_to_pbn
 - block_subsystem.c, [382](#)
- leave_addrs
 - control_words.c, [650](#)
- leave_mark_sp
 - control_words.c, [651](#)
- leave_mark_stack
 - control_words.c, [651](#)
- leave_sp
 - control_words.c, [651](#)
- level
 - Profiler, [124](#)
 - profiler.c, [454](#)
- level_colors
 - log.c, [411](#)
- level_names
 - log.c, [411](#)
- lidt
 - interrupts.c, [476](#)
- LIKELY
 - vm.h, [314](#)
- limit
 - arch.c, [474](#)
- gdtr, [92](#)
- idtr, [104](#)
- interrupts.c, [478](#)
- limit_low
 - arch.c, [475](#)
 - gdt_entry, [92](#)
- line_ptr
 - editor_words.c, [672](#)
- link
 - DictEntry, [71](#)
 - vm.h, [336](#)
- load_avg_milli
 - physics_host_snapshot, [117](#)
- load_cr3
 - vmm.c, [533](#)
- load_kernel_from_esp
 - uefi_loader.c, [494](#)
- loaded
 - cache_slot_t, [65](#)
- LOADER_BLOCK_SIZE
 - capsule_loader.c, [502](#)
- LoadImage
 - _EFI_BOOT_SERVICES, [47](#)
- LoadOptions
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- LoadOptionsSize
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
- LocateDevicePath
 - _EFI_BOOT_SERVICES, [47](#)
- LocateHandle
 - _EFI_BOOT_SERVICES, [47](#)
- log.c
 - color_reset, [411](#)
 - current_level, [411](#)
 - get_timestamp, [409](#)
 - level_colors, [411](#)
 - level_names, [411](#)
 - log_get_level, [409](#)
 - log_message, [409](#)
 - log_set_level, [410](#)
 - log_test_result, [410](#)
 - test_fail_color, [411](#)
 - test_pass_color, [411](#)
 - test_skip_color, [411](#)
- log.h
 - LOG_DEBUG, [195](#)
 - LOG_ERROR, [195](#)
 - log_get_level, [195](#)
 - LOG_INFO, [195](#)
 - LOG_LINE_MAX, [194](#)
 - log_message, [195](#)
 - LOG_NONE, [195](#)
 - log_set_level, [196](#)
 - log_set_vm, [197](#)
 - LOG_TEST, [195](#)
 - log_test_result, [197](#)
 - LOG_WARN, [195](#)
 - LogLevel, [194](#)

- TEST_ERROR, 195
- TEST_FAIL, 195
- TEST_PASS, 195
- TEST_SKIP, 195
- TestResult, 195
- LOG_BLOCKS
 - vm.h, 314
- LOG_BLOCKS_END
 - vm.h, 314
- LOG_BLOCKS_START
 - vm.h, 315
- LOG_DEBUG
 - log.h, 195
- LOG_ERROR
 - log.h, 195
- log_get_level
 - log.c, 409
 - log.h, 195
- LOG_INFO
 - log.h, 195
- LOG_LAYER1_MAX_LINES
 - vm.h, 315
- log_level
 - CLIConfig, 70
- log_level_explicitly_set
 - CLIConfig, 70
- LOG_LINE_MAX
 - log.h, 194
 - vm.h, 315
- LOG_LINES_PER_BLOCK
 - vm.h, 315
- log_message
 - log.c, 409
 - log.h, 195
- LOG_NONE
 - log.h, 195
- log_set_level
 - log.c, 410
 - log.h, 196
- log_set_vm
 - log.h, 197
- LOG_TEST
 - log.h, 195
- log_test_result
 - log.c, 410
 - log.h, 197
- LOG_WARN
 - log.h, 195
- logical_word_and
 - logical_words.c, 708
- logical_word_equals
 - logical_words.c, 708
- logical_word_false
 - logical_words.c, 708
- logical_word_greater_than
 - logical_words.c, 708
- logical_word_less_than
 - logical_words.c, 708
- logical_word_not
 - logical_words.c, 709
- logical_word_not_equals
 - logical_words.c, 709
- logical_word_or
 - logical_words.c, 709
- logical_word_suites
 - logical_words_test.c, 589
- logical_word_true
 - logical_words.c, 709
- logical_word_u_greater_than
 - logical_words.c, 709
- logical_word_u_less_than
 - logical_words.c, 709
- logical_word_within
 - logical_words.c, 709
- logical_word_xor
 - logical_words.c, 710
- logical_word_zero_equals
 - logical_words.c, 710
- logical_word_zero_greater
 - logical_words.c, 710
- logical_word_zero_less
 - logical_words.c, 710
- logical_word_zero_not_equal
 - logical_words.c, 710
- logical_words.c
 - FORTH_FALSE, 708
 - FORTH_TRUE, 708
 - logical_word_and, 708
 - logical_word_equals, 708
 - logical_word_false, 708
 - logical_word_greater_than, 708
 - logical_word_less_than, 708
 - logical_word_not, 709
 - logical_word_not_equals, 709
 - logical_word_or, 709
 - logical_word_true, 709
 - logical_word_u_greater_than, 709
 - logical_word_u_less_than, 709
 - logical_word_within, 709
 - logical_word_xor, 710
 - logical_word_zero_equals, 710
 - logical_word_zero_greater, 710
 - logical_word_zero_less, 710
 - logical_word_zero_not_equal, 710
 - register_logical_words, 710
- logical_words.h
 - register_logical_words, 691
- logical_words_test.c
 - logical_word_suites, 589
 - run_logical_words_tests, 589
- LogLevel
 - log.h, 194
- lookup_strategy
 - VM, 150
- loop_iterations
 - Profiler, 124

- lru_index
 - HotwordsCache_s, 100
- LVT_MASKED
 - apic.c, 469
- LVT_MODE_ONESHOT
 - apic.c, 469
- LVT_MODE_PERIODIC
 - apic.c, 469
- m5_time_trust
 - HeartbeatState, 96
- m5_variance
 - HeartbeatState, 96
- magic
 - blk_meta_t, 55
 - blk_volume_meta_t, 58
 - CapsuleDirHeader, 66
 - kmalloc_prefix_t, 107
 - physics_analytics_header, 112
- main
 - main.c, 413
- main.c
 - __attribute__, 413
 - cleanup_and_exit, 413
 - cleanup_blkio, 413
 - g_blkio_opened, 415
 - g_ram_base, 415
 - g_ram_bytes, 415
 - g_ram_is_mmap, 415
 - init_blkio, 413
 - init_vm_and_subsystem, 413
 - main, 413
 - print_usage, 414
 - run_benchmark_mode, 414
 - run_doe_experiment, 414
 - run_optional_tests, 414
 - signal_handler, 414
 - STARFORTH_STATE_BYTES, 413
- make_pte
 - vmm.c, 533
- mama_forth_words_test.c
 - MAMA_IMPLEMENTED, 590
 - mama_word_suites, 590
 - run_mama_forth_words_tests, 590
- MAMA_IMPLEMENTED
 - mama_forth_words_test.c, 590
- mama_word_suites
 - mama_forth_words_test.c, 590
- mark_buffer_dirty
 - block_words.c, 646
 - block_words.h, 684
- mass_bytes
 - DictPhysics, 73
- math_portable.c
 - erf_q48, 415
 - exp_q48, 416
 - sqrt_q48, 416
- math_portable.h
 - erf_q48, 198
 - Q48_SCALE, 198
 - sqrt_q48, 198
- MAX_BLOCKS
 - block_words.h, 681
 - vm.h, 315
- max_bucket_search_ns
 - HotwordsStats, 102
- max_cache_hit_ns
 - HotwordsStats, 103
- MAX_DICT_ENTRIES
 - parity.c, 538
- MAX_TEST_INPUT
 - test_common.h, 555
- MAX_TEST_OUTPUT
 - test_common.h, 555
- MAX_TESTS_PER_WORD
 - test_common.h, 555
- max_time
 - ProfileTimer, 126
 - WordProfile, 157
- max_time_ns
 - WordStats, 158
- max_transition_probability_q48
 - WordTransitionMetrics, 161
- MAX_VMS
 - capsule_birth.c, 495
- MaxAllocateType
 - uefi.h, 300
- mean_ns_q48
 - BayesianLatencyPosterior, 54
- median_ns_q48
 - BayesianLatencyPosterior, 54
- memory
 - VM, 150
- memory.c
 - hal_mem_alloc, 513
 - hal_mem_alloc_aligned, 513
 - hal_mem_free, 514
- memory_allocated_bytes
 - DoeMetrics, 78
- memory_allocations
 - Profiler, 124
- memory_management.c
 - vm_addr_to_block, 416
 - vm_align, 417
 - vm_allot, 417
 - vm_get_block_addr, 417
- memory_management.h
 - vm_align, 199
 - vm_allot, 200
- memory_map
 - BootInfo, 64
- memory_map_descriptor_size
 - BootInfo, 64
- memory_map_size
 - BootInfo, 65
- MEMORY_SIZE
 - io.h, 192

- memory_stats
 - Profiler, [124](#)
- memory_word_2fetch
 - memory_words.c, [711](#)
- memory_word_2store
 - memory_words.c, [712](#)
- memory_word_cells
 - memory_words.c, [712](#)
- memory_word_cfetch
 - memory_words.c, [712](#)
- memory_word_cstore
 - memory_words.c, [712](#)
- memory_word_erase
 - memory_words.c, [713](#)
- memory_word_fetch
 - memory_words.c, [713](#)
- memory_word_fill
 - memory_words.c, [713](#)
- memory_word_minus_store
 - memory_words.c, [713](#)
- memory_word_move
 - memory_words.c, [714](#)
- memory_word_plus_store
 - memory_words.c, [714](#)
- memory_word_store
 - memory_words.c, [714](#)
- memory_word_suites
 - memory_words_test.c, [591](#)
- memory_words.c
 - memory_word_2fetch, [711](#)
 - memory_word_2store, [712](#)
 - memory_word_cells, [712](#)
 - memory_word_cfetch, [712](#)
 - memory_word_cstore, [712](#)
 - memory_word_erase, [713](#)
 - memory_word_fetch, [713](#)
 - memory_word_fill, [713](#)
 - memory_word_minus_store, [713](#)
 - memory_word_move, [714](#)
 - memory_word_plus_store, [714](#)
 - memory_word_store, [714](#)
 - register_memory_words, [714](#)
- memory_words.h
 - register_memory_words, [692](#)
- memory_words_test.c
 - memory_word_suites, [591](#)
 - run_memory_words_tests, [591](#)
- MemoryProfile, [108](#)
 - cache_hits, [108](#)
 - cache_misses, [108](#)
 - reads, [108](#)
 - total_bytes_read, [108](#)
 - total_bytes_written, [108](#)
 - writes, [108](#)
- meta
 - cache_slot_t, [65](#)
- meta_dirty
 - cache_slot_t, [66](#)
- meta_from_slice
 - block_subsystem.c, [382](#)
- META_PER_BLOCK
 - block_subsystem.c, [376](#)
- META_REGION_OFFSET
 - block_subsystem.c, [376](#)
- META_REGION_SIZE
 - block_subsystem.c, [377](#)
- meta_to_slice
 - block_subsystem.c, [382](#)
- metrics_from_vm
 - doe_metrics.c, [397](#)
 - doe_metrics.h, [185](#)
- metrics_get_cpu_freq_mhz
 - doe_metrics.c, [398](#)
 - doe_metrics.h, [185](#)
- metrics_get_cpu_temp_c
 - doe_metrics.c, [398](#)
 - doe_metrics.h, [186](#)
- metrics_get_timestamp
 - doe_metrics.c, [398](#)
 - doe_metrics.h, [186](#)
- metrics_print_text
 - doe_metrics.c, [398](#)
 - doe_metrics.h, [186](#)
- metrics_write_csv_header
 - doe_metrics.c, [399](#)
 - doe_metrics.h, [187](#)
- metrics_write_csv_row
 - doe_metrics.c, [399](#)
 - doe_metrics.h, [187](#)
- metrics_write_james_law_csv_row
 - doe_metrics.c, [399](#)
 - doe_metrics.h, [187](#)
- min_bucket_search_ns
 - HotwordsStats, [103](#)
- min_cache_hit_ns
 - HotwordsStats, [103](#)
- MIN_SAMPLES_FOR_SPECULATION
 - physics_pipelining_metrics.h, [208](#)
- minSplittable_payload
 - kmalloc.c, [525](#)
- min_time
 - ProfileTimer, [126](#)
 - WordProfile, [157](#)
- min_time_ns
 - WordStats, [158](#)
- MINIMUM_PREFETCH_ROI
 - physics_pipelining_metrics.h, [208](#)
- Minute
 - EFI_TIME, [85](#)
- minzu
 - block_subsystem.c, [382](#)
- MISPREDICTION_COST_Q48
 - physics_pipelining_metrics.h, [209](#)
- misprediction_cost_q48
 - WordTransitionMetrics, [161](#)
- misses

- HotwordsStats, 103
- mixed_arithmetic_word_suites
 - mixed_arithmetic_words_test.c, 592
- mixed_arithmetic_words.c
 - mixed_math_word_m_minus, 715
 - mixed_math_word_m_plus, 715
 - mixed_math_word_m_slash_mod, 715
 - mixed_math_word_m_star, 716
 - mixed_math_word_mod, 716
 - mixed_math_word_slash_mod, 716
 - mixed_math_word_star_slash, 716
 - mixed_math_word_star_slash_mod, 716
 - register_mixed_arithmetic_words, 716
- mixed_arithmetic_words.h
 - register_mixed_arithmetic_words, 692
- mixed_arithmetic_words_test.c
 - mixed_arithmetic_word_suites, 592
 - run_mixed_arithmetic_words_tests, 592
- mixed_math_word_m_minus
 - mixed_arithmetic_words.c, 715
- mixed_math_word_m_plus
 - mixed_arithmetic_words.c, 715
- mixed_math_word_m_slash_mod
 - mixed_arithmetic_words.c, 715
- mixed_math_word_m_star
 - mixed_arithmetic_words.c, 716
- mixed_math_word_mod
 - arithmetic_words.c, 642
 - mixed_arithmetic_words.c, 716
- mixed_math_word_slash_mod
 - mixed_arithmetic_words.c, 716
- mixed_math_word_star_slash
 - mixed_arithmetic_words.c, 716
- mixed_math_word_star_slash_mod
 - mixed_arithmetic_words.c, 716
- Mode
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51
- mode
 - VM, 150
- MODE_COMPILE
 - vm.h, 317
- MODE_INTERPRET
 - vm.h, 317
- mode_str
 - vm_debug.c, 623
- ModificationTime
 - EFI_FILE_INFO, 81
- modified_time
 - blk_meta_t, 56
- module_name
 - TestModule, 137
- monotonic_ns
 - VMHostServices, 154
- monotonic_time_ns
 - physics_host_snapshot, 117
- Month
 - EFI_TIME, 85
- most_likely_next_word_id
 - WordTransitionMetrics, 161
- mounted_time
 - blk_volume_meta_t, 58
- muldiv64
 - timer.c, 485
- mutex_destroy
 - VMHostServices, 154
- mutex_init
 - VMHostServices, 154
- mutex_lock
 - VMHostServices, 154
- mutex_unlock
 - VMHostServices, 154
- name
 - CapsuleNameEntry, 67
 - DictEntry, 72
 - physics_seed_definition, 120
 - ProfileTimer, 126
 - TestCase, 136
 - vm.h, 336
- name_count
 - CapsuleDirHeader, 67
- name_len
 - DictEntry, 72
 - vm.h, 336
- Nanosecond
 - EFI_TIME, 85
- next
 - heap_block, 93
- NEXT_ARM64
 - vm_inner_interp_arm64.h, 352
- NEXT_ASM
 - vm_inner_interp_asm.h, 353
- next_block
 - blk_meta_t, 56
- NEXT_OPT
 - arch_detect.h, 166
- NEXT_RV64
 - vm_inner_interp_riscv64.h, 360
- next_vm_id
 - capsule_birth.c, 498
- next_word_id
 - VM, 150
- nonempty
 - blkio_factory.c, 369
- normalize_alignment
 - kmalloc.c, 525
- ns_to_q48
 - physics_pipelining_metrics.c, 427
- NumberOfPages
 - EFI_MEMORY_DESCRIPTOR, 83
- NumberOfTableEntries
 - _EFI_SYSTEM_TABLE, 53
- offset_high
 - idt_entry, 104
 - interrupts.c, 478
- offset_low

- idt_entry, [104](#)
 - interrupts.c, [478](#)
- offset_mid
 - idt_entry, [104](#)
 - interrupts.c, [478](#)
- opaque
 - blkio_params_t, [61](#)
- Open
 - _EFI_FILE_PROTOCOL, [48](#)
- open
 - blkio_vtable_t, [63](#)
- OpenVolume
 - _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL, [50](#)
- outb
 - console.c, [513](#)
 - interrupts.c, [477](#)
 - timer.c, [485](#)
- OutputString
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, [51](#)
- owner_id
 - blk_meta_t, [56](#)
- owns_xt_entry
 - VMHostServices, [154](#)
- p_align
 - Elf64_Phdr, [87](#)
- p_filesz
 - Elf64_Phdr, [87](#)
- p_flags
 - Elf64_Phdr, [87](#)
- p_memsz
 - Elf64_Phdr, [87](#)
- p_offset
 - Elf64_Phdr, [87](#)
- p_paddr
 - Elf64_Phdr, [87](#)
- p_type
 - Elf64_Phdr, [87](#)
- p_vaddr
 - Elf64_Phdr, [87](#)
- Pad1
 - EFI_TIME, [85](#)
- Pad2
 - EFI_TIME, [85](#)
- padding
 - blk_meta_t, [56](#)
- paddr_to_virt
 - vmm.c, [533](#)
- page_rva
 - reloc_block, [127](#)
 - reloc_stub.c, [492](#)
- panic
 - VMHostServices, [154](#)
- parent_block
 - blk_meta_t, [56](#)
- parity.c
 - fnv1a_64, [539](#)
 - fnv1a_64_u32, [539](#)
 - fnv1a_64_u8, [539](#)
 - hash_colon_body, [539](#)
 - MAX_DICT_ENTRIES, [538](#)
 - print_hex64, [540](#)
 - print_nl, [540](#)
 - print_str, [540](#)
 - print_u32, [540](#)
 - sk_dict_canonical_hash, [540](#)
 - sk_dict_word_count, [541](#)
 - sk_parity_collect, [541](#)
 - SK_PARITY_DEBUG, [538](#)
 - sk_parity_debug_check_ptr, [538](#)
 - sk_parity_debug_log_msg, [538](#)
 - sk_parity_print, [541](#)
- parity.h
 - fnv1a_64, [302](#)
 - FNV1A_64_OFFSET_BASIS, [302](#)
 - FNV1A_64_PRIME, [302](#)
 - SK_BOOTSTRAP_ARENA_FAIL, [302](#)
 - SK_BOOTSTRAP_DICT_FAIL, [302](#)
 - SK_BOOTSTRAP_INIT_FAIL, [302](#)
 - SK_BOOTSTRAP_OK, [302](#)
 - sk_dict_canonical_hash, [303](#)
 - sk_dict_word_count, [303](#)
 - sk_parity_collect, [304](#)
 - sk_parity_print, [304](#)
- PARITY_MODE
 - vm_internal.h, [626](#)
- parity_mode
 - VMHostServices, [154](#)
- parity_put_hex64
 - capsule_run.c, [507](#)
- parity_put_u32
 - capsule_run.c, [507](#)
- parity_put_u64
 - capsule_run.c, [508](#)
- parity_putc
 - capsule_run.c, [508](#)
- parity_puts
 - capsule_run.c, [508](#)
- ParityPacket, [108](#)
 - bootstrap_result, [109](#)
 - header_hash64, [109](#)
 - here_offset, [109](#)
 - latest_word_id, [109](#)
 - tests_errors, [109](#)
 - tests_failed, [109](#)
 - tests_passed, [109](#)
 - tests_skipped, [109](#)
 - tests_total, [109](#)
 - window_hash64, [110](#)
 - word_count, [110](#)
- parse_avg_milli
 - physics_runtime.c, [434](#)
- parse_u32
 - cli.c, [388](#)
- path
 - blkio_file_state_t, [60](#)
- PATH_MAX

- blkio_file.c, 370
- physics_runtime.c, 433
- pattern_diversity_check_count
 - RollingWindowOfTruth, 129
- payload_bytes
 - physics_analytics_event_header, 110
- pd_index
 - vmm.c, 533
- pdpt_index
 - vmm.c, 533
- peak_bytes
 - kmalloc_stats_t, 107
 - sf_alloc_stats, 130
- pending_mode
 - ssm_l8_state_t, 135
- percentage
 - WordProfile, 157
- permissions
 - blk_meta_t, 56
- perturb_physics
 - test_contracts.c, 606
 - test_contracts.h, 565
- PF_R
 - elf64.h, 270
- PF_W
 - elf64.h, 270
- PF_X
 - elf64.h, 270
- phys_sector_size
 - blkio_info_t, 61
- phys_size_bytes
 - blkio_info_t, 61
- PhysicalSize
 - EFI_FILE_INFO, 81
- PhysicalStart
 - EFI_MEMORY_DESCRIPTOR, 83
- physics
 - DictEntry, 72
 - vm.h, 336
- PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT
 - physics_runtime.h, 216
- PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES
 - physics_runtime.h, 216
- physics_analytics_event_header, 110
 - channel, 110
 - payload_bytes, 110
 - reserved, 110
 - timestamp_ns, 110
- physics_analytics_event_header_t
 - physics_runtime.h, 216
- physics_analytics_header, 111
 - consume_seq, 112
 - dropped_events, 112
 - flags, 112
 - heap_bytes, 112
 - magic, 112
 - produce_seq, 112
 - read_offset, 112
 - ring_bytes, 112
 - ring_offset, 112
 - scratch_bytes, 112
 - scratch_offset, 112
 - summary_bytes, 113
 - summary_offset, 113
 - version_major, 113
 - version_minor, 113
 - write_offset, 113
- physics_analytics_header_t
 - physics_runtime.h, 216
- physics_analytics_heap, 113
 - base, 114
 - bytes, 114
 - header, 114
 - ring, 114
 - ring_bytes, 114
 - scratch, 114
 - scratch_bytes, 114
 - summary, 114
 - summary_bytes, 115
- physics_analytics_heap_info
 - physics_runtime.c, 434
 - physics_runtime.h, 217
- physics_analytics_heap_t
 - physics_runtime.h, 217
- physics_analytics_publish_event
 - physics_runtime.c, 435
 - physics_runtime.h, 217
- physics_benchmark_words.c
 - forth_BENCH_DICT_LOOKUP, 717
 - forth_PHYSICS_BAYESIAN_REPORT, 717
 - forth_PHYSICS_BUILD_INFO, 718
 - forth_PHYSICS_CACHE_STATS, 718
 - forth_PHYSICS_RESET_STATS, 718
 - forth_PHYSICS_TOGGLE_CACHE, 718
 - register_physics_benchmark_words, 718
 - sf_fc_count, 719
 - sf_fc_list, 719
- physics_benchmark_words.h
 - register_physics_benchmark_words, 693
- physics_decay_slope_load
 - physics_metadata.h, 206
- physics_diagnostic_words.c
 - forth_PHYSICS_BURN, 719
 - forth_PHYSICS_CALC_KNOBS, 720
 - forth_PHYSICS_SHOW_FEEDBACK, 720
 - forth_PHYSICS_WORD_METRICS, 720
 - register_physics_diagnostic_words, 720
- physics_diagnostic_words.h
 - forth_PHYSICS_BURN, 693
 - forth_PHYSICS_CALC_KNOBS, 694
 - forth_PHYSICS_SHOW_FEEDBACK, 694
 - forth_PHYSICS_WORD_METRICS, 694
 - register_physics_diagnostic_words, 694
- physics_execution_heat_increment
 - physics_metadata.h, 206
- physics_execution_heat_load

- physics_metadata.h, 206
- physics_execution_hooks.c
 - physics_on_lookup, 418
 - physics_post_execute, 418
 - physics_pre_execute, 419
- physics_execution_hooks.h
 - physics_on_lookup, 201
 - physics_post_execute, 201
 - physics_pre_execute, 201
- physics_freeze_words.c
 - forth_ALL_HEATS, 721
 - forth_DECAY_RATE_FETCH, 721
 - forth_FREEZE_CRITICAL, 721
 - forth_FREEZE_WORD, 721
 - forth_FROZEN_QUERY, 721
 - forth_HEAT_FETCH, 722
 - forth_HEAT_STORE, 722
 - forth_SHOW_HEAT, 722
 - forth_UNFREEZE_WORD, 722
 - register_physics_freeze_words, 722
- physics_freeze_words.h
 - register_physics_freeze_words, 695
- physics_global_stats
 - doe_metrics.c, 400
- physics_host_backend
 - physics_runtime.h, 217
- PHYSICS_HOST_BACKEND_FAKE
 - physics_runtime.h, 217
- PHYSICS_HOST_BACKEND_L4RE
 - physics_runtime.h, 217
- PHYSICS_HOST_BACKEND_POSIX
 - physics_runtime.h, 217
- physics_host_backend_t
 - physics_runtime.h, 217
- PHYSICS_HOST_FLAG_CGROUP
 - physics_runtime.h, 216
- PHYSICS_HOST_FLAG_CPU_STAT
 - physics_runtime.h, 216
- PHYSICS_HOST_FLAG_PSI
 - physics_runtime.h, 216
- physics_host_snapshot, 115
 - backend, 116
 - backend_seq, 116
 - cgroup_cpu_usage_us, 116
 - cgroup_memory_current_bytes, 116
 - cpu_count, 117
 - cpu_idle_jiffies, 117
 - cpu_total_jiffies, 117
 - flags, 117
 - load_avg_milli, 117
 - monotonic_time_ns, 117
 - physics_runtime.c, 435
 - physics_runtime.h, 218
 - psi_cpu_full_avg10_milli, 117
 - psi_cpu_full_avg300_milli, 117
 - psi_cpu_full_avg60_milli, 117
 - psi_cpu_some_avg10_milli, 117
 - psi_cpu_some_avg300_milli, 118
 - psi_cpu_some_avg60_milli, 118
 - psi_io_full_avg10_milli, 118
 - psi_io_full_avg300_milli, 118
 - psi_io_full_avg60_milli, 118
 - psi_io_some_avg10_milli, 118
 - psi_io_some_avg300_milli, 118
 - psi_io_some_avg60_milli, 118
 - psi_mem_some_avg10_milli, 118
 - psi_mem_some_avg300_milli, 118
 - psi_mem_some_avg60_milli, 119
 - realtime_ns, 119
 - runnable_threads, 119
 - scheduler_policy, 119
 - scheduler_priority, 119
 - scheduler_quantum_ns, 119
- physics_host_snapshot_t
 - physics_runtime.h, 217
- physics_hotwords_cache.c
 - calculate_posterior, 420
 - g_hotwords_cache, 423
 - hotwords_bayesian_report, 420
 - hotwords_bucket_reorder, 420
 - hotwords_cache_cleanup, 421
 - hotwords_cache_init, 421
 - hotwords_cache_lookup, 421
 - hotwords_cache_promote, 421
 - hotwords_cache_set_enabled, 421
 - hotwords_posterior_bucket_searches, 421
 - hotwords_posterior_cache_hits, 422
 - hotwords_speedup_estimate, 422
 - hotwords_stats_print, 422
 - hotwords_stats_reset, 422
 - q48_to_ns_double, 422
- physics_hotwords_cache.h
 - hotwords_bayesian_report, 203
 - hotwords_bucket_reorder, 203
 - hotwords_cache_cleanup, 203
 - hotwords_cache_init, 203
 - hotwords_cache_lookup, 203
 - hotwords_cache_promote, 204
 - hotwords_cache_set_enabled, 204
 - HOTWORDS_CACHE_SIZE, 202
 - HOTWORDS_EXECUTION_HEAT_DELTA_THRESHOLD, 202
 - HOTWORDS_EXECUTION_HEAT_THRESHOLD, 202
 - hotwords_posterior_bucket_searches, 204
 - hotwords_posterior_cache_hits, 204
 - hotwords_speedup_estimate, 204
 - hotwords_stats_print, 205
 - hotwords_stats_reset, 205
 - HotwordsCache, 203
- physics_metadata.c
 - clamp_u32, 424
 - derive_state_flags, 424
 - physics_metadata_apply_linear_decay, 424
 - physics_metadata_apply_seed, 424
 - physics_metadata_init, 424

- physics_metadata_record_latency, [424](#)
- physics_metadata_refresh_state, [424](#)
- physics_metadata_set_mass, [425](#)
- physics_metadata_touch, [425](#)
- physics_seed_definition_t, [423](#)
- physics_seed_table, [425](#)
- temperature_from_execution_heat, [425](#)
- physics_metadata.h
 - physics_decay_slope_load, [206](#)
 - physics_execution_heat_increment, [206](#)
 - physics_execution_heat_load, [206](#)
 - physics_metadata_apply_linear_decay, [206](#)
 - physics_metadata_apply_seed, [206](#)
 - physics_metadata_init, [206](#)
 - physics_metadata_record_latency, [206](#)
 - physics_metadata_refresh_state, [207](#)
 - physics_metadata_set_mass, [207](#)
 - physics_metadata_touch, [207](#)
- physics_metadata_apply_linear_decay
 - physics_metadata.c, [424](#)
 - physics_metadata.h, [206](#)
- physics_metadata_apply_seed
 - physics_metadata.c, [424](#)
 - physics_metadata.h, [206](#)
- physics_metadata_init
 - physics_metadata.c, [424](#)
 - physics_metadata.h, [206](#)
- physics_metadata_record_latency
 - physics_metadata.c, [424](#)
 - physics_metadata.h, [206](#)
- physics_metadata_refresh_state
 - physics_metadata.c, [424](#)
 - physics_metadata.h, [207](#)
- physics_metadata_set_mass
 - physics_metadata.c, [425](#)
 - physics_metadata.h, [207](#)
- physics_metadata_touch
 - physics_metadata.c, [425](#)
 - physics_metadata.h, [207](#)
- physics_on_lookup
 - physics_execution_hooks.c, [418](#)
 - physics_execution_hooks.h, [201](#)
- physics_pipelining_diagnostic_words.c
 - forth_PIPELINING_ANALYZE_WORD, [723](#)
 - forth_PIPELINING_ENABLE, [723](#)
 - forth_PIPELINING_RESET_ALL, [723](#)
 - forth_PIPELINING_SHOW_STATS, [723](#)
 - forth_PIPELINING_SHOW_TOP_TRANSITIONS, [724](#)
 - forth_PIPELINING_STATS, [724](#)
 - register_physics_pipelining_diagnostic_words, [724](#)
- physics_pipelining_diagnostic_words.h
 - register_physics_pipelining_diagnostic_words, [695](#)
- physics_pipelining_metrics.c
 - ns_to_q48, [427](#)
 - q48_div_u64, [427](#)
 - q48_mul_q48, [427](#)
 - q48_to_ns, [427](#)
 - transition_metrics_binary_chop_suggest_window, [427](#)
 - transition_metrics_cleanup, [428](#)
 - transition_metrics_context_accuracy_string, [428](#)
 - transition_metrics_get_probability_q48, [428](#)
 - transition_metrics_init, [429](#)
 - transition_metrics_record, [429](#)
 - transition_metrics_record_context, [429](#)
 - transition_metrics_record_prefetch_hit, [430](#)
 - transition_metrics_record_prefetch_miss, [430](#)
 - transition_metrics_reset, [431](#)
 - transition_metrics_should_speculate, [431](#)
 - transition_metrics_stats_string, [431](#)
 - transition_metrics_update_cache, [432](#)
 - transition_metrics_update_context_window, [432](#)
- physics_pipelining_metrics.h
 - ENABLE_PIPELINING, [208](#)
 - MIN_SAMPLES_FOR_SPECULATION, [208](#)
 - MINIMUM_PREFETCH_ROI, [208](#)
 - MISPREDICTION_COST_Q48, [209](#)
 - SPECULATION_DEPTH, [209](#)
 - SPECULATION_THRESHOLD_Q48, [209](#)
 - transition_metrics_binary_chop_suggest_window, [209](#)
 - transition_metrics_cleanup, [210](#)
 - transition_metrics_context_accuracy_string, [210](#)
 - transition_metrics_get_probability_q48, [210](#)
 - transition_metrics_init, [211](#)
 - transition_metrics_record, [211](#)
 - transition_metrics_record_context, [212](#)
 - transition_metrics_record_prefetch_hit, [212](#)
 - transition_metrics_record_prefetch_miss, [212](#)
 - transition_metrics_reset, [213](#)
 - transition_metrics_should_speculate, [213](#)
 - transition_metrics_stats_string, [213](#)
 - transition_metrics_update_cache, [214](#)
 - transition_metrics_update_context_window, [214](#)
 - TRANSITION_WINDOW_SIZE, [209](#)
 - WordTransitionMetrics, [209](#)
- physics_post_execute
 - physics_execution_hooks.c, [418](#)
 - physics_execution_hooks.h, [201](#)
- physics_pre_execute
 - physics_execution_hooks.c, [419](#)
 - physics_execution_hooks.h, [201](#)
- physics_runtime.c
 - align_up, [434](#)
 - analytics_heap_allocate, [434](#)
 - analytics_heap_release, [434](#)
 - analytics_ring_free, [434](#)
 - g_runtime, [438](#)
 - parse_avg_milli, [434](#)
 - PATH_MAX, [433](#)
 - physics_analytics_heap_info, [434](#)
 - physics_analytics_publish_event, [435](#)
 - physics_host_snapshot, [435](#)
 - physics_runtime_init, [435](#)
 - physics_runtime_shutdown, [437](#)

- physics_runtime_state_t, [434](#)
- read_cgroup_cpu_usage_us, [437](#)
- read_cgroup_memory_current, [437](#)
- read_proc_stat_totals, [437](#)
- read_psi_file, [437](#)
- resolve_cgroup_path, [437](#)
- snapshot_posix, [438](#)
- physics_runtime.h
 - HOLA_PROTOCOL_MAJOR, [216](#)
 - HOLA_PROTOCOL_MINOR, [216](#)
 - HOLA_SHARED_MAGIC, [216](#)
 - PHYSICS_ANALYTICS_CHANNEL_HOST_SNAPSHOT, [216](#)
 - PHYSICS_ANALYTICS_DEFAULT_HEAP_BYTES, [216](#)
 - physics_analytics_event_header_t, [216](#)
 - physics_analytics_header_t, [216](#)
 - physics_analytics_heap_info, [217](#)
 - physics_analytics_heap_t, [217](#)
 - physics_analytics_publish_event, [217](#)
 - physics_host_backend, [217](#)
 - PHYSICS_HOST_BACKEND_FAKE, [217](#)
 - PHYSICS_HOST_BACKEND_L4RE, [217](#)
 - PHYSICS_HOST_BACKEND_POSIX, [217](#)
 - physics_host_backend_t, [217](#)
 - PHYSICS_HOST_FLAG_CGROUP, [216](#)
 - PHYSICS_HOST_FLAG_CPU_STAT, [216](#)
 - PHYSICS_HOST_FLAG_PSI, [216](#)
 - physics_host_snapshot, [218](#)
 - physics_host_snapshot_t, [217](#)
 - physics_runtime_init, [218](#)
 - physics_runtime_shutdown, [218](#)
- physics_runtime_init
 - physics_runtime.c, [435](#)
 - physics_runtime.h, [218](#)
- physics_runtime_shutdown
 - physics_runtime.c, [437](#)
 - physics_runtime.h, [218](#)
- physics_runtime_state, [119](#)
 - backend_seq, [119](#)
 - heap, [119](#)
 - heap_ready, [120](#)
- physics_runtime_state_t
 - physics_runtime.c, [434](#)
- physics_seed_definition, [120](#)
 - acl_hint, [120](#)
 - avg_latency_ns, [120](#)
 - name, [120](#)
 - pubsub_mask, [120](#)
 - temperature_q8, [120](#)
- physics_seed_definition_t
 - physics_metadata.c, [423](#)
- physics_seed_table
 - physics_metadata.c, [425](#)
- PHYSICS_STATE_COMPILED
 - vm.h, [315](#)
- PHYSICS_STATE_HIDDEN
 - vm.h, [315](#)
- PHYSICS_STATE_IMMEDIATE
 - vm.h, [315](#)
- PHYSICS_STATE_PINNED
 - vm.h, [315](#)
- PhysicsSnapshot, [120](#)
 - decay_slope_q48, [121](#)
 - eff_window_size, [121](#)
 - heat_25th, [121](#)
 - heat_50th, [121](#)
 - heat_75th, [121](#)
 - prefetch_attempts, [121](#)
 - tick_target_ns, [121](#)
 - total_executions, [121](#)
- pic_disable
 - interrupts.c, [477](#)
- pipeline_metrics
 - VM, [150](#)
- PipelineGlobalMetrics, [121](#)
 - last_checked_accuracy, [122](#)
 - last_checked_window_size, [122](#)
 - prefetch_attempts, [122](#)
 - prefetch_hits, [122](#)
 - suggested_next_size, [122](#)
 - window_tuning_checks, [122](#)
- pit_available
 - timer.c, [489](#)
- PIT_CHANNEL0_DATA
 - timer.c, [481](#)
- PIT_COMMAND
 - timer.c, [481](#)
- PIT_FREQ_HZ
 - timer.c, [481](#)
- pit_hz_mean
 - timer_calibration_record, [140](#)
- pit_probe_running
 - timer.c, [485](#)
- pit_program_reload
 - timer.c, [485](#)
- pit_read_counter16
 - timer.c, [486](#)
- pit_tsc_hz_mean
 - timer.c, [489](#)
- pixels_per_scanline
 - FramebufferInfo, [91](#)
- platform_alloc.h
 - sf_alloc_get_stats, [219](#)
 - sf_alloc_init, [219](#)
 - sf_alloc_stats_t, [219](#)
 - sf_calloc, [219](#)
 - sf_free, [219](#)
 - sf_malloc, [220](#)
 - sf_realloc, [220](#)
- platform_init.c
 - sf_time_backend, [445](#)
 - sf_time_init, [445](#)
- platform_lock.h
 - sf_mutex_destroy, [221](#)
 - sf_mutex_init, [221](#)

- sf_mutex_lock, [221](#)
- sf_mutex_t, [220](#)
- sf_mutex_unlock, [221](#)
- platform_time.h
 - sf_format_timestamp, [222](#)
 - sf_has_rtc, [222](#)
 - sf_monotonic_ns, [222](#)
 - sf_ns_to_ms, [223](#)
 - sf_ns_to_seconds, [223](#)
 - sf_ns_to_us, [223](#)
 - sf_realtime_ns, [223](#)
 - sf_seconds_to_ns, [223](#)
 - sf_set_realtime_ns, [223](#)
 - sf_time_backend, [223](#)
 - sf_time_backend_l4re, [223](#)
 - sf_time_backend_posix, [224](#)
 - sf_time_init, [223](#)
 - sf_time_ns_t, [222](#)
 - SF_TIME_STAMP_SIZE, [222](#)
- pml4_index
 - vmm.c, [533](#)
- pmm.c
 - __kernel_end, [529](#)
 - __kernel_start, [529](#)
 - bitmap_clear, [527](#)
 - bitmap_mark_range, [527](#)
 - bitmap_set, [527](#)
 - bitmap_test, [527](#)
 - find_contiguous_free, [527](#)
 - is_ram_type, [528](#)
 - is_usable_memory, [528](#)
 - pmm_alloc_contiguous, [528](#)
 - pmm_alloc_floor_page, [530](#)
 - pmm_alloc_page, [528](#)
 - pmm_bitmap, [530](#)
 - PMM_BITMAP_BITS, [527](#)
 - pmm_bytes_to_pages, [528](#)
 - pmm_free_contiguous, [528](#)
 - pmm_free_page, [528](#)
 - pmm_free_pages, [530](#)
 - pmm_get_stats, [529](#)
 - pmm_init, [529](#)
 - pmm_initialized, [530](#)
 - pmm_is_initialized, [529](#)
 - PMM_MAX_BITMAP_BYTES, [527](#)
 - pmm_memset, [529](#)
 - pmm_total_pages, [530](#)
 - pmm_tracked_pages, [530](#)
 - recompute_free_pages, [529](#)
- pmm.h
 - pmm_alloc_contiguous, [276](#)
 - pmm_alloc_page, [276](#)
 - pmm_free_contiguous, [276](#)
 - pmm_free_page, [276](#)
 - pmm_get_stats, [277](#)
 - pmm_init, [277](#)
 - pmm_is_initialized, [277](#)
 - PMM_PAGE_SIZE, [276](#)
 - pmm_alloc_contiguous
 - pmm.c, [528](#)
 - pmm.h, [276](#)
 - pmm_alloc_floor_page
 - pmm.c, [530](#)
 - pmm_alloc_page
 - pmm.c, [528](#)
 - pmm.h, [276](#)
 - pmm_bitmap
 - pmm.c, [530](#)
 - PMM_BITMAP_BITS
 - pmm.c, [527](#)
 - pmm_bytes_to_pages
 - pmm.c, [528](#)
 - pmm_free_contiguous
 - pmm.c, [528](#)
 - pmm.h, [276](#)
 - pmm_free_page
 - pmm.c, [528](#)
 - pmm.h, [276](#)
 - pmm_free_pages
 - pmm.c, [530](#)
 - pmm_get_stats
 - pmm.c, [529](#)
 - pmm.h, [277](#)
 - pmm_init
 - pmm.c, [529](#)
 - pmm.h, [277](#)
 - pmm_initialized
 - pmm.c, [530](#)
 - pmm_is_initialized
 - pmm.c, [529](#)
 - pmm.h, [277](#)
 - PMM_MAX_BITMAP_BYTES
 - pmm.c, [527](#)
 - pmm_memset
 - pmm.c, [529](#)
 - PMM_PAGE_SIZE
 - pmm.h, [276](#)
 - pmm_stats_t, [122](#)
 - free_bytes, [123](#)
 - free_pages, [123](#)
 - total_bytes, [123](#)
 - total_pages, [123](#)
 - used_bytes, [123](#)
 - used_pages, [123](#)
 - pmm_total_pages
 - pmm.c, [530](#)
 - pmm_tracked_pages
 - pmm.c, [530](#)
 - pmtimer_delta
 - timer.c, [486](#)
 - PMTIMER_FREQ_HZ
 - timer.c, [481](#)
 - PMTIMER_IO_PORT
 - timer.c, [481](#)
 - PMTIMER_MASK
 - timer.c, [481](#)

pmtimer_read
 timer.c, [486](#)
 PMTIMER_WIDTH_BITS
 timer.c, [481](#)
 pn_buf
 format_words.c, [678](#)
 pn_cap
 format_words.c, [678](#)
 pointer_within_heap
 kmallocc.c, [525](#)
 pos
 time_window, [139](#)
 posix_format_timestamp
 time.c, [444](#)
 posix_get_monotonic_ns
 time.c, [444](#)
 posix_get_realtime_ns
 time.c, [444](#)
 posix_has_rtc
 time.c, [444](#)
 posix_set_realtime_ns
 time.c, [445](#)
 post_dict_hash
 CapsuleRunRecord, [68](#)
 pre_dict_hash
 CapsuleRunRecord, [68](#)
 predicted_label_hits
 HeartbeatTickSnapshot, [98](#)
 prefetch_accuracy_percent
 DoeMetrics, [78](#)
 prefetch_attempts
 DoeMetrics, [78](#)
 InferenceInputs, [105](#)
 PhysicsSnapshot, [121](#)
 PipelineGlobalMetrics, [122](#)
 WordTransitionMetrics, [161](#)
 prefetch_hits
 DoeMetrics, [78](#)
 InferenceInputs, [105](#)
 PipelineGlobalMetrics, [122](#)
 WordTransitionMetrics, [161](#)
 prefetch_latency_saved_q48
 WordTransitionMetrics, [161](#)
 prefetch_misses
 WordTransitionMetrics, [161](#)
 present
 vmm_page_info_t, [155](#)
 prev
 heap_block, [93](#)
 prev_block
 blk_meta_t, [56](#)
 PRIM_2DROP
 vm_inner_interp_asm.h, [354](#)
 PRIM_2DROP_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_2DROP_RV64
 vm_inner_interp_riscv64.h, [360](#)
 PRIM_2DUP
 vm_inner_interp_asm.h, [354](#)
 PRIM_2DUP_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_2DUP_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_AND
 vm_inner_interp_asm.h, [354](#)
 PRIM_AND_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_AND_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_BRANCH
 vm_inner_interp_asm.h, [354](#)
 PRIM_BRANCH_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_BRANCH_OPT
 arch_detect.h, [166](#)
 PRIM_BRANCH_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_C_FETCH
 vm_inner_interp_asm.h, [354](#)
 PRIM_C_FETCH_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_C_FETCH_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_C_STORE
 vm_inner_interp_asm.h, [354](#)
 PRIM_C_STORE_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_C_STORE_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_DROP
 vm_inner_interp_asm.h, [355](#)
 PRIM_DROP_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_DROP_OPT
 arch_detect.h, [166](#)
 PRIM_DROP_RV64
 vm_inner_interp_riscv64.h, [361](#)
 PRIM_DUP
 vm_inner_interp_asm.h, [355](#)
 PRIM_DUP_ARM64
 Fast Primitive Word Implementations, [27](#)
 PRIM_DUP_OPT
 arch_detect.h, [166](#)
 PRIM_DUP_RV64
 vm_inner_interp_riscv64.h, [362](#)
 PRIM_EQUALS
 vm_inner_interp_asm.h, [355](#)
 PRIM_EQUALS_ARM64
 Fast Primitive Word Implementations, [28](#)
 PRIM_EQUALS_RV64
 vm_inner_interp_riscv64.h, [362](#)
 PRIM_EXECUTE
 vm_inner_interp_asm.h, [355](#)
 PRIM_EXECUTE_ARM64
 Fast Primitive Word Implementations, [28](#)
 PRIM_EXECUTE_RV64

- vm_inner_interp_riscv64.h, [362](#)
- PRIM_FETCH
 - vm_inner_interp_asm.h, [355](#)
- PRIM_FETCH_ARM64
 - Fast Primitive Word Implementations, [28](#)
- PRIM_FETCH_OPT
 - arch_detect.h, [166](#)
- PRIM_FETCH_RV64
 - vm_inner_interp_riscv64.h, [362](#)
- PRIM_GREATER
 - vm_inner_interp_asm.h, [355](#)
- PRIM_GREATER_ARM64
 - Fast Primitive Word Implementations, [28](#)
- PRIM_GREATER_RV64
 - vm_inner_interp_riscv64.h, [362](#)
- PRIM_INVERT
 - vm_inner_interp_asm.h, [356](#)
- PRIM_INVERT_ARM64
 - Fast Primitive Word Implementations, [28](#)
- PRIM_INVERT_RV64
 - vm_inner_interp_riscv64.h, [362](#)
- PRIM_LESS
 - vm_inner_interp_asm.h, [356](#)
- PRIM_LESS_ARM64
 - Fast Primitive Word Implementations, [28](#)
- PRIM_LESS_RV64
 - vm_inner_interp_riscv64.h, [362](#)
- PRIM_MINUS
 - vm_inner_interp_asm.h, [356](#)
- PRIM_MINUS_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_MINUS_OPT
 - arch_detect.h, [166](#)
- PRIM_MINUS_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_MOD
 - vm_inner_interp_asm.h, [356](#)
- PRIM_MOD_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_MOD_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_NEGATE
 - vm_inner_interp_asm.h, [356](#)
- PRIM_NEGATE_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_NEGATE_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_OR
 - vm_inner_interp_asm.h, [356](#)
- PRIM_OR_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_OR_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_OVER
 - vm_inner_interp_asm.h, [357](#)
- PRIM_OVER_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_OVER_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_PLUS
 - vm_inner_interp_asm.h, [357](#)
- PRIM_PLUS_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_PLUS_OPT
 - arch_detect.h, [167](#)
- PRIM_PLUS_RV64
 - vm_inner_interp_riscv64.h, [363](#)
- PRIM_R_FETCH
 - vm_inner_interp_asm.h, [357](#)
- PRIM_R_FETCH_ARM64
 - Fast Primitive Word Implementations, [29](#)
- PRIM_R_FETCH_OPT
 - arch_detect.h, [167](#)
- PRIM_R_FETCH_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_R_FROM
 - vm_inner_interp_asm.h, [357](#)
- PRIM_R_FROM_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_R_FROM_OPT
 - arch_detect.h, [167](#)
- PRIM_R_FROM_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_ROT
 - vm_inner_interp_asm.h, [357](#)
- PRIM_ROT_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_ROT_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_SLASH
 - vm_inner_interp_asm.h, [357](#)
- PRIM_SLASH_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_SLASH_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_STAR
 - vm_inner_interp_asm.h, [357](#)
- PRIM_STAR_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_STAR_OPT
 - arch_detect.h, [167](#)
- PRIM_STAR_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_STORE
 - vm_inner_interp_asm.h, [358](#)
- PRIM_STORE_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_STORE_OPT
 - arch_detect.h, [167](#)
- PRIM_STORE_RV64
 - vm_inner_interp_riscv64.h, [364](#)
- PRIM_SWAP
 - vm_inner_interp_asm.h, [358](#)
- PRIM_SWAP_ARM64
 - Fast Primitive Word Implementations, [30](#)
- PRIM_SWAP_OPT

arch_detect.h, [167](#)
 PRIM_SWAP_RV64
 vm_inner_interp_riscv64.h, [365](#)
 PRIM_TO_R
 vm_inner_interp_asm.h, [358](#)
 PRIM_TO_R_ARM64
 Fast Primitive Word Implementations, [30](#)
 PRIM_TO_R_OPT
 arch_detect.h, [167](#)
 PRIM_TO_R_RV64
 vm_inner_interp_riscv64.h, [365](#)
 PRIM_XOR
 vm_inner_interp_asm.h, [358](#)
 PRIM_XOR_ARM64
 Fast Primitive Word Implementations, [31](#)
 PRIM_XOR_RV64
 vm_inner_interp_riscv64.h, [365](#)
 PRIM_ZBRANCH
 vm_inner_interp_asm.h, [358](#)
 PRIM_ZBRANCH_ARM64
 Fast Primitive Word Implementations, [31](#)
 PRIM_ZBRANCH_OPT
 arch_detect.h, [167](#)
 PRIM_ZBRANCH_RV64
 vm_inner_interp_riscv64.h, [365](#)
 PRIM_ZERO_EQUALS
 vm_inner_interp_asm.h, [359](#)
 PRIM_ZERO_EQUALS_ARM64
 Fast Primitive Word Implementations, [31](#)
 PRIM_ZERO_EQUALS_RV64
 vm_inner_interp_riscv64.h, [365](#)
 PRIM_ZERO_LESS
 vm_inner_interp_asm.h, [359](#)
 PRIM_ZERO_LESS_ARM64
 Fast Primitive Word Implementations, [31](#)
 PRIM_ZERO_LESS_RV64
 vm_inner_interp_riscv64.h, [365](#)
 print_banner
 kernel_main.c, [519](#)
 print_boot_info
 kernel_main.c, [519](#)
 print_dec
 timer.c, [486](#)
 print_dec_u64
 interrupts.c, [477](#)
 print_heap_stats
 kernel_main.c, [519](#)
 print_hex64
 interrupts.c, [477](#)
 parity.c, [540](#)
 print_line_64
 editor_words.c, [672](#)
 print_module_summary
 test_common.c, [601](#)
 test_common.h, [559](#)
 print_nl
 parity.c, [540](#)
 print_number_formatted
 format_words.c, [677](#)
 print_pmm_stats
 kernel_main.c, [519](#)
 print_q48_16
 interrupts.c, [477](#)
 print_str
 parity.c, [540](#)
 print_u32
 parity.c, [540](#)
 print_uint
 kernel_main.c, [519](#)
 print_usage
 main.c, [414](#)
 print_word_name
 system_words.c, [747](#)
 print_word_name_safe
 profiler.c, [450](#)
 prng_next
 starforth_words.c, [735](#)
 probability_gt_10pct_q48
 SpeedupEstimate, [133](#)
 probability_gt_double_q48
 SpeedupEstimate, [133](#)
 produce_seq
 physics_analytics_header, [112](#)
 PROF_CATEGORY_COUNT
 profiler.h, [227](#)
 PROF_CONTROL_FLOW
 profiler.h, [227](#)
 PROF_DICT_ADD
 profiler.h, [226](#)
 PROF_DICT_LOOKUP
 profiler.h, [226](#)
 PROF_IO_OPS
 profiler.h, [227](#)
 PROF_MEMORY_ACCESS
 profiler.h, [226](#)
 PROF_STACK_OPS
 profiler.h, [226](#)
 PROF_VM_COMPILE
 profiler.h, [226](#)
 PROF_VM_EXECUTE
 profiler.h, [226](#)
 PROF_VM_INIT
 profiler.h, [226](#)
 PROF_WORD_EXEC
 profiler.h, [227](#)
 PROFILE_BASIC
 profiler.h, [227](#)
 PROFILE_DETAILED
 profiler.h, [227](#)
 PROFILE_DISABLED
 profiler.h, [227](#)
 PROFILE_ENABLED
 profiler.h, [225](#)
 PROFILE_END
 profiler.h, [225](#)
 PROFILE_FULL

- profiler.h, [227](#)
- PROFILE_INC_CYCLES
 - profiler.h, [225](#)
- PROFILE_INC_DICT_LOOKUP
 - profiler.h, [225](#)
- PROFILE_INC_STACK_OP
 - profiler.h, [226](#)
- PROFILE_MEMORY_READ
 - profiler.h, [226](#)
- PROFILE_MEMORY_WRITE
 - profiler.h, [226](#)
- PROFILE_START
 - profiler.h, [226](#)
- PROFILE_VERBOSE
 - profiler.h, [227](#)
- PROFILE_WORD_ENTER
 - profiler.h, [226](#)
- PROFILE_WORD_EXIT
 - profiler.h, [226](#)
- ProfileCategory
 - profiler.h, [226](#)
- ProfileLevel
 - profiler.h, [227](#)
- Profiler, [123](#)
 - branch_count, [124](#)
 - category_timers, [124](#)
 - compile_operations, [124](#)
 - counters, [124](#)
 - dict_lookups, [124](#)
 - instruction_count, [124](#)
 - level, [124](#)
 - loop_iterations, [124](#)
 - memory_allocations, [124](#)
 - memory_stats, [124](#)
 - profiling_overhead, [124](#)
 - stack_operations, [124](#)
 - total_execution_time, [124](#)
 - vm_cycles, [125](#)
 - word_profile_capacity, [125](#)
 - word_profile_count, [125](#)
 - word_profiles, [125](#)
- profiler.c
 - _POSIX_C_SOURCE, [448](#)
 - compare_by_calls, [449](#)
 - compare_by_time, [449](#)
 - current_start_time, [454](#)
 - current_word, [454](#)
 - DictEntry, [448](#)
 - enabled, [454](#)
 - g_profiler, [454](#)
 - get_time_ns, [449](#)
 - get_word_stats, [449](#)
 - level, [454](#)
 - print_word_name_safe, [450](#)
 - profiler_analyze_performance, [450](#)
 - profiler_end_timer, [450](#)
 - profiler_generate_report, [450](#)
 - profiler_get_time_ns, [450](#)
 - profiler_inc_branch, [451](#)
 - profiler_inc_dict_lookup, [451](#)
 - profiler_inc_loop_iter, [451](#)
 - profiler_inc_stack_op, [451](#)
 - profiler_inc_vm_cycles, [451](#)
 - profiler_init, [451](#)
 - profiler_memory_alloc, [451](#)
 - profiler_memory_read, [451](#)
 - profiler_memory_write, [451](#)
 - profiler_print_hotspots, [452](#)
 - profiler_print_memory_analysis, [452](#)
 - profiler_print_summary, [452](#)
 - profiler_print_word_analysis, [452](#)
 - profiler_reset, [452](#)
 - profiler_set_level, [452](#)
 - profiler_shutdown, [453](#)
 - profiler_start_timer, [453](#)
 - profiler_state, [454](#)
 - profiler_suggest_optimizations, [453](#)
 - profiler_word_count, [453](#)
 - profiler_word_enter, [453](#)
 - profiler_word_exit, [453](#)
 - total_dict_lookups, [454](#)
 - total_memory_reads, [454](#)
 - total_memory_writes, [454](#)
 - total_stack_ops, [454](#)
 - word_capacity, [454](#)
 - word_count, [455](#)
 - word_stats, [455](#)
- profiler.h
 - DictEntry, [226](#)
 - g_profiler, [231](#)
 - PROF_CATEGORY_COUNT, [227](#)
 - PROF_CONTROL_FLOW, [227](#)
 - PROF_DICT_ADD, [226](#)
 - PROF_DICT_LOOKUP, [226](#)
 - PROF_IO_OPS, [227](#)
 - PROF_MEMORY_ACCESS, [226](#)
 - PROF_STACK_OPS, [226](#)
 - PROF_VM_COMPILE, [226](#)
 - PROF_VM_EXECUTE, [226](#)
 - PROF_VM_INIT, [226](#)
 - PROF_WORD_EXEC, [227](#)
 - PROFILE_BASIC, [227](#)
 - PROFILE_DETAILED, [227](#)
 - PROFILE_DISABLED, [227](#)
 - PROFILE_ENABLED, [225](#)
 - PROFILE_END, [225](#)
 - PROFILE_FULL, [227](#)
 - PROFILE_INC_CYCLES, [225](#)
 - PROFILE_INC_DICT_LOOKUP, [225](#)
 - PROFILE_INC_STACK_OP, [226](#)
 - PROFILE_MEMORY_READ, [226](#)
 - PROFILE_MEMORY_WRITE, [226](#)
 - PROFILE_START, [226](#)
 - PROFILE_VERBOSE, [227](#)
 - PROFILE_WORD_ENTER, [226](#)
 - PROFILE_WORD_EXIT, [226](#)

- ProfileCategory, [226](#)
- ProfileLevel, [227](#)
- profiler_analyze_performance, [227](#)
- profiler_end_timer, [227](#)
- profiler_generate_report, [227](#)
- profiler_get_time_ns, [227](#)
- profiler_inc_branch, [228](#)
- profiler_inc_dict_lookup, [228](#)
- profiler_inc_loop_iter, [228](#)
- profiler_inc_stack_op, [228](#)
- profiler_inc_vm_cycles, [228](#)
- profiler_init, [228](#)
- profiler_memory_alloc, [228](#)
- profiler_memory_read, [229](#)
- profiler_memory_write, [229](#)
- profiler_print_hotspots, [229](#)
- profiler_print_memory_analysis, [229](#)
- profiler_print_summary, [229](#)
- profiler_print_word_analysis, [229](#)
- profiler_reset, [229](#)
- profiler_set_level, [229](#)
- profiler_shutdown, [230](#)
- profiler_start_timer, [230](#)
- profiler_suggest_optimizations, [230](#)
- profiler_word_count, [230](#)
- profiler_word_enter, [230](#)
- profiler_word_exit, [231](#)
- VM, [226](#)
- profiler_analyze_performance
 - profiler.c, [450](#)
 - profiler.h, [227](#)
- profiler_end_timer
 - profiler.c, [450](#)
 - profiler.h, [227](#)
- profiler_generate_report
 - profiler.c, [450](#)
 - profiler.h, [227](#)
- profiler_get_time_ns
 - profiler.c, [450](#)
 - profiler.h, [227](#)
- profiler_inc_branch
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_inc_dict_lookup
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_inc_loop_iter
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_inc_stack_op
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_inc_vm_cycles
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_init
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_memory_alloc
 - profiler.c, [451](#)
 - profiler.h, [228](#)
- profiler_memory_read
 - profiler.c, [451](#)
 - profiler.h, [229](#)
- profiler_memory_write
 - profiler.c, [451](#)
 - profiler.h, [229](#)
- profiler_print_hotspots
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_print_memory_analysis
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_print_summary
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_print_word_analysis
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_reset
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_set_level
 - profiler.c, [452](#)
 - profiler.h, [229](#)
- profiler_shutdown
 - profiler.c, [453](#)
 - profiler.h, [230](#)
- profiler_start_timer
 - profiler.c, [453](#)
 - profiler.h, [230](#)
- profiler_state
 - profiler.c, [454](#)
- profiler_suggest_optimizations
 - profiler.c, [453](#)
 - profiler.h, [230](#)
- profiler_word_count
 - profiler.c, [453](#)
 - profiler.h, [230](#)
- profiler_word_enter
 - profiler.c, [453](#)
 - profiler.h, [230](#)
- profiler_word_exit
 - profiler.c, [453](#)
 - profiler.h, [231](#)
- ProfileScope, [125](#)
 - category, [125](#)
 - word_name, [125](#)
- ProfileTimer, [125](#)
 - call_count, [126](#)
 - max_time, [126](#)
 - min_time, [126](#)
 - name, [126](#)
 - start_time, [126](#)
 - total_time, [126](#)
- profiling_overhead

- Profiler, [124](#)
- promotions
 - HotwordsStats, [103](#)
- psi_cpu_full_avg10_milli
 - physics_host_snapshot, [117](#)
- psi_cpu_full_avg300_milli
 - physics_host_snapshot, [117](#)
- psi_cpu_full_avg60_milli
 - physics_host_snapshot, [117](#)
- psi_cpu_some_avg10_milli
 - physics_host_snapshot, [117](#)
- psi_cpu_some_avg300_milli
 - physics_host_snapshot, [118](#)
- psi_cpu_some_avg60_milli
 - physics_host_snapshot, [118](#)
- psi_io_full_avg10_milli
 - physics_host_snapshot, [118](#)
- psi_io_full_avg300_milli
 - physics_host_snapshot, [118](#)
- psi_io_full_avg60_milli
 - physics_host_snapshot, [118](#)
- psi_io_some_avg10_milli
 - physics_host_snapshot, [118](#)
- psi_io_some_avg300_milli
 - physics_host_snapshot, [118](#)
- psi_io_some_avg60_milli
 - physics_host_snapshot, [118](#)
- psi_mem_some_avg10_milli
 - physics_host_snapshot, [118](#)
- psi_mem_some_avg300_milli
 - physics_host_snapshot, [118](#)
- psi_mem_some_avg60_milli
 - physics_host_snapshot, [119](#)
- PT_DYNAMIC
 - elf64.h, [270](#)
- pt_index
 - vmm.c, [533](#)
- PT_INTERP
 - elf64.h, [270](#)
- PT_LOAD
 - elf64.h, [270](#)
- PT_NOTE
 - elf64.h, [270](#)
- PT_NULL
 - elf64.h, [270](#)
- PT_PHDR
 - elf64.h, [270](#)
- PT_SHLIB
 - elf64.h, [270](#)
- PT_TLS
 - elf64.h, [270](#)
- PTE_ACCESSED
 - vmm.c, [531](#)
- PTE_DIRTY
 - vmm.c, [531](#)
- PTE_GLOBAL
 - vmm.c, [531](#)
- PTE_NX
 - vmm.c, [531](#)
- PTE_PCD
 - vmm.c, [531](#)
- PTE_PRESENT
 - vmm.c, [531](#)
- PTE_PS
 - vmm.c, [532](#)
- PTE_PWT
 - vmm.c, [532](#)
- PTE_USER
 - vmm.c, [532](#)
- PTE_WRITABLE
 - vmm.c, [532](#)
- published_ns
 - HeartbeatSnapshot, [94](#)
- published_tick
 - HeartbeatSnapshot, [94](#)
- pubsub_mask
 - DictPhysics, [73](#)
 - physics_seed_definition, [120](#)
- putc
 - VMHostServices, [154](#)
- puts
 - VMHostServices, [154](#)
- q48_16.c
 - q48_div, [520](#)
 - q48_exp_approx, [520](#)
 - q48_log_approx, [520](#)
 - q48_mul, [521](#)
 - q48_sqrt_approx, [521](#)
- q48_16.h
 - q48_16_t, [278](#), [285](#)
 - q48_abs, [278](#), [285](#)
 - q48_add, [278](#), [285](#)
 - q48_div, [279](#), [285](#)
 - q48_exp_approx, [279](#), [285](#)
 - q48_from_double, [279](#)
 - q48_from_u64, [280](#), [286](#)
 - q48_is_valid, [280](#)
 - q48_log_approx, [280](#), [286](#)
 - q48_mul, [281](#), [286](#)
 - Q48_ONE, [284](#)
 - q48_sqrt_approx, [281](#), [287](#)
 - q48_sub, [281](#), [287](#)
 - q48_to_double, [282](#)
 - q48_to_string, [282](#)
 - q48_to_u64, [282](#), [287](#)
- q48_16_t
 - q48_16.h, [278](#), [285](#)
- q48_16_words.c
 - q48_div, [725](#)
 - q48_exp_approx, [725](#)
 - q48_from_double, [726](#)
 - q48_is_valid, [726](#)
 - q48_log_approx, [726](#)
 - q48_mul, [727](#)
 - q48_sqrt_approx, [727](#)
 - q48_to_double, [728](#)

- q48_to_string, 728
- q48_abs
 - q48_16.h, 278, 285
- q48_add
 - q48_16.h, 278, 285
- q48_div
 - q48_16.c, 520
 - q48_16.h, 279, 285
 - q48_16_words.c, 725
- q48_div_u64
 - physics_pipelining_metrics.c, 427
- q48_exp_approx
 - q48_16.c, 520
 - q48_16.h, 279, 285
 - q48_16_words.c, 725
- q48_from_double
 - q48_16.h, 279
 - q48_16_words.c, 726
 - q48_stubs.c, 542
- q48_from_u64
 - q48_16.h, 280, 286
- q48_is_valid
 - q48_16.h, 280
 - q48_16_words.c, 726
- q48_log_approx
 - q48_16.c, 520
 - q48_16.h, 280, 286
 - q48_16_words.c, 726
- q48_mul
 - q48_16.c, 521
 - q48_16.h, 281, 286
 - q48_16_words.c, 727
- q48_mul_q48
 - physics_pipelining_metrics.c, 427
- Q48_ONE
 - q48_16.h, 284
- Q48_SCALE
 - math_portable.h, 198
- q48_sqrt_approx
 - q48_16.c, 521
 - q48_16.h, 281, 287
 - q48_16_words.c, 727
- q48_stubs.c
 - q48_from_double, 542
 - q48_to_double, 542
- q48_sub
 - q48_16.h, 281, 287
- q48_to_double
 - q48_16.h, 282
 - q48_16_words.c, 728
 - q48_stubs.c, 542
- q48_to_ns
 - physics_pipelining_metrics.c, 427
- q48_to_ns_double
 - physics_hotwords_cache.c, 422
- q48_to_string
 - q48_16.h, 282
 - q48_16_words.c, 728

- q48_to_u64
 - q48_16.h, 282, 287
- QueryMode
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51
- R_AARCH64_ABS64
 - elf64.h, 270
- R_AARCH64_NONE
 - elf64.h, 270
- R_AARCH64_RELATIVE
 - elf64.h, 270
- r_addend
 - Elf64_Rela, 88
- r_info
 - Elf64_Rel, 88
 - Elf64_Rela, 88
- r_offset
 - Elf64_Rel, 88
 - Elf64_Rela, 88
- R_RISCV_64
 - elf64.h, 270
- R_RISCV_NONE
 - elf64.h, 271
- R_RISCV_RELATIVE
 - elf64.h, 271
- R_X86_64_32
 - elf64.h, 271
- R_X86_64_32S
 - elf64.h, 271
- R_X86_64_64
 - elf64.h, 271
- R_X86_64_NONE
 - elf64.h, 271
- R_X86_64_PC32
 - elf64.h, 271
- R_X86_64_RELATIVE
 - elf64.h, 271
- RaiseTPL
 - _EFI_BOOT_SERVICES, 47
- ram_base
 - block_subsystem.c, 385
- ram_close
 - blkio_ram.c, 373
- ram_disk_mb
 - CLIConfig, 70
- ram_flush
 - blkio_ram.c, 373
- ram_info
 - blkio_ram.c, 374
- ram_open
 - blkio_ram.c, 374
- ram_read
 - blkio_ram.c, 374
- ram_size
 - block_subsystem.c, 385
- ram_user
 - block_subsystem.c, 385
- ram_write
 - blkio_ram.c, 374

RAW_LOG
 uefi_loader.c, 493
rdtsc
 timer.c, 486
rdtscp_aux
 timer.c, 486
Read
 _EFI_FILE_PROTOCOL, 48
read
 blkio_vtable_t, 63
read_cgroup_cpu_usage_us
 physics_runtime.c, 437
read_cgroup_memory_current
 physics_runtime.c, 437
read_devblock_4k
 block_subsystem.c, 383
read_header_4k
 block_subsystem.c, 383
read_offset
 physics_analytics_header, 112
read_only
 blkcfg_t, 59
 blkio_file_state_t, 60
 blkio_info_t, 61
 blkio_ram_state_t, 62
read_proc_stat_totals
 physics_runtime.c, 437
read_psi_file
 physics_runtime.c, 437
README.md, 367
reads
 MemoryProfile, 108
realtime_ns
 physics_host_snapshot, 119
recompute_free_pages
 pmm.c, 529
recycled_word_id_count
 VM, 151
recycled_word_ids
 VM, 151
register_acl_words
 acl_words.h, 679
register_arithmetic_words
 arithmetic_words.c, 642
 arithmetic_words.h, 679
register_block_words
 block_words.c, 647
 block_words.h, 685
register_control_words
 Runtime Branch Helpers, 38
register_defining_words
 defining_words.c, 655
 defining_words.h, 686
register_dictionary_heat_diagnostic_words
 dictionary_heat_diagnostic_words.c, 657
 dictionary_heat_diagnostic_words.h, 687
register_dictionary_manipulation_words
 dictionary_manipulation_words.c, 662
 dictionary_manipulation_words.h, 687
register_dictionary_words
 dictionary_words.c, 665
 dictionary_words.h, 688
register_double_words
 double_words.c, 670
 double_words.h, 688
register_editor_words
 editor_words.c, 672
 editor_words.h, 689
register_format_words
 format_words.c, 678
 format_words.h, 689
register_forth79_words
 word_registry.c, 638
 word_registry.h, 366
register_io_words
 io_words.c, 707
 io_words.h, 690
register_logical_words
 logical_words.c, 710
 logical_words.h, 691
register_memory_words
 memory_words.c, 714
 memory_words.h, 692
register_mixed_arithmetic_words
 mixed_arithmetic_words.c, 716
 mixed_arithmetic_words.h, 692
register_physics_benchmark_words
 physics_benchmark_words.c, 718
 physics_benchmark_words.h, 693
register_physics_diagnostic_words
 physics_diagnostic_words.c, 720
 physics_diagnostic_words.h, 694
register_physics_freeze_words
 physics_freeze_words.c, 722
 physics_freeze_words.h, 695
register_physics_pipelining_diagnostic_words
 physics_pipelining_diagnostic_words.c, 724
 physics_pipelining_diagnostic_words.h, 695
register_return_stack_words
 return_stack_words.c, 729
 return_stack_words.h, 696
register_stack_words
 stack_words.c, 731
 stack_words.h, 696
register_starforth_words
 starforth_words.c, 735
 starforth_words.h, 697
register_string_words
 string_words.c, 741
 string_words.h, 701
register_system_words
 system_words.c, 747
 system_words.h, 702
register_vocabulary_words
 vocabulary_words.c, 751
 vocabulary_words.h, 703

- register_word
 - word_registry.c, [638](#)
 - word_registry.h, [367](#)
- RegisterProtocolNotify
 - _EFI_BOOT_SERVICES, [47](#)
- ReinstallProtocolInterface
 - _EFI_BOOT_SERVICES, [47](#)
- reloc_block, [126](#)
 - block_size, [127](#)
 - entry, [127](#)
 - page_rva, [127](#)
- reloc_stub.c
 - __attribute__, [492](#)
 - block_size, [492](#)
 - entry, [492](#)
 - page_rva, [492](#)
- repl.c
 - __attribute__, [535](#)
 - sk_fault_handler, [536](#)
 - sk_readline, [536](#)
 - sk_repl, [537](#)
 - vm_repl, [535](#)
- repl.h
 - sk_repl, [289](#)
 - vm_repl, [288](#)
- report_file
 - break_me_tests.c, [581](#)
- requested
 - heap_block, [93](#)
- Reserved
 - _EFI_BOOT_SERVICES, [47](#)
 - EFI_LOADED_IMAGE_PROTOCOL, [82](#)
 - EFI_TABLE_HEADER, [84](#)
- reserved
 - CapsuleDirHeader, [67](#)
 - CapsuleRunRecord, [68](#)
 - physics_analytics_event_header, [110](#)
 - TestModule, [137](#)
 - timer_calibration_record, [140](#)
 - VMRegistryEntry, [156](#)
- reserved1
 - blk_meta_t, [56](#)
- reserved_disk_lo
 - blk_volume_meta_t, [58](#)
- reserved_int
 - TestModule, [137](#)
- reserved_ram_lo
 - blk_volume_meta_t, [58](#)
- Reset
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, [51](#)
- reset_vm_state
 - system_words.c, [747](#)
- ResetSystem
 - _EFI_RUNTIME_SERVICES, [50](#)
- resolve_cgroup_path
 - physics_runtime.c, [437](#)
- resolve_contract
 - test_common.c, [601](#)
- resolve_pt
 - vmm.c, [534](#)
- restore_dict_state
 - test_common.c, [602](#)
 - test_common.h, [559](#)
- restore_physics
 - test_contracts.c, [606](#)
 - test_contracts.h, [565](#)
- restore_vm_state
 - test_common.c, [602](#)
 - test_common.h, [560](#)
- RestoreTPL
 - _EFI_BOOT_SERVICES, [47](#)
- result_code
 - CapsuleRunRecord, [68](#)
- return_stack
 - VM, [151](#)
- return_stack_word_r_fetch
 - return_stack_words.c, [729](#)
- return_stack_word_r_from
 - return_stack_words.c, [729](#)
- return_stack_word_suites
 - return_stack_words_test.c, [592](#)
- return_stack_word_to_r
 - return_stack_words.c, [730](#)
- return_stack_words.c
 - register_return_stack_words, [729](#)
 - return_stack_word_r_fetch, [729](#)
 - return_stack_word_r_from, [729](#)
 - return_stack_word_to_r, [730](#)
- return_stack_words.h
 - register_return_stack_words, [696](#)
- return_stack_words_test.c
 - return_stack_word_suites, [592](#)
 - run_return_stack_words_tests, [592](#)
- Revision
 - _EFI_FILE_PROTOCOL, [49](#)
 - _EFI_SIMPLE_FILE_SYSTEM_PROTOCOL, [50](#)
 - EFI_TABLE_HEADER, [84](#)
- ring
 - physics_analytics_heap, [114](#)
- ring_bytes
 - physics_analytics_header, [112](#)
 - physics_analytics_heap, [114](#)
- ring_offset
 - physics_analytics_header, [112](#)
- rng_seed
 - test_contracts.c, [606](#)
- rng_state
 - test_contracts.c, [607](#)
- rolling_window
 - VM, [151](#)
- rolling_window_check_adaptive_shrink
 - rolling_window_of_truth.c, [456](#)
 - rolling_window_of_truth.h, [233](#)
- rolling_window_cleanup
 - rolling_window_of_truth.c, [456](#)
 - rolling_window_of_truth.h, [233](#)

- rolling_window_count_transition
 - rolling_window_of_truth.c, 456
 - rolling_window_of_truth.h, 233
- rolling_window_export_execution_history
 - rolling_window_of_truth.c, 457
 - rolling_window_of_truth.h, 234
- rolling_window_find_hottest_word
 - rolling_window_of_truth.c, 457
 - rolling_window_of_truth.h, 234
- rolling_window_get_recent_sequence
 - rolling_window_of_truth.c, 458
 - rolling_window_of_truth.h, 234
- rolling_window_init
 - rolling_window_of_truth.c, 458
 - rolling_window_of_truth.h, 235
- rolling_window_is_warm
 - rolling_window_of_truth.c, 459
 - rolling_window_of_truth.h, 235
- rolling_window_knobs.h
 - ADAPTIVE_CHECK_FREQUENCY, 232
 - ADAPTIVE_GROWTH_THRESHOLD, 232
 - ADAPTIVE_MIN_WINDOW_SIZE, 232
 - ADAPTIVE_SHRINK_RATE, 232
- rolling_window_measure_diversity
 - rolling_window_of_truth.c, 459
 - rolling_window_of_truth.h, 236
- rolling_window_measure_diversity_view
 - rolling_window_of_truth.c, 459
- rolling_window_of_truth.c
 - rolling_window_check_adaptive_shrink, 456
 - rolling_window_cleanup, 456
 - rolling_window_count_transition, 456
 - rolling_window_export_execution_history, 457
 - rolling_window_find_hottest_word, 457
 - rolling_window_get_recent_sequence, 458
 - rolling_window_init, 458
 - rolling_window_is_warm, 459
 - rolling_window_measure_diversity, 459
 - rolling_window_measure_diversity_view, 459
 - rolling_window_pattern_capture_rate, 459
 - rolling_window_publish_snapshot, 460
 - rolling_window_publish_snapshot_if_needed, 460
 - rolling_window_record_execution, 460
 - rolling_window_reset, 461
 - rolling_window_run_adaptive_pass, 461
 - rolling_window_seed_hotwords_cache, 461
 - rolling_window_seed_pipelining_context, 461
 - rolling_window_service, 462
 - rolling_window_service_internal, 462
 - rolling_window_snapshot_view, 462
 - rolling_window_stats_string, 462
- rolling_window_of_truth.h
 - rolling_window_check_adaptive_shrink, 233
 - rolling_window_cleanup, 233
 - rolling_window_count_transition, 233
 - rolling_window_export_execution_history, 234
 - rolling_window_find_hottest_word, 234
 - rolling_window_get_recent_sequence, 234
 - rolling_window_init, 235
 - rolling_window_is_warm, 235
 - rolling_window_measure_diversity, 236
 - rolling_window_pattern_capture_rate, 236
 - rolling_window_record_execution, 237
 - rolling_window_reset, 237
 - rolling_window_service, 238
 - rolling_window_stats_string, 238
- rolling_window_pattern_capture_rate
 - rolling_window_of_truth.c, 459
 - rolling_window_of_truth.h, 236
- rolling_window_publish_snapshot
 - rolling_window_of_truth.c, 460
- rolling_window_publish_snapshot_if_needed
 - rolling_window_of_truth.c, 460
- rolling_window_record_execution
 - rolling_window_of_truth.c, 460
 - rolling_window_of_truth.h, 237
- rolling_window_reset
 - rolling_window_of_truth.c, 461
 - rolling_window_of_truth.h, 237
- rolling_window_run_adaptive_pass
 - rolling_window_of_truth.c, 461
- rolling_window_seed_hotwords_cache
 - rolling_window_of_truth.c, 461
- rolling_window_seed_pipelining_context
 - rolling_window_of_truth.c, 461
- rolling_window_service
 - rolling_window_of_truth.c, 462
 - rolling_window_of_truth.h, 238
- rolling_window_service_internal
 - rolling_window_of_truth.c, 462
- ROLLING_WINDOW_SIZE
 - starforth_config.h, 243
- rolling_window_size
 - DoeMetrics, 78
- rolling_window_snapshot_view
 - rolling_window_of_truth.c, 462
- rolling_window_stats_string
 - rolling_window_of_truth.c, 462
 - rolling_window_of_truth.h, 238
- rolling_window_view_t, 127
 - effective_window_size, 127
 - history, 127
 - is_warm, 127
 - total_executions, 127
 - window_pos, 128
- rolling_window_width
 - DoeMetrics, 78
- RollingWindowOfTruth, 128
 - adaptive_check_accumulator, 128
 - adaptive_pending, 128
 - effective_window_size, 128
 - execution_history, 128
 - is_warm, 129
 - last_pattern_diversity, 129
 - pattern_diversity_check_count, 129
 - snapshot_buffers, 129

- snapshot_effective_window_size, 129
 - snapshot_index, 129
 - snapshot_is_warm, 129
 - snapshot_pending, 129
 - snapshot_total_executions, 129
 - snapshot_window_pos, 129
 - total_executions, 129
 - window_pos, 130
- rsp
 - ExecResult, 91
 - VM, 151
- run_acl_words_tests
 - acl_words_test.c, 577
 - test_runner.h, 568
- run_all_tests
 - test_runner.c, 608
 - test_runner.h, 568
- run_arithmetic_words_tests
 - arithmetic_words_test.c, 579
 - test_runner.h, 568
- run_benchmark_mode
 - main.c, 414
- run_block_words_tests
 - block_words_test.c, 579
 - test_runner.h, 568
- run_break_me_tests
 - break_me_tests.c, 581
 - test_runner.h, 569
- run_control_words_tests
 - control_words_test.c, 582
 - test_runner.h, 569
- run_defining_words_tests
 - defining_words_tests.c, 583
 - test_runner.h, 569
- run_dictionary_manipulation_words_tests
 - dictionary_manipulation_words_test.c, 583
 - test_runner.h, 569
- run_dictionary_words_tests
 - dictionary_words_test.c, 584
 - test_runner.h, 570
- run_doe_experiment
 - main.c, 414
- run_double_words_tests
 - double_words_test.c, 585
 - test_runner.h, 570
- run_format_words_tests
 - format_words_test.c, 586
 - test_runner.h, 571
- run_id
 - CapsuleRunRecord, 68
- run_id_counter
 - capsule_run.c, 508
- run_integration_tests
 - integration_tests.c, 586
 - test_runner.h, 571
- run_io_words_tests
 - io_words_test.c, 588
 - test_runner.h, 571
- run_log
 - capsule_run.c, 508
- run_log_count
 - capsule_run.c, 508
- run_log_head
 - capsule_run.c, 508
- run_logical_words_tests
 - logical_words_test.c, 589
 - test_runner.h, 571
- run_mama_forth_words_tests
 - mama_forth_words_test.c, 590
 - test_runner.h, 572
- run_memory_words_tests
 - memory_words_test.c, 591
 - test_runner.h, 572
- run_mixed_arithmetic_words_tests
 - mixed_arithmetic_words_test.c, 592
 - test_runner.h, 572
- run_module_tests
 - test_runner.c, 609
 - test_runner.h, 573
 - TestModule, 137
- run_optional_tests
 - main.c, 414
- run_return_stack_words_tests
 - return_stack_words_test.c, 592
 - test_runner.h, 573
- run_single_test
 - test_common.c, 602
 - test_common.h, 560
- run_stack_words_tests
 - stack_words_test.c, 593
 - test_runner.h, 573
- run_starforth_words_tests
 - starforth_words_test.c, 594
 - test_runner.h, 574
- run_start_ns
 - HeartbeatState, 96
- run_stress_tests
 - stress_tests.c, 595
 - test_runner.h, 574
- run_string_words_tests
 - string_words_test.c, 596
 - test_runner.h, 574
- run_system_words_tests
 - system_words_test.c, 597
 - test_runner.h, 575
- run_test_suite
 - test_common.c, 603
 - test_common.h, 561
- run_test_suite_m
 - test_common.c, 603
 - test_common.h, 562
- run_tests
 - CLIConfig, 70
- run_tests_flag_observed
 - CLIConfig, 70
- run_vocabulary_words_tests

- test_runner.h, 575
- vocabulary_words_test.c, 598
- run_word_tests
 - test_runner.c, 609
 - test_runner.h, 575
- runnable_threads
 - physics_host_snapshot, 119
- running
 - HeartbeatWorker, 99
- running_under_hypervisor
 - timer.c, 486
- Runtime Architecture Detection, 15
- Runtime Branch Helpers, 32
 - addr, 38
 - control_forth_0branch, 33
 - control_forth_again, 33
 - control_forth_begin, 34
 - control_forth_branch, 34
 - control_forth_case, 34
 - control_forth_do, 34
 - control_forth_else, 34
 - control_forth_endcase, 35
 - control_forth_endof, 35
 - control_forth_EXIT, 35
 - control_forth_I, 35
 - control_forth_if, 35
 - control_forth_J, 35
 - control_forth_leave, 36
 - control_forth_loop, 36
 - control_forth_of, 36
 - control_forth_plus_loop, 36
 - control_forth_qdo, 36
 - control_forth_repeat, 36
 - control_forth_runtime_do, 37
 - control_forth_runtime_leave, 37
 - control_forth_runtime_loop, 37
 - control_forth_runtime_plus_loop, 37
 - control_forth_runtime_qdo, 37
 - control_forth_then, 37
 - control_forth_UNLOOP, 38
 - control_forth_until, 38
 - control_forth_while, 38
 - register_control_words, 38
 - tag, 39
- runtime_abortq
 - system_words.c, 747
- runtime_services
 - BootInfo, 65
- RuntimeServices
 - _EFI_SYSTEM_TABLE, 53
- sample_count
 - BayesianLatencyPosterior, 54
- SAMPLE_WINDOWS
 - timer.c, 481
- sanity_check_hpet_running_quick
 - timer.c, 486
- save_all_buffers
 - block_words.c, 647
- block_words.h, 685
- save_dict_state
 - test_common.c, 604
 - test_common.h, 562
- save_physics
 - test_contracts.c, 607
 - test_contracts.h, 565
- save_vm_state
 - test_common.c, 604
 - test_common.h, 562
- scheduler_policy
 - physics_host_snapshot, 119
- scheduler_priority
 - physics_host_snapshot, 119
- scheduler_quantum_ns
 - physics_host_snapshot, 119
- scr_addr
 - VM, 151
- scratch
 - physics_analytics_heap, 114
- scratch_bytes
 - physics_analytics_header, 112
 - physics_analytics_heap, 114
- scratch_offset
 - physics_analytics_header, 112
- script_mode
 - CLConfig, 70
- Second
 - EFI_TIME, 85
- seed
 - XXHash64State, 163
- selector
 - idt_entry, 104
 - interrupts.c, 478
- SERIAL_SUPPORTED
 - console.c, 512
- set_forth_79_compliance
 - system_words.c, 747
- set_idt_entry
 - interrupts.c, 477
- set_realtime_ns
 - sf_time_backend_t, 132
- set_scr
 - block_words.c, 647
 - editor_words.c, 672
- SetAttribute
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51
- SetCursorPosition
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51
- SetInfo
 - _EFI_FILE_PROTOCOL, 49
- SetMode
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 51
- SetPosition
 - _EFI_FILE_PROTOCOL, 49
- SetTime
 - _EFI_RUNTIME_SERVICES, 50
- SetTimer

- [_EFI_BOOT_SERVICES](#), 47
- [SetVariable](#)
 - [_EFI_RUNTIME_SERVICES](#), 50
- [SetVirtualAddressMap](#)
 - [_EFI_RUNTIME_SERVICES](#), 50
- [SetWakeupTime](#)
 - [_EFI_RUNTIME_SERVICES](#), 50
- [SetWatchdogTimer](#)
 - [_EFI_BOOT_SERVICES](#), 47
- [SF_ALIGN](#)
 - [alloc_kernel.c](#), 440, 442
- [SF_ALIGN_UP](#)
 - [alloc_kernel.c](#), 440, 442
- [sf_alloc_get_stats](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 440, 442
 - [platform_alloc.h](#), 219
- [sf_alloc_init](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 440, 442
 - [platform_alloc.h](#), 219
- [sf_alloc_stats](#), 130
 - [alloc_count](#), 130
 - [free_count](#), 130
 - [peak_bytes](#), 130
 - [total_bytes](#), 130
 - [used_bytes](#), 130
- [sf_alloc_stats_t](#)
 - [platform_alloc.h](#), 219
- [SF_ARENA_SIZE](#)
 - [alloc_kernel.c](#), 440, 442
- [sf_cached_latest](#)
 - [dictionary_management.c](#), 396
- [sf_calloc](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 440, 442
 - [platform_alloc.h](#), 219
- [SF_FC_BUCKETS](#)
 - [dictionary_heat_optimization.h](#), 181
 - [dictionary_management.c](#), 391
 - [vocabulary_words.c](#), 751
- [sf_fc_cap](#)
 - [dictionary_heat_optimization.h](#), 182
 - [dictionary_management.c](#), 396
- [sf_fc_clear_all](#)
 - [dictionary_management.c](#), 391
- [sf_fc_count](#)
 - [dictionary_heat_optimization.h](#), 182
 - [dictionary_management.c](#), 396
 - [physics_benchmark_words.c](#), 719
- [sf_fc_list](#)
 - [dictionary_heat_optimization.h](#), 182
 - [dictionary_management.c](#), 396
 - [physics_benchmark_words.c](#), 719
- [sf_fc_reserve](#)
 - [dictionary_management.c](#), 391
- [sf_format_timestamp](#)
 - [platform_time.h](#), 222
- [sf_free](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 440, 442
 - [platform_alloc.h](#), 219
- [sf_has_rtc](#)
 - [platform_time.h](#), 222
- [sf_malloc](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 440, 442
 - [platform_alloc.h](#), 220
- [sf_monotonic_ns](#)
 - [platform_time.h](#), 222
- [sf_mutex](#), 131
 - [handle](#), 131
- [sf_mutex_destroy](#)
 - [platform_lock.h](#), 221
 - [threading.c](#), 446
- [sf_mutex_init](#)
 - [platform_lock.h](#), 221
 - [threading.c](#), 446
- [sf_mutex_lock](#)
 - [platform_lock.h](#), 221
 - [threading.c](#), 446
- [sf_mutex_t](#)
 - [platform_lock.h](#), 220
- [sf_mutex_unlock](#)
 - [platform_lock.h](#), 221
 - [threading.c](#), 446
- [sf_ns_to_ms](#)
 - [platform_time.h](#), 223
- [sf_ns_to_seconds](#)
 - [platform_time.h](#), 223
- [sf_ns_to_us](#)
 - [platform_time.h](#), 223
- [sf_realloc](#)
 - [alloc_host.c](#), 439
 - [alloc_kernel.c](#), 441, 443
 - [platform_alloc.h](#), 220
- [sf_realtime_ns](#)
 - [platform_time.h](#), 223
- [sf_rebuild_fc_index](#)
 - [dictionary_management.c](#), 391
- [sf_seconds_to_ns](#)
 - [platform_time.h](#), 223
- [sf_set_realtime_ns](#)
 - [platform_time.h](#), 223
- [sf_time_backend](#)
 - [platform_init.c](#), 445
 - [platform_time.h](#), 223
- [sf_time_backend_l4re](#)
 - [platform_time.h](#), 223
 - [time.c](#), 444
- [sf_time_backend_posix](#)
 - [platform_time.h](#), 224
 - [time.c](#), 445
- [sf_time_backend_t](#), 131
 - [format_timestamp](#), 131
 - [get_monotonic_ns](#), 131

- get_realtime_ns, [132](#)
- has_rtc, [132](#)
- set_realtime_ns, [132](#)
- sf_time_init
 - platform_init.c, [445](#)
 - platform_time.h, [223](#)
- sf_time_init_l4re
 - time.c, [443](#)
- sf_time_ns_t
 - platform_time.h, [222](#)
- SF_TIME_STAMP_SIZE
 - platform_time.h, [222](#)
- sf_try_fast_append
 - dictionary_management.c, [392](#)
- sf_xrealloc
 - dictionary_management.c, [392](#)
- sh_addr
 - Elf64_Shdr, [89](#)
- sh_addralign
 - Elf64_Shdr, [89](#)
- sh_entsize
 - Elf64_Shdr, [89](#)
- sh_flags
 - Elf64_Shdr, [89](#)
- sh_info
 - Elf64_Shdr, [89](#)
- sh_link
 - Elf64_Shdr, [89](#)
- sh_name
 - Elf64_Shdr, [89](#)
- sh_offset
 - Elf64_Shdr, [89](#)
- sh_size
 - Elf64_Shdr, [89](#)
- sh_type
 - Elf64_Shdr, [89](#)
- should_error
 - TestCase, [136](#)
- SHT_DYNAMIC
 - elf64.h, [271](#)
- SHT_DYNSYM
 - elf64.h, [271](#)
- SHT_HASH
 - elf64.h, [271](#)
- SHT_NOBITS
 - elf64.h, [271](#)
- SHT_NOTE
 - elf64.h, [271](#)
- SHT_NULL
 - elf64.h, [271](#)
- SHT_PROGBITS
 - elf64.h, [271](#)
- SHT_REL
 - elf64.h, [272](#)
- SHT_RELA
 - elf64.h, [272](#)
- SHT_SHLIB
 - elf64.h, [272](#)
- SHT_STRTAB
 - elf64.h, [272](#)
- SHT_SYMTAB
 - elf64.h, [272](#)
- sig_handler
 - vm_debug.c, [624](#)
- signal_handler
 - main.c, [414](#)
- SignalEvent
 - _EFI_BOOT_SERVICES, [47](#)
- Signature
 - EFI_TABLE_HEADER, [84](#)
- signature
 - blk_meta_t, [56](#)
- Size
 - EFI_FILE_INFO, [81](#)
- size
 - FramebufferInfo, [91](#)
 - heap_block, [93](#)
- size_bytes
 - blkcfg_t, [59](#)
- SK_ALLOC
 - vm_host.h, [349](#)
- SK_ALLOC_ALIGNED
 - vm_host.h, [349](#)
- sk_alloc_fn
 - vm_host.h, [350](#)
- SK_BOOTSTRAP_ARENA_FAIL
 - parity.h, [302](#)
- SK_BOOTSTRAP_DICT_FAIL
 - parity.h, [302](#)
- SK_BOOTSTRAP_INIT_FAIL
 - parity.h, [302](#)
- SK_BOOTSTRAP_OK
 - parity.h, [302](#)
- sk_dict_canonical_hash
 - parity.c, [540](#)
 - parity.h, [303](#)
- sk_dict_word_count
 - parity.c, [541](#)
 - parity.h, [303](#)
- sk_fault_handler
 - repl.c, [536](#)
- SK_FREE
 - vm_host.h, [349](#)
- sk_free_fn
 - vm_host.h, [350](#)
- sk_host_services
 - vm_host.h, [351](#)
- SK_MUTEX_DESTROY
 - vm_host.h, [349](#)
- sk_mutex_destroy_fn
 - vm_host.h, [350](#)
- SK_MUTEX_INIT
 - vm_host.h, [349](#)
- sk_mutex_init_fn
 - vm_host.h, [350](#)
- SK_MUTEX_LOCK

- vm_host.h, 349
- sk_mutex_lock_fn
 - vm_host.h, 350
- SK_MUTEX_UNLOCK
 - vm_host.h, 349
- sk_mutex_unlock_fn
 - vm_host.h, 350
- sk_panic_fn
 - vm_host.h, 350
- sk_parity_collect
 - parity.c, 541
 - parity.h, 304
- SK_PARITY_DEBUG
 - parity.c, 538
 - vm_internal.h, 626
- sk_parity_debug_check_ptr
 - parity.c, 538
- sk_parity_debug_log_msg
 - parity.c, 538
- sk_parity_print
 - parity.c, 541
 - parity.h, 304
- SK_PUTC
 - vm_host.h, 349
- sk_putc_fn
 - vm_host.h, 350
- SK_PUTS
 - vm_host.h, 349
- sk_puts_fn
 - vm_host.h, 350
- sk_readline
 - repl.c, 536
- sk_repl
 - repl.c, 537
 - repl.h, 289
- sk_time_fn
 - vm_host.h, 350
- SK_TIME_NS
 - vm_host.h, 349
- sk_xt_entry_owns_fn
 - vm_host.h, 350
- sk_xt_validate_fn
 - vm_host.h, 350
- slope_fit_quality_q48
 - InferenceOutputs, 106
- snapshot_buffers
 - RollingWindowOfTruth, 129
- snapshot_effective_window_size
 - RollingWindowOfTruth, 129
- snapshot_index
 - HeartbeatState, 96
 - RollingWindowOfTruth, 129
- snapshot_is_warm
 - RollingWindowOfTruth, 129
- snapshot_pending
 - RollingWindowOfTruth, 129
- snapshot_posix
 - physics_runtime.c, 438
- snapshot_total_executions
 - RollingWindowOfTruth, 129
- snapshot_window_pos
 - RollingWindowOfTruth, 129
- snapshots
 - HeartbeatState, 96
- span_var
 - VM, 151
- SPECULATION_DEPTH
 - physics_pipelining_metrics.h, 209
- SPECULATION_THRESHOLD_Q48
 - physics_pipelining_metrics.h, 209
- speedup_factor_q48
 - SpeedupEstimate, 133
- speedup_vs_baseline
 - DoeMetrics, 78
- SpeedupEstimate, 132
 - credible_lower_95_q48, 133
 - credible_upper_95_q48, 133
 - probability_gt_10pct_q48, 133
 - probability_gt_double_q48, 133
 - speedup_factor_q48, 133
- SPIN_IDLE
 - vm.h, 315
- sqrt_q48
 - math_portable.c, 416
 - math_portable.h, 198
- src/blkio_factory.c, 367
- src/blkio_file.c, 369
- src/blkio_ram.c, 372
- src/block_subsystem.c, 374
- src/cli.c, 387
- src/dictionary_heat_optimization.c, 388
- src/dictionary_management.c, 390
- src/doe_metrics.c, 396
- src/heartbeat_export.c, 400
- src/inference_engine.c, 401
- src/io.c, 407
- src/log.c, 408
- src/main.c, 412
- src/math_portable.c, 415
- src/memory_management.c, 416
- src/physics_execution_hooks.c, 418
- src/physics_hotwords_cache.c, 419
- src/physics_metadata.c, 423
- src/physics_pipelining_metrics.c, 426
- src/physics_runtime.c, 432
- src/platform/alloc_host.c, 438
- src/platform/alloc_kernel.c, 439
- src/platform/l4re/time.c, 443
- src/platform/linux/time.c, 444
- src/platform/platform_init.c, 445
- src/platform/threading.c, 446
- src/profiler.c, 446
- src/repl.c, 535
- src/rolling_window_of_truth.c, 455
- src/ssm_jacquard.c, 463
- src/stack_management.c, 464

- src/starkernel/arch/amd64/apic.c, 467
- src/starkernel/arch/amd64/arch.c, 472
- src/starkernel/arch/amd64/interrupts.c, 475
- src/starkernel/arch/amd64/timer.c, 478
- src/starkernel/boot/elf_loader.c, 490
- src/starkernel/boot/reloc_stub.c, 491
- src/starkernel/boot/uefi_loader.c, 493
- src/starkernel/capsule/capsule_birth.c, 494
- src/starkernel/capsule/capsule_find.c, 499
- src/starkernel/capsule/capsule_loader.c, 501
- src/starkernel/capsule/capsule_run.c, 505
- src/starkernel/capsule/capsule_validate.c, 508
- src/starkernel/capsule/capsule_vm_hooks.c, 510
- src/starkernel/capsule/mama_forth_words.c, 511
- src/starkernel/hal/console.c, 511
- src/starkernel/hal/hal.c, 513
- src/starkernel/hal/host_services.c, 513
- src/starkernel/hal/memory.c, 513
- src/starkernel/hash/xxhash64.c, 514
- src/starkernel/kernel_main.c, 517
- src/starkernel/math/q48_16.c, 519
- src/starkernel/memory/kmalloc.c, 522
- src/starkernel/memory/pmm.c, 526
- src/starkernel/memory/vmm.c, 530
- src/starkernel/repl.c, 536
- src/starkernel/vm/alloc_kernel.c, 441
- src/starkernel/vm/arena.c, 537
- src/starkernel/vm/bootstrap/sk_vm_bootstrap.c, 537
- src/starkernel/vm/host/shim.c, 537
- src/starkernel/vm/parity.c, 537
- src/starkernel/vm/q48_stubs.c, 542
- src/starkernel/vm/vm_bootstrap.c, 619
- src/starkernel/vm/vm_core.c, 542
- src/starkernel/vm/vm_internal.h, 625
- src/starkernel/vm/vm_runtime.c, 551
- src/test_runner/include/test_common.h, 554
- src/test_runner/include/test_contracts.h, 563
- src/test_runner/include/test_runner.h, 566
- src/test_runner/modules/acl_words_test.c, 576
- src/test_runner/modules/arithmetic_words_test.c, 578
- src/test_runner/modules/block_words_test.c, 579
- src/test_runner/modules/break_me_tests.c, 580
- src/test_runner/modules/control_words_test.c, 582
- src/test_runner/modules/defining_words_tests.c, 582
- src/test_runner/modules/dictionary_manipulation_words_test.c, 583
- src/test_runner/modules/dictionary_words_test.c, 584
- src/test_runner/modules/double_words_test.c, 585
- src/test_runner/modules/format_words_test.c, 585
- src/test_runner/modules/integration_tests.c, 586
- src/test_runner/modules/io_words_test.c, 588
- src/test_runner/modules/logical_words_test.c, 589
- src/test_runner/modules/mama_forth_words_test.c, 589
- src/test_runner/modules/memory_words_test.c, 591
- src/test_runner/modules/mixed_arithmetic_words_test.c, 591
- src/test_runner/modules/return_stack_words_test.c, 592
- src/test_runner/modules/stack_words_test.c, 593
- src/test_runner/modules/starforth_words_test.c, 593
- src/test_runner/modules/stress_tests.c, 594
- src/test_runner/modules/string_words_test.c, 596
- src/test_runner/modules/system_words_test.c, 597
- src/test_runner/modules/vocabulary_words_test.c, 597
- src/test_runner/test_common.c, 598
- src/test_runner/test_contracts.c, 605
- src/test_runner/test_runner.c, 607
- src/vm.c, 610
- src/vm_api.c, 616
- src/vm_bootstrap.c, 621
- src/vm_debug.c, 622
- src/vm_internal.h, 628
- src/vm_time.c, 633
- src/word_registry.c, 637
- src/word_source/acl_words.c, 639
- src/word_source/arithmetic_words.c, 639
- src/word_source/block_words.c, 643
- src/word_source/control_words.c, 647
- src/word_source/defining_words.c, 651
- src/word_source/dictionary_heat_diagnostic_words.c, 656
- src/word_source/dictionary_manipulation_words.c, 657
- src/word_source/dictionary_words.c, 662
- src/word_source/double_words.c, 666
- src/word_source/editor_words.c, 670
- src/word_source/format_words.c, 673
- src/word_source/include/acl_words.h, 678
- src/word_source/include/arithmetic_words.h, 679
- src/word_source/include/block_words.h, 680
- src/word_source/include/control_words.h, 686
- src/word_source/include/defining_words.h, 686
- src/word_source/include/dictionary_heat_diagnostic_words.h, 686
- src/word_source/include/dictionary_manipulation_words.h, 687
- src/word_source/include/dictionary_words.h, 688
- src/word_source/include/double_words.h, 688
- src/word_source/include/editor_words.h, 689
- src/word_source/include/format_words.h, 689
- src/word_source/include/io_words.h, 690
- src/word_source/include/logical_words.h, 691
- src/word_source/include/memory_words.h, 692
- src/word_source/include/mixed_arithmetic_words.h, 692
- src/word_source/include/physics_benchmark_words.h, 693
- src/word_source/include/physics_diagnostic_words.h, 693
- src/word_source/include/physics_freeze_words.h, 695
- src/word_source/include/physics_pipelining_diagnostic_words.h, 695
- src/word_source/include/return_stack_words.h, 696
- src/word_source/include/stack_words.h, 696
- src/word_source/include/starforth_words.h, 697
- src/word_source/include/string_words.h, 701
- src/word_source/include/system_words.h, 702

src/word_source/include/vocabulary_words.h, 702
 src/word_source/io_words.c, 704
 src/word_source/logical_words.c, 707
 src/word_source/memory_words.c, 711
 src/word_source/mixed_arithmetic_words.c, 715
 src/word_source/physics_benchmark_words.c, 717
 src/word_source/physics_diagnostic_words.c, 719
 src/word_source/physics_freeze_words.c, 721
 src/word_source/physics_pipelining_diagnostic_words.c, 722
 src/word_source/q48_16_words.c, 724
 src/word_source/return_stack_words.c, 728
 src/word_source/stack_words.c, 730
 src/word_source/starforth_words.c, 734
 src/word_source/string_words.c, 739
 src/word_source/system_words.c, 746
 src/word_source/vocabulary_words.c, 750
 ssm_apply_mode
 ssm_jacquard.c, 463
 ssm_jacquard.h, 240
 ssm_config
 VM, 152
 ssm_config_t, 133
 L2_rolling_window, 133
 L3_linear_decay, 133
 L5_window_inference, 133
 L6_decay_inference, 134
 SSM_CV_HIGH_THRESHOLD
 ssm_jacquard.h, 239
 SSM_ENTROPY_HIGH_THRESHOLD
 ssm_jacquard.h, 239
 SSM_HYSTERESIS_TICKS
 ssm_jacquard.h, 239
 ssm_jacquard.c
 ssm_apply_mode, 463
 ssm_l8_init, 463
 ssm_l8_mode_name, 464
 ssm_l8_update, 464
 ssm_jacquard.h
 ssm_apply_mode, 240
 SSM_CV_HIGH_THRESHOLD, 239
 SSM_ENTROPY_HIGH_THRESHOLD, 239
 SSM_HYSTERESIS_TICKS, 239
 ssm_l8_init, 240
 ssm_l8_mode_name, 240
 ssm_l8_mode_t, 239
 ssm_l8_update, 240
 SSM_MODE_C0, 239
 SSM_MODE_C1, 239
 SSM_MODE_C10, 239
 SSM_MODE_C11, 239
 SSM_MODE_C12, 239
 SSM_MODE_C13, 239
 SSM_MODE_C14, 239
 SSM_MODE_C15, 239
 SSM_MODE_C2, 239
 SSM_MODE_C3, 239
 SSM_MODE_C4, 239
 SSM_MODE_C5, 239
 SSM_MODE_C6, 239
 SSM_MODE_C7, 239
 SSM_MODE_C8, 239
 SSM_MODE_C9, 239
 SSM_TEMPORAL_DECAY_LOW_THRESHOLD, 239
 SSM_TEMPORAL_DECAY_THRESHOLD, 239
 ssm_l8_init
 ssm_jacquard.c, 463
 ssm_jacquard.h, 240
 ssm_l8_metrics_t, 134
 cv, 134
 entropy, 134
 stability_score, 134
 temporal_decay, 134
 ssm_l8_mode_name
 ssm_jacquard.c, 464
 ssm_jacquard.h, 240
 ssm_l8_mode_t
 ssm_jacquard.h, 239
 ssm_l8_state
 VM, 152
 ssm_l8_state_t, 134
 current_mode, 135
 hysteresis_counter, 135
 pending_mode, 135
 ssm_l8_update
 ssm_jacquard.c, 464
 ssm_jacquard.h, 240
 SSM_MODE_C0
 ssm_jacquard.h, 239
 SSM_MODE_C1
 ssm_jacquard.h, 239
 SSM_MODE_C10
 ssm_jacquard.h, 239
 SSM_MODE_C11
 ssm_jacquard.h, 239
 SSM_MODE_C12
 ssm_jacquard.h, 239
 SSM_MODE_C13
 ssm_jacquard.h, 239
 SSM_MODE_C14
 ssm_jacquard.h, 239
 SSM_MODE_C15
 ssm_jacquard.h, 239
 SSM_MODE_C2
 ssm_jacquard.h, 239
 SSM_MODE_C3
 ssm_jacquard.h, 239
 SSM_MODE_C4
 ssm_jacquard.h, 239
 SSM_MODE_C5
 ssm_jacquard.h, 239
 SSM_MODE_C6
 ssm_jacquard.h, 239
 SSM_MODE_C7
 ssm_jacquard.h, 239

- SSM_MODE_C8
 - ssm_jacquard.h, [239](#)
- SSM_MODE_C9
 - ssm_jacquard.h, [239](#)
- SSM_TEMPORAL_DECAY_LOW_THRESHOLD
 - ssm_jacquard.h, [239](#)
- SSM_TEMPORAL_DECAY_THRESHOLD
 - ssm_jacquard.h, [239](#)
- st_info
 - Elf64_Sym, [90](#)
- st_name
 - Elf64_Sym, [90](#)
- st_other
 - Elf64_Sym, [90](#)
- st_shndx
 - Elf64_Sym, [90](#)
- st_size
 - Elf64_Sym, [90](#)
- st_value
 - Elf64_Sym, [90](#)
- stability_score
 - ssm_l8_metrics_t, [134](#)
- stack
 - ExecResult, [91](#)
- Stack Operations, [20](#)
 - vm_add_check_overflow, [21](#)
 - vm_cpuid, [21](#)
 - vm_divmod, [22](#)
 - vm_has_sse42, [22](#)
 - vm_max_asm, [22](#)
 - vm_memcpy_asm, [22](#)
 - vm_memzero_asm, [23](#)
 - vm_min_asm, [23](#)
 - vm_mul_double, [23](#)
 - vm_pop_asm, [23](#)
 - vm_push_asm, [24](#)
 - vm_rpop_asm, [24](#)
 - vm_rpush_asm, [25](#)
 - vm_strcmp_asm, [25](#)
 - vm_sub_check_overflow, [25](#)
- stack_management.c
 - vm_pop, [465](#)
 - vm_push, [466](#)
 - vm_rpop, [467](#)
 - vm_rpush, [467](#)
- stack_operations
 - Profiler, [124](#)
- STACK_SIZE
 - vm.h, [315](#)
- stack_word_depth
 - stack_words.c, [731](#)
- stack_word_drop
 - stack_words.c, [731](#)
- stack_word_dup
 - stack_words.c, [731](#)
- stack_word_minus_rot
 - stack_words.c, [732](#)
- stack_word_over
 - stack_words.c, [732](#)
- stack_word_pick
 - stack_words.c, [732](#)
- stack_word_question_dup
 - stack_words.c, [732](#)
- stack_word_roll
 - stack_words.c, [733](#)
- stack_word_rot
 - stack_words.c, [733](#)
- stack_word_suites
 - stack_words_test.c, [593](#)
- stack_word_swap
 - stack_words.c, [733](#)
- stack_words.c
 - register_stack_words, [731](#)
 - stack_word_depth, [731](#)
 - stack_word_drop, [731](#)
 - stack_word_dup, [731](#)
 - stack_word_minus_rot, [732](#)
 - stack_word_over, [732](#)
 - stack_word_pick, [732](#)
 - stack_word_question_dup, [732](#)
 - stack_word_roll, [733](#)
 - stack_word_rot, [733](#)
 - stack_word_swap, [733](#)
- stack_words.h
 - register_stack_words, [696](#)
- stack_words_test.c
 - run_stack_words_tests, [593](#)
 - stack_word_suites, [593](#)
- stale_word_count
 - DoeMetrics, [78](#)
 - HeartbeatSnapshot, [94](#)
 - InferenceInputs, [105](#)
- stale_word_count_at_check
 - VM, [152](#)
- stale_word_ratio
 - DoeMetrics, [78](#)
- Stall
 - _EFI_BOOT_SERVICES, [47](#)
- StandardErrorHandle
 - _EFI_SYSTEM_TABLE, [53](#)
- StarForth, [1](#)
- starforth_config.h
 - ADAPTIVE_CHECK_FREQUENCY, [242](#)
 - ADAPTIVE_GROWTH_THRESHOLD, [242](#)
 - ADAPTIVE_MIN_WINDOW_SIZE, [242](#)
 - ADAPTIVE_SHRINK_RATE, [242](#)
 - DECAY_MIN_INTERVAL, [242](#)
 - DECAY_RATE_PER_US_Q16, [242](#)
 - ENABLE_HOTWORDS_CACHE, [242](#)
 - ENABLE_PIPELINING, [242](#)
 - HEARTBEAT_CHECK_FREQUENCY, [242](#)
 - HEARTBEAT_INFERENCE_FREQUENCY, [242](#)
 - HEARTBEAT_SLOPE_VALIDATION_FREQUENCY, [242](#)
 - HEARTBEAT_THREAD_ENABLED, [242](#)
 - HEARTBEAT_TICK_NS, [242](#)

HEARTBEAT_WINDOW_TUNING_FREQUENCY, [starforth_config.h](#), 243
[242](#)
 INITIAL_DECAY_SLOPE_Q48, [243](#)
 ROLLING_WINDOW_SIZE, [243](#)
 STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
[243](#)
 STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT, [starforth_config.h](#), 243
[243](#)
 STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 243
[243](#)
 STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT, [starforth_config.h](#), 243
[243](#)
 STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT, [starforth_config.h](#), 244
[243](#)
 STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT, [starforth_config.h](#), 244
[244](#)
 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT, [starforth_config.h](#), 244
[244](#)
 STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 244
[244](#)
 STARFORTH_CONFIG_STRICT_PTR_DEFAULT, [starforth_config.h](#), 244
[244](#)
 STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 244
[244](#)
 STRICT_PTR, [244](#)
 TRANSITION_WINDOW_SIZE, [244](#)
 VM_STRICT_PTR, [244](#)
 STARFORTH_CONFIG_ADAPTIVE_CHECK_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ADAPTIVE_GROWTH_THRESHOLD_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ADAPTIVE_MIN_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ADAPTIVE_SHRINK_RATE_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_DECAY_MIN_INTERVAL_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_DECAY_RATE_PER_US_Q16_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ENABLE_HOTWORDS_CACHE_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ENABLE_PIPELINING_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_CHECK_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_INFERENCE_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_SLOPE_VALIDATION_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_THREAD_ENABLED_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_TICK_NS_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_HEARTBEAT_WINDOW_TUNING_FREQUENCY_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_INITIAL_DECAY_SLOPE_Q48_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_ROLLING_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_STRICT_PTR_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_CONFIG_TRANSITION_WINDOW_SIZE_DEFAULT, [starforth_config.h](#), 243
 STARFORTH_STATE_BYTES, [starforth.h](#), 244
 STARFORTH_WORD_ENTROPY_FETCH, [starforth_word_entropy_fetch](#), 698
 STARFORTH_WORD_ENTROPY_STORE, [starforth_word_entropy_store](#), 698
 STARFORTH_WORD_EXECUTION_HEAT_FETCH, [starforth_word_execution_heat_fetch](#), 736
 STARFORTH_WORD_EXECUTION_HEAT_STORE, [starforth_word_execution_heat_store](#), 736
 STARFORTH_WORD_INIT, [starforth_word_init](#), 736
 STARFORTH_WORD_PAREN_DASH, [starforth_words.c](#), 736
 STARFORTH_WORD_RANDOM, [starforth_words.c](#), 736
 STARFORTH_WORD_RESET_ENTROPY, [starforth_words.h](#), 699
 STARFORTH_WORD_RESET_EXECUTION_HEAT, [starforth_words.h](#), 699
 STARFORTH_WORD_SEED, [starforth_words.c](#), 737
 STARFORTH_WORD_SUITES, [starforth_words.c](#), 737
 STARFORTH_WORD_TEST_C, [starforth_words.h](#), 699
 STARFORTH_WORD_TOP_WORDS, [starforth_words.c](#), 737
 STARFORTH_WORD_VERSION, [starforth_words.h](#), 700
 STARFORTH_WORD_WAIT, [starforth_words.c](#), 738
 STARFORTH_WORD_WORD_ENTROPY, [starforth_words.h](#), 700

- starforth_words.h, 701
- starforth_word_word_execution_heat
 - starforth_words.c, 738
- starforth_word_zuse_authenticate
 - starforth_words.c, 739
- starforth_words.c
 - g_prng_state, 739
 - is_valid_dict_entry, 735
 - prng_next, 735
 - register_starforth_words, 735
 - starforth_word_execution_heat_fetch, 736
 - starforth_word_execution_heat_store, 736
 - starforth_word_init, 736
 - starforth_word_paren_dash, 736
 - starforth_word_random, 737
 - starforth_word_reset_execution_heat, 737
 - starforth_word_seed, 737
 - starforth_word_top_words, 737
 - starforth_word_version, 738
 - starforth_word_wait, 738
 - starforth_word_word_execution_heat, 738
 - starforth_word_zuse_authenticate, 739
- starforth_words.h
 - register_starforth_words, 697
 - starforth_word_entropy_fetch, 698
 - starforth_word_entropy_store, 698
 - starforth_word_init, 698
 - starforth_word_paren_dash, 698
 - starforth_word_random, 699
 - starforth_word_reset_entropy, 699
 - starforth_word_seed, 699
 - starforth_word_top_words, 700
 - starforth_word_wait, 700
 - starforth_word_word_entropy, 701
- starforth_words_test.c
 - run_starforth_words_tests, 594
 - starforth_word_suites, 594
- start_time
 - ProfileTimer, 126
- started_ns
 - CapsuleRunRecord, 68
- StartImage
 - _EFI_BOOT_SERVICES, 47
- startup.h
 - startup_blkio_cleanup, 309
 - startup_blkio_init, 309
 - startup_block_subsys_init, 309
 - startup_vm_init, 310
- startup_blkio_cleanup
 - startup.h, 309
- startup_blkio_init
 - startup.h, 309
- startup_block_subsys_init
 - startup.h, 309
- startup_vm_init
 - startup.h, 310
- state
 - blkio_dev, 59
 - VMRegistryEntry, 156
- state_addr
 - VM, 152
- state_flags
 - DictPhysics, 73
- state_var
 - VM, 152
- state_variable
 - dictionary_manipulation_words.c, 662
- stats
 - HotwordsCache_s, 100
- stddev_ns_q48
 - BayesianLatencyPosterior, 54
- StdErr
 - _EFI_SYSTEM_TABLE, 53
- stop_requested
 - HeartbeatWorker, 99
- stress_suites
 - stress_tests.c, 596
- stress_tests.c
 - run_stress_tests, 595
 - stress_suites, 596
- STRICT_PTR
 - starforth_config.h, 244
- string_runtime_s_quote
 - string_words.c, 741
- string_word_bl
 - string_words.c, 741
- string_word_blank
 - string_words.c, 741
- string_word_bracket_literal
 - string_words.c, 742
- string_word_bracket_tick
 - string_words.c, 742
- string_word_cmove
 - string_words.c, 742
- string_word_cmove_greater
 - string_words.c, 742
- string_word_compare
 - string_words.c, 742
- string_word_convert
 - string_words.c, 742
- string_word_count
 - string_words.c, 742
- string_word_enclose
 - string_words.c, 743
- string_word_expect
 - string_words.c, 743
- string_word_literal
 - string_words.c, 743
- string_word_minus_trailing
 - string_words.c, 743
- string_word_number
 - string_words.c, 743
- string_word_query
 - string_words.c, 743
- string_word_s_quote
 - string_words.c, 744

- string_word_scan
 - string_words.c, [744](#)
- string_word_search
 - string_words.c, [744](#)
- string_word_skip
 - string_words.c, [744](#)
- string_word_source
 - string_words.c, [744](#)
- string_word_span
 - string_words.c, [744](#)
- string_word_suites
 - string_words_test.c, [596](#)
- string_word_tib
 - string_words.c, [745](#)
- string_word_tick
 - string_words.c, [745](#)
- string_word_to_in
 - string_words.c, [745](#)
- string_word_word
 - string_words.c, [745](#)
- string_words.c
 - convert_string_to_number, [740](#)
 - ensure_input, [740](#)
 - register_string_words, [741](#)
 - string_runtime_s_quote, [741](#)
 - string_word_bl, [741](#)
 - string_word_blank, [741](#)
 - string_word_bracket_literal, [742](#)
 - string_word_bracket_tick, [742](#)
 - string_word_cmove, [742](#)
 - string_word_cmove_greater, [742](#)
 - string_word_compare, [742](#)
 - string_word_convert, [742](#)
 - string_word_count, [742](#)
 - string_word_enclose, [743](#)
 - string_word_expect, [743](#)
 - string_word_literal, [743](#)
 - string_word_minus_trailing, [743](#)
 - string_word_number, [743](#)
 - string_word_query, [743](#)
 - string_word_s_quote, [744](#)
 - string_word_scan, [744](#)
 - string_word_search, [744](#)
 - string_word_skip, [744](#)
 - string_word_source, [744](#)
 - string_word_span, [744](#)
 - string_word_tib, [745](#)
 - string_word_tick, [745](#)
 - string_word_to_in, [745](#)
 - string_word_word, [745](#)
 - vaddr_from_ptr, [745](#)
 - word_scratch_addr, [746](#)
 - WORD_SCRATCH_CAP, [740](#)
- string_words.h
 - register_string_words, [701](#)
- string_words_test.c
 - run_string_words_tests, [596](#)
 - string_word_suites, [596](#)
- suggested_next_size
 - PipelineGlobalMetrics, [122](#)
- suite_contract
 - WordTestSuite, [159](#)
- summary
 - physics_analytics_heap, [114](#)
- summary_bytes
 - physics_analytics_header, [113](#)
 - physics_analytics_heap, [115](#)
- summary_offset
 - physics_analytics_header, [113](#)
- system_is_running
 - system_words.c, [747](#)
- system_running
 - system_words.c, [750](#)
- system_word_79_standard
 - system_words.c, [748](#)
- system_word_abort
 - system_words.c, [748](#)
- system_word_abort_quote
 - system_words.c, [748](#)
- system_word_bye
 - system_words.c, [748](#)
- system_word_cold
 - system_words.c, [748](#)
- system_word_nop
 - system_words.c, [748](#)
- system_word_page
 - system_words.c, [748](#)
- system_word_quit
 - system_words.c, [749](#)
- system_word_save_system
 - system_words.c, [749](#)
- system_word_see
 - system_words.c, [749](#)
- system_word_suites
 - system_words_test.c, [597](#)
- system_word_vlist
 - system_words.c, [749](#)
- system_word_warm
 - system_words.c, [749](#)
- system_word_words
 - system_words.c, [749](#)
- system_words.c
 - forth_79_standard, [749](#)
 - forth_paren, [747](#)
 - print_word_name, [747](#)
 - register_system_words, [747](#)
 - reset_vm_state, [747](#)
 - runtime_abortq, [747](#)
 - set_forth_79_compliance, [747](#)
 - system_is_running, [747](#)
 - system_running, [750](#)
 - system_word_79_standard, [748](#)
 - system_word_abort, [748](#)
 - system_word_abort_quote, [748](#)
 - system_word_bye, [748](#)
 - system_word_cold, [748](#)

- system_word_nop, 748
- system_word_page, 748
- system_word_quit, 749
- system_word_save_system, 749
- system_word_see, 749
- system_word_vlist, 749
- system_word_warm, 749
- system_word_words, 749
- system_words.h
 - register_system_words, 702
- system_words_test.c
 - run_system_words_tests, 597
 - system_word_suites, 597
- tag
 - Runtime Branch Helpers, 39
- temperature_from_execution_heat
 - physics_metadata.c, 425
- temperature_q8
 - DictPhysics, 73
 - physics_seed_definition, 120
- temporal_decay
 - ssm_l8_metrics_t, 134
- test_acl_adaptive_ttl
 - acl_words_test.c, 577
- test_acl_deny_aborts
 - acl_words_test.c, 577
- test_acl_inherit
 - acl_words_test.c, 577
- test_acl_pin_blocks_shadow
 - acl_words_test.c, 577
- test_acl_pin_immutable
 - acl_words_test.c, 577
- test_acl_strict_mode
 - acl_words_test.c, 578
- test_acl_ttl_hot_path
 - acl_words_test.c, 578
- test_common.c
 - assert_stack_depth, 599
 - assert_stack_top, 600
 - assert_vm_error, 600
 - DIE, 599
 - dump_and_die, 600
 - fail_and_exit, 601
 - fail_fast, 605
 - print_module_summary, 601
 - resolve_contract, 601
 - restore_dict_state, 602
 - restore_vm_state, 602
 - run_single_test, 602
 - run_test_suite, 603
 - run_test_suite_m, 603
 - save_dict_state, 604
 - save_vm_state, 604
 - test_fatal_exit, 599
- test_common.h
 - assert_stack_depth, 555
 - assert_stack_top, 557
 - assert_vm_error, 557
 - fail_fast, 563
 - MAX_TEST_INPUT, 555
 - MAX_TEST_OUTPUT, 555
 - MAX_TESTS_PER_WORD, 555
 - print_module_summary, 559
 - restore_dict_state, 559
 - restore_vm_state, 560
 - run_single_test, 560
 - run_test_suite, 561
 - run_test_suite_m, 562
 - save_dict_state, 562
 - save_vm_state, 562
 - TEST_EDGE_CASE, 555
 - TEST_ERROR_CASE, 555
 - TEST_NORMAL, 555
 - TestType, 555
- test_contracts.c
 - assert_heartbeat_safe, 605
 - capture_exec_result, 605
 - check_physics_transparent, 605
 - contract_rand, 606
 - exec_results_equal, 606
 - global_contract_violations, 607
 - perturb_physics, 606
 - restore_physics, 606
 - rng_seed, 606
 - rng_state, 607
 - save_physics, 607
- test_contracts.h
 - assert_heartbeat_safe, 564
 - capture_exec_result, 564
 - check_physics_transparent, 565
 - CONTRACT_DEFAULT_FUZZ_ROUNDS, 564
 - contract_flags_t, 564
 - CONTRACT_HEARTBEAT_SAFE, 564
 - CONTRACT_NONE, 564
 - CONTRACT_PHYSICS_TRANSPARENT, 564
 - exec_results_equal, 565
 - global_contract_violations, 566
 - perturb_physics, 565
 - restore_physics, 565
 - save_physics, 565
- test_count
 - WordTestSuite, 159
- TEST_EDGE_CASE
 - test_common.h, 555
- test_emergency_console_bypass
 - acl_words_test.c, 578
- test_end_time
 - break_me_tests.c, 581
- TEST_ERROR
 - log.h, 195
- TEST_ERROR_CASE
 - test_common.h, 555
- TEST_FAIL
 - log.h, 195
- test_fail_color
 - log.c, 411

- test_fatal_exit
 - test_common.c, [599](#)
- test_modules
 - test_runner.c, [609](#)
- TEST_NORMAL
 - test_common.h, [555](#)
- TEST_PASS
 - log.h, [195](#)
- test_pass_color
 - log.c, [411](#)
- test_runner.c
 - benchmark_iterations, [609](#)
 - benchmark_mode, [609](#)
 - enable_benchmark_mode, [608](#)
 - get_time_ns, [608](#)
 - global_test_stats, [609](#)
 - run_all_tests, [608](#)
 - run_module_tests, [609](#)
 - run_word_tests, [609](#)
 - test_modules, [609](#)
- test_runner.h
 - enable_benchmark_mode, [567](#)
 - global_test_stats, [576](#)
 - run_acl_words_tests, [568](#)
 - run_all_tests, [568](#)
 - run_arithmetic_words_tests, [568](#)
 - run_block_words_tests, [568](#)
 - run_break_me_tests, [569](#)
 - run_control_words_tests, [569](#)
 - run_defining_words_tests, [569](#)
 - run_dictionary_manipulation_words_tests, [569](#)
 - run_dictionary_words_tests, [570](#)
 - run_double_words_tests, [570](#)
 - run_format_words_tests, [571](#)
 - run_integration_tests, [571](#)
 - run_io_words_tests, [571](#)
 - run_logical_words_tests, [571](#)
 - run_mama_forth_words_tests, [572](#)
 - run_memory_words_tests, [572](#)
 - run_mixed_arithmetic_words_tests, [572](#)
 - run_module_tests, [573](#)
 - run_return_stack_words_tests, [573](#)
 - run_stack_words_tests, [573](#)
 - run_starforth_words_tests, [574](#)
 - run_stress_tests, [574](#)
 - run_string_words_tests, [574](#)
 - run_system_words_tests, [575](#)
 - run_vocabulary_words_tests, [575](#)
 - run_word_tests, [575](#)
- TEST_SKIP
 - log.h, [195](#)
- test_skip_color
 - log.c, [411](#)
- test_start_time
 - break_me_tests.c, [581](#)
- TestCase, [135](#)
 - contract, [135](#)
 - expected, [135](#)
 - implemented, [136](#)
 - input, [136](#)
 - name, [136](#)
 - should_error, [136](#)
 - type, [136](#)
- TestModule, [136](#)
 - module_name, [137](#)
 - reserved, [137](#)
 - reserved_int, [137](#)
 - run_module_tests, [137](#)
- TestResult
 - log.h, [195](#)
- tests
 - WordTestSuite, [159](#)
- tests_errors
 - ParityPacket, [109](#)
- tests_failed
 - acl_words_test.c, [578](#)
 - ParityPacket, [109](#)
- tests_passed
 - acl_words_test.c, [578](#)
 - ParityPacket, [109](#)
- tests_skipped
 - ParityPacket, [109](#)
- tests_total
 - ParityPacket, [109](#)
- TestStats, [137](#)
 - total_error, [137](#)
 - total_fail, [137](#)
 - total_pass, [138](#)
 - total_skip, [138](#)
 - total_tests, [138](#)
- TestString
 - _EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, [51](#)
- TestType
 - test_common.h, [555](#)
- threading.c
 - sf_mutex_destroy, [446](#)
 - sf_mutex_init, [446](#)
 - sf_mutex_lock, [446](#)
 - sf_mutex_unlock, [446](#)
- tib_buf
 - VM, [152](#)
- tib_cap
 - VM, [152](#)
- tick_buffer
 - HeartbeatState, [97](#)
- tick_buffer_size
 - HeartbeatState, [97](#)
- tick_buffer_write_index
 - HeartbeatState, [97](#)
- tick_count
 - DoeMetrics, [79](#)
 - HeartbeatState, [97](#)
- tick_count_total
 - HeartbeatState, [97](#)
- tick_interval_ns
 - HeartbeatTickSnapshot, [99](#)

- tick_ns
 - HeartbeatWorker, 99
- tick_number
 - HeartbeatTickSnapshot, 99
- tick_number_offset
 - HeartbeatState, 97
- tick_target_ns
 - DoeMetrics, 79
 - HeartbeatState, 97
 - PhysicsSnapshot, 121
- ticks
 - time_trust_state, 138
- time.c
 - _POSIX_C_SOURCE, 444
 - posix_format_timestamp, 444
 - posix_get_monotonic_ns, 444
 - posix_get_realtime_ns, 444
 - posix_has_rtc, 444
 - posix_set_realtime_ns, 445
 - sf_time_backend_l4re, 444
 - sf_time_backend_posix, 445
 - sf_time_init_l4re, 443
- TIME_TRUST_HIGH
 - timer.h, 290
- TIME_TRUST_LOW
 - timer.h, 290
- time_trust_state, 138
 - expected_delta, 138
 - last_tsc, 138
 - ticks, 138
 - total_samples, 138
 - trust, 139
 - variance, 139
 - window, 139
- time_trust_t
 - timer.h, 290
- time_window, 139
 - count, 139
 - deltas, 139
 - pos, 139
- TIME_WINDOW_SIZE
 - timer.h, 290
- timer.c
 - calib_record, 489
 - calibrate_tsc_with_pmtimer, 482
 - calibrate_window_hpet, 482
 - calibrate_window_pit, 482
 - converge_tsc_bare_metal, 482
 - cpu_has_invariant_tsc, 483
 - cpu_has_rdtscp, 483
 - cpuid, 483
 - CV_EPSILON_PPM, 480
 - CV_STABLE_COUNT, 480
 - derive_tsc_hz_from_cpuid, 483
 - DRIFT_EPSILON_PPM, 480
 - enable_hpet, 483
 - g_heartbeat, 489
 - heartbeat_init, 483
 - heartbeat_state, 484
 - heartbeat_tick, 484
 - heartbeat_ticks, 484
 - heartbeat_trust, 484
 - hpet_freq_hz, 489
 - HPET_GEN_CAP_ID, 480
 - HPET_GEN_CONFIG, 481
 - HPET_MAIN_COUNTER, 481
 - HPET_PHYS_BASE, 481
 - hpet_read, 484
 - hpet_regs, 489
 - HPET_VIRT_BASE, 481
 - hpet_write, 484
 - inb, 484
 - init_timer_vm_mode, 485
 - inl, 485
 - muldiv64, 485
 - outb, 485
 - pit_available, 489
 - PIT_CHANNEL0_DATA, 481
 - PIT_COMMAND, 481
 - PIT_FREQ_HZ, 481
 - pit_probe_running, 485
 - pit_program_reload, 485
 - pit_read_counter16, 486
 - pit_tsc_hz_mean, 489
 - pmtimer_delta, 486
 - PMTIMER_FREQ_HZ, 481
 - PMTIMER_IO_PORT, 481
 - PMTIMER_MASK, 481
 - pmtimer_read, 486
 - PMTIMER_WIDTH_BITS, 481
 - print_dec, 486
 - rdtsc, 486
 - rdtscp_aux, 486
 - running_under_hypervisor, 486
 - SAMPLE_WINDOWS, 481
 - sanity_check_hpet_running_quick, 486
 - timer_calibration_record, 487
 - timer_check_drift_now, 487
 - timer_drift_check_bare_metal, 487
 - timer_fatal, 487
 - timer_init, 487
 - timer_now_ns, 487
 - timer_now_ns_vm_relative, 487
 - TIMER_REQUIRE_PIT, 482
 - timer_tsc_hz, 488
 - TIMER_VM_STRICT_INVARIANT_TSC, 482
 - tsc_hz_locked, 489
 - update_stats, 488
 - variance_to_trust, 488
 - vm_ns_base, 489
 - vm_pm_start, 489
 - vm_timebase_init_from_pmtimer, 488
 - WARMUP_WINDOWS, 482
 - window_push, 488
 - WINDOW_TICKS_HPET, 482
 - WINDOW_TICKS_PIT, 482

- window_variance_q48, [488](#)
 - windows_used, [489](#)
- timer.h
 - heartbeat_init, [291](#)
 - heartbeat_state, [291](#)
 - heartbeat_tick, [291](#)
 - heartbeat_ticks, [291](#)
 - heartbeat_trust, [291](#)
 - TIME_TRUST_HIGH, [290](#)
 - TIME_TRUST_LOW, [290](#)
 - time_trust_t, [290](#)
 - TIME_WINDOW_SIZE, [290](#)
 - timer_calibration_record, [292](#)
 - timer_calibration_record_t, [290](#)
 - timer_check_drift_now, [292](#)
 - timer_init, [292](#)
 - timer_now_ns, [292](#)
 - TIMER_TRUST_ABSOLUTE, [291](#)
 - timer_trust_level, [291](#)
 - timer_trust_level_t, [290](#)
 - TIMER_TRUST_NONE, [291](#)
 - TIMER_TRUST_RELATIVE, [291](#)
 - timer_tsc_hz, [292](#)
 - TimeTrustState, [290](#)
 - TimeWindow, [290](#)
- timer_calibration_record, [139](#)
 - converged, [140](#)
 - cv_hpet_ppm, [140](#)
 - cv_pit_ppm, [140](#)
 - diff_ppm, [140](#)
 - hpet_hz, [140](#)
 - pit_hz_mean, [140](#)
 - reserved, [140](#)
 - timer.c, [487](#)
 - timer.h, [292](#)
 - trust, [140](#)
 - tsc_hz_mean, [140](#)
 - vm_mode, [140](#)
 - windows_used, [141](#)
- timer_calibration_record_t
 - timer.h, [290](#)
- timer_check_drift_now
 - timer.c, [487](#)
 - timer.h, [292](#)
- TIMER_DIV_1
 - apic.c, [469](#)
- TIMER_DIV_128
 - apic.c, [469](#)
- TIMER_DIV_16
 - apic.c, [469](#)
- TIMER_DIV_2
 - apic.c, [469](#)
- TIMER_DIV_32
 - apic.c, [470](#)
- TIMER_DIV_4
 - apic.c, [470](#)
- TIMER_DIV_64
 - apic.c, [470](#)
- TIMER_DIV_8
 - apic.c, [470](#)
- timer_drift_check_bare_metal
 - timer.c, [487](#)
- timer_fatal
 - timer.c, [487](#)
- timer_init
 - timer.c, [487](#)
 - timer.h, [292](#)
- timer_initial_count
 - apic.c, [472](#)
- timer_now_ns
 - timer.c, [487](#)
 - timer.h, [292](#)
- timer_now_ns_vm_relative
 - timer.c, [487](#)
- timer_period_tsc_ticks
 - apic.c, [472](#)
- TIMER_REQUIRE_PIT
 - timer.c, [482](#)
- timer_tick_hz
 - apic.c, [472](#)
- TIMER_TRUST_ABSOLUTE
 - timer.h, [291](#)
- timer_trust_level
 - timer.h, [291](#)
- timer_trust_level_t
 - timer.h, [290](#)
- TIMER_TRUST_NONE
 - timer.h, [291](#)
- TIMER_TRUST_RELATIVE
 - timer.h, [291](#)
- timer_tsc_hz
 - timer.c, [488](#)
 - timer.h, [292](#)
- TIMER_VM_STRICT_INVARIANT_TSC
 - timer.c, [482](#)
- timestamp_ns
 - physics_analytics_event_header, [110](#)
- TimeTrustState
 - timer.h, [290](#)
- TimeWindow
 - timer.h, [290](#)
- TimeZone
 - EFI_TIME, [85](#)
- total_blkio_blocks_1k
 - block_subsystem.c, [385](#)
- total_blocks
 - blk_volume_meta_t, [58](#)
 - blkio_dev, [59](#)
 - blkio_file_state_t, [60](#)
 - blkio_info_t, [61](#)
 - blkio_params_t, [61](#)
 - blkio_ram_state_t, [62](#)
- total_bytes
 - kmalloc_stats_t, [107](#)
 - pmm_stats_t, [123](#)
 - sf_alloc_stats, [130](#)

- total_bytes_read
 - MemoryProfile, 108
- total_bytes_written
 - MemoryProfile, 108
- total_context_transitions
 - WordTransitionMetrics, 162
- total_devblocks
 - blk_volume_meta_t, 58
- total_dict_lookups
 - profiler.c, 454
- total_error
 - TestStats, 137
- total_execution_time
 - Profiler, 124
- total_executions
 - DoeMetrics, 79
 - PhysicsSnapshot, 121
 - rolling_window_view_t, 127
 - RollingWindowOfTruth, 129
- total_fail
 - TestStats, 137
- total_heat
 - DoeMetrics, 79
 - HeartbeatSnapshot, 94
 - InferenceInputs, 105
- total_heat_at_last_check
 - VM, 152
- total_len
 - XXHash64State, 163
- total_lookups
 - doe_metrics.c, 400
 - DoeMetrics, 79
 - HotwordsStats, 103
- total_memory_reads
 - profiler.c, 454
- total_memory_writes
 - profiler.c, 454
- total_pages
 - pmm_stats_t, 123
- total_pass
 - TestStats, 138
- total_runtime_ms
 - DoeMetrics, 79
- total_samples
 - time_trust_state, 138
- total_skip
 - TestStats, 138
- total_stack_ops
 - profiler.c, 454
- total_tests
 - TestStats, 138
- total_tests_run
 - break_me_tests.c, 581
- total_time
 - ProfileTimer, 126
 - WordProfile, 157
- total_time_ns
 - WordStats, 158
- total_transitions
 - WordTransitionMetrics, 162
- total_user_lbn
 - block_subsystem.c, 385
- total_volumes
 - blk_volume_meta_t, 58
- TPL_APPLICATION
 - uefi.h, 296
- TPL_CALLBACK
 - uefi.h, 296
- TPL_HIGH_LEVEL
 - uefi.h, 296
- TPL_NOTIFY
 - uefi.h, 296
- tracked_blocks
 - blk_volume_meta_t, 58
- trajectory_length
 - InferenceInputs, 105
- transition_heat
 - WordTransitionMetrics, 162
- transition_metrics
 - DictEntry, 72
 - vm.h, 336
- transition_metrics_binary_chop_suggest_window
 - physics_pipelining_metrics.c, 427
 - physics_pipelining_metrics.h, 209
- transition_metrics_cleanup
 - physics_pipelining_metrics.c, 428
 - physics_pipelining_metrics.h, 210
- transition_metrics_context_accuracy_string
 - physics_pipelining_metrics.c, 428
 - physics_pipelining_metrics.h, 210
- transition_metrics_get_probability_q48
 - physics_pipelining_metrics.c, 428
 - physics_pipelining_metrics.h, 210
- transition_metrics_init
 - physics_pipelining_metrics.c, 429
 - physics_pipelining_metrics.h, 211
- transition_metrics_record
 - physics_pipelining_metrics.c, 429
 - physics_pipelining_metrics.h, 211
- transition_metrics_record_context
 - physics_pipelining_metrics.c, 429
 - physics_pipelining_metrics.h, 212
- transition_metrics_record_prefetch_hit
 - physics_pipelining_metrics.c, 430
 - physics_pipelining_metrics.h, 212
- transition_metrics_record_prefetch_miss
 - physics_pipelining_metrics.c, 430
 - physics_pipelining_metrics.h, 212
- transition_metrics_reset
 - physics_pipelining_metrics.c, 431
 - physics_pipelining_metrics.h, 213
- transition_metrics_should_speculate
 - physics_pipelining_metrics.c, 431
 - physics_pipelining_metrics.h, 213
- transition_metrics_stats_string
 - physics_pipelining_metrics.c, 431

- physics_pipelining_metrics.h, 213
- transition_metrics_update_cache
 - physics_pipelining_metrics.c, 432
 - physics_pipelining_metrics.h, 214
- transition_metrics_update_context_window
 - physics_pipelining_metrics.c, 432
 - physics_pipelining_metrics.h, 214
- TRANSITION_WINDOW_SIZE
 - physics_pipelining_metrics.h, 209
 - starforth_config.h, 244
- traverse_name_field
 - dictionary_manipulation_words.c, 662
- TRUE
 - uefi.h, 296
- trust
 - time_trust_state, 139
 - timer_calibration_record, 140
- tsc_hz_locked
 - timer.c, 489
- tsc_hz_mean
 - timer_calibration_record, 140
- tuning_lock
 - VM, 152
- Type
 - EFI_MEMORY_DESCRIPTOR, 83
- type
 - BlockBuffer, 64
 - TestCase, 136
- type_attr
 - idt_entry, 104
 - interrupts.c, 478
- u32_to_dec
 - capsule_loader.c, 504
- ucell_t
 - vm.h, 317
- udiv_floor
 - block_subsystem.c, 383
- uefi.h
 - AllHandles, 300
 - AllocateAddress, 300
 - AllocateAnyPages, 300
 - AllocateMaxAddress, 300
 - BOOLEAN, 297
 - ByProtocol, 300
 - ByRegisterNotify, 300
 - CHAR16, 297
 - EFI_ABORTED, 295
 - EFI_ACPI_20_TABLE_GUID, 300
 - EFI_ACPI_TABLE_GUID, 300
 - EFI_ALLOCATE_PAGES, 297
 - EFI_ALLOCATE_POOL, 297
 - EFI_ALLOCATE_TYPE, 299
 - EFI_BAD_BUFFER_SIZE, 295
 - EFI_BOOT_SERVICES, 297
 - EFI_BUFFER_TOO_SMALL, 295
 - EFI_DEVICE_ERROR, 295
 - EFI_EXIT_BOOT_SERVICES, 297
 - EFI_FILE_CLOSE, 297
 - EFI_FILE_GET_INFO, 297
 - EFI_FILE_INFO_GUID, 301
 - EFI_FILE_MODE_READ, 295
 - EFI_FILE_OPEN, 297
 - EFI_FILE_PROTOCOL, 297
 - EFI_FILE_READ, 297
 - EFI_FREE_PAGES, 297
 - EFI_FREE_POOL, 297
 - EFI_GET_MEMORY_MAP, 297
 - EFI_HANDLE, 298
 - EFI_HANDLE_PROTOCOL, 298
 - EFI_INVALID_PARAMETER, 295
 - EFI_LOAD_ERROR, 295
 - EFI_LOADED_IMAGE_PROTOCOL_GUID, 301
 - EFI_LOCATE_HANDLE, 298
 - EFI_LOCATE_SEARCH_TYPE, 300
 - EFI_MEMORY_RP, 295
 - EFI_MEMORY_RUNTIME, 295
 - EFI_MEMORY_TYPE, 300
 - EFI_MEMORY_UC, 295
 - EFI_MEMORY_UCE, 295
 - EFI_MEMORY_WB, 295
 - EFI_MEMORY_WC, 295
 - EFI_MEMORY_WP, 295
 - EFI_MEMORY_WT, 296
 - EFI_MEMORY_XP, 296
 - EFI_NOT_FOUND, 296
 - EFI_NOT_READY, 296
 - EFI_OUT_OF_RESOURCES, 296
 - EFI_PHYSICAL_ADDRESS, 298
 - EFI_RAISE_TPL, 298
 - EFI_RESTORE_TPL, 298
 - EFI_RUNTIME_SERVICES, 298
 - EFI_SIMPLE_FILE_SYSTEM_PROTOCOL, 298
 - EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_GUID, 301
 - EFI_SIMPLE_FILE_SYSTEM_PROTOCOL_OPEN_VOLUME, 298
 - EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL, 298
 - EFI_STATUS, 298
 - EFI_SUCCESS, 296
 - EFI_SYSTEM_TABLE, 298
 - EFI_TEXT_RESET, 298
 - EFI_TEXT_STRING, 298
 - EFI_TPL, 299
 - EFI_UNSUPPORTED, 296
 - EFI_WRITE_PROTECTED, 296
 - EfiACPIMemoryNVS, 300
 - EfiACPIReclaimMemory, 300
 - EFIAPI, 296
 - EfiBootServicesCode, 300
 - EfiBootServicesData, 300
 - EfiConventionalMemory, 300
 - EfiLoaderCode, 300
 - EfiLoaderData, 300
 - EfiMaxMemoryType, 300
 - EfiMemoryMappedIO, 300
 - EfiMemoryMappedIOPortSpace, 300

- EfiPalCode, 300
- EfiPersistentMemory, 300
- EfiReservedMemoryType, 300
- EfiRuntimeServicesCode, 300
- EfiRuntimeServicesData, 300
- EfiUnusableMemory, 300
- FALSE, 296
- INT16, 299
- INT32, 299
- INT64, 299
- INT8, 299
- INTN, 299
- MaxAllocateType, 300
- TPL_APPLICATION, 296
- TPL_CALLBACK, 296
- TPL_HIGH_LEVEL, 296
- TPL_NOTIFY, 296
- TRUE, 296
- UINT16, 299
- UINT32, 299
- UINT64, 299
- UINT8, 299
- UINTN, 299
- VOID, 299
- uefi_boot_services_exited
 - BootInfo, 65
- uefi_loader.c
 - efi_main, 493
 - g_boot_info, 494
 - guid_equals, 494
 - kernel_elf_buffer, 494
 - kernel_elf_size, 494
 - KERNEL_MAX_SIZE, 493
 - KernelEntry, 493
 - load_kernel_from_esp, 494
 - RAW_LOG, 493
- UINT16
 - uefi.h, 299
- UINT32
 - uefi.h, 299
- UINT64
 - uefi.h, 299
- UINT8
 - uefi.h, 299
- UINTN
 - uefi.h, 299
- Unified API Wrappers, 15
- UninstallProtocolInterface
 - _EFI_BOOT_SERVICES, 47
- UNLIKELY
 - vm.h, 315
- Unload
 - EFI_LOADED_IMAGE_PROTOCOL, 83
- UnloadImage
 - _EFI_BOOT_SERVICES, 47
- update_free_bytes
 - kmalloc.c, 525
- update_stats
 - timer.c, 488
- used_bytes
 - kmalloc_stats_t, 108
 - pmm_stats_t, 123
 - sf_alloc_stats, 130
- used_pages
 - pmm_stats_t, 123
- USER_BLOCKS_START
 - vm.h, 315
- v1
 - XXHash64State, 163
- v2
 - XXHash64State, 163
- v3
 - XXHash64State, 163
- v4
 - XXHash64State, 163
- vaddr_from_ptr
 - string_words.c, 745
- vaddr_t
 - vm.h, 317
- valid
 - cache_slot_t, 66
- validate_result_strings
 - capsule_validate.c, 509
- variance
 - time_trust_state, 139
- variance_to_trust
 - timer.c, 488
- VendorGuid
 - EFI_CONFIGURATION_TABLE, 80
- VendorTable
 - EFI_CONFIGURATION_TABLE, 80
- verbose
 - VMHostServices, 154
- version
 - blk_volume_meta_t, 58
- version_major
 - physics_analytics_header, 113
- version_minor
 - physics_analytics_header, 113
- VirtualStart
 - EFI_MEMORY_DESCRIPTOR, 83
- VM, 141
 - abort_requested, 143
 - base, 143
 - base_addr, 143
 - compile_buffer, 143
 - compile_pos, 144
 - compile_size, 144
 - compiling_word, 144
 - current_executing_entry, 144
 - current_word_name, 144
 - data_stack, 144
 - decay_slope_direction, 144
 - decay_slope_q48, 144
 - dict_fence_here, 144
 - dict_fence_latest, 145

- dict_lock, 145
- doe_row_printing, 145
- dsp, 145
- ecw_nesting, 146
- emergency_console, 146
- error, 146
- exit_colon, 147
- halted, 147
- heartbeat, 147
- heartbeat_decay_cursor_id, 148
- heat_threshold_25th, 148
- heat_threshold_50th, 148
- heat_threshold_75th, 148
- here, 148
- host, 148
- hot_word_count_at_check, 148
- hotwords_cache, 148
- in_var, 149
- input_buffer, 149
- input_length, 149
- input_pos, 149
- interpreter_enabled, 149
- ip, 149
- last_bucket_reorg_ns, 149
- last_decay_check_ns, 149
- last_inference_outputs, 149
- latest, 149
- lookup_strategy, 150
- memory, 150
- mode, 150
- next_word_id, 150
- pipeline_metrics, 150
- profiler.h, 226
- recycled_word_id_count, 151
- recycled_word_ids, 151
- return_stack, 151
- rolling_window, 151
- rsp, 151
- scr_addr, 151
- span_var, 151
- ssm_config, 152
- ssm_l8_state, 152
- stale_word_count_at_check, 152
- state_addr, 152
- state_var, 152
- tib_buf, 152
- tib_cap, 152
- total_heat_at_last_check, 152
- tuning_lock, 152
- vm.h, 317
- word_count_at_check, 152
- word_id_map, 153
- zuse_session, 153
- vm
 - block_subsystem.c, 385
 - InferenceInputs, 105
- VM Input Operations, 15
 - vm_addr_from_ptr, 16
 - vm_input_ensure, 16
 - vm_input_in, 17
 - vm_input_load_line, 17
 - vm_input_source, 18
 - vm_input_span, 18
 - vm_input_tib, 19
- vm.c
 - acl_recheck, 611
 - execute_colon_word, 611
 - vm_addr_ok, 611
 - vm_compile_call, 612
 - vm_compile_exit, 612
 - vm_compile_literal, 612
 - vm_compile_word, 612
 - vm_enter_compile_mode, 612
 - vm_exit_compile_mode, 613
 - vm_get_base, 613
 - vm_interpret, 613
 - vm_interpret_word, 613
 - vm_load_cell, 614
 - vm_load_u8, 614
 - vm_make_immediate, 614
 - vm_parse_number, 614
 - vm_parse_word, 615
 - vm_ptr, 615
 - vm_set_base, 615
 - vm_store_cell, 615
 - vm_store_u8, 616
- vm.h
 - __attribute__, 317, 335
 - acl_allow, 335
 - acl_default, 335
 - acl_mode, 335
 - ACL_MODE_STRICT, 314
 - ACL_MODE_TTL, 314
 - acl_pinned, 336
 - acl_ttl, 336
 - ACL_TTL_OPEN, 314
 - ACL_USER_DEFAULT, 314
 - BLOCK_SIZE, 314
 - CELL, 317
 - cell_t, 316
 - CHARGE_NEUTRAL, 314
 - COMPILE_BUFFER_SIZE, 314
 - DICTIONARY_BLOCKS, 314
 - DICTIONARY_MEMORY_SIZE, 314
 - DICTIONARY_SIZE, 314
 - execute_colon_word, 318
 - execution_heat, 336
 - flags, 336
 - func, 336
 - heartbeat_capture_tick_snapshot, 318
 - heartbeat_emit_tick_row, 318
 - HEARTBEAT_TICK_BUFFER_SIZE, 314
 - HeartbeatWorker, 316
 - HEAT_CACHE_DEMOTION_THRESHOLD, 314
 - HotwordsCache, 317
 - InferenceOutputs, 317

INPUT_BUFFER_SIZE, 314
 LIKELY, 314
 link, 336
 LOG_BLOCKS, 314
 LOG_BLOCKS_END, 314
 LOG_BLOCKS_START, 315
 LOG_LAYER1_MAX_LINES, 315
 LOG_LINE_MAX, 315
 LOG_LINES_PER_BLOCK, 315
 MAX_BLOCKS, 315
 MODE_COMPILE, 317
 MODE_INTERPRET, 317
 name, 336
 name_len, 336
 physics, 336
 PHYSICS_STATE_COMPILED, 315
 PHYSICS_STATE_HIDDEN, 315
 PHYSICS_STATE_IMMEDIATE, 315
 PHYSICS_STATE_PINNED, 315
 SPIN_IDLE, 315
 STACK_SIZE, 315
 transition_metrics, 336
 ucell_t, 317
 UNLIKELY, 315
 USER_BLOCKS_START, 315
 vaddr_t, 317
 VM, 317
 VM_ADDR, 319
 vm_addr_ok, 319
 vm_addr_to_block, 319
 vm_align, 320
 vm_allot, 320
 vm_bootstrap_root_vocabulary, 320
 vm_cleanup, 321
 vm_compile_call, 321
 vm_compile_exit, 321
 vm_compile_literal, 321
 vm_compile_word, 321
 vm_create_word, 322
 vm_dictionary_find_by_func, 322
 vm_dictionary_find_latest_by_func, 323
 vm_dictionary_get_data_field, 323
 vm_dictionary_lookup_by_word_id, 323
 vm_dictionary_track_entry, 323
 vm_dictionary_untrack_entry, 323
 vm_enable_interpreter, 324
 vm_enter_compile_mode, 324
 vm_exit_compile_mode, 324
 vm_find_word, 324
 vm_get_block_addr, 325
 vm_heartbeat_start, 325
 vm_hide_word, 325
 vm_init, 325
 vm_init_with_host, 326
 vm_interpret, 326
 vm_interpret_word, 327
 vm_load_cell, 327
 vm_load_u8, 328
 vm_make_immediate, 328
 VM_MEMORY_SIZE, 315
 vm_mode_t, 317
 vm_parse_number, 328
 vm_parse_word, 328
 vm_pin_execution_heat, 329
 VM_POP, 315
 vm_pop, 329
 vm_pop_fast, 330
 vm_ptr, 330
 VM_PUSH, 316
 vm_push, 330
 vm_push_fast, 331
 vm_repl, 331
 VM_RPOP, 316
 vm_rpop, 332
 vm_rpop_fast, 332
 VM_RPUSH, 316
 vm_rpush, 332
 vm_rpush_fast, 333
 vm_run_smoke_tests, 333
 vm_smudge_word, 333
 vm_snapshot_read, 333
 vm_store_cell, 333
 vm_store_u8, 333
 vm_tick, 334
 vm_tick_inference_engine, 334
 vm_tick_slope_validator, 334
 vm_tick_window_tuner, 335
 vm_unpin_execution_heat, 335
 VMHostServices, 317
 WORD_COMPILED, 316
 WORD_FROZEN, 316
 word_func_t, 317
 WORD_HIDDEN, 316
 word_id, 336
 WORD_ID_INVALID, 316
 WORD_IMMEDIATE, 316
 WORD_NAME_MAX, 316
 WORD_PINNED, 316
 WORD_SMUDGED, 316
 WordTransitionMetrics, 317
 vm_abs_asm
 vm_asm_opt_arm64.h, 339
 vm_asm_opt_riscv64.h, 344
 vm_abs_opt
 arch_detect.h, 167
 vm_add_check_overflow
 Stack Operations, 21
 vm_asm_opt_arm64.h, 339
 vm_asm_opt_riscv64.h, 344
 vm_add_overflow_opt
 arch_detect.h, 167
 VM_ADDR
 vm.h, 319
 vm_addr_from_ptr
 VM Input Operations, 16
 vm_addr_ok

- vm.c, 611
- vm.h, 319
- vm_core.c, 545
- vm_addr_to_block
 - memory_management.c, 416
 - vm.h, 319
- vm_align
 - memory_management.c, 417
 - memory_management.h, 199
 - vm.h, 320
- vm_alloc_fn
 - capsule_birth.c, 498
- vm_allot
 - memory_management.c, 417
 - memory_management.h, 200
 - vm.h, 320
- vm_api.c
 - vm_api_create_word, 617
 - vm_api_find_word, 617
 - vm_api_hide_word, 617
 - vm_api_pop, 618
 - vm_api_push, 618
 - vm_api_rpop, 618
 - vm_api_rpush, 619
 - vm_api_smudge_word, 619
- vm_api_create_word
 - vm_api.c, 617
- vm_api_find_word
 - vm_api.c, 617
- vm_api_hide_word
 - vm_api.c, 617
- vm_api_pop
 - vm_api.c, 618
- vm_api_push
 - vm_api.c, 618
- vm_api_rpop
 - vm_api.c, 618
- vm_api_rpush
 - vm_api.c, 619
- vm_api_smudge_word
 - vm_api.c, 619
- vm_asm_opt_arm64.h
 - vm_abs_asm, 339
 - vm_add_check_overflow, 339
 - vm_clz, 339
 - vm_ctz, 339
 - vm_dc_zva, 339
 - vm_divmod, 339
 - vm_has_neon, 340
 - vm_is_cortex_a72, 340
 - vm_max_asm, 340
 - vm_memcpy_asm, 340
 - vm_memzero_asm, 340
 - vm_min_asm, 341
 - vm_mul_double, 341
 - vm_pop_asm, 341
 - vm_popcnt, 341
 - vm_prefetch, 341
 - vm_prefetch_stream, 341
 - vm_push_asm, 341
 - vm_read_midr, 342
 - vm_rpop_asm, 342
 - vm_rpush_asm, 342
 - vm_strcmp_asm, 342
 - vm_strcmp_short_asm, 343
 - vm_sub_check_overflow, 343
- vm_asm_opt_riscv64.h
 - vm_abs_asm, 344
 - vm_add_check_overflow, 344
 - vm_clz, 344
 - vm_ctz, 344
 - vm_divmod, 344
 - vm_has_riscv_b_ext, 344
 - vm_has_riscv_m_ext, 344
 - vm_max_asm, 345
 - vm_memcpy_asm, 345
 - vm_memzero_asm, 345
 - vm_min_asm, 345
 - vm_mul_double, 345
 - vm_pop_asm, 345
 - vm_popcnt, 345
 - vm_prefetch, 345
 - vm_prefetch_stream, 345
 - vm_push_asm, 345
 - vm_rpop_asm, 346
 - vm_rpush_asm, 346
 - vm_strcmp_asm, 346
 - vm_sub_check_overflow, 346
- vm_assert_interpreter_enabled
 - vm_core.c, 546
- vm_bootstrap.c
 - vm_bootstrap_root_vocabulary, 620
 - vm_bootstrap_scr, 620, 621
 - vm_cleanup, 621
 - vm_init, 620, 621
 - vm_init_with_host, 620
- vm_bootstrap_root_vocabulary
 - vm.h, 320
 - vm_bootstrap.c, 620
- vm_bootstrap_scr
 - vm_bootstrap.c, 620, 621
- vm_check_arena
 - vm_core.c, 544
- vm_cleanup
 - vm.h, 321
 - vm_bootstrap.c, 621
 - vm_core.c, 546
 - vm_internal.h, 629
- vm_clz
 - vm_asm_opt_arm64.h, 339
 - vm_asm_opt_riscv64.h, 344
- vm_clz_opt
 - arch_detect.h, 167
- vm_compile_call
 - vm.c, 612
 - vm.h, 321

- vm_core.c, 546
- vm_compile_exit
 - vm.c, 612
 - vm.h, 321
 - vm_core.c, 546
- vm_compile_literal
 - vm.c, 612
 - vm.h, 321
 - vm_core.c, 546
- vm_compile_word
 - vm.c, 612
 - vm.h, 321
 - vm_core.c, 546
- vm_core.c
 - execute_colon_word, 544
 - hosted_alloc, 544
 - hosted_fake_ns, 550
 - hosted_free, 544
 - hosted_mutex_destroy, 544
 - hosted_mutex_init, 544
 - hosted_mutex_lock, 545
 - hosted_mutex_unlock, 545
 - hosted_panic, 545
 - hosted_putc, 545
 - hosted_puts, 545
 - hosted_services, 550
 - hosted_time_ns, 545
 - hosted_xt_entry_owned, 545
 - hosted_xt_is_executable, 545
 - vm_addr_ok, 545
 - vm_assert_interpreter_enabled, 546
 - vm_check_arena, 544
 - vm_cleanup, 546
 - vm_compile_call, 546
 - vm_compile_exit, 546
 - vm_compile_literal, 546
 - vm_compile_word, 546
 - vm_default_host_services, 546
 - vm_enable_interpreter, 547
 - vm_enter_compile_mode, 547
 - vm_exit_compile_mode, 547
 - vm_get_base, 547
 - vm_host, 547
 - vm_host_alloc, 547
 - vm_host_calloc, 547
 - vm_host_free, 548
 - vm_interpret, 548
 - vm_interpret_word, 548
 - vm_load_cell, 548
 - vm_load_u8, 548
 - vm_make_immediate, 548
 - vm_monotonic_ns, 549
 - vm_parse_number, 549
 - vm_parse_word, 549
 - vm_ptr, 549
 - vm_reset_hosted_fake_ns, 550
 - vm_set_base, 550
 - vm_store_cell, 550
 - vm_store_u8, 550
- vm_cpuid
 - Stack Operations, 21
- vm_create_word
 - dictionary_management.c, 392
 - vm.h, 322
- vm_ctz
 - vm_asm_opt_arm64.h, 339
 - vm_asm_opt_riscv64.h, 344
- vm_ctz_opt
 - arch_detect.h, 167
- vm_cycles
 - Profiler, 125
- vm_dc_zva
 - vm_asm_opt_arm64.h, 339
- vm_debug.c
 - dbg_get_base, 623
 - dump_block_preview, 623
 - g_vm, 625
 - mode_str, 623
 - sig_handler, 624
 - vm_debug_dump_state, 624
 - vm_debug_install_signal_handlers, 624
 - vm_debug_set_current_vm, 624
- vm_debug.h
 - vm_debug_dump_state, 346
 - vm_debug_install_signal_handlers, 347
 - vm_debug_set_current_vm, 347
- vm_debug_dump_state
 - vm_debug.c, 624
 - vm_debug.h, 346
- vm_debug_install_signal_handlers
 - vm_debug.c, 624
 - vm_debug.h, 347
- vm_debug_set_current_vm
 - vm_debug.c, 624
 - vm_debug.h, 347
- vm_default_host_services
 - vm_core.c, 546
 - vm_internal.h, 626
- vm_dict_hash_fn
 - capsule_birth.c, 498
- vm_dictionary_acquire_word_id
 - dictionary_management.c, 392
- vm_dictionary_find_by_func
 - dictionary_management.c, 393
 - vm.h, 322
- vm_dictionary_find_latest_by_func
 - dictionary_management.c, 393
 - vm.h, 323
- vm_dictionary_get_data_field
 - dictionary_management.c, 393
 - vm.h, 323
- vm_dictionary_index_reset
 - dictionary_management.c, 393
- vm_dictionary_lookup_by_word_id
 - dictionary_management.c, 394
 - vm.h, 323

- vm_dictionary_track_entry
 - dictionary_management.c, [394](#)
 - vm.h, [323](#)
- vm_dictionary_untrack_entry
 - dictionary_management.c, [394](#)
 - vm.h, [323](#)
- vm_divmod
 - Stack Operations, [22](#)
 - vm_asm_opt_arm64.h, [339](#)
 - vm_asm_opt_riscv64.h, [344](#)
- vm_divmod_opt
 - arch_detect.h, [167](#)
- vm_docol_arm64
 - vm_inner_interp_arm64.h, [352](#)
- vm_docol_asm
 - vm_inner_interp_asm.h, [359](#)
- vm_docol_rv64
 - vm_inner_interp_riscv64.h, [366](#)
- vm_enable_interpreter
 - vm.h, [324](#)
 - vm_core.c, [547](#)
- vm_enter_compile_mode
 - vm.c, [612](#)
 - vm.h, [324](#)
 - vm_core.c, [547](#)
- vm_exec_fn
 - capsule_birth.c, [498](#)
- vm_exit_arm64
 - vm_inner_interp_arm64.h, [352](#)
- vm_exit_asm
 - vm_inner_interp_asm.h, [359](#)
- vm_exit_compile_mode
 - vm.c, [613](#)
 - vm.h, [324](#)
 - vm_core.c, [547](#)
- vm_exit_rv64
 - vm_inner_interp_riscv64.h, [366](#)
- vm_find_word
 - dictionary_management.c, [394](#)
 - vm.h, [324](#)
- vm_get_arch_name
 - arch_detect.h, [169](#)
- vm_get_base
 - vm.c, [613](#)
 - vm_core.c, [547](#)
 - vm_internal.h, [626](#), [629](#)
- vm_get_block_addr
 - memory_management.c, [417](#)
 - vm.h, [325](#)
- vm_get_build_info
 - arch_detect.h, [169](#)
- vm_has_asm_opt
 - arch_detect.h, [169](#)
- vm_has_direct_threading
 - arch_detect.h, [170](#)
- vm_has_neon
 - vm_asm_opt_arm64.h, [340](#)
- vm_has_riscv_b_ext
 - vm_asm_opt_riscv64.h, [344](#)
- vm_has_riscv_m_ext
 - vm_asm_opt_riscv64.h, [344](#)
- vm_has_sse42
 - Stack Operations, [22](#)
- vm_has_sse42_opt
 - arch_detect.h, [170](#)
- vm_heartbeat_publish_snapshot
 - vm_internal.h, [630](#)
 - vm_time.c, [634](#)
- vm_heartbeat_run_cycle
 - vm_internal.h, [627](#), [630](#)
 - vm_runtime.c, [552](#)
 - vm_time.c, [634](#)
- vm_heartbeat_start
 - vm.h, [325](#)
- vm_heartbeat_start_thread
 - vm_internal.h, [630](#)
 - vm_time.c, [634](#)
- vm_heartbeat_stop_thread
 - vm_internal.h, [630](#)
 - vm_time.c, [634](#)
- vm_heartbeat_update_l8
 - vm_runtime.c, [552](#)
 - vm_time.c, [634](#)
- vm_hide_word
 - dictionary_management.c, [395](#)
 - vm.h, [325](#)
- vm_host
 - vm_core.c, [547](#)
 - vm_internal.h, [627](#)
- vm_host.h
 - SK_ALLOC, [349](#)
 - SK_ALLOC_ALIGNED, [349](#)
 - sk_alloc_fn, [350](#)
 - SK_FREE, [349](#)
 - sk_free_fn, [350](#)
 - sk_host_services, [351](#)
 - SK_MUTEX_DESTROY, [349](#)
 - sk_mutex_destroy_fn, [350](#)
 - SK_MUTEX_INIT, [349](#)
 - sk_mutex_init_fn, [350](#)
 - SK_MUTEX_LOCK, [349](#)
 - sk_mutex_lock_fn, [350](#)
 - SK_MUTEX_UNLOCK, [349](#)
 - sk_mutex_unlock_fn, [350](#)
 - sk_panic_fn, [350](#)
 - SK_PUTC, [349](#)
 - sk_putc_fn, [350](#)
 - SK_PUTS, [349](#)
 - sk_puts_fn, [350](#)
 - sk_time_fn, [350](#)
 - SK_TIME_NS, [349](#)
 - sk_xt_entry_owns_fn, [350](#)
 - sk_xt_validate_fn, [350](#)
 - VMHostServices, [350](#)
- vm_host_alloc
 - vm_core.c, [547](#)

- vm_internal.h, [627](#)
- vm_host_calloc
 - vm_core.c, [547](#)
 - vm_internal.h, [627](#)
- vm_host_free
 - vm_core.c, [548](#)
 - vm_internal.h, [627](#)
- vm_id
 - CapsuleRunRecord, [68](#)
 - VMRegistryEntry, [156](#)
- vm_init
 - vm.h, [325](#)
 - vm_bootstrap.c, [620](#), [621](#)
 - vm_internal.h, [630](#)
- vm_init_with_host
 - vm.h, [326](#)
 - vm_bootstrap.c, [620](#)
- vm_inner_interp_arm64.h
 - NEXT_ARM64, [352](#)
 - vm_docol_arm64, [352](#)
 - vm_exit_arm64, [352](#)
 - vm_save_registers_arm64, [352](#)
 - vm_setup_registers_arm64, [352](#)
- vm_inner_interp_asm.h
 - NEXT_ASM, [353](#)
 - PRIM_2DROP, [354](#)
 - PRIM_2DUP, [354](#)
 - PRIM_AND, [354](#)
 - PRIM_BRANCH, [354](#)
 - PRIM_C_FETCH, [354](#)
 - PRIM_C_STORE, [354](#)
 - PRIM_DROP, [355](#)
 - PRIM_DUP, [355](#)
 - PRIM_EQUALS, [355](#)
 - PRIM_EXECUTE, [355](#)
 - PRIM_FETCH, [355](#)
 - PRIM_GREATER, [355](#)
 - PRIM_INVERT, [356](#)
 - PRIM_LESS, [356](#)
 - PRIM_MINUS, [356](#)
 - PRIM_MOD, [356](#)
 - PRIM_NEGATE, [356](#)
 - PRIM_OR, [356](#)
 - PRIM_OVER, [357](#)
 - PRIM_PLUS, [357](#)
 - PRIM_R_FETCH, [357](#)
 - PRIM_R_FROM, [357](#)
 - PRIM_ROT, [357](#)
 - PRIM_SLASH, [357](#)
 - PRIM_STAR, [357](#)
 - PRIM_STORE, [358](#)
 - PRIM_SWAP, [358](#)
 - PRIM_TO_R, [358](#)
 - PRIM_XOR, [358](#)
 - PRIM_ZBRANCH, [358](#)
 - PRIM_ZERO_EQUALS, [359](#)
 - PRIM_ZERO_LESS, [359](#)
 - vm_docol_asm, [359](#)
 - vm_exit_asm, [359](#)
 - vm_save_registers, [359](#)
 - vm_setup_registers, [359](#)
- vm_inner_interp_riscv64.h
 - NEXT_RV64, [360](#)
 - PRIM_2DROP_RV64, [360](#)
 - PRIM_2DUP_RV64, [361](#)
 - PRIM_AND_RV64, [361](#)
 - PRIM_BRANCH_RV64, [361](#)
 - PRIM_C_FETCH_RV64, [361](#)
 - PRIM_C_STORE_RV64, [361](#)
 - PRIM_DROP_RV64, [361](#)
 - PRIM_DUP_RV64, [362](#)
 - PRIM_EQUALS_RV64, [362](#)
 - PRIM_EXECUTE_RV64, [362](#)
 - PRIM_FETCH_RV64, [362](#)
 - PRIM_GREATER_RV64, [362](#)
 - PRIM_INVERT_RV64, [362](#)
 - PRIM_LESS_RV64, [362](#)
 - PRIM_MINUS_RV64, [363](#)
 - PRIM_MOD_RV64, [363](#)
 - PRIM_NEGATE_RV64, [363](#)
 - PRIM_OR_RV64, [363](#)
 - PRIM_OVER_RV64, [363](#)
 - PRIM_PLUS_RV64, [363](#)
 - PRIM_R_FETCH_RV64, [364](#)
 - PRIM_R_FROM_RV64, [364](#)
 - PRIM_ROT_RV64, [364](#)
 - PRIM_SLASH_RV64, [364](#)
 - PRIM_STAR_RV64, [364](#)
 - PRIM_STORE_RV64, [364](#)
 - PRIM_SWAP_RV64, [365](#)
 - PRIM_TO_R_RV64, [365](#)
 - PRIM_XOR_RV64, [365](#)
 - PRIM_ZBRANCH_RV64, [365](#)
 - PRIM_ZERO_EQUALS_RV64, [365](#)
 - PRIM_ZERO_LESS_RV64, [365](#)
 - vm_docol_rv64, [366](#)
 - vm_exit_rv64, [366](#)
 - vm_save_registers_rv64, [366](#)
 - vm_setup_registers_rv64, [366](#)
- vm_input_ensure
 - VM Input Operations, [16](#)
- vm_input_in
 - VM Input Operations, [17](#)
- vm_input_load_line
 - VM Input Operations, [17](#)
- vm_input_source
 - VM Input Operations, [18](#)
- vm_input_span
 - VM Input Operations, [18](#)
- vm_input_tib
 - VM Input Operations, [19](#)
- vm_internal.h
 - execute_colon_word, [629](#)
 - HEARTBEAT_DECAY_BATCH, [626](#), [629](#)
 - HEARTBEAT_HAS_THREADS, [626](#), [629](#)
 - heartbeat_publish_snapshot, [626](#)

- heartbeat_snapshot_index_load, [626](#), [629](#)
- heartbeat_snapshot_index_store, [626](#), [629](#)
- HeartbeatWorker, [626](#), [629](#)
- PARITY_MODE, [626](#)
- SK_PARITY_DEBUG, [626](#)
- vm_cleanup, [629](#)
- vm_default_host_services, [626](#)
- vm_get_base, [626](#), [629](#)
- vm_heartbeat_publish_snapshot, [630](#)
- vm_heartbeat_run_cycle, [627](#), [630](#)
- vm_heartbeat_start_thread, [630](#)
- vm_heartbeat_stop_thread, [630](#)
- vm_host, [627](#)
- vm_host_alloc, [627](#)
- vm_host_calloc, [627](#)
- vm_host_free, [627](#)
- vm_init, [630](#)
- vm_monotonic_ns, [627](#)
- vm_reset_hosted_fake_ns, [627](#)
- vm_set_base, [628](#), [631](#)
- vm_snapshot_read, [631](#)
- vm_tick, [631](#)
- vm_tick_inference_engine, [632](#)
- vm_tick_slope_validator, [632](#)
- vm_tick_window_tuner, [632](#)
- vm_interpret
 - vm.c, [613](#)
 - vm.h, [326](#)
 - vm_core.c, [548](#)
- vm_interpret_word
 - vm.c, [613](#)
 - vm.h, [327](#)
 - vm_core.c, [548](#)
- vm_is_cortex_a72
 - vm_asm_opt_arm64.h, [340](#)
- vm_load_cell
 - vm.c, [614](#)
 - vm.h, [327](#)
 - vm_core.c, [548](#)
- vm_load_u8
 - vm.c, [614](#)
 - vm.h, [328](#)
 - vm_core.c, [548](#)
- vm_make_immediate
 - vm.c, [614](#)
 - vm.h, [328](#)
 - vm_core.c, [548](#)
- vm_max_asm
 - Stack Operations, [22](#)
 - vm_asm_opt_arm64.h, [340](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_max_opt
 - arch_detect.h, [168](#)
- vm_memcpy_asm
 - Stack Operations, [22](#)
 - vm_asm_opt_arm64.h, [340](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_memcpy_opt
 - arch_detect.h, [168](#)
- VM_MEMORY_SIZE
 - vm.h, [315](#)
- vm_memzero_asm
 - Stack Operations, [23](#)
 - vm_asm_opt_arm64.h, [340](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_memzero_opt
 - arch_detect.h, [168](#)
- vm_min_asm
 - Stack Operations, [23](#)
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_min_opt
 - arch_detect.h, [168](#)
- vm_mode
 - timer_calibration_record, [140](#)
- vm_mode_t
 - vm.h, [317](#)
- vm_monotonic_ns
 - vm_core.c, [549](#)
 - vm_internal.h, [627](#)
- vm_mul_double
 - Stack Operations, [23](#)
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_mul_double_opt
 - arch_detect.h, [168](#)
- vm_ns_base
 - timer.c, [489](#)
- vm_parse_number
 - vm.c, [614](#)
 - vm.h, [328](#)
 - vm_core.c, [549](#)
- vm_parse_word
 - vm.c, [615](#)
 - vm.h, [328](#)
 - vm_core.c, [549](#)
- vm_pin_execution_heat
 - dictionary_management.c, [395](#)
 - vm.h, [329](#)
- vm_pm_start
 - timer.c, [489](#)
- VM_POP
 - vm.h, [315](#)
- vm_pop
 - stack_management.c, [465](#)
 - vm.h, [329](#)
- vm_pop_asm
 - Stack Operations, [23](#)
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_pop_fast
 - vm.h, [330](#)
- vm_pop_opt
 - arch_detect.h, [168](#)
- vm_popcnt
 - vm_asm_opt_arm64.h, [341](#)

- vm_asm_opt_riscv64.h, [345](#)
- vm_popcnt_opt
 - arch_detect.h, [168](#)
- vm_prefetch
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_prefetch_opt
 - arch_detect.h, [168](#)
- vm_prefetch_stream
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_prefetch_stream_opt
 - arch_detect.h, [168](#)
- vm_ptr
 - vm.c, [615](#)
 - vm.h, [330](#)
 - vm_core.c, [549](#)
- VM_PUSH
 - vm.h, [316](#)
- vm_push
 - stack_management.c, [466](#)
 - vm.h, [330](#)
- vm_push_asm
 - Stack Operations, [24](#)
 - vm_asm_opt_arm64.h, [341](#)
 - vm_asm_opt_riscv64.h, [345](#)
- vm_push_fast
 - vm.h, [331](#)
- vm_push_opt
 - arch_detect.h, [168](#)
- vm_read_midr
 - vm_asm_opt_arm64.h, [342](#)
- vm_registry
 - capsule_birth.c, [499](#)
- vm_registry_alloc
 - capsule_birth.c, [498](#)
- vm_registry_count
 - capsule_birth.c, [499](#)
- vm_repl
 - repl.c, [535](#)
 - repl.h, [288](#)
 - vm.h, [331](#)
- vm_reset_hosted_fake_ns
 - vm_core.c, [550](#)
 - vm_internal.h, [627](#)
- VM_RPOP
 - vm.h, [316](#)
- vm_rpop
 - stack_management.c, [467](#)
 - vm.h, [332](#)
- vm_rpop_asm
 - Stack Operations, [24](#)
 - vm_asm_opt_arm64.h, [342](#)
 - vm_asm_opt_riscv64.h, [346](#)
- vm_rpop_fast
 - vm.h, [332](#)
- vm_rpop_opt
 - arch_detect.h, [169](#)
- VM_RPUSH
 - vm.h, [316](#)
- vm_rpush
 - stack_management.c, [467](#)
 - vm.h, [332](#)
- vm_rpush_asm
 - Stack Operations, [25](#)
 - vm_asm_opt_arm64.h, [342](#)
 - vm_asm_opt_riscv64.h, [346](#)
- vm_rpush_fast
 - vm.h, [333](#)
- vm_rpush_opt
 - arch_detect.h, [169](#)
- vm_run_smoke_tests
 - vm.h, [333](#)
- vm_runtime.c
 - heartbeat_publish_snapshot, [551](#)
 - heartbeat_snapshot_index_load, [551](#)
 - heartbeat_snapshot_index_store, [551](#)
 - vm_heartbeat_run_cycle, [552](#)
 - vm_heartbeat_update_l8, [552](#)
 - vm_snapshot_read, [552](#)
 - vm_tick, [552](#)
 - vm_tick_apply_background_decay, [553](#)
 - vm_tick_inference_engine, [553](#)
 - vm_tick_slope_validator, [553](#)
 - vm_tick_window_tuner, [553](#)
- vm_save_registers
 - vm_inner_interp_asm.h, [359](#)
- vm_save_registers_arm64
 - vm_inner_interp_arm64.h, [352](#)
- vm_save_registers_opt
 - arch_detect.h, [169](#)
- vm_save_registers_rv64
 - vm_inner_interp_riscv64.h, [366](#)
- vm_set_base
 - vm.c, [615](#)
 - vm_core.c, [550](#)
 - vm_internal.h, [628](#), [631](#)
- vm_setup_registers
 - vm_inner_interp_asm.h, [359](#)
- vm_setup_registers_arm64
 - vm_inner_interp_arm64.h, [352](#)
- vm_setup_registers_opt
 - arch_detect.h, [169](#)
- vm_setup_registers_rv64
 - vm_inner_interp_riscv64.h, [366](#)
- vm_smudge_word
 - dictionary_management.c, [395](#)
 - vm.h, [333](#)
- vm_snapshot_read
 - vm.h, [333](#)
 - vm_internal.h, [631](#)
 - vm_runtime.c, [552](#)
 - vm_time.c, [635](#)
- VM_STATE_DEAD
 - capsule_run.h, [262](#)
- VM_STATE_EMBRYO

- capsule_run.h, 262
- VM_STATE_LIVE
 - capsule_run.h, 262
- VM_STATE_STILLBORN
 - capsule_run.h, 262
- vm_store_cell
 - vm.c, 615
 - vm.h, 333
 - vm_core.c, 550
- vm_store_u8
 - vm.c, 616
 - vm.h, 333
 - vm_core.c, 550
- vm_strcmp_asm
 - Stack Operations, 25
 - vm_asm_opt_arm64.h, 342
 - vm_asm_opt_riscv64.h, 346
- vm_strcmp_opt
 - arch_detect.h, 169
- vm_strcmp_short_asm
 - vm_asm_opt_arm64.h, 343
- VM_STRICT_PTR
 - starforth_config.h, 244
- vm_sub_check_overflow
 - Stack Operations, 25
 - vm_asm_opt_arm64.h, 343
 - vm_asm_opt_riscv64.h, 346
- vm_sub_overflow_opt
 - arch_detect.h, 169
- vm_tick
 - vm.h, 334
 - vm_internal.h, 631
 - vm_runtime.c, 552
 - vm_time.c, 635
- vm_tick_apply_background_decay
 - vm_runtime.c, 553
 - vm_time.c, 635
- vm_tick_inference_engine
 - vm.h, 334
 - vm_internal.h, 632
 - vm_runtime.c, 553
 - vm_time.c, 635
- vm_tick_slope_validator
 - vm.h, 334
 - vm_internal.h, 632
 - vm_runtime.c, 553
 - vm_time.c, 637
- vm_tick_window_tuner
 - vm.h, 335
 - vm_internal.h, 632
 - vm_runtime.c, 553
 - vm_time.c, 637
- vm_time.c
 - vm_heartbeat_publish_snapshot, 634
 - vm_heartbeat_run_cycle, 634
 - vm_heartbeat_start_thread, 634
 - vm_heartbeat_stop_thread, 634
 - vm_heartbeat_update_l8, 634
 - vm_snapshot_read, 635
 - vm_tick, 635
 - vm_tick_apply_background_decay, 635
 - vm_tick_inference_engine, 635
 - vm_tick_slope_validator, 637
 - vm_tick_window_tuner, 637
- vm_timebase_init_from_pmtimer
 - timer.c, 488
- vm_unpin_execution_heat
 - dictionary_management.c, 395
 - vm.h, 335
- vm_vocabulary_find_word
 - vocabulary_words.c, 752
- vm_workload_duration_ns_q48
 - DoeMetrics, 79
- VMHostServices, 153
 - alloc, 153
 - free, 153
 - is_executable_ptr, 154
 - monotonic_ns, 154
 - mutex_destroy, 154
 - mutex_init, 154
 - mutex_lock, 154
 - mutex_unlock, 154
 - owns_xt_entry, 154
 - panic, 154
 - parity_mode, 154
 - putc, 154
 - puts, 154
 - verbose, 154
 - vm.h, 317
 - vm_host.h, 350
- vmm.c
 - ADDR_MASK, 531
 - alloc_page_zeroed, 532
 - get_or_alloc_table, 532
 - get_table, 532
 - hpet_phys_base, 535
 - invlpg, 532
 - is_ram_type, 532
 - lapic_phys_base, 535
 - load_cr3, 533
 - make_pte, 533
 - paddr_to_virt, 533
 - pd_index, 533
 - pdpt_index, 533
 - pml4_index, 533
 - pt_index, 533
 - PTE_ACCESSED, 531
 - PTE_DIRTY, 531
 - PTE_GLOBAL, 531
 - PTE_NX, 531
 - PTE_PCD, 531
 - PTE_PRESENT, 531
 - PTE_PS, 532
 - PTE_PWT, 532
 - PTE_USER, 532
 - PTE_WRITABLE, 532

- resolve_pt, [534](#)
- vmm_get_paddr, [534](#)
- vmm_init, [534](#)
- vmm_map_page, [534](#)
- vmm_map_range, [534](#)
- vmm_query_page, [534](#)
- vmm_root_pml4_phys, [535](#)
- vmm_unmap_page, [535](#)
- vmm.h
 - VMM_FLAG_NX, [305](#)
 - VMM_FLAG_PRESENT, [305](#)
 - VMM_FLAG_USER, [305](#)
 - VMM_FLAG_WRITABLE, [305](#)
 - vmm_get_paddr, [305](#)
 - vmm_init, [305](#)
 - vmm_map_page, [305](#)
 - vmm_map_range, [306](#)
 - VMM_PAGE_SIZE, [305](#)
 - vmm_query_page, [306](#)
 - vmm_unmap_page, [306](#)
- VMM_FLAG_NX
 - vmm.h, [305](#)
- VMM_FLAG_PRESENT
 - vmm.h, [305](#)
- VMM_FLAG_USER
 - vmm.h, [305](#)
- VMM_FLAG_WRITABLE
 - vmm.h, [305](#)
- vmm_get_paddr
 - vmm.c, [534](#)
 - vmm.h, [305](#)
- vmm_init
 - vmm.c, [534](#)
 - vmm.h, [305](#)
- vmm_map_page
 - vmm.c, [534](#)
 - vmm.h, [305](#)
- vmm_map_range
 - vmm.c, [534](#)
 - vmm.h, [306](#)
- vmm_page_info_t, [155](#)
 - executable, [155](#)
 - present, [155](#)
 - writable, [155](#)
- VMM_PAGE_SIZE
 - vmm.h, [305](#)
- vmm_query_page
 - vmm.c, [534](#)
 - vmm.h, [306](#)
- vmm_root_pml4_phys
 - vmm.c, [535](#)
- vmm_unmap_page
 - vmm.c, [535](#)
 - vmm.h, [306](#)
- VMRegistryEntry, [155](#)
 - birth_capsule_id, [155](#)
 - birth_dict_hash, [155](#)
 - birth_timestamp_ns, [155](#)
 - flags, [155](#)
 - reserved, [156](#)
 - state, [156](#)
 - vm_id, [156](#)
- VMState
 - capsule_run.h, [262](#)
- vocab_find_word
 - vocabulary_words.c, [752](#)
- vocab_safe_len
 - vocabulary_words.c, [752](#)
- vocab_sync_vm_vars
 - vocabulary_words.c, [752](#)
- vocabulary_create_vocabulary_direct
 - vocabulary_words.c, [753](#)
 - vocabulary_words.h, [703](#)
- vocabulary_debug_print_hex
 - vocabulary_words.c, [753](#)
- vocabulary_select_runtime
 - vocabulary_words.c, [753](#)
- vocabulary_word_context
 - vocabulary_words.c, [753](#)
- vocabulary_word_current
 - vocabulary_words.c, [753](#)
- vocabulary_word_definitions
 - vocabulary_words.c, [754](#)
 - vocabulary_words.h, [703](#)
- vocabulary_word_forth
 - vocabulary_words.c, [754](#)
 - vocabulary_words.h, [704](#)
- vocabulary_word_order
 - vocabulary_words.c, [754](#)
- vocabulary_word_paren_find
 - vocabulary_words.c, [755](#)
- vocabulary_word_suites
 - vocabulary_words_test.c, [598](#)
- vocabulary_word_vocabulary
 - vocabulary_words.c, [755](#)
- vocabulary_words.c
 - context_var_addr, [755](#)
 - context_vocab, [755](#)
 - ctx_cached_head, [755](#)
 - ctx_fc, [755](#)
 - ctx_n, [756](#)
 - current_var_addr, [756](#)
 - current_vocab, [756](#)
 - fc_free, [751](#)
 - fc_rebuild, [751](#)
 - forth_cached_head, [756](#)
 - forth_fc, [756](#)
 - forth_n, [756](#)
 - forth_vocab, [756](#)
 - init_vocabulary_system, [751](#)
 - register_vocabulary_words, [751](#)
 - SF_FC_BUCKETS, [751](#)
 - vm_vocabulary_find_word, [752](#)
 - vocab_find_word, [752](#)
 - vocab_safe_len, [752](#)
 - vocab_sync_vm_vars, [752](#)

- vocabulary_create_vocabulary_direct, 753
- vocabulary_debug_print_hex, 753
- vocabulary_select_runtime, 753
- vocabulary_word_context, 753
- vocabulary_word_current, 753
- vocabulary_word_definitions, 754
- vocabulary_word_forth, 754
- vocabulary_word_order, 754
- vocabulary_word_paren_find, 755
- vocabulary_word_vocabulary, 755
- vocabulary_words.h
 - register_vocabulary_words, 703
 - vocabulary_create_vocabulary_direct, 703
 - vocabulary_word_definitions, 703
 - vocabulary_word_forth, 704
- vocabulary_words_test.c
 - run_vocabulary_words_tests, 598
 - vocabulary_word_suites, 598
- VOID
 - uefi.h, 299
- vol_meta
 - block_subsystem.c, 387
- vol_meta_dirty
 - block_subsystem.c, 387
- volmeta_from_buf
 - block_subsystem.c, 383
- volmeta_to_buf
 - block_subsystem.c, 383
- vt
 - blkio_dev, 60
- WaitForEvent
 - _EFI_BOOT_SERVICES, 48
- WARMUP_WINDOWS
 - timer.c, 482
- width
 - FramebufferInfo, 91
- window
 - InferenceInputs, 105
 - time_trust_state, 139
- window_diversity_percent
 - DoeMetrics, 79
- window_final_size_bytes
 - DoeMetrics, 79
- window_hash64
 - ParityPacket, 110
- window_pos
 - rolling_window_view_t, 128
 - RollingWindowOfTruth, 130
- window_push
 - timer.c, 488
- WINDOW_TICKS_HPET
 - timer.c, 482
- WINDOW_TICKS_PIT
 - timer.c, 482
- window_tuning_checks
 - DoeMetrics, 79
 - PipelineGlobalMetrics, 122
- window_variance_q48
 - DoeMetrics, 79
 - InferenceOutputs, 106
 - timer.c, 488
- window_width
 - HeartbeatSnapshot, 94
 - HeartbeatTickSnapshot, 99
- windows_used
 - timer.c, 489
 - timer_calibration_record, 141
- word_capacity
 - profiler.c, 454
- WORD_COMPILED
 - vm.h, 316
- word_count
 - InferenceInputs, 106
 - ParityPacket, 110
 - profiler.c, 455
- word_count_at_check
 - VM, 152
- word_divide
 - arithmetic_words.c, 643
- word_divmod
 - arithmetic_words.c, 643
- word_executions_delta
 - HeartbeatTickSnapshot, 99
- WORD_FROZEN
 - vm.h, 316
- word_func_t
 - vm.h, 317
- WORD_HIDDEN
 - vm.h, 316
- word_id
 - DictEntry, 72
 - vm.h, 336
- WORD_ID_INVALID
 - vm.h, 316
- word_id_map
 - VM, 153
- WORD_IMMEDIATE
 - vm.h, 316
- word_name
 - ProfileScope, 125
 - WordProfile, 157
 - WordTestSuite, 159
- WORD_NAME_MAX
 - vm.h, 316
- WORD_PINNED
 - vm.h, 316
- word_profile_capacity
 - Profiler, 125
- word_profile_count
 - Profiler, 125
- word_profiles
 - Profiler, 125
- word_registry.c
 - register_forth79_words, 638
 - register_word, 638
- word_registry.h

- register_forth79_words, 366
- register_word, 367
- word_scratch_addr
 - string_words.c, 746
- WORD_SCRATCH_CAP
 - string_words.c, 740
- WORD_SMUDGED
 - vm.h, 316
- word_stats
 - profiler.c, 455
- WordContract, 156
 - flags, 156
 - fuzz_rounds, 156
- WordProfile, 156
 - avg_time, 157
 - call_count, 157
 - max_time, 157
 - min_time, 157
 - percentage, 157
 - total_time, 157
 - word_name, 157
- words_executed
 - DoeMetrics, 79
 - HeartbeatState, 97
- WordStats, 157
 - call_count, 158
 - entry, 158
 - max_time_ns, 158
 - min_time_ns, 158
 - total_time_ns, 158
- WordTestSuite, 158
 - suite_contract, 159
 - test_count, 159
 - tests, 159
 - word_name, 159
- WordTransitionMetrics, 159
 - actual_window_size, 160
 - context_transitions, 160
 - context_window, 160
 - context_window_pos, 160
 - max_transition_probability_q48, 161
 - misprediction_cost_q48, 161
 - most_likely_next_word_id, 161
 - physics_pipelining_metrics.h, 209
 - prefetch_attempts, 161
 - prefetch_hits, 161
 - prefetch_latency_saved_q48, 161
 - prefetch_misses, 161
 - total_context_transitions, 162
 - total_transitions, 162
 - transition_heat, 162
 - vm.h, 317
- worker
 - HeartbeatState, 97
- writable
 - vmm_page_info_t, 155
- Write
 - _EFI_FILE_PROTOCOL, 49
- write
 - blkio_vtable_t, 63
- write_count
 - blk_meta_t, 56
- write_devblock_4k
 - block_subsystem.c, 383
- write_header_4k
 - block_subsystem.c, 383
- write_offset
 - physics_analytics_header, 113
- write_ramdrive_block
 - capsule_loader.c, 504
- writes
 - MemoryProfile, 108
- xxh64_avalanche
 - xxhash64.c, 515
- xxh64_merge_round
 - xxhash64.c, 515
- xxh64_read32
 - xxhash64.c, 515
- xxh64_read64
 - xxhash64.c, 515
- xxh64_rotl
 - xxhash64.c, 516
- xxh64_round
 - xxhash64.c, 516
- xxhash64
 - xxhash64.c, 516
 - xxhash64.h, 307
- xxhash64.c
 - xxh64_avalanche, 515
 - xxh64_merge_round, 515
 - xxh64_read32, 515
 - xxh64_read64, 515
 - xxh64_rotl, 516
 - xxh64_round, 516
 - xxhash64, 516
 - xxhash64_digest, 516
 - XXHASH64_PRIME1, 515
 - XXHASH64_PRIME2, 515
 - XXHASH64_PRIME3, 515
 - XXHASH64_PRIME4, 515
 - XXHASH64_PRIME5, 515
 - xxhash64_reset, 517
 - xxhash64_update, 517
- xxhash64.h
 - xxhash64, 307
 - xxhash64_capsule, 307
 - XXHASH64_CAPSULE_SEED, 307
 - xxhash64_digest, 308
 - xxhash64_reset, 308
 - xxhash64_update, 308
- xxhash64_capsule
 - xxhash64.h, 307
- XXHASH64_CAPSULE_SEED
 - xxhash64.h, 307
- xxhash64_digest
 - xxhash64.c, 516

- xxhash64.h, [308](#)
- XXHASH64_PRIME1
 - xxhash64.c, [515](#)
- XXHASH64_PRIME2
 - xxhash64.c, [515](#)
- XXHASH64_PRIME3
 - xxhash64.c, [515](#)
- XXHASH64_PRIME4
 - xxhash64.c, [515](#)
- XXHASH64_PRIME5
 - xxhash64.c, [515](#)
- xxhash64_reset
 - xxhash64.c, [517](#)
 - xxhash64.h, [308](#)
- xxhash64_update
 - xxhash64.c, [517](#)
 - xxhash64.h, [308](#)
- XXHash64State, [162](#)
 - buffer, [162](#)
 - buffer_size, [162](#)
 - seed, [163](#)
 - total_len, [163](#)
 - v1, [163](#)
 - v2, [163](#)
 - v3, [163](#)
 - v4, [163](#)
- Year
 - EFI_TIME, [85](#)
- zero
 - idt_entry, [104](#)
 - interrupts.c, [478](#)
- zuse_session
 - VM, [153](#)